

Foundations to Deep Learning

From foundations to deep learning — a one-stop study book

Table of contents

Foundations to Deep Learning	1
How to study from this book	1
 Part I — Mathematical Foundations	 3
Chapter 1 — Sets, functions, and mathematical preliminaries	5
1.1 What this chapter is for	5
1.2 The number sets and intervals	5
1.3 Sets: elements, cardinality, subsets	6
1.4 Building sets: set-builder notation	6
1.5 Set operations, Venn diagrams, De Morgan	6
1.6 Cartesian products and relations	7
1.7 Functions: the core definition	8
1.8 Three important kinds of functions	8
1.9 Composition of functions	9
1.10 Inverse functions	9
1.11 Logic: quantifiers, implies, equivalent	10
1.12 A note on proof techniques (not covered here)	10
1.13 Worked example set	10
1.14 Problem set	12
1.15 Where this goes next	13
Solutions — Chapter 1: Sets, functions, and mathematical preliminaries	13
Problem 1	13
Problem 2	13
Problem 3	13
Problem 4	14
Problem 5	14
Problem 6	14
Problem 7	14
Problem 8	15
Problem 9	15
Problem 10	15
Problem 11	15
Problem 12	16
 2. Vectors and matrices	 17
2.1 A vector = a list of numbers	17
2.2 The geometric view: arrows	17
2.3 Addition, subtraction, scalar multiplication	18
2.4 Magnitude (norm) and unit vectors	19

2.5 The dot product	19
2.6 Linear combinations	20
2.7 What is a matrix?	21
2.8 Special matrices	21
2.9 Matrix addition and scalar multiplication	22
2.10 Matrix multiplication: the heart of the chapter	22
2.11 The basic rules (and one famous non-rule)	24
Problem set	24
Where this goes next	25
Solutions to Chapter 2: Vectors and matrices	25
3. Linear systems, determinants, inverses	27
3.1 What this chapter is for	27
3.2 Linear equations and systems of linear equations	27
3.3 The matrix form $A\mathbf{x} = \mathbf{b}$, and the three possible outcomes	28
3.4 Elementary row operations and echelon forms	29
3.5 Gaussian elimination and back-substitution	30
3.6 The matrix inverse	32
3.7 Determinants	33
3.8 Cramer's rule	34
3.9 Worked example set	34
3.10 Problem set	35
3.11 Where this goes next	36
Solutions — Chapter 3: Linear systems, determinants, inverses	36
4. Vector spaces and the four fundamental subspaces	39
4.1 What is a vector space?	39
4.2 Subspaces: spaces inside spaces	39
4.3 Span: everything a set can reach	40
4.4 Linear independence and dependence	40
4.5 Basis and dimension	41
4.6 The column space $C(A)$: where $Ax = \mathbf{b}$ can land	41
4.7 The null space $N(A)$: everything A kills	42
4.8 Reading bases off the RREF: rank and nullity	42
4.9 The other two: row space and left null space	43
4.10 The big payoff: the three outcomes of $Ax = \mathbf{b}$, explained	45
Problem set	45
Where this goes next	45
Solutions — Chapter 4. Vector spaces and the four fundamental subspaces	46
5. Orthogonality, projections, least squares	49
5.1 What this chapter is for	49
5.2 Orthogonal vectors	49
5.3 Orthogonal subspaces	50
5.4 Orthonormal sets, and the $Q^T Q = I$ fact	50
5.5 Why project? The inconsistent case	51
5.6 Projection onto a line	51
5.7 Projection onto a subspace	52
5.8 Least squares: the two routes to the normal equations	54
5.9 Worked example set	55
5.10 Problem set	56
5.11 Where this goes next	57
Solutions — Chapter 5. Orthogonality, projections, least squares	57
6. Eigenvalues, eigenvectors, diagonalization	59

6.1 What this chapter is for	59
6.2 The definition	59
6.3 How to find them: the characteristic equation	60
6.4 Worked example: eigenvalues and eigenvectors of a 2×2	61
6.5 A useful shift (and a warning)	62
6.6 Eigenspaces, and why distinct eigenvalues give independence	62
6.7 Diagonalization: $S^{-1}AS = \Lambda$	62
6.8 Powers become easy: $A^k = S\Lambda^kS^{-1}$	64
6.9 Case study: the Fibonacci sequence via diagonalization	64
6.10 When diagonalization fails: defective matrices	65
6.11 Symmetric matrices: the spectral theorem	65
6.12 Why ML cares	66
6.13 Where this goes next	66
Problem set	67
Solutions — Chapter 6. Eigenvalues, eigenvectors, diagonalization	67
7. SVD and positive definite matrices	71
7.1 Why eigendecomposition is not enough	71
7.2 The SVD, stated	71
7.3 Where U , Σ , V come from: the A^TA bridge	71
7.4 Worked example: a square but non-diagonalizable 2×2 matrix	72
7.5 Worked example: a rectangular 2×3 matrix	73
7.6 Geometry of the SVD	74
7.7 Why SVD matters in ML	75
7.8 Positive definiteness: from quadratics to matrices	75
7.9 Positive definite matrices	76
7.10 Bowls, minima, and a look ahead at optimization	77
7.11 Where this goes next	77
Problem set	77
Chapter 7 — Solutions	78
8. Single-variable calculus: continuity, differentiability, derivatives	81
8.1 Limits: the “approach” idea	81
8.1.1 The ε - δ definition, stated carefully	81
8.1.2 One-sided limits	82
8.2 Continuity: no jumps	82
8.2.1 Continuity is preserved by the usual operations	83
8.2.2 The intermediate value theorem (one paragraph)	83
8.3 The derivative: how fast is it changing?	83
8.3.1 Definition: a limit of difference quotients	83
8.3.2 The geometric twin: slope of the tangent line	84
8.4 Differentiable $\Rightarrow$ continuous (but not conversely)	85
8.5 The derivative rules: a toolbox	85
8.6 Linear approximation: replace the curve by its tangent	87
8.7 Finding maxima and minima	87
8.7.1 Increasing, decreasing, and turning points	87
8.7.2 Critical points and the second derivative test	88
8.7.3 Closed intervals and global extrema	88
8.8 Where this goes next	89
Problem set	89
Chapter 8 — Solutions	89
Solution 1	89
Solution 2	89
Solution 3	90

Solution 4	90
Solution 5	90
Solution 6	90
Solution 7	90
Solution 8	91
Solution 9	91
Solution 10	91
Solution 11	91
Solution 12	91
9. Multivariable calculus: partial derivatives, gradients, Taylor series	93
9.1 Partial derivatives: freeze everything but one variable	93
9.2 The gradient: all partials packaged into one vector	94
9.3 Directional derivatives: rate of change along any direction	95
9.4 The gradient is the direction of steepest ascent	95
9.5 The gradient is perpendicular to the level sets	96
9.6 Linear (first-order) approximation and the tangent plane	97
9.7 The multivariable chain rule: differentiating through a moving point	98
9.8 Taylor series: adding the quadratic term (the Hessian)	99
9.9 Critical points: where the gradient vanishes	99
9.10 Where this goes next	100
Problem set	100
Solutions — Chapter 9: Multivariable calculus	100
Solution 1	100
Solution 2	101
Solution 3	101
Solution 4	101
Solution 5	102
Solution 6	102
Solution 7	102
Solution 8	102
Solution 9	102
Solution 10	103
Solution 11	103
Solution 12	103
10. Unconstrained optimization and gradient descent	105
10.1 The problem: minimize, with no strings attached	105
10.2 The school method, and where it breaks	105
10.3 The iterative idea: pick a direction, take a step, repeat	106
10.4 The step-size problem: too big oscillates, too small stalls	106
10.5 The gradient descent algorithm (one dimension)	107
10.6 Why the negative derivative: the Taylor argument	108
10.7 The multivariate version: replace the derivative by the gradient	108
10.8 Optimality conditions: the gradient finds suspects, the Hessian interrogates them	110
10.9 What gradient descent promises — and what it does not	111
10.10 This is training — and where this goes next	112
Problem set	112
Solutions — Chapter 10: Unconstrained optimization and gradient descent	113
Solution 1	113
Solution 2	113
Solution 3	113
Solution 4	113
Solution 5	114

Solution 6	114
Solution 7	114
Solution 8	115
Solution 9	115
Solution 10	115
Solution 11	115
11. Constrained optimization: Lagrange multipliers, projected gradient descent	117
11.1 The problem: minimize, with strings attached	117
11.2 The optimality check: no descent direction is feasible	117
11.3 The Lagrangian and the first-order conditions (equality constraints)	118
11.4 eg 2 — warm-up: closest point on a line (full steps)	119
11.5 eg 3 — the lecture’s example: on the unit circle (full steps)	119
11.6 The geometry: level sets tangent, gradients parallel	119
11.7 Inequality constraints: active or not, and complementary slackness	120
11.8 Projected gradient descent: step, then snap back	121
11.9 Why ML cares	122
11.10 Where this goes next	122
Problem set	122
Solutions — Chapter 11: Constrained optimization	123
Problem 1	123
Problem 2	123
Problem 3	123
Problem 4	124
Problem 5	124
Problem 6	124
Problem 7	124
Problem 8	124
Problem 9	125
Problem 10	125
Problem 11	125
12. Convex sets and convex functions	127
12.1 Convex sets: the line-segment test	127
12.2 Examples and non-examples, with verdicts	127
12.3 Hyperplanes and half-spaces are convex	128
12.4 The intersection property: convexity survives $\cap$	129
12.5 Convex combinations and the convex hull	129
12.6 Convex functions I: epigraph and the chord inequality	129
12.7 Convex functions II: first-order and second-order tests	130
12.8 The gallery: who is convex?	131
12.9 THE theorem: every local minimum is a global minimum	131
12.10 First-order optimality: $\nabla f(\mathbf{x}^*) = \mathbf{0}$ is necessary <i>and</i> sufficient	132
12.11 Building bigger convex functions: sums and compositions	132
12.12 Why ML cares	132
12.13 Where this goes next	133
Problem set	133
Solutions — Chapter 12. Convex sets and convex functions	134
13. Duality and KKT conditions	137
13.1 The problem, revisited	137
13.2 The Lagrangian, made formal	137
13.3 The primal problem: maxing over λ is the constraint	138
13.4 Swap the min and the max: the dual function and the dual problem	138

13.5 The dual function is concave (the lecturer's exercise)	139
13.6 Weak duality: the dual always under-promises ($d^* \leq p^*$)	139
13.7 Strong duality: for convex problems the gap closes ($d^* = p^*$)	140
13.8 The KKT conditions, derived from strong duality	140
13.9 The general KKT form: m inequalities, n equalities	141
13.10 Necessary, sufficient — and the certificate	142
13.11 Reading complementary slackness: active vs. inactive	142
13.12 eg 1 — the tutorial's LP, solved end-to-end with KKT (full steps)	142
13.13 eg 2 — a quadratic with an inequality constraint, primal <i>and</i> dual (full steps)	143
13.14 eg 3 — the diet problem: a primal/dual pair from the tutorial (full steps)	143
13.15 Why this matters: SVMs, and the Part I closing shot	144
Problem set	144
Solutions — 13. Duality and KKT conditions	145

Part II — Probability and Statistics 147

14. Probability basics: experiments, sample spaces, Bayes	149
14.1 Experiments and outcomes	149
14.2 The sample space	149
14.3 Events: subsets of the sample space	150
14.4 Combining events: union, intersection, complement	150
14.5 Disjoint events and partitions	150
14.6 The probability function and its three axioms	151
14.7 Consequences of the axioms	151
14.8 Equally likely outcomes and counting	152
14.9 Conditional probability	153
14.10 The law of total probability	153
14.11 Bayes' theorem	154
14.12 Independence	155
14.13 Repeated independent trials	156
14.14 Why this matters: the language every later chapter speaks	156
Problem set	157
Solutions — 14. Probability basics: experiments, sample spaces, Bayes	157
15. Discrete random variables and key distributions	161
15.1 Random variables: functions on the sample space	161
15.2 Range of a random variable	162
15.3 Discrete random variables	162
15.4 The PMF: probability mass function	162
15.5 The CDF for discrete random variables	163
15.6 Functions of a random variable	164
15.7 Expectation	164
15.8 Linearity of expectation	165
15.9 Variance and standard deviation	165
15.10 The uniform distribution	166
15.11 The Bernoulli distribution	167
15.12 The binomial distribution	168
15.13 The geometric distribution	168
15.14 The negative binomial distribution	169
15.15 The Poisson distribution	170
15.16 The hypergeometric distribution	170
15.17 Why this matters: the language every later chapter speaks	171
Problem set	171

Solutions — 15. Discrete random variables and key distributions	172
16. Continuous random variables	175
16.1 Continuous random variables	175
16.2 Why the PMF breaks: $P(X = x) = 0$	175
16.3 The PDF: probability density function	176
16.4 The CDF for continuous random variables	177
16.5 Expectation	177
16.6 Variance and standard deviation	178
16.7 The uniform distribution	179
16.8 The exponential distribution	179
16.9 The normal distribution	180
16.10 Functions of a continuous random variable	181
16.11 Why this matters: the same language, a bigger world	182
Problem set	183
Solutions — 16. Continuous random variables	183
17. Joint distributions: two random variables	185
17.1 Two random variables, one experiment	185
17.2 The joint PMF	185
17.3 Marginal PMFs: summing out	186
17.4 Conditional PMFs and the multiplication rule	187
17.5 Independence of random variables	188
17.6 The joint CDF	189
17.7 Expectation of functions of two random variables	189
17.8 The distribution of $g(X, Y)$: pooling over contours	190
17.9 Covariance	190
17.10 The correlation coefficient	191
17.11 Variance of a sum	192
17.12 The joint PDF	192
17.13 Marginal PDFs: integrating out	193
17.14 Conditional PDFs	194
17.15 Why this matters: the two-variable toolkit	194
Problem set	195
Solutions — 17. Joint distributions: two random variables	196
18. Joint continuous distributions	199
18.1 The continuous two-variable toolkit (recap)	199
18.2 The CDF method for $Z = g(X, Y)$	199
18.3 Convolution: the distribution of a sum	201
18.4 Max and min of independent variables	202
18.5 The Jacobian method: transforming two variables	203
18.6 The joint moment generating function	204
18.7 Why this matters: the two-variable machine	205
Problem set	205
Solutions — 18. Joint continuous distributions	206
19. Random vectors and the multivariate normal	209
19.1 Random vectors: the mean vector and the covariance matrix	209
19.2 Affine transformations: $\mathbf{Y} = \mathbf{AX} + \mathbf{b}$	210
19.3 The standard normal vector	211
19.4 The bivariate normal: the source's construction	211
19.5 Marginals and conditionals of the bivariate normal	212
19.6 The general multivariate normal: definition	213
19.7 Affine maps stay normal: properties	214

19.8 Marginal and conditional distributions in d dimensions	215
19.9 Why this matters: the normal workhorse	216
Problem set	216
Solutions — Chapter 19. Random vectors and the multivariate normal	216
Problem 1	216
Problem 2	217
Problem 3	217
Problem 4	217
Problem 5	218
Problem 6	218
Problem 7	218
Problem 8	218
Problem 9	219
Problem 10	219
20. Estimation: MLE and Bayesian/MAP	221
20.1 The estimation setup	221
20.2 Likelihood, log-likelihood, negative log-likelihood	221
20.3 Maximum likelihood estimation	222
20.4 Worked: Bernoulli bias	223
20.5 Worked: Uniform $[a, b]$	223
20.6 Worked: Normal — mean, then mean and variance	224
20.7 Estimating μ and Σ of a multivariate normal	225
20.8 ML for linear regression with Gaussian noise	225
20.9 The Bayesian setup: prior, likelihood, posterior	225
20.10 Point estimates: posterior mean and MAP	226
20.11 Worked: Beta prior + Bernoulli data (the conjugate pair)	226
20.12 Why this matters: the two workhorses of ML	228
Problem set	228
Solutions — 20. Estimation: MLE and Bayesian/MAP	229
21. Inequalities and the central limit theorem	231
21.1 The two questions	231
21.2 Markov's inequality	231
21.3 Chebyshev's inequality	232
21.4 Two ways a sequence of random variables can converge	233
21.5 The weak law of large numbers	234
21.6 The central limit theorem	234
21.7 Worked: the deck's uniform demo	235
21.8 Worked: CLT approximation for a binomial	236
21.9 Why this matters: the normal's ubiquity, and the Part II closing shot	237
Problem set	237
Solutions — Chapter 21: Inequalities and the central limit theorem	238
Part III — Machine Learning Foundations	241
22. What is ML: data, models, tasks; supervised vs unsupervised	243
22.1 What is machine learning?	243
22.2 The task hierarchy: manual labour $\rightarrow$ programming $\rightarrow$ machine learning	243
22.3 Data: a collection of vectors	245
22.4 Models: mathematical simplifications of reality	245
22.5 Learning algorithms and the ML task, precisely	246
22.6 Notation for the rest of Part III	247

22.7 Supervised learning: curve-fitting with answers	247
22.8 Regression: predicting a real number	247
22.9 Classification: predicting a discrete label	249
22.10 Training loss is not the goal: test data	251
22.11 Unsupervised learning: understanding data, no labels	251
22.12 Dimensionality reduction: compress, then reconstruct	252
22.13 Density estimation: scoring reality with probabilities	254
22.14 How it all connects — and what comes next	255
Problem set	255
Solutions — Chapter 22. What is ML: data, models, tasks; supervised vs unsupervised	256
Problem 1	256
Problem 2	256
Problem 3	257
Problem 4	257
Problem 5	257
Problem 6	257
Problem 7	258
Problem 8	258
Problem 9	258
Problem 10	258
23. Linear and polynomial regression	259
23.1 The setup: data, model, loss	259
23.2 The feature matrix	260
23.3 Deriving the normal equations (the calculus route)	260
23.4 The same equations from projections	261
23.5 The closed-form solution and the full-rank condition	261
23.6 Why squared loss is right: the Gaussian-noise story	261
23.7 Worked example set	262
23.8 Polynomial regression: curved fits with the same machine	264
23.9 More capacity is not always better: overfitting, and the validation discipline	265
23.10 The lecture's remark: ridge regularization (a pointer)	266
23.11 Where this goes next	266
Problem set	266
Solutions — Chapter 23: Linear and polynomial regression	267
24. PCA: variance maximization, eigendecomposition, kernel PCA	271
24.1 The problem: the best m -dimensional home for the data	271
24.2 The covariance matrix C — the only matrix that matters	271
24.3 First viewpoint: minimizing the reconstruction error	272
24.4 Second viewpoint: maximizing the variance of the projection	273
24.5 The PCA algorithm: keep the top m , drop the bottom $d - m$	273
24.6 Worked example set	274
24.7 The encoder-decoder reading: the optimal linear pair	275
24.8 How many components? Reading the eigenvalues	276
24.9 PCA in high dimensions: the $n \times n$ trick	276
24.10 Kernel PCA: when the structure isn't linear	277
24.11 Where this goes next	280
Problem set	280
Solutions — Chapter 24. PCA	281
Problem 1	281
Problem 2	281
Problem 3	281
Problem 4	281

Problem 5	282
Problem 6	282
Problem 7	282
Problem 8	282
Problem 9	282
Problem 10	283
25. GMMs and the EM algorithm	285
25.1 The other answer to “understand the data”	285
25.2 The motivating example: three types of students	285
25.3 The Gaussian mixture model	286
25.4 First caution: the component labels are meaningless	287
25.5 The latent-variable view: naming the hidden crowd	287
25.6 The MLE question — and why it is ugly	288
25.7 The chicken-and-egg insight	288
25.8 The E-step: responsibilities	289
25.9 The M-step: responsibility-weighted MLE	289
25.10 The EM algorithm: alternate until convergence	290
25.11 Worked: two EM iterations by hand	291
25.12 Where this goes next — and the close of Part III	292
Problem set	293
Solutions — Chapter 25: GMMs and the EM algorithm	293
Part IV — Machine Learning Techniques	297
26. K-means clustering	299
26.1 The clustering task: grouping unlabelled data	299
26.2 How many partitions? Counting the assignments	299
26.3 The k-means objective: within-cluster sum of squares	300
26.4 Lloyd’s algorithm: assign, re-mean, repeat	301
26.5 Worked: one Lloyd iteration by hand	302
26.6 The shape of the clusters: half-spaces and bisectors	303
26.7 Initialization: uniform vs k-means++	304
26.8 Choosing K : the lecture’s penalty idea	304
26.9 The hard-assignment cousin of GMM/EM	305
26.10 Cautions from the practice bench	305
26.11 Where this goes next	306
Problem set	306
Solutions — Chapter 26: K-means clustering	306
Problem 1 — Objective arithmetic	306
Problem 2 — One Lloyd step	307
Problem 3 — Monotonicity, proved	307
Problem 4 — Bisector geometry	307
Problem 5 — k-means++ scoring	307
Problem 6 — Choosing K	308
Problem 7 — True or false	308
Problem 8 — Hard vs soft	308
27. Kernel methods and kernel regression	309
27.1 Notation	309
27.2 The kernel machinery, briefly recapped	309
27.3 Motivation: linear models are limited	310
27.4 Ridge regression in feature space: the objective	310

27.5 The dual form: the solution lives in the span of the data	310
27.6 The closed-form solution	311
27.7 What it means: prediction as a weighted sum of similarities	311
27.8 Worked example: kernel ridge by hand	312
27.9 Practical notes: the kernel, λ , and the $O(n^3)$ bill	313
27.10 Where this goes next	313
Problem set	313
Solutions — Chapter 27: Kernel methods and kernel regression	314
Problem 1 — Cubic-kernel ϕ , by hand	314
Problem 2 — Guided derivation	314
Problem 3 — Kernel ridge numeric	315
Problem 4 — Interpolation at $\lambda = 0$	315
Problem 5 — Polynomial kernel = polynomial regression	316
Problem 6 — RBF σ , qualitatively	316
Problem 7 — True or false	316
Problem 8 — Cost accounting	316
28. Regularization: ridge and lasso	317
28.1 The problem, restated: overfitting wears big weights	317
28.2 The fix: charge rent on big weights	317
28.3 Ridge regression: the ℓ_2 penalty	318
28.4 The ridge solution, by hand	318
28.5 Gradient descent for ridge	319
28.6 What λ does: shrinkage, then underfitting	319
28.7 Lasso: the ℓ_1 penalty, and exact zeros	320
28.8 Ridge vs lasso, side by side	321
28.9 The bias–variance trade-off, tied to λ	321
28.10 Choosing λ : the validation discipline	322
28.11 Worked example: one tiny dataset, three fits	323
28.12 Where this goes next	324
Problem set	325
Solutions — Chapter 28: Regularization: ridge and lasso	325
Problem 1 — Ridge gradient, by hand	325
Problem 2 — The two endpoints	326
Problem 3 — Soft-thresholding in 1-D	326
Problem 4 — One GD step vs the closed form	326
Problem 5 — Read a λ table	326
Problem 6 — Why the diamond zeroes and the ball doesn't	326
Problem 7 — sklearn practice (MLP slides)	327
29. KNN and decision trees	329
29.1 The task both solve: §22.9 classification	329
29.2 KNN: the algorithm — no model, just neighbours	329
29.3 The distance metric: what “near” means	330
29.4 Voting (classification) and averaging (regression)	330
29.5 Worked: 3-NN on seven 2-D points	331
29.6 Decision boundaries: what k does to the shape	332
29.7 Choosing k : §28.10's discipline, reused	332
29.8 KNN: strengths and limits (as the lecture lists them)	332
29.9 Decision trees: recursive partitioning	333
29.10 ID3: the greedy top-down algorithm	333
29.11 Measuring impurity: how mixed is this node?	334
29.12 Information gain: the splitting score	334
29.13 Worked: one full split computation by hand	335

29.14 The lecture's tennis example (numbers verified)	335
29.15 CART: regression trees — constants per region	337
29.16 Classification trees: swap the loss, keep the growing	337
29.17 Overfitting: deep trees memorize	338
29.18 Pre-pruning: stop before it overgrows	339
29.19 Post-pruning: grow big, then cut back (cost-complexity)	339
29.20 KNN vs decision trees	339
29.21 Trees: advantages and disadvantages (as the lecture lists them)	340
29.22 Where this goes next	340
Problem set	341
Solutions — Chapter 29: KNN and decision trees	341
Problem 1 — 3-NN vote, by hand ($q_2 = (4, 2)$)	341
Problem 2 — The metric can flip the answer	342
Problem 3 — KNN regression	342
Problem 4 — The effect of k	342
Problem 5 — Verify the lecture's Wind number	342
Problem 6 — Finish the §29.13 comparison	342
Problem 7 — Why Gini beats misclassification for growing	343
Problem 8 — Cost-complexity arithmetic	343
30. Naive Bayes: generative vs discriminative models	345
30.1 Two philosophies: model $P(y x)$, or model $P(x, y)$	345
30.2 Bayes' rule for classification	346
30.3 The Naive Bayes assumption	346
30.4 Modelling $p(x_j y_c)$: one distribution per feature type	347
30.5 Inference: decide in log space	348
30.6 Training = maximizing the log-likelihood	348
30.7 The MLE estimates: training is counting	349
30.8 The zero-count problem and Laplace smoothing	349
30.9 Worked example: six emails, three words (every number recomputed)	350
30.10 Worked example: Gaussian NB on one continuous feature	351
30.11 Strengths and limits (as the sources list them)	352
30.12 Where this goes next	352
Problem set	353
Solutions — Chapter 30: Naive Bayes: generative vs discriminative models	353
Problem 1 — Classify $[0, 0, 1]$ (§30.9's data, no smoothing)	353
Problem 2 — Parameter counting ($k = 4$, $d = 5$ binary features)	354
Problem 3 — Laplace smoothing on $x = [0, 1, 1]$	354
Problem 4 — Spam posterior (practice-assignment numbers)	354
Problem 5 — Gaussian NB boundary	354
Problem 6 — Generative or discriminative?	355
Problem 7 — Why log space	355
31. Perceptron and logistic regression	357
31.1 The shared shape: a linear boundary, and what its parts mean	357
31.2 The perceptron: a neuron that learns from mistakes	358
31.3 The perceptron loss: pay only for mistakes	358
31.4 The perceptron update rule: learn only when wrong	359
31.5 Convergence: guaranteed on separable data, hopeless otherwise	359
31.6 Worked example: three points, two updates (every number recomputed)	360
31.7 Why probabilities? The perceptron's blind spot	361
31.8 Logistic regression: the model	362
31.9 The loss: cross-entropy is the negative log-likelihood	363
31.10 The gradient, and training by gradient descent	364

31.11 Worked example: one gradient-descent step (every number recomputed)	365
31.12 Multiclass: softmax	365
31.13 Generative vs discriminative, revisited: Naive Bayes against logistic regression	366
31.14 Strengths and limits (as the sources list them)	366
31.15 Where this goes next	366
Problem set	367
Solutions — Chapter 31: Perceptron and logistic regression	367
Problem 1 — Perceptron by hand	367
Problem 2 — Mistake bound	368
Problem 3 — Flip the labels	368
Problem 4 — Sigmoid arithmetic	368
Problem 5 — One GD step, new numbers	368
Problem 6 — Same wall, two derivations	368
Problem 7 — Count the knobs	369
Chapter 32: Hard-margin SVM	371
32.1 The problem with “any” wall: which separator?	371
32.2 The margin: functional, geometric, and the canonical form	371
32.3 The hard-margin primal: from “widest street” to $\min \frac{1}{2} \ w\ ^2$	372
32.4 The dual: same problem, mirrored	373
32.5 KKT and support vectors: only the wall-huggers vote	373
32.6 Worked example: three points, the dual solved by hand	375
32.7 Strengths and limits (as the sources list them)	376
32.8 Where this goes next	377
Problem set	377
Solutions — Chapter 32: Hard-margin SVM	377
Problem 1 — Width and supporting lines	377
Problem 2 — Scaling, canonical and otherwise	378
Problem 3 — The symmetric square	378
Problem 4 — Complementary slackness, both directions	378
Problem 5 — Delete the bystander	378
Problem 6 — Three linear classifiers	379
Chapter 33: Soft-margin SVM and the kernel trick	381
33.1 The lecture’s closing question	381
33.2 Slack variables: the bribe	381
33.3 The soft-margin primal	382
33.4 The soft-margin dual: the box constraint	382
33.5 Worked example: one outlier, two values of C	383
33.6 The kernel trick for SVMs	384
33.7 Worked example: quadratic kernel on three points	385
33.8 The pattern, third time	386
33.9 Strengths and limits (as the sources list them)	387
33.10 Where this goes next	387
Problem set	388
Solutions — Chapter 33: Soft-margin SVM and the kernel trick	388
Problem 1 — Reading the slack	388
Problem 2 — The box, derived	388
Problem 3 — Three α regimes	389
Problem 4 — The C endpoints	389
Problem 5 — Gram matrix and Mercer	390
Problem 6 — A two-point kernel dual	390
Problem 7 — True or false	390

Chapter 34: Ensembles — bagging, boosting, AdaBoost	391
34.1 The ensemble idea — the answer to trees’ variance problem	391
34.2 Bagging: bootstrap aggregation	391
34.3 Bagging needs disagreement — the diversity condition	393
34.4 Out-of-bag error: a free test-error estimate	394
34.5 Boosting: the sequential idea — fix mistakes, don’t average them away	394
34.6 AdaBoost: adaptive boosting, the algorithm	395
34.7 Hand-worked: three AdaBoost rounds on eight points	395
34.8 Bagging vs boosting — the sources’ contrast	397
34.9 Strengths and limits — as the sources list them	397
34.10 Where this goes next	398
Problem set	398
Solutions — Chapter 34: Ensembles — bagging, boosting, AdaBoost	398
Problem 1 — Vote arithmetic	398
Problem 2 — OOB counting	399
Problem 3 — AdaBoost by hand (Q5-style)	399
Problem 4 — Reading α	399
Problem 5 — Bagging vs boosting, sourced	399
Problem 6 — The m -knob	399
Chapter 35: Loss functions for classification	401
35.1 The ideal loss — and why it’s unusable	401
35.2 One yardstick: the margin	402
35.3 Zero-one loss: the ideal, restated in margin form	402
35.4 Hinge loss: the SVM’s bribe	402
35.5 Squared loss: regression doing classification	403
35.6 Logistic loss: cross-entropy in margin form	403
35.7 Exponential loss: AdaBoost’s engine	404
35.8 The perceptron’s loss: hinge without the margin	405
35.9 Surrogate losses: the trade	405
35.10 The big comparison: one dataset, five losses	406
35.11 Why AdaBoost’s weights update the way they do	407
35.12 Strengths and limits (as the sources list them)	408
35.13 Where this goes next	409
Problem set	409
Solutions — Chapter 35: Loss functions for classification	410
Problem 1 — Margin and 0/1	410
Problem 2 — Hinge’s margin demand	410
Problem 3 — Squared loss’s confidence tax	410
Problem 4 — Logistic, two faces	410
Problem 5 — Perceptron = modified-hinge SGD	410
Problem 6 — AdaBoost weights from exponential loss	411
Problem 7 — True or false	411
Part V — Machine Learning in Practice	413
Chapter 36: NumPy and Pandas for ML	415
36.1 The ndarray: the book’s data structure	415
36.2 Vectorized ops vs Python loops	416
36.3 Broadcasting: the silent shape rule	416
36.4 Indexing, slicing, and the axis semantics	417
36.5 Reductions, sorting, stacking, comparing	417
36.6 Random numbers: seeded and reproducible	418

36.7 pandas: the labeled table	418
36.8 Worked end-to-end: load, clean, describe, extract	419
36.9 Where this goes next	420
Problem set	420
Solutions — Chapter 36: NumPy and Pandas for ML	421
Problem 1 — Broadcast or break	421
Problem 2 — Reproduce the speedup	421
Problem 3 — Read the slice	421
Problem 4 — Axis by hand	422
Problem 5 — Clean the table	422
Problem 6 — Seeds	422
Problem 7 — Course-sourced	423
Chapter 37: scikit-learn: regression workflows	425
37.1 The estimator contract: <code>fit</code> , <code>predict</code> , <code>score</code>	425
37.2 The workflow: split first, then fit	426
37.3 Two engines for the same line	426
37.4 Model inspection: reading the weights back out	427
37.6 Polynomial regression: expand the features, keep the machine	428
37.7 Regularization in sklearn: the <code>alpha</code> dial	430
37.8 Transformers and pipelines: learn on train, apply everywhere	431
37.9 Choosing the dial: validation, briefly	432
37.10 Where this goes next	432
Problem set	433
Solutions — Chapter 37: scikit-learn: regression workflows	433
Problem 1 — The contract	433
Problem 2 — R^2 by hand	434
Problem 3 — Pick the metric	434
Problem 4 — Course-sourced	434
Problem 5 — Score or error?	434
Problem 6 — Spot the leak	434
Problem 7 — The <code>alpha</code> translation	435
Problem 8 — Diagnose the fit	435
Chapter 38: scikit-learn: classification workflows	437
38.1 The classifier contract: <code>fit</code> , <code>predict</code> , <code>decision_function</code> , <code>predict_proba</code> , <code>score</code>	437
38.2 The zoo, as the course presents it	438
38.3 Least-squares classification: <code>RidgeClassifier</code>	438
38.4 The perceptron, as code	439
38.6 <code>SGDClassifier</code> : one engine, many classifiers	441
38.7 Multiclass, multilabel, multioutput	442
38.9 Thresholds, imbalance, and multiclass metrics	445
38.10 The full classification pipeline: three course experiments	445
38.11 Where this goes next	446
Problem set	447
Solutions — Chapter 38: scikit-learn: classification workflows	447
Problem 1 — The contract	447
Problem 2 — Confusion matrix by hand	448
Problem 3 — The <code>C</code> translation	448
Problem 4 — Course-sourced (<code>LabelBinarizer</code>)	448
Problem 5 — Threshold moving	448
Problem 6 — Spot the leak	449
Problem 7 — OVR vs OVO	449
Problem 8 — Diagnose the fit	449

Chapter 39: Model evaluation, cross-validation, hyperparameter tuning	451
39.1 One test split is a noisy number	451
39.2 Cross-validation: the robust estimate	452
39.3 <code>cross_val_score</code> vs <code>cross_validate</code> : what they return	454
39.4 Learning curves: §23.9 as code	455
39.5 Stratified iterators: the classification addition	456
39.6 The three-way split doctrine: train / validation / test	457
39.7 <code>GridSearchCV</code> vs <code>RandomizedSearchCV</code>	458
39.8 Model-specific HPT: <code>RidgeCV</code> , <code>LassoCV</code> , <code>ElasticNetCV</code> , <code>LogisticRegressionCV</code>	459
39.9 The pipeline + search combination: leak-proof HPT	460
39.10 Where this goes next	461
Problem set	461
Solutions — Chapter 39: Model evaluation, cross-validation, hyperparameter tuning	462
 Chapter 40: End-to-end model training and debugging	 467
40.1 The end-to-end runbook: the course’s eight steps	467
40.2 The debugging playbook: symptom → diagnosis → fix	469
40.3 Reading the instruments	471
40.4 The “suspiciously good” checklist (the book’s)	471
40.5 Where this goes next	472
Problem set	472
Chapter 40 solutions: End-to-end model training and debugging	473
1. The eight steps	473
2. The broadcast that tells on you	473
3. The shuffle case	474
4. Spot the leak	474
5. Read the dashboard	474
6. Search arithmetic	474
7. Spend the test set once	475
8. Error analysis, the deck’s way	475
9. The suspiciously-good audit	475
 Part VI — Bridge to Deep Learning	 477
 Chapter 41: The artificial neuron and MLPs from scratch	 479
41.1 The neuron’s two steps	479
41.2 The lineage: a calculator, a learner, and a neuron	479
41.3 Why smooth matters: differentiability → gradient descent	480
41.4 From one neuron to a layer: the two vectorized equations	480
41.5 Worked example: one layer, every number recomputed	481
41.6 Why depth needs non-linearity	482
41.7 Activations compared	482
41.8 XOR: why no linear model can do it	483
41.9 The 2-2-1 network, worked by hand	484
41.10 The training loop in numpy: the whole MLP, re-run	485
41.11 The output layer + loss pairing	487
41.12 The contract survives: sklearn’s <code>MLPClassifier</code> / <code>MLPRegressor</code>	487
41.13 PyTorch: tensors in, the same loop out	488
41.14 Four traps worth knowing (the course’s own warnings)	489
41.15 Where this goes next	490
Problem set	490
Chapter 41 solutions — The artificial neuron and MLPs from scratch	491

Chapter 42: Backpropagation, worked end-to-end	493
42.1 The question backprop answers	493
42.2 The computational graph: any expression is a graph of basic ops	493
42.3 The 2-2-1 backward pass, every number by hand	495
42.4 The general layer-wise rules (the deck's derivation)	496
42.5 The output-layer special case: the pairing cancellation	497
42.6 Gradient checking: trust, then verify	498
42.7 Why one backward pass beats perturbing every weight	499
42.8 Where this goes next	499
Problem set	499
Chapter 42 solutions — Backpropagation, worked end-to-end	500
Problem 1 — the deck's scalar graph, re-derived	500
Problem 2 — the three components	501
Problem 3 — batch-form translation	501
Problem 4 — the XOR backward pass, a second example	501
Problem 5 — the $(g - y)$ check	502
Problem 6 — softmax pairing	502
Problem 7 — gradient check by hand	502
Problem 8 — credit where it's due	503
Problem 9 — vanishing, read backwards	503
Problem 10 — map the notebook	503
Chapter 43: Optimization for neural nets — SGD variants, init, normalization	505
43.1 Why plain gradient descent needs help in deep nets	505
43.2 The three gradient descents: batch, stochastic, mini-batch	506
43.3 Momentum: remember where you were going	506
43.4 Nesterov: look before you leap (notebook-only)	507
43.5 Adaptive methods I: AdaGrad, RMSProp, AdaDelta	507
43.6 Adam: momentum + RMSProp, with bias correction	508
43.7 Weight decay: the λ that rides inside the optimizer	509
43.8 Learning-rate schedules: shrink the steps as you settle	510
43.9 Normalization: fix the scale before the optimizer has to	510
43.10 Initialization: where the weights start	511
43.11 The optimizers, traced: numpy on the deck's own problem	511
43.12 The practitioner's cheat sheet (notebook, [recorded])	513
43.13 Where this goes next	514
Problem set	514
Solutions — Chapter 43: Optimization for neural nets	515
1. The ravine, by hand	515
2. Velocity is a leaky average	516
3. Mini-batch noise	516
4. AdaGrad vs RMSProp, one step each	516
5. Adam's bias correction, with and without	516
6. Weight decay = L2, until Adam	516
7. Read the schedules	517
8. Init scale through the δ recursion	517
9. Scaling as preconditioning	517
10. Triage	517
Chapter 44: CNNs — convolution, pooling, transfer learning	519
44.1 Why dense layers fail on images	519
44.2 Convolution as local pattern matching	520
44.3 eg: a 2D convolution, worked by hand [verified-NumPy]	520
44.4 Channels, stride, padding: the shape bookkeeping	521

44.5 Pooling: downsampling with a purpose	522
44.6 The CNN stack: feature hierarchy, then a dense head	522
44.7 The parameter-sharing payoff	523
44.8 Transfer learning: don't reinvent the wheel	524
44.9 A second head for CNNs: segmentation (the U-Net note)	525
44.10 Where this goes next	525
Problem set	525
Chapter 44 solutions	526
Solution 1 — Parameter explosion, quantified	526
Solution 2 — The convolution, by hand	527
Solution 3 — Name the operation	527
Solution 4 — Stride and padding	527
Solution 5 — Trace the MNIST stack	528
Solution 6 — Pooling vs PCA	528
Solution 7 — Equivariance, then invariance	528
Solution 8 — Freeze vs fine-tune	528
Solution 9 — The transfer-learning head	529
Solution 10 — Design a small CNN	529
Chapter 45: RNNs — sequences and vanishing gradients	531
45.1 Sequence data: order matters	531
45.2 Before the RNN: Markov, windows, and why they break	532
45.3 The RNN: one cell, reused across time	532
45.4 eg: one forward step — next-character prediction [verified-NumPy]	533
45.5 BPTT: the same backprop, unrolled	534
45.6 eg: BPTT by hand on a 1-neuron net [verified-NumPy]	535
45.7 The price of sharing: vanishing and exploding gradients	535
45.8 LSTM: gates and the additive cell	536
45.9 GRU: two gates, no separate cell	537
45.10 Reading both ways, speaking at length	538
45.11 Judging sequences: perplexity, BLEU, beam search	539
45.12 The PyTorch face	540
45.13 Where this goes next	540
Problem set	541
Solutions — Chapter 45: RNNs — sequences and vanishing gradients	542
1. Factor and cut	542
2. One recurrent step, by hand	542
3. The sharing payoff, counted	543
4. The two-component recursion	543
5. Finish the BPTT eg	543
6. Name the regime	543
7. Clip vs truncate	543
8. The LSTM's gradient path	544
9. The GRU's mix	544
10. Generate and judge	544
Chapter 46: Attention and the transformer	545
46.1 Two promises this chapter cashes	545
46.2 Embeddings: turning words into vectors	546
46.3 Attention, in one formula: queries, keys, values	546
46.4 From kernels to the dot product	547
46.5 Why the $\sqrt{d_k}$: scaled dot-product attention	547
46.6 The matrix formula, by hand	548
46.7 Self-attention, cross-attention, and the Bahdanau original	548

46.8 The masked softmax	548
46.9 Multi-head attention	549
46.10 Order is gone — positional encoding	549
46.11 Training deep stacks: residuals and layer norm	550
46.12 The transformer, end to end	550
46.13 Three flavors: BERT, GPT, and the full stack	551
46.14 The Vision Transformer	552
46.15 The notebooks: a mini-GPT and pretrained models in one line	553
46.16 Where this goes next	553
Problem set	553
Solutions — Chapter 46: Attention and the transformer	554
1. One query, three keys	555
2. Why scaling matters	555
3. The Gaussian-to-dot-product derivation	555
4. Multi-head parameter count	555
5. The causal mask	555
6. Positional encoding by hand	556
7. Masked softmax	556
8. Patch math	556
9. Cross-attention shapes	556
10. The quadratic bill	556
Chapter 47: Generative models: GANs and diffusion	559
47.1 Two promises this chapter cashes	559
47.2 What a generative model is	560
47.3 The latent space: from enumeration to noise	560
47.4 The GAN: a two-player game	561
47.5 Training: alternate, freeze, repeat	561
47.6 The losses, worked by hand [verified-NumPy]	562
47.7 The notebook: a GAN on FashionMNIST, from scratch [recorded — torch]	562
47.8 Scaling up: DCGAN and StyleGAN [recorded]	563
47.9 When GANs break [recorded]	563
47.10 The diffusion pitch: many easy steps	564
47.11 The forward process: the math	564
47.12 eg: two steps of noise, one jump [verified-NumPy]	565
47.13 The reverse process: learning to denoise	565
47.14 The denoiser: a U-Net that knows what time it is	566
47.15 eg: a time embedding, by hand [verified-NumPy]	566
47.16 DDIM: sampling 20–100× faster	567
47.17 eg: one DDIM step, by hand [verified-NumPy]	567
47.18 Guidance: steering the denoiser	567
47.19 Latent diffusion: Stable Diffusion	568
47.20 Judging the forgeries: FID, IS, CLIP	569
47.21 Where this goes next: closing Part VI (and the book)	570
Problem set	570
Solutions — Chapter 47: Generative models: GANs and diffusion	572
1. The 2×2 marginal	572
2. GAN losses by hand	572
3. The notebook’s parameter counts	572
4. The Nash point	572
5. Forward diffusion numbers	572
6. The linear schedule’s endpoints	572
7. One DDIM step	573
8. FID by hand	573

9. Classifier-free guidance numbers	573
10. Inception Score by hand	573
11. The book-ender	573

Foundations to Deep Learning

From foundations to deep learning — a one-stop study book

This book takes you from mathematical foundations (linear algebra, calculus, probability) through classical machine learning to neural networks and transformers — everything needed before the core deep learning and AI courses.

How to study from this book

- i) Read a chapter front to back: concept first, then the worked **eg** blocks.
- ii) Hard topics carry a “**Basically, ...**” line — the same idea restated as simply as possible.
- iii) Do the **Practice set** at the end of each chapter *before* looking at solutions: on the web, each solution is hidden behind a click; in the PDF, solutions follow each chapter.

There is nowhere else you need to go. Links out appear only where the book itself could not explain something sufficiently.

Part I — Mathematical Foundations

Chapter 1 — Sets, functions, and mathematical preliminaries

1.1 What this chapter is for

Everything in this book is built out of two things: *sets* of numbers and *functions* that map one set to another.

- A dataset is a *set* of points, each point living in $\mathbb{R}^d$ (a d -dimensional vector).
- A model is a *function* $f : \mathbb{R}^d \rightarrow \mathbb{R}$: it takes a data point and returns a prediction.
- A loss is also a *function*: it takes the model's parameters and returns one real number measuring how bad the model is. Training a model means minimizing that function.
- The sentences in later chapters (“for every $\epsilon > 0$ there exists...”, “if f is differentiable then...”) are written in the *logic* of *quantifiers* and *implications* covered at the end of this chapter.

So this chapter collects the vocabulary the rest of the book speaks. Nothing here is decoration; every idea below is used again.

1.2 The number sets and intervals

Def. The standard *number sets* are:

$$\mathbb{N} = \{0, 1, 2, 3, \dots\} \quad (\text{natural numbers, includes } 0)$$

$$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\} \quad (\text{integers})$$

$$\mathbb{Q} = \left\{ \frac{p}{q} : p, q \in \mathbb{Z}, q \neq 0 \right\} \quad (\text{rational numbers})$$

$$\mathbb{R} = \text{all rationals plus irrationals like } \sqrt{2}, \pi \quad (\text{real numbers})$$

They nest inside each other:

$$\mathbb{N} \subseteq \mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R}.$$

Two more sets appear constantly in ML (this book follows the MLF convention):

- $\mathbb{R}_+ = \{x \in \mathbb{R} : x \geq 0\}$, the non-negative reals (note: 0 is included).
- $\mathbb{Z}_+ = \{0, 1, 2, \dots\}$, the non-negative integers.

Note: In this book $\mathbb{R}_+$ and $\mathbb{Z}_+$ include 0. Some other books exclude it; watch for that when reading elsewhere.

Intervals carve out pieces of $\mathbb{R}$:

- *Closed interval*: $[a, b] = \{x \in \mathbb{R} : a \leq x \leq b\}$ — endpoints included.
- *Open interval*: $(a, b) = \{x \in \mathbb{R} : a < x < b\}$ — endpoints excluded.

Square brackets $[]$ mean “include the endpoint”; round brackets $()$ mean “exclude it”.

Basically, ... $\mathbb{R}$ is the whole number line. An interval is just a segment of that line, and the bracket style tells you whether the cut points themselves are in the segment.

1.3 Sets: elements, cardinality, subsets

Def. A *set* = a collection of well-defined items. The items are called *elements*.

- Write $x \in S$ for “ x is an element of S ”.
- *Order* does not matter: $\{1, 2, 3\} = \{3, 1, 2\}$.
- *Duplicates* do not matter: $\{1, 1, 2\} = \{1, 2\}$.

Def. The *cardinality* of a set S , written $|S|$, = the number of elements in S .

- $S = \{1, 2, 5, 7, 9, 300\}$ has $|S| = 6$.
- $A = \{\text{Srikanth, Keerthana, Balloon, Cell phone, } \pi\}$ has $|A| = 5$.

Def. X is a *subset* of Y , written $X \subseteq Y$, = every element of X is also an element of Y .

- $\{1, 2, 5\} \subseteq \{0, 1, 2, 5, 7\}$.
- $\mathbb{N} \subseteq \mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R}$.

Def. X is a *proper subset* of Y , written $X \subset Y$, = $X \subseteq Y$ but $X \neq Y$.

- $\{1, 2, 5\} \subset \{0, 1, 2, 5, 7\}$ (the second set has extra elements).
- $\{4, 16, 32, 64\}$ is **not** a proper subset of $\{64, 16, 32, 4\}$ — the two sets are equal.

Basically, ... A set is a bag of distinct things. A subset is a smaller bag whose contents all came from the bigger bag. “Proper” just means strictly smaller.

1.4 Building sets: set-builder notation

Instead of listing elements, describe the rule that picks them:

$$\{x \in S : \text{condition on } x\} = \text{“all } x \text{ in } S \text{ satisfying the condition”}.$$

- $\{x \in \mathbb{R} : a \leq x \leq b\}$ is the closed interval $[a, b]$.
- $\{x \in \mathbb{R}^d : x_i \in [a, b] \text{ for all } i = 1, \dots, d\}$ is the d -dimensional box $[a, b]^d$.

Read “:” or “|” as “such that”. This is how almost every set in this book is written, so get comfortable reading it left to right: “the set of all x in S such that ...”.

Basically, ... Listing $\{2, 4, 6, 8, \dots\}$ gets old fast. Set-builder notation says the recipe instead of the ingredients: “all x from here that pass this test”.

1.5 Set operations, Venn diagrams, De Morgan

Fix a *universe* U : the big set everything lives in. For sets $A, B \subseteq U$:

- *Union*: $A \cup B = \{x : x \in A \text{ or } x \in B\}$ — elements in either.
- *Intersection*: $A \cap B = \{x : x \in A \text{ and } x \in B\}$ — elements in both.
- *Complement*: $A^c = \{x \in U : x \notin A\}$ — everything in the universe except A . Also written $U \setminus A$.

- *Set difference*: $A \setminus B = \{x : x \in A \text{ and } x \notin B\}$.

A *Venn diagram* draws sets as overlapping regions inside a rectangle (the universe). Shading a region shows which set an expression describes. Two expressions that shade the same region are the same set — this is how you check identities visually.

The two big identities are *De Morgan's laws*:

$$(A \cup B)^c = A^c \cap B^c \quad \text{and} \quad (A \cap B)^c = A^c \cup B^c.$$

In words: “not in (either one)” is the same as “in neither one”; “not in (both)” is the same as “missing from at least one”. The Venn picture below shows the first law: both sides shade exactly the region outside both circles.

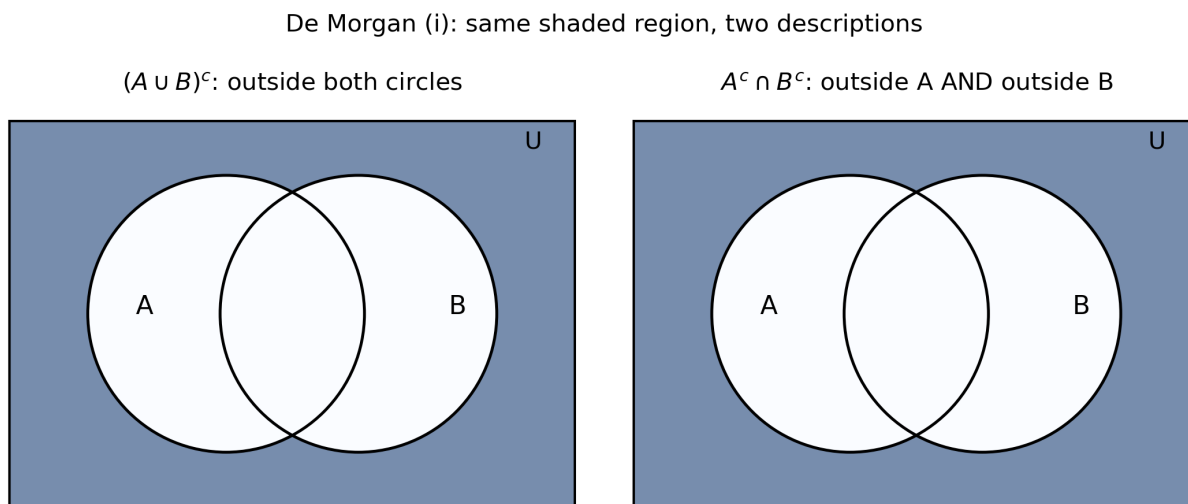


Figure 1: Venn diagrams showing De Morgan's law: $(A \cup B)^c$ and $A^c \cap B^c$ shade the same region

Basically, ... De Morgan is about flipping “and/or” when a “not” passes through. “NOT (tall AND dark)” = “short OR light”. Same flip works for sets.

1.6 Cartesian products and relations

Def. The *Cartesian product* of non-empty sets X and Y is

$$X \times Y = \{(x, y) : x \in X, y \in Y\},$$

the set of all *ordered pairs*. Order matters: $(a, 1) \neq (1, a)$.

- i) If $A = \{a, b\}$ and $B = \{1, 2, 3\}$, then $A \times B = \{(a, 1), (a, 2), (a, 3), (b, 1), (b, 2), (b, 3)\}$.
- ii) $\mathbb{R}^d = \underbrace{\mathbb{R} \times \cdots \times \mathbb{R}}_{d \text{ times}}$ is the set of d -dimensional vectors; e.g. $(1, 2, 3.3) \in \mathbb{R}^3$.

Def. A *relation* R between X and Y = any collection of ordered pairs with one element from each set, i.e. $R \subseteq X \times Y$.

- i) With A, B as above, $R_1 = \{(a, 1), (b, 2), (b, 3)\}$, $R_2 = \{(a, 2), (b, 1)\}$ are relations from A to B .

A relation is just “some arrows drawn from X to Y ” — possibly several arrows out of one element, possibly none. Functions (next section) are the special relations with exactly one arrow per element.

Basically, ... $X \times Y$ is every possible pairing of an x with a y — the full menu. A relation is whatever subset of that menu you actually choose.

Relations on a single set S (so $R \subseteq S \times S$) can have three useful *properties*:

- *Reflexive*: every element relates to itself: $(x, x) \in R$ for all $x \in S$.
- *Symmetric*: arrows go both ways: $(x, y) \in R \Rightarrow (y, x) \in R$.
- *Transitive*: arrows chain: $(x, y) \in R$ and $(y, z) \in R \Rightarrow (x, z) \in R$.

Def. An *equivalence relation* = a relation that is reflexive, symmetric, and transitive.

Basically, ... An equivalence relation is a mathematically strict way of saying “these things are the same for our purposes” — it groups elements into clumps where everything in a clump is mutually interchangeable. (Equality itself is the model example.)

1.7 Functions: the core definition

Def. A *function* $f : X \rightarrow Y$ = a rule assigning to each $x \in X$ exactly one $y \in Y$.

Three names, never mix them up:

- *Domain* = the set of allowed inputs (X).
- *Codomain* = the set of possible outputs (Y) — declared up front.
- *Range* = the outputs that actually occur: $\{f(x) : x \in X\}$ — always a subset of the codomain.

As a relation: a function is a relation with no two pairs sharing the same first element.

The *graph* of $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is the set $G_f = \{(x, f(x)) : x \in \mathbb{R}^d\}$ — input glued to its output. For $d = 1$ this is the familiar curve in the plane.

Vertical line test: a curve in the plane is the graph of a function iff every vertical line meets it at most once. (Two hits = one input with two outputs = not a function.)

Basically, ... A function is a machine with one input slot and one output slot: every input you are allowed to feed it produces exactly one output. Domain = what you may feed it; range = what can come out.

1.8 Three important kinds of functions

Def. Let $f : X \rightarrow Y$.

- *Injective (one-one)*: different inputs give different outputs. Formally, $f(x_1) = f(x_2) \Rightarrow x_1 = x_2$.
- *Surjective (onto)*: every $y \in Y$ gets hit, i.e. *range* = *codomain*.
- *Bijjective*: both injective and surjective — a perfect one-to-one pairing.

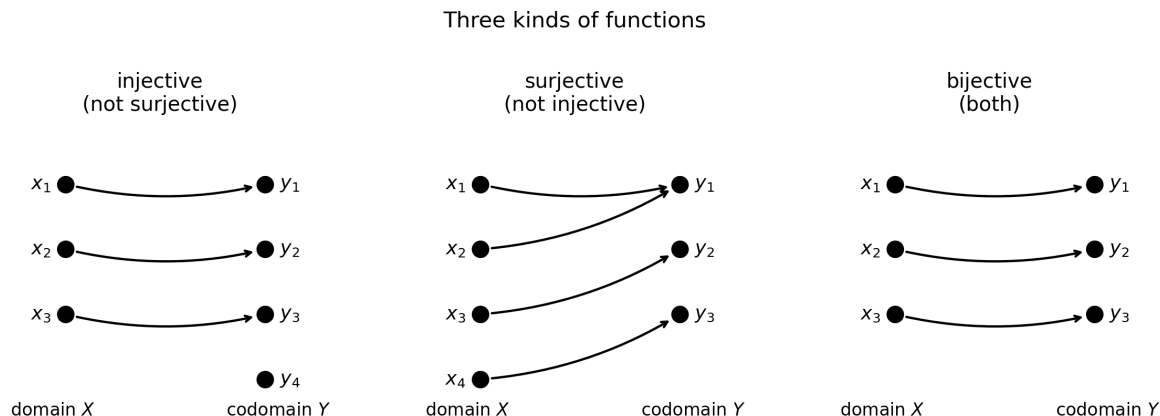


Figure 2: Arrow diagrams of injective, surjective, and bijective functions between finite sets

Horizontal line test: f is one-one iff every horizontal line meets its graph at most once.

One handy fact: a *strictly increasing function* ($x_1 < x_2 \Rightarrow f(x_1) < f(x_2)$) or a *strictly decreasing function* ($x_1 < x_2 \Rightarrow f(x_1) > f(x_2)$) is always one-one: it never takes the same value twice, so distinct inputs always give distinct outputs.

Note: the non-strict “ $\leq$ ” versions of these definitions allow flat stretches — a constant function satisfies $x_1 \leq x_2 \Rightarrow f(x_1) \leq f(x_2)$ but is not one-one. Strictness is what rules out repeats.

Note: “one-one” and “injective” mean the same thing; “onto” and “surjective” mean the same thing.

Basically, ... Injective = no two inputs share an output (no collisions). Surjective = no output is left unused (full coverage). Bijective = perfect pairing, every input matched to a unique output and nothing left over.

1.9 Composition of functions

Def. The *composition* $f \circ g$ = “apply g first, then f ”:

$$(f \circ g)(x) = f(g(x)).$$

Read $f \circ g$ as “ f after g ”. Order matters: $f \circ g$ and $g \circ f$ are usually different functions.

Domain rules (both must hold for x to be allowed into $f \circ g$):

- i) x is in the domain of g ;
- ii) $g(x)$ is in the domain of f .

ML pointer: a neural network is a long composition $f_3 \circ f_2 \circ f_1$ of simple functions (layers). Every “stack” you meet later is composition wearing a costume.

Basically, ... Composition is piping: the output of one machine becomes the input of the next. If the first machine’s output doesn’t fit the second machine’s input slot, the pipe is broken there.

1.10 Inverse functions

Def. If f is one-one, its *inverse* f^{-1} = the function that undoes it:

$$f^{-1}(f(x)) = x \quad \text{for all } x \in \text{domain}(f),$$

$$f(f^{-1}(y)) = y \quad \text{for all } y \in \text{range}(f).$$

So f^{-1} exists exactly when f is one-one (a bijection between its domain and its range).

Three warnings:

- i) $f^{-1} \neq \frac{1}{f}$. The “ -1 ” is not an exponent; it means “undo”.
- ii) The domain of f^{-1} is the range of f , and the range of f^{-1} is the domain of f — inputs and outputs swap.
- iii) The graph of f^{-1} is the mirror image of the graph of f across the line $y = x$: a point $(a, f(a))$ on f becomes $(f(a), a)$ on f^{-1} .

Basically, ... The inverse is the “reverse gear” of a function: it takes you from output back to the input that produced it. Reverse gear only exists if the function never merged two different inputs into one output (one-one) — otherwise reversing is ambiguous.

1.11 Logic: quantifiers, implies, equivalent

Later chapters state definitions and theorems in a compressed logical shorthand. Four symbols do most of the work:

Symbol	Read as	Meaning
$\forall$	“for all”	the claim holds for every element
$\exists$	“there exists”	the claim holds for at least one element
$\Rightarrow$	“implies”	if the left side is true, the right side must be true
$\Leftrightarrow$	“is equivalent to”	each side implies the other

Examples:

- i) $\forall x \in \mathbb{R}, x^2 \geq 0$ reads “for all real x , x^2 is non-negative”.
- ii) $\exists n \in \mathbb{N}$ such that $n > 100$ reads “there exists a natural number bigger than 100” (e.g. $n = 101$).
- iii) $x > 2 \Rightarrow x > 1$: being bigger than 2 implies being bigger than 1. (The reverse implication is false.)
- iv) $x \in A \cap B \Leftrightarrow (x \in A \text{ and } x \in B)$: the two sides say exactly the same thing.

Note: “ $\Rightarrow$ ” is one-directional; “ $\Leftrightarrow$ ” is both directions at once. Confusing them is one of the most common reading errors in math — always check which arrow is meant.

Basically, ... $\forall$ = “every single one, no exceptions”. $\exists$ = “at least one exists somewhere”. $\Rightarrow$ = “this forces that”. $\Leftrightarrow$ = “these are two names for the same fact”.

1.12 A note on proof techniques (not covered here)

You will sometimes see the words *direct proof*, *proof by contrapositive*, *proof by contradiction*, and *proof by induction* named in mathematical writing. This chapter deliberately does not teach them: the source materials for this chapter (Maths 1 Vol. 1; MLF Week 2) do not develop proof techniques, and this book does not invent content its sources lack. Wherever a later chapter needs a proof idea, it will be introduced at the point of use.

1.13 Worked example set

eg 1 — Subsets and cardinality. Let $A = \{2, 4, 6, 8\}$ and $B = \{1, 2, 3, 4, 5, 6, 7, 8\}$.

Step 1: Check every element of A : $2 \in B$, $4 \in B$, $6 \in B$, $8 \in B$. So $A \subseteq B$.

Step 2: Is $A = B$? No — e.g. $1 \in B$ but $1 \notin A$. So $A \subset B$ (proper).

Step 3: $|A| = 4$, $|B| = 8$.

eg 2 — Union and intersection of intervals. Let $A = [2, 5]$ and $B = [4, 7]$.

$A \cup B$: x is in A or in B , i.e. $2 \leq x \leq 5$ or $4 \leq x \leq 7$. Since the intervals overlap ($4 \leq 5$), together they cover $2 \leq x \leq 7$. So $A \cup B = [2, 7]$.

$A \cap B$: x is in both, i.e. $2 \leq x \leq 5$ and $4 \leq x \leq 7$. Both hold exactly when $4 \leq x \leq 5$. So $A \cap B = [4, 5]$.

eg 3 — Verify De Morgan's first law with intervals. Take universe $U = [0, 10]$, $A = [2, 5]$, $B = [4, 7]$.

Left side: $(A \cup B)^c$. From eg 2, $A \cup B = [2, 7]$. Removing $[2, 7]$ from $[0, 10]$ leaves $[0, 2) \cup (7, 10]$. (The endpoints 2 and 7 are excluded because they belong to $A \cup B$.)

Right side: $A^c \cap B^c$.

$A^c = [0, 2) \cup (5, 10]$ (everything in $[0, 10]$ except $[2, 5]$).

$B^c = [0, 4) \cup (7, 10]$ (everything in $[0, 10]$ except $[4, 7]$).

Intersect: x must be in both. Below 2: $[0, 2)$ qualifies for both. Between 2 and 4: in B^c but not in A^c . Between 4 and 5: in neither. Between 5 and 7: in A^c but not in B^c . Above 7: $(7, 10]$ qualifies for both. So $A^c \cap B^c = [0, 2) \cup (7, 10]$.

Both sides equal $[0, 2) \cup (7, 10]$. Verified.

eg 4 — Relation vs function. Let $X = \{a, b\}$, $Y = \{1, 2, 3\}$. Then $X \times Y = \{(a, 1), (a, 2), (a, 3), (b, 1), (b, 2), (b, 3)\}$.

Consider $R_1 = \{(a, 1), (b, 2), (b, 3)\}$ and $R_2 = \{(a, 1), (a, 2)\}$.

R_1 : b is paired with two different outputs (2 and 3), so R_1 is **not** a function.

R_2 : a is paired with two different outputs (1 and 2), so R_2 is **not** a function either.

Take instead $R_3 = \{(a, 1), (b, 2)\}$: each of a, b appears exactly once as a first element. So R_3 **is** a function, with domain $\{a, b\}$ and range $\{1, 2\}$.

eg 5 — Checking relation properties. On $S = \{1, 2, 3\}$, let $R = \{(1, 1), (2, 2), (3, 3), (1, 2), (2, 1)\}$.

Reflexive? Need $(1, 1), (2, 2), (3, 3)$ — all present. Yes.

Symmetric? Check each pair: $(1, 2) \in R$ and $(2, 1) \in R$. Diagonals are symmetric with themselves. Yes.

Transitive? Check all chains: $(1, 2), (2, 1) \in R \Rightarrow$ need $(1, 1) \in R$ — present. $(2, 1), (1, 2) \in R \Rightarrow$ need $(2, 2) \in R$ — present. All other chains involve only diagonals. Yes.

Reflexive + symmetric + transitive $\Rightarrow R$ is an equivalence relation.

eg 6 — Domain and range; a non-surjective function. Let $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = x^2$.

Range: $x^2 \geq 0$ for all real x , and every $y \geq 0$ is hit (take $x = \sqrt{y}$). So range = $[0, \infty)$.

Surjective? Codomain is $\mathbb{R}$ but range is $[0, \infty) \neq \mathbb{R}$ (e.g. -1 is never hit). So f is **not** surjective.

Injective? $f(2) = 4 = f(-2)$ but $2 \neq -2$. So f is **not** injective either.

eg 7 — Classifying three functions. i) $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = 2x + 1$.

Injective: $f(x_1) = f(x_2) \Rightarrow 2x_1 + 1 = 2x_2 + 1 \Rightarrow x_1 = x_2$. Yes.

Surjective: for any $y \in \mathbb{R}$, take $x = \frac{y-1}{2}$; then $f(x) = y$. Yes. So f is bijective.

ii) $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = x^2$. Neither injective nor surjective (eg 6).

iii) $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = x^3$.

Injective: f is strictly increasing ($x_1 < x_2 \Rightarrow x_1^3 < x_2^3$), and an increasing function is one-one. Yes.

Surjective: for any $y \in \mathbb{R}$, $x = y^{1/3} \in \mathbb{R}$ satisfies $f(x) = y$. Yes. So f is bijective.

eg 8 — Composition, both orders. Let $f(x) = 3x - 4$ and $g(x) = x^2$.

$$(f \circ g)(x) = f(g(x)) = f(x^2) = 3x^2 - 4.$$

$$(g \circ f)(x) = g(f(x)) = g(3x - 4) = (3x - 4)^2 = 9x^2 - 24x + 16.$$

Note $(f \circ g)(x) \neq (g \circ f)(x)$ — order matters.

eg 9 — Domain of a composition. Let $f(x) = \frac{3}{x-1}$ and $g(x) = \frac{3}{x}$.

$$\text{Step 1: } (f \circ g)(x) = f(g(x)) = \frac{3}{g(x)-1} = \frac{3}{\frac{3}{x}-1} = \frac{3x}{3-x}.$$

Step 2: Domain rule i): x must be in $\text{domain}(g)$, so $x \neq 0$.

Step 3: Domain rule ii): $g(x)$ must be in $\text{domain}(f)$, i.e. $g(x) \neq 1$, so $\frac{3}{x} \neq 1$, giving $x \neq 3$.

Step 4: Domain of $f \circ g$ is $\mathbb{R} \setminus \{0, 3\}$.

eg 10 — Finding and verifying an inverse. Let $f: \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = 2x + 3$. (Increasing, hence one-one, so f^{-1} exists.)

Step 1: Set $y = 2x + 3$ and solve for x : $x = \frac{y-3}{2}$. So $f^{-1}(y) = \frac{y-3}{2}$, i.e. $f^{-1}(x) = \frac{x-3}{2}$.

Step 2: Verify $f^{-1}(f(x)) = \frac{(2x+3)-3}{2} = \frac{2x}{2} = x$.

Step 3: Verify $f(f^{-1}(x)) = 2\left(\frac{x-3}{2}\right) + 3 = (x-3) + 3 = x$.

eg 11 — Reading quantifier statements. i) “ $\forall x \in \mathbb{R}, x^2 \geq 0$ ” reads: “for every real number x , x^2 is at least 0.” (True.)

ii) Formalize “some natural number exceeds 100”: $\exists n \in \mathbb{N}$ such that $n > 100$. (True: $n = 101$ works.)

iii) “ $x \in A \cap B \Leftrightarrow x \in A$ and $x \in B$ ” reads: “ x is in the intersection exactly when x is in A and also in B .”
The $\Leftrightarrow$ says the two sides are interchangeable.

1.14 Problem set

Full worked solutions are in the companion solutions volume (`solutions/01-sets-functions-preliminaries.md`).

- Let $A = \{x \in \mathbb{N} : x < 5\}$. List the elements of A and give $|A|$.
- Decide whether each statement is true or false, with one line of justification:
 - $\{4, 16\} \subseteq \{16, 4, 32\}$;
 - $\{4, 16\} \subset \{16, 4\}$;
 - $\{4, 16, 32\} \subset \{16, 4, 32\}$.
- Take universe $U = [0, 10]$, $A = [2, 5]$, $B = [4, 7]$. Compute $A \cap B$, A^c , and B^c , and use them to verify De Morgan’s second law $(A \cap B)^c = A^c \cup B^c$.
- Let $A = \{1, 2\}$ and $B = \{x, y\}$. List all elements of $A \times B$ and count them.
- Let $X = \{1, 2, 3\}$, $Y = \{2, 3\}$, $R_1 = \{(1, 2), (2, 3), (3, 3)\}$, $R_2 = \{(1, 2), (1, 3), (2, 2)\}$.
 - Which of R_1, R_2 is a function from X to Y ? Why does the other fail?
 - For the one that is a function, state its domain and its range.
- For each function, decide injective / surjective / bijective / neither:
 - $f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = x^2 + 1$;
 - $f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = 5x - 7$;
 - $f: \mathbb{N} \rightarrow \mathbb{N}, f(n) = n + 1$.
- Find the domain of each (as a subset of $\mathbb{R}$):
 - $f(x) = \frac{1}{x-2}$;

- ii) $g(x) = \sqrt{x+1}$;
- iii) $h(x) = \frac{1}{\sqrt{x-1}}$.
- 8. Let $f(x) = x+1$ and $g(x) = 2x$. Find $(f \circ g)(x)$, $(g \circ f)(x)$, and $(f \circ f)(x)$. Are any two of them equal?
- 9. Let $f(x) = \sqrt{x}$ and $g(x) = x-5$. Find $(f \circ g)(x)$ and its domain, checking both domain rules.
- 10. Let $f: \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = \frac{x+1}{3}$. Find f^{-1} and verify both $f^{-1}(f(x)) = x$ and $f(f^{-1}(x)) = x$.
- 11. On $S = \{1, 2, 3\}$, let $R = \{(1, 1), (2, 2), (3, 3), (1, 3), (3, 1)\}$. Is R reflexive? Symmetric? Transitive? Is it an equivalence relation?
- 12. i) Write “every integer is a rational number” as a quantified statement using $\forall$.
 ii) True or false: $\exists x \in \mathbb{R}$ such that $x^2 = -1$. Justify in one line.

1.15 Where this goes next

- $\mathbb{R}^d$ and Cartesian products return in Chapter 2 (vectors and matrices): a dataset is a subset of $\mathbb{R}^d$.
- Functions $f: \mathbb{R}^d \rightarrow \mathbb{R}$, domains, and composition are the language of Chapters 8–10 (calculus and optimization); the loss functions of Chapters 23 and 31 are exactly such functions.
- Quantifiers ($\forall$, $\exists$) and implication ($\Rightarrow$) reappear in every limit, continuity, and convergence definition from Chapter 8 onward.
- Bijectivity and inverses return with matrix inverses (Chapter 3) and change-of-variables ideas later.

Solutions — Chapter 1: Sets, functions, and mathematical preliminaries

Problem 1

$$A = \{x \in \mathbb{N} : x < 5\}.$$

Since this book takes $\mathbb{N} = \{0, 1, 2, \dots\}$:

Step 1: List naturals below 5: 0, 1, 2, 3, 4.

Step 2: $A = \{0, 1, 2, 3, 4\}$, so $|A| = 5$.

Note: If you learned $\mathbb{N}$ starting at 1 elsewhere, you would get $\{1, 2, 3, 4\}$ — the convention matters, which is why the book states it up front.

Problem 2

$$\text{i) } \{4, 16\} \subseteq \{16, 4, 32\}.$$

Check each element of the left set: 4 is in the right set, 16 is in the right set. True.

$$\text{ii) } \{4, 16\} \subset \{16, 4\}.$$

The left set is a subset of the right set (same check as i), but the sets are equal — order does not matter, so $\{16, 4\} = \{4, 16\}$. A proper subset must be strictly smaller. False.

$$\text{iii) } \{4, 16, 32\} \subset \{16, 4, 32\}.$$

Again equal sets (order irrelevant). False — for the same reason as ii).

Problem 3

$$U = [0, 10], A = [2, 5], B = [4, 7].$$

Step 1: $A \cap B$. Need $2 \leq x \leq 5$ and $4 \leq x \leq 7$: both hold exactly on $[4, 5]$.

Step 2: $(A \cap B)^c = [4, 5]^c$ inside $[0, 10]$: everything except $[4, 5]$, i.e. $[0, 4) \cup (5, 10]$.

Step 3: $A^c = [0, 2) \cup (5, 10]$ (remove $[2, 5]$ from $[0, 10]$).

Step 4: $B^c = [0, 4) \cup (7, 10]$ (remove $[4, 7]$ from $[0, 10]$).

Step 5: $A^c \cup B^c = ([0, 2) \cup (5, 10]) \cup ([0, 4) \cup (7, 10])$.

Collect: below 4, $[0, 4)$ is covered by B^c ; above 5, $(5, 10]$ is covered by A^c . Check the boundary points: $4 \in A$ and $4 \in B$, so 4 is in neither complement; $5 \in A$ and $5 \in B$, so 5 is in neither complement. Hence $A^c \cup B^c = [0, 4) \cup (5, 10]$.

Step 6: This equals $(A \cap B)^c$ from Step 2. De Morgan's second law verified.

Problem 4

$A \times B = \{(x, y) : x \in A, y \in B\}$.

List: $A \times B = \{(1, x), (1, y), (2, x), (2, y)\}$.

Count: 4 elements (every one of the 2 choices from A pairs with every one of the 2 choices from B).

Problem 5

i) $R_1 = \{(1, 2), (2, 3), (3, 3)\}$: the first elements 1, 2, 3 each occur exactly once, so each input has exactly one output — R_1 is a function.

$R_2 = \{(1, 2), (1, 3), (2, 2)\}$: the input 1 is paired with two different outputs (2 and 3), violating “exactly one output per input” — R_2 is not a function.

ii) For R_1 : domain = $\{1, 2, 3\}$ (all first elements); range = $\{2, 3\}$ (the second elements that occur).

Problem 6

i) $f : \mathbb{R} \rightarrow \mathbb{R}, f(x) = x^2 + 1$.

Injective? $f(1) = 2 = f(-1)$ with $1 \neq -1$. No.

Surjective? $f(x) \geq 1$ always, so e.g. $0 \in \mathbb{R}$ is never hit. No.

Neither.

ii) $f : \mathbb{R} \rightarrow \mathbb{R}, f(x) = 5x - 7$.

Injective: $f(x_1) = f(x_2) \Rightarrow 5x_1 - 7 = 5x_2 - 7 \Rightarrow 5x_1 = 5x_2 \Rightarrow x_1 = x_2$. Yes.

Surjective: for any $y \in \mathbb{R}$, $x = \frac{y+7}{5}$ satisfies $f(x) = 5 \cdot \frac{y+7}{5} - 7 = y$. Yes.

Bijjective.

iii) $f : \mathbb{N} \rightarrow \mathbb{N}, f(n) = n + 1$.

Injective: $n_1 + 1 = n_2 + 1 \Rightarrow n_1 = n_2$. Yes.

Surjective? $0 \in \mathbb{N}$: is there $n \in \mathbb{N}$ with $n + 1 = 0$? That needs $n = -1 \notin \mathbb{N}$. No.

Injective but not surjective.

Basically, ... shifting every natural number up by one leaves a hole at 0 — nothing maps to it.

Problem 7

i) $f(x) = \frac{1}{x-2}$.

The denominator cannot be zero: $x - 2 \neq 0$. Domain = $\mathbb{R} \setminus \{2\}$.

ii) $g(x) = \sqrt{x+1}$.

Need $x + 1 \geq 0$, i.e. $x \geq -1$. Domain = $[-1, \infty)$.

$$\text{iii) } h(x) = \frac{1}{\sqrt{x-1}}.$$

Two requirements: $x - 1 \geq 0$ (square root) and $\sqrt{x-1} \neq 0$ (denominator). Together: $x - 1 > 0$. Domain = $(1, \infty)$.

Problem 8

$$f(x) = x + 1, g(x) = 2x.$$

$$(f \circ g)(x) = f(g(x)) = f(2x) = 2x + 1.$$

$$(g \circ f)(x) = g(f(x)) = g(x + 1) = 2(x + 1) = 2x + 2.$$

$$(f \circ f)(x) = f(f(x)) = f(x + 1) = (x + 1) + 1 = x + 2.$$

No two are equal: $2x + 1$, $2x + 2$, and $x + 2$ are all different functions. (Domains are all $\mathbb{R}$.)

Problem 9

$$f(x) = \sqrt{x}, g(x) = x - 5.$$

$$\text{Step 1: } (f \circ g)(x) = f(g(x)) = \sqrt{x-5}.$$

Step 2: Domain rule i): $x \in \text{domain}(g) = \mathbb{R}$ — no restriction.

Step 3: Domain rule ii): $g(x) \in \text{domain}(f)$, i.e. $x - 5 \geq 0$, so $x \geq 5$.

Step 4: Domain of $f \circ g$ is $[5, \infty)$.

Note: Even though g accepts every real, the composition rejects $x < 5$ because g 's output would not fit f 's input slot.

Problem 10

$$f : \mathbb{R} \rightarrow \mathbb{R}, f(x) = \frac{x+1}{3}.$$

Step 1: Set $y = \frac{x+1}{3}$ and solve for x : $3y = x + 1$, so $x = 3y - 1$. Hence $f^{-1}(x) = 3x - 1$.

$$\text{Step 2: } f^{-1}(f(x)) = 3\left(\frac{x+1}{3}\right) - 1 = (x+1) - 1 = x.$$

$$\text{Step 3: } f(f^{-1}(x)) = \frac{(3x-1)+1}{3} = \frac{3x}{3} = x.$$

Problem 11

$$R = \{(1, 1), (2, 2), (3, 3), (1, 3), (3, 1)\} \text{ on } S = \{1, 2, 3\}.$$

Reflexive? $(1, 1), (2, 2), (3, 3)$ all present. Yes.

Symmetric? The non-diagonal pairs are $(1, 3)$ and $(3, 1)$ — each other's reverse is present. Yes.

Transitive? Check chains through distinct elements: $(1, 3), (3, 1) \in R \Rightarrow$ need $(1, 1) \in R$ — present. $(3, 1), (1, 3) \in R \Rightarrow$ need $(3, 3) \in R$ — present. Chains involving only diagonals are automatic. Yes.

All three hold, so R is an equivalence relation.

Problem 12

- i) $\forall x \in \mathbb{Z}, x \in \mathbb{Q}$ — equivalently, $\forall x (x \in \mathbb{Z} \Rightarrow x \in \mathbb{Q})$.
- ii) False. For every real x , $x^2 \geq 0$, so $x^2 = -1$ has no real solution.

2. Vectors and matrices

Everything in deep learning is vectors and matrices. A *dataset* is a matrix, one row per *data point*. A *data point* is a vector — its *features* listed in order. A *neural network layer* is a matrix multiplying a vector, plus a vector *bias*. So this chapter is the vocabulary of the whole subject. Read it slowly; the rest of the book keeps pointing back here.

2.1 A vector = a list of numbers

A *vector* = an ordered list of numbers. The numbers are called *components* (also *coordinates* or *entries*).

- i) A *row vector*: $(3, 2), (1, 2, 3.3)$.
- ii) A *column vector*: $\begin{pmatrix} 3 \\ 2 \end{pmatrix}, \begin{pmatrix} 5 \\ -1 \\ 4 \end{pmatrix}$.
- iii) From Chapter 1 (§1.6): a vector with n real entries is an element of $\mathbb{R}^n$ — a point of n -dimensional space. Vectors with n entries = vectors in $\mathbb{R}^n$ = points in $\mathbb{R}^n$.

Why lists? Real data arrives as tables, and a vector is just a row or a column of a table:

- The cricket row $(64.35, 31.92, 33.30, 26.00, 49.87) = V.$ Kohli, Dhoni, Rohit, Rahul, Dhawan’s averages vs South Africa — five numbers, one vector.
- A grocery-shop column: $(150, 50, 35, 70, 25) =$ stock (in kg/L/packets/bars) of rice, dal, oil, biscuits, soap.

Note: *order* matters in a vector. $(3, 2) \neq (2, 3)$ — same reason $(a, 1) \neq (1, a)$ in §1.6.

Basically, ... A vector is a shopping list with a fixed order. “(3 kg rice, 2 kg dal)” tells you everything; shuffling it to “(2 kg rice, 3 kg dal)” changes the meaning. ML just calls these lists “vectors” and the items “features”.

2.2 The geometric view: arrows

For $\mathbb{R}^2$ and $\mathbb{R}^3$ we can draw vectors. The vector (a, b) is drawn as an *arrow* from the *origin* $(0, 0)$ to the point (a, b) .

- The *length* of the arrow = the vector’s *magnitude* (size).
- The *arrowhead* points in the vector’s *direction*.

Two ways to see the same vector:

- i) As a *point*: $(3, 2)$ is the location “3 right, 2 up”.
- ii) As a *direction*: $(3, 2)$ is the instruction “go 3 right, 2 up”.

A vector has magnitude and direction but *no position*: sliding it parallel to itself (same length, same direction) does not change it. Two arrows drawn in different places are the same vector as long as their lengths and directions match.

Note: the standard *unit vectors* $\hat{\mathbf{i}} = (1, 0)$ and $\hat{\mathbf{j}} = (0, 1)$ are arrows of length 1 along the axes. Any $(a, b) = a\hat{\mathbf{i}} + b\hat{\mathbf{j}}$; e.g. $(1, 2) = 1\hat{\mathbf{i}} + 2\hat{\mathbf{j}}$. (What “length 1” means is in §2.4.)

Basically, ... A vector is an arrow. Length says how far, the tip says which way, and where you draw it on the page doesn’t matter — only the push it represents.

2.3 Addition, subtraction, scalar multiplication

Vector *addition* = add the matching entries (*coordinatewise*). You can only add vectors of the same length.

Def. $(a_1, \dots, a_n) + (b_1, \dots, b_n) = (a_1 + b_1, \dots, a_n + b_n)$.

eg. Arun buys $(3, 2)$ (3 kg rice, 2 kg dal), Neela buys $(5, 6)$. Together:

$$(3, 2) + (5, 6) = (3 + 5, 2 + 6) = (8, 8).$$

8 kg rice, 8 kg dal — nothing more than adding two lists entry by entry.

eg. The grocery stock-take. Stock plus new stock, minus three buyers’ purchases:

$$\begin{aligned} (150, 50, 35, 70, 25) &+ (-8, -8, -4, -10, -4) + (-12, -5, -7, -10, -2) \\ &+ (-3, -2, -5, -5, -1) + (100, 75, 30, 80, 30). \end{aligned}$$

Add each column: rice $150 - 8 - 12 - 3 + 100 = 227$; dal $50 - 8 - 5 - 2 + 75 = 110$; oil $35 - 4 - 7 - 5 + 30 = 49$; biscuits $70 - 10 - 10 - 5 + 80 = 125$; soap $25 - 4 - 2 - 1 + 30 = 48$.

$$= (227, 110, 49, 125, 48).$$

Scalar multiplication = multiply every entry by one number (the *scalar*). It stretches (or shrinks, or flips) the arrow without changing its direction — unless the scalar is negative, which reverses it.

Def. $c(a_1, \dots, a_n) = (ca_1, \dots, ca_n)$ for any scalar $c \in \mathbb{R}$.

eg. Buyer A repeats yesterday’s purchase exactly:

$$2(8, 8, 4, 10, 4) = (16, 16, 8, 20, 8).$$

Vector *subtraction* = add the negative: $\mathbf{u} - \mathbf{v} = \mathbf{u} + (-1)\mathbf{v}$, done coordinatewise. **eg.** $(5, 6) - (3, 2) = (5 - 3, 6 - 2) = (2, 4)$.

The *zero vector* $\mathbf{0} = (0, 0, \dots, 0)$ = adding nothing. It satisfies $\mathbf{v} + \mathbf{0} = \mathbf{v}$ for every $\mathbf{v}$.

Geometrically, addition has two equivalent pictures:

- i) *Head-to-tail*: put the tail of the second arrow at the head of the first; the sum is the arrow from the very start to the very end.
- ii) *Parallelogram law*: put both tails at the origin; the sum is the diagonal of the parallelogram they span.

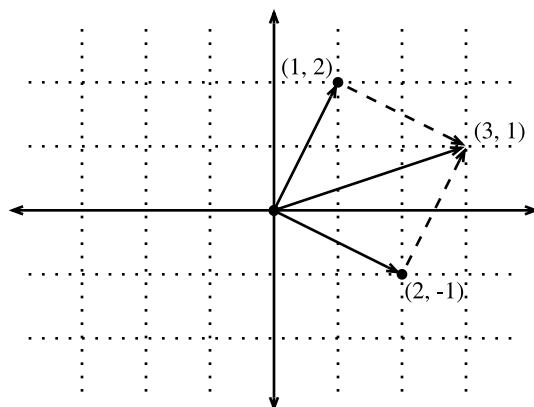


Figure 3: Vector addition by following one vector then the other: head-to-tail

eg. $(1, 2) + (2, 1) = (3, 3)$ — the diagonal of the parallelogram with sides $(1, 2)$ and $(2, 1)$ lands on $(3, 3)$.

Note: $-\mathbf{v}$ is the arrow of $\mathbf{v}$ reversed, and $\mathbf{u} - \mathbf{v}$ is the direction from the tip of $\mathbf{v}$ to the tip of $\mathbf{u}$. Check: $\mathbf{v} + (\mathbf{u} - \mathbf{v}) = \mathbf{u}$.

Basically, ... Adding vectors = adding the lists item by item. On paper, it is walking: follow the first arrow, then follow the second from where you landed — where you finish is the sum. Scaling = making the arrow longer or shorter (a negative number also flips it around).

2.4 Magnitude (norm) and unit vectors

The *magnitude* (or *length*, or *norm*) of a vector, written $\|\mathbf{v}\|$, is the length of its arrow. For $(3, 4)$ in $\mathbb{R}^2$, Pythagoras gives $\|(3, 4)\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$.

Def. For $\mathbf{v} = (v_1, \dots, v_n) \in \mathbb{R}^n$,

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}.$$

eg. $\|(1, 2, 2)\| = \sqrt{1 + 4 + 4} = \sqrt{9} = 3$.

A *unit vector* = a vector of length 1. To make any nonzero $\mathbf{v}$ into a unit vector pointing the same way, divide by its length: $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$.

eg. $\mathbf{v} = (3, 4)$ has $\|\mathbf{v}\| = 5$, so $\hat{\mathbf{v}} = (3/5, 4/5)$. Check: $\|\hat{\mathbf{v}}\| = \sqrt{9/25 + 16/25} = \sqrt{1} = 1$.

Note: norms are never negative, and $\|c\mathbf{v}\| = |c| \|\mathbf{v}\|$.

Basically, ... The norm is just “how long is the arrow” — the straight-line distance from the origin to the point, computed by Pythagoras in however many dimensions you have. A unit vector is the same arrow shrunk to length 1: pure direction, no size.

2.5 The dot product

The *dot product* takes two vectors and returns a single number (a *scalar*). It is the bridge between algebra and geometry in this chapter.

Def. For $\mathbf{u} = (u_1, \dots, u_n)$, $\mathbf{v} = (v_1, \dots, v_n)$ in $\mathbb{R}^n$,

$$\mathbf{u} \cdot \mathbf{v} = u_1 v_1 + u_2 v_2 + \dots + u_n v_n.$$

eg. $(2, 4) \cdot (3, 5) = 2 \times 3 + 4 \times 5 = 6 + 20 = 26$. **eg.** $(1, 2, 3) \cdot (2, 0, 1) = 1 \times 2 + 2 \times 0 + 3 \times 1 = 2 + 0 + 3 = 5$.

Three facts that make the dot product earn its keep:

i) *Length from dot product*: $\mathbf{v} \cdot \mathbf{v} = v_1^2 + \cdots + v_n^2 = \|\mathbf{v}\|^2$, so

$$\|\mathbf{v}\| = \sqrt{\mathbf{v} \cdot \mathbf{v}}.$$

eg. $(3, 4) \cdot (3, 4) = 9 + 16 = 25$, and $\sqrt{25} = 5 = \|(3, 4)\|$.

ii) *Symmetry*: $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$ — order does not matter.

iii) *Angle*: if θ is the angle between nonzero $\mathbf{u}$ and $\mathbf{v}$,

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta, \quad \text{so} \quad \cos \theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

eg (angle, full steps). Find the angle between $\mathbf{u} = (1, 0)$ and $\mathbf{v} = (1, 1)$.

$$\mathbf{u} \cdot \mathbf{v} = 1 \times 1 + 0 \times 1 = 1,$$

$$\|\mathbf{u}\| = \sqrt{1 + 0} = 1, \quad \|\mathbf{v}\| = \sqrt{1 + 1} = \sqrt{2},$$

$$\cos \theta = \frac{1}{1 \cdot \sqrt{2}} = \frac{1}{\sqrt{2}} \Rightarrow \theta = 45^\circ.$$

Exactly what the picture says: $(1, 1)$ points diagonally between the axes.

eg (angle, full steps). $\mathbf{a} = (4, 3)$, $\mathbf{b} = (-4, 3)$.

$$\mathbf{a} \cdot \mathbf{b} = 4 \times (-4) + 3 \times 3 = -16 + 9 = -7,$$

$$\|\mathbf{a}\| = 5, \quad \|\mathbf{b}\| = \sqrt{16 + 9} = 5,$$

$$\cos \theta = \frac{-7}{5 \times 5} = \frac{-7}{25} = -0.28 \Rightarrow \theta = \cos^{-1}(-0.28) \approx 106.3^\circ.$$

The negative dot product means the angle is obtuse — the vectors lean away from each other.

Note (ML): $\cos \theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$ is called *cosine similarity*. It is 1 when the vectors point the same way, -1 when they point opposite, and 0 when they are at right angles. In ML, two document or image vectors with cosine similarity near 1 “say the same thing” even if one is longer (brighter image, longer document) — the angle ignores length. This is why dot products show up in attention and recommendation: they measure agreement.

Note (preview): $\|\mathbf{v}\| \cos \theta$ is the length of the *projection* of $\mathbf{v}$ onto the direction of $\mathbf{u}$ — how much of $\mathbf{v}$ lies along $\mathbf{u}$. The full projection story waits in Chapter 5.

Basically, ... The dot product = multiply matching entries, then add. That’s the whole computation. What it tells you: how much two arrows agree. Big positive = pointing together; zero = at right angles; negative = pointing apart. ML uses it as a “similarity score” between two feature vectors.

2.6 Linear combinations

A *linear combination* = scale some vectors, then add them. One operation, two ingredients (scaling + adding).

Def. Let $\mathbf{v}_1, \dots, \mathbf{v}_k$ be vectors and $\alpha_1, \dots, \alpha_k$ scalars. Then

$$\alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \cdots + \alpha_k \mathbf{v}_k$$

is a *linear combination* of the $\mathbf{v}_i$ with *coefficients* α_i . The result is again a vector.

eg. $2(1, 2) + 3(2, 1)$: scale first, $2(1, 2) = (2, 4)$ and $3(2, 1) = (6, 3)$; then add, $(2, 4) + (6, 3) = (8, 7)$. **eg.** $(4, 5) = 2(1, 2) + 1(2, 1)$: check, $2(1, 2) + (2, 1) = (2 + 2, 4 + 1) = (4, 5)$.

Note: vector addition is the linear combination $1\mathbf{v}_1 + 1\mathbf{v}_2$; scalar multiplication is $c\mathbf{v}$. Everything in §2.3 is a special case of this one idea.

Basically, ... A linear combination is a recipe: “take twice of this arrow and three times of that arrow, add them up.” The numbers in front (coefficients) say how much of each ingredient. This single idea quietly powers all of matrix multiplication in §2.10.

2.7 What is a matrix?

A *matrix* = a rectangular array of numbers, arranged in *rows* and *columns*. (Plural: *matrices*.)

- i) An $m \times n$ *matrix* has m rows and n columns. Read “ m by n ”.
- ii) The (i, j) -th entry A_{ij} is the number in row i , column j . Rows and columns are numbered from 1.

eg.

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \end{pmatrix}$$

is a 2×3 matrix. Its $(1, 2)$ -th entry is $A_{12} = 2$ and its $(2, 3)$ -th entry is $A_{23} = 4$.

Two ML faces of a matrix:

- i) A matrix is a *table of data*: stack data vectors as rows. One row = one data point, one column = one feature. The cricket table of §2.1 is a 6×5 matrix.
- ii) A matrix is a *machine* that transforms vectors: $m \times n$ matrix times an n -vector gives an m -vector (§2.10). A neural layer $\mathbf{y} = W\mathbf{x} + \mathbf{b}$ is exactly this.

Basically, ... A matrix is a table of numbers with an address system: row i , column j picks out one cell. ML reads a matrix two ways — as a pile of data (rows = examples) and as a machine that converts one vector into another.

2.8 Special matrices

- *Square matrix* = number of rows = number of columns ($n \times n$).
- The i -th *diagonal entry* of a square matrix is A_{ii} ; the *diagonal* is the set of all of them. **eg.** In $\begin{pmatrix} 0.3 & 5 & -7 \\ 2.8 & 0 & 1 \\ 0 & -2.5 & -1 \end{pmatrix}$ the diagonal entries are 0.3, 0, -1 .
- *Zero matrix* = every entry is 0. Written 0 (or $0_{m \times n}$ when the shape matters).
- *Diagonal matrix* = square, everything off the diagonal is 0. **eg.** $\begin{pmatrix} 1 & 0 & 0 \\ 0 & -3 & 0 \\ 0 & 0 & 4.2 \end{pmatrix}$ is diagonal.
- *Scalar matrix* = diagonal, and all diagonal entries are equal. **eg.** $\begin{pmatrix} -3 & 0 & 0 \\ 0 & -3 & 0 \\ 0 & 0 & -3 \end{pmatrix}$ is scalar.
- *Identity matrix* I = scalar matrix with all diagonal entries 1 — the “do nothing” matrix.

$$I_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

- *Transpose* A^T = flip rows and columns: $(A^T)_{ij} = A_{ji}$. **eg.** $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$. A row vector's transpose is a column vector and vice versa.
- *Symmetric matrix* = square matrix equal to its own transpose: $A^T = A$ (mirror across the diagonal). **eg.** $\begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ is symmetric; $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ is not ($2 \neq 3$ off the diagonal).

Basically, ... Special matrices are the greatest hits: the zero matrix (all zeros), the identity (1s on the diagonal — multiplying by it changes nothing, like $\times 1$), and the transpose (rotate the table so rows become columns). Symmetric = the table looks the same after that flip.

2.9 Matrix addition and scalar multiplication

Exactly like vectors, done *entrywise*. Addition needs the same shape on both sides.

Def. For $m \times n$ matrices A, B : $(A + B)_{ij} = A_{ij} + B_{ij}$. For a scalar c : $(cA)_{ij} = cA_{ij}$.

eg.

$$\begin{pmatrix} 2 & -1 & 0 \\ 3 & 4 & 5 \end{pmatrix} + \begin{pmatrix} 1 & 3 & -2 \\ 0 & -4 & 1 \end{pmatrix} = \begin{pmatrix} 2+1 & -1+3 & 0-2 \\ 3+0 & 4-4 & 5+1 \end{pmatrix} = \begin{pmatrix} 3 & 2 & -2 \\ 3 & 0 & 6 \end{pmatrix}.$$

eg.

$$3 \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} = \begin{pmatrix} 3 & 6 & 9 \\ 12 & 15 & 18 \end{pmatrix}.$$

Note: $A + B = B + A$ (addition commutes), and $\lambda(A + B) = \lambda A + \lambda B$. Adding a 2×3 matrix to a 3×2 matrix is simply not defined — the entries would not line up.

Basically, ... Matrix addition = lay the tables on top of each other and add cell by cell. Scaling = multiply every cell by one number. Same as vectors, because a matrix is just vectors stacked.

2.10 Matrix multiplication: the heart of the chapter

Matrix multiplication looks odd at first. It is *not* entrywise — and that is deliberate: it is built so that multiplying matrices composes the machines of §2.7.

Compatibility: $A_{m \times n} B_{n \times p} = (AB)_{m \times p}$. The inner dimensions must match (columns of A = rows of B); the result takes the outer ones.

Def (row-times-column). $(AB)_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$ = row i of A dotted with column j of B .

eg (fully worked). Multiply $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ by $B = \begin{pmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \end{pmatrix}$. A is 2×2 , B is 2×3 ; the inner 2s match, so AB is 2×3 . Six dot products:

- $(AB)_{11} = \text{row 1 of } A \cdot \text{col 1 of } B = (1, 2) \cdot (1, 3) = 1 + 6 = 7$
- $(AB)_{12} = \text{row 1 of } A \cdot \text{col 2 of } B = (1, 2) \cdot (2, 4) = 2 + 8 = 10$
- $(AB)_{13} = \text{row 1 of } A \cdot \text{col 3 of } B = (1, 2) \cdot (3, 5) = 3 + 10 = 13$
- $(AB)_{21} = \text{row 2 of } A \cdot \text{col 1 of } B = (3, 4) \cdot (1, 3) = 3 + 12 = 15$
- $(AB)_{22} = \text{row 2 of } A \cdot \text{col 2 of } B = (3, 4) \cdot (2, 4) = 6 + 16 = 22$
- $(AB)_{23} = \text{row 2 of } A \cdot \text{col 3 of } B = (3, 4) \cdot (3, 5) = 9 + 20 = 29$

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \end{pmatrix} = \begin{pmatrix} 7 & 10 & 13 \\ 15 & 22 & 29 \end{pmatrix}.$$

eg (matrix times vector). $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \end{pmatrix}$: row 1 $\cdot (5, 6) = 5 + 12 = 17$; row 2 $\cdot (5, 6) = 15 + 24 = 39$. Result: $\begin{pmatrix} 17 \\ 39 \end{pmatrix}$.

There is a second view of the same computation, and it is the more useful one in ML.

The columns view. Write A as its columns $[\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_n]$. Then

$$A\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n$$

— a *linear combination* of the columns of A , with the entries of $\mathbf{x}$ as coefficients.

eg (same product, columns view). Columns of $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ are $\mathbf{a}_1 = (1, 3)$, $\mathbf{a}_2 = (2, 4)$. Then

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \end{pmatrix} = 5\mathbf{a}_1 + 6\mathbf{a}_2 = 5(1, 3) + 6(2, 4) = (5 + 12, 15 + 24) = (17, 39).$$

Same $(17, 39)$ as the row-times-column computation above. Two views, one answer.

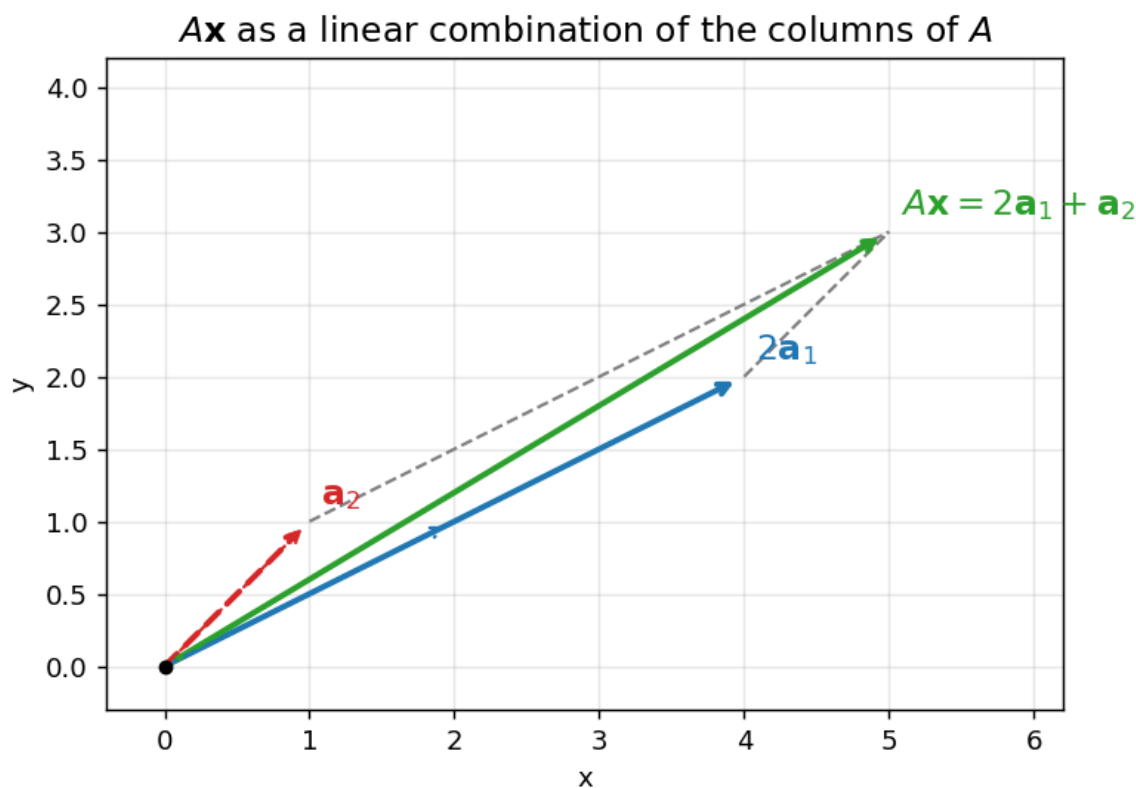


Figure 4: Matrix-vector product as a linear combination of the columns of A

Two handy special cases:

- i) Multiplying by a scalar matrix = scalar multiplication: $\begin{pmatrix} c & 0 & 0 \\ 0 & c & 0 \\ 0 & 0 & c \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} = \begin{pmatrix} c & 2c \\ 3c & 4c \\ 5c & 6c \end{pmatrix} = c \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}.$
- ii) Multiplying by the identity changes nothing: $IA = A = AI$ (with the right-sized I on each side).

Basically, ... Matrix multiplication = every row of the left table dotted with every column of the right table. And the ML-flavoured re-read: $A\mathbf{x}$ is a recipe — “take x_1 of column 1, x_2 of column 2, ...” and add. The columns of A are the ingredients; the entries of $\mathbf{x}$ are how much of each.

2.11 The basic rules (and one famous non-rule)

For matrices of compatible shapes:

- i) $(A + B) + C = A + (B + C)$ (*associativity* of addition).
- ii) $(AB)C = A(BC)$ (*associativity* of multiplication) — brackets don’t matter in a product chain, which is why ABC is unambiguous.
- iii) $A + B = B + A$ (*commutativity* of addition).
- iv) $A(B + C) = AB + AC$ and $(A + B)C = AC + BC$ (*distributivity*).
- v) $\lambda(AB) = (\lambda A)B = A(\lambda B)$ — scalars slide freely through a product.
- vi) The famous non-rule: in general $AB \neq BA$ (when both even make sense). Multiplication does *not* commute. **eg (counterexample, full steps).** $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, B = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}:$

$$AB = \begin{pmatrix} 1 \cdot 1 + 2 \cdot 0 & 1 \cdot 0 + 2 \cdot 0 \\ 3 \cdot 1 + 4 \cdot 0 & 3 \cdot 0 + 4 \cdot 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 3 & 0 \end{pmatrix},$$

$$BA = \begin{pmatrix} 1 \cdot 1 + 0 \cdot 3 & 1 \cdot 2 + 0 \cdot 4 \\ 0 \cdot 1 + 0 \cdot 3 & 0 \cdot 2 + 0 \cdot 4 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 0 & 0 \end{pmatrix}.$$

$AB \neq BA$ — same ingredients, different order, different answer.

Basically, ... Addition of matrices behaves like addition of numbers. Multiplication keeps associativity and distributivity — but forgets commutativity: order matters. “Rotate then stretch” is not “stretch then rotate”. (In ML this is why $W_2(W_1\mathbf{x})$ means apply W_1 first.)

Problem set

1. Compute $(5, -2, 4) + 3(1, 0, -1) - (2, 3, 0)$.
2. Let $\mathbf{u} = (6, 8)$. Find $\|\mathbf{u}\|$ and the unit vector pointing in the same direction as $\mathbf{u}$.
3. Compute the dot product of $(3, -1, 2)$ and $(-2, 4, 5)$. What does the answer say about the angle between them?
4. Find the angle between $(1, 2)$ and $(2, 1)$, in degrees.
5. Write $(7, 5)$ as a linear combination of $(1, 2)$ and $(2, 1)$, i.e. find scalars a, b with $(7, 5) = a(1, 2) + b(2, 1)$.
6. For $A = \begin{pmatrix} 2 & 0 & -1 \\ 1 & 3 & 4 \end{pmatrix}$: what is its shape? What is the $(2, 1)$ -th entry? Is A square? Is it diagonal?
7. With $A = \begin{pmatrix} 1 & -2 \\ 0 & 3 \end{pmatrix}$ and $B = \begin{pmatrix} 4 & 1 \\ 2 & -1 \end{pmatrix}$, compute $A + B$ and $2A - B$.
8. Multiply $\begin{pmatrix} 2 & -1 \\ 0 & 3 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & -1 \end{pmatrix}$, showing each entry’s computation.
9. For $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and $B = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$, compute AB and BA . Are they equal?

10. Let $\mathbf{v} = (5, 6)$ and $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$. Compute $A\mathbf{v}$ two ways — row-times-column and as a linear combination of the columns of A — and confirm both give the same vector.
11. True or false, with a reason: “If A is $m \times n$ and B is $n \times p$, then AB is defined and has shape $p \times m$.”
12. Is $\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$ symmetric? Is the transpose of a row vector a column vector?

Where this goes next

This chapter gave you the objects and the operations. The coming chapters put them to work:

- **Chapter 3 (linear systems):** the $Ax = \mathbf{b}$ in §2.10’s columns view — “which recipe of columns makes $\mathbf{b}$?” — becomes solving systems of linear equations.
- **Chapter 4 (vector spaces):** the loose facts about adding and scaling vectors (§2.3, §2.6) get their proper home: the axioms every vector space obeys.
- **Chapter 5 (orthogonality):** the dot product’s zero (§2.5) gets a name — *orthogonal* — plus projections and the geometry they unlock.

Solutions to Chapter 2: Vectors and matrices

1. $(5, -2, 4) + 3(1, 0, -1) - (2, 3, 0)$. Scale first, then add coordinatewise:

$$3(1, 0, -1) = (3, 0, -3),$$

$$(5, -2, 4) + (3, 0, -3) = (5 + 3, -2 + 0, 4 - 3) = (8, -2, 1),$$

$$(8, -2, 1) - (2, 3, 0) = (8 - 2, -2 - 3, 1 - 0) = (6, -5, 1).$$

2. $\mathbf{u} = (6, 8)$. Norm:

$$\|\mathbf{u}\| = \sqrt{6^2 + 8^2} = \sqrt{36 + 64} = \sqrt{100} = 10.$$

Unit vector in the same direction: divide by the length,

$$\hat{\mathbf{u}} = \frac{(6, 8)}{10} = \left(\frac{3}{5}, \frac{4}{5}\right).$$

Check: $\left\|\left(\frac{3}{5}, \frac{4}{5}\right)\right\| = \sqrt{\frac{9}{25} + \frac{16}{25}} = \sqrt{1} = 1.$

3. $(3, -1, 2) \cdot (-2, 4, 5) = 3 \times (-2) + (-1) \times 4 + 2 \times 5 = -6 - 4 + 10 = 0$. The dot product is zero, so $\cos \theta = 0$ and the angle between them is 90° — the vectors are at right angles to each other (this gets the name *orthogonal* in Chapter 5).
4. $\mathbf{u} = (1, 2)$, $\mathbf{v} = (2, 1)$:

$$\mathbf{u} \cdot \mathbf{v} = 1 \times 2 + 2 \times 1 = 2 + 2 = 4,$$

$$\|\mathbf{u}\| = \sqrt{1 + 4} = \sqrt{5}, \quad \|\mathbf{v}\| = \sqrt{4 + 1} = \sqrt{5},$$

$$\cos \theta = \frac{4}{\sqrt{5} \cdot \sqrt{5}} = \frac{4}{5} = 0.8 \Rightarrow \theta = \cos^{-1}(0.8) \approx 36.87^\circ.$$

5. Want $(7, 5) = a(1, 2) + b(2, 1)$. Expanding the right side: $(a + 2b, 2a + b)$. Matching components:

$$\begin{cases} a + 2b = 7 \\ 2a + b = 5 \end{cases}$$

From the first, $a = 7 - 2b$. Substituting into the second: $2(7 - 2b) + b = 5$, i.e. $14 - 3b = 5$, so $3b = 9$ and $b = 3$. Then $a = 7 - 2 \times 3 = 1$. Check: $1(1, 2) + 3(2, 1) = (1 + 6, 2 + 3) = (7, 5)$.

6. $A = \begin{pmatrix} 2 & 0 & -1 \\ 1 & 3 & 4 \end{pmatrix}$ has 2 rows and 3 columns: shape 2×3 . The $(2, 1)$ -th entry is the row-2, column-1 entry: $A_{21} = 1$. Not square ($2 \neq 3$), hence not diagonal either.

7. $A + B$ entrywise:

$$\begin{pmatrix} 1 & -2 \\ 0 & 3 \end{pmatrix} + \begin{pmatrix} 4 & 1 \\ 2 & -1 \end{pmatrix} = \begin{pmatrix} 1+4 & -2+1 \\ 0+2 & 3+(-1) \end{pmatrix} = \begin{pmatrix} 5 & -1 \\ 2 & 2 \end{pmatrix}.$$

$$2A = \begin{pmatrix} 2 & -4 \\ 0 & 6 \end{pmatrix}, \text{ so}$$

$$2A - B = \begin{pmatrix} 2-4 & -4-1 \\ 0-2 & 6-(-1) \end{pmatrix} = \begin{pmatrix} -2 & -5 \\ -2 & 7 \end{pmatrix}.$$

8. 3×2 times 2×2 : inner dimensions match ($2 = 2$), so the result is 3×2 . Each entry = row of left dotted with column of right:

- $(1, 1)$: $(2, -1) \cdot (1, 3) = 2 \times 1 + (-1) \times 3 = 2 - 3 = -1$
- $(1, 2)$: $(2, -1) \cdot (2, -1) = 2 \times 2 + (-1) \times (-1) = 4 + 1 = 5$
- $(2, 1)$: $(0, 3) \cdot (1, 3) = 0 \times 1 + 3 \times 3 = 9$
- $(2, 2)$: $(0, 3) \cdot (2, -1) = 0 \times 2 + 3 \times (-1) = -3$
- $(3, 1)$: $(1, 1) \cdot (1, 3) = 1 \times 1 + 1 \times 3 = 4$
- $(3, 2)$: $(1, 1) \cdot (2, -1) = 1 \times 2 + 1 \times (-1) = 1$

$$\begin{pmatrix} 2 & -1 \\ 0 & 3 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & -1 \end{pmatrix} = \begin{pmatrix} -1 & 5 \\ 9 & -3 \\ 4 & 1 \end{pmatrix}.$$

- 9.

$$AB = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} = \begin{pmatrix} 0 \cdot 2 + 1 \cdot 0 & 0 \cdot 0 + 1 \cdot 3 \\ 1 \cdot 2 + 0 \cdot 0 & 1 \cdot 0 + 0 \cdot 3 \end{pmatrix} = \begin{pmatrix} 0 & 3 \\ 2 & 0 \end{pmatrix},$$

$$BA = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 \cdot 0 + 0 \cdot 1 & 2 \cdot 1 + 0 \cdot 0 \\ 0 \cdot 0 + 3 \cdot 1 & 0 \cdot 1 + 3 \cdot 0 \end{pmatrix} = \begin{pmatrix} 0 & 2 \\ 3 & 0 \end{pmatrix}.$$

$AB \neq BA$ — another concrete case of non-commutativity.

10. Row-times-column: row 1 $\cdot (5, 6) = 1 \times 5 + 2 \times 6 = 5 + 12 = 17$; row 2 $\cdot (5, 6) = 3 \times 5 + 4 \times 6 = 15 + 24 = 39$. So $A\mathbf{v} = (17, 39)$.

Columns view: the columns of A are $\mathbf{a}_1 = (1, 3)$, $\mathbf{a}_2 = (2, 4)$, and

$$A\mathbf{v} = 5\mathbf{a}_1 + 6\mathbf{a}_2 = 5(1, 3) + 6(2, 4) = (5, 15) + (12, 24) = (17, 39).$$

Both views agree.

11. False. The product of an $m \times n$ matrix and an $n \times p$ matrix is $m \times p$ — the *outer* dimensions: rows from A , columns from B . ($p \times m$ is wrong in both order and meaning; e.g. a 2×3 times a 3×4 gives 2×4 , not 4×2 .)

12. Transpose flips rows and columns: $\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}^T = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$ — it equals itself, so yes, symmetric. And yes: a $1 \times n$ row vector's transpose is $n \times 1$, i.e. a column vector, directly from $(A^T)_{ij} = A_{ji}$.

3. Linear systems, determinants, inverses

3.1 What this chapter is for

Most of machine learning is, quietly, the business of solving $A\mathbf{x} = \mathbf{b}$.

- Fitting a line through data points (Chapter 5) ends in a system of equations — the *normal equations* $(A^T A)\mathbf{x} = A^T \mathbf{b}$.
- A neural network layer computes $\mathbf{y} = W\mathbf{x} + \mathbf{b}$; asking “which input produced this output?” is solving a linear system.
- Every optimizer step solves (approximately) a linear system built from derivatives.

So this chapter answers three questions about $A\mathbf{x} = \mathbf{b}$: does a solution exist, is it unique, and how do we compute it? The tools are *Gaussian elimination* (the practical workhorse), the *matrix inverse* (the clean way to think about the answer), and the *determinant* (the one number that detects whether an inverse exists).

3.2 Linear equations and systems of linear equations

Def. A *linear equation* in the *variables* (or *unknowns*) $x_1, x_2, \dots, x_n$ = an equation of the form

$$a_1x_1 + a_2x_2 + \dots + a_nx_n = b,$$

where the *coefficients* $a_1, a_2, \dots, a_n$ and the right-hand side b are real numbers.

- i) $2x + 3y + 5z = -9$ is linear: the variables are x, y, z and the coefficients are 2, 3, 5.
- ii) $x^2 + y = 1$ is **not** linear (the x^2 breaks it); $xy = 4$ is **not** linear (a product of two variables breaks it).

Def. A *system of linear equations* = a collection of one or more linear equations involving the same set of variables.

Def. A *solution* = an assignment of values to the variables that makes every equation in the system true at the same time.

eg. Consider

$$x + 2y + z = 8, \quad 2x + y + z = 7, \quad x + y + 2z = 9.$$

Claim: $(x, y, z) = (1, 2, 3)$ is a solution. Check each equation: - $1 + 2(2) + 3 = 1 + 4 + 3 = 8$ - $2(1) + 2 + 3 = 2 + 2 + 3 = 7$ - $1 + 2 + 2(3) = 1 + 2 + 6 = 9$

All three hold simultaneously, so $(1, 2, 3)$ is indeed a solution.

The general shape — m equations in n unknowns — is:

$$\begin{aligned}
a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\
a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\
&\vdots \\
a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m
\end{aligned}$$

Basically, ... A linear equation is a flat recipe: each variable appears plain, multiplied by one number, everything added up — no squares, no products of variables. A system is several such recipes that must all balance at once, and a solution is one set of values that balances every one of them.

3.3 The matrix form $A\mathbf{x} = \mathbf{b}$, and the three possible outcomes

Writing the general system above with matrices compresses it to one line.

Def. The *matrix form* of a linear system is $A\mathbf{x} = \mathbf{b}$, where:

- A is the $m \times n$ *coefficient matrix* (the a_{ij} arranged in a table),
- $\mathbf{x} = (x_1, \dots, x_n)^T$ is the column vector of unknowns,
- $\mathbf{b} = (b_1, \dots, b_m)^T$ is the column vector of right-hand sides.

eg. The system of §3.2 in matrix form:

$$\begin{pmatrix} 1 & 2 & 1 \\ 2 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 8 \\ 7 \\ 9 \end{pmatrix},$$

so $A = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}$ (3×3), $\mathbf{x} = (x, y, z)^T$, $\mathbf{b} = (8, 7, 9)^T$.

Def. The *augmented matrix*, written $[A \mid \mathbf{b}]$, = the coefficient matrix with $\mathbf{b}$ glued on as one extra last column. It carries the whole system in one table.

Now recall the columns view from §2.10: if $\mathbf{a}_1, \dots, \mathbf{a}_n$ are the columns of A , then

$$A\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n.$$

So solving $A\mathbf{x} = \mathbf{b}$ is asking: “**which recipe of the columns of A makes $\mathbf{b}$?**” The unknowns x_i are the amounts of each column-ingredient.

There are exactly three possible answers to that question:

- a *unique solution* — exactly one recipe works;
- no solution* — no recipe can make $\mathbf{b}$ (the system is *inconsistent*);
- infinitely many solutions* — whole families of recipes work.

eg (unique solution).

$$2x + y = 215$$

$$3x + y = 260$$

Subtract the first equation from the second: $(3x+y) - (2x+y) = 260 - 215$, so $x = 45$. Then $y = 215 - 2(45) = 215 - 90 = 125$. Check: $3(45) + 125 = 135 + 125 = 260$. One solution: $(45, 125)$.

eg (infinitely many solutions).

$$2x + y = 215$$

$$4x + 2y = 430$$

The second equation is just twice the first ($2 \times (2x + y) = 2 \times 215$). It adds no new information: every (x, y) on the line $y = 215 - 2x$ satisfies both. E.g. $x = 100$ gives $y = 15$, and $4(100) + 2(15) = 430$.

eg (no solution).

$$\begin{aligned} 2x + y &= 215 \\ 4x + 2y &= 400 \end{aligned}$$

Twice the first equation demands $4x + 2y = 430$, but the second demands $4x + 2y = 400$. Both cannot hold: $430 \neq 400$. No (x, y) satisfies both.

The geometry: with two variables, each equation is a line in $\mathbb{R}^2$, so two equations = two lines. The three cases are exactly the three ways two lines can relate:

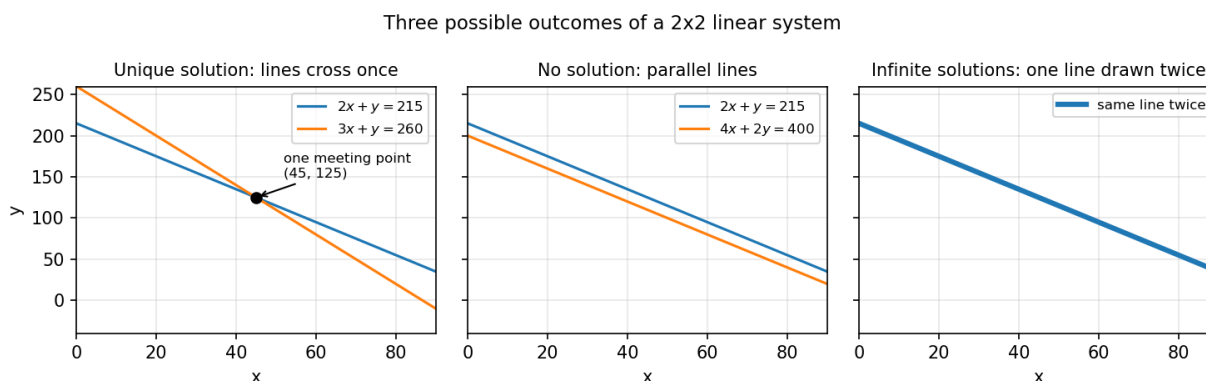


Figure 5: Three panels: two lines crossing at $(45, 125)$ for a unique solution; two parallel lines for no solution; a single line drawn twice for infinitely many solutions

With three variables each equation is a plane in $\mathbb{R}^3$; the same three outcomes occur (intersecting at one point / the same plane twice / parallel planes).

Basically, ... Two equations in two unknowns = two lines drawn on paper. Either they cross at exactly one point (one answer), they are the same line drawn twice (infinite answers — every point on it), or they run parallel forever and never meet (no answer). All the machinery in this chapter is just a reliable way to decide which of these three you are looking at, and to find the point — or points — when they exist.

3.4 Elementary row operations and echelon forms

The strategy for solving: reshape the augmented matrix into a simpler system with the same solutions, using moves that never change the solution set.

Def. The three *elementary row operations* on a matrix are:

1. $R_i \leftrightarrow R_j$: interchange two rows (reorder two equations — harmless).
2. $R_i \leftarrow cR_i$ with $c \neq 0$: multiply a row by a nonzero constant (scale both sides of an equation — harmless).
3. $R_i \leftarrow R_i + cR_j$: add a multiple of one row to another (combine two equations — harmless).

Applying any of these to $[A \mid \mathbf{b}]$ leaves the solution set of $A\mathbf{x} = \mathbf{b}$ unchanged.

The target shape is a staircase.

Def. A matrix is in *row echelon form (REF)* if:

- i) the first nonzero entry of each nonzero row (its *leading entry* or *pivot*) sits in a column strictly to the right of the leading entry of the row above;

ii) all-zero rows, if any, sit at the bottom.

It is in *reduced row echelon form (RREF)* if, in addition:

- iii) every leading entry equals 1;
- iv) each leading 1 is the only nonzero entry in its column.

eg.

$$A_{\text{ref}} = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

is in REF: leading entries at columns 1, 3, 4 form a staircase. It is **not** RREF: the leading 1 in column 3 has a 3 above it.

$$A_{\text{rref}} = \begin{pmatrix} 1 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

is in RREF: leading 1s at columns 1, 3, 4, each alone in its column.

$$\begin{pmatrix} 0 & 1 & 2 \\ 1 & 0 & 3 \end{pmatrix}$$

is not even REF: the staircase runs the wrong way (row 1 leads in column 2, row 2 in column 1).

Note: the source slides bundle conditions (i)–(iv) into a single “(Reduced)Row echelon form” definition, requiring leading entries to be 1 from the start. This chapter uses the standard split: REF = staircase only, RREF = staircase + clean leading 1s.

Basically, ... Echelon form is “stairs of equations”: each equation starts further right than the one above, and dead all-zero rows get swept to the bottom. Reduced means the stairs are extra tidy — every step starts with a clean 1 and nothing else in its column. Once the system looks like stairs, the answers are easy to read off.

3.5 Gaussian elimination and back-substitution

Def. *Gaussian elimination* = the procedure:

1. Form the augmented matrix $[A \mid \mathbf{b}]$.
2. Apply elementary row operations until the left block is in (reduced) row echelon form: $[A \mid \mathbf{b}] \rightarrow [R \mid \mathbf{c}]$.
3. Solve $R\mathbf{x} = \mathbf{c}$ — its solutions are exactly the solutions of $A\mathbf{x} = \mathbf{b}$.

Reading the answer off $[R \mid \mathbf{c}]$:

- If some row reads $0 = c_i$ with $c_i \neq 0$ (a zero row of R paired with a nonzero entry of $\mathbf{c}$), the system has **no solution** — that row demands $0 = c_i$, impossible.
- Otherwise: a variable x_i is *dependent* if its column contains a leading 1, and *independent (free)* if not. Give each independent variable any value you like; then each dependent variable is forced by the unique row in which it leads. Every solution arises this way.

eg (full 3×3 elimination). Solve

$$\begin{aligned} 2x + y - z &= 1 \\ x + y + z &= 6 \\ 3x - y + z &= 4. \end{aligned}$$

Step 1 — augmented matrix:

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 1 \\ 1 & 1 & 1 & 6 \\ 3 & -1 & 1 & 4 \end{array} \right]$$

Step 2 — $R_1 \leftrightarrow R_2$ (get a 1 in the top-left):

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & 1 & -1 & 1 \\ 3 & -1 & 1 & 4 \end{array} \right]$$

Step 3 — $R_2 \leftarrow R_2 - 2R_1$: $[2 - 2, 1 - 2, -1 - 2 \mid 1 - 12] = [0, -1, -3 \mid -11]$:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & -1 & -3 & -11 \\ 3 & -1 & 1 & 4 \end{array} \right]$$

Step 4 — $R_3 \leftarrow R_3 - 3R_1$: $[3 - 3, -1 - 3, 1 - 3 \mid 4 - 18] = [0, -4, -2 \mid -14]$:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & -1 & -3 & -11 \\ 0 & -4 & -2 & -14 \end{array} \right]$$

Step 5 — $R_2 \leftarrow -R_2$ (leading 1 in row 2):

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & 3 & 11 \\ 0 & -4 & -2 & -14 \end{array} \right]$$

Step 6 — $R_3 \leftarrow R_3 + 4R_2$: $[0, -4 + 4, -2 + 12 \mid -14 + 44] = [0, 0, 10 \mid 30]$:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & 3 & 11 \\ 0 & 0 & 10 & 30 \end{array} \right] \quad (\text{row echelon form})$$

Step 7 — $R_3 \leftarrow R_3/10$: $z = 3$.

Step 8 — *back-substitution* (solve bottom-up). From row 2: $y + 3z = 11$, so $y = 11 - 3(3) = 11 - 9 = 2$. From row 1: $x + y + z = 6$, so $x = 6 - 2 - 3 = 1$.

Solution: $(x, y, z) = (1, 2, 3)$. Verify: $2(1) + 2 - 3 = 1$; $1 + 2 + 3 = 6$; $3(1) - 2 + 3 = 4$.

eg (reading RREF: infinitely many solutions).

$$\left[\begin{array}{ccc|c} 1 & 0 & 2 & 5 \\ 0 & 1 & -1 & 3 \end{array} \right]$$

reads $x + 2z = 5$, $y - z = 3$. Leading 1s sit in columns 1 and 2, so x, y are dependent; column 3 has none, so z is independent. Set $z = t$ (any real number): then $x = 5 - 2t$, $y = 3 + t$. Infinitely many solutions, one per value of t . Check $t = 0$: $(5, 3, 0)$ gives $5 + 0 = 5$ and $3 - 0 = 3$.

eg (reading RREF: no solution).

$$\left[\begin{array}{cc|c} 1 & 2 & 4 \\ 0 & 0 & 7 \end{array} \right]$$

reads $x + 2y = 4$ and $0 = 7$. The second row is impossible, so the system has no solution.

Basically, ... Gaussian elimination is “tidy the equations, then read the answers.” You combine and scale equations until they form stairs; the bottom stair tells you the last variable, and you climb back up substituting. If a stair ever says $0 = 7$, there is no answer; if some variable never gets a stair of its own, it is free and you get infinitely many answers.

3.6 The matrix inverse

Recall §1.10: a one-one function f has an inverse f^{-1} that undoes it — $f^{-1}(f(x)) = x$. A (square) matrix is a machine that transforms vectors; its inverse, when it exists, is the machine that reverses the transformation.

Def. Let A be an $n \times n$ matrix. Its *inverse*, written A^{-1} , is the $n \times n$ matrix satisfying

$$AA^{-1} = A^{-1}A = I.$$

Only square matrices can have inverses, and not every square matrix does.

For 2×2 matrices there is a direct formula. With $\det(A) = ad - bc$:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \quad \text{provided } ad - bc \neq 0.$$

Mnemonic: swap the diagonal entries, flip the signs of the off-diagonal entries, divide everything by the determinant.

eg. $A = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$. $\det(A) = 2(1) - 1(1) = 1 \neq 0$, so

$$A^{-1} = \frac{1}{1} \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix}.$$

Verify $AA^{-1} = I$:

$$\begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 2(1) + 1(-1) & 2(-1) + 1(2) \\ 1(1) + 1(-1) & 1(-1) + 1(2) \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

The undo machine works.

If A^{-1} exists, solving $A\mathbf{x} = \mathbf{b}$ is one step: multiply both sides on the left by A^{-1} :

$$A\mathbf{x} = \mathbf{b} \quad \Rightarrow \quad \mathbf{x} = A^{-1}\mathbf{b}.$$

Note (one honest line): forming A^{-1} costs roughly three times the arithmetic of plain elimination and rounds worse, so nobody solves $A\mathbf{x} = \mathbf{b}$ by inverting in practice — elimination (and fancier factorizations) is the workhorse; the inverse is the clean way to *think* about the solution.

When does the inverse fail to exist? A matrix with no inverse is called *singular*. Preview of §3.7: A is singular exactly when $\det(A) = 0$.

eg (singular matrix). $S = \begin{pmatrix} 2 & 4 \\ 1 & 2 \end{pmatrix}$ has $\det(S) = 2(2) - 4(1) = 0$ — no inverse exists. Notice why: row 1 is $2 \times$ row 2, so the two rows carry the same information; S squashes distinct vectors together and cannot be undone.

Two facts worth filing: $\det(A^{-1}) = \frac{1}{\det(A)}$, and for larger matrices A^{-1} is found by row-reducing $[A \mid I]$ to $[I \mid A^{-1}]$ — elimination again, solving $A\mathbf{x} = \mathbf{e}_i$ for each column.

Note (*homogeneous systems*): $A\mathbf{x} = \mathbf{0}$ always has the *trivial solution* $\mathbf{x} = \mathbf{0}$. If A is invertible it is the *only* solution ($\mathbf{x} = A^{-1}\mathbf{0} = \mathbf{0}$); if $\det(A) = 0$ there are infinitely many others. And if there are more unknowns than equations, a nonzero solution is guaranteed.

Basically, ... The inverse is the “undo button” for a matrix: whatever A does to a vector, A^{-1} reverses. Just like functions in §1.10, undoing only works when nothing got merged — a matrix that squashes two different vectors into one (determinant zero) cannot be undone, so it has no inverse.

3.7 Determinants

Def. The *determinant* of a square matrix A , written $\det(A)$ or $|A|$, = a single number computed from the entries of A . Only square matrices have determinants. It is the test for invertibility ($\det(A) \neq 0 \Leftrightarrow A$ invertible) and appears in solution formulas.

1×1 : $\det([a]) = a$.

2×2 :

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc.$$

eg. $\det \begin{pmatrix} 2 & 3 \\ 6 & 10 \end{pmatrix} = 2(10) - 3(6) = 20 - 18 = 2$.

For 3×3 and beyond, the determinant is built recursively. The (i, j) -th *minor* M_{ij} = the determinant of the submatrix left after deleting row i and column j ; the (i, j) -th *cofactor* is $C_{ij} = (-1)^{i+j} M_{ij}$ (the sign follows a checkerboard: $\begin{smallmatrix} + & - & + \\ - & + & - \\ + & - & + \end{smallmatrix}$). Expanding along the first row:

$$\det(A) = a_{11}C_{11} + a_{12}C_{12} + a_{13}C_{13}.$$

In fact you may expand along *any* row or column — pick the one with the most zeros.

eg (3×3 , full steps). $A = \begin{pmatrix} 2 & 4 & 1 \\ 3 & 8 & 7 \\ 5 & 6 & 9 \end{pmatrix}$, expanding along row 1:

$$\begin{aligned} \det(A) &= 2 \det \begin{pmatrix} 8 & 7 \\ 6 & 9 \end{pmatrix} - 4 \det \begin{pmatrix} 3 & 7 \\ 5 & 9 \end{pmatrix} + 1 \det \begin{pmatrix} 3 & 8 \\ 5 & 6 \end{pmatrix} \\ &= 2(72 - 42) - 4(27 - 35) + (18 - 40) \\ &= 2(30) - 4(-8) + (-22) \\ &= 60 + 32 - 22 = 70. \end{aligned}$$

eg (*triangular shortcut*). For an upper (or lower) triangular matrix the determinant is just the product of the diagonal entries:

$$\det \begin{pmatrix} 2 & 4 & 3 \\ 0 & 8 & 7 \\ 0 & 0 & 9 \end{pmatrix} = 2 \cdot 8 \cdot 9 = 144.$$

Geometric meaning (one paragraph): $|\det(A)|$ is the factor by which the linear map $\mathbf{x} \mapsto A\mathbf{x}$ scales areas (2×2) or volumes (3×3) — the unit square becomes a parallelogram of area $|\det(A)|$, the unit cube a parallelepiped of volume $|\det(A)|$. The sign records *orientation*: $\det(A) > 0$ preserves it, $\det(A) < 0$ flips it (a mirror). $\det(A) = 0$ means the map flattens space — area collapses to zero — which is exactly why a singular matrix cannot be undone.

eg. $A = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$ has $\det(A) = 4$: it doubles every length, so the unit square (area 1) becomes a 2×2 square (area 4).

Basically, ... The determinant is one number that says “how much does this matrix stretch space?” A determinant of 4 means every area becomes 4 times bigger; a determinant of 0 means the matrix flattens everything into something with no area at all — which is exactly why such a matrix can’t be undone.

The properties we will reuse later:

- i) $\det(AB) = \det(A)\det(B)$ — determinants multiply across products.
- ii) $\det(A^T) = \det(A)$ — transposing changes nothing.

- iii) $\det(I) = 1$ and $\det(A^{-1}) = \frac{1}{\det(A)}$.
- iv) Swapping two rows (or two columns) flips the sign.
- v) Adding a multiple of one row to another leaves the determinant unchanged.
- vi) Multiplying a row by t multiplies the determinant by t .
- vii) A zero row or column, two identical rows, or one row that is a combination of the others gives $\det = 0$.
- viii) Determinants can be computed by row reduction: reduce to triangular form, tracking how each operation changed the value:

Operation	Effect on det
$R_i \leftrightarrow R_j$ (swap)	sign flips
$R_i \leftarrow cR_i$ (scale)	multiplied by c
$R_i \leftarrow R_i + cR_j$ (add multiple)	unchanged

eg. $\det \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ via row reduction: $R_2 \leftarrow R_2 - 3R_1$ (type 3, det unchanged) gives $\begin{pmatrix} 1 & 2 \\ 0 & -2 \end{pmatrix}$, triangular, so $\det = 1 \cdot (-2) = -2$. Direct check: $1(4) - 2(3) = -2$.

3.8 Cramer's rule

When A is invertible ($\det(A) \neq 0$), each unknown has a closed-form expression in determinants. Let A_i be the matrix formed by replacing column i of A with $\mathbf{b}$. Then:

$$x_i = \frac{\det(A_i)}{\det(A)} \quad i = 1, \dots, n.$$

This is *Cramer's rule*. It needs $\det(A) \neq 0$ (division by zero is not allowed), and it is mostly a theoretical tool: each x_i costs a full determinant, so for large systems it is far more work than elimination.

eg. Solve $2x + y = 7$, $x + 3y = 11$. $A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$, $\det(A) = 2(3) - 1(1) = 5$. $A_1 = \begin{pmatrix} 7 & 1 \\ 11 & 3 \end{pmatrix}$ (column 1 replaced by $\mathbf{b}$): $\det(A_1) = 7(3) - 1(11) = 21 - 11 = 10$, so $x = 10/5 = 2$. $A_2 = \begin{pmatrix} 2 & 7 \\ 1 & 11 \end{pmatrix}$ (column 2 replaced by $\mathbf{b}$): $\det(A_2) = 2(11) - 7(1) = 22 - 7 = 15$, so $y = 15/5 = 3$. Check: $2(2) + 3 = 7$; $2 + 3(3) = 11$.

Basically, ... Cramer's rule solves the system with pure determinants: each variable is a ratio of two determinants — swap in the answer column, take the determinant, divide. Elegant on paper, impractical at scale: it needs $n + 1$ determinants, each of which is itself expensive to compute.

3.9 Worked example set

eg 1 — 3×3 determinant by cofactor expansion. Compute $\det(A)$ for $A = \begin{pmatrix} 2 & 4 & 1 \\ 3 & 8 & 7 \\ 5 & 6 & 9 \end{pmatrix}$.

Step 1: Expand along row 1 (signs +, −, +):

$$\det(A) = 2 \cdot M_{11} - 4 \cdot M_{12} + 1 \cdot M_{13}.$$

Step 2: Minors (delete the row and column of each entry): $- M_{11} = \det \begin{pmatrix} 8 & 7 \\ 6 & 9 \end{pmatrix} = 8(9) - 7(6) = 72 - 42 = 30$.

$- M_{12} = \det \begin{pmatrix} 3 & 7 \\ 5 & 9 \end{pmatrix} = 3(9) - 7(5) = 27 - 35 = -8$. $- M_{13} = \det \begin{pmatrix} 3 & 8 \\ 5 & 6 \end{pmatrix} = 3(6) - 8(5) = 18 - 40 = -22$.

Step 3: Combine: $\det(A) = 2(30) - 4(-8) + 1(-22) = 60 + 32 - 22 = 70$.

eg 2 — Inverse and $\mathbf{x} = A^{-1}\mathbf{b}$, full steps. Solve $\begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}$ via the inverse.

Step 1: $\det(A) = 2(1) - 1(1) = 1 \neq 0$, so A^{-1} exists.

Step 2: $A^{-1} = \frac{1}{1} \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix}$ (swap diagonal, negate off-diagonal).

Step 3: $\begin{pmatrix} x \\ y \end{pmatrix} = A^{-1}\mathbf{b} = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 5 \\ 3 \end{pmatrix} = \begin{pmatrix} 1(5) + (-1)(3) \\ (-1)(5) + 2(3) \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$.

Step 4: Verify in the original system: $2(2) + 1 = 5$; $2 + 1 = 3$.

eg 3 — Reading infinitely many solutions from RREF. The augmented matrix of a system reduces to

$$\left[\begin{array}{ccc|c} 1 & 0 & 2 & 5 \\ 0 & 1 & -1 & 3 \\ 0 & 0 & 0 & 0 \end{array} \right].$$

Step 1: No row reads $0 = \text{nonzero}$ (the zero row pairs with 0), so solutions exist.

Step 2: Leading 1s are in columns 1 and 2 $\rightarrow x, y$ dependent; column 3 $\rightarrow z$ independent.

Step 3: Set $z = t$. The rows read $x + 2z = 5$ and $y - z = 3$, so $x = 5 - 2t$, $y = 3 + t$.

Step 4: Solution family: $(x, y, z) = (5 - 2t, 3 + t, t)$ for any $t \in \mathbb{R}$.

eg 4 — Spotting “no solution” early. Augmented matrix:

$$\left[\begin{array}{cc|c} 1 & 2 & 4 \\ 2 & 4 & 10 \end{array} \right].$$

$R_2 \leftarrow R_2 - 2R_1$: $[2 - 2, 4 - 4 \mid 10 - 8] = [0, 0 \mid 2]$. The second row reads $0 = 2$ — impossible. No solution. (Geometrically: $x + 2y = 4$ and $x + 2y = 5$ are parallel lines.)

eg 5 — A singular system has no inverse to save it. $A = \begin{pmatrix} 3 & 6 \\ 1 & 2 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} 9 \\ 4 \end{pmatrix}$.

Step 1: $\det(A) = 3(2) - 6(1) = 0$. Singular: no inverse, and neither $\mathbf{x} = A^{-1}\mathbf{b}$ nor Cramer’s rule applies.

Step 2: Eliminate anyway: $R_2 \leftarrow 3R_2 - R_1$ gives $[3 - 3, 6 - 6 \mid 12 - 9] = [0, 0 \mid 3]$, i.e. $0 = 3$. No solution.

Step 3: Moral: $\det(A) = 0$ does not automatically mean “no solution” — with $\mathbf{b} = (9, 3)^T$ the same A would give $0 = 0$ and infinitely many solutions. Zero determinant means “elimination must decide”; the inverse simply does not exist to decide it.

3.10 Problem set

Full worked solutions are in the companion solutions volume (`solutions/03-linear-systems-determinants-inverses.md`).

- Write the system $3x + 2y = 12$, $x - y = 1$ in the form $A\mathbf{x} = \mathbf{b}$ (state A , $\mathbf{x}$, $\mathbf{b}$) and give its augmented matrix $[A \mid \mathbf{b}]$.
- Solve by Gaussian elimination, showing every row operation:

$$x + y + z = 5, \quad 2x - y + 3z = 17, \quad x + 2y - z = -4.$$

- For each system, say whether it has a unique solution, no solution, or infinitely many solutions — and give the solution(s):
 - $x + 2y = 4$, $3x + 6y = 12$;

- ii) $x + 2y = 4$, $3x + 6y = 10$;
 iii) $x + 2y = 4$, $x - y = 1$.
4. Let $A = \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix}$.
- i) Compute A^{-1} using the 2×2 formula, and verify $AA^{-1} = I$.
 ii) Use it to solve $A \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}$.
5. Explain in two lines why $A = \begin{pmatrix} 2 & 4 \\ 1 & 2 \end{pmatrix}$ has no inverse.
6. Compute:
- i) $\det \begin{pmatrix} 5 & -2 \\ 3 & 4 \end{pmatrix}$;
 ii) $\det \begin{pmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{pmatrix}$ (expand along a row or column of your choice);
 iii) $\det \begin{pmatrix} 3 & 1 & 2 \\ 0 & -2 & 5 \\ 0 & 0 & 4 \end{pmatrix}$ (use the triangular shortcut).
7. If A is 3×3 with $\det(A) = 5$, what is $\det(2A)$? Justify using the row-scaling property.
8. Use Cramer's rule to solve $2x + y = 7$, $x + 3y = 11$. (Compare with §3.8.)
9. Describe all solutions of the homogeneous system $x + 2y = 0$, $3x + 6y = 0$. Why must there be infinitely many?
10. True or false, with a one-line reason:
- i) A matrix with a zero row has determinant 0.
 ii) If $\det(A) \neq 0$ then $A\mathbf{x} = \mathbf{b}$ has exactly one solution.
 iii) Swapping two rows of a matrix changes the sign of its determinant.
 iv) $\det(A + B) = \det(A) + \det(B)$ for all square A, B .
11. The RREF of an augmented matrix is $\left[\begin{array}{ccc|c} 1 & 2 & 0 & 4 \\ 0 & 0 & 1 & 3 \end{array} \right]$. Identify the dependent and independent variables and write down all solutions.

3.11 Where this goes next

- **Chapter 4 (vector spaces):** “When does $A\mathbf{x} = \mathbf{b}$ have solutions?” gets its proper answer: $\mathbf{b}$ must lie in the *span* of the columns of A (the *column space*), and the solutions of $A\mathbf{x} = \mathbf{0}$ form the *null space*. Unique / none / infinite stops being a list and becomes geometry.
- **Chapter 5 (least squares):** real data gives a tall A (more equations than unknowns) — no inverse exists, and usually no exact solution either. The best-fit answer solves the *normal equations* $(A^T A)\mathbf{x} = A^T \mathbf{b}$, where the inverse (of $A^T A$) finally earns its keep.
- Determinants return later with change-of-variables: when a transformation stretches space, probability densities must be reweighted by exactly $|\det|$ — the scaling factor of §3.7 doing real work.

Solutions — Chapter 3: Linear systems, determinants, inverses

Worked solutions to the problem set in `chapters/03-linear-systems-determinants-inverses.md`. Same numbering, full steps.

1. The unknowns are x, y ; the coefficient table and right-hand side are read straight off the equations:

$$A = \begin{pmatrix} 3 & 2 \\ 1 & -1 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 12 \\ 1 \end{pmatrix},$$

so $A\mathbf{x} = \mathbf{b}$ is $\begin{pmatrix} 3 & 2 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 12 \\ 1 \end{pmatrix}$. The augmented matrix glues $\mathbf{b}$ on as a last column:

$$[A \mid \mathbf{b}] = \left[\begin{array}{cc|c} 3 & 2 & 12 \\ 1 & -1 & 1 \end{array} \right].$$

2. Augmented matrix:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 5 \\ 2 & -1 & 3 & 17 \\ 1 & 2 & -1 & -4 \end{array} \right]$$

Step 1 — $R_2 \leftarrow R_2 - 2R_1$: $[2 - 2, -1 - 2, 3 - 2 \mid 17 - 10] = [0, -3, 1 \mid 7]$.

Step 2 — $R_3 \leftarrow R_3 - R_1$: $[1 - 1, 2 - 1, -1 - 1 \mid -4 - 5] = [0, 1, -2 \mid -9]$.

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 5 \\ 0 & -3 & 1 & 7 \\ 0 & 1 & -2 & -9 \end{array} \right]$$

Step 3 — $R_2 \leftrightarrow R_3$ (nicer pivot):

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 5 \\ 0 & 1 & -2 & -9 \\ 0 & -3 & 1 & 7 \end{array} \right]$$

Step 4 — $R_3 \leftarrow R_3 + 3R_2$: $[0, -3 + 3, 1 - 6 \mid 7 - 27] = [0, 0, -5 \mid -20]$, so $-5z = -20$ and $z = 4$.

Step 5 — back-substitution. Row 2: $y - 2z = -9$, so $y = -9 + 2(4) = -1$. Row 1: $x + y + z = 5$, so $x = 5 - (-1) - 4 = 2$.

Solution: $(x, y, z) = (2, -1, 4)$. Verify: $2 + (-1) + 4 = 5$; $2(2) - (-1) + 3(4) = 4 + 1 + 12 = 17$; $2 + 2(-1) - 4 = 2 - 2 - 4 = -4$.

3. i) Infinitely many. The second equation is $3 \times$ the first ($3x + 6y = 3 \cdot 4 = 12$): one equation in disguise. Solutions: all (x, y) with $x + 2y = 4$, e.g. $(x, y) = (4 - 2t, t)$. Check: $(4 - 2t) + 2t = 4$; $3(4 - 2t) + 6t = 12$.

ii) No solution. $3 \times$ the first equation demands $3x + 6y = 12$, but the second demands $3x + 6y = 10$: $12 \neq 10$, contradiction. (Parallel lines.)

iii) Unique solution. Subtract: $(x + 2y) - (x - y) = 4 - 1$, so $3y = 3$ and $y = 1$; then $x = 1 + y = 2$. Check: $2 + 2(1) = 4$; $2 - 1 = 1$. Solution: $(2, 1)$.

4. i) $\det(A) = 3(1) - 1(2) = 1 \neq 0$. By the 2×2 formula (swap diagonal, negate off-diagonal, divide by $\det$):

$$A^{-1} = \frac{1}{1} \begin{pmatrix} 1 & -1 \\ -2 & 3 \end{pmatrix} = \begin{pmatrix} 1 & -1 \\ -2 & 3 \end{pmatrix}.$$

Verify:

$$AA^{-1} = \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ -2 & 3 \end{pmatrix} = \begin{pmatrix} 3 - 2 & -3 + 3 \\ 2 - 2 & -2 + 3 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I.$$

ii) $\begin{pmatrix} x \\ y \end{pmatrix} = A^{-1}\mathbf{b} = \begin{pmatrix} 1 & -1 \\ -2 & 3 \end{pmatrix} \begin{pmatrix} 5 \\ 3 \end{pmatrix} = \begin{pmatrix} 5 - 3 \\ -10 + 9 \end{pmatrix} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$. Check: $3(2) + 1(-1) = 5$; $2(2) + 1(-1) = 3$

5. $\det(A) = 2(2) - 4(1) = 4 - 4 = 0$. A matrix has an inverse only if its determinant is nonzero; here it is zero, so A is singular — no inverse exists. (Equivalently: row 1 is $2 \times$ row 2, so the rows carry redundant information.)

6. i) $\det \begin{pmatrix} 5 & -2 \\ 3 & 4 \end{pmatrix} = 5(4) - (-2)(3) = 20 + 6 = 26$.

ii) Expand along row 1 (signs +, −, +):

$$\begin{aligned}\det &= 1 \det \begin{pmatrix} 4 & 5 \\ 0 & 6 \end{pmatrix} - 2 \det \begin{pmatrix} 0 & 5 \\ 1 & 6 \end{pmatrix} + 3 \det \begin{pmatrix} 0 & 4 \\ 1 & 0 \end{pmatrix} \\ &= 1(24 - 0) - 2(0 - 5) + 3(0 - 4) \\ &= 24 + 10 - 12 = 22.\end{aligned}$$

iii) Upper triangular: determinant = product of the diagonal = $3 \cdot (-2) \cdot 4 = -24$.

7. $\det(2A) = 40$. Reason: $2A$ scales every row of A by 2; scaling one row by t scales the determinant by t (property vi), and there are 3 rows, so $\det(2A) = 2 \cdot 2 \cdot 2 \cdot \det(A) = 8 \cdot 5 = 40$. (In general $\det(tA) = t^n \det(A)$ for $n \times n$ A .)

8. $A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} 7 \\ 11 \end{pmatrix}$, $\det(A) = 2(3) - 1(1) = 5 \neq 0$. - $A_1 = \begin{pmatrix} 7 & 1 \\ 11 & 3 \end{pmatrix}$: $\det(A_1) = 7(3) - 1(11) = 21 - 11 = 10$, so $x = 10/5 = 2$. - $A_2 = \begin{pmatrix} 2 & 7 \\ 1 & 11 \end{pmatrix}$: $\det(A_2) = 2(11) - 7(1) = 22 - 7 = 15$, so $y = 15/5 = 3$.

Check: $2(2) + 3 = 7$; $2 + 3(3) = 11$.

9. From the first equation $x = -2y$; the second is $3 \times$ the first, hence redundant. Put $y = t$ (free): all solutions are $(x, y) = (-2t, t)$, $t \in \mathbb{R}$ — infinitely many. Why infinitely many: $\det \begin{pmatrix} 1 & 2 \\ 3 & 6 \end{pmatrix} = 6 - 6 = 0$, so the coefficient matrix is singular; a homogeneous system with singular coefficient matrix always has nontrivial solutions beyond $\mathbf{0}$.

10. i) True — a zero row forces $\det = 0$ (property vii). ii) True — $\det(A) \neq 0$ means A is invertible, and then $\mathbf{x} = A^{-1}\mathbf{b}$ is the one and only solution. iii) True — swapping two rows flips the sign (property iv). iv) False. Counterexample: $A = B = I_2$. Then $\det(A + B) = \det \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} = 4$, but $\det(A) + \det(B) = 1 + 1 = 2 \neq 4$.

11. The rows read $x + 2y = 4$ and $z = 3$. Leading 1s are in columns 1 and 3, so x and z are dependent; column 2 has no leading 1, so y is independent. Set $y = t$: then $x = 4 - 2t$ and $z = 3$. All solutions: $(x, y, z) = (4 - 2t, t, 3)$, $t \in \mathbb{R}$ — infinitely many.

4. Vector spaces and the four fundamental subspaces

Chapter 3 asked “when does $Ax = \mathbf{b}$ have a solution?” and answered it with elimination, dependent and independent variables, and the three possible outcomes. This chapter gives those answers their proper home: *vector spaces*. The set of all $\mathbf{x}$ with $Ax = \mathbf{0}$, the set of all $\mathbf{b}$ for which $Ax = \mathbf{b}$ is solvable — both are *subspaces*. And every $m \times n$ matrix carries not one but four of them, tied together by one number: the *rank*.

4.1 What is a vector space?

The idea: a set where you can add things and scale things, and the results always stay inside the set, with the familiar arithmetic rules behaving.

Def. A *vector space* = a nonempty set V with two operations — *addition* ($+: V \times V \rightarrow V$) and *scalar multiplication* ($\cdot: \mathbb{R} \times V \rightarrow V$) — satisfying, for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ and all scalars a, b :

- i) *closure*: $\mathbf{u} + \mathbf{v} \in V$ and $a\mathbf{u} \in V$ — the operations never leave the set;
- ii) there is a *zero vector* $\mathbf{0}$ with $\mathbf{u} + \mathbf{0} = \mathbf{u}$;
- iii) every $\mathbf{u}$ has a *negative* $-\mathbf{u}$ with $\mathbf{u} + (-\mathbf{u}) = \mathbf{0}$;
- iv) addition behaves: $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$ and $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$;
- v) scaling behaves: $a(\mathbf{u} + \mathbf{v}) = a\mathbf{u} + a\mathbf{v}$, $(a + b)\mathbf{u} = a\mathbf{u} + b\mathbf{u}$, and $1\mathbf{u} = \mathbf{u}$.

A *vector* = an element of a vector space. (It is standard to write $a\mathbf{u}$ instead of $a \cdot \mathbf{u}$.)

eg (the working example). $\mathbb{R}^n$ with the coordinatewise addition and scaling of §2.3 is a vector space. Its zero vector is $(0, \dots, 0)$; the negative of $(v_1, \dots, v_n)$ is $(-v_1, \dots, -v_n)$.

eg (vectors need not be lists). The set $M_{2 \times 3}(\mathbb{R})$ of all 2×3 matrices of real numbers is a vector space, with matrix addition and scalar multiplication (§2.9). Its zero vector is the 2×3 zero matrix. So “vectors” are any objects with a good addition and scaling.

Note: the real number 0 and the zero vector $\mathbf{0}$ share a symbol; the context tells you which one is meant.

Basically, ... A vector space = a collection where “add two things” and “stretch one thing” always give you back something in the collection, and all the usual arithmetic rules hold. $\mathbb{R}^n$ is the main one in this book — but even plain matrices qualify as vectors.

4.2 Subspaces: spaces inside spaces

Def. A *subspace* = a nonempty subset W of a vector space V that is itself a vector space under the same operations.

You never need to check every axiom to prove something is a subspace — two checks are enough.

Def (the subspace test). A nonempty $W \subseteq V$ is a subspace iff:

- i) it is *closed under addition*: $\mathbf{w}_1, \mathbf{w}_2 \in W \Rightarrow \mathbf{w}_1 + \mathbf{w}_2 \in W$;
- ii) it is *closed under scaling*: $\mathbf{w} \in W, c \in \mathbb{R} \Rightarrow c\mathbf{w} \in W$.

Shortcut: if $\mathbf{0} \notin W$, it fails at once — since $0\mathbf{w} = \mathbf{0}$ must stay in W by (ii).

eg (subspace). The x -axis $W = \{(x, 0) : x \in \mathbb{R}\}$ in $\mathbb{R}^2$. Addition: $(x_1, 0) + (x_2, 0) = (x_1 + x_2, 0) \in W$. Scaling: $c(x, 0) = (cx, 0) \in W$. Verdict: subspace.

eg (not a subspace). The line $y = x + 1$ in $\mathbb{R}^2$: it misses the origin, $(0, 0) \notin W$. Verdict: not a subspace.

eg (not a subspace). The first quadrant $\{(x, y) : x \geq 0, y \geq 0\}$: closed under addition, but $(-1)(1, 1) = (-1, -1)$ escapes it. Verdict: not a subspace — scaling by negatives breaks it.

eg (subspace). Every V has two *trivial subspaces*: V itself, and $\{\mathbf{0}\}$ (the zero vector alone). Check $\{\mathbf{0}\}$: $\mathbf{0} + \mathbf{0} = \mathbf{0}$ and $c\mathbf{0} = \mathbf{0}$. Verdict: subspace.

Basically, ... A subspace = a flat piece through the origin that is “sealed”: add or stretch anything inside and you stay inside. A line through the origin qualifies; a line that misses the origin, or a quadrant, does not.

4.3 Span: everything a set can reach

Def. The *span* of a set S = the set of all (finite) *linear combinations* of elements of S :

$$\text{span}(S) = \left\{ \sum_{i=1}^k \alpha_i \mathbf{v}_i : \mathbf{v}_i \in S, \alpha_i \in \mathbb{R} \right\}.$$

S is a *spanning set* for V if $\text{span}(S) = V$.

Note: $\text{span}(S)$ is always a subspace — adding two combinations gives a combination, and scaling one gives a combination, so the subspace test passes.

eg (full steps). $S = \{(1, 1, 0), (1, 0, 0)\} \subset \mathbb{R}^3$:

$$\text{span}(S) = \{a(1, 1, 0) + b(1, 0, 0) : a, b \in \mathbb{R}\} = \{(a+b, a, 0) : a, b \in \mathbb{R}\}.$$

Write $c = a + b$: then every vector is $(c, a, 0)$ with c, a free — exactly the xy -plane (all vectors with third coordinate 0).

eg. $\text{span}\{(1, 0)\} = \{(a, 0) : a \in \mathbb{R}\} =$ the x -axis in $\mathbb{R}^2$. And $\{(1, 0), (0, 1)\}$ spans all of $\mathbb{R}^2$, since any $(x, y) = x(1, 0) + y(0, 1)$.

Basically, ... The span of a set = “everything you can cook from these ingredients.” One nonzero vector in $\mathbb{R}^2$ cooks a whole line; two non-parallel ones cook a whole plane; $(1, 0)$ and $(0, 1)$ cook all of $\mathbb{R}^2$.

4.4 Linear independence and dependence

The key question about any set of vectors: is anyone redundant?

Def. $\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ is *linearly dependent* if the zero vector can be written as a linear combination with *not all coefficients zero*:

$$\alpha_1 \mathbf{v}_1 + \dots + \alpha_k \mathbf{v}_k = \mathbf{0} \quad \text{with some } \alpha_i \neq 0.$$

It is *linearly independent* if the only way to make $\mathbf{0}$ is $\alpha_1 = \dots = \alpha_k = 0$.

The intuition: dependent = some vector is a combination of the others (redundant); independent = no vector can be built from the rest. Two vectors on the same line, or three vectors on the same plane, are always dependent.

eg (independence, full steps). Test $\{(1, 1), (1, -1)\}$. Set

$$a(1, 1) + b(1, -1) = (0, 0).$$

That reads $a + b = 0$ and $a - b = 0$. Adding the two equations: $2a = 0 \Rightarrow a = 0$; then $b = 0$. Only the trivial coefficients work — the set is *independent*.

eg (dependence, full steps). Test $\{(1, 2, 4), (2, -1, 2), (5, 0, 8)\}$. Set

$$a(1, 2, 4) + b(2, -1, 2) + c(5, 0, 8) = (0, 0, 0),$$

which reads

$$\begin{aligned} a + 2b + 5c &= 0 \\ 2a - b &= 0 \\ 4a + 2b + 8c &= 0. \end{aligned}$$

Try $a = -1$, $b = -2$, $c = 1$: first, $-1 - 4 + 5 = 0$; second, $-2 + 2 = 0$; third, $-4 - 4 + 8 = 0$. Nonzero coefficients make $\mathbf{0}$ — the set is *dependent*. In fact the third vector was redundant all along: $(5, 0, 8) = (1, 2, 4) + 2(2, -1, 2)$ — check $(1 + 4, 2 - 2, 4 + 4) = (5, 0, 8)$.

Note: any set containing the zero vector is dependent at once: $1 \cdot \mathbf{0} + 0 \cdot (\text{the rest}) = \mathbf{0}$ uses the nonzero coefficient 1.

Basically, ... Independence = “nobody here is a copy of the others.” To test: try to build the zero vector from the set. If the only recipe is “use zero of everything,” it is independent. If you find any real recipe with nonzero amounts, it is dependent — and one vector is secretly a mix of the rest.

4.5 Basis and dimension

Def. A *basis* of V = a set that is linearly independent **and** spans V . It is the smallest set that still describes the whole space — no redundancies, nothing missing.

Def. The *dimension* of V , $\dim(V)$, = the number of vectors in a basis. (Every basis of the same space has the same size.)

eg. The *standard basis* of $\mathbb{R}^n$: $\mathbf{e}_1 = (1, 0, \dots, 0)$, $\mathbf{e}_2 = (0, 1, \dots, 0)$, and so on. Independent (each brings a coordinate no other has), and spanning, since any $(v_1, \dots, v_n) = v_1 \mathbf{e}_1 + \dots + v_n \mathbf{e}_n$. So $\dim(\mathbb{R}^n) = n$.

eg (finding a basis, full steps). Let $W = \text{span}\{(1, 0, 0), (0, 1, 0), (3, 5, 0)\}$ in $\mathbb{R}^3$. The third vector is redundant:

$$(3, 5, 0) = 3(1, 0, 0) + 5(0, 1, 0) \quad \text{— check: } (3 + 0, 0 + 5, 0) = (3, 5, 0)$$

Drop it. The remaining $\{(1, 0, 0), (0, 1, 0)\}$ are independent (neither is a multiple of the other) and still span W (every vector of W was a combination of all three, hence of these two). So it is a basis, and $\dim(W) = 2$ — a plane inside $\mathbb{R}^3$.

Basically, ... A basis = the “skeleton crew” of a space: enough vectors to reach everywhere, but nobody redundant. The dimension = how many crew members. A line has dimension 1, a plane 2, all of $\mathbb{R}^3$ has 3.

4.6 The column space $C(A)$: where $Ax = \mathbf{b}$ can land

Back to Chapter 3’s question: for which $\mathbf{b}$ does $Ax = \mathbf{b}$ have a solution? Write $A = [\mathbf{u}_1 \ \dots \ \mathbf{u}_n]$ with columns $\mathbf{u}_i$. By the columns view (§2.10),

$$A\mathbf{x} = x_1 \mathbf{u}_1 + \dots + x_n \mathbf{u}_n.$$

Def. The *column space* $C(A)$ = the span of the columns of A :

$$C(A) = \text{span}(\mathbf{u}_1, \dots, \mathbf{u}_n) = \{\text{all linear combinations of the columns}\}.$$

It is a subspace of $\mathbb{R}^m$ (each column has m entries).

The payoff, straight from the definition: $Ax = \mathbf{b}$ is solvable iff $\mathbf{b} \in C(A)$ — solving is asking “which recipe of columns makes $\mathbf{b}$?”, and $\mathbf{b}$ must be reachable by some recipe.

eg (the lecture's example).

$$A = \begin{pmatrix} 1 & 1 & 2 \\ 2 & 1 & 3 \\ 3 & 1 & 4 \\ 4 & 1 & 5 \end{pmatrix} \quad (4 \times 3).$$

Spot the redundancy: column 3 = column 1 + column 2. Check: $(1 + 1, 2 + 1, 3 + 1, 4 + 1) = (2, 3, 4, 5)$. So

$$C(A) = \text{span}(\text{col } 1, \text{col } 2) = \text{span} \left\{ \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \right\},$$

a two-dimensional subspace of $\mathbb{R}^4$ (a plane inside 4-D space). Not every $\mathbf{b} \in \mathbb{R}^4$ lies in it — no surprise, since $Ax = \mathbf{b}$ is 4 equations in 3 unknowns.

Basically, ... The column space = every output the machine A can ever produce. Asking “does $Ax = \mathbf{b}$ have a solution?” = asking “is $\mathbf{b}$ on the menu?” If $\mathbf{b}$ is off the menu (outside the column space), no solution exists — no recipe of columns can make it.

4.7 The null space $N(A)$: everything A kills

Def. The *null space* $N(A)$ = the set of all $\mathbf{x}$ with $A\mathbf{x} = \mathbf{0}$:

$$N(A) = \{\mathbf{x} \mid A\mathbf{x} = \mathbf{0}\}.$$

It is a subspace of $\mathbb{R}^n$. Why? Take $\mathbf{x}_1, \mathbf{x}_2 \in N(A)$: then $A\mathbf{x}_1 = \mathbf{0}$ and $A\mathbf{x}_2 = \mathbf{0}$, so

$$A(\mathbf{x}_1 + \mathbf{x}_2) = A\mathbf{x}_1 + A\mathbf{x}_2 = \mathbf{0} + \mathbf{0} = \mathbf{0}$$

— closed under addition. And $A(\alpha\mathbf{x}) = \alpha A\mathbf{x} = \mathbf{0}$ — closed under scaling. Both halves of the subspace test (§4.2) pass.

eg (same A as §4.6). Find $\mathbf{x}$ with $A\mathbf{x} = \mathbf{0}$: a combination of the columns that makes the zero vector. Since $\text{col } 3 = \text{col } 1 + \text{col } 2$,

$$1 \cdot \text{col } 1 + 1 \cdot \text{col } 2 - 1 \cdot \text{col } 3 = \mathbf{0},$$

so $\mathbf{x} = (1, 1, -1)^T \in N(A)$. Check: $A\mathbf{x} = (1 + 1 - 2, 2 + 1 - 3, 3 + 1 - 4, 4 + 1 - 5)^T = (0, 0, 0, 0)^T$. So

$$N(A) = \{t(1, 1, -1)^T : t \in \mathbb{R}\},$$

a line in $\mathbb{R}^3$.

Note (the two extreme cases). i) If A is *invertible*: $N(A) = \{\mathbf{0}\}$ only, $C(A)$ is the whole space, and $Ax = \mathbf{b}$ has the unique solution $\mathbf{x} = A^{-1}\mathbf{b}$ (§3.6). ii) Otherwise $N(A)$ holds some nonzero $\mathbf{x}_n$, and every solution of $Ax = \mathbf{b}$ looks like $\mathbf{x} = \mathbf{x}_p + \mathbf{x}_n$ with $A\mathbf{x}_p = \mathbf{b}$ and $A\mathbf{x}_n = \mathbf{0}$ — one particular solution plus anything the machine kills. Check: $A(\mathbf{x}_p + \mathbf{x}_n) = \mathbf{b} + \mathbf{0} = \mathbf{b}$.

Basically, ... The null space = the directions the machine A squashes flat to zero. If nothing nonzero gets squashed ($N(A) = \{\mathbf{0}\}$), each output comes from exactly one input — unique solutions. If some direction does get squashed, you can slide along it without changing the output — infinitely many solutions.

4.8 Reading bases off the RREF: rank and nullity

Gaussian elimination (§3.5) doesn't just solve $Ax = \mathbf{b}$ — its pivot positions hand you bases for $C(A)$ and $N(A)$.

eg (full elimination).

$$A = \begin{pmatrix} 1 & 2 & 2 & 2 \\ 2 & 4 & 6 & 8 \\ 3 & 6 & 8 & 10 \end{pmatrix}.$$

Eliminate: $R_2 \leftarrow R_2 - 2R_1$ gives $[0, 0, 2, 4]$; $R_3 \leftarrow R_3 - 3R_1$ gives $[0, 0, 2, 4]$; then $R_3 \leftarrow R_3 - R_2$ gives $[0, 0, 0, 0]$:

$$U = \begin{pmatrix} 1 & 2 & 2 & 2 \\ 0 & 0 & 2 & 4 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Pivots sit in columns 1 and 3 — the *pivot columns*; x_2 and x_4 are the *free variables*.

i) $C(A)$: the columns of the *original* A in the pivot positions form a basis:

$$C(A) = \text{span} \left\{ \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \begin{pmatrix} 2 \\ 6 \\ 8 \end{pmatrix} \right\}, \quad \dim C(A) = 2.$$

ii) $N(A)$: solve $U\mathbf{x} = \mathbf{0}$ — $x_1 + 2x_2 + 2x_3 + 2x_4 = 0$ and $2x_3 + 4x_4 = 0$, so $x_3 = -2x_4$. Give each free variable its turn:

- Set $x_2 = 1, x_4 = 0$: then $x_3 = 0$ and $x_1 = -2(1) - 2(0) - 2(0) = -2$, so $\mathbf{u} = (-2, 1, 0, 0)^T \in N(A)$.
- Set $x_2 = 0, x_4 = 1$: then $x_3 = -2$ and $x_1 = -2(0) - 2(-2) - 2(1) = 2$, so $\mathbf{v} = (2, 0, -2, 1)^T \in N(A)$.

Verify against the original A : $A\mathbf{u} = (-2+2, -4+4, -6+6)^T = \mathbf{0}$; $A\mathbf{v} = (2-4+2, 4-12+8, 6-16+10)^T = \mathbf{0}$. Every solution is a combination of $\mathbf{u}$ and $\mathbf{v}$:

$$N(A) = \text{span}(\mathbf{u}, \mathbf{v}), \quad \dim N(A) = 2.$$

Def. The *rank* of A = the number of pivot columns = $\dim C(A)$. The *nullity* of A = the number of free variables = $\dim N(A)$.

Here $\text{rank}(A) = 2$ and $\text{nullity}(A) = 2$. Every column is either a pivot column or a free column, so for an $m \times n$ matrix:

The rank-nullity theorem. $\text{rank}(A) + \text{nullity}(A) = n$ (the number of columns). In subspace language: $\dim C(A) + \dim N(A) = n$.

Basically, ... Elimination sorts the columns into two teams: pivot columns (the independent crew — they span the column space, and their count is the rank) and free columns (each one spawns a null-space direction — their count is the nullity). Every column joins exactly one team, so $\text{rank} + \text{nullity} = \text{number of columns}$. The free variables of §3.5 were null-space basis vectors all along: one basis vector per free variable.

4.9 The other two: row space and left null space

Everything so far came from A . Transpose it, and you get two more subspaces.

Def. The *row space* of A = the column space of A^T — the span of the *rows* of A . (The lecture calls it $R(A) = C(A^T)$.) It is a subspace of $\mathbb{R}^n$.

Def. The *left null space* of A = $N(A^T) = \{\mathbf{y} \mid A^T\mathbf{y} = \mathbf{0}\} = \{\mathbf{y} \mid \mathbf{y}^T A = \mathbf{0}\}$ — the combinations of the *rows* of A that make the zero vector. It is a subspace of $\mathbb{R}^m$.

An important fact: *column rank* = $\dim C(A)$ equals *row rank* = $\dim C(A^T)$ — both are the rank r . Applying rank-nullity to A^T (which is $n \times m$) gives $\dim C(A^T) + \dim N(A^T) = m$, so $\dim N(A^T) = m - r$. The full picture:

Subspace	Symbol	Lives in	Dimension
Column space	$C(A)$	$\mathbb{R}^m$	r
Null space	$N(A)$	$\mathbb{R}^n$	$n - r$
Row space	$C(A^T)$	$\mathbb{R}^n$	r
Left null space	$N(A^T)$	$\mathbb{R}^m$	$m - r$

eg (left null space). For the 3×4 A of §4.8, the row combination $[1 \ 1 \ -1]$ kills every column:

$$1 \cdot (1, 2, 2, 2) + 1 \cdot (2, 4, 6, 8) - 1 \cdot (3, 6, 8, 10) = (0, 0, 0, 0).$$

Check entry by entry: $1 + 2 - 3 = 0$; $2 + 4 - 6 = 0$; $2 + 6 - 8 = 0$; $2 + 8 - 10 = 0$. So $(1, 1, -1)^T \in N(A^T)$ — a line in $\mathbb{R}^3$, and indeed $\dim N(A^T) = m - r = 3 - 2 = 1$.

eg (all four at once). $A = \begin{pmatrix} 1 & 2 \\ 3 & 6 \end{pmatrix}$ ($m = n = 2$):

- $C(A)$ = line through $\begin{pmatrix} 1 \\ 3 \end{pmatrix}$ (col 2 = 2 · col 1) — $\text{rank}(A) = r = 1$.
- $N(A)$ = line through $\begin{pmatrix} -2 \\ 1 \end{pmatrix}$ (from $x + 2y = 0$); check $A \begin{pmatrix} -2 \\ 1 \end{pmatrix} = (-2 + 2, -6 + 6)^T = \mathbf{0}$ — nullity = $n - r = 1$.
- Row space $C(A^T)$ = line through $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$ (row 2 = 3 · row 1) — dimension $r = 1$.
- $N(A^T)$ = line through $\begin{pmatrix} -3 \\ 1 \end{pmatrix}$ (from $y_1 + 3y_2 = 0$ on A^T); check $A^T \begin{pmatrix} -3 \\ 1 \end{pmatrix} = (-3 + 3, -6 + 6)^T = \mathbf{0}$ — dimension $m - r = 1$.

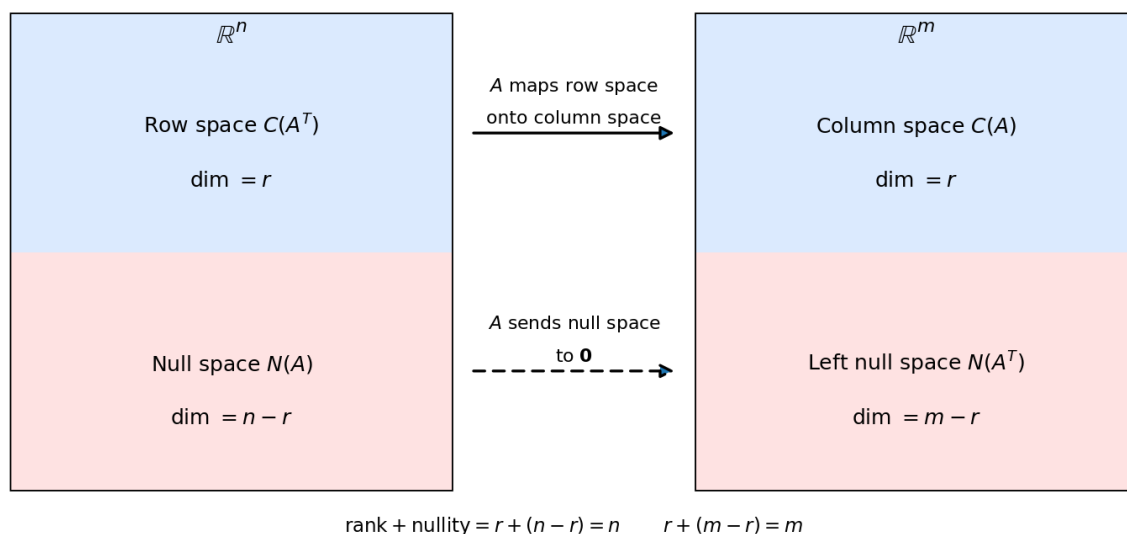


Figure 6: The big picture: $\mathbb{R}^n$ splits into row space ($\dim r$) and null space ($\dim n-r$); $\mathbb{R}^m$ splits into column space ($\dim r$) and left null space ($\dim m-r$); A maps the row space onto the column space and sends the null space to zero

Basically, ... A matrix has four natural “clubs”: the outputs it can make (column space), the inputs it kills (null space), the independent rows it was built from (row space), and the row-recipes that cancel out (left

null space). Their sizes always add up: $r + (n - r) = n$ columns' worth inside $\mathbb{R}^n$, $r + (m - r) = m$ rows' worth inside $\mathbb{R}^m$.

4.10 The big payoff: the three outcomes of $Ax = \mathbf{b}$, explained

Chapter 3 met the three outcomes by elimination. The subspaces name them:

- i) *No solution* $\iff \mathbf{b} \notin C(A)$. (The $0 = c_i \neq 0$ row of §3.5 is elimination discovering that $\mathbf{b}$ is off the menu.)
- ii) *Unique solution* $\iff \mathbf{b} \in C(A)$ **and** $N(A) = \{\mathbf{0}\}$ — then $\mathbf{x} = A^{-1}\mathbf{b}$ when A is square invertible. $N(A) = \{\mathbf{0}\}$ means no free variables: every column is a pivot column ($r = n$), so each recipe of columns is the only one.
- iii) *Infinitely many solutions* $\iff \mathbf{b} \in C(A)$ **and** $N(A)$ holds a nonzero vector. Every solution is $\mathbf{x} = \mathbf{x}_p + \mathbf{x}_n$: one particular recipe $\mathbf{x}_p$ ($A\mathbf{x}_p = \mathbf{b}$) plus any direction the machine kills ($A\mathbf{x}_n = \mathbf{0}$).

And §3.5's "independent (free) variables" are exactly the null space made concrete: each free variable contributes one basis vector of $N(A)$, so

$$(\text{number of free variables}) = \text{nullity}(A) = \dim N(A).$$

"Infinitely many solutions" in Chapter 3 = "some variable is free" = "the null space is bigger than $\{\mathbf{0}\}$ " here.

Basically, ... Chapter 3 gave you the *procedure* (eliminate, count free variables); this chapter gives you the *meaning*. Solvable = " $\mathbf{b}$ is in the column space." Unique = "nothing gets killed." Infinite = "something gets killed, so you can slide along it." The free variables of §3.5 were null-space basis vectors all along.

Problem set

- One-line verdict for each: is it a subspace of the stated space? (a) The line $y = 3x$ in $\mathbb{R}^2$. (b) The first quadrant $\{(x, y) : x \geq 0, y \geq 0\}$ in $\mathbb{R}^2$. (c) $\{(0, 0, 0)\}$ in $\mathbb{R}^3$. (d) The line $y = x + 1$ in $\mathbb{R}^2$.
- Describe $\text{span}\{(1, 1)\}$ in $\mathbb{R}^2$ and $\text{span}\{(1, 1, 1)\}$ in $\mathbb{R}^3$ geometrically, with parametric descriptions.
- Test $\{(1, 2), (-1, 2)\}$ for linear independence, showing all steps.
- Find a basis and the dimension for $W = \text{span}\{(1, 1, 0), (1, 0, 0), (2, 1, 0)\}$ in $\mathbb{R}^3$.
- A is 5×7 with $\text{rank}(A) = 4$. Find $\text{nullity}(A)$, $\dim C(A^T)$, and $\dim N(A^T)$.
- Explain why the columns of A are linearly independent iff $N(A) = \{\mathbf{0}\}$.
- For $A = \begin{pmatrix} 1 & 2 \\ 3 & 6 \end{pmatrix}$, describe exactly which $\mathbf{b} = (b_1, b_2)^T$ make $Ax = \mathbf{b}$ solvable, as a condition on b_1, b_2 .
- (The lecture's homework.) Work out the four fundamental subspaces of $A = \begin{pmatrix} 1 & 3 & 3 & 2 \\ 2 & 6 & 9 & 7 \\ -1 & -3 & 3 & 4 \end{pmatrix}$: give a basis and the dimension for each of $C(A)$, $N(A)$, $C(A^T)$, $N(A^T)$, plus $\text{rank}(A)$ and $\text{nullity}(A)$.
- True or false, with a reason: (a) $\text{rank}(A) = \text{number of pivot columns} = \text{number of nonzero rows in the RREF of } A$. (b) If A is $m \times n$ with $m < n$, then $Ax = \mathbf{b}$ always has infinitely many solutions. (c) $N(A)$ is always a subspace of $\mathbb{R}^n$.
- A is 4×4 and invertible. State the dimension of each of the four fundamental subspaces, and say how many solutions $Ax = \mathbf{b}$ has for any $\mathbf{b}$.

Where this goes next

- Chapter 5 (orthogonality):** the four subspaces come in orthogonal pairs — $C(A)$ with $N(A^T)$, and $C(A^T)$ with $N(A)$. That geometry is what makes projections and least squares work.
- Chapter 6 (eigen-stuff):** eigenvectors live in special subspaces of a square matrix — $N(A - \lambda I)$ for each eigenvalue λ .

Solutions — Chapter 4. Vector spaces and the four fundamental subspaces

1. Apply the subspace test (§4.2): closed under addition and scaling, and $\mathbf{0}$ must be in.

(a) $W = \{(x, y) : y = 3x\}$. Addition: $(x_1, 3x_1) + (x_2, 3x_2) = (x_1 + x_2, 3(x_1 + x_2)) \in W$. Scaling: $c(x, 3x) = (cx, 3(cx)) \in W$. Verdict: subspace.

(b) First quadrant: $(1, 1)$ is in it, but $(-1)(1, 1) = (-1, -1)$ is not. Verdict: not a subspace.

(c) $\{(0, 0, 0)\}$: $\mathbf{0} + \mathbf{0} = \mathbf{0}$, $c\mathbf{0} = \mathbf{0}$. Verdict: subspace (the trivial one).

(d) Line $y = x + 1$: $(0, 0)$ is not on it ($0 \neq 0 + 1$). Verdict: not a subspace.

2. $\text{span}\{(1, 1)\} = \{t(1, 1) : t \in \mathbb{R}\} = \{(t, t) : t \in \mathbb{R}\}$ — the line $y = x$ through the origin in $\mathbb{R}^2$.

$\text{span}\{(1, 1, 1)\} = \{t(1, 1, 1) : t \in \mathbb{R}\} = \{(t, t, t) : t \in \mathbb{R}\}$ — the line through the origin in the direction $(1, 1, 1)$ in $\mathbb{R}^3$.

3. Set $a(1, 2) + b(-1, 2) = (0, 0)$. That reads

$$\begin{aligned} a - b &= 0 \\ 2a + 2b &= 0. \end{aligned}$$

From the first, $b = a$. Substitute into the second: $2a + 2a = 4a = 0 \Rightarrow a = 0$, hence $b = 0$. Only the trivial coefficients work — the set is linearly independent.

4. $W = \text{span}\{(1, 1, 0), (1, 0, 0), (2, 1, 0)\}$. Spot the redundancy:

$$(1, 1, 0) + (1, 0, 0) = (1 + 1, 1 + 0, 0) = (2, 1, 0)$$

so the third vector adds nothing. Drop it: $\{(1, 1, 0), (1, 0, 0)\}$ still spans W . Check independence: $a(1, 1, 0) + b(1, 0, 0) = (0, 0, 0)$ reads $a + b = 0$ and $a = 0$, so $a = b = 0$ — independent. Hence $\{(1, 1, 0), (1, 0, 0)\}$ is a basis and $\dim(W) = 2$ (a plane in $\mathbb{R}^3$).

5. $m = 5$, $n = 7$, $r = 4$.

- $\text{nullity}(A) = n - r = 7 - 4 = 3$ (rank-nullity).
- $\dim C(A^T) = \text{row rank} = r = 4$.
- $\dim N(A^T) = m - r = 5 - 4 = 1$.

6. $A\mathbf{x} = x_1(\text{col } 1) + \cdots + x_n(\text{col } n)$ — multiplying A by $\mathbf{x}$ is exactly forming the linear combination of the columns with coefficients x_i (§2.10 columns view). So $A\mathbf{x} = \mathbf{0}$ with $\mathbf{x} \neq \mathbf{0}$ is the same thing as “a nontrivial combination of the columns makes $\mathbf{0}$,” which is the definition of dependence (§4.4). Therefore: columns independent $\iff$ no nonzero $\mathbf{x}$ gives $A\mathbf{x} = \mathbf{0} \iff N(A) = \{\mathbf{0}\}$.

7. Solvable $\iff \mathbf{b} \in C(A)$. The columns are $(1, 3)^T$ and $(2, 6)^T = 2(1, 3)^T$, so

$$C(A) = \text{span} \left\{ \begin{pmatrix} 1 \\ 3 \end{pmatrix} \right\} = \left\{ t \begin{pmatrix} 1 \\ 3 \end{pmatrix} : t \in \mathbb{R} \right\} = \{(t, 3t) : t \in \mathbb{R}\}.$$

So $A\mathbf{x} = \mathbf{b}$ is solvable iff $\mathbf{b} = (b_1, b_2)^T$ satisfies $b_2 = 3b_1$ — $\mathbf{b}$ must lie on the line through $(1, 3)$. Check: with $b_2 = 3b_1$, take $x_1 = b_1$, $x_2 = 0$; then $A\mathbf{x} = b_1(1, 3)^T = (b_1, 3b_1)^T = \mathbf{b}$. If $b_2 \neq 3b_1$, no recipe of columns can produce $\mathbf{b}$.

8. $A = \begin{pmatrix} 1 & 3 & 3 & 2 \\ 2 & 6 & 9 & 7 \\ -1 & -3 & 3 & 4 \end{pmatrix}$ (3×4). Eliminate:

$$\begin{aligned} R_2 &\leftarrow R_2 - 2R_1 : [2 - 2, 6 - 6, 9 - 6, 7 - 4] = [0, 0, 3, 3], \\ R_3 &\leftarrow R_3 + R_1 : [-1 + 1, -3 + 3, 3 + 3, 4 + 2] = [0, 0, 6, 6], \\ R_3 &\leftarrow R_3 - 2R_2 : [0, 0, 6 - 6, 6 - 6] = [0, 0, 0, 0]. \end{aligned}$$

$$U = \begin{pmatrix} 1 & 3 & 3 & 2 \\ 0 & 0 & 3 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Pivots in columns 1 and 3: $\text{rank}(A) = 2$, $\text{nullity}(A) = 4 - 2 = 2$.

- $C(A)$: pivot columns of A :

$$C(A) = \text{span} \left\{ \begin{pmatrix} 1 \\ 2 \\ -1 \end{pmatrix}, \begin{pmatrix} 3 \\ 9 \\ 3 \end{pmatrix} \right\}, \quad \dim = 2.$$

- $N(A)$: from U , $x_1 + 3x_2 + 3x_3 + 2x_4 = 0$ and $3x_3 + 3x_4 = 0$, so $x_3 = -x_4$. Free variables x_2, x_4 :
 - $x_2 = 1, x_4 = 0$: $x_3 = 0, x_1 = -3(1) - 3(0) - 2(0) = -3 \rightarrow \mathbf{u} = (-3, 1, 0, 0)^T$.
 - $x_2 = 0, x_4 = 1$: $x_3 = -1, x_1 = -3(0) - 3(-1) - 2(1) = 1 \rightarrow \mathbf{v} = (1, 0, -1, 1)^T$.
 Verify: $A\mathbf{u} = (-3 + 3, -6 + 6, 3 - 3)^T = \mathbf{0}$; $A\mathbf{v} = (1 - 3 + 2, 2 - 9 + 7, -1 - 3 + 4)^T = \mathbf{0}$.

$$N(A) = \text{span}(\mathbf{u}, \mathbf{v}), \quad \dim = 2.$$

- Row space $C(A^T)$: the nonzero rows of U :

$$C(A^T) = \text{span}\{(1, 3, 3, 2), (0, 0, 3, 3)\}, \quad \dim = 2.$$

- $N(A^T)$: $\mathbf{y}^T A = \mathbf{0}$ with $\mathbf{y} = (y_1, y_2, y_3)^T$ reads $y_1 + 2y_2 - y_3 = 0$, $3y_1 + 9y_2 + 3y_3 = 0$, $2y_1 + 7y_2 + 4y_3 = 0$ (the second column equation is a multiple of the first). From the first: $y_1 = y_3 - 2y_2$. Substitute into the third-coefficient equation: $3(y_3 - 2y_2) + 9y_2 + 3y_3 = 6y_3 + 3y_2 = 0 \Rightarrow y_2 = -2y_3$. Then $y_1 = y_3 - 2(-2y_3) = 5y_3$. So $\mathbf{y} = y_3(5, -2, 1)^T$. Check: $5(1, 3, 3, 2) - 2(2, 6, 9, 7) + 1(-1, -3, 3, 4) = (5 - 4 - 1, 15 - 12 - 3, 15 - 18 + 3, 10 - 14 + 4) = (0, 0, 0, 0)$.

$$N(A^T) = \text{span}\{(5, -2, 1)^T\}, \quad \dim = m - r = 3 - 2 = 1.$$

9. (a) True: each pivot column holds exactly one pivot and each nonzero RREF row holds exactly one pivot, so both counts equal the number of pivots, which is the rank. (b) False: $m < n$ means more unknowns than equations, but consistency is not guaranteed — e.g. $A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$, $\mathbf{b} = (0, 1)^T$: the second row reads $0 = 1$, no solution. (c) True: proved in §4.7 — $N(A)$ passes both halves of the subspace test.

10. Invertible 4×4 : $\text{rank}(A) = 4$.

- $\dim C(A) = 4$ (the whole $\mathbb{R}^4$), $\dim N(A) = 4 - 4 = 0$ (just $\{\mathbf{0}\}$).
- $\dim C(A^T) = 4$, $\dim N(A^T) = 4 - 4 = 0$.

Since $C(A) = \mathbb{R}^4$, every $\mathbf{b}$ is on the menu; since $N(A) = \{\mathbf{0}\}$, each recipe is unique. $A\mathbf{x} = \mathbf{b}$ has exactly one solution $\mathbf{x} = A^{-1}\mathbf{b}$ for every $\mathbf{b}$.

5. Orthogonality, projections, least squares

5.1 What this chapter is for

Most of machine learning is, quietly, the business of solving $A\mathbf{x} = \mathbf{b}$. Chapter 3 gave three possible outcomes for that system; Chapter 4 named them geometrically: a solution exists iff $\mathbf{b}$ is in the *column space* $C(A)$ (§4.10).

But real data is uncooperative. You measure more data points than you have unknowns, the measurements carry *noise*, and the tall system $A\mathbf{x} = \mathbf{b}$ has **no** solution — $\mathbf{b}$ is not in $C(A)$. Throwing away equations until it solves is a bad deal: it dumps all the error onto the equations you dropped.

The better deal is the *least squares idea*: keep every equation, and find the $\mathbf{x}$ that makes the total squared miss $\|A\mathbf{x} - \mathbf{b}\|^2$ as small as possible. The engine that finds it is the *orthogonal projection* of $\mathbf{b}$ onto $C(A)$. So this chapter answers three questions, in order:

- i) What does *orthogonal* mean — for vectors, and for whole subspaces?
- ii) How do we *project* a vector onto a line, and onto a subspace?
- iii) How do projections solve the *least squares problem*?

Basically, ... When the target is off the shelf of reachable answers, don't stretch — drop straight down onto the nearest point of the shelf. Orthogonality tells you which direction is “straight down.”

5.2 Orthogonal vectors

From Chapter 2 (§2.5): for $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the *dot product* is $\mathbf{x}^T \mathbf{y} = x_1 y_1 + \cdots + x_n y_n$, and the *length* is $\|\mathbf{x}\| = \sqrt{\mathbf{x}^T \mathbf{x}}$.

Def. $\mathbf{x}$ is *orthogonal* to $\mathbf{y}$ if $\mathbf{x}^T \mathbf{y} = 0$.

This is exactly the *right angle* from school geometry, and *Pythagoras theorem* proves it. If $\mathbf{x}$ and $\mathbf{y}$ meet at a right angle, then $\|\mathbf{x} + \mathbf{y}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{y}\|^2$. Expand the left side:

$$(\mathbf{x} + \mathbf{y})^T (\mathbf{x} + \mathbf{y}) = \mathbf{x}^T \mathbf{x} + \mathbf{y}^T \mathbf{y} + 2 \mathbf{x}^T \mathbf{y}.$$

The cross term dies iff $\mathbf{x}^T \mathbf{y} = 0$. So: right angle $\iff$ dot product zero.

eg (full steps). $(1, 2)$ and $(-2, 1)$: $(1)(-2) + (2)(1) = -2 + 2 = 0$ — orthogonal. Pythagoras check: $\|(1, 2)\|^2 = 5$, $\|(-2, 1)\|^2 = 5$, and their sum is $(-1, 3)$ with $\|(-1, 3)\|^2 = 1 + 9 = 10 = 5 + 5$.

Two small facts the lecture flags:

- i) $\mathbf{0}$ is orthogonal to every $\mathbf{x}$: $\mathbf{0}^T \mathbf{x} = 0$ always.
- ii) *Mutually orthogonal* nonzero vectors are *linearly independent*. Proof: suppose $c_1 \mathbf{v}_1 + \cdots + c_k \mathbf{v}_k = \mathbf{0}$. Dot both sides with $\mathbf{v}_1$: everything dies except $c_1 \mathbf{v}_1^T \mathbf{v}_1 = 0$. Since $\mathbf{v}_1 \neq \mathbf{0}$, $\mathbf{v}_1^T \mathbf{v}_1 > 0$, so $c_1 = 0$. Repeat with $\mathbf{v}_2, \dots, \mathbf{v}_k$.

Basically, ... Orthogonal = at right angles = dot product is zero. That's the whole test, and it's why the zero vector is "perpendicular to everything" (its dot product with anything is 0).

5.3 Orthogonal subspaces

Def. Two subspaces U and V are *orthogonal* if $\mathbf{x}^T \mathbf{y} = 0$ for **every** $\mathbf{x} \in U$ and **every** $\mathbf{y} \in V$.

eg. $U = \text{span}\{(1, 0, 0, 0), (0, 1, 0, 0)\}$ (the x_1x_2 -plane in $\mathbb{R}^4$) and $V = \text{span}\{(0, 0, 1, 0), (0, 0, 0, 1)\}$ (the x_3x_4 -plane): any vector of U has the form $(x_1, x_2, 0, 0)$, any vector of V the form $(0, 0, y_3, y_4)$, and their dot product is always 0.

Now the payoff from Chapter 4: the four *fundamental subspaces* pair up orthogonally.

- i) *Row space* $\perp$ *null space*: $C(A^T) \perp N(A)$. Proof: take $\mathbf{x} \in N(A)$, i.e. $A\mathbf{x} = \mathbf{0}$. Row by row, that equation says $(\text{row}_i \text{ of } A)^T \mathbf{x} = 0$ — every row of A is orthogonal to $\mathbf{x}$. Any *linear combination* of rows is therefore orthogonal to $\mathbf{x}$ too, and linear combinations of rows are exactly the row space.
- ii) *Column space* $\perp$ *left null space*: $C(A) \perp N(A^T)$. This is the same claim applied to A^T .

They are not just orthogonal — they are *orthogonal complements*. The dimensions add up to the full space (Chapter 4's table: $r + (n - r) = n$ and $r + (m - r) = m$), so every vector of $\mathbb{R}^n$ splits uniquely into a row-space piece plus a null-space piece, and every vector of $\mathbb{R}^m$ into a column-space piece plus a left-null-space piece. That split $\mathbf{b} = \mathbf{p} + \mathbf{e}$ is exactly what least squares computes.

eg (the lecture's example, full steps). $A = \begin{pmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{pmatrix}$. Column 2 = 2 · column 1, so $C(A)$ is the line

through $(1, 2, 3)^T$ (rank 1). $N(A)$: $x + 2y = 0$ gives the line through $(-2, 1)^T$. The row space: rows are $(1, 2)$, $2(1, 2)$, $3(1, 2)$ — the line through $(1, 2)$ in $\mathbb{R}^2$. Check pair i): $(1, 2) \cdot (-2, 1) = -2 + 2 = 0$. $N(A^T)$: $\mathbf{y}^T A = \mathbf{0}$ reads $y_1 + 2y_2 + 3y_3 = 0$ — a plane in $\mathbb{R}^3$; pair ii) holds there by the same row argument. Dimension check: $\dim C(A^T) + \dim N(A) = 1 + 1 = 2 = n$; $\dim C(A) + \dim N(A^T) = 1 + 2 = 3 = m$.

Basically, ... The four subspaces from Chapter 4 come in two right-angle pairs: input side (*row space* *null space*) and output side (*column space* *left null space*). Whatever A can reach is perpendicular to whatever kills its transpose.

5.4 Orthonormal sets, and the $Q^T Q = I$ fact

Def. Vectors are *orthonormal* if they are *orthogonal* and each has *unit length*.

eg. $(1, 0)$ and $(0, 1)$. **eg.** $(\cos \theta, \sin \theta)$ and $(-\sin \theta, \cos \theta)$: dot product = $-\cos \theta \sin \theta + \sin \theta \cos \theta = 0$; each length squared = $\cos^2 \theta + \sin^2 \theta = 1$.

Now put orthonormal vectors $\mathbf{q}_1, \dots, \mathbf{q}_n$ as the columns of a matrix Q . Then

$$Q^T Q = I.$$

Why: the (i, j) entry of $Q^T Q$ is $\mathbf{q}_i^T \mathbf{q}_j$, which is 1 when $i = j$ (unit length) and 0 when $i \neq j$ (orthogonal). That is the whole content of the identity matrix.

eg ($Q^T Q = I$, full steps). $Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, columns $\mathbf{q}_1 = \frac{1}{\sqrt{2}}(1, 1)^T$, $\mathbf{q}_2 = \frac{1}{\sqrt{2}}(1, -1)^T$. Check orthonormality: $\mathbf{q}_1^T \mathbf{q}_2 = \frac{1}{2}(1 - 1) = 0$; $\|\mathbf{q}_1\|^2 = \frac{1}{2}(1 + 1) = 1$, same for $\mathbf{q}_2$. Direct multiply:

$$Q^T Q = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I.$$

Basically, ... Orthonormal = "unit-length directions at right angles to each other." $Q^T Q = I$ is the matrix way of writing exactly that — the columns agree to be perpendicular unit vectors.

5.5 Why project? The inconsistent case

Chapter 4 §4.10 gave the verdict: $A\mathbf{x} = \mathbf{b}$ has no solution iff $\mathbf{b} \notin C(A)$. Now suppose that is the situation — for instance

$$2x = b_1, \quad 3x = b_2, \quad 4x = b_3, \quad \text{or} \quad x + 2y = 4, \quad x + 3y = 5, \quad 2x + 4y = 6.$$

Three (or more) equations, fewer unknowns, and in general the data does not line up. The lecture calls these *inconsistent systems*.

You could solve any two of the equations exactly and throw the rest away. The problem: the discarded equations then carry huge errors while the kept ones carry none — an arbitrary, lopsided answer. The reasonable alternative is to **spread the error**: keep every equation and make the total *squared error* $\|A\mathbf{x} - \mathbf{b}\|^2$ as small as possible. That is *least squares*, and its geometric form is: find the point $\mathbf{p}$ of $C(A)$ closest to $\mathbf{b}$ — the *projection* of $\mathbf{b}$ onto the column space.

Basically, ... No exact answer exists, so find the nearest reachable one. “Nearest” needs a notion of perpendicular — which is why this whole chapter runs on orthogonality.

5.6 Projection onto a line

Start with the simplest case: a line through a vector $\mathbf{a}$. Given $\mathbf{b}$, its *projection* $\mathbf{p}$ is the point on the line closest to $\mathbf{b}$; the *error* $\mathbf{e} = \mathbf{b} - \mathbf{p}$ sticks out at a right angle to the line.

Write $\mathbf{p} = \hat{x}\mathbf{a}$ (some scalar multiple of $\mathbf{a}$) and enforce the right angle:

$$\mathbf{a}^T \mathbf{e} = 0 \Rightarrow \mathbf{a}^T (\mathbf{b} - \hat{x}\mathbf{a}) = 0 \Rightarrow \hat{x} = \frac{\mathbf{a}^T \mathbf{b}}{\mathbf{a}^T \mathbf{a}}.$$

So the projection is

$$\mathbf{p} = \frac{\mathbf{a}^T \mathbf{b}}{\mathbf{a}^T \mathbf{a}} \mathbf{a} = \left(\frac{\mathbf{a}\mathbf{a}^T}{\mathbf{a}^T \mathbf{a}} \right) \mathbf{b}.$$

eg (full steps). $\mathbf{b} = (1, 2, 3)^T$, $\mathbf{a} = (1, 1, 1)^T$. $\mathbf{a}^T \mathbf{b} = 1 + 2 + 3 = 6$; $\mathbf{a}^T \mathbf{a} = 1 + 1 + 1 = 3$; $\hat{x} = 6/3 = 2$. $\mathbf{p} = 2(1, 1, 1)^T = (2, 2, 2)^T$. $\mathbf{e} = \mathbf{b} - \mathbf{p} = (-1, 0, 1)^T$. Check the right angle: $\mathbf{a}^T \mathbf{e} = -1 + 0 + 1 = 0$.

The *projection matrix* $P = \frac{\mathbf{a}\mathbf{a}^T}{\mathbf{a}^T \mathbf{a}}$ turns any $\mathbf{b}$ into its projection: $\mathbf{p} = P\mathbf{b}$.

eg (the lecture's P , full steps). $\mathbf{a} = (1, 1, 1)^T$: $P = \frac{1}{3} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$. Apply to $\mathbf{b} = (1, 2, 3)^T$: $P\mathbf{b} = \frac{1}{3}(6, 6, 6)^T = (2, 2, 2)^T$ — same $\mathbf{p}$ as above. Apply to $(1, 0, 0)^T$: $P(1, 0, 0)^T = (1/3, 1/3, 1/3)^T$ — projecting onto $(1, 1, 1)$ returns the **average** of the entries.

Three things to notice about P (the lecture checks each):

- i) $P^T = P$ — P is *symmetric* (transpose the formula and it is unchanged).
- ii) $P^2 = P$ — projecting twice is the same as projecting once: $P\mathbf{b}$ already lies on the line, so projecting again does nothing.
- iii) $C(P)$ is the line through $\mathbf{a}$, $N(P)$ is the plane orthogonal to $\mathbf{a}$, $\text{rank}(P) = 1$.

Also: replacing $\mathbf{a}$ by $2\mathbf{a}$ (same line!) leaves P unchanged — the 4's cancel top and bottom. The projection matrix cares about the **line**, not the vector you named it with.

eg (the mean as a projection). $x = 1$, $x = 2$ is inconsistent ($A = (1, 1)^T$, $\mathbf{b} = (1, 2)^T$). $\hat{x} = \frac{\mathbf{a}^T \mathbf{b}}{\mathbf{a}^T \mathbf{a}} = \frac{1+2}{2} = \frac{3}{2}$ — the average of the two measurements. Least squares on one unknown is just averaging.

Note (a free theorem): the projection proof gives *Cauchy-Schwarz* almost for free. $\|\mathbf{e}\|^2 \geq 0$ with $\mathbf{e} = \mathbf{b} - \frac{\mathbf{a}^T \mathbf{b}}{\mathbf{a}^T \mathbf{a}} \mathbf{a}$ expands to $\mathbf{b}^T \mathbf{b} - \frac{(\mathbf{a}^T \mathbf{b})^2}{\mathbf{a}^T \mathbf{a}} \geq 0$, i.e.

$$|\mathbf{a}^T \mathbf{b}| \leq \|\mathbf{a}\| \|\mathbf{b}\|.$$

Equality holds exactly when $\mathbf{b}$ is already a multiple of $\mathbf{a}$ (so $\mathbf{e} = \mathbf{0}$).

Basically, ... Projection = the shadow of $\mathbf{b}$ cast straight down onto the line. P is the shadow machine: feed it any vector, it returns the shadow. Shadows of shadows don't move ($P^2 = P$), and the machine is symmetric ($P^T = P$).

5.7 Projection onto a subspace

Now the general case: A is $m \times n$ with more equations than unknowns, and we want the projection $\mathbf{p}$ of $\mathbf{b}$ onto the *column space* $C(A)$.

The logic is identical to the line case, one dimension up. Write $\mathbf{p} = A\hat{\mathbf{x}}$ (some combination of the columns) and demand the error be orthogonal to the whole space:

$$\mathbf{e} = \mathbf{b} - A\hat{\mathbf{x}} \perp C(A).$$

The lecture derives what this forces in two ways:

- i) *Orthogonal-complement route*: $\mathbf{e}$ is orthogonal to every vector of $C(A)$, and §5.3 says the orthogonal complement of $C(A)$ is $N(A^T)$. So $\mathbf{e} \in N(A^T)$, i.e. $A^T \mathbf{e} = \mathbf{0}$.
- ii) *First-principles route*: write $A = [\mathbf{a}_1 \cdots \mathbf{a}_n]$. $\mathbf{e} \perp C(A)$ means $\mathbf{a}_i^T \mathbf{e} = 0$ for every column; stacking those rows gives $A^T \mathbf{e} = \mathbf{0}$.

Either way, with $\mathbf{e} = \mathbf{b} - A\hat{\mathbf{x}}$:

$$A^T(\mathbf{b} - A\hat{\mathbf{x}}) = \mathbf{0} \implies A^T A\hat{\mathbf{x}} = A^T \mathbf{b}.$$

These are the *normal equations* — the single most important equation of this chapter (and the one Chapter 3 §3.1 previewed). Even when $A\mathbf{x} = \mathbf{b}$ is inconsistent, $A^T A\hat{\mathbf{x}} = A^T \mathbf{b}$ always has a solution.

If the columns of A are *linearly independent*, $A^T A$ is *invertible* (proof: $A^T A\mathbf{x} = \mathbf{0} \Rightarrow \mathbf{x}^T A^T A\mathbf{x} = \|A\mathbf{x}\|^2 = 0 \Rightarrow A\mathbf{x} = \mathbf{0} \Rightarrow \mathbf{x} = \mathbf{0}$, so $A^T A$ kills nothing). Then

$$\hat{\mathbf{x}} = (A^T A)^{-1} A^T \mathbf{b}, \quad \mathbf{p} = A\hat{\mathbf{x}} = \underbrace{A(A^T A)^{-1} A^T}_P \mathbf{b}.$$

The general *projection matrix* is $P = A(A^T A)^{-1} A^T$. Check its two properties:

- i) $P^T = P$: transpose reverses the product, $(A^T A)^{-1}$ is symmetric because $A^T A$ is ($(A^T A)^T = A^T (A^T)^T = A^T A$), and everything folds back to P .
- ii) $P^2 = P$: $A(A^T A)^{-1} \underbrace{A^T A(A^T A)^{-1}}_I A^T = A(A^T A)^{-1} A^T$ — the middle pair cancels to I .

The converse holds too: $P^2 = P$ and $P^T = P$ imply P is the projection matrix onto $C(P)$, because $(\mathbf{b} - P\mathbf{b})^T (P\mathbf{c}) = \mathbf{b}^T (I - P)P\mathbf{c} = \mathbf{b}^T (P - P^2)\mathbf{c} = 0$ — the error is orthogonal to everything in the column space.

Four sanity checks (the lecture's special cases):

- $\mathbf{b} \in C(A)$, say $\mathbf{b} = A\mathbf{x}$: $P\mathbf{b} = A(A^T A)^{-1} A^T A\mathbf{x} = A\mathbf{x} = \mathbf{b}$ — already in the space, projection does nothing.
- $\mathbf{b} \in N(A^T)$: $A^T \mathbf{b} = \mathbf{0} \Rightarrow P\mathbf{b} = \mathbf{0}$ — fully perpendicular, projection kills it.
- A square and invertible: $P = A(A^T A)^{-1} A^T = I$ — the column space is all of $\mathbb{R}^n$, every $\mathbf{b}$ is already there.

- Rank 1 ($A = \mathbf{a}$): $P = \mathbf{a}\mathbf{a}^T / \mathbf{a}^T \mathbf{a}$ — back to §5.6, as it must.

Orthonormal-basis shortcut: if $\mathbf{q}_1, \dots, \mathbf{q}_r$ is an orthonormal basis of the subspace, then $Q^T Q = I$ collapses the formula to $P = QQ^T$, and

$$\mathbf{p} = \sum_{i=1}^r (\mathbf{q}_i^T \mathbf{b}) \mathbf{q}_i.$$

Project onto each basis direction (a line projection, §5.6), add the pieces.

eg (projection onto a plane, full steps — this is the figure below). $C(A) =$ the xy -plane, spanned by the orthonormal $\mathbf{q}_1 = \frac{1}{\sqrt{2}}(1, 1, 0)^T$, $\mathbf{q}_2 = \frac{1}{\sqrt{2}}(1, -1, 0)^T$ (§5.4). Project $\mathbf{b} = (1, 2, 3)^T$:

$$\mathbf{q}_1^T \mathbf{b} = \frac{3}{\sqrt{2}}, \quad \mathbf{q}_2^T \mathbf{b} = \frac{-1}{\sqrt{2}},$$

$$\mathbf{p} = \frac{3}{\sqrt{2}}\mathbf{q}_1 - \frac{1}{\sqrt{2}}\mathbf{q}_2 = \frac{3}{2}(1, 1, 0)^T - \frac{1}{2}(1, -1, 0)^T = (1, 2, 0)^T.$$

The projection matrix $P = \mathbf{q}_1\mathbf{q}_1^T + \mathbf{q}_2\mathbf{q}_2^T$:

$$P = \frac{1}{2} \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} + \frac{1}{2} \begin{pmatrix} 1 & -1 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Check: $P^2 = P$ (diagonal of 0s and 1s), $P^T = P$; $P\mathbf{b} = (1, 2, 0)^T$. The error $\mathbf{e} = (0, 0, 3)^T$ is orthogonal to the plane. (With $A = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix}$, the normal equations give $A^T A = 4I$, $A^T \mathbf{b} = (2, 4)^T$, $\hat{\mathbf{x}} = (1/2, 1)^T$, $A\hat{\mathbf{x}} = (1, 2, 0)^T$ — same answer, no orthonormal shortcut needed.)

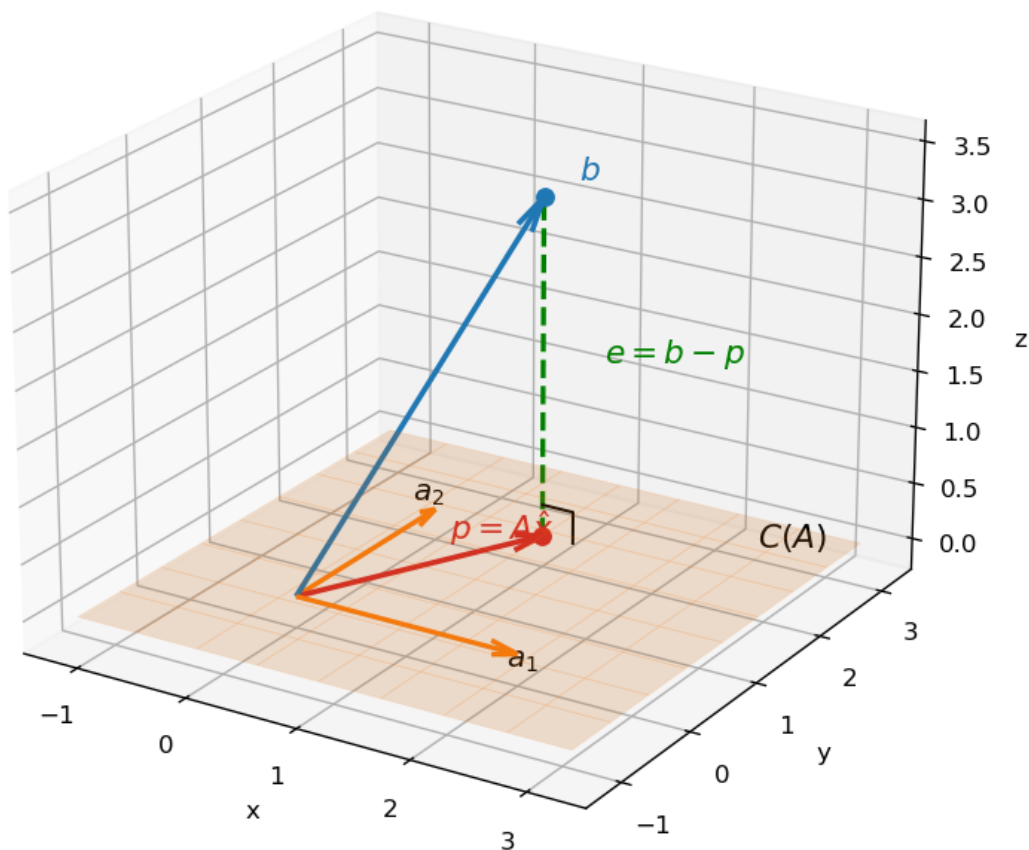


Figure 7: Least-squares geometry: $\mathbf{b}$ sits off the column space $C(A)$ (the plane spanned by the columns $\mathbf{a}_1, \mathbf{a}_2$); $\mathbf{p} = A\hat{\mathbf{x}}$ is its orthogonal projection onto the plane; the error $\mathbf{e} = \mathbf{b} - \mathbf{p}$ is perpendicular to the plane, i.e. $\mathbf{e}$ lies in $N(A^T)$

Basically, ... Projecting onto a subspace = running the line-projection in every basis direction and adding the results. The normal equations $A^T A \hat{\mathbf{x}} = A^T \mathbf{b}$ are the “which combination of columns?” question, answered in one shot.

5.8 Least squares: the two routes to the normal equations

The *least squares problem*: A is $m \times n$ with $m > n$ (tall — more equations than unknowns), $\mathbf{b} \notin C(A)$, and we want

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|^2.$$

The calculus route (the lecture’s 1-D example). Take $2x = b_1$, $3x = b_2$, $4x = b_3$. The squared error:

$$E = (2x - b_1)^2 + (3x - b_2)^2 + (4x - b_3)^2.$$

Set $\frac{dE}{dx} = 0$:

$$2(2x - b_1) \cdot 2 + 2(3x - b_2) \cdot 3 + 2(4x - b_3) \cdot 4 = 0.$$

Drop the common factor 2: $2(2x - b_1) + 3(3x - b_2) + 4(4x - b_3) = 0$, i.e. $(4 + 9 + 16)x = 2b_1 + 3b_2 + 4b_3$. So

$$\hat{x} = \frac{2b_1 + 3b_2 + 4b_3}{29} = \frac{\mathbf{a}^T \mathbf{b}}{\mathbf{a}^T \mathbf{a}}, \quad \mathbf{a} = (2, 3, 4)^T.$$

This is exactly the §5.6 projection formula. The lecture's bottom line: **taking derivatives and finding the minimum is the same operation as projecting.**

The geometric route. The closest point of $C(A)$ to $\mathbf{b}$ is its orthogonal projection $\mathbf{p} = A\hat{\mathbf{x}}$ (§5.7). The error $\mathbf{e} = \mathbf{b} - A\hat{\mathbf{x}}$ is orthogonal to $C(A)$, which forces the normal equations:

$$A^T A \hat{\mathbf{x}} = A^T \mathbf{b}.$$

When the columns of A are linearly independent, $A^T A$ is invertible and

$$\hat{\mathbf{x}} = (A^T A)^{-1} A^T \mathbf{b}.$$

Note: $A^T A$ is always *symmetric* ($(A^T A)^T = A^T A$); it is invertible exactly when A 's columns are independent (§5.7's proof). With dependent columns there are infinitely many least-squares $\hat{\mathbf{x}}$ — the normal equations still have solutions, but no unique one.

Note (ML): least squares **is** the engine of *linear regression*: fitting a line (or plane, or hyperplane) to data points is exactly “minimize $\|A\mathbf{x} - \mathbf{b}\|^2$,” and the normal equations are the training step. The full regression story is Chapter 23; projection matrices resurface in *PCA* (Chapter 24).

Basically, ... No exact answer exists, so take the answer with the smallest total miss. Two ways to say the same thing: (1) calculus — differentiate the squared error, set to zero; (2) geometry — drop $\mathbf{b}$ perpendicularly onto the column space. Both land on $A^T A \hat{\mathbf{x}} = A^T \mathbf{b}$.

5.9 Worked example set

eg (the lecture's least-squares example, recomputed). Fit the best line $y = \theta'x + \theta''$ through the points $(-1, 1)$, $(1, 1)$, $(2, 3)$:

$$A\theta = \mathbf{b}, \quad A = \begin{pmatrix} -1 & 1 \\ 1 & 1 \\ 2 & 1 \end{pmatrix}, \quad \theta = \begin{pmatrix} \theta' \\ \theta'' \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 1 \\ 3 \end{pmatrix}.$$

Step 1 — is it even solvable? Eliminate on $[A \mid \mathbf{b}]$:

$$\left(\begin{array}{cc|c} -1 & 1 & 1 \\ 1 & 1 & 1 \\ 2 & 1 & 3 \end{array} \right) \xrightarrow[R_3 \leftarrow R_3 + 2R_1]{R_2 \leftarrow R_2 + R_1} \left(\begin{array}{cc|c} -1 & 1 & 1 \\ 0 & 2 & 2 \\ 0 & 3 & 5 \end{array} \right) \xrightarrow{R_3 \leftarrow R_3 - \frac{3}{2}R_2} \left(\begin{array}{cc|c} -1 & 1 & 1 \\ 0 & 2 & 2 \\ 0 & 0 & 2 \end{array} \right).$$

Last row: $0 = 2$ — inconsistent, so $\mathbf{b} \notin C(A)$. Least squares it is.

Step 2 — build the normal equations. $A^T = \begin{pmatrix} -1 & 1 & 2 \\ 1 & 1 & 1 \end{pmatrix}$:

$$A^T A = \begin{pmatrix} 1+1+4 & -1+1+2 \\ -1+1+2 & 1+1+1 \end{pmatrix} = \begin{pmatrix} 6 & 2 \\ 2 & 3 \end{pmatrix}, \quad A^T \mathbf{b} = \begin{pmatrix} -1+1+6 \\ 1+1+3 \end{pmatrix} = \begin{pmatrix} 6 \\ 5 \end{pmatrix}.$$

Step 3 — solve $6\hat{\theta}' + 2\hat{\theta}'' = 6$, $2\hat{\theta}' + 3\hat{\theta}'' = 5$. From the first, $\hat{\theta}'' = 3 - 3\hat{\theta}'$; substitute: $2\hat{\theta}' + 3(3 - 3\hat{\theta}') = 5 \Rightarrow -7\hat{\theta}' = -4$. So

$$\hat{\theta}' = \frac{4}{7}, \quad \hat{\theta}'' = 3 - \frac{12}{7} = \frac{9}{7}.$$

The best-fit line: $y = \frac{4}{7}x + \frac{9}{7}$.

Step 4 — the projections (fitted values): at $x = -1$: $\frac{4}{7}(-1) + \frac{9}{7} = \frac{5}{7}$; at $x = 1$: $\frac{13}{7}$; at $x = 2$: $\frac{17}{7}$. So $\mathbf{p} = A\hat{\theta} = (5/7, 13/7, 17/7)^T$ — check row 1: $-\frac{4}{7} + \frac{9}{7} = \frac{5}{7}$.

Step 5 — the error: $\mathbf{e} = \mathbf{b} - \mathbf{p} = (2/7, -6/7, 4/7)^T$. Verify $\mathbf{e} \perp C(A)$: against column 1, $(-1)\frac{2}{7} + \frac{-6}{7} + 2 \cdot \frac{4}{7} = \frac{-2-6+8}{7} = 0$; against column 2, $\frac{2}{7} - \frac{6}{7} + \frac{4}{7} = 0$. So $\mathbf{e} \in N(A^T)$, exactly as §5.7 demands.

Step 6 — the minimum squared error: $\|\mathbf{e}\|^2 = \frac{4+36+16}{49} = \frac{56}{49} = \frac{8}{7}$. (Had the three points lain exactly on a line, we would have $\mathbf{e} = \mathbf{0}$ and error 0 — the “consistent” case.)

eg (the projection matrix for the same example). $M = (A^T A)^{-1} = \frac{1}{14} \begin{pmatrix} 3 & -2 \\ -2 & 6 \end{pmatrix}$ (check: $\begin{pmatrix} 6 & 2 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} 3 & -2 \\ -2 & 6 \end{pmatrix} = 14I$). With rows $\mathbf{r}_1 = (-1, 1)$, $\mathbf{r}_2 = (1, 1)$, $\mathbf{r}_3 = (2, 1)$ of A , $P_{ij} = \mathbf{r}_i M \mathbf{r}_j^T$:

$$P = \begin{pmatrix} 13/14 & 3/14 & -1/7 \\ 3/14 & 5/14 & 3/7 \\ -1/7 & 3/7 & 5/7 \end{pmatrix}.$$

Verify $P\mathbf{b} = \mathbf{p}$: row 1 gives $\frac{13}{14} + \frac{3}{14} - \frac{3}{7} = \frac{16}{14} - \frac{6}{14} = \frac{10}{14} = \frac{5}{7}$; row 2 gives $\frac{3}{14} + \frac{5}{14} + \frac{9}{7} = \frac{8}{14} + \frac{18}{14} = \frac{26}{14} = \frac{13}{7}$; row 3 gives $-\frac{1}{7} + \frac{3}{7} + \frac{15}{7} = \frac{17}{7}$. Sanity: $\text{tr}(P) = \frac{13}{14} + \frac{5}{14} + \frac{10}{14} = 2 = \text{number of independent columns} = \text{rank of } P$.

5.10 Problem set

- Orthogonality check, with full steps: (a) Are $(1, 2)$ and $(-2, 1)$ orthogonal? Verify Pythagoras on them. (b) Are $(1, 2, 3)$ and $(2, -1, -1)$ orthogonal?
- Are $\frac{1}{\sqrt{2}}(1, 1)$ and $\frac{1}{\sqrt{2}}(1, -1)$ orthonormal? Show every step (dot product, both lengths).
- For $Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$: (a) verify $Q^T Q = I$ by direct multiplication; (b) use $P = \mathbf{q}_1 \mathbf{q}_1^T$ to project $(3, 1)$ onto the line through $\mathbf{q}_1$, and check the error is orthogonal to $\mathbf{q}_1$.
- Project $\mathbf{b} = (4, 2)$ onto the line through $\mathbf{a} = (1, 3)$: find $\hat{x}$, $\mathbf{p}$, $\mathbf{e}$, and verify $\mathbf{a}^T \mathbf{e} = 0$.
- Show that replacing $\mathbf{a}$ by $3\mathbf{a}$ in $P = \frac{\mathbf{a}\mathbf{a}^T}{\mathbf{a}^T \mathbf{a}}$ leaves P unchanged (use $\mathbf{a} = (1, 1, 1)^T$ and compute both).
- For $P = \frac{1}{3} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$: verify $P^2 = P$ and $P^T = P$ by direct multiplication; state $C(P)$, $N(P)$, and $\text{rank}(P)$; compute $P(1, 2, 3)^T$ and $P(1, 0, 0)^T$.
- $A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (1, 2, 2)^T$. Compute $A^T A$ and $A^T \mathbf{b}$, solve the normal equations, write the fitted line, compute the residual $\mathbf{e}$, and verify it is orthogonal to both columns of A .
- Solve the least-squares problem $2x = 3$, $4x = 5$, $6x = 6$; then check the error vector is orthogonal to $(2, 4, 6)^T$.
- Prove: if $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ are nonzero and pairwise orthogonal, they are linearly independent.
- True or false, with a reason: (a) $P^2 = P$ alone implies P is an (orthogonal) projection matrix. (b) If the columns of A are dependent, then $A^T A$ is not invertible. (c) The projection of $\mathbf{b}$ onto $C(A)$ does not depend on which basis of $C(A)$ you use.
- The system $x = 1$, $x = 2$ is inconsistent. Show the least-squares answer is the mean $\hat{x} = 3/2$, and describe the geometry in one line.
- Solve the least-squares problem $x + y = 1$, $x + 2y = 2$, $x + 3y = 4$: find $\hat{x}, \hat{y}$, the residual vector, and verify the residual is orthogonal to both columns of A .

5.11 Where this goes next

- **Chapter 6 (eigen-stuff):** $Q^T Q = I$ returns in its nicest form — a *symmetric matrix* has *orthonormal eigenvectors*, so it diagonalizes as $A = Q \Lambda Q^T$ with a genuine rotation Q . Orthogonality is what makes symmetric matrices so well-behaved.
- **Chapter 23 (regression):** least squares grows up into *linear regression*. Fitting a line today becomes fitting hyperplanes to real datasets; the normal equations $A^T A \hat{\mathbf{x}} = A^T \mathbf{b}$ are still the training step, and the residual $\mathbf{e}$ becomes the thing statistics is built on.
- **Chapter 24 (PCA):** projection matrices come back as the tool that drops high-dimensional data onto its most informative directions — $P\mathbf{b}$ keeping the signal, $(I - P)\mathbf{b}$ thrown away as noise.

Solutions — Chapter 5. Orthogonality, projections, least squares

1. (a) $(1, 2) \cdot (-2, 1) = 1 \cdot (-2) + 2 \cdot 1 = -2 + 2 = 0$ — orthogonal. Pythagoras: $\|(1, 2)\|^2 = 1 + 4 = 5$; $\|(-2, 1)\|^2 = 4 + 1 = 5$; sum $(1, 2) + (-2, 1) = (-1, 3)$ with $\|(-1, 3)\|^2 = 1 + 9 = 10 = 5 + 5$.

(b) $(1, 2, 3) \cdot (2, -1, -1) = 2 - 2 - 3 = -3 \neq 0$ — not orthogonal.

2. Let $\mathbf{u} = \frac{1}{\sqrt{2}}(1, 1)$, $\mathbf{v} = \frac{1}{\sqrt{2}}(1, -1)$. Dot: $\mathbf{u}^T \mathbf{v} = \frac{1}{2}(1 \cdot 1 + 1 \cdot (-1)) = \frac{1}{2}(1 - 1) = 0$. Lengths: $\|\mathbf{u}\|^2 = \frac{1}{2}(1 + 1) = 1$, $\|\mathbf{v}\|^2 = \frac{1}{2}(1 + 1) = 1$. Orthogonal + unit length = orthonormal.

3. (a) $Q^T = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ (symmetric), so

$$Q^T Q = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} = I.$$

(b) $\mathbf{q}_1 = \frac{1}{\sqrt{2}}(1, 1)^T$. $P = \mathbf{q}_1 \mathbf{q}_1^T = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$. Project $(3, 1)^T$: $P(3, 1)^T = \frac{1}{2}(4, 4)^T = (2, 2)^T$. Via the formula: $\hat{x} = \frac{\mathbf{q}_1^T \mathbf{b}}{\mathbf{q}_1^T \mathbf{q}_1} = \mathbf{q}_1^T \mathbf{b} = \frac{3+1}{\sqrt{2}} = 2\sqrt{2}$ (since $\mathbf{q}_1^T \mathbf{q}_1 = 1$), $\mathbf{p} = 2\sqrt{2} \cdot \frac{1}{\sqrt{2}}(1, 1)^T = (2, 2)^T$. Error: $\mathbf{e} = (3, 1)^T - (2, 2)^T = (1, -1)^T$; $\mathbf{q}_1^T \mathbf{e} = \frac{1}{\sqrt{2}}(1 - 1) = 0$.

4. $\mathbf{a}^T \mathbf{b} = 4 \cdot 1 + 2 \cdot 3 = 10$; $\mathbf{a}^T \mathbf{a} = 1 + 9 = 10$. $\hat{x} = 10/10 = 1$. $\mathbf{p} = 1 \cdot (1, 3)^T = (1, 3)^T$. $\mathbf{e} = (4, 2)^T - (1, 3)^T = (3, -1)^T$. Check: $\mathbf{a}^T \mathbf{e} = 3 \cdot 1 + (-1) \cdot 3 = 3 - 3 = 0$.

5. With $\mathbf{a} = (1, 1, 1)^T$: $\mathbf{a} \mathbf{a}^T$ is the 3×3 all-ones matrix, $\mathbf{a}^T \mathbf{a} = 3$, so $P = \frac{1}{3} \mathbf{1} \mathbf{1}^T$. With $3\mathbf{a} = (3, 3, 3)^T$: $(3\mathbf{a})(3\mathbf{a})^T = 9 \cdot \mathbf{1} \mathbf{1}^T$, $(3\mathbf{a})^T (3\mathbf{a}) = 27$; ratio = $\frac{9}{27} \mathbf{1} \mathbf{1}^T = \frac{1}{3} \mathbf{1} \mathbf{1}^T$ — identical . The line is what matters, not the label vector.

6. P^2 : entry $(i, j) = \sum_k \frac{1}{3} \cdot \frac{1}{3} = 3 \cdot \frac{1}{9} = \frac{1}{3}$ — every entry is $1/3$, so $P^2 = P$. $P^T = P$ is immediate (all entries equal) . $C(P)$: every column is $(1/3, 1/3, 1/3)^T$ — the line through $(1, 1, 1)^T$. $N(P)$: $P\mathbf{x} = \mathbf{0} \iff x_1 + x_2 + x_3 = 0$ — the plane orthogonal to $(1, 1, 1)^T$. $\text{rank}(P) = 1$. $P(1, 2, 3)^T = \frac{1}{3}(6, 6, 6)^T = (2, 2, 2)^T$; $P(1, 0, 0)^T = (1/3, 1/3, 1/3)^T$ (the average of the entries).

7. Columns: $\mathbf{c}_1 = (1, 1, 1)^T$, $\mathbf{c}_2 = (1, 2, 3)^T$. $A^T A = \begin{pmatrix} \mathbf{c}_1^T \mathbf{c}_1 & \mathbf{c}_1^T \mathbf{c}_2 \\ \mathbf{c}_2^T \mathbf{c}_1 & \mathbf{c}_2^T \mathbf{c}_2 \end{pmatrix} = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}$ (dot products: $1 + 1 + 1 = 3$; $1 + 2 + 3 = 6$; $1 + 4 + 9 = 14$). $A^T \mathbf{b} = (\mathbf{c}_1^T \mathbf{b}, \mathbf{c}_2^T \mathbf{b})^T = (1 + 2 + 2, 1 + 4 + 6)^T = (5, 11)^T$. Normal equations: $\begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix} \hat{\theta} = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$. $\det = 42 - 36 = 6$; inverse $\frac{1}{6} \begin{pmatrix} 14 & -6 \\ -6 & 3 \end{pmatrix}$. $\hat{\theta} = \frac{1}{6} \begin{pmatrix} 14 \cdot 5 - 6 \cdot 11 \\ -6 \cdot 5 + 3 \cdot 11 \end{pmatrix} = \frac{1}{6} (4, 3)^T = (2/3, 1/2)^T$. Fitted line (first column is the constant): $y = \frac{1}{2}x + \frac{2}{3}$. Residual: $\mathbf{p} = \frac{2}{3}\mathbf{c}_1 + \frac{1}{2}\mathbf{c}_2 = (7/6, 5/3, 13/6)^T$; $\mathbf{e} = (1, 2, 2)^T - \mathbf{p} = (-1/6, 1/3, -1/6)^T$. Checks: $\mathbf{e}^T \mathbf{c}_1 = -1/6 + 1/3 - 1/6 = 0$; $\mathbf{e}^T \mathbf{c}_2 = -1/6 + 2/3 - 3/6 = (-1 + 4 - 3)/6 = 0$. Minimum squared error $\|\mathbf{e}\|^2 = 1/36 + 4/36 + 1/36 = 6/36 = 1/6$.

8. $A = (2, 4, 6)^T$. $A^T A = 4 + 16 + 36 = 56$; $A^T \mathbf{b} = 2 \cdot 3 + 4 \cdot 5 + 6 \cdot 6 = 6 + 20 + 36 = 62$. $\hat{x} = 62/56 = 31/28$. Error: $\hat{x}A = (62/28, 124/28, 186/28)^T$; $\mathbf{e} = (3, 5, 6)^T - \hat{x}A = (22/28, 16/28, -18/28)^T$. Check: $(2, 4, 6) \cdot \mathbf{e} = \frac{44+64-108}{28} = 0$.

9. Suppose $c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + c_3\mathbf{v}_3 = \mathbf{0}$. Dot with $\mathbf{v}_1$: $c_1\mathbf{v}_1^T\mathbf{v}_1 + c_2\mathbf{v}_2^T\mathbf{v}_1 + c_3\mathbf{v}_3^T\mathbf{v}_1 = 0$. Pairwise orthogonality kills the last two terms; $\mathbf{v}_1 \neq \mathbf{0}$ gives $\mathbf{v}_1^T\mathbf{v}_1 > 0$, so $c_1 = 0$. Dotting with $\mathbf{v}_2$ gives $c_2 = 0$; with $\mathbf{v}_3$, $c_3 = 0$. Only the trivial combination works — independent.

10. (a) **False.** $P^2 = P$ alone makes P *idempotent* (an oblique projection onto its column space along its null space). For the *orthogonal* projection matrix we built, you also need $P^T = P$ — the lecture's converse requires both. (b) **True.** Dependent columns: $A\mathbf{x} = \mathbf{0}$ for some $\mathbf{x} \neq \mathbf{0}$. Then $A^T A\mathbf{x} = A^T \mathbf{0} = \mathbf{0}$ with $\mathbf{x} \neq \mathbf{0}$, so $A^T A$ is singular. (c) **True.** $P = A(A^T A)^{-1}A^T$ projects onto the whole space $C(A)$; changing basis $A \rightarrow AQ$ (Q invertible) gives $AQ((AQ)^T AQ)^{-1}(AQ)^T = AQ(Q^T A^T AQ)^{-1}Q^T A^T = A(A^T A)^{-1}A^T$ — the same P .

11. $A = (1, 1)^T$, $\mathbf{b} = (1, 2)^T$: $A^T A = 2$, $A^T \mathbf{b} = 3$, $\hat{x} = 3/2$. Residual: $(1 - 3/2, 2 - 3/2)^T = (-1/2, 1/2)^T$, orthogonal to $(1, 1)^T$. Geometry: one unknown, two data points on the x -axis of the unknown — the closest point of the line $\text{span}\{(1, 1)\}$ to $(1, 2)$ sits at the midpoint, the mean.

12. $A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (1, 2, 4)^T$. Same $A^T A = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}$ as problem 7. $A^T \mathbf{b} = (1 + 2 + 4, 1 + 4 + 12)^T = (7, 17)^T$. $\hat{\mathbf{x}} = \frac{1}{6} \begin{pmatrix} 14 & -6 \\ -6 & 3 \end{pmatrix} \begin{pmatrix} 7 \\ 17 \end{pmatrix} = \frac{1}{6}(98 - 102, -42 + 51)^T = \frac{1}{6}(-4, 9)^T = (-2/3, 3/2)^T$.

So $\hat{x} = -2/3$, $\hat{y} = 3/2$. Fitted values: $\mathbf{p} = -\frac{2}{3}(1, 1, 1)^T + \frac{3}{2}(1, 2, 3)^T = (5/6, 7/3, 23/6)^T$. Residual: $\mathbf{e} = (1 - 5/6, 2 - 7/3, 4 - 23/6)^T = (1/6, -1/3, 1/6)^T$. Checks: $\mathbf{e}^T(1, 1, 1)^T = 1/6 - 1/3 + 1/6 = 0$; $\mathbf{e}^T(1, 2, 3)^T = 1/6 - 2/3 + 3/6 = (1 - 4 + 3)/6 = 0$.

6. Eigenvalues, eigenvectors, diagonalization

6.1 What this chapter is for

A matrix A is a machine that moves vectors: $\mathbf{x} \mapsto A\mathbf{x}$. Most vectors get turned as well as stretched. But a few special vectors are only stretched — never turned. Those are the *eigenvectors*, and the stretch factors are the *eigenvalues*.

Why care? The lecture’s motivation: a system of differential equations like $\frac{dv}{dt} = 4v - 5w$, $\frac{dw}{dt} = 2v - 3w$ can be written $\frac{d\mathbf{u}}{dt} = A\mathbf{u}$. Guessing solutions of the form $\mathbf{u}(t) = e^{\lambda t}\mathbf{x}$ and substituting gives $\lambda e^{\lambda t}\mathbf{x} = Ae^{\lambda t}\mathbf{x}$ — and the $e^{\lambda t}$ cancels, leaving $A\mathbf{x} = \lambda\mathbf{x}$. So solving the whole differential equation comes down to solving this one algebraic equation. The same trick will unlock the Fibonacci sequence in §6.9, matrix powers in §6.8, and — further down the road — PCA in Chapter 24.

This chapter answers four questions, in order:

- i) What are *eigenvalues* and *eigenvectors*?
- ii) How do we *find* them?
- iii) What is *diagonalization* — and why does it make powers trivial?
- iv) What is special about *symmetric* matrices?

Basically, ... Every matrix has a few “favorite directions.” Feed it a favorite direction and it just scales it — no rotation. This chapter is about finding those directions and using them to take the matrix apart.

6.2 The definition

Def. For a square matrix A , a scalar λ is an *eigenvalue* and a nonzero vector $\mathbf{x}$ is an *eigenvector* (for λ) if

$$A\mathbf{x} = \lambda\mathbf{x}.$$

This is the *eigen equation*.

Two things to lock in:

- i) $\mathbf{x} \neq \mathbf{0}$ is required. Without it, $A\mathbf{0} = \lambda\mathbf{0}$ would make every λ an “eigenvalue” — meaningless.
- ii) Geometrically: A either *stretches* $\mathbf{x}$ (if $|\lambda| > 1$), *shrinks* it (if $|\lambda| < 1$), *flips* it (if $\lambda < 0$) — but it does **not change its direction**.

$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$: Ax stays on x 's line, Aw does not stay on w 's line

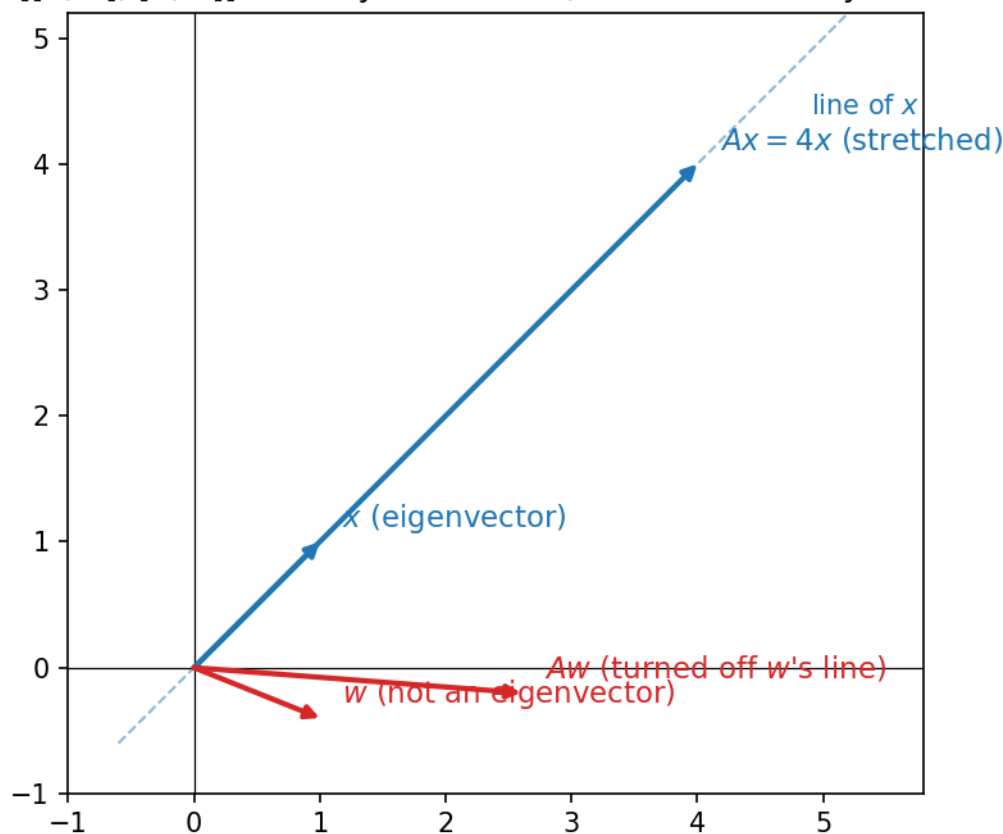


Figure 8: Eigenvectors under $A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$: the eigenvector $x = (1, 1)$ is stretched by 4 and stays on its line (dashed); the non-eigenvector w is turned off its line by A

Note the $\lambda = 0$ case: $Ax = 0$ with $x \neq 0$ means x lives in the *null space* $N(A)$ (§4.7). So “0 is an eigenvalue” is just a fancy way of saying “ A kills some nonzero vector.”

Basically, ... An eigenvector is a direction the matrix respects: it scales the arrow but refuses to turn it. The eigenvalue is the scaling number on the dial.

6.3 How to find them: the characteristic equation

Start from the eigen equation and rearrange:

$$Ax = \lambda x \iff (A - \lambda I)x = 0.$$

We need a **nonzero** x satisfying this. That is possible iff $A - \lambda I$ is *singular* — iff its determinant is zero (§3.6–3.7):

$$\det(A - \lambda I) = 0.$$

This is the *characteristic equation*. The left side is the *characteristic polynomial*: for an $n \times n$ matrix it has degree n , so (counting multiplicity) there are n roots — the eigenvalues.

The recipe:

- i) Solve $\det(A - \lambda I) = 0$ for the eigenvalues $\lambda_1, \dots, \lambda_n$.

- ii) For each λ_i , solve $(A - \lambda_i I)\mathbf{x} = \mathbf{0}$ — i.e. find $N(A - \lambda_i I)$ (§4.7) — by elimination. Any nonzero vector there is an eigenvector.

Note (handy 2×2 check, from the lecture): the eigenvalues satisfy

$$\lambda_1 + \lambda_2 = \text{tr}(A) = a_{11} + a_{22}, \quad \lambda_1 \lambda_2 = \det(A).$$

If your computed eigenvalues fail either, recompute.

eg (eyeball it). A projection P onto a plane: for any $\mathbf{x}$ in the plane, $P\mathbf{x} = \mathbf{x}$ — so $\lambda = 1$ is an eigenvalue, and every in-plane $\mathbf{x}$ is an eigenvector. For $\mathbf{x}$ perpendicular to the plane, $P\mathbf{x} = \mathbf{0}$ — so $\lambda = 0$ is an eigenvalue, with every perpendicular $\mathbf{x}$ an eigenvector. No determinant needed when the geometry is this clear.

eg (eyeball it). The permutation matrix $B = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ swaps coordinates: $B \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (so $\lambda = 1$, eigenvector $(1, 1)^T$) and $B \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} = -\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ (so $\lambda = -1$, eigenvector $(1, -1)^T$).

eg (not always real). The rotation $R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ turns every vector by 90° — no real direction survives.

The algebra agrees: $\det(R - \lambda I) = \begin{vmatrix} -\lambda & -1 \\ 1 & -\lambda \end{vmatrix} = \lambda^2 + 1 = 0$, so $\lambda = \pm i$. Eigenvalues can be complex. (Symmetric matrices are the safe case — always real eigenvalues, §6.11.)

Basically, ... Finding eigenvalues = solving $\det(A - \lambda I) = 0$. Finding eigenvectors = solving the homogeneous system $(A - \lambda I)\mathbf{x} = \mathbf{0}$ — the vectors the shifted matrix kills.

6.4 Worked example: eigenvalues and eigenvectors of a 2×2

eg (full steps). $A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$.

Step 1 — characteristic equation:

$$A - \lambda I = \begin{bmatrix} 3 - \lambda & 1 \\ 1 & 3 - \lambda \end{bmatrix}, \quad \det(A - \lambda I) = (3 - \lambda)^2 - 1 = \lambda^2 - 6\lambda + 8.$$

$$\lambda^2 - 6\lambda + 8 = (\lambda - 4)(\lambda - 2) = 0 \implies \lambda_1 = 4, \lambda_2 = 2.$$

Check: $\lambda_1 + \lambda_2 = 6 = \text{tr}(A)$; $\lambda_1 \lambda_2 = 8 = \det(A)$.

Step 2 — eigenvector for $\lambda_1 = 4$:

$$A - 4I = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}, \quad (A - 4I)\mathbf{x} = \mathbf{0} \iff -x_1 + x_2 = 0 \iff x_2 = x_1.$$

Pick $x_1 = 1$: $\mathbf{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Step 3 — eigenvector for $\lambda_2 = 2$:

$$A - 2I = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad (A - 2I)\mathbf{x} = \mathbf{0} \iff x_1 + x_2 = 0 \iff x_2 = -x_1.$$

Pick $x_1 = 1$: $\mathbf{x}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

Step 4 — verify $A\mathbf{x} = \lambda\mathbf{x}$:

$$A\mathbf{x}_1 = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix} = 4\mathbf{x}_1 \checkmark, \quad A\mathbf{x}_2 = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ -2 \end{bmatrix} = 2\mathbf{x}_2 \checkmark.$$

Note: any nonzero multiple is also an eigenvector — $[2, 2]^T$, $[-1, 1]^T$, etc. An eigenvector names a direction, not a length. The full solution set for $\lambda = 4$ is $\text{span}\{[1, 1]^T\} = N(A - 4I)$, the *eigenspace* (§6.6).

Basically, ... Two systems, solved in order: first the determinant gives the stretch factors, then each stretch factor gives a small linear system whose solutions are the directions.

6.5 A useful shift (and a warning)

eg (full steps). Let $B = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ (the permutation matrix from §6.3) and note $B + 3I = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} = A$, the matrix of §6.4. If $B\mathbf{x} = \mu\mathbf{x}$, then

$$(B + 3I)\mathbf{x} = B\mathbf{x} + 3\mathbf{x} = \mu\mathbf{x} + 3\mathbf{x} = (\mu + 3)\mathbf{x}.$$

So A has the **same eigenvectors** as B , with eigenvalues **shifted by 3**: B 's $(1, [1, 1]^T)$ and $(-1, [1, -1]^T)$ become A 's $(4, [1, 1]^T)$ and $(2, [1, -1]^T)$ — exactly what §6.4 found. Adding a multiple of I slides the spectrum without touching the directions.

Warning (from the lecture — do not skip). It is tempting to generalize: if $A\mathbf{x}_1 = \lambda_1\mathbf{x}_1$ and $B\mathbf{x}_2 = \lambda_2\mathbf{x}_2$, is $(\lambda_1 + \lambda_2)$ an eigenvalue of $A + B$? **No** — because $\mathbf{x}_1$ need not equal $\mathbf{x}_2$. The shift trick above works only because one and the same $\mathbf{x}$ is an eigenvector of both B and $3I$ (every vector is an eigenvector of I). Sharing eigenvectors is a strong condition — it is exactly what diagonalization (§6.7) will exploit.

Basically, ... $A = B + cI$ keeps every eigenvector of B and just adds c to every eigenvalue. But you can't add eigenvalues across two arbitrary matrices — the eigenvectors have to match.

6.6 Eigenspaces, and why distinct eigenvalues give independence

Def. The *eigenspace* for λ is $N(A - \lambda I)$: every eigenvector for λ , plus $\mathbf{0}$. It is a subspace (§4.2): it is exactly “everything $A - \lambda I$ kills.”

Claim. Eigenvectors belonging to *distinct* eigenvalues are *linearly independent*.

Proof for two (from the lecture; the n -case follows the same argument): suppose $A\mathbf{x}_1 = \lambda_1\mathbf{x}_1$, $A\mathbf{x}_2 = \lambda_2\mathbf{x}_2$ with $\lambda_1 \neq \lambda_2$, and

$$c_1\mathbf{x}_1 + c_2\mathbf{x}_2 = \mathbf{0} \tag{1}$$

Apply A to both sides:

$$c_1\lambda_1\mathbf{x}_1 + c_2\lambda_2\mathbf{x}_2 = \mathbf{0} \tag{2}$$

Subtract $\lambda_2 \times (1)$ from (2):

$$c_1(\lambda_1 - \lambda_2)\mathbf{x}_1 = \mathbf{0}.$$

Since $\lambda_1 \neq \lambda_2$ and $\mathbf{x}_1 \neq \mathbf{0}$, we must have $c_1 = 0$; similarly $c_2 = 0$. Hence $\{\mathbf{x}_1, \mathbf{x}_2\}$ is independent (§4.4).

Corollary: if an $n \times n$ matrix has n **distinct** eigenvalues, it has n independent eigenvectors — and (next section) that is all diagonalization needs.

Basically, ... Different eigenvalues pull in genuinely different directions — no eigenvector can be built out of eigenvectors for other eigenvalues. Enough distinct eigenvalues, enough independent directions.

6.7 Diagonalization: $S^{-1}AS = \Lambda$

Def. An $n \times n$ matrix A is *diagonalizable* if there is an invertible S with

$$S^{-1}AS = \Lambda,$$

where Λ is diagonal. Equivalently $A = S\Lambda S^{-1}$ and $AS = S\Lambda$.

Why independent eigenvectors are the whole game: suppose A has n independent eigenvectors $\mathbf{x}_1, \dots, \mathbf{x}_n$. Stack them as columns: $S = [\mathbf{x}_1 \ \dots \ \mathbf{x}_n]$. Independent columns in a square matrix $\Rightarrow S$ is invertible (§4.5). Then

$$AS = A[\mathbf{x}_1 \ \dots \ \mathbf{x}_n] = [A\mathbf{x}_1 \ \dots \ A\mathbf{x}_n] = [\lambda_1\mathbf{x}_1 \ \dots \ \lambda_n\mathbf{x}_n] = [\mathbf{x}_1 \ \dots \ \mathbf{x}_n] \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{bmatrix} = S\Lambda.$$

Left-multiply by S^{-1} : $S^{-1}AS = \Lambda$. Done. So:

- i) n independent eigenvectors $\Rightarrow A$ is diagonalizable;
- ii) in particular, n distinct eigenvalues $\Rightarrow A$ is diagonalizable (§6.6).

Two remarks from the lecture:

- i) S is **not unique**: scaling any eigenvector still gives an eigenvector, so many S 's work.
- ii) Λ is (up to ordering): its diagonal entries must be the eigenvalues of A . Why? From $AS = S\Lambda$, column 1 reads $A\mathbf{y}_1 = \lambda_1\mathbf{y}_1$ — column 1 of S is forced to be an eigenvector for whatever sits at Λ_{11} .

eg (full steps). $A = \begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix}$.

Step 1 — eigenvalues:

$$\det(A - \lambda I) = \begin{vmatrix} 1 - \lambda & 4 \\ 2 & 3 - \lambda \end{vmatrix} = (1 - \lambda)(3 - \lambda) - 8 = \lambda^2 - 4\lambda - 5 = (\lambda - 5)(\lambda + 1).$$

$$\lambda_1 = 5, \quad \lambda_2 = -1.$$

Distinct $\Rightarrow$ diagonalizable.

Step 2 — eigenvectors. For $\lambda_1 = 5$:

$$A - 5I = \begin{bmatrix} -4 & 4 \\ 2 & -2 \end{bmatrix} \Rightarrow -4x_1 + 4x_2 = 0 \Rightarrow \mathbf{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

For $\lambda_2 = -1$:

$$A + I = \begin{bmatrix} 2 & 4 \\ 2 & 4 \end{bmatrix} \Rightarrow 2x_1 + 4x_2 = 0 \Rightarrow \mathbf{x}_2 = \begin{bmatrix} -2 \\ 1 \end{bmatrix}.$$

Step 3 — assemble $S = [\mathbf{x}_1 \ \mathbf{x}_2] = \begin{bmatrix} 1 & -2 \\ 1 & 1 \end{bmatrix}$, $\Lambda = \begin{bmatrix} 5 & 0 \\ 0 & -1 \end{bmatrix}$ (order matched: column 1 $\leftrightarrow$ eigenvalue 5).

Step 4 — verify $S^{-1}AS = \Lambda$. $\det S = 1 \cdot 1 - (-2) \cdot 1 = 3$, so $S^{-1} = \frac{1}{3} \begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix}$.

$$AS = \begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 5 & 2 \\ 5 & -1 \end{bmatrix},$$

$$S^{-1}(AS) = \frac{1}{3} \begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 5 & 2 \\ 5 & -1 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 15 & 0 \\ 0 & -3 \end{bmatrix} = \begin{bmatrix} 5 & 0 \\ 0 & -1 \end{bmatrix} = \Lambda \checkmark.$$

Basically, ... Diagonalization = describing the matrix in the language of its own eigenvectors. S is the dictionary (its columns are the new basis); in that language A is just “ λ_1 along direction 1, λ_2 along direction 2, ...” — pure scaling, no mixing. Same map, better basis.

6.8 Powers become easy: $A^k = S\Lambda^k S^{-1}$

From $S^{-1}AS = \Lambda$:

$$S^{-1}A^2S = (S^{-1}AS)(S^{-1}AS) = \Lambda^2,$$

and the same for any k : $A^k = S\Lambda^k S^{-1}$. Since Λ is diagonal, Λ^k is just the diagonal entries raised to k . (Also: $A\mathbf{x} = \lambda\mathbf{x} \Rightarrow A^2\mathbf{x} = A(\lambda\mathbf{x}) = \lambda A\mathbf{x} = \lambda^2\mathbf{x}$ — the same S diagonalizes A^2 .)

eg (full steps). $A = \begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix}$ from §6.7, with $S = \begin{bmatrix} 1 & -2 \\ 1 & 1 \end{bmatrix}$, $\Lambda = \text{diag}(5, -1)$, $S^{-1} = \frac{1}{3} \begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix}$.

$$\Lambda^2 = \begin{bmatrix} 25 & 0 \\ 0 & 1 \end{bmatrix}, \quad S\Lambda^2 = \begin{bmatrix} 1 & -2 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 25 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 25 & -2 \\ 25 & 1 \end{bmatrix},$$

$$A^2 = S\Lambda^2 S^{-1} = \frac{1}{3} \begin{bmatrix} 25 & -2 \\ 25 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 27 & 48 \\ 24 & 51 \end{bmatrix} = \begin{bmatrix} 9 & 16 \\ 8 & 17 \end{bmatrix}.$$

Direct check: $A^2 = \begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix} = \begin{bmatrix} 9 & 16 \\ 8 & 17 \end{bmatrix}$. For A^{100} the direct route is hopeless; the diagonal route is three small multiplications.

Basically, ... Powers of a diagonal matrix are trivial — just power each diagonal entry. Diagonalization moves that triviality over to A .

6.9 Case study: the Fibonacci sequence via diagonalization

The Fibonacci recurrence $F_{k+2} = F_{k+1} + F_k$ with $F_0 = 0$, $F_1 = 1$ is linear — so pack two consecutive terms into a vector and watch it evolve by one matrix.

Let $\mathbf{u}_k = \begin{bmatrix} F_k \\ F_{k+1} \end{bmatrix}$. Then

$$\mathbf{u}_{k+1} = \begin{bmatrix} F_{k+1} \\ F_{k+2} \end{bmatrix} = \begin{bmatrix} F_{k+1} \\ F_k + F_{k+1} \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}}_A \begin{bmatrix} F_k \\ F_{k+1} \end{bmatrix} = A\mathbf{u}_k,$$

so $\mathbf{u}_k = A^k \mathbf{u}_0$ with $\mathbf{u}_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.

Step 1 — diagonalize A . Characteristic equation:

$$\det(A - \lambda I) = \begin{vmatrix} -\lambda & 1 \\ 1 & 1 - \lambda \end{vmatrix} = -\lambda(1 - \lambda) - 1 = \lambda^2 - \lambda - 1 = 0,$$

$$\lambda_1 = \frac{1 + \sqrt{5}}{2}, \quad \lambda_2 = \frac{1 - \sqrt{5}}{2}.$$

Distinct $\Rightarrow$ diagonalizable.

Step 2 — eigenvectors. Solve $(A - \lambda I)\mathbf{x} = \mathbf{0}$: $\begin{bmatrix} -\lambda & 1 \\ 1 & 1 - \lambda \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \mathbf{0}$. Take $x_1 = 1$, $x_2 = \lambda$: the first row gives $-\lambda + \lambda = 0$; the second gives $1 + (1 - \lambda)\lambda = 1 + \lambda - \lambda^2 = 0$ (since $\lambda^2 = \lambda + 1$ from the characteristic equation). So

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ \lambda_1 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 1 \\ \lambda_2 \end{bmatrix}.$$

Step 3 — write $\mathbf{u}_0$ in the eigenbasis: $\mathbf{u}_0 = c_1 \mathbf{x}_1 + c_2 \mathbf{x}_2$:

$$\begin{bmatrix} c_1 + c_2 \\ c_1 \lambda_1 + c_2 \lambda_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

From the first row $c_2 = -c_1$; the second gives $c_1(\lambda_1 - \lambda_2) = 1$. Since $\lambda_1 - \lambda_2 = \sqrt{5}$: $c_1 = \frac{1}{\sqrt{5}}$, $c_2 = -\frac{1}{\sqrt{5}}$.

Step 4 — iterate. Applying A k times scales each eigen-component by its eigenvalue (§6.8):

$$\mathbf{u}_k = c_1 \lambda_1^k \mathbf{x}_1 + c_2 \lambda_2^k \mathbf{x}_2.$$

The first component is F_k :

$$F_k = \frac{\lambda_1^k - \lambda_2^k}{\sqrt{5}} \quad (\text{Binet's formula}).$$

Step 5 — the approximation. $|\lambda_2| = \frac{\sqrt{5}-1}{2} \approx 0.618 < 1$, so $\lambda_2^k \rightarrow 0$ as k grows. For large k :

$$F_k \approx \frac{\lambda_1^k}{\sqrt{5}}.$$

For $k = 100$: $F_{100} \approx \frac{\left(\frac{1+\sqrt{5}}{2}\right)^{100}}{\sqrt{5}} \approx 3.54 \times 10^{20}$ (the exact value is 354224848179261915075 — the dropped term is far below 1).

Note the lecture's bottom line: this works because the recurrence is **linear** — $\mathbf{u}_{k+1}$ is a linear function of $\mathbf{u}_k$. A nonlinear recurrence breaks the whole analysis.

Basically, ... Fibonacci is a little 2-D machine applied over and over. Diagonalization splits the machine into two independent 1-D scalars: one grows (the golden ratio $\lambda_1 \approx 1.618$), one shrinks ($\lambda_2 \approx -0.618$). After 100 steps the shrinker has vanished — the golden ratio runs the whole show.

6.10 When diagonalization fails: defective matrices

Not every matrix has enough independent eigenvectors. If an eigenvalue repeats but its eigenspace is too small, no invertible S exists.

eg. $A = \begin{bmatrix} 3 & 1 \\ 0 & 3 \end{bmatrix}$: $\det(A - \lambda I) = (3 - \lambda)^2 = 0$, so $\lambda = 3$ twice. But

$$A - 3I = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad (A - 3I)\mathbf{x} = \mathbf{0} \iff x_2 = 0,$$

so the eigenspace is just $\text{span}\{[1, 0]^T\}$ — one independent eigenvector for a 2×2 matrix. No S can be built; A is *defective* (not diagonalizable).

Note: a repeated eigenvalue does not *automatically* break things — the identity matrix has $\lambda = 1$ twice and is perfectly diagonalizable. The problem is only when the repeated eigenvalue comes with too few independent directions.

Basically, ... Diagonalization needs n independent directions for an $n \times n$ matrix. If an eigenvalue “should” supply two directions but only supplies one, the matrix can't be un-mixed — it's defective.

6.11 Symmetric matrices: the spectral theorem

For a *real symmetric* matrix ($A^T = A$) the lecture states the *spectral theorem* (the proof needs complex-matrix background, taken up later — here we take it as a fact):

- i) all eigenvalues of A are **real**;
- ii) eigenvectors for distinct eigenvalues are linearly independent;
- iii) A is **orthogonally diagonalizable**: there is an orthogonal Q with

$$A = Q\Lambda Q^T, \quad Q^T Q = I.$$

Since $Q^T Q = I$ means $Q^{-1} = Q^T$ (§5.4), this is diagonalization with a free inverse — no S^{-1} to compute. The columns of Q are **orthonormal** eigenvectors (§5.4).

Contrast with §6.10: no symmetric matrix is defective. Symmetric $\Rightarrow$ always diagonalizable, always real spectrum. (A general real matrix has no such guarantee — §6.3's rotation and §6.10's defective matrix.)

eg (full steps). $A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$ is symmetric ($A^T = A$). From §6.4: eigenvalues 4, 2 with eigenvectors $[1, 1]^T$, $[1, -1]^T$ — already orthogonal ($[1, 1]^T [1, -1] = 1 - 1 = 0$; §5.2: orthogonal nonzero vectors are independent).

Step 1 — normalize to unit eigenvectors:

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{q}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad Q = [\mathbf{q}_1 \ \mathbf{q}_2] = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Step 2 — sanity check $Q^T Q = I$ (the check the lecture insists on):

$$Q^T Q = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \checkmark.$$

So $Q^{-1} = Q^T$.

Step 3 — verify $A = Q \Lambda Q^T$ with $\Lambda = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}$:

$$\begin{aligned} \Lambda Q^T &= \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 4 & 4 \\ 2 & -2 \end{bmatrix}, \\ Q \Lambda Q^T &= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 4 & 4 \\ 2 & -2 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 6 & 2 \\ 2 & 6 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} = A \checkmark. \end{aligned}$$

Note: stacking the *unnormalized* eigenvectors $[1, 1]^T$, $[1, -1]^T$ would still diagonalize A (§6.7) but would **not** give $Q^T Q = I$. The orthonormality is what makes the inverse free.

Basically, ... Symmetric matrices are the well-behaved citizens: real eigenvalues, and you can always pick eigenvectors that are unit-length and mutually perpendicular. Then diagonalization reads $A = Q \Lambda Q^T$ — and Q 's inverse is just its transpose.

6.12 Why ML cares

- i) *Covariance matrices* are symmetric. Their eigendecomposition $C = Q \Lambda Q^T$ exists with real eigenvalues and orthonormal eigenvectors — this is the engine of PCA (Chapter 24): the eigenvectors are the principal directions, the eigenvalues the variances along them.
- ii) *Quadratic forms* $x^T A x$: for symmetric A , writing $\mathbf{x}$ in the eigenbasis decouples the form into $\sum_i \lambda_i y_i^2$ — the sign pattern of the eigenvalues tells you whether a critical point is a minimum, maximum, or saddle. Preview for the optimization chapters (10/12).
- iii) $A^T A$ is symmetric, so it diagonalizes as $Q \Lambda Q^T$ — and its eigenvalues turn out to be the **squared singular values** of A . That fact is the bridge to the SVD in Chapter 7 (stated here as a pointer, derived there).

6.13 Where this goes next

- **Chapter 7 (SVD):** eigendecomposition needs a square matrix. The singular value decomposition $A = U \Sigma V^T$ extends the same idea — “special directions, pure scaling” — to *every* matrix, rectangular included. The eigenpairs of $A^T A$ (§6.12(iii)) are the bridge.
- **Chapter 24 (PCA):** the covariance matrix is symmetric, so §6.11 applies directly: its orthonormal eigenvectors are the principal components, its eigenvalues the explained variances.

Problem set

- For $A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$: (a) find the characteristic polynomial and both eigenvalues; (b) find an eigenvector for each; (c) verify $A\mathbf{x} = \lambda\mathbf{x}$ for each pair; (d) check $\lambda_1 + \lambda_2 = \text{tr}(A)$ and $\lambda_1\lambda_2 = \det(A)$.
- Let $A = \begin{bmatrix} 2 & 2 \\ 1 & 3 \end{bmatrix}$. Which of these claims are true? (a) $\lambda = 4$, $\mathbf{x} = [1, 1]^T$ is an eigenpair; (b) $\lambda = 2$, $\mathbf{x} = [1, -1]^T$ is an eigenpair; (c) $\lambda = 1$, $\mathbf{x} = [2, -1]^T$ is an eigenpair. Justify each by direct computation.
- Suppose $P^2 = P$ (a projection matrix, §6.3) and $P\mathbf{x} = \lambda\mathbf{x}$ with $\mathbf{x} \neq \mathbf{0}$. Show λ can only be 0 or 1.
- Let $B = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ with eigenpairs $(1, [1, 1]^T)$ and $(-1, [1, -1]^T)$. (a) Without computing any determinant, find the eigenvalues and eigenvectors of $A = B + 3I$. (b) Explain in one line why the same trick does not give the eigenvalues of $B + C$ for a general matrix C .
- Diagonalize $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$: find eigenvalues, eigenvectors, build S and Λ , and verify $S^{-1}AS = \Lambda$ explicitly.
- Using your diagonalization from Problem 5, compute $A^3 = S\Lambda^3S^{-1}$ and check the result by direct multiplication.
- For $A = \begin{bmatrix} 3 & 1 \\ 0 & 3 \end{bmatrix}$: (a) find the eigenvalues; (b) find $N(A - \lambda I)$ and its dimension; (c) explain why A is not diagonalizable.
- Orthogonally diagonalize $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$: give unit eigenvectors, form Q , check $Q^TQ = I$, and verify $A = Q\Lambda Q^T$.
- For $R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$: (a) find the eigenvalues from $\det(R - \lambda I) = 0$; (b) verify $\lambda_1 + \lambda_2 = \text{tr}(R)$ and $\lambda_1\lambda_2 = \det(R)$; (c) why does this not contradict the spectral theorem (§6.11)?
- (a) Write down Binet's formula from §6.9. (b) Use it to estimate F_{20} , and bound the size of the neglected term $|\lambda_2|^{20}/\sqrt{5}$. (c) In one line: why is the neglected term so small?

Solutions — Chapter 6. Eigenvalues, eigenvectors, diagonalization

- $A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$.
 - $A - \lambda I = \begin{bmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{bmatrix}$, $\det = (4-\lambda)(3-\lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda-5)(\lambda-2)$. Eigenvalues: $\lambda_1 = 5$, $\lambda_2 = 2$.
 - $\lambda_1 = 5$: $A - 5I = \begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix}$; $-x_1 + x_2 = 0$, so $\mathbf{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. $\lambda_2 = 2$: $A - 2I = \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix}$; $2x_1 + x_2 = 0$, so $\mathbf{x}_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$.
 - $A\mathbf{x}_1 = \begin{bmatrix} 4+1 \\ 2+3 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix} = 5\mathbf{x}_1$. $A\mathbf{x}_2 = \begin{bmatrix} 4-2 \\ 2-6 \end{bmatrix} = \begin{bmatrix} 2 \\ -4 \end{bmatrix} = 2\mathbf{x}_2$.
 - $5 + 2 = 7 = 4 + 3 = \text{tr}(A)$; $5 \cdot 2 = 10 = 4 \cdot 3 - 1 \cdot 2 = \det(A)$.
- $A = \begin{bmatrix} 2 & 2 \\ 1 & 3 \end{bmatrix}$.
 - True: $A \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix} = 4 \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.
 - False: $A \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ -2 \end{bmatrix}$, but $2 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ -2 \end{bmatrix}$.

(c) True: $A \begin{bmatrix} 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 4-2 \\ 2-3 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \end{bmatrix} = 1 \cdot \begin{bmatrix} 2 \\ -1 \end{bmatrix}$. (Indeed the characteristic polynomial is $\lambda^2 - 5\lambda + 4 = (\lambda - 4)(\lambda - 1)$, so 2 is not even an eigenvalue.)

3. $P\mathbf{x} = \lambda\mathbf{x}$, $\mathbf{x} \neq \mathbf{0}$, and $P^2 = P$. Apply P to both sides of $P\mathbf{x} = \lambda\mathbf{x}$: $P^2\mathbf{x} = \lambda P\mathbf{x} = \lambda^2\mathbf{x}$. But $P^2 = P$, so $P^2\mathbf{x} = P\mathbf{x} = \lambda\mathbf{x}$. Hence $\lambda^2\mathbf{x} = \lambda\mathbf{x}$, i.e. $(\lambda^2 - \lambda)\mathbf{x} = \mathbf{0}$. Since $\mathbf{x} \neq \mathbf{0}$, $\lambda^2 - \lambda = 0$, so $\lambda = 0$ or $\lambda = 1$. (This is exactly the plane-projection example of §6.3: $\lambda = 1$ in the plane, $\lambda = 0$ perpendicular to it.)

4. (a) By the shift rule (§6.5): $A = B + 3I$ keeps B 's eigenvectors and adds 3 to each eigenvalue. Eigenpairs of A : $(1 + 3, [1, 1]^T) = (4, [1, 1]^T)$ and $(-1 + 3, [1, -1]^T) = (2, [1, -1]^T)$ — matching the §6.4 computation. (b) The rule used $(B + cI)\mathbf{x} = B\mathbf{x} + c\mathbf{x}$, which needs $\mathbf{x}$ to be an eigenvector of **both** B and cI — true since every vector is an eigenvector of I . A general C does not share B 's eigenvectors, so $(B + C)\mathbf{x} \neq (\mu + \nu)\mathbf{x}$ in general (§6.5 warning).

5. $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$: $\det(A - \lambda I) = (2 - \lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 3)(\lambda - 1)$; $\lambda_1 = 3$, $\lambda_2 = 1$.

$\lambda_1 = 3$: $A - 3I = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}$, $\mathbf{x}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. $\lambda_2 = 1$: $A - I = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$, $\mathbf{x}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

$S = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, $\Lambda = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$. $\det S = -2$, $S^{-1} = -\frac{1}{2} \begin{bmatrix} -1 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix}$.

$$AS = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 3 & -1 \end{bmatrix},$$

$$S^{-1}AS = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 3 & -1 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} = \Lambda \checkmark.$$

6. $\Lambda^3 = \text{diag}(27, 1)$. $S\Lambda^3 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 27 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 27 & 1 \\ 27 & -1 \end{bmatrix}$.

$$A^3 = S\Lambda^3S^{-1} = \begin{bmatrix} 27 & 1 \\ 27 & -1 \end{bmatrix} \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} = \begin{bmatrix} 14 & 13 \\ 13 & 14 \end{bmatrix}.$$

Direct: $A^2 = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}$, $A^3 = A^2A = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 14 & 13 \\ 13 & 14 \end{bmatrix}$.

7. $A = \begin{bmatrix} 3 & 1 \\ 0 & 3 \end{bmatrix}$.

(a) $\det(A - \lambda I) = (3 - \lambda)^2$: $\lambda = 3$ (double root).

(b) $A - 3I = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$; $(A - 3I)\mathbf{x} = \mathbf{0}$ gives $x_2 = 0$, x_1 free. $N(A - 3I) = \text{span}\{[1, 0]^T\}$, dimension 1.

(c) A 2×2 matrix needs 2 independent eigenvectors to diagonalize; there is only one independent direction here. A is defective — not diagonalizable.

8. $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ is symmetric; from Problem 5, eigenvalues 3, 1 with (orthogonal) eigenvectors $[1, 1]^T$, $[1, -1]^T$.

Unit eigenvectors: $\mathbf{q}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\mathbf{q}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$; $Q = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$.

$$Q^T Q = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = I \checkmark.$$

With $\Lambda = \text{diag}(3, 1)$: $\Lambda Q^T = \frac{1}{\sqrt{2}} \begin{bmatrix} 3 & 3 \\ 1 & -1 \end{bmatrix}$,

$$Q\Lambda Q^T = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 3 & 3 \\ 1 & -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = A \checkmark.$$

9. (a) $\det(R - \lambda I) = \begin{vmatrix} -\lambda & -1 \\ 1 & -\lambda \end{vmatrix} = \lambda^2 + 1 = 0$: $\lambda_1 = i$, $\lambda_2 = -i$. (b) $i + (-i) = 0 = \text{tr}(R)$; $i \cdot (-i) = -i^2 = 1 = \det(R)$. (The trace/determinant relations hold over $\mathbb{C}$ too.) (c) No contradiction: the spectral theorem applies to **symmetric** matrices, and $R^T = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} = -R \neq R$. A non-symmetric real matrix may have complex eigenvalues — exactly what §6.3 warned about.

10. (a) $F_k = \frac{\lambda_1^k - \lambda_2^k}{\sqrt{5}}$ with $\lambda_1 = \frac{1+\sqrt{5}}{2}$, $\lambda_2 = \frac{1-\sqrt{5}}{2}$. (b) Dropping the λ_2 term: $F_{20} \approx \lambda_1^{20}/\sqrt{5} \approx 6765.0000296$.

The neglected term is $|\lambda_2|^{20}/\sqrt{5} \approx 2.96 \times 10^{-5}$ — the estimate is correct to 4 decimal places, and rounding gives the exact integer $F_{20} = 6765$. (c) Because $|\lambda_2| \approx 0.618 < 1$: a number smaller than 1 raised to the 20th power is tiny.

7. SVD and positive definite matrices

Chapter 6 ended with a limitation: eigendecomposition wants a *square* matrix with enough independent eigenvectors, and symmetric matrices are the only ones that always cooperate (§6.11). This chapter removes both restrictions. First, the **singular value decomposition** — a factorization that works for *every* matrix, rectangular included. Then **positive definite matrices** — the quadratic-form test that decides whether a stationary point is a minimum, a workhorse of optimization.

7.1 Why eigendecomposition is not enough

- i) Eigendecomposition $A = S\Lambda S^{-1}$ needs a square $n \times n$ matrix (§6.7).
- ii) It needs n independent eigenvectors — defective matrices (§6.10) simply have no such factorization.
- iii) Only symmetric matrices are *guaranteed* to cooperate (§6.11).

The SVD drops all of this: *any* real $m \times n$ matrix gets factored, no symmetry, no squareness, no eigenvector-counting required.

7.2 The SVD, stated

Def. Every real $m \times n$ matrix A can be written as

$$A = U\Sigma V^T$$

where:

- i) U is $m \times m$ with **orthonormal** columns ($U^T U = I$);
- ii) V is $n \times n$ with **orthonormal** columns ($V^T V = I$);
- iii) Σ is $m \times n$, all zeros except a diagonal $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ down its main diagonal.

Terminology: the σ_i are the *singular values* of A . The columns of V are the *right singular vectors*; the columns of U are the *left singular vectors*.

eg. For a 2×3 matrix, the shapes read

$$\begin{bmatrix} * & * & * \\ * & * & * \end{bmatrix}_{2 \times 3} = U_{2 \times 2} \Sigma_{2 \times 3} V_{2 \times 3}^T, \quad \Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \end{bmatrix}.$$

U and V are square and orthogonal; the “diagonal” Σ is rectangular, with the singular values sitting on its main diagonal (§7.2’s only rectangle-with-a-diagonal).

Basically, ... SVD says: every matrix, no matter how lopsided, is just “turn $\rightarrow$ stretch $\rightarrow$ turn”. V^T and U are pure rotations/reflections (they never change lengths); all the stretching is done by the diagonal Σ .

7.3 Where U , Σ , V come from: the $A^T A$ bridge

Chapter 6, §6.12(iii) promised this: the eigenvalues of $A^T A$ are the squared singular values. Here is the bridge, built from the lecture’s argument.

Step 0 — $A^T A$ is symmetric. For *any* A , $(A^T A)^T = A^T (A^T)^T = A^T A$. By the spectral theorem (§6.11) it has n real eigenvalues and an orthonormal eigenbasis $\mathbf{x}_1, \dots, \mathbf{x}_n$ of $\mathbb{R}^n$:

$$A^T A \mathbf{x}_i = \lambda_i \mathbf{x}_i, \quad \mathbf{x}_i^T \mathbf{x}_j = 0 \quad (i \neq j), \quad \|\mathbf{x}_i\| = 1.$$

Step 1 — the eigenvalues are ≥ 0 . Dotting the eigen equation with $\mathbf{x}_i$:

$$\lambda_i = \mathbf{x}_i^T A^T A \mathbf{x}_i = (A \mathbf{x}_i)^T (A \mathbf{x}_i) = \|A \mathbf{x}_i\|^2 \geq 0.$$

So order them $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_r > 0 = \lambda_{r+1} = \dots = \lambda_n$. The *singular values* are

$$\sigma_i = \sqrt{\lambda_i} \quad (i = 1, \dots, r),$$

placed as the diagonal of Σ . (This is why the square root appears: Σ will hold σ_i , and σ_i^2 are the eigenvalues.)

Step 2 — V and U . Put the orthonormal $\mathbf{x}_i$ as the columns of V . For $i \leq r$ define

$$\mathbf{y}_i = \frac{1}{\sigma_i} A \mathbf{x}_i.$$

Then each $\mathbf{y}_i$ is unit length ($\|A \mathbf{x}_i\| = \sqrt{\lambda_i} = \sigma_i$), and they are mutually orthogonal:

$$\mathbf{y}_i^T \mathbf{y}_j = \frac{1}{\sigma_i \sigma_j} \mathbf{x}_i^T A^T A \mathbf{x}_j = \frac{\lambda_j}{\sigma_i \sigma_j} \mathbf{x}_i^T \mathbf{x}_j = 0 \quad (i \neq j).$$

Extend $\mathbf{y}_1, \dots, \mathbf{y}_r$ to a full orthonormal basis of $\mathbb{R}^m$ (always possible); these m vectors are the columns of U .

Step 3 — $U \Sigma V^T = A$. The (i, j) entry of $U^T A V$ is $\mathbf{y}_i^T A \mathbf{x}_j$: it equals σ_j when $i = j \leq r$ (since $A \mathbf{x}_j = \sigma_j \mathbf{y}_j$) and 0 otherwise (for $j > r$, $\lambda_j = 0$ gives $A \mathbf{x}_j = \mathbf{0}$). So $U^T A V = \Sigma$, i.e. $A = U \Sigma V^T$.

Note: from the factorization, $A A^T = U \Sigma \Sigma^T U^T$, so the columns of U are orthonormal eigenvectors of the symmetric $A A^T$ — the left side mirrors the right side.

Basically, ... A itself may have no eigenvectors at all (try the 90° rotation of §6.3). But $A^T A$ is *always* symmetric, so it always has a full set of real eigenvalues and orthonormal eigenvectors. The SVD just reads its ingredients off $A^T A$: take square roots of the eigenvalues to get the stretches, take the eigenvectors as V , and push them through A to get U .

7.4 Worked example: a square but non-diagonalizable 2×2 matrix

Take $A = \begin{bmatrix} \sqrt{2} & 1 \\ 0 & \sqrt{2} \end{bmatrix}$ (the lecture's example). First a sanity check: A has eigenvalue $\sqrt{2}$ twice, but $A - \sqrt{2}I = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ has only a one-dimensional eigenspace — A is defective, so no eigendecomposition exists (§6.10). The SVD has no such problem.

Step 1 — form $A^T A$ and find its eigenpairs:

$$A^T A = \begin{bmatrix} \sqrt{2} & 0 \\ 1 & \sqrt{2} \end{bmatrix} \begin{bmatrix} \sqrt{2} & 1 \\ 0 & \sqrt{2} \end{bmatrix} = \begin{bmatrix} 2 & \sqrt{2} \\ \sqrt{2} & 3 \end{bmatrix}.$$

Trace = 5, determinant = $6 - 2 = 4$, so the characteristic equation is $\lambda^2 - 5\lambda + 4 = 0$, giving $\lambda_1 = 4$, $\lambda_2 = 1$. Singular values: $\sigma_1 = \sqrt{4} = 2$, $\sigma_2 = \sqrt{1} = 1$.

Eigenvectors. For $\lambda_1 = 4$: $A^T A - 4I = \begin{bmatrix} -2 & \sqrt{2} \\ \sqrt{2} & -1 \end{bmatrix}$, and $\begin{bmatrix} 1 \\ \sqrt{2} \end{bmatrix}$ kills it ($-2 + \sqrt{2} \cdot \sqrt{2} = 0$). For $\lambda_2 = 1$:

$A^T A - I = \begin{bmatrix} 1 & \sqrt{2} \\ \sqrt{2} & 2 \end{bmatrix}$, and $\begin{bmatrix} \sqrt{2} \\ -1 \end{bmatrix}$ kills it ($\sqrt{2} - \sqrt{2} = 0$). Normalize (both have length $\sqrt{3}$):

$$\mathbf{x}_1 = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ \sqrt{2} \end{bmatrix}, \quad \mathbf{x}_2 = \frac{1}{\sqrt{3}} \begin{bmatrix} \sqrt{2} \\ -1 \end{bmatrix}, \quad V = [\mathbf{x}_1 \ \mathbf{x}_2] = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & \sqrt{2} \\ \sqrt{2} & -1 \end{bmatrix}.$$

Sanity check $V^T V = I$: $\mathbf{x}_1^T \mathbf{x}_2 = \frac{1}{3}(\sqrt{2} - \sqrt{2}) = 0$, $\|\mathbf{x}_1\|^2 = \frac{1}{3}(1 + 2) = 1$, $\|\mathbf{x}_2\|^2 = \frac{1}{3}(2 + 1) = 1$.

Step 2 — build U from $\mathbf{y}_i = A\mathbf{x}_i/\sigma_i$:

$$A\mathbf{x}_1 = \frac{1}{\sqrt{3}} \begin{bmatrix} \sqrt{2} & 1 \\ 0 & \sqrt{2} \end{bmatrix} \begin{bmatrix} 1 \\ \sqrt{2} \end{bmatrix} = \frac{1}{\sqrt{3}} \begin{bmatrix} 2\sqrt{2} \\ 2 \end{bmatrix}, \quad \mathbf{y}_1 = \frac{1}{2} A\mathbf{x}_1 = \begin{bmatrix} \sqrt{2/3} \\ 1/\sqrt{3} \end{bmatrix},$$

$$A\mathbf{x}_2 = \frac{1}{\sqrt{3}} \begin{bmatrix} \sqrt{2} & 1 \\ 0 & \sqrt{2} \end{bmatrix} \begin{bmatrix} \sqrt{2} \\ -1 \end{bmatrix} = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ -\sqrt{2} \end{bmatrix}, \quad \mathbf{y}_2 = \frac{1}{1} A\mathbf{x}_2 = \begin{bmatrix} 1/\sqrt{3} \\ -\sqrt{2/3} \end{bmatrix}.$$

Check: $\|\mathbf{y}_1\|^2 = 2/3 + 1/3 = 1$, $\|\mathbf{y}_2\|^2 = 1/3 + 2/3 = 1$, $\mathbf{y}_1^T \mathbf{y}_2 = \frac{\sqrt{2}}{3} - \frac{\sqrt{2}}{3} = 0$. So

$$U = \begin{bmatrix} \sqrt{2/3} & 1/\sqrt{3} \\ 1/\sqrt{3} & -\sqrt{2/3} \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}.$$

Step 3 — verify $A = U\Sigma V^T$, entry by entry. First $\Sigma V^T = \frac{1}{\sqrt{3}} \begin{bmatrix} 2 & 2\sqrt{2} \\ \sqrt{2} & -1 \end{bmatrix}$. Then:

$$(1,1): \frac{1}{\sqrt{3}} \left(\sqrt{\frac{2}{3}} \cdot 2 + \frac{1}{\sqrt{3}} \cdot \sqrt{2} \right) = \frac{1}{\sqrt{3}} \cdot \frac{3\sqrt{2}}{\sqrt{3}} = \sqrt{2} \checkmark$$

$$(1,2): \frac{1}{\sqrt{3}} \left(\sqrt{\frac{2}{3}} \cdot 2\sqrt{2} + \frac{1}{\sqrt{3}} \cdot (-1) \right) = \frac{1}{\sqrt{3}} \left(\frac{4}{\sqrt{3}} - \frac{1}{\sqrt{3}} \right) = 1 \checkmark$$

$$(2,1): \frac{1}{\sqrt{3}} \left(\frac{1}{\sqrt{3}} \cdot 2 - \sqrt{\frac{2}{3}} \cdot \sqrt{2} \right) = \frac{1}{\sqrt{3}} \left(\frac{2}{\sqrt{3}} - \frac{2}{\sqrt{3}} \right) = 0 \checkmark$$

$$(2,2): \frac{1}{\sqrt{3}} \left(\frac{1}{\sqrt{3}} \cdot 2\sqrt{2} + \sqrt{\frac{2}{3}} \right) = \frac{1}{\sqrt{3}} \cdot \frac{3\sqrt{2}}{\sqrt{3}} = \sqrt{2} \checkmark$$

All four entries of $A = \begin{bmatrix} \sqrt{2} & 1 \\ 0 & \sqrt{2} \end{bmatrix}$ recovered.

Note: here $\det(U) = -\frac{2}{3} - \frac{1}{3} = -1$ — U contains a reflection, not just a rotation. Orthogonal matrices are allowed to flip (§7.6).

Basically, ... Even when a matrix is too broken for eigenvectors (defective), $A^T A$ is always well-behaved. The recipe never changes: eigenpairs of $A^T A \rightarrow$ square-root the eigenvalues $\rightarrow$ normalize $\rightarrow$ push through A to get U .

7.5 Worked example: a rectangular 2×3 matrix

Take $A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$.

Step 1 — $A^T A$. Columns of A : $\mathbf{c}_1 = (1, 0)^T$, $\mathbf{c}_2 = (0, 1)^T$, $\mathbf{c}_3 = (1, 1)^T$; entry (i, j) is $\mathbf{c}_i \cdot \mathbf{c}_j$:

$$A^T A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

$\det(A^T A - \lambda I) = (1 - \lambda)[(1 - \lambda)(2 - \lambda) - 1] - (1 - \lambda) = (1 - \lambda)(\lambda^2 - 3\lambda) = \lambda(1 - \lambda)(\lambda - 3)$. So $\lambda_1 = 3$, $\lambda_2 = 1$, $\lambda_3 = 0$; singular values $\sigma_1 = \sqrt{3}$, $\sigma_2 = 1$, $\sigma_3 = 0$.

Eigenvectors (normalized): for $\lambda_1 = 3$, $(A^T A - 3I)\mathbf{x} = \mathbf{0}$ gives $x_3 = 2x_1 = 2x_2$, so $\mathbf{x}_1 = \frac{1}{\sqrt{6}}(1, 1, 2)^T$ (check: $(1 + 1 + 4)/6 = 1$). For $\lambda_2 = 1$: $x_3 = 0$, $x_1 + x_2 = 0$, so $\mathbf{x}_2 = \frac{1}{\sqrt{2}}(1, -1, 0)^T$. For $\lambda_3 = 0$: $x_1 + x_3 = 0$, $x_2 + x_3 = 0$, so $\mathbf{x}_3 = \frac{1}{\sqrt{3}}(1, 1, -1)^T$. These are mutually orthogonal (pairwise dot products are 0).

Step 2 — U from the two *nonzero* singular values:

$$A\mathbf{x}_1 = \frac{1}{\sqrt{6}}A(1, 1, 2)^T = \frac{1}{\sqrt{6}}(3, 3)^T, \quad \mathbf{u}_1 = \frac{1}{\sqrt{3}}A\mathbf{x}_1 = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)^T,$$

$$A\mathbf{x}_2 = \frac{1}{\sqrt{2}}A(1, -1, 0)^T = \frac{1}{\sqrt{2}}(1, -1)^T, \quad \mathbf{u}_2 = \frac{1}{1}A\mathbf{x}_2 = \left(\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right)^T.$$

$\mathbf{u}_1, \mathbf{u}_2$ are already an orthonormal basis of $\mathbb{R}^2$ — no extension needed.

Step 3 — assemble and verify. With $r = 2$,

$$U = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{bmatrix}, \quad \Sigma = \begin{bmatrix} \sqrt{3} & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad V = \begin{bmatrix} 1/\sqrt{6} & 1/\sqrt{2} & 1/\sqrt{3} \\ 1/\sqrt{6} & -1/\sqrt{2} & 1/\sqrt{3} \\ 2/\sqrt{6} & 0 & -1/\sqrt{3} \end{bmatrix}.$$

$\Sigma V^T = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} & \sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} & 0 \end{bmatrix}$ (since $\sqrt{3}/\sqrt{6} = 1/\sqrt{2}$ and $2\sqrt{3}/\sqrt{6} = \sqrt{2}$). Multiplying by U , entry by entry:
 $(1, 1) = \frac{1}{2} + \frac{1}{2} = 1$, $(1, 2) = \frac{1}{2} - \frac{1}{2} = 0$, $(1, 3) = \frac{1}{\sqrt{2}} \cdot \sqrt{2} = 1$, $(2, 1) = 0$, $(2, 2) = 1$, $(2, 3) = 1$ — all six entries of A recovered.

Note: $\mathbf{x}_3$ (the null-space direction of $A^T A$) is annihilated: $A\mathbf{x}_3 = \mathbf{0}$, which is why $\sigma_3 = 0$ contributes nothing.

Reduced (economy) SVD. The zero column of Σ and the third column of V do no work. Dropping them:

$$A = U_r \Sigma_r V_r^T, \quad U_r = U \ (2 \times 2), \ \Sigma_r = \begin{bmatrix} \sqrt{3} & 0 \\ 0 & 1 \end{bmatrix}, \ V_r = [\mathbf{x}_1 \ \mathbf{x}_2] \ (3 \times 2).$$

In general: full SVD keeps $U \ m \times m$, $\Sigma \ m \times n$, $V \ n \times n$; reduced SVD keeps only the r nonzero singular values, so U_r is $m \times r$, Σ_r is $r \times r$, V_r is $n \times r$.

7.6 Geometry of the SVD

Apply $A = U\Sigma V^T$ to a vector $\mathbf{x}$ in three stages: V^T rotates/reflects into the right-singular-vector basis, Σ stretches each coordinate by σ_i , U rotates/reflects into the output basis. Since orthogonal maps preserve shapes up to turning, the *only* distortion A ever causes is pure axis-aligned stretching.

Consequence: A sends the unit sphere to an **ellipsoid** whose semiaxes are $\sigma_i \mathbf{u}_i$ — i.e. $A\mathbf{v}_i = \sigma_i \mathbf{u}_i$, the directions where A acts by pure scaling.

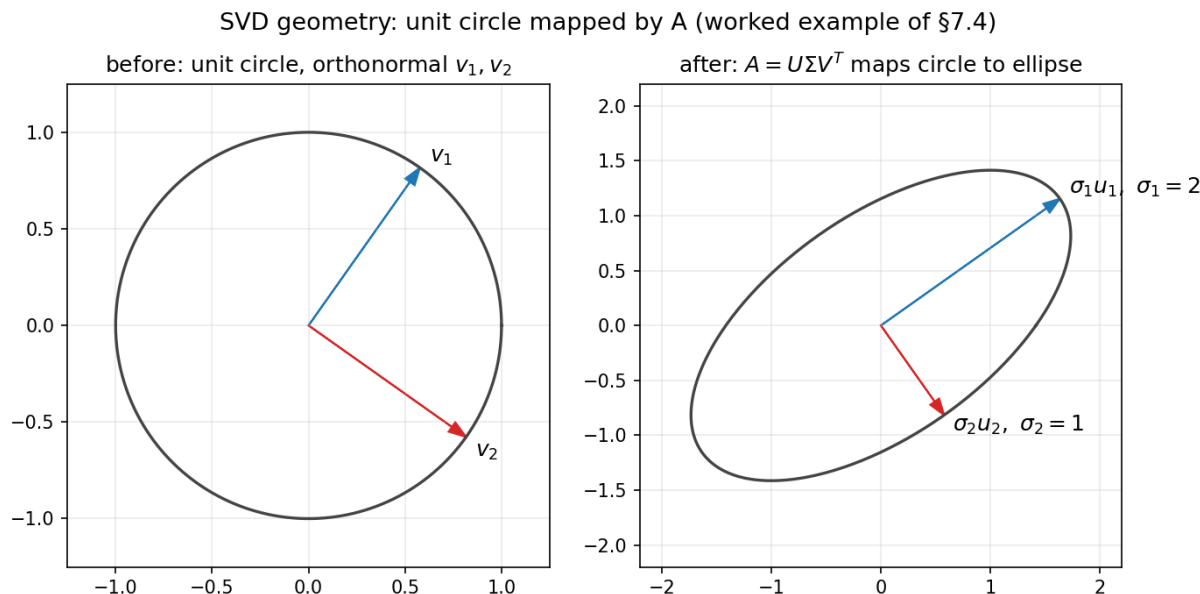


Figure 9: SVD geometry for the §7.4 example: the unit circle with orthonormal $\mathbf{v}_1, \mathbf{v}_2$ maps under A to an ellipse whose semi-axes are $\sigma_1 \mathbf{u}_1$ (length 2) and $\sigma_2 \mathbf{u}_2$ (length 1)

Rank. From the bridge: $\sigma_i = 0 \iff \lambda_i = 0 \iff A\mathbf{x}_i = \mathbf{0}$. So the number of *nonzero* singular values equals $\text{rank}(A)$. In §7.5, $\sigma_3 = 0$ because the two rows of A span only a 2-dimensional space — $\text{rank}(A) = 2$.

Basically, ... Every matrix squashes or stretches a ball into an ellipsoid. The SVD reads off the ellipsoid’s axes: σ_i are the axis lengths, $\mathbf{v}_i$ the input directions that get stretched, $\mathbf{u}_i$ the output directions they land on. Rank is just “how many axes have nonzero length.”

7.7 Why SVD matters in ML

Eigendecomposition is picky (square, enough eigenvectors). SVD applies to *every* data matrix — and data matrices are almost never square: m samples $\times$ n features.

One more payoff: the SVD is a sum of rank-one pieces,

$$A = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T + \cdots + \sigma_r \mathbf{u}_r \mathbf{v}_r^T,$$

each term a matrix with a single nonzero singular value. Keeping only the first k terms gives A_k , the **best rank- k approximation** of A — the rank- k matrix closest to A . Since $\sigma_1 \geq \sigma_2 \geq \cdots$, the dropped tail is the smallest possible error. This is the engine behind dimensionality reduction (keep the big stretches, discard the noise), latent-factor recommendation systems (user–item matrix $\approx$ low-rank structure), and — coming in Chapter 24 — PCA, where the principal components are exactly the right singular vectors of the centered data matrix.

Basically, ... Rank- k approximation is lossy compression for matrices: keep the k biggest stretches of the SVD, throw away the rest, and the leftover error is as small as any rank- k matrix can manage.

7.8 Positive definiteness: from quadratics to matrices

Before matrices, look at the quadratic function $f(x, y) = ax^2 + 2bxy + cy^2$.

eg (the motivating case). Take $f(x, y) = 2x^2 + 4xy + y^2$. Its partials are $f_x = 4x + 4y$, $f_y = 4x + 2y$ — both vanish at $(0, 0)$, as they do for *any* quadratic of this form, whatever its coefficients. The question

is what kind of stationary point $(0, 0)$ is: minimum, maximum, or saddle? Its second partials are $f_{xx} = 4$, $f_{xy} = 4$, $f_{yy} = 2$ — constant, so the same at $(0, 0)$ as everywhere. We will come back to this f in §7.10.

Def. A function f with $f(0, 0) = 0$ that is strictly positive everywhere else is *positive definite*.

Geometric picture: a bowl sitting on the origin, like $f(x, y) = x^2 + y^2$.

- i) $f > 0 \Rightarrow a > 0$: evaluate at $(1, 0)$; $f(1, 0) = a$ must be positive.
- ii) $f > 0 \Rightarrow c > 0$: same argument at $(0, 1)$.
- iii) Complete the square:

$$ax^2 + 2bxy + cy^2 = a\left(x + \frac{b}{a}y\right)^2 + \left(c - \frac{b^2}{a}\right)y^2.$$

The first term is ≥ 0 ; for the whole thing to be > 0 at every nonzero (x, y) , the second coefficient must be > 0 too, i.e. $c - b^2/a > 0$, or $ac > b^2$.

So: $f(x, y) = ax^2 + 2bxy + cy^2$ is positive definite **iff** $a > 0$ and $ac > b^2$.

The other cases: if $ac = b^2$ (with $a > 0$), f is *positive semi-definite* — ≥ 0 , but zero along a whole line, not just at the origin (a flat-bottomed valley). If $a < 0$ with $ac = b^2$, *negative semi-definite*. If $ac < b^2$, the origin is a *saddle point* — up in some directions, down in others.

eg. $f(x, y) = x^2 - 10xy + y^2$ is *not* positive definite: $f(1, 1) = 1 - 10 + 1 = -8 < 0$, even though $a, c > 0$.

eg. $f(x, y) = 2xy$: at $(1, 1)$ it is 2, at $(1, -1)$ it is -2 — a saddle at the origin.

Basically, ... Positive definite means “a bowl”: zero at the origin, strictly uphill in every direction. If the bowl is dented so it dips below the origin somewhere, or flat along a line, it’s not positive definite anymore.

The matrix connection: with $V = (x, y)^T$ and $A = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$,

$$f(x, y) = V^T A V.$$

Positive definiteness of the *function* becomes a property of the *matrix*.

7.9 Positive definite matrices

Def. A (real, *symmetric* — assumed from here on) $n \times n$ matrix A is *positive definite* if

$$\mathbf{x}^T A \mathbf{x} > 0 \quad \text{for every } \mathbf{x} \neq \mathbf{0}.$$

(The symmetry assumption matters: only the symmetric part of a matrix contributes to $\mathbf{x}^T A \mathbf{x}$, so definiteness is defined for symmetric matrices.)

Theorem (eigenvalue test). A is positive definite **iff** all its eigenvalues are strictly positive.

Proof ($\Rightarrow$). If $A\mathbf{x} = \lambda\mathbf{x}$ with $\mathbf{x} \neq \mathbf{0}$: $\mathbf{x}^T A \mathbf{x} = \lambda\|\mathbf{x}\|^2 > 0$ by definiteness, and $\|\mathbf{x}\|^2 > 0$, so $\lambda > 0$.

Proof ($\Leftarrow$). By the spectral theorem (§6.11) there is an orthonormal eigenbasis $\mathbf{x}_1, \dots, \mathbf{x}_n$. Write any $\mathbf{x} = c_1\mathbf{x}_1 + \dots + c_n\mathbf{x}_n$; then $A\mathbf{x} = c_1\lambda_1\mathbf{x}_1 + \dots + c_n\lambda_n\mathbf{x}_n$, and orthonormality kills the cross terms:

$$\mathbf{x}^T A \mathbf{x} = c_1^2\lambda_1 + \dots + c_n^2\lambda_n > 0,$$

since every $\lambda_i > 0$ and at least one $c_i^2 > 0$.

For 2×2 , the quadratic conditions translate directly (with $A = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$): A is positive definite **iff** $a > 0$ and $\det(A) = ac - b^2 > 0$.

eg (two ways). $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

Way 1 — the 2×2 test: $a = 2 > 0$, $\det(A) = 4 - 1 = 3 > 0$. Positive definite .

Way 2 — eigenvalues: $\det(A - \lambda I) = (2 - \lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = 0$ gives $\lambda = 3, 1$, both > 0 . Equivalently, $f(x, y) = \mathbf{x}^T A \mathbf{x} = 2x^2 + 2xy + 2y^2 = 2(x + y/2)^2 + \frac{3}{2}y^2 > 0$ for $(x, y) \neq (0, 0)$.

eg (not definite). $B = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$: $\det(B) = 1 - 4 = -3 < 0$, so not positive definite. Its eigenvalues are 3 and -1 — mixed signs means the origin is a saddle of $\mathbf{x}^T B \mathbf{x}$: along $(1, 1)^T$ the form is $6 > 0$, along $(1, -1)^T$ it is $-2 < 0$.

Basically, ... A symmetric matrix is positive definite iff it stretches *every* direction by a positive amount — all its eigenvalues are positive. If even one eigenvalue is negative or zero, some direction goes downhill or stays flat, and the bowl is broken.

7.10 Bowls, minima, and a look ahead at optimization

Return to the motivating example $f(x, y) = 2x^2 + 4xy + y^2$ from §7.8. Its second partials ($f_{xx} = 4$, $f_{xy} = 4$, $f_{yy} = 2$) are constant, so the same at the stationary point $(0, 0)$ as everywhere. Writing $f = ax^2 + 2bxy + cy^2$ gives $a = 2$, $b = 2$, $c = 1$ — note the 2×2 test uses the *halved* mixed coefficient ($2bxy = 4xy$, so $b = 2$). Then $\det = ac - b^2 = 2 - 4 = -2 < 0$: the origin is a **saddle**, not a minimum, despite $a > 0$.

The general principle (developed in Chapters 10 and 12): at a stationary point, the matrix of second derivatives — the *Hessian* — plays the role of A above. Positive definite Hessian $\Rightarrow$ bowl $\Rightarrow$ local minimum; indefinite Hessian $\Rightarrow$ saddle. In ML, that Hessian is exactly what second-order optimizers examine, and covariance matrices (symmetric, always at least positive *semi*-definite since $\mathbf{w}^T C \mathbf{w} = \text{Var}(\mathbf{w}^T \mathbf{x}) \geq 0$) are the other place definiteness shows up constantly.

7.11 Where this goes next

- **Chapter 8 (single-variable calculus):** Part I continues with calculus — derivatives as local linear maps, the one-dimensional version of the “bowl” idea.
- **Chapter 10 (optimization):** Hessians and second-order conditions — §7.10’s sketch becomes the full minimum/maximum/saddle test for functions of many variables.
- **Chapter 24 (PCA):** the SVD returns as the engine of principal component analysis: the right singular vectors of the centered data matrix are the principal components, and σ_i^2 the explained variances.

Problem set

1. Find the SVD of $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$: compute $A^T A$, its eigenpairs, the singular values, V , then $U = AV\Sigma^{-1}$, and verify $A = U\Sigma V^T$.
2. Find the SVD of $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. (Hint: $A^T A$ is trivial here.)
3. For the U in §7.4: verify $U^T U = I$ explicitly and compute $\det(U)$. What does the sign of the determinant tell you geometrically?
4. For the rectangular example in §7.5, verify $V^T V = I$ for the three columns $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$.
5. A matrix has singular values 5, 2, 0. What is its rank? Explain in one line using §7.6.
6. Write the best rank-1 approximation $A_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T$ of the §7.4 matrix $A = \begin{bmatrix} \sqrt{2} & 1 \\ 0 & \sqrt{2} \end{bmatrix}$ as an explicit 2×2 matrix (entries may involve $\sqrt{2}$), and compare it entry-wise with A .
7. Prove $A^T A$ is always positive *semi*-definite, for any real A . Conclude its eigenvalues are ≥ 0 (this is the fact §7.3 started from).
8. Test $A = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$ for positive definiteness two ways: (a) the 2×2 test $a > 0$, $\det(A) > 0$; (b) by finding both eigenvalues.

9. Show $B = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ is not positive definite: find its eigenvalues, and find one nonzero vector $\mathbf{x}$ with $\mathbf{x}^T B \mathbf{x} < 0$.
10. Complete the square for $f(x, y) = x^2 + 4xy + 5y^2$ to show it is positive definite. Write down its coefficient matrix and check the 2×2 test on it.
11. Let A be symmetric and positive definite. Using the eigenvalue test and the fact that $\det(A)$ is the product of the eigenvalues (§6.4), show $\det(A) > 0$.

Chapter 7 — Solutions

1. $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$. $A^T A = \begin{bmatrix} 4 & 0 \\ 0 & 9 \end{bmatrix}$ — already diagonal, eigenvalues 9 and 4 (trace = 13, determinant = 36). Ordered: $\lambda_1 = 9$, $\lambda_2 = 4$; singular values $\sigma_1 = 3$, $\sigma_2 = 2$. Eigenvectors: for $\lambda_1 = 9$, $(0, 1)^T$; for $\lambda_2 = 4$, $(1, 0)^T$ — already orthonormal. $V = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, $\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$.

$$\mathbf{u}_1 = \frac{1}{3} A(0, 1)^T = \frac{1}{3} (0, 3)^T = (0, 1)^T, \quad \mathbf{u}_2 = \frac{1}{2} A(1, 0)^T = \frac{1}{2} (2, 0)^T = (1, 0)^T,$$

$$U = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \text{ Verify: } U\Sigma = \begin{bmatrix} 0 & 2 \\ 3 & 0 \end{bmatrix}, U\Sigma V^T = \begin{bmatrix} 0 & 2 \\ 3 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} = A.$$

2. $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. $A^T A = I$, so $\lambda_1 = \lambda_2 = 1$, $\sigma_1 = \sigma_2 = 1$. Take $V = I$ (the standard basis is already orthonormal), $\Sigma = I$. Then $U = AV\Sigma^{-1} = A$. Check $U^T U = A^T A = I$, and $U\Sigma V^T = AII = A$.
3. $U = \begin{bmatrix} \sqrt{2/3} & 1/\sqrt{3} \\ 1/\sqrt{3} & -\sqrt{2/3} \end{bmatrix}$. $U^T U$: $(1, 1)$: $(\sqrt{2/3})^2 + (1/\sqrt{3})^2 = \frac{2}{3} + \frac{1}{3} = 1$; $(2, 2)$: $(1/\sqrt{3})^2 + (\sqrt{2/3})^2 = 1$; $(1, 2)$: $\sqrt{2/3} \cdot \frac{1}{\sqrt{3}} + \frac{1}{\sqrt{3}} \cdot (-\sqrt{2/3}) = \frac{\sqrt{2}}{3} - \frac{\sqrt{2}}{3} = 0$. So $U^T U = I$. $\det(U) = -\frac{2}{3} - \frac{1}{3} = -1$. Geometrically: an orthogonal matrix with determinant -1 is a reflection (orientation-reversing) rather than a pure rotation — still length- and angle-preserving, so it qualifies as one of the SVD's "turns".
4. Columns: $\mathbf{x}_1 = \frac{1}{\sqrt{6}}(1, 1, 2)^T$, $\mathbf{x}_2 = \frac{1}{\sqrt{2}}(1, -1, 0)^T$, $\mathbf{x}_3 = \frac{1}{\sqrt{3}}(1, 1, -1)^T$. Unit lengths: $\|\mathbf{x}_1\|^2 = (1+1+4)/6 = 1$; $\|\mathbf{x}_2\|^2 = (1+1)/2 = 1$; $\|\mathbf{x}_3\|^2 = (1+1+1)/3 = 1$. Orthogonality: $\mathbf{x}_1^T \mathbf{x}_2 = (1-1+0)/\sqrt{12} = 0$; $\mathbf{x}_1^T \mathbf{x}_3 = (1+1-2)/\sqrt{18} = 0$; $\mathbf{x}_2^T \mathbf{x}_3 = (1-1+0)/\sqrt{6} = 0$. The (i, j) entry of $V^T V$ is $\mathbf{x}_i^T \mathbf{x}_j$, so $V^T V = I$.
5. Rank = 2. By §7.6, $\text{rank}(A)$ equals the number of *nonzero* singular values — $\sigma_3 = 0$ contributes a zero outer product to the sum, adding no new direction.
6. $A_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T = 2 \begin{bmatrix} \sqrt{2/3} \\ 1/\sqrt{3} \end{bmatrix} \frac{1}{\sqrt{3}} [1 \quad \sqrt{2}] = 2 \begin{bmatrix} \sqrt{2/3} & 2/3 \\ 1/3 & \sqrt{2/3} \end{bmatrix} = \begin{bmatrix} 2\sqrt{2}/3 & 4/3 \\ 2/3 & 2\sqrt{2}/3 \end{bmatrix}$. Numerically $A_1 \approx \begin{bmatrix} 0.943 & 1.333 \\ 0.667 & 0.943 \end{bmatrix}$ vs $A \approx \begin{bmatrix} 1.414 & 1 \\ 0 & 1.414 \end{bmatrix}$. Note $\text{rank}(A_1) = 1$: both columns are multiples of $\mathbf{u}_1$.
7. For any $\mathbf{x}$: $\mathbf{x}^T A^T A \mathbf{x} = (A\mathbf{x})^T (A\mathbf{x}) = \|A\mathbf{x}\|^2 \geq 0$. So $A^T A$ is positive semi-definite (the inequality is not strict, since $A\mathbf{x} = \mathbf{0}$ is possible for $\mathbf{x} \neq \mathbf{0}$). If $A^T A \mathbf{x} = \lambda \mathbf{x}$ with $\mathbf{x} \neq \mathbf{0}$, then $\lambda = \mathbf{x}^T A^T A \mathbf{x} / \|\mathbf{x}\|^2 = \|A\mathbf{x}\|^2 / \|\mathbf{x}\|^2 \geq 0$ — eigenvalues of $A^T A$ are never negative.
8. (a) $a = 4 > 0$; $\det(A) = 12 - 4 = 8 > 0$. Both 2×2 conditions hold $\rightarrow$ positive definite. (b) $\det(A - \lambda I) = (4 - \lambda)(3 - \lambda) - 4 = \lambda^2 - 7\lambda + 8 = 0$, so $\lambda = \frac{7 \pm \sqrt{49 - 32}}{2} = \frac{7 \pm \sqrt{17}}{2} \approx 5.56, 1.44$. Both $> 0 \rightarrow$ positive definite (agrees with (a)).
9. $\det(B - \lambda I) = (1 - \lambda)^2 - 4 = \lambda^2 - 2\lambda - 3 = 0 \rightarrow \lambda = 3, -1$. Mixed signs $\rightarrow$ not positive definite. Explicitly, for $\mathbf{x} = (1, -1)^T$: $\mathbf{x}^T B \mathbf{x} = [1 \quad -1] \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = [1 \quad -1] \begin{bmatrix} -1 \\ 1 \end{bmatrix} = -2 < 0$. (Equivalently, $\det(B) = -3 < 0$ fails the 2×2 test.)

10. $x^2 + 4xy + 5y^2 = (x^2 + 4xy + 4y^2) + y^2 = (x + 2y)^2 + y^2$. This is a sum of squares, zero only when $x + 2y = 0$ and $y = 0$, i.e. only at $(0, 0)$ — hence positive definite. Coefficient matrix: $A = \begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix}$ ($f = \mathbf{x}^T A \mathbf{x} = x^2 + 2 \cdot 2xy + 5y^2$). $a = 1 > 0$, $\det(A) = 5 - 4 = 1 > 0$.
11. By the eigenvalue test (§7.9), all eigenvalues $\lambda_1, \dots, \lambda_n$ of A are > 0 . By §6.4, $\det(A) = \lambda_1 \lambda_2 \cdots \lambda_n$ — a product of positive numbers, hence > 0 .

8. Single-variable calculus: continuity, differentiability, derivatives

Chapter 7 ended with a one-dimensional sketch: derivatives as local linear maps, and the “bowl” idea that decides minima. This chapter builds that sketch into the full one-variable machinery — limits, continuity, the derivative, and how to *find* maxima and minima. Everything in machine learning is, at bottom, a function you want to minimize (the loss) or a rate at which something changes (the loss with respect to a parameter). This chapter is the instrument panel for reading those functions.

The single-variable story has three acts:

- i) **Limits** — what it means for $f(x)$ to “approach” a value.
- ii) **Continuity** — no jumps: the limit equals the value.
- iii) **Differentiability** — how fast f changes, and where it bottoms out.

8.1 Limits: the “approach” idea

Intuitively, we write

$$\lim_{x \rightarrow a} f(x) = L$$

to mean: as x gets closer and closer to a (from either side), the values $f(x)$ get closer and closer to L . The key question this answers: *what would f equal at a if it behaved nicely there?* — regardless of what $f(a)$ actually is, or whether $f(a)$ is even defined.

Note: The limit as $x \rightarrow a$ has nothing to do with the value (or existence) of $f(a)$. A limit only looks at x near a , never at a .

8.1.1 The ε - δ definition, stated carefully

“Gets closer and closer” is too vague to prove anything with. The formal definition pins down *how close* with two tolerances: ε (how close the outputs must be to L) and δ (how close the inputs must be to a).

Def. $\lim_{x \rightarrow a} f(x) = L$ means: **for every** $\varepsilon > 0$ **there exists** a $\delta > 0$ such that, for every x in the domain with $0 < |x - a| < \delta$,

$$|f(x) - L| < \varepsilon.$$

Read it with the quantifier shorthand from §1.11:

$$\forall \varepsilon > 0, \exists \delta > 0 \text{ such that } (0 < |x - a| < \delta \Rightarrow |f(x) - L| < \varepsilon).$$

The order matters: *first* someone hands you any tolerance ε on the output ($\forall \varepsilon > 0$); *then* you must produce a window δ around a ($\exists \delta > 0$) that forces every output inside it to satisfy the tolerance. The δ is allowed to depend on ε — smaller tolerance, tighter window. The $0 <$ part says $x = a$ itself is excluded: the limit does not care about $f(a)$.

Basically, ... Pick any bullseye size ε around L on the y -axis. The definition promises you can find a (possibly tiny) window δ around a on the x -axis so that the whole graph inside that window — except possibly the point at a itself — lands inside the bullseye.

Remark on the sources. The course book builds limits from *sequences*: $\lim_{x \rightarrow a} f(x) = L$ iff $f(x_n) \rightarrow L$ for every sequence $x_n \rightarrow a$. That sequential definition and the ε - δ one above are equivalent — they certify the same limits. We use ε - δ here because it is the standard working form (and it is exactly the language the MLF lectures use when they say “as x approaches a ”).

eg 1 — proving a limit with ε - δ (full steps). Claim: $\lim_{x \rightarrow 1} x^2 = 1$.

Let $\varepsilon > 0$ be given. We need $\delta > 0$ with $0 < |x - 1| < \delta \Rightarrow |x^2 - 1| < \varepsilon$.

- i) Factor the output distance: $|x^2 - 1| = |x - 1| |x + 1|$.
- ii) Tame the $|x + 1|$ factor: first demand $\delta \leq 1$, so $|x - 1| < 1$ gives $0 < x < 2$, hence $|x + 1| = x + 1 < 3$.
- iii) Then $|x^2 - 1| = |x - 1| |x + 1| < 3|x - 1|$. To make this $< \varepsilon$, it suffices to have $|x - 1| < \varepsilon/3$.
- iv) Choose $\delta = \min(1, \varepsilon/3) > 0$. Then $0 < |x - 1| < \delta$ gives $|x^2 - 1| < 3\delta \leq 3(\varepsilon/3) = \varepsilon$.

Basically, ... An ε - δ proof is a recipe: “you give me the output tolerance, I hand you back the input window.” Step ii) is the standard trick — first lock x into a small fixed neighborhood so the extra factor can’t misbehave, then shrink δ further to hit the tolerance.

8.1.2 One-sided limits

x can approach a from the left ($x < a$) or the right ($x > a$). If the two sides disagree, the two-sided limit does not exist.

Def. $\lim_{x \rightarrow a^-} f(x) = L_1$ (left limit) and $\lim_{x \rightarrow a^+} f(x) = L_2$ (right limit) are defined the same way, except x is restricted to $x < a$ (resp. $x > a$). The two-sided limit exists **iff** both one-sided limits exist *and are equal* — and then it equals their common value.

eg 2 — a limit that does not exist: the floor function. Let $f(x) = \lfloor x \rfloor$ and $a = -1$. From the left, x is just below -1 (e.g. -1.1), so $\lfloor x \rfloor = -2$: $\lim_{x \rightarrow -1^-} \lfloor x \rfloor = -2$. From the right, x is just above -1 (e.g. -0.9), so $\lfloor x \rfloor = -1$: $\lim_{x \rightarrow -1^+} \lfloor x \rfloor = -1$. The sides disagree, so $\lim_{x \rightarrow -1} \lfloor x \rfloor$ does not exist. Verdict: *no limit*.

eg 3 — limits at infinity. $\lim_{x \rightarrow \infty} \frac{1}{x} = 0$: as x grows larger and larger, $\frac{1}{x}$ hugs 0 (same for $x \rightarrow -\infty$). The idea is identical — only the “approach” is “toward ∞ ” instead of “toward a ”. Verdict: *limit 0*.

eg 4 — oscillation kills the limit. $f(x) = \sin \frac{1}{x}$ on $\mathbb{R} \setminus \{0\}$: as $x \rightarrow 0$, the function swings between -1 and 1 faster and faster, never settling. Along $x_n = \frac{1}{\pi/2 + 2\pi n} \rightarrow 0$ we get $f(x_n) = 1$ always; along $y_n = \frac{1}{3\pi/2 + 2\pi n} \rightarrow 0$ we get $f(y_n) = -1$ always. Verdict: *no limit at 0* — approaching does not mean settling.

8.2 Continuity: no jumps

Def. f is **continuous at a** (in its domain) if

$$\lim_{x \rightarrow a} f(x) = f(a),$$

i.e. (i) both one-sided limits exist, and (ii) they equal each other *and* the actual value $f(a)$. f is continuous on its domain if it is continuous at every point of it.

Three things must hold: $f(a)$ is defined; the limit exists; the limit equals the value. In words: **continuity means the limit can be found by just plugging in** — the graph has no jump, hole, or break at a .

Basically, ... A continuous function is one you can draw without lifting the pen. Discontinuous means the pen has to jump.

eg 5 — $|x|$ is continuous at 0. From either side, $|x| \rightarrow 0$ as $x \rightarrow 0$, and $f(0) = 0$. Left limit = right limit = value. Verdict: *continuous at 0* (in fact everywhere).

eg 6 — a jump: discontinuous. $f(x) = x + 1$ for $-4 \leq x < 2$, $f(x) = x^2 - 4$ for $2 \leq x \leq 3$, at $a = 2$. Left limit: $x + 1 \rightarrow 3$ as $x \rightarrow 2^-$. Right limit: $x^2 - 4 \rightarrow 0$ as $x \rightarrow 2^+$. $3 \neq 0$, so condition (ii) fails. Verdict: *not continuous at $x = 2$* — a genuine jump.

eg 7 — $\lfloor x \rfloor$ fails at every integer. We saw in eg 2 that the one-sided limits at -1 disagree; the same happens at every integer n . Verdict: *discontinuous at every integer, continuous everywhere else*.

8.2.1 Continuity is preserved by the usual operations

Theorem. If f and g are continuous at a , then $f \pm g$, $f \cdot g$ are continuous at a ; if additionally $g(a) \neq 0$, then f/g is continuous at a . And composition preserves continuity: if g is continuous at a and f is continuous at $g(a)$, then $f(g(x))$ is continuous at a .

eg 8 — chaining the theorems (full steps). Is $f(x) = \frac{\sin(x^2)}{x^2 - 1}$ continuous at $x = 0$?

- i) x^2 is continuous at 0; $\sin$ is continuous at $0 = 0^2$; so $\sin(x^2)$ is continuous at 0 (composition).
- ii) x^2 and 1 are continuous at 0, so $x^2 - 1$ is continuous at 0 (difference).
- iii) $x^2 - 1$ equals $-1 \neq 0$ at $x = 0$, so the quotient is continuous at 0. Verdict: *continuous at 0* — no ε - δ needed, just the algebra of continuity.

Basically, ... Anything you can build out of continuous pieces with $+$, $-$, $\times$, $\div$ (no division by zero), and nesting stays continuous. Polynomials, $\sin$, $\cos$, e^x , $\log$ — all continuous wherever they are defined.

8.2.2 The intermediate value theorem (one paragraph)

If f is continuous on a closed interval $[a, b]$ and L is any number between $f(a)$ and $f(b)$, then there is some $c \in (a, b)$ with $f(c) = L$. This is the “no lifting the pen” idea made precise: a continuous graph cannot jump over a value on its way from one endpoint to the other. (The course book invokes it only as a hint — e.g. to argue that $x^3 + 5$ attains every real value — but the statement is standard.) **eg.** $f(x) = x^3 + x - 5$ is a continuous polynomial with $f(1) = -3 < 0 < 5 = f(2)$, so some $c \in (1, 2)$ satisfies $f(c) = 0$: a root is *guaranteed* even though we never solved for it.

8.3 The derivative: how fast is it changing?

Average rate of change is easy; *instantaneous* rate of change is what calculus is for. The course book opens with a truck: Jalandhar to Trichy, 2900 km in 72 hours, so the *average* speed is $\frac{2900}{72} \approx 40.28$ km/h. But the driver was fined for speeding near Nagpur — on one 260 km stretch covered in 3 hours his average was $\frac{260}{3} \approx 86.67$ km/h, over the limit. Average speed over a long trip says nothing about the speedometer *at one instant*. To get the instantaneous speed, shrink the time window Δt toward 0:

$$\text{instantaneous speed} = \lim_{\Delta t \rightarrow 0} \frac{\text{distance in time } \Delta t}{\Delta t}.$$

That limit is the model for the derivative.

8.3.1 Definition: a limit of difference quotients

For f defined on an interval around a , the increment of f when we step from a to $a + h$ is $f(a + h) - f(a)$, and the *average* rate of change over that step is the difference quotient

$$\frac{f(a + h) - f(a)}{h}.$$

Def. f is **differentiable at a** if the limit

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a + h) - f(a)}{h}$$

exists. The number $f'(a)$ is the **derivative** of f at a — the instantaneous rate of change of f at a .

Basically, ... The derivative is what the average rate of change *settles down to* as you shrink the measurement window to a point. Speedometer reading = derivative of the odometer.

8.3.2 The geometric twin: slope of the tangent line

The same limit has a picture. The secant line through $(a, f(a))$ and $(a + h, f(a + h))$ has slope exactly the difference quotient. As $h \rightarrow 0$, the second point slides into the first, and the secant *settles* onto the **tangent line** — the line that “just touches” the graph at $(a, f(a))$ and points in the graph’s instantaneous direction.

Ch 8: the derivative as a limit of secant slopes

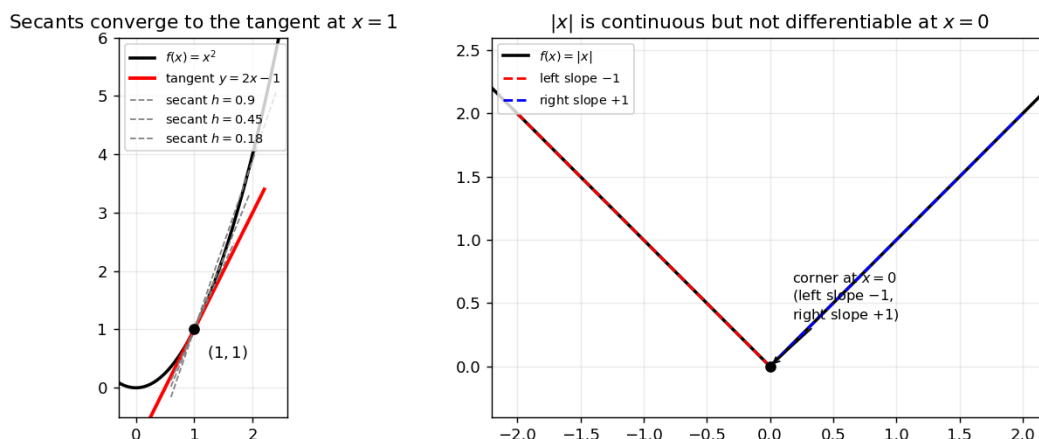


Figure 10: Left: secants of $f(x)=x^2$ through $(1,1)$ for shrinking h converging to the tangent $y=2x-1$. Right: $f(x)=|x|$ has a corner at $x=0$, with left slope -1 and right slope $+1$.

If $f'(a)$ exists, the tangent exists, is unique, and its equation is

$$y - f(a) = f'(a)(x - a) \quad \text{i.e.} \quad y = f(a) + f'(a)(x - a).$$

Conversely, a non-vertical tangent at a is differentiability at a . So: **derivative = limit of secant slopes = slope of the tangent line**. Two names for one idea — analytic and geometric.

eg 9 — from the definition (full steps): $f(x) = x^2$.

$$f'(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} = \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} = \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} = \lim_{h \rightarrow 0} (2x + h) = 2x.$$

The h cancels, then $h \rightarrow 0$ kills the leftover. At $x = 1$: $f'(1) = 2$ — the tangent slope in the figure.

eg 10 — from the definition (full steps): $f(x) = \sin x$.

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{\sin(x+h) - \sin x}{h} \\ &= \lim_{h \rightarrow 0} \frac{\sin x \cos h + \cos x \sin h - \sin x}{h} \\ &= \lim_{h \rightarrow 0} \left(\sin x \cdot \frac{\cos h - 1}{h} + \cos x \cdot \frac{\sin h}{h} \right) \\ &= \sin x \cdot 0 + \cos x \cdot 1 = \cos x, \end{aligned}$$

using the two standard limits $\lim_{h \rightarrow 0} \frac{\sin h}{h} = 1$ and $\lim_{h \rightarrow 0} \frac{\cos h - 1}{h} = 0$.

eg 11 — a tangent line (full steps). Tangent to $f(x) = \cos x$ at $x = \frac{\pi}{3}$: $f(\frac{\pi}{3}) = \cos \frac{\pi}{3} = \frac{1}{2}$; $f'(x) = -\sin x$, so $f'(\frac{\pi}{3}) = -\sin \frac{\pi}{3} = -\frac{\sqrt{3}}{2}$. The tangent is

$$y = -\frac{\sqrt{3}}{2} \left(x - \frac{\pi}{3} \right) + \frac{1}{2}.$$

Recipe, always: evaluate f , evaluate f' , plug into $y = f(a) + f'(a)(x - a)$.

8.4 Differentiable $\Rightarrow$ continuous (but not conversely)

Theorem. If f is differentiable at a , then f is continuous at a .

Proof (brief). Differentiability gives $L = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$. Then

$$\lim_{h \rightarrow 0} (f(a+h) - f(a)) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h} \cdot h = L \cdot 0 = 0,$$

so $\lim_{h \rightarrow 0} f(a+h) = f(a)$: both one-sided limits equal the value.

Basically, ... If the graph is smooth enough to have a well-defined slope at a , it certainly cannot jump there. Smoothness is *stronger* than no-jumps.

The converse is false, and the counterexample is famous:

eg 12 — $|x|$ at 0: continuous, but NOT differentiable (full reasoning). Continuity at 0 was eg 5. For differentiability,

$$\frac{f(0+h) - f(0)}{h} = \frac{|h|}{h}.$$

Now $|h| = h$ for $h \geq 0$ and $|h| = -h$ for $h < 0$, so the one-sided limits of the quotient are

$$\lim_{h \rightarrow 0^+} \frac{|h|}{h} = \lim_{h \rightarrow 0^+} \frac{h}{h} = 1, \quad \lim_{h \rightarrow 0^-} \frac{|h|}{h} = \lim_{h \rightarrow 0^-} \frac{-h}{h} = -1.$$

They disagree, so the two-sided limit does not exist: $f'(0)$ does not exist. Geometrically, the graph has a *corner* at 0 — the left “slope” is -1 , the right “slope” is $+1$, and no single tangent line fits (right panel of the figure). Verdict: *continuous at 0, not differentiable at 0*.

Two more one-line verdicts from the course book: (i) $f(x) = x^{1/3}$ at 0: the difference quotient is $h^{-2/3}$, which blows up as $h \rightarrow 0$ — the tangent would be *vertical*, so no (finite) derivative. Verdict: *not differentiable at 0*. (ii) $f(x) = \lfloor x \rfloor$ at any integer: not continuous there, hence by the theorem cannot be differentiable. Verdict: *not differentiable at integers*.

Note (for ML): differentiability is the ticket that lets gradients exist. The $|x|$ corner is exactly the shape of the ReLU activation at 0 — a kink where the derivative is ambiguous, and practical ML just picks a one-sided value there. Corners need care; smooth functions do not.

8.5 The derivative rules: a toolbox

Computing every derivative from the limit definition would be miserable. These rules reduce differentiation to algebra.

Power rule. $(x^n)' = nx^{n-1}$ (for the usual n). From eg 9: $(x^2)' = 2x$. Also $(x)' = 1$, $(c)' = 0$ for constant c .

Linearity. If f, g are differentiable at a : $(f \pm g)'(a) = f'(a) \pm g'(a)$, and $(cf)'(a) = c f'(a)$ for constant c .

eg 13 — a polynomial (full steps). $p(x) = 3x^3 - 2x^2 + x - 5$:

$$p'(x) = 3 \cdot 3x^2 - 2 \cdot 2x + 1 \cdot 1 - 0 = 9x^2 - 4x + 1.$$

Term by term, power rule on each, constants die.

Product rule. $(fg)'(a) = f'(a)g(a) + f(a)g'(a)$. Mnemonic: differentiate the first, leave the second; leave the first, differentiate the second; add.

eg 14 — product (full steps). $h(x) = x^7 \sin x$. Take $f(x) = x^7$, $g(x) = \sin x$; then $f'(x) = 7x^6$, $g'(x) = \cos x$, and

$$h'(x) = f'(x)g(x) + f(x)g'(x) = 7x^6 \sin x + x^7 \cos x.$$

Quotient rule. $\left(\frac{f}{g}\right)'(a) = \frac{f'(a)g(a) - f(a)g'(a)}{g(a)^2}$, provided $g(a) \neq 0$.

eg 15 — quotient (full steps). $h(x) = \tan x = \frac{\sin x}{\cos x}$. With $f = \sin x$, $g = \cos x$ ($f' = \cos x$, $g' = -\sin x$):

$$h'(x) = \frac{\cos x \cdot \cos x - \sin x \cdot (-\sin x)}{\cos^2 x} = \frac{\cos^2 x + \sin^2 x}{\cos^2 x} = \frac{1}{\cos^2 x} = \sec^2 x.$$

Chain rule. If f and g are differentiable, so is the composition, and

$$(f(g(x)))' = f'(g(x)) \cdot g'(x).$$

Outer derivative *evaluated at the inner function*, times the inner derivative. The course book states this without proof — it is the workhorse rule, so it gets extra care below.

eg 16 — chain rule (full steps). $h(x) = \tan(2x)$: outer $f(u) = \tan u$, inner $g(x) = 2x$. Then $f'(u) = \sec^2 u$, $g'(x) = 2$, so

$$h'(x) = f'(g(x)) g'(x) = \sec^2(2x) \cdot 2 = 2 \sec^2(2x).$$

eg 17 — chain rule again (full steps). $h(x) = \sin(x^2)$: outer $f(u) = \sin u$, inner $g(x) = x^2$. Then $f'(u) = \cos u$, $g'(x) = 2x$, so

$$h'(x) = \cos(x^2) \cdot 2x = 2x \cos(x^2).$$

Peel from the outside in: derivative of the outer layer, with the inner layer left untouched inside it, times the derivative of the inner layer.

Basically, ... The chain rule is for onions. $h(x) = \sin(x^2)$ is “sine of a square”. Differentiate the sine (getting cosine), keep the square intact inside it, then multiply by the derivative of the square. Every nested function differentiates layer by layer, outside $\rightarrow$ in.

Note — why this rule matters for ML. A neural network is a long chain of compositions: weights feed activations feed more weights feed the loss. The derivative of the *loss with respect to an early weight* is a chain rule applied dozens of times — that repeated peeling is exactly **backpropagation** (Chapter 42). Master the chain rule here and backprop is bookkeeping, not magic.

A short table of standard derivatives (verify each yourself from the definition or the rules):

$f(x)$	$f'(x)$
x^n	nx^{n-1}
$\sin(ax)$	$a \cos(ax)$
$\cos(ax)$	$-a \sin(ax)$
e^{ax}	ae^{ax}
$\ln(ax)$	$\frac{1}{x}$
$\log_b(ax)$	$\frac{1}{x \ln b}$

8.6 Linear approximation: replace the curve by its tangent

Start from the derivative at a point x^* :

$$f'(x^*) = \lim_{x \rightarrow x^*} \frac{f(x) - f(x^*)}{x - x^*}.$$

When x is *close to* x^* , the limit is nearly attained, so

$$f'(x^*) \approx \frac{f(x) - f(x^*)}{x - x^*} \quad (x \text{ near } x^*).$$

Rearranging,

$$f(x) \approx f(x^*) + f'(x^*)(x - x^*) \quad (x \text{ near } x^*).$$

The right-hand side is a *linear* function of x (slope $f'(x^*)$, a constant). This is the **linear approximation** — the course book’s “best linear approximation”, and the MLF lectures’ “key expression in first-order calculus”. Its graph is exactly the tangent line: the tangent is the *geometric* object, the linear approximation the *functional* one.

eg 18 — linear approximations (full steps). $f(x) = x^3$ at $a = 1$: $f(1) = 1$, $f'(x) = 3x^2$, $f'(1) = 3$. So

$$L(x) = 1 + 3(x - 1) = 3x - 2,$$

and $x^3 \approx 3x - 2$ for x near 1. Check: at $x = 1.1$, $L = 1.3$ vs $1.1^3 = 1.331$ — close.

eg 19 — a flat one. $f(x) = \sec x$ at $a = 0$: $f(0) = 1$, $f'(x) = \sec x \tan x$, $f'(0) = 0$. So $L(x) = 1 + 0 \cdot x = 1$: near 0, $\sec x \approx 1$. (The tangent is horizontal — consistent with §8.7’s story.)

The famous small- x approximations all come from this one formula, expanded at 0:

- i) $\sin x \approx x$ near 0 (since $\sin 0 = 0$, $(\sin)' = \cos$, $\cos 0 = 1$).
- ii) $e^x \approx 1 + x$ near 0 (since $e^0 = 1$, $(e^x)' = e^x$, value 1 at 0).
- iii) $\ln(1 + x) \approx x$ near 0 (since $\ln 1 = 0$, derivative $\frac{1}{1+x}$, value 1 at 0).
- iv) $(1 + x)^r \approx 1 + rx$ near 0 (since value 1 at 0, derivative $r(1 + x)^{r-1}$, value r at 0).

eg 20 — using the approximation (full steps). Which is closest to 0.99^7 : 0.95, 0.93, 0.91, 0.90? Write $0.99^7 = (1 + x)^7$ with $x = -0.01$, close enough to 0 for the linear approximation: $(1 + x)^7 \approx 1 + 7x = 1 + 7(-0.01) = 0.93$. Answer: 0.93. (This is simple interest standing in for compound interest — valid only because the rate is small.)

On the error. The approximation is *exact* at $x = x^*$ and degrades as x moves away — the tangent hugs the curve near the touch point and drifts off farther out. Everything above is only trustworthy “around” the expansion point; the formula itself never warns you when you have wandered too far. That is the honest limitation of every first-order method in ML, including the one in Chapter 10.

Basically, ... Zoom in close enough on any smooth curve and it looks straight. The linear approximation is that straight line, written down: value at the point, plus slope times how far you walked.

8.7 Finding maxima and minima

8.7.1 Increasing, decreasing, and turning points

On an interval, f is **increasing** if $x_1 < x_2 \Rightarrow f(x_1) < f(x_2)$, and **decreasing** if $x_1 < x_2 \Rightarrow f(x_1) > f(x_2)$. A point where an increase ends and a decrease begins (or vice versa) is a **turning point**: the first kind is a **local maximum**, the second a **local minimum**.

The derivative reads monotonicity directly: where $f'(x) > 0$ the function climbs (f increasing), where $f'(x) < 0$ it falls (f decreasing). Just as f' checks the monotonicity of f , the second derivative f'' checks the monotonicity of f' .

At a turning point the graph levels out — the tangent, if it exists, is horizontal:

Theorem. If a is a turning point and $f'(a)$ exists, then $f'(a) = 0$.

Idea of proof. Left of a local max, the difference quotient $\frac{f(a)-f(a-h)}{h} \geq 0$; right of it, $\frac{f(a+h)-f(a)}{h} \leq 0$. Both tend to $f'(a)$, which is therefore both ≥ 0 and ≤ 0 : hence 0.

8.7.2 Critical points and the second derivative test

Def. a is a **critical point** of f if f is not differentiable at a or $f'(a) = 0$. Every turning point is a critical point — so the hunt for maxima/minima starts by solving $f'(x) = 0$ (and listing points of non-differentiability).

But not every critical point turns: a critical point that is neither a max nor a min is a **saddle point** (the course book's example: $f(x) = (x^2 - 4x + 3.8)(x + 2)x^3$ has $f'(0) = 0$ at $x = 0$ with no turn there).

Second derivative test. Let f be twice differentiable at c with $f'(c) = 0$: - $f''(c) < 0 \Rightarrow$ local maximum at c (f' falling through 0: climb then fall). - $f''(c) > 0 \Rightarrow$ local minimum at c (f' rising through 0: fall then climb). - $f''(c) = 0 \Rightarrow$ test inconclusive (could be max, min, or saddle).

The **first derivative test** says the same thing via sign changes of f' : $+$ $\rightarrow$ $-$ across c means local max; $-$ $\rightarrow$ $+$ means local min; no sign change means saddle.

eg 21 — max/min of a cubic (full steps). $f(x) = x^3 - 12x$.

- i) $f'(x) = 3x^2 - 12$. Set $= 0$: $3x^2 = 12$, $x^2 = 4$, so critical points $x = 2$ and $x = -2$. (f is a polynomial, differentiable everywhere, so these are all.)
- ii) $f''(x) = 6x$. At $x = 2$: $f''(2) = 12 > 0 \Rightarrow$ **local minimum**, value $f(2) = 8 - 24 = -16$. At $x = -2$: $f''(-2) = -12 < 0 \Rightarrow$ **local maximum**, value $f(-2) = -8 + 24 = 16$.
- iii) Sanity via f' signs: for $x < -2$, $f' > 0$ (climbing); between -2 and 2 , $f' < 0$ (falling); after 2 , $f' > 0$ (climbing again). Climb-fall-climb: max at -2 , min at 2 . Agrees.

8.7.3 Closed intervals and global extrema

On a closed interval $[a, b]$, the endpoints can be extrema too (there is no “other side” to turn from). The **global** (absolute) maximum/minimum of a continuous f on a closed, bounded interval always exists. To find it: list all critical points *inside* the interval plus the two boundary points, evaluate f at each, and take the largest/smallest values.

eg 22 — endpoints matter (full steps). $f(x) = x^2$ on $[-1, 1]$: $f'(x) = 2x = 0$ gives critical point $x = 0$, with $f''(0) = 2 > 0$: local minimum, $f(0) = 0$. Boundary values: $f(-1) = 1$, $f(1) = 1$ — both are local (indeed global) maxima on this interval. The interior critical point is the min; the max lives at the boundary.

eg 23 — global max/min (full steps). $f(x) = x^3 - 12x$ on $[-3, 3]$. Critical points inside: ± 2 (from eg 21). Evaluate everywhere relevant:

$$f(-3) = -27 + 36 = 9, \quad f(-2) = 16, \quad f(2) = -16, \quad f(3) = 27 - 36 = -9.$$

Global maximum 16 at $x = -2$; global minimum -16 at $x = 2$.

A word on concavity. $f'' > 0$ means the graph curves upward like a bowl ($\cup$); $f'' < 0$ means it curves downward ($\cap$). A point where the curvature flips sign is an **inflection point** — the graph changes from bowl-up to bowl-down there (it may or may not be a saddle in the critical-point sense). That is the whole story at the level this chapter needs; the bowl picture returns with the Hessian in Chapter 10.

Basically, ... To find the peaks and valleys: (1) solve $f'(x) = 0$ — those are your suspects; (2) check each suspect with f'' (or the sign change of f'); (3) on a closed interval, also check the endpoints. Suspects, verdicts, boundaries — that is the complete procedure.

8.8 Where this goes next

- **Chapter 9 (multivariable calculus):** everything here gains more variables. The derivative becomes the *gradient* (a vector of partial derivatives), the tangent line becomes the *tangent plane*, and the linear approximation becomes the first-order approximation of a surface.
- **Chapter 10 (optimization):** the linear approximation iterated — step a little along the tangent, re-approximate, repeat — is *gradient descent*. The second-derivative test grows into the Hessian test for minima, maxima, and saddles in many dimensions.
- **Chapter 42 (backpropagation):** the chain rule of §8.5, applied layer by layer through a computation graph. Every gradient in deep learning is §8.5 on an industrial scale.

Problem set

1. Prove with the ε - δ definition that $\lim_{x \rightarrow 2} (2x+1) = 5$. (State your δ in terms of ε and verify the implication.)
2. Let $f(x) = 1$ for $x > 0$, $f(0) = 0$, $f(x) = -1$ for $x < 0$. Find $\lim_{x \rightarrow 0^+} f(x)$ and $\lim_{x \rightarrow 0^-} f(x)$. Does $\lim_{x \rightarrow 0} f(x)$ exist? Is f continuous at 0?
3. Evaluate $\lim_{x \rightarrow \infty} \frac{3x^2 + 1}{2x^2 - x}$.
4. Let $f(x) = \frac{x^2 - 9}{x - 3}$ for $x \neq 3$, and $f(3) = 6$. Is f continuous at $x = 3$? Justify with one-sided limits (they coincide here) and the value.
5. Using only the continuity theorems of §8.2.1 (no ε - δ), explain why $g(x) = e^{x^2} \cos x$ is continuous at every real x .
6. From the limit definition of the derivative, compute $f'(x)$ for $f(x) = \frac{1}{x}$ ($x \neq 0$). Show every algebraic step.
7. Differentiate: (a) $x^2 e^x$; (b) $\frac{x}{x^2 + 1}$. Name the rule used in each part.
8. Use the chain rule to find $h'(x)$ for $h(x) = (2x^3 - 1)^4$. Identify the outer and inner functions.
9. Find the equation of the tangent line to $f(x) = x^3 - 2x$ at $x = 2$.
10. Use $(1+x)^r \approx 1+rx$ to approximate $\sqrt{1.02}$. Compute the true value to 4 decimal places and give the absolute error of the approximation.
11. Find the critical points of $f(x) = x^3 + x^2 - x + 5$ and classify each as a local maximum, local minimum, or neither, using the second derivative test.
12. Find the global maximum and global minimum of $f(x) = x^3 - 3x$ on the closed interval $[-2, 2]$.

Chapter 8 — Solutions

Solution 1

Claim: $\lim_{x \rightarrow 2} (2x+1) = 5$.

Let $\varepsilon > 0$ be given. We need $\delta > 0$ such that $0 < |x - 2| < \delta \Rightarrow |(2x+1) - 5| < \varepsilon$.

- i) Simplify the output distance: $|(2x+1) - 5| = |2x - 4| = 2|x - 2|$.
- ii) We want $2|x - 2| < \varepsilon$, i.e. $|x - 2| < \varepsilon/2$.
- iii) Choose $\delta = \varepsilon/2 > 0$. Then $0 < |x - 2| < \delta$ gives $|(2x+1) - 5| = 2|x - 2| < 2\delta = \varepsilon$.

Basically, ... The output distance is just twice the input distance, so the window is half the tolerance.

Solution 2

This is the sign function.

- i) For $x > 0$ (however close to 0), $f(x) = 1$: $\lim_{x \rightarrow 0^+} f(x) = 1$.
- ii) For $x < 0$ (however close to 0), $f(x) = -1$: $\lim_{x \rightarrow 0^-} f(x) = -1$.
- iii) $1 \neq -1$: the one-sided limits disagree, so $\lim_{x \rightarrow 0} f(x)$ does **not** exist.

- iv) Continuity at 0 requires the limit to equal $f(0) = 0$ — it fails already at existence. Verdict: *not continuous at 0* (a jump discontinuity).

Solution 3

Divide numerator and denominator by the highest power, x^2 :

$$\lim_{x \rightarrow \infty} \frac{3x^2 + 1}{2x^2 - x} = \lim_{x \rightarrow \infty} \frac{3 + \frac{1}{x^2}}{2 - \frac{1}{x}}.$$

As $x \rightarrow \infty$, $\frac{1}{x^2} \rightarrow 0$ and $\frac{1}{x} \rightarrow 0$ (eg 3 of the chapter). The denominator tends to $2 \neq 0$, so the quotient rule for limits applies:

$$= \frac{3 + 0}{2 - 0} = \frac{3}{2}.$$

Basically, ... At infinity, only the leading terms survive: $\frac{3x^2}{2x^2} = \frac{3}{2}$.

Solution 4

For $x \neq 3$:

$$\frac{x^2 - 9}{x - 3} = \frac{(x - 3)(x + 3)}{x - 3} = x + 3.$$

So $\lim_{x \rightarrow 3^-} f(x) = \lim_{x \rightarrow 3^+} f(x) = 3 + 3 = 6$ — the hole at $x = 3$ does not affect the limit. Since $f(3) = 6$ is defined to equal that limit, all three continuity conditions hold. Verdict: *continuous at $x = 3$* (the value was chosen precisely to “fill the hole”).

Solution 5

- i) x^2 is a polynomial, hence continuous at every real x ; e^u is continuous at every real u , in particular at $u = x^2$. By the composition part of the continuity theorems, e^{x^2} is continuous at every x .
- ii) $\cos x$ is continuous at every real x .
- iii) By the product part of the theorems, $e^{x^2} \cdot \cos x$ is continuous at every real x . No ε - δ argument needed — the theorems do the work.

Solution 6

$$f'(x) = \lim_{h \rightarrow 0} \frac{\frac{1}{x+h} - \frac{1}{x}}{h}.$$

- i) Combine the numerator: $\frac{1}{x+h} - \frac{1}{x} = \frac{x - (x+h)}{x(x+h)} = \frac{-h}{x(x+h)}$.
- ii) Divide by h : $\frac{-h}{x(x+h) \cdot h} = \frac{-1}{x(x+h)}$ (valid for $h \neq 0$).
- iii) Take the limit: $\lim_{h \rightarrow 0} \frac{-1}{x(x+h)} = \frac{-1}{x \cdot x} = -\frac{1}{x^2}$.

So $\left(\frac{1}{x}\right)' = -\frac{1}{x^2}$ for $x \neq 0$ — consistent with the power rule for $n = -1$.

Solution 7

- (a) $x^2 e^x$ — **product rule**. With $f = x^2$, $g = e^x$ ($f' = 2x$, $g' = e^x$):

$$\frac{d}{dx}(x^2 e^x) = 2x \cdot e^x + x^2 \cdot e^x = x e^x (x + 2).$$

- (b) $\frac{x}{x^2 + 1}$ — **quotient rule**. With $f = x$, $g = x^2 + 1$ ($f' = 1$, $g' = 2x$; note $g(x) \geq 1 \neq 0$ always):

$$\frac{d}{dx} \left(\frac{x}{x^2 + 1} \right) = \frac{1 \cdot (x^2 + 1) - x \cdot 2x}{(x^2 + 1)^2} = \frac{1 - x^2}{(x^2 + 1)^2}.$$

Solution 8

$h(x) = (2x^3 - 1)^4$: outer $f(u) = u^4$, inner $g(x) = 2x^3 - 1$. $f'(u) = 4u^3$, $g'(x) = 6x^2$. Chain rule:

$$h'(x) = f'(g(x)) \cdot g'(x) = 4(2x^3 - 1)^3 \cdot 6x^2 = 24x^2(2x^3 - 1)^3.$$

Basically, ... Bring down the 4, keep the inside untouched, then multiply by the derivative of the inside.

Solution 9

$f(x) = x^3 - 2x$ at $a = 2$.

- i) $f(2) = 8 - 4 = 4$ — the touch point is $(2, 4)$.
- ii) $f'(x) = 3x^2 - 2$, so $f'(2) = 12 - 2 = 10$ — the slope.
- iii) Tangent: $y = f(2) + f'(2)(x - 2) = 4 + 10(x - 2) = 10x - 16$.

Solution 10

$\sqrt{1.02} = (1 + 0.02)^{1/2}$: take $r = \frac{1}{2}$, $x = 0.02$ (close to 0, so the linear approximation applies):

$$(1 + x)^r \approx 1 + rx = 1 + \frac{1}{2}(0.02) = 1.01.$$

True value: $\sqrt{1.02} \approx 1.00995049 \dots$, i.e. 1.0100 to 4 decimal places. Absolute error: $|1.01 - 1.00995049 \dots| \approx 0.0000495 \approx 5 \times 10^{-5}$ — tiny, because 0.02 is close to the expansion point 0.

Solution 11

$f(x) = x^3 + x^2 - x + 5$.

- i) $f'(x) = 3x^2 + 2x - 1$. Set = 0: $3x^2 + 2x - 1 = 0$. Discriminant $4 + 12 = 16$; roots $x = \frac{-2 \pm 4}{6}$, i.e. $x = -1$ and $x = \frac{1}{3}$. Both are critical points (f is a polynomial, differentiable everywhere).
- ii) $f''(x) = 6x + 2$. At $x = -1$: $f''(-1) = -4 < 0 \Rightarrow$ **local maximum**, value $f(-1) = -1 + 1 + 1 + 5 = 6$.
At $x = \frac{1}{3}$: $f''(\frac{1}{3}) = 4 > 0 \Rightarrow$ **local minimum**, value $f(\frac{1}{3}) = \frac{1}{27} + \frac{1}{9} - \frac{1}{3} + 5 = \frac{130}{27} \approx 4.815$.

Solution 12

$f(x) = x^3 - 3x$ on $[-2, 2]$.

- i) $f'(x) = 3x^2 - 3 = 3(x^2 - 1) = 0$ gives critical points $x = 1$ and $x = -1$, both inside $[-2, 2]$.
- ii) Evaluate f at the critical points and the boundary points:

$$f(-2) = -8 + 6 = -2, \quad f(-1) = -1 + 3 = 2, \quad f(1) = 1 - 3 = -2, \quad f(2) = 8 - 6 = 2.$$

- iii) Largest value 2 (at $x = -1$ and $x = 2$); smallest value -2 (at $x = -2$ and $x = 1$). So: **global maximum 2, global minimum -2** — each attained twice, once interior and once at the boundary.

9. Multivariable calculus: partial derivatives, gradients, Taylor series

Chapter 8 built the one-variable machinery: derivative, tangent line, linear approximation. Machine learning almost never has one variable — the loss depends on thousands or millions of weights. This chapter replays Chapter 8 with many variables: the derivative becomes a *vector* of partial derivatives (the gradient), the tangent line becomes a *tangent plane*, and the linear approximation grows into Taylor’s quadratic approximation.

The chapter has six acts:

- i) **Partial derivatives** — differentiate in one direction, freeze the rest.
- ii) **The gradient** — package all the partials into one vector.
- iii) **Directional derivatives** — rate of change along any direction.
- iv) **Linear approximation and tangent planes** — the tangent line $\rightarrow$ tangent plane upgrade (§8.6’s formula, generalized).
- v) **The multivariable chain rule** — differentiating through a moving point.
- vi) **Taylor series** — first-order (linear) and second-order (quadratic, with the Hessian).

9.1 Partial derivatives: freeze everything but one variable

Take $f(x_1, x_2) = x_1^2 + x_2^2$. Ask: “how fast does f change if I nudge *only* x_1 ?” Fix x_2 at a constant — then f is an ordinary one-variable function of x_1 , and the ordinary derivative applies. That is a **partial derivative**.

Def. Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and $\mathbf{v} \in \mathbb{R}^d$. The **partial derivative** of f with respect to x_i at $\mathbf{v}$ is

$$\frac{\partial f}{\partial x_i}(\mathbf{v}) = \lim_{\alpha \rightarrow 0} \frac{f(\mathbf{v} + \alpha \mathbf{e}_i) - f(\mathbf{v})}{\alpha},$$

where $\mathbf{e}_i$ is the coordinate vector (1 in the i -th slot, 0 elsewhere). Moving along $\mathbf{v} + \alpha \mathbf{e}_i$ changes *only* the i -th coordinate, so the limit is exactly the single-variable derivative of §8.3 in that one direction.

The working rule is simpler than the definition: **hold all other variables fixed (treat them as constants) and differentiate with respect to the chosen variable** using the §8.5 rules. Alternative notation: f_{x_i} or $\frac{\partial f}{\partial x_i}$.

eg 1 — partials of a quadratic (full steps). $f(x_1, x_2) = x_1^2 + x_2^2$.

- i) $\frac{\partial f}{\partial x_1}$: freeze x_2 . Then x_2^2 is a constant, and $\frac{d}{dx_1} x_1^2 = 2x_1$ by the power rule. So $\frac{\partial f}{\partial x_1} = 2x_1$.
- ii) $\frac{\partial f}{\partial x_2}$: freeze x_1 . Then x_1^2 is a constant, and $\frac{d}{dx_2} x_2^2 = 2x_2$. So $\frac{\partial f}{\partial x_2} = 2x_2$.

eg 2 — product + chain in the partials (full steps). $f(x, y) = xe^{xy}$.

- i) $\frac{\partial f}{\partial x}$: freeze y . Product rule on x and e^{xy} : $\frac{d}{dx} [e^{xy}] = ye^{xy}$ (chain rule, y constant). So

$$\frac{\partial f}{\partial x} = 1 \cdot e^{xy} + x \cdot ye^{xy} = e^{xy}(1 + xy).$$

- ii) $\frac{\partial f}{\partial y}$: freeze x . Then x is a constant factor, and $\frac{d}{dy}[e^{xy}] = xe^{xy}$. So

$$\frac{\partial f}{\partial y} = x \cdot xe^{xy} = x^2 e^{xy}.$$

At $(1, 0)$: $\frac{\partial f}{\partial x}(1, 0) = e^0(1 + 0) = 1$, $\frac{\partial f}{\partial y}(1, 0) = 1 \cdot e^0 = 1$.

eg 3 — quick two-variable partials (full steps). $f(x, y) = x^2 - xy$.

- i) $\frac{\partial f}{\partial x}$: freeze y . Derivative of x^2 is $2x$; derivative of xy (with y constant) is y . So $\frac{\partial f}{\partial x} = 2x - y$.
 ii) $\frac{\partial f}{\partial y}$: freeze x . Derivative of x^2 is 0; derivative of $-xy$ (with x constant) is $-x$. So $\frac{\partial f}{\partial y} = -x$.

Basically, ... A partial derivative is a one-variable derivative with blinkers on: pick the variable you care about, pretend every other variable is just a number, and differentiate normally. The ∂ (“curly d”) is there to remind you the others are being held still.

Note: Partial derivatives are *rates along the coordinate axes only*. Along a diagonal direction, x and y move together — that needs the directional derivative (§9.3).

9.2 The gradient: all partials packaged into one vector

At a point $\mathbf{v}$, there are d partial derivatives. Stack them into a column vector and you get the **gradient**:

Def. For $f : \mathbb{R}^d \rightarrow \mathbb{R}$,

$$\nabla f(\mathbf{v}) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{v}) \\ \frac{\partial f}{\partial x_2}(\mathbf{v}) \\ \vdots \\ \frac{\partial f}{\partial x_d}(\mathbf{v}) \end{pmatrix}.$$

∇ is read “nabla” or “del”. (The row-vector version $\frac{\partial f}{\partial \mathbf{x}}$ is the same package written as a row; the gradient is its transpose, hence a column.)

This is the *computational* interpretation of the gradient: it is the vector that collects every partial derivative. The geometric interpretations — steepest ascent, perpendicular to level sets — come in §9.4–9.5.

eg 4 — gradient of a quadratic at a point (full steps). $f(x_1, x_2) = x_1^2 + x_2^2$, at $\mathbf{v} = (6, 2)$. From eg 1: $\frac{\partial f}{\partial x_1} = 2x_1$, $\frac{\partial f}{\partial x_2} = 2x_2$. Evaluate at $(6, 2)$:

$$\nabla f(6, 2) = \begin{pmatrix} 2 \cdot 6 \\ 2 \cdot 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 4 \end{pmatrix}.$$

eg 5 — a linear function has a constant gradient (full steps). $f(\mathbf{x}) = x_1 + 2x_2 + 3x_3$.

- i) $\frac{\partial f}{\partial x_1}$: freeze x_2, x_3 ; only x_1 survives differentiation, giving 1.
 ii) $\frac{\partial f}{\partial x_2} = 2$; $\frac{\partial f}{\partial x_3} = 3$. Hence

$$\nabla f(\mathbf{x}) = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \quad \text{everywhere.}$$

A constant gradient means the function is linear: it climbs at a steady rate everywhere, so its rate of change has nothing new to say at any point.

Basically, ... The gradient is the complete first-order report on a function at one point: “here is how fast f changes in each coordinate direction, all at once.” One vector, d numbers, no information lost.

Note (ML): In training, f is the loss and $\mathbf{x}$ the vector of model weights. ∇f then says *how each weight nudges the loss* — it is the quantity every training step computes. This chapter builds the object; Chapter 10 uses it.

9.3 Directional derivatives: rate of change along any direction

Partials measure change along the axes. What about change along an arbitrary direction $\mathbf{u}$?

Def. The **directional derivative** of f at $\mathbf{v}$ along $\mathbf{u}$ is

$$D_{\mathbf{u}}f(\mathbf{v}) = \lim_{\alpha \rightarrow 0} \frac{f(\mathbf{v} + \alpha\mathbf{u}) - f(\mathbf{v})}{\alpha}.$$

In words: start at $\mathbf{v}$, walk a tiny step α along $\mathbf{u}$, and measure the rate of change of f .

Because α is tiny, $f(\mathbf{v} + \alpha\mathbf{u})$ can be replaced by the linear approximation of §9.6 (valid near $\mathbf{v}$):

$$f(\mathbf{v} + \alpha\mathbf{u}) \approx f(\mathbf{v}) + \nabla f(\mathbf{v})^T(\alpha\mathbf{u}).$$

Plug into the definition: the $f(\mathbf{v})$ cancels, the α cancels, and

$$D_{\mathbf{u}}f(\mathbf{v}) = \nabla f(\mathbf{v})^T \mathbf{u} = \nabla f(\mathbf{v}) \cdot \mathbf{u}.$$

So the directional derivative is just the dot product of the gradient with the direction (§2.5) — a *weighted sum of partials*, weighted by the direction's components. The formula is most meaningful when $\mathbf{u}$ is a **unit vector** ($\|\mathbf{u}\| = 1$); otherwise the “rate” is polluted by the direction's length.

eg 6 — directional derivative, normalizing the direction (full steps). $f(x, y) = x \cos y$, direction $\mathbf{u} = [2, 1]$.

- i) Partials: $\frac{\partial f}{\partial x} = \cos y$ (x differentiated, $\cos y$ constant); $\frac{\partial f}{\partial y} = -x \sin y$ (x constant factor, derivative of $\cos y$ is $-\sin y$). So $\nabla f = (\cos y, -x \sin y)^T$.
- ii) $\mathbf{u}$ is not unit length: $\|\mathbf{u}\| = \sqrt{2^2 + 1^2} = \sqrt{5}$. Unit direction: $\hat{\mathbf{u}} = \left(\frac{2}{\sqrt{5}}, \frac{1}{\sqrt{5}}\right)$.
- iii)

$$D_{\hat{\mathbf{u}}}f(x, y) = \nabla f \cdot \hat{\mathbf{u}} = \frac{2}{\sqrt{5}} \cos y - \frac{1}{\sqrt{5}} x \sin y.$$

eg 7 — directional derivative at a point (full steps). $f(x, y) = x^2 - xy$ at $(2, -3)$, direction $\mathbf{u} = 0.6\mathbf{i} + 0.8\mathbf{j}$.

- i) From eg 3: $\nabla f(x, y) = (2x - y, -x)^T$. At $(2, -3)$: $\nabla f(2, -3) = (2(2) - (-3), -2)^T = (7, -2)^T$.
- ii) Check $\mathbf{u}$ is unit: $0.6^2 + 0.8^2 = 0.36 + 0.64 = 1$. (no normalization needed).
- iii)

$$D_{\mathbf{u}}f(2, -3) = (7, -2) \cdot (0.6, 0.8) = 7(0.6) + (-2)(0.8) = 4.2 - 1.6 = 2.6.$$

Basically, ... Stand at $\mathbf{v}$, point your finger along $\mathbf{u}$, and ask “how fast does the ground rise in that direction?” The answer is the gradient dotted with your finger: each partial contributes, weighted by how much your finger leans along its axis.

9.4 The gradient is the direction of steepest ascent

Among all directions $\mathbf{u}$ (unit length), which makes f increase fastest? That is: maximize $D_{\mathbf{u}}f(\mathbf{v}) = \nabla f(\mathbf{v})^T \mathbf{u}$ over $\|\mathbf{u}\| = 1$.

The tool is the **Cauchy–Schwarz inequality**: for vectors $\mathbf{a}, \mathbf{b}$,

$$-\|\mathbf{a}\| \|\mathbf{b}\| \leq \mathbf{a}^T \mathbf{b} \leq \|\mathbf{a}\| \|\mathbf{b}\|,$$

with the upper bound attained exactly when $\mathbf{a}$ is a *positive* scalar multiple of $\mathbf{b}$. Apply it with $\mathbf{a} = \nabla f(\mathbf{v})$ and $\mathbf{b} = \mathbf{u}$ (unit, so $\|\mathbf{u}\| = 1$):

$$D_{\mathbf{u}}f(\mathbf{v}) = \nabla f(\mathbf{v})^T \mathbf{u} \leq \|\nabla f(\mathbf{v})\|.$$

The maximum rate of increase equals the gradient's length, and it is attained at

$$\mathbf{u} = \frac{\nabla f(\mathbf{v})}{\|\nabla f(\mathbf{v})\|},$$

the unit vector along the gradient. Hence: **the gradient points in the direction of steepest ascent**. Flip the sign: $-\nabla f(\mathbf{v})$ is the direction of steepest descent, and every $\mathbf{u}$ with $\nabla f(\mathbf{v})^T \mathbf{u} < 0$ is a **descent direction** (a direction along which f decreases).

eg 8 — steepest ascent, verified (full steps). $f(x_1, x_2) = x_1^2 + x_2^2$ at $(3, 1)$.

- i) $\nabla f(3, 1) = (6, 2)^T$ (eg 1 with $2x_1, 2x_2$).
- ii) Unit steepest-ascent direction: $\|(6, 2)\| = \sqrt{36 + 4} = \sqrt{40} = 2\sqrt{10}$, so $\mathbf{u}^* = \frac{(6, 2)}{2\sqrt{10}} = \frac{(3, 1)}{\sqrt{10}}$.
- iii) Max rate: $D_{\mathbf{u}^*} f = \nabla f \cdot \mathbf{u}^* = \frac{18+2}{\sqrt{10}} = \frac{20}{\sqrt{10}} = \sqrt{40} \approx 6.32 = \|\nabla f\|$.
- iv) Sanity check against a worse direction: along $\mathbf{u} = (1, 0)$, $D = (6, 2) \cdot (1, 0) = 6 < 6.32$. The gradient direction really is fastest. The steepest-descent direction is $-\mathbf{u}^* = \frac{(-3, -1)}{\sqrt{10}}$.

Basically, ... If f is a hill and you are standing on it, the gradient is the arrow a compass would draw saying “climb this way for the fastest gain in height.” Walk the opposite way to descend fastest. Every direction at right angles to the gradient is locally flat — you neither climb nor fall.

Note (forward pointer): This is the entire engine of Chapter 10: gradient descent is just “repeatedly step a little along $-\nabla f$ ” — the steepest-descent direction re-applied at each new point.

9.5 The gradient is perpendicular to the level sets

Draw the curves where f is constant — the **level sets** (contour lines) $\{\mathbf{x} : f(\mathbf{x}) = c\}$. At any point $\mathbf{v}$, the gradient $\nabla f(\mathbf{v})$ is **perpendicular** to the level set through $\mathbf{v}$.

The easy version to prove: $\nabla f(\mathbf{v})$ is perpendicular to the *linearized* level set $\{\mathbf{x} : L_{\mathbf{v}}(f)(\mathbf{x}) = f(\mathbf{v})\}$, where $L_{\mathbf{v}}(f)$ is the linear approximation of §9.6. That set is

$$\{\mathbf{x} : f(\mathbf{v}) + \nabla f(\mathbf{v})^T(\mathbf{x} - \mathbf{v}) = f(\mathbf{v})\} = \{\mathbf{x} : \nabla f(\mathbf{v})^T \mathbf{x} = \nabla f(\mathbf{v})^T \mathbf{v}\},$$

a hyperplane of the form $\mathbf{w}^T \mathbf{x} = b$ — and a hyperplane’s normal vector is $\mathbf{w}$ itself (from the lines-and-planes geometry: the vector $\mathbf{w}$ pierces $\mathbf{w}^T \mathbf{x} = b$ at 90°). Here $\mathbf{w} = \nabla f(\mathbf{v})$. For the true (curved) level set, the same perpendicularity holds at the point of contact — the right panel of the figure shows it directly.

eg 9 — perpendicularity on a circle (full steps). $f(x_1, x_2) = x_1^2 + x_2^2$ at $\mathbf{v} = (-6, 2)$.

- i) $\nabla f(-6, 2) = (-12, 4)^T$. Level value: $f(-6, 2) = 36 + 4 = 40$, so the level set is the circle $x_1^2 + x_2^2 = 40$.
- ii) The circle’s tangent at $(-6, 2)$ is perpendicular to the radius vector $(-6, 2)$ (a radius always meets its circle at 90°).
- iii) $\nabla f(-6, 2) = (-12, 4) = 2 \cdot (-6, 2)$ is parallel to the radius, hence perpendicular to the tangent — i.e. perpendicular to the level set at that point.

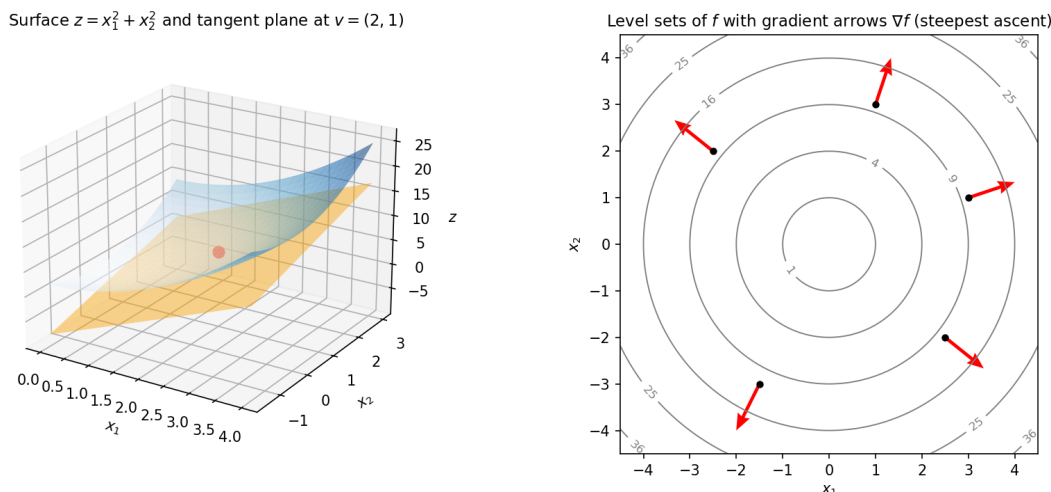


Figure 11: Left: surface $z = x_1^2 + x_2^2$ with its tangent plane at $\mathbf{v} = (2, 1)$, touching at the red dot. Right: contour plot of $x_1^2 + x_2^2$ with red gradient arrows at several points, perpendicular to the level sets and pointing outward (steepest ascent).

Basically, ... Contours are the “same height” lines on a map. The gradient arrow always stabs straight through the contour line at right angles, pointing from lower contours toward higher ones — it is the direction that crosses height-lines fastest, which is exactly why it is the steepest way up.

9.6 Linear (first-order) approximation and the tangent plane

Recall §8.6: near x^* ,

$$f(x) \approx f(x^*) + f'(x^*)(x - x^*).$$

The multivariable upgrade keeps the shape and swaps the parts: the slope $f'(x^*)$ becomes the gradient vector, and multiplication becomes the dot product. For $f : \mathbb{R}^d \rightarrow \mathbb{R}$, near $\mathbf{v}$:

$$f(\mathbf{x}) \approx f(\mathbf{v}) + \nabla f(\mathbf{v})^T (\mathbf{x} - \mathbf{v}) \quad (\mathbf{x} \text{ near } \mathbf{v}).$$

Writing the dot product out:

$$f(\mathbf{x}) \approx f(\mathbf{v}) + \sum_{i=1}^d \frac{\partial f}{\partial x_i}(\mathbf{v}) (x_i - v_i).$$

Why does this hold? Move along x_1 alone: the one-variable linear approximation says the change is $\frac{\partial f}{\partial x_1}(\mathbf{v})(x_1 - v_1)$. Move along x_2 alone: the change is $\frac{\partial f}{\partial x_2}(\mathbf{v})(x_2 - v_2)$. If all coordinates move at once, the total change is the *sum* of the individual changes — and that sum is exactly the formula above. This is the **linear approximation** $L_{\mathbf{v}}(f)$: valid when $\mathbf{x}$ is near $\mathbf{v}$, exact at $\mathbf{x} = \mathbf{v}$.

Tangent line $\rightarrow$ tangent plane. In §8.6 the graph of the linear approximation was the tangent *line*. Here the graph of $f : \mathbb{R}^d \rightarrow \mathbb{R}$ lives in $\mathbb{R}^{d+1}$, and the graph of $L_{\mathbf{v}}(f)$ — a linear function — is a *plane* (hyperplane) that touches the graph of f at the point $(\mathbf{v}, f(\mathbf{v}))$: the **tangent plane**. Same object as in §8.6, one dimension up (left panel of the figure).

eg 10 — linear approximation around (6, 2) (full steps). $f(x_1, x_2) = x_1^2 + x_2^2$, $\mathbf{v} = (6, 2)$.

- i) Value: $f(6, 2) = 36 + 4 = 40$.
- ii) Gradient at $\mathbf{v}$ (eg 4): $\nabla f(6, 2) = (12, 4)^T$.
- iii)

$$L(x_1, x_2) = 40 + (12, 4) \cdot (x_1 - 6, x_2 - 2) = 40 + 12(x_1 - 6) + 4(x_2 - 2).$$

Simplify: $40 + 12x_1 - 72 + 4x_2 - 8 = 12x_1 + 4x_2 - 40$.

- iv) Check: at $(6, 2)$, $L = 72 + 8 - 40 = 40 = f(6, 2)$. (exact at the expansion point).
- v) Numerical check near $\mathbf{v}$: at $(6.1, 2.05)$, $L = 12(6.1) + 4(2.05) - 40 = 73.2 + 8.2 - 40 = 41.4$, while $f = 6.1^2 + 2.05^2 = 37.21 + 4.2025 = 41.4125$. Error ≈ 0.0125 — the plane hugs the surface near the touch point.

eg 11 — linearize and estimate (full steps). $f(x, y) = xe^{xy}$ at $(1, 0)$; approximate $f(1.1, -0.1)$.

- i) $f(1, 0) = 1 \cdot e^0 = 1$.
- ii) Partial derivatives at $(1, 0)$ (eg 2): both equal 1. So $\nabla f(1, 0) = (1, 1)^T$.
- iii)

$$L(x, y) = 1 + 1 \cdot (x - 1) + 1 \cdot (y - 0) = x + y.$$

- iv) $f(1.1, -0.1) \approx L(1.1, -0.1) = 1.1 - 0.1 = 1.0$. True value: $1.1 \cdot e^{-0.11} \approx 1.1 \times 0.89583 \approx 0.98542$. The plane is a good stand-in this close to $(1, 0)$.

Basically, ... Zoom in close enough on any smooth surface and it looks flat. The linear approximation is that flat sheet written down: value at the point, plus (rate in each direction) $\times$ (how far you walked in that direction), added up. The gradient is the complete instruction manual for the sheet.

Note: Like §8.6's warning — the approximation is *exact* at the expansion point and degrades as you wander away. The formula never tells you when you have walked too far; every first-order method in ML (including Chapter 10's) inherits this honest limitation.

9.7 The multivariable chain rule: differentiating through a moving point

§9.3 derived the directional derivative along a *straight line* $\mathbf{v} + \alpha \mathbf{u}$. The same linear-approximation machinery handles a point moving along a *curve*. Let $z = f(x(t), y(t))$: x and y each depend on t , and z depends on both. Then

$$\frac{dz}{dt} = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt} = \nabla f(\mathbf{x}(t)) \cdot \mathbf{x}'(t).$$

Read it as: each moving coordinate contributes its own chain of “how f responds to x ” $\times$ “how x moves with t ” — exactly the “if both move, add the changes” reasoning of §9.6, now with the changes happening continuously. This is the *scalar prototype* of backpropagation's chain rule: a neural network is a long composition of such steps, and Chapter 42 replays this formula layer by layer in vector form. (The vector/matrix version is not derived here.)

eg 12 — chain rule along a curve (full steps). $z = x_1^2 + x_2^2$ with $x_1(t) = 3t$, $x_2(t) = 1 - t$. Find $\frac{dz}{dt}$.

Method A — substitute first, then differentiate.

$$z(t) = (3t)^2 + (1 - t)^2 = 9t^2 + 1 - 2t + t^2 = 10t^2 - 2t + 1,$$

$$\frac{dz}{dt} = 20t - 2.$$

Method B — the chain rule. i) $\frac{\partial z}{\partial x_1} = 2x_1 = 6t$; $\frac{dx_1}{dt} = 3$. ii) $\frac{\partial z}{\partial x_2} = 2x_2 = 2(1 - t)$; $\frac{dx_2}{dt} = -1$. iii)

$$\frac{dz}{dt} = (6t)(3) + (2(1 - t))(-1) = 18t - 2 + 2t = 20t - 2. \quad \checkmark$$

Both routes agree — Method B is the one that scales to long compositions where substitution is impossible.

Basically, ... t pulls two levers (x and y), and each lever pulls z . The total pull on z is (how hard t pulls x) $\times$ (how hard x pulls z), plus the same for y . The gradient is the “how hard each lever pulls z ” part; the curve's velocity is the “how hard t pulls each lever” part.

9.8 Taylor series: adding the quadratic term (the Hessian)

§9.6 is the *first-order* (linear) Taylor approximation. Just as Chapter 8's quadratics improved on its line, we can add a quadratic term in many variables. For $f: \mathbb{R}^d \rightarrow \mathbb{R}$, near $\mathbf{v}$:

$$f(\mathbf{x}) \approx f(\mathbf{v}) + \nabla f(\mathbf{v})^T (\mathbf{x} - \mathbf{v}) + \frac{1}{2} (\mathbf{x} - \mathbf{v})^T \mathbf{H}(\mathbf{v}) (\mathbf{x} - \mathbf{v}),$$

where $\mathbf{H}$ is the **Hessian matrix** — the $d \times d$ matrix of second-order partial derivatives:

$$\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j},$$

i.e. differentiate with respect to x_j first, then x_i . The term $\frac{1}{2}(\mathbf{x} - \mathbf{v})^T \mathbf{H}(\mathbf{v}) (\mathbf{x} - \mathbf{v})$ is a scalar (row $\times$ matrix $\times$ column), exactly analogous to the $\frac{1}{2}f''(a)(x-a)^2$ term of the one-variable Taylor series. For smooth functions the mixed partials agree ($\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x}$), so $\mathbf{H}$ is symmetric — check it in the example below.

eg 13 — Hessian and quadratic approximation (full steps). $f(x, y) = x^2 - xy$, expanded around $\mathbf{v} = (2, -3)$.

- i) First-order pieces (eg 3, eg 7): $f(2, -3) = 4 - (2)(-3) = 10$; $\nabla f(2, -3) = (7, -2)^T$.
- ii) Second partials: $\frac{\partial^2 f}{\partial x^2} = \frac{\partial}{\partial x}(2x - y) = 2$; $\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial}{\partial x}(-x) = -1$; $\frac{\partial^2 f}{\partial y \partial x} = \frac{\partial}{\partial y}(2x - y) = -1$; $\frac{\partial^2 f}{\partial y^2} = \frac{\partial}{\partial y}(-x) = 0$. So

$$\mathbf{H} = \begin{pmatrix} 2 & -1 \\ -1 & 0 \end{pmatrix},$$

symmetric, as expected.

- iii) Set $\mathbf{u} = (x - 2, y + 3)^T$. The quadratic term:

$$\frac{1}{2} \mathbf{u}^T \mathbf{H} \mathbf{u} = \frac{1}{2} (2(x-2)^2 - 2(x-2)(y+3)) = (x-2)^2 - (x-2)(y+3).$$

- iv) Full quadratic approximation:

$$Q(x, y) = 10 + 7(x-2) - 2(y+3) + (x-2)^2 - (x-2)(y+3).$$

- v) Consistency check — expand to verify it reproduces f exactly (it should: f is itself quadratic, and the second-order Taylor of a quadratic is exact):

$$(x-2)^2 - (x-2)(y+3) = x^2 - 4x + 4 - xy - 3x + 2y + 6 = x^2 - xy - 7x + 2y + 10,$$

$$\text{and } 10 + 7(x-2) - 2(y+3) = 7x - 2y - 10. \text{ Adding: } x^2 - xy - 7x + 2y + 10 + 7x - 2y - 10 = x^2 - xy.$$

Basically, ... The linear approximation pretends the world is flat; the quadratic term adds the *curvature*. The Hessian is the “curvature report”: how the slope itself is changing in every pair of directions. First order tells you which way is downhill; second order tells you how the hill bends — which is exactly the extra information second-order optimizers buy (Chapter 10/12).

Note: The quadratic approximation is the best quadratic fit at the point, and it is *exact* for functions that are already quadratic (eg 13's check). For everything else it beats the linear approximation near the point — at the cost of computing d^2 second partials instead of d first ones.

9.9 Critical points: where the gradient vanishes

From §8.7: at a one-variable turning point, $f'(a) = 0$. The multivariable version: if f is minimized (or maximized) at $\mathbf{v}$, then every partial derivative is 0 there, i.e.

$$\nabla f(\mathbf{v}) = \mathbf{0}.$$

A point with $\nabla f(\mathbf{v}) = \mathbf{0}$ (the *vector* zero) is a **critical point** — the first-order necessary condition for optimality. But the converse fails: a vanishing gradient also marks maxima and saddle points, so critical points are *candidates* to be checked, not verdicts. (The Hessian-based check that tells minima from maxima from saddles is Chapter 10's.)

eg 14 — finding a critical point (full steps). $f(x_1, x_2) = x_1^2 + x_2^2$.

- i) $\nabla f(\mathbf{x}) = (2x_1, 2x_2)^T$ (eg 1).
- ii) Set $\nabla f(\mathbf{v}) = \mathbf{0}$: $2v_1 = 0$ and $2v_2 = 0$, so $\mathbf{v} = (0, 0)$.
- iii) This is the unique critical point — and (from the bowl shape) the global minimum, $f(0, 0) = 0$.

Basically, ... A critical point is where the surface goes locally flat in *every* direction at once: every arrow of steepest ascent shrinks to nothing. Peaks, valley bottoms, and mountain passes all look flat if you zoom in close — so “gradient zero” narrows the search to suspects, and the Hessian interrogates them (Chapter 10).

9.10 Where this goes next

- **Chapter 10 (optimization):** the gradient becomes an algorithm. Gradient descent iterates the linear approximation of §9.6 — step a little along $-\nabla f$ (the steepest-descent direction of §9.4), re-approximate, repeat. The Hessian of §9.8 grows into the second-derivative test for minima, maxima, and saddles in many dimensions, and into second-order (Newton-style) methods.
- **Chapter 42 (backpropagation):** the chain rule of §9.7, industrialized. Every gradient in deep learning is §9.7 applied layer by layer through a computation graph — “outer derivative at the inner value, times inner derivative” (§8.5), once per layer.

The through-line of the whole chapter: the gradient is the object that turns a many-variable function into something you can *navigate* — it points uphill, its negative points downhill, its length sets the pace, and its vanishing marks the interesting points.

Problem set

1. Compute the partial derivatives of $f(x, y) = 3x^2y - 5xy^3 + 2x$: find $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$.
2. Let $f(x, y, z) = xy + yz + zx$. Write down ∇f as a column vector.
3. For $f(x_1, x_2) = \sin x_1 \cos x_2$, compute ∇f at $(\frac{\pi}{2}, 0)$. What kind of point is it (see §9.9)?
4. Let $f(x, y) = x^2 - xy$. Compute the directional derivative at $(2, -3)$ along $\mathbf{u} = (4, 3)$ (normalize it first).
5. Find the linear approximation of $f(x, y) = xy + y^2$ at $(1, 2)$, and use it to approximate $f(1.05, 1.9)$.
6. Let $z = e^{xy}$ with $x = t^2$, $y = 1 - t$. Compute $\frac{dz}{dt}$ by the chain rule, and verify by substituting first.
7. For $f(x, y) = 3x + 4y - x^2 - y^2$ at $(1, 1)$: what is the unit direction of steepest ascent, and what is the maximum rate of increase? What is the steepest-descent direction?
8. Compute the Hessian matrix of $f(x, y) = x^3 + y^2 + xy$ at $(1, 2)$. Is it symmetric?
9. Write the second-order Taylor (quadratic) approximation of $f(x, y) = e^{x+y}$ around $(0, 0)$.
10. For $f(x, y) = x^2 + 4y^2$ at $(1, 1)$: (a) compute $\nabla f(1, 1)$; (b) verify it is perpendicular to the level set $f(x, y) = 5$ at that point by dotting it with a tangent direction to the level set.
11. Find all critical points of $f(x, y) = x^2 + y^2 - 2x + 4y$.
12. Show that the directional derivative along the coordinate direction $\mathbf{e}_1 = (1, 0, \dots, 0)$ is exactly the first partial derivative: $D_{\mathbf{e}_1} f(\mathbf{v}) = \frac{\partial f}{\partial x_1}(\mathbf{v})$.

Solutions — Chapter 9: Multivariable calculus

Solution 1

$$f(x, y) = 3x^2y - 5xy^3 + 2x.$$

$\frac{\partial f}{\partial x}$ — freeze y (treat y as a constant), differentiate term by term:

$$\frac{\partial}{\partial x}(3x^2y) = 3y \cdot 2x = 6xy, \quad \frac{\partial}{\partial x}(-5xy^3) = -5y^3 \cdot 1 = -5y^3, \quad \frac{\partial}{\partial x}(2x) = 2.$$

$$\frac{\partial f}{\partial x} = 6xy - 5y^3 + 2.$$

$\frac{\partial f}{\partial y}$ — freeze x :

$$\frac{\partial}{\partial y}(3x^2y) = 3x^2 \cdot 1 = 3x^2, \quad \frac{\partial}{\partial y}(-5xy^3) = -5x \cdot 3y^2 = -15xy^2, \quad \frac{\partial}{\partial y}(2x) = 0.$$

$$\frac{\partial f}{\partial y} = 3x^2 - 15xy^2.$$

Solution 2

$$f(x, y, z) = xy + yz + zx.$$

- i) $\frac{\partial f}{\partial x}$: freeze y, z . $\frac{\partial}{\partial x}(xy) = y$, $\frac{\partial}{\partial x}(yz) = 0$, $\frac{\partial}{\partial x}(zx) = z$. So $\frac{\partial f}{\partial x} = y + z$.
- ii) $\frac{\partial f}{\partial y}$: freeze x, z . $\frac{\partial}{\partial y}(xy) = x$, $\frac{\partial}{\partial y}(yz) = z$, $\frac{\partial}{\partial y}(zx) = 0$. So $\frac{\partial f}{\partial y} = x + z$.
- iii) $\frac{\partial f}{\partial z}$: freeze x, y . $\frac{\partial}{\partial z}(xy) = 0$, $\frac{\partial}{\partial z}(yz) = y$, $\frac{\partial}{\partial z}(zx) = x$. So $\frac{\partial f}{\partial z} = x + y$.

$$\nabla f = \begin{pmatrix} y + z \\ x + z \\ x + y \end{pmatrix}.$$

Solution 3

$$f(x_1, x_2) = \sin x_1 \cos x_2.$$

- i) $\frac{\partial f}{\partial x_1} = \cos x_1 \cos x_2$ (differentiate $\sin x_1$, freeze $\cos x_2$).
- ii) $\frac{\partial f}{\partial x_2} = -\sin x_1 \sin x_2$ ($\sin x_1$ frozen; derivative of $\cos x_2$ is $-\sin x_2$).

At $(\frac{\pi}{2}, 0)$:

$$\frac{\partial f}{\partial x_1} = \cos \frac{\pi}{2} \cdot \cos 0 = 0 \cdot 1 = 0, \quad \frac{\partial f}{\partial x_2} = -\sin \frac{\pi}{2} \cdot \sin 0 = -1 \cdot 0 = 0.$$

$$\nabla f(\frac{\pi}{2}, 0) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

By §9.9 this is a **critical point** — a candidate for a max/min/saddle, not a verdict (the Hessian would have to interrogate it).

Solution 4

$$f(x, y) = x^2 - xy \text{ at } (2, -3), \text{ direction } \mathbf{u} = (4, 3).$$

- i) Normalize: $\|\mathbf{u}\| = \sqrt{16 + 9} = 5$, so $\hat{\mathbf{u}} = (\frac{4}{5}, \frac{3}{5}) = (0.8, 0.6)$.
- ii) $\nabla f(x, y) = (2x - y, -x)^T$ (eg 3). At $(2, -3)$: $\nabla f(2, -3) = (2(2) - (-3), -2)^T = (7, -2)^T$.
- iii)

$$D_{\hat{\mathbf{u}}}f(2, -3) = (7, -2) \cdot (0.8, 0.6) = 7(0.8) + (-2)(0.6) = 5.6 - 1.2 = 4.4.$$

(Compare eg 7: along $(0.6, 0.8)$ the rate was 2.6; along $(0.8, 0.6)$ it is 4.4 — different directions, different rates, same formula.)

Solution 5

$f(x, y) = xy + y^2$ at $(1, 2)$.

- i) $f(1, 2) = 1 \cdot 2 + 2^2 = 2 + 4 = 6$.
- ii) $\frac{\partial f}{\partial x} = y$, so at $(1, 2)$: 2. $\frac{\partial f}{\partial y} = x + 2y$, so at $(1, 2)$: $1 + 4 = 5$.
- iii)

$$L(x, y) = 6 + 2(x - 1) + 5(y - 2) = 6 + 2x - 2 + 5y - 10 = 2x + 5y - 6.$$

- iv) $f(1.05, 1.9) \approx L(1.05, 1.9) = 2(1.05) + 5(1.9) - 6 = 2.1 + 9.5 - 6 = 5.6$. Check: true value = $1.05 \times 1.9 + 1.9^2 = 1.995 + 3.61 = 5.605$. Error 0.005.

Solution 6

$z = e^{xy}$, $x = t^2$, $y = 1 - t$.

Chain rule:

$$\frac{dz}{dt} = \frac{\partial z}{\partial x} \frac{dx}{dt} + \frac{\partial z}{\partial y} \frac{dy}{dt}.$$

$\frac{\partial z}{\partial x} = ye^{xy}$, $\frac{dx}{dt} = 2t$; $\frac{\partial z}{\partial y} = xe^{xy}$, $\frac{dy}{dt} = -1$. So

$$\frac{dz}{dt} = ye^{xy} \cdot 2t + xe^{xy} \cdot (-1) = e^{xy}(2ty - x) = e^{t^2(1-t)}(2t(1-t) - t^2).$$

Verify by substitution: $z(t) = e^{t^2(1-t)} = e^{t^2-t^3}$, so $\frac{dz}{dt} = (2t - 3t^2)e^{t^2-t^3}$. And $2t(1-t) - t^2 = 2t - 2t^2 - t^2 = 2t - 3t^2$. The two routes agree.

Solution 7

$f(x, y) = 3x + 4y - x^2 - y^2$ at $(1, 1)$.

- i) $\nabla f(x, y) = (3 - 2x, 4 - 2y)^T$. At $(1, 1)$: $\nabla f(1, 1) = (1, 2)^T$.
- ii) $\|\nabla f\| = \sqrt{1 + 4} = \sqrt{5}$.
- iii) Unit steepest-ascent direction: $\frac{(1, 2)}{\sqrt{5}}$. Maximum rate of increase: $\|\nabla f\| = \sqrt{5} \approx 2.236$.
- iv) Steepest-descent direction: $-\frac{(1, 2)}{\sqrt{5}} = \frac{(-1, -2)}{\sqrt{5}}$.

Solution 8

$f(x, y) = x^3 + y^2 + xy$.

- i) First partials: $\frac{\partial f}{\partial x} = 3x^2 + y$; $\frac{\partial f}{\partial y} = 2y + x$.
- ii) Second partials: $\frac{\partial^2 f}{\partial x^2} = 6x$; $\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial}{\partial x}(2y + x) = 1$; $\frac{\partial^2 f}{\partial y \partial x} = \frac{\partial}{\partial y}(3x^2 + y) = 1$; $\frac{\partial^2 f}{\partial y^2} = 2$.
- iii) At $(1, 2)$:

$$\mathbf{H}(1, 2) = \begin{pmatrix} 6 & 1 \\ 1 & 2 \end{pmatrix}.$$

Yes — symmetric ($\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x} = 1$).

Solution 9

$f(x, y) = e^{x+y}$ around $(0, 0)$.

- i) $f(0, 0) = e^0 = 1$.
- ii) $\frac{\partial f}{\partial x} = e^{x+y}$, $\frac{\partial f}{\partial y} = e^{x+y}$, so $\nabla f(0, 0) = (1, 1)^T$.

iii) All second partials are e^{x+y} , so $\mathbf{H}(0,0) = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$.

iv) With $\mathbf{u} = (x, y)^T$:

$$\frac{1}{2} \mathbf{u}^T \mathbf{H} \mathbf{u} = \frac{1}{2} (x^2 + 2xy + y^2) = \frac{1}{2} (x + y)^2.$$

v)

$$Q(x, y) = 1 + x + y + \frac{1}{2}(x + y)^2.$$

Sanity check: expand e^{x+y} as a series in $s = x + y$: $e^s = 1 + s + \frac{1}{2}s^2 + \dots$ — matches.

Solution 10

$f(x, y) = x^2 + 4y^2$ at $(1, 1)$.

(a) $\nabla f(x, y) = (2x, 8y)^T$, so $\nabla f(1, 1) = (2, 8)^T$.

(b) Level value: $f(1, 1) = 1 + 4 = 5$, so the level set is $x^2 + 4y^2 = 5$. A tangent direction: implicit differentiation gives $2x + 8y y' = 0$, so $y' = -\frac{x}{4y} = -\frac{1}{4}$ at $(1, 1)$. Take the tangent vector $(4, -1)$ (run 4, rise -1). Then

$$\nabla f(1, 1) \cdot (4, -1) = 2 \cdot 4 + 8 \cdot (-1) = 8 - 8 = 0.$$

Zero dot product — perpendicular. (This is §9.5 in action.)

Solution 11

$f(x, y) = x^2 + y^2 - 2x + 4y$.

$\frac{\partial f}{\partial x} = 2x - 2$; $\frac{\partial f}{\partial y} = 2y + 4$. Set both to zero:

$$2x - 2 = 0 \Rightarrow x = 1, \quad 2y + 4 = 0 \Rightarrow y = -2.$$

The unique critical point is $(1, -2)$. (Completing the square: $f = (x - 1)^2 + (y + 2)^2 - 5$, so this point is in fact the global minimum — but §9.9 alone only certifies it as a candidate; the Hessian verdict waits in Chapter 10.)

Solution 12

$$D_{\mathbf{e}_1} f(\mathbf{v}) = \nabla f(\mathbf{v}) \cdot \mathbf{e}_1 = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{v}) \\ \vdots \\ \frac{\partial f}{\partial x_d}(\mathbf{v}) \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} = \frac{\partial f}{\partial x_1}(\mathbf{v}) \cdot 1 + 0 + \dots + 0.$$

Hence $D_{\mathbf{e}_1} f(\mathbf{v}) = \frac{\partial f}{\partial x_1}(\mathbf{v})$. The directional derivative along a coordinate axis *is* the corresponding partial derivative — the partials are just the directional derivatives in the axis directions.

10. Unconstrained optimization and gradient descent

10.1 The problem: minimize, with no strings attached

An optimization problem has an **objective** f — a function you want to make as small (or as large) as possible — evaluated at a **variable** (in ML parlance, a **parameter**) $\mathbf{x} \in \mathbb{R}^d$, plus possibly some **constraints** $g_j(\mathbf{x}) \leq 0$ or $h_j(\mathbf{x}) = 0$ restricting where $\mathbf{x}$ may live.

Def. The **unconstrained** optimization problem =

$$\min_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x}),$$

with *no* constraints on $\mathbf{x}$: every point of $\mathbb{R}^d$ is allowed. Compare: the “cow and grass” setup from the lectures — minimize $(x_1 - 40)^2 + (x_2 - 40)^2$ — is *constrained* by the rope and the fence. Here, those constraints are dropped: the cow can roam the whole plane.

Note: Maximizing is the same problem in disguise: $\max f(\mathbf{x}) = -\min(-f(\mathbf{x}))$. Flip the sign and you are back to minimization.

Why ML cares. Training a model *is* an unconstrained minimization: the variable $\mathbf{x}$ is the vector of model parameters, the objective f is the loss on the training data, and the “best classifier” of the lectures is the one with the least loss. This chapter builds the algorithm that does the minimizing. (Chapter 11 adds the constraints back; Chapter 43 scales the algorithm up to neural nets.)

Basically, ... You have a hilly landscape f and you may stand anywhere. Find the lowest point. Machine learning is exactly this: the landscape is the loss, the position is the model’s weights, and “training” is walking downhill until you can’t go lower.

10.2 The school method, and where it breaks

The method from school: write $f'(x)$, set it to 0, solve for x .

eg 1 — the easy case (full steps). Minimize $f(x) = (x - 5)^2$ over $x \in \mathbb{R}$.

- i) $f'(x) = 2(x - 5)$. Set = 0: $x = 5$.
- ii) $f''(x) = 2 > 0$, so (Chapter 8, §8.7.2) this is a local minimum.
- iii) Since $f(x) \geq 0$ everywhere and $f(5) = 0$, nothing can beat it: $x = 5$ is the **global minimum**, value 0.

Basically, ... For $(x - 5)^2$ you can almost see the answer: squares are never negative, and 5 makes it zero. The derivative method agrees: $x = 5$.

But this method only works when the equation $f'(x) = 0$ is *solvable*:

eg 2 — where the school method stalls. Minimize $f(x) = 3x^6 + 2x^5 + 3x^3 + 5x^2 + 2$.

- i) $f'(x) = 18x^5 + 10x^4 + 9x^2 + 10x$.

- ii) Set = 0: a degree-5 equation, $18x^5 + 10x^4 + 9x^2 + 10x = 0$.
- iii) There is no general formula for degree 5 — so the “set derivative to zero and solve” recipe dead-ends. Worse: even if you found roots, you’d still have to decide which (if any) is the minimum.

Note: Computers cannot do the clever logical reasoning of eg 1 (“squares are non-negative, so 5 must be the minimum”). They need a *mechanical* procedure: an algorithm that starts somewhere and improves, step by step. That is what we build next.

10.3 The iterative idea: pick a direction, take a step, repeat

Start with any guess $x_0 \in \mathbb{R}$ (arbitrary — say $x_0 = 10$) and improve it:

$$x_{t+1} = x_t + d,$$

where d is a **direction**: a signed step added to the current point. In one dimension there are only two directions — left ($d < 0$) or right ($d > 0$).

For $f(x) = (x-5)^2$: if $x > 5$, the minimum lies to the left, so d must be negative; if $x < 5$, d must be positive. Two conclusions:

- i) The good direction **depends on where you are**: d must be a *function of x* , not a constant.
- ii) The derivative carries the needed sign information: $f'(x) = 2(x-5)$ is positive when $x > 5$ and negative when $x < 5$ — exactly the *opposite* of the direction we want. So choose

$$d = -f'(x).$$

Then $d < 0$ when $x > 5$ and $d > 0$ when $x < 5$: always the correct side.

eg 3 — the first iteration (full steps). $f(x) = (x-5)^2$, $x_0 = 10$.

- i) $f'(10) = 2(10-5) = 10$. Direction $d = -10$ (left — correct, the minimum is at 5).
- ii) $x_1 = x_0 + d = 10 - 10 = 0$. We moved — but did we move *well*?

Basically, ... You are somewhere on a hill and want to go down. The direction d must change depending on where you stand — and the negative of the slope, $-f'(x)$, always points the right way: downhill. This is the seed of the whole chapter.

10.4 The step-size problem: too big oscillates, too small stalls

Continue eg 3 with the rule $x_{t+1} = x_t - f'(x_t)$ (this is $d = -f'(x)$ with step size 1):

- i) $x_0 = 10 \rightarrow x_1 = 10 - 10 = 0$.
- ii) $f'(0) = 2(0-5) = -10$, so $x_2 = 0 - (-10) = 10$.
- iii) $x_3 = 0$, $x_4 = 10$, ... — we bounce between 10 and 0 forever and *never reach* 5.

Note: The *direction* was right every time (left from 10, right from 0). The failure is the *amount* moved: each step overshoots the minimum and lands on the mirror image of where it started.

The fix: scale the direction by a **step size** $\eta_t > 0$ that may differ each round:

$$x_{t+1} = x_t - \eta_t f'(x_t).$$

If $\eta_t = 1$ always, we recover the oscillating rule. The question is how to choose the sequence $\eta_0, \eta_1, \dots$

Attempt 1: halve every time. $\eta_t = \frac{1}{2^t}$: $1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \dots$. Seems safe — steps shrink, no oscillation. But:

eg 4 — shrinking too fast (full steps). Suppose some function always returns direction $d = 1$ (always “move right”), and the minimum is at $x^* = 5$ while we start at $x_0 = 2$.

- i) With $\eta_t = \frac{1}{2^t}$: $x_1 = 2 + 1 = 3$, $x_2 = 3 + \frac{1}{2} = 3.5$, $x_3 = 3.5 + \frac{1}{4} = 3.75$, ...

- ii) Total distance ever travelled: $1 + \frac{1}{2} + \frac{1}{4} + \dots = \frac{1}{1-1/2} = 2$ (geometric series).
- iii) So the iterates approach $2 + 2 = 4$ — and *stop there*, forever short of $x^* = 5$. The direction was right all along; the steps became too small too fast.

Attempt 2: shrink, but slower. $\eta_t = \frac{1}{t+1}$: $1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots$. Steps still shrink (no oscillation), but now the total budget $\sum_{t=0}^{\infty} \frac{1}{t+1} = 1 + \frac{1}{2} + \frac{1}{3} + \dots = \infty$ (the harmonic series diverges). No matter how far x^* is, the cumulative step budget is infinite — you can always get there.

Basically, ... Step size has two enemies pulling opposite ways. Too big: you bounce back and forth across the valley forever (the $10 \rightarrow 0 \rightarrow 10$ dance). Too small: you run out of gas halfway there (stuck at 4, minimum at 5). The winning recipe: steps that shrink — so you stop bouncing — but whose *total* never runs out — so you always have enough road left to reach the minimum.

10.5 The gradient descent algorithm (one dimension)

Collecting the ideas: initialize anywhere, then iterate

$$\boxed{x_{t+1} = x_t - \eta_t f'(x_t)}, \quad \text{e.g. } \eta_t = \frac{1}{t+1}.$$

All it needs is the ability to compute the derivative at a point — something a computer does mechanically. It is a **first-order** method: it uses only first-derivative information, never f'' .

Def. This procedure is **gradient descent**. (“Gradient” is the higher-dimensional name for the derivative — §10.7. The name is explained there; the algorithm is already here.)

eg 5 — step size 1 diverges (full steps). $f(x) = (3x - 9)^2$, $x_0 = 2$, constant $\eta = 1$.

- i) $f'(x) = 2(3x - 9) \cdot 3 = 18x - 54$. (Check: minimum where $18x - 54 = 0$, i.e. $x = 3$, $f(3) = 0$.)
- ii) $x_1 = 2 - 1 \cdot (18 \cdot 2 - 54) = 2 - (-18) = 20$. Overshot: we leapt from 2 past 3 all the way to 20.
- iii) $x_2 = 20 - (18 \cdot 20 - 54) = 20 - 306 = -286$. Worse. f explodes: $9 \rightarrow 2601 \rightarrow 751689 \approx 7.5 \times 10^5$.

eg 6 — a small step size converges (full steps). Same f , same $x_0 = 2$, but $\eta = 0.05$.

- i) $x_1 = 2 - 0.05(18 \cdot 2 - 54) = 2 - 0.05(-18) = 2.9$.
- ii) $x_2 = 2.9 - 0.05(18 \cdot 2.9 - 54) = 2.9 - 0.05(-1.8) = 2.99$.
- iii) $x_3 = 2.99 - 0.05(18 \cdot 2.99 - 54) = 2.99 - 0.05(-0.18) = 2.999$.
- iv) f : $9 \rightarrow 0.09 \rightarrow 0.0009 \rightarrow 0.000009$ — homing in on 3.

Note: Same function, same start, same direction — only η changed, and the outcome flipped from blow-up to convergence. Step size is not a detail; it is half the algorithm. (This is why η , the **learning rate**, becomes a hyperparameter to tune in Chapter 39.)

eg 7 — converging to a local minimum (full steps). $f(x) = x^3 - x^2 - x + 5$, $x_0 = 0.75$, $\eta = 0.25$.

- i) $f'(x) = 3x^2 - 2x - 1$. Critical points: $3x^2 - 2x - 1 = (3x + 1)(x - 1) = 0$, so $x = 1$ and $x = -\frac{1}{3}$. With $f''(x) = 6x - 2$: $f''(1) = 4 > 0$ (local min), $f''(-\frac{1}{3}) = -4 < 0$ (local max).
- ii) $f'(0.75) = 3(0.5625) - 1.5 - 1 = -0.8125$. $x_1 = 0.75 - 0.25(-0.8125) = 0.953125 \approx 0.953$.
- iii) $f'(0.953125) = 3(0.9084473) - 2(0.953125) - 1 \approx -0.1809$. $x_2 = 0.953125 - 0.25(-0.1809) \approx 0.998$.
- iv) The iterates crawl toward $x = 1$ — the *local* minimum nearest the start. The local maximum at $-\frac{1}{3}$ is never visited. Direction right, step size fine, destination local.

Basically, ... Gradient descent is a blindfolded hiker who can only feel the slope under their feet: step downhill a little, feel again, repeat. The step size is the stride length — too long and you leap over the valley and bounce; too short and you fall asleep halfway. With a sensible stride, you reach the bottom of *your* valley.

10.6 Why the negative derivative: the Taylor argument

We pulled $d = -f'(x)$ out of a hat in §10.3 (it happened to work for $(x - 5)^2$). Here is the real justification, from the Taylor series of Chapter 8.

Setup. We are at x and consider stepping to $x + \eta d$: direction d , step size $\eta > 0$ (small). First-order Taylor (§8.6, and the lectures' key expansion):

$$f(x + \eta d) \approx f(x) + \eta d f'(x),$$

dropping the $\eta^2, \eta^3, \dots$ higher-order terms, which are negligible when η is small. The change in f is therefore

$$f(x + \eta d) - f(x) \approx \eta d f'(x).$$

The requirement. We minimize, so we want the new value *smaller*: $f(x + \eta d) - f(x) < 0$, i.e.

$$\eta d f'(x) < 0.$$

Since $\eta > 0$ cannot change the sign, this is equivalent to

$$d \cdot f'(x) < 0 :$$

the direction must point *against* the derivative.

The choice. Take $d = -f'(x)$. Then

$$d \cdot f'(x) = -f'(x) \cdot f'(x) = -f'(x)^2 \leq 0,$$

strictly negative unless $f'(x) = 0$ (in which case the algorithm stops — §10.9). So for small η , the Taylor approximation *guarantees* the step decreases f .

Gradient descent as iterated linear approximation. Chapter 9, §9.6 promised this, and here is the delivery. At x_t , the linear approximation is

$$L_{x_t}(x) = f(x_t) + f'(x_t)(x - x_t).$$

Write the next point as $x = x_t + \eta u$ with $|u| = 1$ (a fixed-size step in a unit direction u):

$$L_{x_t}(x_t + \eta u) = f(x_t) + \eta f'(x_t) u.$$

Minimizing *the linearization* over the unit direction u : the term $f'(x_t) u$ is smallest when u points opposite to $f'(x_t)$, i.e. $u = -f'(x_t)/|f'(x_t)|$ (the one-dimensional Cauchy–Schwarz argument of §9.4). So the linear model's minimizer is a step in the direction $-f'(x_t)$ — and absorbing $|f'(x_t)|$ into the step size gives the update

$$x_{t+1} = x_t - \eta f'(x_t).$$

That is: **gradient descent = minimize the linear approximation at the current point, step there, re-approximate, repeat.** The honest limitation of §8.6/§9.6 carries over: the linearization is only trustworthy *near* x_t — which is exactly why η must be small.

Basically, ... Zoom in on the curve at your current point: it looks like a straight line with slope $f'(x)$. Walk downhill along that line a little. You will land slightly lower (the line is accurate up close). Now zoom in at the new point and do it again. That loop *is* gradient descent.

10.7 The multivariate version: replace the derivative by the gradient

For $f : \mathbb{R}^d \rightarrow \mathbb{R}$, the derivative generalizes to the **gradient** ∇f (Chapter 9, §9.2) — the vector of partial derivatives. Everything carries over with f' swapped for ∇f :

$$\boxed{\mathbf{x}_{t+1} = \mathbf{x}_t - \eta_t \nabla f(\mathbf{x}_t)},$$

a vector minus a scalar times a vector. This is why the algorithm is called **gradient** descent — in one dimension we were secretly doing it all along.

The Taylor justification upgrades too (§9.8’s first-order term):

$$f(\mathbf{x} + \eta \mathbf{d}) \approx f(\mathbf{x}) + \eta \mathbf{d}^T \nabla f(\mathbf{x}),$$

so the decrease condition becomes $\mathbf{d}^T \nabla f(\mathbf{x}) < 0$. Choosing $\mathbf{d} = -\nabla f(\mathbf{x})$ gives

$$\mathbf{d}^T \nabla f(\mathbf{x}) = -\nabla f(\mathbf{x})^T \nabla f(\mathbf{x}) = -\|\nabla f(\mathbf{x})\|^2 \leq 0,$$

strict unless the gradient is $\mathbf{0}$.

Descent directions and the steepest one. Any $\mathbf{d}$ with $\mathbf{d}^T \nabla f(\mathbf{x}) < 0$ is a **descent direction** — a small step along it is guaranteed (by the linear approximation) to reduce f . There are infinitely many: at a point in $\mathbb{R}^2$, they fill the entire half-plane on the far side of the line perpendicular to ∇f (§9.4’s picture: the gradient partitions space into “uphill” and “downhill” halves). Among unit directions, the one minimizing the linearization is $-\nabla f(\mathbf{x})/\|\nabla f(\mathbf{x})\|$ — the **steepest descent** direction (§9.4, Cauchy–Schwarz). So gradient descent is also called the **steepest descent** algorithm: at every step it picks the locally fastest way down.

eg 8 — three kinds of directions (full steps). $f(x_1, x_2) = x_1^2 + x_2^2$ at $(3, 1)$; $\nabla f(3, 1) = (6, 2)^T$.

- i) $\mathbf{d} = (-6, -2)$: $\mathbf{d}^T \nabla f = -36 - 4 = -40 < 0 \rightarrow$ descent (this is $-\nabla f$, steepest descent).
- ii) $\mathbf{d} = (1, 1)$: $\mathbf{d}^T \nabla f = 6 + 2 = 8 > 0 \rightarrow$ ascent, f increases.
- iii) $\mathbf{d} = (-2, 6)$: $\mathbf{d}^T \nabla f = -12 + 12 = 0 \rightarrow$ perpendicular to the gradient: locally flat, neither up nor down.

eg 9 — two-dimensional iterations (full steps). Minimize the “cow” distance $d(x_1, x_2) = (x_1 - 40)^2 + (x_2 - 40)^2$ from $\mathbf{x}_0 = (5, 2)$, with constant $\eta = 0.1$.

- i) $\nabla d(\mathbf{x}) = (2(x_1 - 40), 2(x_2 - 40))^T$. At $(5, 2)$: $(-70, -76)^T$.
- ii) $\mathbf{x}_1 = (5, 2) - 0.1(-70, -76) = (12, 9.6)$. Both coordinates moved *up* — toward 40. (The negative gradient $(70, 76)$ points straight at the grass.)
- iii) $\nabla d(12, 9.6) = (-56, -60.8)^T$. $\mathbf{x}_2 = (12, 9.6) + 0.1(56, 60.8) = (17.6, 15.68)$.
- iv) $\nabla d(17.6, 15.68) = (-44.8, -48.64)^T$. $\mathbf{x}_3 = (17.6, 15.68) + 0.1(44.8, 48.64) = (22.08, 20.544)$.
- v) Check the values: $d(5, 2) = 35^2 + 38^2 = 2669$; $d(12, 9.6) = 28^2 + 30.4^2 = 1708.16$; $d(17.6, 15.68) \approx 1093.22$ — monotone decrease, as promised. Each coordinate’s error $(x_i - 40)$ shrinks by the factor $(1 - 2\eta) = 0.8$ per step: $-35 \rightarrow -28 \rightarrow -22.4 \rightarrow -17.92$.

The picture (right panel): every iterate cuts straight across the contours toward $(40, 40)$.

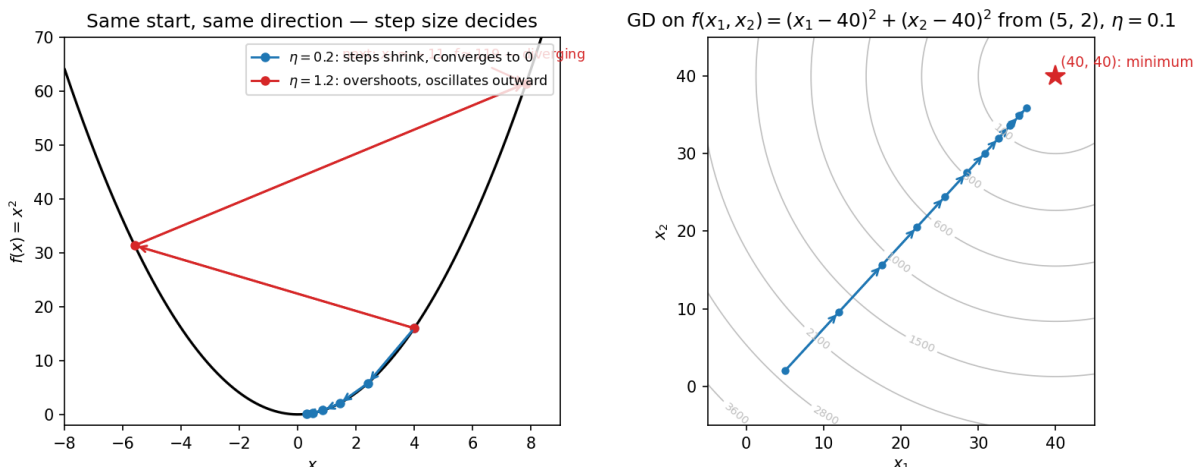


Figure 12: Left: gradient descent on $f(x) = x^2$ from $x_0 = 4$ — with $\eta = 0.2$ the iterates shrink toward 0, with $\eta = 1.2$ they overshoot and diverge outward. Right: contour plot of $(x_1 - 40)^2 + (x_2 - 40)^2$ with the GD iterates from $(5, 2)$ ($\eta = 0.1$) marching toward the minimum at $(40, 40)$.

Basically, ... In many dimensions the slope becomes an arrow — the gradient — pointing the steepest way *up*. Flip it and walk a little: you descend as fast as locally possible. Recompute the arrow at the new spot (it will have turned a bit), flip, walk again. The contour picture shows the path: always crossing the “same height” lines at right angles, heading for the bottom of the bowl.

10.8 Optimality conditions: the gradient finds suspects, the Hessian interrogates them

First-order necessary condition (Chapter 9, §9.9). If f is minimized (or maximized) at $\mathbf{v}$, then

$$\nabla f(\mathbf{v}) = \mathbf{0}.$$

A point with $\nabla f(\mathbf{v}) = \mathbf{0}$ is a **critical point**. Necessary, not sufficient: maxima and saddles also have vanishing gradient.

Second-order conditions. At a critical point $\mathbf{v}$, the linear term of the Taylor expansion vanishes, so what remains is the quadratic term (§9.8):

$$f(\mathbf{v} + \mathbf{u}) \approx f(\mathbf{v}) + \frac{1}{2} \mathbf{u}^T \mathbf{H}(\mathbf{v}) \mathbf{u},$$

where $\mathbf{H}(\mathbf{v})$ is the **Hessian** matrix of second partials at $\mathbf{v}$. The sign of the quadratic form $\mathbf{u}^T \mathbf{H}(\mathbf{v}) \mathbf{u}$ decides the verdict — this is Chapter 7’s bowl machinery (§7.9–§7.10), now applied at $\mathbf{v}$:

- i) $\mathbf{H}(\mathbf{v})$ **positive definite** $\rightarrow$ the quadratic term is > 0 in every direction $\rightarrow$ bowl $\rightarrow$ **local minimum**.
- ii) $\mathbf{H}(\mathbf{v})$ **negative definite** ($-\mathbf{H}(\mathbf{v})$ positive definite) $\rightarrow$ quadratic term < 0 in every direction $\rightarrow$ upside-down bowl $\rightarrow$ **local maximum**.
- iii) $\mathbf{H}(\mathbf{v})$ **indefinite** (positive in some directions, negative in others) $\rightarrow$ up some ways, down others $\rightarrow$ **saddle point**.
- iv) $\mathbf{H}(\mathbf{v})$ merely semi-definite $\rightarrow$ **inconclusive** (flat in some direction; higher-order terms decide — the multivariable twin of $f''(c) = 0$ in §8.7.2).

For 2×2 , the eigenvalue test of §7.9 becomes the quick check: with $\mathbf{H} = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$ — positive definite iff $a > 0$ and $\det \mathbf{H} = ac - b^2 > 0$; negative definite iff $a < 0$ and $\det \mathbf{H} > 0$; indefinite iff $\det \mathbf{H} < 0$.

eg 10 — classifying all critical points of a 2-variable function (full steps). $f(x, y) = x^3 - 3x + y^3 - 3y$.

- i) First-order: $\frac{\partial f}{\partial x} = 3x^2 - 3$, $\frac{\partial f}{\partial y} = 3y^2 - 3$. Set both to 0: $x = \pm 1$, $y = \pm 1$. Four critical points: $(1, 1)$, $(-1, -1)$, $(1, -1)$, $(-1, 1)$.
- ii) Hessian: $\frac{\partial^2 f}{\partial x^2} = 6x$, $\frac{\partial^2 f}{\partial x \partial y} = 0$, $\frac{\partial^2 f}{\partial y^2} = 6y$. So $\mathbf{H}(x, y) = \begin{pmatrix} 6x & 0 \\ 0 & 6y \end{pmatrix}$ (symmetric).
- iii) At $(1, 1)$: $\mathbf{H} = \begin{pmatrix} 6 & 0 \\ 0 & 6 \end{pmatrix}$, $a = 6 > 0$, $\det = 36 > 0 \rightarrow$ positive definite $\rightarrow$ **local minimum**, $f(1, 1) = 1 - 3 + 1 - 3 = -4$.
- iv) At $(-1, -1)$: $\mathbf{H} = \begin{pmatrix} -6 & 0 \\ 0 & -6 \end{pmatrix}$, $a = -6 < 0$, $\det = 36 > 0 \rightarrow$ negative definite $\rightarrow$ **local maximum**, $f(-1, -1) = -1 + 3 - 1 + 3 = 4$.
- v) At $(1, -1)$: $\mathbf{H} = \begin{pmatrix} 6 & 0 \\ 0 & -6 \end{pmatrix}$, $\det = -36 < 0 \rightarrow$ indefinite $\rightarrow$ **saddle**, $f(1, -1) = 1 - 3 - 1 + 3 = 0$.
- vi) At $(-1, 1)$: $\mathbf{H} = \begin{pmatrix} -6 & 0 \\ 0 & 6 \end{pmatrix}$, $\det = -36 < 0 \rightarrow$ indefinite $\rightarrow$ **saddle**, $f(-1, 1) = -1 + 3 + 1 - 3 = 0$.

Basically, ... The gradient going quiet only means “flat here” — it can’t tell a valley floor from a peak or a mountain pass. The Hessian is the follow-up detective: it measures the curvature in every direction. Curved up everywhere = valley (minimum); curved down everywhere = peak (maximum); up some ways, down others = pass (saddle).

10.9 What gradient descent promises — and what it does not

What it promises. With a sensible step-size schedule (e.g. $\eta_t = \frac{1}{t+1}$), the oscillations of §10.4 disappear: one can show the iterates converge, and the Taylor argument of §10.6 gives monotone decrease $f(\mathbf{x}_{t+1}) < f(\mathbf{x}_t)$ for small η_t (the **descent** in gradient *descent*). The algorithm needs only differentiability and a gradient oracle.

Why it stops at local minima. Look at the update: $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta_t \nabla f(\mathbf{x}_t)$. If $\nabla f(\mathbf{x}_t) = \mathbf{0}$, then $\mathbf{x}_{t+1} = \mathbf{x}_t$ — the algorithm *cannot move*. It halts at the first critical point it slides into, which (eg 7) is typically a *local* minimum:

Def. $\hat{\mathbf{x}}$ is a **local minimum** = there exists $\varepsilon > 0$ such that $f(\hat{\mathbf{x}}) \leq f(\mathbf{x})$ for all $\mathbf{x}$ with $\|\mathbf{x} - \hat{\mathbf{x}}\| < \varepsilon$ (lowest in *some* neighbourhood). The **global minimum** $\mathbf{x}^*$ satisfies $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for *all* $\mathbf{x}$.

The honest scope.

- i) Gradient descent finds **stationary points** (critical points), not guaranteed global minima. A wiggly f has many valleys; starting in one valley, you descend to *its* bottom and stop — the deeper valley next door stays invisible.
- ii) It can even converge to a **saddle point** (tutorial caveat): at a saddle the gradient is also $\mathbf{0}$, so the update also freezes there.
- iii) With a bad η it may not converge at all — oscillation (eg 5) or, for some functions and starts, an infinite cycle.
- iv) No general-purpose algorithm can guarantee the global minimum of an arbitrary function — gradient descent settling for local minima is the best one can hope for without extra assumptions on f .

The good news for ML. For the functions ML actually minimizes, local is often good enough: there is a large, important class — the **convex** functions of Chapter 12 — for which *every* local minimum is automatically a global minimum (the $(x - 5)^2$ of §10.2 is the baby example: one valley, so the local minimum *is* the global one). That is why gradient descent, despite its modest guarantees, powers machine learning.

A second-order alternative (one paragraph). Newton’s method uses curvature too: $x_{t+1} = x_t - f'(x_t)/f''(x_t)$ (in d dimensions it needs the *inverse Hessian*). On $f(x) = (3x - 9)^2$ from $x_0 = 2$ it lands on $x = 3$ in a *single* iteration — because f is quadratic, its second-order Taylor model is exact (§9.8’s note). The price, per the tutorial: it needs second derivatives (expensive, and a Hessian inverse per step in higher dimensions), it fails when the Hessian is not invertible, it can cycle or land on a saddle, and each iteration costs far more. Gradient descent stays first-order — cheaper per step, slower to arrive.

Basically, ... Gradient descent is a ball rolling downhill that stops at the bottom of whichever valley it started in — it can't teleport to a deeper valley, and on a mountain pass it can get stuck too. For most ML losses, any valley bottom is fine (and for convex losses there is only one valley). Newton's method is the same ball with a curvature sensor: faster, but heavier, and it can misfire.

Note: “Converges to a local minimum” also quietly assumes the gradient exists everywhere the iterates go — non-differentiable points (kinks like $|x|$) need their own machinery, outside this chapter.

10.10 This is training — and where this goes next

Unconstrained gradient descent is not a warm-up exercise: **it is the training loop**. Every time a model is “trained” in this book, some version of

$$\theta_{t+1} = \theta_t - \eta_t \nabla L(\theta_t)$$

runs — parameters θ , loss L , learning rate η_t . The loss landscape is the hill; the weights are the hiker.

Where this goes next

- **Chapter 11 (constrained optimization):** the cow's rope and fence return. When $-\nabla f$ points outside the allowed region, pure gradient descent breaks — Lagrange multipliers and projected gradient descent fix it.
- **Chapter 12 (convexity):** the functions for which “local minimum = global minimum”, where gradient descent's modest guarantee becomes the full prize.
- **Chapter 39 (hyperparameter tuning):** η — the learning rate — is the hyperparameter you will tune most. §10.4–10.5 showed why: it decides between convergence and chaos.
- **Chapter 43 (optimization for neural nets):** the industrial version — stochastic gradients, momentum, adaptive step sizes (Adam and friends) — gradient descent scaled to millions of parameters.

The through-line: Chapter 9 turned the gradient into a compass; this chapter turned the compass into an algorithm — step a little along $-\nabla f$, re-approximate, repeat — and the Hessian into the judge that classifies where you land.

Problem set

1. Minimize $f(x) = x^2 - 6x + 5$ by the school method: compute f' , solve $f'(x) = 0$, classify with f'' , and argue the critical point is the global minimum.
2. Run two gradient-descent iterations on $f(x) = x^2$ from $x_0 = 4$ with constant $\eta = 0.1$. Verify f decreases at each step.
3. For $f(x) = (x - 5)^2$ from $x_0 = 10$: (a) with constant $\eta = 1$, compute x_1, x_2, x_3 and describe the behaviour; (b) with the schedule $\eta_t = \frac{1}{t+1}$ ($t = 0, 1$), compute x_1, x_2 and describe the behaviour.
4. Let $S_n = 1 + \frac{1}{2} + \dots + \frac{1}{2^{n-1}}$ and $H_n = 1 + \frac{1}{2} + \dots + \frac{1}{n}$. Show $S_n < 2$ for every n (the $\frac{1}{2^t}$ budget is finite), while $H_n \rightarrow \infty$ (hint: $\frac{1}{3} + \frac{1}{4} > \frac{1}{2}$, $\frac{1}{5} + \dots + \frac{1}{8} > \frac{1}{2}$, ...). What does this say about the two step-size schedules of §10.4?
5. For $f(x_1, x_2) = x_1^2 + 4x_2^2$: (a) write down ∇f ; (b) run two GD iterations from $(2, 3)$ with $\eta = 0.2$; (c) check that f decreased at each step.
6. For $f(x, y) = x^2 + y^2$ at the point $(2, -1)$: which of $\mathbf{d}_1 = (1, 1)$, $\mathbf{d}_2 = (-2, 1)$, $\mathbf{d}_3 = (1, 2)$ are descent directions? Interpret each dot product geometrically.
7. Find all critical points and classify each (local min / local max / saddle) using the Hessian: (a) $f(x, y) = x^2 + y^2 - 2x - 4y + 5$; (b) $f(x, y) = -x^2 - y^2 + 4x$; (c) $f(x, y) = x^2 - y^2$.
8. Starting from the first-order Taylor approximation $f(x + \eta d) \approx f(x) + \eta d^T \nabla f(x)$, derive the sign condition that d must satisfy for the step to decrease f . Then verify that $d = -\nabla f(x)$ satisfies it strictly whenever $\nabla f(x) \neq 0$.
9. For $f(x) = (x - a)^2$ with a *constant* step size η : (a) show $x_{t+1} - a = (1 - 2\eta)(x_t - a)$; (b) deduce the exact range of η for which the iterates converge to a , and explain what goes wrong outside it.

10. Take one Newton step $x_1 = x_0 - f'(x_0)/f''(x_0)$ on $f(x) = (3x - 9)^2$ from $x_0 = 2$. Where do you land? Explain why it is exact in one step (hint: §9.8's note on quadratics).
11. Suppose gradient descent has “converged”, i.e. $\mathbf{x}_{t+1} = \mathbf{x}_t$ to machine precision. (a) What does the update rule imply about $\nabla f(\mathbf{x}_t)$? (b) Why does this mean GD finds stationary points rather than guaranteed minima — and why might such a point be a saddle?

Solutions — Chapter 10: Unconstrained optimization and gradient descent

Solution 1

$$f(x) = x^2 - 6x + 5.$$

- i) $f'(x) = 2x - 6$. Set $= 0$: $2x = 6$, so $x = 3$ — the only critical point (f is a polynomial, differentiable everywhere).
- ii) $f''(x) = 2 > 0$: local minimum at $x = 3$ (§8.7.2).
- iii) Complete the square: $f(x) = x^2 - 6x + 9 - 4 = (x - 3)^2 - 4 \geq -4 = f(3)$ for all x . So $x = 3$ is the **global minimum**, value -4 .

Solution 2

$$f(x) = x^2, f'(x) = 2x, x_0 = 4, \eta = 0.1.$$

- i) $x_1 = x_0 - \eta f'(x_0) = 4 - 0.1 \cdot 8 = 4 - 0.8 = 3.2$.
- ii) $f'(3.2) = 6.4$. $x_2 = 3.2 - 0.1 \cdot 6.4 = 3.2 - 0.64 = 2.56$.
- iii) Values: $f(4) = 16$, $f(3.2) = 10.24$, $f(2.56) = 6.5536$. $16 > 10.24 > 6.5536$: decreasing at each step, homing in on 0.

Solution 3

$$f(x) = (x - 5)^2, f'(x) = 2(x - 5), x_0 = 10.$$

- (a) Constant $\eta = 1$:
 - $x_1 = 10 - 1 \cdot 2(10 - 5) = 10 - 10 = 0$.
 - $x_2 = 0 - 1 \cdot 2(0 - 5) = 0 + 10 = 10$.
 - $x_3 = 10 - 10 = 0$. Behaviour: pure oscillation $10 \rightarrow 0 \rightarrow 10 \rightarrow 0 \rightarrow \dots$ — the step always overshoots the minimum at 5 by the same amount, landing on the mirror image. It never converges.
- (b) Schedule $\eta_t = \frac{1}{t+1}$:
 - $t = 0$: $\eta_0 = 1$. $x_1 = 10 - 1 \cdot 10 = 0$.
 - $t = 1$: $\eta_1 = \frac{1}{2}$. $f'(x_1) = 2(0 - 5) = -10$. $x_2 = 0 - \frac{1}{2}(-10) = 5$. Behaviour: $x_2 = 5$ — the exact minimum, reached in two steps. Shrinking the step size killed the oscillation.

Solution 4

$S_n = 1 + \frac{1}{2} + \dots + \frac{1}{2^{n-1}}$ is geometric with ratio $\frac{1}{2}$:

$$S_n = \frac{1 - (1/2)^n}{1 - 1/2} = 2(1 - 2^{-n}) < 2 \quad \text{for every } n,$$

and $S_n \rightarrow 2$. So a step-size schedule $\eta_t = \frac{1}{t+1}$ has a *finite* total budget: starting anywhere, you can travel at most 2 units — possibly stranding you short of the minimum (§10.4, eg 4).

$H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$: group the terms after the first two:

$$\underbrace{\frac{1}{3} + \frac{1}{4}}_{>2 \cdot \frac{1}{4} = \frac{1}{2}} + \underbrace{\frac{1}{5} + \cdots + \frac{1}{8}}_{>4 \cdot \frac{1}{8} = \frac{1}{2}} + \underbrace{\frac{1}{9} + \cdots + \frac{1}{16}}_{>8 \cdot \frac{1}{16} = \frac{1}{2}} + \cdots$$

Each block exceeds $\frac{1}{2}$, and there are infinitely many blocks, so $H_n \rightarrow \infty$. The schedule $\eta_t = \frac{1}{t+1}$ still shrinks (no oscillation) but its *total* budget is infinite — you can reach a minimum no matter how far away it is. This is the two-enemies compromise of §10.4.

Solution 5

$$f(x_1, x_2) = x_1^2 + 4x_2^2.$$

(a) $\nabla f = \begin{pmatrix} 2x_1 \\ 8x_2 \end{pmatrix}.$

(b) From $(2, 3)$, $\eta = 0.2$:

- $\nabla f(2, 3) = (4, 24)^T$. $\mathbf{x}_1 = (2, 3) - 0.2(4, 24) = (2 - 0.8, 3 - 4.8) = (1.2, -1.8)$.
- $\nabla f(1.2, -1.8) = (2.4, -14.4)^T$. $\mathbf{x}_2 = (1.2, -1.8) - 0.2(2.4, -14.4) = (1.2 - 0.48, -1.8 + 2.88) = (0.72, 1.08)$.

(c) $f(2, 3) = 4 + 4 \cdot 9 = 40$. $f(1.2, -1.8) = 1.44 + 4 \cdot 3.24 = 1.44 + 12.96 = 14.4$. $f(0.72, 1.08) = 0.5184 + 4 \cdot 1.1664 = 0.5184 + 4.6656 = 5.184$. $40 > 14.4 > 5.184$: decreased both times. (The true minimum is $(0, 0)$ with value 0.)

Solution 6

$$f(x, y) = x^2 + y^2, \nabla f = (2x, 2y)^T. \text{ At } (2, -1): \nabla f(2, -1) = (4, -2)^T.$$

- $\mathbf{d}_1^T \nabla f = (1, 1) \cdot (4, -2) = 4 - 2 = 2 > 0$: **not** a descent direction — it points partly uphill (angle $< 90^\circ$ with the gradient).
- $\mathbf{d}_2^T \nabla f = (-2, 1) \cdot (4, -2) = -8 - 2 = -10 < 0$: **descent** direction — it makes an obtuse angle with the gradient, pointing into the downhill half-plane.
- $\mathbf{d}_3^T \nabla f = (1, 2) \cdot (4, -2) = 4 - 4 = 0$: **neither** — perpendicular to the gradient, so a small step is locally flat (it runs along the contour, §9.5).

Solution 7

(a) $f(x, y) = x^2 + y^2 - 2x - 4y + 5$.

- $\nabla f = (2x - 2, 2y - 4)^T = \mathbf{0}$ gives $(x, y) = (1, 2)$ — the only critical point.
- Hessian: $\mathbf{H} = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$, constant.
- $a = 2 > 0$, $\det = 4 > 0 \rightarrow$ positive definite $\rightarrow$ **local minimum**. $f(1, 2) = 1 + 4 - 2 - 8 + 5 = 0$. (Completing the square: $f = (x - 1)^2 + (y - 2)^2 \geq 0$ — it is in fact the global minimum.)

(b) $f(x, y) = -x^2 - y^2 + 4x$.

- $\nabla f = (-2x + 4, -2y)^T = \mathbf{0}$ gives $(x, y) = (2, 0)$.
- $\mathbf{H} = \begin{pmatrix} -2 & 0 \\ 0 & -2 \end{pmatrix}$.
- $a = -2 < 0$, $\det = 4 > 0 \rightarrow$ negative definite $\rightarrow$ **local maximum**. $f(2, 0) = -4 - 0 + 8 = 4$. (Indeed $f = 4 - (x - 2)^2 - y^2 \leq 4$: the global maximum.)

(c) $f(x, y) = x^2 - y^2$.

- $\nabla f = (2x, -2y)^T = \mathbf{0}$ gives $(0, 0)$.

ii) $\mathbf{H} = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}$.

iii) $\det = -4 < 0 \rightarrow$ indefinite $\rightarrow$ **saddle point**. Check: along the x -axis $f = x^2 \geq 0$ (uphill), along the y -axis $f = -y^2 \leq 0$ (downhill) — up some ways, down others.

Solution 8

We want the step to a smaller value: $f(x + \eta d) - f(x) < 0$.

From the Taylor approximation,

$$f(x + \eta d) - f(x) \approx \eta d f'(x),$$

so the requirement becomes $\eta d f'(x) < 0$. Since $\eta > 0$ is positive, it cannot flip the sign, so equivalently:

$$d \cdot f'(x) < 0$$

(the direction must oppose the derivative — §10.6).

Check $d = -f'(x)$: $d \cdot f'(x) = (-f'(x)) \cdot f'(x) = -f'(x)^2$. If $f'(x) \neq 0$, then $f'(x)^2 > 0$ and $-f'(x)^2 < 0$ strictly. So the condition holds strictly: for small enough η , the step $x \rightarrow x + \eta d$ is guaranteed to decrease f . If $f'(x) = 0$, then $d = \mathbf{0}$ and the algorithm does not move (§10.9).

Solution 9

$$f(x) = (x - a)^2, \quad f'(x) = 2(x - a).$$

(a) $x_{t+1} = x_t - \eta \cdot 2(x_t - a)$. Subtract a from both sides:

$$x_{t+1} - a = x_t - a - 2\eta(x_t - a) = (1 - 2\eta)(x_t - a).$$

The error $e_t = x_t - a$ evolves as $e_{t+1} = (1 - 2\eta)e_t$, so $e_t = (1 - 2\eta)^t e_0$.

(b) $|e_t| \rightarrow 0$ iff $|1 - 2\eta| < 1$, i.e. $-1 < 1 - 2\eta < 1$, i.e. $0 < \eta < 1$.

- If $\eta = 1$: $1 - 2\eta = -1$, so $e_t = (-1)^t e_0$ — pure oscillation, the §10.4 example ($10 \rightarrow 0 \rightarrow 10$).
- If $\eta > 1$: $|1 - 2\eta| > 1$ — the error grows geometrically: divergence (eg 5's blow-up).
- If $\eta \leq 0$: the “step” moves uphill or nowhere.

Inside $(0, 1)$ the error shrinks by a constant factor each step — linear (geometric) convergence to a .

Solution 10

$$f(x) = (3x - 9)^2: \quad f'(x) = 18x - 54, \quad f''(x) = 18.$$

$$x_1 = x_0 - \frac{f'(x_0)}{f''(x_0)} = 2 - \frac{18 \cdot 2 - 54}{18} = 2 - \frac{-18}{18} = 2 + 1 = 3.$$

One step lands exactly on $x = 3$, the minimizer ($f(3) = 0$). Why: Newton's step minimizes the *second-order* Taylor model $f(x_t) + f'(x_t)(x - x_t) + \frac{1}{2}f''(x_t)(x - x_t)^2$, and since f is itself quadratic, that model is *exact* (§9.8's note) — minimizing the model is minimizing f .

Solution 11

(a) The update is $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta_t \nabla f(\mathbf{x}_t)$. If $\mathbf{x}_{t+1} = \mathbf{x}_t$, then $\eta_t \nabla f(\mathbf{x}_t) = \mathbf{0}$, and since $\eta_t > 0$:

$$\nabla f(\mathbf{x}_t) = \mathbf{0}.$$

The algorithm has reached a critical (stationary) point.

- (b) “Gradient zero” is the *first-order* condition shared by minima, maxima, and saddles (§10.8): the update cannot distinguish them, because at all of them the step is $\mathbf{0}$. In particular, at a saddle point (e.g. $(0, 0)$ for $f = x^2 - y^2$, where $\nabla f = \mathbf{0}$ but the Hessian is indefinite) gradient descent freezes just as it does at a minimum — the tutorial’s caveat. So GD’s guarantee is “converges to a stationary point (typically a local minimum)”, never “converges to the global minimum”.

11. Constrained optimization: Lagrange multipliers, projected gradient descent

11.1 The problem: minimize, with strings attached

Chapter 10 solved $\min_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x})$ with no restrictions. Now the constraints return — the cow and the grass from the lectures (§10.1):

eg 1 — the cow and the grass (the setup). A cow stands at $\mathbf{x} = (x_1, x_2)$ and wants the grass patch at $(40, 40)$:

$$f(\mathbf{x}) = (x_1 - 40)^2 + (x_2 - 40)^2.$$

The cow is tied by a rope of length R to a peg at the origin — it cannot roam further than R from the peg:

$$g(\mathbf{x}) = x_1^2 + x_2^2 - R^2 \leq 0.$$

The full problem is

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) \quad \text{subject to} \quad g(\mathbf{x}) \leq 0.$$

Def. The **constrained** optimization problem =

$$\min_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x}) \quad \text{subject to} \quad \begin{cases} h_i(\mathbf{x}) = 0, & i = 1, \dots, m, \\ g_j(\mathbf{x}) \leq 0, & j = 1, \dots, p, \end{cases}$$

where the h_i are **equality constraints** and the g_j are **inequality constraints**. The **feasible set** is

$$S = \{\mathbf{x} : h_i(\mathbf{x}) = 0 \text{ all } i, g_j(\mathbf{x}) \leq 0 \text{ all } j\}$$

— the points the cow is allowed to stand on. An inequality $g_j(\mathbf{x}) \leq 0$ is **active** (binding) at $\mathbf{x}$ if $g_j(\mathbf{x}) = 0$ (the rope is taut), and **inactive** if $g_j(\mathbf{x}) < 0$ (there is slack).

- i) If the unconstrained best point lies inside S , the constraint changes nothing: minimize as in Chapter 10.
- ii) If it lies outside S , the optimum sits on the boundary of S — the cow pulls the rope taut.

Basically, ... Same hill as before, but now there is a fence. You still want the lowest point, except you are only allowed to stand inside the fence. If the valley bottom happens to be inside the fence, great — nothing changes. If it is outside, you end up pressed against the fence at the closest reachable spot.

11.2 The optimality check: no descent direction is feasible

Before computing anything, the lectures ask a sharper question: if someone *claims* a point $\mathbf{x}^*$ is optimal, how do you catch them lying?

- i) First check: is $\mathbf{x}^*$ even feasible? If $g(\mathbf{x}^*) > 0$, the claim is bogus.
- ii) Second check: could you improve? A **descent direction** is a direction that lowers f — from the Taylor series (§10.6), any $\mathbf{d}$ with $\mathbf{d}^T \nabla f(\mathbf{x}^*) < 0$ is a descent direction for small enough steps. A **feasible direction** is a direction in which a small step keeps the constraint satisfied.

Key condition. If $\mathbf{x}^*$ is optimal, then *no descent direction is feasible*: if some direction both reduced f and stayed inside S , you could move there and beat $\mathbf{x}^*$ — contradicting optimality.

Now apply this at $\mathbf{x}^*$ with one inequality constraint $g(\mathbf{x}) \leq 0$. The descent directions for f are the half-space $\mathbf{d}^T \nabla f(\mathbf{x}^*) < 0$. What about feasible directions? If g decreases along $\mathbf{d}$ ($\mathbf{d}^T \nabla g(\mathbf{x}^*) < 0$), the step stays feasible — so the half-space $\mathbf{d}^T \nabla g(\mathbf{x}^*) < 0$ is certainly feasible.

So: when do these two half-spaces overlap? If $\nabla f(\mathbf{x}^*)$ and $\nabla g(\mathbf{x}^*)$ point in unrelated directions, the two half-spaces intersect — there *is* a direction that is both descent and feasible, and $\mathbf{x}^*$ cannot be optimal. If they point in the *same* direction, it is even worse: *every* descent direction is feasible. The only configuration with no overlap is when the two gradients point in **opposite** directions:

$$\nabla f(\mathbf{x}^*) = -\lambda \nabla g(\mathbf{x}^*), \quad \lambda > 0$$

for some positive scalar λ . This is a **necessary condition** for optimality (not sufficient — it also holds at maxima, as we will see). The scalar λ is the **Lagrange multiplier**.

Note: $\lambda > 0$ here, not just any scalar — the gradients must be *anti-parallel*. The sign matters for inequalities, and §11.7 explains why.

11.3 The Lagrangian and the first-order conditions (equality constraints)

For an equality constraint $h(\mathbf{x}) = 0$, the same argument runs, with one difference: feasibility to first order means $\mathbf{d}^T \nabla h(\mathbf{x}^*) = 0$ — the feasible directions form a *line*, not a half-space. Both the parallel *and* the anti-parallel configurations avoid overlap with the descent half-space. So λ gets no sign restriction:

$$\nabla f(\mathbf{x}^*) = -\lambda \nabla h(\mathbf{x}^*), \quad h(\mathbf{x}^*) = 0, \quad \lambda \text{ any real number.}$$

Def. For $\min f(\mathbf{x})$ subject to $h(\mathbf{x}) = 0$, the **Lagrangian** is

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda h(\mathbf{x}).$$

Setting its $\mathbf{x}$ -derivative to zero gives $\nabla f(\mathbf{x}) - \lambda \nabla h(\mathbf{x}) = \mathbf{0}$, i.e. $\nabla f(\mathbf{x}^*) = \lambda \nabla h(\mathbf{x}^*)$ — the same condition as above, since λ ranges over all of $\mathbb{R}$ in both forms.

Note (sign convention): The lectures write the condition as $\nabla f(\mathbf{x}^*) = -\lambda \nabla g(\mathbf{x}^*)$. The Lagrangian form $L = f - \lambda h$ gives $\nabla f = \lambda \nabla h$. These are the *same* condition: rename $\lambda \rightarrow -\lambda$ (an unrestricted multiplier absorbs the sign). Other textbooks write $L = f + \lambda h$; only consistency matters, not the sign.

The recipe — the **method of Lagrange multipliers**:

- i) Write $L(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda h(\mathbf{x})$.
- ii) Solve the system $\nabla_{\mathbf{x}} L = \mathbf{0}$ and $h(\mathbf{x}) = 0$ — that is, $\nabla f = \lambda \nabla h$ together with the constraint.
- iii) The solutions are *candidates*: plug each into f and pick the smallest (the conditions are necessary, not sufficient — a candidate can be a maximizer, as eg 3 will show).

Basically, ... At the optimum, the hill's steepest-ascent arrow and the constraint's steepest-ascent arrow must point along the same line (parallel or anti-parallel). Why: if they pointed in different directions, you could slide along the constraint in a direction that also goes downhill on the objective — and then you wouldn't be at the optimum. The multiplier λ just measures the ratio of the two arrows' lengths.

11.4 eg 2 — warm-up: closest point on a line (full steps)

Minimize $f(x, y) = x^2 + y^2$ subject to $x + y = 1$.

- i) Constraint: $h(x, y) = x + y - 1 = 0$. Lagrangian:

$$L(x, y, \lambda) = x^2 + y^2 - \lambda(x + y - 1).$$

- ii) Stationarity: $\frac{\partial L}{\partial x} = 2x - \lambda = 0$, $\frac{\partial L}{\partial y} = 2y - \lambda = 0$ — so $x = y = \lambda/2$.
 iii) Feasibility: $x + y = 1$ gives $\lambda/2 + \lambda/2 = 1$, so $\lambda = 1$, $x = y = \frac{1}{2}$.
 iv) Value: $f(\frac{1}{2}, \frac{1}{2}) = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$. Check: $\nabla f = (1, 1) = 1 \cdot (1, 1) = \lambda \nabla h$.

Only one candidate, and $f \rightarrow \infty$ at infinity along the line, so this is the global minimum: the point on the line $x + y = 1$ closest to the origin.

11.5 eg 3 — the lecture's example: on the unit circle (full steps)

Minimize $f(x_1, x_2) = x_1^2 + 2x_2 + 4x_2^2$ subject to $g(x_1, x_2) = x_1^2 + x_2^2 - 1 = 0$.

- i) Lagrangian: $L(x_1, x_2, \lambda) = x_1^2 + 2x_2 + 4x_2^2 - \lambda(x_1^2 + x_2^2 - 1)$.
 ii) Lagrange equations ($\nabla f = \lambda \nabla g$):

$$\begin{cases} 2x_1 = \lambda \cdot 2x_1, \\ 2 + 8x_2 = \lambda \cdot 2x_2, \end{cases} \quad x_1^2 + x_2^2 = 1.$$

- iii) First equation: $2x_1(1 - \lambda) = 0$ — so either $x_1 = 0$ or $\lambda = 1$.
 iv) **Case** $\lambda = 1$. Second equation: $2 + 8x_2 = 2x_2$, so $6x_2 = -2$, $x_2 = -\frac{1}{3}$. From the circle: $x_1^2 = 1 - \frac{1}{9} = \frac{8}{9}$, $x_1 = \pm \frac{\sqrt{8}}{3}$. Two candidates: $(\frac{\sqrt{8}}{3}, -\frac{1}{3})$ and $(-\frac{\sqrt{8}}{3}, -\frac{1}{3})$.
 v) **Case** $x_1 = 0$. From the circle: $x_2 = \pm 1$. Two candidates: $(0, 1)$ and $(0, -1)$.
 vi) Plug all four into f :

$$\begin{aligned} f(0, 1) &= 0 + 2 + 4 = 6, \\ f(0, -1) &= 0 - 2 + 4 = 2, \\ f(\pm \frac{\sqrt{8}}{3}, -\frac{1}{3}) &= \frac{8}{9} - \frac{2}{3} + \frac{4}{9} = \frac{12}{9} - \frac{6}{9} = \frac{2}{3}. \end{aligned}$$

- vii) The minimum $\frac{2}{3}$ is attained at the two points $(\pm \frac{\sqrt{8}}{3}, -\frac{1}{3})$ — the **minimizers**. $(0, 1)$ with value 6 is the **global maximizer**, and $(0, -1)$ with value 2 is a **local maximizer** along the circle: moving either way round the circle from $(0, -1)$ decreases f toward $\frac{2}{3}$ (check: with $x_1 = \cos \theta$, $x_2 = \sin \theta$, $\frac{d^2 f}{d\theta^2} = -4 < 0$ at $\theta = -\pi/2$).

Note: This is why step iii) of the recipe says *candidates*: the Lagrange equations alone found the maximum and the saddle too. The equations are necessary, not sufficient — the final plug-in decides.

11.6 The geometry: level sets tangent, gradients parallel

Here is the picture behind everything above, and it comes straight from Chapter 9. From §9.5: the gradient is **perpendicular to the level sets**. Now:

- i) The constraint $h(\mathbf{x}) = 0$ is itself a level set — of h . So $\nabla h(\mathbf{x}^*)$ is perpendicular to the constraint curve at $\mathbf{x}^*$.
 ii) At the optimum, the level set of f through $\mathbf{x}^*$ must be **tangent** to the constraint curve: if it crossed the constraint transversally, moving along the constraint would change f — and one direction would decrease it, contradicting optimality.
 iii) Two tangent curves share a tangent line, hence share a perpendicular direction — so $\nabla f(\mathbf{x}^*)$ and $\nabla h(\mathbf{x}^*)$ are parallel. That is exactly $\nabla f = \lambda \nabla h$.

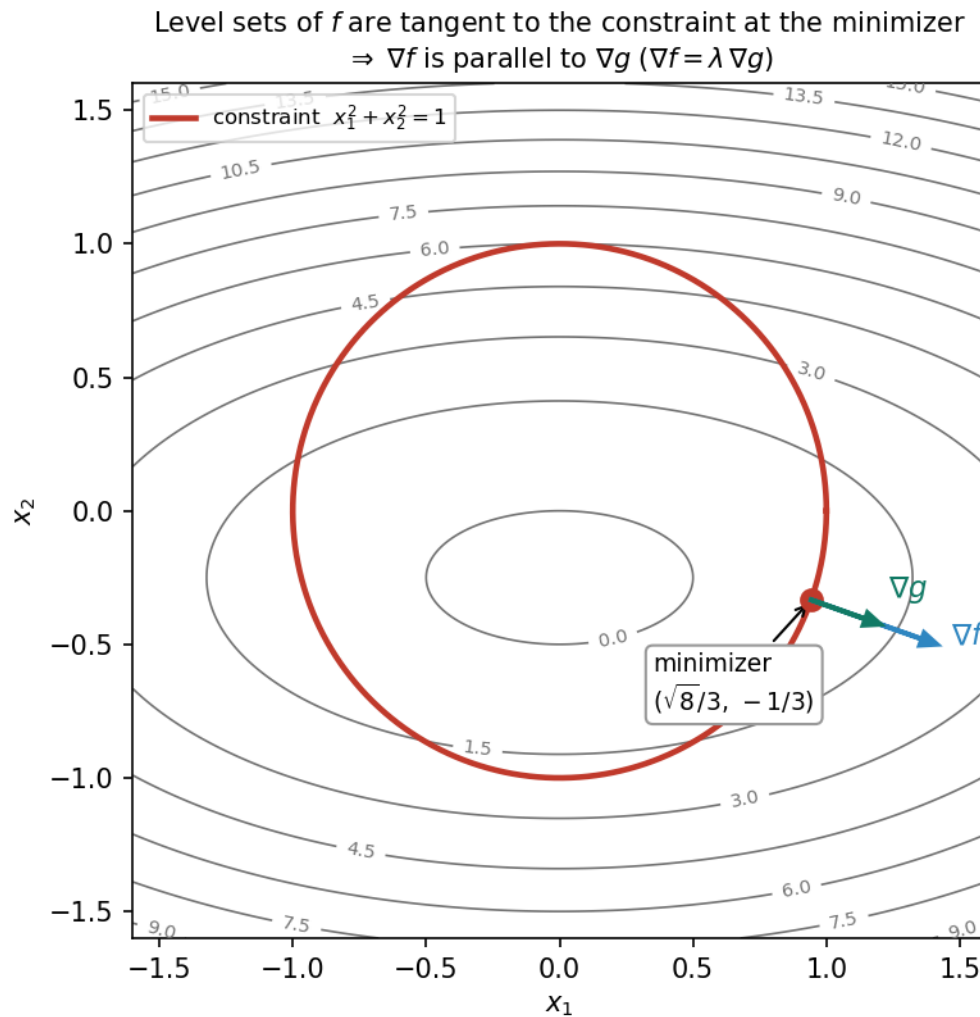


Figure 13: Level sets of $f(x_1, x_2) = x_1^2 + 2x_2 + 4x_2^2$ (thin grey contours) with the constraint circle $x_1^2 + x_2^2 = 1$ (red). The minimizer $(\sqrt{8}/3, -1/3)$ is marked; the blue arrow is $\text{grad } f$ and the green arrow is $\text{grad } g$, parallel to each other, perpendicular to both the circle and the tangent level set.

Basically, ... Draw the contour lines of the objective and the constraint curve on the same map. At the best feasible point, the contour line just *kisses* the constraint curve (tangent) — it can't cut through it, because cutting through would mean you could slide along the constraint to a lower contour. Tangent curves have parallel normal arrows, and those arrows are the gradients. That's the whole method in one picture.

11.7 Inequality constraints: active or not, and complementary slackness

Now $g(\mathbf{x}) \leq 0$. The lectures' treatment stays at one key idea: at the optimum, each inequality constraint is either doing something or nothing.

- i) **Inactive:** $g(\mathbf{x}^*) < 0$. The constraint is slack — locally you can move freely — so the problem behaves like the unconstrained one: $\nabla f(\mathbf{x}^*) = \mathbf{0}$.
- ii) **Active:** $g(\mathbf{x}^*) = 0$. The rope is taut; treat it as an equality: $\nabla f(\mathbf{x}^*) = -\lambda \nabla g(\mathbf{x}^*)$ with $\lambda \geq 0$

(anti-parallel — the §11.2 picture: only the anti-parallel configuration leaves no descent direction feasible).

One line captures both cases — **complementary slackness**:

$$\lambda \cdot g(\mathbf{x}^*) = 0, \quad \lambda \geq 0$$

either the constraint binds ($g(\mathbf{x}^*) = 0$) or the multiplier vanishes ($\lambda = 0$; the constraint might as well not exist).

eg 4 — inactive constraint (full steps). Minimize $f(x, y) = x^2 + y^2$ subject to $x + y \leq 1$.

- i) The unconstrained minimum is $(0, 0)$, and $g(0, 0) = 0 + 0 - 1 = -1 < 0$ — feasible with slack.
- ii) Nothing to do: $\mathbf{x}^* = (0, 0)$, $f = 0$ (and $f \geq 0$ everywhere). $\lambda = 0$, and $\lambda \cdot g = 0 \cdot (-1) = 0$.

eg 5 — active constraint (full steps). Minimize $f(x, y) = x^2 + y^2$ subject to $x + y \leq -1$.

- i) The unconstrained minimum $(0, 0)$ gives $g(0, 0) = 1 > 0$ — infeasible. The constraint must be active.
- ii) Solve as equality, $h(x, y) = x + y + 1 = 0$: $\nabla f = (2x, 2y) = -\lambda(1, 1) = -\lambda \nabla g$.
- iii) So $2x = -\lambda$, $2y = -\lambda$ — $x = y = -\lambda/2$. With $x + y = -1$: $-\lambda = -1$, $\lambda = 1 \geq 0$ (anti-parallel, as required).
- iv) $\mathbf{x}^* = (-\frac{1}{2}, -\frac{1}{2})$, $f = \frac{1}{2}$. Check anti-parallelism: $\nabla f = (-1, -1)$, $\nabla g = (1, 1)$ — opposite directions.

Note: For equalities λ could be anything; for inequalities $\lambda \geq 0$ is load-bearing: it filters maximizers out of the candidate list. Problem 7 shows this directly — the equality problem there has two stationary points, and the sign condition rejects the one with $\lambda < 0$. The full version of these conditions (with several constraints and the Lagrangian) is the Karush–Kuhn–Tucker theory — Chapter 13.

Basically, ... An inequality constraint is a rope, not a rail: it only pushes back when taut. At the optimum, either the rope is slack (then ignore it — you’re at an ordinary unconstrained minimum) or it’s taut (then solve it like an equality, with the extra rule that the multiplier can’t be negative). “Complementary slackness” is just this sentence in algebra: $\lambda \cdot g = 0$ means at least one of the two is zero.

11.8 Projected gradient descent: step, then snap back

Lagrange multipliers solve the equations *analytically* — but in general those equations are unsolvable (eg 2 of §10.2’s warning applies here too). What we want is an *iterative* method, in the spirit of Chapter 10. The problem with plain gradient descent: the step

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta_t \nabla f(\mathbf{x}_t)$$

can land **outside** the feasible set S , and tuning η_t to always stay inside is hopeless in general.

The fix is almost embarrassingly simple: take the gradient step anyway, then **project** back onto S — move to the closest feasible point.

Def. The **projection** of $\mathbf{z}$ onto a set S is the closest point of S :

$$\Pi_S(\mathbf{z}) = \arg \min_{\mathbf{y} \in S} \|\mathbf{z} - \mathbf{y}\|^2.$$

For simple sets it has a closed form: $\Pi_{(-\infty, a]}(z) = \min(z, a)$ (snap down to a), $\Pi_{[0, \infty)}(z) = \max(z, 0)$ (snap up to 0), $\Pi_{\text{unit ball}}(\mathbf{z}) = \mathbf{z} / \max(1, \|\mathbf{z}\|)$ (shrink to the ball).

Def. Projected gradient descent =

$$\mathbf{x}_{t+1} = \Pi_S(\mathbf{x}_t - \eta_t \nabla f(\mathbf{x}_t)) \quad (\text{gradient step, then projection step}).$$

Every iterate stays in S by construction — the projection guarantees it.

eg 6 — projected GD by hand (full steps). Minimize $f(x) = (x - 5)^2$ subject to $x \leq 2$. (The unconstrained minimum $x = 5$ is infeasible; the constrained optimum is $x^* = 2$, the closest feasible point to 5.)

Take $\eta = 0.1$, $x_0 = 0$. Projection: $\Pi_S(z) = \min(z, 2)$.

- i) Gradient step: $x_t - 0.1 \cdot 2(x_t - 5) = 0.8x_t + 1$. Then snap: $x_{t+1} = \min(0.8x_t + 1, 2)$.
- ii) $x_1 = \min(0.8 \cdot 0 + 1, 2) = 1$. f : 25 $\rightarrow$ 16.
- iii) $x_2 = \min(0.8 \cdot 1 + 1, 2) = 1.8$. f : 16 $\rightarrow$ 10.24.
- iv) $x_3 = \min(0.8 \cdot 1.8 + 1, 2) = \min(2.44, 2) = 2$. $f = 9$.
- v) $x_4 = \min(0.8 \cdot 2 + 1, 2) = \min(2.6, 2) = 2$. Frozen at the optimum.

Note: Without the projection, step iv) would have gone to 2.44 — infeasible — and kept marching toward 5. The projection is what holds the iterates inside the fence.

The catch (from the lectures). The method is only as good as the projection:

- i) You must be able to compute Π_S efficiently — for messy sets, the projection itself can be as hard as the original problem.
- ii) Convergence to the true optimum is guaranteed when the constraint set is *nice* — specifically, when S is **convex** (Chapter 12 defines this). If f is also nice (convex), the method even converges fast.

Basically, ... Projected gradient descent is gradient descent with a bouncer. You take your usual downhill step; if you stepped outside the allowed region, the bouncer (the projection) drags you back to the nearest allowed point. It works great when the allowed region is a “nice” shape (convex — next chapter) and the projection is cheap to compute.

11.9 Why ML cares

Constrained problems are everywhere once you look:

- i) **Regularization as a constraint.** Training with a weight budget, $\min L(\mathbf{w})$ subject to $\|\mathbf{w}\| \leq R$, is the constrained form of penalized training — the penalty version of Chapter 28 is its close cousin. Projected gradient descent on the norm ball is literally “take a step, then shrink the weights back into the ball”.
- ii) **SVMs (Chapters 32–33).** The support vector machine is posed as a constrained optimization problem (maximize the margin subject to classification constraints) — and it is solved through the Lagrangian machinery of this chapter, pushed all the way to duality in Chapter 13.
- iii) Any time parameters must live somewhere specific — probabilities summing to 1, non-negative weights, bounded weights — this chapter’s tools apply.

11.10 Where this goes next

- **Chapter 12 (convexity):** the “nice” sets and functions this chapter kept gesturing at — convex sets make projected gradient descent converge; convex functions make every local minimum global.
- **Chapter 13 (duality and KKT):** the full Karush–Kuhn–Tucker conditions for many constraints, and the dual problem — the machinery that actually solves SVMs.
- **Chapter 28 (regularization):** the penalty view of the weight-budget constraint of §11.9.
- **Chapters 32–33 (SVMs):** the payoff — a real ML model built on Lagrange multipliers.

Problem set

1. Starting from $L(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda h(\mathbf{x})$, derive the two first-order conditions for $\min f(\mathbf{x})$ s.t. $h(\mathbf{x}) = 0$. Explain in one line why this is the same condition as the lectures’ $\nabla f(\mathbf{x}^*) = -\lambda \nabla h(\mathbf{x}^*)$.
2. Minimize $f(x, y) = (x - 3)^2 + (y - 4)^2$ subject to $x + y = 3$ by Lagrange multipliers. (Hint: geometrically this is the closest point on the line to $(3, 4)$ — use that to check your answer and its value.)

3. A rectangular box without a lid is to be made from 12m^2 of cardboard. Maximize the volume $f(x, y, z) = xyz$ subject to $2xz + 2yz + xy = 12$, $x, y, z > 0$. (Hint: multiply the three Lagrange equations by x, y, z respectively and compare.)
4. Minimize $f(x, y) = x^2 + xy + y^2$ subject to $x^2 + y^2 = 1$. Find all stationary points from the Lagrange system, evaluate f at each, and identify the minimizers and maximizers.
5. Minimize $f(x, y) = x^2 + y^2$ subject to $x + y \leq 1$. Is the constraint active or inactive at the optimum? Verify complementary slackness.
6. Minimize $f(x, y) = x^2 + y^2$ subject to $x + y \leq -1$. Solve with the constraint active, find λ , and verify $\lambda \geq 0$ and that ∇f and ∇g are anti-parallel at the optimum.
7. Minimize $f(x, y) = (x - 2)^2 + (y - 2)^2$ subject to $x^2 + y^2 \leq 1$. (a) Show the unconstrained minimum is infeasible, so the constraint is active. (b) Find both stationary points of the equality problem and their multipliers; use the sign condition $\lambda \geq 0$ to pick the minimizer.
8. Run three projected-gradient-descent iterations for $\min f(x) = (x - 4)^2$ subject to $x \leq 1$, from $x_0 = 0$ with constant $\eta = 0.25$. What is the constrained optimum, and do the iterates reach it?
9. For $\min f(x) = (x - 5)^2$ subject to $x \leq 2$ with $\eta = 0.1$, $x_0 = 0$: (a) compute x_1, x_2, x_3 of *plain* gradient descent (no projection) and show x_3 is infeasible; (b) apply one projection step to x_2 's successor and compare with eg 6.
10. Minimize $x^2 + y^2 + z^2$ subject to the two constraints $x + y + z = 1$ and $x - y = 0$, using one multiplier per constraint. Interpret the optimum geometrically.
11. Explain, with the §11.2 half-space picture, why an equality constraint allows λ of either sign while an inequality constraint $g(\mathbf{x}) \leq 0$ forces $\lambda \geq 0$. (Hint: for the equality, the feasible directions form the line $\mathbf{d}^T \nabla h = 0$; for the inequality, the half-space $\mathbf{d}^T \nabla g < 0$.)

Solutions — Chapter 11: Constrained optimization

Problem 1

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda h(\mathbf{x}).$$

- i) $\nabla_{\mathbf{x}} L = \nabla f(\mathbf{x}) - \lambda \nabla h(\mathbf{x}) = \mathbf{0}$, i.e. $\nabla f(\mathbf{x}^*) = \lambda \nabla h(\mathbf{x}^*)$.
- ii) $\frac{\partial L}{\partial \lambda} = -h(\mathbf{x}) = 0$, i.e. $h(\mathbf{x}^*) = 0$.

One line: the lectures write $\nabla f = -\lambda \nabla h$ with λ unrestricted, and since λ ranges over all of $\mathbb{R}$, renaming $\lambda \rightarrow -\lambda$ turns one form into the other — they are the same condition.

Problem 2

$$f(x, y) = (x - 3)^2 + (y - 4)^2, \text{ constraint } x + y = 3. \quad L = (x - 3)^2 + (y - 4)^2 - \lambda(x + y - 3).$$

- i) $2(x - 3) - \lambda = 0$, $2(y - 4) - \lambda = 0$ — so $2(x - 3) = 2(y - 4)$, i.e. $y = x + 1$.
- ii) Constraint: $x + (x + 1) = 3$ — $x = 1$, $y = 2$, $\lambda = 2(1 - 3) = -4$.
- iii) Value: $f(1, 2) = 4 + 4 = 8$. Single candidate; $f \rightarrow \infty$ at infinity along the line — the global minimum.

Check: distance from $(3, 4)$ to the line $x + y - 3 = 0$ is $\frac{|3+4-3|}{\sqrt{2}} = \frac{4}{\sqrt{2}} = 2\sqrt{2}$; squared distance = 8. Also $\nabla f(1, 2) = (-4, -4) = -4(1, 1) = \lambda \nabla h$.

Problem 3

Maximize $f = xyz$ subject to $g(x, y, z) = 2xz + 2yz + xy - 12 = 0$, $x, y, z > 0$ (lectures' sign form $\nabla f = -\lambda \nabla g$).

- i) $\nabla f = (yz, xz, xy)$, $\nabla g = (2z + y, 2z + x, 2x + 2y)$. Equations:

$$yz = -\lambda(2z + y), \quad xz = -\lambda(2z + x), \quad xy = -\lambda(2x + 2y).$$

(The sign of λ is free for an equality; $-\lambda$ versus λ makes no difference.)

- ii) Multiply by x, y, z respectively:

$$xyz = -\lambda(2xz + xy) = -\lambda(2yz + xy) = -\lambda(2xz + 2yz).$$

- iii) Since $xyz > 0$ (nonzero volume, positive sides): from the first two, $2xz + xy = 2yz + xy - 2xz = 2yz - x = y$. From the second and third, $2yz + xy = 2xz + 2yz - xy = 2xz - y = 2z$. So $x = y = 2z$.
- iv) Constraint: $2xz + 2yz + xy = 2(2z)(z) + 2(2z)(z) + (2z)(2z) = 12z^2 = 12 \implies z = 1$ (positive root), $(x, y, z) = (2, 2, 1)$.
- v) Volume: $f = 2 \cdot 2 \cdot 1 = 4 \text{ m}^3$, the maximum (boundary cases with a zero side give volume 0).

Problem 4

$$f(x, y) = x^2 + xy + y^2, \quad x^2 + y^2 = 1. \quad L = x^2 + xy + y^2 - \lambda(x^2 + y^2 - 1).$$

- i) $(2 - 2\lambda)x + y = 0, x + (2 - 2\lambda)y = 0$. Put $\mu = 2 - 2\lambda$: $\mu x + y = 0, x + \mu y = 0$.
- ii) $y = -\mu x$ into the second: $x - \mu^2 x = x(1 - \mu^2) = 0$. $x = 0$ forces $y = 0$ (infeasible), so $\mu = \pm 1$.
- iii) $\mu = 1$: $y = -x$; circle gives $2x^2 = 1 \implies (\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}), (-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$. $f = \frac{1}{2} - \frac{1}{2} + \frac{1}{2} = \frac{1}{2}$ at both — the **minimizers**.
- iv) $\mu = -1$: $y = x$; $x = \pm \frac{1}{\sqrt{2}} \implies (\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}), (-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}})$. $f = \frac{1}{2} + \frac{1}{2} + \frac{1}{2} = \frac{3}{2}$ at both — the **maximizers**.

Problem 5

$f(x, y) = x^2 + y^2, g(x, y) = x + y - 1 \leq 0$. The unconstrained minimum $(0, 0)$ has $g(0, 0) = -1 < 0$ — feasible with slack. So the constraint is **inactive** at the optimum: $\mathbf{x}^* = (0, 0), f = 0$ (global, since $f \geq 0$ everywhere). Multiplier $\lambda = 0$; complementary slackness: $\lambda \cdot g(\mathbf{x}^*) = 0 \cdot (-1) = 0$.

Problem 6

$f(x, y) = x^2 + y^2, g(x, y) = x + y + 1 \leq 0$. Unconstrained minimum $(0, 0)$ has $g(0, 0) = 1 > 0$ — infeasible, so the constraint is **active**: solve $x + y + 1 = 0$.

- i) $\nabla f = (2x, 2y) = -\lambda(1, 1) = -\lambda \nabla g$ — so $2x = -\lambda, 2y = -\lambda, x = y = -\lambda/2$.
- ii) $x + y = -1$ gives $-\lambda = -1 \implies \lambda = 1 \geq 0$.
- iii) $\mathbf{x}^* = (-\frac{1}{2}, -\frac{1}{2}), f = \frac{1}{2}$. Anti-parallel check: $\nabla f(-\frac{1}{2}, -\frac{1}{2}) = (-1, -1), \nabla g = (1, 1)$ — opposite directions. Complementary slackness: $g(\mathbf{x}^*) = 0$.

Problem 7

$$f(x, y) = (x - 2)^2 + (y - 2)^2, \quad g(x, y) = x^2 + y^2 - 1 \leq 0.$$

- (a) Unconstrained minimum: $\nabla f = (2(x - 2), 2(y - 2)) = \mathbf{0} \implies (2, 2)$. $g(2, 2) = 8 - 1 = 7 > 0$ — infeasible. So the constraint is active.
- (b) Equality problem $x^2 + y^2 = 1$, lecture form $\nabla f = -\lambda \nabla g$:

$$2(x - 2) = -\lambda \cdot 2x, \quad 2(y - 2) = -\lambda \cdot 2y.$$

$$\text{So } x(1 + \lambda) = 2, \quad y(1 + \lambda) = 2 \implies x = y = \frac{2}{1 + \lambda}. \quad \text{Circle: } 2x^2 = 1 \implies x = \pm \frac{1}{\sqrt{2}}.$$

- $x = y = \frac{1}{\sqrt{2}}$: $\frac{1}{\sqrt{2}}(1 + \lambda) = 2 \implies \lambda = 2\sqrt{2} - 1 \approx 1.83 \geq 0$. Minimizer. Value: $f = 2(\frac{1}{\sqrt{2}} - 2)^2 = 2(\frac{9}{2} - 2\sqrt{2}) = 9 - 4\sqrt{2} \approx 3.34$.
- $x = y = -\frac{1}{\sqrt{2}}$: $\lambda = 1 - 2\sqrt{2} \approx -1.83 < 0$ — fails the sign condition: the maximizer. Value: $9 + 4\sqrt{2} \approx 14.66$.

The $\lambda \geq 0$ rule picks the minimizer and rejects the maximizer.

Problem 8

$$f(x) = (x - 4)^2, \quad x \leq 1, \quad \eta = 0.25, \quad x_0 = 0. \quad \text{Update: } x_{t+1} = \min(x_t - 0.25 \cdot 2(x_t - 4), 1) = \min(0.5x_t + 2, 1).$$

- i) $x_1 = \min(0.5 \cdot 0 + 2, 1) = \min(2, 1) = 1$.
- ii) $x_2 = \min(0.5 \cdot 1 + 2, 1) = \min(2.5, 1) = 1$.

iii) $x_3 = 1$.

Constrained optimum: the closest point of $(-\infty, 1]$ to 4 is $x^* = 1$, $f = 9$ — the iterates reach it in one step and stay.

Problem 9

$f(x) = (x - 5)^2$, $x \leq 2$, $\eta = 0.1$, $x_0 = 0$.

- (a) Plain GD: $x_{t+1} = x_t - 0.2(x_t - 5) = 0.8x_t + 1$. $x_1 = 1$, $x_2 = 0.8 + 1 = 1.8$, $x_3 = 1.44 + 1 = 2.44$. $2.44 > 2$ — infeasible; plain GD leaves the feasible set on the third step (and would keep marching to 5).
- (b) Projecting x_3 : $\min(2.44, 2) = 2$ — exactly eg 6's x_3 . With projection every step, the iterates are $0 \rightarrow 1 \rightarrow 1.8 \rightarrow 2 \rightarrow 2$, frozen at the constrained optimum $x^* = 2$.

Problem 10

$L = x^2 + y^2 + z^2 - \lambda_1(x + y + z - 1) - \lambda_2(x - y)$.

- i) $2x = \lambda_1 + \lambda_2$, $2y = \lambda_1 - \lambda_2$, $2z = \lambda_1$.
- ii) Constraint $x - y = 0$ gives $x = y$, so $\lambda_2 = 0$ and $x = z = \lambda_1/2$.
- iii) $x + y + z = 1$ gives $\frac{3\lambda_1}{2} = 1$ — $\lambda_1 = \frac{2}{3}$, $x = y = z = \frac{1}{3}$.
- iv) Value: $f = 3 \cdot \frac{1}{9} = \frac{1}{3}$ — the global minimum (single candidate, $f \rightarrow \infty$ at infinity).

Geometrically: the two planes intersect in a line; the optimum is the point of that line closest to the origin, with f the squared distance.

Problem 11

Equality $h(\mathbf{x}) = 0$: to first order, feasible directions satisfy $\mathbf{d}^T \nabla h(\mathbf{x}^*) = 0$ — a *line*, symmetric: if $\mathbf{d}$ is feasible, so is $-\mathbf{d}$. The descent directions are the half-space $\mathbf{d}^T \nabla f < 0$. These avoid each other exactly when the line is perpendicular to ∇f — i.e. $\nabla f \parallel \nabla h$ — and parallelism covers *both* directions ($\lambda > 0$ and $\lambda < 0$).

Inequality $g(\mathbf{x}) \leq 0$: feasible directions include the *half-space* $\mathbf{d}^T \nabla g < 0$ — one-sided, not symmetric. Only the anti-parallel configuration ($\nabla f = -\lambda \nabla g$, $\lambda > 0$) points the descent half-space entirely away from the feasible half-space. With $\lambda < 0$ (parallel), every descent direction would be feasible — the point could be improved, so it cannot be a minimum.

12. Convex sets and convex functions

Chapter 10 ended with a promise and a warning: gradient descent reliably finds *local* minima, but a wiggly function has many valleys and the deepest one may stay invisible (§10.9). Chapter 11 added a second promise: projected gradient descent converges when the constraint set is “nice” (§11.8). This chapter cashes both cheques. The nice sets are **convex sets**; the nice functions are **convex functions** — the single class for which *every local minimum is automatically a global minimum*.

12.1 Convex sets: the line-segment test

Def. A set $S \subseteq \mathbb{R}^d$ is **convex** = for every pair $\mathbf{x}_1, \mathbf{x}_2 \in S$ and every $\lambda \in [0, 1]$,

$$\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2 \in S.$$

The point $\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2$ slides along the line segment joining $\mathbf{x}_1$ to $\mathbf{x}_2$ as λ moves from 1 to 0 ($\lambda = 1$ gives $\mathbf{x}_1$, $\lambda = 0$ gives $\mathbf{x}_2$, $\lambda = \frac{1}{2}$ gives the midpoint). Convexity says: pick any two points of the set — the whole segment between them never leaves the set.

- i) In $\mathbb{R}$, the convex sets are exactly the intervals $[a, b]$ (with $a = -\infty$ or $b = +\infty$ allowed). A union of two disjoint intervals, like $[a, b] \cup [c, d]$ with $b < c$, fails: pick one point from each interval and the segment between them crosses the gap.
- ii) The empty set and all of $\mathbb{R}^d$ are convex (the condition holds vacuously / trivially).
- iii) To *disprove* convexity you need only one bad pair: two points in S and one $\lambda \in [0, 1]$ whose mixture escapes S . To *prove* it you must handle every pair and every λ .

Basically, ... A convex set has no dents, no holes, no separate pieces. If you and a friend both stand inside it, you can walk to each other in a straight line without ever stepping outside.

12.2 Examples and non-examples, with verdicts

Convex. - Intervals on the line (§12.1). - **Euclidean balls** $B = \{\mathbf{x} : \|\mathbf{x}\| \leq \theta\}$ — the disk, the solid sphere, and their higher-dimensional cousins (eg 4 proves it). - **Boxes** $[a_1, b_1] \times \cdots \times [a_d, b_d]$. - **Half-spaces** $\{\mathbf{x} : \mathbf{w}^T \mathbf{x} \leq b\}$ and **hyperplanes** $\{\mathbf{x} : \mathbf{w}^T \mathbf{x} = b\}$ — eg 2 and eg 3 prove these. - The feasible sets from Chapter 11: $\{x : x \leq 2\}$ (a half-space), $\{x_1^2 + x_2^2 \leq R^2\}$ (a ball), and $\{\mathbf{x} : A\mathbf{x} = \mathbf{b}\}$ (eg 5).

Not convex. - The union of two disjoint intervals $[0, 1] \cup [2, 3]$ — the segment from 1 to 2 crosses the gap. - The unit circle $\{\mathbf{x} : \|\mathbf{x}\| = 1\}$ in $\mathbb{R}^2$ (the *boundary* only): the midpoint of $(1, 0)$ and $(-1, 0)$ is $(0, 0)$, which is not on the circle. - An annulus (donut) $\{1 \leq \|\mathbf{x}\| \leq 2\}$ — the hole swallows segments. - A crescent / moon shape — two points on the horns join across the missing bite (top-right panel of the figure below).

Convex vs non-convex: sets (top) and functions (bottom)

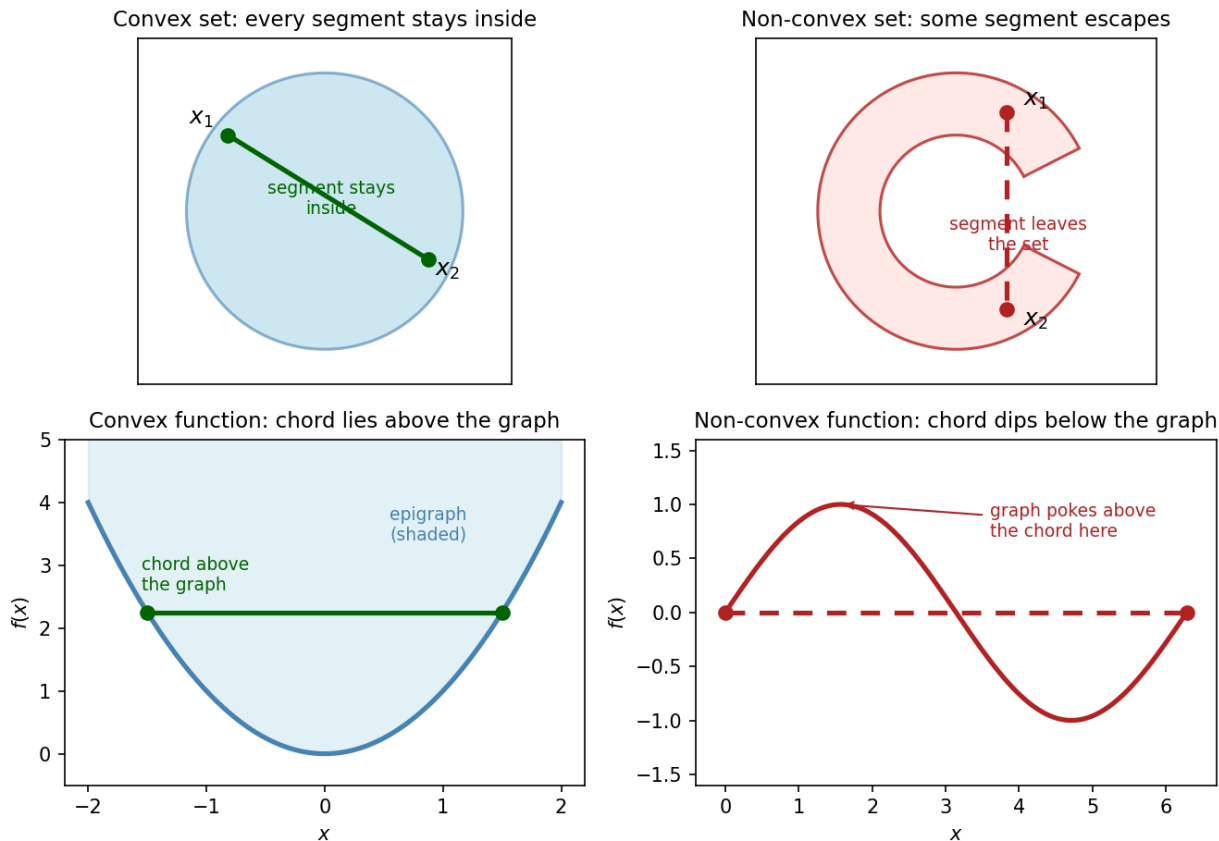


Figure 14: Top row: a disk (convex set) with the segment between two points staying inside, and a crescent (non-convex set) with a segment escaping through the missing bite. Bottom row: $f(x) = x^2$ with its epigraph shaded and a chord lying above the graph (convex function), and $f(x) = \sin x$ with the graph poking above the chord (non-convex function).

eg 1 — an interval is convex (full steps). Let $S = [a, b]$, $\mathbf{x}_1, \mathbf{x}_2 \in [a, b]$, $\lambda \in [0, 1]$. Then

$$\lambda x_1 + (1 - \lambda)x_2 \geq \lambda a + (1 - \lambda)a = a, \quad \lambda x_1 + (1 - \lambda)x_2 \leq \lambda b + (1 - \lambda)b = b,$$

so the mixture lies in $[a, b]$. Verdict: **convex**.

12.3 Hyperplanes and half-spaces are convex

These two appear constantly in ML (linear classifiers, linear constraints), so they get their own proofs — both straight from the definition.

eg 2 — a hyperplane is convex (full steps). Let $S = \{\mathbf{x} : \mathbf{w}^T \mathbf{x} = b\}$ for fixed $\mathbf{w}, b$.

- i) Take $\mathbf{x}_1, \mathbf{x}_2 \in S$, so $\mathbf{w}^T \mathbf{x}_1 = b$ and $\mathbf{w}^T \mathbf{x}_2 = b$. Take any $\lambda \in [0, 1]$.
- ii) $\mathbf{w}^T (\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2) = \lambda \mathbf{w}^T \mathbf{x}_1 + (1 - \lambda) \mathbf{w}^T \mathbf{x}_2 = \lambda b + (1 - \lambda)b = b$.
- iii) So the mixture satisfies the defining equation: it lies in S . Verdict: **convex**.

eg 3 — a half-space is convex (full steps). Let $S = \{\mathbf{x} : \mathbf{w}^T \mathbf{x} \leq b\}$.

- i) Take $\mathbf{x}_1, \mathbf{x}_2 \in S$, so $\mathbf{w}^T \mathbf{x}_1 \leq b$ and $\mathbf{w}^T \mathbf{x}_2 \leq b$. Take any $\lambda \in [0, 1]$.
- ii) $\mathbf{w}^T(\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2) = \lambda \mathbf{w}^T \mathbf{x}_1 + (1 - \lambda) \mathbf{w}^T \mathbf{x}_2 \leq \lambda b + (1 - \lambda)b = b$ — the inequality survives because $\lambda \geq 0$ and $1 - \lambda \geq 0$.
- iii) So the mixture lies in S . Verdict: **convex**.

eg 4 — a Euclidean ball is convex (full steps). Let $B = \{\mathbf{x} : \|\mathbf{x}\| \leq \theta\}$.

- i) Take $\mathbf{x}_1, \mathbf{x}_2 \in B$, so $\|\mathbf{x}_1\| \leq \theta$, $\|\mathbf{x}_2\| \leq \theta$. Take any $\lambda \in [0, 1]$.
- ii) $\|\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2\| \leq \|\lambda \mathbf{x}_1\| + \|(1 - \lambda)\mathbf{x}_2\|$ (triangle inequality) $= \lambda \|\mathbf{x}_1\| + (1 - \lambda)\|\mathbf{x}_2\|$ (homogeneity, $\lambda \geq 0$) $\leq \lambda\theta + (1 - \lambda)\theta = \theta$.
- iii) So the mixture lies in B . Verdict: **convex**.

Basically, ... A hyperplane is flat — of course a segment between two points on it stays on it. A half-space is “one side of a flat wall”, and averaging two points on the same side can’t push you through the wall. A ball is round with no dents, so the straight path between two inside points stays inside.

12.4 The intersection property: convexity survives $\cap$

Theorem. If $S_1, S_2 \subseteq \mathbb{R}^d$ are convex, so is $S_1 \cap S_2$.

Proof. Take $\mathbf{x}_1, \mathbf{x}_2 \in S_1 \cap S_2$ and $\lambda \in [0, 1]$. Then $\mathbf{x}_1, \mathbf{x}_2$ lie in S_1 , so $\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \in S_1$ by convexity of S_1 ; the same argument puts it in S_2 . Belonging to both, it belongs to the intersection.

Note: This is a workhorse: whenever a set looks complicated, try writing it as an intersection of simple convex sets you already trust.

eg 5 — solutions of $A\mathbf{x} = \mathbf{b}$ are convex (full steps). Let $S = \{\mathbf{x} : A\mathbf{x} = \mathbf{b}\}$ with A an $m \times d$ matrix.

- i) Write the rows of A as $\mathbf{a}_1^T, \dots, \mathbf{a}_m^T$ and the entries of $\mathbf{b}$ as $b_1, \dots, b_m$. Then $A\mathbf{x} = \mathbf{b}$ is the system $\mathbf{a}_i^T \mathbf{x} = b_i$, $i = 1, \dots, m$.
- ii) Each $\{\mathbf{x} : \mathbf{a}_i^T \mathbf{x} = b_i\}$ is a hyperplane — convex by eg 2.
- iii) S is the intersection of these m hyperplanes — convex by the intersection theorem.

Note: A box $[a_1, b_1] \times \dots \times [a_d, b_d]$ is convex the same way: each strip $\{a_i \leq x_i \leq b_i\}$ is an intersection of two half-spaces, and the box is the intersection of all $2d$ of them.

Basically, ... Convexity is contagious under intersection: pile convex conditions on top of each other (“satisfy this AND this AND this”) and the surviving set is still convex.

12.5 Convex combinations and the convex hull

So far we mixed *two* points. Mixing finitely many works the same way.

Def. $\mathbf{z}$ is a **convex combination** of $\mathbf{x}_1, \dots, \mathbf{x}_n = \mathbf{z} = \sum_{i=1}^n \lambda_i \mathbf{x}_i$ for some $\lambda_i \geq 0$ with $\sum_{i=1}^n \lambda_i = 1$.

Def. The **convex hull** $\text{CH}(S)$ of a set of points $S = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ = the set of *all* their convex combinations. It is the smallest convex set containing S — equivalently, the intersection of all convex sets that contain S (Problem 5 proves both halves).

Basically, ... The convex hull is what you get by shrink-wrapping the points: pull a rubber sheet tight around them and fill in everything inside. For three non-collinear points in the plane it is the solid triangle; for the probability vectors it is the simplex.

12.6 Convex functions I: epigraph and the chord inequality

Now the functions. A convex function needs a convex domain first — if the domain itself has gaps, the chord test below cannot even be stated. (On all of $\mathbb{R}^d$ this is automatic.) The lectures give four equivalent definitions; each is the right weapon for a different fight.

Def 1 (epigraph). The **epigraph** of $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is the set of points on or above the graph:

$$\text{epi}(f) = \{(\mathbf{x}, z) \in \mathbb{R}^{d+1} : z \geq f(\mathbf{x})\}.$$

f is **convex** = $\text{epi}(f)$ is a convex set. (Bottom-left panel of the figure: the shaded region above x^2 has no dents.)

Def 2 (chord below the graph). f is **convex** = for all $\mathbf{x}_1, \mathbf{x}_2$ and all $\lambda \in [0, 1]$,

$$f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2).$$

The left side is f evaluated on the segment between the points; the right side is the *chord* — the straight line interpolating between the two function values. Convexity says the graph always lies on or below its chords (bottom row of the figure).

eg 6 — $f(x) = x^2$ is convex, from the chord inequality (full steps). Take any x_1, x_2 and $\lambda \in [0, 1]$:

$$\lambda f(x_1) + (1 - \lambda) f(x_2) - f(\lambda x_1 + (1 - \lambda) x_2) = \lambda x_1^2 + (1 - \lambda) x_2^2 - (\lambda x_1 + (1 - \lambda) x_2)^2.$$

Expand the square: $\lambda^2 x_1^2 + 2\lambda(1 - \lambda)x_1 x_2 + (1 - \lambda)^2 x_2^2$. Subtracting,

$$(\lambda - \lambda^2) x_1^2 - 2\lambda(1 - \lambda) x_1 x_2 + ((1 - \lambda) - (1 - \lambda)^2) x_2^2 = \lambda(1 - \lambda)(x_1^2 - 2x_1 x_2 + x_2^2) = \lambda(1 - \lambda)(x_1 - x_2)^2 \geq 0.$$

So $f(\lambda x_1 + (1 - \lambda) x_2) \leq \lambda f(x_1) + (1 - \lambda) f(x_2)$. Verdict: **convex**.

eg 7 — $f(x) = x^3$ is NOT convex on $\mathbb{R}$ (full steps). One bad triple suffices. Take $x_1 = -2$, $x_2 = 0$, $\lambda = \frac{1}{2}$:

$$f(\tfrac{1}{2}(-2) + \tfrac{1}{2}(0)) = f(-1) = -1, \quad \tfrac{1}{2}f(-2) + \tfrac{1}{2}f(0) = \tfrac{1}{2}(-8) + 0 = -4.$$

The inequality demands $-1 \leq -4$ — false. Verdict: **not convex**.

Basically, ... A convex function is bowl-shaped: stretch a rubber band between any two points on the curve and the curve sags below it (or touches it). x^3 fails because on the negative side it bends the wrong way — the rubber band cuts underneath the curve there.

12.7 Convex functions II: first-order and second-order tests

Def 3 (tangent lower bound; f differentiable). f is **convex** iff for all $\mathbf{x}, \mathbf{y}$,

$$f(\mathbf{y}) \geq f(\mathbf{x}) + (\mathbf{y} - \mathbf{x})^T \nabla f(\mathbf{x}).$$

The right side is the first-order Taylor (tangent plane) approximation at $\mathbf{x}$ — and for a convex function this linear approximation *globally underestimates* f : the whole graph sits above every one of its tangents, no matter how far $\mathbf{y}$ is from $\mathbf{x}$. (Compare §10.6: Taylor’s approximation is only *locally* accurate for general f ; convexity upgrades “local” to “everywhere”.)

eg 8 — tangent lower bound for x^2 , checked numerically. Take $f(x) = x^2$, $\mathbf{x} = 1$, $\mathbf{y} = -2$. Then $f(\mathbf{y}) = 4$ and $f(\mathbf{x}) + f'(\mathbf{x})(\mathbf{y} - \mathbf{x}) = 1 + 2(-3) = -5$. Indeed $4 \geq -5$ — the tangent line $y = 2x - 1$ at $x = 1$ lies far below the parabola at $y = -2$.

Def 4 (Hessian PSD; f twice differentiable). f is **convex** iff its Hessian $\mathbf{H}(\mathbf{x}) = \nabla^2 f(\mathbf{x})$ is **positive semi-definite (PSD)** at every $\mathbf{x}$ — i.e. all its eigenvalues are ≥ 0 (§7.9’s eigenvalue test). Intuition: PSD means the function curves *upward* (never downward) in every direction; a single direction of negative curvature breaks the bowl.

Note: This is the test you will actually reach for. Positive *definite* (eigenvalues > 0 , §10.8’s $a > 0$, $\det > 0$ test in 2-D) is stronger than needed — it implies convexity, and even strict convexity. Merely PSD (some zero eigenvalues, like a constant function) still means convex; and kinky convex functions like $|x|$ have no Hessian at the kink at all, so Def 4 simply does not apply there (Problem 6).

eg 9 — $f(x, y) = 4x^2 + 2y^2$ is **convex (full steps)**. i) First derivatives: $f_x = 8x$, $f_y = 4y$. ii) Second derivatives: $f_{xx} = 8$, $f_{xy} = 0$, $f_{yy} = 4$, so $\mathbf{H} = \begin{pmatrix} 8 & 0 \\ 0 & 4 \end{pmatrix}$ (symmetric). iii) Eigenvalue test: $\mathbf{H}$ is diagonal, so its eigenvalues are 8 and 4, both > 0 — positive definite, hence PSD. Determinant test agrees: $a = 8 > 0$, $\det = 8 \cdot 4 - 0 = 32 > 0$. iv) Verdict: **convex**.

eg 10 — $f(x, y) = x^2 - y^2$ is **NOT convex (full steps)**. i) $f_x = 2x$, $f_y = -2y$; $f_{xx} = 2$, $f_{xy} = 0$, $f_{yy} = -2$, so $\mathbf{H} = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}$. ii) Eigenvalues: 2 and -2 — mixed signs, so $\mathbf{H}$ is indefinite (saddle geometry, §10.8). iii) Verdict: **not convex**.

Basically, ... The Hessian is the bowl inspector: it measures curvature in every direction at once. All curvatures non-negative = bowl (convex). One negative curvature = a saddle or a hill somewhere = not convex.

12.8 The gallery: who is convex?

Convex (with one-line verdicts). - $f(x) = x^2$ — eg 6 / eg 9 (baby case). - $f(x) = e^x$ — $f''(x) = e^x > 0$ everywhere. - $f(\mathbf{x}) = \|\mathbf{x}\|$ (any norm) — $\|\lambda\mathbf{x}_1 + (1-\lambda)\mathbf{x}_2\| \leq \lambda\|\mathbf{x}_1\| + (1-\lambda)\|\mathbf{x}_2\|$ by the triangle inequality and homogeneity, exactly the chord inequality. - $f(x) = |x|$ — the 1-D norm; note the kink at 0 means Def 3 and Def 4 do not apply there, but Def 2 works fine. - Every affine function $f(\mathbf{x}) = \mathbf{a}^T \mathbf{x} + b$ — the chord inequality holds with *equality* (both convex and concave). - $f(x) = -\ln x$ on $(0, \infty)$ — $f''(x) = 1/x^2 > 0$.

Not convex (with one-line verdicts). - $f(x) = x^3$ on $\mathbb{R}$ — eg 7 exhibits the violating triple. - $f(x) = \sin x$ on $[0, 2\pi]$ — bottom-right panel of the figure: the graph pokes above the chord (Problem 7 makes this numerical). - $f(x) = e^{-x^2}$ (the Gaussian bell) — convex near the tails is not enough; the middle bends downward, so some chord cuts through the graph. - $f(x, y) = x^2 - y^2$ — eg 10, the saddle.

Note: Non-convexity is a *global* verdict but a *local* witness suffices: one bad pair $(\mathbf{x}_1, \mathbf{x}_2, \lambda)$ kills convexity for the whole function.

12.9 THE theorem: every local minimum is a global minimum

Theorem. If f is convex, every local minimum of f is a global minimum.

This is the payoff Chapter 10 was waiting for (§10.9): gradient descent can only promise *local* minima, but for convex f the local minima *are* the global minima — so gradient descent finds the true optimum.

Proof (by contradiction, following the lectures). Suppose $\mathbf{x}^*$ is a local minimum but *not* a global minimum, and let $\mathbf{z}$ be a point with $f(\mathbf{z}) < f(\mathbf{x}^*)$ — such a $\mathbf{z}$ must exist, since $\mathbf{x}^*$ is not global.

- i) Since $\mathbf{x}^*$ is a local minimum, there is a ball of radius $\delta > 0$ around it inside which $f(\mathbf{x}^*) \leq f(\mathbf{y})$ for all $\mathbf{y}$ in the ball.
- ii) Walk from $\mathbf{x}^*$ toward $\mathbf{z}$ but stop inside that ball: pick $\lambda \in (0, 1)$ close enough to 1 that $\mathbf{y}_\lambda = \lambda\mathbf{x}^* + (1-\lambda)\mathbf{z}$ lies in the ball. By (i), $f(\mathbf{x}^*) \leq f(\mathbf{y}_\lambda)$.
- iii) By convexity of f (Def 2): $f(\mathbf{y}_\lambda) \leq \lambda f(\mathbf{x}^*) + (1-\lambda)f(\mathbf{z})$.
- iv) Since $f(\mathbf{z}) < f(\mathbf{x}^*)$ strictly: $\lambda f(\mathbf{x}^*) + (1-\lambda)f(\mathbf{z}) < \lambda f(\mathbf{x}^*) + (1-\lambda)f(\mathbf{x}^*) = f(\mathbf{x}^*)$.
- v) Chain (ii)–(iv): $f(\mathbf{x}^*) \leq f(\mathbf{y}_\lambda) < f(\mathbf{x}^*)$ — absurd. The assumption was false: no local-but-not-global minimum exists.

Note: “Every local minimum is global” does *not* mean the global minimum is unique. A flat-bottomed convex function (picture a valley with a level floor) has a whole interval of global minima, all sharing the same value — and the set of all global minimizers is itself convex (Problem 11).

Basically, ... A convex function has exactly one valley (possibly with a flat floor). If you’re at the bottom of *your* valley and the landscape is convex, there is no deeper valley anywhere — you’re at the global bottom. Gradient descent, the ball that rolls downhill and stops (§10.9), therefore lands at the true optimum.

12.10 First-order optimality: $\nabla f(\mathbf{x}^*) = \mathbf{0}$ is necessary *and* sufficient

For general functions, $\nabla f(\mathbf{x}^*) = \mathbf{0}$ is only a *necessary* condition for a minimum (it also holds at maxima and saddles — §10.8’s interrogation exists precisely because of this). Convexity promotes it to a full characterization:

Theorem. Let f be differentiable and convex. Then $\mathbf{x}^*$ is a global minimum **iff** $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

Proof sketch. ($\Rightarrow$) If $\mathbf{x}^*$ is global, it is local, and the Taylor argument from gradient descent applies: a nonzero gradient would give a descent direction $-\nabla f(\mathbf{x}^*)$ strictly decreasing f nearby — contradiction. (Convexity is not used here.) ($\Leftarrow$) If $\nabla f(\mathbf{x}^*) = \mathbf{0}$, plug $\mathbf{x} = \mathbf{x}^*$ into Def 3: $f(\mathbf{y}) \geq f(\mathbf{x}^*) + (\mathbf{y} - \mathbf{x}^*)^T \nabla f(\mathbf{x}^*) = f(\mathbf{x}^*)$ for every $\mathbf{y}$. So $\mathbf{x}^*$ beats everything — it is global.

Why this matters. Gradient descent halts exactly where $\nabla f = \mathbf{0}$. For convex f , this theorem certifies that wherever it halts is the global optimum — no Hessian interrogation needed, no valley-doubt. “Set the gradient to zero and solve” is not a heuristic here; it is the complete answer.

Basically, ... For a general function, “gradient is zero” means “flat here — could be a valley, a peak, or a pass”. For a convex function it means exactly one thing: “you are at the bottom of the only valley”.

12.11 Building bigger convex functions: sums and compositions

Real ML objectives look scary — but they factor into small convex pieces glued by operations that preserve convexity.

Property 1 — sums. If f, g are convex, so is $h(\mathbf{x}) = f(\mathbf{x}) + g(\mathbf{x})$.

Proof. $h(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) = f(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) + g(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1-\lambda)f(\mathbf{y}) + \lambda g(\mathbf{x}) + (1-\lambda)g(\mathbf{y}) = \lambda h(\mathbf{x}) + (1-\lambda)h(\mathbf{y})$.

Property 2 — composition (convex over linear). If $f: \mathbb{R} \rightarrow \mathbb{R}$ is convex and $g(\mathbf{x}) = \mathbf{a}^T \mathbf{x} + c$ is affine, then $h(\mathbf{x}) = f(g(\mathbf{x}))$ is convex.

Proof. g preserves mixtures exactly: $g(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) = \lambda g(\mathbf{x}) + (1-\lambda)g(\mathbf{y})$. So $h(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) = f(\lambda g(\mathbf{x}) + (1-\lambda)g(\mathbf{y})) \leq \lambda f(g(\mathbf{x})) + (1-\lambda)f(g(\mathbf{y})) = \lambda h(\mathbf{x}) + (1-\lambda)h(\mathbf{y})$, using convexity of f on the two scalars $g(\mathbf{x}), g(\mathbf{y})$.

Note: Composition of convex with convex is *not* convex in general — it needs an extra condition (e.g. the outer function also non-decreasing). The classic counterexample is $f(z) = e^{-z}$ (convex) over $g(x) = x^2$ (convex): $h(x) = e^{-x^2}$ is the Gaussian bell, not convex (§12.8).

eg 11 — least squares is convex (full steps). $f(\mathbf{w}) = \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}_i - y_i)^2$, the linear-regression objective (Chapters 5, 23).

- i) Split it: $f(\mathbf{w}) = \sum_{i=1}^n h_i(\mathbf{w})$ with $h_i(\mathbf{w}) = (\mathbf{w}^T \mathbf{x}_i - y_i)^2$.
- ii) Each h_i is a composition: inner $g_i(\mathbf{w}) = \mathbf{w}^T \mathbf{x}_i - y_i$ is affine in $\mathbf{w}$; outer $f_0(z) = z^2$ is convex (§12.8). By Property 2, each h_i is convex.
- iii) By Property 1, the sum f is convex.

So minimizing squared error is a convex optimization problem: gradient descent (§10) provably lands at the global optimum, and “set the gradient to zero” gives the normal equations directly.

Basically, ... Don’t wrestle the whole objective at once. Chop it into per-datapoint pieces, check each small piece is convex (usually a convex function of a linear score), then add the pieces up. Sums and “convex-of-affine” never break convexity.

12.12 Why ML cares

Convexity is why several of this book’s workhorses are trustworthy:

- i) **Least squares (Chapters 5, 23)** — convex by eg 11. Closed-form optimum, gradient descent guaranteed to find it.
- ii) **Logistic regression (Chapter 31)** — its loss is convex in the weights (proved there from these same tools). Same guarantee: no bad local minima.
- iii) **SVMs (Chapters 32–33)** — posed as convex optimization problems (convex objective, convex feasible set); the Lagrangian machinery of Chapter 11 and the duality of Chapter 13 apply precisely because everything is convex.
- iv) **Projected gradient descent (§11.8)** — converges to the true constrained optimum when the feasible set S is convex (the projection stays meaningful) and f is convex (the target is the global minimum). The “nice sets” promise is now defined.

The honest one-liner: neural-network losses are **not** convex in their weights — the landscape has many valleys, saddles, and symmetries. Gradient descent there finds *a* good local minimum, not a certified global one (and in practice that is enough — Chapter 43).

Basically, ... Convexity is the certificate that says “the optimizer can’t get trapped”. Least squares, logistic regression, and SVMs carry this certificate; neural nets don’t — which is exactly why training them is an art as well as a science.

12.13 Where this goes next

- **Chapter 13 (duality and KKT):** convexity is what makes the *dual* problem meaningful — for convex problems the duality gap closes (strong duality), and the KKT conditions become necessary *and* sufficient for optimality. That is the machinery that actually solves SVMs.
- **Chapter 11 revisited:** every feasible set there — half-spaces, balls, $\{A\mathbf{x} = \mathbf{b}\}$ — is now certified convex, so projected gradient descent (§11.8) is on solid ground.
- **Chapter 25 (EM) and MLT:** Jensen’s inequality, the probabilist’s workhorse built on convexity, appears there — this chapter’s chord inequality is its ancestor.

Problem set

1. Prove from the definition that the half-space $\{\mathbf{x} : \mathbf{a}^T \mathbf{x} \leq b\}$ is convex.
2. Prove that the Euclidean ball $B = \{\mathbf{x} : \|\mathbf{x}\| \leq \theta\}$ is convex. (Hint: triangle inequality + homogeneity, as in eg 4.)
3. Is the unit circle $C = \{\mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x}\| = 1\}$ convex? Give an explicit violating pair and λ , with the verdict.
4. Prove that the intersection of an *arbitrary collection* (not just two) of convex sets is convex.
5. Prove that the convex hull $\text{CH}(S)$ of a finite point set S is a convex set. (Harder follow-up: show it equals the intersection of all convex sets containing S .)
6. Prove $f(x) = |x|$ is convex on $\mathbb{R}$ using the chord inequality (Def 2). Where do Def 3 and Def 4 break down for this f ?
7. Show $f(x) = \sin x$ is *not* convex on $[0, 2\pi]$: use $x_1 = 0$, $x_2 = \pi$, $\lambda = \frac{1}{2}$ and compute both sides of the chord inequality.
8. Classify $f(x, y) = x^2 + xy + y^2$ via the Hessian: compute all second partials, write down $\mathbf{H}$, run the 2×2 determinant test ($a > 0$, $\det > 0$), and give the verdict.
9. Show $f(x) = -\ln x$ is convex on $(0, \infty)$ using the second-derivative test.
10. Let $f(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|^2$ with X an $n \times d$ matrix. (a) Compute $\nabla^2 f(\mathbf{w})$ and show it equals $2X^T X$. (b) For any $\mathbf{z}$, show $\mathbf{z}^T (2X^T X) \mathbf{z} = 2\|X\mathbf{z}\|^2 \geq 0$, so the Hessian is PSD. (c) Conclude f is convex — a second proof of eg 11.
11. Prove that the set of all global minimizers of a convex function f is a convex set.
12. For $f(x, y) = (x - 3)^2 + (y + 1)^2$: (a) find the point where $\nabla f = \mathbf{0}$; (b) use the theorem of §12.10 to certify it is the global minimum — no Hessian needed; (c) write out the tangent-plane lower bound (Def 3) at that point explicitly.

Solutions — Chapter 12. Convex sets and convex functions

Full worked solutions to the Chapter 12 problem set. Same numbering as the chapter.

1. Prove from the definition that the half-space $\{\mathbf{x} : \mathbf{a}^T \mathbf{x} \leq b\}$ is convex.

Solution. Let $S = \{\mathbf{x} : \mathbf{a}^T \mathbf{x} \leq b\}$.

- i) Take $\mathbf{x}_1, \mathbf{x}_2 \in S$: then $\mathbf{a}^T \mathbf{x}_1 \leq b$ and $\mathbf{a}^T \mathbf{x}_2 \leq b$. Take any $\lambda \in [0, 1]$.
- ii) $\mathbf{a}^T(\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2) = \lambda \mathbf{a}^T \mathbf{x}_1 + (1-\lambda) \mathbf{a}^T \mathbf{x}_2$. Since $\lambda \geq 0$ and $1-\lambda \geq 0$, multiplying the two inequalities keeps their direction: $\lambda \mathbf{a}^T \mathbf{x}_1 \leq \lambda b$ and $(1-\lambda) \mathbf{a}^T \mathbf{x}_2 \leq (1-\lambda)b$.
- iii) Adding: $\mathbf{a}^T(\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2) \leq \lambda b + (1-\lambda)b = b$. So the mixture lies in S . Since $\mathbf{x}_1, \mathbf{x}_2, \lambda$ were arbitrary, S is convex.

2. Prove that the Euclidean ball $B = \{\mathbf{x} : \|\mathbf{x}\| \leq \theta\}$ is convex.

Solution.

- i) Take $\mathbf{x}_1, \mathbf{x}_2 \in B$: $\|\mathbf{x}_1\| \leq \theta$, $\|\mathbf{x}_2\| \leq \theta$. Take any $\lambda \in [0, 1]$.
- ii) Triangle inequality: $\|\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2\| \leq \|\lambda \mathbf{x}_1\| + \|(1-\lambda)\mathbf{x}_2\|$.
- iii) Homogeneity with non-negative scalars: $= \lambda \|\mathbf{x}_1\| + (1-\lambda)\|\mathbf{x}_2\| \leq \lambda\theta + (1-\lambda)\theta = \theta$.
- iv) So $\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2 \in B$. Arbitrary pair and $\lambda \in B$ is convex.

3. Is the unit circle $C = \{\mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x}\| = 1\}$ convex?

Solution. No. Take $\mathbf{x}_1 = (1, 0)$ and $\mathbf{x}_2 = (-1, 0)$, both in C since $\|\mathbf{x}_1\| = \|\mathbf{x}_2\| = 1$. With $\lambda = \frac{1}{2}$:

$$\frac{1}{2}\mathbf{x}_1 + \frac{1}{2}\mathbf{x}_2 = (0, 0), \quad \|(0, 0)\| = 0 \neq 1,$$

so the midpoint is not in C . One violating triple is enough: **not convex**. (The disk $\|\mathbf{x}\| \leq 1$ is convex — Problem 2 — but the boundary alone is not.)

4. Prove that the intersection of an *arbitrary collection* of convex sets is convex.

Solution. Let $\{S_i\}_{i \in I}$ be convex sets (any index set I) and $S = \bigcap_{i \in I} S_i$.

- i) Take $\mathbf{x}_1, \mathbf{x}_2 \in S$ and $\lambda \in [0, 1]$.
- ii) For *each* $i \in I$: $\mathbf{x}_1, \mathbf{x}_2 \in S_i$ (they lie in the intersection, hence in every member), so $\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2 \in S_i$ by convexity of S_i .
- iii) The mixture lies in every S_i , hence in $\bigcap_{i \in I} S_i = S$. So S is convex.

5. Prove that the convex hull $\text{CH}(S)$ of a finite point set S is a convex set. (Follow-up: show it equals the intersection of all convex sets containing S .)

Solution. Write $S = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$.

Convexity. Take $\mathbf{z}_1, \mathbf{z}_2 \in \text{CH}(S)$: $\mathbf{z}_1 = \sum_{i=1}^n \lambda_i \mathbf{x}_i$, $\mathbf{z}_2 = \sum_{i=1}^n \mu_i \mathbf{x}_i$ with $\lambda_i, \mu_i \geq 0$, $\sum_i \lambda_i = \sum_i \mu_i = 1$. For $t \in [0, 1]$:

$$t\mathbf{z}_1 + (1-t)\mathbf{z}_2 = \sum_{i=1}^n (t\lambda_i + (1-t)\mu_i) \mathbf{x}_i.$$

The new coefficients satisfy $t\lambda_i + (1-t)\mu_i \geq 0$ and $\sum_i (t\lambda_i + (1-t)\mu_i) = t \cdot 1 + (1-t) \cdot 1 = 1$ — so the mixture is again a convex combination of the $\mathbf{x}_i$, i.e. in $\text{CH}(S)$.

Follow-up (equivalence). Let $\mathcal{C} = \bigcap \{C : C \text{ convex}, S \subseteq C\}$.

- i) $\text{CH}(S) \subseteq \mathcal{C}$: take any convex $C \supseteq S$ and any $\mathbf{z} = \sum_{i=1}^n \lambda_i \mathbf{x}_i \in \text{CH}(S)$. By induction on n , convex combinations of points of C stay in C (base $n = 2$ is the definition; for $n + 1$, if $\lambda_{n+1} = 1$ then $\mathbf{z} = \mathbf{x}_{n+1} \in C$, else $\mathbf{z} = (1 - \lambda_{n+1}) \sum_{i=1}^n \frac{\lambda_i}{1 - \lambda_{n+1}} \mathbf{x}_i + \lambda_{n+1} \mathbf{x}_{n+1}$, a mixture of two points of C by the induction hypothesis). So $\mathbf{z} \in C$ for every such C , hence $\mathbf{z} \in \mathcal{C}$.
- ii) $\mathcal{C} \subseteq \text{CH}(S)$: $\text{CH}(S)$ is convex (proved above) and contains S (take $\lambda_i = 1$ for $\mathbf{x}_i$), so it is one of the sets being intersected — and an intersection is contained in each of its members.
- iii) Both inclusions give $\text{CH}(S) = \mathcal{C}$.

6. Prove $f(x) = |x|$ is convex on $\mathbb{R}$ using the chord inequality. Where do Def 3 and Def 4 break down?

Solution. Take any x_1, x_2 and $\lambda \in [0, 1]$:

$$f(\lambda x_1 + (1 - \lambda)x_2) = |\lambda x_1 + (1 - \lambda)x_2| \leq |\lambda x_1| + |(1 - \lambda)x_2| = \lambda|x_1| + (1 - \lambda)|x_2| = \lambda f(x_1) + (1 - \lambda)f(x_2),$$

using the triangle inequality then $|\lambda x| = \lambda|x|$ for $\lambda \geq 0$. So Def 2 holds: **convex**.

Def 3 needs f differentiable everywhere — $|x|$ has no derivative at $x = 0$ (left slope $-1 \neq$ right slope $+1$), so $\nabla f(0)$ does not exist and the tangent lower bound cannot even be stated there. Def 4 needs f twice differentiable — it fails at 0 for the same reason (it is not even once differentiable there). Both definitions are inapplicable at the kink; Def 2 works globally.

7. Show $f(x) = \sin x$ is *not* convex on $[0, 2\pi]$.

Solution. Take $x_1 = 0$, $x_2 = \pi$, $\lambda = \frac{1}{2}$. Left side:

$$f(\tfrac{1}{2} \cdot 0 + \tfrac{1}{2} \cdot \pi) = \sin(\pi/2) = 1.$$

Right side:

$$\tfrac{1}{2}f(0) + \tfrac{1}{2}f(\pi) = \tfrac{1}{2} \cdot 0 + \tfrac{1}{2} \cdot 0 = 0.$$

Convexity demands $1 \leq 0$ — false. Verdict: **not convex**. (Geometrically: the chord from $(0, 0)$ to $(\pi, 0)$ is the x -axis, and the graph bulges above it.)

8. Classify $f(x, y) = x^2 + xy + y^2$ via the Hessian.

Solution.

- i) First partials: $f_x = 2x + y$, $f_y = x + 2y$.
- ii) Second partials: $f_{xx} = 2$, $f_{xy} = 1$, $f_{yx} = 1$, $f_{yy} = 2$. So $\mathbf{H} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ (symmetric).
- iii) 2×2 test: $a = f_{xx} = 2 > 0$ and $\det(\mathbf{H}) = 2 \cdot 2 - 1 \cdot 1 = 3 > 0$ — positive definite, hence PSD. (Eigenvalue check: $\det(\mathbf{H} - \lambda I) = (2 - \lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = 0$ gives $\lambda = 3, 1$, both > 0 .)
- iv) Verdict: **convex**. (Indeed $f(x, y) = \frac{1}{2}(x + y)^2 + \frac{1}{2}(x^2 + y^2)$, a sum of convex pieces.)

9. Show $f(x) = -\ln x$ is convex on $(0, \infty)$.

Solution. $f'(x) = -1/x$, $f''(x) = 1/x^2$. For every $x > 0$, $f''(x) = 1/x^2 > 0 \geq 0$. By the 1-D second-derivative test (Def 4 with $d = 1$), f is **convex** on $(0, \infty)$.

10. Least squares via the Hessian: $f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$.

Solution.

- (a) Expand: $f(\mathbf{w}) = (X\mathbf{w} - \mathbf{y})^T(X\mathbf{w} - \mathbf{y}) = \mathbf{w}^T X^T X \mathbf{w} - 2\mathbf{y}^T X \mathbf{w} + \mathbf{y}^T \mathbf{y}$. The matrix $X^T X$ is symmetric. Gradient: $\nabla f(\mathbf{w}) = 2X^T X \mathbf{w} - 2X^T \mathbf{y}$ (using $\nabla_{\mathbf{w}} \mathbf{w}^T A \mathbf{w} = 2A\mathbf{w}$ for symmetric A , and $\nabla_{\mathbf{w}} \mathbf{c}^T \mathbf{w} = \mathbf{c}$). Differentiating once more: $\nabla^2 f(\mathbf{w}) = 2X^T X$, constant in $\mathbf{w}$.

- (b) For any $\mathbf{z} \in \mathbb{R}^d$:

$$\mathbf{z}^T (2X^T X) \mathbf{z} = 2 (X\mathbf{z})^T (X\mathbf{z}) = 2\|X\mathbf{z}\|^2 \geq 0.$$

So the Hessian is positive semi-definite (this is exactly the §7.9 eigenvalue picture: all eigenvalues of $X^T X$ are ≥ 0).

- (c) A twice-differentiable function with PSD Hessian everywhere is convex (Def 4). Hence f is **convex** — the Hessian route to the same conclusion as eg 11.

11. Prove that the set of all global minimizers of a convex function f is a convex set.

Solution. Let $f^* = \min_{\mathbf{x}} f(\mathbf{x})$ (assume the minimum is attained) and $M = \{\mathbf{x} : f(\mathbf{x}) = f^*\}$.

- i) Take $\mathbf{x}_1, \mathbf{x}_2 \in M$: $f(\mathbf{x}_1) = f(\mathbf{x}_2) = f^*$. Take $\lambda \in [0, 1]$.
- ii) By convexity: $f(\lambda\mathbf{x}_1 + (1-\lambda)\mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1-\lambda)f(\mathbf{x}_2) = \lambda f^* + (1-\lambda)f^* = f^*$.
- iii) But f^* is the *global minimum value*, so $f(\cdot) \geq f^*$ everywhere — combined with (ii), $f(\lambda\mathbf{x}_1 + (1-\lambda)\mathbf{x}_2) = f^*$, i.e. the mixture lies in M .
- iv) So M is convex. (This is why a flat-bottomed convex function has an *interval* of minimizers in 1-D.)

12. For $f(x, y) = (x-3)^2 + (y+1)^2$: (a) find where $\nabla f = \mathbf{0}$; (b) certify it is the global minimum; (c) write the tangent-plane lower bound there.

Solution.

- (a) $\nabla f(x, y) = (2(x-3), 2(y+1))$. Setting to zero: $x = 3, y = -1$. The unique stationary point is $\mathbf{x}^* = (3, -1)$ with $f(\mathbf{x}^*) = 0$.
- (b) f is differentiable, and convex: it is a sum of the convex quadratics $(x-3)^2$ and $(y+1)^2$ (each has second derivative $2 > 0$; sums of convex are convex, Property 1). By the §12.10 theorem, for differentiable convex f , $\nabla f(\mathbf{x}^*) = \mathbf{0}$ **iff** $\mathbf{x}^*$ is a global minimum. Hence $(3, -1)$ is certified the global minimizer — no Hessian interrogation needed.
- (c) Def 3 at $\mathbf{x}^*$: $f(\mathbf{y}) \geq f(\mathbf{x}^*) + (\mathbf{y} - \mathbf{x}^*)^T \nabla f(\mathbf{x}^*) = 0 + 0 = 0$ for all $\mathbf{y} \in \mathbb{R}^2$. Explicitly: $(y_1-3)^2 + (y_2+1)^2 \geq 0$ — the tangent “plane” at the bottom of the bowl is the flat plane $z = 0$, and the bowl sits above it everywhere.

13. Duality and KKT conditions

This is the last chapter of Part I, and it ties the whole part together. Chapter 10 said: for unconstrained problems, $\nabla f(\mathbf{x}^*) = \mathbf{0}$ finds candidates, and gradient descent finds local minima (§10.8–10.9). Chapter 11 said: constraints change the game — the Lagrangian encodes them, and each inequality is either active or inactive at the optimum (§11.3, §11.7). Chapter 12 said: if everything is convex, local minima *are* global minima (§12.9), and $\nabla f = \mathbf{0}$ becomes necessary *and* sufficient (§12.10). This chapter finishes the story: every constrained problem has a **mirror image** — the **dual** problem — and for convex problems the two mirrors agree (strong duality). Out of that agreement drop the **KKT conditions**: four equations that *completely characterize* the optimum of a convex constrained problem — the constrained analogue of “set the gradient to zero”.

The lectures build this in the simplest setting first — one inequality constraint — and only then generalize. This chapter follows that order.

13.1 The problem, revisited

Def. The (single-inequality) constrained problem =

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{s.t.} \quad h(\mathbf{x}) \leq 0,$$

with $f, h : \mathbb{R}^d \rightarrow \mathbb{R}$. f is the objective, $h(\mathbf{x}) \leq 0$ the constraint. The feasible set $\{\mathbf{x} : h(\mathbf{x}) \leq 0\}$ is a sublevel set of h .

Note: Chapter 11 handled *equality* constraints $l(\mathbf{x}) = 0$ with the Lagrangian (§11.3). Inequalities need one extra discipline: the multiplier must be non-negative. That sign is the whole subject of this chapter.

13.2 The Lagrangian, made formal

Def. The **Lagrangian** for the problem above =

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda h(\mathbf{x}), \quad \lambda \geq 0.$$

It takes *two* inputs: the variable $\mathbf{x}$ (a vector) and one scalar **Lagrange multiplier** λ per constraint — here just one, since there is one constraint. (With m inequalities and n equalities there will be $m + n$ multipliers; §13.9.)

Why $\lambda \geq 0$? Think of $\lambda h(\mathbf{x})$ as a *penalty* for violating the constraint. If $h(\mathbf{x}) > 0$ (violation) and $\lambda > 0$, the penalty is positive — bad. If $h(\mathbf{x}) \leq 0$ (feasible) and $\lambda \geq 0$, the penalty is ≤ 0 — never a reward for leaving. A negative λ would *reward* violations, which is backwards. The sign discipline $\lambda \geq 0$ is what makes the Lagrangian honest.

Basically, ... The Lagrangian is the objective plus a fine for breaking the rule. The fine is λ rupees per unit of violation, and λ can’t be negative — you never get *paid* for breaking the rule.

13.3 The primal problem: maxing over λ *is* the constraint

Here is the lecture's key move. Fix an $\mathbf{x}$ and consider

$$\max_{\lambda \geq 0} L(\mathbf{x}, \lambda) = \max_{\lambda \geq 0} [f(\mathbf{x}) + \lambda h(\mathbf{x})].$$

$f(\mathbf{x})$ is now a constant; only the penalty term moves. Two cases:

- i) $h(\mathbf{x}) \leq 0$ (feasible): $\lambda h(\mathbf{x}) \leq 0$ for every $\lambda \geq 0$, so the biggest it can be is 0 — attained at $\lambda = 0$. The max equals $f(\mathbf{x})$.
- ii) $h(\mathbf{x}) > 0$ (infeasible): crank $\lambda \rightarrow \infty$ and $\lambda h(\mathbf{x}) \rightarrow \infty$. The max is ∞ .

So $\max_{\lambda \geq 0} L(\mathbf{x}, \lambda)$ equals $f(\mathbf{x})$ inside the feasible set and ∞ outside — exactly the function

$$J(\mathbf{x}) = \begin{cases} f(\mathbf{x}) & h(\mathbf{x}) \leq 0 \\ \infty & \text{otherwise.} \end{cases}$$

Def. The **primal problem** =

$$\min_{\mathbf{x}} \left[\max_{\lambda \geq 0} L(\mathbf{x}, \lambda) \right] = \min_{\mathbf{x}} J(\mathbf{x}).$$

It is *equivalent* to the original constrained problem: any $\mathbf{x}$ outside the feasible set scores ∞ and can never be the minimum, so the minimizer must be feasible, and among feasible points $J = f$. Call the primal solution $\mathbf{x}^*$ and the primal optimal value $p^* = f(\mathbf{x}^*)$.

Note: We have not made anything easier — the lectures are explicit that this min-max “can be hard to solve”. The payoff comes from *swapping* the min and the max.

Basically, ... The inner max is a bouncer: show a feasible $\mathbf{x}$ and it lets you through with your exact $f(\mathbf{x})$; show an infeasible one and it throws you out with an infinite bill. Minimizing over $\mathbf{x}$ afterwards therefore solves the original problem — with the constraint now *encoded* in the objective.

13.4 Swap the min and the max: the dual function and the dual problem

Out of curiosity, swap the order:

$$\max_{\lambda \geq 0} \left[\min_{\mathbf{x}} L(\mathbf{x}, \lambda) \right].$$

Def. The **dual function** =

$$g(\lambda) = \min_{\mathbf{x}} L(\mathbf{x}, \lambda) = \min_{\mathbf{x}} [f(\mathbf{x}) + \lambda h(\mathbf{x})].$$

Def. The **dual problem** = $\max_{\lambda \geq 0} g(\lambda)$. Call the dual solution λ^* and the dual optimal value $d^* = g(\lambda^*)$.

Why would anyone do this? Two reasons (both from the lectures):

- i) The inner problem $\min_{\mathbf{x}} L(\mathbf{x}, \lambda)$ is *unconstrained* — the whole Chapter 10 toolbox applies.
- ii) The outer constraint is just $\lambda \geq 0$ — far simpler than the original $h(\mathbf{x}) \leq 0$.

So the dual is “typically easier” than the primal. The catch: swapping min and max changes the problem, and nothing yet says the two optima agree. That is what §§13.6–13.7 settle.

Basically, ... The primal does the hard part (constraints) on the inside and the easy part outside; the dual flips it — unconstrained on the inside, a trivial $\lambda \geq 0$ on the outside. Same ingredients, much friendlier order.

13.5 The dual function is concave (the lecturer's exercise)

Theorem. $g(\lambda) = \min_{\mathbf{x}} L(\mathbf{x}, \lambda)$ is a **concave** function of λ .

Proof. Fix $\mathbf{x}$. As a function of λ , $L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda h(\mathbf{x})$ is *affine* (a straight line in λ). So g is the pointwise *minimum* of a family of affine functions — one line per $\mathbf{x}$. Take λ_1, λ_2 and $\theta \in [0, 1]$:

$$g(\theta\lambda_1 + (1 - \theta)\lambda_2) = \min_{\mathbf{x}} [\theta L(\mathbf{x}, \lambda_1) + (1 - \theta)L(\mathbf{x}, \lambda_2)].$$

For every $\mathbf{x}$, $\theta L(\mathbf{x}, \lambda_1) + (1 - \theta)L(\mathbf{x}, \lambda_2) \geq \theta g(\lambda_1) + (1 - \theta)g(\lambda_2)$ (each term is at least its own minimum). Taking $\min_{\mathbf{x}}$ on the left preserves the inequality:

$$g(\theta\lambda_1 + (1 - \theta)\lambda_2) \geq \theta g(\lambda_1) + (1 - \theta)g(\lambda_2),$$

which is exactly the concavity chord inequality (graph above its chords).

So the dual problem is: **maximize a concave function over** $\lambda \geq 0$ — a well-behaved problem (maximizing concave is the mirror of minimizing convex).

Basically, ... Each fixed $\mathbf{x}$ gives a straight line in λ ; the dual function is the *lower envelope* of all those lines — and the lower envelope of straight lines always sags downward like a frown, i.e. it is concave.

13.6 Weak duality: the dual always under-promises ($d^* \leq p^*$)

Theorem (weak duality). For *any* f, h (no convexity needed),

$$\boxed{d^* \leq p^*}.$$

The dual optimal value never exceeds the primal optimal value. The difference $p^* - d^* \geq 0$ is the **duality gap**.

Proof (following the lectures). Fix any $\lambda \geq 0$.

- i) From §13.3: $L(\mathbf{x}, \lambda) \leq J(\mathbf{x})$ for *every* $\mathbf{x}$ — if $h(\mathbf{x}) \leq 0$ then $L = f + \lambda h \leq f = J$; if $h(\mathbf{x}) > 0$ then $L \leq \infty = J$.
- ii) Take $\min_{\mathbf{x}}$ on both sides: $\min_{\mathbf{x}} L(\mathbf{x}, \lambda) \leq \min_{\mathbf{x}} J(\mathbf{x})$. The left side is $g(\lambda)$; the right side is p^* (the min of J is the constrained optimum $f(\mathbf{x}^*)$).
- iii) This holds for *every* $\lambda \geq 0$, so it holds for the λ that maximizes g : $\max_{\lambda \geq 0} g(\lambda) \leq p^*$, i.e. $d^* \leq p^*$.

Note: The panel (a) of the figure is this proof drawn: for the problem $\min x^2$ s.t. $x \geq 1$ (i.e. $h(x) = 1 - x \leq 0$), every Lagrangian curve $L(x, \lambda) = x^2 + \lambda(1 - x)$ lies on or below $J(x)$ — which is x^2 for $x \geq 1$ and ∞ elsewhere.

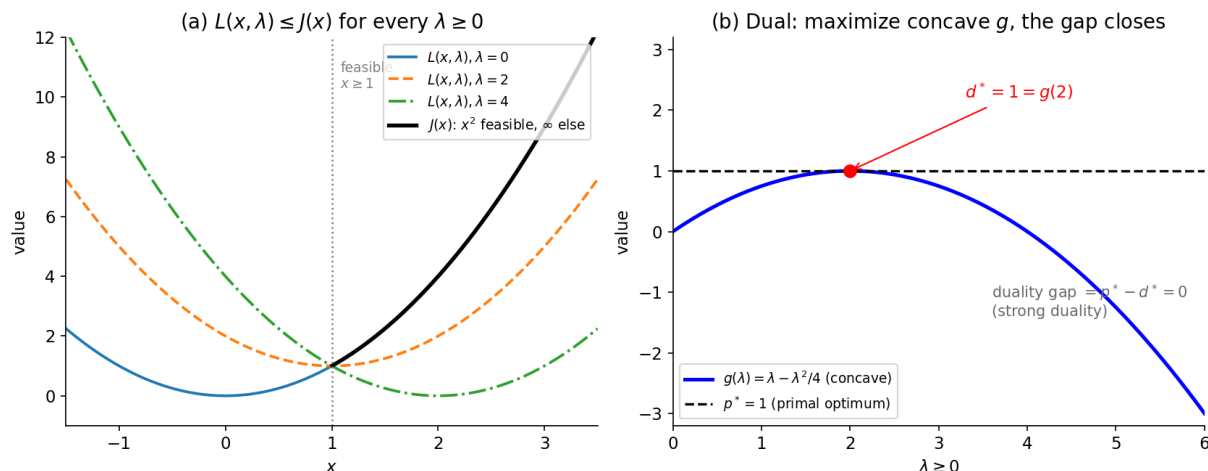


Figure 15: (a) For $f(x) = x^2$ with constraint $x \geq 1$, the Lagrangian curves $L(x, \lambda)$ for $\lambda = 0, 2, 4$ all lie on or below $J(x)$ (x^2 on the feasible set, infinity outside) — the picture behind weak duality. (b) The dual function $g(\lambda) = \lambda - \lambda^2/4$ is concave; its maximum $d^* = 1$ at $\lambda^* = 2$ equals the primal optimum $p^* = 1$, so the duality gap is zero (strong duality).

Corollary. If the primal is *unbounded below* ($p^* = -\infty$), the dual must be *infeasible* ($d^* = -\infty$); if the dual is *unbounded above* ($d^* = +\infty$), the primal must be *infeasible* ($p^* = +\infty$). (The tutorial’s primal/dual table says exactly this for LPs; it follows from $d^* \leq p^*$ in general.)

Basically, ... Weak duality is a free lunch with a catch: solve the (easier) dual and you get a *guaranteed lower bound* on the true optimum — but not the optimum itself. The gap between the bound and the truth is the duality gap, and nothing so far closes it.

13.7 Strong duality: for convex problems the gap closes ($d^* = p^*$)

Theorem (strong duality). If f and h are **convex**, then (up to some regularity conditions)

$$d^* = p^*.$$

Solve the dual, and you have solved the primal. The lectures state this as “a very, very powerful theorem” and do not prove it.

Note on the fine print. The lectures only ever say “up to some regularity conditions” / “modulo regularity” — they never name them. In the standard literature these are the *constraint qualifications*, the most famous being **Slater’s condition** (roughly: some strictly feasible point exists, i.e. some $\mathbf{x}$ with $h(\mathbf{x}) < 0$ strictly). For this book’s purposes — convex f , convex constraints, nothing pathological — strong duality holds, and we use it.

Basically, ... Weak duality says the dual’s answer is *at most* the primal’s answer. Strong duality says: if everything is convex (and the problem isn’t degenerate), the two answers are *equal*. The mirror reflects perfectly — so you may solve whichever side is easier.

13.8 The KKT conditions, derived from strong duality

Assume f, h convex, strong duality holds, and $\mathbf{x}^*, \lambda^*$ are the primal/dual optima. What *must* this pair satisfy? The lectures derive two non-trivial conditions (call them C1, C2) plus the two obvious ones.

C1 — stationarity. By strong duality, $f(\mathbf{x}^*) = g(\lambda^*) = \min_{\mathbf{x}} [f(\mathbf{x}) + \lambda^* h(\mathbf{x})]$. So $\mathbf{x}^*$ minimizes the *unconstrained* function $L(\cdot, \lambda^*)$ — and an unconstrained minimizer has zero gradient (§10.8):

$$\boxed{\nabla f(\mathbf{x}^*) + \lambda^* \nabla h(\mathbf{x}^*) = \mathbf{0}}. \quad (\text{a})$$

C2 — complementary slackness. The same equality, read differently: $f(\mathbf{x}^*) = \min_{\mathbf{x}} [f(\mathbf{x}) + \lambda^* h(\mathbf{x})]$. The minimum over *all* $\mathbf{x}$ is $\leq$ the value at any particular $\mathbf{x}$ — in particular at $\mathbf{x}^*$ itself:

$$f(\mathbf{x}^*) \leq f(\mathbf{x}^*) + \lambda^* h(\mathbf{x}^*).$$

But $\lambda^* \geq 0$ and $h(\mathbf{x}^*) \leq 0$ (dual and primal feasibility), so the extra term is ≤ 0 , i.e. the right side is $\leq f(\mathbf{x}^*)$. Sandwiched between $f(\mathbf{x}^*)$ and $f(\mathbf{x}^*)$, it must *equal* $f(\mathbf{x}^*)$ — forcing the extra term to zero:

$$\boxed{\lambda^* h(\mathbf{x}^*) = 0}. \quad (\text{b})$$

The remaining two are the feasibility conditions we already knew:

$$\boxed{h(\mathbf{x}^*) \leq 0} \quad [\text{primal feasibility}] \quad (\text{c})$$

$$\boxed{\lambda^* \geq 0} \quad [\text{dual feasibility}] \quad (\text{d})$$

These four are the **Karush–Kuhn–Tucker (KKT) conditions**. The lectures name them after the three people who formulated them.

Basically, ... (a) says: at the optimum, the Lagrangian looks flat — same first-order idea as $\nabla f = 0$, but for the penalized objective. (b) says: either the constraint is *tight* ($h = 0$) or its multiplier is *off* ($\lambda = 0$) — you don't pay a fine on a rule you weren't breaking. (c) and (d) say: the answer must be feasible, and the fine must be non-negative.

13.9 The general KKT form: m inequalities, n equalities

The one-constraint derivation was for exposition. For the general problem

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{s.t.} \quad h_i(\mathbf{x}) \leq 0 \quad (i = 1, \dots, m), \quad l_j(\mathbf{x}) = 0 \quad (j = 1, \dots, n),$$

give every constraint its own multiplier: $u_i \geq 0$ for the inequalities, v_j (free sign) for the equalities. The Lagrangian is

$$L(\mathbf{x}, \mathbf{u}, \mathbf{v}) = f(\mathbf{x}) + \sum_{i=1}^m u_i h_i(\mathbf{x}) + \sum_{j=1}^n v_j l_j(\mathbf{x}).$$

The same argument goes through, and the KKT conditions read:

- (a) **Stationarity:** $\nabla f(\mathbf{x}^*) + \sum_{i=1}^m u_i^* \nabla h_i(\mathbf{x}^*) + \sum_{j=1}^n v_j^* \nabla l_j(\mathbf{x}^*) = \mathbf{0}$ (the zero *vector*).
- (b) **Complementary slackness:** $u_i^* h_i(\mathbf{x}^*) = 0$ for every i (only the inequalities need it — for equalities $l_j(\mathbf{x}^*) = 0$ already).
- (c) **Primal feasibility:** $h_i(\mathbf{x}^*) \leq 0$ for every i ; $l_j(\mathbf{x}^*) = 0$ for every j .
- (d) **Dual feasibility:** $u_i^* \geq 0$ for every i (the v_j^* are unrestricted).

Note: With no inequality constraints this collapses to §11.3's Lagrange conditions (stationarity only) — the KKT conditions are their proper generalization.

13.10 Necessary, sufficient — and the certificate

The lectures are careful about *which direction* the KKT conditions work:

- i) **Necessity (convex case).** If f and the constraints are convex (modulo regularity), every optimal $(\mathbf{x}^*, \mathbf{u}^*, \mathbf{v}^*)$ must satisfy KKT — that is what §13.8 proved.
- ii) **Sufficiency — read the fine print.** The lectures assert that KKT $\Rightarrow$ local optimum “irrespective of convex or not” (asserted, not proved). That is overstated: without convexity a KKT point can be a saddle or even a local *maximum*. Counterexample: $\min_x -x^2$ s.t. $x \leq 0$ — stationarity ($-2x + u = 0$), complementary slackness ($ux = 0$), primal feasibility ($x \leq 0$), and dual feasibility ($u \geq 0$) all hold at $(x^*, u^*) = (0, 0)$, yet $x = 0$ is a local *maximum* ($f(-0.1) = -0.01 < f(0) = 0$). What is standard: under the regularity conditions, KKT is *necessary* for a local optimum — even without convexity; but *sufficiency* needs convexity, which is exactly what (iii) supplies.
- iii) **Both (convex case).** For convex problems, (i) + (ii) + §12.9 (every local minimum of a convex function is global) combine: the KKT conditions are **necessary and sufficient** for global optimality. The transcript’s words: for convex problems the KKT conditions “completely characterize the optimal solution”; in the KKT-continued lecture, they are “both necessary and sufficient for most convex optimization problems”.

This makes KKT a **certificate**: hand me a candidate $(\mathbf{x}^*, \mathbf{u}^*)$ for a *convex* problem; if it passes the four checks, I *know* it is optimal — I never had to run an optimizer. Compare §10.8: there $\nabla f = \mathbf{0}$ was only a *necessary* condition (it also holds at maxima and saddles). For convex constrained problems, KKT upgrades “suspect” to “convicted”.

Basically, ... The KKT conditions are the constrained version of “ $\nabla f = 0$ means you’re at the bottom” — except better: for convex problems they don’t just *suggest* optimality, they *prove* it. Find numbers satisfying (a)–(d) and the case is closed.

13.11 Reading complementary slackness: active vs. inactive

Condition (b), $u_i^* h_i(\mathbf{x}^*) = 0$, is the formal version of §11.7’s active/inactive discussion:

- i) If $u_i^* > 0$ (the multiplier is “on”), then $h_i(\mathbf{x}^*) = 0$ — the constraint is **active** (binding): the optimum sits exactly on that constraint’s boundary.
- ii) If $h_i(\mathbf{x}^*) < 0$ (strictly inside — **inactive**), then $u_i^* = 0$ — the multiplier is “off”: the constraint might as well not exist.
- iii) Both zero is allowed: a constraint can be exactly tight yet carry no penalty.

So at the optimum, *inactive constraints drop out of the Lagrangian* — only the active ones shape the answer. This is why the tutorial’s LP below only ever uses two of its four constraints.

Basically, ... Think of u_i^* as the price you’d pay to relax constraint i a little. A slack constraint has price zero — loosening a rule you’re not even touching changes nothing. A binding constraint can have a positive price — it is genuinely holding the optimum back.

13.12 eg 1 — the tutorial’s LP, solved end-to-end with KKT (full steps)

The Week 9 tutorial’s canonical example:

$$\min_{x_1, x_2} 3x_1 + x_2 \quad \text{s.t.} \quad x_1 - x_2 + 4 \leq 0, \quad -3x_1 + 2x_2 + 10 \leq 0, \quad x_1, x_2 \geq 0.$$

- i) **Standard form.** $h_1 = x_1 - x_2 + 4 \leq 0$, $h_2 = -3x_1 + 2x_2 + 10 \leq 0$, $h_3 = -x_1 \leq 0$, $h_4 = -x_2 \leq 0$, with multipliers $u_1, \dots, u_4 \geq 0$.
- ii) **Lagrangian.** $L = 3x_1 + x_2 + u_1(x_1 - x_2 + 4) + u_2(-3x_1 + 2x_2 + 10) - u_3x_1 - u_4x_2$.

iii) **Stationarity.**

$$\frac{\partial L}{\partial x_1} = 3 + u_1 - 3u_2 - u_3 = 0, \quad \frac{\partial L}{\partial x_2} = 1 - u_1 + 2u_2 - u_4 = 0. \quad (1, 2)$$

iv) **Guess which constraints are active.** Try h_1, h_2 active (i.e. $u_1, u_2 > 0$) and the non-negativity constraints slack ($u_3 = u_4 = 0$; we will verify). Then (1, 2) become:

$$3 + u_1 - 3u_2 = 0, \quad 1 - u_1 + 2u_2 = 0.$$

From the second: $u_1 = 1 + 2u_2$. Substitute into the first: $3 + (1 + 2u_2) - 3u_2 = 0$, so $4 - u_2 = 0$, giving $\boxed{u_2 = 4}$ and $\boxed{u_1 = 9}$. Dual feasibility: $9 \geq 0, 4 \geq 0$.

v) **Complementary slackness.** $u_1 > 0 \Rightarrow h_1 = 0$: $x_1 - x_2 + 4 = 0$, so $x_1 = x_2 - 4$. $u_2 > 0 \Rightarrow h_2 = 0$: $-3x_1 + 2x_2 + 10 = 0$. Substitute: $-3(x_2 - 4) + 2x_2 + 10 = 0$, i.e. $-x_2 + 22 = 0$, so $\boxed{x_2 = 22}$ and $\boxed{x_1 = 18}$.

vi) **Verify the guess.** Primal feasibility: $h_1 = 18 - 22 + 4 = 0 \leq 0$; $h_2 = -54 + 44 + 10 = 0 \leq 0$; $x_1 = 18 \geq 0, x_2 = 22 \geq 0$ (so $u_3 = u_4 = 0$ was consistent: $0 \cdot (-18) = 0, 0 \cdot (-22) = 0$). Complementary slackness: $9 \cdot 0 = 0, 4 \cdot 0 = 0$.

vii) **Optimum.** $\boxed{f^* = 3 \cdot 18 + 22 = 76}$. The problem is an LP — hence convex — so KKT is necessary and sufficient: this *is* the global optimum, certified without any search.

Note: The solution strategy is always this two-step dance: *guess* the active set (which $u_i > 0$), *solve* the resulting equations, then *verify* all four KKT conditions. A wrong guess fails verification — that is the method telling you to try another active set (Problem 8 shows a failed guess).

13.13 eg 2 — a quadratic with an inequality constraint, primal *and* dual (full steps)

$$\min_x x^2 \quad \text{s.t.} \quad x \geq 1 \iff h(x) = 1 - x \leq 0.$$

- i) **KKT.** $L(x, u) = x^2 + u(1 - x)$. Stationarity: $2x - u = 0$. Complementary slackness: $u(1 - x) = 0$. Primal feasibility: $1 - x \leq 0$. Dual feasibility: $u \geq 0$.
- ii) **Solve.** If $u = 0$, stationarity gives $x = 0$, but $1 - 0 = 1 \not\leq 0$ — violates primal feasibility. So $u > 0$, hence $1 - x = 0$: $\boxed{x^* = 1}, \boxed{u^* = 2}$ (from $2x = u$). Check: $2(1 - 1) = 0$, $1 - 1 = 0 \leq 0$, $2 \geq 0$. $\boxed{f^* = 1}$.
- iii) **The dual, explicitly.** $g(u) = \min_x [x^2 + u(1 - x)]$: unconstrained min at $x = u/2$, so $g(u) = \frac{u^2}{4} + u(1 - \frac{u}{2}) = u - \frac{u^2}{4}$ — concave, as §13.5 promised (panel (b) of the figure). $\max_{u \geq 0} g(u)$: $g'(u) = 1 - u/2 = 0$ gives $u^* = 2, d^* = g(2) = 2 - 1 = 1$.
- iv) **The gap closes.** $d^* = 1 = p^*$ — strong duality in numbers, since f is convex ($f'' = 2 > 0$, §12.7) and h is linear.

Basically, ... Without the constraint the answer would be $x = 0$; the constraint $x \geq 1$ drags the optimum to the boundary $x = 1$, and the multiplier $u^* = 2$ is exactly the “price” of that drag — relax the constraint a touch and the objective improves at rate 2.

13.14 eg 3 — the diet problem: a primal/dual pair from the tutorial (full steps)

The tutorial’s interpretation example. Primal (minimize cost subject to nutrition requirements):

$$\min_{x_1, x_2} 50x_1 + 80x_2 \quad \text{s.t.} \quad 3x_1 \geq 6, \quad 2x_1 + 4x_2 \geq 10, \quad 2x_1 + 5x_2 \geq 8, \quad x_1, x_2 \geq 0.$$

Its **dual** (given in the tutorial):

$$\max_{y_1, y_2, y_3} 6y_1 + 10y_2 + 8y_3 \quad \text{s.t.} \quad 3y_1 + 2y_2 + 2y_3 \leq 50, \quad 4y_2 + 5y_3 \leq 80, \quad y_1, y_2, y_3 \geq 0.$$

- i) **Primal optimum.** From $3x_1 \geq 6$: $x_1 \geq 2$. The first two constraints bind: $x_1 = 2$, $2(2) + 4x_2 = 10 \Rightarrow x_2 = 1.5$. Check the third: $4 + 7.5 = 11.5 \geq 8$. Value: $50(2) + 80(1.5) = 100 + 120 = 220$.
- ii) **Dual optimum.** Complementary slackness suggests: primal constraints 1, 2 binding and $x_1, x_2 > 0$, so try dual constraints binding with $y_3 = 0$ (primal constraint 3 slack). Then $3y_1 + 2y_2 = 50$, $4y_2 = 80$: $y_2 = 20$, $3y_1 = 50 - 40 = 10$, $y_1 = 10/3$. Feasible. Value: $6(10/3) + 10(20) + 8(0) = 20 + 200 = 220$.
- iii) **Verdict.** $d^* = 220 = p^*$: strong duality again (LPs are convex problems). And the certificate argument (Problem 10): a primal-feasible point with value 220 and a dual-feasible point with value 220 *prove* each other optimal via weak duality — $220 \leq d^* \leq p^* \leq 220$.

Basically, ... The primal asks “cheapest menu meeting the diet”; the dual asks the mirror question. Both arrive at 220 — two different problems, one answer. That is strong duality you can taste.

13.15 Why this matters: SVMs, and the Part I closing shot

The lectures end the optimization part of the course with one machine-learning example — the **support vector machine** (Chapters 32–33 will build it properly):

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i \mathbf{w}^T \mathbf{x}_i \geq 1 \quad \forall i,$$

i.e. $h_i(\mathbf{w}) = 1 - y_i \mathbf{w}^T \mathbf{x}_i \leq 0$ for each datapoint $(\mathbf{x}_i, y_i)$.

- i) The objective is quadratic — convex (§12.7: its Hessian is the identity, eigenvalues $1 > 0$). The constraints are linear — convex. So **strong duality holds**: the primal and dual have the same optimal value.
- ii) The dual is therefore fair game — and the lectures promise it is the *better* game: solving the dual “will help one go from linear models to non-linear models in a very smooth way”, leading to **kernel methods** (Chapter 33).
- iii) The KKT conditions will do real work there: complementary slackness will tell us that only the *active* constraints matter — the datapoints sitting exactly on the margin. Those are the **support vectors**; every other point gets multiplier zero and drops out of the answer. (§13.11’s “price” intuition, weaponized.)

Basically, ... The SVM is a quadratic objective with linear constraints — exactly the kind of problem this chapter was built for. Duality lets us solve the mirror problem instead, and complementary slackness then reveals that only a few special datapoints (the support vectors) actually decide the answer.

Part I, in one paragraph. Unconstrained optimization (Ch 10): follow the gradient down; $\nabla f = \mathbf{0}$ marks the suspects. Constraints (Ch 11): the Lagrangian folds constraints into the objective; inequalities are active or inactive. Convexity (Ch 12): the bowl shape that makes every local minimum global. Duality and KKT (this chapter): every constrained problem has a concave mirror image; for convex problems the mirror agrees, and the four KKT conditions certify optimality outright. With this, Part I’s machinery is complete — Part II puts probability underneath it.

Problem set

1. For $\min_x x^2$ s.t. $x \geq 2$: write h , L , the primal and the dual function $g(\lambda)$. Solve the dual, report p^* , d^* and the gap, and verify all four KKT conditions at the optimum.
2. For the tutorial LP of §13.12, compute each complementary-slackness product $u_i^* h_i(\mathbf{x}^*)$ numerically and each primal-feasibility residual $h_i(\mathbf{x}^*)$, and confirm stationarity holds — i.e. verify KKT by hand at $((18, 22), (9, 4, 0, 0))$.

3. Diet problem (§13.14): (a) check $(x_1, x_2) = (2, 1.5)$ is primal feasible with value 220; (b) check $(y_1, y_2, y_3) = (10/3, 20, 0)$ is dual feasible with value 220; (c) use weak duality to prove both are optimal and $p^* = d^* = 220$.
4. Prove the dual function $g(\lambda) = \min_{\mathbf{x}} L(\mathbf{x}, \lambda)$ is concave (the lecturer's exercise) — see §13.5 for the skeleton; write every step.
5. For $\min_x (x-3)^2$ s.t. $x \leq 1$: compute $\max_{\lambda \geq 0} L(x, \lambda)$ for (i) $x = 0$ and (ii) $x = 2$, and verify each equals $J(x)$ from §13.3.
6. For $\min_{x,y} x^2 + y^2$ s.t. $x + y \geq 1$: (a) write h and L ; (b) compute $g(\lambda)$ and d^* ; (c) state the weak-duality lower bound on p^* ; (d) solve the primal (hint: Cauchy–Schwarz) and check whether the gap closes.
7. KKT verification: for $\min_x x^2$ s.t. $1 - x \leq 0$, check all four KKT conditions at $(x^*, u^*) = (1, 2)$, one line each.
8. Someone proposes $(x, u) = (0, 0)$ for the problem in Problem 7. Test each KKT condition — which one(s) fail? What does this say about trusting stationarity alone (cf. §10.8)?
9. From scratch: $\min_{x_1, x_2} 2x_1 + 3x_2$ s.t. $x_1 + x_2 \geq 4$, $x_1, x_2 \geq 0$. (a) Put it in standard form $h_i \leq 0$. (b) Write L and all four KKT conditions. (c) Guess the active set, solve for $(\mathbf{x}^*, \mathbf{u}^*)$, verify, and report f^* .
10. The certificate theorem: for a *convex* problem, prove that any $(\mathbf{x}^*, \mathbf{u}^*)$ satisfying all KKT conditions must satisfy $p^* = d^* = f(\mathbf{x}^*)$. (Hint: chain $f(\mathbf{x}^*) = g(\mathbf{u}^*) \leq d^* \leq p^* \leq f(\mathbf{x}^*)$, justifying each link.)

Solutions — 13. Duality and KKT conditions

1. $\min_x x^2$ s.t. $x \geq 2$.

$h(x) = 2 - x \leq 0$. Lagrangian: $L(x, \lambda) = x^2 + \lambda(2 - x)$, $\lambda \geq 0$. Primal: $\min_x \max_{\lambda \geq 0} L(x, \lambda)$; geometrically the answer is $x^* = 2$, $p^* = 4$. Dual function: $g(\lambda) = \min_x [x^2 + \lambda(2 - x)]$. Unconstrained min: $\frac{d}{dx} = 2x - \lambda = 0 \Rightarrow x = \lambda/2$. So

$$g(\lambda) = \frac{\lambda^2}{4} + \lambda \left(2 - \frac{\lambda}{2}\right) = 2\lambda - \frac{\lambda^2}{4}.$$

Dual: $\max_{\lambda \geq 0} g(\lambda)$: $g'(\lambda) = 2 - \lambda/2 = 0 \Rightarrow \lambda^* = 4 \geq 0$; $g'' = -1/2 < 0$ so this is the max. $d^* = g(4) = 8 - 4 = 4$. Gap: $p^* - d^* = 4 - 4 = 0$ — strong duality (convex: $f'' = 2 > 0$, h linear). KKT at $(2, 4)$: stationarity $2(2) - 4 = 0$; CS $4(2 - 2) = 0$; PF $2 - 2 = 0 \leq 0$; DF $4 \geq 0$.

2. Tutorial LP at $\mathbf{x}^* = (18, 22)$, $\mathbf{u}^* = (9, 4, 0, 0)$, with $h_1 = x_1 - x_2 + 4$, $h_2 = -3x_1 + 2x_2 + 10$, $h_3 = -x_1$, $h_4 = -x_2$.

Residuals: $h_1 = 18 - 22 + 4 = 0$; $h_2 = -54 + 44 + 10 = 0$; $h_3 = -18$; $h_4 = -22$. All ≤ 0 — primal feasibility. CS products: $u_1 h_1 = 9 \cdot 0 = 0$; $u_2 h_2 = 4 \cdot 0 = 0$; $u_3 h_3 = 0 \cdot (-18) = 0$; $u_4 h_4 = 0 \cdot (-22) = 0$. Stationarity: $\frac{\partial L}{\partial x_1} = 3 + u_1 - 3u_2 - u_3 = 3 + 9 - 12 - 0 = 0$; $\frac{\partial L}{\partial x_2} = 1 - u_1 + 2u_2 - u_4 = 1 - 9 + 8 - 0 = 0$. Dual feasibility: $9, 4, 0, 0 \geq 0$. All four KKT conditions hold — and since an LP is convex, $(18, 22)$ is certified globally optimal with $f^* = 76$.

3. Diet problem.

- (a) Primal feasibility of $(2, 1.5)$: $3(2) = 6 \geq 6$; $2(2) + 4(1.5) = 10 \geq 10$; $2(2) + 5(1.5) = 11.5 \geq 8$; $2, 1.5 \geq 0$. Value: $50(2) + 80(1.5) = 220$.
- (b) Dual feasibility of $(10/3, 20, 0)$: $3(10/3) + 2(20) + 2(0) = 10 + 40 = 50 \leq 50$; $4(20) + 5(0) = 80 \leq 80$; all ≥ 0 . Value: $6(10/3) + 10(20) + 8(0) = 20 + 200 = 220$.
- (c) For a minimization primal, any feasible point upper-bounds the optimum: $p^* \leq 220$. For the maximization dual, any feasible point lower-bounds it: $d^* \geq 220$. Weak duality: $d^* \leq p^*$. Chain: $220 \leq d^* \leq p^* \leq 220$ — equality throughout, so $p^* = d^* = 220$ and both points are optimal. This is the certificate argument in action: matching feasible values prove optimality on both sides.

4. Concavity of g (full steps). Fix $\theta \in [0, 1]$, $\lambda_1, \lambda_2 \geq 0$.

$$g(\theta\lambda_1 + (1-\theta)\lambda_2) = \min_{\mathbf{x}} L(\mathbf{x}, \theta\lambda_1 + (1-\theta)\lambda_2) = \min_{\mathbf{x}} [\theta L(\mathbf{x}, \lambda_1) + (1-\theta)L(\mathbf{x}, \lambda_2)],$$

using that L is affine in λ for fixed $\mathbf{x}$. Now for *every* $\mathbf{x}$: $\theta L(\mathbf{x}, \lambda_1) + (1-\theta)L(\mathbf{x}, \lambda_2) \geq \theta \min_{\mathbf{z}} L(\mathbf{z}, \lambda_1) + (1-\theta) \min_{\mathbf{z}} L(\mathbf{z}, \lambda_2) = \theta g(\lambda_1) + (1-\theta)g(\lambda_2)$, since each $L(\mathbf{x}, \cdot)$ is at least its own minimum and $\theta, 1-\theta \geq 0$. The

left side, minimized over $\mathbf{x}$, is still $\geq$ the right side (a constant in $\mathbf{x}$):

$$g(\theta\lambda_1 + (1-\theta)\lambda_2) \geq \theta g(\lambda_1) + (1-\theta)g(\lambda_2),$$

the concavity chord inequality.

5. $\min_x (x-3)^2$ s.t. $x \leq 1$: $h(x) = x-1 \leq 0$, $L(x, \lambda) = (x-3)^2 + \lambda(x-1)$, $\lambda \geq 0$.

- (i) $x = 0$: $L(0, \lambda) = 9 + \lambda(-1) = 9 - \lambda$. Over $\lambda \geq 0$ the max is at $\lambda = 0$: value 9. $J(0) = (0-3)^2 = 9$ since $0 \leq 1$ (feasible) .
- (ii) $x = 2$: $L(2, \lambda) = 1 + \lambda(1) = 1 + \lambda \rightarrow \infty$ as $\lambda \rightarrow \infty$. $J(2) = \infty$ since $2 > 1$ (infeasible) .

6. $\min_{x,y} x^2 + y^2$ s.t. $x + y \geq 1$.

- (a) $h(x, y) = 1 - x - y \leq 0$; $L = x^2 + y^2 + \lambda(1 - x - y)$, $\lambda \geq 0$.
- (b) $\frac{\partial L}{\partial x} = 2x - \lambda = 0$, $\frac{\partial L}{\partial y} = 2y - \lambda = 0 \Rightarrow x = y = \lambda/2$. Then $g(\lambda) = 2(\lambda^2/4) + \lambda(1 - \lambda) = \lambda - \lambda^2/2$. Dual $\max_{\lambda \geq 0} g$: $g' = 1 - \lambda = 0 \Rightarrow \lambda^* = 1$; $d^* = g(1) = 1 - 1/2 = 1/2$.
- (c) Weak duality: $p^* \geq d^* = 1/2$.
- (d) Primal: by Cauchy-Schwarz, $x^2 + y^2 \geq (x+y)^2/2 \geq 1/2$ on the feasible set (since $x+y \geq 1$). Equality at $x = y = 1/2$, which is feasible ($1/2 + 1/2 = 1 \geq 1$). So $p^* = 1/2 = d^*$ — gap 0. (Convex: Hessian $\text{diag}(2, 2) \succ 0$; h linear — strong duality applies.)

7. KKT at $(1, 2)$ for $\min x^2$ s.t. $1 - x \leq 0$: stationarity $2(1) - 2 = 0$; complementary slackness $2(1 - 1) = 0$; primal feasibility $1 - 1 = 0 \leq 0$; dual feasibility $2 \geq 0$.

8. Testing $(x, u) = (0, 0)$: stationarity $2(0) - 0 = 0$ (holds!); CS $0 \cdot (1 - 0) = 0$ (holds!); dual feasibility $0 \geq 0$ (holds!); primal feasibility $1 - 0 = 1 \leq 0$ **fails**. The proposal is not even feasible. Moral: stationarity alone certifies nothing — exactly §10.8's warning that first-order conditions need the full interrogation. Here the missing interrogation is feasibility, and KKT supplies it.

9. $\min 2x_1 + 3x_2$ s.t. $x_1 + x_2 \geq 4$, $x_1, x_2 \geq 0$.

- (a) Standard form: $h_1 = 4 - x_1 - x_2 \leq 0$, $h_2 = -x_1 \leq 0$, $h_3 = -x_2 \leq 0$.
- (b) $L = 2x_1 + 3x_2 + u_1(4 - x_1 - x_2) - u_2x_1 - u_3x_2$. KKT: stationarity $2 - u_1 - u_2 = 0$, $3 - u_1 - u_3 = 0$; CS $u_1(4 - x_1 - x_2) = 0$, $u_2x_1 = 0$, $u_3x_2 = 0$; PF the three $h_i \leq 0$; DF $u_1, u_2, u_3 \geq 0$.
- (c) Guess: h_1 active, $x_2 = 0$ (cheaper variable x_1 does all the work), so $u_3 = 0$ free, $u_1 > 0$, $u_2 = 0$ (since $x_1 > 0$ expected). Then stationarity: $2 - u_1 = 0 \Rightarrow u_1 = 2$; $3 - 2 - u_3 = 0 \Rightarrow u_3 = 1$. CS: $h_1 = 0 \Rightarrow x_1 + x_2 = 4$; with $x_2 = 0$: $\mathbf{x}^* = (4, 0)$. Verify: PF: $4 - 4 - 0 = 0 \leq 0$, $-4 \leq 0$, $0 \leq 0$. CS: $2(0) = 0$, $0 \cdot 4 = 0$, $1 \cdot 0 = 0$. DF: $2, 0, 1 \geq 0$. $f^* = 2(4) + 3(0) = 8$. (Sanity: on $x_1 + x_2 = 4$, $f = 2x_1 + 3(4 - x_1) = 12 - x_1$, minimized at the largest feasible $x_1 = 4$.) LP convex globally optimal.

10. Certificate theorem (full steps). Let the problem be convex (f convex, each h_i convex, equalities affine) and $(\mathbf{x}^*, \mathbf{u}^*)$ satisfy KKT.

- i) Primal feasibility gives $\mathbf{x}^*$ feasible, so $p^* \leq f(\mathbf{x}^*)$ (the min over the feasible set is at most the value at one feasible point).
- ii) For fixed $\mathbf{u}^* \geq 0$, $L(\cdot, \mathbf{u}^*) = f + \sum u_i^* h_i$ is convex (nonneg-weighted sum of convex functions). Stationarity $\nabla_{\mathbf{x}} L(\mathbf{x}^*, \mathbf{u}^*) = \mathbf{0}$ plus convexity $\mathbf{x}^*$ globally minimizes $L(\cdot, \mathbf{u}^*)$ (§12.10). Hence $g(\mathbf{u}^*) = \min_{\mathbf{x}} L(\mathbf{x}, \mathbf{u}^*) = L(\mathbf{x}^*, \mathbf{u}^*) = f(\mathbf{x}^*) + \sum_i u_i^* h_i(\mathbf{x}^*) = f(\mathbf{x}^*)$, using complementary slackness $u_i^* h_i(\mathbf{x}^*) = 0$.
- iii) $d^* = \max_{\mathbf{u} \geq 0} g(\mathbf{u}) \geq g(\mathbf{u}^*) = f(\mathbf{x}^*)$.
- iv) Chain: $f(\mathbf{x}^*) = g(\mathbf{u}^*) \leq d^* \leq p^* \leq f(\mathbf{x}^*)$, the middle inequality being weak duality. Equality throughout: $\boxed{p^* = d^* = f(\mathbf{x}^*)}$. So KKT alone — no optimizer run — certifies $\mathbf{x}^*$ primal-optimal, $\mathbf{u}^*$ dual-optimal, and zero duality gap.

Part II — Probability and Statistics

14. Probability basics: experiments, sample spaces, Bayes

Part I built the machinery of optimization: follow the gradient down, fold constraints into the Lagrangian, exploit convexity, certify with KKT (§13.15 closed the arc: “Part II puts probability underneath it”). Why underneath? Because machine learning runs on uncertainty — noisy measurements, random sampling, models that output beliefs rather than certainties. Before any of that machinery can touch data, we need a mathematical language for *chance*. That is Part II, and it starts here, at the very beginning: what does it mean to assign a number to how likely something is?

The Stats 2 lectures frame the whole course with a running example — IPL cricket, 2008–2020, 816 matches of ball-by-ball data. The first step is descriptive: average runs per powerplay over climb from 6.00 (Over 1) to 8.30 (Over 6), with standard deviations around 4. The next step is modeling: “Are there models that explain the observed patterns in the data? How will you simulate powerplay overs in an IPL innings?” Simulation needs probability — a “mathematical” theory that assigns chances to events: define the basic objects precisely, assume a few things (the **axioms**), and deduce everything else by logical proof. This chapter builds exactly that.

14.1 Experiments and outcomes

Def. An **experiment** = a process or phenomenon we wish to study statistically. An **outcome** = the result of the experiment, described in as much detail as necessary.

- i) Toss a coin: outcomes heads, tails.
- ii) Throw a die: outcomes 1, 2, 3, 4, 5, 6.
- iii) IPL powerplay: the outcome could be the full yaml file of ball-by-ball data — “lots of details”. What counts as an outcome depends on what you are studying.

Basically, ... The experiment is “what you do”; the outcome is “what happened”. The same real-world process can be a different experiment depending on how much detail you record — one delivery’s runs, or the whole match file.

14.2 The sample space

Def. The **sample space** S = the set of *all* outcomes of an experiment.

- i) Toss a coin: $S = \{\text{heads}, \text{tails}\}$.
- ii) Throw a die: $S = \{1, 2, 3, 4, 5, 6\}$.
- iii) IPL: runs scored off one delivery: $S = \{0, 1, 2, 3, \dots\}$. Winner of a season: $S = \{\text{CSK}, \text{MI}, \text{DC}, \dots\}$.
- iv) Urn with 3 red, 3 white, 3 blue marbles; draw one: $S = \{\text{red}, \text{white}, \text{blue}\}$.
- v) Draw a card from a shuffled pack: $S = \{\text{spades}, \text{hearts}, \text{diamonds}, \text{clubs}\} \times \{2, 3, \dots, 10, J, Q, K, A\}$ — a Cartesian product (§1.6).

Sample spaces come in three sizes: **finite** (a die); **countably infinite** (the number of coin tosses until the first head: $\{1, 2, 3, \dots\}$ — the outcomes can be listed first, second, third, ...); and **uncountably infinite** (the exact landing point of a thrown dart, or the exact rainfall in a year — a continuum, no listing possible).

Note: In practice you rarely write S down completely — for the IPL it is “too big to write down”. Quite often it is enough to *imagine* a sample space. But the lectures give good advice: when interpreting probabilities becomes confusing, thinking of the sample space can be of use.

Basically, ... The sample space is the guest list: every possible result, listed once. It can be tiny (2 outcomes), infinite but listable, or a continuum. You don’t always write the whole list — but you should always know the list exists.

14.3 Events: subsets of the sample space

Def. An **event** = a subset of the sample space. An event is said to have **occurred** if the actual outcome belongs to the subset.

- i) Coin: $S = \{H, T\}$ has $2^2 = 4$ events: $\emptyset, \{H\}, \{T\}, \{H, T\}$.
- ii) Die: $S = \{1, \dots, 6\}$ has $2^6 = 64$ events — e.g. “even number” = $\{2, 4, 6\}$, “multiple of 3” = $\{3, 6\}$.
- iii) Fisherman: “catch is more than 100 Kg”, “pomfret is in the catch” — perfectly good events, even though nobody wrote down S .

Note (the technical restriction). The lectures mention “a technical restriction on what subsets can be events”, ignored “for now, and pointed out later when necessary”. Its standard name: the events must form a **σ -algebra** — a collection closed under complements and countable unions, so anything you build with set operations stays an event. For every finite or countable sample space in this chapter, “every subset is an event” works fine; the restriction only bites for uncountable sample spaces (Chapter 16 will say more).

Basically, ... An event is a *question* about the outcome, phrased as a set: “did the outcome land in this set?” If yes, the event occurred. That’s all an event is.

14.4 Combining events: union, intersection, complement

Events are sets, so all of §1.5 applies:

- i) **Union** $A \cup B$ = “or”: outcomes in A or in B (or both). Die: “even or a multiple of 3” = $\{2, 3, 4, 6\}$.
- ii) **Intersection** $A \cap B$ = “and”: outcomes in both. Die: “even and a multiple of 3” = $\{6\}$.
- iii) **Complement** $A^c = S \setminus A$ = “not”: outcomes not in A . Die: $\text{even}^c = \text{odd} = \{1, 3, 5\}$. If A occurred, A^c did not, and vice versa.

Translating English into events is a core skill. The lectures’ IPL example: one over of six deliveries, each worth 0, 1, 2, 3, 4 or 6 runs. Let A = “no 4s in the over”, B = “no 6s in the over”, C = “exactly 20 runs scored”. Then $A \cup B$ = “the over had no 4s or no 6s”, $A \cap B$ = “no 4s and no 6s”. Complements follow **De Morgan’s laws**:

$$(A \cup B)^c = A^c \cap B^c, \quad (A \cap B)^c = A^c \cup B^c.$$

So $(A \cup B)^c$ = “at least one 4 *and* at least one 6”, and $(A \cap B)^c$ = “at least one 4 *or* at least one 6”. (Problem 3 asks whether $A \cap B \cap C$ can occur.)

eg 1. Let $S = \{a, b, c, d, e\}$, $E = \{a, b, e\}$, $F = \{b, c\}$. Then $E^c = \{c, d\}$, $F^c = \{a, d, e\}$, $E \cup F = \{a, b, c, e\}$, $E \cap F = \{b\}$, $E \setminus F = E \cap F^c = \{a, e\}$, $F \setminus E = \{c\}$.

Basically, ... “Or” = union, “and” = intersection, “not” = complement. De Morgan says: the opposite of “A or B” is “not A *and* not B” — the *and/or flip* when you negate.

14.5 Disjoint events and partitions

Def. Two events are **disjoint** (mutually exclusive) if $A \cap B = \emptyset$.

- i) Die: even and odd are disjoint. Fisherman: “more than 200 Kg” and “less than 50 Kg” are disjoint.
- ii) A and A^c are disjoint, and together they cover everything: $A \cup A^c = S$ — either A occurs or A^c occurs.
- iii) Events $E_1, E_2, \dots$ are (pairwise) disjoint if $E_i \cap E_j = \emptyset$ for every $i \neq j$. Drawing a card: spades, hearts, diamonds, clubs are disjoint.

Def. A **partition** of S = a collection of disjoint events whose union is S . The simplest partition is $\{B, B^c\}$; a finer one is B_1, B_2, B_3, B_4 (panel (b) of the figure).

Basically, ... Disjoint events can’t happen together. A partition chops the whole sample space into non-overlapping pieces that cover everything — like cutting a cake with no crumbs left over. Partitions are the scaffolding the law of total probability (§14.10) is built on.

14.6 The probability function and its three axioms

Def. Probability P = a function that assigns to each event a real number in $[0, 1]$, satisfying:

1. **Non-negativity:** $P(E) \geq 0$ for every event E .
2. **Normalization:** $P(S) = 1$.
3. **Countable additivity:** if $E_1, E_2, \dots$ are disjoint, then

$$P(E_1 \cup E_2 \cup \dots) = P(E_1) + P(E_2) + \dots$$

The sample space, the events, and P together are called a **probability space**. (The lectures present Axioms 2 and 3 as “the two axioms”, with the $[0, 1]$ codomain stated in the definition itself; written as three, these are Kolmogorov’s axioms — the standard form used everywhere.)

0 means the event cannot occur, 1 means it always occurs; a higher value means a higher chance. One warning from the lectures, worth keeping: this *meaning* of probability is not part of the mathematics. The theory assumes only the axioms — everything else is deduced by proof.

eg 2. Coin, $S = \{H, T\}$. Valid: $P(\emptyset) = 0$, $P(\{H\}) = p$, $P(\{T\}) = 1 - p$, $P(S) = 1$ for any $0 \leq p \leq 1$ — a *fair* coin is $p = 1/2$. **Invalid:** $P(\{H\}) = 0.5$, $P(\{T\}) = 0.6$: the singletons are disjoint with union S , so Axiom 3 forces $P(S) = 0.5 + 0.6 = 1.1 \neq 1$, contradicting Axiom 2.

Basically, ... The axioms are the rules of the game: chances can’t be negative, *something* must happen (total chance 1), and chances of non-overlapping events add up. Every formula in this chapter is a logical consequence of these three.

14.7 Consequences of the axioms

Four properties, each proved from the axioms alone (the lectures’ proofs, in the lectures’ order):

- i) **Empty set.** $P(\emptyset) = 0$. *Proof.* $\emptyset^c = S$, and $\emptyset, S$ are disjoint with $\emptyset \cup S = S$. By Axiom 3, $P(S) = P(\emptyset) + P(S)$, so $P(\emptyset) = 0$.
- ii) **Complement.** $P(E^c) = 1 - P(E)$. *Proof.* E, E^c are disjoint and $E \cup E^c = S$, so $1 = P(S) = P(E) + P(E^c)$.
- iii) **Subset / monotonicity.** If $E \subseteq F$ then $P(F) = P(E) + P(F \setminus E)$, hence $P(E) \leq P(F)$. *Proof.* E and $F \setminus E = F \cap E^c$ are disjoint with union F ; apply Axiom 3. (The lectures’ intuition check: if $A \subseteq B$, $P(B)$ “should be higher than” $P(A)$ — and it is.)
- iv) **Difference form.** $P(E) = P(E \cap F) + P(E \setminus F)$ (and symmetrically $P(F) = P(E \cap F) + P(F \setminus E)$). *Proof.* $E \cap F \subseteq E$; apply (iii), noting $E \setminus (E \cap F) = E \setminus F$.
- v) **Inclusion–exclusion (union formula).**

$$P(E \cup F) = P(E) + P(F) - P(E \cap F).$$

Proof. $E \cup F$ splits into *three* disjoint pieces — $E \setminus F$, $E \cap F$, $F \setminus E$ (panel (a) of the figure) — so by Axiom 3, $P(E \cup F) = P(E \setminus F) + P(E \cap F) + P(F \setminus E)$. Substituting $P(E \setminus F) = P(E) - P(E \cap F)$ and $P(F \setminus E) = P(F) - P(E \cap F)$ from (iv) gives the formula.

eg 3 (fishing town). Chance of catching more than 400 Kg of fish in a day: 35%; more than 500 Kg: 10%. Let $F = \text{“catch} > 400\text{”}$, $E = \text{“catch} > 500\text{”}$; then $E \subseteq F$. By (iii), $P(F \setminus E) = P(F) - P(E) = 0.35 - 0.10 = 0.25$. So the chance of catching *between* 400 and 500 Kg is 25%.

eg 4 (weather forecast). $P(\text{rain}) = 0.6$, $P(\text{max temp above } 30^\circ) = 0.7$, $P(\text{rain and temp above } 30^\circ) = 0.4$. By inclusion–exclusion, $P(\text{rain} \cup \text{hot}) = 0.6 + 0.7 - 0.4 = 0.9$. “No rain and temp below 30° ” is the complement, so $P = 1 - 0.9 = \text{0.10}$ — a 10% chance.

Basically, ... Never add probabilities of overlapping events naively — that double-counts the overlap. Inclusion–exclusion says: add both, then subtract the overlap once. And “A but not B” is just $P(A) - P(\text{both})$.

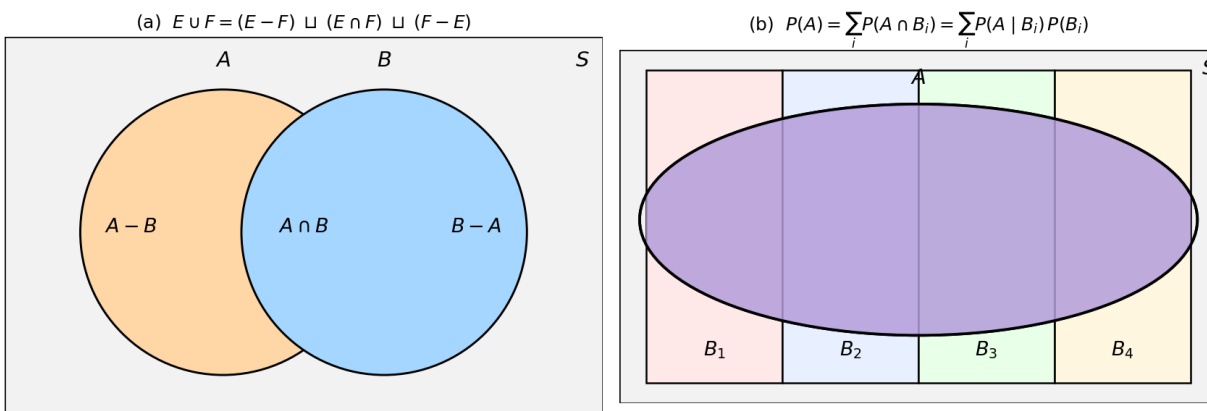


Figure 16: (a) Two events E and F inside the sample space S , split into the three disjoint pieces E minus F , E intersect F , F minus E — the picture behind inclusion–exclusion. (b) A partition $B_1..B_4$ of S with an event A overlaid; A is the disjoint union of the pieces A intersect B_i — the picture behind the law of total probability.

14.8 Equally likely outcomes and counting

When the sample space is countable, one can assign probabilities to *individual outcomes* — and for a finite S with symmetry, the natural choice is the **uniform distribution**: every outcome gets probability $1/|S|$, and

$$P(\text{event}) = \frac{\text{number of outcomes in the event}}{\text{number of outcomes in } S}.$$

Why this works: the $|S|$ singleton outcomes are disjoint with union S , so Axiom 3 forces their probabilities to sum to 1; “equally likely” makes them all equal, hence $1/|S|$ each.

eg 5 (two dice). $S =$ the 36 ordered pairs — a Cartesian product (§1.6). “Sum is 8”: $\{(2, 6), (3, 5), (4, 4), (5, 3), (6, 2)\}$ — 5 outcomes. $P = \text{5/36}$.

eg 6 (urn). 5 red and 8 blue marbles; draw one at random. S has 13 equally likely outcomes, so $P(\text{red}) = \text{5/13}$.

eg 7 (mixed-up hats, 3 persons). Three identical hats get mixed; each person picks one at random. $S =$ the $3! = 6$ orderings, equally likely. “Nobody gets their own hat” = the derangements $(2, 3, 1)$ and $(3, 1, 2)$ —

2 outcomes. $P = \boxed{2/6 = 1/3}$.

eg 8 (lost keys). An apartment key is lost; security hands over 50 possible keys, tried one after another until the match. The natural outcome is *which* key is the matching one: $S = \{1, 2, \dots, 50\}$. Is the uniform distribution reasonable? Yes — with no information distinguishing the keys, symmetry gives each $P = 1/50$. (The uniform choice is an *assumption* justified by symmetry; real information, like remembering the key was bent, would change it.)

Basically, ... “Picked at random” almost always means “every outcome equally likely”. Then probability is just counting: how many outcomes you like, divided by how many there are. The whole game is setting up the *right* sample space — the counting after that is mechanical.

14.9 Conditional probability

Many experiments run in *steps* — toss a coin three times, throw a die twice, one IPL delivery after another. The initial sample space for three tosses is $S = \{HHH, HHT, \dots, TTT\}$ (8 outcomes). Now suppose you *observe* the first toss: event $B =$ “first toss is tails” occurred. For the remaining tosses the relevant sample space shrinks to $B = \{THH, THT, TTH, TTT\}$ — and the events and probabilities must be redefined to account for B having occurred. This is the **conditional probability space given B** .

Def. For an event B with $P(B) > 0$, the **conditional probability of A given B** =

$$P(A | B) = \frac{P(A \cap B)}{P(B)}.$$

Equivalently, the **multiplication rule**: $P(A \cap B) = P(B) P(A | B)$. Read $P(A | B)$ as “the probability of A in the world where B is known to have occurred”.

Note: In ordinary language the word “conditional” is often dropped — “the probability both are girls, given at least one is a girl” is stated as if it were unconditional. Watch for the hidden “given”.

eg 9 (die, given even). Fair die, $E = \{2, 4, 6\}$, $P(E) = 1/2$. Then $P(\{2\} | E) = P(\{2\} \cap E)/P(E) = (1/6)/(1/2) = \boxed{1/3}$; likewise $P(\{4\} | E) = P(\{6\} | E) = 1/3$, while $P(\{1\} | E) = P(\emptyset)/P(E) = 0$. Also $P(\{2, 5\} | E) = 1/3$ and $P(\{2, 3, 4\} | E) = P(\{2, 4\})/P(E) = (1/3)/(1/2) = \boxed{2/3}$.

eg 10 (family with two children). $S = \{GG, GB, BG, BB\}$ (birth order), equally likely. $B =$ “at least one girl” = $\{GG, GB, BG\}$, $P(B) = 3/4$. $A =$ “both girls” = $\{GG\}$. Then $P(A | B) = (1/4)/(3/4) = \boxed{1/3}$ — *not* $1/2$. Conditioning shrinks the denominator, not just the numerator.

eg 11 (sequential draws: the multiplication chain). A class of 15 students: 4 from State 1, 8 from State 2, 3 from State 3. Three different students are chosen one after another. $P(\text{State 1, then State 3, then State 1})$:

$$\frac{4}{15} \cdot \frac{3}{14} \cdot \frac{3}{13} = \frac{36}{2730} = \boxed{\frac{6}{455} \approx 0.0132}.$$

Each factor is a conditional probability given the earlier draws — the multiplication rule applied twice. (Drawing *without* replacement changes the conditional probabilities at each step; that shrinking denominator is the whole content of “without replacement”.)

Basically, ... Conditioning = zooming in. Once B is known, throw away every outcome outside B , and rescale the survivors’ probabilities so they still sum to 1 — the rescaling factor is $1/P(B)$.

14.10 The law of total probability

A is the disjoint union of $A \cap B$ and $A \cap B^c$, so by Axiom 3 and the multiplication rule:

$$\boxed{P(A) = P(A \cap B) + P(A \cap B^c) = P(A | B) P(B) + P(A | B^c) P(B^c)}.$$

More generally, for a partition $B_1, B_2, \dots$ of S (panel (b) of the figure):

$$P(A) = \sum_i P(A \cap B_i) = \sum_i P(A | B_i) P(B_i).$$

The law computes an *unconditional* probability by cases: weigh each case's conditional probability by the case's own probability.

eg 12 (two urns). Pick an urn at random, then pick a marble from it. $P(\text{red} | \text{urn 1}) = 7/13$, $P(\text{red} | \text{urn 2}) = 5/13$, $P(\text{urn 1}) = P(\text{urn 2}) = 1/2$. Then

$$P(\text{red}) = \frac{7}{13} \cdot \frac{1}{2} + \frac{5}{13} \cdot \frac{1}{2} = \frac{12}{26} = \boxed{\frac{6}{13}}.$$

eg 13 (economic model). If interest rates rise, 60% chance unemployment rises; if not, 30% chance. The economist believes rates rise with 40% chance. $P(\text{unemployment rises}) = 0.6 \cdot 0.4 + 0.3 \cdot 0.6 = 0.24 + 0.18 = \boxed{0.42}$.

eg 14 (tricks with coins). 5 coins: 2 double-headed, 1 double-tailed, 2 normal. Pick one at random and toss it. Partitioning by which coin was picked:

$$P(\text{head}) = 1 \cdot \frac{2}{5} + 0 \cdot \frac{1}{5} + \frac{1}{2} \cdot \frac{2}{5} = \frac{2}{5} + \frac{1}{5} = \boxed{\frac{3}{5}}.$$

Basically, ... The law of total probability is “compute by cases”. You don’t know which urn, coin, or scenario you’re in, so you average over all possibilities — each weighted by how likely it is.

14.11 Bayes’ theorem

The law of total probability goes from *causes to effects* ($P(A | B)$ known, want $P(A)$). Bayes’ theorem goes *backwards* — from an observed effect to the probable cause:

$$P(B | A) = \frac{P(B) P(A | B)}{P(A)}, \quad P(A) > 0, P(B) > 0.$$

Proof. $P(A \cap B) = P(B) P(A | B) = P(A) P(B | A)$ — the multiplication rule written two ways. Divide by $P(A)$.

The denominator $P(A)$ is usually expanded by the law of total probability over $\{B, B^c\}$ or a finer partition — the two results always travel together.

eg 15 (two urns, reversed). Same setup as eg 12. A red marble was drawn — which urn did it likely come from?

$$P(\text{urn 1} | \text{red}) = \frac{P(\text{urn 1}) P(\text{red} | \text{urn 1})}{P(\text{red})} = \frac{\frac{1}{2} \cdot \frac{7}{13}}{\frac{6}{13}} = \boxed{\frac{7}{12}}.$$

Slightly above $1/2$ — urn 1 is redder, so a red draw is weak evidence for urn 1.

eg 16 (diagnostic test: the flu test, full steps). 1% of people in a city have Swine Flu. The test detects 95% of true cases ($P(\text{positive} | \text{flu}) = 0.95$), but 2% of healthy people also test positive ($P(\text{positive} | \text{no flu}) = 0.02$). A randomly chosen person tests positive. What is $P(\text{flu} | \text{positive})$?

- i) Name the events: B = “has Swine Flu”, A = “tests positive”. Given: $P(B) = 0.01$, $P(B^c) = 0.99$, $P(A | B) = 0.95$, $P(A | B^c) = 0.02$.
- ii) Denominator by the law of total probability:

$$P(A) = P(A | B)P(B) + P(A | B^c)P(B^c) = 0.95 \cdot 0.01 + 0.02 \cdot 0.99 = 0.0095 + 0.0198 = 0.0293.$$

iii) Bayes:

$$P(B | A) = \frac{0.01 \cdot 0.95}{0.0293} = \frac{0.0095}{0.0293} \approx \boxed{0.3242 = 32.42\%}.$$

The surprise is genuine: a 95%-accurate test, a positive result — and still only about one in three actually has the flu. Why? The disease is rare (1% base rate): the 2% false-positive rate, applied to the 99% healthy majority, produces *more* positives (0.0198) than the true cases do (0.0095). **Base rates dominate.** This is the single most important intuition Bayes' theorem teaches — and it recurs in medical testing, spam filters (Chapter 30), and Bayesian inference (Chapter 20).

eg 17 (die, then coins). Roll a fair die, then toss as many fair coins as the number shown. Given 5 heads were obtained, what is $P(\text{die showed } 5)$?

i) Let $D = \text{"die showed } 5\text{"}$, $F = \text{"5 heads obtained"}$. $P(D) = 1/6$. Given the die shows 5: $P(F | D) = \binom{5}{5}(1/2)^5 = 1/32$.

ii) Denominator: partition over die faces $k = 1, \dots, 6$. $P(F | k) = \binom{k}{5}(1/2)^k$ for $k \geq 5$, and 0 otherwise:

$$P(F) = \frac{1}{6} \left[\binom{5}{5} \left(\frac{1}{2}\right)^5 + \binom{6}{5} \left(\frac{1}{2}\right)^6 \right] = \frac{1}{6} \left[\frac{1}{32} + \frac{6}{64} \right] = \frac{1}{6} \cdot \frac{4}{32} = \frac{1}{48}.$$

iii) Bayes: $P(D | F) = \frac{(1/32)(1/6)}{1/48} = \frac{1/192}{1/48} = \boxed{\frac{1}{4}}.$

Basically, ... Bayes' theorem flips a conditional around: you know how likely the evidence is *under each hypothesis* (the easy direction — the test's specs), and you want how likely *each hypothesis is, given the evidence* (what you actually care about). The price of the flip is the denominator — the total probability of the evidence, computed by cases.

14.12 Independence

Motivation: if $P(B) > 0$ and $P(A | B) = P(A)$, then learning that B occurred tells you *nothing* about A — the conditional world agrees with the original on A .

Def. Events A and B are **independent** if

$$\boxed{P(A \cap B) = P(A)P(B)}.$$

Two rules of thumb from the lectures — keep them on different shelves: - Independent E_1, E_2 : **multiply** for “and”: $P(E_1 \cap E_2) = P(E_1)P(E_2)$. - Disjoint A, B : **add** for “or”: $P(A \cup B) = P(A) + P(B)$.

eg 18 (coin, three tosses). S = the 8 equally likely outcomes. $A = \text{"first toss heads"} = \{HHH, HHT, HTH, HTT\}$, $P(A) = 1/2$; $B = \text{"second toss heads"} = \{HHH, HHT, THH, THT\}$, $P(B) = 1/2$; $A \cap B = \{HHH, HHT\}$, $P = 1/4 = \frac{1}{2} \cdot \frac{1}{2}$ — independent, as intuition demands. (A and B^c are independent too: $A \cap B^c = \{HTH, HTT\}$, $P = 1/4$. No accident — see the theorem below.)

eg 19 (die: even and multiple of 3). $A = \{2, 4, 6\}$, $P(A) = 1/2$; $B = \{3, 6\}$, $P(B) = 1/3$; $A \cap B = \{6\}$, $P = 1/6 = \frac{1}{2} \cdot \frac{1}{3}$ — **independent**. Two typical confusions, answered: (1) yes, they have an intersection — independence is about *probabilities multiplying*, not about sets avoiding each other; (2) yes, with a single throw of one die — the two events are two *questions* about the same outcome, and the questions can be independent of each other.

eg 20 (card: spade and king). $P(\text{spade}) = 13/52 = 1/4$, $P(\text{king}) = 4/52 = 1/13$, $P(\text{spade-king}) = 1/52 = \frac{1}{4} \cdot \frac{1}{13}$ — independent, despite there being exactly one spade-king card. Same lesson as eg 19.

Disjoint $\neq$ independent — the crucial contrast. Die: $A = \text{even}$, $B = \text{odd}$. $A \cap B = \emptyset$, so $P(A \cap B) = 0 \neq \frac{1}{2} \cdot \frac{1}{2}$. Disjoint events of positive probability are *never* independent: if B occurs, A definitely did not — B 's occurrence maximally informs A . (Fine print: this needs $P(A), P(B) > 0$. The empty set is disjoint from

everything *and* independent of everything, since $P(\emptyset \cap B) = 0 = 0 \cdot P(B)$. A degenerate case worth knowing, not worth worrying about.)

Theorem (independence survives complementing). A, B independent $\Rightarrow A$ and B^c independent; applying twice, A^c and B^c independent. *Proof.* $P(A \cap B^c) = P(A \setminus B) = P(A) - P(A \cap B)$ (by §14.7(iv)) $= P(A) - P(A)P(B) = P(A)(1 - P(B)) = P(A)P(B^c)$. Intuition: if B tells you nothing about A , then “not B ” tells you nothing either.

Mutual independence (stronger than pairwise). Three events are **mutually independent** if every sub-collection multiplies:

$$P(A \cap B) = P(A)P(B), P(A \cap C) = P(A)P(C), P(B \cap C) = P(B)P(C), \text{ and } P(A \cap B \cap C) = P(A)P(B)P(C).$$

The last condition is *not* redundant. **eg 21:** toss a fair coin twice, $S = \{HH, HT, TH, TT\}$. $A = \{HH, TT\}$ (“tosses agree”), $B = \{HH, HT\}$ (“first is heads”), $C = \{HH, TH\}$ (“second is heads”). $P(A) = P(B) = P(C) = 1/2$; $A \cap B = A \cap C = B \cap C = \{HH\}$, each with $P = 1/4$ — pairwise independent. But $A \cap B \cap C = \{HH\}$, $P = 1/4 \neq 1/8 = P(A)P(B)P(C)$. Pairwise independent, **not** mutually independent.

For n events: mutually independent means the intersection of *any* sub-collection has probability equal to the product of the probabilities — “almost 2^n constraints”, as the lectures put it. And the complement theorem extends: any sub-collection, with or without complementing, stays independent.

Basically, ... Independence = “knowing one tells you nothing about the other”, made precise as “the ‘and’ probability is the product”. Disjoint is about *sets* (no overlap); independence is about *information* (no influence) — they are nearly opposites. And “pairwise independent” is weaker than “mutually independent”: always check the full intersection too.

14.13 Repeated independent trials

Repeat the same experiment n times independently — toss a coin n times, test n people for a disease, send n bits. Call one trial a success with probability p (failure $1 - p$); write 1 for success, 0 for failure.

- i) Sample space: all 2^n binary strings; e.g. $n = 3$: $S = \{000, 001, 010, 011, 100, 101, 110, 111\}$.
- ii) Probability of one *specific* sequence — multiply by independence. For $n = 3$: $P(000) = (1 - p)^3$, $P(101) = p(1 - p)p = p^2(1 - p)$. In general $P(b_1 \dots b_n) = p^w(1 - p)^{n-w}$ where w = the number of 1s. Only the *count* of successes matters, not their positions.
- iii) Fair coin, $n = 5$ ($p = 1/2$): all 32 sequences equally likely, each $1/32$. $P(0 \text{ tails}) = P(HHHHH) = 1/32$. $P(1 \text{ tail}) = P(\{THHHH, HTHHH, HHTHH, HHHTH, HHHTT\}) = 5/32$ — the five sequences are disjoint, so add (Axiom 3). $P(\text{at least 4 tails}) = P(4 \text{ tails}) + P(5 \text{ tails}) = 5/32 + 1/32 = 6/32$.
- iv) Biased coin, $n = 5$, $P(H) = 1/3$, $P(T) = 2/3$: $P(HHHHH) = (1/3)^5$, $P(TTTTT) = (2/3)^5$, and $P(1 \text{ tail}) = 5 \cdot \frac{2}{3} \left(\frac{1}{3}\right)^4$ — five disjoint sequences, each with probability $\frac{2}{3} \left(\frac{1}{3}\right)^4$.

Note: Counting “how many sequences have exactly k successes” in general, and naming the resulting distributions (Bernoulli, Binomial, Geometric), is Chapter 15’s job — this section only sets up the *machinery* (product sample spaces plus independence) that makes those distributions tick.

Basically, ... n independent trials = 2^n sequences, and each sequence’s probability is just the product of its trials. Add up the sequences you like (they’re disjoint). That is the entire engine behind every “repeated experiment” calculation.

14.14 Why this matters: the language every later chapter speaks

This chapter built the vocabulary — experiment, sample space, event, probability, conditioning, independence, Bayes — that the rest of Part II (and the ML chapters) speak fluently:

- i) **Bayes’ theorem** is the engine of *Bayesian inference*: Chapter 20’s MAP estimation is Bayes’ theorem with the parameters as the hypothesis; Chapter 30’s naive Bayes classifier is Bayes’ theorem with a strong independence assumption doing the heavy lifting.

- ii) **Independence** is the “iid” in “iid data” — the assumption, used in nearly every ML chapter, that datapoints are independent and identically distributed. §14.13’s repeated trials are the prototype.
- iii) **The law of total probability** becomes *marginalization*: summing (or integrating) out the variables you don’t care about — the central move of Chapters 17–18 on joint distributions.
- iv) **Conditional probability** is how ML models express uncertainty: $P(\text{label} \mid \text{features})$ is a conditional distribution, and Chapters 15–19 develop the random-variable machinery to work with it.

Part II, opening shot. Part I assumed the world hands you a function to optimize. The real world hands you *data* — noisy, partial, random. Probability is the mathematics of reasoning under that randomness, and statistics is its application: collect data, use probabilistic analysis, make inferences. The next chapter introduces the workhorse object of the whole enterprise: the **random variable** (Chapter 15).

Problem set

- (Restaurant hiring) Four applicants: David and Megha from Delhi, Rajesh and Veronica from Mumbai. One is hired at random as waiter, another from the remaining as cashier. (a) Write the sample space (12 ordered pairs). (b) Write as subsets: A = “cashier is from Delhi”, B = “exactly one position is filled by a Delhiite”, C = “neither position is filled by a Delhiite”. (c) Under the uniform distribution, find $P(A)$, $P(B)$, $P(C)$.
- (Mixed-up hats, 5 persons) Five identical hats get mixed; each person picks one at random. A = “no person gets their own hat”, B = “every person gets their own hat”, C = “at least one person does not get their own hat”, D = “at least one person gets their own hat”. (a) What are A^c and B^c ? (b) Are A and B disjoint? (c) What is $A \cap B^c$?
- (IPL over) One over, six deliveries, each worth 0, 1, 2, 3, 4 or 6 runs. A = “no 4s in the over”, B = “no 6s in the over”, C = “exactly 20 runs scored”. (a) Describe $A \cup B$ and $A \cap B$ in words. (b) Can $A \cap B \cap C$ occur? Justify. (c) Write $(A \cup B)^c$ and $(A \cap B)^c$ in words, using De Morgan’s laws.
- (Fishing town) $P(\text{catch} > 400 \text{ Kg}) = 0.35$, $P(\text{catch} > 500 \text{ Kg}) = 0.10$. Find $P(400 < \text{catch} \leq 500 \text{ Kg})$, naming the property used.
- (Weather forecast) $P(\text{rain}) = 0.6$, $P(\text{max temp above } 30^\circ) = 0.7$, $P(\text{both}) = 0.4$. Find $P(\text{no rain and temp below } 30^\circ)$.
- (Independence and complements) Prove: if A and B are independent, then A and B^c are independent, and hence A^c and B^c are independent.
- (Two urns) $P(\text{red} \mid \text{urn 1}) = 7/13$, $P(\text{red} \mid \text{urn 2}) = 5/13$, and the urn is picked at random. (a) Find $P(\text{red})$. (b) Find $P(\text{urn 1} \mid \text{red})$. (c) Find $P(\text{urn 2} \mid \text{blue})$.
- (Economic model) $P(\text{unemployment rises} \mid \text{rates rise}) = 0.6$, $P(\text{unemployment rises} \mid \text{rates don't rise}) = 0.3$, $P(\text{rates rise}) = 0.4$. Find $P(\text{unemployment rises})$.
- (Tricks with coins) 2 double-headed, 1 double-tailed, 2 normal coins; one is picked at random and tossed. Find $P(\text{head})$.
- (MCQ) A student knows the answer to a 4-choice MCQ with probability $3/4$; otherwise she guesses uniformly at random. Given she answered correctly, find $P(\text{she knew the answer})$.
- (Die, then coins) Roll a fair die, then toss as many fair coins as the number shown. Given 5 heads were obtained, find $P(\text{die showed 5})$.
- (Pairwise vs mutual) A fair coin is tossed twice; $A = \{HH, TT\}$, $B = \{HH, HT\}$, $C = \{HH, TH\}$. (a) Verify pairwise independence. (b) Show the three are not mutually independent.
- (Basketball free throws) Player 1 (a 40% free-throw shooter) and Player 2 (a 70% shooter) alternate throws, Player 1 first, until the first basket is scored; throws are independent. (a) Find $P(\text{Player 1 wins before the 3rd round})$. (b) Find $P(\text{Player 1 eventually wins})$.

Solutions — 14. Probability basics: experiments, sample spaces, Bayes

1. Restaurant hiring. Write D = David, M = Megha (Delhi); R = Rajesh, V = Veronica (Mumbai). Ordered pairs are (waiter, cashier).

- (a) $S = \{(D, M), (D, R), (D, V), (M, D), (M, R), (M, V), (R, D), (R, M), (R, V), (V, D), (V, M), (V, R)\}$ — $4 \cdot 3 = 12$ outcomes, equally likely.
- (b) $A = \text{cashier} \in \{D, M\}$: $\{(D, M), (M, D), (R, D), (R, M), (V, D), (V, M)\}$ — 6 outcomes. $B = \text{exactly one of the two positions is Delhi: waiter Delhi \& cashier Mumbai}$ $\{(D, R), (D, V), (M, R), (M, V)\}$ plus waiter Mumbai & cashier Delhi $\{(R, D), (R, M), (V, D), (V, M)\}$ — 8 outcomes. $C = \text{neither is Delhi:}$ $\{(R, V), (V, R)\}$ — 2 outcomes.
- (c) $P(A) = 6/12 = 1/2$; $P(B) = 8/12 = 2/3$; $P(C) = 2/12 = 1/6$.

2. Mixed-up hats, 5 persons.

- (a) $A^c = \text{"at least one person gets their own hat"} = D$. $B^c = \text{"at least one person does not get their own hat"} = C$.
- (b) Yes, disjoint. B is the single identity ordering (everyone keeps their own hat), which is not a derangement, so $A \cap B = \emptyset$.
- (c) Since $A \cap B = \emptyset$, $A \subseteq B^c$, so $A \cap B^c = A$.

3. IPL over.

- (a) $A \cup B$: “the over had no 4s or no 6s” (at least one of the two holds). $A \cap B$: “the over had no 4s and no 6s”.
- (b) No. Under $A \cap B$, every delivery scores at most 3 runs, so the over total is at most $6 \cdot 3 = 18 < 20$; C requires exactly 20. Hence $A \cap B \cap C = \emptyset$.
- (c) $(A \cup B)^c = A^c \cap B^c$: “the over had at least one 4 and at least one 6”. $(A \cap B)^c = A^c \cup B^c$: “the over had at least one 4 or at least one 6”.

4. Let $F = \text{"catch} > 400\text{"}$, $E = \text{"catch} > 500\text{"}$. Then $E \subseteq F$, and “between 400 and 500 Kg” is $F \setminus E$. By the subset property (§14.7(iii)): $P(F \setminus E) = P(F) - P(E) = 0.35 - 0.10 = \boxed{0.25}$ (25%).

5. Let $R = \text{rain}$, $H = \text{temp above } 30^\circ$. Inclusion–exclusion: $P(R \cup H) = 0.6 + 0.7 - 0.4 = 0.9$. “No rain and temp at most 30° ” = $(R \cup H)^c$, so by the complement property: $P = 1 - 0.9 = \boxed{0.10}$.

6. Given $P(A \cap B) = P(A)P(B)$. Then

$$P(A \cap B^c) = P(A \setminus B) = P(A) - P(A \cap B) = P(A) - P(A)P(B) = P(A)(1 - P(B)) = P(A)P(B^c),$$

using §14.7(iv) for $P(A \setminus B) = P(A) - P(A \cap B)$ and §14.7(ii) for $P(B^c) = 1 - P(B)$. So A and B^c are independent. Applying the same argument to the independent pair (A, B^c) : A^c and $(B^c)^c = B$ are independent; applying once more to (A^c, B) : A^c and B^c are independent.

7. Two urns, $P(\text{urn 1}) = P(\text{urn 2}) = 1/2$.

- (a) Law of total probability: $P(\text{red}) = P(\text{red} \mid \text{urn 1})P(\text{urn 1}) + P(\text{red} \mid \text{urn 2})P(\text{urn 2}) = \frac{7}{13} \cdot \frac{1}{2} + \frac{5}{13} \cdot \frac{1}{2} = \frac{12}{26} = \boxed{6/13}$.
- (b) Bayes: $P(\text{urn 1} \mid \text{red}) = \frac{P(\text{urn 1})P(\text{red} \mid \text{urn 1})}{P(\text{red})} = \frac{\frac{1}{2} \cdot \frac{7}{13}}{\frac{6}{13}} = \boxed{7/12}$.
- (c) $P(\text{blue} \mid \text{urn 1}) = 6/13$, $P(\text{blue} \mid \text{urn 2}) = 8/13$. $P(\text{blue}) = \frac{6}{13} \cdot \frac{1}{2} + \frac{8}{13} \cdot \frac{1}{2} = \frac{14}{26} = 7/13$. Bayes: $P(\text{urn 2} \mid \text{blue}) = \frac{\frac{1}{2} \cdot \frac{8}{13}}{\frac{7}{13}} = \boxed{4/7}$. (Sanity: $P(\text{urn 1} \mid \text{blue}) = 3/7$, and $3/7 + 4/7 = 1$.)

8. Let $U = \text{unemployment rises}$, $R = \text{rates rise}$. Law of total probability: $P(U) = P(U \mid R)P(R) + P(U \mid R^c)P(R^c) = 0.6 \cdot 0.4 + 0.3 \cdot 0.6 = 0.24 + 0.18 = \boxed{0.42}$.

9. Partition by coin type: $P(\text{double-headed}) = 2/5$, $P(\text{double-tailed}) = 1/5$, $P(\text{normal}) = 2/5$. $P(\text{head}) = 1 \cdot \frac{2}{5} + 0 \cdot \frac{1}{5} + \frac{1}{2} \cdot \frac{2}{5} = \frac{2}{5} + \frac{1}{5} = \boxed{3/5}$.

10. Let $K = \text{knows the answer}$, $C = \text{answered correctly}$. $P(K) = 3/4$, $P(K^c) = 1/4$, $P(C \mid K) = 1$, $P(C \mid K^c) = 1/4$. Denominator: $P(C) = 1 \cdot \frac{3}{4} + \frac{1}{4} \cdot \frac{1}{4} = \frac{3}{4} + \frac{1}{16} = \frac{13}{16}$. Bayes: $P(K \mid C) = \frac{(3/4) \cdot 1}{13/16} = \frac{3}{4} \cdot \frac{16}{13} =$

$$\boxed{12/13 \approx 0.923}.$$

11. Let D = “die showed 5”, F = “5 heads obtained”. $P(D) = 1/6$, $P(F | D) = \binom{5}{5}(1/2)^5 = 1/32$. Partition over die faces $k = 1, \dots, 6$: $P(F | k) = \binom{k}{5}(1/2)^k$ for $k \geq 5$, else 0. So $P(F) = \frac{1}{6} \left[\binom{5}{5}(1/2)^5 + \binom{6}{5}(1/2)^6 \right] = \frac{1}{6} \left[\frac{1}{32} + \frac{6}{64} \right] = \frac{1}{6} \cdot \frac{4}{32} = \frac{1}{48}$. Bayes: $P(D | F) = \frac{(1/32)(1/6)}{1/48} = \frac{1/192}{1/48} = \boxed{1/4}$.

12. $S = \{HH, HT, TH, TT\}$, uniform, so each singleton has $P = 1/4$.

- (a) $P(A) = P(\{HH, TT\}) = 1/2$, $P(B) = P(\{HH, HT\}) = 1/2$, $P(C) = P(\{HH, TH\}) = 1/2$. $A \cap B = \{HH\}$, $P = 1/4 = \frac{1}{2} \cdot \frac{1}{2}$; $A \cap C = \{HH\}$, $P = 1/4$; $B \cap C = \{HH\}$, $P = 1/4$. All three pairs independent.
- (b) $A \cap B \cap C = \{HH\}$, $P = 1/4 \neq 1/8 = P(A)P(B)P(C)$. The triple-product condition fails, so A, B, C are pairwise but not mutually independent.

13. Let $q_1 = 0.4$ (Player 1 scores), $q_2 = 0.7$ (Player 2 scores); misses 0.6, 0.3.

- (a) “Wins before the 3rd round” = wins in round 1 or round 2. Round 1: $P = 0.4$. Round 2: Player 1 misses, Player 2 misses, Player 1 scores: $0.6 \cdot 0.3 \cdot 0.4 = 0.072$. Total: $0.4 + 0.072 = \boxed{0.472}$.
- (b) Player 1 wins in round r iff both miss the first $r - 1$ rounds and he scores in round r : $P_r = (0.6 \cdot 0.3)^{r-1} \cdot 0.4 = 0.4 \cdot 0.18^{r-1}$ (rounds are disjoint events). Sum the geometric series: $\sum_{r=1}^{\infty} 0.4 \cdot 0.18^{r-1} =$

$$\frac{0.4}{1 - 0.18} = \frac{0.4}{0.82} = \boxed{\frac{20}{41} \approx 0.4878}.$$

15. Discrete random variables and key distributions

Chapter 14 ended with a deliberate cliffhanger: §14.13 set up n repeated independent trials — 2^n sequences, each sequence's probability a product of its trials — and then said the general counting and the *names* of the resulting distributions are this chapter's job. The setup was complete; the language was missing. That language is the **random variable**.

The motivation comes from the same place as before. The Stats 2 lectures open with IPL cricket — ball-by-ball data, runs per over, wickets per innings. An outcome of “one powerplay over” is a huge, messy object (six deliveries, wides, no-balls, runs, wickets). But what you actually want to *study* is usually a number computed from that outcome: how many runs? how many wickets? Those numbers are random variables, and this chapter builds their entire toolkit: what they are, how probabilities attach to them (the PMF and CDF), how to average them (expectation) and measure their spread (variance), and the seven named discrete distributions that model almost every “repeated experiment” you will ever meet.

15.1 Random variables: functions on the sample space

Def. A **random variable** = a function whose domain is the sample space S of an experiment and whose range is the real numbers — a function from the sample space to the real line.

Not “random” (it is a fixed function) and not quite a “variable” (it is a mapping). The name is historical; the object is precise.

- i) Toss a coin, $S = \{H, T\}$. $X(H) = 0$, $X(T) = 1$ is a random variable. So is $Y(H) = -10$, $Y(T) = 200$. So is $Z(H) = \sqrt{2}$, $Z(T) = \pi$. So is $U(H) = 0$, $U(T) = 0$. All valid — though in practice, *meaningful* functions are the ones studied.
- ii) Throw a die, $S = \{1, 2, 3, 4, 5, 6\}$. Pick any six real numbers $x_1, \dots, x_6$ and set $X(i) = x_i$: a random variable. If the x_i are distinct, X is one-to-one — essentially a relabeling of the sample space. They need not be distinct: $E(2) = E(4) = E(6) = 1$, $E(1) = E(3) = E(5) = 0$ is a perfectly good random variable (“even indicator”).

Random variables turn questions about outcomes into questions about numbers. For a random variable X :

- i) $(X < x) = \{s \in S : X(s) < x\}$ is an event, for every real x . Likewise $(X > x)$, $(X = x)$, $(X \leq x)$, $(X \geq x)$ are all events.
- ii) This is the “technical condition” the lectures mention: a function counts as a random variable only if every such set is genuinely an event (for finite and countable S , where every subset is an event, it is automatic).
- iii) Not every event can be expressed through every random variable. With the even-indicator E above: $(E = 0) = \{1, 3, 5\}$, $(E = 1) = \{2, 4, 6\}$ — but the event $\{2\}$ cannot be written using E alone. A coarser random variable sees less.

Why random variables at all? One IPL powerplay over supports many: number of runs, number of actual deliveries, number of wickets, number of boundaries, number of dot balls. Instead of assigning probabilities

to the entire messy outcome, assign probabilities to events defined *through* these random variables. It reduces the detail to something simpler — and with limited data, only such summaries can be studied at all.

Basically, ... A random variable is a “measurement function”: feed it the outcome of the experiment, it returns a number. Probability then becomes a question about that number — “what are the chances the measurement comes out below 4?” — instead of a question about the whole outcome.

15.2 Range of a random variable

Def. The **range** of a random variable = the set of values it takes. The range is a subset of the real line.

- i) Throw a die, X = number shown: range $\{1, 2, 3, 4, 5, 6\}$.
- ii) The even-indicator E : range $\{0, 1\}$.
- iii) IPL powerplay over: number of actual deliveries — range 6, 7, 8, 9, ... (wides and no-balls add deliveries); number of runs — 0, 1, 2, 3, ...; number of wickets — 0, 1, ..., 10; number of boundaries — 0, 1, 2, ...; number of dot balls — 0, 1, ..., 6.

The range is where the probability story will live: instead of tracking *which* outcome occurred, track *which value* the random variable took.

15.3 Discrete random variables

Def. A random variable is **discrete** if its range is a **discrete** set.

Which sets are discrete?

- i) Every finite subset of the real line is discrete.
- ii) The integers are discrete.
- iii) Integer multiples of any real number are discrete (e.g. $\{0, 0.5, 1, 1.5, \dots\}$).
- iv) Any subset of a discrete set is discrete.

Which are not? Any interval (a, b) with $a < b$ is not discrete — and any set containing a non-discrete set is not discrete.

Every random variable in §§15.1–15.2 is discrete: die faces, coin indicators, IPL counts. The non-discrete case — exact rainfall, exact dart position — waits for Chapter 16.

Basically, ... “Discrete” means the possible values can be listed (perhaps an infinite list): 0, 1, 2, If the values fill a continuum — every real number between 0 and 1 — it is not discrete, and a different toolkit is needed.

15.4 The PMF: probability mass function

If X is discrete, $(X = t)$ is an event for each value t in its range T . Define

$$P(X = t) = P(\text{all outcomes that make } X \text{ take the value } t).$$

For a set B of values, $(X \in B)$ is an event with

$$P(X \in B) = \sum_{t \in B} P(X = t).$$

Def. The **probability mass function (PMF)** of a discrete random variable X with range T = the function $f_X : T \rightarrow [0, 1]$ defined by

$$f_X(t) = P(X = t) \quad \text{for } t \in T.$$

Two properties every PMF satisfies (they are just the axioms, §14.6, wearing new clothes):

- i) $0 \leq f_X(t) \leq 1$ for all t — each value is a probability.
- ii) $\sum_{t \in T} f_X(t) = 1$ — the values $(X = t)$ are disjoint events whose union is “ X takes some value in T ”, i.e. the whole sample space.

Note: From here on, random variables and their PMFs are usually studied without much mention of the experiment or sample space. Events are defined *using* random variables, and probabilities are evaluated *using* the PMF. The sample space did its job — it is now scaffolding you can stop looking at.

eg 1 (3-digit lottery). A three-digit number is drawn uniformly from 000 to 999 (1000 equally likely outcomes). A ticket matching exactly wins Rs. 2 lakhs; matching exactly two of the three digits wins Rs. 20000; otherwise nothing. Let X = the ticket’s value. The PMF:

- i) $P(X = 200000) = 1/1000 = 0.001$ — one winning number.
- ii) $P(X = 20000)$: choose which two digits match — $\binom{3}{2} = 3$ choices — and the third digit must differ from the drawn one — 9 choices. 27 outcomes. $P = 27/1000 = 0.027$.
- iii) $P(X = 0) = 1 - 0.001 - 0.027 = 0.972$.

So $f_X(0) = 0.972$, $f_X(20000) = 0.027$, $f_X(200000) = 0.001$; range $\{0, 20000, 200000\}$. Check: the three values are disjoint and sum to 1.

eg 2 (working with a PMF). A random variable X has PMF given partially:

t	-1	1	2	4
$f_X(t)$	0.5	0.25	0.125	?

- i) $f_X(4) = 1 - (0.5 + 0.25 + 0.125) = \boxed{0.125}$ — the PMF must sum to 1.
- ii) Range of X : $\{-1, 1, 2, 4\}$.
- iii) $P(X > 3) = f_X(4) = \boxed{0.125}$ — only 4 exceeds 3.
- iv) $P(X < 3/2) = f_X(-1) + f_X(1) = 0.5 + 0.25 = \boxed{0.75}$.

eg 3 (finding the constant). $f_X(k) = c/3^k$ for $k = 1, 2, 3, \dots$. Find c ; find $P(X > 10)$ and $P(X > 10 \mid X > 5)$.

- i) The probabilities must sum to 1:

$$1 = \sum_{k=1}^{\infty} \frac{c}{3^k} = c \cdot \frac{1/3}{1 - 1/3} = c \cdot \frac{1}{2}, \quad \text{so } \boxed{c = 2}.$$

- ii) $P(X > 10) = \sum_{k=11}^{\infty} 2/3^k = \frac{2/3^{11}}{1 - 1/3} = \frac{2}{3^{11}} \cdot \frac{3}{2} = \boxed{(1/3)^{10}}$.
- iii) By the same tail computation, $P(X > 5) = (1/3)^5$. Then

$$P(X > 10 \mid X > 5) = \frac{P(X > 10)}{P(X > 5)} = \frac{(1/3)^{10}}{(1/3)^5} = \boxed{(1/3)^5}.$$

Note: (iii) says $P(X > 10 \mid X > 5) = P(X > 5)$ — surviving 5 more rounds after surviving 5 is exactly as likely as surviving 5 fresh. The past is forgotten. This is the memoryless property in action; §15.13 proves it in general.

15.5 The CDF for discrete random variables

Def. The **cumulative distribution function (CDF)** of X =

$$F_X(x) = P(X \leq x) = \sum_{t \leq x} f_X(t), \quad x \in \mathbb{R}.$$

The PMF answers “exactly t ”; the CDF answers “at most x ”. For discrete X , F_X is a step function:

- i) $0 \leq F_X(x) \leq 1$, non-decreasing; $F_X(x) \rightarrow 0$ as $x \rightarrow -\infty$, $F_X(x) \rightarrow 1$ as $x \rightarrow +\infty$.
- ii) At each range value t , F_X jumps by exactly $f_X(t)$; between range values it is flat.
- iii) Interval probabilities come from differences: $P(a < X \leq b) = F_X(b) - F_X(a)$.

eg 4 (CDF of eg 2's PMF). With $f_X(-1) = 0.5$, $f_X(1) = 0.25$, $f_X(2) = 0.125$, $f_X(4) = 0.125$:

$$F_X(x) = \begin{cases} 0 & x < -1, \\ 0.5 & -1 \leq x < 1, \\ 0.75 & 1 \leq x < 2, \\ 0.875 & 2 \leq x < 4, \\ 1 & x \geq 4. \end{cases}$$

(Panel (d) of the figure draws this staircase.) Then $P(1 < X \leq 4) = F_X(4) - F_X(1) = 1 - 0.75 = 0.25$ — and indeed $f_X(2) + f_X(4) = 0.125 + 0.125 = 0.25$.

Basically, ... The PMF is the full story (“how much mass sits at each point”); the CDF is the running total (“how much mass has piled up so far”). Anything you can ask with one, you can answer with the other.

15.6 Functions of a random variable

If X is a random variable with range T and PMF f_X , and f is any function $\mathbb{R} \rightarrow \mathbb{R}$, then $f(X)$ — apply f to the value X takes — is a *composition* of two functions, hence another random variable on the same probability space.

Its PMF comes straight from X 's:

$$f_{f(X)}(a) = P(f(X) = a) = P(X \in \{t : f(t) = a\}) = \sum_{t: f(t)=a} f_X(t).$$

Group together all t that f maps to the same value a , and add their probabilities.

eg 5. $X \sim \text{Uniform}(\{-2, -1, 0, 1, 2\})$ (each value with probability $1/5$), and $Y = X^2$. Then Y 's range is $\{0, 1, 4\}$: - $f_Y(0) = P(X = 0) = 1/5$, - $f_Y(1) = P(X \in \{-1, 1\}) = 2/5$, - $f_Y(4) = P(X \in \{-2, 2\}) = 2/5$.

Sum: 1. Note Y is *not* uniform — squaring folds the values together.

eg 6. $X \sim \text{Geometric}(1/2)$ (trials until the first head; §15.13), and $Y = \min(X, 5)$ — “cap the wait at 5”. Then: - for $k = 1, 2, 3, 4$: $P(Y = k) = P(X = k) = (1/2)^k$, - $P(Y = 5) = P(X \geq 5) = (1/2)^4 = 1/16$ — everything from 5 on piles up at the cap.

Check: $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{16} = 1$.

Basically, ... $f(X)$ inherits its probabilities from X : every value of X that f sends to the same place pools its probability there. This is the engine behind §15.7's averages — $E[X^2]$ is just the expectation of the function $f(X) = X^2$.

15.7 Expectation

The lectures' IPL framing started with *averages*: mean runs per powerplay over climbing from 6.00 to 8.30. The expectation is the theoretical version of that average — the “center of mass” of the PMF.

Def. The **expectation** (expected value, mean) of a discrete random variable $X =$

$$E[X] = \sum_{t \in T} t \cdot f_X(t).$$

Note (convergence): for an infinite range, this sum must converge absolutely ($\sum |t|f_X(t) < \infty$); otherwise $E[X]$ is undefined. Every distribution in this chapter has a well-defined mean.

Two consequences of §15.6 used constantly:

- i) **Functions:** $E[f(X)] = \sum_{t \in T} f(t) f_X(t)$ — average the transformed values with the original probabilities (no need to find $f(X)$'s PMF first).
- ii) In particular $E[X^2] = \sum t^2 f_X(t)$ — needed for variance (§15.9).

eg 7 (lottery, continued). From eg 1: $f_X(0) = 0.972$, $f_X(20000) = 0.027$, $f_X(200000) = 0.001$.

$$E[X] = 0(0.972) + 20000(0.027) + 200000(0.001) = 540 + 200 = \boxed{\text{Rs. 740}}.$$

Interpretation: over many, many tickets, the average payout per ticket settles at Rs. 740. **Note:** no ticket ever pays Rs. 740 — the expectation is an average, not a possible outcome. Confusing the two is the single most common expectation mistake.

Basically, ... Expectation = probability-weighted average. Multiply each possible value by how likely it is, add up. It is the one number that summarizes “where” the distribution sits.

15.8 Linearity of expectation

Theorem (linearity). For constants a, b and any random variables X, Y (no independence needed):

$$\boxed{E[aX + b] = a E[X] + b, \quad E[X + Y] = E[X] + E[Y].}$$

Proof. For the affine case:

$$E[aX + b] = \sum_t (at + b) f_X(t) = a \sum_t t f_X(t) + b \sum_t f_X(t) = a E[X] + b,$$

since $\sum_t f_X(t) = 1$. The sum rule follows by the same idea applied to the joint PMF of (X, Y) (joint distributions are Chapter 17's subject): $E[X + Y] = \sum_{t,s} (t + s) f_{X,Y}(t, s) = \sum_t t f_X(t) + \sum_s s f_Y(s)$.

This is the workhorse theorem of the chapter. It turns hard sums into easy ones: the expectation of a *count* is the sum of the expectations of its indicators — used for the binomial (§15.12), negative binomial (§15.14), and hypergeometric (§15.16) means.

eg 8 (linearity, checked both ways). With eg 2's PMF, $E[X] = (-1)(0.5) + (1)(0.25) + (2)(0.125) + (4)(0.125) = -0.5 + 0.25 + 0.25 + 0.5 = 0.5$. By linearity, $E[2X + 1] = 2(0.5) + 1 = \boxed{2}$. Direct check: $2X + 1$ takes values $-1, 3, 5, 9$ with the same probabilities, and $(-1)(0.5) + 3(0.25) + 5(0.125) + 9(0.125) = -0.5 + 0.75 + 0.625 + 1.125 = 2.0$.

Basically, ... Averages add: the average of a sum is the sum of the averages — always, even when the parts depend on each other. Whenever you see “expected number of ...”, try writing the count as a sum of 0/1 indicators first.

15.9 Variance and standard deviation

Expectation says where the distribution sits; **variance** says how spread out it is. The IPL framing had both: average runs per over *and* their standard deviations (≈ 4).

Def. The **variance** of X =

$$\boxed{\text{Var}(X) = E[(X - E[X])^2]},$$

the expected squared distance from the mean. The **standard deviation** $\sigma = \sqrt{\text{Var}(X)}$ restores the original units.

Theorem. $\boxed{\text{Var}(X) = E[X^2] - (E[X])^2.}$

Proof. Write $\mu = E[X]$ and expand, using linearity (§15.8):

$$\text{Var}(X) = E[(X - \mu)^2] = E[X^2 - 2\mu X + \mu^2] = E[X^2] - 2\mu E[X] + \mu^2 = E[X^2] - \mu^2. \quad \blacksquare$$

Corollary. $\text{Var}(aX + b) = a^2 \text{Var}(X)$: shifting by b does not change the spread; scaling by a scales squared distances by a^2 . (Proof: $E[(aX + b) - (a\mu + b)]^2 = E[a^2(X - \mu)^2] = a^2 \text{Var}(X)$.)

eg 9. With eg 2's PMF, $E[X] = 0.5$ (eg 8). Then

$$E[X^2] = (1)(0.5) + (1)(0.25) + (4)(0.125) + (16)(0.125) = 0.5 + 0.25 + 0.5 + 2 = 3.25,$$

$$\text{Var}(X) = 3.25 - (0.5)^2 = \boxed{3.0}, \quad \sigma = \sqrt{3} \approx \boxed{1.73}.$$

Basically, ... Variance = “average of the squares minus the square of the average”. The squaring penalizes far-from-mean values, so variance measures *spread*, not location. Standard deviation is the same idea in the original units (runs, not runs-squared).

15.10 The uniform distribution

Def. $X \sim \text{Uniform}(T)$, T a finite set: range T , PMF

$$f_X(t) = \frac{1}{|T|} \quad \text{for all } t \in T.$$

This is the “equally likely outcomes” model from §14.8, now as a named distribution: fair coin $\sim \text{Uniform}(\{0, 1\})$, fair die $\sim \text{Uniform}(\{1, 2, 3, 4, 5, 6\})$.

For the standard case $T = \{1, 2, \dots, n\}$:

$$E[X] = \frac{1}{n} \sum_{k=1}^n k = \frac{n+1}{2}, \quad E[X^2] = \frac{1}{n} \sum_{k=1}^n k^2 = \frac{(n+1)(2n+1)}{6},$$

$$\text{Var}(X) = \frac{(n+1)(2n+1)}{6} - \frac{(n+1)^2}{4} = \frac{n^2 - 1}{12}.$$

eg 10 (fair die). $E[X] = 7/2 = 3.5$; $\text{Var}(X) = 35/12 \approx 2.92$, $\sigma \approx 1.71$. Sanity check via the definition: $E[(X - 3.5)^2] = \frac{1}{6}(2(2.5^2) + 2(1.5^2) + 2(0.5^2)) = \frac{1}{6}(12.5 + 4.5 + 0.5) = 17.5/6 = 35/12$.

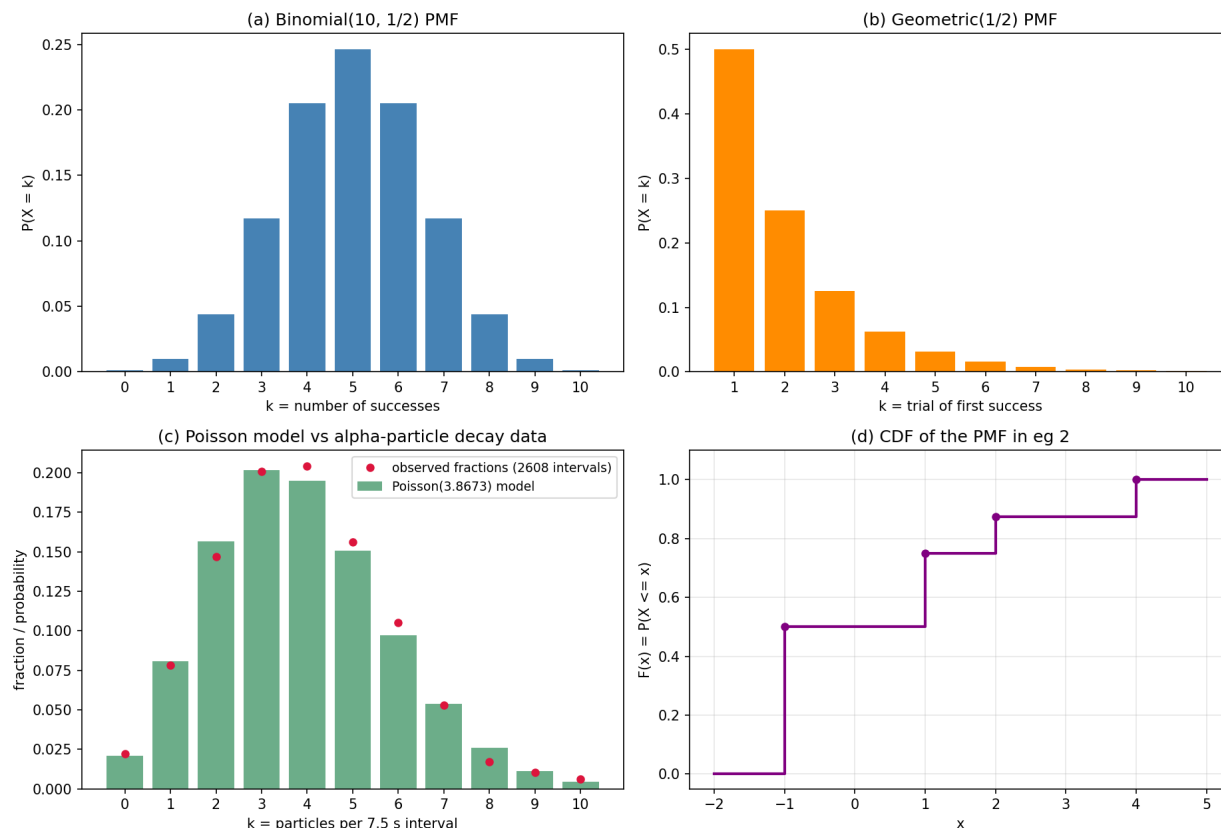


Figure 17: (a) PMF of Binomial(10, 1/2). (b) PMF of Geometric(1/2), the “ski slope” of waiting-time probabilities. (c) Poisson(3.8673) model bars against the observed alpha-particle decay fractions from the source slides — the model fits the data. (d) CDF staircase of the PMF in eg 2, jumping by $f_X(t)$ at each range value.

15.11 The Bernoulli distribution

Def. $X \sim \text{Bernoulli}(p)$, $0 \leq p \leq 1$: range $\{0, 1\}$,

$$f_X(0) = 1 - p, \quad f_X(1) = p.$$

The model of a single success/failure trial: $X = 1$ = “success” with probability p . It is the atom everything else in this chapter is built from — and $\text{Binomial}(1, p) = \text{Bernoulli}(p)$ (§15.12).

$$E[X] = 0 \cdot (1 - p) + 1 \cdot p = p, \quad \text{Var}(X) = E[X^2] - (E[X])^2 = p - p^2 = p(1 - p)$$

(using $X^2 = X$ for a 0/1 variable — the oldest trick in the chapter).

Basically, ... Bernoulli = one yes/no question. Its mean is p (the chance of “yes”), and its variance $p(1 - p)$ is largest at $p = 1/2$ — maximum uncertainty when the question is a coin flip.

15.12 The binomial distribution

Def. $X \sim \text{Binomial}(n, p)$, n a positive integer, $0 \leq p \leq 1$: range $\{0, 1, \dots, n\}$,

$$f_X(k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

The story: number of successes in n *independent* Bernoulli(p) trials. This is exactly the counting argument §14.13 promised:

- i) One *specific* sequence with k successes has probability $p^k(1-p)^{n-k}$ — multiply by independence (§14.13: only the count of 1s matters, not their positions).
- ii) How many such sequences are there? Choose the k success positions among n : $\binom{n}{k}$.
- iii) The $\binom{n}{k}$ sequences are disjoint, so add (Axiom 3, §14.6): $\binom{n}{k} p^k (1-p)^{n-k}$.

The PMF is valid: $\sum_{k=0}^n \binom{n}{k} p^k (1-p)^{n-k} = (p + (1-p))^n = 1$ by the binomial theorem.

Mean and variance — the indicator trick. Write $X = X_1 + \dots + X_n$ where X_i is 1 if trial i succeeds, else 0; each $X_i \sim \text{Bernoulli}(p)$. By linearity (§15.8):

$$E[X] = \sum_{i=1}^n E[X_i] = \boxed{np}.$$

For the variance, $E[X^2] = \sum_i E[X_i^2] + \sum_{i \neq j} E[X_i X_j]$. Now $X_i^2 = X_i$ so $E[X_i^2] = p$; and for $i \neq j$, independence (§14.12) gives $E[X_i X_j] = E[X_i] E[X_j] = p^2$. Hence $E[X^2] = np + n(n-1)p^2$, and

$$\text{Var}(X) = np + n(n-1)p^2 - n^2 p^2 = \boxed{np(1-p)}.$$

eg 11 (3 fair tosses — the deck's question). “How many heads will appear?” $X \sim \text{Binomial}(3, 1/2)$:

$$f_X(0) = \frac{1}{8}, f_X(1) = \frac{3}{8}, f_X(2) = \frac{3}{8}, f_X(3) = \frac{1}{8}.$$

E.g. $P(\text{exactly 2 heads}) = \binom{3}{2} (1/2)^3 = \boxed{3/8}$ — the three sequences HHT, HTH, THH .

Basically, ... Binomial = “how many yeses in n independent tries”. Every sequence with k yeses is equally likely; count the sequences with $\binom{n}{k}$. The mean np is just “ n tries times p chance each” — linearity doing the counting for you.

15.13 The geometric distribution

Def. $X \sim \text{Geometric}(p)$, $0 < p \leq 1$: range $\{1, 2, 3, \dots\}$,

$$f_X(k) = (1-p)^{k-1} p.$$

The story: number of trials until the *first* success in repeated independent Bernoulli(p) trials. For $X = k$: $k-1$ failures then a success — multiply by independence. The PMF is valid: $p \sum_{k=1}^{\infty} (1-p)^{k-1} = p \cdot \frac{1}{1-(1-p)} = 1$ (geometric series — the distribution's namesake).

Mean and variance. Using $\sum_{k=1}^{\infty} kx^{k-1} = 1/(1-x)^2$ for $|x| < 1$:

$$E[X] = \sum_{k=1}^{\infty} k(1-p)^{k-1} p = \frac{p}{(1-(1-p))^2} = \boxed{\frac{1}{p}}.$$

For the variance, the $E[X(X-1)]$ trick avoids differentiating twice: $\sum_{k=1}^{\infty} k(k-1)x^{k-2} = 2/(1-x)^3$, so

$$E[X(X-1)] = p(1-p) \cdot \frac{2}{(1-(1-p))^3} = \frac{2(1-p)}{p^2}.$$

Then $E[X^2] = E[X(X-1)] + E[X] = \frac{2(1-p)}{p^2} + \frac{1}{p} = \frac{2-p}{p^2}$, and

$$\text{Var}(X) = \frac{2-p}{p^2} - \frac{1}{p^2} = \boxed{\frac{1-p}{p^2}}.$$

Note (the other convention). Some sources count *failures* before the first success: range $\{0, 1, 2, \dots\}$, $f_X(k) = (1-p)^k p$. Same distribution shifted by 1 (mean $(1-p)/p$, same variance). This chapter uses the trials-until-success convention unless stated.

Note (memorylessness). The geometric forgets past failures: for $m, n \geq 0$,

$$P(X > m+n \mid X > m) = \frac{P(X > m+n)}{P(X > m)} = \frac{(1-p)^{m+n}}{(1-p)^m} = (1-p)^n = P(X > n).$$

($P(X > k) = \sum_{j=k+1}^{\infty} (1-p)^{j-1} p = (1-p)^k$.) Knowing you already failed m times tells you nothing about the next n — eg 3's $P(X > 10 \mid X > 5) = (1/3)^5 = P(X > 5)$ was exactly this. The geometric is the *only* discrete distribution with this property.

eg 12. Fair coin; X = toss of the first head, $X \sim \text{Geometric}(1/2)$. $P(X = 5) = (1/2)^4(1/2) = \boxed{1/32}$ — four tails, then a head. $E[X] = 2$ tosses on average; $\text{Var}(X) = (1/2)/(1/4) = 2$.

Also the deck's second question — “which is the first flip (if any) that shows heads?” in 3 tosses: $P(X = k) = (1/2)^k$ for $k = 1, 2, 3$, and $P(\text{no heads}) = (1/2)^3 = 1/8$. A geometric cut off at 3.

Basically, ... Geometric = “how long until the first yes”. The PMF is a “ski slope” (panel (b) of the figure): each extra failure multiplies the probability by $(1-p)$. And it has no memory — a coin that failed 10 times is not “due”.

15.14 The negative binomial distribution

Def. $X \sim \text{NegativeBinomial}(r, p)$, r a positive integer, $0 < p \leq 1$: range $\{r, r+1, r+2, \dots\}$,

$$f_X(k) = \binom{k-1}{r-1} (1-p)^{k-r} p^r.$$

The story: number of trials until the r -th success. The counting argument: for $X = k$, the k -th trial *must* be a success, and among the first $k-1$ trials exactly $r-1$ are successes — $\binom{k-1}{r-1}$ ways, each with probability $p^{r-1}(1-p)^{k-r}$ — times p for the final success. With $r = 1$ this collapses to the geometric: $\text{NegativeBinomial}(1, p) = \text{Geometric}(p)$.

Mean and variance — via geometrics. The wait for r successes is the sum of r independent geometric waits (wait for the 1st, then the 2nd, ...). By linearity (§15.8) and independence (§14.12):

$$E[X] = r \cdot \frac{1}{p} = \boxed{\frac{r}{p}}, \quad \text{Var}(X) = r \cdot \frac{1-p}{p^2} = \boxed{\frac{r(1-p)}{p^2}}.$$

eg 13. $p = 1/2$; X = trials until the 3rd head. $P(X = 5) = \binom{4}{2}(1/2)^2(1/2)^3 = 6/32 = \boxed{3/16}$ — two heads among the first four tosses ($\binom{4}{2}$ ways), then a head.

15.15 The Poisson distribution

Def. $X \sim \text{Poisson}(\lambda)$, $\lambda > 0$: range $\{0, 1, 2, \dots\}$,

$$f_X(k) = \frac{e^{-\lambda} \lambda^k}{k!}.$$

The story (from the source deck): events occurring over time — arrivals at a queue, website visitors, radioactive emissions, meteorites entering the atmosphere. What these share: the arrival *rate* can be assumed constant, and given one arrival, the time until the next is independent of the past. Under those assumptions, the number of arrivals in a fixed period is Poisson. The PMF is valid: $\sum_{k=0}^{\infty} \lambda^k/k! = e^\lambda$.

Mean and variance. $E[X(X-1)] = \sum_{k=0}^{\infty} k(k-1) e^{-\lambda} \lambda^k/k! = \lambda^2 \sum_{j=0}^{\infty} e^{-\lambda} \lambda^j/j! = \lambda^2$ (with $j = k-2$). Similarly, $E[X] = \sum k e^{-\lambda} \lambda^k/k! = \lambda \sum e^{-\lambda} \lambda^{k-1}/(k-1)! = \lambda$. So

$$E[X] = \lambda, \quad \text{Var}(X) = \lambda^2 + \lambda - \lambda^2 = \lambda.$$

Mean equals variance — the Poisson’s signature.

Note (Poisson as the limit of the binomial). Take $\text{Binomial}(n, p)$ with $n \rightarrow \infty$, $p \rightarrow 0$ holding $np = \lambda$ fixed (many trials, rare successes). Then

$$\binom{n}{k} \left(\frac{\lambda}{n}\right)^k \left(1 - \frac{\lambda}{n}\right)^{n-k} = \underbrace{\frac{n(n-1)\cdots(n-k+1)}{n^k}}_{\rightarrow 1} \cdot \frac{\lambda^k}{k!} \cdot \underbrace{\frac{(1-\lambda/n)^n}{(1-\lambda/n)^k}}_{\rightarrow e^{-\lambda}} \rightarrow \frac{e^{-\lambda} \lambda^k}{k!}.$$

This is why Poisson models “rare events”: it is the binomial pushed to the rare-event limit. (The same limit is stated on distribution-explorer’s Binomial page — the named cross-check resource for this chapter; the site’s PMF/mean/variance tables were used to verify every distribution in §§15.10–15.16.)

eg 14 (radioactive decay — the deck’s data). In 2608 intervals of 7.5 seconds, alpha-particle counts were recorded; the average rate $\lambda = (\text{total particles})/2608 = 3.8673$. Then

$$P(X=3) = \frac{e^{-3.8673} 3.8673^3}{3!} \approx \boxed{0.2016},$$

against the observed fraction 0.201. Panel (c) of the figure plots model vs observed fractions for $k = 0, \dots, 10$ — they track each other closely: the Poisson model *fits*.

eg 15 (fireballs). Over 276 months, meteorite (“fireball”) counts average $\lambda = 2.5217$ per month. $P(X=0) = e^{-2.5217} \approx \boxed{0.0803}$ — a month with no fireballs happens about 8% of the time; the observed fraction was 0.087.

Basically, ... Poisson = “how many events in a fixed window, when events drip in at a steady rate independently of each other”. One parameter λ does everything: it is the mean, the variance, and (estimated from data) the observed average rate.

15.16 The hypergeometric distribution

Def. $X \sim \text{Hypergeometric}(N, r, m)$, N, r, m positive integers: a population of N items holds r of Type 1 and $N-r$ of Type 2; draw m items *uniformly at random without replacement*; X = number of Type-1 items drawn. Range: $\max(0, m - (N-r)), \dots, \min(r, m)$,

$$f_X(k) = \frac{\binom{r}{k} \binom{N-r}{m-k}}{\binom{N}{m}}.$$

The counting argument: choose the k Type-1 items ($\binom{r}{k}$ ways), choose the $m - k$ Type-2 items ($\binom{N-r}{m-k}$ ways), divide by all $\binom{N}{m}$ equally likely draws. The deck's range examples: $N = 100, r = 50, m = 20 \Rightarrow X \in \{0, \dots, 20\}$; $N = 100, r = 10, m = 20 \Rightarrow X \in \{0, \dots, 10\}$ (can't draw more Type-1 than exist); $N = 100, r = 90, m = 20 \Rightarrow X \in \{10, \dots, 20\}$ (at least 10 must be Type-1).

Mean and variance. Write $X = X_1 + \dots + X_m$ with X_j the indicator that draw j is Type 1. Each draw is equally likely to be any of the N items, so $E[X_j] = r/N$, and by linearity

$$E[X] = m \frac{r}{N}.$$

The draws are *not* independent (sampling without replacement — §14.9's shrinking denominator), so the variance is not $m \frac{r}{N} (1 - \frac{r}{N})$; it carries a **finite-population correction**:

$$\text{Var}(X) = m \frac{r}{N} \frac{N-r}{N} \frac{N-m}{N-1}.$$

When the population dwarfs the sample ($N \gg m$), $\frac{N-m}{N-1} \approx 1$ and the hypergeometric is well approximated by Binomial($m, r/N$) — without replacement barely differs from with replacement.

eg 16. $N = 100$ items, $r = 50$ Type 1, draw $m = 20$. $E[X] = 20 \cdot \frac{1}{2} = 10$. $P(X = 10) = \binom{50}{10} \binom{50}{10} / \binom{100}{20} \approx 0.1969$. $\text{Var}(X) = 20 \cdot \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{80}{99} = 400/99 \approx 4.04$ (vs 5 without the correction).

Basically, ... Hypergeometric = binomial's “without replacement” sibling. Drawing 20 from 100 and counting Type-1s: the *average* is the same as with replacement ($m \cdot r/N$), but the *spread* shrinks — each draw “uses up” some randomness.

15.17 Why this matters: the language every later chapter speaks

- i) **Expectation and variance are the ML vocabulary.** Loss functions are expectations over data; “noise” is variance; the bias–variance tradeoff is a variance decomposition. Linearity of expectation is the single most-used proof move in randomized algorithms and ML theory.
- ii) **These distributions are the models.** Bernoulli/Binomial sit behind every binary outcome (click/no-click, spam/ham — Chapter 30's naive Bayes uses such likelihoods); the geometric behind waiting times; the Poisson behind count data and arrival processes; the hypergeometric behind sampling audits.
- iii) **The pattern continues.** Chapter 16 lifts everything here to continuous random variables: PMF $\rightarrow$ PDF, sums $\rightarrow$ integrals, same expectation/variance definitions. Chapters 17–18 put two random variables together (joint PMFs — the object §15.8's sum-rule proof already leaned on). Chapter 20's estimation and Chapter 21's central limit theorem (“averages of many iid variables concentrate at $E[X]$ ”) both start from this chapter's definitions.

Part II, second shot. Events and axioms (§14) said *what* chance is. Random variables say *what chance is about*: numbers measured on experiments, with PMFs for their probabilities, expectations for their averages, variances for their spread. The next chapter keeps the same ideas and lets the values fill a continuum (Chapter 16).

Problem set

1. (Random variables and ranges) Consider one IPL powerplay over. (a) Which of these are random variables: the number of wickets; the number of dot balls; “whether the over was exciting”? (b) Give the range of the number of wickets and of the number of dot balls.
2. (PMF validity) $f_X(k) = c/2^k$ for $k = 1, 2, 3, \dots$. Find c . Then find $P(X \leq 2)$.
3. (Expectation and variance) X has PMF $f_X(0) = 0.3, f_X(1) = 0.5, f_X(2) = 0.2$. Find $E[X]$ and $\text{Var}(X)$.
4. (CDF) For X as in Problem 3: (a) write $F_X(x)$ piecewise; (b) find $P(0 < X \leq 2)$ using the CDF, and check against the PMF.

5. (Linearity) $X \sim \text{Binomial}(100, 1/4)$. Find $E[2X + 3]$.
6. (Variance scaling) $X \sim \text{Geometric}(1/2)$. Find $\text{Var}(3X - 1)$.
7. (Binomial) A biased coin with $P(H) = 1/3$ is tossed 5 times. Find $P(\text{exactly 2 heads})$.
8. (Memorylessness) $X \sim \text{Geometric}(0.3)$: a component's lifetime in years. Given it has survived 4 years, find $P(X > 10 \mid X > 4)$.
9. (Negative binomial) Trials with $p = 1/4$ are repeated until the 3rd success. Find $P(X = 6)$.
10. (Poisson fit) For the alpha-decay data ($\lambda = 3.8673$): (a) compute $P(X \leq 1)$ under the Poisson model; (b) compare with the observed fraction of intervals with 0 or 1 particles ($0.022 + 0.078 = 0.100$).
11. (Hypergeometric) $N = 100$, $r = 50$, $m = 20$: find $E[X]$ and $\text{Var}(X)$.
12. (Functions of a random variable) $X \sim \text{Geometric}(1/2)$ and $Y = \min(X, 3)$. Find the PMF of Y and $E[Y]$.

Solutions — 15. Discrete random variables and key distributions

1. (Random variables and ranges) (a) The number of wickets and the number of dot balls are random variables: each is a function from the over's outcome (the ball-by-ball record) to a real number. "Whether the over was exciting" is not — it is not a numerical function of the outcome (no rule assigns it a real number).

(b) Number of wickets: $\{0, 1, \dots, 10\}$ (at most ten wickets can fall). Number of dot balls: $\{0, 1, \dots, 6\}$ (at most six legal deliveries in the over; wides/no-balls add deliveries but not dot balls).

2. (PMF validity) Sum to 1:

$$1 = \sum_{k=1}^{\infty} \frac{c}{2^k} = c \cdot \frac{1/2}{1 - 1/2} = c,$$

so $\boxed{c = 1}$ — i.e. $X \sim \text{Geometric}(1/2)$ in the trials-until-success convention (§15.13). Then $P(X \leq 2) = f_X(1) + f_X(2) = \frac{1}{2} + \frac{1}{4} = \boxed{3/4}$.

3. (Expectation and variance)

$$E[X] = 0(0.3) + 1(0.5) + 2(0.2) = 0.9.$$

$$E[X^2] = 0^2(0.3) + 1^2(0.5) + 2^2(0.2) = 0.5 + 0.8 = 1.3.$$

$$\text{Var}(X) = E[X^2] - (E[X])^2 = 1.3 - 0.81 = \boxed{0.49}.$$

So $\boxed{E[X] = 0.9}$, $\sigma = 0.7$.

4. (CDF) (a) The CDF jumps by $f_X(t)$ at each range value:

$$F_X(x) = \begin{cases} 0 & x < 0, \\ 0.3 & 0 \leq x < 1, \\ 0.8 & 1 \leq x < 2, \\ 1 & x \geq 2. \end{cases}$$

(b) $P(0 < X \leq 2) = F_X(2) - F_X(0) = 1 - 0.3 = \boxed{0.7}$. Check via the PMF: $f_X(1) + f_X(2) = 0.5 + 0.2 = 0.7$.

5. (Linearity) $E[X] = np = 100 \cdot \frac{1}{4} = 25$ (§15.12). By linearity (§15.8): $E[2X + 3] = 2E[X] + 3 = 2(25) + 3 = \boxed{53}$.

6. (Variance scaling) $\text{Var}(X) = (1 - p)/p^2 = (1/2)/(1/4) = 2$ (§15.13). By the corollary in §15.9: $\text{Var}(3X - 1) = 3^2 \cdot \text{Var}(X) = 9 \cdot 2 = \boxed{18}$.

7. (Binomial) $X \sim \text{Binomial}(5, 1/3)$:

$$P(X = 2) = \binom{5}{2} \left(\frac{1}{3}\right)^2 \left(\frac{2}{3}\right)^3 = 10 \cdot \frac{1}{9} \cdot \frac{8}{27} = \frac{80}{243} \approx \boxed{0.3292}.$$

(Ten sequences with exactly 2 heads, each with probability $(1/3)^2(2/3)^3$ — the §15.12 counting argument.)

8. (Memorylessness) $P(X > k) = (1 - p)^k = 0.7^k$. By memorylessness (§15.13):

$$P(X > 10 \mid X > 4) = \frac{0.7^{10}}{0.7^4} = 0.7^6 = \boxed{0.117649}.$$

The four survived years are forgotten — this equals $P(X > 6)$, the fresh 6-year survival probability.

9. (Negative binomial) $X \sim \text{NegativeBinomial}(3, 1/4)$:

$$P(X = 6) = \binom{5}{2} \left(\frac{3}{4}\right)^3 \left(\frac{1}{4}\right)^3 = 10 \cdot \frac{27}{64} \cdot \frac{1}{64} = \frac{270}{4096} = \frac{135}{2048} \approx 0.0659.$$

(The 6th trial is the 3rd success; among the first 5 trials exactly 2 are successes: $\binom{5}{2}$ ways.)

10. (Poisson fit) (a) $P(X \leq 1) = P(X = 0) + P(X = 1) = e^{-\lambda} + \lambda e^{-\lambda} = e^{-3.8673}(1 + 3.8673)$. With $e^{-3.8673} \approx 0.02094$: $P(X \leq 1) \approx 0.02094 \times 4.8673 \approx \boxed{0.1018}$.

(b) The observed fraction is $0.022 + 0.078 = 0.100$ — the model (0.1018) matches the data to within 0.002. The Poisson fit is good (see panel (c) of the figure).

11. (Hypergeometric) $E[X] = m \cdot r/N = 20 \cdot \frac{50}{100} = \boxed{10}$.

$$\text{Var}(X) = m \cdot \frac{r}{N} \cdot \frac{N-r}{N} \cdot \frac{N-m}{N-1} = 20 \cdot \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{80}{99} = \frac{400}{99} \approx \boxed{4.04}.$$

(Without the finite-population correction $\frac{80}{99}$, the variance would be 5.)

12. (Functions of a random variable) $Y = \min(X, 3)$ caps X at 3: - $P(Y = 1) = P(X = 1) = 1/2$, - $P(Y = 2) = P(X = 2) = 1/4$, - $P(Y = 3) = P(X \geq 3) = 1 - \frac{1}{2} - \frac{1}{4} = 1/4$.

Check: $\frac{1}{2} + \frac{1}{4} + \frac{1}{4} = 1$. Then

$$E[Y] = 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} + 3 \cdot \frac{1}{4} = 0.5 + 0.5 + 0.75 = \boxed{1.75}.$$

(Compare $E[X] = 2$: capping the long tail lowers the mean, as it should.)

16. Continuous random variables

Chapter 15 closed with a promise (§15.17): everything built for discrete random variables lifts to the continuous case — PMF $\rightarrow$ PDF, sums $\rightarrow$ integrals, same expectation/variance definitions. This chapter is that lift.

The motivation comes from the Stats 2 lectures. Consider meteorite weights: 45000+ data points spread from 0.01 grams to 60 tons. Listing every individual value is unwieldy; instead you take logarithms, split the range into ≈ 100 intervals, count how many values fall in each interval, and describe the *shape* of the resulting histogram instead of the individual values. Or consider $\text{Binomial}(n, p)$ with $n = 100$: the PMF formula is exact but the calculations are not pleasant. Continuous random variables are what you reach for when the values fill a continuum and counting individual outcomes stops being the right tool.

16.1 Continuous random variables

Def. A random variable is **continuous** if its range is a **continuum** — not a discrete set (§15.3).

Which ranges are continuous?

- i) Any interval (a, b) , $[a, b]$, (a, ∞) with $a < b$ is continuous.
- ii) The whole real line $\mathbb{R}$ is continuous.
- iii) A range containing any interval is continuous.

The MLF lectures put it as: the range of a discrete random variable is countable; the range of a continuous random variable is not (uncountably infinite). The two definitions agree for every case in this chapter.

Note (mixed random variables). Not every random variable is discrete or continuous. The Stats 2 deck gives $F(x) = 0$ for $x < 0$, $F(x) = 0.5 + 0.1x$ for $0 \leq x \leq 5$, $F(x) = 1$ for $x > 5$: here $P(X = 0) = 0.5$ (a genuine atom at 0) while on $(0, 5]$ the probability smears out continuously — $P(1.99 < X \leq 2.01) = F(2.01) - F(1.99) = 0.002$, and $P(1.999999 < X \leq 2.000001) = 0.00000002$. As the precision increases, the probability decreases. This is a *mixed* random variable. This chapter handles the pure continuous case; the discrete case was Chapter 15.

Basically, ... Discrete = the values can be listed: 0, 1, 2, Continuous = the values fill stretches of the number line: every real number between 0 and 1. A different toolkit is needed because there are too many values to give each one its own probability.

16.2 Why the PMF breaks: $P(X = x) = 0$

The MLF lecture's example: you reach a bus stop at a random time; X = how long you wait for the bus. Ask “what is the probability you wait *exactly* 4 minutes 29 seconds?” The answer is 0 — there are so many possible wait times, why would it be exactly that one?

- i) For a continuous X , $P(X = x) = 0$ for *every* single value x .
- ii) But interval questions are meaningful: $P(\text{wait between 4:29 and 4:35})$ can be small yet nonzero.

- iii) So probability lives on *intervals*, not points. The PMF (§15.4), which assigns probability to individual values, cannot work — something that measures probability *per unit length* must take its place.

Note: $P(X = x) = 0$ does *not* mean “ x is impossible”. Waiting exactly 4:29 is possible — it is just one of uncountably many possibilities. Zero probability $\neq$ impossible, for continuous random variables.

Basically, ... A PMF puts lumps of probability on individual points. In the continuous world there are too many points for lumps: each point gets probability zero, and the probability lives in the stretches between points. You need a *density* — probability per unit of x — instead.

16.3 The PDF: probability density function

Def. The **probability density function (PDF)** of a continuous random variable X = the function f_X such that

$$P(X \in [x, x + dx]) \approx f_X(x) dx$$

for a small interval dx around x . Equivalently, for any event $A \subseteq \mathbb{R}$ (the Stats 2 deck’s formulation):

$$P(X \in A) = \int_A f_X(x) dx.$$

Three properties every PDF satisfies:

- i) $f_X(x) \geq 0$ for all x — it is probability divided by a positive dx .
- ii) $\int_{-\infty}^{\infty} f_X(x) dx = 1$ — the discrete PMF’s “sums to 1” (§15.4), with the sum replaced by an integral.
- iii) (Stats 2 deck’s third condition) f_X is piecewise continuous.

Two things that surprise newcomers:

- i) $f_X(x)$ **can exceed 1**. Only the *numerator* (probability) is bounded by 1; the denominator dx can be tiny, so the ratio — the density — can be arbitrarily large. Densities are not probabilities.
- ii) **Densities have units**. Probability is unitless, but dividing by dx creates a unit: if X is a waiting time in minutes, f_X is in “per minute”; if X is a height in meters, f_X is in “per meter”. (The MLF lecture’s point: 1/15 per minute is the same density as 4 per hour.)

Def (support). The **support** of $X = \text{supp}(X) = \{x : f_X(x) > 0\}$ — the intervals in which X can fall with positive probability (Stats 2 deck).

eg 1 (validity check — the deck’s problem). Is

$$f(x) = \begin{cases} 3x^2 & 0 < x < 1, \\ 0 & \text{otherwise} \end{cases}$$

a valid density? Check the three properties: $3x^2 \geq 0$; $\int_{-\infty}^{\infty} f = \int_0^1 3x^2 dx = [x^3]_0^1 = 1$; piecewise continuous . Valid. Then $P(X = 1/5) = \boxed{0}$ — every single point has probability zero (§16.2) — while for a small ϵ , $P(X \in [1/5 - \epsilon, 1/5 + \epsilon]) \approx f(1/5) \cdot 2\epsilon = 3(1/25) \cdot 2\epsilon = 6\epsilon/25$: positive, and shrinking with ϵ .

eg 2 (finding the constant — the deck’s problem).

$$f(x) = \begin{cases} k & 0 \leq x < 1/4, \\ 2k & 1/4 \leq x < 3/4, \\ 3k & 3/4 \leq x < 1, \\ 0 & \text{otherwise.} \end{cases}$$

Integrate to 1:

$$1 = k \cdot \frac{1}{4} + 2k \cdot \frac{1}{2} + 3k \cdot \frac{1}{4} = \frac{k}{4} + k + \frac{3k}{4} = 2k,$$

so $\boxed{k = 1/2}$. (The area under the three rectangles must total 1.)

eg 3 (the bus-waiting density — MLF lecture). You arrive uniformly between 7:15 and 7:30; buses leave every 15 minutes. Then X = waiting time is equally likely to be any value between 0 and 15 minutes, so $f_X(x) = C$ on $[0, 15]$, 0 elsewhere. Find C :

$$1 = \int_0^{15} C \, dx = 15C \quad \Rightarrow \quad \boxed{C = 1/15 \text{ per minute}}.$$

In hours (0 to 0.25 h): $C = 4$ per hour — same density, different unit. The rectangle's area is 1 either way.

Basically, ... The PDF is the continuous version of the PMF: where the PMF says “this much probability sits at t ”, the PDF says “probability is packed this densely around x ”. Probabilities are areas under the PDF curve; the total area is 1.

16.4 The CDF for continuous random variables

Def. The **cumulative distribution function (CDF)** of X =

$$\boxed{F_X(x) = P(X \leq x) = \int_{-\infty}^x f_X(u) \, du, \quad x \in \mathbb{R}.}$$

Same definition as §15.5; the sum over $t \leq x$ became an integral.

- i) The CDF is the *integral* of the PDF; the PDF is the *derivative* of the CDF wherever the derivative exists: $f_X(x) = F'_X(x)$ (Stats 2 deck).
- ii) $F_X(-\infty) = 0$, $F_X(+\infty) = 1$; F_X is non-decreasing (MLF lecture: $P(X \leq 5) \geq P(X \leq 3.6)$, always).
- iii) Interval probabilities are differences: $P(a < X \leq b) = F_X(b) - F_X(a)$ — and since single points have probability 0, the endpoints never matter: $P(a \leq X \leq b) = P(a < X < b) = F_X(b) - F_X(a)$.

eg 4 (linear density $\rightarrow$ quadratic CDF — MLF lecture). $f_X(x) = x/2$ for $0 \leq x \leq 2$ (0 otherwise). First check validity: $\int_0^2 (x/2) \, dx = [x^2/4]_0^2 = 1$ (a triangle of base 2, height 1). The CDF, for $0 \leq x \leq 2$:

$$F_X(x) = \int_0^x \frac{u}{2} \, du = \left[\frac{u^2}{4} \right]_0^x = \boxed{\frac{x^2}{4}},$$

with $F_X(x) = 0$ for $x < 0$ and $F_X(x) = 1$ for $x > 2$ (check: $F_X(2) = 4/4 = 1$). Panel (d) of the figure draws both. Then $P(0.5 < X \leq 1.5) = F_X(1.5) - F_X(0.5) = (2.25 - 0.25)/4 = \boxed{0.5}$.

Basically, ... The CDF is the running total — “how much probability has piled up so far” — and it works identically for discrete and continuous variables. The PDF is its slope: where the CDF climbs steeply, probability is dense.

16.5 Expectation

Def. The **expectation** of a continuous random variable X =

$$\boxed{E[X] = \int_{-\infty}^{\infty} x f_X(x) \, dx},$$

the discrete definition (§15.7) with $\sum$ replaced by $\int$.

The MLF lecture's image: treat the PDF curve as a physical object whose mass at each point is proportional to the curve's height — the point where you can balance it is $E[X]$. Same center-of-mass idea as §15.7.

Theorem (LOTUS). For $Y = g(X)$,

$$E[g(X)] = \int_{-\infty}^{\infty} g(x) f_X(x) dx$$

— average the transformed values with the *original* density, no need to find Y 's PDF first (MLF lecture's "powerful shortcut"; Stats 2 deck states it as a theorem, "whenever the integral exists"). In particular $E[X^2] = \int x^2 f_X(x) dx$.

Theorem (linearity). $E[aX + b] = aE[X] + b$ and $E[X + Y] = E[X] + E[Y]$ — same as §15.8, no independence needed. (The affine case is one line from the integral definition; the sum rule leans on the joint density, Chapter 17's object, exactly as §15.8 leaned on the joint PMF.)

eg 5 (triangular density — MLF lecture). With $f_X(x) = x/2$ on $[0, 2]$ (eg 4):

$$E[X] = \int_0^2 x \cdot \frac{x}{2} dx = \frac{1}{2} \left[\frac{x^3}{3} \right]_0^2 = \frac{1}{2} \cdot \frac{8}{3} = \boxed{\frac{4}{3}}.$$

Sanity check from the lecture: the density rises from 0 to 2, so larger values are likelier — $E[X]$ should exceed the midpoint 1 but stay below the maximum 2. $4/3 \approx 1.33$ (marked on panel (d) of the figure).

eg 6 (total expectation — MLF lecture). You arrive uniformly between 7:10 and 7:30; buses at 7:15 and 7:30. Let X = waiting time. Rather than finding X 's density directly, condition on A = "arrive before 7:15":

- i) Given A : arrival uniform on $[7:10, 7:15]$, so waiting time uniform on $[0, 5]$: $E[X | A] = 2.5$.
- ii) Given A^c : arrival uniform on $[7:15, 7:30]$, so waiting time uniform on $[0, 15]$: $E[X | A^c] = 7.5$.
- iii) $P(A) = 5/20 = 1/4$, $P(A^c) = 3/4$.

By the law of total expectation:

$$E[X] = E[X | A] P(A) + E[X | A^c] P(A^c) = \frac{1}{4}(2.5) + \frac{3}{4}(7.5) = \boxed{6.25 \text{ minutes}}.$$

Note: this is exactly the discrete law of total probability from §14.10, with expectations in place of probabilities — split the outcomes into easy groups, condition, and recombine.

Basically, ... Expectation = probability-weighted average, now with an integral instead of a sum. Everything the discrete expectation could do — linearity, LOTUS, total expectation — the continuous one does too.

16.6 Variance and standard deviation

Def. The **variance** of X =

$$\text{Var}(X) = E[(X - E[X])^2],$$

and $\sigma = \sqrt{\text{Var}(X)}$ is the **standard deviation** — identical to §15.9.

Theorem. $\text{Var}(X) = E[X^2] - (E[X])^2$ (same one-line expansion as §15.9, using linearity).

Corollary. $\text{Var}(aX + b) = a^2 \text{Var}(X)$ (MLF lecture). **Note:** unlike expectation, variance is *not* linear: $\text{Var}(X + Y) \neq \text{Var}(X) + \text{Var}(Y)$ in general — the lecture's counterexample: $Y = -X$ gives $\text{Var}(X + Y) = 0$ while $\text{Var}(X) + \text{Var}(Y) > 0$. (The sum-of-independent case, where it does add, is Chapter 17's.)

eg 7 (uniform variance — MLF lecture). Let $X \sim \text{Uniform}[a, b]$ (PDF $1/(b-a)$ on $[a, b]$; §16.7). Using LOTUS:

$$E[X^2] = \int_a^b x^2 \frac{1}{b-a} dx = \frac{1}{b-a} \left[\frac{x^3}{3} \right]_a^b = \frac{b^3 - a^3}{3(b-a)} = \frac{a^2 + ab + b^2}{3}.$$

With $E[X] = (a+b)/2$ (eg 5's method, or §16.7):

$$\text{Var}(X) = \frac{a^2 + ab + b^2}{3} - \frac{(a+b)^2}{4} = \boxed{\frac{(b-a)^2}{12}}.$$

When $b = a$, the variance is 0: X is always a , a constant — the only random variables with zero variance (MLF lecture).

Basically, ... Variance = “average of the squares minus the square of the average” — the same formula as the discrete case, computed with integrals. Spread, not location.

16.7 The uniform distribution

Def. $X \sim \text{Uniform}[a, b]$ ($a < b$):

$$f_X(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b, \\ 0 & \text{otherwise,} \end{cases} \quad F_X(x) = \begin{cases} 0 & x \leq a, \\ \frac{x-a}{b-a} & a < x < b, \\ 1 & x \geq b. \end{cases}$$

The continuous version of §15.10’s discrete uniform: “equally likely” now means equal-length intervals get equal probability. Mean and variance (eg 7 and the calculation in §16.5):

$$E[X] = \frac{a+b}{2}, \quad \text{Var}(X) = \frac{(b-a)^2}{12}.$$

eg 8 (the deck’s problem). $X \sim \text{Uniform}[-10, 10]$, so $f_X(x) = 1/20$ on $[-10, 10]$:

- i) $P(-3 \leq X \leq 2) = \frac{2 - (-3)}{20} = \boxed{\frac{1}{4}}$ — interval length over total length.
- ii) $P(5 < |X| < 7)$: $|X| \in (5, 7)$ means $X \in (-7, -5) \cup (5, 7)$, total length 4: $4/20 = \boxed{1/5}$.
- iii) $P(X > 7 \mid X > 3) = \frac{P(X > 7)}{P(X > 3)} = \frac{(10-7)/20}{(10-3)/20} = \boxed{\frac{3}{7}}$.

Basically, ... Continuous uniform = “pick a point at random in $[a, b]$ ”. Every probability is a length ratio: favorable length divided by $b - a$. The mean is the midpoint; the spread grows with $(b - a)^2$.

16.8 The exponential distribution

Def. $X \sim \text{Exponential}(\lambda)$, $\lambda > 0$:

$$f_X(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0, \\ 0 & x < 0. \end{cases}$$

The continuous analogue of the geometric (§15.13): where the geometric models *number of trials* until success, the exponential models *time* until an event — the MLF lecture’s example is a bulb’s running time, $\lambda = 0.5$ per year. (Geometric: tries; exponential: time. Same shape of idea.)

Validity check (MLF lecture): $\int_0^\infty \lambda e^{-\lambda x} dx = [-e^{-\lambda x}]_0^\infty = 1$.

The CDF, for $x > 0$:

$$F_X(x) = \int_0^x \lambda e^{-\lambda u} du = 1 - e^{-\lambda x}$$

(0 for $x \leq 0$), so the tail is clean: $P(X > x) = e^{-\lambda x}$.

Mean and variance. $E[X] = \int_0^\infty x \lambda e^{-\lambda x} dx$. Integration by parts ($u = x$, $dv = e^{-\lambda x} dx$; MLF lecture’s derivation):

$$= \lambda \left(\left[-\frac{x}{\lambda} e^{-\lambda x} \right]_0^\infty + \int_0^\infty \frac{1}{\lambda} e^{-\lambda x} dx \right) = 0 + \int_0^\infty e^{-\lambda x} dx = \boxed{\frac{1}{\lambda}}.$$

(The boundary term vanishes: $e^{-\lambda x}$ beats x to zero.) The lecture's reading: small λ (rare failures) $\rightarrow$ large expected lifetime; $\lambda = 0.5/\text{year} \rightarrow E[X] = 2$ years. A second round of parts gives $E[X^2] = 2/\lambda^2$, hence (Stats 2 deck's table):

$$E[X] = \frac{1}{\lambda}, \quad \text{Var}(X) = \frac{1}{\lambda^2}.$$

Note: λ 's unit is “per unit time” — $E[X] = 1/\lambda$ then comes out in units of time, as it must.

Theorem (memorylessness). For $a > b \geq 0$:

$$P(X > a \mid X > b) = P(X > a - b).$$

Proof (MLF lecture): the left side is $\frac{P(X > a)}{P(X > b)} = \frac{e^{-\lambda a}}{e^{-\lambda b}} = e^{-\lambda(a-b)} = P(X > a - b)$. A bulb that has survived 1 year is, probabilistically, a brand-new bulb — the past is forgotten. The exponential is the *only* continuous distribution with this property (just as the geometric is the only discrete one, §15.13).

Note (minimum of exponentials — MLF lecture). If $X \sim \text{Exponential}(\lambda)$, $Y \sim \text{Exponential}(\tau)$ are independent, then $Z = \min(X, Y) \sim \text{Exponential}(\lambda + \tau)$: $P(Z > z) = P(X > z)P(Y > z) = e^{-\lambda z}e^{-\tau z} = e^{-(\lambda+\tau)z}$. The lecture's reading: a laptop whose CPU fails at rate 0.25/year and RAM at 0.25/year fails (first component down = laptop down) at rate 0.5/year — expected laptop life 2 years, not 4.

eg 9 (the deck's problem). $X \sim \text{Exponential}(2)$:

- i) $P(5 < X < 7) = F_X(7) - F_X(5) = (1 - e^{-14}) - (1 - e^{-10}) = e^{-10} - e^{-14} \approx \boxed{4.5 \times 10^{-5}}$.
- ii) $P(X > 7 \mid X > 3) = P(X > 4)$ by memorylessness $= e^{-2 \cdot 4} = e^{-8} \approx \boxed{3.35 \times 10^{-4}}$ — no need to touch the joint event.

Basically, ... Exponential = “waiting time for the next event, when events strike at a steady rate λ with no memory”. Mean $1/\lambda$, tail $e^{-\lambda x}$, and the past never matters — surviving b more years after surviving b is exactly as likely as surviving b fresh.

16.9 The normal distribution

Def. $Z \sim \text{Normal}(0, 1)$ (**standard normal**):

$$f_Z(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}, \quad z \in \mathbb{R}.$$

Is this a valid density? The constant $1/\sqrt{2\pi}$ must make the integral 1 — i.e. $\int_{-\infty}^{\infty} e^{-z^2/2} dz$ must equal $\sqrt{2\pi}$. This integral has no elementary antiderivative, but the MLF lecture's polar-coordinates trick evaluates it: with $A = \int_{-\infty}^{\infty} e^{-x^2/2} dx$,

$$A^2 = \iint_{\mathbb{R}^2} e^{-(x^2+y^2)/2} dx dy \stackrel{x=r\cos\theta, y=r\sin\theta}{=} \int_0^{2\pi} \int_0^{\infty} e^{-r^2/2} r dr d\theta.$$

With $u = r^2/2$ ($du = r dr$), the inner integral is $\int_0^{\infty} e^{-u} du = 1$; the outer is 2π . So $A^2 = 2\pi$, $A = \sqrt{2\pi}$. (The $r dr$ from $dx dy = r dr d\theta$ is what makes the integral solvable.)

Def (general normal). Put $X = \sigma Z + \mu$ with $Z \sim \text{Normal}(0, 1)$, $\sigma > 0$. By the change-of-variables rule ($z = (x - \mu)/\sigma$, Jacobian factor $1/\sigma$; MLF lecture):

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad X \sim \text{Normal}(\mu, \sigma^2).$$

Then, by linearity (§16.5) and the variance corollary (§16.6) — no integration needed:

$$E[X] = \sigma E[Z] + \mu = \mu, \quad \text{Var}(X) = \sigma^2 \text{Var}(Z) = \sigma^2,$$

since Z is symmetric about 0 ($E[Z] = 0$) and $\text{Var}(Z) = 1$. So μ is the **mean parameter**, σ^2 the **variance parameter** — the parameters *are* the mean and variance.

Standardization. If $X \sim \text{Normal}(\mu, \sigma^2)$, then

$$Z = \frac{X - \mu}{\sigma} \sim \text{Normal}(0, 1)$$

(Stats 2 deck). The normal CDF has no closed form, so probabilities are computed by standardizing and looking up $\Phi(z) = P(Z \leq z)$ in a normal table (or a computing system — Stats 2 deck).

Note (why the normal is everywhere). The MLF lecture’s answer: the central limit theorem (Chapter 21) — whenever many small independent effects add up, the total is approximately normal. Measurement noise, e.g. an ammeter’s reading with no voltage applied, is the textbook example. “Gaussian” = normal, same distribution.

eg 10. $X \sim \text{Normal}(2, 5)$ (so $\sigma = \sqrt{5} \approx 2.236$). Standardize:

- i) $P(X < 5) = P\left(Z < \frac{5-2}{\sqrt{5}}\right) = \Phi(1.3416) \approx \boxed{0.9101}$.
- ii) $P(X > 10) = 1 - \Phi\left(\frac{10-2}{\sqrt{5}}\right) = 1 - \Phi(3.5777) \approx \boxed{1.7 \times 10^{-4}}$.

eg 11. $X \sim \text{Normal}(3, 1)$:

- i) $P(5 < X < 7) = \Phi(4) - \Phi(2) \approx 0.99997 - 0.97725 = \boxed{0.0227}$.
- ii) $P(X > 7 \mid X > 3) = \frac{P(X > 7)}{P(X > 3)} = \frac{1 - \Phi(4)}{1 - \Phi(0)} \approx \frac{0.0000317}{0.5} = \boxed{6.3 \times 10^{-5}}$. No memorylessness here — the normal remembers; the ratio must be computed directly (contrast eg 9(ii)).

Basically, ... Normal = the bell curve. Two parameters that *are* the mean and variance. To compute any probability: subtract the mean, divide by the standard deviation, look up the standard-normal table. And it shows up everywhere because sums of many small effects are approximately normal (Chapter 21 makes this precise).

16.10 Functions of a continuous random variable

If X has CDF F_X and $Y = g(X)$, the CDF method (Stats 2 deck):

$$F_Y(y) = P(Y \leq y) = P(g(X) \leq y) = P(X \in \{x : g(x) \leq y\}),$$

then $f_Y = F'_Y$ — the continuous twin of §15.6’s “pool the probabilities” rule, now via calculus.

eg 12 (the deck’s example). $X \sim \text{Uniform}[0, 1]$, $Y = 2X$. For $0 \leq y \leq 2$:

$$F_Y(y) = P(2X \leq y) = P(X \leq \frac{y}{2}) = \int_0^{y/2} 1 \, dx = \frac{y}{2},$$

so $f_Y(y) = F'_Y(y) = 1/2$ on $[0, 2]$: $Y \sim \text{Uniform}[0, 2]$. (Scaling stretches the interval and thins the density so the area stays 1.)

For a *strictly monotone* g there is a direct formula: with $x = g^{-1}(y)$,

$$f_Y(y) = f_X(g^{-1}(y)) \left| \frac{d}{dy} g^{-1}(y) \right|$$

on Y 's range. This is what §16.9 used: $x = \sigma z + \mu$ has inverse $z = (x - \mu)/\sigma$ with derivative $1/\sigma$, giving the general normal PDF from the standard one.

Basically, ... $g(X)$ inherits its distribution from X 's: the CDF method (F_Y first, differentiate) always works; for monotone g the Jacobian formula does it in one step. And LOTUS (§16.5) means you usually need $g(X)$'s *expectation*, not its whole distribution.

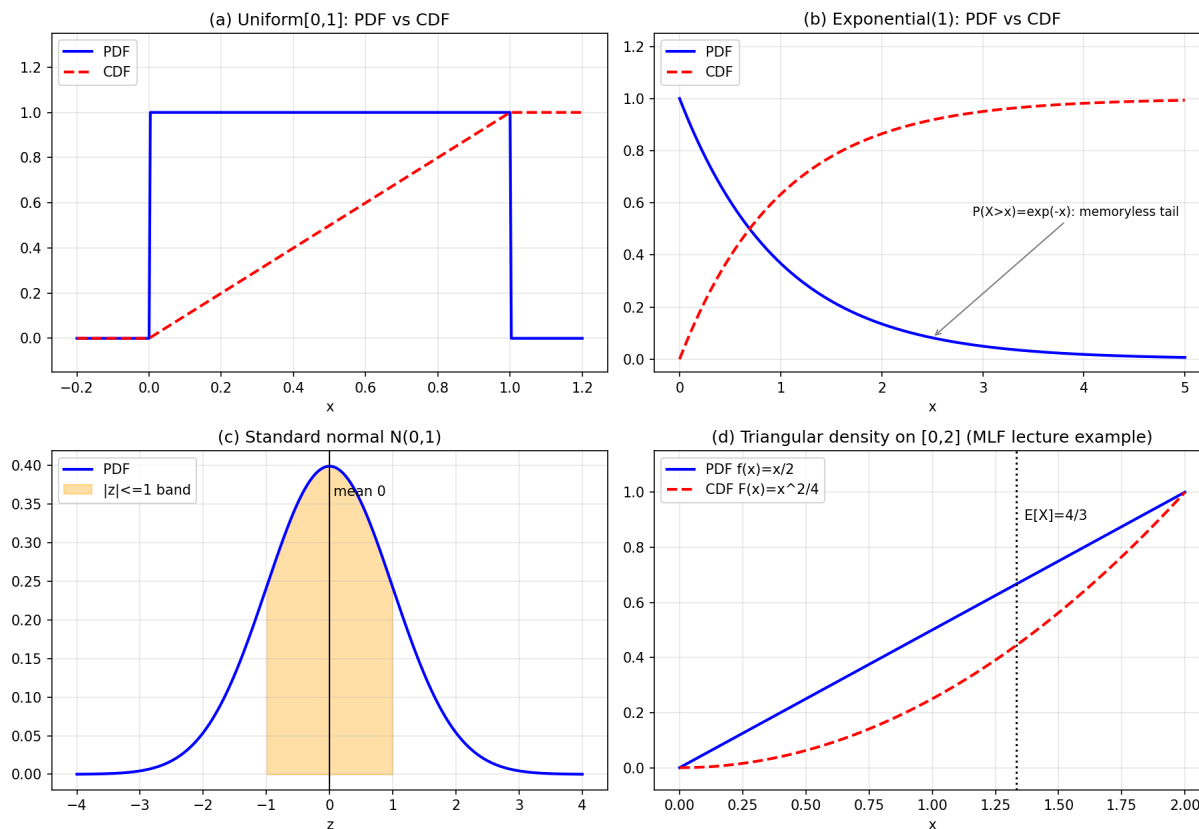


Figure 18: (a) Uniform[0,1]: flat PDF vs ramp CDF. (b) Exponential(1): decaying PDF vs saturating CDF, with the memoryless tail $P(X > x) = \exp(-x)$. (c) Standard normal PDF with the $|z| \leq 1$ band shaded. (d) The MLF lecture's triangular density $f(x) = x/2$ on $[0, 2]$ and its CDF $F(x) = x^2/4$, with $E[X] = 4/3$ marked.

16.11 Why this matters: the same language, a bigger world

- i) **The lift is complete.** PMF $\rightarrow$ PDF (§16.3), CDF the same definition (§16.4), sums $\rightarrow$ integrals (§§16.5–16.6), and the three workhorse distributions — uniform, exponential, normal — each the continuous sibling of a discrete one (§§15.10, 15.13, and the normal's own story). §15.17's promise, delivered.
- ii) **These are the ML distributions.** Uniform: random initialization ranges. Exponential: waiting times, survival/failure modeling. Normal: measurement noise, weight priors, the limiting shape of averaged quantities (Chapter 21). You will meet all three constantly from here on.
- iii) **What comes next.** Chapters 17–18 put two random variables together (joint densities — the object the linearity proof already leaned on); Chapter 19 builds the multivariate normal; Chapter 20 estimates parameters (MLE) of exactly these distributions; Chapter 21's central limit theorem explains *why* the normal keeps appearing.

Problem set

- (PDF validity) $f(x) = 3x^2$ for $0 < x < 1$ (0 otherwise). (a) Show f is a valid density. (b) Find $P(X = 1/5)$ and $P(X \in [1/5 - \epsilon, 1/5 + \epsilon])$ for small $\epsilon > 0$.
- (Finding the constant) $f(x) = k$ on $[0, 1/4)$, $2k$ on $[1/4, 3/4)$, $3k$ on $[3/4, 1)$ (0 otherwise). Find k ; then find $P(X \leq 1/2)$.
- (CDF from PDF) $f_X(x) = x/2$ for $0 \leq x \leq 2$ (0 otherwise). (a) Write $F_X(x)$ piecewise. (b) Find $P(0.5 < X \leq 1.5)$.
- (Uniform) $X \sim \text{Uniform}[-10, 10]$. Find $P(-3 \leq X \leq 2)$, $P(5 < |X| < 7)$, and $P(X > 7 \mid X > 3)$.
- (Expectation and variance) $f_X(x) = 2x$ for $0 < x < 1$ (0 otherwise). Find $E[X]$ and $\text{Var}(X)$.
- (Exponential probabilities) $X \sim \text{Exponential}(2)$. Find $P(5 < X < 7)$ and $P(X > 7 \mid X > 3)$.
- (Exponential mean) A bulb's lifetime X is $\text{Exponential}(\lambda)$ with $\lambda = 0.5$ per year. Find $E[X]$ and $P(X > 2)$.
- (Memorylessness) $X \sim \text{Exponential}(0.3)$: a component's lifetime in years. Given it has survived 4 years, find $P(X > 10 \mid X > 4)$.
- (Normal) $X \sim \text{Normal}(3, 1)$. Find $P(5 < X < 7)$ and $P(X > 7 \mid X > 3)$. (Use a standard-normal table.)
- (Minimum of exponentials) A laptop's CPU and RAM have independent failure times $X, Y \sim \text{Exponential}(0.25)$ per year; the laptop fails when the first component fails, $Z = \min(X, Y)$. Find the distribution of Z and $P(Z \leq 1)$.

Solutions — 16. Continuous random variables

1. (PDF validity) (a) $f(x) = 3x^2 \geq 0$ on $(0, 1)$; $\int_{-\infty}^{\infty} f(x) dx = \int_0^1 3x^2 dx = [x^3]_0^1 = 1$; piecewise continuous. All three density properties (§16.3) hold, so f is valid.

(b) $P(X = 1/5) = \boxed{0}$ — every single point has probability zero for a continuous random variable (§16.2). For the interval,

$$P(X \in [1/5 - \epsilon, 1/5 + \epsilon]) = \int_{1/5 - \epsilon}^{1/5 + \epsilon} 3x^2 dx = [x^3]_{1/5 - \epsilon}^{1/5 + \epsilon} = \boxed{(1/5 + \epsilon)^3 - (1/5 - \epsilon)^3} \approx \frac{6\epsilon}{25}$$

for small ϵ (using $P \approx f(1/5) \cdot 2\epsilon$). Positive, but shrinking to 0 with ϵ — precision costs probability.

2. (Finding the constant) Integrate to 1:

$$1 = k \cdot \frac{1}{4} + 2k \cdot \frac{1}{2} + 3k \cdot \frac{1}{4} = \frac{k}{4} + k + \frac{3k}{4} = 2k,$$

so $\boxed{k = 1/2}$. Then

$$P(X \leq 1/2) = \int_0^{1/4} \frac{1}{2} dx + \int_{1/4}^{1/2} 1 dx = \frac{1}{2} \cdot \frac{1}{4} + 1 \cdot \frac{1}{4} = \frac{1}{8} + \frac{1}{4} = \boxed{\frac{3}{8}}.$$

3. (CDF from PDF) (a) For $0 \leq x \leq 2$: $F_X(x) = \int_0^x (u/2) du = x^2/4$. So

$$F_X(x) = \begin{cases} 0 & x < 0, \\ x^2/4 & 0 \leq x \leq 2, \\ 1 & x > 2. \end{cases}$$

Check: $F_X(2) = 1$, and $F'_X = x/2 = f_X$ on $(0, 2)$.

(b) $P(0.5 < X \leq 1.5) = F_X(1.5) - F_X(0.5) = \frac{2.25}{4} - \frac{0.25}{4} = \boxed{0.5}$.

4. (Uniform) $f_X(x) = 1/20$ on $[-10, 10]$ (§16.7).

$$P(-3 \leq X \leq 2) = \frac{2 - (-3)}{20} = \boxed{1/4} \text{ (length ratio).}$$

$$P(5 < |X| < 7): |X| \in (5, 7) \text{ iff } X \in (-7, -5) \cup (5, 7), \text{ total length } 2 + 2 = 4: 4/20 = \boxed{1/5}.$$

$$P(X > 7 \mid X > 3) = \frac{P(X > 7)}{P(X > 3)} = \frac{(10 - 7)/20}{(10 - 3)/20} = \boxed{3/7}.$$

5. (Expectation and variance) $E[X] = \int_0^1 x \cdot 2x \, dx = \int_0^1 2x^2 \, dx = \left[\frac{2x^3}{3} \right]_0^1 = \boxed{2/3}.$

$$E[X^2] = \int_0^1 x^2 \cdot 2x \, dx = \int_0^1 2x^3 \, dx = \left[\frac{x^4}{2} \right]_0^1 = 1/2.$$

$$\text{Var}(X) = E[X^2] - (E[X])^2 = \frac{1}{2} - \frac{4}{9} = \frac{9-8}{18} = \boxed{1/18}, \sigma = 1/\sqrt{18} \approx 0.236.$$

6. (Exponential probabilities) $F_X(x) = 1 - e^{-2x}$ for $x > 0$ (§16.8).

$$P(5 < X < 7) = F_X(7) - F_X(5) = (1 - e^{-14}) - (1 - e^{-10}) = e^{-10} - e^{-14} \approx \boxed{4.5 \times 10^{-5}}.$$

$$P(X > 7 \mid X > 3) = P(X > 7 - 3) = P(X > 4) \text{ by memorylessness} = e^{-2 \cdot 4} = e^{-8} \approx \boxed{3.35 \times 10^{-4}}.$$

7. (Exponential mean) $E[X] = 1/\lambda = 1/0.5 = \boxed{2 \text{ years}}$ (§16.8). $P(X > 2) = e^{-\lambda \cdot 2} = e^{-1} \approx \boxed{0.3679}$ — about a 37% chance the bulb outlives its own mean lifetime (the exponential's right skew at work).

8. (Memorylessness) $P(X > 10 \mid X > 4) = P(X > 10 - 4) = P(X > 6) = e^{-0.3 \cdot 6} = e^{-1.8} \approx \boxed{0.1653}$ (§16.8). The 4 survived years are forgotten — same answer as for a brand-new component asked to survive 6 years. (Compare §15's Problem 8: the geometric version.)

9. (Normal) Standardize with $Z = (X - 3)/1 = X - 3$.

$$P(5 < X < 7) = P(2 < Z < 4) = \Phi(4) - \Phi(2) \approx 0.99997 - 0.97725 = \boxed{0.0227}.$$

$$P(X > 7 \mid X > 3) = \frac{P(X > 7)}{P(X > 3)} = \frac{1 - \Phi(4)}{1 - \Phi(0)} \approx \frac{0.0000317}{0.5} = \boxed{6.3 \times 10^{-5}}. \text{ (The normal has no memorylessness — the ratio must be computed directly; contrast Problem 8.)}$$

10. (Minimum of exponentials) $P(Z > z) = P(X > z)P(Y > z) = e^{-0.25z}e^{-0.25z} = e^{-0.5z}$ for $z \geq 0$ (§16.8, Note). So $\boxed{Z \sim \text{Exponential}(0.5)}$ per year — the laptop fails twice as fast as either part alone, with $E[Z] = 2$ years.

$$P(Z \leq 1) = 1 - e^{-0.5} \approx \boxed{0.3935} \text{ — about a 39\% chance the laptop dies within its first year.}$$

17. Joint distributions: two random variables

Chapters 15–16 built the full toolkit for *one* random variable: PMF/PDF, CDF, expectation, variance, and the named distributions. Real experiments rarely carry just one quantity of interest, though: toss a coin and you may care about the outcome of *each* toss; pick a two-digit number and you may care about its units digit *and* its remainder mod 4. Chapter 17 puts two random variables together — the joint PMF/PDF, marginals, conditionals, independence, the joint CDF, $E[g(X, Y)]$, covariance, correlation, and $\text{Var}(X + Y)$. This is also the object that §§15.8 and 16.5 leaned on: the proof of linearity of expectation for sums was deferred until the joint distribution existed. It exists now.

17.1 Two random variables, one experiment

Def. X and Y are **jointly distributed** (or “defined on the same probability space”) = X and Y are both functions of the outcome of the *same* experiment (§15.1).

The Stats 2 deck’s three motivating examples:

- i) **Toss a coin thrice.** $X_i = 1$ if the i -th toss is heads, 0 if tails, $i = 1, 2, 3$. Three random variables on one experiment; together X_1, X_2, X_3 completely describe the outcome.
- ii) **A random 2-digit number, 00–99.** X = units digit, Y = remainder on division by 4. Two random variables on one experiment. Here knowing X tells you something about Y (a number ending in 1 cannot be a multiple of 4).
- iii) **An IPL powerplay over.** X = runs scored, Y = wickets fallen. A model that ignores the X – Y relationship is a bad model: expect larger X when $Y = 0$ than when $Y = 2$.

So the new question is not “what does X do?” or “what does Y do?” but “what do X and Y do *together*?” — and in particular, how one changes what you know about the other.

Note. Everything here is stated for two variables. The same ideas extend to n (the deck’s §1.5), but this chapter stays at two; Chapter 19’s multivariate normal is the first place this book needs more.

Basically, ... One random variable = one question about an experiment. Two random variables = two questions about the *same* experiment. The interesting part is the relationship between the two answers: sometimes they have nothing to do with each other (separate coin tosses), sometimes one pins down the other (units digit vs. remainder mod 4).

17.2 The joint PMF

Def. Let X, Y be discrete random variables on the same probability space, with ranges T_X, T_Y . Their **joint PMF** is the function $f_{X,Y} : T_X \times T_Y \rightarrow [0, 1]$,

$$\boxed{f_{X,Y}(x, y) = P(X = x \text{ and } Y = y) = P(X = x, Y = y)}, \quad x \in T_X, y \in T_Y.$$

Two properties, straight from the probability axioms (§14.6):

- i) Every value satisfies $0 \leq f_{X,Y}(x, y) \leq 1$ — it *is* a probability.
- ii) All the values sum to 1: $\sum_{x \in T_X} \sum_{y \in T_Y} f_{X,Y}(x, y) = 1$ — the (x, y) pairs partition the sample space (§14.5).

The joint PMF is usually written as a **table** (or matrix): X -values along one axis, Y -values along the other, $P(X = x, Y = y)$ in cell (x, y) .

eg 1 (two coin tosses — the deck's example). Toss a fair coin twice; $X_i = 1$ if toss i is heads, 0 if tails. The four outcomes are equally likely:

$$f_{X_1, X_2}(0, 0) = P(X_1 = 0, X_2 = 0) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4},$$

and the same for the other three cells. The joint PMF table:

$X_1 \setminus X_2$	0	1
0	1/4	1/4
1	1/4	1/4

eg 2 (random 2-digit number — the deck's example). A number from 00 to 99 is equally likely. X = units digit (0–9), Y = number mod 4 (0–3):

- i) $f_{X,Y}(0, 0)$: units digit 0 *and* divisible by 4 — the numbers 00, 20, 40, 60, 80. That's 5 of 100: $\boxed{1/20}$.
- ii) $f_{X,Y}(1, 0)$: units digit 1 *and* divisible by 4 — impossible (odd numbers are never 0 mod 4): $\boxed{0}$.
- iii) $f_{X,Y}(4, 2)$: units digit 4 *and* $\equiv 2 \pmod{4}$ — the numbers 14, 34, 54, 74, 94: $\boxed{1/20}$.

Every cell is 0 or 1/20 — a checkerboard with holes where the X – Y combination is impossible (panel (a) of the figure). The 100 cells sum to 1 .

Basically, ... The joint PMF is the PMF of the *pair*: one probability for every (x, y) combination. The table form is the working tool — row for one variable, column for the other, joint probability in each cell, everything summing to 1.

17.3 Marginal PMFs: summing out

Def. Given the joint PMF $f_{X,Y}$, the PMFs of the individual variables X and Y are called the **marginal PMFs**. To get them, **sum out** the other variable:

$$f_X(x) = \sum_{y \in T_Y} f_{X,Y}(x, y), \quad f_Y(y) = \sum_{x \in T_X} f_{X,Y}(x, y).$$

Why it works (the deck's proof): fix x ; the events $\{X = x, Y = y\}$ as y ranges over T_Y are disjoint and their union is $\{X = x\}$ — a partition (§14.5). So by the law of total probability (§14.10),

$$P(X = x) = \sum_{y \in T_Y} P(X = x, Y = y).$$

This step is called **marginalization**. In the table: f_X is the row totals, f_Y the column totals.

- i) **Joint $\rightarrow$ marginal is unique.** One joint table gives exactly one pair of marginals.
- ii) **Marginal joint.** The reverse is *not* unique: many different joint tables can produce the same marginals. You cannot reconstruct the relationship between X and Y from their individual distributions alone.

eg 3 (marginals from the table — the deck’s example). The joint PMF

$X_1 \setminus X_2$	0	1
0	0.05	0.35
1	0.25	0.35

is valid: all entries ≥ 0 , sum = 1.00 . Summing out:

$$f_{X_1}(0) = 0.05 + 0.25 = \boxed{0.30}, \quad f_{X_1}(1) = 0.35 + 0.35 = \boxed{0.70},$$

$$f_{X_2}(0) = 0.05 + 0.35 = \boxed{0.40}, \quad f_{X_2}(1) = 0.25 + 0.35 = \boxed{0.60}.$$

eg 4 (same marginals, different joints — the deck’s example). For any $x \in [0, 1/2]$ the table

$X_1 \setminus X_2$	0	1
0	x	$1/2 - x$
1	$1/2 - x$	x

has marginals $f_{X_1}(0) = f_{X_1}(1) = f_{X_2}(0) = f_{X_2}(1) = 1/2$ — *regardless of x* . Infinitely many different joints, identical marginals. (The deck’s warning made concrete: seeing the marginals and writing down $f_{X,Y}(x, y) = f_X(x) f_Y(y)$ is only *one* of these possibilities — the $x = 1/4$ one — and it may be wrong. §17.5 names when it is right.)

Basically, ... Marginal = “the X story alone”, obtained by adding up each row (or column) of the joint table and forgetting the other variable. But the marginals don’t remember how the two variables interact — that information lives only in the full joint table.

17.4 Conditional PMFs and the multiplication rule

Def. The **conditional PMF of Y given $X = x$** is

$$f_{Y|X=x}(y) = P(Y = y \mid X = x) = \frac{f_{X,Y}(x, y)}{f_X(x)}, \quad f_X(x) > 0.$$

It is exactly the conditional probability of §14.9, with the joint PMF in the numerator.

- i) For fixed x , $f_{Y|X=x}$ is a genuine PMF: $\sum_y f_{Y|X=x}(y) = \sum_y f_{X,Y}(x, y)/f_X(x) = f_X(x)/f_X(x) = 1$.
- ii) **Conditioning can change the range.** ($Y \mid X = x$) only takes values y with $f_{X,Y}(x, y) > 0$ — knowing X can rule out Y -values entirely (as eg 2’s zeros showed).
- iii) Rearranging the definition gives the **multiplication rule**:

$$f_{X,Y}(x, y) = f_X(x) f_{Y|X=x}(y),$$

the joint built from a marginal and a conditional — the workhorse for *constructing* joint PMFs when the experiment has a natural order.

eg 5 (conditionals from the table — the deck’s example). Joint PMF on $X \in \{0, 1, 2\}$, $Y \in \{0, 1\}$:

$X \setminus Y$	0	1
0	$1/4$	$1/8$

$X \setminus Y$	0	1
1	$1/8$	$1/8$
2	$1/8$	$1/4$

Marginals: $f_X(0) = 3/8$, $f_X(1) = 1/4$, $f_X(2) = 3/8$; $f_Y(0) = f_Y(1) = 1/2$.

$$f_{Y|X=0}(0) = \frac{1/4}{3/8} = \boxed{\frac{2}{3}}, \quad f_{Y|X=0}(1) = \frac{1/8}{3/8} = \boxed{\frac{1}{3}} \quad (\text{sums to } 1),$$

$$f_{X|Y=1}(0) = \frac{1/8}{1/2} = \boxed{\frac{1}{4}}.$$

eg 6 (die, then coins — the deck’s example). Throw a die; let X be the number shown. Toss a coin that many times; let Y be the number of heads. Then

$$f_X(t) = \frac{1}{6} \quad (t = 1, \dots, 6), \quad (Y \mid X = t) \sim \text{Binomial}(t, \frac{1}{2}) \quad (\S 15.12),$$

so $f_{Y|X=t}(t') = \binom{t}{t'} (1/2)^t$ for $t' = 0, \dots, t$. By the multiplication rule:

$$f_{X,Y}(t, t') = \frac{1}{6} \binom{t}{t'} \left(\frac{1}{2}\right)^t, \quad t = 1, \dots, 6, \quad t' = 0, \dots, t.$$

The conditional was easy to write down; the joint follows for free. This is the standard modeling pattern: specify what happens first, then what happens *given* it.

Basically, ... Conditional PMF = “zoom in on one column (or row) of the joint table and re-normalize it to sum to 1”. The multiplication rule reads the same table backwards: joint = marginal $\times$ conditional. If your experiment has an order (die first, coins after), build the joint from the conditional — don’t try to write the joint directly.

17.5 Independence of random variables

Def. X and Y are **independent** if their joint PMF factors into the product of the marginals, for *every* pair:

$$f_{X,Y}(x, y) = f_X(x) f_Y(y) \quad \text{for all } x \in T_X, \quad y \in T_Y.$$

This lifts §14.12 (independence of *events*) to random variables: X and Y are independent iff every event $\{X = x\}$ is independent of every event $\{Y = y\}$.

- i) **To prove independence:** check the factorization at *all* (x, y) .
- ii) **To disprove independence:** *one* counterexample suffices: $f_{X,Y}(x, y) \neq f_X(x) f_Y(y)$ for a single pair.
- iii) **The zero-cell shortcut** (the deck’s special case): if $f_{X,Y}(x, y) = 0$ while $f_X(x) > 0$ and $f_Y(y) > 0$, then X and Y are dependent — independence would require $f_{X,Y}(x, y) = f_X(x) f_Y(y) > 0$. Impossible combinations are the fastest dependence detector.

eg 7 (independence tests — the deck’s examples).

- i) **Two coin tosses (eg 1):** $f_{X_1, X_2}(0, 0) = 1/4 = (1/2)(1/2) = f_{X_1}(0) f_{X_2}(0)$, and the same at all four cells. Independent.
- ii) **The 2-digit number (eg 2):** $f_{X,Y}(1, 0) = 0 \neq (1/10)(1/4) = 1/40 = f_X(1) f_Y(0)$. Not independent — one counterexample is enough (and it’s the zero-cell case).
- iii) **The x -family (eg 4):** $f_{X_1, X_2}(0, 0) = x$ vs. $f_{X_1}(0) f_{X_2}(0) = 1/4$. Independent iff $\boxed{x = 1/4}$; dependent for every other $x \in [0, 1/2]$.

Basically, ... Independent = “the table is the product of its margins” — knowing X changes nothing about Y ’s distribution, so $f_{Y|X=x}(y) = f_Y(y)$ for every x . To *break* independence you only need a single cell where the product rule fails; a 0 in a cell whose row and column both have positive mass is the classic giveaway.

17.6 The joint CDF

Def. The **joint CDF** of X and Y is

$$F_{X,Y}(x, y) = P(X \leq x, Y \leq y) = \sum_{u \leq x} \sum_{v \leq y} f_{X,Y}(u, v), \quad x, y \in \mathbb{R}.$$

Same definition as §15.5, in two dimensions: probability of the rectangle $(-\infty, x] \times (-\infty, y]$.

- i) Limits: $F_{X,Y} \rightarrow 0$ when $x \rightarrow -\infty$ or $y \rightarrow -\infty$; $F_{X,Y} \rightarrow 1$ when both $x, y \rightarrow +\infty$.
- ii) Non-decreasing in each argument separately.
- iii) **Rectangle formula.** For $a < b, c < d$:

$$P(a < X \leq b, c < Y \leq d) = F_{X,Y}(b, d) - F_{X,Y}(a, d) - F_{X,Y}(b, c) + F_{X,Y}(a, c)$$

(add the whole rectangle, subtract the two strips, add back the doubly-subtracted corner — inclusion-exclusion, §14.7).

- iv) The marginal CDFs fall out: $F_X(x) = \lim_{y \rightarrow +\infty} F_{X,Y}(x, y)$ — push one bound to infinity and the other variable is summed out.

eg 8 (joint CDF of two coin tosses). With eg 1's table (1/4 in each cell):

$$F_{X_1, X_2}(0, 0) = \frac{1}{4}, \quad F_{X_1, X_2}(0, 1) = \frac{1}{2}, \quad F_{X_1, X_2}(1, 0) = \frac{1}{2}, \quad F_{X_1, X_2}(1, 1) = 1.$$

Check the rectangle formula on $(0, 1] \times (0, 1]$:

$$P(X_1 = 1, X_2 = 1) = F(1, 1) - F(0, 1) - F(1, 0) + F(0, 0) = 1 - \frac{1}{2} - \frac{1}{2} + \frac{1}{4} = \boxed{\frac{1}{4}}.$$

Basically, ... The joint CDF is the running total in two dimensions — “how much probability has piled up in the rectangle below-left of (x, y) ”. Differences of CDF values give rectangle probabilities, exactly as $F_X(b) - F_X(a)$ gives interval probabilities (§15.5).

17.7 Expectation of functions of two random variables

Def (two-variable LOTUS). For $Z = g(X, Y)$,

$$E[g(X, Y)] = \sum_{x \in T_X} \sum_{y \in T_Y} g(x, y) f_{X,Y}(x, y)$$

— average the transformed values with the *joint* PMF, no need to find Z 's distribution first. (The continuous version swaps the sums for a double integral; §17.12.)

The headline application is the sum — and with the joint PMF in hand, linearity finally gets its proof:

Theorem (linearity for sums — the deferred proof from §§15.8, 16.5).

$$E[X + Y] = E[X] + E[Y],$$

no independence needed. *Proof:*

$$\begin{aligned} E[X + Y] &= \sum_x \sum_y (x + y) f_{X,Y}(x, y) \\ &= \sum_x x \underbrace{\sum_y f_{X,Y}(x, y)}_{f_X(x)} + \sum_y y \underbrace{\sum_x f_{X,Y}(x, y)}_{f_Y(y)} \quad (\text{marginalization, §17.3}) \\ &= E[X] + E[Y]. \quad \blacksquare \end{aligned}$$

The joint PMF was the missing piece all along: the double sum splits because marginalization collapses each inner sum.

Note (total expectation, two-variable form). Conditioning the other way: $E[Y] = \sum_x E[Y | X = x] f_X(x)$ — the law of total probability (§14.10) with expectations. This is how eg 6-style “two-stage” experiments are averaged.

eg 9 ($E[XY]$ on the table). On eg 5’s table:

$$E[XY] = \sum_x \sum_y xy f_{X,Y}(x, y) = 1 \cdot 1 \cdot \frac{1}{8} + 2 \cdot 1 \cdot \frac{1}{4} = \boxed{\frac{5}{8}},$$

the other terms being 0. (Watch for this number — §17.9 uses it.)

eg 10 (die, then coins — total expectation). With eg 6’s setup, $(Y | X = t) \sim \text{Binomial}(t, 1/2)$ has mean $t/2$ (§15.12), and $E[X] = 7/2$:

$$E[Y] = \sum_{t=1}^6 E[Y | X = t] f_X(t) = \sum_{t=1}^6 \frac{t}{2} \cdot \frac{1}{6} = \frac{E[X]}{2} = \boxed{\frac{7}{4}}.$$

No joint table was ever written — the conditional did all the work.

Basically, ... Two-variable LOTUS: to average any function of the pair, weight each cell’s $g(x, y)$ by the cell’s probability and add up. And linearity $E[X + Y] = E[X] + E[Y]$ is *not* a separate axiom — it falls out of the double sum the moment the joint PMF exists. Always true, independence irrelevant.

17.8 The distribution of $g(X, Y)$: pooling over contours

LOTUS (§17.7) gives *expectations* without the distribution of $Z = g(X, Y)$. When the distribution itself is needed, pool the joint PMF over the **contours** $\{g(x, y) = z\}$ (the deck’s §2.3.3):

$$P(Z = z) = \sum_{(x, y): g(x, y) = z} f_{X,Y}(x, y)$$

— add up every cell of the joint table that the function maps to z . (One-variable version: §15.6.)

eg 11 (sum of two uniforms — the deck’s example). $X, Y \stackrel{\text{i.i.d.}}{\sim} \text{Uniform}\{0, 1, 2, 3\}$, $Z = X + Y$. The joint cells are all $1/16$ (independence, §17.5). Counting the (x, y) with $x + y = z$:

z	0	1	2	3	4	5	6
$f_Z(z)$	1/16	2/16	3/16	4/16	3/16	2/16	1/16

Note: the sum of two uniforms is *not* uniform — it peaks in the middle (the deck’s “Try it yourself” warning). Pooling over the anti-diagonal contours makes the triangular shape obvious: more (x, y) pairs land on middle sums.

Basically, ... The distribution of $g(X, Y)$ = walk the joint table, and for each value z , collect all cells where g equals z . Sums, maxima, minima all work this way; the contours tell you which cells to pool. (The continuous version — convolution integrals and max/min via CDFs — is Chapter 18.)

17.9 Covariance

Def. The **covariance** of X and Y is

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])],$$

the average product of the *deviations* from the means. Expanding (linearity, §17.7):

$$\boxed{\text{Cov}(X, Y) = E[XY] - E[X]E[Y]}.$$

- i) Sign: $\text{Cov} > 0$ means X and Y tend to be above (or below) their means *together*; $\text{Cov} < 0$ means one is above when the other is below; $\text{Cov} = 0$ means no *linear* co-movement — X and Y are **uncorrelated**.
- ii) $\text{Cov}(X, X) = \text{Var}(X)$ — variance is self-covariance (§15.9).
- iii) Symmetric: $\text{Cov}(X, Y) = \text{Cov}(Y, X)$.
- iv) $\text{Cov}(aX + b, cY + d) = ac \text{Cov}(X, Y)$ — shifts don't matter, scales multiply.

Theorem (independence uncorrelated). If X, Y are independent, $\text{Cov}(X, Y) = 0$. *Proof:* $E[XY] = \sum_x \sum_y xy f_X(x) f_Y(y) = (\sum_x x f_X(x))(\sum_y y f_Y(y)) = E[X]E[Y]$.

Warning — the converse is false. Zero covariance does *not* imply independence. Covariance only sees *linear* co-movement; a perfect *nonlinear* relationship can have $\text{Cov} = 0$.

eg 12 (covariance on the table). On eg 5's table: $E[X] = 1$, $E[Y] = 1/2$ (check: $0 \cdot \frac{3}{8} + 1 \cdot \frac{1}{4} + 2 \cdot \frac{3}{8} = 1$), and $E[XY] = 5/8$ (eg 9):

$$\text{Cov}(X, Y) = \frac{5}{8} - 1 \cdot \frac{1}{2} = \boxed{\frac{1}{8}} > 0$$

— positive: larger X comes with larger Y (look at the table: the mass sits on $(0, 0)$ and $(2, 1)$).

eg 13 (uncorrelated but dependent — a hand-checked counterexample). Let $X \in \{-1, 0, 1\}$ with probability $1/3$ each, and $Y = X^2$. Then $E[X] = 0$, $E[Y] = (1 + 0 + 1)/3 = 2/3$, and

$$E[XY] = E[X^3] = \frac{(-1)^3 + 0^3 + 1^3}{3} = 0,$$

so $\text{Cov}(X, Y) = 0 - 0 \cdot \frac{2}{3} = \boxed{0}$ — uncorrelated. But Y is *determined by* X : e.g. $f_{X,Y}(0, 1) = 0 \neq (1/3)(2/3) = f_X(0)f_Y(1)$ (zero-cell test, §17.5). Dependence with exactly zero covariance — panel (d) of the figure.

Basically, ... Covariance = “do the two variables move together?” measured as the average product of their deviations. Independent variables always have covariance 0 — but 0 covariance doesn't mean independent: it only rules out *linear* relationships. A *U-shaped* relationship (like $Y = X^2$) slips through with covariance exactly 0.

17.10 The correlation coefficient

Def. The **correlation coefficient** (Pearson) of X and Y is

$$\boxed{\rho(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}}, \quad \sigma_X, \sigma_Y > 0.$$

Covariance has awkward units (units of $X \times$ units of Y); dividing by the standard deviations (§15.9) makes ρ **unitless** and comparable across problems.

Theorem. $\boxed{-1 \leq \rho(X, Y) \leq 1}$. *Proof:* standardize $U = (X - E[X])/\sigma_X$, $V = (Y - E[Y])/\sigma_Y$, so $\text{Var}(U) = \text{Var}(V) = 1$ and $\text{Cov}(U, V) = \rho$. Then $0 \leq \text{Var}(U + V) = 2 + 2\rho$ gives $\rho \geq -1$, and $0 \leq \text{Var}(U - V) = 2 - 2\rho$ gives $\rho \leq 1$.

- i) $|\rho| = 1$ iff Y is an exact affine function of X ($Y = aX + b$): $+1$ for $a > 0$, -1 for $a < 0$.
- ii) $\rho = 0$ = uncorrelated — same warning as §17.9: linear silence, not independence.

eg 14. On eg 5's table: $\text{Cov}(X, Y) = 1/8$ (eg 12). $E[X^2] = 0 + 1 \cdot \frac{1}{4} + 4 \cdot \frac{3}{8} = 7/4$, so $\text{Var}(X) = 7/4 - 1 = 3/4$; $\text{Var}(Y) = 1/2 - 1/4 = 1/4$ (Bernoulli-ish, §15.11). Hence

$$\rho(X, Y) = \frac{1/8}{\sqrt{3/4} \sqrt{1/4}} = \frac{1/8}{\sqrt{3}/4} = \boxed{\frac{1}{2\sqrt{3}} \approx 0.29},$$

a mild positive linear relationship — consistent with the table’s diagonal lean.

Basically, ... Correlation = covariance rescaled to live in $[-1, 1]$: $+1$ = perfect lockstep, -1 = perfect mirror, 0 = no linear relationship. It inherits covariance’s blind spot for nonlinear patterns, so never read it as “dependence strength” in general.

17.11 Variance of a sum

Theorem. For any two random variables,

$$\boxed{\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2 \text{Cov}(X, Y)}.$$

Proof: write $\mu_X = E[X]$, $\mu_Y = E[Y]$ and expand:

$$\text{Var}(X + Y) = E[((X - \mu_X) + (Y - \mu_Y))^2] = E[(X - \mu_X)^2] + E[(Y - \mu_Y)^2] + 2E[(X - \mu_X)(Y - \mu_Y)].$$

Corollary (the deferred case from §16.6). If X and Y are *independent*, $\text{Cov}(X, Y) = 0$ (§17.9), so

$$\boxed{\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)} \quad (\text{independent } X, Y).$$

Variances add for independent variables — the fact §16.6 promised.

- i) Positively correlated variables: the sum is *more* variable than the parts ($\text{Var}(X+Y) > \text{Var}(X) + \text{Var}(Y)$); negatively correlated: less.
- ii) The general affine form: $\text{Var}(aX + bY) = a^2 \text{Var}(X) + b^2 \text{Var}(Y) + 2ab \text{Cov}(X, Y)$ — note $\text{Var}(X - Y) = \text{Var}(X) + \text{Var}(Y) - 2 \text{Cov}(X, Y)$.
- iii) Sanity check on §16.6’s counterexample: $Y = -X$ gives $\text{Cov}(X, Y) = -\text{Var}(X)$, so $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(X) - 2\text{Var}(X) = 0$ — exactly what §16.6 computed directly.

eg 15 (variances add — checked two ways). X_1, X_2 = two fair-coin indicators (§17.2, eg 1): each $\text{Var} = 1/4$ (§15.11). Independence gives $\text{Var}(X_1 + X_2) = 1/4 + 1/4 = \boxed{1/2}$. Direct check: $Z = X_1 + X_2$ takes 0, 1, 2 with probabilities 1/4, 1/2, 1/4; $E[Z] = 1$, $E[Z^2] = 0 + 1/2 + 1 = 3/2$, so $\text{Var}(Z) = 3/2 - 1 = 1/2$.

Basically, ... Variance of a sum = sum of variances *plus twice the covariance*. The covariance term is the correction for the two variables moving together. For independent variables there’s no co-movement to correct for, and variances simply add — the cleanest case, and the one §16.6 was waiting for.

17.12 The joint PDF

Def. For two *continuous* random variables X, Y (MLF Week 11, Part 4), the **joint PDF** is the function $f_{X,Y}$ such that for any region $A \subseteq \mathbb{R}^2$,

$$\boxed{P((X, Y) \in A) = \iint_A f_{X,Y}(x, y) dx dy}.$$

- i) $f_{X,Y}(x, y) \geq 0$ everywhere — it is probability divided by a positive area $dx dy$.
- ii) $\iint_{\mathbb{R}^2} f_{X,Y} = 1$ — the discrete “table sums to 1” (§17.2) with sums replaced by integrals, as in §16.3.
- iii) $f_{X,Y}$ is a *density* (probability per unit area), not a probability — it can exceed 1, exactly as in §16.3.

eg 16 (the MLF Week 11 tutorial’s problem).

$$f_{X,Y}(x, y) = \begin{cases} cxy & 0 < x < 2, 1 < y < 3, \\ 0 & \text{otherwise.} \end{cases}$$

Find c : integrate to 1 over the rectangle.

$$1 = \int_0^2 \int_1^3 cxy \, dy \, dx = c \left[\frac{x^2}{2} \right]_0^2 \left[\frac{y^2}{2} \right]_1^3 = c \cdot 2 \cdot \frac{9-1}{2} = 8c,$$

so $\boxed{c = 1/8}$. (The rectangle factors into separate x and y integrals — a hint of independence, confirmed in §17.13.)

eg 17 (rectangle probability — the tutorial's question).

$$P(0 < X < 1, 1 < Y < 2) = \int_0^1 \int_1^2 \frac{xy}{8} \, dy \, dx = \frac{1}{8} \cdot \frac{1}{2} \cdot \frac{4-1}{2} = \boxed{\frac{3}{32}}.$$

eg 18 (a non-rectangular region — the tutorial's question). $P(X + Y < 3)$: the region is $0 < x < 2$, $1 < y < \min(3, 3 - x) = 3 - x$ (panel (b) of the figure):

$$\begin{aligned} P(X + Y < 3) &= \int_0^2 \int_1^{3-x} \frac{xy}{8} \, dy \, dx = \frac{1}{8} \int_0^2 x \left[\frac{(3-x)^2 - 1}{2} \right] dx \\ &= \frac{1}{16} \int_0^2 (x^3 - 6x^2 + 8x) \, dx = \frac{1}{16} \left[\frac{x^4}{4} - 2x^3 + 4x^2 \right]_0^2 \\ &= \frac{1}{16} (4 - 16 + 16) = \boxed{\frac{1}{4}}. \end{aligned}$$

Basically, ... The joint PDF is the continuous joint table: probability is *volume* under the joint density surface over a region. Rectangles are easy (separate the integrals); slanted regions need the limits written carefully — draw the region first, then integrate.

17.13 Marginal PDFs: integrating out

Def. The **marginal PDFs** come from **integrating out** the other variable (MLF Week 11, Part 4):

$$\boxed{f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x,y) \, dy, \quad f_Y(y) = \int_{-\infty}^{\infty} f_{X,Y}(x,y) \, dx.}$$

Same idea as §17.3 — “row totals and column totals” — with sums replaced by integrals (§16.3's lift). Marginal CDFs follow: $F_X(x) = \int_{-\infty}^x f_X(u) \, du$ (§16.4).

eg 19 (marginals of the tutorial density). For $f_{X,Y}(x,y) = xy/8$ on $(0, 2) \times (1, 3)$:

$$f_X(x) = \int_1^3 \frac{xy}{8} \, dy = \frac{x}{8} \left[\frac{y^2}{2} \right]_1^3 = \boxed{\frac{x}{2}}, \quad 0 < x < 2,$$

$$f_Y(y) = \int_0^2 \frac{xy}{8} \, dx = \frac{y}{8} \left[\frac{x^2}{2} \right]_0^2 = \boxed{\frac{y}{4}}, \quad 1 < y < 3.$$

Check they are valid densities: $\int_0^2 (x/2) \, dx = 1$, $\int_1^3 (y/4) \, dy = (9-1)/8 = 1$. Then

$$F_X(1) = \int_0^1 \frac{x}{2} \, dx = \boxed{\frac{1}{4}}, \quad F_Y(2) = \int_1^2 \frac{y}{4} \, dy = \boxed{\frac{3}{8}}$$

— the tutorial's answers, confirmed.

Note (independence in the continuous case). Here $f_{X,Y}(x,y) = xy/8 = (x/2)(y/4) = f_X(x)f_Y(y)$ for every (x,y) , on a *rectangular* region — so X and Y are independent (§17.5's rule, with integrals). Two ingredients, both needed: the density factors *and* the support is a product set. A factorizing density on a non-rectangular support is *not* independent.

Basically, ... Integrating out = the continuous version of summing out. Slice the joint density along one axis, add up the slice, and you get the other variable's density alone. And the independence test is unchanged: does the joint equal the product of the marginals — with a rectangular support?

17.14 Conditional PDFs

Def. The **conditional PDF of Y given $X = x$** is (MLF Week 11, Part 2/4)

$$f_{Y|X=x}(y) = \frac{f_{X,Y}(x,y)}{f_X(x)}, \quad f_X(x) > 0,$$

and the **multiplication rule** holds as in §17.4:

$$f_{X,Y}(x,y) = f_X(x) f_{Y|X=x}(y).$$

For fixed x , $f_{Y|X=x}$ is a genuine density ($\int f_{Y|X=x} = 1$ — the joint integrates to $f_X(x)$). If X, Y are independent, $f_{Y|X=x}(y) = f_Y(y)$: conditioning changes nothing.

eg 20 (conditional on a triangular region — hand-checked). Let $X \sim \text{Uniform}[0, 1]$ and, given $X = x$, let $(Y | X = x) \sim \text{Uniform}[0, x]$:

$$f_{X,Y}(x,y) = f_X(x) f_{Y|X=x}(y) = 1 \cdot \frac{1}{x} = \frac{1}{x}, \quad 0 \leq y \leq x \leq 1.$$

Validity: $\int_0^1 \int_0^x (1/x) dy dx = \int_0^1 1 dx = 1$. The marginal of Y :

$$f_Y(y) = \int_y^1 \frac{1}{x} dx = \boxed{-\ln y}, \quad 0 < y < 1 \quad \left(\int_0^1 -\ln y dy = 1 \right).$$

And the reverse conditional, on $y \leq x \leq 1$:

$$f_{X|Y=y}(x) = \frac{1/x}{-\ln y} = \boxed{\frac{1}{-x \ln y}}; \quad \int_y^1 \frac{dx}{-x \ln y} = \frac{[\ln x]_y^1}{-\ln y} = 1.$$

Knowing $Y = y$ rules out $X < y$ — conditioning reshapes the range, exactly as in §17.4 — and the density is *not* uniform on $[y, 1]$: smaller x (tighter Y -intervals) get more weight.

Basically, ... Conditional PDF = slice the joint density at $X = x$ and re-normalize the slice to area 1. Same table-column idea as §17.4, now with integrals. And the multiplication rule is the modeling workhorse in the continuous world too: specify the marginal of the first variable, then the conditional of the second, and the joint follows.

17.15 Why this matters: the two-variable toolkit

- i) **The toolkit is complete.** From one joint object you get everything: marginals by summing/integrating out (§§17.3, 17.13), conditionals by dividing (§§17.4, 17.14), the joint back by multiplying (§§17.4, 17.14), independence by the factorization test (§17.5). Two slogans to keep: *joint* $\rightarrow$ *marginal is unique*; *marginal* $\rightarrow$ *joint*.

- ii) **Linearity, proved.** $E[X + Y] = E[X] + E[Y]$ (§17.7) needed the joint PMF — that was the debt §§15.8 and 16.5 carried, now paid. And covariance/correlation (§§17.9–17.10) is the first *measure of relationship* between variables: the feature-correlation matrices and PCA of later chapters are this idea at scale.
- iii) **What comes next.** Chapter 18 goes deeper on joint *continuous* distributions (sums via convolution, min/max via CDFs, transformed variables); Chapter 19 builds the multivariate normal, where the covariance becomes a matrix; Chapter 20 estimates these joint models from data (MLE); Chapter 21's central limit theorem explains why sums of many variables are normal — with Var adding (§17.11) all the way there.

Problem set

- (Joint PMF validity and marginals) The joint PMF of $X_1, X_2 \in \{0, 1\}$ is: $f(0, 0) = 0.05$, $f(0, 1) = 0.35$, $f(1, 0) = 0.25$, $f(1, 1) = 0.35$. (a) Verify it is a valid joint PMF. (b) Find the marginal PMFs f_{X_1} and f_{X_2} . (c) Are X_1 and X_2 independent?
- (The 2-digit number) X = units digit, Y = remainder mod 4 of a random 2-digit number (eg 2). (a) Find $f_{X,Y}(2, 3)$. (b) Are X and Y independent? Give a one-cell justification.
- (Conditional PMFs) On eg 5's table ($X \in \{0, 1, 2\}$, $Y \in \{0, 1\}$): (a) find $f_{Y|X=1}$ and check it sums to 1; (b) find $f_{X|Y=0}(2)$.
- (Joint CDF) For two fair-coin indicators X_1, X_2 (eg 1): (a) write F_{X_1, X_2} at the four points $(0, 0), (0, 1), (1, 0), (1, 1)$; (b) use the rectangle formula to recover $P(X_1 = 1, X_2 = 1)$.
- ($E[g(X, Y)]$ and covariance) Die-then-coins (eg 6/eg 10): X = die outcome, Y = heads in X tosses. (a) Find $E[Y]$. (b) Find $\text{Cov}(X, Y)$.
- (Covariance and correlation) On eg 5's table: compute $\text{Cov}(X, Y)$ and $\rho(X, Y)$.
- (Variance of sums) $X, Y \stackrel{\text{i.i.d.}}{\sim} \text{Bernoulli}(p)$. Find $\text{Var}(X + Y)$ and $\text{Var}(X - Y)$.
- (Joint PDF) $f_{X,Y}(x, y) = cxy$ for $0 < x < 2$, $1 < y < 3$ (0 otherwise). (a) Find c . (b) Find $P(X < 1, Y > 2)$.
- (Marginal and conditional PDFs) For the density in Problem 8: (a) find f_X and f_Y ; (b) find $f_{Y|X=x}$; (c) are X and Y independent?
- (IPL powerplay) $Y \in \{0, 1, 2\}$ = wickets in an over, with $f_Y(0) = 13/16$, $f_Y(1) = 1/8$, $f_Y(2) = 1/16$. Given $Y = y$, X = runs is uniform on $\{6, \dots, 12\}$, $\{2, \dots, 8\}$, $\{0, \dots, 6\}$ respectively. Find $E[X]$.

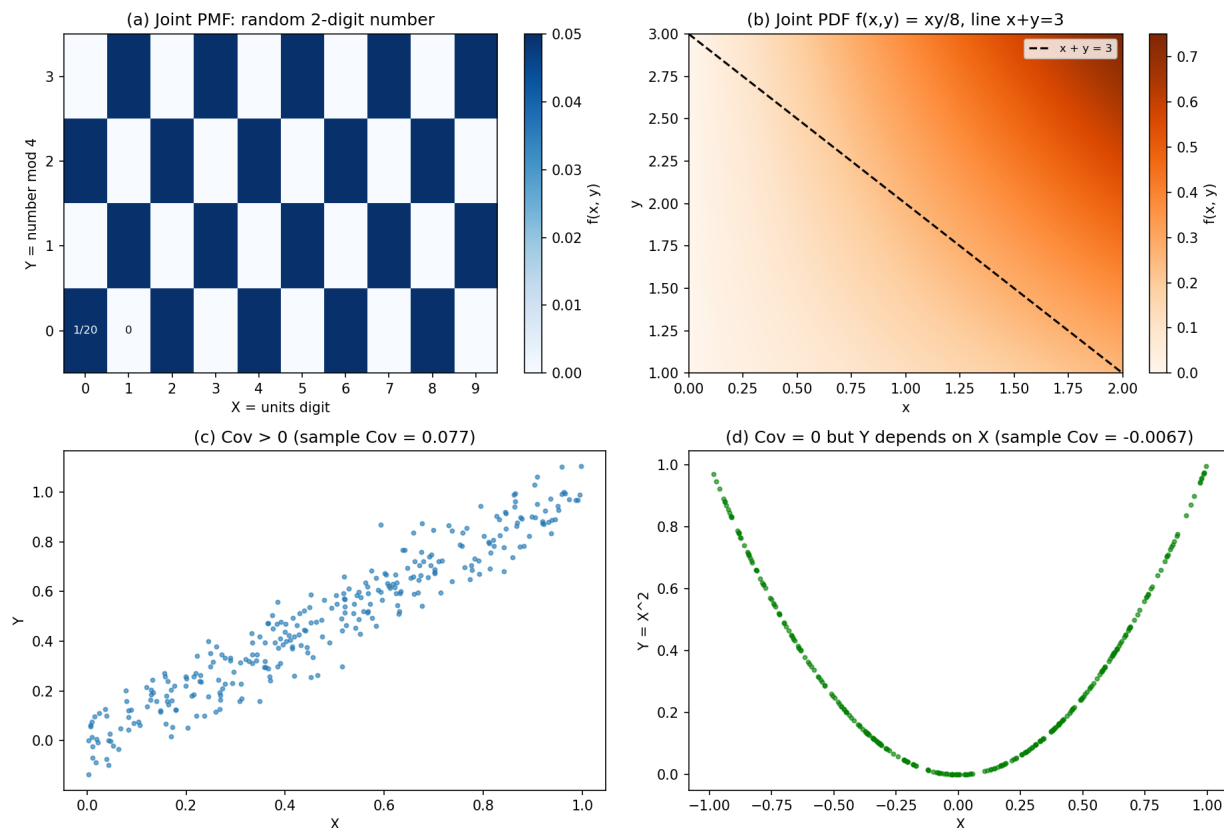


Figure 19: (a) Joint PMF of the 2-digit number (eg 2): checkerboard of $1/20$ cells with impossible (x,y) holes — X and Y are not independent. (b) Joint PDF $xy/8$ on $(0,2) \times (1,3)$ with the line $x+y=3$ (eg 18's region). (c) Positively correlated scatter: $\text{Cov} > 0$. (d) Uncorrelated but dependent: $Y=X^2$ (eg 13), $\text{Cov}=0$ yet Y is a function of X .

Solutions — 17. Joint distributions: two random variables

1. (Joint PMF validity and marginals) The table:

$X_1 \setminus X_2$	0	1
0	0.05	0.35
1	0.25	0.35

(a) Validity: every entry is in $[0, 1]$; the entries sum to $0.05 + 0.35 + 0.25 + 0.35 = 1.00$ (§17.2). Valid joint PMF.

(b) Summing out (§17.3):

$$f_{X_1}(0) = 0.05 + 0.25 = \boxed{0.30}, \quad f_{X_1}(1) = 0.35 + 0.35 = \boxed{0.70},$$

$$f_{X_2}(0) = 0.05 + 0.35 = \boxed{0.40}, \quad f_{X_2}(1) = 0.25 + 0.35 = \boxed{0.60}.$$

(c) Independence test (§17.5): $f_{X_1, X_2}(0, 0) = 0.05$ but $f_{X_1}(0) f_{X_2}(0) = 0.30 \times 0.40 = 0.12$. Since $0.05 \neq 0.12$, not independent — one failing cell is enough.

2. (The 2-digit number) (a) $f_{X,Y}(2,3)$: units digit 2 and number $\equiv 3 \pmod{4}$. A number ending in 2 is even, hence 0 or 2 mod 4 — never 3. The count is 0: $f_{X,Y}(2,3) = 0$.

(b) One-cell justification (§17.5, zero-cell shortcut): $f_{X,Y}(2,3) = 0$ while $f_X(2) = 1/10 > 0$ and $f_Y(3) = 1/4 > 0$, so $f_{X,Y}(2,3) \neq f_X(2)f_Y(3) = 1/40$. $\boxed{\text{Not independent.}}$

3. (Conditional PMFs) On eg 5's table, $f_X(1) = 1/4$, $f_Y(0) = 1/2$.

(a) $f_{Y|X=1}(0) = \frac{f_{X,Y}(1,0)}{f_X(1)} = \frac{1/8}{1/4} = \boxed{\frac{1}{2}}$, $f_{Y|X=1}(1) = \frac{1/8}{1/4} = \boxed{\frac{1}{2}}$. Check: $1/2 + 1/2 = 1$ — a genuine PMF (§17.4).

(b) $f_{X|Y=0}(2) = \frac{f_{X,Y}(2,0)}{f_Y(0)} = \frac{1/8}{1/2} = \boxed{\frac{1}{4}}$.

4. (Joint CDF) (a) With $1/4$ in every cell of eg 1's table:

$$\boxed{F_{X_1,X_2}(0,0) = 1/4}, \quad \boxed{F_{X_1,X_2}(0,1) = 1/2}, \quad \boxed{F_{X_1,X_2}(1,0) = 1/2}, \quad \boxed{F_{X_1,X_2}(1,1) = 1}.$$

(b) Rectangle formula (§17.6) on $(0,1] \times (0,1]$:

$$P(X_1 = 1, X_2 = 1) = F(1,1) - F(0,1) - F(1,0) + F(0,0) = 1 - \frac{1}{2} - \frac{1}{2} + \frac{1}{4} = \boxed{\frac{1}{4}},$$

matching the table .

5. ($E[g(X,Y)]$ and covariance) (a) By total expectation (§17.7): $E[Y | X = t] = t/2$ for $(Y | X = t) \sim \text{Binomial}(t, 1/2)$, and $E[X] = 7/2$, so

$$E[Y] = \sum_{t=1}^6 \frac{t}{2} \cdot \frac{1}{6} = \frac{1}{2}E[X] = \boxed{\frac{7}{4}}.$$

(b) $E[XY] = E_X[X \cdot E[Y | X]] = E[X^2]/2$. For a fair die, $E[X^2] = (1 + 4 + 9 + 16 + 25 + 36)/6 = 91/6$, so $E[XY] = 91/12$. Then

$$\text{Cov}(X,Y) = E[XY] - E[X]E[Y] = \frac{91}{12} - \frac{7}{2} \cdot \frac{7}{4} = \frac{91}{12} - \frac{49}{8} = \frac{182 - 147}{24} = \boxed{\frac{35}{24} \approx 1.46},$$

positive — more die pips mean more tosses, hence more heads, as expected.

6. (Covariance and correlation) On eg 5's table (§17.9–17.10):

$$E[X] = 0 \cdot \frac{3}{8} + 1 \cdot \frac{1}{4} + 2 \cdot \frac{3}{8} = 1, \quad E[Y] = \frac{1}{2},$$

$$E[XY] = 1 \cdot 1 \cdot \frac{1}{8} + 2 \cdot 1 \cdot \frac{1}{4} = \frac{5}{8},$$

$$\boxed{\text{Cov}(X,Y) = \frac{5}{8} - 1 \cdot \frac{1}{2} = \frac{1}{8}}.$$

For ρ : $E[X^2] = 0 + 1 \cdot \frac{1}{4} + 4 \cdot \frac{3}{8} = 7/4$, so $\text{Var}(X) = 7/4 - 1 = 3/4$; $E[Y^2] = 1/2$, so $\text{Var}(Y) = 1/2 - 1/4 = 1/4$. Hence

$$\boxed{\rho(X,Y) = \frac{1/8}{\sqrt{3/4}\sqrt{1/4}} = \frac{1/8}{\sqrt{3/4}} = \frac{1}{2\sqrt{3}} = \frac{\sqrt{3}}{6} \approx 0.29}.$$

7. (Variance of sums) $X, Y \stackrel{\text{i.i.d.}}{\sim} \text{Bernoulli}(p)$ are independent, so $\text{Cov}(X,Y) = 0$ (§17.9) and $\text{Var}(X) = \text{Var}(Y) = p(1-p)$ (§15.11). By §17.11:

$$\boxed{\text{Var}(X+Y) = p(1-p) + p(1-p) + 0 = 2p(1-p)},$$

$$\boxed{\text{Var}(X - Y) = p(1 - p) + p(1 - p) - 0 = 2p(1 - p)}.$$

Note the sign of the sum doesn't matter for independent variables — both give $2p(1 - p)$.

8. (Joint PDF) (a) $1 = \int_0^2 \int_1^3 cxy \, dy \, dx = c \cdot 2 \cdot \frac{9-1}{2} = 8c$, so $\boxed{c = 1/8}$.

(b) $P(X < 1, Y > 2) = \int_0^1 \int_2^3 \frac{xy}{8} \, dy \, dx = \frac{1}{8} \cdot \left[\frac{x^2}{2} \right]_0^1 \cdot \left[\frac{y^2}{2} \right]_2^3 = \frac{1}{8} \cdot \frac{1}{2} \cdot \frac{9-4}{2} = \boxed{\frac{5}{32}}.$

9. (Marginal and conditional PDFs) (a) Integrating out (§17.13):

$$f_X(x) = \int_1^3 \frac{xy}{8} \, dy = \frac{x}{8} \cdot \frac{9-1}{2} = \boxed{\frac{x}{2}}, \quad 0 < x < 2,$$

$$f_Y(y) = \int_0^2 \frac{xy}{8} \, dx = \frac{y}{8} \cdot \frac{4}{2} = \boxed{\frac{y}{4}}, \quad 1 < y < 3.$$

(b) $f_{Y|X=x}(y) = \frac{f_{X,Y}(x,y)}{f_X(x)} = \frac{xy/8}{x/2} = \boxed{\frac{y}{4}}, \quad 1 < y < 3 \text{ (§17.14)}.$

(c) $\boxed{\text{Yes, independent}}$: $f_{Y|X=x}(y) = y/4 = f_Y(y)$ for every x , equivalently $f_{X,Y}(x,y) = (x/2)(y/4) = f_X(x)f_Y(y)$, on the rectangular support $(0, 2) \times (1, 3)$ (§17.13's Note).

10. (IPL powerplay) By total expectation (§17.7's Note): $E[X | Y = 0] = (6 + 12)/2 = 9$, $E[X | Y = 1] = (2 + 8)/2 = 5$, $E[X | Y = 2] = (0 + 6)/2 = 3$ (means of discrete uniforms, §15.10). So

$$E[X] = \frac{13}{16} \cdot 9 + \frac{1}{8} \cdot 5 + \frac{1}{16} \cdot 3 = \frac{117 + 10 + 3}{16} = \boxed{\frac{65}{8} = 8.125}.$$

Sanity check: most overs have 0 wickets (13/16) with mean 9 runs, so $E[X]$ should sit just below 9 — 8.125 .

18. Joint continuous distributions

Chapter 17 built the full two-variable toolkit — joint PMF/PDF, marginals, conditionals, independence, $E[g(X, Y)]$, covariance — but one question kept getting deferred: *if you transform the pair, what is the distribution of the result?* The discrete version was answered in §17.8 (pool the joint PMF over contours); this chapter does the continuous version. The three workhorses: the CDF method for $Z = g(X, Y)$ (§18.2), convolution for sums of independent variables (§18.3), and the Jacobian method for invertible transformations (§18.5). Max/min via CDFs (§18.4) and the joint moment generating function (§18.6) round out the machinery. The one-variable CDF method (§16.10) is the special case everything here generalizes.

18.1 The continuous two-variable toolkit (recap)

Everything from §§17.12–17.14, in one place:

- i) **Joint PDF** (§17.12): $P((X, Y) \in A) = \iint_A f_{X,Y}(x, y) dx dy$ for any region A ; $f_{X,Y} \geq 0$; $\iint_{\mathbb{R}^2} f_{X,Y} = 1$.
- ii) **Marginals** (§17.13): $f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x, y) dy$ (integrate out y), and symmetrically for f_Y .
- iii) **Conditionals** (§17.14): $f_{Y|X=x}(y) = f_{X,Y}(x, y)/f_X(x)$; the multiplication rule $f_{X,Y}(x, y) = f_X(x) f_{Y|X=x}(y)$.
- iv) **Independence** (§17.5, continuous form): X, Y independent iff $f_{X,Y}(x, y) = f_X(x) f_Y(y)$ for all (x, y) — and the support is a product set (a factorizing density on a non-rectangular support is not independent; §17.13's Note).

Two formulas that were stated for the discrete case lift to integrals:

Def (continuous two-variable LOTUS). For $Z = g(X, Y)$,

$$E[g(X, Y)] = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} g(x, y) f_{X,Y}(x, y) dx dy$$

— §17.7 with sums replaced by integrals. The covariance formula is unchanged in shape: $\text{Cov}(X, Y) = E[XY] - E[X]E[Y]$ (§17.9), computed with this integral.

Basically, ... Same toolkit as Chapter 17, sums swapped for integrals: the joint PDF spreads probability over regions, marginals come from integrating out, conditionals from dividing, independence from the product test. The one new formula here is continuous LOTUS — average $g(x, y)$ against the joint density, no need to find g 's distribution first.

18.2 The CDF method for $Z = g(X, Y)$

§17.8 pooled the joint PMF over the contours $\{g(x, y) = z\}$. The continuous version pools *probability mass* over the region $\{g(x, y) \leq z\}$ — the CDF method (§16.10), now in two dimensions:

Recipe. For $Z = g(X, Y)$:

$$F_Z(z) = P(g(X, Y) \leq z) = \iint_{\{(x, y) : g(x, y) \leq z\}} f_{X,Y}(x, y) dx dy, \quad f_Z(z) = F'_Z(z).$$

- i) Sketch the region $\{(x, y) : g(x, y) \leq z\}$ and intersect it with the support of $f_{X,Y}$ — the picture dictates the integral's limits.
- ii) Integrate, then differentiate with respect to z (Leibniz rule lets the derivative pass inside when the limits are constant).
- iii) If (X, Y) is uniform on a region D , the integral collapses to an area ratio: $P((X, Y) \in A) = |A|/|D|$ (the Stats 2 deck's §3.1).

eg 1 (the deck's 2-D uniform triangle). Let $(X, Y) \sim \text{Uniform}(D)$ with $D = \{(x, y) : x + y < 2, x > 0, y > 0\}$ — a right triangle with legs 2, area $|D| = 2$, so $f_{X,Y}(x, y) = 1/2$ on D .

$P(X + Y < 1)$: the region $A = \{(x, y) : x + y < 1, x > 0, y > 0\}$ is a triangle with legs 1, area $|A| = 1/2$. By the area ratio,

$$P(X + Y < 1) = \frac{|A|}{|D|} = \frac{1/2}{2} = \frac{1}{4}.$$

The same via integration (the deck's “Method 2” — y runs 0 to $1 - x$, x runs 0 to 1):

$$P(X + Y < 1) = \int_0^1 \int_0^{1-x} \frac{1}{2} dy dx = \frac{1}{2} \int_0^1 (1 - x) dx = \frac{1}{2} \cdot \frac{1}{2} = \boxed{\frac{1}{4}}.$$

$P(X + 2Y > 1)$: work with the complement inside D . The set $\{x + 2y < 1, x > 0, y > 0\}$ is a triangle with x -intercept 1 and y -intercept $1/2$, area $\frac{1}{2} \cdot 1 \cdot \frac{1}{2} = 1/4$. So

$$P(X + 2Y > 1) = \frac{|D| - 1/4}{|D|} = \frac{2 - 1/4}{2} = \boxed{\frac{7}{8}}.$$

eg 2 (the deck's $f_{X,Y}(x, y) = x + y$). On the unit square, $f_{X,Y}(x, y) = x + y$ ($0 < x, y < 1$). Valid density: $\int_0^1 \int_0^1 (x + y) dy dx = \int_0^1 (x + 1/2) dx = 1$.

$P(X < 1/2, Y < 1/2)$: integrate over the quarter square —

$$\int_0^{1/2} \int_0^{1/2} (x + y) dy dx = \int_0^{1/2} \left(\frac{x}{2} + \frac{1}{8} \right) dx = \left[\frac{x^2}{4} + \frac{x}{8} \right]_0^{1/2} = \frac{1}{16} + \frac{1}{16} = \boxed{\frac{1}{8}}.$$

(Note: the deck's printed answer $3/16$ integrates $x/2$ to $x^2/2$ instead of $x^2/4$ — an arithmetic slip; $1/8$ is correct. Logged in the review.)

$P(X + Y < 1)$: the region is $0 < x < 1, 0 < y < 1 - x$ —

$$\int_0^1 \int_0^{1-x} (x + y) dy dx = \int_0^1 \left(x(1 - x) + \frac{(1 - x)^2}{2} \right) dx = \int_0^1 \left(\frac{1}{2} - \frac{x^2}{2} \right) dx = \boxed{\frac{1}{3}}.$$

Basically, ... The CDF method in 2-D = “shade the region where $g(x, y) \leq z$, integrate the joint density over it, differentiate”. Drawing the region is the whole battle — once the limits are right, the rest is bookkeeping. For uniform joints, shading *is* the answer: probability = shaded area $\div$ total area.

18.3 Convolution: the distribution of a sum

The most-used $g(X, Y)$ is the sum. Let X, Y be *independent* continuous variables with PDFs f_X, f_Y , and $Z = X + Y$. Apply the CDF method (§18.2): the region $\{x + y \leq z\}$ is everything below the line $y = z - x$, so

$$\begin{aligned} F_Z(z) &= P(X + Y \leq z) = \int_{-\infty}^{\infty} \int_{-\infty}^{z-x} f_X(x) f_Y(y) dy dx \\ &= \int_{-\infty}^{\infty} F_Y(z-x) f_X(x) dx \quad (\text{independence: joint} = \text{product}). \end{aligned}$$

Differentiating under the integral (the limits are constant):

$$f_Z(z) = \int_{-\infty}^{\infty} f_X(x) f_Y(z-x) dx = (f_X * f_Y)(z).$$

This is the **convolution** of f_X and f_Y , written $f_X * f_Y$. Sliding-window reading: fix z ; as x runs over X 's range, $f_Y(z-x)$ is Y 's density shifted to sit at $z-x$; the integral multiplies the two densities at every split of z into $x + (z-x)$ and adds them up — exactly §17.8's contour pooling, made continuous.

- i) Symmetry: $f_Z(z) = \int_{-\infty}^{\infty} f_X(z-y) f_Y(y) dy$ — convolve in whichever order is easier.
- ii) Discrete twin (VOL1 deck §2.3.4; §17.8's contour version): $f_Z(z) = \sum_x f_X(x) f_Y(z-x)$.
- iii) Three or more: convolution is associative — $f_{X+Y+Z} = (f_X * f_Y) * f_Z$. Add variables two at a time.

eg 3 (sum of two i.i.d. Uniform[0, 1] — the continuous twin of §17.8's eg 11). $f_X = f_Y = 1$ on $(0, 1)$. The integrand is 1 exactly when $0 < x < 1$ and $0 < z-x < 1$, i.e. $x \in (0, 1) \cap (z-1, z)$:

- for $0 < z < 1$: the overlap is $(0, z)$, so $f_Z(z) = \int_0^z 1 dx = \boxed{z}$;
- for $1 \leq z < 2$: the overlap is $(z-1, 1)$, so $f_Z(z) = \int_{z-1}^1 1 dx = \boxed{2-z}$;
- 0 elsewhere.

$$f_{X+Y}(z) = \begin{cases} z & 0 < z < 1, \\ 2-z & 1 \leq z < 2, \\ 0 & \text{otherwise.} \end{cases}$$

The **triangular distribution** — panel (a) of the figure. Middle sums have more (x, y) splits landing on them, so the density ramps up then down. Same moral as the discrete eg 11: sums of uniforms are *not* uniform.

eg 4 (sum of two i.i.d. Exp(λ)). $f_X(x) = \lambda e^{-\lambda x}$ for $x > 0$ (§16.8). For $z > 0$, the overlap of $(0, \infty)$ with $(-\infty, z)$ is $(0, z)$:

$$f_Z(z) = \int_0^z \lambda e^{-\lambda x} \lambda e^{-\lambda(z-x)} dx = \lambda^2 e^{-\lambda z} \int_0^z dx = \boxed{\lambda^2 z e^{-\lambda z}}, \quad z > 0.$$

This is the **Gamma**(2, λ) (Erlang-2) density: the waiting time for *two* arrivals when arrivals come at rate λ . Sanity check via LOTUS (§18.1): $E[Z] = E[X] + E[Y] = 2/\lambda$ (§17.7), and $\int_0^{\infty} z \cdot \lambda^2 z e^{-\lambda z} dz = \frac{1}{\lambda} \int_0^{\infty} t^2 e^{-t} dt = 2/\lambda$.

eg 5 (discrete convolution: sum of independent Poissons). $X \sim \text{Poisson}(\lambda_1)$, $Y \sim \text{Poisson}(\lambda_2)$,

independent (§15.15). For $z = 0, 1, 2, \dots$, the discrete convolution (item ii)) sums over the splits $k + (z - k) = z$:

$$\begin{aligned} f_Z(z) &= \sum_{k=0}^z \frac{e^{-\lambda_1} \lambda_1^k}{k!} \cdot \frac{e^{-\lambda_2} \lambda_2^{z-k}}{(z-k)!} \\ &= \frac{e^{-(\lambda_1+\lambda_2)}}{z!} \sum_{k=0}^z \frac{z!}{k! (z-k)!} \lambda_1^k \lambda_2^{z-k} \\ &= \frac{e^{-(\lambda_1+\lambda_2)}}{z!} \sum_{k=0}^z \binom{z}{k} \lambda_1^k \lambda_2^{z-k} = \boxed{\frac{e^{-(\lambda_1+\lambda_2)} (\lambda_1 + \lambda_2)^z}{z!}}, \end{aligned}$$

by the binomial theorem (§15.12). So $\boxed{X + Y \sim \text{Poisson}(\lambda_1 + \lambda_2)}$ — independent Poisson counts add their rates, exactly as independent exponential waiting times convolve to a Gamma (eg 4).

Basically, ... The PDF of a sum of independent variables = convolution: “for each way of splitting z into $x + y$, multiply the two densities, and add over all splits”. Two uniforms give a triangle (peaks in the middle), two exponentials give a Gamma (the two-arrival waiting time). Whenever you see “total of independent waiting times / errors / measurements”, think convolution.

18.4 Max and min of independent variables

For $M = \max(X, Y)$ and $m = \min(X, Y)$ with X, Y independent, the CDF method (§18.2) collapses beautifully — no integrals needed:

$$\boxed{F_M(z) = P(M \leq z) = P(X \leq z) P(Y \leq z) = F_X(z) F_Y(z)}$$

(the max is small iff *both* are small), and

$$\boxed{F_m(z) = 1 - P(m > z) = 1 - P(X > z) P(Y > z) = 1 - (1 - F_X(z))(1 - F_Y(z))}$$

(the min is large iff *both* are large). For n independent variables: $F_{\max} = \prod_i F_{X_i}$, $F_{\min} = 1 - \prod_i (1 - F_{X_i})$. Differentiate for the PDFs.

eg 6 (max and min of i.i.d. Uniform[0, 1]). $F_X(z) = z$ on $(0, 1)$ (§16.7):

$$F_M(z) = z^2, \quad \boxed{f_M(z) = 2z}, \quad F_m(z) = 1 - (1 - z)^2 = 2z - z^2, \quad \boxed{f_m(z) = 2(1 - z)},$$

$0 < z < 1$ — panel (b) of the figure. The max piles up near 1, the min near 0, exactly as intuition says. For n i.i.d. uniforms: $f_{\max}(z) = nz^{n-1}$, $f_{\min}(z) = n(1 - z)^{n-1}$.

eg 7 (min of independent exponentials — competing clocks). $X \sim \text{Exp}(\lambda_1)$, $Y \sim \text{Exp}(\lambda_2)$ independent (§16.8: $P(X > z) = e^{-\lambda_1 z}$). Then

$$P(\min(X, Y) > z) = e^{-\lambda_1 z} e^{-\lambda_2 z} = e^{-(\lambda_1 + \lambda_2)z},$$

so $\boxed{\min(X, Y) \sim \text{Exp}(\lambda_1 + \lambda_2)}$ — the minimum of independent exponentials is exponential with the *summed* rate. Two components in series fail at the combined rate; two competing clocks ring at the combined rate.

eg 8 (the deck’s $P(X > Y)$). Same setup as eg 7. Integrate over $\{y < x\}$:

$$\begin{aligned} P(X > Y) &= \int_0^\infty \int_0^x \lambda_1 e^{-\lambda_1 x} \lambda_2 e^{-\lambda_2 y} dy dx \\ &= \int_0^\infty \lambda_1 e^{-\lambda_1 x} (1 - e^{-\lambda_2 x}) dx = 1 - \frac{\lambda_1}{\lambda_1 + \lambda_2} = \boxed{\frac{\lambda_2}{\lambda_1 + \lambda_2}}. \end{aligned}$$

Each clock's chance of ringing *second* is its rate's share of the total rate — the memoryless property (§16.8) makes the race fair at every instant.

Note (reliability reading). Max = *parallel* system (works while *any* component works — the last failure kills it); min = *series* system (fails at the *first* failure). These two CDF formulas are the whole of two-component reliability theory.

Basically, ... Max and min of independent variables need no integration: the max's CDF is the *product* of the CDFs (both must be small), the min's CDF is one minus the product of the *survival* functions (both must be large). Exponentials are the showcase: the minimum of competing exponential clocks is exponential at the summed rate.

18.5 The Jacobian method: transforming two variables

§18.2's CDF method always works but can be painful. When $(U, V) = (g_1(X, Y), g_2(X, Y))$ is *invertible* with smooth partials, the **Jacobian method** gives the joint PDF directly — the 2-D upgrade of §16.10's monotone formula.

Theorem (change of variables). Let g be one-to-one on the support of (X, Y) , with continuous partial derivatives and nonzero Jacobian determinant

$$J = \frac{\partial(u, v)}{\partial(x, y)} = \det \begin{pmatrix} \partial u / \partial x & \partial u / \partial y \\ \partial v / \partial x & \partial v / \partial y \end{pmatrix} \neq 0.$$

Write the inverse as $(x, y) = h(u, v) = (h_1(u, v), h_2(u, v))$. Then

$$f_{U,V}(u, v) = f_{X,Y}(h_1(u, v), h_2(u, v)) \left| \frac{\partial(x, y)}{\partial(u, v)} \right|$$

on the transformed support, 0 elsewhere.

Why the extra factor. A small rectangle of area $du dv$ at (u, v) is the image of a small parallelogram of area $|\partial(x, y) / \partial(u, v)| du dv$ at (x, y) — the Jacobian determinant *is* the local area-distortion factor (§9.7). Probability mass is conserved, so the density rescales by exactly that factor. In 1-D, $|dg^{-1}/dy|$ (§16.10) is the same idea with lengths instead of areas.

Steps. i) Solve for x, y in terms of u, v (the inverse map). ii) Compute the Jacobian determinant *of the inverse* and take $|\cdot|$. iii) Substitute into $f_{X,Y}$ and multiply by $|J_{\text{inv}}|$. iv) Transform the support: push the (x, y) inequalities through the inverse. (For linear g , $|J_{\text{inv}}|$ is constant — the easy case.)

eg 9 (linear: $U = X + Y, V = X - Y$). $X, Y \stackrel{\text{i.i.d.}}{\sim} \text{Uniform}[0, 1]$. Invert: $x = (u + v)/2, y = (u - v)/2$.

$$\frac{\partial(x, y)}{\partial(u, v)} = \det \begin{pmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{pmatrix} = -\frac{1}{4} - \frac{1}{4} = -\frac{1}{2}, \quad |J_{\text{inv}}| = \frac{1}{2}.$$

Support: $0 < x < 1, 0 < y < 1$ becomes $0 < u + v < 2, 0 < u - v < 2$ — i.e. $0 < u < 2$ and $|v| < \min(u, 2 - u)$, a diamond (panel (c) of the figure). Hence

$$f_{U,V}(u, v) = 1 \cdot \frac{1}{2} = \frac{1}{2} \quad \text{on the diamond}$$

(check: diamond area = 2, so it integrates to 1). The marginal of U :

$$f_U(u) = \int_{-\min(u, 2-u)}^{\min(u, 2-u)} \frac{1}{2} dv = \min(u, 2 - u) = \begin{cases} u & 0 < u < 1, \\ 2 - u & 1 \leq u < 2, \end{cases}$$

exactly the triangular density from §18.3's convolution — two methods, one answer.

eg 10 (nonlinear classic: sum and proportion of exponentials). $X, Y \stackrel{\text{i.i.d.}}{\sim} \text{Exp}(1)$; $U = X + Y$, $V = X/(X + Y)$. Invert: $x = uv$, $y = u(1 - v)$.

$$\frac{\partial(x, y)}{\partial(u, v)} = \det \begin{pmatrix} v & u \\ 1 - v & -u \end{pmatrix} = -uv - u(1 - v) = -u, \quad |J_{\text{inv}}| = u.$$

Support: $x > 0$, $y > 0$ becomes $u > 0$, $0 < v < 1$. Now

$$f_{U,V}(u, v) = \underbrace{e^{-uv} e^{-u(1-v)}}_{f_{X,Y}} \cdot u = u e^{-u} = [u e^{-u} \mathbf{1}_{\{u>0\}}] \cdot [1 \cdot \mathbf{1}_{\{0<v<1\}}].$$

It *factors*: $[U \perp V]$, with $V \sim \text{Uniform}[0, 1]$ and $f_U(u) = u e^{-u}$ on $u > 0$ — which is §18.3's eg 4 density at $\lambda = 1$ (Gamma(2, 1)) , a second cross-check between convolution and Jacobian. The sum and the proportion are independent: knowing the total tells you nothing about the split.

Basically, ... The Jacobian method = “substitute the inverse into the joint density, then multiply by the area-stretch factor $|\partial(x, y)/\partial(u, v)|$ ”. Always transform the support too — most mistakes happen there, not in the determinant. And when the transformed joint factors, you’ve discovered an independence (§17.5) hiding inside the transformation, like eg 10’s sum-vs-proportion.

18.6 The joint moment generating function

A compact encoding of the joint distribution, and a second route to sums:

Def. The **joint MGF** of (X, Y) is

$$M_{X,Y}(s, t) = E[e^{sX+tY}],$$

wherever the expectation exists.

- i) Marginals fall out: $M_X(s) = M_{X,Y}(s, 0)$, $M_Y(t) = M_{X,Y}(0, t)$.
- ii) **Independence test:** $X \perp Y$ iff $M_{X,Y}(s, t) = M_X(s) M_Y(t)$ (for MGFs existing near 0).
- iii) **Sums without integrating.** For independent X, Y ,

$$M_{X+Y}(s) = M_X(s) M_Y(s)$$

since $E[e^{s(X+Y)}] = E[e^{sX} e^{sY}] = E[e^{sX}] E[e^{sY}]$ by independence (§17.5). Convolution (§18.3) does the sum in density space; the MGF does it in one line of algebra.

eg 11 (sum of independent normals). For $X \sim \text{Normal}(\mu_1, \sigma_1^2)$, $M_X(s) = e^{\mu_1 s + \sigma_1^2 s^2/2}$ (standard formula), and similarly for Y . If $X \perp Y$,

$$M_{X+Y}(s) = e^{\mu_1 s + \sigma_1^2 s^2/2} e^{\mu_2 s + \sigma_2^2 s^2/2} = e^{(\mu_1 + \mu_2)s + (\sigma_1^2 + \sigma_2^2)s^2/2},$$

which is the MGF of $\text{Normal}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$ — so by MGF uniqueness (quoted as a fact),

$$[X + Y \sim \text{Normal}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)] \quad (\text{independent } X, Y).$$

Means add, *variances* add (§17.11’s corollary, now as a distribution statement). The normal family is closed under independent sums — the fact Chapter 21’s central limit theorem will lean on. (The full multivariate normal, where the *covariance* becomes a matrix, is Chapter 19.)

Basically, ... The joint MGF packs the whole joint distribution into one function $E[e^{sX+tY}]$. Its killer feature: for independent variables, the MGF of a sum is the *product* of the MGFs — no convolution integral needed. Identify the product, and you’ve identified the sum’s distribution (as with the normals above).

18.7 Why this matters: the two-variable machine

- i) **Sums are everywhere.** Total noise, total waiting time, accumulated error — convolution (§18.3) and the MGF product (§18.6) are the two ways to get a sum's distribution. Chapter 21's CLT is the limit of this idea as the number of summands grows.
- ii) **Extremes are cheap.** Max/min need only CDFs (§18.4) — no integrals at all. Reliability (series vs parallel), best/worst-case analysis, and the order statistics behind quantiles all start here.
- iii) **The Jacobian is the general engine.** Any invertible transformation of the pair goes through §18.5; the linear case (constant Jacobian) is exactly what Chapter 19's multivariate normal needs for affine transformations.
- iv) **What comes next.** Chapter 19 builds the multivariate normal (covariance as a matrix, §17.9 at scale); Chapter 20 estimates these joint models from data (MLE); Chapter 21 explains why sums keep turning normal.

Problem set

- (2-D uniform triangle — the deck's Q1) $(X, Y) \sim \text{Uniform}(D)$, $D = \{(x, y) : x + y < 2, x > 0, y > 0\}$. Find (a) $P(X + Y < 1)$, (b) $P(X + 2Y > 1)$.
- (Region integration — the deck's Q2) $f_{X,Y}(x, y) = x + y$ for $0 < x, y < 1$ (0 otherwise). (a) Verify it is a valid joint density. (b) Find $P(X < 1/2, Y < 1/2)$. (c) Find $P(X + Y < 1)$.
- (Marginals — the deck's 3.5 Q3) $f_{X,Y}(x, y) = \frac{6}{5}(x + y^2)$ for $0 < x, y < 1$ (0 otherwise). (a) Find f_X and f_Y . (b) Are X and Y independent?
- (Uniform marginals, dependent joint — the deck's §3.2 Q2) (X, Y) is uniform on $D = [0, \frac{1}{2}]^2 \cup [\frac{1}{2}, 1]^2$. (a) Find the marginal densities of X and Y . (b) Are X and Y independent?
- (Convolution: uniforms) $X, Y \stackrel{\text{i.i.d.}}{\sim} \text{Uniform}[0, 1]$. Find the PDF of $Z = X + Y$.
- (Convolution: exponentials) $X, Y \stackrel{\text{i.i.d.}}{\sim} \text{Exp}(\lambda)$. Find the PDF of $Z = X + Y$.
- (Max and min) $X, Y \stackrel{\text{i.i.d.}}{\sim} \text{Uniform}[0, 1]$. Find the PDFs of $\max(X, Y)$ and $\min(X, Y)$.
- (Min of exponentials) $X \sim \text{Exp}(\lambda_1)$, $Y \sim \text{Exp}(\lambda_2)$, independent. (a) Find the distribution of $\min(X, Y)$. (b) Find $P(X > Y)$.
- (Jacobian) $X, Y \stackrel{\text{i.i.d.}}{\sim} \text{Uniform}[0, 1]$; $U = X + Y$, $V = X - Y$. (a) Find $f_{U,V}$. (b) Find the marginal PDF of U and compare with Problem 5.
- (MGF of a sum) $X \sim \text{Normal}(\mu_1, \sigma_1^2)$, $Y \sim \text{Normal}(\mu_2, \sigma_2^2)$, independent. Use MGFs to find the distribution of $X + Y$.

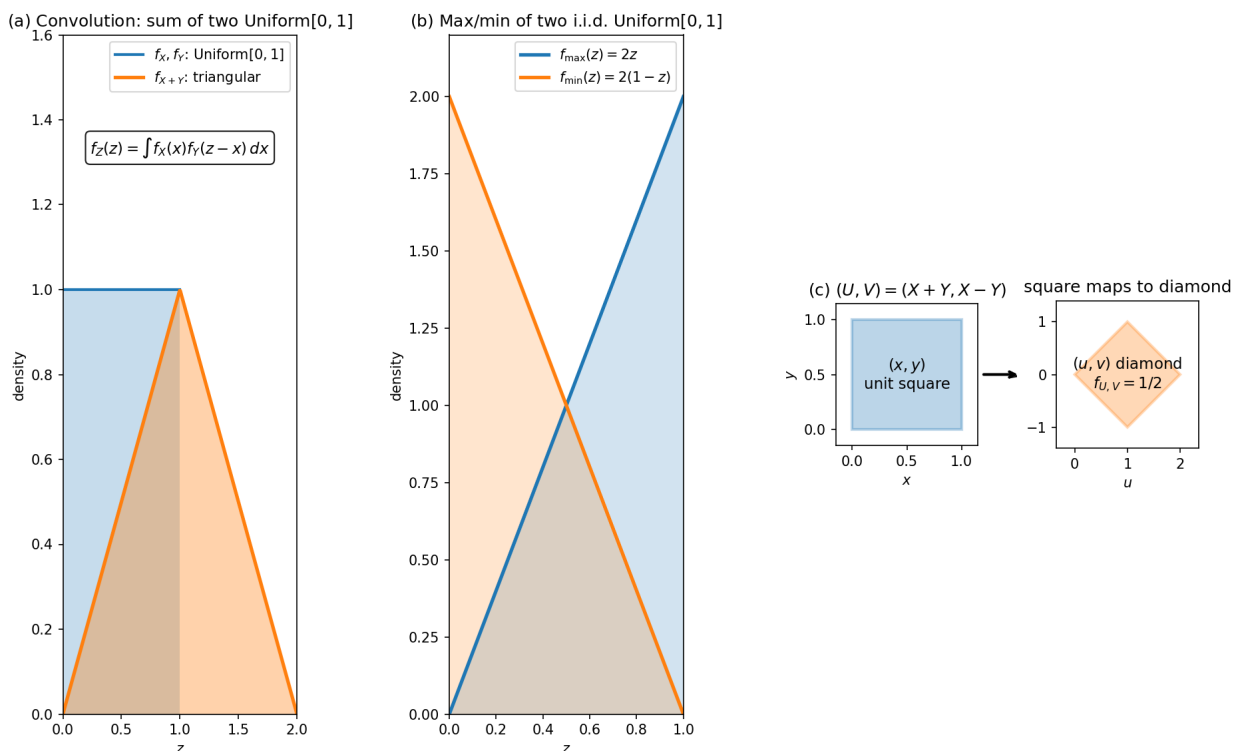


Figure 20: (a) Convolution of two Uniform[0,1] densities: the sum $Z=X+Y$ has the triangular PDF $f(z)=z$ on $(0,1)$, $2-z$ on $(1,2)$. (b) Max and min of two i.i.d. Uniform[0,1]: $f_{\max}(z)=2z$ piles near 1, $f_{\min}(z)=2(1-z)$ piles near 0. (c) Jacobian example (eg 9): the unit square in (x,y) maps to the diamond $|v|<\min(u,2-u)$, $0<u<2$ in (u,v) under $(U,V)=(X+Y,X-Y)$, with constant density $1/2$.

Solutions — 18. Joint continuous distributions

1. (2-D uniform triangle — the deck's Q1) D is the right triangle with vertices $(0,0)$, $(2,0)$, $(0,2)$; $|D| = \frac{1}{2} \cdot 2 \cdot 2 = 2$, so $f_{X,Y}(x,y) = 1/2$ on D (§18.2).

(a) $A = \{x + y < 1, x > 0, y > 0\}$ is a triangle with legs 1: $|A| = 1/2$. By the area ratio,

$$P(X + Y < 1) = \frac{1/2}{2} = \frac{1}{4}.$$

(b) The complement inside D : $\{x + 2y < 1, x > 0, y > 0\}$ is a triangle with x -intercept 1 and y -intercept $1/2$, area $\frac{1}{2} \cdot 1 \cdot \frac{1}{2} = 1/4$. Hence

$$P(X + 2Y > 1) = \frac{2 - 1/4}{2} = \frac{7/4}{2} = \frac{7}{8}.$$

2. (Region integration — the deck's Q2) (a) Non-negative on the unit square, and

$$\int_0^1 \int_0^1 (x + y) dy dx = \int_0^1 \left(x + \frac{1}{2}\right) dx = \frac{1}{2} + \frac{1}{2} = \boxed{1},$$

so it is a valid joint density (§18.2, eg 2).

(b)

$$P(X < \tfrac{1}{2}, Y < \tfrac{1}{2}) = \int_0^{1/2} \int_0^{1/2} (x+y) dy dx = \int_0^{1/2} \left(\frac{x}{2} + \frac{1}{8}\right) dx = \left[\frac{x^2}{4} + \frac{x}{8}\right]_0^{1/2} = \frac{1}{16} + \frac{1}{16} = \boxed{\frac{1}{8}}.$$

(The deck prints $3/16$, integrating $x/2$ to $x^2/2$ instead of $x^2/4$ — an arithmetic slip; $1/8$ is correct.)(c) The region $\{x+y < 1\} \cap [0, 1]^2$: $0 < x < 1$, $0 < y < 1-x$:

$$\begin{aligned} P(X+Y < 1) &= \int_0^1 \int_0^{1-x} (x+y) dy dx = \int_0^1 \left(x(1-x) + \frac{(1-x)^2}{2}\right) dx \\ &= \int_0^1 \left(\frac{1}{2} - \frac{x^2}{2}\right) dx = \frac{1}{2} - \frac{1}{6} = \boxed{\frac{1}{3}}. \end{aligned}$$

3. (Marginals — the deck's 3.5 Q3) (a) Integrating out (§17.13):

$$f_X(x) = \int_0^1 \frac{6}{5}(x+y^2) dy = \frac{6}{5} \left[x + \frac{1}{3}\right], \quad \boxed{f_X(x) = \frac{6}{5} \left(x + \frac{1}{3}\right)}, \quad 0 < x < 1,$$

$$f_Y(y) = \int_0^1 \frac{6}{5}(x+y^2) dx = \frac{6}{5} \left[\frac{1}{2} + y^2\right], \quad \boxed{f_Y(y) = \frac{6}{5} \left(\frac{1}{2} + y^2\right)}, \quad 0 < y < 1.$$

Check: $\int_0^1 \frac{6}{5}(x+1/3) dx = \frac{6}{5}(1/2+1/3) = 1$; $\int_0^1 \frac{6}{5}(1/2+y^2) dy = \frac{6}{5}(1/2+1/3) = 1$.(b) Independence test (§17.5): at $(0,0)$, $f_{X,Y}(0,0) = 0$ but $f_X(0)f_Y(0) = \frac{6}{5} \cdot \frac{1}{3} \cdot \frac{6}{5} \cdot \frac{1}{2} = \frac{6}{25} \neq 0$.

Not independent.

4. (Uniform marginals, dependent joint — the deck's §3.2 Q2) $|D| = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$, so $f_{X,Y} = 2$ on D (the deck's §3.1).(a) For $0 < x < \frac{1}{2}$: $f_X(x) = \int_0^{1/2} 2 dy = 1$. For $\frac{1}{2} < x < 1$: $f_X(x) = \int_{1/2}^1 2 dy = 1$. So

$f_X(x) = 1, 0 < x < 1$

: $X \sim \text{Uniform}[0, 1]$. By symmetry,

$f_Y(y) = 1, 0 < y < 1$

: $Y \sim \text{Uniform}[0, 1]$.(b) Zero-cell test (§17.5): $f_{X,Y}(1/4, 3/4) = 0$ (off the diagonal squares), while $f_X(1/4)f_Y(3/4) = 1 \cdot 1 = 1 \neq 0$.

Not independent

 — uniform marginals do not imply a uniform (independent) joint.**5. (Convolution: uniforms)** §18.3, eg 3. With $f_X = f_Y = \mathbf{1}_{(0,1)}$,

$$f_Z(z) = \int_{-\infty}^{\infty} f_X(x) f_Y(z-x) dx,$$

the integrand is 1 on $x \in (0, 1) \cap (z-1, z)$. For $0 < z < 1$ the overlap is $(0, z)$: $f_Z(z) = z$. For $1 \leq z < 2$ the overlap is $(z-1, 1)$: $f_Z(z) = 2-z$. Hence

$$f_Z(z) = \begin{cases} z & 0 < z < 1, \\ 2-z & 1 \leq z < 2, \\ 0 & \text{otherwise,} \end{cases}$$

the triangular distribution — panel (a) of the figure.

6. (Convolution: exponentials) §18.3, eg 4. For $z > 0$:

$$f_Z(z) = \int_0^z \lambda e^{-\lambda x} \lambda e^{-\lambda(z-x)} dx = \lambda^2 e^{-\lambda z} \int_0^z dx = \boxed{\lambda^2 z e^{-\lambda z}}, \quad z > 0$$

(0 for $z \leq 0$): the Gamma(2, λ) density. Check the mean: $E[Z] = \int_0^\infty \lambda^2 z^2 e^{-\lambda z} dz = \frac{1}{\lambda} \cdot 2! = 2/\lambda = E[X] + E[Y]$ (§17.7).

7. (Max and min) §18.4, eg 6. $F_X(z) = z$ on $(0, 1)$. max: $F_M(z) = z^2$, so $f_{\max}(z) = 2z$, $0 < z < 1$. min: $F_m(z) = 1 - (1 - z)^2 = 2z - z^2$, so $f_{\min}(z) = 2(1 - z)$, $0 < z < 1$. (Panel (b) of the figure.)

8. (Min of exponentials) (a) §18.4, eg 7:

$$P(\min(X, Y) > z) = P(X > z) P(Y > z) = e^{-\lambda_1 z} e^{-\lambda_2 z} = e^{-(\lambda_1 + \lambda_2)z}, \quad z > 0,$$

so $\min(X, Y) \sim \text{Exp}(\lambda_1 + \lambda_2)$.

(b) §18.4, eg 8:

$$\begin{aligned} P(X > Y) &= \int_0^\infty \int_0^x \lambda_1 e^{-\lambda_1 x} \lambda_2 e^{-\lambda_2 y} dy dx = \int_0^\infty \lambda_1 e^{-\lambda_1 x} (1 - e^{-\lambda_2 x}) dx \\ &= 1 - \frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{\lambda_2}{\lambda_1 + \lambda_2}. \end{aligned}$$

9. (Jacobian) §18.5, eg 9. (a) $U = X + Y$, $V = X - Y$ inverts to $x = (u + v)/2$, $y = (u - v)/2$;

$$\left| \frac{\partial(x, y)}{\partial(u, v)} \right| = \left| \det \begin{pmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{pmatrix} \right| = \frac{1}{2}.$$

The unit square maps to the diamond $0 < u < 2$, $|v| < \min(u, 2 - u)$ (panel (c) of the figure). Hence

$$f_{U,V}(u, v) = \frac{1}{2} \text{ on the diamond, } 0 \text{ elsewhere.}$$

(b) $f_U(u) = \int_{-\min(u, 2-u)}^{\min(u, 2-u)} \frac{1}{2} dv = \min(u, 2 - u)$, i.e.

$$f_U(u) = \begin{cases} u & 0 < u < 1, \\ 2 - u & 1 \leq u < 2, \end{cases}$$

identical to Problem 5's convolution result — the Jacobian and convolution routes agree.

10. (MGF of a sum) §18.6, eg 11. $M_X(s) = e^{\mu_1 s + \sigma_1^2 s^2/2}$, $M_Y(s) = e^{\mu_2 s + \sigma_2^2 s^2/2}$. By independence,

$$M_{X+Y}(s) = M_X(s) M_Y(s) = e^{(\mu_1 + \mu_2)s + (\sigma_1^2 + \sigma_2^2)s^2/2},$$

the MGF of Normal($\mu_1 + \mu_2$, $\sigma_1^2 + \sigma_2^2$). By MGF uniqueness,

$$X + Y \sim \text{Normal}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2).$$

19. Random vectors and the multivariate normal

Chapter 18 worked with a pair (X, Y) ; this chapter stacks d variables into one column vector $\mathbf{X} = (X_1, \dots, X_d)^T$ and builds the single most-used distribution in machine learning on top of it: the multivariate normal. Chapter 18's §18.6 already showed the *sum* of independent normals is normal; here the *whole vector* is normal, the covariance becomes a matrix, and everything — affine maps, marginals, conditionals, standardization — stays inside the normal family. The core construction is exactly the source's: start from independent standard normals and apply a linear map.

19.1 Random vectors: the mean vector and the covariance matrix

Def. A **random vector** is $\mathbf{X} = (X_1, \dots, X_d)^T$, a column of d random variables on the same experiment. Its joint PDF $f_{\mathbf{X}}(\mathbf{x})$ works exactly like §17.12's joint PDF, one dimension higher:

$$P(\mathbf{X} \in A) = \int_A f_{\mathbf{X}}(\mathbf{x}) d\mathbf{x}.$$

Two summaries replace the scalar mean and variance:

Def (mean vector). $E[\mathbf{X}] = (E[X_1], \dots, E[X_d])^T = \mu$ — expectation acts coordinate-wise.

Def (covariance matrix). With $\mu = E[\mathbf{X}]$,

$$\Sigma = \text{Cov}(\mathbf{X}) = E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)^T],$$

a $d \times d$ matrix whose (i, j) entry is

$$\Sigma_{ij} = E[(X_i - \mu_i)(X_j - \mu_j)] = \text{Cov}(X_i, X_j) = E[X_i X_j] - E[X_i]E[X_j] \quad (\S 17.9).$$

- i) **The diagonal is variance:** $\Sigma_{ii} = \text{Var}(X_i) \geq 0$.
- ii) **Symmetry:** $\Sigma_{ij} = \Sigma_{ji}$, so $\Sigma^T = \Sigma$ — the matrix is symmetric.
- iii) **The short form.** Expanding the outer product,

$$\Sigma = E[\mathbf{X}\mathbf{X}^T] - \mu\mu^T$$

(proof: $E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)^T] = E[\mathbf{X}\mathbf{X}^T] - \mu\mu^T - \mu\mu^T + \mu\mu^T$ — Problem 2 asks you to write it out).

Theorem (a covariance matrix is always positive semi-definite). For *any* fixed vector $\mathbf{a} \in \mathbb{R}^d$,

$$\mathbf{a}^T \Sigma \mathbf{a} = \text{Var}(\mathbf{a}^T \mathbf{X}) \geq 0,$$

so Σ is positive semi-definite (§7.9's PSD concept).

Proof. $\mathbf{a}^T \Sigma \mathbf{a} = \mathbf{a}^T E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)^T] \mathbf{a} = E[\mathbf{a}^T (\mathbf{X} - \mu)(\mathbf{X} - \mu)^T \mathbf{a}] = E[(\mathbf{a}^T (\mathbf{X} - \mu))^2]$. But $\mathbf{a}^T (\mathbf{X} - \mu) = \mathbf{a}^T \mathbf{X} - \mathbf{a}^T \mu$ is a scalar random variable with mean 0, so its second moment is its variance, and a variance is never negative.

Note. This is the same argument as §7.9's Problem 7 ($\mathbf{A}^T \mathbf{A}$ is PSD for any real $\mathbf{A}$): a covariance is a sum/average of outer products $\mathbf{v}\mathbf{v}^T$, and $\mathbf{a}^T \mathbf{v}\mathbf{v}^T \mathbf{a} = (\mathbf{a}^T \mathbf{v})^2 \geq 0$ term by term. Consequences: every eigenvalue of Σ is ≥ 0 (§7.9), and $\det(\Sigma) \geq 0$.

Note (correlation form). Dividing by the standard deviations recovers §17.10's correlation coefficient entry-wise: $\rho_{ij} = \Sigma_{ij}/(\sigma_i \sigma_j)$, i.e. $\Sigma = \mathbf{D}\mathbf{R}\mathbf{D}$ with $\mathbf{D} = \text{diag}(\sigma_1, \dots, \sigma_d)$ and $\mathbf{R}$ the correlation matrix ($R_{ii} = 1$, $|R_{ij}| \leq 1$).

eg 1 (from a joint table). (X, Y) discrete with $P(0, 0) = 0.1$, $P(0, 1) = 0.2$, $P(1, 0) = 0.3$, $P(1, 1) = 0.4$ (first index X , second Y). - Means: $E[X] = 0(0.3) + 1(0.7) = 0.7$; $E[Y] = 0(0.4) + 1(0.6) = 0.6$. So $\mu = (0.7, 0.6)^T$. - Variances: $E[X^2] = E[X] = 0.7$ (since $X^2 = X$ for 0–1 values), $\text{Var}(X) = 0.7 - 0.7^2 = 0.21$; $\text{Var}(Y) = 0.6 - 0.6^2 = 0.24$. - $E[XY] = P(X = 1, Y = 1) = 0.4$; $\text{Cov}(X, Y) = 0.4 - 0.7 \cdot 0.6 = -0.02$.

$$\Sigma = \begin{pmatrix} 0.21 & -0.02 \\ -0.02 & 0.24 \end{pmatrix}.$$

Symmetric . PSD check: diagonal > 0 and $\det = 0.21 \cdot 0.24 - 0.0004 = 0.05 > 0$ (§7.9's 2×2 test) . Correlation $\rho = -0.02/\sqrt{0.21 \cdot 0.24} \approx -0.089$ — nearly uncorrelated.

Basically, ... A random vector's covariance matrix is just “§17.9's covariance, arranged in a table”: diagonal = each variable's variance, off-diagonal = each pair's covariance. It is always symmetric and positive semi-definite (the PSD fact is just “ $\mathbf{a}^T \Sigma \mathbf{a}$ is a variance, and variances are non-negative”). If you remember one thing: *a covariance matrix can never have a negative eigenvalue.*

19.2 Affine transformations: $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{b}$

Every later result in this chapter is an application of two formulas. Let $\mathbf{X}$ have mean μ and covariance Σ , $\mathbf{A}$ an $m \times d$ matrix, $\mathbf{b}$ a vector.

Theorem. For $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{b}$:

$$E[\mathbf{Y}] = \mathbf{A}\mu + \mathbf{b}, \quad \text{Cov}(\mathbf{Y}) = \mathbf{A}\Sigma\mathbf{A}^T.$$

Proof. Mean: $E[\mathbf{A}\mathbf{X} + \mathbf{b}] = \mathbf{A}E[\mathbf{X}] + \mathbf{b}$ by linearity of expectation, coordinate-wise. Covariance: $\mathbf{Y} - E[\mathbf{Y}] = \mathbf{A}(\mathbf{X} - \mu)$ (the $\mathbf{b}$ cancels), so

$$\text{Cov}(\mathbf{Y}) = E[\mathbf{A}(\mathbf{X} - \mu)(\mathbf{X} - \mu)^T \mathbf{A}^T] = \mathbf{A} E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)^T] \mathbf{A}^T = \mathbf{A}\Sigma\mathbf{A}^T,$$

pulling the constant matrices out of the expectation.

- i) Scalar special case: for $\mathbf{a}^T \mathbf{X}$ (one linear combination), $\text{Var}(\mathbf{a}^T \mathbf{X}) = \mathbf{a}^T \Sigma \mathbf{a}$ — exactly the PSD proof's quantity in §19.1.
- ii) §17.11 is the $d = 2$ case: $\text{Var}(X + Y)$ is $(1, 1) \Sigma (1, 1)^T = \sigma_1^2 + \sigma_2^2 + 2\text{Cov}(X, Y)$.
- iii) The shift $\mathbf{b}$ moves the mean but never touches the covariance — spreading is translation-invariant.

eg 2. $\mu = (0, 1)^T$, $\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$, and $\mathbf{Y} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \mathbf{X}$ (sum and difference — §18.5's eg 9 transform, no

shift). - $E[\mathbf{Y}] = \mathbf{A}\mu = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$. - $\Sigma\mathbf{A}^T = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} 3 & 1 \\ 2 & 0 \end{pmatrix}$; then

$$\text{Cov}(\mathbf{Y}) = \mathbf{A}(\Sigma\mathbf{A}^T) = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 3 & 1 \\ 2 & 0 \end{pmatrix} = \begin{pmatrix} 5 & 1 \\ 1 & 3 \end{pmatrix}.$$

Sanity: $\text{Var}(X_1 + X_2) = 2 + 1 + 2(1) = 5$ (§17.11); symmetric ; $\det = 15 - 1 = 14 > 0$, still PSD .

Basically, ... An affine map does the obvious thing to the mean (map it, then shift) and sandwiches the covariance ($\mathbf{A}\Sigma\mathbf{A}^T$). The sandwich is the whole story of this chapter: the multivariate normal is *built* by sandwiching the identity covariance.

19.3 The standard normal vector

Def. Let $Z_1, \dots, Z_d \stackrel{\text{i.i.d.}}{\sim} N(0, 1)$ (§16.9). The **standard normal vector** is $\mathbf{Z} = (Z_1, \dots, Z_d)^T$. By independence the joint density is the product:

$$f_{\mathbf{Z}}(\mathbf{z}) = \prod_{i=1}^d \frac{1}{\sqrt{2\pi}} e^{-z_i^2/2} = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{1}{2}\|\mathbf{z}\|^2\right).$$

- i) $E[\mathbf{Z}] = \mathbf{0}$ and $\text{Cov}(\mathbf{Z}) = \mathbf{I}$ — variance 1 on the diagonal, independence 0 off it (§17.9).
- ii) The density depends on $\mathbf{z}$ only through $\|\mathbf{z}\|^2 = \mathbf{z}^T \mathbf{z}$: it is **rotationally symmetric** — rotating $\mathbf{z}$ changes nothing. (This is why the construction in §19.6 works with *any* square root $\mathbf{A}$ of Σ .)

Basically, ... The standard normal vector = “ d independent copies of the bell curve, packed into one column”. Its density is a perfect round hill: $(2\pi)^{-d/2}$ normalizer, $\exp(-\|\mathbf{z}\|^2/2)$ shape. Mean zero, covariance the identity — the blank canvas everything else is painted on.

19.4 The bivariate normal: the source’s construction

The MLF deck builds the bivariate normal in one concrete case, then generalizes — this chapter follows the same path. Take $Z_1, Z_2 \stackrel{\text{i.i.d.}}{\sim} N(0, 1)$ and, for a fixed $\rho \in (-1, 1)$,

$$\boxed{X_1 = Z_1, \quad X_2 = \rho Z_1 + \sqrt{1 - \rho^2} Z_2}, \quad \text{i.e. } \mathbf{X} = \mathbf{A}\mathbf{Z}, \quad \mathbf{A} = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1 - \rho^2} \end{pmatrix}.$$

Step 1 — mean and covariance (§19.2). $E[X_1] = E[Z_1] = 0$; $E[X_2] = \rho \cdot 0 + \sqrt{1 - \rho^2} \cdot 0 = 0$. $\text{Var}(X_1) = 1$. $\text{Var}(X_2) = \rho^2 \text{Var}(Z_1) + (1 - \rho^2) \text{Var}(Z_2) = \rho^2 + 1 - \rho^2 = 1$ (independence kills the cross term, §17.11). And

$$\text{Cov}(X_1, X_2) = E[X_1 X_2] = E[Z_1(\rho Z_1 + \sqrt{1 - \rho^2} Z_2)] = \rho E[Z_1^2] + \sqrt{1 - \rho^2} E[Z_1]E[Z_2] = \rho \cdot 1 + 0 = \rho.$$

So the parameter ρ is the correlation coefficient (§17.10), and

$$\boxed{\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}}.$$

Check against the sandwich: $\mathbf{A}\mathbf{A}^T = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1 - \rho^2} \end{pmatrix} \begin{pmatrix} 1 & \rho \\ 0 & \sqrt{1 - \rho^2} \end{pmatrix} = \begin{pmatrix} 1 & \rho \\ \rho & \rho^2 + 1 - \rho^2 \end{pmatrix} = \Sigma$.

Step 2 — the density via the Jacobian (§18.5). $\mathbf{A}$ is invertible ($\det \mathbf{A} = \sqrt{1 - \rho^2} > 0$), with

$$\mathbf{A}^{-1} = \begin{pmatrix} 1 & 0 \\ -\rho/\sqrt{1 - \rho^2} & 1/\sqrt{1 - \rho^2} \end{pmatrix}, \quad |\det \mathbf{A}^{-1}| = \frac{1}{\sqrt{1 - \rho^2}}.$$

The linear change of variables (§18.5, constant-Jacobian case) gives

$$f_{\mathbf{X}}(\mathbf{x}) = f_{\mathbf{Z}}(\mathbf{A}^{-1}\mathbf{x}) |\det \mathbf{A}^{-1}| = \frac{1}{2\pi\sqrt{1 - \rho^2}} \exp\left(-\frac{1}{2}\|\mathbf{A}^{-1}\mathbf{x}\|^2\right).$$

Since $\|\mathbf{A}^{-1}\mathbf{x}\|^2 = \mathbf{x}^T \mathbf{A}^{-T} \mathbf{A}^{-1} \mathbf{x}$ and $\mathbf{A}^{-T} \mathbf{A}^{-1} = (\mathbf{A} \mathbf{A}^T)^{-1} = \Sigma^{-1}$,

$$f_{\mathbf{X}}(\mathbf{x}) = \frac{1}{2\pi\sqrt{\det \Sigma}} \exp\left(-\frac{1}{2} \mathbf{x}^T \Sigma^{-1} \mathbf{x}\right), \quad \Sigma^{-1} = \frac{1}{1-\rho^2} \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix}.$$

Note $\det \Sigma = 1 - \rho^2 = (\det \mathbf{A})^2$ — the Jacobian factor is exactly $1/\sqrt{\det \Sigma}$.

eg 3 (the density written out, $\rho = 1/2$). $\det \Sigma = 3/4$, $\Sigma^{-1} = \frac{4}{3} \begin{pmatrix} 1 & -1/2 \\ -1/2 & 1 \end{pmatrix} = \begin{pmatrix} 4/3 & -2/3 \\ -2/3 & 4/3 \end{pmatrix}$.

Hence

$$f(x_1, x_2) = \frac{1}{\pi\sqrt{3}} \exp\left(-\frac{2}{3}(x_1^2 - x_1x_2 + x_2^2)\right)$$

(normalizer: $2\pi\sqrt{3/4} = \pi\sqrt{3}$; exponent: $-\frac{1}{2} \cdot \frac{4}{3}(x_1^2 - x_1x_2 + x_2^2)$).

The figure shows this family for $\rho \in \{-0.7, 0, 0.7\}$: the contours are ellipses whose tilt follows the sign of ρ ; at $\rho = 0$ they are circles and the two coordinates are independent.

Basically, ... The bivariate normal = “take two independent standard normals, tilt one into the other by the correlation ρ ”. Everything — the $\begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$ covariance, the $1/\sqrt{1-\rho^2}$ normalizer, the elliptical contours — falls out of that one construction plus the Jacobian rule. The general multivariate normal is the same idea with a $d \times d$ matrix.

19.5 Marginals and conditionals of the bivariate normal

The deck’s payoff move: factor the §19.4 density. Since $\|\mathbf{A}^{-1}\mathbf{x}\|^2 = x_1^2 + (x_2 - \rho x_1)^2/(1 - \rho^2)$,

$$\begin{aligned} f_{\mathbf{X}}(x_1, x_2) &= \frac{1}{2\pi\sqrt{1-\rho^2}} \exp\left(-\frac{x_1^2}{2}\right) \exp\left(-\frac{(x_2 - \rho x_1)^2}{2(1-\rho^2)}\right) \\ &= \underbrace{\frac{1}{\sqrt{2\pi}} e^{-x_1^2/2}}_{N(0,1) \text{ density in } x_1} \cdot \underbrace{\frac{1}{\sqrt{2\pi}\sqrt{1-\rho^2}} \exp\left(-\frac{(x_2 - \rho x_1)^2}{2(1-\rho^2)}\right)}_{N(\rho x_1, 1-\rho^2) \text{ density in } x_2}. \end{aligned}$$

The joint factors as marginal $\times$ conditional (§17.4’s multiplication rule), and both factors are normal:

$$\boxed{X_1 \sim N(0, 1)}, \quad \boxed{X_2 \mid X_1 = x_1 \sim N(\rho x_1, 1 - \rho^2)}.$$

- i) The conditional *mean* $E[X_2 \mid X_1 = x_1] = \rho x_1$ is a straight line through the origin — the green line in the figure. Observing x_1 shifts your best guess of X_2 toward it, scaled by ρ .
- ii) The conditional *variance* $1 - \rho^2$ is *smaller* than the marginal variance 1 (unless $\rho = 0$): knowing X_1 removes the ρ^2 share of X_2 ’s uncertainty.
- iii) By symmetry of the joint density in x_1, x_2 (swap the labels — the deck leaves this as an exercise): $X_2 \sim N(0, 1)$ and $\boxed{X_1 \mid X_2 = x_2 \sim N(\rho x_2, 1 - \rho^2)}$.

General bivariate normal. Rescaling (§19.7) lifts this to $X_i \sim N(\mu_i, \sigma_i^2)$ with correlation ρ :

$$\boxed{X_1 \sim N(\mu_1, \sigma_1^2)}, \quad \boxed{X_2 \mid X_1 = x_1 \sim N\left(\mu_2 + \rho \frac{\sigma_2}{\sigma_1}(x_1 - \mu_1), \sigma_2^2(1 - \rho^2)\right)}.$$

(Hand-derived from the standard case by $X_i = \mu_i + \sigma_i Z_i$ -style scaling; logged in the review.)

eg 4 (conditioning, $\rho = 0.5$, standard case). Observe $X_1 = 2$. Then $X_2 \mid X_1 = 2 \sim N(0.5 \cdot 2, 1 - 0.25) = N(1, 0.75)$. Hence

$$P(X_2 > 1.5 \mid X_1 = 2) = P\left(Z > \frac{1.5 - 1}{\sqrt{0.75}}\right) = P(Z > 0.5774) = 1 - \Phi(0.5774) \approx \boxed{0.282}.$$

Note how conditioning *helped*: the marginal $P(X_2 > 1.5) = P(Z > 1.5) \approx 0.067$; the observation $X_1 = 2$ more than quadruples it.

Basically, ... For the bivariate normal, marginals and conditionals are all normal — that is the special property the deck highlights. Condition on $X_1 = x_1$ and X_2 is still a bell curve, just recentered at ρx_1 (the regression line) and squeezed to variance $1 - \rho^2$. Correlation = “how much of X_2 ’s wiggle X_1 explains”.

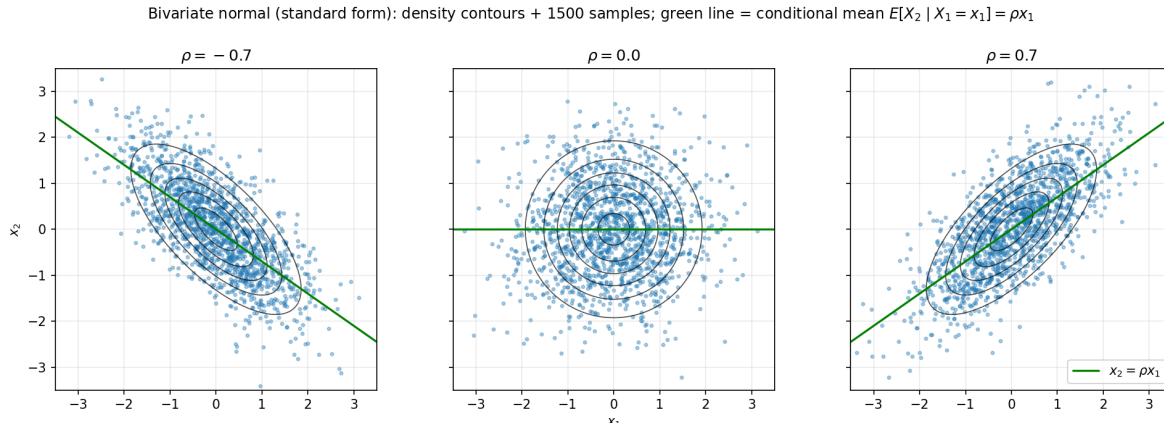


Figure 21: Bivariate normal (standard form) for $\rho = -0.7, 0, 0.7$: density contour ellipses plus 1500 samples per panel drawn via the chapter’s construction $X_1 = Z_1$, $X_2 = \rho Z_1 + \sqrt{1 - \rho^2} Z_2$; the green line in each panel is the conditional mean $E[X_2 | X_1 = x_1] = \rho x_1$. At $\rho = 0$ the contours are circles and X_1, X_2 are independent.

19.6 The general multivariate normal: definition

Nothing in §19.4 used that $d = 2$ except the matrix size. Let $\mathbf{Z} \in \mathbb{R}^d$ be the standard normal vector (§19.3), $\mathbf{A}$ any invertible $d \times d$ matrix, $\mu \in \mathbb{R}^d$:

$$\boxed{\mathbf{X} = \mathbf{A}\mathbf{Z} + \mu \sim \mathcal{N}_d(\mu, \Sigma)}, \quad \Sigma = \mathbf{A}\mathbf{A}^T.$$

Def (multivariate normal PDF). For positive *definite* Σ ,

$$\boxed{f_{\mathbf{X}}(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} \sqrt{\det \Sigma}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right)}.$$

The derivation is §19.4’s Jacobian computation with d in place of 2: $f_{\mathbf{X}}(\mathbf{x}) = f_{\mathbf{Z}}(\mathbf{A}^{-1}(\mathbf{x} - \mu)) |\det \mathbf{A}^{-1}|$, and $|\det \mathbf{A}^{-1}| = 1/\sqrt{\det \Sigma}$ since $\det \Sigma = (\det \mathbf{A})^2$. Then $E[\mathbf{X}] = \mu$ and $\text{Cov}(\mathbf{X}) = \mathbf{A}\mathbf{A}^T = \Sigma$ by §19.2.

- i) **Two parameters only.** The whole distribution is fixed by the mean vector μ (d numbers) and the covariance matrix Σ ($d(d+1)/2$ numbers by symmetry).
- ii) **The density sees $\mathbf{A}$ only through Σ .** Different square roots of the same Σ give different random vectors with the *same* density — the deck’s scalar example: Z and $-Z$ are different variables with identical $N(0, 1)$ densities.
- iii) **Σ must be positive definite** (§7.9): $\Sigma = \mathbf{A}\mathbf{A}^T$ with $\mathbf{A}$ invertible forces every eigenvalue > 0 , so $\det \Sigma > 0$ and Σ^{-1} exists. The deck’s 2×2 form: $\begin{pmatrix} a^2 & \rho ab \\ \rho ab & b^2 \end{pmatrix}$ with $|\rho| < 1$; $\begin{pmatrix} 1 & 3 \\ 3 & 1 \end{pmatrix}$ is *not* a valid covariance ($\rho = 3$ is impossible — compare §7.9’s non-definite $\begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$).

Def (Mahalanobis distance). The exponent's quadratic form defines

$$d_{\Sigma}(\mathbf{x}, \mu) = \sqrt{(\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu)},$$

so $f_{\mathbf{X}}(\mathbf{x}) \propto \exp(-d_{\Sigma}^2/2)$ — the **Mahalanobis form**.

- i) When $\Sigma = \mathbf{I}$, this is ordinary Euclidean distance: the standard normal is “Mahalanobis with the identity”.
- ii) Contours $d_{\Sigma} = c$ are **ellipsoids** centered at μ whose axes point along Σ 's eigenvectors (§6.11), with half-lengths $c\sqrt{\lambda_i}$ — the figure's ellipses, in d dimensions. (Chapter 24's PCA reads these axes as the principal components.)
- iii) Reading: distance measured in units of *spread along each direction* — a point 3 units out along a high-variance direction is “closer” (more likely) than 3 units along a low-variance one.

Basically, ... The multivariate normal = “take the round standard-normal hill, stretch it by $\mathbf{A}$, move it to μ ”. Its density needs only the mean vector and the covariance matrix. The Mahalanobis distance is the ruler that knows about the stretching: points are judged not by raw Euclidean distance from the mean, but by distance *relative to how spread out each direction is* — which is why the contours are ellipses, not spheres.

19.7 Affine maps stay normal: properties

The deck lists the key properties; all follow from $\mathbf{X} = \mathbf{AZ} + \mu$ plus §19.2.

Theorem (affine closure). If $\mathbf{X} \sim \mathcal{N}_d(\mu, \Sigma)$, $\mathbf{B}$ is $k \times d$, and $\mathbf{c} \in \mathbb{R}^k$, then

$$\mathbf{Y} = \mathbf{BX} + \mathbf{c} \sim \mathcal{N}_k(\mathbf{B}\mu + \mathbf{c}, \mathbf{B}\Sigma\mathbf{B}^T).$$

Why. Write $\mathbf{X} = \mathbf{AZ} + \mu$; then $\mathbf{Y} = (\mathbf{BA})\mathbf{Z} + (\mathbf{B}\mu + \mathbf{c})$ is again “linear map of $\mathbf{Z}$ plus shift” — the *definition* of a (possibly lower-dimensional) normal. Mean and covariance come from §19.2.

- i) **Linear combinations are (univariate) normal.** For a fixed vector $\mathbf{a}$: $\mathbf{a}^T \mathbf{X} \sim N(\mathbf{a}^T \mu, \mathbf{a}^T \Sigma \mathbf{a})$. In particular the sum of independent normals is normal — §18.6's MGF result, now as a one-line corollary.
- ii) **Standardization / whitening.** With $\Sigma = \mathbf{Q}\Lambda\mathbf{Q}^T$ (§6.11), $\mathbf{W} = \Sigma^{-1/2}(\mathbf{X} - \mu) \sim \mathcal{N}_d(\mathbf{0}, \mathbf{I})$ where $\Sigma^{-1/2} = \mathbf{Q}\Lambda^{-1/2}\mathbf{Q}^T$; equivalently $\mathbf{Z} = \mathbf{A}^{-1}(\mathbf{X} - \mu)$. The deck's instance: $\mathbf{B} = \text{diag}(1/a, 1/b)$ turns the general bivariate $\Sigma = \begin{pmatrix} a^2 & \rho ab \\ \rho ab & b^2 \end{pmatrix}$ into the correlation form $\begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$ (check: $\mathbf{B}\Sigma\mathbf{B}^T$ divides row/column i by the i -th scale).
- iii) **Zero covariance independence (normal only!).** If $\mathbf{X} \sim \mathcal{N}_d(\mu, \Sigma)$, then $X_i \perp X_j$ iff $\Sigma_{ij} = 0$. The direction holds for *any* variables (§17.9: independence $\Rightarrow$ Cov = 0); the direction is special to the normal — for general variables, uncorrelated does *not* imply independent (§17.10's warning).

eg 5 (the deck tutorial's N_3 question). $\mathbf{Y} \sim \mathcal{N}_3(\mu, \Sigma)$ with $\mu = (3, 1, 4)^T$ and $\Sigma = \begin{pmatrix} 6 & 1 & -2 \\ 1 & 13 & 4 \\ -2 & 4 & 4 \end{pmatrix}$.

Find the distribution of $Z = 2Y_1 - Y_2 + 3Y_3 = \mathbf{c}^T \mathbf{Y}$, $\mathbf{c} = (2, -1, 3)^T$. By (i): $Z \sim N(\mathbf{c}^T \mu, \mathbf{c}^T \Sigma \mathbf{c})$. - Mean: $2(3) + (-1)(1) + 3(4) = 6 - 1 + 12 = 17$. - $\Sigma \mathbf{c} = \begin{pmatrix} 6 & 1 & -2 \\ 1 & 13 & 4 \\ -2 & 4 & 4 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \\ 3 \end{pmatrix} = \begin{pmatrix} 12 - 1 - 6 \\ 2 - 13 + 12 \\ -4 - 4 + 12 \end{pmatrix} = \begin{pmatrix} 5 \\ 1 \\ 4 \end{pmatrix}$; variance $= \mathbf{c}^T (\Sigma \mathbf{c}) = 2(5) + (-1)(1) + 3(4) = 10 - 1 + 12 = 21$.

$$Z \sim N(17, 21)$$

(the tutorial's answer A — e.g. $P(Z > 22) = P(Z_{\text{std}} > 5/\sqrt{21}) \approx P(Z_{\text{std}} > 1.091) \approx 0.138$).

eg 6 (whitening a diagonal case). $\mathbf{X} \sim \mathcal{N}_2((1, 2)^T, \text{diag}(4, 9))$. Standardize coordinate-wise: $W_1 = (X_1 - 1)/2$, $W_2 = (X_2 - 2)/3$, i.e. $\mathbf{W} = \text{diag}(1/2, 1/3)(\mathbf{X} - \mu)$. By the theorem, $\mathbf{W} \sim \mathcal{N}_2(\mathbf{0}, \mathbf{B}\Sigma\mathbf{B}^T)$ with $\mathbf{B}\Sigma\mathbf{B}^T = \text{diag}(1/2, 1/3) \text{diag}(4, 9) \text{diag}(1/2, 1/3) = \mathbf{I}$. So $\boxed{\mathbf{W} \sim \mathcal{N}_2(\mathbf{0}, \mathbf{I})}$ — Problem 7 does the correlated case via the eigendecomposition.

Note (degenerate case). If $\mathbf{B}\Sigma\mathbf{B}^T$ is singular (e.g. $\mathbf{B}$ has fewer rows than d , or maps into a lower-dimensional subspace), $\mathbf{Y}$ is still normal in the “ $\mathbf{AZ} + \mu$ ” sense, but it has no density on $\mathbb{R}^k$ — all its mass sits on a lower-dimensional affine subspace. The mean/covariance formulas stay valid; only the PDF formula (§19.6) needs a positive definite covariance.

Basically, ... “Normal in, normal out”: any affine transformation of a normal vector — a weighted sum, a rescaling, a projection — is still normal, with mean and covariance from §19.2’s formulas. Whitening is the reverse gear: subtract the mean, multiply by $\Sigma^{-1/2}$, and any normal becomes the round standard one. And the famous freebie: for normals (only for normals), “uncorrelated” and “independent” are the same thing.

19.8 Marginal and conditional distributions in d dimensions

Theorem (marginals). Partition $\mathbf{X} = (\mathbf{X}_{(1)}, \mathbf{X}_{(2)})$ with $\mu = (\mu_1, \mu_2)$ and $\Sigma = \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}$. Then

$\boxed{\mathbf{X}_{(1)} \sim \mathcal{N}(\mu_1, \Sigma_{11})}$ — just keep the matching rows of μ and the matching block of Σ .

Why. $\mathbf{X}_{(1)} = \mathbf{B}\mathbf{X}$ with $\mathbf{B} = [\mathbf{I} \ 0]$; apply §19.7. (So: any single coordinate $X_i \sim N(\mu_i, \Sigma_{ii})$.)

Theorem (conditionals). With the same partition (Σ_{11} invertible),

$$\boxed{\mathbf{X}_{(2)} \mid \mathbf{X}_{(1)} = \mathbf{x}_1 \sim \mathcal{N}(\mu_2 + \Sigma_{21}\Sigma_{11}^{-1}(\mathbf{x}_1 - \mu_1), \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12})}.$$

- i) This is the d -dimensional version of §19.5’s $N(\rho x_1, 1 - \rho^2)$: the conditional mean is *linear* in the observation, and the conditional covariance does *not* depend on $\mathbf{x}_1$ — observing data shifts the bell but never changes its shape.
- ii) The matrix $\Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12}$ is the **Schur complement** — the leftover uncertainty in block 2 after block 1 explains what it can. It is PSD (it is itself a covariance), and $\leq \Sigma_{22}$ in PSD order: conditioning never *adds* uncertainty.
- iii) Not proved here (the deck doesn’t prove it either): it follows from completing the square in the Mahalanobis exponent after the change of variables $\mathbf{y}_2 = \mathbf{x}_2 - \Sigma_{21}\Sigma_{11}^{-1}\mathbf{x}_1$, which decorrelates the blocks. The §19.5 bivariate case *is* the proof for $d = 2$ — check: $\Sigma_{21}\Sigma_{11}^{-1} = \rho/1 = \rho$, mean ρx_1 , Schur complement $1 - \rho \cdot 1 \cdot \rho = 1 - \rho^2$.

eg 7 (conditioning in 3-D). $\mathbf{X} \sim \mathcal{N}_3(\mathbf{0}, \Sigma)$, $\Sigma = \begin{pmatrix} 4 & 2 & 1 \\ 2 & 3 & 0.5 \\ 1 & 0.5 & 2 \end{pmatrix}$ (PD: leading minors $4 > 0$, $12 - 4 = 8 > 0$,

$\det = 14 > 0$). Find the distribution of X_3 given $(X_1, X_2) = (x_1, x_2)$. - Blocks: $\Sigma_{11} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$,

$\Sigma_{11}^{-1} = \frac{1}{8} \begin{pmatrix} 3 & -2 \\ -2 & 4 \end{pmatrix}$, $\Sigma_{21} = (1, 0.5)$. - $\Sigma_{21}\Sigma_{11}^{-1} = (1, 0.5) \begin{pmatrix} 0.375 & -0.25 \\ -0.25 & 0.5 \end{pmatrix} = (0.375 - 0.125, -0.25 + 0.25) = (0.25, 0)$. - Conditional mean: $0 + 0.25x_1 + 0 \cdot x_2 = 0.25x_1$. Conditional variance: $2 - (1, 0.5) \cdot (0.25, 0)^T = 2 - 0.25 = 1.75$.

$$\boxed{X_3 \mid (X_1, X_2) = (x_1, x_2) \sim N(0.25x_1, 1.75)}.$$

Note. The 0 coefficient on x_2 says: once X_1 is known, X_2 tells you *nothing more* about X_3 — the partial correlation of X_2, X_3 given X_1 is zero. Marginal: $(X_1, X_2) \sim \mathcal{N}_2(\mathbf{0}, \Sigma_{11})$ by the marginal theorem.

Basically, ... Marginals: cross out the rows you don’t want. Conditionals: the mean becomes a *linear function* of what you observed ($\mu_2 + (\text{regression coefficients}) \times (\text{observation} - \text{its mean})$), and the leftover covariance is the Schur complement — always smaller than before, never depending on the observed values. The bivariate formulas of §19.5 are exactly this with 1×1 blocks.

19.9 Why this matters: the normal workhorse

- i) **Everything linear stays normal.** §19.7's affine closure is why the normal is the default noise model in ML: least squares (§5), linear regression (§23), and Gaussian noise assumptions all run on “normal in, normal out”. The deck uses it immediately for linear regression with Gaussian noise (Week 12, part 2 — Chapter 20's territory).
- ii) **Estimation targets these parameters.** Chapter 20's MLE estimates exactly μ and Σ from data — this chapter defined what is being estimated.
- iii) **Mixtures build on it.** Chapter 25's Gaussian mixture models are weighted sums of the densities defined here (§19.6).
- iv) **The CLT explains its ubiquity.** Chapter 21 shows sums of many small independent effects converge to a normal — the reason the bell curve in §16.9 deserved a whole vector-valued upgrade.
- v) **What comes next.** Chapter 20 estimates μ, Σ (MLE/MAP); Chapter 21 explains *why* normals appear (CLT); Chapter 24 reads Σ 's eigenvectors as principal components.

Problem set

- (X, Y) has joint PMF $P(0, 0) = 0.1$, $P(0, 1) = 0.2$, $P(1, 0) = 0.3$, $P(1, 1) = 0.4$ (first index X). Find the mean vector μ and the covariance matrix Σ . Verify symmetry and positive semi-definiteness, and compute the correlation coefficient.
- Prove the short form: $\Sigma = E[\mathbf{X}\mathbf{X}^T] - \mu\mu^T$, where $\mu = E[\mathbf{X}]$.
- $\mathbf{X}$ has $\mu = (1, 2)^T$, $\Sigma = \begin{pmatrix} 4 & 1 \\ 1 & 2 \end{pmatrix}$. For $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{b}$ with $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, $\mathbf{b} = (0, 1)^T$, find $E[\mathbf{Y}]$ and $\text{Cov}(\mathbf{Y})$.
- Write the full bivariate normal PDF for $\mu = (1, -1)^T$, $\sigma_1 = 2$, $\sigma_2 = 1$, $\rho = 0.6$ (give the normalizer numerically and the exponent's quadratic form explicitly).
- (X_1, X_2) is bivariate normal with $\mu_1 = \mu_2 = 0$, $\sigma_1 = \sigma_2 = 1$, $\rho = 0.5$. (a) Give the distribution of $X_2 \mid X_1 = 2$. (b) Compute $P(X_2 > 1.5 \mid X_1 = 2)$.
- $\mathbf{Y} \sim \mathcal{N}_3(\mu, \Sigma)$ with $\mu = (2, -1, 5)^T$, $\Sigma = \begin{pmatrix} 9 & 2 & -1 \\ 2 & 4 & 1 \\ -1 & 1 & 6 \end{pmatrix}$. Find the distribution of $Z = Y_1 + 2Y_2 - Y_3$.
- $\mathbf{X} \sim \mathcal{N}_2((1, 2)^T, \begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix})$. Find $\Sigma^{-1/2}$ via the eigendecomposition and write the whitened vector $\mathbf{W} = \Sigma^{-1/2}(\mathbf{X} - \mu)$; verify $\text{Cov}(\mathbf{W}) = \mathbf{I}$.
- $\mathbf{X} \sim \mathcal{N}_3(\mathbf{0}, \Sigma)$, $\Sigma = \begin{pmatrix} 4 & 0 & 1.5 \\ 0 & 9 & -2 \\ 1.5 & -2 & 16 \end{pmatrix}$. (a) Which pairs of coordinates are independent? (b) Compute the correlation coefficients ρ_{13} and ρ_{23} .
- $\mathbf{X} \sim \mathcal{N}_3(\mu, \Sigma)$ with $\mu = (1, 0, 2)^T$, $\Sigma = \begin{pmatrix} 2 & 1 & 0.5 \\ 1 & 2 & 0.5 \\ 0.5 & 0.5 & 1 \end{pmatrix}$. Find the distribution of X_3 given $X_1 = 3$, $X_2 = -1$.
- $\mathbf{X} \sim \mathcal{N}_2((1, 2)^T, \begin{pmatrix} 4 & 1 \\ 1 & 1 \end{pmatrix})$. Compute the squared Mahalanobis distance of $\mathbf{x} = (3, 3)^T$ from μ , and hence the density value $f_{\mathbf{X}}(3, 3)$.

Solutions — Chapter 19. Random vectors and the multivariate normal

Problem 1

Marginals of X : $P(X = 0) = 0.1 + 0.2 = 0.3$, $P(X = 1) = 0.3 + 0.4 = 0.7$; of Y : $P(Y = 0) = 0.1 + 0.3 = 0.4$, $P(Y = 1) = 0.2 + 0.4 = 0.6$.

- $\mu = (E[X], E[Y])^T = (0.7, 0.6)^T$.
- $E[X^2] = 0^2(0.3) + 1^2(0.7) = 0.7$, so $\text{Var}(X) = 0.7 - 0.49 = 0.21$. $E[Y^2] = 0.6$, $\text{Var}(Y) = 0.6 - 0.36 = 0.24$.
- $E[XY] = \sum xy P(x, y) = 1 \cdot 1 \cdot 0.4 = 0.4$ (all other terms have $x = 0$ or $y = 0$). $\text{Cov}(X, Y) = 0.4 - 0.7 \cdot 0.6 = -0.02$.

$$\Sigma = \begin{pmatrix} 0.21 & -0.02 \\ -0.02 & 0.24 \end{pmatrix}.$$

Symmetry: off-diagonals equal . PSD: diagonal entries > 0 and $\det = 0.21 \cdot 0.24 - (-0.02)^2 = 0.0504 - 0.0004 = 0.05 > 0$, so by §7.9's 2×2 test the matrix is positive definite (hence PSD) . Correlation: $\rho = -0.02/\sqrt{0.21 \cdot 0.24} = -0.02/\sqrt{0.0504} \approx -0.02/0.2245 \approx \boxed{-0.089}$.

Problem 2

$$\begin{aligned} \Sigma &= E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)^T] \\ &= E[\mathbf{X}\mathbf{X}^T - \mathbf{X}\mu^T - \mu\mathbf{X}^T + \mu\mu^T] \\ &= E[\mathbf{X}\mathbf{X}^T] - E[\mathbf{X}]\mu^T - \mu E[\mathbf{X}]^T + \mu\mu^T \quad (\text{linearity, } \mu \text{ constant}) \\ &= E[\mathbf{X}\mathbf{X}^T] - \mu\mu^T - \mu\mu^T + \mu\mu^T \\ &= \boxed{E[\mathbf{X}\mathbf{X}^T] - \mu\mu^T}. \end{aligned}$$

This is the matrix version of $\text{Var}(X) = E[X^2] - (E[X])^2$ (§16.6).

Problem 3

$$E[\mathbf{Y}] = \mathbf{A}\mu + \mathbf{b} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ -1 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \boxed{\begin{pmatrix} 3 \\ 0 \end{pmatrix}}.$$

$\text{Cov}(\mathbf{Y}) = \mathbf{A}\Sigma\mathbf{A}^T$. First $\Sigma\mathbf{A}^T = \begin{pmatrix} 4 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} 5 & 3 \\ 3 & -1 \end{pmatrix}$; then

$$\mathbf{A}(\Sigma\mathbf{A}^T) = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 5 & 3 \\ 3 & -1 \end{pmatrix} = \boxed{\begin{pmatrix} 8 & 2 \\ 2 & 4 \end{pmatrix}}.$$

Checks: $\text{Var}(Y_1) = \text{Var}(X_1 + X_2) = 4 + 2 + 2(1) = 8$; symmetric ; $\det = 32 - 4 = 28 > 0$, PSD . Note $Y_1 = X_1 + X_2 + 0$ and $Y_2 = X_1 - X_2 + 1$ — the shift 1 appears only in the mean.

Problem 4

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix} = \begin{pmatrix} 4 & 1.2 \\ 1.2 & 1 \end{pmatrix}.$$

- $\det \Sigma = 4 \cdot 1 - 1.44 = 2.56$; $\sqrt{\det \Sigma} = 1.6$. Normalizer: $\boxed{(2\pi) \cdot 1.6 = 3.2\pi \approx 10.053}$.
- $\Sigma^{-1} = \frac{1}{2.56} \begin{pmatrix} 1 & -1.2 \\ -1.2 & 4 \end{pmatrix} = \begin{pmatrix} 0.390625 & -0.46875 \\ -0.46875 & 1.5625 \end{pmatrix}$.

$$\boxed{f(x_1, x_2) = \frac{1}{3.2\pi} \exp\left(-\frac{1}{2} \left[0.390625 (x_1 - 1)^2 - 0.9375 (x_1 - 1)(x_2 + 1) + 1.5625 (x_2 + 1)^2 \right] \right)}$$

(the cross term: $2 \cdot (-0.46875)(x_1 - 1)(x_2 + 1) = -0.9375 (x_1 - 1)(x_2 + 1)$).

Problem 5

(a) By §19.5: $X_2 \mid X_1 = x_1 \sim N(\rho x_1, 1 - \rho^2)$. With $\rho = 0.5$, $x_1 = 2$:

$$\boxed{X_2 \mid X_1 = 2 \sim N(1, 0.75)}.$$

(b) Standardize: $P(X_2 > 1.5 \mid X_1 = 2) = P\left(Z > \frac{1.5-1}{\sqrt{0.75}}\right) = P(Z > 0.5774) = 1 - \Phi(0.5774) \approx 0.71815$, so the answer is $\boxed{\approx 0.282}$.

Compare with the marginal: $P(X_2 > 1.5) = 1 - \Phi(1.5) \approx 0.0668$ — conditioning on the large X_1 more than quadruples the probability.

Problem 6

$Z = \mathbf{c}^T \mathbf{Y}$ with $\mathbf{c} = (1, 2, -1)^T$. By §19.7(i), $Z \sim N(\mathbf{c}^T \mu, \mathbf{c}^T \Sigma \mathbf{c})$.

- Mean: $1(2) + 2(-1) + (-1)(5) = 2 - 2 - 5 = \boxed{-5}$.
- $\Sigma \mathbf{c} = \begin{pmatrix} 9 & 2 & -1 \\ 2 & 4 & 1 \\ -1 & 1 & 6 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ -1 \end{pmatrix} = \begin{pmatrix} 9+4+1 \\ 2+8-1 \\ -1+2-6 \end{pmatrix} = \begin{pmatrix} 14 \\ 9 \\ -5 \end{pmatrix}$; variance = $\mathbf{c}^T (\Sigma \mathbf{c}) = 1(14) + 2(9) + (-1)(-5) = 14 + 18 + 5 = \boxed{37}$.

$$\boxed{Z \sim N(-5, 37)}.$$

(Sanity: Σ is PD — leading minors $9 > 0$, $36 - 4 = 32 > 0$, $\det = 175 > 0$ — so it is a valid covariance.)

Problem 7

Eigendecomposition of $\Sigma = \begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}$: characteristic equation $(5 - \lambda)^2 - 9 = 0$ gives $\boxed{\lambda_1 = 8, \lambda_2 = 2}$. Unit eigenvectors: $\mathbf{q}_1 = (1, 1)^T / \sqrt{2}$ (for $\lambda = 8$: $(5 - 8)x + 3y = 0 \Rightarrow y = x$), $\mathbf{q}_2 = (1, -1)^T / \sqrt{2}$.

$$\Sigma^{-1/2} = \frac{1}{\sqrt{8}} \mathbf{q}_1 \mathbf{q}_1^T + \frac{1}{\sqrt{2}} \mathbf{q}_2 \mathbf{q}_2^T = \frac{1}{2\sqrt{2}} \cdot \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} + \frac{1}{\sqrt{2}} \cdot \frac{1}{2} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

Combining over the common denominator $4\sqrt{2}$:

$$\boxed{\Sigma^{-1/2} = \frac{1}{4\sqrt{2}} \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix} \approx \begin{pmatrix} 0.5303 & -0.1768 \\ -0.1768 & 0.5303 \end{pmatrix}}.$$

Verification: $\text{Cov}(\mathbf{W}) = \Sigma^{-1/2} \Sigma \Sigma^{-1/2}$. Since $\Sigma^{-1/2}$ is symmetric and $\Sigma^{-1/2} \Sigma \Sigma^{-1/2} = \Sigma^{-1/2} (\Sigma^{1/2} \Sigma^{1/2}) \Sigma^{-1/2} = \mathbf{I}$ (both built from the same $\mathbf{Q}$), $\boxed{\text{Cov}(\mathbf{W}) = \mathbf{I}}$ — and by §19.7, $\mathbf{W} \sim \mathcal{N}_2(\mathbf{0}, \mathbf{I})$. Numerically, $\Sigma^{-1/2} \Sigma \Sigma^{-1/2} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ to machine precision.

Problem 8

(a) For a normal vector, $X_i \perp X_j \iff \Sigma_{ij} = 0$ (§19.7(iii)). The only zero off-diagonal is $\Sigma_{12} = \Sigma_{21} = 0$, so $\boxed{X_1 \perp X_2}$; the other two pairs are dependent (nonzero covariance).

(b) $\rho_{13} = \Sigma_{13} / \sqrt{\Sigma_{11} \Sigma_{33}} = 1.5 / \sqrt{4 \cdot 16} = 1.5/8 = \boxed{0.1875}$. $\rho_{23} = \Sigma_{23} / \sqrt{\Sigma_{22} \Sigma_{33}} = -2 / \sqrt{9 \cdot 16} = -2/12 = \boxed{-1/6 \approx -0.1667}$.

(Validity: $\det \Sigma = 539.75 > 0$ with positive leading minors, so Σ is PD.)

Problem 9

Partition with $\mathbf{X}_{(1)} = (X_1, X_2)$, $\mathbf{X}_{(2)} = X_3$: $\mu_1 = (1, 0)^T$, $\mu_2 = 2$, $\Sigma_{11} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, $\Sigma_{21} = (0.5, 0.5)$, $\Sigma_{22} = 1$.

- $\Sigma_{11}^{-1} = \frac{1}{3} \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$.
- Regression coefficients: $\Sigma_{21}\Sigma_{11}^{-1} = (0.5, 0.5) \cdot \frac{1}{3} \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} = \frac{1}{3}(0.5, 0.5) = (1/6, 1/6)$.
- Conditional mean: $2 + \frac{1}{6}(3-1) + \frac{1}{6}(-1-0) = 2 + \frac{1}{3} - \frac{1}{6} = 2 + \frac{1}{6} = \boxed{13/6 \approx 2.1667}$.
- Conditional variance (Schur complement): $1 - (0.5, 0.5) \cdot (1/6, 1/6)^T = 1 - \frac{1}{6} = \boxed{5/6 \approx 0.8333}$.

$$\boxed{X_3 \mid X_1 = 3, X_2 = -1 \sim N(13/6, 5/6)}.$$

Problem 10

$\mathbf{d} = \mathbf{x} - \mu = (3-1, 3-2)^T = (2, 1)^T$. $\det \Sigma = 4 \cdot 1 - 1 = 3$, so $\Sigma^{-1} = \frac{1}{3} \begin{pmatrix} 1 & -1 \\ -1 & 4 \end{pmatrix}$.

Squared Mahalanobis distance:

$$d_{\Sigma}^2 = \mathbf{d}^T \Sigma^{-1} \mathbf{d} = \frac{1}{3}(2, 1) \begin{pmatrix} 1 & -1 \\ -1 & 4 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \frac{1}{3}(2, 1) \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \frac{1}{3}(2+2) = \boxed{\frac{4}{3}}.$$

Density: $f_{\mathbf{X}}(3, 3) = \frac{1}{2\pi\sqrt{3}} \exp(-d_{\Sigma}^2/2) = \frac{1}{2\pi\sqrt{3}} e^{-2/3} \approx \frac{0.51342}{10.88280} \approx \boxed{0.0472}$.

(Reading: Euclidean distance from μ is $\sqrt{5} \approx 2.236$, but the Mahalanobis distance is $\sqrt{4/3} \approx 1.155$ — the point lies mostly along the high-variance direction, so the density judges it “closer” than raw distance suggests.)

20. Estimation: MLE and Bayesian/MAP

Chapter 19 defined what is being estimated — the mean vector μ and the covariance matrix Σ of a distribution. This chapter answers the next question: given data, how do you actually *find* those numbers? Two philosophies, one from the MLF deck and one from the MLT slides: maximum likelihood (treat the parameter as a fixed unknown and pick the one that makes the data most likely) and Bayesian estimation (treat the parameter as a random variable, state a prior belief, and update it with data). The Bayesian answer’s “best single number” is the MAP estimate — the mode of the posterior.

20.1 The estimation setup

Def. A **family of distributions** is $\mathcal{P} = \{P_\theta : \theta \in \Theta\}$ — a *set* of distributions, one for each value of the **parameter** θ , where Θ is the **parameter space** (the set of allowed values). You observe data $X_1, \dots, X_n$ drawn i.i.d. from P_θ for *some* $\theta \in \Theta$, but you do not know which. The **estimation problem** = use the data to find the best θ (the MLF transcript’s words: “use this data and find out what is the best θ ”).

Def (estimator). An **estimator** is a rule or method applied to the sample that produces a guess for the parameter (the MLT notes’ definition). The guess itself — a number computed from the data — is the **estimate**. Estimators get a hat: $\hat{\theta}$.

Def (i.i.d.). The data are **independent and identically distributed**: i) **Independent**: $P(x_i | x_j) = P(x_i)$ — knowing one observation tells you nothing about another. ii) **Identically distributed**: every X_i follows the *same* distribution, $P(x_i) = P(x_j)$ for all i, j .

Note. Everything in this chapter stands on the i.i.d. assumption — it is what turns the likelihood into a product (§20.2) and the log-likelihood into a sum. When it fails (time series, correlated samples), the recipes below need reworking.

eg 1 (the setup, concretely). You have a coin of unknown bias. Family: $\mathcal{P} = \{\text{Bernoulli}(p) : p \in [0, 1]\}$, so $\theta = p$, $\Theta = [0, 1]$. You toss it $n = 4$ times and see $\{1, 0, 1, 1\}$. The problem: which p best explains this? (The deck’s own framing — slide: “there is some probabilistic mechanism that generates data, about which we don’t know ‘something’; given data, find/estimate what we don’t know”.)

Basically, ... Estimation = “some distribution from a known menu generated my data; the menu item is unknown; use the data to pick the menu item”. The parameter θ is the menu item, Θ is the menu, and the data is your evidence. MLE picks the menu item that makes your evidence look the most plausible.

20.2 Likelihood, log-likelihood, negative log-likelihood

Fix the data $x_1, \dots, x_n$. Now ask, for each candidate parameter θ : how likely was *this* data under P_θ ?

Def (likelihood). The **likelihood** of a parameter θ is

$$L(\theta) = P(X_1 = x_1, \dots, X_n = x_n \mid \theta),$$

i.e. “the probability of this data set being generated, if θ were the true parameter” (the transcript’s phrasing). It is a function of θ — the data are fixed, θ varies.

i) **i.i.d. product.** By independence,

$$L(\theta) = \prod_{i=1}^n f(x_i \mid \theta),$$

where f is the PMF (discrete) or PDF (continuous) of P_θ . The transcript uses one notation for both; this chapter does the same.

ii) **Likelihood is not a distribution over θ .** It is *not* required to integrate (or sum) to 1 over θ — it is “how well each θ explains the data”, not “the probability of θ ”.

Def (log-likelihood). Since log is monotone increasing, maximizing $L(\theta)$ is the same as maximizing

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^n \log f(x_i \mid \theta),$$

the **log-likelihood**. The log turns the ugly product into a sum — the transcript’s “you can immediately see why we did this, because log nicely decomposes products”.

Def (negative log-likelihood / risk). Equivalently, minimize

$$R(\theta) = -\log L(\theta) = -\sum_{i=1}^n \log f(x_i \mid \theta),$$

which the transcript calls the **risk function** $R(\theta)$. Maximizing the log-likelihood = minimizing the negative log-likelihood: “it is just an issue of which you are comfortable with”.

Note. log of a product is a sum of logs — the one algebraic move this whole chapter runs on. Every derivation below is: write the likelihood, take the log, differentiate, set to zero, solve.

Basically, ... Likelihood = “score each possible parameter by how probable it makes the observed data”. Log-likelihood = “the same score, but in log form so the product becomes a sum and calculus works”. Negative log-likelihood = “flip the sign so it’s a minimization problem, like every loss function you’ve met”. All three give the same answer.

20.3 Maximum likelihood estimation

Def (MLE). The **maximum likelihood estimate** is

$$\hat{\theta}_{ML} = \arg \max_{\theta \in \Theta} L(\theta) = \arg \max_{\theta \in \Theta} \ell(\theta) = \arg \min_{\theta \in \Theta} R(\theta).$$

(The deck attributes the principle to Fisher: “Fisher’s principle of maximum likelihood”).

- i) You never learn the *true* θ — “the moment you enter the realm of probability you can never be 100% certain” (transcript). MLE gives the parameter that *most explains* the data, not a guarantee.
- ii) In practice: write $\ell(\theta)$ (or $R(\theta)$), take the derivative, set to 0, solve — the first-order necessary condition from §10.8. Check it is actually a max (min of R); in the examples below there is a unique critical point and it is the answer.

The next four sections are the transcript’s four worked examples, done end to end.

20.4 Worked: Bernoulli bias

Family: $P_\theta = \text{Bernoulli}(\theta)$, $\theta \in [0, 1]$. Data: $x_1, \dots, x_n \in \{0, 1\}$.

Step 1 — the model in compact form. $P_\theta(x) = \theta$ if $x = 1$, $1 - \theta$ if $x = 0$. Since x is only 0 or 1,

$$P_\theta(x) = \theta^x (1 - \theta)^{1-x}$$

(check: $x = 1$ gives $\theta^1(1 - \theta)^0 = \theta$; $x = 0$ gives $\theta^0(1 - \theta)^1 = 1 - \theta$).

Step 2 — negative log-likelihood. Let $a = \sum_{i=1}^n x_i$ = number of heads. Then

$$\begin{aligned} R(\theta) &= - \sum_{i=1}^n \log(\theta^{x_i} (1 - \theta)^{1-x_i}) \\ &= - \sum_{i=1}^n [x_i \log \theta + (1 - x_i) \log(1 - \theta)] \\ &= a \log \frac{1}{\theta} + (n - a) \log \frac{1}{1 - \theta}. \end{aligned}$$

Step 3 — minimize. $\frac{dR}{d\theta} = -\frac{a}{\theta} + \frac{n-a}{1-\theta} = 0$ gives $\frac{a}{\theta} = \frac{n-a}{1-\theta}$, i.e. $a(1 - \theta) = (n - a)\theta$, so $a = n\theta$:

$$\hat{\theta}_{ML} = \frac{a}{n} = \frac{1}{n} \sum_{i=1}^n x_i.$$

eg 2 (the deck's numeric slide). Data $\{1, 0, 1, 1\}$: $n = 4$, $a = 3$, so $\hat{p}_{ML} = 3/4$. The figure's left panel plots $L(p) = p^3(1 - p)$ — the peak sits exactly at $p = 0.75$ (check: $L(0.75) = 0.75^3 \cdot 0.25 \approx 0.1055$, while $L(0.5) = 0.0625$ and $L(0.9) = 0.0729$ — the peak really is at $3/4$).

Note (zero-count warning). If $a = 0$, $\hat{\theta}_{ML} = 0$ — the model now says heads are *impossible*, even though you only tossed the coin a few times. This overconfidence is exactly what a prior smooths away in §20.11.

Basically, ... For a coin, MLE says “the bias is the fraction of heads you saw”. You knew this before the math — the derivation just shows the maximum-likelihood principle *agrees* with common sense here, which is why the principle is trusted on harder problems.

20.5 Worked: Uniform $[a, b]$

Family: $P_\theta = \text{Uniform}(a, b)$; the parameter is the *pair* $\theta = (a, b)$, $a < b$.

Step 1 — the model. $P_\theta(x) = \frac{1}{b-a} \mathbf{1}_{\{x \in [a, b]\}}$ (indicator: 1 inside $[a, b]$, 0 outside).

Step 2 — negative log-likelihood. For each observation,

$$-\log P_\theta(x_i) = \log(b - a) - \log \mathbf{1}_{\{x_i \in [a, b]\}}.$$

If *any* x_i falls outside $[a, b]$, its indicator is 0 and $-\log 0 = +\infty$. So:

$$R(a, b) = \begin{cases} n \log(b - a), & a \leq \min_i x_i \text{ and } b \geq \max_i x_i, \\ +\infty, & \text{otherwise.} \end{cases}$$

Step 3 — minimize. Avoiding $+\infty$ forces $a \leq \min x_i$, $b \geq \max x_i$; then minimize $n \log(b - a)$, i.e. minimize the width $b - a$. The smallest allowed interval hugs the data:

$$\hat{a}_{ML} = \min_i x_i, \quad \hat{b}_{ML} = \max_i x_i.$$

eg 3. Data $\{2.1, 3.4, 1.8, 4.0\}$: $\boxed{\hat{a}_{ML} = 1.8, \hat{b}_{ML} = 4.0}$. Any wider interval spreads the same probability mass $\frac{1}{b-a}$ thinner at each data point, so the likelihood drops — the tightest box wins.

Note. Not every MLE looks like an average: here it is an order statistic (min/max). The shape of the family decides the shape of the estimator.

Basically, ... For “uniform between a and b ”, MLE picks the tightest box that still contains all the data. Widen the box and each point gets less density — so shrink-wrap the observations.

20.6 Worked: Normal — mean, then mean and variance

Family: $P_\theta = N(\mu, \sigma^2)$.

Case A — variance known ($\sigma^2 = 1$), **estimate** μ . $P_\theta(x) = \frac{1}{\sqrt{2\pi}} \exp(-\frac{1}{2}(x - \mu)^2)$. Dropping the μ -free constant $-\log(1/\sqrt{2\pi})$,

$$R(\mu) = \frac{1}{2} \sum_{i=1}^n (x_i - \mu)^2 + \text{const}.$$

$\frac{dR}{d\mu} = -\sum_{i=1}^n (x_i - \mu) = 0$ gives

$$\hat{\mu}_{ML} = \frac{1}{n} \sum_{i=1}^n x_i = \bar{x}$$

— the sample mean (the tutorial’s Q5).

Case B — estimate μ and σ^2 jointly (the transcript’s final example).

$$P_\theta(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right),$$

and, writing $n \log \sigma = \frac{n}{2} \log \sigma^2$,

$$R(\mu, \sigma^2) = \frac{n}{2} \log \sigma^2 + \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2 + \text{const}.$$

i) $\frac{\partial R}{\partial \mu} = -\frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) = 0$ $\boxed{\hat{\mu}_{ML} = \bar{x}}$. ii) $\frac{\partial R}{\partial \sigma^2} = \frac{n}{2\sigma^2} - \frac{1}{2\sigma^4} \sum_{i=1}^n (x_i - \mu)^2 = 0$ multiply by $2\sigma^4$:
 $n\sigma^2 = \sum_{i=1}^n (x_i - \mu)^2$

$$\hat{\sigma}_{ML}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu}_{ML})^2.$$

Note. The MLE variance divides by n , not $n - 1$ — it is the *average* squared deviation, the “sampled variance computed using the empirical mean” (transcript). (The $n - 1$ version is a different, unbiased, estimator — not what MLE gives.)

eg 4 (numeric, full steps). Data $\{4, 5, 7, 8\}$, $n = 4$. - $\hat{\mu}_{ML} = (4 + 5 + 7 + 8)/4 = 24/4 = \boxed{6}$. - Squared deviations from 6: $(4 - 6)^2 = 4$, $(5 - 6)^2 = 1$, $(7 - 6)^2 = 1$, $(8 - 6)^2 = 4$; sum = 10. - $\hat{\sigma}_{ML}^2 = 10/4 = \boxed{2.5}$. - Sanity check (it really is the min): $R(\mu) = \frac{1}{2\sigma^2} \sum (x_i - \mu)^2 + \frac{n}{2} \log \sigma^2$ with $\sigma^2 = 2.5$: at $\mu = 6$, $R = \frac{10}{5} + 2 \log 2.5 = 2 + 1.8326 = 3.8326$; at $\mu = 5$, $\sum = 1 + 0 + 4 + 9 = 14$, $R = \frac{14}{5} + 1.8326 = 4.6326 > 3.8326$.

Basically, ... For a normal, MLE rediscovers the two numbers you’d compute anyway: the mean of the data, and the average squared distance from that mean. The derivation’s real lesson: the $\frac{n}{2} \log \sigma^2$ term stops σ^2 from blowing up to infinity (a wider bell fits each point worse) — the likelihood balances “center me here” against “don’t spread me too thin”.

20.7 Estimating μ and Σ of a multivariate normal

This is §19.9’s promise: Chapter 19 defined μ, Σ ; now estimate them. The deck states the extension to the multivariate normal $P_\theta = \mathcal{N}_d(\mu, \Sigma)$ without working the matrix calculus (gradients with respect to matrices go beyond §9’s vector-gradient toolkit):

$$\hat{\mu}_{ML} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \bar{\mathbf{x}}, \quad \hat{\Sigma}_{ML} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \hat{\mu}_{ML})(\mathbf{x}_i - \hat{\mu}_{ML})^T.$$

- i) The mean estimate is the coordinate-wise sample mean; the covariance estimate is the average of the outer products of the centered observations — the empirical version of §19.1’s $\Sigma = E[(\mathbf{X} - \mu)(\mathbf{X} - \mu)^T]$, with the expectation replaced by the average over the data.
- ii) Same n -not- $(n-1)$ denominator as §20.6.
- iii) $\hat{\Sigma}_{ML}$ is symmetric and PSD by construction (a sum of outer products — §19.1’s note).

eg 5 (2-D, full steps). Observations $\mathbf{x}_1 = (0, 0)^T$, $\mathbf{x}_2 = (1, 2)^T$, $\mathbf{x}_3 = (2, 1)^T$. - $\hat{\mu}_{ML} = \left(\frac{0+1+2}{3}, \frac{0+2+1}{3}\right)^T = (1, 1)^T$. - Centered: $(-1, -1)^T$, $(0, 1)^T$, $(1, 0)^T$. Outer products: $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, $\begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$, $\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$; sum = $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. - $\hat{\Sigma}_{ML} = \frac{1}{3} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 2/3 & 1/3 \\ 1/3 & 2/3 \end{pmatrix}$. - Sanity: symmetric ; PSD check via §7.9’s 2×2 test: diagonal > 0 , $\det = \frac{4}{9} - \frac{1}{9} = \frac{1}{3} > 0$.

Basically, ... Same story as the 1-D normal, one dimension higher: the ML mean is the average of the points, the ML covariance is the average of “how far each point is from the mean, times itself” (the outer product). That one formula *is* §19.1’s covariance matrix with “expectation” swapped for “average over data”.

20.8 ML for linear regression with Gaussian noise

The deck closes its estimation block with the first genuinely ML example. Model: $y = \mathbf{w}^T \mathbf{x} + \varepsilon$, $\varepsilon \sim N(0, \sigma^2)$ — i.e. $P(y | \mathbf{x}) = N(\mathbf{w}^T \mathbf{x}, \sigma^2)$, with $\mathbf{w}$ the unknown parameter. Data: $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$.

The log-likelihood (dropping the $\mathbf{w}$ -free constant):

$$\ell(\mathbf{w}) = -\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \text{const.}$$

Maximizing it = minimizing the sum of squared errors:

$$\hat{\mathbf{w}}_{ML} = \arg \min_{\mathbf{w}} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2.$$

Note. “Maximum likelihood under Gaussian noise IS least squares” (§5). The noise model *chooses* the loss function: normal noise squared error. Chapter 23 builds the full regression story on this.

Basically, ... Assume your errors are bell-curved, and the most-likely weights are exactly the least-squares weights. This is the bridge from probability (Part II) to machine learning (Part III): fitting a model = estimating parameters = minimizing a loss.

20.9 The Bayesian setup: prior, likelihood, posterior

MLE treats θ as a fixed unknown. The Bayesian approach treats the parameter *as a random variable* and updates a belief about it with data (the MLT deck: “think of the parameter to estimate as a ‘random’ variable”).

Def. Three ingredients (the MLT TA notes): i) **Prior** $f(\theta)$ (or $P(\theta)$): your belief about θ *before* seeing data — “hunch codified using a probability distribution over θ ” (deck slide). ii) **Likelihood** $P(D | \theta)$: how likely the observed data D is, given θ — the same $L(\theta)$ from §20.2. iii) **Posterior** $P(\theta | D)$: your *updated* belief about θ after seeing the data — “updated hunch codified using a probability distribution”.

Bayes’ theorem (§14.11) links them:

$$P(\theta | D) = \frac{P(D | \theta) f(\theta)}{P(D)}.$$

Note. $P(D)$, the **marginal likelihood** (or evidence), does not depend on θ — it just normalizes the posterior to integrate to 1. For comparing θ values you can drop it:

$$\text{posterior} \propto \text{likelihood} \times \text{prior}.$$

Basically, ... Bayesian estimation = “start with a hunch (prior), look at data (likelihood), get an updated hunch (posterior)”. The posterior blends both: strong prior + little data → prior dominates; lots of data → the data washes the prior out. The whole update rule is one line: posterior is proportional to likelihood times prior.

20.10 Point estimates: posterior mean and MAP

Bayesian methods return a *distribution* over θ , not a single number. Two standard ways to extract one number (the MLT TA notes):

- i) **Posterior mean** $E[\theta | D]$ — the average of the posterior.
- ii) **Posterior mode** — the peak of the posterior.

Def (MAP). The **maximum a posteriori** estimate is the mode of the posterior:

$$\hat{\theta}_{MAP} = \arg \max_{\theta} P(\theta | D) = \arg \max_{\theta} [\log P(D | \theta) + \log f(\theta)].$$

Note (MAP vs MLE). Compare with §20.3: MAP = MLE’s objective *plus* a $\log f(\theta)$ term from the prior. The prior acts as a penalty pulling the estimate toward a-priori plausible values. And when the prior is *flat* (constant over Θ — e.g. the deck’s Beta(1, 1) uniform prior on $(0, 1)$), the $\log f(\theta)$ term is a constant and $\arg \max \text{posterior} = \arg \max \text{likelihood}$:

$$\text{flat prior} \Rightarrow \hat{\theta}_{MAP} = \hat{\theta}_{ML}$$

— MLE is the special case of MAP with no prior opinion. (Problem 9 proves this for the Bernoulli case.)

Basically, ... If MLE is “trust the data completely”, MAP is “trust the data, but keep one eye on what you believed before”. The prior is a tug toward your hunch; the likelihood is a tug toward the data; the posterior peak is where they balance. No hunch at all (flat prior) → the tug-of-war is one-sided → you get MLE back.

20.11 Worked: Beta prior + Bernoulli data (the conjugate pair)

The MLT deck’s worked Bayesian example. Prior on the coin bias $p \in (0, 1)$:

Def (Beta distribution). For $\alpha, \beta > 0$,

$$f(p; \alpha, \beta) = \frac{1}{B(\alpha, \beta)} p^{\alpha-1} (1-p)^{\beta-1}, \quad 0 < p < 1,$$

where $B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}$ is the normalizer.

i) **Mean:** $E[p] = \frac{\alpha}{\alpha + \beta}$ (read: “ α pseudo-heads, β pseudo-tails”).

ii) **Mode** (the MLT notes’ derivation): $\frac{d}{dp} \log f = \frac{\alpha-1}{p} - \frac{\beta-1}{1-p} = 0$ gives $\text{mode} = \frac{\alpha-1}{\alpha+\beta-2}$ (valid for $\alpha, \beta > 1$; the notes list the boundary cases: mode 0 if $\alpha \leq 1 < \beta$, mode 1 if $\beta \leq 1 < \alpha$, any $p \in (0, 1)$ if $\alpha = \beta = 1$, and $\{0, 1\}$ if both < 1).

iii) The deck’s sketch: Beta(0.5, 0.5) piles mass at the edges (bias near 0 or 1), Beta(2, 5) leans left (mean 2/7), Beta(2, 2) is symmetric about 1/2, and the flat red line is the uniform prior Beta(1, 1).

Step 1 — posterior. Bernoulli likelihood with n_1 heads, n_0 tails ($n_1 + n_0 = n$):

$$f(p | D) \propto \underbrace{p^{n_1}(1-p)^{n_0}}_{\text{likelihood}} \cdot \underbrace{p^{\alpha-1}(1-p)^{\beta-1}}_{\text{prior}} = p^{n_1+\alpha-1}(1-p)^{n_0+\beta-1}.$$

The right side has the *same functional form* as the prior — so the posterior is Beta again:

$$\text{Beta}(\alpha, \beta) \text{ prior} \xrightarrow{\text{Bernoulli data}} \text{Beta}(\alpha + n_1, \beta + n_0) \text{ posterior}$$

(the deck’s slide, verbatim in content).

Def (conjugate prior). A prior is **conjugate** for a likelihood if the posterior comes out in the *same family* as the prior (MLT notes). Beta is conjugate to Bernoulli/Binomial: updating = just adding head/tail counts to α, β .

Step 2 — the estimates. For posterior Beta($\alpha + n_1, \beta + n_0$):

$$\hat{p}_{MAP} = \frac{\alpha + n_1 - 1}{\alpha + \beta + n - 2}, \quad E[p | D] = \frac{\alpha + n_1}{\alpha + \beta + n}.$$

eg 6 (all three estimates, side by side). Prior Beta(2, 2) (weakly favors $p = 1/2$); data: 10 tosses, 7 heads ($n_1 = 7, n_0 = 3$). Posterior = Beta(9, 5). - MLE: $\hat{p}_{ML} = 7/10 = 0.7$. - MAP: $\hat{p}_{MAP} = 8/12 = 2/3 \approx 0.667$. - Posterior mean: $E[p | D] = 9/14 \approx 0.643$. The prior ($\alpha = \beta = 2$, i.e. “two pseudo-heads, two pseudo-tails”) drags the estimate toward 1/2; MAP drags less than the posterior mean here. With 1000 tosses the +2s would be invisible — data swamps the prior. The figure’s right panel draws this exact update.

Note. Revisit §20.4’s zero-count warning: with 0 heads in 4 tosses, MLE says $\hat{p} = 0$, but Beta(2, 2) gives posterior Beta(2, 6) and $\hat{p}_{MAP} = 1/6$ — the prior keeps the estimate sane. This smoothing is the practical reason priors exist.

Basically, ... The Beta distribution is a “belief about a coin’s bias” you can do arithmetic with: α = imaginary heads seen before, β = imaginary tails. Real data just adds to the imaginary counts. Then MAP = “the most believable bias after pooling imaginary and real tosses”, and it’s one clean fraction. When a prior updates this cleanly (same family out as in), it’s called *conjugate* — Beta and Bernoulli are the textbook conjugate pair.

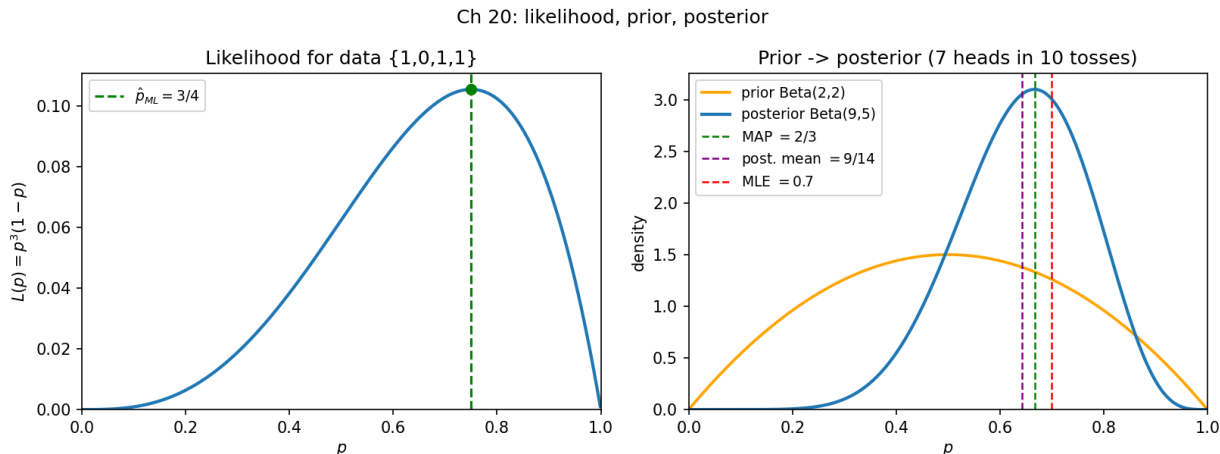


Figure 22: Left: likelihood curve $L(p) = p^3(1-p)$ for coin data $\{1,0,1,1\}$, peaking at $p = 3/4$ (the MLE). Right: Beta(2,2) prior (orange) updated with 7 heads in 10 tosses to a Beta(9,5) posterior (blue), with vertical lines at the MAP $2/3$, the posterior mean $9/14$, and the MLE 0.7 .

20.12 Why this matters: the two workhorses of ML

- i) **MLE is the default fitting rule.** Least squares (§20.8, §5, §23), logistic regression (§31), neural network training (Part VI) — all minimize a negative log-likelihood, i.e. maximize a likelihood. “Even for very complicated models, maximum likelihood forms the first point from which you build more complicated algorithms” (transcript).
- ii) **MAP = regularization.** Chapter 28’s ridge regression turns out to be MAP estimation with a Gaussian prior — the $\log f(\theta)$ penalty of §20.10 becomes the $\lambda \|\mathbf{w}\|^2$ term. Priors are where “keep the model simple” enters the math.
- iii) **The normal parameters are now estimable.** §19.9’s promise is kept: μ and Σ are estimated by sample mean and sample covariance (§20.7) — and Chapter 25’s GMMs estimate *several* such pairs at once.
- iv) **What comes next.** Chapter 21 explains *why* the normal family deserved all this attention (central limit theorem); the Bayesian story continues wherever priors appear (Chapters 28, 30).

Problem set

1. (Deck slide.) Data $\{1, 0, 1, 1\}$ from Bernoulli(p). Write $L(p)$ explicitly, take $\log L(p)$, differentiate, and show $\hat{p}_{ML} = 3/4$. Evaluate $L(0.75)$ and $L(0.5)$ to confirm the peak.
2. (MLT notes Q2.) A 0–1 dataset is modeled as Bernoulli(p). The MLE is $\hat{p}_{ML} = 0.25$ and the number of ones is 10. Find the number of zeros.
3. (MLT notes Q3.) Four points in $[0, 1]$ have density $f(x; \theta) = \theta x^{\theta-1}$ for $0 \leq x \leq 1$ (0 otherwise), $\theta > 0$. (i) Write the log-likelihood $\ell(\theta)$. (ii) For $x_1 = e^{-1}, x_2 = e^{-2}, x_3 = e^{-3}, x_4 = e^{-4}$, find $\hat{\theta}_{ML}$.
4. (Tutorial.) $X_1, \dots, X_n \stackrel{i.i.d.}{\sim} N(\mu, \sigma^2)$ with μ known. Derive $\hat{\sigma}_{ML}^2$ from first principles (write $R(\sigma^2)$, differentiate, solve).
5. Data $\{2.1, 3.4, 1.8, 4.0\}$ from Uniform(a, b). Find $\hat{a}_{ML}, \hat{b}_{ML}$ and explain in one line why $[a, b] = [1.0, 5.0]$ has strictly smaller likelihood.
6. Data $\{4, 5, 7, 8\}$ from $N(\mu, \sigma^2)$. Compute $\hat{\mu}_{ML}$ and $\hat{\sigma}_{ML}^2$ with full steps, then verify $\hat{\mu}_{ML}$ minimizes $R(\mu)$ by comparing $R(6)$ and $R(5)$ (use $\hat{\sigma}_{ML}^2$ in R).
7. $\mathbf{x}_1 = (0, 0)^T$, $\mathbf{x}_2 = (1, 2)^T$, $\mathbf{x}_3 = (2, 1)^T$ from $\mathcal{N}_2(\mu, \Sigma)$. Compute $\hat{\mu}_{ML}$ and $\hat{\Sigma}_{ML}$ with full steps, and verify $\hat{\Sigma}_{ML}$ is symmetric PSD.

8. Prior $\text{Beta}(3, 3)$; 20 tosses, 12 heads. Find (i) the posterior, (ii) $\hat{p}_{MAP}$, (iii) the posterior mean, (iv) $\hat{p}_{ML}$; say in one line how the prior moved the estimate.
9. Prove: with a flat $\text{Beta}(1, 1)$ prior and Bernoulli data (n_1 heads, n_0 tails), $\hat{p}_{MAP} = \hat{p}_{ML}$. (This is §20.10's boxed claim for the Bernoulli case.)
10. Model $y = wx + \varepsilon$, $\varepsilon \sim N(0, \sigma^2)$ (line through the origin), data $\{(1, 2), (2, 4), (3, 5)\}$. Write $R(w)$, derive $\hat{w}_{ML} = \frac{\sum x_i y_i}{\sum x_i^2}$, and compute it numerically.

Solutions — 20. Estimation: MLE and Bayesian/MAP

1. $L(p) = P_{\text{Bern}}(1) P_{\text{Bern}}(0) P_{\text{Bern}}(1) P_{\text{Bern}}(1) = p \cdot (1-p) \cdot p \cdot p = \boxed{p^3(1-p)}$. $\log L(p) = 3 \log p + \log(1-p)$. Differentiate:

$$\frac{d}{dp} \log L = \frac{3}{p} - \frac{1}{1-p} \stackrel{!}{=} 0 \Rightarrow 3(1-p) = p \Rightarrow 3 = 4p \Rightarrow \boxed{\hat{p}_{ML} = 3/4}.$$

Second derivative $-\frac{3}{p^2} - \frac{1}{(1-p)^2} < 0$ everywhere, so this is the (unique) maximizer. Numeric check: $L(0.75) = 0.75^3 \cdot 0.25 = 0.421875 \cdot 0.25 = 0.10546875$; $L(0.5) = 0.5^4 = 0.0625 < 0.10546875$ — the peak is at $3/4$.

2. $\hat{p}_{ML} = \frac{\# \text{ones}}{n} = \frac{10}{10+n_0} = 0.25$. Solve: $10 = 0.25(10 + n_0)$ $10 = 2.5 + 0.25n_0$ $7.5 = 0.25n_0$ $\boxed{n_0 = 30}$ zeros (so $n = 40$ total; check: $10/40 = 0.25$).

3. (i) $L(\theta) = \prod_{i=1}^4 \theta x_i^{\theta-1} = \theta^4 \prod_{i=1}^4 x_i^{\theta-1}$, so

$$\ell(\theta) = 4 \log \theta + (\theta - 1) \sum_{i=1}^4 \log x_i.$$

(ii) $\log x_i = \log e^{-i} = -i$, so $\sum \log x_i = -(1 + 2 + 3 + 4) = -10$:

$$\ell(\theta) = 4 \log \theta - 10(\theta - 1), \quad \frac{d\ell}{d\theta} = \frac{4}{\theta} - 10 \stackrel{!}{=} 0 \Rightarrow \boxed{\hat{\theta}_{ML} = \frac{4}{10} = 0.4}.$$

Second derivative $-4/\theta^2 < 0$ maximum. (Sanity: $\theta = 0.4 < 1$ makes $x^{\theta-1} = x^{-0.6}$ decreasing in x ... the density piles toward 0, matching data clustered near 0: $e^{-4} \approx 0.018$.)

4. With μ known, only σ^2 is free:

$$R(\sigma^2) = \frac{n}{2} \log \sigma^2 + \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2 + \text{const.}$$

Let $v = \sigma^2$. $\frac{dR}{dv} = \frac{n}{2v} - \frac{1}{2v^2} \sum_{i=1}^n (x_i - \mu)^2$ (using $\frac{d}{dv} \frac{1}{v} = -\frac{1}{v^2}$). Set to 0:

$$\frac{n}{2v} = \frac{\sum_{i=1}^n (x_i - \mu)^2}{2v^2} \Rightarrow nv = \sum_{i=1}^n (x_i - \mu)^2 \Rightarrow \boxed{\hat{\sigma}_{ML}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}.$$

(Only the known μ , not the estimated $\bar{x}$, appears inside — this is the tutorial's answer B.)

5. $\boxed{\hat{a}_{ML} = 1.8}$, $\boxed{\hat{b}_{ML} = 4.0}$ (min and max of the data). For $[1.0, 5.0]$: $L = \left(\frac{1}{5-1}\right)^4 = \frac{1}{256} \approx 0.0039$, versus $L(\hat{a}, \hat{b}) = \left(\frac{1}{2.2}\right)^4 \approx 0.0427$ — ten times larger. One line: the likelihood is $(b-a)^{-n}$, strictly decreasing in the width $b-a$, so any strictly wider interval containing the data has strictly smaller likelihood.

6. $\hat{\mu}_{ML} = (4 + 5 + 7 + 8)/4 = \boxed{6}$; squared deviations 4, 1, 1, 4 sum to 10, so $\hat{\sigma}_{ML}^2 = 10/4 = \boxed{2.5}$. $R(\mu) = \frac{1}{2\hat{\sigma}_{ML}^2} \sum_{i=1}^4 (x_i - \mu)^2 + \frac{4}{2} \log \hat{\sigma}_{ML}^2$ with $\frac{4}{2} \log 2.5 = 2(0.9162907) = 1.8326$: - $R(6) = \frac{10}{5} + 1.8326 = 2 + 1.8326 = \boxed{3.8326}$, - $R(5) = \frac{1+0+4+9}{5} + 1.8326 = \frac{14}{5} + 1.8326 = 2.8 + 1.8326 = \boxed{4.6326} > 3.8326$. So $\mu = 6$ really gives the smaller negative log-likelihood.

7. $\hat{\mu}_{ML} = \left(\frac{0+1+2}{3}, \frac{0+2+1}{3}\right)^T = \boxed{(1, 1)^T}$. Centered observations: $(-1, -1)^T, (0, 1)^T, (1, 0)^T$. Outer products:

$$\begin{pmatrix} -1 \\ -1 \end{pmatrix}(-1, -1) = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \begin{pmatrix} 0 \\ 1 \end{pmatrix}(0, 1) = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 0 \end{pmatrix}(1, 0) = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix},$$

sum = $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, so $\hat{\Sigma}_{ML} = \frac{1}{3} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 2/3 & 1/3 \\ 1/3 & 2/3 \end{pmatrix}$. Symmetric (equal off-diagonals). PSD by §7.9's 2×2 test: $2/3 > 0$ and $\det = \frac{4}{9} - \frac{1}{9} = \frac{1}{3} > 0$.

8. (i) Posterior = Beta(3 + 12, 3 + 8) = $\boxed{\text{Beta}(15, 11)}$. (ii) $\hat{p}_{MAP} = \frac{15-1}{15+11-2} = \frac{14}{24} = \boxed{7/12 \approx 0.5833}$.

(iii) Posterior mean = $\frac{15}{15+11} = \boxed{15/26 \approx 0.5769}$. (iv) $\hat{p}_{ML} = 12/20 = \boxed{0.6}$. One line: the Beta(3, 3) prior (centered at 1/2) pulled the estimate slightly down from the MLE 0.6 toward 0.5 — with only 3 + 3 pseudo-observations against 20 real ones, the pull is small.

9. Prior Beta(1, 1) = uniform on (0, 1): $f(p) = 1$. Posterior $\propto p^{n_1}(1-p)^{n_0} \cdot 1 = p^{n_1}(1-p)^{n_0}$, i.e. Beta(1 + n_1 , 1 + n_0) (conjugacy, §20.11). Its mode:

$$\hat{p}_{MAP} = \frac{(1 + n_1) - 1}{(1 + n_1) + (1 + n_0) - 2} = \frac{n_1}{n_1 + n_0} = \frac{n_1}{n} = \boxed{\hat{p}_{ML}}. \blacksquare$$

Equivalently from §20.10's boxed form: with $f(\theta)$ constant, $\arg \max_{\theta} [\log P(D \mid \theta) + \log f(\theta)] = \arg \max_{\theta} \log P(D \mid \theta)$.

10. $P(y_i \mid x_i) = N(wx_i, \sigma^2)$, so $R(w) = -\sum_{i=1}^3 \log \left[\frac{1}{\sigma\sqrt{2\pi}} e^{-(y_i - wx_i)^2 / (2\sigma^2)} \right]$

$$= \boxed{R(w) = \frac{1}{2\sigma^2} \sum_{i=1}^3 (y_i - wx_i)^2 + 3 \log(\sigma\sqrt{2\pi})}.$$

(The second term is w -free.) $\frac{dR}{dw} = -\frac{1}{\sigma^2} \sum_{i=1}^3 x_i(y_i - wx_i) \stackrel{!}{=} 0 \quad \sum x_i y_i = w \sum x_i^2 \quad \boxed{\hat{w}_{ML} = \frac{\sum_i x_i y_i}{\sum_i x_i^2}}.$

Numerically: $\sum x_i y_i = 1 \cdot 2 + 2 \cdot 4 + 3 \cdot 5 = 25$; $\sum x_i^2 = 1 + 4 + 9 = 14$; $\boxed{\hat{w}_{ML} = 25/14 \approx 1.7857}$.

21. Inequalities and the central limit theorem

Chapter 19 defined the normal family and Chapter 20 showed how to estimate its parameters from data — and both chapters ended with a promise: Chapter 21 explains *why* the normal deserves all that attention. This chapter keeps it. It is also the finale of Part II, so it answers the last two questions probability owes us: *how far can a random variable stray from its mean?* (Markov’s and Chebyshev’s inequalities — tail bounds) and *what happens to averages when you have a lot of data?* (the weak law of large numbers and the central limit theorem — the two limit theorems of the course). All of it comes from the MLF Week 12 lecture on “Laws of large numbers: Markov, Chebyshev, Hoeffding, Central limit”.

21.1 The two questions

- i) **Tail bounds.** Given a random variable, how far can it be from its mean? The mean (§15.7) is the center of mass of the distribution — but a center of mass tells you nothing about how far out the mass can sit. Markov’s inequality (§21.2) and Chebyshev’s inequality (§21.3) bound the probability of large deviations using only the mean, or the mean and variance.
- ii) **Limit theorems.** Take n i.i.d. draws (§20.1) and average them. The weak law of large numbers (§21.5) says the average converges to the true mean — in the *convergence in probability* sense of §21.4. The central limit theorem (§21.6) says something stronger and stranger: the *fluctuations* of that average around the mean, scaled up by $\sqrt{n}$, look normal — no matter what distribution the draws came from.

Note. The lecture also names Hoeffding’s inequality — an exponential-rate bound for the sample mean that needs the extra assumption that the variables are bounded. It is stated, not proved, in the lecture, and this chapter treats it the same way: a pointer in §21.5’s note, not a theorem. The strong law of large numbers (convergence *almost surely*) is likewise outside this chapter’s scope — the lecture explicitly sets it aside.

21.2 Markov’s inequality

Def. X is a **positive random variable** if it takes no negative values: $X \geq 0$ (the lecture: “we cannot take negative values”).

Theorem (Markov’s inequality). Let X be a positive random variable with $E[X] = \mu$ (finite). For any $t > 0$,

$$P(X \geq t) \leq \frac{E[X]}{t}.$$

Proof (the lecture’s, for the continuous case — it holds for discrete X too). Write the expectation as an integral and split it at t :

$$E[X] = \int_0^\infty x f_X(x) dx = \int_0^t x f_X(x) dx + \int_t^\infty x f_X(x) dx.$$

The first integral is non-negative ($x \geq 0$, $f_X \geq 0$), so dropping it only makes the right side smaller:

$$E[X] \geq \int_t^\infty x f_X(x) dx.$$

On $[t, \infty)$ we have $x \geq t$, so replace x by the smaller t (again shrinking the right side):

$$E[X] \geq t \int_t^\infty f_X(x) dx = t P(X \geq t),$$

since $\int_t^\infty f_X = P(X \geq t)$. Rearranging gives the inequality.

Note (when it is vacuous). If $t \leq \mu = E[X]$, the bound is $E[X]/t \geq 1$ — and every probability is ≤ 1 anyway, so the inequality says nothing. It only bites for $t > \mu$ (the lecture’s “meaningful only for t greater than or equal to the mean”).

eg 1 (the lecture’s example). X measures something in meters, $E[X] = 10$. What is $P(X \geq 50)$? Markov says immediately, with no other information:

$$P(X \geq 50) \leq \frac{10}{50} = \frac{1}{5} = 20\%.$$

There is only a 20% chance of exceeding 50 meters — from the mean alone.

eg 2 (the bound is tight — the lecture’s discrete example). Let X take only two values: $P(X = 0) = 4/5$, $P(X = 50) = 1/5$. Then

$$E[X] = 0 \cdot \frac{4}{5} + 50 \cdot \frac{1}{5} = \boxed{10}, \quad P(X \geq 50) = \boxed{\frac{1}{5}}.$$

Markov predicts $P(X \geq 50) \leq 10/50 = 1/5$ — and the true probability is *exactly* $1/5$. Given only the mean, you cannot do better than Markov: this distribution shows the bound is the best possible in general. (Problem 9 generalizes this construction.)

Basically, ... Markov = “if the average is μ , then being k times the average can happen at most $1/k$ of the time”. The proof’s whole idea: the expectation is an integral; throw away the part below t ; on the part above t , every x is at least t , so the leftover integral is at least $t \times$ (probability of being above t). And the two-point example shows you can’t sharpen it without more information — the mean alone only buys you this much.

21.3 Chebyshev’s inequality

Markov needs a positive X and bounds a one-sided tail. Chebyshev drops the positivity requirement and bounds the two-sided deviation from the mean, using one more piece of information: the variance (§15.9).

Theorem (Chebyshev’s inequality). Let $E[X] = \mu$ and $\text{Var}(X) = \sigma^2$ (finite). For any $t > 0$,

$$P(|X - \mu| \geq t) \leq \frac{\sigma^2}{t^2}.$$

Proof (the lecture’s — one line, via Markov). The event $\{|X - \mu| \geq t\}$ is the same as $\{(X - \mu)^2 \geq t^2\}$, and $(X - \mu)^2$ is a positive random variable, so Markov applies to it:

$$P(|X - \mu| \geq t) = P((X - \mu)^2 \geq t^2) \leq \frac{E[(X - \mu)^2]}{t^2} = \frac{\sigma^2}{t^2},$$

since $E[(X - \mu)^2]$ is exactly the variance.

- i) **No positivity needed.** The lecture stresses this: X may take negative values — squaring makes $(X - \mu)^2$ positive, which is all Markov needs. Markov is the one-sided, mean-only bound; Chebyshev is the two-sided, mean-and-variance bound.
- ii) **The $k\sigma$ form.** Put $t = k\sigma$ ($k > 0$): then $\sigma^2/t^2 = 1/k^2$, so

$$P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}.$$

The probability of being k standard deviations from the mean is at most $1/k^2$ — for *any* distribution with a variance.

eg 3 (the practice assignment's Q8). For any X with mean μ and standard deviation σ ,

$$P(|X - \mu| \geq 2\sigma) \leq \frac{1}{4} \quad \text{hence} \quad P(|X - \mu| < 2\sigma) \geq \frac{3}{4}.$$

Three-quarters of *any* distribution's mass lies within two standard deviations of its mean — no shape assumed.

Basically, ... Chebyshev = “the variance controls the tails: straying k standard deviations out happens at most $1/k^2$ of the time”. The proof is a trick worth memorizing: square the deviation so Markov applies, and the numerator becomes the variance by definition. Whenever a problem asks “how far from the mean”, and you know the variance, reach for Chebyshev.

21.4 Two ways a sequence of random variables can converge

Before the limit theorems, the lecture fixes the vocabulary — “random variables are not real numbers, they are functions from Ω to $\mathbb{R}$, so can we have notions of convergence?” Two of them (a third, convergence *almost surely*, is stronger; the lecture names it and moves on).

Def (convergence in probability). $X_n \xrightarrow{p} X$ if, for every fixed $\varepsilon > 0$,

$$\lim_{n \rightarrow \infty} P(|X_n - X| \geq \varepsilon) = 0.$$

Fix any (nonzero) tolerance ε ; the probability of missing it by more than ε dies out as n grows.

Def (convergence in distribution). $X_n \xrightarrow{d} X$ if their CDFs (§15.5, §16.4) match in the limit:

$$\lim_{n \rightarrow \infty} F_{X_n}(x) = F_X(x) \quad \text{for all } x.$$

Only the *distributions* have to line up — the random variables themselves need not get close to each other.

- i) **Hierarchy.** Convergence in probability *implies* convergence in distribution, but not conversely (the lecture). In-distribution is the weaker claim: it is about histograms lining up, not about X_n pinning down X .
- ii) **Why the distinction matters.** The weak law (§21.5) ends at a *constant* μ — convergence in probability is the natural language. The CLT (§21.6) ends at a *distribution* $N(0, \sigma^2)$ — convergence in probability cannot even be stated there (a distribution is not a number to get close to), so the theorem uses convergence in distribution.

Basically, ... Convergence in probability = “the random variable itself settles down onto a target”. Convergence in distribution = “only its *shape* settles down — the histogram freezes, even though each draw is still random”. The law of large numbers is the first kind; the central limit theorem is the second.

21.5 The weak law of large numbers

Def (sample mean). For $X_1, \dots, X_n$ i.i.d. (§20.1), $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ — the empirical average, itself a random variable.

You already suspect $\bar{X}_n$ should sit near $\mu = E[X_1]$. The weak law makes it precise.

Theorem (weak law of large numbers). Let $X_1, X_2, \dots$ be i.i.d. with $E[X_i] = \mu$. Then

$$\boxed{\bar{X}_n \xrightarrow{p} \mu} \quad \text{i.e.} \quad \lim_{n \rightarrow \infty} P(|\bar{X}_n - \mu| \geq \varepsilon) = 0 \quad \forall \varepsilon > 0.$$

Proof (the lecture's, under one extra assumption: the variance is finite, $\text{Var}(X_i) = \sigma^2 < \infty$). Two facts about $\bar{X}_n$: - $E[\bar{X}_n] = \frac{1}{n} \sum E[X_i] = \mu$ (linearity of expectation, §15.8). - $\text{Var}(\bar{X}_n) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{\sigma^2}{n}$ — independence makes the variance of the sum the sum of the variances (§17.11), and the $1/n$ factor squares (§15.9's corollary).

Apply Chebyshev (§21.3) to the random variable $\bar{X}_n$:

$$P(|\bar{X}_n - \mu| \geq \varepsilon) \leq \frac{\text{Var}(\bar{X}_n)}{\varepsilon^2} = \frac{\sigma^2}{n\varepsilon^2} \xrightarrow{n \rightarrow \infty} 0. \quad \blacksquare$$

Note (the assumptions, and Hoeffding). The lecture flags three things. First, the proof assumed finite variance — the theorem is true without it, but that proof is harder. Second, Chebyshev's bound dies only as $1/n$: doubling n from 10 to 100 cuts the bound from $\sim 1/10$ to $\sim 1/100$. The *true* concentration of $\bar{X}_n$ is far faster — exponential in n — and Hoeffding's inequality captures that, but it demands the stronger assumption that each X_i is bounded in $[a, b]$:

$$\boxed{P(|\bar{X}_n - \mu| \geq \varepsilon) \leq 2 \exp\left(-\frac{2n\varepsilon^2}{(b-a)^2}\right)} \quad (\text{Hoeffding — stated, not proved}).$$

The lecture does not prove it; neither does this chapter. Third, boundedness is strictly stronger than finite variance: a normal has $\sigma^2 = 1$ but is unbounded — Hoeffding cannot touch it, which is why the lecture proves the weak law with Chebyshev instead.

eg 4 (how slow is $1/n$? — the lecture's comparison, made numeric). Chebyshev says $P(|\bar{X}_n - \mu| \geq \varepsilon) \leq \sigma^2/(n\varepsilon^2)$. Suppose you want this ≤ 0.01 at tolerance $\varepsilon = 0.1\sigma$:

$$\frac{\sigma^2}{n(0.1\sigma)^2} \leq 0.01 \Rightarrow \frac{100}{n} \leq 0.01 \Rightarrow \boxed{n \geq 10,000}.$$

Ten thousand samples for a 99%-confidence, tenth-of-a-standard-deviation guarantee — Chebyshev's rate is honest but weak. (Problem 6.) The lecture's point: the bound is valid, the reality (Hoeffding's exponential rate) is much better.

Basically, ... The weak law = “averages work”. Flip a coin n times and the fraction of heads *will* get arbitrarily close to the true bias — not certainly for any fixed n , but with probability tending to 1 as n grows. The proof is Chebyshev applied to the average itself, using the one miracle fact $\text{Var}(\bar{X}_n) = \sigma^2/n$: averaging n independent copies shrinks the variance by n . That $1/n$ shrinkage is the whole engine — and it is also why the lecture's exponential-rate refinement (Hoeffding) needs bounded variables.

21.6 The central limit theorem

Same setup as the weak law — $X_1, X_2, \dots$ i.i.d., $E[X_i] = \mu$, $\text{Var}(X_i) = \sigma^2$ — but a different question. The weak law asks *where* the average settles (μ). The CLT asks: *zoom in* on the fluctuations around μ — what do they look like?

Def. $Y_n = \frac{1}{\sqrt{n}} \sum_{i=1}^n (X_i - \mu)$ — a *scaled* sum. Compare with the sample mean $\bar{X}_n = \frac{1}{n} \sum X_i$: the CLT's Y_n uses $1/\sqrt{n}$, not $1/n$ (and centers each term at μ).

Theorem (central limit theorem — quoted, not proved). The lecture states it without proof:

$$Y_n \xrightarrow{d} N(0, \sigma^2).$$

The scaled sum converges *in distribution* to a normal with mean 0 and variance σ^2 — the variance of the original X_i 's, whatever distribution they came from.

Corollary (the standardized form). Since $Y_n = \sqrt{n}(\bar{X}_n - \mu)$,

$$Z_n = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} = \frac{Y_n}{\sigma} \xrightarrow{d} N(0, 1).$$

Equivalently, for large n : $\bar{X}_n \dot{\sim} N\left(\mu, \frac{\sigma^2}{n}\right)$ — the sample mean is approximately normal, centered at the true mean, with the $1/n$ -shrunk variance of §21.5.

Note (why $\sqrt{n}$, not n). Compute $\text{Var}(Y_n)$: by independence (§17.11), $\text{Var}(\sum (X_i - \mu)) = n\sigma^2$, and the $1/\sqrt{n}$ factor squares to $1/n$ (§15.9):

$$\text{Var}(Y_n) = \frac{1}{n} \cdot n\sigma^2 = \boxed{\sigma^2} \quad \text{for every } n.$$

The $\sqrt{n}$ scaling is the *one* scaling that keeps the variance constant — the fluctuations neither collapse to a point (as with $1/n$, the weak law's scaling, where $\text{Var} \rightarrow 0$) nor blow up (as with no scaling). It is a microscope tuned exactly to the scale where the shape of the randomness stays visible. And that shape is always the bell curve.

Note (a limit theorem, but an early one). The CLT is a statement about $n \rightarrow \infty$, yet the lecture stresses it “begins to hold even for n in the range of 10 to 20” — §21.7's demo shows $n = 10$ already looking normal.

Basically, ... WLLN = “the average converges to the mean” (the *location*). CLT = “blow up the wiggles around the mean by $\sqrt{n}$ and they look like a bell curve” (the *shape*). The $\sqrt{n}$ is a zoom level: zoom out ($1/n$) and the wiggles vanish into the mean — that's the weak law; don't zoom at all and they explode; zoom by exactly $\sqrt{n}$ and the wiggles freeze into a normal distribution. Any starting distribution — uniform, Bernoulli, anything with a variance — ends up at the same bell curve. That is why the normal is everywhere (§21.9).

21.7 Worked: the deck's uniform demo

The lecture's own illustration. Take X_i i.i.d. $\text{Uniform}[-0.5, 0.5]$ — flat, about as non-normal as possible. Then $\mu = E[X_i] = 0$ and $\sigma^2 = (1)^2/12 = 1/12$ (§16.7), so the CLT predicts $Y_n = \frac{1}{\sqrt{n}} \sum_{i=1}^n X_i \xrightarrow{d} N(0, 1/12)$.

Deck demo: X_i iid Uniform $[-0.5, 0.5]$, $Y_n = \frac{1}{\sqrt{n}} \sum_{i=1}^n X_i$ approaches $N(0, 1/12)$ (red)

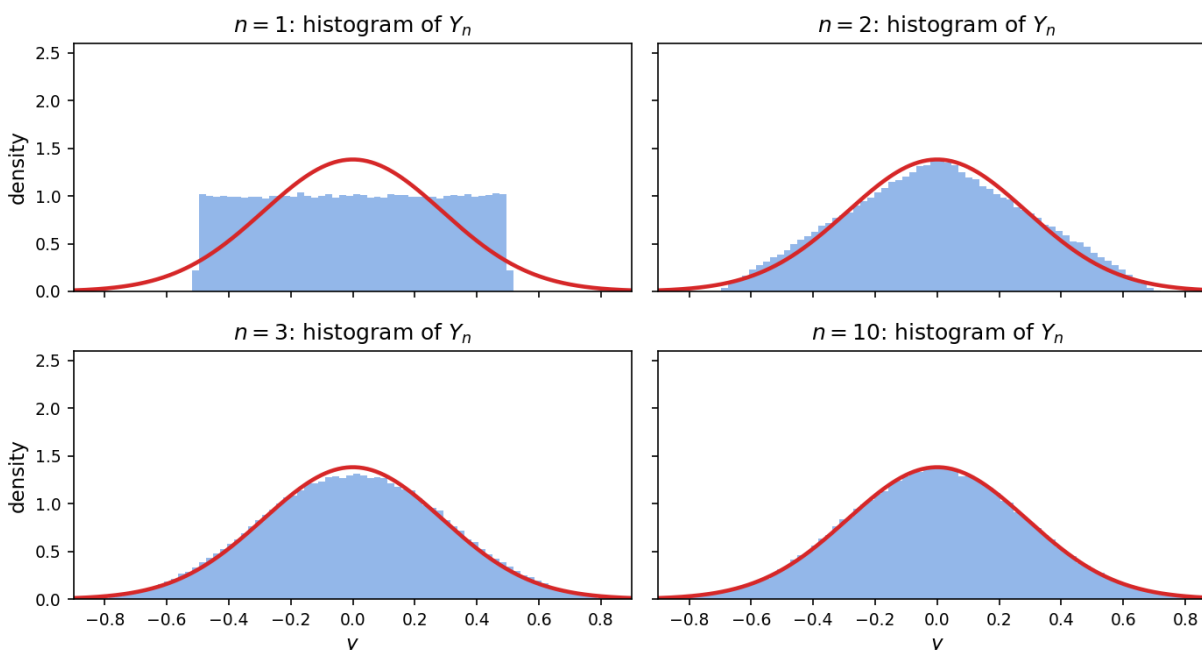


Figure 23: Histograms of $Y_n = (1/\sqrt{n})\sum X_i$ for X_i iid Uniform $[-0.5, 0.5]$ at $n = 1, 2, 3, 10$, with the $N(0, 1/12)$ density overlaid in red. At $n = 1$ the histogram is flat; at $n = 2$ it is triangular; at $n = 3$ it is rounded; at $n = 10$ it matches the normal curve.

- i) $n = 1$: $Y_1 = X_1$ — just the flat uniform histogram. No normality at all.
- ii) $n = 2$: $Y_2 = (X_1 + X_2)/\sqrt{2}$ — the histogram turns *triangular* (the sum of two uniforms is triangular; the $1/\sqrt{2}$ only rescales).
- iii) $n = 3$: a rounded, piecewise-quadratic hump — visibly bell-ish.
- iv) $n = 10$: the histogram sits almost exactly on the red $N(0, 1/12)$ curve. Ten uniforms in, bell curve out — the lecture’s “even $n = 10$ to 20” claim, verified.

eg 5 (reading the limit off the figure). The red curve is $N(0, 1/12)$: standard deviation $1/\sqrt{12} \approx 0.289$. At $n = 10$ the histogram’s peak height is ≈ 1.38 — and the normal peak is $1/(\sigma\sqrt{2\pi}) = 1/(0.289 \cdot 2.507) \approx 1.38$. The variance stayed $\sigma^2 = 1/12$ at every n (the §21.6 calculation) — only the *shape* changed, from flat to bell.

21.8 Worked: CLT approximation for a binomial

The most-used CLT calculation in practice: a count is a sum of indicators (§15.12), so counts are approximately normal.

Setup. Flip a fair coin 100 times; X = number of heads. Write $X = \sum_{i=1}^{100} X_i$ with $X_i \sim \text{Bernoulli}(1/2)$ i.i.d. (§15.11): $E[X_i] = 1/2$, $\text{Var}(X_i) = 1/4$. Hence $E[X] = 100 \cdot \frac{1}{2} = 50$, $\text{Var}(X) = 100 \cdot \frac{1}{4} = 25$ (§17.11), $\sigma_X = 5$.

CLT approximation. For $n = 100$ (large), $X \sim N(50, 25)$:

$$P(X \geq 60) \approx P\left(Z \geq \frac{60 - 50}{5}\right) = 1 - \Phi(2) \approx 1 - 0.97725 = \boxed{0.0228},$$

using the $\Phi(2)$ table value from §16.9. (No continuity correction — the sources don't use one; see the review log.)

How good is it? Three ways to bound/estimate the same number.

- i) **Markov** (§21.2): $P(X \geq 60) \leq E[X]/60 = 50/60 \approx \boxed{0.833}$. Valid — and useless.
- ii) **Chebyshev** (§21.3): $P(X \geq 60) = P(X - 50 \geq 10) \leq P(|X - 50| \geq 10) \leq 25/10^2 = \boxed{0.25}$. Better, still loose.
- iii) **Exact** (binomial sum, §15.12): $P(X \geq 60) = \sum_{k=60}^{100} \binom{100}{k} / 2^{100} \approx \boxed{0.0284}$.

So: Markov 0.83, Chebyshev 0.25, CLT 0.0228, truth 0.0284. The inequalities are *guarantees* — always true, often loose. The CLT is an *approximation* — not guaranteed, but far tighter. That is the practical division of labor in this chapter: bounds when you must be certain, CLT when you want a number.

Note. The CLT approximation is one-sided here only for convenience — $P(X \leq 40) \approx \Phi(-2) = 0.0228$ by symmetry, and two-sided versions work the same way (Problem 8).

Basically, ... “100 fair flips, at least 60 heads” “a normal with mean 50 and sd 5 exceeds 60” = “a standard normal exceeds 2” 2.3%. The recipe never changes: mean = $n\mu$, sd = $\sigma\sqrt{n}$, standardize, look up Φ . Compare against Markov/Chebyshev to feel the trade: guarantees are loose, approximations are tight.

21.9 Why this matters: the normal's ubiquity, and the Part II closing shot

- i) **The promise is kept.** §19.9(iv), §20.12(iv), and §16.9's note all pointed here: the CLT is *why* the normal family deserved two chapters. The lecture's physical picture: measure a wire's current with a sensitive ammeter and you get a normal — because the current is the sum of *many electrons* each moving left or right (a Bernoulli each, §15.11), and a sum of many small independent effects is normal by the CLT. Measurement noise, heights, test scores — whenever the observed quantity is an additive pile-up of small independent contributions, the bell curve appears. “Gaussian” is just the name the pile-up goes by.
- ii) **The weak law underwrites all of estimation.** Every estimator in Chapter 20 is built from sample averages — $\hat{\mu}_{ML} = \bar{x}$ (§20.6), the empirical covariance (§20.7). The weak law is the silent license: those averages converge to the true parameters as n grows, so *more data means better estimates*. §20.10's “lots of data washes the prior out” is the same phenomenon in Bayesian dress.
- iii) **What comes next.** Part III turns this machinery into machine learning: linear and polynomial regression (Ch 23) inherit §20.8's “Gaussian noise — least squares”, and every generalization argument from here on leans on averages concentrating (§21.5) and fluctuations being normal (§21.6).

Part II, in one paragraph. Probability basics (Ch 14): experiments, sample spaces, conditional probability, Bayes. Discrete and continuous random variables (Ch 15–16): PMF/PDF/CDF, expectation, variance, the named distributions — Bernoulli, binomial, geometric, uniform, exponential, normal. Joint distributions (Ch 17–18): two variables at once, independence, covariance, “variances add for independent variables”. Random vectors (Ch 19): the multivariate normal and its parameters μ, Σ — “everything linear stays normal”. Estimation (Ch 20): MLE picks the parameter that makes the data most likely; MAP adds a prior's tug; Gaussian noise turns fitting into least squares. Inequalities and limits (this chapter): Markov and Chebyshev bound the tails from the mean and variance alone; the weak law says averages converge to the mean; the central limit theorem says the $\sqrt{n}$ -scaled fluctuations around it are always normal — which is why the bell curve ran this entire part. With this, Part II's machinery is complete — Part III builds machine learning on top of it.

Problem set

1. (Practice assignment Q1.) A biased coin lands heads with probability 1/10; flipped 200 times. Use Markov's inequality to bound $P(X \geq 120)$, where X = number of heads. Give the bound as a reduced

fraction.

2. (Tutorial Q1.) $X \sim \text{Binomial}(n, p)$. Use Markov's inequality to bound $P(X \geq \alpha n)$ where $p < \alpha < 1$; show the bound is p/α . Evaluate for $p = 1/2$, $\alpha = 3/4$.
3. (Practice assignment Q2.) $X \sim \text{Binomial}(n, p)$. Use Chebyshev's inequality to bound $P(X \geq \alpha n)$: show $P(X \geq \alpha n) \leq \frac{p(1-p)}{n(\alpha-p)^2}$. Evaluate for $p = 1/2$, $\alpha = 3/4$, $n = 8$, and compare with Problem 2's Markov bound — which is tighter here?
4. (Tutorial Q4.) X has the geometric distribution $p(x) = 2^{-x}$, $x = 1, 2, \dots$ (i) Use Chebyshev to show $P(|X - 2| \leq 2) \geq 1/2$. (ii) Compute the exact $P(|X - 2| \leq 2)$ and compare with the bound.
5. Prove the $k\sigma$ rule from Chebyshev: for $E[X] = \mu$, $\text{Var}(X) = \sigma^2$, $P(|X - \mu| \geq k\sigma) \leq 1/k^2$. Then give the lower bound on $P(|X - \mu| < 3\sigma)$ that holds for *every* distribution with finite variance.
6. $X_1, \dots, X_n$ i.i.d. with variance σ^2 . How large must n be for Chebyshev to guarantee $P(|\bar{X}_n - \mu| \geq 0.1\sigma) \leq 0.01$? What does the answer say about Chebyshev's $1/n$ rate?
7. $X_1, \dots, X_{36}$ i.i.d. with $\mu = 10$, $\sigma^2 = 4$. (i) State the CLT's approximate distributions of Y_{36} and of $\bar{X}_{36}$. (ii) Approximate $P(\bar{X}_{36} \geq 10.5)$.
8. 100 fair coin flips, X = number of heads. Use the CLT to approximate $P(X \leq 42)$ (standardize and look up Φ ; no continuity correction). The exact binomial value is 0.0666 — how far off is the approximation?
9. Fix $\mu > 0$ and $t > \mu$. Construct a random variable attaining equality in Markov's inequality: $P(X = 0) = 1 - \mu/t$, $P(X = t) = \mu/t$. Verify $E[X] = \mu$ and $P(X \geq t) = \mu/t = E[X]/t$.
10. Explain in words: the WLLN uses the $1/n$ scaling and ends at the constant μ (convergence in probability); the CLT uses the $1/\sqrt{n}$ scaling and ends at $N(0, \sigma^2)$ (convergence in distribution). Why can't the CLT use $1/n$? (Hint: what is $\text{Var}(\bar{X}_n)$ as $n \rightarrow \infty$, and what does a "distribution" limit with zero variance look like?)

Solutions — Chapter 21: Inequalities and the central limit theorem

1. X = heads in 200 flips, $X \sim \text{Binomial}(200, 1/10)$ (§15.12). $E[X] = np = 200 \cdot \frac{1}{10} = 20$. $X \geq 0$, so Markov (§21.2):

$$P(X \geq 120) \leq \frac{E[X]}{120} = \frac{20}{120} = \boxed{\frac{1}{6}}.$$

(The practice assignment's answer: $A = 1/6$.)

2. $X \sim \text{Binomial}(n, p)$ has $E[X] = np$ (§15.12). Markov (§21.2), with $t = \alpha n > 0$:

$$P(X \geq \alpha n) \leq \frac{E[X]}{\alpha n} = \frac{np}{\alpha n} = \boxed{\frac{p}{\alpha}}.$$

The condition $p < \alpha < 1$ makes the bound < 1 (non-vacuous) and $\alpha n > E[X]$ (the §21.2 note). For $p = 1/2$, $\alpha = 3/4$:

$$P(X \geq 3n/4) \leq \frac{1/2}{3/4} = \frac{2}{3} \approx \boxed{0.667}.$$

(The tutorial's answer: $A = 2/3$.)

3. First shift to a deviation from the mean, then apply Chebyshev (§21.3):

$$P(X \geq \alpha n) = P(X - np \geq n\alpha - np) \leq P(|X - np| \geq n(\alpha - p)) \leq \frac{\text{Var}(X)}{[n(\alpha - p)]^2}.$$

The first inequality holds because $\{X - np \geq t\} \subseteq \{|X - np| \geq t\}$ for $t = n(\alpha - p) > 0$. With $\text{Var}(X) = np(1-p)$ (§15.12):

$$P(X \geq \alpha n) \leq \frac{np(1-p)}{n^2(\alpha-p)^2} = \boxed{\frac{p(1-p)}{n(\alpha-p)^2}}.$$

For $p = 1/2$, $\alpha = 3/4$, $n = 8$: $\alpha - p = 1/4$,

$$\frac{(1/2)(1/2)}{8 \cdot (1/4)^2} = \frac{1/4}{8/16} = \frac{1/4}{1/2} = \boxed{0.5}.$$

(The practice assignment's answer: 0.5, equivalently $4/n$.) Comparison: Markov (Problem 2) gave $2/3 \approx 0.667$; Chebyshev gives 0.5 — **Chebyshev is tighter here**, because it exploits the variance, not just the mean.

4. (i) $p(x) = 2^{-x}$, $x = 1, 2, \dots$ is Geometric($1/2$) (first success on trial x). From §15.13 (eg 12): $E[X] = 1/p = \boxed{2}$, $\text{Var}(X) = (1-p)/p^2 = (1/2)/(1/4) = \boxed{2}$. Chebyshev (§21.3) with $t = 2$:

$$P(|X - 2| \geq 2) \leq \frac{2}{2^2} = \frac{1}{2} \quad \Rightarrow \quad \boxed{P(|X - 2| \leq 2) \geq 1 - \frac{1}{2} = \frac{1}{2}}.$$

(ii) Exact: $|X - 2| \leq 2 \iff 0 \leq X \leq 4$, and $X \geq 1$ always, so

$$P(|X - 2| \leq 2) = P(X \leq 4) = \sum_{k=1}^4 2^{-k} = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} = \boxed{\frac{15}{16} = 0.9375}.$$

Comparison: Chebyshev guarantees ≥ 0.5 ; the truth is 0.9375. The bound is valid but loose — Chebyshev knows only the mean and variance, and the geometric's long right tail forces it to be pessimistic.

5. Chebyshev (§21.3): $P(|X - \mu| \geq t) \leq \sigma^2/t^2$. Set $t = k\sigma$ ($k > 0$):

$$P(|X - \mu| \geq k\sigma) \leq \frac{\sigma^2}{k^2\sigma^2} = \boxed{\frac{1}{k^2}}.$$

For $k = 3$: $P(|X - \mu| \geq 3\sigma) \leq 1/9$, so $\boxed{P(|X - \mu| < 3\sigma) \geq 1 - 1/9 = 8/9 \approx 0.889}$ — for every distribution with finite variance. (For $k = 2$ this recovers eg 3's $1/4$ / $3/4$, the practice assignment's Q8.)

6. Chebyshev on $\bar{X}_n$ (the §21.5 calculation): $P(|\bar{X}_n - \mu| \geq 0.1\sigma) \leq \frac{\sigma^2}{n(0.1\sigma)^2} = \frac{100}{n}$. Demand ≤ 0.01 :

$$\frac{100}{n} \leq 0.01 \quad \Rightarrow \quad n \geq \frac{100}{0.01} = \boxed{10,000}.$$

The answer says Chebyshev's $1/n$ rate is *slow*: cutting the tolerance by 10 costs $100\times$ the samples, and cutting the failure probability by 10 costs $10\times$ the samples. The lecture's point (§21.5 note): the bound is honest, but the true concentration (Hoeffding's exponential rate, for bounded variables) needs far fewer samples.

7. (i) $Y_{36} = \frac{1}{\sqrt{36}} \sum_{i=1}^{36} (X_i - 10) \xrightarrow{d} N(0, \sigma^2) = \boxed{N(0, 4)}$ (§21.6). Equivalently (§21.6 corollary),

$$\bar{X}_{36} \sim \boxed{N\left(10, \frac{4}{36}\right) = N(10, 1/9)}, \quad \text{sd} = \frac{2}{6} = \frac{1}{3}.$$

(ii) Standardize:

$$P(\bar{X}_{36} \geq 10.5) \approx P\left(Z \geq \frac{10.5 - 10}{1/3}\right) = P(Z \geq 1.5) = 1 - \Phi(1.5) \approx 1 - 0.93319 = \boxed{0.0668}.$$

8. $X = \sum_{i=1}^{100} X_i$, $X_i \sim \text{Bernoulli}(1/2)$: $E[X] = 50$, $\text{Var}(X) = 25$, $\text{sd} = 5$ (§21.8 setup). CLT: $X \sim N(50, 25)$:

$$P(X \leq 42) \approx P\left(Z \leq \frac{42 - 50}{5}\right) = \Phi(-1.6) = 1 - \Phi(1.6) \approx 1 - 0.94520 = \boxed{0.0548}.$$

Exact binomial value: 0.0666. Error: $|0.0666 - 0.0548| = 0.0118$ — about 1.2 percentage points, i.e. the CLT is off by $\approx 18\%$ relative. Decent for $n = 100$ with no continuity correction; it improves as n grows. (The CLT is an approximation, not a bound — §21.8.)

9. $E[X] = 0 \cdot P(X = 0) + t \cdot P(X = t) = 0 \cdot (1 - \mu/t) + t \cdot (\mu/t) = \boxed{\mu}$. And $P(X \geq t) = P(X = t) = \boxed{\mu/t} = E[X]/t$ — equality in Markov (§21.2). Conditions: $\mu > 0$, $t > \mu$ ensure $0 < \mu/t < 1$, so the probabilities are valid ($1 - \mu/t > 0$). This is the general form of eg 2's $\{0, 4/5; 50, 1/5\}$ example ($\mu = 10$, $t = 50$): no bound using only the mean can beat Markov, because this distribution attains it.

10. The $1/n$ scaling in $\bar{X}_n$ averages: $\text{Var}(\bar{X}_n) = \sigma^2/n \rightarrow 0$, so the distribution collapses onto the single point μ — the only possible limit of a “distribution” with vanishing variance is a constant, and “convergence to a constant” is convergence in probability (the WLLN). If you fed $\bar{X}_n$ to the CLT’s machinery instead, you’d get the degenerate claim “ $\bar{X}_n \xrightarrow{d} N(\mu, 0)$ ” — a normal with zero variance is just the point μ again; all shape information is lost. The CLT’s $1/\sqrt{n}$ scaling is the unique zoom where $\text{Var}(Y_n) = \sigma^2$ stays fixed (§21.6 note): the center still converges (that part is the WLLN), but the *fluctuations* stay visible and converge in distribution to $N(0, \sigma^2)$. In short: $1/n$ asks “where does it settle?” (answer: μ , in probability); $1/\sqrt{n}$ asks “what do the wiggles look like on the way there?” (answer: normal, in distribution).

Part III — Machine Learning Foundations

22. What is ML: data, models, tasks; supervised vs unsupervised

Chapter 21 closed Part II by explaining *why* averages concentrate and fluctuations look normal. This chapter opens Part III: it turns that machinery into machine learning. Everything here comes from the MLF Week 1 lectures — “Introduction, Terminology and Setup” by Harish Guruprasad Ramaswamy — whose job is to fix the vocabulary (data, model, learning algorithm, task) and sketch the four learning problems this part of the book is about: regression and classification (supervised), dimensionality reduction and density estimation (unsupervised). The algorithms themselves are Chapters 23–25’s job; this chapter frames.

22.1 What is machine learning?

Def. Machine learning (ML) = “the study of computer algorithms that improve automatically through experience and by the use of data” (the Wikipedia definition the lecture opens with).

Three phrases do the work: *computer algorithms* (it is a computation problem, not philosophy), *improve automatically through experience* (the system gets better without a human rewriting it), and *by the use of data* (experience = data — past examples, not a textbook of rules).

Basically, ... Ordinary software = a human writes the rules; the computer executes them. Machine learning = the computer *derives* the rules from examples. You don’t teach it face detection by describing a face in code — you show it a million faces and let it figure out the pattern.

Note (the ML vs everything-else test). The lecture’s rule of thumb: if you can solve a task with manual labour or plain programming, *do that* — don’t reach for ML. ML earns its keep only where both fail. That decision is made by a task analysis (§22.2).

22.2 The task hierarchy: manual labour → programming → machine learning

Def (task). A **task** = a process converting an **input** into an **output**. The same task can be performed at increasing levels of abstraction:

- i) **Manual labour.** The human directly converts input to output. (Input → Human → Output.)
- ii) **Programming / tool usage.** The human builds a *tool* that converts input to output. (Input → Software → Output.)
- iii) **Machine learning.** The human doesn’t even build the tool. The human gives a broad blueprint to a *tool design*, which uses **data** plus the human’s blueprint to construct the tool. (Input → Model → Output, where the model was built by a learning algorithm from data.)

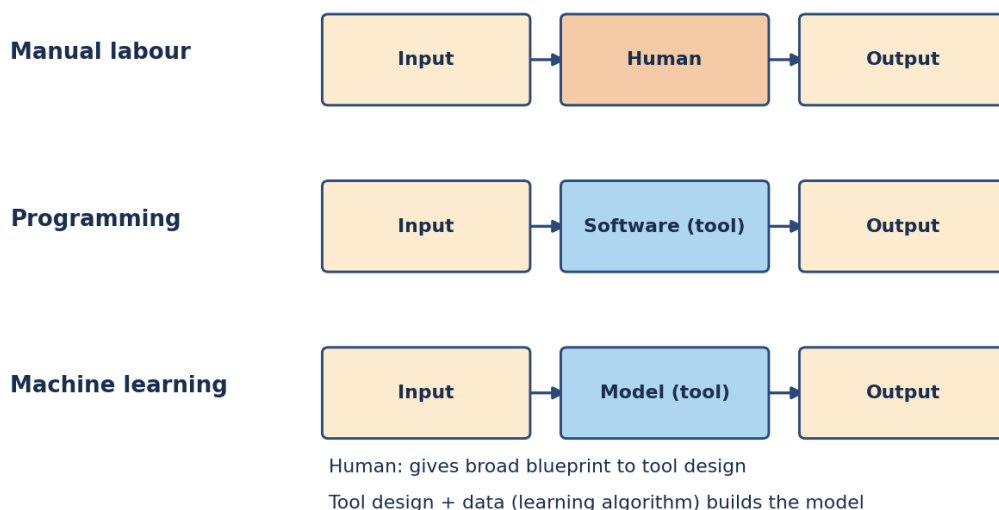


Figure 24: Three rows of block diagrams: Manual labour (Input -> Human -> Output), Programming (Input -> Software (tool) -> Output), Machine learning (Input -> Model (tool) -> Output), with a note that the human gives a broad blueprint to tool design and the learning algorithm builds the model from data.

Why programming/human labour fail. Two failure modes, and one success condition for ML:

- i) **Scale/speed/cost.** Humans can verify passwords in principle — but not for billions of logins.
- ii) **Can't express the rules.** You know a face when you see one (a baby learns it from a few examples), but you cannot write down “what pixel patterns constitute a face” in a programming language. Or you don't even know the rules at all — nobody fully knows the function mapping today's radar map to tomorrow's rain.
- iii) **Hope for ML.** Lots of example data, plus *some* structural idea about the rule (you don't know the exact rule, but you know what shape it might take).

eg 1 (task analysis: password verify — no ML needed). Input: password; output: authentication. Manual labour: conceivable but fails on scale (billions of humans needed). Programming: trivial — compare input against the password on file. Since programming works, ML is never considered.

eg 2 (task analysis: face detection — the ML case). Input: image (pixels); output: face / no face (+ location). Manual labour: conceivable (a human drawing boxes on every photo) but absurd on scale. Programming: fails — the idea of “face” cannot be conveyed to a computer that sees only pixels, even though such a function clearly exists. ML: succeeds — the human never writes the face rule; a learning algorithm builds the tool from lots of face/non-face images (practically unlimited on the internet).

eg 3 (task analysis: weather prediction — the ML case). Input: radar map (wind, moisture); output: rain/shine tomorrow. Manual labour fails — humans don't know the rule themselves (physics implements it; nobody wrote it down). Programming fails for the same reason — you can't code what you don't know. ML works because a rule *exists* (nature runs it), and there is immense historical weather data to learn it from.

The wonders (the lecture's tour). Once you see the hierarchy, you see ML everywhere: **inbox** (spam classifiers — “one of the biggest success cases of machine learning”), **shopping cart** (recommender systems: “customers who bought this also bought...”), **smart assistant** (sound waveform → text → command: “increase volume by 10%”), **robots** (a Mars rover can't be remote-controlled with 10–15 minute radio delays — it must decide actions from its environment), **games** (chess, then Go: “impossibly complex” until an ML system beat the best human), **marketing** (who to advertise to on a fixed budget).

Basically, ... Ask two questions about any task. (1) Can a human do it at scale? Can a programmer write

the rule? If either is yes — stop, use that. (2) If both are no — is there lots of example data and some structural idea of the rule? If yes — that’s a machine learning task. Password verify stops at question 1; face detection and weather prediction pass both gates.

22.3 Data: a collection of vectors

Def. In ML, **data** = a collection of vectors. Each data point is a vector in $\mathbb{R}^d$ (the lecture: “data will almost always mean a collection of vectors”).

The lecture’s running example: six houses, each a 4-dimensional vector:

	# rooms	Area (100 sq.ft)	Distance to metro (km)	Price (10 lakhs)
House 1	3	9	1.9	5.0
House 2	2	7	2.1	3.2
House 3	4	12	2.8	6.6
House 4	5	16	0.9	9.8
House 5	5	15	3.1	8.5
House 6	4	11	1.6	6.9

Def (metadata). **Metadata** = information *on* the data — what each number means. Here: coordinate 1 = number of rooms, coordinate 2 = area in hundreds of square feet, coordinate 3 = distance to metro in km, coordinate 4 = price in tens of lakhs.

- i) **Metadata is for humans.** “House 1 is a three-bedroom house with 900 sq.ft, 1.9 km from the metro, priced at 50 lakhs” — that story is human-readable; the computer only needs the numbers.
- ii) **Consistency is for the computer.** The computer doesn’t need to know what “rooms” means — but coordinate 1 must mean rooms for *every* house. As long as the meaning is consistent across rows, the data is usable.

eg 4 (reading the table with metadata). House 4 = (5, 16, 0.9, 9.8) reads as: 5 rooms, 1600 sq.ft, 0.9 km from the metro, price 98 lakhs (9.8 tens of lakhs). The vector (5, 16, 0.9, 9.8) is the data; the sentence is the metadata’s work.

Note (the data assumption). Learning algorithms treat the data points as independent draws of the same kind — the examples are generated “independently”, in the lecture’s phrasing for the tweet stream (§22.13) — and expect *future* data to look like the training data (that’s what “predict on unseen data” in §22.10 means). The formal i.i.d. machinery is §20.1’s; here it is just the standing bet every ML task makes: tomorrow’s data resembles today’s.

Basically, ... Data = a table of numbers (vectors). Metadata = the column headers that tell *you* what the numbers mean; the computer doesn’t care about headers, it only cares that column 2 means the same thing in every row. Every ML chapter from here on starts from such a table.

22.4 Models: mathematical simplifications of reality

Def. A **model** = a mathematical simplification of reality. It represents reality but is simpler and more compact than reality — never exact, sometimes useful.

The lecture’s scientific examples: the **ideal gas model** ($PV = nRT$ — no gas is truly ideal, but it captures the important behaviour); the **inverse-square law** for gravity (force $\propto 1/R^2$ — ignores relativity, still runs spaceflight); **Moore’s law** (transistor counts doubling — not a physical law, but a trend that held for decades); the **Cobb–Douglas model** in economics.

“All models are wrong, but some are useful.” — George Box

ML models are the same idea, aimed at a narrower job. Two kinds:

- i) **Predictive models** — predict the future from the present. Two subtypes:
- **Regression model.** Predicts a *real-valued* quantity. The lecture’s example good model:

$$\text{Price} = 0.5 \times \text{Area} - \text{Distance}.$$

Price rises with area, falls with distance from the metro — a trend with exceptions, but useful: on a fixed budget you can now reason “big house far away, or small house close in”.

- **Classification model.** Predicts a *discrete* quantity. The lecture’s example good model: for whether a house is closer than 2 km to a metro,

$$\text{Close if } 2 \times \text{ROOMS} - \text{PRICE} < 1, \quad \text{Far otherwise.}$$

Reading: a small expensive house (few rooms, high price) is probably close to the city; the rule encodes that hunch as arithmetic.

- ii) **Probabilistic models** — don’t predict; they *score reality*. Give them any configuration and they say how likely it is. Two of the lecture’s examples: “what is the probability that a randomly chosen person is at lat-long (25°N, 30°E)?” (high in Bombay, low in the Sahara desert); “what is the probability that a given tweet was generated by Mr. Chopra?” (Chopra-like tweets score high; random digit strings score low).

Note (predictive vs probabilistic). A regression model answers “what will this new house cost?”. A probabilistic model answers “how likely is this configuration?”. Both are functions; they differ in what the output *means*. Density estimation (§22.13) builds the second kind; regression and classification build the first.

eg 5 (using the deck’s models on new data). Take the regression model $\text{Price} = 0.5 \times \text{Area} - \text{Distance}$ (area in 100 sq.ft, distance in km, price in 10 lakhs). A new house: 700 sq.ft at 3 km from the metro → area coordinate 7:

$$\text{Price} = 0.5 \times 7 - 3 = 3.5 - 3 = \boxed{0.5} \text{ (in 10 lakhs)} = 5 \text{ lakhs.}$$

You’ve never seen this house, but the model predicts its price anyway — that is what makes it *predictive*.

eg 6 (using the deck’s classification model). Rule: Close if $2 \times \text{ROOMS} - \text{PRICE} < 1$. House A: 2 rooms, price 8 (in 10 lakhs) → $2(2) - 8 = -4 < 1 \rightarrow$ **Close**. House B: 4 rooms, price 3 → $2(4) - 3 = 5 \not< 1 \rightarrow$ **Far**. Small-expensive lands near the metro; big-cheap lands far out — exactly the hunch the arithmetic encodes.

Basically, ... A model = a compact equation that captures a trend (price rises with area, falls with distance) while ignoring the exceptions. Predictive models answer “what’s the value for this new input?”; probabilistic models answer “how likely is this configuration?”. ML’s job is to *find* such equations from data instead of a human writing them by hand.

22.5 Learning algorithms and the ML task, precisely

Def. A **learning algorithm** = $\text{Data} \rightarrow \text{Models}$. It does not invent a model from nothing; it **chooses from a collection of models with the same structure but different parameters**, using the data to pick the “best” one.

The lecture’s example. Fix the structure *before* seeing any data:

$$\text{Price} = a \times (\text{area}) + b \times (\# \text{ rooms}) + c \times (\text{distance to metro}).$$

The numbers a, b, c are the **parameters** — every choice of (a, b, c) is a different model in the collection, all sharing the same structure. The learning algorithm looks at the data and decides the values of a, b, c .

The ML task, revisited (§22.2’s diagram, now with names). The *tool* is the **model** (e.g. the price-prediction equation). The *tool design* is the **learning algorithm**: the human supplies only the broad blueprint (the structure, e.g. “price is linear in area, rooms, distance”), and the algorithm combines it with past data to construct the actual model. At prediction time, the model converts a new input into an output.

- i) **Structure is the human's job.** “Price depends linearly on area, rooms, distance” — chosen by human intuition *before* any data is seen.
- ii) **Parameters are the data's job.** The exact a, b, c — chosen by the learning algorithm *from* the data.
- iii) **The collection is the hypothesis space.** The set of all models the algorithm is allowed to pick from (all linear models, all sign-of-linear models, ...). The algorithm never leaves this collection; if the truth isn't in it, the best it can do is the closest member. (Choosing *which* collection — model selection — is §22.10's validation data.)

Basically, ... The learning algorithm is a very disciplined shopper: the human decides the shop (“all straight-line price rules”), the data decides which item to buy (the specific a, b, c that best explains the houses seen so far). Everything in Chapters 23–25 is this pattern with the details filled in: what collection, what “best” means, and how to find it.

22.6 Notation for the rest of Part III

The regression lecture fixes notation once, used through Chapters 23–25:

- i) $\mathbb{R}$ = real numbers; $\mathbb{R}_+$ = positive reals; $\mathbb{R}^d$ = d -dimensional vectors of reals.
- ii) $\mathbf{x}$ (the lecture writes x) = a vector; x_j = its j -th coordinate; $\|\mathbf{x}\|$ = Euclidean length, $\|\mathbf{x}\|^2 = x_1^2 + x_2^2 + \dots + x_d^2$.
- iii) $\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^n$ = a collection of n vectors — **superscript** indexes the data point, **subscript** the coordinate. So x_j^i = the j -th coordinate of the i -th vector (eg: if $\mathbf{x}^3 = (7, 7, 8)$ then $x_2^3 = 7$). The lecture warns this collides with powers — it writes $(x^1)_2$ (parenthesized) for the *square of the first coordinate* of the vector x , versus x_1^2 for the *first coordinate of the second vector*; read from context.
- iv) $\mathbf{1}(\cdot)$ = the **indicator**: $\mathbf{1}(\text{predicate}) = 1$ if true, 0 if false. Eg: $\mathbf{1}(2 \text{ is even}) = 1$, $\mathbf{1}(2 \text{ is odd}) = 0$ — the standard device for turning English (“was it classified correctly?”) into math.

22.7 Supervised learning: curve-fitting with answers

Def. Supervised learning = learning from **labelled** data. You are given

$$\{(\mathbf{x}^1, y^1), (\mathbf{x}^2, y^2), \dots, (\mathbf{x}^n, y^n)\},$$

each $\mathbf{x}^i \in \mathbb{R}^d$ an **instance** (the input vector), each y^i its **label** (the answer). Find a model f such that $f(\mathbf{x}^i)$ is “close” to y^i .

The extreme simplification (the lecture's). Supervised learning = **curve-fitting**. You have points; you fit a curve through them so the curve passes as close to the points as possible. Everything that makes it harder than school curve-fitting — high dimensions, the choice of “close”, generalizing to new points — is detail on this picture.

The two supervised tasks differ only in what the labels y^i can be:

- i) **Regression** (§22.8): $y^i \in \mathbb{R}$ — labels are real numbers (prices, temperatures). Model: $f: \mathbb{R}^d \rightarrow \mathbb{R}$.
- ii) **Classification** (§22.9): $y^i \in \{+1, -1\}$ — labels are discrete classes (spam/not-spam, close/far). Model: $f: \mathbb{R}^d \rightarrow \{+1, -1\}$.

Note (“supervised” because...). The labels supervise: they tell the algorithm what the right answer was for each training instance, so it can measure how wrong its curve is. Unsupervised learning (§22.11) has no labels — no answers to check against.

22.8 Regression: predicting a real number

Setup. Training data $\{(\mathbf{x}^i, y^i)\}$ with $\mathbf{x}^i \in \mathbb{R}^d$, $y^i \in \mathbb{R}$ (eg: $\mathbf{x}^i$ = (rooms, area, distance), y^i = price). The algorithm outputs $f: \mathbb{R}^d \rightarrow \mathbb{R}$.

Def (squared loss). How good is a candidate f ? Measure the deviation at each point and square it:

$$L(f) = \frac{1}{n} \sum_{i=1}^n (f(\mathbf{x}^i) - y^i)^2.$$

Squaring keeps the loss non-negative and punishes overshoot and undershoot equally. $L(f) = 0$ iff $f(\mathbf{x}^i) = y^i$ for all i — the smallest possible loss. The learning algorithm's job: find an f with small $L(f)$.

The linear parameterization. The most common structure: f is linear in its input,

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = \sum_{j=1}^d w_j x_j + b,$$

with **parameters** $w_1, \dots, w_d, b$. For houses: $f = w_1(\text{rooms}) + w_2(\text{area}) + w_3(\text{distance}) + b$. Every choice of (w_1, w_2, w_3, b) is one model in the collection; the algorithm picks the best one (§22.5). *Finding* it from an infinite collection is Chapter 23's job — here we just *evaluate* candidates.

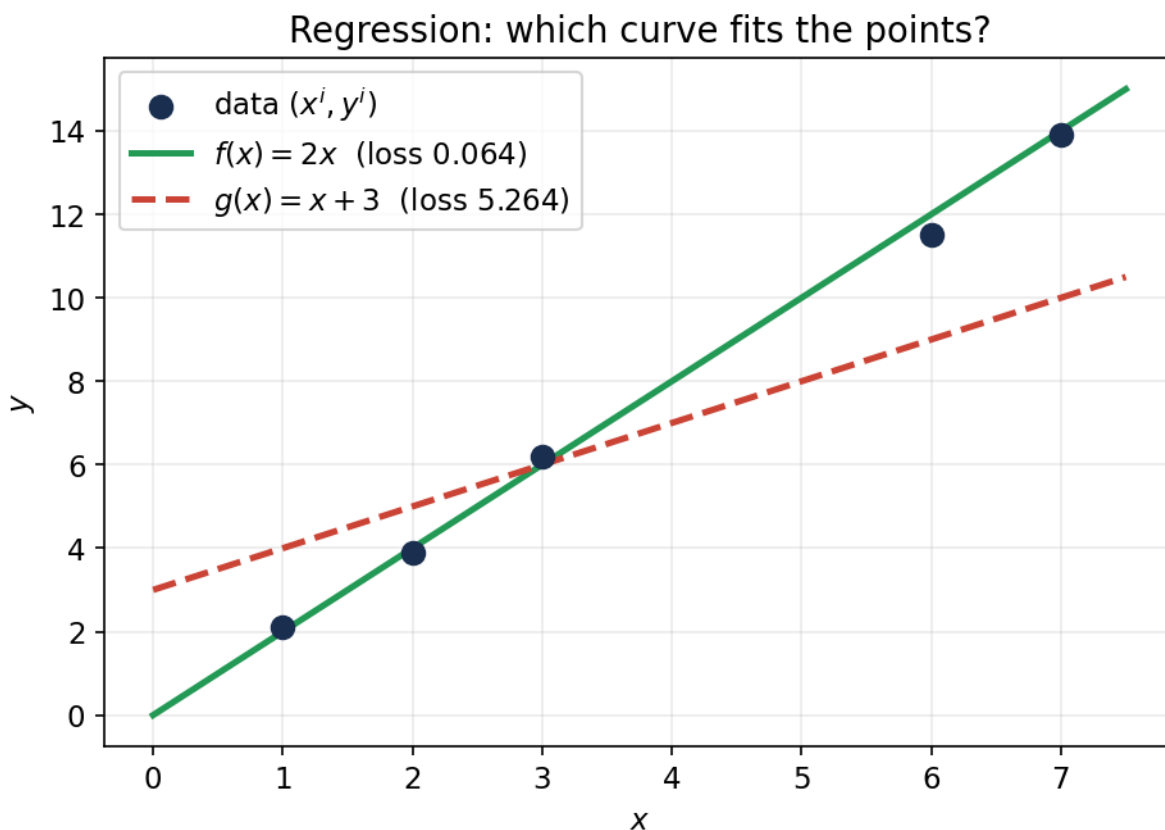


Figure 25: Scatter plot of the five points $(1, 2.1), (2, 3.9), (3, 6.2), (6, 11.5), (7, 13.9)$. A green line $f(x) = 2x$ passes close to all points; a dashed red line $g(x) = x + 3$ misses them badly.

eg 7 (the lecture's 1-D illustration, worked fully). Data ($d = 1$, written as vectors for form's sake): $(x^1, y^1) = (1, 2.1), (2, 3.9), (3, 6.2), (6, 11.5), (7, 13.9)$. Two candidate models: $f(x) = 2x_1$ and $g(x) = x_1 + 3$. Evaluate:

$$f : 2, 4, 6, 12, 14 \quad g : 4, 5, 6, 9, 10.$$

Loss of f :

$$L(f) = \frac{1}{5} [(2 - 2.1)^2 + (4 - 3.9)^2 + (6 - 6.2)^2 + (12 - 11.5)^2 + (14 - 13.9)^2]$$

$$= \frac{1}{5}[0.01 + 0.01 + 0.04 + 0.25 + 0.01] = \frac{0.32}{5} = \boxed{0.064}.$$

Loss of g :

$$\begin{aligned} L(g) &= \frac{1}{5}[(4 - 2.1)^2 + (5 - 3.9)^2 + (6 - 6.2)^2 + (9 - 11.5)^2 + (10 - 13.9)^2] \\ &= \frac{1}{5}[3.61 + 1.21 + 0.04 + 6.25 + 15.21] = \frac{26.32}{5} = \boxed{5.264}. \end{aligned}$$

$L(f) \ll L(g)$ — the algorithm prefers f . The picture says why: f threads through the points; g sits far below most of them. (A real algorithm wouldn't choose between two models but among an infinite collection — the principle is identical.)

eg 8 (the lecture's house-price illustration, worked fully). Same house table as §22.3. Candidates: $f(\mathbf{x}) = 2(\text{rooms}) - 0.5(\text{distance})$ and $g(\mathbf{x}) = (\text{rooms}) + 2(\text{distance})$. Predictions of f : $6 - 0.95 = 5.05$; $4 - 1.05 = 2.95$; $8 - 1.4 = 6.6$; $10 - 0.45 = 9.55$; $10 - 1.55 = 8.45$; $8 - 0.8 = 7.2$. Against true prices (5.0, 3.2, 6.6, 9.8, 8.5, 6.9):

$$L(f) = \frac{1}{6}[0.05^2 + (-0.25)^2 + 0^2 + (-0.25)^2 + (-0.05)^2 + 0.3^2] = \frac{0.22}{6} \approx \boxed{0.0367}.$$

Predictions of g : $3 + 3.8 = 6.8$; $2 + 4.2 = 6.2$; $4 + 5.6 = 9.6$; $5 + 1.8 = 6.8$; $5 + 6.2 = 11.2$; $4 + 3.2 = 7.2$:

$$L(g) = \frac{1}{6}[1.8^2 + 3.0^2 + 3.0^2 + (-3.0)^2 + 2.7^2 + 0.3^2] = \frac{37.62}{6} = \boxed{6.27}.$$

f wins decisively — and notice what f says: price rises with rooms and falls with distance to the metro, the sensible trend from §22.4. The loss picked the model a human would also call reasonable.

Basically, ... Regression = “draw the best curve through the points, where ‘best’ = smallest average squared miss”. The lecture's two demos are the whole idea in miniature: compute each candidate's misses, square, average, pick the smaller number. Chapter 23 automates the picking over all straight lines (and curves) at once.

22.9 Classification: predicting a discrete label

Setup. Same instances $\mathbf{x}^i \in \mathbb{R}^d$, but now the labels are just two values: $y^i \in \{+1, -1\}$. The algorithm outputs $f: \mathbb{R}^d \rightarrow \{+1, -1\}$. (The lecture's example: from a house's area and price, predict whether its room count is > 3 or ≤ 3 .)

Def (0–1 / misclassification loss). The natural loss: the **fraction of misclassified instances**,

$$L(f) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(f(\mathbf{x}^i) \neq y^i),$$

using the indicator from §22.6. Each point contributes 1 if wrong, 0 if right; divide by n . Perfect classification $L(f) = 0$.

The linear separator. A linear $\mathbf{w}^T \mathbf{x} + b$ outputs reals, not ± 1 — so put a sign on top:

$$\boxed{f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)} \quad (\text{a linear separator}).$$

Not the only possible classifier, but the canonical one: it carves the input space into a $+1$ region and a -1 region with a straight boundary ($\mathbf{w}^T \mathbf{x} + b = 0$).

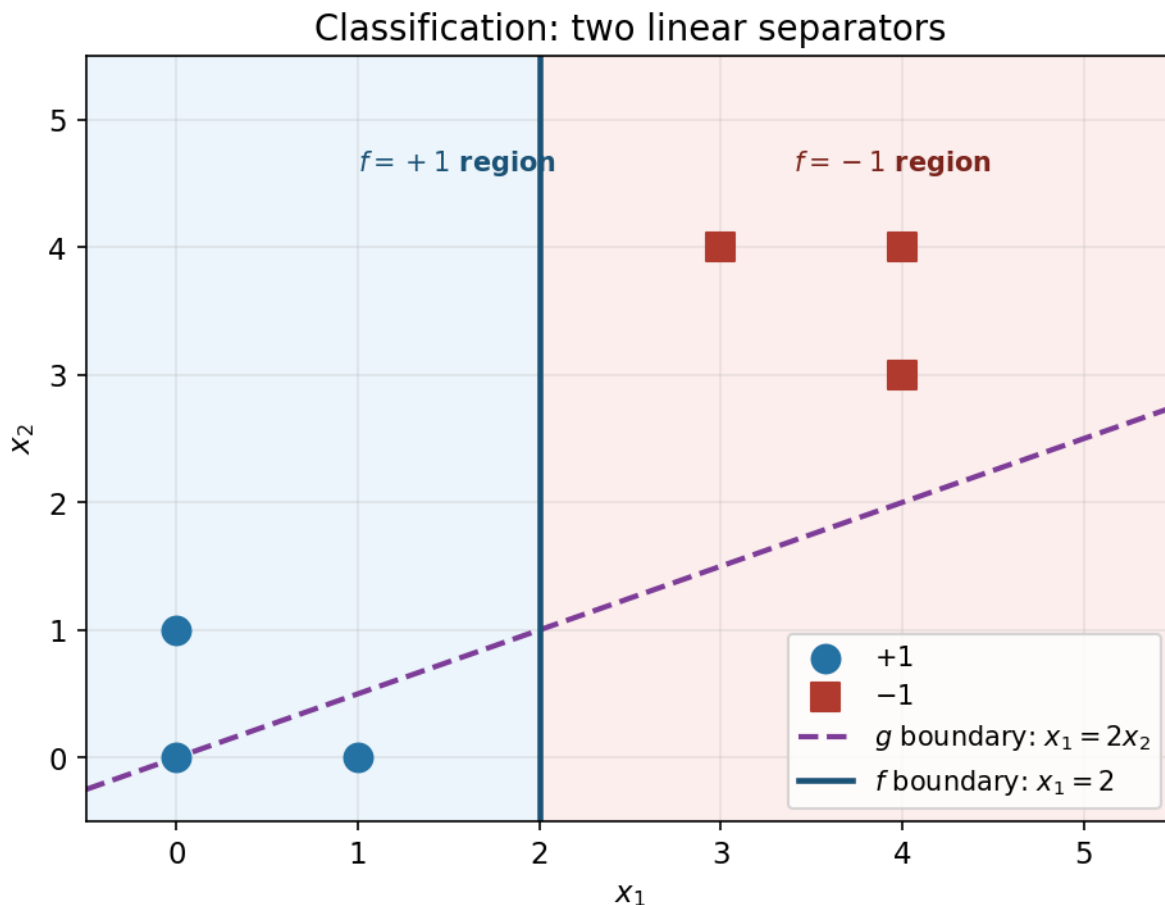


Figure 26: Six points in the plane: three blue circles (+1) at $(0,0), (1,0), (0,1)$ and three red squares (-1) at $(4,4), (3,4), (4,3)$. A vertical blue line at $x_1=2$ separates them perfectly (f 's boundary, regions shaded); a dashed purple line $x_1=2x_2$ through the origin misclassifies one point (g 's boundary).

eg 9 (the lecture's 2-D illustration, worked fully). Points: $\mathbf{x}^1 = (0,0), \mathbf{x}^2 = (1,0), \mathbf{x}^3 = (0,1)$ labelled +1; $\mathbf{x}^4 = (4,4), \mathbf{x}^5 = (3,4), \mathbf{x}^6 = (4,3)$ labelled -1. Candidates: $f(\mathbf{x}) = \text{sign}(2-x_1)$ and $g(\mathbf{x}) = \text{sign}(x_1-2x_2)$ (convention: $\text{sign}(0) = +1$).

$$f : \text{sign}(2), \text{sign}(1), \text{sign}(2), \text{sign}(-2), \text{sign}(-1), \text{sign}(-2) = (+1, +1, +1, -1, -1, -1).$$

Matches every label $\rightarrow \boxed{L(f) = 0}$. Geometrically: f predicts +1 left of the line $x_1 = 2$ and -1 right of it — all +1 points lie left, all -1 points right.

$$g : \text{sign}(0), \text{sign}(1), \text{sign}(-2), \text{sign}(-4), \text{sign}(-5), \text{sign}(-2) = (+1, +1, -1, -1, -1, -1).$$

The third point $\mathbf{x}^3 = (0,1)$ is truly +1 but g says -1 — one mistake $\rightarrow \boxed{L(g) = 1/6}$. The algorithm prefers f . (A real algorithm searches all separators, not just two.)

eg 10 (the lecture's house-rooms illustration, worked fully). Encode rooms ≤ 3 as -1, rooms > 3 as +1. From §22.3's table the true labels are $(-1, -1, +1, +1, +1, +1)$. Three candidates:

$$f(\mathbf{x}) = \text{sign}(\text{area} - 10), \quad g(\mathbf{x}) = \text{sign}(\text{price} - 6), \quad h(\mathbf{x}) = \text{sign}(\text{price} - 9).$$

f says “area above 10 (hundred sq.ft) more than 3 rooms”; g, h threshold the price instead. Predictions:

$$f : (-1, -1, +1, +1, +1, +1) \quad (\text{areas } 9, 7, 12, 16, 15, 11),$$

$g : (-1, -1, +1, +1, +1, +1)$ (prices 5.0, 3.2, 6.6, 9.8, 8.5, 6.9),

$h : (-1, -1, -1, +1, -1, -1)$ (only house 4 clears price 9).

$L(f) = 0$, $L(g) = 0$, $L(h) = 3/6 = 1/2$ (h misses houses 3, 5, 6). The algorithm rejects h — but f and g tie at zero loss, and on training data alone it cannot choose between them. That tie is a first hint that training loss isn't the whole story.

Basically, ... Classification = regression's discrete twin: labels are ± 1 , “close” becomes “exactly right”, and the loss is just the fraction you got wrong. The linear separator is one straight cut through the data — sign of a linear function. Compare candidates by counting mistakes; fewest mistakes wins.

22.10 Training loss is not the goal: test data

The trap. A model can score $L(f) = 0$ on the training data and still be useless. The lecture's counterexample, on eg 9's data:

$$f(\mathbf{x}) = \begin{cases} +1 & \mathbf{x} \in \{(0, 0), (1, 0), (0, 1)\}, \\ -1 & \text{otherwise.} \end{cases}$$

It memorizes the three positive points and says -1 everywhere else — zero training loss. But a new point near the origin, say $(0.1, 0.1)$, is clearly $+1$ -ish by any sane logic, and this f calls it -1 . It learned the *data*, not the *principle*.

Def. Evaluate on **test data** — data *not* used in training. Training picks the model; testing judges it. Judging on the training set gives an overoptimistic estimate, because the model has already seen those points.

The three-way split. i) **Training data** — pick the best model *within* a collection (eg: the best a, b, c among all linear models). ii) **Validation data** — pick the best *collection* (eg: linear in area/rooms/distance vs. a fancier formula like $a \cdot \text{area/rooms} + b \cdot \text{distance}^2$). Choosing the collection is **model selection**, done by humans, guided by held-out validation data. iii) **Test data** — final evaluation of the learned model, on data it never saw during training *or* selection. (Eg: learn on 1990–2020 house prices; judge on 2021 houses — predicting a 1990 price you already saw proves nothing.)

Note. This chapter's worked examples all picked among a few hand-given candidates on the training data — the lecture is explicit that this is a *simplified* learning algorithm. Real algorithms search the whole infinite collection (§22.5), and real evaluation always holds out test data. Every chapter from here on inherits this discipline: fit on train, select on validation, report on test.

Basically, ... “Zero mistakes on the practice exam” “knows the subject”. The memorizer aces the practice set by rote and fails every new question. ML's report card is always the *unseen* test set — that's the number that counts.

22.11 Unsupervised learning: understanding data, no labels

Def. **Unsupervised learning** works on **unlabelled** data — just $\{\mathbf{x}^1, \dots, \mathbf{x}^n\}$, no y^i . Goal, vaguely: build models that **compress, explain and group** the data. The lecture's one-liner: unsupervised learning is “understanding data”.

- i) **No answers to check.** Supervised learning measures $f(\mathbf{x}^i)$ against the known y^i ; unsupervised has no y^i , so “loss” has to be redefined per task (§§22.12–22.13 each invent their own).
- ii) **Usually a preprocessing step, not the end goal.** Its outputs become valuable after human interpretation or inside another pipeline. The lecture's example: a Coca-Cola marketing manager faces a million tweets a week — unreadable. Group them into 10 clusters (selfies-with-Coke, co-branding, paid promotions, ...); *then* a human reads the 10 groups and briefs the boss. The grouping is the algorithm's job; giving the groups meaning is the human's.
- iii) **The two unsupervised tasks of this course: dimensionality reduction** (§22.12 — compress) and **density estimation** (§22.13 — score/explain).

Basically, ... Supervised = “here are questions with answers; learn the pattern”. Unsupervised = “here is a mountain of data with no answers; find its structure”. Nobody grades unsupervised output directly — its value shows up downstream, when the compressed or grouped data makes the real task easy.

22.12 Dimensionality reduction: compress, then reconstruct

The problem. Data lives in $\mathbb{R}^D$ with D huge. The lecture’s example: a million people $\times$ a million gene-expression levels — a $10^6 \times 10^6$ matrix nobody can transmit or store. Goal: represent each person with just (say) 100 numbers.

Def (encoder, decoder). Unlike every task so far, the algorithm learns **two** models: - **Encoder** $f : \mathbb{R}^D \rightarrow \mathbb{R}^{d'}$ — compresses ($d' \ll D$ typically); - **Decoder** $g : \mathbb{R}^{d'} \rightarrow \mathbb{R}^D$ — tries to undo the encoder. Goal: $\boxed{g(f(\mathbf{x}^i)) \approx \mathbf{x}^i}$
— encode, decode, get (almost) the original back.

Def (reconstruction loss). Measure the failure with the squared norm, averaged:

$$\boxed{L(f, g) = \frac{1}{n} \sum_{i=1}^n \|g(f(\mathbf{x}^i)) - \mathbf{x}^i\|^2}.$$

Zero loss = perfect round-trip; the algorithm hunts for the encoder–decoder pair minimizing it. *How* to search the infinite space of pairs is Chapter 24’s job (PCA).

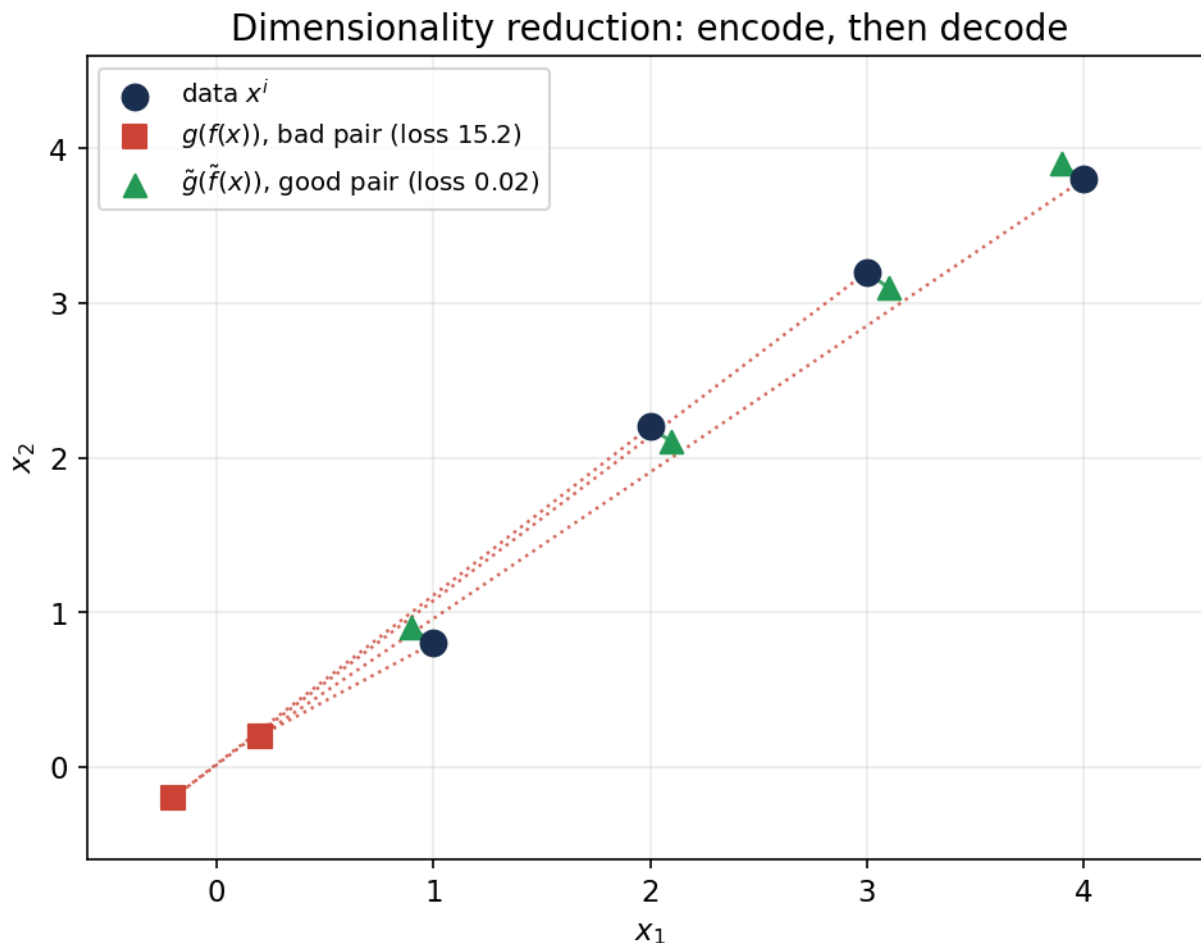


Figure 27: Four dark data points near the diagonal from $(1, 0.8)$ to $(4, 3.8)$. Red squares (bad pair reconstructions at $(0.2, 0.2)$ and $(-0.2, -0.2)$) sit far from the data with long dotted connectors; green triangles (good pair reconstructions at $(0.9, 0.9), (2.1, 2.1), (3.1, 3.1), (3.9, 3.9)$) sit almost on top of the data.

eg 11 (the lecture's illustration, worked fully). $D = 2$, $d' = 1$, $n = 4$: $\mathbf{x}^1 = (1, 0.8)$, $\mathbf{x}^2 = (2, 2.2)$, $\mathbf{x}^3 = (3, 3.2)$, $\mathbf{x}^4 = (4, 3.8)$. Two encoder-decoder pairs: - Bad pair: $f(\mathbf{x}) = x_1 - x_2$, $g(u) = (u, u)$. Encodings: $0.2, -0.2, -0.2, 0.2$. Reconstructions: $(0.2, 0.2), (-0.2, -0.2), (-0.2, -0.2), (0.2, 0.2)$ — two *different* inputs collapse to the same output; nothing is recovered.

$$L = \frac{1}{4}[0.8^2 + 0.6^2 + 2.2^2 + 2.4^2 + 3.2^2 + 3.4^2 + 3.8^2 + 3.6^2] = \frac{60.8}{4} = \boxed{15.2}.$$

- Good pair: $\tilde{f}(\mathbf{x}) = (x_1 + x_2)/2$, $\tilde{g}(u) = (u, u)$. Encodings: $0.9, 2.1, 3.1, 3.9$. Reconstructions: $(0.9, 0.9), (2.1, 2.1), (3.1, 3.1), (3.9, 3.9)$ — each hugs its original.

$$L = \frac{1}{4}[0.1^2 + 0.1^2] \times 4 = \frac{0.08}{4} = \boxed{0.02}.$$

15.2 vs 0.02: the loss sees what the picture shows. The points lie near the diagonal $x_1 \approx x_2$, so averaging the coordinates keeps almost all the information — the encoder discovered the data's one-dimensional shape.

Note. The lecture presents this as “choose the better of two pairs” — again the simplified-algorithm trick. The real algorithm searches all encoders/decoders; Chapter 24 does it optimally for the linear case.

Basically, ... Dimensionality reduction = “squeeze each data point through a narrow pipe, then try to rebuild it; the best pipe loses the least”. The loss is just reconstruction error — how far the round-trip lands from where it started.

22.13 Density estimation: scoring reality with probabilities

The problem. Output a **probabilistic model** (§22.4) — a function scoring *every* possible configuration. The lecture’s example: build a robot that tweets like Mr. Chopra (wisdomofchopra.com — “profound sounding words put together in a random order”). A tweet is a 128-character string over 27 symbols (26 lowercase + space), so the model must score all 27^{128} possibilities — high for Chopra-like tweets, low for random digit strings.

Def. Data $\{\mathbf{x}^1, \dots, \mathbf{x}^n\}$, $\mathbf{x}^i \in \mathbb{R}^d$. Learn a **probability mapping**

$$P : \mathbb{R}^d \rightarrow \mathbb{R}_+,$$

non-negative everywhere and **summing (or integrating) to 1** over all configurations. Goal: $P(\mathbf{x})$ large if $\mathbf{x}$ looks like the data, low otherwise. The “sums to 1” is the catch — you can’t give everything a high score; probability mass spent in one place is mass denied elsewhere.

Def (negative log-likelihood loss). Same object as §20.2’s risk, now as a loss:

$$L(P) = - \sum_{i=1}^n \log P(\mathbf{x}^i).$$

Each data point wants $P(\mathbf{x}^i)$ as large as possible (so $-\log P(\mathbf{x}^i)$ is as small as possible); sum over the data. Small loss = the model assigns high probability to what was actually observed.

eg 12 (the lecture’s uniform-model illustration, worked fully). $d = 1$, data $\{2.3, 2.7, 4.6, 4.9\}$ (the transcript once misprints 2.7 as 2.6 — the deck’s figure shows 2.7). Three candidate probability models (all integrate to 1):

$$P_1(x) = \frac{1}{10} \mathbf{1}(0 \leq x \leq 10), \quad P_2(x) = \frac{1}{5} \mathbf{1}(0 \leq x \leq 5), \quad P_3(x) = \frac{1}{5} \mathbf{1}(3 \leq x \leq 8).$$

Scores on the four points: P_1 : $\frac{1}{10}$ each; P_2 : $\frac{1}{5}$ each; P_3 : 0, 0, $\frac{1}{5}$, $\frac{1}{5}$.

$$L(P_1) = -4 \log \frac{1}{10} = 4 \ln 10 \approx \boxed{9.21}, \quad L(P_2) = -4 \log \frac{1}{5} = 4 \ln 5 \approx \boxed{6.44},$$

$$L(P_3) = -\log 0 - \dots = \boxed{\infty}.$$

P_3 assigns zero probability to observed data — infinitely bad, instantly disqualified. Between the survivors, P_2 beats P_1 : “the data came from Uniform[0, 5]” concentrates its unit mass where the data actually is, instead of spreading it over [0, 10]. The loss rewards putting mass on the data — the whole game of density estimation in one line.

Note (Gaussian mixtures — a pointer, not a lesson). The lecture’s second illustration compares two Gaussian mixture models on 9 planar points — one with centers (0, 0), (4, 1), (7, 3), another with centers (5, 5), (8, 9), (−1, −2) — and asks which explains the data better (the first, by inspection: its centers sit on the visible clumps). The lecture deliberately skips the numerics (“I am not going to explain what the Gaussian mixture model is right now”). So does this chapter: GMMs and how to fit them (EM) are Chapter 25. The takeaway here is only the *framing*: density estimation = pick the probability model with the smallest negative log-likelihood.

Basically, ... Density estimation = “learn a scoring function over all possible inputs, with total score fixed at 1, so the observed data scores as high as possible”. The Chopra-tweet generator is the mascot: feed it real tweets, get back a machine that scores (and can generate) Chopra-like text. The loss is negative log-likelihood — the same object §20.2 minimized for estimation.

22.14 How it all connects — and what comes next

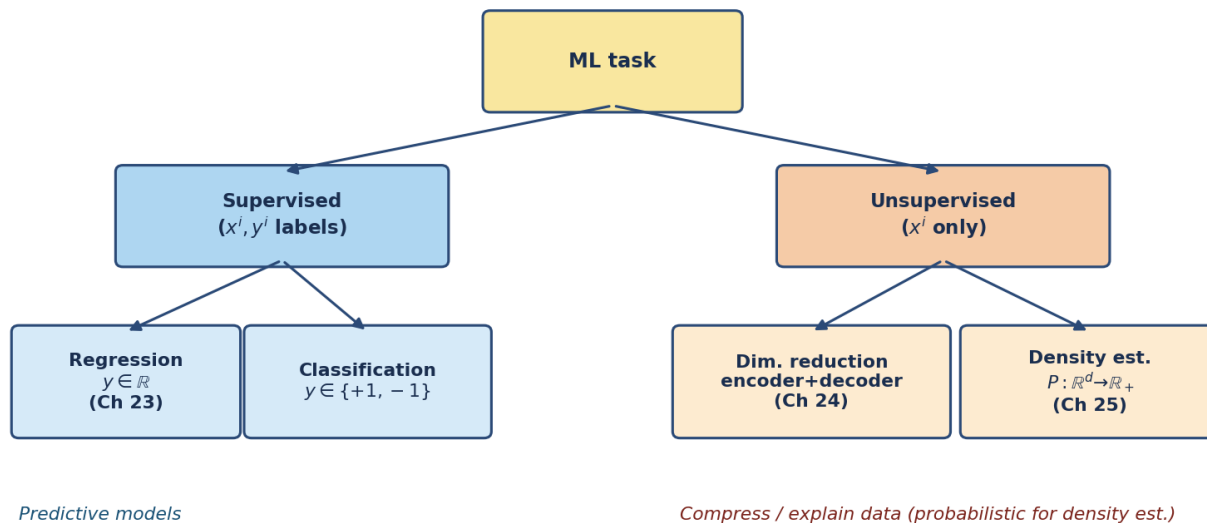


Figure 28: Taxonomy diagram: ML task splits into Supervised ($\hat{x}^i, \hat{y}^i$ labels) and Unsupervised ($\hat{x}^i$ only). Supervised splits into Regression (y in $\mathbb{R}$, Ch 23) and Classification (y in $\{+1, -1\}$). Unsupervised splits into Dimensionality reduction (encoder+decoder, Ch 24) and Density estimation ($P: \mathbb{R}^d \rightarrow \mathbb{R}_+$, Ch 25).

- i) **The map.** Every ML task in this course is one of four things: supervised + real labels (regression), supervised + discrete labels (classification), unsupervised + compression (dimensionality reduction), unsupervised + scoring (density estimation). Regression and classification build *predictive* models; dimensionality reduction compresses; density estimation builds *probabilistic* models.
- ii) **The shared skeleton.** All four follow §22.5’s pattern: fix a collection of models (structure), use data to pick parameters, judge candidates with a loss — squared loss (§22.8), misclassification fraction (§22.9), reconstruction error (§22.12), negative log-likelihood (§22.13). Only the model family and the loss change.
- iii) **The shared discipline.** Fit on training data, choose the model family on validation data, report on test data (§22.10). Zero training loss can be pure memorization.
- iv) **What Part III fills in.** This chapter framed; the next three chapters compute: Chapter 23 finds the best regression model over *all* linear (and polynomial) models — and §20.8 will explain why squared loss is the right choice when the noise is Gaussian. Chapter 24 finds the optimal linear encoder–decoder (PCA: the directions of maximum variance). Chapter 25 fits Gaussian mixtures with the EM algorithm — the “entire parametric family” version of eg 12’s multiple choice.

Part III, in one paragraph (the opener’s version). Part II built the probability toolkit — distributions, expectation, estimation, and the limit theorems that say averages concentrate (§21.5) and fluctuations go normal (§21.6). Part III spends it: machine learning is what happens when a learning algorithm, armed with data and a loss function, searches a model collection for the member that best explains the data — then proves itself on data it never saw. Regression first.

Problem set

1. State the lecture’s definition of machine learning. Draw the three-level task hierarchy (manual labour → programming → machine learning) and say, for each level, who builds the tool that converts input to output.

2. Task analysis. For each of (a) password verification, (b) face detection, (c) weather prediction: say which level of the hierarchy the task belongs at, and give the lecture's reason (scale/speed/cost, inability to express rules, unknown rules, or "programming already works").
3. From the house table in §22.3 and its metadata: (i) write House 2 as a vector in $\mathbb{R}^4$; (ii) translate it into a sentence like eg 4; (iii) explain why the computer needs the coordinates to be consistent across houses but does not need the metadata.
4. (i) Is "Price = $0.5 \times \text{Area} - \text{Distance}$ " a predictive or a probabilistic model? Why? (ii) Is "the probability that a random tweet was generated by Mr. Chopra" a predictive or a probabilistic model? Why? (iii) Give one use of each.
5. The 1-D regression data of eg 7: $(1, 2.1), (2, 3.9), (3, 6.2), (6, 11.5), (7, 13.9)$. A third candidate $k(x) = 1.5x_1 + 1$. Compute $L(k)$ fully and say whether the learning algorithm prefers $f(x) = 2x_1$, $g(x) = x_1 + 3$, or k .
6. On eg 9's classification data, evaluate the new separator $m(\mathbf{x}) = \text{sign}(x_2 - 1)$ (use $\text{sign}(0) = +1$). Compute $L(m)$ and rank f, g, m by loss.
7. Explain in words why the memorizer of §22.10 (+1 on the three training points, -1 elsewhere) is a bad model despite $L(f) = 0$, and say exactly what data should be used to expose it.
8. On eg 11's dimensionality-reduction data, consider a third pair: $f(\mathbf{x}) = x_1 + x_2$, $g(u) = (u/2, u/2)$. Compute its reconstruction loss fully and compare with the two pairs of eg 11. What do you notice?
9. Density estimation on new data $\{1.0, 2.0\}$: compare $P_2(x) = \frac{1}{5}\mathbf{1}(0 \leq x \leq 5)$ with $P_4(x) = \frac{1}{2}\mathbf{1}(0 \leq x \leq 2)$ via the negative log-likelihood. Which wins, and why does the reasoning match eg 12?
10. Classify each as regression, classification, dimensionality reduction, or density estimation, and name the model type (predictive/probabilistic) and the loss the lecture would use: (a) predicting tomorrow's maximum temperature from today's readings; (b) spam vs not-spam for incoming email; (c) compressing 1000-dimensional sensor readings to 10 numbers per reading; (d) scoring how likely a tweet was written by a given author.

Solutions — Chapter 22. What is ML: data, models, tasks; supervised vs unsupervised

Problem 1

Machine learning = "the study of computer algorithms that improve automatically through experience and by the use of data" (Wikipedia definition, MLF Week 1 lecture 1).

Level	Who builds the tool?
Manual labour	No tool — the human directly converts input to output.
Programming	The human builds the tool (software) that converts input to output.
Machine learning	The human gives only a broad blueprint to a <i>tool design</i> (the learning algorithm); the tool design uses data to construct the tool (the model).

Problem 2

- (a) **Password verification → programming.** Manual labour fails on scale/speed/cost (billions of logins); but a program comparing the input against the password on file is trivial. Since programming works, ML is never considered.
- (b) **Face detection → machine learning.** Manual labour fails on scale (a human drawing boxes on every photo). Programming fails because the rule "which pixel patterns are a face" cannot be expressed in code — humans know faces (a baby learns from examples) but cannot convey the idea to a pixel-seeing computer. ML works: lots of face/non-face images exist, and the algorithm builds the tool from them.

- (c) **Weather prediction → machine learning.** Manual labour fails — humans don't know the rule mapping today's radar map to tomorrow's rain (physics implements it; nobody wrote it down). Programming fails for the same reason: you can't code a rule you don't know. ML works because the rule exists and there is immense historical weather data.

Problem 3

- (i) House 2 as a vector: $\boxed{(2, 7, 2.1, 3.2)} \in \mathbb{R}^4$.
- (ii) In words: a two-bedroom house with 700 sq.ft of area, 2.1 km from the metro, priced at 32 lakhs (3.2 tens of lakhs).
- (iii) The computer never reads the metadata — it only manipulates numbers. What it needs is *consistency*: coordinate 2 must mean “area in 100 sq.ft” for house 2 *and* every other house, otherwise arithmetic across rows is meaningless. The metadata (the sentence) is for the human, to make the data interpretable.

Problem 4

- (i) **Predictive** (regression): given a new house's area and distance, it outputs a predicted price — a real-valued prediction about the future/unseen.
- (ii) **Probabilistic**: it doesn't predict a value for a new input; it *scores* configurations — how likely is this tweet under Chopra's tweet distribution.
- (iii) Uses: the regression model prices houses you've never seen (eg 5: 700 sq.ft at 3 km → 5 lakhs). The probabilistic model powers the Chopra-tweet generator (§22.13): score candidate tweets, keep/generate the high-scoring ones.

Problem 5

Predictions of $k(x) = 1.5x_1 + 1$: 2.5, 4, 5.5, 10, 11.5. Against (2.1, 3.9, 6.2, 11.5, 13.9):

$$\begin{aligned} L(k) &= \frac{1}{5}[(2.5 - 2.1)^2 + (4 - 3.9)^2 + (5.5 - 6.2)^2 + (10 - 11.5)^2 + (11.5 - 13.9)^2] \\ &= \frac{1}{5}[0.16 + 0.01 + 0.49 + 2.25 + 5.76] = \frac{8.67}{5} = \boxed{1.734}. \end{aligned}$$

Ranking: $L(f) = 0.064 < L(k) = 1.734 < L(g) = 5.264$. The algorithm still prefers $f(x) = 2x_1$; k is better than g but worse than f .

Problem 6

$m(\mathbf{x}) = \text{sign}(x_2 - 1)$, true labels (+1, +1, +1, -1, -1, -1):

	$x_2 - 1$	m	truth
(0, 0)	-1	-1	+1 wrong
(1, 0)	-1	-1	+1 wrong
(0, 1)	0	+1	+1 right
(4, 4)	3	+1	-1 wrong
(3, 4)	3	+1	-1 wrong
(4, 3)	2	+1	-1 wrong

Five mistakes out of six: $\boxed{L(m) = 5/6}$. Ranking: $f(0) < g(1/6) < m(5/6)$. (m draws a horizontal cut at $x_2 = 1$; it keeps only (0, 1) on the right side.)

Problem 7

Zero training loss means the model is perfect *on points it has already seen* — and this f achieves it by pure memorization: it hard-codes +1 exactly at the three positive training points and defaults to -1 everywhere else. It learned no principle (e.g. “positives live near the origin”), so on a *new* point like $(0.1, 0.1)$ — which any reasonable rule calls +1 — it predicts -1 .

To expose it, evaluate on **test data**: fresh points drawn the same way but never shown during training. The memorizer will fail there while a genuine separator (like eg 9’s f) succeeds. Training loss measures fit; test loss measures learning.

Problem 8

Encodings $f(\mathbf{x}) = x_1 + x_2$: 1.8, 4.2, 6.2, 7.8. Reconstructions $g(u) = (u/2, u/2)$: $(0.9, 0.9)$, $(2.1, 2.1)$, $(3.1, 3.1)$, $(3.9, 3.9)$ — identical to the good pair’s reconstructions in eg 11.

$$L = \frac{1}{4}[(0.1^2 + 0.1^2) \times 4] = \frac{0.08}{4} = \boxed{0.02},$$

exactly tying eg 11’s good pair, and far below the bad pair’s 15.2.

What you notice: the loss only sees the *round trip* $g \circ f$. Here $g(f(\mathbf{x})) = \tilde{g}(\tilde{f}(\mathbf{x})) = (\frac{x_1+x_2}{2}, \frac{x_1+x_2}{2})$ — the factor of 2 moved from the encoder into the decoder, but the composition (and hence the loss) is unchanged. Different encoder–decoder pairs can be equivalent.

Problem 9

P_2 : scores $\frac{1}{5}, \frac{1}{5}$ on $\{1.0, 2.0\}$:

$$L(P_2) = -\log \frac{1}{5} - \log \frac{1}{5} = 2 \ln 5 \approx \boxed{3.219}.$$

P_4 : scores $\frac{1}{2}, \frac{1}{2}$:

$$L(P_4) = -\log \frac{1}{2} - \log \frac{1}{2} = 2 \ln 2 \approx \boxed{1.386}.$$

P_4 wins. Same reasoning as eg 12: both integrate to 1, but P_4 concentrates its unit mass on $[0, 2]$ where the data actually sits, instead of spreading it thinly over $[0, 5]$. Negative log-likelihood rewards putting probability mass on observed data.

Problem 10

- (a) **Regression** — predictive model; squared loss $L(f) = \frac{1}{n} \sum (f(\mathbf{x}^i) - y^i)^2$. Labels are real numbers (temperatures).
- (b) **Classification** — predictive model; misclassification fraction $L(f) = \frac{1}{n} \sum \mathbf{1}(f(\mathbf{x}^i) \neq y^i)$. Labels are discrete (spam/not-spam).
- (c) **Dimensionality reduction** — compression (neither predictive nor probabilistic in the lecture’s taxonomy); reconstruction loss $L(f, g) = \frac{1}{n} \sum \|g(f(\mathbf{x}^i)) - \mathbf{x}^i\|^2$. No labels at all.
- (d) **Density estimation** — probabilistic model P ; negative log-likelihood $L(P) = -\sum \log P(\mathbf{x}^i)$. No labels; the goal is scoring configurations, not predicting values.

23. Linear and polynomial regression

Chapter 22 promised (§22.14): Chapter 23 finds the best regression model over *all* linear (and polynomial) models — not by comparing two hand-picked candidates, but by solving for the optimum over the entire infinite collection. This chapter is that solution. Everything here comes from the MLF Week 4 lecture “Linear and Polynomial Regression” (Prof. Prashanth L A, IIT Madras). The lecture’s headline idea: solving linear regression by taking derivatives gives *exactly* the system of equations you would get by projecting onto the column space — §5.8’s two routes to the normal equations, now wearing ML clothes. Polynomial regression turns out to need no new machinery at all: transform the features, then run the same machine.

23.1 The setup: data, model, loss

Data. $\{(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n)\}$ with $x^i \in \mathbb{R}^d$, $y^i \in \mathbb{R}$. The superscript indexes the data point, the subscript (when needed) the coordinate — §22.6’s convention, kept through Part III.

Model. A linear function of the input:

$$\boxed{f_\theta(x) = \theta^T x}, \quad \theta \in \mathbb{R}^d.$$

The parameters θ are what the learning algorithm chooses (§22.5).

Note (where is the bias b ?). Chapter 22 wrote the model as $\mathbf{w}^T \mathbf{x} + b$. The lecture folds b into θ : append a constant coordinate 1 to every x (so $x = (1, x_1, \dots, x_{d-1})^T$), and b becomes θ_0 (0-based indexing, used throughout this chapter). It is the same model — an intercept feature is just a feature that never changes.

Loss. The squared-error objective the lecture minimizes:

$$\boxed{L(\theta) = \frac{1}{2} \sum_{i=1}^n (\theta^T x^i - y^i)^2}.$$

Note (the $\frac{1}{2}$ vs the $\frac{1}{n}$). Chapter 22 wrote the loss as $\frac{1}{n} \sum (\cdot)^2$; Sejal’s lecture notes and the deck write $\frac{1}{2} \sum (\cdot)^2$. The constant in front is a bookkeeping choice — $\frac{1}{2}$ cancels the 2 that differentiation produces — and it never changes the minimizer: $\arg \min \frac{1}{2} \sum = \arg \min \frac{1}{n} \sum$. The learning algorithm picks the *model*, not the number.

Basically, ... “How far is the line’s prediction from each answer? Square each miss, add up. Find the line with the smallest total.” That is the entire regression problem — §22.8’s curve-fitting, now with a plan to *find* the best curve.

23.2 The feature matrix

Stack the data into a matrix and the bookkeeping collapses. **Def (feature matrix).** A is the $n \times d$ matrix whose rows are the data points as row vectors:

$$A = \begin{pmatrix} (x^1)^T \\ (x^2)^T \\ \vdots \\ (x^n)^T \end{pmatrix}, \quad Y = \begin{pmatrix} y^1 \\ y^2 \\ \vdots \\ y^n \end{pmatrix}.$$

The lecture calls A the **feature matrix**; Y is the target vector.

- i) $A\theta$ = **all predictions at once**. Row i of $A\theta$ is $(x^i)^T\theta = \theta^T x^i = f_\theta(x^i)$.
- ii) $A\theta - Y$ = **all residuals at once**. A vector of the n misses.
- iii) **The loss, in one line:**

$$L(\theta) = \frac{1}{2}(A\theta - Y)^T(A\theta - Y) = \frac{1}{2}\|A\theta - Y\|^2.$$

Basically, ... A is the data table, Y the answer column. Multiply the table by the parameters to get predictions; subtract the answers to get the misses; the loss is half the squared length of the miss vector.

23.3 Deriving the normal equations (the calculus route)

To minimize L : set the gradient to zero (§10.1's recipe). Write the loss with matrix entries:

$$L(\theta) = \frac{1}{2} \sum_{i=1}^n \left(\sum_{j=1}^d A_{ij}\theta_j - y^i \right)^2.$$

The partial derivative in θ_k (chain rule; the $\frac{1}{2}$ cancels the 2):

$$\frac{\partial L}{\partial \theta_k} = \sum_{i=1}^n \underbrace{\left(\sum_{j=1}^d A_{ij}\theta_j - y^i \right)}_{\text{residual } i} \cdot \underbrace{A_{ik}}_{\text{column } k} = \sum_{i=1}^n A_{ik} (A\theta - Y)_i = [A^T(A\theta - Y)]_k.$$

So the gradient is

$$\nabla_\theta L = A^T(A\theta - Y).$$

Setting it to zero and rearranging:

$$A^T A \theta = A^T Y \quad (\text{the normal equations}).$$

These are d equations in the d unknowns θ — the lecture's entire linear-regression training step in one line.

Note (why this is a minimum, not just a stationary point). The Hessian is $\nabla^2 L = A^T A$. For any z :

$$z^T (A^T A) z = (Az)^T (Az) = \|Az\|^2 \geq 0,$$

so the Hessian is positive semidefinite (§10.8's interrogation). A convex quadratic's stationary point is its global minimum (§12.9) — the normal equations don't just find a candidate, they find *the* optimum.

Basically, ... Differentiate the total miss, set to zero, and the equations tidy themselves into $A^T A \theta = A^T Y$. The $d \times d$ system on the left is all the data, compressed into one matrix.

23.4 The same equations from projections

This is the lecture’s headline, and §5.8 already proved the mechanism:

- i) **Calculus route** (§23.3): differentiate L , set $\nabla L = 0 \rightarrow A^T A \theta = A^T Y$.
- ii) **Geometric route** (§5.7): the predictions $\mathbf{p} = A\hat{\theta}$ are the closest point of the column space $C(A)$ to Y — i.e. the orthogonal projection of Y onto $C(A)$. The error $Y - A\hat{\theta}$ is orthogonal to $C(A)$, which forces $A^T(Y - A\hat{\theta}) = 0$ — the same equations.

The lecture’s point. Two completely different arguments — “minimize the squared miss” and “drop Y perpendicularly onto the column space” — land on the identical system. Least squares is projection; the ML loss is the geometry of §5.

Basically, ... “The best prediction vector is the shadow Y casts on the column space” and “the parameters with the smallest squared error” are the same sentence in two languages.

23.5 The closed-form solution and the full-rank condition

If A has **full column rank** (independent columns — §4.6), then $A^T A$ is invertible and the normal equations have one solution:

$$\hat{\theta} = (A^T A)^{-1} A^T Y.$$

Why full rank of A invertibility of $A^T A$ (the lecture’s homework hint, worked out). It is enough to show the two matrices have the same null space (§4.7):

- i) $Ax = 0 \Rightarrow A^T Ax = A^T 0 = 0$. Trivial direction: $N(A) \subseteq N(A^T A)$.
- ii) $A^T Ax = 0 \Rightarrow x^T A^T Ax = 0 \Rightarrow (Ax)^T (Ax) = \|Ax\|^2 = 0 \Rightarrow Ax = 0$. So $N(A^T A) \subseteq N(A)$.

Same null space same rank (rank-nullity, §4.8). If A has full column rank d , then $A^T A$ is a $d \times d$ matrix of rank d — hence invertible.

Note (dependent columns). If A ’s columns are dependent, the normal equations still *have* solutions — but infinitely many (§5.8 said the same): there are many least-squares $\hat{\theta}$, and the closed form breaks because $(A^T A)^{-1}$ doesn’t exist. §23.10 shows the lecture’s fix.

Basically, ... If every feature says something new (independent columns), there is exactly one best θ , given by one matrix inverse. If two features say the same thing, credit can’t be split uniquely — no single answer.

23.6 Why squared loss is right: the Gaussian-noise story

Why square the misses — why not absolute values, or something else? The lecture gives the probabilistic answer (the full machinery is §20.8; here is the meaning).

The generative model. Suppose nature produces labels as

$$y = \theta^T x + \varepsilon, \quad \varepsilon \sim N(0, 1/\beta),$$

i.e. the truth is linear, plus **Gaussian noise** with precision β (precision = 1/variance). The data $\{(x^i, y^i)\}$ are drawn independently from this model. Maximum likelihood asks: which θ makes the observed data most probable? The log-likelihood (dropping θ -free constants):

$$\ell(\theta) = -\frac{\beta}{2} \sum_{i=1}^n (y^i - \theta^T x^i)^2 + \text{const.}$$

Maximizing $\ell(\theta)$ = minimizing $\frac{\beta}{2} \sum (y^i - \theta^T x^i)^2$ = minimizing the least-squares objective of §23.1 (the $\beta/2$ is another harmless constant).

The lecture’s message. “Least squares regression solves a maximum likelihood estimation problem under a linear model.” The noise model *chooses* the loss: **Gaussian noise squared error**. This is §20.8 made concrete — the bridge from Part II’s probability to Part III’s algorithms.

Basically, ... If you believe the errors are bell-curved, squared error isn’t an arbitrary choice — it’s exactly what “the most likely explanation” works out to be.

23.7 Worked example set

eg 1 (the normal equations on §22.8’s data — beating both hand-picked candidates). Data (with intercept feature): $(x^1, y^1) = (1, 2.1), (2, 3.9), (3, 6.2), (6, 11.5), (7, 13.9)$. Chapter 22 compared $f(x) = 2x$ ($L = 0.064$) with $g(x) = x + 3$ ($L = 5.264$) — both guesses. Now solve over *all* lines $f(x) = \theta_0 + \theta_1 x$.

Step 1 — build A, Y :

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 6 \\ 1 & 7 \end{pmatrix}, \quad Y = \begin{pmatrix} 2.1 \\ 3.9 \\ 6.2 \\ 11.5 \\ 13.9 \end{pmatrix}.$$

Step 2 — compress into the normal equations:

$$A^T A = \begin{pmatrix} 5 & 19 \\ 19 & 99 \end{pmatrix}, \quad A^T Y = \begin{pmatrix} 37.6 \\ 194.8 \end{pmatrix}.$$

(Check a couple of entries: $(A^T A)_{22} = 1 + 4 + 9 + 36 + 49 = 99$; $(A^T Y)_2 = 2.1 + 2(3.9) + 3(6.2) + 6(11.5) + 7(13.9) = 2.1 + 7.8 + 18.6 + 69 + 97.3 = 194.8$.)

Step 3 — invert. $\det(A^T A) = 5 \cdot 99 - 19^2 = 495 - 361 = 134 \neq 0$ — the columns are independent, so the solution is unique (§23.5):

$$(A^T A)^{-1} = \frac{1}{134} \begin{pmatrix} 99 & -19 \\ -19 & 5 \end{pmatrix}, \quad \hat{\theta} = \frac{1}{134} \begin{pmatrix} 99(37.6) - 19(194.8) \\ -19(37.6) + 5(194.8) \end{pmatrix} = \frac{1}{134} \begin{pmatrix} 21.2 \\ 259.6 \end{pmatrix}.$$

$$\boxed{\hat{\theta}_0 \approx 0.158, \quad \hat{\theta}_1 \approx 1.937} \quad \text{best line: } f(x) = 1.937x + 0.158.$$

Step 4 — verify it wins. Predictions: 2.0955, 4.0328, 5.9701, 11.7821, 13.7194. Residuals (pred $- y$): $-0.0045, 0.1328, -0.2299, 0.2821, -0.1806$. Squared sum = 0.1827; in §22.8’s $\frac{1}{n}$ convention:

$$L(\hat{\theta}) = \frac{0.1827}{5} \approx \boxed{0.0365} < 0.064 = L(f = 2x).$$

The algorithm didn’t pick between two candidates — it searched the whole infinite collection and found something strictly better than both.

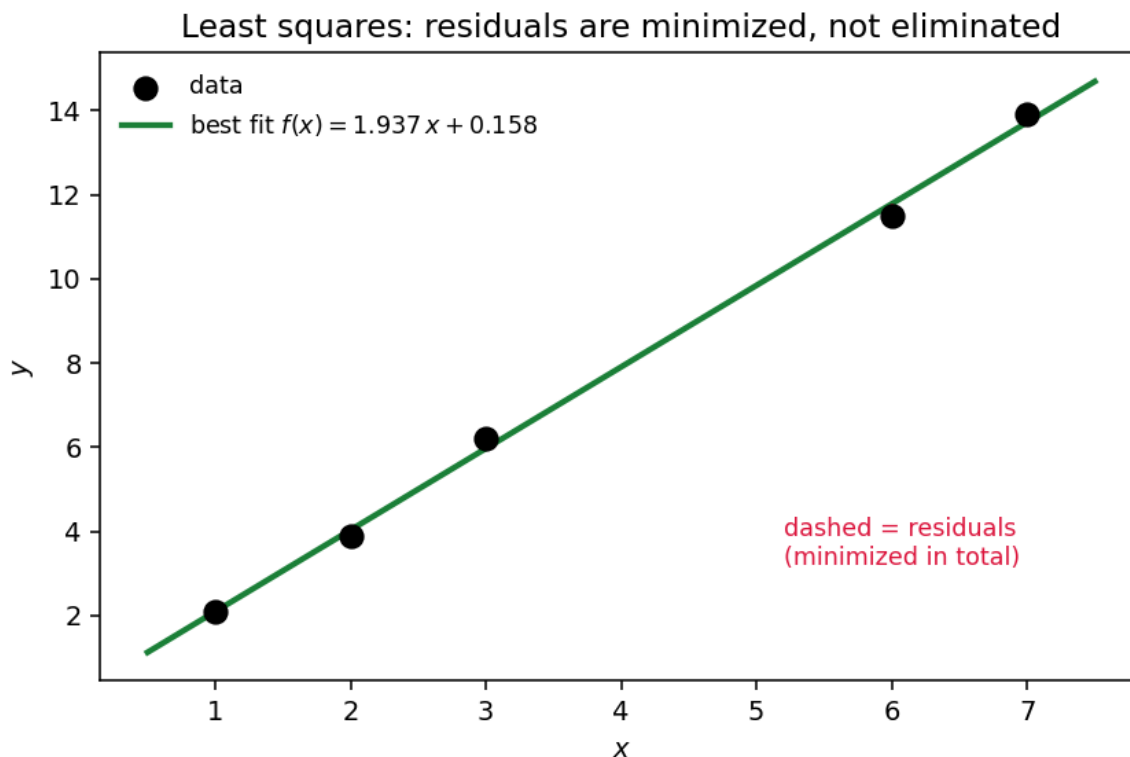


Figure 29: Scatter of the five points $(1,2.1), (2,3.9), (3,6.2), (6,11.5), (7,13.9)$ with the green best-fit line $f(x)=1.937x+0.158$; short dashed red vertical segments join each point to the line, showing the minimized residuals.

eg 2 (invertible vs singular $A^T A$). (i) $A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}$: columns independent,

$$A^T A = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}, \quad \det = 42 - 36 = 6 \neq 0 \Rightarrow (A^T A)^{-1} \text{ exists.}$$

(ii) $B = \begin{pmatrix} 1 & 2 \\ 1 & 2 \\ 1 & 2 \end{pmatrix}$: column 2 = $2 \times$ column 1 (the “feature” says nothing new),

$$B^T B = \begin{pmatrix} 3 & 6 \\ 6 & 12 \end{pmatrix}, \quad \det = 36 - 36 = 0 \Rightarrow \text{no inverse.}$$

In (ii) the normal equations still have infinitely many solutions — every θ with $\theta_0 + 2\theta_1$ fixed gives the same predictions — but no unique $\hat{\theta}$.

eg 3 (quadratic regression by hand). Fit a degree-2 polynomial through $(0, 1), (1, 3), (2, 7)$. Transformed features (§23.8): $\phi(x) = (1, x, x^2)^T$:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad Y = \begin{pmatrix} 1 \\ 3 \\ 7 \end{pmatrix}.$$

Normal equations:

$$A^T A = \begin{pmatrix} 3 & 3 & 5 \\ 3 & 5 & 9 \\ 5 & 9 & 17 \end{pmatrix}, \quad A^T Y = \begin{pmatrix} 11 \\ 17 \\ 31 \end{pmatrix}, \quad \text{i.e.} \quad \begin{cases} 3\theta_0 + 3\theta_1 + 5\theta_2 = 11 \\ 3\theta_0 + 5\theta_1 + 9\theta_2 = 17 \\ 5\theta_0 + 9\theta_1 + 17\theta_2 = 31 \end{cases}.$$

A is full rank (Vandermonde in disguise — distinct x 's), so try $\hat{\theta} = (1, 1, 1)^T$: $3 + 3 + 5 = 11$; $3 + 5 + 9 = 17$; $5 + 9 + 17 = 31$. Unique solution: $\hat{y} = 1 + x + x^2$ — and indeed $1 + 0 + 0 = 1$, $1 + 1 + 1 = 3$, $1 + 2 + 4 = 7$: the parabola threads all three points, residuals 0. **Note.** With $m = n - 1$ (degree 2, three points), the polynomial *can* hit every point exactly — training error 0. Whether that's good is §23.9's question.

eg 4 (the degree ladder: train error falls, test error bends). Five training points near $y = x^2$ (mild noise): $x = (-2, -1, 0, 1, 2)$, $y = (4.2, 0.9, -0.1, 1.2, 3.8)$. Three held-out test points: $(-1.5, 2.3)$, $(0.5, 0.3)$, $(1.5, 2.2)$. Fit polynomials of degree $m = 1, 2, 4$ (§23.8's machine):

degree m	train MSE	test MSE
1	2.863	0.977
2	0.0229	0.0030
4	0.0000	0.0191

Read it: $m = 1$ can't bend (underfits — high train *and* test error). $m = 2$ nails the true shape (test error 0.0030 — best). $m = 4$ has 5 parameters for 5 points, so it interpolates the training data exactly (train error 0) — yet its test error is *worse* than $m = 2$'s: it wiggled between the training points to fit the noise. This is the overfitting curve; §23.9 turns it into a discipline.

23.8 Polynomial regression: curved fits with the same machine

The lecture's generalization of last week's line fitting: instead of a line, fit a degree- m polynomial.

Data. Now one-dimensional: $\{(x^1, y^1), \dots, (x^n, y^n)\}$, $x^i, y^i \in \mathbb{R}$.

Def (transformed features). For each input, build the vector of its powers:

$$\phi(x) = \begin{pmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^m \end{pmatrix}, \quad \hat{y}(x) = \theta^T \phi(x) = \sum_{j=0}^m \theta_j x^j, \quad \theta = (\theta_0, \dots, \theta_m)^T.$$

The procedure. Assemble $\tilde{A}$ with rows $\phi(x^i)^T$ (same Y as before) and solve the *same* normal equations:

$$\tilde{A}^T \tilde{A} \theta = \tilde{A}^T Y.$$

“The only difference with respect to regular linear regression,” the lecture says, “is that we are transforming the features and then performing regression.” No new derivation is needed — §23.3's algebra never assumed the features were raw.

Note (“linear” = linear in θ). A parabola is curved in x but linear in the parameters $(\theta_0, \theta_1, \theta_2)$ — and least squares only ever sees the parameters. That is why the same machine fits curves.

Basically, ... Want a parabola? Square your x 's yourself, hand the lecture's machine the features $[1, x, x^2]$, and it fits the “line” in that taller space. Polynomial regression is linear regression in disguise.

23.9 More capacity is not always better: overfitting, and the validation discipline

Eg 4's table is the whole story in miniature. Raising the degree m always *can* lower the training error (a degree-4 polynomial contains every degree-2 one as a special case — set the extra θ_j to 0). But:

- i) **Training error is optimistic.** The $m = 4$ curve scored 0.0000 on the data it was fitted to and 0.0191 on data it wasn't — the same trap as §22.10's memorizer, now wearing a polynomial costume.
- ii) **Too simple is also bad.** $m = 1$ underfits: it can't express the U-shape, so even its training error (2.863) is poor. Capacity too low — underfitting; capacity too high — overfitting.
- iii) **The discipline (§22.10).** Fit each degree on the *training* data, compare degrees on held-out *validation* data, report the winner's error on a third split — the *test* data. Choosing the degree is **model selection**: §22.10's job (ii) — picking the collection, not the parameters. Eg 4's verdict: $m = 2$ wins on the held-out data — here that data plays the *validation* role (0.0030); a real deployment would then report on a third, untouched *test* split.

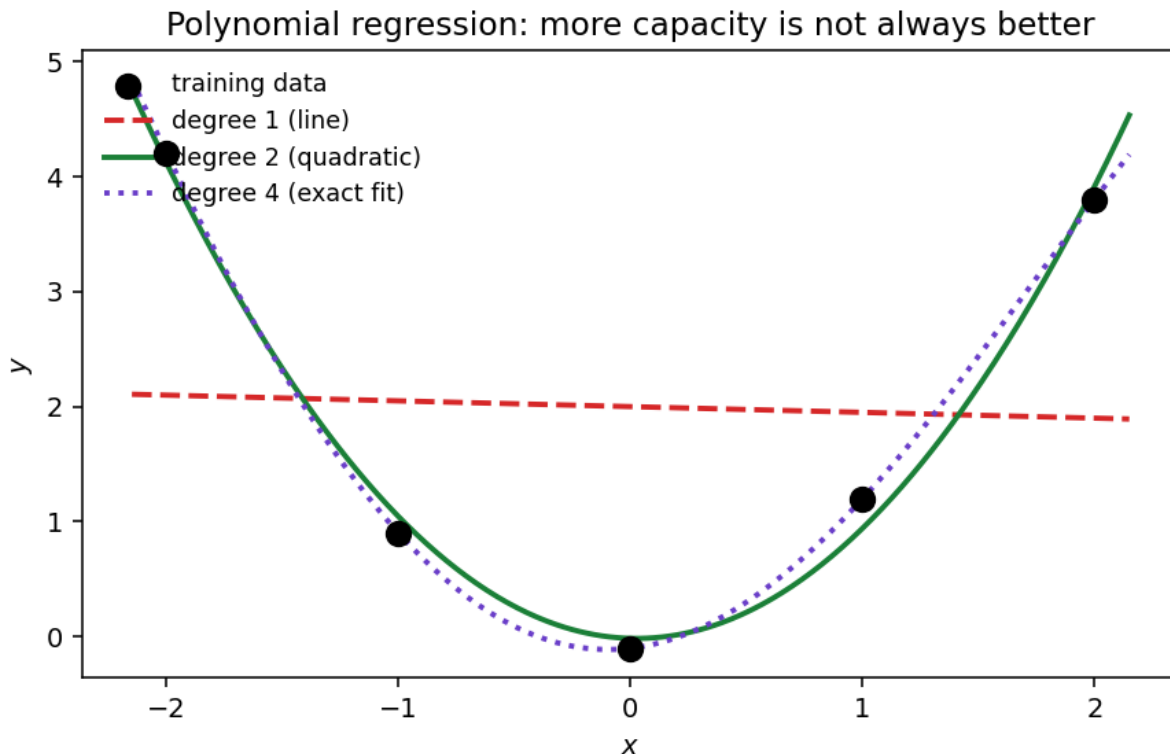


Figure 30: Scatter of the five training points near $y=x^2$. A dashed red line (degree 1) cuts straight through; a solid green parabola (degree 2) tracks the points; a dotted purple degree-4 curve hits every point exactly but wiggles between them.

Note (the lecture's hint). The lecture mentions overfitting only to motivate its closing remark (§23.10): the regularization parameter λ “controls overfitting — too small λ could lead to a lot of overfitting, too large λ leads to underfitting.” There are two cures for an over-ambitious polynomial: choose a smaller m by validation (this section), or penalize large parameters directly (§23.10). The full story of the second cure is Chapter 28.

Basically, ... “The degree-4 curve aced the practice exam and flunked the real one.” Bigger model — better model — judge capacity on data the model never saw during fitting.

23.10 The lecture’s remark: ridge regularization (a pointer)

The lecture ends with “a very short remark” — ridge regression — kept short here too, because it is Chapter 28’s subject. Instead of the plain least-squares objective, minimize the **regularized** version:

$$\bar{L}(\theta) = \frac{1}{2} \sum_{i=1}^n ((x^i)^T \theta - y^i)^2 + \lambda \|\theta\|^2, \quad \lambda > 0.$$

The extra term penalizes large parameters. Repeating §23.3’s derivation, the gradient picks up $2\lambda\theta$, so stationarity gives $(A^T A + 2\lambda I)\theta_{\text{reg}} = A^T Y$ — and since $\lambda > 0$ is just a constant, absorbing the 2 into it recovers the lecture’s compact form:

$$(A^T A + \lambda I) \theta_{\text{reg}} = A^T Y, \quad \hat{\theta}_{\text{reg}} = (A^T A + \lambda I)^{-1} A^T Y.$$

Homework (the lecture’s). Show $A^T A + \lambda I$ is invertible *even when A is not full rank*: for $\lambda > 0$ and $z \neq 0$,

$$z^T (A^T A + \lambda I) z = \|Az\|^2 + \lambda \|z\|^2 > 0,$$

so the matrix is positive definite, hence invertible. The λI term is a purely algebraic fix — it guarantees a unique answer — that also happens to be the overfitting cure the lecture promises. Small λ — near-ordinary least squares (overfitting risk stays); large λ — parameters squeezed toward 0 (underfitting). How to pick λ in practice — by validation, §23.9’s discipline — and the lasso cousin belong to Chapter 28.

Basically, ... Add a penalty for big weights, and two things happen at once: the equations always have a unique solution, and the model stops chasing noise. The dial λ trades overfitting against underfitting.

23.11 Where this goes next

- i) **Projections return.** The geometric reading of §23.4 — predictions as projections onto $C(A)$ — is the seed of Chapter 24 (PCA): there the “column space” is learned, not given.
- ii) **Regularization, properly.** Chapter 28 takes §23.10’s remark and makes it a subject: ridge and lasso, the bias–variance trade-off, and choosing λ by validation.
- iii) **The iterative alternative.** Forming $A^T A$ costs $O(nd^2)$ and inverting it $O(d^3)$ — for enormous data that is too slow. Chapter 10 (§10.7, §10.9) gives the scalable route: gradient descent walks downhill on $L(\theta)$ directly, no inverse needed. Same optimum (§23.3’s convexity guarantees it), different engine.
- iv) **The atom of the neuron.** $f_\theta(x) = \theta^T x$ is the linear core inside every artificial neuron of Chapter 41 — this chapter is where the pattern “parameters $\times$ features, chosen by minimizing a loss” first appears in full.

Problem set

1. For the 1-D data $(x^1, y^1) = (1, 3), (2, 5), (4, 9)$ with an intercept feature: (i) write A and Y ; (ii) for $\theta = (1, 2)^T$ (intercept 1, slope 2), compute $A\theta$ and $A\theta - Y$; (iii) evaluate $L(\theta) = \frac{1}{2} \|A\theta - Y\|^2$. What does your answer say about the fit?
2. Scalar derivation (the lecture’s 1-D case). $L(w, b) = \frac{1}{2} \sum_{i=1}^n (wx^i + b - y^i)^2$. Compute $\partial L / \partial w$ and $\partial L / \partial b$ by hand, set both to zero, and show the result is the 2×2 case of the normal equations.
3. Lecture homework, the hard direction: prove that if $A^T Ax = 0$ then $Ax = 0$ (hence $N(A^T A) \subseteq N(A)$). Hint: multiply by x^T .
4. Fit the best line $f(x) = \theta_0 + \theta_1 x$ through $(0, 0), (1, 2), (2, 3)$ via the normal equations, showing all steps. Compute the minimum squared error $\|A\hat{\theta} - Y\|^2$.
5. Show the normal-equation solution is a *global* minimizer of L : (i) compute the Hessian $\nabla^2 L$; (ii) show it is positive semidefinite; (iii) conclude with §12.9.

6. Fit a quadratic $\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$ through $(0, 2)$, $(1, 1)$, $(2, 2)$: (i) write $\tilde{A}$ and Y ; (ii) write the 3×3 normal equations; (iii) solve them (verify your answer satisfies all three equations); (iv) explain why the residual is exactly 0.
7. True or false, with one-line justification: (i) $A^T A$ is always symmetric. (ii) Full column rank of A guarantees a unique $\hat{\theta}$. (iii) Raising the polynomial degree never increases the training error. (iv) Raising the polynomial degree never increases the test error.
8. For each A , say whether $(A^T A)^{-1}$ exists and give the ML reason: (i) $\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}$; (ii) $\begin{pmatrix} 1 & 2 & 2 \\ 1 & 3 & 3 \\ 1 & 4 & 4 \end{pmatrix}$ (features: intercept, x , and a copy of x); (iii) $\begin{pmatrix} 1 & 0 \\ 2 & 0 \\ 3 & 0 \end{pmatrix}$ (features: x , and an always-0 feature).
9. A colleague fits polynomials of degree 1–4 and reports:

m	train MSE	validation MSE
1	2.90	1.00
2	0.05	0.01
3	0.02	0.03
4	0.00	0.09

- (i) Which degree should she pick, and on what data was that decision made? (ii) What phenomenon does $m = 4$ display? (iii) If she must quote one number as “the model’s error” to a user, which split’s number should it be, and why (§22.10)?
10. Guided MLE→least squares (§20.8, §23.6). Data generated as $y^i = \theta^T x^i + \varepsilon^i$ with $\varepsilon^i \sim N(0, \sigma^2)$ i.i.d.
- (i) Write the likelihood $L(\theta)$ as a product of Gaussian densities. (ii) Take the log and drop θ -free terms. (iii) Show the maximizer equals $\arg \min_{\theta} \sum_i (y^i - \theta^T x^i)^2$.

Solutions — Chapter 23: Linear and polynomial regression

1. (i) Rows $(1, x^i)$:

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 4 \end{pmatrix}, \quad Y = \begin{pmatrix} 3 \\ 5 \\ 9 \end{pmatrix}.$$

(ii) $A\theta = \begin{pmatrix} 1+2(1) \\ 1+2(2) \\ 1+2(4) \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \\ 9 \end{pmatrix} = Y$, so $A\theta - Y = (0, 0, 0)^T$. (iii) $L(\theta) = \frac{1}{2}\|0\|^2 = \boxed{0}$. The line $f(x) = 2x + 1$

passes exactly through all three points — the consistent case (§5.8): no least squares needed, the loss is already at its floor.

2. Differentiate term by term:

$$\frac{\partial L}{\partial w} = \sum_{i=1}^n (wx^i + b - y^i) \cdot x^i, \quad \frac{\partial L}{\partial b} = \sum_{i=1}^n (wx^i + b - y^i) \cdot 1.$$

Set both to zero:

$$\sum_i x^i (wx^i + b) = \sum_i x^i y^i, \quad \sum_i (wx^i + b) = \sum_i y^i.$$

With $A = \begin{pmatrix} 1 & x^1 \\ \vdots & \vdots \\ 1 & x^n \end{pmatrix}$ and $\theta = (b, w)^T$, these are exactly

$$A^T A \theta = A^T Y : \quad \begin{pmatrix} n & \sum x^i \\ \sum x^i & \sum (x^i)^2 \end{pmatrix} \begin{pmatrix} b \\ w \end{pmatrix} = \begin{pmatrix} \sum y^i \\ \sum x^i y^i \end{pmatrix},$$

the 2×2 normal equations. (Check: $(A^T A)_{11} = n$, $(A^T A)_{12} = \sum x^i$, $(A^T A)_{22} = \sum (x^i)^2$; $(A^T Y)_1 = \sum y^i$, $(A^T Y)_2 = \sum x^i y^i$.)

3. Assume $A^T A x = 0$. Multiply on the left by x^T :

$$0 = x^T (A^T A x) = (x^T A^T)(Ax) = (Ax)^T (Ax) = \|Ax\|^2.$$

A vector of length 0 is the zero vector, so $Ax = 0$. Hence every x killed by $A^T A$ is killed by A : $N(A^T A) \subseteq N(A)$. (The reverse inclusion is immediate: $Ax = 0 \Rightarrow A^T A x = A^T 0 = 0$.) The two null spaces are equal, so by rank-nullity (§4.8) A and $A^T A$ have the same rank.

4.

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 \\ 2 \\ 3 \end{pmatrix}, \quad A^T A = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}, \quad A^T Y = \begin{pmatrix} 5 \\ 8 \end{pmatrix}.$$

$\det(A^T A) = 15 - 9 = 6 \neq 0$ (independent columns — §23.5):

$$\hat{\theta} = \frac{1}{6} \begin{pmatrix} 5 & -3 \\ -3 & 3 \end{pmatrix} \begin{pmatrix} 5 \\ 8 \end{pmatrix} = \frac{1}{6} \begin{pmatrix} 25 - 24 \\ -15 + 24 \end{pmatrix} = \begin{pmatrix} 1/6 \\ 3/2 \end{pmatrix}.$$

$$\boxed{f(x) = \frac{3}{2}x + \frac{1}{6}}.$$

Predictions: $\frac{1}{6}, \frac{10}{6}, \frac{19}{6}$; residuals (pred $- y$): $\frac{1}{6}, -\frac{1}{3}, \frac{1}{6}$. Minimum squared error:

$$\|A\hat{\theta} - Y\|^2 = \frac{1}{36} + \frac{4}{36} + \frac{1}{36} = \frac{6}{36} = \boxed{\frac{1}{6}}.$$

(And $Y \notin C(A)$ — no line hits all three points — so this is genuinely the least-squares compromise, §5.8.)

5. (i) From §23.3, $\nabla L = A^T(A\theta - Y)$, which is linear in θ ; differentiating once more:

$$\nabla^2 L = A^T A.$$

(ii) For any z : $z^T(A^T A)z = (Az)^T(Az) = \|Az\|^2 \geq 0$ — positive semidefinite, so L is convex (§12.7's second-order test). (iii) A stationary point of a convex function is a global minimum (§12.9): the $\hat{\theta}$ solving the normal equations minimizes L over all of $\mathbb{R}^d$.

6. (i)

$$\tilde{A} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad Y = \begin{pmatrix} 2 \\ 1 \\ 2 \end{pmatrix}.$$

$$(ii) \tilde{A}^T \tilde{A} = \begin{pmatrix} 3 & 3 & 5 \\ 3 & 5 & 9 \\ 5 & 9 & 17 \end{pmatrix}, \quad \tilde{A}^T Y = \begin{pmatrix} 5 \\ 5 \\ 9 \end{pmatrix} \text{ (sums: } 2+1+2=5; 0+1+4=5; 0+1+8=9\text{):}$$

$$\begin{cases} 3\theta_0 + 3\theta_1 + 5\theta_2 = 5 \\ 3\theta_0 + 5\theta_1 + 9\theta_2 = 5 \\ 5\theta_0 + 9\theta_1 + 17\theta_2 = 9 \end{cases}.$$

(iii) Try $\theta = (2, -2, 1)^T$: $6 - 6 + 5 = 5$; $6 - 10 + 9 = 5$; $10 - 18 + 17 = 9$. $\tilde{A}$ has distinct x 's (Vandermonde — full rank), so the solution is unique: $\boxed{\hat{y} = 2 - 2x + x^2}$. Check on the data: $2 - 0 + 0 = 2$; $2 - 2 + 1 = 1$; $2 - 4 + 4 = 2$. (iv) Three parameters, three points, full-rank design matrix — the quadratic interpolates exactly (eg 3's lesson): $m = n - 1$ can always thread every point, so the residual is 0. (Whether that's wise is §23.9's question.)

7. (i) **True:** $(A^T A)^T = A^T (A^T)^T = A^T A$. (ii) **True:** full column rank $A^T A$ invertible (§23.5) exactly one $\hat{\theta}$. (iii) **True:** the degree- m family contains the degree- $(m-1)$ family (pad extra $\theta_j = 0$), and minimizing over a bigger set can't give a bigger minimum. (iv) **False:** eg 4 — test MSE rose $0.0030 \rightarrow 0.0191$ from $m = 2$ to $m = 4$. That's overfitting.

8. (i) $A^T A = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}$, $\det = 42 - 36 = 6 \neq 0$ — **exists**. ML reading: intercept and x are independent

features; both coefficients are pinned down. (ii) $A^T A = \begin{pmatrix} 3 & 9 & 9 \\ 9 & 29 & 29 \\ 9 & 29 & 29 \end{pmatrix}$ — columns 2 and 3 identical, rank

$\leq 2 < 3$, singular — **no inverse**. ML reading: the “feature” was duplicated; the model can't decide how to split credit between θ_2 and θ_3 (any shift $\theta_2 \rightarrow \theta_2 + t$, $\theta_3 \rightarrow \theta_3 - t$ gives identical predictions). (iii)

$A^T A = \begin{pmatrix} 14 & 0 \\ 0 & 0 \end{pmatrix}$, $\det = 0$ — **no inverse**. ML reading: the always-0 feature contributes nothing to any prediction, so its coefficient is completely free.

9. (i) $m = 2$ — decided on the **validation** MSE (0.01, the smallest column-3 entry). The training column was only for fitting; the degree is model selection (§23.9). (ii) **Overfitting:** train MSE 0.00 (memorized) but validation MSE 0.09 (worst of the four). (iii) The **test** split's number — on data untouched by both fitting and selection. The validation number is already optimistic (it chose the winner); §22.10's three-way split exists exactly so the reported number is honest.

10. (i) Each $y^i \mid x^i \sim N(\theta^T x^i, \sigma^2)$, independent:

$$L(\theta) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^i - \theta^T x^i)^2}{2\sigma^2}\right).$$

(ii)

$$\ell(\theta) = \log L(\theta) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y^i - \theta^T x^i)^2.$$

(iii) The first term doesn't involve θ , and $1/(2\sigma^2) > 0$, so

$$\arg \max_{\theta} \ell(\theta) = \arg \min_{\theta} \sum_{i=1}^n (y^i - \theta^T x^i)^2,$$

the least-squares objective (§23.1). Maximum likelihood under Gaussian noise **is** least squares — §20.8's result, §23.6's meaning.

24. PCA: variance maximization, eigendecomposition, kernel PCA

Two promises converge here. §22.12 defined the reconstruction loss $L(f, g) = \frac{1}{n} \sum_i \|g(f(\mathbf{x}^i)) - \mathbf{x}^i\|^2$ for an encoder-decoder pair and said the algorithm that searches *all* pairs is Chapter 24's job. §23.11 added that the geometric reading of least squares — predictions as projections — returns here, except the “column space” is no longer given: it is *learned* from the data. This chapter is both promises kept. Everything in §§24.1–24.9 comes from the MLF Week 6 lectures “Principal Component Analysis” (Prof. Prashanth L A, IIT Madras) and the Week 6 tutorial; §§24.10–24.11's kernel material comes from the MLT Week 2 slides. The headline, stated in the first lecture: PCA has two viewpoints — **minimizing reconstruction error** and **maximizing variance of the projected data** — and they are the *same algorithm*.

Notation. The lectures write data points with subscripts — $x_1, \dots, x_n, x_i \in \mathbb{R}^d$ — kept through this chapter (a change from §22.6's superscripts, flagged here so nothing is misread). The target dimension is m , an **input parameter** fixed before the algorithm runs. The mean is $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$.

24.1 The problem: the best m -dimensional home for the data

Data. $D = \{x_1, \dots, x_n\}, x_i \in \mathbb{R}^d$. No labels — this is unsupervised learning (§22.11).

Goal. Fix $m < d$. Find the **best m -dimensional subspace** to project D onto — “best” in a sense made precise in §§24.3–24.4. Two things must be decided: (i) *which* subspace, (ii) *how* to project each point onto it.

The broad context. In ML-flavoured feature selection, you start by collecting as many features as you can — high-dimensional data — then reduce dimensionality to a good subset of features. PCA is the classic dimensionality-reduction method for this: project the data onto a lower-dimensional subspace chosen so that reconstruction error is minimized, or equivalently so that the variance of the projected data is maximized.

Basically, ... “The data lives in a tall room. Pick the m -dimensional floor that fits it best, then drop every point straight down onto that floor.” PCA is the recipe for picking the floor.

24.2 The covariance matrix C — the only matrix that matters

PCA's entire computation is the eigendecomposition of one matrix. **Def (sample covariance).**

$$C = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(x_i - \bar{x})^T, \quad C \in \mathbb{R}^{d \times d}.$$

This is §19.1's covariance matrix with the expectation replaced by the data average.

- i) **Symmetric:** $C^T = C$ — each term $(x_i - \bar{x})(x_i - \bar{x})^T$ is symmetric.
- ii) **Positive semi-definite:** for any u , $u^T C u = \frac{1}{n} \sum_i (u^T (x_i - \bar{x}))^2 \geq 0$ — the same argument as §19.1's theorem, and §7.9's PSD fact.

- iii) **Spectral theorem applies (§6.11):** all eigenvalues of C are real (and ≥ 0 by (ii)), and there is an **orthonormal basis of eigenvectors** $\{u_1, \dots, u_d\}$ with eigenvalues ordered $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$.

Basically, ... C is the “spread table” of the data: diagonal = variance along each axis, off-diagonal = how pairs of axes vary together. PCA reads everything off this one table’s eigenvalues and eigenvectors.

24.3 First viewpoint: minimizing the reconstruction error

Fix a candidate m -dimensional subspace with orthonormal basis $B = \{u_1, \dots, u_m\}$; extend it to a full orthonormal basis $B' = \{u_1, \dots, u_m, u_{m+1}, \dots, u_d\}$ of $\mathbb{R}^d$. Every point expands as $x_i = \sum_{j=1}^d (x_i^T u_j) u_j$ (orthonormal-basis coordinates, §5.4).

Def (reconstruction error). Approximate each point by

$$\tilde{x}_i = \sum_{j=1}^m z_{ij} u_j + \sum_{j=m+1}^d \beta_j u_j$$

and measure

$$J = \frac{1}{n} \sum_{i=1}^n \|x_i - \tilde{x}_i\|^2.$$

The coefficients z_{ij}, β_j are free — choose them to minimize J .

The optimal coefficients. Because B' is orthonormal, cross terms vanish when the norm is expanded (the $\|c_1 u_1 + c_2 u_2\|^2 = c_1^2 + c_2^2$ logic, applied d -wide):

$$J = \frac{1}{n} \sum_{i=1}^n \left[\sum_{j=1}^m (x_i^T u_j - z_{ij})^2 + \sum_{j=m+1}^d (x_i^T u_j - \beta_j)^2 \right].$$

Each coefficient appears in exactly one square, so differentiating is trivial: $-\frac{\partial J}{\partial z_{ij}} = 0 \Rightarrow \boxed{z_{ij} = x_i^T u_j}$ — the encoded coordinates are just dot products with the basis. $-\frac{\partial J}{\partial \beta_j} = 0 \Rightarrow \boxed{\beta_j = \bar{x}^T u_j}$ — the mean’s dot product with each dropped direction.

So for the given subspace, the best reconstruction is

$$\tilde{x}_i = \sum_{j=1}^m (x_i^T u_j) u_j + \sum_{j=m+1}^d (\bar{x}^T u_j) u_j.$$

Note (centered data). If the data is already centered ($\bar{x} = 0$) — “which is what usually people do when you do PCA,” per the lecture — the second term vanishes and $\tilde{x}_i$ is simply the orthogonal projection onto the m -dimensional subspace.

The optimal error, in terms of C . Since the first m terms cancel in $x_i - \tilde{x}_i$,

$$x_i - \tilde{x}_i = \sum_{j=m+1}^d ((x_i - \bar{x})^T u_j) u_j, \quad \|x_i - \tilde{x}_i\|^2 = \sum_{j=m+1}^d ((x_i - \bar{x})^T u_j)^2$$

(orthonormality again). Swapping the sums and writing the square as a product:

$$J^* = \frac{1}{n} \sum_{i=1}^n \sum_{j=m+1}^d ((x_i - \bar{x})^T u_j)^2 = \sum_{j=m+1}^d u_j^T \underbrace{\left[\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(x_i - \bar{x})^T \right]}_C u_j.$$

$$J^* = \sum_{j=m+1}^d u_j^T C u_j \quad (\text{best error for the subspace spanned by } u_1, \dots, u_m).$$

Basically, ... “For a fixed floor, the best reconstruction keeps each point’s coordinates along the floor and replaces the lost coordinates with the mean’s. The leftover error is a sum of one quadratic form $u^T C u$ per dropped direction.”

24.4 Second viewpoint: maximizing the variance of the projection

Totally different starting point. Project onto the line along a unit vector u : point x_i goes to $(x_i^T u) u$, the projected mean to $(\bar{x}^T u) u$. The variance of the projected points:

$$\frac{1}{n} \sum_{i=1}^n (x_i^T u - \bar{x}^T u)^2 = \frac{1}{n} \sum_{i=1}^n u^T (x_i - \bar{x})(x_i - \bar{x})^T u = \boxed{u^T C u}.$$

The problem: maximize $u^T C u$ subject to $u^T u = 1$.

Solution (Lagrangian). $L(u, \lambda) = u^T C u + \lambda(1 - u^T u)$; $\frac{\partial L}{\partial u} = 2Cu - 2\lambda u = 0$ gives $\boxed{Cu = \lambda u}$, and then $u^T C u = \lambda$. The maximizer is an eigenvector of C , and the maximum value is its eigenvalue — so pick the **largest** eigenvalue.

Note (the lecture’s calculus alternative). Maximize the ratio $\frac{u^T C u}{u^T u}$ unconstrained instead: differentiating with the quotient rule, $\frac{\partial}{\partial u_{(i)}} \frac{u^T C u}{u^T u} = 0$ gives $(u^T u)Cu = (u^T C u)u$, i.e. $Cu = \lambda u$ with $\lambda = \frac{u^T C u}{u^T u}$ — the same eigen-equation, no multipliers needed.

Basically, ... “Which direction, when you squash the data onto it, keeps the data most spread out?” The answer is an eigenvector of C — specifically the one with the biggest eigenvalue, and that eigenvalue *is* the spread you keep.

24.5 The PCA algorithm: keep the top m , drop the bottom $d - m$

The two viewpoints now collapse into one recipe. From §24.3, the subspace problem is: choose the *dropped* directions $u_{m+1}, \dots, u_d$ to minimize $J^* = \sum_{j=m+1}^d u_j^T C u_j$. The single-direction problem (§24.4 with “minimize”) says each $u^T C u$ is minimized by the eigenvector of the **smallest** eigenvalue. So:

- i) Compute C (§24.2).
- ii) Eigendecompose: eigenvalues $\lambda_1 \geq \dots \geq \lambda_d$, orthonormal eigenvectors $u_1, \dots, u_d$.
- iii) **Keep** the top m ($u_1, \dots, u_m$ — largest eigenvalues, §24.4’s maximizers); **drop** the bottom $d - m$ (smallest eigenvalues, §24.3’s minimizers).
- iv) Encode: $\alpha_j = x_i^T u_j$, $j = 1, \dots, m$; reconstruct $\tilde{x}_i = \sum_{j=1}^m \alpha_j u_j$ (+ the mean term if uncentered, §24.3).

The two bookkeeping numbers (the tutorial’s Step 8):

$$\boxed{\text{Reconstruction error } J^* = \sum_{j=m+1}^d \lambda_j}, \quad \boxed{\text{Projected variance} = \sum_{j=1}^m \lambda_j}.$$

Same algorithm, two readings: it minimizes the first and maximizes the second.

Basically, ... “Find the directions where the data varies most; throw away the directions where it barely varies. The eigenvalues tell you exactly how much each direction was worth.”

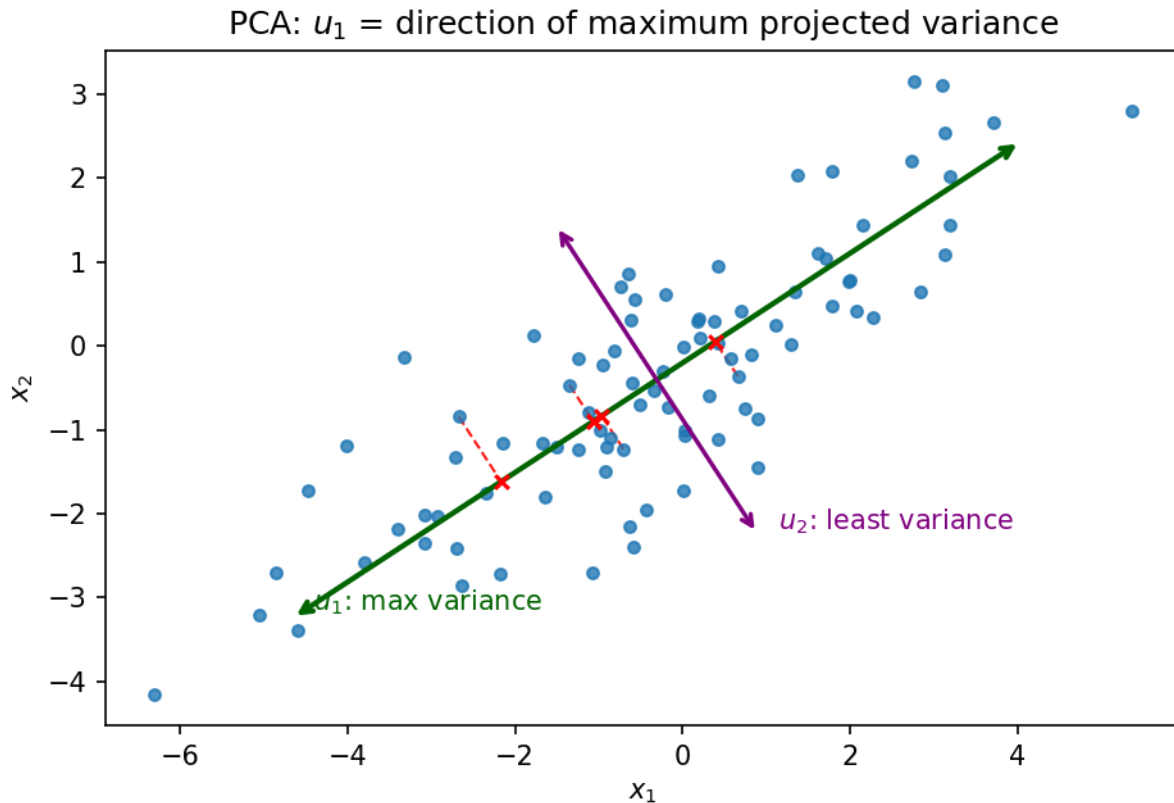


Figure 31: Elongated blue point cloud in 2-D with a green double arrow u_1 along the long axis labelled “max variance”, a purple double arrow u_2 along the short axis labelled “least variance”, and dashed red segments dropping five sample points onto the u_1 line.

24.6 Worked example set

eg 1 (the lecture’s example — data already on a line). $D = \{(-1, -1)^T, (0, 0)^T, (1, 1)^T\}$, $m = 1$.

Step 1 — mean: $\bar{x} = \frac{1}{3}[(-1, -1) + (0, 0) + (1, 1)] = (0, 0)^T$. Already centered.

Step 2 — covariance:

$$C = \frac{1}{3} \sum_{i=1}^3 x_i x_i^T = \frac{1}{3} \left[\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} + 0 + \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \right] = \frac{2}{3} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

Step 3 — eigenpairs. $\text{tr}(C) = \frac{4}{3}$, $\det(C) = 0$ (rank 1), so $\lambda_1 = \frac{4}{3}$, $\lambda_2 = 0$. For λ_1 : $\frac{2}{3} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \frac{4}{3} \begin{pmatrix} a \\ b \end{pmatrix} \Rightarrow$

$$a = b; \text{ normalized } \boxed{u_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}}, u_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Step 4 — project. $\tilde{x}_i = (x_i^T u_1) u_1$: $x_1^T u_1 = -\sqrt{2} \Rightarrow \tilde{x}_1 = -\sqrt{2} \cdot \frac{1}{\sqrt{2}} (1, 1)^T = (-1, -1)^T = x_1$; $\tilde{x}_2 = (0, 0)^T = x_2$; $\tilde{x}_3 = (1, 1)^T = x_3$.

Step 5 — books: $J^* = \lambda_2 = 0$ (nothing lost — the data was already one-dimensional, so PCA changes nothing); projected variance = $\lambda_1 = \frac{4}{3}$. Sanity check: $\frac{1}{3}[(-\sqrt{2})^2 + 0 + (\sqrt{2})^2] = \frac{4}{3}$.

eg 2 (a hand-worked 2-D PCA with real loss). $D = \{(1, 2), (2, 1), (3, 4), (4, 3)\}$, $m = 1$.

Step 1 — mean: $\bar{x} = (2.5, 2.5)^T$. Centered points: $(-1.5, -0.5), (-0.5, -1.5), (0.5, 1.5), (1.5, 0.5)$.

Step 2 — covariance:

$$C = \frac{1}{4} \sum_i (x_i - \bar{x})(x_i - \bar{x})^T = \frac{1}{4} \begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix} = \begin{pmatrix} 1.25 & 0.75 \\ 0.75 & 1.25 \end{pmatrix}.$$

(Check: $\sum (x_1\text{-coords})^2 = 2.25 + 0.25 + 0.25 + 2.25 = 5$.)

Step 3 — eigenpairs. $\text{tr} = 2.5$, $\det = 1.25^2 - 0.75^2 = 1$: $\lambda = \frac{2.5 \pm \sqrt{6.25 - 4}}{2} = \frac{2.5 \pm 1.5}{2}$, so $\boxed{\lambda_1 = 2, \lambda_2 = 0.5}$. For

$$\lambda_1 = 2: \begin{pmatrix} -0.75 & 0.75 \\ 0.75 & -0.75 \end{pmatrix} u = 0 \Rightarrow \boxed{u_1 = \frac{1}{\sqrt{2}}(1, 1)^T}, u_2 = \frac{1}{\sqrt{2}}(1, -1)^T.$$

Step 4 — project (uncentered: recenter, project, add $\bar{x}$ back). E.g. $x_1 = (1, 2)$: centered $(-1.5, -0.5)$, dot with u_1 : $-2/\sqrt{2} = -\sqrt{2}$, so $\tilde{x}_1 = -\sqrt{2}u_1 + \bar{x} = (-1, -1) + (2.5, 2.5) = (1.5, 1.5)$. Similarly $\tilde{x}_2 = (1.5, 1.5)$, $\tilde{x}_3 = \tilde{x}_4 = (3.5, 3.5)$.

Step 5 — books: $J^* = \lambda_2 = 0.5$. Check directly: each $\|x_i - \tilde{x}_i\|^2 = 0.5^2 + 0.5^2 = 0.5$, average = 0.5 . Projected variance = $\lambda_1 = 2$; fraction kept = $\frac{2}{2.5} = 80\%$ (§24.8).

eg 3 (the tutorial's 3-D example — corrected). $x_1 = (3, 6, 9)^T$, $x_2 = (2, 4, 6)^T$, $x_3 = (1, 2, 3)^T$. Mean $\bar{x} = (2, 4, 6)^T$; centered: $(1, 2, 3), (0, 0, 0), (-1, -2, -3)$. With $v = (1, 2, 3)^T$:

$$C = \frac{1}{3} [vv^T + (-v)(-v)^T] = \frac{2}{3} \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{pmatrix}.$$

C is rank 1, so two eigenvalues are 0; the nonzero one is $\frac{2}{3}\|v\|^2 = \frac{2}{3} \cdot 14 = \boxed{\lambda_1 = \frac{28}{3}}$ (the tutorial slide writes 14 — it forgot the $\frac{2}{3}$ factor; the eigenvectors it gives are right). $u_1 = v/\|v\| = \frac{1}{\sqrt{14}}(1, 2, 3)^T$. Projection: $x_1^T u_1 = 42/\sqrt{14}$, $\tilde{x}_1 = \frac{42}{14}(1, 2, 3)^T = (3, 6, 9)^T = x_1$ (data already 1-D). $J^* = 0$, projected variance = $28/3$.

Note (a tutorial slip, flagged). The tutorial's second example (four 2-D points, $C = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$) mixes an *uncentered* projection with a *centered* covariance — inconsistent — and then botches the arithmetic, writing $J = \frac{1}{4}[0^2 + 2^2 + 2^2 + 2^2] = 24$ (the bracket is 12, not 24, and the x_3 term is really $\|(2, 4) - (2, 0)\|^2 = 16$, not 2^2). Done consistently (§24.3, mean term included), the reconstructions are $(2, 2), (0, 2), (2, 2), (4, 2)$; the per-point squared errors are 4, 0, 4, 0; and $J^* = 2 = \lambda_2$ for the dropped direction, as §24.5 predicts.

24.7 The encoder–decoder reading: the optimal linear pair

§22.12 promised an algorithm that searches all encoder–decoder pairs for the one minimizing the reconstruction loss. For *linear* pairs, PCA is that algorithm. Write $U_m = [u_1 \cdots u_m]$ ($d \times m$):

- **Encoder** $f(x) = U_m^T(x - \bar{x}) \in \mathbb{R}^m$ — the m dot products $x^T u_j$ of §24.3.
- **Decoder** $g(z) = U_m z + \bar{x} \in \mathbb{R}^d$ — rebuild as a combination of the kept directions, plus the mean.

Then $g(f(x_i)) = \tilde{x}_i$ of §24.3, and §22.12's loss is exactly J^* :

$$\boxed{L(f, g) = \frac{1}{n} \sum_{i=1}^n \|g(f(x_i)) - x_i\|^2 = \sum_{j=m+1}^d \lambda_j}.$$

PCA doesn't compare two hand-picked pairs (the way §22.12's illustration did) — it solves for the optimum over *all* linear pairs. And §23.11's remark lands: least squares projected Y onto a *given* column space; here the subspace itself — the span of the top- m eigenvectors — is learned from the data's own variance.

Basically, ... “The best linear pipe: squeeze through the m fattest-variance directions, rebuild from those. The loss you can't avoid equals the eigenvalues you threw away.”

24.8 How many components? Reading the eigenvalues

Total variance in the data: $\sum_{j=1}^d \lambda_j = \text{tr}(C)$ (the trace is the sum of the diagonal = sum of per-axis variances; eigenvalues sum to the trace, §6.11). Kept: $\sum_{j=1}^m \lambda_j$. Lost: $\sum_{j=m+1}^d \lambda_j$. So the **fraction of variance explained** by the first m components is

$$\frac{\sum_{j=1}^m \lambda_j}{\sum_{j=1}^d \lambda_j}.$$

Two ways to pick m from it: (i) a threshold — keep adding components until the fraction crosses, say, 95%; (ii) the elbow — stop where the eigenvalues flatten out and each new component adds almost nothing. (The lectures never name “explained variance” or draw a scree plot; the ratio above is just their projected-variance and trace numbers divided.)

Note. PCA has no validation split the way §23.9’s polynomial degrees did — there is no “test error” for an unsupervised method (§22.11: nobody grades unsupervised output directly). The eigenvalue tail *is* the selection criterion: it measures exactly what reconstruction would cost.

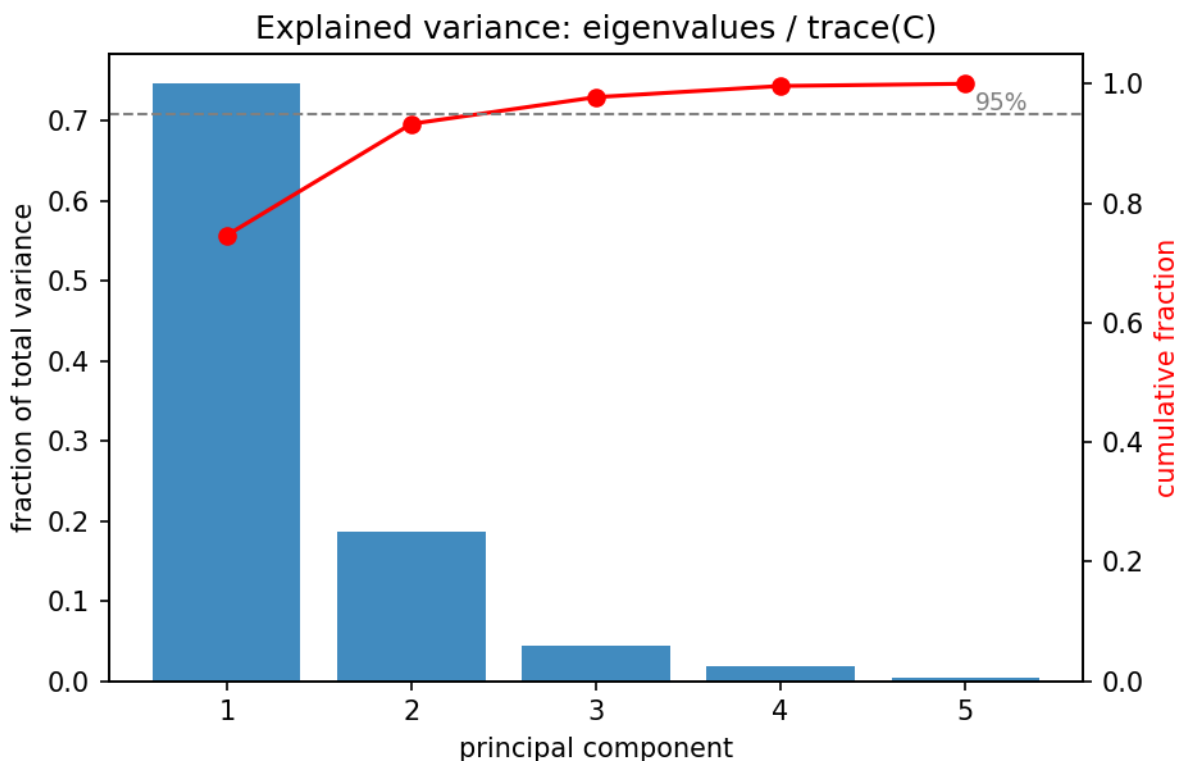


Figure 32: Bar chart of five components’ fractions of total variance (0.75, 0.19, 0.045, 0.019, 0.004) with a red cumulative curve crossing a dashed grey 95% line at component 2.

24.9 PCA in high dimensions: the $n \times n$ trick

When $d \gg n$ (few points, huge features — the MLT slide’s example: face recognition, “eigenfaces”), eigendecomposing the $d \times d$ matrix C at $O(d^3)$ is wasteful. Let A be the $n \times d$ matrix with rows $(x_i - \bar{x})^T$; then $C = \frac{1}{n} A^T A$.

- i) **Most eigenvalues are 0.** C is a sum of n rank-1 matrices, so $\text{rank}(C) \leq n$ — at least $d - n$ eigenvalues are 0. No need to find them.
- ii) **The nonzero spectrum lives in an $n \times n$ matrix.** If $Cu = \lambda u$ ($\lambda \geq 0$), then

$$\frac{1}{n}AA^T(Au) = A\left(\frac{1}{n}A^TAu\right) = A(\lambda u) = \lambda(Au),$$

- so λ is also an eigenvalue of the $n \times n$ matrix $\frac{1}{n}AA^T$. Conversely, if $\frac{1}{n}AA^Tv = \lambda v$ with $\lambda > 0$, then $C(A^Tv) = \frac{1}{n}A^TAA^Tv = A^T(\lambda v) = \lambda(A^Tv)$ — recover the d -dimensional eigenvector as $u = A^Tv/\|A^Tv\|$.
- iii) **The recipe:** eigendecompose $\frac{1}{n}AA^T$ ($n \times n$, $O(n^3)$), keep the top- m eigenpairs, map back with A^T . Same PCA, no $d \times d$ matrix ever formed.

Basically, ... “With 50 faces of a million pixels each, don’t diagonalize a million-by-million matrix — diagonalize the 50-by-50 one and translate the answers back.”

24.10 Kernel PCA: when the structure isn’t linear

The problem (MLT W2, “Issue 2”). Data may not lie in a low-dimensional *linear* subspace. The slide’s example: points on a circle $(f_1 - a)^2 + (f_2 - b)^2 = r^2$. Plain PCA finds u_1, u_2 both important (nearly equal eigenvalues) — the one-dimensional circular structure is invisible to it.

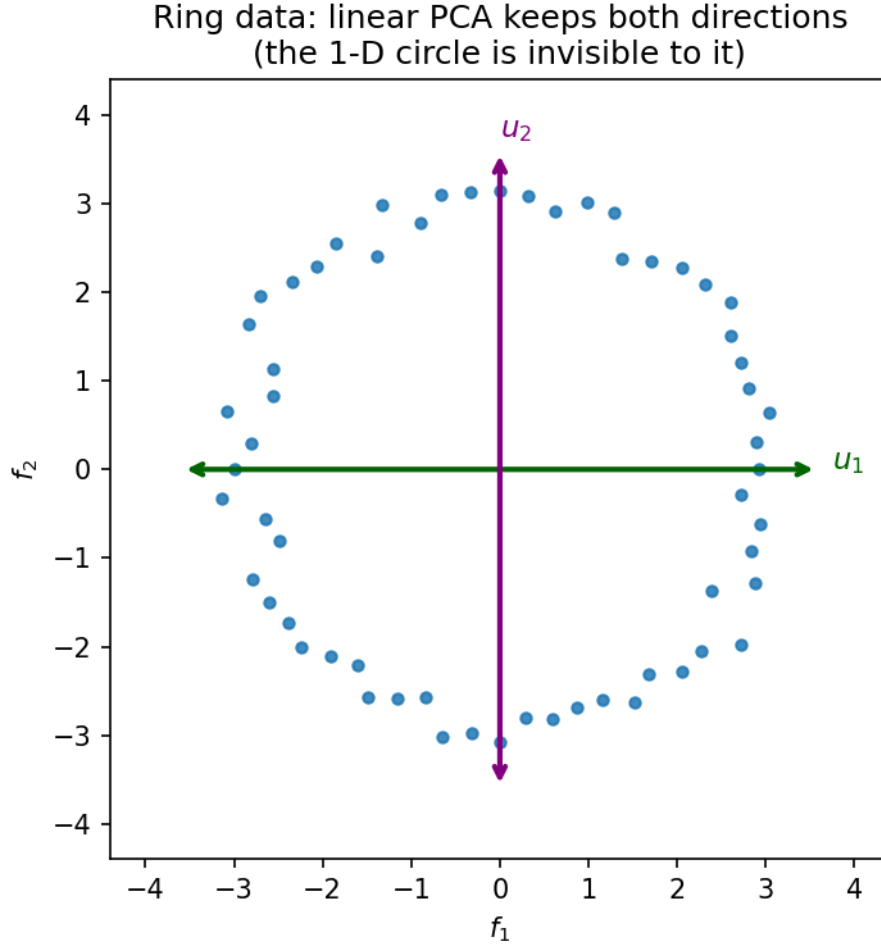


Figure 33: Ring of blue points in the (f_1, f_2) plane with a green double arrow u_1 and a purple double arrow u_2 of equal length through the centre; the caption notes linear PCA keeps both directions and misses the 1-D circular structure.

The idea. Lift $x \in \mathbb{R}^d$ through a map ϕ to a high-dimensional space $\mathbb{R}^D$ where the structure *becomes* linear, then run the §24.9 machinery (we already know how to handle $D \gg n$). For the circle: with $\phi(x) = [1, f_1^2, f_2^2, f_1 f_2, f_1, f_2]^T$,

$$\phi(x)^T u = 0 \quad \text{for} \quad u = [a^2 + b^2 - r^2, 1, 1, 0, -2a, -2b]^T,$$

which is exactly $(f_1 - a)^2 + (f_2 - b)^2 = r^2$ expanded — the points lie in a *linear* subspace of $\mathbb{R}^6$. But explicit ϕ can be brutal: cubic features of d variables number $\approx O(d^3)$, and the RBF kernel's ϕ is infinite-dimensional. The slides' way out has two steps.

Step 1 — the dual form. An eigenvector w_k of $C = \frac{1}{n} X X^T$ (data assumed centered; X is $d \times n$ with columns x_i) satisfies

$$C w_k = \lambda_k w_k \Rightarrow w_k = \sum_{i=1}^n \frac{x_i^T w_k}{n \lambda_k} x_i = X \alpha_k \quad \text{for some } \alpha_k \in \mathbb{R}^n.$$

Every eigenvector is a linear combination of the data points. Plugging $w_k = X \alpha_k$ into $C w_k = \lambda_k w_k$ and premultiplying by X^T :

$$\boxed{K \alpha_k = (n \lambda_k) \alpha_k}, \quad K = X^T X \in \mathbb{R}^{n \times n}.$$

Unit-norm w_k needs $\alpha_k^T K \alpha_k = 1$, i.e. $\alpha_k = \beta_k / \sqrt{n\lambda_k}$ where β_k is a unit eigenvector of K . (The linear-algebra fact behind this: XX^T and $X^T X$ share their nonzero eigenvalues.)

Step 2 — the kernel trick. The dual form never needs $\phi(x_i)$ alone — only inner products $K_{ij} = \phi(x_i)^T \phi(x_j)$. So *replace* the dot product by a **kernel function** $\kappa(x, x')$ that computes it directly:

eg (the slide's computation). For $x = (f_1, f_2)$, $x' = (g_1, g_2)$:

$$\kappa(x, x') = (x^T x' + 1)^2 = (f_1 g_1 + f_2 g_2 + 1)^2 = f_1^2 g_1^2 + f_2^2 g_2^2 + 1 + 2f_1 g_1 f_2 g_2 + 2f_1 g_1 + 2f_2 g_2.$$

That equals $\phi(x)^T \phi(x')$ with $\phi(x) = [f_1^2, f_2^2, 1, \sqrt{2}f_1 f_2, \sqrt{2}f_1, \sqrt{2}f_2]^T$. “We managed to compute $\phi(x)^T \phi(x')$ **without explicitly computing** $\phi(x)$ ” — $O(d)$ work instead of $O(d^2)$.

Def (kernel function, valid kernel). Any $\kappa : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ that is some $\phi(x)^T \phi(x')$ is a **kernel function**. How to tell without exhibiting ϕ — Mercer's theorem (informal): κ is valid iff (a) it is symmetric, $\kappa(x, x') = \kappa(x', x)$, and (b) for *every* dataset, the matrix $K_{ij} = \kappa(x_i, x_j)$ is positive semi-definite (all eigenvalues ≥ 0). - **Polynomial kernel:** $\kappa(x, x') = (x^T x' + 1)^p$, $p \geq 1$. - **RBF / Gaussian kernel:** $\kappa(x, x') = \exp(-\|x - x'\|^2 / (2\sigma^2))$, $\sigma > 0$ — valid, and its ϕ maps into an *infinite*-dimensional space (technicalities aside: think of a point mapped to a “function,” with dot products becoming integrals).

The kernel PCA algorithm. Input $\{x_1, \dots, x_n\}$, kernel κ . - Step 1: $K_{ij} = \kappa(x_i, x_j)$, then **center the kernel** — PCA needs zero-mean data, but ϕ 's mean is uncomputable, so center its Gram matrix instead: $K_{ij}^c = K_{ij} - \theta_i - \theta_j + P$ with $\theta_i = \frac{1}{n} \sum_k K_{ik}$, $P = \frac{1}{n^2} \sum_{ij} K_{ij}$. - Step 2: eigendecompose K^c : unit eigenvectors $\beta_1, \dots, \beta_\ell$ with eigenvalues $n\lambda_1 \geq \dots \geq n\lambda_\ell$; set $\alpha_k = \beta_k / \sqrt{n\lambda_k}$. - Step 3: the eigenvectors w_k of the feature-space covariance *cannot* be reconstructed (that needs ϕ) — but the **compressed representation** can: $\phi(x_i)^T w_k = \sum_{j=1}^n \alpha_{kj} K_{ij}^c$. “For downstream tasks, this is usually good enough.”

Basically, ... “Linear PCA only sees straight-line structure. Lift the data into a bigger space where curves become straight, do PCA there — and use the kernel trick so you never actually visit the bigger space. You lose the reconstruction, but you keep the compressed coordinates, which is what downstream tasks eat.”

eg (kernel PCA by hand — three points on a parabola). $x_1 = (-1, 1)$, $x_2 = (0, 0)$, $x_3 = (1, 1)$ ($y = x^2$), quadratic kernel $\kappa(x, z) = (x^T z + 1)^2$.

Step 1 — Gram matrix: $K_{11} = ((1+1)+1)^2 = 9$, $K_{12} = K_{13} = K_{22} = K_{23} = 1$, $K_{33} = 9$:

$$K = \begin{pmatrix} 9 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 9 \end{pmatrix}.$$

Center it: $\theta = (11/3, 1, 11/3)^T$, $P = 25/9$, so

$$K^c = \frac{1}{9} \begin{pmatrix} 40 & -8 & -32 \\ -8 & 16 & -8 \\ -32 & -8 & 40 \end{pmatrix}.$$

(Check one entry: $K_{11}^c = 9 - 2(11/3) + 25/9 = (81 - 66 + 25)/9 = 40/9$.)

Step 2 — eigendecompose. The inner matrix has trace 96; $(1, 1, 1)^T \mapsto 0$ (eigenvalue 0 — centering kills the all-ones direction); $(1, 0, -1)^T \mapsto 72(1, 0, -1)^T$; $(1, -2, 1)^T \mapsto 24(1, -2, 1)^T$. So K^c 's eigenvalues are 8, $\frac{8}{3}$, 0, i.e. $n\lambda_1 = 8$, $n\lambda_2 = 8/3$ with $n = 3$: $\lambda_1 = 8/3$, $\lambda_2 = 8/9$. Unit eigenvector for $n\lambda_1$: $\beta_1 = \frac{1}{\sqrt{2}}(-1, 0, 1)^T$; normalized $\alpha_1 = \beta_1 / \sqrt{8} = (-1/4, 0, 1/4)^T$ (eigenvector signs are arbitrary). Check $\alpha_1^T K^c \alpha_1 = 1$.

Step 3 — compressed 1-D coordinates $z_{i1} = \sum_j \alpha_{1j} K_{ij}^c = K^c \alpha_1 = (-2, 0, 2)^T$. Their variance: $\frac{4+0+4}{3} = \frac{8}{3} = \lambda_1$ — exactly the projected-variance rule of §24.5, now in feature space. The three parabola points collapse to a clean 1-D line $(-2, 0, 2)$ that a linear PCA in the original plane could never produce.

24.11 Where this goes next

- i) **Eigenfaces.** The MLT slide’s motivating example for §24.9: each face image is a point in a million-dimensional pixel space; with far fewer images than pixels, the $n \times n$ trick makes PCA feasible, and the top eigenvectors — “eigenfaces” — are the directions of maximum variance across faces.
- ii) **The SVD connection.** The MLF lecture flags it without deriving: the SVD of the centered data matrix (Chapter 7) computes PCA’s directions without ever forming $C = \frac{1}{n}A^T A$ — the numerically careful way to do all of §24.5.
- iii) **A different answer to “understand the data.”** PCA compresses by *subspaces*. Chapter 25 (GMMs and EM) compresses by *probability*: fit a mixture of Gaussians to the data instead of a flat floor through it. Same unsupervised spirit (§22.11), different model of what “structure” means.

Problem set

- The lecture’s example, fully worked: $D = \{(-1, -1)^T, (0, 0)^T, (1, 1)^T\}$, $m = 1$. (i) Compute $\bar{x}$ and C . (ii) Find the eigenvalues and (unit) eigenvectors of C . (iii) Write the projected points $\tilde{x}_i$. (iv) Compute J^* two ways — directly from $\frac{1}{n} \sum_i \|x_i - \tilde{x}_i\|^2$ and from the eigenvalue formula — and the projected variance.
- Baby Lagrangian. $C = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}$. Write a unit vector as $u = (\cos \theta, \sin \theta)^T$, compute $u^T C u$ as a function of θ , and find the maximizer, the minimizer, and both values. Which eigenvectors do they correspond to?
- Prove every eigenvalue of the covariance matrix C is ≥ 0 . (Hint: §19.1’s PSD argument — what is $u^T C u$?)
- PCA by hand. Data $(1, 2), (2, 1), (3, 4), (4, 3)$, $m = 1$. (i) Compute $\bar{x}$ and C . (ii) Find its eigenvalues and unit eigenvectors. (iii) Compute the four reconstructions $\tilde{x}_i$. (iv) Compute J^* directly and check it against the eigenvalue formula. (v) What fraction of the total variance does the first component explain?
- Guided proof. Starting from the lecture’s $J^* = \sum_{j=m+1}^d u_j^T C u_j$ with $u_{m+1}, \dots, u_d$ the eigenvectors for $\lambda_{m+1}, \dots, \lambda_d$: (i) show $J^* = \sum_{j=m+1}^d \lambda_j$; (ii) explain why any *other* choice of m kept directions gives reconstruction error $\geq$ this.
- The $n \times n$ trick, proved. Let $C = \frac{1}{n}A^T A$ with A the $n \times d$ centered data matrix. (i) Show: if $Cu = \lambda u$ then $\frac{1}{n}AA^T(Au) = \lambda(Au)$. (ii) Show: if $\frac{1}{n}AA^T v = \lambda v$ with $\lambda > 0$ then $A^T v$ is an eigenvector of C with eigenvalue λ . (iii) Why does this let PCA skip the $d \times d$ eigendecomposition when $d \gg n$?
- The kernel identity. For $x = (f_1, f_2)$, $x' = (g_1, g_2)$, expand $(x^T x' + 1)^2$ fully and exhibit an explicit $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^6$ with $(x^T x' + 1)^2 = \phi(x)^T \phi(x')$.
- Kernel PCA numeric. $K^c = \frac{1}{9} \begin{pmatrix} 40 & -8 & -32 \\ -8 & 16 & -8 \\ -32 & -8 & 40 \end{pmatrix}$ (from §24.10’s parabola example, $n = 3$). (i) Find its eigenvalues (hint: try $(1, 1, 1)^T$, $(1, 0, -1)^T$, $(1, -2, 1)^T$). (ii) Deduce λ_1, λ_2 . (iii) Compute the normalized α_1 for the top component. (iv) Compute the compressed coordinates $K^c \alpha_1$ and verify their variance equals λ_1 .
- Centering the kernel, derived. Let $\tilde{\phi}(x_i) = \phi(x_i) - \frac{1}{n} \sum_k \phi(x_k)$. (i) Write $K_{ij}^c = \tilde{\phi}(x_i)^T \tilde{\phi}(x_j)$ in terms of $K_{ij} = \phi(x_i)^T \phi(x_j)$. (ii) Simplify to $K_{ij}^c = K_{ij} - \theta_i - \theta_j + P$ with $\theta_i = \frac{1}{n} \sum_k K_{ik}$, $P = \frac{1}{n^2} \sum_{k\ell} K_{k\ell}$.
- True or false, one-line justification: (i) C is always symmetric. (ii) The first principal component is the eigenvector of the *smallest* eigenvalue of C . (iii) J^* never increases when m increases. (iv) Plain PCA can discover the 1-D circular structure of ring data. (v) Kernel PCA can explicitly reconstruct the feature-space eigenvectors w_k .

Solutions — Chapter 24. PCA

Problem 1

$D = \{(-1, -1)^T, (0, 0)^T, (1, 1)^T\}$, $m = 1$.

(i) $\bar{x} = \frac{1}{3}[(-1, -1) + (0, 0) + (1, 1)] = (0, 0)^T$.

$$C = \frac{1}{3} \left[\begin{pmatrix} -1 \\ -1 \end{pmatrix} (-1 \quad -1) + 0 + \begin{pmatrix} 1 \\ 1 \end{pmatrix} (1 \quad 1) \right] = \frac{2}{3} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

(ii) $\text{tr}(C) = 4/3$, $\det(C) = (2/3)^2(1 \cdot 1 - 1 \cdot 1) = 0$. Characteristic: $\lambda^2 - \frac{4}{3}\lambda = 0$, so $\lambda_1 = 4/3$, $\lambda_2 = 0$. For λ_1 : $\frac{2}{3} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} u = \frac{4}{3}u \Rightarrow u_1 + u_2$ direction; unit: $u_1 = \frac{1}{\sqrt{2}}(1, 1)^T$. For $\lambda_2 = 0$: orthogonal direction, $u_2 = \frac{1}{\sqrt{2}}(1, -1)^T$.

(iii) $\tilde{x}_i = (x_i^T u_1) u_1$ (centered, so no mean term): $x_1^T u_1 = -2/\sqrt{2} = -\sqrt{2}$, $\tilde{x}_1 = -\sqrt{2} u_1 = (-1, -1)^T = x_1$; $\tilde{x}_2 = (0, 0)^T = x_2$; $x_3^T u_1 = \sqrt{2}$, $\tilde{x}_3 = (1, 1)^T = x_3$.

(iv) Direct: every $\|x_i - \tilde{x}_i\| = 0$, so $J^* = 0$. Eigenvalue formula: $J^* = \lambda_2 = 0$. Projected variance $= \lambda_1 = 4/3$; check: $\frac{1}{3}[(\sqrt{2})^2 + 0 + (\sqrt{2})^2] = 4/3$.

Problem 2

$u^T C u = 3 \cos^2 \theta + \sin^2 \theta = 3 \cos^2 \theta + (1 - \cos^2 \theta) = 1 + 2 \cos^2 \theta$. Max at $\cos^2 \theta = 1$, i.e. $\theta = 0, \pi$: $u = \pm(1, 0)^T = \pm e_1$, value 3 — the eigenvector for the largest eigenvalue $\lambda = 3$. Min at $\cos^2 \theta = 0$, i.e. $\theta = \pi/2, 3\pi/2$: $u = \pm(0, 1)^T = \pm e_2$, value 1 — the eigenvector for the smallest eigenvalue $\lambda = 1$. This is §24.4's claim in miniature: extrema of $u^T C u$ on the unit circle sit on eigenvectors, with values equal to the eigenvalues.

Problem 3

Let $Cu = \lambda u$ with $u \neq 0$. Then $\lambda \|u\|^2 = u^T(\lambda u) = u^T C u = \frac{1}{n} \sum_{i=1}^n (u^T(x_i - \bar{x}))^2 \geq 0$ (a sum of squares). Since $\|u\|^2 > 0$, $\lambda \geq 0$. This is §19.1's covariance-is-PSD argument applied to the sample covariance — the same reason a covariance matrix “can never have a negative eigenvalue.”

Problem 4

(i) $\bar{x} = \frac{1}{4}[(1, 2) + (2, 1) + (3, 4) + (4, 3)] = (2.5, 2.5)^T$. Centered: $(-1.5, -0.5), (-0.5, -1.5), (0.5, 1.5), (1.5, 0.5)$. $\sum (x_1\text{-coords})^2 = 2.25 + 0.25 + 0.25 + 2.25 = 5$; same for x_2 ; cross terms: $(-1.5)(-0.5) + (-0.5)(-1.5) + (0.5)(1.5) + (1.5)(0.5) = 0.75 + 0.75 + 0.75 + 0.75 = 3$. So $C = \frac{1}{4} \begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix} = \begin{pmatrix} 1.25 & 0.75 \\ 0.75 & 1.25 \end{pmatrix}$.

(ii) $\text{tr} = 2.5$, $\det = 1.25^2 - 0.75^2 = 1.5625 - 0.5625 = 1$. $\lambda = \frac{2.5 \pm \sqrt{6.25 - 4}}{2} = \frac{2.5 \pm 1.5}{2}$: $\lambda_1 = 2$, $\lambda_2 = 0.5$. For λ_1 : $\begin{pmatrix} -0.75 & 0.75 \\ 0.75 & -0.75 \end{pmatrix} u = 0 \Rightarrow u_1 = \frac{1}{\sqrt{2}}(1, 1)^T$; $u_2 = \frac{1}{\sqrt{2}}(1, -1)^T$.

(iii) Recenter, project, add $\bar{x}$: $x_1 = (1, 2) \rightarrow (-1.5, -0.5)$, dot with u_1 is $-2/\sqrt{2} = -\sqrt{2}$, $\tilde{x}_1 = -\sqrt{2} u_1 + \bar{x} = (-1, -1) + (2.5, 2.5) = (1.5, 1.5)$. Similarly $\tilde{x}_2 = (1.5, 1.5)$, $\tilde{x}_3 = \tilde{x}_4 = (3.5, 3.5)$.

(iv) $\|x_1 - \tilde{x}_1\|^2 = (-0.5)^2 + (0.5)^2 = 0.5$; same 0.5 for all four. $J^* = 0.5 = \lambda_2$ (the dropped direction's eigenvalue).

(v) Total variance $\text{tr}(C) = 2.5$; first component explains $2/2.5 = 0.8 = 80\%$.

Problem 5

- (i) Each dropped direction is an eigenvector: $Cu_j = \lambda_j u_j$ with $\|u_j\| = 1$, so $u_j^T Cu_j = u_j^T (\lambda_j u_j) = \lambda_j$. Hence $J^* = \sum_{j=m+1}^d \lambda_j$.
- (ii) J^* is a sum of $d - m$ terms $u^T Cu$ over orthonormal dropped directions. The lecture's single-direction result says the minimum of $u^T Cu$ over unit u is the smallest eigenvalue λ_d , achieved at u_d ; removing that direction, the minimum over the remaining orthogonal complement is λ_{d-1} , and so on. Any other orthonormal choice replaces some λ_j by a larger value — e.g. keeping u_d instead of u_1 puts λ_1 into the dropped sum. Formally this is the same greedy argument the lecture uses for the variance-maximization side (§24.4 extended to $m \geq 1$: the top- m eigenvectors maximize the projected variance).

Problem 6

- (i) $\frac{1}{n}AA^T(Au) = A(\frac{1}{n}A^T Au) = A(Cu) = A(\lambda u) = \lambda(Au)$. So Au is an eigenvector of $\frac{1}{n}AA^T$ with the same eigenvalue λ .
- (ii) $C(A^T v) = \frac{1}{n}A^T A(A^T v) = A^T(\frac{1}{n}AA^T v) = A^T(\lambda v) = \lambda(A^T v)$. Since $\lambda > 0$, $A^T v \neq 0$ (else $\frac{1}{n}AA^T v = 0 \neq \lambda v$), so it is a genuine eigenvector of C .
- (iii) When $d \gg n$, $\text{rank}(C) \leq n$: all but at most n eigenvalues of the $d \times d$ matrix C are 0. By (i)–(ii) the nonzero spectrum is recovered exactly from the $n \times n$ matrix $\frac{1}{n}AA^T$ — eigendecompose that ($O(n^3)$, not $O(d^3)$) and map eigenvectors back via $u = A^T v / \|A^T v\|$.

Problem 7

$(x^T x' + 1)^2 = (f_1 g_1 + f_2 g_2 + 1)^2$. Expanding $(a + b + c)^2 = a^2 + b^2 + c^2 + 2ab + 2ac + 2bc$:

$$= f_1^2 g_1^2 + f_2^2 g_2^2 + 1 + 2f_1 g_1 f_2 g_2 + 2f_1 g_1 + 2f_2 g_2.$$

Take $\phi(x) = (f_1^2, f_2^2, 1, \sqrt{2}f_1 f_2, \sqrt{2}f_1, \sqrt{2}f_2)^T \in \mathbb{R}^6$. Then $\phi(x)^T \phi(x') = f_1^2 g_1^2 + f_2^2 g_2^2 + 1 + (\sqrt{2}f_1 f_2)(\sqrt{2}g_1 g_2) + (\sqrt{2}f_1)(\sqrt{2}g_1) + (\sqrt{2}f_2)(\sqrt{2}g_2)$, which is exactly the expansion above. So $(x^T x' + 1)^2$ is a valid kernel — it computes a 6-D dot product in $O(d)$ time without forming ϕ .

Problem 8

- (i) $(1, 1, 1)^T$: rows sum to $40 - 8 - 32 = 0$, so eigenvalue 0. $(1, 0, -1)^T$: first row $40 + 32 = 72$, second $-8 + 8 = 0$, third $-32 - 40 = -72 = 72 \cdot (-1)$ — eigenvalue 72. $(1, -2, 1)^T$: first $40 + 16 - 32 = 24$, second $-8 - 32 - 8 = -48 = 24 \cdot (-2)$, third $-32 + 16 + 40 = 24$ — eigenvalue 24. So the inner matrix has eigenvalues 72, 24, 0; K^c (divided by 9) has 8, 8/3, 0. Check: $8 + 8/3 + 0 = 32/3 = \text{tr}(K^c) = (40 + 16 + 40)/9$.
- (ii) K^c 's eigenvalues are $n\lambda_k$ with $n = 3$: $n\lambda_1 = 8 \Rightarrow \lambda_1 = 8/3$; $n\lambda_2 = 8/3 \Rightarrow \lambda_2 = 8/9$.
- (iii) Unit eigenvector for 8: $\beta_1 = \frac{1}{\sqrt{2}}(-1, 0, 1)^T$. Normalized: $\alpha_1 = \beta_1/\sqrt{8} = (-1/4, 0, 1/4)^T$ (eigenvector signs are arbitrary — $(1/4, 0, -1/4)^T$ is equally valid). Check: $\alpha_1^T K^c \alpha_1 = \frac{1}{16} \cdot \frac{1}{9} \cdot (-1, 0, 1) \cdot (-72, 0, 72)^T = \frac{144}{144} = 1$.
- (iv) Compressed coordinates $K^c \alpha_1 = (-2, 0, 2)^T$ (the sign-flipped $(2, 0, -2)^T$ is equally valid). Variance: $\frac{4+0+4}{3} = 8/3 = \lambda_1$ — the feature-space projected-variance rule.

Problem 9

- (i) $\tilde{\phi}(x_i) = \phi(x_i) - \mu$ with $\mu = \frac{1}{n} \sum_k \phi(x_k)$. Then

$$K_{ij}^c = (\phi(x_i) - \mu)^T (\phi(x_j) - \mu) = \underbrace{\phi(x_i)^T \phi(x_j)}_{K_{ij}} - \phi(x_i)^T \mu - \mu^T \phi(x_j) + \mu^T \mu.$$

- (ii) $\phi(x_i)^T \mu = \frac{1}{n} \sum_k \phi(x_i)^T \phi(x_k) = \frac{1}{n} \sum_k K_{ik} = \theta_i$; similarly $\mu^T \phi(x_j) = \theta_j$; $\mu^T \mu = \frac{1}{n^2} \sum_{k\ell} \phi(x_k)^T \phi(x_\ell) = \frac{1}{n^2} \sum_{k\ell} K_{k\ell} = P$. So $K_{ij}^c = K_{ij} - \theta_i - \theta_j + P$. This centers the data *in feature space* using only kernel evaluations — ϕ itself never appears.

Problem 10

- (i) **True.** $C^T = \frac{1}{n} \sum_i ((x_i - \bar{x})(x_i - \bar{x})^T)^T = C$ term by term.
- (ii) **False.** The first principal component is the eigenvector of the *largest* eigenvalue — the direction of maximum projected variance (§24.4); the smallest eigenvalue's eigenvector is the *first direction dropped* (§24.5).
- (iii) **True.** $J^*(m) = \sum_{j=m+1}^d \lambda_j$; raising m removes a nonnegative term ($\lambda_j \geq 0$ by Problem 3) from the sum.
- (iv) **False.** On ring data the eigenvalues are nearly equal — PCA keeps both directions and finds no 1-D structure (§24.10's figure); the circular relation is nonlinear.
- (v) **False.** Reconstructing $w_k = \sum_j \phi(x_j) \alpha_{kj}$ needs ϕ explicitly, which defeats the kernel trick's purpose; only the compressed coordinates $\sum_j \alpha_{kj} K_{ij}^c$ are computable (§24.10, Step 3).

25. GMMs and the EM algorithm

Two promises converge here. §24.11(iii) promised a different answer to “understand the data”: PCA compresses by *subspaces*; Chapter 25 compresses by *probability* — fit a mixture of Gaussians to the data instead of a flat floor through it. §22.13 sketched the density-estimation task (learn a probability mapping P with the smallest negative log-likelihood) and named the Gaussian mixture model as its illustration, deferring the model and how to fit it to this chapter. This chapter is both promises kept: the mixture model (§§25.2–25.5), why fitting it by maximum likelihood is hard (§25.6), and the iterative algorithm that fits it anyway (§§25.7–25.10), with every number worked by hand (§25.11). Everything comes from the MLF Week 12 lecture “Gaussian Mixture Models and Expectation Maximisation” (Prof. Harish Guruprasad Ramaswamy, IIT Madras) and the MLT Week 4 live-session problems on EM.

Notation. Data $D = \{x_1, \dots, x_n\}$, $x_i \in \mathbb{R}^d$ (same subscript convention as Chapter 24). The number of mixture components is K , fixed before fitting. Parameters are packaged as $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$. $N(x \mid \mu_k, \Sigma_k)$ is the Gaussian density from §19.6: in d dimensions,

$$N(x \mid \mu_k, \Sigma_k) = \frac{1}{(2\pi)^{d/2} \sqrt{\det \Sigma_k}} \exp\left(-\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)\right),$$

and in $d = 1$, $N(x \mid \mu_k, \sigma_k^2) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right)$.

25.1 The other answer to “understand the data”

PCA (§24) looked at unlabelled data and asked for the best low-dimensional *floor* through it — a subspace. There is a second, equally natural question: **what probability distribution generated these points?** That is density estimation (§22.13): learn a function $p(x)$ that is large where the data is dense and small elsewhere, integrating to 1.

- i) **PCA compresses by geometry.** Keep the directions of maximum variance; throw away the rest.
- ii) **GMMs compress by probability.** Don’t project anything — instead, describe the data’s *shape* as a weighted sum of Gaussian hills.
- iii) Same unsupervised spirit (§22.11): no labels, no y_i . Different model of what “structure” means.

Basically, ... “PCA says: the data is a cloud — squash it onto the best flat floor. GMMs say: the data is several overlapping clouds — describe each cloud by its own bell curve, and say how much of the data each bell owns.”

25.2 The motivating example: three types of students

The lecture’s motivating example. An exam (multiple choice, with negative marking) is written by three types of students:

- i) Type 1 mostly picks option A (doesn’t study): scores $\sim$ normal with mean -4 , variance 0.5 — the **red** curve.

- ii) Type 2 picks A/B/C/D uniformly at random: scores $\sim$ normal with mean 0, variance 1 — the **green** curve.
- iii) Type 3 studies and answers properly: scores $\sim$ normal with mean 5, variance 1 — the **blue** curve.

40% of students are type 1, 30% type 2, 30% type 3. Nobody advertises their type: the teacher only sees a column of marks. The histogram of many marks has **three peaks** — near -4 , near 0, near 5 — a *multimodal* distribution.

The point. Every distribution met so far (normal, exponential, uniform...) has a *single* peak. No single-peak density can capture three-peaked data. The right model is a **mixture of Gaussians**: $0.4 \times$ (red curve) $+ 0.3 \times$ (green curve) $+ 0.3 \times$ (blue curve). Not plain addition — each curve already integrates to 1, so adding them integrates to 3; the weights 0.4, 0.3, 0.3 rescale so the total integrates to 1, and they record *what fraction of students live in each mode*.

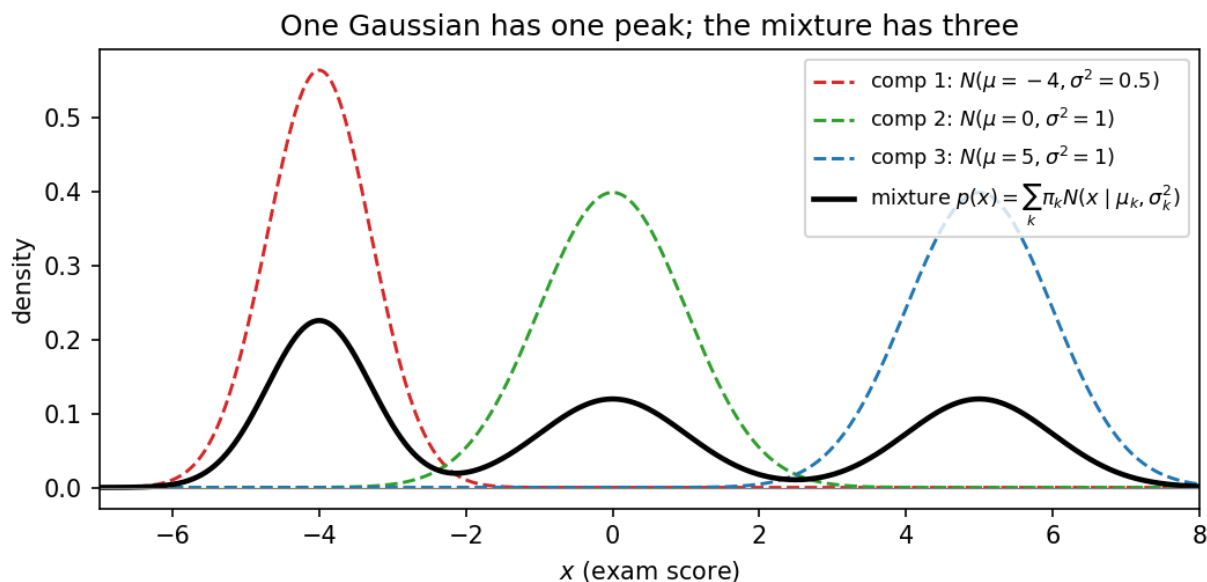


Figure 34: The three Gaussian components of the lecture’s exam-score example (dashed red, green, blue) and their weighted mixture (solid black): one Gaussian has one peak, the mixture has three.

Basically, ... “The class is secretly three different crowds mixed together. Each crowd’s scores form one bell curve. The overall histogram is the three bells stacked with weights 0.4, 0.3, 0.3 — which is why it has three humps no single bell could make.”

Note (adding densities $\neq$ adding random variables). If you *add* normal random variables you get another normal (§19.7) — one peak, always. Adding *densities* is a different operation entirely: it produces the mixture. Don’t confuse the two.

25.3 The Gaussian mixture model

Def (Gaussian mixture density). With K components,

$$f_X(x) = \sum_{k=1}^K \pi_k N(x | \mu_k, \Sigma_k).$$

Each component k is a full Gaussian (§19.6); the **mixture weights** π_k blend them.

The parameters. For each component k : - π_k — a scalar: *what fraction of the data comes from component k* ; $\pi_k \geq 0$ and $\sum_{k=1}^K \pi_k = 1$. - μ_k — the component’s mean (d numbers). - Σ_k — the component’s covariance matrix ($d(d+1)/2$ numbers, §19.6).

- i) **Why the weights.** Each $N(x \mid \mu_k, \Sigma_k)$ integrates to 1; the weights make the mixture integrate to $\sum_k \pi_k = 1$ — a valid density.
- ii) **The exam example is $K = 3$, $d = 1$:** $\pi = (0.4, 0.3, 0.3)$, $\mu = (-4, 0, 5)$, $\sigma^2 = (0.5, 1, 1)$.
- iii) **K is an input, not estimated.** The lecture assumes the number of components is known a priori; in practice people try several values of K .

eg 1 (reading the mixture density). For the exam example, the density at $x = 0$: - red: $0.4 \cdot N(0 \mid -4, 0.5) = 0.4 \cdot \frac{1}{\sqrt{2\pi \cdot 0.5}} e^{-16} \approx 2.5 \times 10^{-8}$, - green: $0.3 \cdot N(0 \mid 0, 1) = 0.3 \cdot \frac{1}{\sqrt{2\pi}} \approx 0.1197$, - blue: $0.3 \cdot N(0 \mid 5, 1) = 0.3 \cdot \frac{1}{\sqrt{2\pi}} e^{-12.5} \approx 4.5 \times 10^{-7}$,

$$f_X(0) \approx \boxed{0.1197},$$

almost entirely the green component’s doing — a score of 0 is green territory. (Problem 1 asks for $f_X(-4)$ and $f_X(5)$: same arithmetic, different winner.)

Basically, ... “A GMM is K bell curves plus K percentages. To score a point, ask each bell how likely the point is, weight by that bell’s percentage, and add. The percentages are the model’s way of saying how much of the data each bell owns.”

25.4 First caution: the component labels are meaningless

Def (unidentifiability by label swap). Reordering the components changes nothing about the density.

eg 2 (the lecture’s swap). Two-component model, parameter set A: $\pi_1 = 0.5, \mu_1 = 1, \sigma_1^2 = 1$; $\pi_2 = 0.5, \mu_2 = -1, \sigma_2^2 = 1$. Parameter set B: $\pi_1 = 0.5, \mu_1 = -1, \sigma_1^2 = 1$; $\pi_2 = 0.5, \mu_2 = 1, \sigma_2^2 = 1$. As vectors these look different — but

$$f_A(x) = 0.5 N(x \mid 1, 1) + 0.5 N(x \mid -1, 1) = f_B(x),$$

purely because addition commutes. No data can ever distinguish A from B.

The rule. Indexing is bookkeeping, nothing more: pick an order and stay consistent. (This also means the fitted model is never unique — a fact §25.10 will need.)

Basically, ... “Calling the left bell ‘component 1’ and the right bell ‘component 2’ — or the other way round — describes the same mixture. The names are stickers; the density doesn’t read them.”

25.5 The latent-variable view: naming the hidden crowd

Same model, more useful notation. Introduce a **latent (hidden) variable** $Z = \text{which component generated the point}$.

- i) **Component choice:** $P(Z = k) = \pi_k$ — rolling a K -faced die with face probabilities $\pi_1, \dots, \pi_K$ (in the exam story: which *type* of student you are).
- ii) **Point given the component:** $X \mid Z = k \sim N(\mu_k, \Sigma_k)$ — once the type is fixed, the score is that type’s Gaussian.
- iii) **Recovering the mixture (marginalization):** Z is summed out,

$$P(X) = \sum_{k=1}^K P(Z = k) P(X \mid Z = k) = \sum_{k=1}^K \pi_k N(x \mid \mu_k, \Sigma_k),$$

exactly §25.3’s density. Nothing new mathematically — just a name for the crowd membership.

Generative story. To draw one data point: (1) sample $Z = k$ with probability π_k ; (2) sample x from $N(\mu_k, \Sigma_k)$. If you could *see* Z , the data would arrive colour-coded — red points from the red bell, green from green, blue from blue — and everything would be easy (§25.7). You can't: Z is hidden. The lecture is explicit that “probabilities and densities” are used interchangeably here; the level of rigour doesn't need more.

Basically, ... “Give every point a secret colour = which bell made it. If you knew the colours, fitting would be trivial. You don't — the colours are invisible — but naming them (Z) lets us talk precisely about what we're missing.”

25.6 The MLE question — and why it is ugly

We fit parametric families by maximum likelihood (§20.3). Try it here. Data $x_1, \dots, x_n$, K known, parameters $\theta = \{\pi_k, \mu_k, \Sigma_k\}$.

Likelihood. By independence (§20.2), the likelihood is the product of one mixture density per point:

$$L(\theta) = \prod_{i=1}^n \left(\sum_{k=1}^K \pi_k N(x_i \mid \mu_k, \Sigma_k) \right).$$

Negative log-likelihood (the risk $R(\theta)$, §20.2):

$$R(\theta) = - \sum_{i=1}^n \log \left(\sum_{k=1}^K \pi_k N(x_i \mid \mu_k, \Sigma_k) \right).$$

The problem. There is a **summation inside the log**. A product inside a log splits into a sum (§20.2: $\log \prod = \sum \log$) — clean. A sum inside a log does not split — ugly. You can still write down the gradient of $R(\theta)$, but setting it to 0 and solving for the parameters is not possible in closed form: every parameter appears inside every log-of-sum. (Problem 3 makes this concrete for μ_1 .)

- i) **Not hopeless, just unsatisfying.** Generic optimizers (gradient descent, §10) could attack $R(\theta)$ directly — but the lecture wants a more intuitive solution.
- ii) **The intuition comes from §25.5's colours.** The difficulty is entirely the hidden Z : if we knew which point came from which component, estimation would be Chapter 20 again. That observation is the whole algorithm.

Basically, ... “log(sum) is the villain. log of a product breaks apart nicely; log of a sum refuses. So the usual move — differentiate, set to zero, solve — dies on the spot. The way out: stop fighting the sum, and use the hidden colours instead.”

25.7 The chicken-and-egg insight

Two facts, both easy, each assuming the other:

- i) **If you knew the colours (Z), the parameters are trivial.** Take all red points: their sample mean estimates μ_{red} (§20.6), their sample covariance estimates Σ_{red} (§20.7), and the fraction of red points estimates π_{red} . Ridiculously easy — just Chapter 20 per colour.
- ii) **If you knew the parameters, the colours are (mostly) easy.** Handed $x = 0.2$ with the exam parameters, you'd bet green with near-certainty; $x = 6.2$ screams blue; $x = -5.3$ screams red. Only boundary points are genuinely ambiguous — $x = -2$ could be red or green (§25.8 computes exactly how ambiguous).

The catch. You have neither. This is the **chicken-and-egg problem**: parameters give colours, colours give parameters, and you're holding nothing.

The standard escape (the lecture's analogy): *create a random chicken first*. Initialize the parameters to some valid random values. From that chicken hatch an egg (compute colours). From that egg hatch a better

chicken (recompute parameters). Repeat — and hope the generations converge to a realistic chicken *and* egg. That loop is the EM algorithm.

Basically, ... “Parameters and colour-assignments each determine the other. Since you have neither, guess one at random, derive the other, derive the first back — and keep going. Each round’s guess is a little less random than the last.”

25.8 The E-step: responsibilities

Fix the parameters. For point x_i , ask: *what is the probability it came from component k ?* That is $P(Z_i = k | x_i)$ — and Bayes’ rule (§14) answers it, since we know both pieces ($P(Z_i = k) = \pi_k$, $X_i | Z_i = k \sim N(\mu_k, \Sigma_k)$):

$$\gamma(z_{ik}) := P(Z_i = k | x_i) = \frac{\pi_k N(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x_i | \mu_j, \Sigma_j)}.$$

Def (responsibilities). $\gamma(z_{ik})$ is the **responsibility** component k takes for point x_i : how much of the blame/credit for that point belongs to bell k .

- i) **Rows sum to 1:** $\sum_{k=1}^K \gamma(z_{ik}) = 1$ — the numerator summed over k is the denominator. Every point’s total responsibility is fully distributed.
- ii) **Soft, not hard:** $\gamma(z_{ik}) \in [0, 1]$. A point is not *assigned* to one component; each component takes a *share* of it.
- iii) **The extreme case:** if $\gamma(z_{ik})$ is 1 for exactly one k and 0 for the rest, the point is fully assigned — keep this case in mind for §25.9.

eg 3 (the lecture’s ambiguous point, computed). Exam parameters; $x = -2$: - numerator: red $0.4 \cdot N(-2 | -4, 0.5) = 0.4 \cdot 0.010333 = 0.004133$; green $0.3 \cdot N(-2 | 0, 1) = 0.3 \cdot 0.053991 = 0.016197$; blue ≈ 0 ; - denominator: $0.004133 + 0.016197 \approx 0.020331$;

$$\gamma = (0.2033, 0.7967, 0.0000).$$

So $x = -2$ is 79.7% green, 20.3% red — exactly the lecture’s “could have come from either” made quantitative.

eg 4 (the unambiguous point). $x = 0.2$: green numerator $0.3 \cdot 0.391043 = 0.117313$, denominator ≈ 0.117314 ,

$$\gamma \approx (0.00000, 0.99999, 0.00001),$$

i.e. green takes essentially full responsibility — the lecture’s “almost certainly the green cluster”, now with the number attached.

Basically, ... “The E-step re-colours every point *softly*: instead of stamping ‘red’ on it, you write down a probability vector — 20% red, 80% green. Bayes’ rule does the stamping: each bell’s claim is its weight times its density at the point, normalized.”

25.9 The M-step: responsibility-weighted MLE

Now reverse direction: responsibilities known, re-estimate the parameters. Start from the extreme case (§25.8(iii)), where each point belongs to exactly one component:

- μ_k = sample mean of the points assigned to k (§20.6),
- Σ_k = sample covariance of the points assigned to k (§20.7),
- π_k = fraction of points assigned to k .

With soft responsibilities, “the points assigned to k ” becomes “all points, weighted by $\gamma(z_{ik})$ ”. Define the **effective count**

$$N_k = \sum_{i=1}^n \gamma(z_{ik})$$

(“a proxy for the number of points assigned to component k ” — the lecture’s phrase). Then:

$$\mu_k = \frac{1}{N_k} \sum_{i=1}^n \gamma(z_{ik}) x_i, \quad \pi_k = \frac{N_k}{n},$$

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^n \gamma(z_{ik}) (x_i - \mu_k)(x_i - \mu_k)^T.$$

- i) **Each is Chapter 20’s MLE with weights.** Set all $\gamma(z_{ik}) \in \{0, 1\}$ and these collapse to §20.6’s $\hat{\mu} = \bar{x}$, §20.7’s $\hat{\Sigma}$, and the plain fraction — the unweighted MLEs.
- ii) **Denominator n , not $n - 1$** — same as §20.6’s Note: MLE averages, it doesn’t debias.
- iii) **Where they come from.** Holding the responsibilities fixed, each update maximizes the responsibility-weighted log-likelihood. For μ_k in $d = 1$: minimizing $-\sum_i \gamma(z_{ik}) \log N(x_i | \mu_k, \sigma_k^2)$ and setting the derivative to 0 gives $-\sum_i \gamma(z_{ik})(x_i - \mu_k)/\sigma_k^2 = 0$, i.e. the μ_k formula above. The Σ_k formula is the same weighted-MLE argument as §20.7; $\pi_k = N_k/n$ follows from maximizing $\sum_i \sum_k \gamma(z_{ik}) \log \pi_k$ under $\sum_k \pi_k = 1$ (Lagrange, §11 — Problem 7).

Note. The lecture presents these updates intuitively (“the sample mean/covariance of the points the component is responsible for”) and notes the full theory behind *why* they are the right updates belongs to a later course. The formulas above *are* those updates.

Basically, ... “The M-step rebuilds each bell from the points it is responsible for — but a point that is 80% green counts as 0.8 of a green point. Mean = weighted average, covariance = weighted spread, weight = total responsibility divided by n . It is Chapter 20’s MLE, wearing responsibility-weights.”

25.10 The EM algorithm: alternate until convergence

Put §§25.8–25.9 together. **EM = Expectation–Maximisation.**

- i) **Initialize** π_k, μ_k, Σ_k to valid random values (the random chicken).
- ii) **E-step:** compute all responsibilities $\gamma(z_{ik})$ from the current parameters (§25.8).
- iii) **M-step:** recompute π_k, μ_k, Σ_k from the responsibilities (§25.9).
- iv) **Repeat (ii)–(iii)** until convergence — the parameters (or the log-likelihood) stop changing.

What the lecture actually promises. Honestly, not much beyond the loop: “you can keep on alternating till you have reached convergence”, with the *hope* that each generation’s chicken is more realistic than the last. The theorem-level story (why these updates are principled, what exactly converges) is deferred to a later course. So here is the precise, no-more-than-is-known status:

- i) **No global-optimum guarantee.** EM climbs to a *local* optimum / stationary point of the likelihood, not necessarily the best one. Where you land depends on where you start — **initialization matters**.
- ii) **Non-uniqueness is built in.** §25.4’s label swaps mean the optimum was never unique to begin with; different runs may return permuted-but-identical solutions.
- iii) **K stays fixed** throughout — it was your input (§25.3(iii)), not something EM discovers.

Basically, ... “Guess, re-colour, rebuild, repeat — until nothing moves anymore. It works remarkably well in practice, but it is hill-climbing from a random start: different starts can reach different hills. The lecture shows you the loop; the proof that the loop is sensible is a later course’s job.”

25.11 Worked: two EM iterations by hand

Data (1-D), $n = 6$: $x = \{-3, -2, -1, 4, 5, 6\}$ — two visible clusters. $K = 2$. Deliberately mediocre initialization:

$$\pi_1 = \pi_2 = 0.5, \quad \mu_1 = -1, \mu_2 = 2, \quad \sigma_1^2 = \sigma_2^2 = 4.$$

(Initial log-likelihood: -17.468 .)

Iteration 1, E-step. Responsibilities $\gamma(z_{i1}), \gamma(z_{i2})$:

x_i	-3	-2	-1	4	5	6
$\gamma(z_{i1})$	0.9325	0.8670	0.7549	0.0675	0.0331	0.0159
$\gamma(z_{i2})$	0.0675	0.1330	0.2451	0.9325	0.9669	0.9841

eg, $x = -3$: $N(-3 | -1, 4) = \frac{1}{\sqrt{8\pi}} e^{-4/8} = 0.199471 \cdot e^{-0.5} = 0.120985$; $N(-3 | 2, 4) = 0.199471 \cdot e^{-25/8} = 0.008764$; $\gamma(z_{i1}) = \frac{0.5 \cdot 0.120985}{0.5 \cdot 0.120985 + 0.5 \cdot 0.008764} = \frac{0.120985}{0.129749} = \boxed{0.9325}$. (Every row sums to 1 — check the first: $0.9325 + 0.0675 = 1.0000$.)

Iteration 1, M-step. Effective counts: $N_1 = 0.9325 + 0.8670 + 0.7549 + 0.0675 + 0.0331 + 0.0159 = \boxed{2.6709}$, $N_2 = 6 - 2.6709 = \boxed{3.3291}$. - $\mu_1 = \frac{0.9325(-3) + 0.8670(-2) + 0.7549(-1) + 0.0675(4) + 0.0331(5) + 0.0159(6)}{2.6709} = \frac{-4.7553}{2.6709} = \boxed{-1.78}$, - $\mu_2 = \frac{0.0675(-3) + 0.1330(-2) + 0.2451(-1) + 0.9325(4) + 0.9669(5) + 0.9841(6)}{3.3291} = \frac{13.7553}{3.3291} = \boxed{4.13}$, - $\sigma_1^2 = 2.48$, $\sigma_2^2 = 5.73$ (weighted average squared deviations about the *new* means), - $\pi_1 = 2.6709/6 = \boxed{0.445}$, $\pi_2 = \boxed{0.555}$. Log-likelihood after the M-step: $\boxed{-14.360}$ (up from -17.468).

Iteration 2, E-step (with the updated parameters):

x_i	-3	-2	-1	4	5	6
$\gamma(z_{i1})$	0.9871	0.9698	0.9148	0.0015	0.0001	0.0000
$\gamma(z_{i2})$	0.0129	0.0302	0.0852	0.9985	0.9999	1.0000

Iteration 2, M-step. $N_1 = 2.8734$, $N_2 = 3.1266$;

$$\boxed{\mu_1 = -2.02, \mu_2 = 4.74}, \quad \boxed{\sigma_1^2 = 0.68, \sigma_2^2 = 2.29}, \quad \boxed{\pi_1 = 0.479, \pi_2 = 0.521}.$$

Log-likelihood: $\boxed{-12.294}$ — up again.

Reading the numbers. The means walk toward the true clusters: $\mu_1 : -1 \rightarrow -1.78 \rightarrow -2.02$ (left cluster sits at ≈ -2), $\mu_2 : 2 \rightarrow 4.13 \rightarrow 4.74$ (right cluster at ≈ 5). Responsibilities *sharpen* — $x = -3$ goes from 93% to 98.7% component-1 — and the inflated initial variances collapse as each bell stops trying to cover both clusters. Run to 20 iterations: $\mu = (-2.00, 5.00)$, $\sigma^2 = (0.667, 0.667)$, $\pi = (0.5, 0.5)$, log-likelihood -11.456 , stable to 4 decimals — the sample statistics of the two clusters, recovered from a bad start.

EM pulls the two weighted Gaussians onto the two clusters

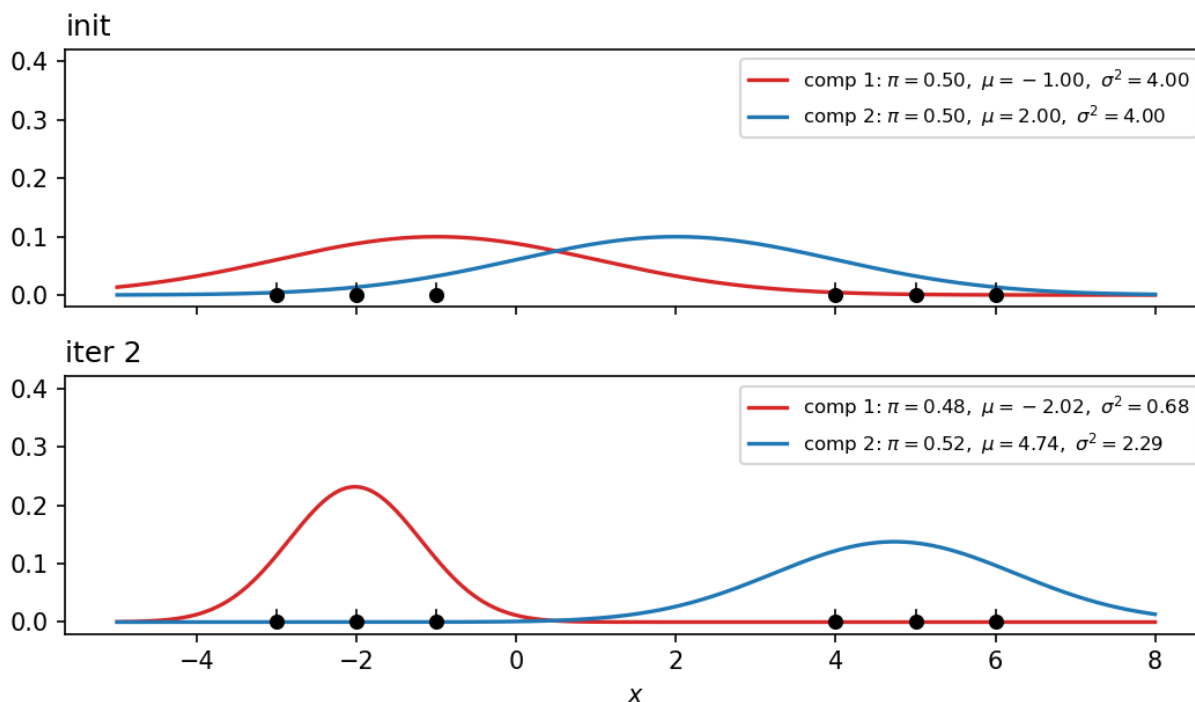


Figure 35: Weighted component Gaussians of the hand-worked example at initialization (top: two broad overlapping bells) and after two EM iterations (bottom: one bell on each cluster), with the six data points as black dots.

Basically, ... “Two rounds are enough to watch the magic: the bells slide onto the clusters, the colour-assignments harden, the likelihood climbs. EM is just §§25.8–25.9 on repeat — and the numbers above are the proof that repeating works.”

25.12 Where this goes next — and the close of Part III

- i) **Clustering, the soft way.** The lecture’s closing application: run EM on a GMM and you have “one of the building blocks of clustering”. Each point ends with a responsibility vector, not a single label — *soft* clustering. (The hard-assignment cousin, k-means, is Chapter 26’s job.)
- ii) **The theory behind the updates.** The lecture is upfront: the updates are intuitively right, and they are also “theoretically well founded” — but that foundation is a later course’s material, not this chapter’s.
- iii) **Part III, closed.** §22.11 framed unsupervised learning as “understanding data” and named two tasks: dimensionality reduction and density estimation. Chapter 24 answered with *subspaces* (PCA: the best floor through the data). This chapter answers with *probability* (GMMs: the best mixture of bells for the data, fitted by EM). Same spirit, different model of structure — the unsupervised arc is complete.
- iv) **What Part IV adds.** The models so far were few and classical — regression, PCA, mixtures. Part IV (MLT) widens the toolbox: k-means, kernels, regularization, trees, SVMs, ensembles. The probability machinery of Part II stays underneath all of it.

Basically, ... “Part III in one line: Chapter 22 asked what ML is, 23 fitted lines, 24 found floors, 25 fitted bells. You now own the classical unsupervised toolkit — compress by subspace, or describe by mixture. Part IV spends that toolkit on the modern zoo.”

Problem set

- Trimodal arithmetic (the exam example).** With $\pi = (0.4, 0.3, 0.3)$, $\mu = (-4, 0, 5)$, $\sigma^2 = (0.5, 1, 1)$: (i) compute $f_X(-4)$ and $f_X(5)$ the way eg 1 computed $f_X(0)$. (ii) The three values are $\approx 0.2257, 0.1197, 0.1197$ — which peak of the histogram is tallest, and why (weights $\times$ widths)?
- Labels don't matter.** Write the mixture density for the lecture's parameter set A ($\pi_1 = 0.5, \mu_1 = 1, \sigma_1^2 = 1$; $\pi_2 = 0.5, \mu_2 = -1, \sigma_2^2 = 1$) and for set B (same, with μ_1, μ_2 swapped). Show the two densities are identical functions.
- Why direct MLE is ugly (guided).** 1-D data $\{a, b\}$, $K = 2$, $\sigma_1^2 = \sigma_2^2 = 1$ and $\pi_1 = \pi_2 = \frac{1}{2}$ known; estimate μ_1, μ_2 . (i) Write $R(\mu_1, \mu_2)$. (ii) Compute $\partial R / \partial \mu_1$ and express it through the responsibilities $\gamma(z_{i1})$. (iii) Set it to 0: what equation must μ_1 satisfy? Explain why this is not a closed-form solution (where does μ_1 appear?).
- E-step arithmetic (MLT Week 4 live session).** A 3-component GMM; for $x = 6$: $P(x | z = 1) = 0.2$, $P(x | z = 2) = 0.3$, $P(x | z = 3) = 0.5$; $\pi_1 = 0.4$, $\pi_2 = 0.3$, $\pi_3 = 0.3$. Compute the posterior $P(z = k | x)$ for $k = 1, 2, 3$. Which component most likely generated $x = 6$?
- Fill the responsibilities (MLT Week 4 live session).** 1-D data $X = [-3, -1, 1, 3]$, $K = 3$. An E-step produced

$$R = \begin{pmatrix} 0.3 & a & 0.5 \\ b & 1 & b \\ c & 0.5 & c \\ 0.1 & 0.4 & d \end{pmatrix},$$

each row = one point's responsibilities. (i) Use $\sum_k R_{ik} = 1$ to find a, b, c, d . (ii) Compute the M-step means μ_1, μ_2, μ_3 and their sum.

- M-step arithmetic (MLT Week 4 live session).** Data $\{2, 1, 3, 2, 1, 2, 0, -3, 0, 2\}$ ($n = 10$), $K = 3$. At some EM round the E-step gave, for component 3, $\lambda_{3i} = P(z_i = 3 | x_i)$: $[0.2, 0.3, 0.1, 0.6, 0.1, 0.5, 0.4, 0.7, 0.1, 0.2]$. Compute π_3 and μ_3 after the M-step.
- The π_k update, derived.** With responsibilities fixed, the π -dependent part of the objective is $\sum_{i=1}^n \sum_{k=1}^K \gamma(z_{ik}) \log \pi_k$, subject to $\sum_{k=1}^K \pi_k = 1$. Use a Lagrange multiplier (§11) to show $\pi_k = N_k/n$.
- True or false, one-line justification:** (i) $\sum_{k=1}^K \gamma(z_{ik}) = 1$ for every data point i . (ii) EM is guaranteed to find the global maximum of the likelihood. (iii) If every $\gamma(z_{ik}) \in \{0, 1\}$, the M-step is exactly the per-cluster sample mean, sample covariance, and data fraction. (iv) Swapping two components' labels changes the fitted mixture density. (v) A GMM with $K = 1$ can capture trimodal data.

Solutions — Chapter 25: GMMs and the EM algorithm

1. Trimodal arithmetic.

(i) $f_X(x) = 0.4 N(x | -4, 0.5) + 0.3 N(x | 0, 1) + 0.3 N(x | 5, 1)$.

At $x = -4$: - red: $0.4 \cdot \frac{1}{\sqrt{2\pi \cdot 0.5}} e^0 = 0.4 \cdot 0.564190 = 0.225676$, - green: $0.3 \cdot \frac{1}{\sqrt{2\pi}} e^{-(-4)^2/2} = 0.3 \cdot 0.398942 \cdot e^{-8} = 0.119683 \cdot 0.000335 = 0.000040$, - blue: $0.3 \cdot \frac{1}{\sqrt{2\pi}} e^{-(-9)^2/2} \approx 0$,

$$\boxed{f_X(-4) \approx 0.2257}.$$

At $x = 5$: - blue: $0.3 \cdot \frac{1}{\sqrt{2\pi}} e^0 = 0.119683$, - green: $0.3 \cdot \frac{1}{\sqrt{2\pi}} e^{-25/2} = 0.119683 \cdot e^{-12.5} = 0.119683 \cdot 3.73 \times 10^{-6} \approx 0.000000$, - red ≈ 0 ,

$$\boxed{f_X(5) \approx 0.1197}.$$

- (ii) Tallest peak at $x = -4$ ($0.2257 > 0.1197$). Each peak's height is $\approx \pi_k / \sqrt{2\pi\sigma_k^2}$ (the other components contribute negligibly at a peak): red $\frac{0.4}{\sqrt{\pi}} = 0.2257$, green $\frac{0.3}{\sqrt{2\pi}} = 0.1197$, blue the same 0.1197 . Red wins twice over: largest weight (0.4) and narrowest bell ($\sigma^2 = 0.5$ concentrates its mass into a taller spike).

This is §22.13's "mass spent in one place is mass denied elsewhere" — the narrow red bell spends its 0.4 of mass in a tight neighbourhood, so its density there is high.

2. Labels don't matter.

Set A: $f_A(x) = 0.5 N(x | 1, 1) + 0.5 N(x | -1, 1)$.

Set B: $f_B(x) = 0.5 N(x | -1, 1) + 0.5 N(x | 1, 1)$.

For every x , $f_B(x) - f_A(x) = 0$ by commutativity of addition: the two summands are identical, only their order differs. Hence $f_A \equiv f_B$ as functions — no dataset, however large, can distinguish the two parameter vectors. This is §25.4's unidentifiability.

3. Why direct MLE is ugly.

(i) $N(x | \mu, 1) = \frac{1}{\sqrt{2\pi}} e^{-(x-\mu)^2/2}$, so

$$R(\mu_1, \mu_2) = -\log\left(\frac{1}{2}N(a | \mu_1, 1) + \frac{1}{2}N(a | \mu_2, 1)\right) - \log\left(\frac{1}{2}N(b | \mu_1, 1) + \frac{1}{2}N(b | \mu_2, 1)\right).$$

(ii) $\frac{\partial}{\partial \mu_1} \log N(x | \mu_1, 1) = x - \mu_1$. By the chain rule on each log-of-sum, with $D_i = \frac{1}{2}N(x_i | \mu_1, 1) + \frac{1}{2}N(x_i | \mu_2, 1)$ ($x_1 = a, x_2 = b$):

$$\frac{\partial R}{\partial \mu_1} = -\sum_{i=1}^2 \frac{\frac{1}{2} \partial_{\mu_1} N(x_i | \mu_1, 1)}{D_i} = -\sum_{i=1}^2 (x_i - \mu_1) \frac{N(x_i | \mu_1, 1)}{N(x_i | \mu_1, 1) + N(x_i | \mu_2, 1)}.$$

The fraction is exactly the responsibility (with $\pi_1 = \pi_2$ cancelling):

$$\boxed{\frac{\partial R}{\partial \mu_1} = -(a - \mu_1) \gamma(z_{11}) - (b - \mu_1) \gamma(z_{12})}.$$

(iii) Setting to zero: $\gamma(z_{11})(a - \mu_1) + \gamma(z_{12})(b - \mu_1) = 0$, i.e.

$$\mu_1 = \frac{\gamma(z_{11})a + \gamma(z_{12})b}{\gamma(z_{11}) + \gamma(z_{12})}.$$

Not a closed-form solution: $\gamma(z_{i1}) = N(x_i | \mu_1, 1) / [N(x_i | \mu_1, 1) + N(x_i | \mu_2, 1)]$ itself contains μ_1 (and μ_2) inside the Gaussian densities — μ_1 appears on both sides of the equation. It is a *fixed-point equation*, not a formula. This is precisely the lecture's point: you can write the gradient, but "you cannot set it to 0 and solve for it". (Note the silver lining: the fixed-point equation *is* the M-step update — EM solves it by iteration, freezing γ in the E-step and solving for μ_1 in the M-step.)

4. E-step arithmetic.

Numerators $\pi_k P(x | z = k)$: - $k = 1$: $0.4 \times 0.2 = 0.08$, - $k = 2$: $0.3 \times 0.3 = 0.09$, - $k = 3$: $0.3 \times 0.5 = 0.15$.

Denominator: $0.08 + 0.09 + 0.15 = 0.32$. Posteriors:

$$\boxed{P(z = 1 | x) = \frac{0.08}{0.32} = 0.25}, \quad \boxed{P(z = 2 | x) = \frac{0.09}{0.32} = 0.28125}, \quad \boxed{P(z = 3 | x) = \frac{0.15}{0.32} = 0.46875}.$$

Component 3 has the highest posterior — most likely to have generated $x = 6$. (Note: the largest *likelihood* alone doesn't decide; the posterior weights it by π_k . Here they agree, but in general the π_k factor matters.)

5. Fill the responsibilities.

(i) Rows must sum to 1:

- Row 1: $0.3 + a + 0.5 = 1 \Rightarrow \boxed{a = 0.2}$.
- Row 2: $b + 1 + b = 1 \Rightarrow \boxed{b = 0}$.
- Row 3: $c + 0.5 + c = 1 \Rightarrow \boxed{c = 0.25}$.

- Row 4: $0.1 + 0.4 + d = 1 \Rightarrow \boxed{d = 0.5}$.

$$R = \begin{pmatrix} 0.3 & 0.2 & 0.5 \\ 0 & 1 & 0 \\ 0.25 & 0.5 & 0.25 \\ 0.1 & 0.4 & 0.5 \end{pmatrix}.$$

(ii) M-step means $\mu_k = \sum_i R_{ik} x_i / \sum_i R_{ik}$ with $X = [-3, -1, 1, 3]$:

- $N_1 = 0.3 + 0 + 0.25 + 0.1 = 0.65$; $\mu_1 = \frac{0.3(-3) + 0(-1) + 0.25(1) + 0.1(3)}{0.65} = \frac{-0.9 + 0 + 0.25 + 0.3}{0.65} = \frac{-0.35}{0.65} = \boxed{-0.5385}$,
- $N_2 = 0.2 + 1 + 0.5 + 0.4 = 2.1$; $\mu_2 = \frac{0.2(-3) + 1(-1) + 0.5(1) + 0.4(3)}{2.1} = \frac{-0.6 - 1 + 0.5 + 1.2}{2.1} = \frac{0.1}{2.1} = \boxed{0.0476}$,
- $N_3 = 0.5 + 0 + 0.25 + 0.5 = 1.25$; $\mu_3 = \frac{0.5(-3) + 0(-1) + 0.25(1) + 0.5(3)}{1.25} = \frac{-1.5 + 0 + 0.25 + 1.5}{1.25} = \frac{0.25}{1.25} = \boxed{0.2}$.

$$\boxed{\mu_1 + \mu_2 + \mu_3 = -0.5385 + 0.0476 + 0.2 = -0.2909 \approx -0.29}.$$

(The MLT live-session slide reports $-0.53 + 0.04 + 0.2 = -0.29$ — same answer up to its one-decimal rounding.)

6. M-step arithmetic.

$\lambda_{3i} = [0.2, 0.3, 0.1, 0.6, 0.1, 0.5, 0.4, 0.7, 0.1, 0.2]$, $x = [2, 1, 3, 2, 1, 2, 0, -3, 0, 2]$, $n = 10$.

Effective count: $N_3 = \sum_i \lambda_{3i} = 0.2 + 0.3 + 0.1 + 0.6 + 0.1 + 0.5 + 0.4 + 0.7 + 0.1 + 0.2 = \boxed{3.2}$.

$$\boxed{\pi_3 = N_3/n = 3.2/10 = 0.32}.$$

Weighted sum: $\sum_i \lambda_{3i} x_i = 0.2(2) + 0.3(1) + 0.1(3) + 0.6(2) + 0.1(1) + 0.5(2) + 0.4(0) + 0.7(-3) + 0.1(0) + 0.2(2)$
 $= 0.4 + 0.3 + 0.3 + 1.2 + 0.1 + 1.0 + 0 - 2.1 + 0 + 0.4 = \boxed{1.6}$.

$$\boxed{\mu_3 = 1.6/3.2 = 0.5}.$$

7. The π_k update, derived.

Maximize $J(\pi) = \sum_{i=1}^n \sum_{k=1}^K \gamma(z_{ik}) \log \pi_k$ subject to $g(\pi) = \sum_{k=1}^K \pi_k - 1 = 0$. Lagrangian (§11):

$$\mathcal{L}(\pi, \lambda) = \sum_{i,k} \gamma(z_{ik}) \log \pi_k - \lambda \left(\sum_{k=1}^K \pi_k - 1 \right).$$

$$\frac{\partial \mathcal{L}}{\partial \pi_j} = \frac{\sum_{i=1}^n \gamma(z_{ij})}{\pi_j} - \lambda = \frac{N_j}{\pi_j} - \lambda = 0 \Rightarrow \pi_j = \frac{N_j}{\lambda}.$$

Summing over j and using the constraint: $1 = \sum_j \pi_j = \frac{1}{\lambda} \sum_j N_j$. But $\sum_j N_j = \sum_j \sum_i \gamma(z_{ij}) = \sum_i \underbrace{\sum_j \gamma(z_{ij})}_{=1} = n$. Hence $\lambda = n$ and

$$\boxed{\pi_k = \frac{N_k}{n}}.$$

8. True or false.

- (i) **True.** $\sum_k \gamma(z_{ik}) = \sum_k \frac{\pi_k N(x_i | \mu_k, \Sigma_k)}{\sum_j \pi_j N(x_i | \mu_j, \Sigma_j)} = 1$ — numerator sums to denominator (§25.8(i)).
- (ii) **False.** EM converges to a *local* optimum / stationary point of the likelihood; which one depends on the initialization. No global guarantee (§25.10).
- (iii) **True.** With $\gamma(z_{ik}) \in \{0, 1\}$, N_k is the count of points in cluster k , and the M-step formulas become exactly the per-cluster sample mean (§20.6), sample covariance (§20.7), and fraction (§25.9(i)).

- (iv) **False.** The mixture density is a sum — addition commutes — so permuted labels give the identical function (§25.4).
- (v) **False.** $K = 1$ is a single Gaussian: one peak, always. Trimodal data needs (at least) three components (§25.2).

Part IV — Machine Learning Techniques

26. K-means clustering

Part IV opens here. §25.12(iv) previewed this part: the modern ML toolbox — k-means, kernels, regularization, trees, SVMs, ensembles — with Part II’s probability machinery underneath. The toolbox opens with the oldest and simplest clustering algorithm there is, and it keeps a promise §25.12(i) made: EM on a GMM clusters *softly*, handing every point a responsibility vector; the hard-assignment cousin, k-means, is this chapter’s job. Same unsupervised spirit as Chapters 24–25 (§22.11) — a different goal: not compressing the data, not scoring it, but *grouping* it.

Everything in §§26.1–26.10 comes from the MLT Week 3 lecture notes on k-means (Lloyd’s algorithm) and the Week 3 practice assignment; the hand-worked example in §26.5 follows the lecture’s step order exactly, with every number recomputed independently (review log).

Notation. Data $D = \{x_1, \dots, x_n\}$, $x_i \in \mathbb{R}^d$ (subscript convention, Chapters 24–25). The number of clusters K is fixed *before* the algorithm runs — an input, like GMM’s K (§25.3(iii)). Each point carries a **cluster indicator** $z_i \in \{1, \dots, K\}$: $z_i = k$ means “ x_i belongs to cluster k ”. Cluster k ’s center is $\mu_k \in \mathbb{R}^d$, the **centroid**.

26.1 The clustering task: grouping unlabelled data

Def (clustering). Given unlabelled $x_1, \dots, x_n$, partition them into K groups — **clusters** — so that points in the same group are similar and points in different groups are dissimilar.

- i) **Unsupervised, §22.11’s third verb.** §22.11 says unsupervised learning compresses, explains, and *groups*. PCA compressed (§24), GMMs explained via probability (§25); k-means *groups*. No labels, no y_i — the groups are discovered, not taught.
- ii) **The use case (§22.11).** §22.11’s marketing manager: a million tweets a week, unreadable — group them into 10 clusters (selfies-with-Coke, co-branding, paid promotions, ...), then a human reads the 10 groups. The grouping is the algorithm’s job; giving the groups meaning is the human’s.
- iii) **K is your call.** The algorithm does not discover how many clusters exist; you hand it K up front (§26.8 is about choosing it).

Basically, ... “You have a pile of unlabelled points and a number K . Sort the pile into K heaps so each heap’s points huddle together. Nobody tells you the right heaps — ‘right’ has to be defined by the algorithm itself, which is §26.3’s job.”

26.2 How many partitions? Counting the assignments

Fix K . An assignment is a choice of $z_1, \dots, z_n$, each in $\{1, \dots, K\}$.

Count. n points, K independent choices each (clusters allowed to be empty):

K^n possible assignments

— the lecture’s count, stated exactly this way.

- i) **Brute force is hopeless.** K^n grows brutally: $n = 100$, $K = 3$ gives 3^{100} assignments. Nobody enumerates.
- ii) **So we iterate, not enumerate.** Lloyd's algorithm (§26.4) walks through assignments — each step either moves to a strictly better one or stops — instead of checking all K^n .
- iii) **Labels are bookkeeping.** As with GMM components (§25.4), permuting cluster labels changes nothing — assignments differing only by renaming clusters describe the same partition. The K^n count ignores this; it is an upper bound on *distinct* partitions, which is all the convergence argument (§26.4) needs.

eg 1 (counting, tiny). $n = 3$ points, $K = 2$: $2^3 = 8$ assignments — $(1, 1, 1), (1, 1, 2), (1, 2, 1), (1, 2, 2), (2, 1, 1), (2, 1, 2), (2, 2, 1), (2, 2, 2)$. Note $(1, 1, 2)$ and $(2, 2, 1)$ are the same partition with renamed labels — the double-counting (iii) mentions.

Basically, ... “Every point independently picks one of K buckets: K^n ways to fill them. For any real dataset that number is astronomical, so the algorithm never lists assignments — it improves its way to a good one instead.”

26.3 The k-means objective: within-cluster sum of squares

Given an assignment, how good is it? The lecture's answer: add up, over all points, the squared distance from each point to its own cluster's center.

Def (k-means objective). With μ_{z_i} = the center of the cluster assigned to x_i ,

$$F(z_1, \dots, z_n) = \sum_{i=1}^n \|x_i - \mu_{z_i}\|^2.$$

Also called the **distortion** or the **within-cluster sum of squares (WCSS)**: it measures how scattered each cluster is around its center. **Goal: minimize F .**

Def (centroid). The center μ_k is the *mean* of the points assigned to cluster k :

$$\mu_k = \frac{\sum_{i=1}^n x_i \mathbf{1}(z_i = k)}{\sum_{i=1}^n \mathbf{1}(z_i = k)}$$

— the lecture's formula: the cluster's points summed, divided by the cluster's headcount ($\mathbf{1}(\cdot)$ is the indicator, 1 when true, 0 when false).

- i) **Why the mean?** For a fixed assignment, the mean is what *minimizes* that cluster's contribution to F : differentiating $\sum_{i:z_i=k} \|x_i - \mu\|^2$ w.r.t. μ (componentwise, the §20.6 calculation) gives $-2 \sum_{i:z_i=k} (x_i - \mu) = 0$, i.e. $\mu = \text{average}$. The centroid isn't an arbitrary “center” — it is the optimal one for the points it serves.
- ii) **F is a loss, §22.11-style.** Unsupervised learning has no y_i to check against, so each task invents its own loss (§22.11(i)): PCA invented reconstruction error (§22.12), density estimation invented negative log-likelihood (§22.13), and clustering invents F .
- iii) **$F \geq 0$ always**, and $F = 0$ means every point sits exactly on its centroid — possible only if each cluster's points coincide, or (trivially) with $K = n$ (§26.8).

eg 2 (reading F). 1-D data $\{0, 2, 9, 10\}$, $K = 2$, assignment $z = (1, 1, 2, 2)$: - cluster 1: $\{0, 2\}$, $\mu_1 = 1$: contributions $(0 - 1)^2 + (2 - 1)^2 = 2$; - cluster 2: $\{9, 10\}$, $\mu_2 = 9.5$: contributions $0.25 + 0.25 = 0.5$;

$$F = 2.5.$$

A worse grouping scores worse — Problem 1 works this out on similar data, where a bad split scores $\frac{146}{3}$ against 1 for the good one. That sensitivity is the point: F *scores* assignments.

Basically, ... “ F is the total ‘unhappiness’: every point measures its squared distance to its cluster's center, and you add them all up. Good clustering = small F = every point snuggled close to its own center. And the center is just the average of its points — the one location that minimizes that cluster's unhappiness.”

26.4 Lloyd's algorithm: assign, re-mean, repeat

Minimizing F over K^n assignments by brute force is out (§26.2). The lecture's algorithm — **Lloyd's algorithm** — alternates two steps, each trivial, each unable to increase F .

Initialization. Pick $z_1^0, \dots, z_n^0 \in \{1, \dots, K\}$ — an initial assignment (§26.7 is about choosing it well).

Until convergence, repeat: - **Compute means.** For each k ,

$$\forall k, \quad \mu_k^t = \frac{\sum_{i=1}^n x_i \mathbf{1}(z_i = k)}{\sum_{i=1}^n \mathbf{1}(z_i = k)}$$

(the centroid formula, applied to the current assignment). - **Reassignment step.** For each i ,

$$\forall i, \quad z_i^{t+1} = \arg \min_k \|x_i - \mu_k^t\|^2$$

— every point moves to its *nearest* centroid. (Ties: keep the old label; it changes nothing — a convention, not in the lecture.)

Why each step is safe (neither can increase F): i) **The mean step** fixes the assignment and re-centers: by §26.3(i), the mean minimizes each cluster's squared-deviation sum, so recomputing μ_k can only *lower* (or leave) F . ii) **The reassignment step** fixes the centers and re-assigns: each point independently picks the k minimizing its own term $\|x_i - \mu_k^t\|^2$, so the sum F can only *lower* (or leave). Hence, with $F^t =$ the objective after iteration t :

$$F^{t+1} \leq F^t$$

— the practice assignment's answer, and the lecture's “every reassignment reduces the value of the objective function”, stated with exactly the strictness the sources support: *strictly* smaller whenever at least one point actually changes cluster — a point moves only if it finds a *strictly* closer center (the lecture's “unhappy” points).

Why it must stop (the lecture's convergence argument). In the lecture's four lines: i) In every iteration, only unhappy points change clusters. ii) Every reassignment that moves a point reduces F . iii) Every reassignment results in a *new* partition — F strictly dropped, so this partition cannot be one already visited (the practice solution's (a)+(c)+(e): partitions never repeat, and a change in F means the partition changed). iv) There are only finitely many partitions ($\leq K^n$, §26.2) — a strictly decreasing sequence over a finite set must end. The algorithm converges.

What “converges” does NOT mean (practice assignment, Q6/Q9 — stated exactly as sourced): i) **No global optimum.** Different initializations can land in different local minima of F — “one initialization might get stuck in local minima, while another may lead to global minima” (Q9(b)). ii) **Init affects speed.** The number of iterations to converge depends on the start (Q9(d)). iii) **It always stops.** Q9(c) (“one initialization may converge while another may not”) is *false* — the finiteness argument holds for every start. iv) **K^n bounds partitions, not iterations** — Q6(f) is false: K^n is only the upper limit on possible partitions, not the iteration count (in practice Lloyd stops far, far earlier).

Note (empty clusters). The K^n count allows empty clusters, but the centroid formula divides by the cluster's headcount — a cluster that empties out breaks the formula. The lecture prescribes no fix; in practice, re-seed or drop the empty center. (Flagged in the review log as a point the lecture leaves open.)

Basically, ... “Two dance steps, repeated: (1) each cluster's center moves to its points' average; (2) each point defects to its nearest center. Neither step can ever make F worse — so F ratchets down every round. There are only finitely many assignments, so the ratchet must click to a stop. *Where* it stops depends on where you started: Lloyd finds a local valley of F , not necessarily the deepest one.”

26.5 Worked: one Lloyd iteration by hand

Six points in 2-D, $K = 2$ — deliberately bad initial assignment so things move:

$$x_1 = (1, 1), x_2 = (1, 2), x_3 = (2, 1), x_4 = (6, 6), x_5 = (6, 7), x_6 = (7, 6),$$

$$z^0 = (1, 1, 2, 1, 2, 2)$$

(points 1, 2, 4 start in cluster 1; 3, 5, 6 in cluster 2).

Step 1 — compute means. Cluster 1: $\{x_1, x_2, x_4\}$, $\mu_1^0 = (\frac{1+1+6}{3}, \frac{1+2+6}{3}) = \boxed{(\frac{8}{3}, 3)}$. Cluster 2: $\{x_3, x_5, x_6\}$, $\mu_2^0 = (\frac{2+6+7}{3}, \frac{1+7+6}{3}) = \boxed{(5, \frac{14}{3})}$.

Initial objective (each point to its *assigned* center): - cluster 1: $\|x_1 - \mu_1^0\|^2 = \frac{61}{9}$, $\|x_2 - \mu_1^0\|^2 = \frac{34}{9}$, $\|x_4 - \mu_1^0\|^2 = \frac{181}{9} \rightarrow \frac{276}{9}$; - cluster 2: $\|x_3 - \mu_2^0\|^2 = \frac{202}{9}$, $\|x_5 - \mu_2^0\|^2 = \frac{58}{9}$, $\|x_6 - \mu_2^0\|^2 = \frac{52}{9} \rightarrow \frac{312}{9}$;

$$F^0 = \frac{276+312}{9} = \frac{196}{3} \approx 65.33.$$

Step 2 — reassign (squared distances to the two centers; smaller wins):

point	to μ_1^0	to μ_2^0	new z
x_1	$\frac{61}{9} \approx 6.78$	$\frac{265}{9} \approx 29.44$	1
x_2	$\frac{34}{9} \approx 3.78$	$\frac{208}{9} \approx 23.11$	1
x_3	$\frac{40}{9} \approx 4.44$	$\frac{202}{9} \approx 22.44$	1 (was 2)
x_4	$\frac{181}{9} \approx 20.11$	$\frac{25}{9} \approx 2.78$	2 (was 1)
x_5	$\frac{244}{9} \approx 27.11$	$\frac{58}{9} \approx 6.44$	2
x_6	$\frac{250}{9} \approx 27.78$	$\frac{52}{9} \approx 5.78$	2

eg, $x_3 = (2, 1)$: to μ_1^0 : $(2 - \frac{8}{3})^2 + (1 - 3)^2 = \frac{4}{9} + 4 = \frac{40}{9}$; to μ_2^0 : $(2 - 5)^2 + (1 - \frac{14}{3})^2 = 9 + \frac{121}{9} = \frac{202}{9}$. Closer to μ_1^0 — it defects. Two unhappy points move (x_3 : 2 $\rightarrow$ 1, x_4 : 1 $\rightarrow$ 2); the rest stay.

$$z^1 = (1, 1, 1, 2, 2, 2).$$

Step 3 — recompute means. $\mu_1^1 = (\frac{1+1+2}{3}, \frac{1+2+1}{3}) = \boxed{(\frac{4}{3}, \frac{4}{3})}$; $\mu_2^1 = (\frac{6+6+7}{3}, \frac{6+7+6}{3}) = \boxed{(\frac{19}{3}, \frac{19}{3})}$ — the centers have slid onto the two natural blobs.

$$F^1 = \underbrace{\frac{2}{9} + \frac{5}{9} + \frac{5}{9}}_{\text{cluster 1}} + \underbrace{\frac{2}{9} + \frac{5}{9} + \frac{5}{9}}_{\text{cluster 2}} = \boxed{\frac{8}{3} \approx 2.67},$$

down from 65.33. **Convergence check:** reassigning with μ_1^1, μ_2^1 moves no point (eg x_3 : $\frac{5}{9}$ vs $\frac{425}{9}$ — stays), so $z^2 = z^1$ and the algorithm stops. Every number above was recomputed independently (fractions and numpy) in the review log.

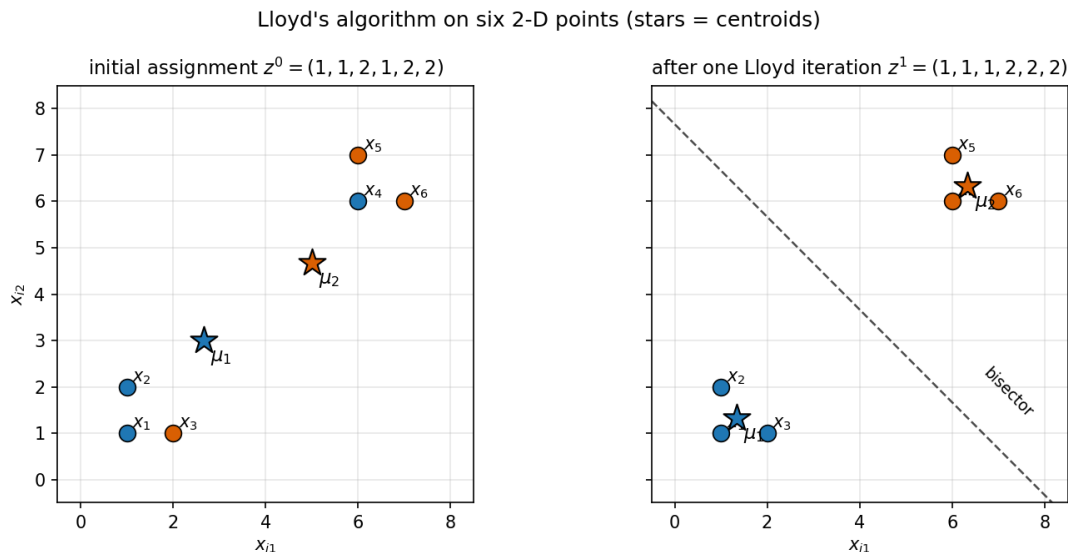


Figure 36: Lloyd's algorithm on the six 2-D points of the hand-worked example: initial assignment (left) vs after one iteration (right); stars are centroids; the dashed line is the perpendicular bisector of the final centers.

Basically, ... “One round is enough to watch it work: the centers jump to their points' averages, the two misplaced points defect to the nearer center, the centers jump again onto the real blobs, and F collapses from 65.3 to 2.7. Then nothing wants to move — done.”

26.6 The shape of the clusters: half-spaces and bisectors

Fix the final centers. Which points of the plane would join cluster 1 vs cluster 2? The lecture's geometric answer:

- i) **The boundary between two centers is a perpendicular bisector.** The set of points equidistant from μ_1, μ_2 — $\|x - \mu_1\| = \|x - \mu_2\|$ — is the hyperplane perpendicular to the segment $\mu_1\mu_2$ through its midpoint (practice Q3; Problem 4 proves it).
- ii) **Each cluster's region is an intersection of half-spaces.** Cluster k 's region = points closer to μ_k than to *every* other center = the intersection of the $K - 1$ half-spaces cut by the bisectors with centers $1, \dots, K$ (excluding k) — the lecture's statement.
- iii) **Consequence.** Every cluster region is convex (an intersection of convex half-spaces, §§12.3–12.4) and polygonal/polyhedral. k-means can only draw *straight* boundaries — it will never carve out a crescent or a ring.

eg 3 (the worked example's bisector). Final centers $\mu_1 = (\frac{4}{3}, \frac{4}{3})$, $\mu_2 = (\frac{19}{3}, \frac{19}{3})$. Midpoint: $(\frac{23}{6}, \frac{23}{6})$. Equidistance $\|x - \mu_1\|^2 = \|x - \mu_2\|^2$ gives $2x^T(\mu_2 - \mu_1) = \|\mu_2\|^2 - \|\mu_1\|^2$, i.e. $10(x_1 + x_2) = \frac{230}{3}$:

$$x_1 + x_2 = \frac{23}{3}$$

— the dashed line in the figure: perpendicular to the segment joining the centers (direction $(1, 1)$), through the midpoint (check: $\frac{23}{6} + \frac{23}{6} = \frac{23}{3}$). Points with $x_1 + x_2 < \frac{23}{3}$ join cluster 1 — all of x_1, x_2, x_3 satisfy this ($2, 3, 3 < 7.67$); x_4, x_5, x_6 don't ($12, 13, 13 > 7.67$).

Basically, ... “Once the centers freeze, the plane is divided like a cake cut with straight knives: each pair of centers contributes the perpendicular bisector of the segment between them, and your cluster is the piece containing your center. Straight cuts only — k-means can't do curves.”

26.7 Initialization: uniform vs k-means++

Lloyd's start matters (§26.4: local minima, iteration count). The lecture gives two ways to choose the initial centers:

- i) **Uniformly at random.** Pick the initial centers (or assignments) with equal probability over the data. Simple; sometimes unlucky.
- ii) **k-means++.** Idea, in the lecture's words: "cluster centers should be as far apart as possible." Procedure:
 1. Choose the first center μ_1^0 uniformly at random from $x_1, \dots, x_n$.
 2. For $l = 2, 3, \dots, K$: choose μ_l^0 *probabilistically*, with probability proportional to the score

$$S(x) = \min_{j=1, \dots, l-1} \|x - \mu_j^0\|^2 \quad \forall x$$

— the squared distance to the *nearest* already-chosen center.

Reading the score. $S(x)$ is big exactly when x is far from all existing centers — so far-flung points are *likely* (not certain) to become new centers, and the centers spread out. Two edge facts: a point sitting exactly on an existing center has $S = 0$ and is never re-chosen; and "proportional to" means *probabilities*, not a maximum — the highest-score point is the most likely pick but not a guaranteed one (practice Q5: scores 25, 67, 89, 24, 56 $\rightarrow$ the 89 point has the highest *probability*, yet might not be chosen).

eg 4 (k-means++ scoring, on §26.5's data). Suppose the first pick is $\mu_1^0 = x_4 = (6, 6)$. Then $S(x) = \|x - (6, 6)\|^2$: $S(x_1) = 50$, $S(x_2) = 41$, $S(x_3) = 41$, $S(x_4) = 0$, $S(x_5) = 1$, $S(x_6) = 1$; total 134.

$$P(\mu_2^0 = x_1) = \frac{50}{134} \approx 0.373, \quad P(x_2) = P(x_3) = \frac{41}{134} \approx 0.306, \quad P(x_5) = P(x_6) = \frac{1}{134} \approx 0.0075, \quad P(x_4) = 0.$$

The far blob (x_1, x_2, x_3) soaks up $\approx 98.5\%$ of the probability — the second center will almost surely land far from the first, as intended.

Basically, ... "Random start = throw K darts blindfolded. k-means++ = throw the first dart blindfolded, then throw each next dart *aiming away* from where darts already landed — 'away' measured by squared distance, a dice roll deciding exactly where. Spread-out starts, fewer bad valleys."

26.8 Choosing K : the lecture's penalty idea

K is an input (§26.1(iii)) — but which K ? The lecture reasons from the objective:

- i) $K = n$ **trivializes** F . Every point its own cluster, each point its own centroid: $\|x_i - \mu_{z_i}\| = 0$ for all i , so $F = 0$ (practice Q8). Perfect score, useless clustering.
- ii) **More centers never hurt the optimum.** With $K + 1$ clusters you can always match the best K -cluster solution (leave one cluster empty), so the *optimal* F is non-increasing in K (practice Q7: among $K = 1, 10, 100$, $K = 1$ gives the largest F).
- iii) **So F alone can't pick K .** The lecture's resolution, stated as a slogan rather than a formula:

$$\text{minimize } F \text{ (objective value)} + \text{Penalty}(K)$$

— want F small *and* K small; penalize large K to block the $K = n$ triviality.

Note (the lecture stops here). No concrete penalty and no selection rule appear in the lecture — the elbow method and other criteria belong to later material, not these sources (flagged in the review log). The takeaway is the *principle*: fit quality and model complexity pull in opposite directions, and K sits at the tradeoff.

Basically, ... "Asking ' F alone, which K is best?' always answers ' $K = n$, perfect zero' — which is cheating. So you pay a tax for every extra cluster and minimize score + tax. The lecture states the tax idea but doesn't hand you the tax table."

26.9 The hard-assignment cousin of GMM/EM

§25.12(i)’s promise, kept. The two algorithms side by side — same skeleton, one decisive difference:

	GMM + EM (§25)	k-means (§26)
Assignment	soft: responsibility vector $\gamma(z_{ik}) \in [0, 1], \sum_k \gamma(z_{ik}) = 1$ (§25.8)	hard: single label $z_i \in \{1, \dots, K\}$
“E”-step	Bayes: $\gamma(z_{ik}) \propto \pi_k N(x_i \mu_k, \Sigma_k)$	arg min: nearest centroid, a 0/1 stamp
“M”-step	responsibility-weighted mean/covariance/weights (§25.9)	plain mean of the assigned points (§26.3)
K	fixed input (§25.3(iii))	fixed input (§26.1(iii))
Convergence	to a local optimum; init matters (§25.10)	to a local minimum of F ; init matters (§26.4)

- i) **The bridge is exact in one direction.** Take EM’s M-step (§25.9) and force every responsibility to 0 or 1: $\mu_k = \frac{1}{N_k} \sum_i \gamma(z_{ik}) x_i$ becomes *the mean of the points assigned to k* — exactly k-means’ centroid formula — and N_k becomes the headcount (Problem 8). §25.9(i) already noted this collapse; k-means is what EM’s M-step looks like when the E-step stops hedging.
- ii) **What k-means gives up.** No π_k (cluster sizes aren’t modeled, just counted), no Σ_k (clusters are implicitly spherical — every direction counts equally in $\|x_i - \mu_k\|^2$), no uncertainty for boundary points (a point is *in* or *out*; §25.8’s $x = -2$ would get stamped instead of split 20%/80%).
- iii) **What k-means gains.** Simplicity and speed: no densities to evaluate, no covariances to maintain — just distances and means. The lecture’s closing application of EM was “one of the building blocks of clustering” (§25.12(i)); k-means is the other, harder-edged block.

Note (the $\sigma^2 \rightarrow 0$ limit). The textbook story — k-means as the zero-variance limit of a spherical GMM — is *not* in these lectures, so this chapter doesn’t claim it (flagged in the review log).

Basically, ... “EM says: every point belongs to every cluster a little bit — here are the percentages. k-means says: pick one. Same alternating skeleton (assign-ish, then re-mean), same fixed K , same init-sensitivity, same local-optimum honesty. Freeze EM’s percentages into 0s and 1s and its update formulas turn into k-means’ — that’s the family resemblance, proved in Problem 8.”

26.10 Cautions from the practice bench

Two warnings the Week 3 practice assignment insists on:

- i) **Outliers hurt.** k-means minimizes *squared* distances with *means* — both are outlier-magnets. One far-flung point drags its cluster’s centroid toward itself and can wreck the grouping of the honest points (practice Q10: yes, sensitive).
- ii) **Geometry can fool it.** Two parallel lines of points, $K = 2$ (practice Q4): with a lucky initialization you get the two lines as clusters; with an unlucky one you get a left-half/right-half split — and if the lines are far apart relative to the gaps between points, initialization stops mattering. The clusters you get depend on the start *and* on the data’s geometry, because F has multiple valleys.

Basically, ... “Squared distance + mean = outlier-sensitive by construction. And F ’s landscape can have several valleys — which one you land in depends on where you start and how the data is shaped. k-means is simple, not magic.”

26.11 Where this goes next

- i) **The unsupervised arc, complete.** §22.11 asked for models that compress, explain, and group: PCA compressed (§24), GMMs explained (§25), k-means groups (§26). Three answers, one spirit — no labels anywhere.
- ii) **Part IV continues.** Next in the toolbox: kernel methods, regularization, trees, SVMs, ensembles (§25.12(iv)'s list) — and the coming chapters will reuse this chapter's moves: alternating minimization, distance-based decisions, and the ever-present question of how complex a model to fit.

Problem set

1. **Objective arithmetic.** 1-D data $\{0, 1, 9, 10\}$, $K = 2$. (i) Compute F for $z = (1, 1, 2, 2)$. (ii) Compute F for $z = (1, 2, 2, 2)$. (iii) Which assignment is better, and what does “better” mean here?
2. **One Lloyd step.** 1-D data $\{1, 2, 9, 10\}$, $K = 2$, initial assignment $z^0 = (1, 1, 1, 2)$. (i) Compute μ_1^0, μ_2^0 . (ii) Perform the reassignment step: give z^1 . (iii) Compute F^0 and F^1 ; verify $F^1 < F^0$.
3. **Monotonicity, proved (guided).** (i) For a fixed assignment, show the centroid μ_k minimizes cluster k 's contribution $\sum_{i: z_i=k} \|x_i - \mu\|^2$ (differentiate w.r.t. μ). (ii) Conclude the compute-means step cannot increase F . (iii) Conclude the reassignment step cannot increase F (each point minimizes its own term). (iv) Conclude $F^{t+1} \leq F^t$, with strict inequality iff at least one point changes cluster.
4. **Bisector geometry.** Centers $\mu_1 = (0, 0)$, $\mu_2 = (4, 0)$. (i) Find the equation of the boundary between the clusters. (ii) Which cluster does $(1, 1)$ join? (iii) Prove the general claim: the boundary is perpendicular to the segment $\mu_1\mu_2$ and passes through its midpoint.
5. **k-means++ scoring.** 1-D data $\{0, 5, 6\}$; first center $\mu_1^0 = 0$. (i) Compute $S(x)$ for each point. (ii) Give $P(\mu_2^0 = x)$ for each point. (iii) Which point is most likely chosen as μ_2^0 , and can you *guarantee* it will be?
6. **Choosing K .** (i) Show that $K = n$ achieves $F = 0$ for any dataset. (ii) For 100 points, which of $K = 1, 10, 100$ gives the largest optimal F , and why? (iii) In one line: why does the lecture say to penalize large K ?
7. **True or false, one-line justification:** (i) Two runs of Lloyd's algorithm from different initializations always return the same clusters. (ii) F can never increase from one iteration to the next. (iii) Lloyd's algorithm can visit the same partition twice. (iv) k-means is insensitive to outliers. (v) Every k-means cluster region is convex.
8. **Hard vs soft (the §25.12(i) bridge).** (i) In EM's M-step (§25.9), suppose every responsibility $\gamma(z_{ik}) \in \{0, 1\}$. Show the μ_k update becomes exactly k-means' centroid formula, and interpret N_k and $\pi_k = N_k/n$. (ii) In one line: what does k-means' reassignment step do that EM's E-step does not?

Solutions — Chapter 26: K-means clustering

Problem 1 — Objective arithmetic

Data $\{0, 1, 9, 10\}$, $K = 2$.

(i) $z = (1, 1, 2, 2)$. Cluster 1: $\{0, 1\}$, $\mu_1 = 0.5$; contributions $(0 - 0.5)^2 + (1 - 0.5)^2 = 0.25 + 0.25 = 0.5$. Cluster 2: $\{9, 10\}$, $\mu_2 = 9.5$; contributions $0.25 + 0.25 = 0.5$.

$$\boxed{F = 1}.$$

(ii) $z = (1, 2, 2, 2)$. Cluster 1: $\{0\}$, $\mu_1 = 0$; contribution 0. Cluster 2: $\{1, 9, 10\}$, $\mu_2 = \frac{20}{3}$; contributions

$$\left(1 - \frac{20}{3}\right)^2 + \left(9 - \frac{20}{3}\right)^2 + \left(10 - \frac{20}{3}\right)^2 = \frac{289}{9} + \frac{49}{9} + \frac{100}{9} = \frac{438}{9} = \frac{146}{3}.$$

$$\boxed{F = \frac{146}{3} \approx 48.67}.$$

(iii) $z = (1, 1, 2, 2)$ is better. “Better” here means *smaller* F : the points sit tighter around their centroids (two natural pairs), whereas $z = (1, 2, 2, 2)$ lumps $\{1, 9, 10\}$ into one stretched cluster. F is the score; smaller wins.

Problem 2 — One Lloyd step

Data $\{1, 2, 9, 10\}$, $K = 2$, $z^0 = (1, 1, 1, 2)$.

(i) $\mu_1^0 = \frac{1+2+9}{3} = \boxed{4}$, $\mu_2^0 = \boxed{10}$.

(ii) Squared distances to $(4, 10)$: - $x = 1$: 9 vs $81 \rightarrow 1$; - $x = 2$: 4 vs $64 \rightarrow 1$; - $x = 9$: 25 vs $1 \rightarrow 2$; - $x = 10$: 36 vs $0 \rightarrow 2$.

$$\boxed{z^1 = (1, 1, 2, 2)}.$$

(iii) F^0 : cluster 1 = $\{1, 2, 9\}$ about $\mu_1^0 = 4$: $9+4+25 = 38$; cluster 2 = $\{10\}$: 0 . $\boxed{F^0 = 38}$. Recompute means for z^1 : $\mu_1^1 = 1.5$, $\mu_2^1 = 9.5$. F^1 : cluster 1: $0.25 + 0.25 = 0.5$; cluster 2: $0.25 + 0.25 = 0.5$. $\boxed{F^1 = 1 < 38 = F^0}$ — the unhappy point $x = 9$ defected and F collapsed.

Problem 3 — Monotonicity, proved

(i) Fix the assignment; for cluster k minimize $g(\mu) = \sum_{i:z_i=k} \|x_i - \mu\|^2$. $\nabla_\mu g = -2 \sum_{i:z_i=k} (x_i - \mu) = 0$ gives $\mu \sum_{i:z_i=k} 1 = \sum_{i:z_i=k} x_i$, i.e.

$$\mu_k = \frac{\sum_{i:z_i=k} x_i}{\#\{i : z_i = k\}},$$

the centroid. (Hessian $2m_k I \succ 0$: it is the minimizer.)

(ii) The compute-means step replaces each μ_k by the minimizer of that cluster’s contribution to F while the assignment is frozen — each cluster’s sum can only drop, so F cannot increase.

(iii) The reassignment step freezes the centers; each point’s new label is $\arg \min_k \|x_i - \mu_k^t\|^2$, the minimizer of that point’s term in F . Every term can only drop, so F cannot increase.

(iv) Both steps are non-increasing, so $\boxed{F^{t+1} \leq F^t}$. Strictness: the mean step is strictly decreasing unless the centers were already the centroids; the reassignment step is strictly decreasing iff at least one point moves (a point moves only to a *strictly* closer center). So $F^{t+1} < F^t$ iff at least one point changes cluster in iteration t .

Problem 4 — Bisector geometry

(i) $\|x - \mu_1\|^2 = \|x - \mu_2\|^2$: $x_1^2 + x_2^2 = (x_1 - 4)^2 + x_2^2$, i.e. $0 = -8x_1 + 16$:

$$\boxed{x_1 = 2}.$$

(ii) $(1, 1)$: to μ_1 : $1 + 1 = 2$; to μ_2 : $9 + 1 = 10$. Closer to $\mu_1 \rightarrow \boxed{\text{cluster 1}}$ (also: $1 < 2$, left of the boundary).

(iii) $\|x - \mu_1\|^2 = \|x - \mu_2\|^2 \iff -2x^T \mu_1 + \|\mu_1\|^2 = -2x^T \mu_2 + \|\mu_2\|^2 \iff 2x^T(\mu_2 - \mu_1) = \|\mu_2\|^2 - \|\mu_1\|^2$. This is a hyperplane with normal vector $\mu_2 - \mu_1$ — hence perpendicular to the segment $\mu_1 \mu_2$. The midpoint $m = \frac{\mu_1 + \mu_2}{2}$ satisfies it: $2m^T(\mu_2 - \mu_1) = (\mu_1 + \mu_2)^T(\mu_2 - \mu_1) = \|\mu_2\|^2 - \|\mu_1\|^2$. So the boundary is the perpendicular bisector.

Problem 5 — k-means++ scoring

(i) $S(x) = \|x - 0\|^2 = x^2$: $\boxed{S(0) = 0, S(5) = 25, S(6) = 36}$.

(ii) Total = 61:

$$P(\mu_2^0 = 0) = 0, \quad P(\mu_2^0 = 5) = \frac{25}{61} \approx 0.410, \quad P(\mu_2^0 = 6) = \frac{36}{61} \approx 0.590.$$

(iii) $x = 6$ is most likely (highest score $\rightarrow$ highest probability). No — the choice is *probabilistic*, not a max: 6 is picked with probability $\frac{36}{61}$, not 1.

Problem 6 — Choosing K

(i) With $K = n$, put each x_i in its own cluster; then $\mu_{z_i} = x_i$ and every term $\|x_i - \mu_{z_i}\|^2 = 0$. $\boxed{F = 0}$ for any dataset.

(ii) $\boxed{K = 1}$ gives the largest optimal F : one centroid must serve all 100 points, so every point's distance is measured from the single global mean; adding centers can only let points sit closer to their own center (practice Q7's argument).

(iii) Because F alone is minimized by the trivial $K = n$ ($F = 0$, useless) — without a penalty on K there is nothing stopping the degenerate solution.

Problem 7 — True or false

- (i) **False.** Different initializations can converge to different local minima of F (practice Q9(b)).
- (ii) **True.** $F^{t+1} \leq F^t$ every iteration — neither Lloyd step can increase F (Problem 3; practice Q2/Q6).
- (iii) **False.** Every reassignment strictly reduces F , so a visited partition — with its F value — can never recur (practice Q6(a)).
- (iv) **False.** k-means minimizes squared distances about means; both are outlier-sensitive (practice Q10).
- (v) **True.** Each cluster region is an intersection of $K - 1$ half-spaces (§26.6); half-spaces are convex (§12.3) and intersections preserve convexity (§12.4).

Problem 8 — Hard vs soft

(i) EM's M-step (§25.9): $\mu_k = \frac{1}{N_k} \sum_{i=1}^n \gamma(z_{ik}) x_i$ with $N_k = \sum_i \gamma(z_{ik})$. If $\gamma(z_{ik}) \in \{0, 1\}$, the sum keeps exactly the points with $\gamma(z_{ik}) = 1$ — i.e. the points *assigned* to k — and N_k is their headcount:

$$\mu_k = \frac{\sum_{i: z_i = k} x_i}{\#\{i : z_i = k\}},$$

k-means' centroid formula (§26.3). $\pi_k = N_k/n$ becomes the *fraction of data points in cluster k* — the empirical cluster proportion.

(ii) k-means' reassignment stamps each point with a single hard label (nearest centroid); EM's E-step instead distributes each point *softly* across components via Bayes' rule.

27. Kernel methods and kernel regression

Chapter 23’s closing message was that polynomial regression needs no new machinery — transform the features, run the same least-squares machine. §24.10 built the other half of the story: when the feature map ϕ is expensive or even infinite-dimensional, the kernel trick lets you work with inner products $\kappa(x, x')$ alone. This chapter joins the two: *ridge regression in feature space, solved without ever visiting feature space*. Everything here comes from the MLT Week 2 slides (the kernel machinery, recapped in §27.2) and the MLT Week 5 lectures and programming assignment on kernel regression (the model, the coefficient vector α , the weighted-sum predictions).

27.1 Notation

Notation. Data $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$ with $x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$ (subscript convention, Chapters 24–26). The kernel, Gram matrix, and feature map follow §24.10’s definitions exactly:

$$\boxed{\kappa(x, x') = \phi(x)^T \phi(x')}, \quad \boxed{K_{ij} = \kappa(x_i, x_j)} \in \mathbb{R}^{n \times n}, \quad y = (y_1, \dots, y_n)^T.$$

Φ is the $n \times D$ **feature matrix** whose i -th row is $\phi(x_i)^T$ (the §23.2 convention: rows are points), so $\Phi\Phi^T = K$ and $(\Phi w)_i = w^T \phi(x_i)$. The Week 5 assignment’s $w = \phi(X)\alpha$ is the same relation transposed.

27.2 The kernel machinery, briefly recapped

§24.10 derived the full machinery; here is only what kernel regression needs:

- i) **Lifting.** Map $x \in \mathbb{R}^d$ through $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$ into a space where the structure is linear, then run linear methods there.
- ii) **The dual form.** In kernel PCA, every eigenvector lay in the span of the data: $w_k = X\alpha_k$. Regression will have the same shape: the solution w lies in the span of the *transformed* training points (§27.5).
- iii) **The kernel trick.** Only inner products ever appear, so replace $\phi(x_i)^T \phi(x_j)$ by $\kappa(x_i, x_j)$, computed directly: $\boxed{\text{polynomial } \kappa(x, x') = (x^T x' + 1)^p}$, $\boxed{\text{RBF } \kappa(x, x') = \exp(-\|x - x'\|^2 / (2\sigma^2))}$. A kernel is valid iff it is symmetric and every Gram matrix is positive semi-definite (Mercer, §24.10).
- iv) **The point of the trick.** For the quadratic kernel on $\mathbb{R}^2$, $\kappa(x, x') = (x^T x' + 1)^2$ buys the 6-dimensional ϕ at $O(d)$ cost; the RBF kernel’s ϕ is infinite-dimensional — no explicit ϕ exists at all, yet κ computes the inner product in $O(d)$.

Basically, ... “Anything that only needs dot products of lifted features can run on kernel values instead.” Kernel regression is exactly such an algorithm.

27.3 Motivation: linear models are limited

§23's machine fits $f_\theta(x) = \theta^T x$ — a line, or a hyperplane. Polynomial regression (§23.8) escapes lines by hand-building $\phi(x) = (1, x, x^2, \dots, x^m)^T$ and reusing the same machine. But:

- i) **Explicit ϕ scales badly.** Cubic features of d variables number $\approx O(d^3)$; each one costs a column in the feature matrix and a parameter in θ .
- ii) **Explicit ϕ can be impossible.** The RBF kernel's ϕ is infinite-dimensional (§24.10) — there is no finite feature vector to build. Yet it is a perfectly computable function: $\exp(-\|x - x'\|^2/(2\sigma^2))$ is just arithmetic on two d -vectors.
- iii) **The dilemma is §24.10's, again.** There the answer was: find the dual form, then kernelize it. The Week 5 lectures motivate regression the same way — “impose *structure* to reduce search space” — and the structure here is *ridge-regularized linear regression in feature space, with the kernel trick doing the heavy lifting*.

Basically, ... “You want curves, but you refuse to build the curve-features. Do linear regression in the imaginary lifted space anyway — arrange the algebra so the imaginary space only ever appears as $\kappa(x, x')$, and it was never imaginary to begin with.”

27.4 Ridge regression in feature space: the objective

Start with §23.10's ridge objective, but on the lifted features. **Def (kernel ridge objective).** For $w \in \mathbb{R}^D$:

$$J(w) = \frac{1}{2} \sum_{i=1}^n (w^T \phi(x_i) - y_i)^2 + \frac{\lambda}{2} \|w\|^2, \quad \lambda > 0,$$

i.e. $J(w) = \frac{1}{2} \|\Phi w - y\|^2 + \frac{\lambda}{2} \|w\|^2$. It is §23.10's objective with $\phi(x_i)$ in place of x^i (the $\frac{\lambda}{2}$ vs §23.10's λ is the same bookkeeping choice as §23.1's note — the constant in front never changes the minimizer).

Why ridge, not plain least squares? Two reasons, both sourced: §23.10 shows the λI term guarantees a unique solution even when the feature matrix is rank-deficient (with an infinite-dimensional ϕ , rank considerations are hopeless), and the lecture promises λ “controls overfitting — too small λ could lead to a lot of overfitting, too large λ leads to underfitting.” The Week 5 lecture's warning — “by *memorizing*, we can get zero error on training data; what we care about is test performance” — is exactly what the penalty term guards against.

Basically, ... “Fit the lifted points with least squares, but charge λ for big weights. The charge does two jobs: it keeps the equations solvable and it stops the model from memorizing.”

27.5 The dual form: the solution lives in the span of the data

The gradient of J (§10.1's recipe, as in §23.3):

$$\nabla_w J = \sum_{i=1}^n (w^T \phi(x_i) - y_i) \phi(x_i) + \lambda w = \Phi^T (\Phi w - y) + \lambda w.$$

Setting $\nabla_w J = 0$ and solving for w :

$$\lambda w = \Phi^T (y - \Phi w) \quad \Rightarrow \quad \boxed{w = \Phi^T \alpha}, \quad \alpha := \frac{y - \Phi w}{\lambda} \in \mathbb{R}^n.$$

The optimal weight vector is a linear combination of the transformed training points:

$$\boxed{w = \sum_{i=1}^n \alpha_i \phi(x_i)}.$$

This is the regression twin of §24.10's "every eigenvector is a linear combination of the data points" ($w_k = X\alpha_k$): the D -dimensional unknown w is parametrized by the n numbers $\alpha_1, \dots, \alpha_n$. And since J is a convex quadratic (Hessian $\Phi^T\Phi + \lambda I$ is positive definite for $\lambda > 0$), the stationary point is the global minimum — §23.3's argument, unchanged.

Basically, ... "Don't search all of $\mathbb{R}^D$ for w . The gradient equations force the answer to be a mix of the training points' lifted features — so search the n mixing weights α instead."

27.6 The closed-form solution

Substitute $w = \Phi^T\alpha$ into the definition $\alpha = (y - \Phi w)/\lambda$:

$$\lambda\alpha = y - \Phi\Phi^T\alpha = y - K\alpha,$$

since $\Phi\Phi^T$ is exactly the Gram matrix $K_{ij} = \phi(x_i)^T\phi(x_j) = \kappa(x_i, x_j)$. Rearranging:

$$\boxed{(K + \lambda I)\alpha = y}, \quad \boxed{\hat{\alpha} = (K + \lambda I)^{-1}y}.$$

Note (always solvable). K is positive semi-definite (Mercer, §24.10), so for $\lambda > 0$ and $z \neq 0$:

$$z^T(K + \lambda I)z = \underbrace{z^TKz}_{\geq 0} + \underbrace{\lambda\|z\|^2}_{> 0} > 0,$$

hence $K + \lambda I$ is positive definite and invertible — the same one-line proof as §23.10's homework for $A^TA + \lambda I$.

The model at a new point x needs only κ :

$$f(x) = w^T\phi(x) = \alpha^T\Phi\phi(x) = \sum_{i=1}^n \alpha_i \phi(x_i)^T\phi(x) = \boxed{f(x) = \sum_{i=1}^n \alpha_i \kappa(x, x_i)}.$$

The Week 5 assignment's prediction loop is exactly this formula ($\hat{y}_i = \sum_j \alpha_j \ker(x_i, x_j)$), and its $\alpha = K^{-1}y$ is the $\lambda \rightarrow 0$ limit of $\hat{\alpha} = (K + \lambda I)^{-1}y$.

The kernel regression algorithm. Input $D = \{(x_i, y_i)\}$, kernel κ , $\lambda > 0$: - Step 1: build $K_{ij} = \kappa(x_i, x_j)$ (n^2 kernel evaluations). - Step 2: solve $(K + \lambda I)\alpha = y$ for $\alpha \in \mathbb{R}^n$. - Step 3: predict $f(x) = \sum_i \alpha_i \kappa(x, x_i)$.

Basically, ... "Two steps, no ϕ anywhere: solve one $n \times n$ linear system for α , then predict by adding up kernel values weighted by α . The lifted space was scaffolding — the building stands on κ alone."

27.7 What it means: prediction as a weighted sum of similarities

Read $f(x) = \sum_i \alpha_i \kappa(x, x_i)$ as a **similarity-weighted vote**:

- i) **The kernel is a similarity score.** $\kappa(x, x_i)$ is large when x resembles the training point x_i : for the RBF kernel it decays with distance $\|x - x_i\|$; for the polynomial kernel it grows with alignment $x^T x_i$. A prediction is a weighted sum of "how similar is x to each training point," with weights α_i .
- ii) **The weights can be negative.** α_i is not an importance score — it is a coefficient. In §27.8's example, $\alpha_2 = -1/4$ pulls the fit down near $x_2 = 0$. Positive α_i copies y_i 's influence up; negative α_i pushes the opposite way.
- iii) **The kernel shapes the function.** Polynomial kernel of degree p : $\kappa(x, x_i) = (x^T x_i + 1)^p$ is a degree- p polynomial in x , so $f(x)$ is a degree- p polynomial — it searches the same curves as §23.8's polynomial regression, minus the explicit features (Problem 5). RBF kernel: each term $\alpha_i \exp(-\|x - x_i\|^2/(2\sigma^2))$ is a bump centered at x_i with width σ — the prediction is a sum of bumps, smooth and local: points far from all training data get near-zero contributions, and σ sets how far each bump reaches.
- iv) **The λ knob, in similarity language.** $\lambda \rightarrow 0$ forces $f(x_i) \approx y_i$ at every training point — the bumps interpolate, memorizing (the Week 5 lecture's zero-training-error memorizer). Larger λ shrinks every α_i toward 0, flattening the fit — §23.10's "too large λ leads to underfitting."

Basically, ... “To predict at x : ask every training point ‘how similar am I to you?’ ($\kappa(x, x_i)$), multiply by its coefficient α_i , add up. Near neighbours dominate; strangers don’t vote.”

eg (RBF bumps). Training points $x_1 = -1, x_2 = 0, x_3 = 1$, RBF kernel with $\sigma = 0.5$: the three similarity functions $\kappa(x, x_i)$ are bumps of height 1 at each training point, decaying to $\exp(-(1)^2/(2 \cdot 0.25)) = e^{-2} \approx 0.135$ one unit away. The prediction $f(x) = \sum_i \alpha_i \kappa(x, x_i)$ stacks these bumps — wherever the training points are dense, the bumps add up; far away, all bumps die and the prediction fades to 0.

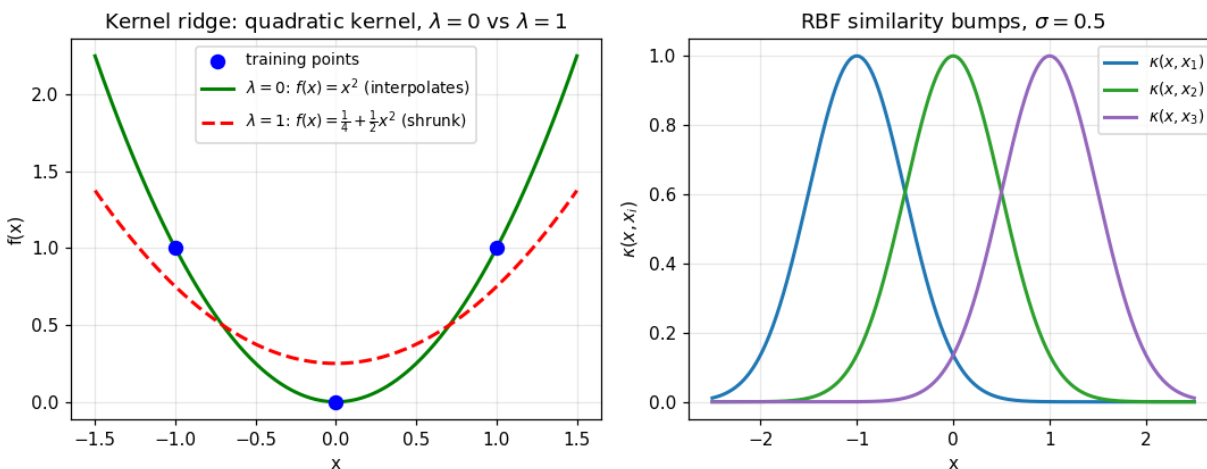


Figure 37: Left panel: the three training points $(-1, 1), (0, 0), (1, 1)$ with two fits — the $\lambda=0$ curve $f(x)=x^2$ threading all three points exactly, and the $\lambda=1$ curve $f(x)=1/4+x^2/2$ sitting slightly below at the outer points and slightly above at the centre. Right panel: three RBF bumps $\kappa(x, x_i)$ for $x_i = -1, 0, 1$ with $\sigma = 0.5$, each peaking at 1 at its centre and decaying to near zero one unit away.

27.8 Worked example: kernel ridge by hand

Data: $x_1 = -1, x_2 = 0, x_3 = 1$; $y = (1, 0, 1)^T$. Quadratic kernel $\kappa(x, z) = (xz + 1)^2$ (the same kernel as §24.10’s parabola example).

Step 1 — Gram matrix. $K_{ij} = (x_i x_j + 1)^2$:

$$K = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 4 \end{pmatrix} \quad (\text{eg } K_{11} = (1+1)^2 = 4, \quad K_{13} = (-1+1)^2 = 0).$$

Step 2a — no regularization ($\lambda = 0$, the assignment’s setting). Solve $K\alpha = y$:

$$\begin{cases} 4\alpha_1 + \alpha_2 = 1 \\ \alpha_1 + \alpha_2 + \alpha_3 = 0 \\ \alpha_2 + 4\alpha_3 = 1 \end{cases}.$$

The data is symmetric in $x_1 \leftrightarrow x_3$, so try $\alpha_1 = \alpha_3 = a, \alpha_2 = b$: $4a + b = 1$ and $2a + b = 0$. Subtracting: $2a = 1$, so $\alpha = (1/2, -1, 1/2)^T$.

$$f(x) = \frac{1}{2}(1-x)^2 - 1 + \frac{1}{2}(1+x)^2 = x^2.$$

Check: $f(-1) = 1, f(0) = 0, f(1) = 1$ — with $\lambda = 0$ the model *interpolates*: it threads all three points, training error 0. (It found $y = x^2$, the exact parabola the points sit on — kernel regression with the quadratic kernel recovering what §23.8’s polynomial regression would fit.)

Step 2b — regularized ($\lambda = 1$). Solve $(K + I)\alpha = y$:

$$\begin{pmatrix} 5 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 5 \end{pmatrix} \alpha = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}.$$

Again $\alpha_1 = \alpha_3 = a$: $5a + b = 1$, $2a + 2b = 0 \Rightarrow b = -a$, so $4a = 1$: $\alpha = (1/4, -1/4, 1/4)^T$.

$$f(x) = \frac{1}{4}(1-x)^2 - \frac{1}{4} + \frac{1}{4}(1+x)^2 = \frac{1}{4} + \frac{1}{2}x^2.$$

Check on training points: $f(-1) = 3/4$, $f(0) = 1/4$, $f(1) = 3/4$. Residuals $(-1/4, 1/4, -1/4)$ — the fit no longer interpolates; every coefficient shrank ($|1/4| < |1/2|$, $|-1/4| < |-1|$), pulling the curve toward 0. The price of $\lambda = 1$ is training error $\frac{1}{4^2} \cdot 3/3 = 1/16$ per point; the purchase is a flatter, less committal model — §23.10's trade in miniature.

Basically, ... “Same data, same kernel, two λ 's: $\lambda = 0$ memorizes the parabola through every point; $\lambda = 1$ shrinks the weights and accepts small misses. That dial is the whole regularization story.”

27.9 Practical notes: the kernel, λ , and the $O(n^3)$ bill

Only what the sources support:

- i) **Choosing the kernel = choosing the function's shape.** The polynomial kernel of degree p commits to degree- p polynomials — §23.8's world, without the explicit features. The RBF kernel commits to smooth, local fits (sums of bumps, §27.7(iii)) and is the only option whose ϕ *cannot* be built explicitly (§24.10). The course gives no kernel-selection procedure beyond this qualitative guidance — the disciplined way to compare choices is §23.9's: fit candidates on training data, compare on held-out validation data.
- ii) **Choosing λ .** §23.10's remark is the complete guidance: too small λ — near-interpolation, overfitting risk (the Week 5 lecture's memorizer); too large λ — weights squeezed toward 0, underfitting. The assignment's $\alpha = K^{-1}y$ is the $\lambda \rightarrow 0$ endpoint — used with `pinv` there precisely because K can be singular without the λI term.
- iii) **Computational cost.** Step 1 costs n^2 kernel evaluations; Step 2 solves an $n \times n$ dense system — $O(n^3)$, the same bill as §24.9's $n \times n$ eigendecomposition. The kernel trick removed the D -dependence (even infinite D is fine) but left an n -dependence: for very large n , the $O(n^3)$ solve is the bottleneck. The lectures give no alternative here — no low-rank or iterative shortcut appears in these sources.
- iv) **What centering?** None. §24.10 centered the Gram matrix because PCA needs zero-mean data; kernel regression has no such requirement — the algorithm of §27.6 uses K as-is.

27.10 Where this goes next

- i) **Regularization, properly.** §27.4 borrowed §23.10's remark; Chapter 28 makes ridge and lasso a full subject — the bias-variance trade-off, and how λ is actually chosen.
- ii) **The kernel trick, again.** The pattern “ $\$w = \$(\text{span of data}) + \text{Gram matrix } K$ ” appeared twice now (§24.10's $w_k = X\alpha_k$, §27.5's $w = \Phi^T\alpha$). Chapters 32–33 (hard/soft-margin SVM) run it a third time, for classification.
- iii) **From bumps to neurons.** The RBF picture — predictions as weighted sums of local bumps — is worth keeping in mind for Part VI: a neural network's hidden layer will build its predictions out of learned, adaptive “bumps” instead of fixed kernel ones.

Problem set

1. Cubic-kernel ϕ , by hand (the assignment's kernel). For scalar x, z , expand $\kappa(x, z) = (xz + 1)^3$ fully and exhibit an explicit $\phi : \mathbb{R} \rightarrow \mathbb{R}^4$ with $(xz + 1)^3 = \phi(x)^T \phi(z)$.

2. Guided derivation. Starting from $J(w) = \frac{1}{2}\|\Phi w - y\|^2 + \frac{\lambda}{2}\|w\|^2$ with Φ the $n \times D$ feature matrix: (i) compute $\nabla_w J$; (ii) show the stationary point satisfies $w = \Phi^T \alpha$ for some $\alpha \in \mathbb{R}^n$; (iii) substitute to get $(K + \lambda I)\alpha = y$; (iv) argue $(K + \lambda I)$ is invertible for $\lambda > 0$.
3. Kernel ridge numeric. Data $x_1 = 0, x_2 = 2; y = (2, 4)^T$; kernel $\kappa(x, z) = xz + 1$ (linear kernel with intercept); $\lambda = 1$. (i) Write K . (ii) Solve $(K + \lambda I)\alpha = y$. (iii) Write $f(x)$ and simplify to a line; check $f(0), f(2)$.
4. Interpolation at $\lambda = 0$. Assume K is invertible. (i) Show the $\lambda \rightarrow 0$ solution $\hat{\alpha} = K^{-1}y$ gives training predictions $\hat{y}_i = y_i$ exactly (training error 0). (ii) Connect this to the Week 5 lecture's "by *memorizing*, we can get zero error on training data."
5. Polynomial kernel = polynomial regression. For scalar x and kernel $\kappa(x, z) = (xz + 1)^p$: (i) show each $\kappa(x, x_i)$ is a polynomial in x of degree $\leq p$; (ii) conclude $f(x) = \sum_i \alpha_i \kappa(x, x_i)$ is a polynomial of degree $\leq p$; (iii) what does this say about kernel regression with this kernel vs §23.8's polynomial regression?
6. RBF σ , qualitatively. With the RBF kernel, fix the data and α . (i) What happens to each term $\alpha_i \kappa(x, x_i)$ as $\sigma \rightarrow 0$? (ii) As $\sigma \rightarrow \infty$? (iii) Which regime risks the memorizer of §27.4, and why?
7. True or false, one-line justification: (i) Kernel regression ever needs the explicit $\phi(x)$. (ii) $(K + \lambda I)$ is invertible for every $\lambda > 0$. (iii) A negative α_i means training point i was an outlier. (iv) The RBF kernel's ϕ can be written down as a finite vector. (v) Kernel regression with the linear kernel $\kappa(x, z) = x^T z$ (no +1) can learn a nonzero intercept.
8. Cost accounting. For the §27.6 algorithm with n training points in $\mathbb{R}^d$: (i) how many kernel evaluations does Step 1 need, and what is each evaluation's cost for the RBF kernel? (ii) What is Step 2's asymptotic cost? (iii) At prediction time, what does one $f(x)$ cost?

Solutions — Chapter 27: Kernel methods and kernel regression

Problem 1 — Cubic-kernel ϕ , by hand

Expand $(xz + 1)^3$ for scalars x, z :

$$(xz + 1)^3 = (xz)^3 + 3(xz)^2 + 3(xz) + 1 = x^3 z^3 + 3x^2 z^2 + 3xz + 1.$$

Take

$$\phi(x) = \begin{pmatrix} x^3 \\ \sqrt{3}x^2 \\ \sqrt{3}x \\ 1 \end{pmatrix}.$$

Then

$$\phi(x)^T \phi(z) = x^3 z^3 + (\sqrt{3}x^2)(\sqrt{3}z^2) + (\sqrt{3}x)(\sqrt{3}z) + 1 = x^3 z^3 + 3x^2 z^2 + 3xz + 1 = (xz + 1)^3 \quad \checkmark$$

This is the explicit 4-dimensional feature map behind the Week 5 assignment's `ker = ((xi.T@xj)+1)**3`. Note the $\sqrt{3}$ weights — the middle features must be *scaled* for the dot product to reproduce the kernel exactly (same point as §24.10's $\sqrt{2}$ factors).

Problem 2 — Guided derivation

(i) $J(w) = \frac{1}{2}(\Phi w - y)^T(\Phi w - y) + \frac{\lambda}{2}w^T w$. Differentiating (the §23.3 calculation with Φ in place of A):

$$\nabla_w J = \Phi^T(\Phi w - y) + \lambda w.$$

(Entrywise: $\frac{\partial}{\partial w_k} \frac{1}{2} \sum_i (\sum_j \Phi_{ij} w_j - y_i)^2 = \sum_i \Phi_{ik} (\Phi w - y)_i = [\Phi^T(\Phi w - y)]_k$; the penalty contributes λw_k .)

(ii) Setting $\nabla_w J = 0$: $\Phi^T(\Phi w - y) + \lambda w = 0$, i.e.

$$\lambda w = \Phi^T(y - \Phi w) \quad \Rightarrow \quad \boxed{w = \Phi^T \alpha}, \quad \alpha := \frac{y - \Phi w}{\lambda} \in \mathbb{R}^n.$$

So the stationary point is a linear combination of the rows of Φ — the transformed training points.

(iii) Substitute $w = \Phi^T \alpha$ into the definition of α :

$$\lambda \alpha = y - \Phi(\Phi^T \alpha) = y - (\Phi \Phi^T) \alpha = y - K \alpha,$$

with $K_{ij} = \phi(x_i)^T \phi(x_j) = \kappa(x_i, x_j)$. Hence $\boxed{(K + \lambda I) \alpha = y}$.

(iv) K is positive semi-definite (Mercer, §24.10): $z^T K z \geq 0$ for all z . For $\lambda > 0$ and $z \neq 0$:

$$z^T (K + \lambda I) z = z^T K z + \lambda z^T z \geq 0 + \lambda \|z\|^2 > 0,$$

so $K + \lambda I$ is positive definite, hence invertible. $\boxed{\hat{\alpha} = (K + \lambda I)^{-1} y}$ — the same one-line proof as §23.10's homework for $A^T A + \lambda I$.

Problem 3 — Kernel ridge numeric

(i) $\kappa(x, z) = xz + 1$:

$$K = \begin{pmatrix} 0 \cdot 0 + 1 & 0 \cdot 2 + 1 \\ 2 \cdot 0 + 1 & 2 \cdot 2 + 1 \end{pmatrix} = \boxed{\begin{pmatrix} 1 & 1 \\ 1 & 5 \end{pmatrix}}.$$

(ii) With $\lambda = 1$:

$$K + I = \begin{pmatrix} 2 & 1 \\ 1 & 6 \end{pmatrix}, \quad \det = 12 - 1 = 11, \quad (K + I)^{-1} = \frac{1}{11} \begin{pmatrix} 6 & -1 \\ -1 & 2 \end{pmatrix}.$$

$$\hat{\alpha} = \frac{1}{11} \begin{pmatrix} 6 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 4 \end{pmatrix} = \frac{1}{11} \begin{pmatrix} 12 - 4 \\ -2 + 8 \end{pmatrix} = \boxed{\begin{pmatrix} 8/11 \\ 6/11 \end{pmatrix}}.$$

(iii) $f(x) = \alpha_1 \kappa(x, 0) + \alpha_2 \kappa(x, 2) = \frac{8}{11}(x \cdot 0 + 1) + \frac{6}{11}(2x + 1)$:

$$\boxed{f(x) = \frac{12}{11}x + \frac{14}{11}}.$$

Check: $f(0) = 14/11 \approx 1.27$ (vs $y_1 = 2$), $f(2) = (24 + 14)/11 = 38/11 \approx 3.45$ (vs $y_2 = 4$). Both miss — regularization shrinks the fit, exactly as in §27.8. **Note.** The +1 in the kernel is what permits a nonzero intercept: without it, $\kappa(x, z) = xz$ gives $f(x) = x(\sum_i \alpha_i x_i)$, always 0 at $x = 0$.

Problem 4 — Interpolation at $\lambda = 0$

(i) Training predictions are $\hat{y} = K \hat{\alpha}$ (row i : $\hat{y}_i = \sum_j K_{ij} \hat{\alpha}_j = \sum_j \hat{\alpha}_j \kappa(x_i, x_j) = f(x_i)$). With $\hat{\alpha} = K^{-1} y$:

$$\hat{y} = K(K^{-1} y) = y,$$

so $\boxed{\hat{y}_i = y_i}$ for all i — training error exactly 0.

(ii) The Week 5 lecture warned: “by *memorizing*, we can get zero error on training data.” $\lambda = 0$ is that memorizer in kernel form — the model spends all its freedom hitting the training labels and says nothing about test points. The assignment’s `pinv(K)@y` is precisely this endpoint (with `pinv` standing in for K^{-1} when K is singular).

Problem 5 — Polynomial kernel = polynomial regression

- (i) $\kappa(x, x_i) = (xx_i + 1)^p$ expands by the binomial theorem to $\sum_{k=0}^p \binom{p}{k} (xx_i)^k$ — a polynomial in x of degree $\leq p$ (coefficients depend on x_i).
- (ii) $f(x) = \sum_i \alpha_i \kappa(x, x_i)$ is a sum of degree- $\leq p$ polynomials, hence a polynomial in x of degree $\leq p$.
- (iii) Kernel regression with the degree- p polynomial kernel searches (a subset of) the same degree- p polynomials as §23.8's polynomial regression — but without ever forming the $(1, x, \dots, x^p)$ features. For large p or multivariate x (where explicit features number $\approx O(d^p)$), the kernel route is the only feasible one. The price: §23.8's explicit θ is interpretable coefficient-by-coefficient, while kernel regression's α is not.

Problem 6 — RBF σ , qualitatively

- (i) $\sigma \rightarrow 0$: $\kappa(x, x_i) = \exp(-\|x - x_i\|^2 / (2\sigma^2)) \rightarrow 1$ if $x = x_i$ and $\rightarrow 0$ otherwise — each bump collapses to a spike at its training point. The prediction becomes a sum of isolated spikes.
- (ii) $\sigma \rightarrow \infty$: $\kappa(x, x_i) \rightarrow 1$ for every x — each bump flattens to a constant, and $f(x) \rightarrow \sum_i \alpha_i$, a constant function.
- (iii) The $\sigma \rightarrow 0$ regime risks the §27.4 memorizer: each training point gets its own private spike, so the model can hit every training label exactly ($f(x_i) \approx \alpha_i$) while predicting near-zero everywhere else — zero training error, useless generalization. Narrow bumps = high capacity, same disease as §23.9's degree-4 polynomial.

Problem 7 — True or false

- (i) **False.** The algorithm (§27.6) evaluates κ only — Step 1 builds $K_{ij} = \kappa(x_i, x_j)$, Step 3 sums $\alpha_i \kappa(x, x_i)$. No $\phi(x)$ is ever formed; that is the kernel trick's whole point.
- (ii) **True.** K is PSD and $\lambda > 0$; $z^T(K + \lambda I)z = z^T K z + \lambda \|z\|^2 > 0$ for $z \neq 0$, so $K + \lambda I$ is positive definite invertible (the §27.6 Note).
- (iii) **False.** Negative weights are ordinary coefficients, not outlier flags: §27.8's $\alpha_2 = -1/4$ simply pulls the fit *down* near $x_2 = 0$.
- (iv) **False.** §24.10: the RBF kernel's ϕ is infinite-dimensional — no finite vector exists. The kernel computes the inner product anyway.
- (v) **False.** $f(x) = \sum_i \alpha_i x^T x_i = x^T (\sum_i \alpha_i x_i)$ satisfies $f(0) = 0$ for every α — homogeneous linear, no intercept. The $+1$ in $(x^T x' + 1)^p$ is what smuggles the intercept in (see Problem 3's Note).

Problem 8 — Cost accounting

- (i) Step 1 fills an $n \times n$ matrix: n^2 kernel evaluations ($n(n+1)/2$ distinct, by symmetry). Each RBF evaluation computes $\|x - x'\|^2$ — $O(d)$ arithmetic.
- (ii) Solving the dense $n \times n$ system $(K + \lambda I)\alpha = y$: $\boxed{O(n^3)}$ — the same bill as §24.9's $n \times n$ eigendecomposition. The kernel trick killed the D -dependence (even infinite D is fine) but left the n -dependence.
- (iii) One prediction $f(x) = \sum_{i=1}^n \alpha_i \kappa(x, x_i)$ needs n kernel evaluations: $\boxed{O(nd)}$ for the RBF kernel — linear in the training-set size, the price of the “sum of similarities” form.

28. Regularization: ridge and lasso

§23.10 ended with a promise and a deferral: add a penalty on large parameters to least squares, get ridge regression, and leave the full story — ridge *and* lasso, the bias–variance trade-off, how λ is actually chosen — to this chapter. §27.4 already borrowed ridge for kernel regression. This chapter pays both debts. Everything here comes from the MLT Week 3 slides (Ashish Tendulkar) — the regularization block that closes the regression unit — with the practical λ -selection half from the MLP Week 4 slides.

28.1 The problem, restated: overfitting wears big weights

§23.9’s verdict: capacity too low — underfitting; capacity too high — overfitting. The lecture makes the mechanism concrete with a degree-9 polynomial fit on a small dataset. The fitted weights come out grotesque — $w_4 = -3332.88$, $w_5 = 2993.67$, $w_6 = -1304.04$ — and the lecture’s observation is the whole diagnosis:

- i) The curve threads every training point (training error ≈ 0).
- ii) Between the points it whipsaws wildly — it memorized the noise, §22.10’s memorizer in polynomial costume.
- iii) **The signature of the disease is the weights themselves:** higher-degree features carry enormous, cancelling weights. Overfitting *looks like* huge parameters.

Basically, ... “The degree-9 curve aced the practice exam and flunked the real one — and you can see the cheating in the weights: thousands, cancelling each other out.”

There are two cures (§23.9 named the first): pick a smaller degree m by validation, or **penalize large parameters directly** — keep the degree-9 features, but forbid the weights from exploding. That second cure is regularization.

28.2 The fix: charge rent on big weights

Def (regularized objective). The lecture modifies the least-squares loss by adding a penalty term:

$$J(w) = \frac{1}{2}(Xw - y)^T(Xw - y) + \lambda \cdot \text{penalty}, \quad \lambda > 0.$$

Two components, and the lecture names both:

- i) **The penalty** = a function of the weight vector w — it measures “how big” the weights are.
- ii) **The regularization rate** λ = how much penalty to add — the dial that controls the strength of the cure.

Regularizing changes the loss, so it changes the loss’s derivative, so it changes the weight-update rule in gradient descent. That is the entire machinery: one extra term, and both the closed-form solution and the iterative one shift.

Basically, ... “Fit the data, but pay a tax on big weights. The tax bill has two parts: *what counts as big* (the penalty) and *the tax rate* (λ).”

28.3 Ridge regression: the ℓ_2 penalty

Def (ridge objective). Ridge uses the squared ℓ_2 norm of the weight vector as the penalty:

$$J(w) = \frac{1}{2} \sum_{i=1}^n (w^T x^{(i)} - y^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^m w_j^2, \quad \lambda > 0,$$

in vectorized form

$$J(w) = \frac{1}{2} (Xw - y)^T (Xw - y) + \frac{\lambda}{2} \|w\|_2^2 = \frac{1}{2} (Xw - y)^T (Xw - y) + \frac{\lambda}{2} w^T w.$$

Note (the λ convention, flagged as §27.4 did). This chapter follows the lecture: the penalty carries $\frac{\lambda}{2}$, so the gradient below comes out clean as λw with no stray factor of 2. §23.10 wrote the penalty as $\lambda \|\theta\|^2$ and then absorbed the 2 into λ ; §27.4 uses $\frac{\lambda}{2}$ exactly as here. All three are the same objective up to renaming λ — the minimizer never cares about the bookkeeping.

Basically, ... “Ridge = least squares, plus a tax of $\lambda/2$ on every squared weight. Big weight — big tax bill the optimizer keeps weights small.”

28.4 The ridge solution, by hand

The lecture differentiates term by term (same steps as §23.3’s least-squares gradient, plus the penalty). Expanding,

$$J(w) = \frac{1}{2} (w^T X^T X w - 2w^T X^T y + y^T y + \lambda w^T w),$$

and differentiating each piece (§10.1’s recipe):

$$\begin{aligned} \nabla_w J &= \frac{1}{2} (2X^T X w - 2X^T y + 0 + 2\lambda w) \\ &= \boxed{\nabla_w J = X^T X w - X^T y + \lambda w}. \end{aligned}$$

Eg (one GD step, fully worked). Data: one feature, no intercept, points $(x, y) = (1, 2), (2, 3)$; $\lambda = 2$, step size $\alpha = 0.1$, start $w_0 = 0$. The lecture’s update is $w_{k+1} := w_k - \alpha(X^T(Xw_k - y) + \lambda w_k)$. Here $X^T X = 1^2 + 2^2 = 5$ and $X^T y = 1 \cdot 2 + 2 \cdot 3 = 8$. Then $w_1 = 0 - 0.1(X^T(0 - y) + 0) = 0.1 \cdot 8 = 0.8$. One step already walks from 0 toward the exact answer (Problem 4 checks it against the closed form).

Set the gradient to zero for the closed form:

$$X^T X w - X^T y + \lambda w = 0 \quad \implies \quad \boxed{(X^T X + \lambda I)w = X^T y},$$

$$\boxed{\hat{w}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y}.$$

This is §23.10’s compact form, now earned from the lecture’s $\frac{\lambda}{2}$ convention. Two endpoints, straight from the lecture:

- i) **No regularization.** $\lambda \rightarrow 0$: $\hat{w} \rightarrow (X^T X)^{-1} X^T y$ — the ordinary least-squares solution. The tax is switched off.
- ii) **Infinite regularization.** $\lambda \rightarrow \infty$: the λI term dominates, $\hat{w} \rightarrow 0$ — every weight taxed into oblivion.

And §23.10’s homework result is why the inverse always exists: for $\lambda > 0$ and $z \neq 0$, $z^T(X^T X + \lambda I)z = \|Xz\|^2 + \lambda \|z\|^2 > 0$, so $X^T X + \lambda I$ is positive definite — invertible even when X ’s columns are dependent and plain least squares has infinitely many answers.

Basically, ... “Ridge has a formula, not just an algorithm: $(X^T X + \lambda I)^{-1} X^T y$. Turn λ to zero and you get plain least squares back; crank it to infinity and every weight dies to zero.”

Note (the Bayesian reading: ridge = MAP with a Gaussian prior). §20.12(ii) promised this connection; the MLT lectures never state it — the four lines below are the book’s own derivation, from definitions already in Chapters 19–20. Assume Gaussian noise, $y \mid X, w \sim N(Xw, \sigma^2 I)$ (§19.6):

$$\log P(y \mid X, w) = -\frac{1}{2\sigma^2} \|Xw - y\|^2 + c_1.$$

Put a Gaussian prior on the weights, $w \sim N(0, \tau^2 I)$ — “before seeing data, I believe the weights are small”:

$$\log f(w) = -\frac{1}{2\tau^2} \|w\|^2 + c_2.$$

MAP (§20.10) maximizes the sum:

$$\hat{w}_{MAP} = \arg \max_w \left[-\frac{1}{2\sigma^2} \|Xw - y\|^2 - \frac{1}{2\tau^2} \|w\|^2 \right] = \arg \min_w \left[\frac{1}{2} \|Xw - y\|^2 + \frac{\sigma^2}{2\tau^2} \|w\|^2 \right],$$

exactly the ridge objective (§28.3) with $\boxed{\lambda = \sigma^2/\tau^2}$ — the noise variance over the prior variance. The “tax rate” λ suddenly has a meaning: how noisy you think the data is, divided by how strongly you believed the weights should be small. (Lasso’s counterpart runs the same four lines with a Laplace prior $f(w_j) \propto e^{-c|w_j|}$, whose log is the ℓ_1 penalty — stated as the standard counterpart, not derived.)

28.5 Gradient descent for ridge

From the gradient, the lecture’s update rule:

$$\boxed{w_{k+1} := w_k - \alpha(X^T(Xw_k - y) + \lambda w_k)}.$$

Read it against §23.3’s unregularized update $w_{k+1} := w_k - \alpha X^T(Xw_k - y)$: the only change is the extra $-\alpha\lambda w_k$ — every step, each weight is dragged a little toward zero *in addition* to following the data’s gradient. That drag is the tax, collected per-iteration.

Basically, ... “Gradient descent with ridge = ordinary gradient descent, plus a small tug toward zero on every step. The tug is λ -sized.”

28.6 What λ does: shrinkage, then underfitting

The lecture’s slide on the effect of λ is blunt: **as λ increases, the model loses capacity, and for very large λ it underfits.** The degree-9 polynomial experiment makes it visible:

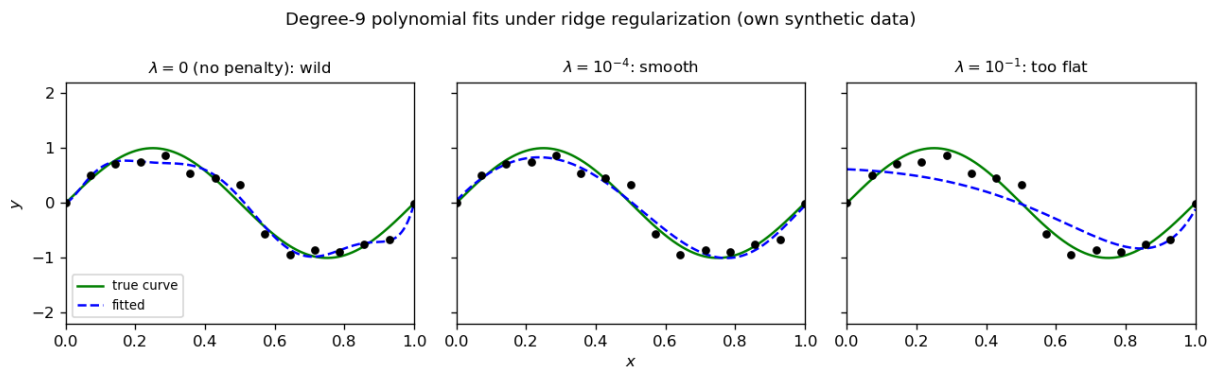


Figure 38: Three panels: degree-9 polynomial fits. Left ($\lambda=0$): dashed fitted curve whipsaws between points. Middle ($\lambda=1e-4$): fitted curve tracks the true sine smoothly. Right ($\lambda=1e-1$): fitted curve is nearly flat, missing the sine’s shape.

- i) $\lambda = 0$: the penalty is off — the wild interpolating curve, §28.1’s disease.
- ii) Small $\lambda > 0$: weights shrink just enough to kill the whipsaw; the curve tracks the true shape. The sweet spot.
- iii) Large λ : weights crushed toward zero; the fit goes nearly flat and misses the signal — underfitting, exactly as the lecture warns.

Note. Shrinking $\neq$ zeroing: ridge makes weights *small*, essentially never *exactly* zero. If you want coefficients killed outright, that is lasso’s job (§28.7).

Basically, ... “ λ is a dimmer switch on model flexibility: 0 = blinding overfit, a little = just right, a lot = the model goes blind and underfits.”

28.7 Lasso: the ℓ_1 penalty, and exact zeros

Def (lasso objective). Lasso swaps the squared penalty for the ℓ_1 norm:

$$J(w) = \frac{1}{2}(Xw - y)^T(Xw - y) + \frac{\lambda}{2}\|w\|_1 = \frac{1}{2}(Xw - y)^T(Xw - y) + \frac{\lambda}{2} \sum_{j=1}^m |w_j|.$$

The lecture is honest about the cost: *estimating the lasso weights needs specialized optimization algorithms, beyond this course’s scope* — the $|w_j|$ kink at zero means no clean gradient-and-inverse route like §28.4. (The MLP course just calls sklearn’s **Lasso**.)

But the headline property needs no optimizer to state: **lasso sets some coefficients *exactly* to zero**. It does feature selection by itself — out of a pile of candidate features, it keeps a few and discards the rest. Two ways to see why:

- i) **The 1-D case (Problem 3 proves it).** With one feature, the lasso solution is the *soft-thresholded* least-squares answer: $\hat{w} = \text{sign}(\rho) (|\rho| - \lambda/2)_+ / \|x\|^2$ with $\rho = x^T y$. If $|\rho| \leq \lambda/2$ — the feature’s pull on the data is weaker than the tax — the weight is not shrunk, it is *deleted*: exactly 0. Ridge’s 1-D answer $\rho / (\|x\|^2 + \lambda)$ only ever shrinks.
- ii) **The geometry.** Minimizing J equals minimizing the squared error subject to a penalty budget $\|w\| \leq t$. The ℓ_1 budget region is a *diamond* — it has corners on the axes. The error contours (ellipses around the OLS point) generically first touch a diamond at a corner, and a corner means some $w_j = 0$. The ℓ_2 budget is a *ball* — smooth, no corners — so the touch point keeps every coordinate nonzero:

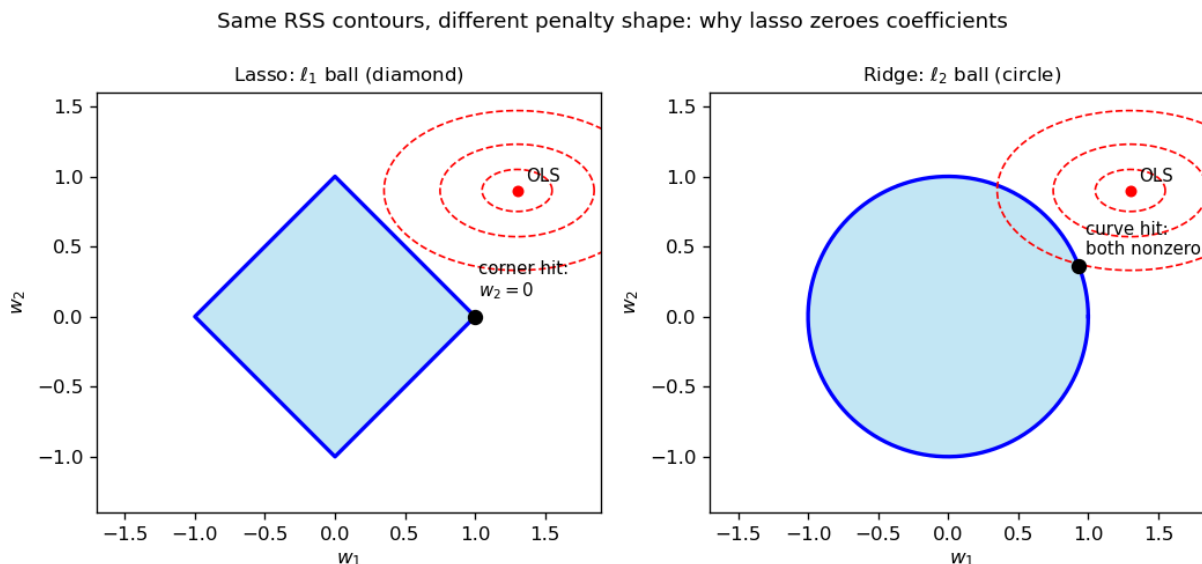


Figure 39: Two panels. Left: diamond $|w_1| + |w_2| = 1$ with elliptical RSS contours; the smallest ellipse touching the diamond touches at the corner (1,0), so $w_2 = 0$. Right: circle $w_1^2 + w_2^2 = 1$; the touching ellipse meets it at a smooth point with both coordinates nonzero.

Basically, ... “Ridge puts weights on a diet; lasso fires some of them. The diamond has corners and the circle doesn’t — corners are where coordinates are exactly zero.”

28.8 Ridge vs lasso, side by side

	Ridge (ℓ_2)	Lasso (ℓ_1)
Penalty	$\frac{\lambda}{2} \ w\ _2^2$	$\frac{\lambda}{2} \ w\ _1$
Closed form	Yes: $(X^T X + \lambda I)^{-1} X^T y$	No — specialized optimizers (beyond scope; sklearn in MLP)
Effect on weights	Shrinks toward 0, never exactly 0	Shrinks <i>and</i> zeroes: automatic feature selection
$\lambda \rightarrow 0$	Least squares	Least squares
$\lambda \rightarrow \infty$	All weights $\rightarrow 0$	All weights $\rightarrow 0$

Note (elastic net). The lecture lists a third type — the *combination* of ℓ_1 and ℓ_2 , elastic net — with no further detail; the MLP slides add its cross-validated estimator **ElasticNetCV**. It inherits lasso’s sparsity and ridge’s stability, and belongs to the same choose-by-validation discipline as everything in this chapter.

Basically, ... “Many correlated features and you want them all, small? Ridge. A pile of features and you suspect only a few matter? Lasso.”

28.9 The bias–variance trade-off, tied to λ

§27.10(i) promised this section; here it is, in exactly the terms the sources support. The lecture never writes a bias–variance decomposition formula — what it gives, twice, is the trade-off itself:

- i) **Too small λ** the fit bends to every wiggle of *this particular* training sample — the degree-9 whipsaw, the memorizer. The model is over-sensitive to the sample’s noise. Call that the **high-variance** end: refit on a fresh sample and the curve swings wildly.

- ii) **Too large λ** the fit is too stiff to follow the true shape — the nearly-flat panel, the lecture’s “the model loses its capacity... underfits.” The model systematically misses the signal no matter the sample. Call that the **high-bias** end.
- iii) **The sweet spot** sits between them, and the λ -vs-validation-error curve (§28.10) draws the trade-off: error falls as the noise-chasing stops, then rises as stiffness sets in. **λ is the dial of the trade-off.**

Note (scope). The formal $\mathbb{E} = \text{bias}^2 + \text{variance} + \text{noise}$ decomposition is not in these lectures; this section stays with their qualitative reading — flexibility against the sample’s noise on one side, stiffness against the signal on the other, λ choosing the balance. Nothing here contradicts the formal version; the course simply never states it.

Basically, ... “Small λ = a nervous student who memorizes the textbook’s typos; large λ = a student who only learned the chapter headings. You want the one in between — and λ picks which student you get.”

28.10 Choosing λ : the validation discipline

λ is a hyperparameter — §22.10’s job (ii), picking the collection, not the parameters. The lecture gives the complete procedure, four steps:

1. **Construct a set of candidate λ values** to experiment with (a grid, usually logarithmic: $10^{-4}, 10^{-3}, \dots, 10^2$).
2. **For each candidate:** train the model, and compute the error on the **validation set** (cross-validation error). Pick the λ with the **least validation error**.
3. **Retrain** with the chosen λ on the **entire training set**.
4. **Report** the final model’s performance on the **test set** — untouched until now (§23.9’s discipline, unchanged).

The validation-error curve is typically **bowl-shaped (U-shaped)**: high at tiny λ (overfitting end), high at huge λ (underfitting end), minimum at the sweet spot. The lecture’s own figure finds the most appropriate $\lambda = 0.1$ at the bowl’s bottom:

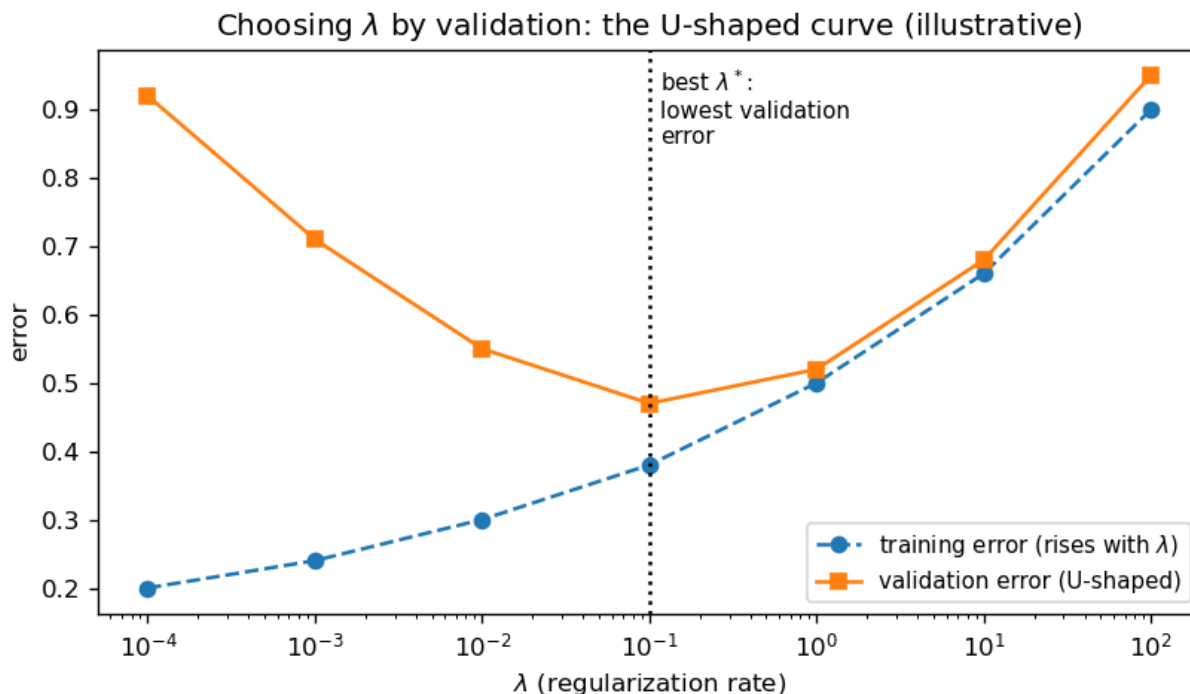


Figure 40: Training error rises monotonically with lambda; validation error is U-shaped with its minimum marked as the chosen lambda*.

Note (the MLP practice). In sklearn the regularization rate is the `alpha` argument — `Ridge(alpha=1e-3)` sets $\lambda = 10^{-3}$ (and `SGDRegressor(alpha=1e-3, penalty='l2')` is the same idea by gradient descent). Doing the four steps by hand for every λ is tedious, so the MLP slides point at the estimators that automate step 2: **RidgeCV / LassoCV / ElasticNetCV perform the cross-validated λ search internally**, and **GridSearchCV / RandomizedSearchCV** generalize the same grid-search pattern to any hyperparameter.

Basically, ... “Don’t guess λ — audition a lineup of them on held-out data, hire the winner, then give it the full training set and report its score on data it has never seen. The U-curve tells you the auditions worked.”

28.11 Worked example: one tiny dataset, three fits

Setup (deliberately nasty): two *nearly collinear* features, three points, no intercept.

$$x_1 = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad x_2 = \begin{pmatrix} 1 \\ 1.9 \\ 3.1 \end{pmatrix}, \quad y = \begin{pmatrix} 2 \\ 4.3 \\ 5.7 \end{pmatrix}.$$

Correlation of the columns ≈ 0.997 — x_2 is almost a copy of x_1 . The sufficient statistics:

$$X^T X = \begin{pmatrix} 14 & 14.1 \\ 14.1 & 14.22 \end{pmatrix}, \quad X^T y = \begin{pmatrix} 27.7 \\ 27.84 \end{pmatrix}, \quad \det(X^T X) = 14 \cdot 14.22 - 14.1^2 = 0.27.$$

That tiny determinant is the warning light: $X^T X$ is nearly singular, so plain least squares will be unstable.

Fit 1 — unregularized ($\lambda = 0$). $(X^T X)^{-1} = \frac{1}{0.27} \begin{pmatrix} 14.22 & -14.1 \\ -14.1 & 14 \end{pmatrix}$, so

$$\hat{w}_1 = \frac{14.22 \cdot 27.7 - 14.1 \cdot 27.84}{0.27} = \frac{1.35}{0.27} = 5, \quad \hat{w}_2 = \frac{-14.1 \cdot 27.7 + 14 \cdot 27.84}{0.27} = \frac{-0.81}{0.27} = -3.$$

Predictions: $X\hat{w} = (5-3, 10-5.7, 15-9.3)^T = (2, 4.3, 5.7)^T = y$ — training MSE = 0. **The trap, exactly as §23.9 warns:** zero training error from weights $(5, -3)$ that violently cancel — one feature’s coefficient is large positive, its near-twin’s large negative. Any fresh noise would swing this fit wildly.

Fit 2 — ridge, $\lambda = 1$. $X^T X + I = \begin{pmatrix} 15 & 14.1 \\ 14.1 & 15.22 \end{pmatrix}$, $\det = 15 \cdot 15.22 - 14.1^2 = 29.49$:

$$\hat{w}_1 = \frac{15.22 \cdot 27.7 - 14.1 \cdot 27.84}{29.49} = \frac{29.05}{29.49} \approx 0.985, \quad \hat{w}_2 = \frac{-14.1 \cdot 27.7 + 15 \cdot 27.84}{29.49} = \frac{27.03}{29.49} \approx 0.917.$$

Predictions $\hat{y} \approx (1.902, 3.712, 5.797)^T$; residuals $\approx (-0.098, -0.588, 0.097)$; training MSE ≈ 0.122 . The wild $(5, -3)$ collapsed to a calm $(0.985, 0.917)$ — both features share the load, same sign, no cancellation. Push further to $\lambda = 5$: $\hat{w} \approx (0.841, 0.832)$, MSE ≈ 0.515 — shrinkage deepening, error rising: the underfitting end of §28.6.

Fit 3 — lasso, $\lambda = 1$. No closed form, so coordinate descent with soft-thresholding (threshold $\lambda/2 = 0.5$; column norms $\|x_1\|^2 = 14$, $\|x_2\|^2 = 14.22$). Start $w = (0, 0)$: - Sweep 1: $w_1 \leftarrow S(27.7, 0.5)/14 = 27.2/14 = 68/35 \approx 1.943$; then $w_2 \leftarrow S(27.84 - 14.1 \cdot 1.942857, 0.5)/14.22 = S(0.4457, 0.5)/14.22 = 0$ — below threshold, *deleted*. - Sweep 2: w_1 recomputes to the same 1.943; w_2 stays 0. Converged: $\hat{w} = (1.943, 0)^T$.

Check (KKT): gradient of the squared-error half at $\hat{w}$ is $X^T(X\hat{w} - y) \approx (-0.5, -0.4457)^T$; for $w_1 \neq 0$ this equals $-\frac{\lambda}{2}\text{sign}(w_1) = -0.5$, and $|-0.4457| \leq 0.5 = \lambda/2$ permits $w_2 = 0$. Training MSE ≈ 0.064 . The redundant twin feature is gone — sparsity, on a hand-checkable example.

Fit	$\hat{w}_1$	$\hat{w}_2$	Train MSE	Reading
OLS ($\lambda = 0$)	5	-3	0	Memorizer: exact on training, unstable weights
Ridge $\lambda = 1$	0.985	0.917	0.122	Shrunk: stable, shares load across twins
Ridge $\lambda = 5$	0.841	0.832	0.515	Over-shrunk: sliding toward underfit
Lasso $\lambda = 1$	1.943	0	0.064	Sparse: redundant feature deleted

Basically, ... “Same three points, three personalities: OLS memorized them with crazy cancelling weights, ridge calmed both weights down, lasso fired the redundant feature outright.”

28.12 Where this goes next

- i) **Ridge, kernelized.** §27.4–§27.6 already ran this chapter’s ridge objective through the kernel trick: $(K + \lambda I)\alpha = y$. Regularization is what keeps that infinite-dimensional fit solvable and honest.
- ii) **The λ discipline generalizes.** Everything in §28.10 — candidates, validation error, retrain, test — is §22.10’s model-selection job (ii) in full; later chapters reuse it for every hyperparameter, not just λ .
- iii) **Priors, derived.** §20.12(ii) promised that ridge’s penalty has a probabilistic reading — MAP with a Gaussian prior. The MLT lectures never state that connection, so §28.4 derives it as the book’s own four-line argument (not a lecture claim): with Gaussian noise and a $N(0, \tau^2 I)$ prior on w , the MAP estimate is exactly the ridge objective, with $\lambda = \sigma^2/\tau^2$.

Problem set

- Ridge gradient, by hand.** For $J(w) = \frac{1}{2}(Xw - y)^T(Xw - y) + \frac{\lambda}{2}w^T w$: (i) expand J fully; (ii) differentiate term by term to get $\nabla_w J = X^T Xw - X^T y + \lambda w$; (iii) write the stationary condition as $(X^T X + \lambda I)w = X^T y$.
- The two endpoints.** From $\hat{w}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y$: (i) argue that as $\lambda \rightarrow 0$, $\hat{w} \rightarrow (X^T X)^{-1} X^T y$ (assume $X^T X$ invertible); (ii) argue that as $\lambda \rightarrow \infty$, $\hat{w} \rightarrow 0$. One line each, citing which term dominates.
- Soft-thresholding in 1-D.** Single feature $x \in \mathbb{R}^n$, no intercept: $J(w) = \frac{1}{2}\|xw - y\|^2 + \frac{\lambda}{2}|w|$, $w \in \mathbb{R}$. With $\rho = x^T y$: (i) show the minimizer is $\hat{w} = \text{sign}(\rho) (|\rho| - \lambda/2)_+ / \|x\|^2$, where $(z)_+ = \max(z, 0)$; (ii) contrast with ridge's 1-D answer $\hat{w}_{\text{ridge}} = \rho / (\|x\|^2 + \lambda)$ — when does lasso give *exactly* zero while ridge never does?
- One GD step vs the closed form.** Data $(x, y) = (1, 2), (2, 3)$, one feature, no intercept; $\lambda = 2$, $\alpha = 0.1$, $w_0 = 0$. (i) Compute w_1 from the lecture's update $w_{k+1} := w_k - \alpha(X^T(Xw_k - y) + \lambda w_k)$. (ii) Compute the exact ridge $\hat{w}$ from §28.4 and say whether w_1 moved toward it.
- Read a λ table.** A grid search reports:

λ	10^{-4}	10^{-3}	10^{-2}	10^{-1}	1	10
Train MSE	0.20	0.24	0.30	0.38	0.50	0.66
Validation MSE	0.92	0.71	0.55	0.47	0.52	0.68

- Which λ does the lecture's procedure pick, and why? (ii) Why is “pick the λ with smallest *training* MSE” wrong — name the section that explains the trap. (iii) After picking, what are the next two steps before reporting?
- Why the diamond zeroes and the ball doesn't.** In one paragraph, using the soft-thresholding result of Problem 3 and the §28.7 figure: explain why the ℓ_1 penalty produces *exactly* zero coefficients while the ℓ_2 penalty only shrinks them.
 - sklearn practice (MLP slides).** (i) In `Ridge(alpha=1e-3)`, what does `alpha` correspond to in this chapter's notation? (ii) What do `RidgeCV` / `LassoCV` automate from §28.10's four steps? (iii) Name the lecture's third regularization type and its CV estimator from the MLP slides.

Solutions — Chapter 28: Regularization: ridge and lasso

Problem 1 — Ridge gradient, by hand

- Expand. $(Xw - y)^T(Xw - y) = (w^T X^T - y^T)(Xw - y) = w^T X^T Xw - w^T X^T y - y^T Xw + y^T y$. The middle two terms are scalars and transposes of each other: $y^T Xw = (w^T X^T y)^T = w^T X^T y$. So

$$J(w) = \frac{1}{2}(w^T X^T Xw - 2w^T X^T y + y^T y + \lambda w^T w).$$

- Differentiate term by term (§10.1's recipe; $X^T X$ is symmetric):

- $\nabla_w \frac{1}{2} w^T X^T Xw = X^T Xw$,
- $\nabla_w \frac{1}{2} (-2w^T X^T y) = -X^T y$,
- $\nabla_w \frac{1}{2} y^T y = 0$,
- $\nabla_w \frac{1}{2} \lambda w^T w = \lambda w$.

Adding: $\boxed{\nabla_w J = X^T Xw - X^T y + \lambda w}$.

- Stationarity $\nabla_w J = 0$ gives $X^T Xw - X^T y + \lambda w = 0$, i.e. $\boxed{(X^T X + \lambda I)w = X^T y}$.

Problem 2 — The two endpoints

- (i) $\lambda \rightarrow 0$: the λI term vanishes, so $(X^T X + \lambda I)^{-1} \rightarrow (X^T X)^{-1}$ (here we use the assumed invertibility), and $\hat{w} \rightarrow \boxed{(X^T X)^{-1} X^T y}$ — ordinary least squares.
- (ii) $\lambda \rightarrow \infty$: factor λ out, $(X^T X + \lambda I)^{-1} = \frac{1}{\lambda}(\frac{1}{\lambda} X^T X + I)^{-1} \rightarrow \frac{1}{\lambda} I \rightarrow 0$ as a matrix, so $\hat{w} = (X^T X + \lambda I)^{-1} X^T y \rightarrow \boxed{0}$. The penalty term dominates the data term.

Problem 3 — Soft-thresholding in 1-D

- (i) $J(w) = \frac{1}{2}(w^2 \|x\|^2 - 2w x^T y + \|y\|^2) + \frac{\lambda}{2}|w|$. Write $\rho = x^T y$ and $s = \|x\|^2 > 0$. For $w \neq 0$ the (sub)derivative is $sw - \rho + \frac{\lambda}{2}\text{sign}(w)$; at $w = 0$ the subdifferential of $\frac{\lambda}{2}|w|$ is the interval $[-\lambda/2, \lambda/2]$.
- If $\rho > \lambda/2$: optimum at $w > 0$ with $sw - \rho + \lambda/2 = 0$, i.e. $w = (\rho - \lambda/2)/s$.
 - If $\rho < -\lambda/2$: optimum at $w < 0$ with $sw - \rho - \lambda/2 = 0$, i.e. $w = (\rho + \lambda/2)/s$.
 - If $|\rho| \leq \lambda/2$: $w = 0$ is optimal, because 0 lies in the subdifferential: $-\rho \in [-\lambda/2, \lambda/2]$.

Together: $\boxed{\hat{w} = \text{sign}(\rho) (|\rho| - \lambda/2)_+ / s}$ with $s = \|x\|^2$.

- (ii) Ridge's 1-D answer $\rho/(s + \lambda)$ is a *smooth rescaling*: it equals 0 only if $\rho = 0$ exactly — otherwise merely smaller than the OLS value ρ/s . Lasso's answer has a **dead zone**: whenever $|\rho| \leq \lambda/2$ (the feature's correlation with y is weaker than half the tax rate), the coefficient is *exactly* 0, not just small. That dead zone is the algebraic face of the diamond's corners (§28.7).

Problem 4 — One GD step vs the closed form

$X = (1, 2)^T$, $y = (2, 3)^T$: $X^T X = 5$, $X^T y = 1 \cdot 2 + 2 \cdot 3 = 8$.

- (i) $w_1 = w_0 - \alpha(X^T(Xw_0 - y) + \lambda w_0) = 0 - 0.1(X^T(-y) + 0) = 0.1 \cdot 8 = \boxed{0.8}$.
- (ii) Exact ridge: $\hat{w} = X^T y / (X^T X + \lambda) = 8 / (5 + 2) = \boxed{8/7 \approx 1.143}$. Yes — $w_1 = 0.8$ moved from 0 toward 1.143 (about 70% of the way in one step, since this 1-D problem's optimal step would be $1/(5 + 2) = 1/7 \approx 0.143$).

Problem 5 — Read a λ table

- (i) The lecture's step 2 picks the λ with the **least validation error**: $\boxed{\lambda^* = 10^{-1}}$, validation MSE 0.47 (the bowl's bottom).
- (ii) Training MSE is smallest at $\lambda = 10^{-4}$ (0.20), but training error is *optimistic* — §23.9's trap: the $\lambda \rightarrow 0$ end memorizes the training sample (§28.1, §28.9's high-variance end), as the validation column proves (0.92, the worst). Never select on training error.
- (iii) Step 3: retrain with $\lambda^* = 0.1$ on the **entire training set** (train + validation pooled). Step 4: report performance on the **test set**, untouched until now.

Problem 6 — Why the diamond zeroes and the ball doesn't

Lasso's 1-D solution (Problem 3) is $\text{sign}(\rho) (|\rho| - \lambda/2)_+ / \|x\|^2$: the $(\cdot)_+$ builds in a dead zone — any coordinate whose data-pull $|\rho|$ falls below the tax threshold $\lambda/2$ is set to *exactly* zero rather than merely shrunk. Geometrically (§28.7's figure), minimizing the lasso objective is minimizing squared error inside the diamond $\|w\|_1 \leq t$, and the error ellipses generically first touch a diamond at a *corner* — a corner sits on an axis, i.e. some $w_j = 0$ exactly. Ridge's 1-D answer $\rho / (\|x\|^2 + \lambda)$ has no dead zone (zero only if $\rho = 0$ exactly), and its budget region is a smooth ball with no corners, so the ellipse touch-point keeps every coordinate nonzero: shrinkage without selection.

Problem 7 — sklearn practice (MLP slides)

- (i) **alpha** is this chapter's λ : `Ridge(alpha=1e-3)` sets the regularization rate to 10^{-3} (same for `SGDRegressor(alpha=1e-3, penalty='l2')`).
- (ii) They automate step 2 of §28.10's procedure: `RidgeCV` / `LassoCV` run the cross-validated search over candidate λ values internally and return the λ with the least CV error — no hand-written grid loop needed. (`GridSearchCV` / `RandomizedSearchCV` generalize the same pattern to any hyperparameter.)
- (iii) The lecture's third type is **elastic net** — the $\ell_1 + \ell_2$ combination — and the MLP slides' CV estimator is `ElasticNetCV`.

29. KNN and decision trees

Two classifiers from the MLT lectures, and a study in opposites. KNN (Week 7 slides) never builds a model at all — it keeps the whole training set and decides each new point by asking its neighbours. Decision trees (Week 9 slides) do the reverse: they compress the training set into an explicit set of if-then rules — a partition of the feature space — and throw the data away at prediction time. Both are §22.9 classifiers ($f : \mathbb{R}^d \rightarrow \{\text{classes}\}$), both handle regression too (the lectures say so for each), and both carry a complexity hyperparameter — k for KNN, tree size/depth for trees — chosen by exactly the discipline §28.12(ii) promised would generalize: §28.10’s candidates $\rightarrow$ validation error $\rightarrow$ retrain $\rightarrow$ test, applied to every hyperparameter, not just λ .

Everything in §§29.2–29.8 comes from the MLT Week 7 KNN slides; §§29.9–29.21 from the MLT Week 9 decision-tree slides (Dr. Ashish Tendulkar). The hand-worked examples in §§29.5 and 29.13 use the chapter’s own tiny datasets, with every number recomputed independently (review log); the lecture’s Play-Tennis numbers in §29.14 are verified, not copied blind.

Notation. The lectures write the i -th example as $x^{(i)}$ (superscript, §22’s convention) with label $y^{(i)}$; $x_j^{(i)}$ is feature j of example i . $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ is the training set. A tree node covers a region R_i holding N_i samples.

29.1 The task both solve: §22.9 classification

Setup. Same as §22.9: instances $x^{(i)} \in \mathbb{R}^d$, but the labels are discrete — $y^{(i)} \in \{1, \dots, K\}$ for K classes (the tree lectures write it this way; §22.9’s ± 1 is the $K = 2$ case). The algorithm outputs $f : \mathbb{R}^d \rightarrow \{1, \dots, K\}$.

- i) **The loss is still §22.9’s.** Misclassification fraction $L(f) = \frac{1}{n} \sum_i \mathbf{1}(f(x^{(i)}) \neq y^{(i)})$ — fraction wrong. KNN and trees just carve the input space into class regions differently than a linear separator does: not one straight cut (§22.9), but many small local decisions.
- ii) **Both also do regression.** Each lecture says so explicitly: KNN averages neighbours’ outputs (§29.4), trees fit a constant per region (§29.15). The chapter leads with classification because both lectures do.

Basically, ... “§22.9’s game — predict a discrete label, count your mistakes — played two new ways: KNN asks ‘who are this point’s neighbours?’, trees ask ‘which box does this point fall in?’”

29.2 KNN: the algorithm — no model, just neighbours

Def (k-nearest neighbours). KNN is a **supervised** algorithm for classification and regression. It is an **instance-based** learning technique: **there is no explicit model**. To label a new example, KNN compares it with the existing training examples, finds the k **nearest neighbours**, and assigns an output based on those neighbours’ labels.

- i) **The key insight** (the lecture’s one-liner): “*An example is labeled by the company it keeps.*” A new point inherits the majority opinion of its neighbourhood.

- ii) **Two hyperparameters** (the lecture names exactly two): **how many neighbours (k)** to consult, and **which distance metric** to compare examples with (§29.3).
- iii) **Nothing is learned up front.** Training = storing the dataset. All the work happens when a prediction is requested — the lecture’s limitation (i) in §29.8 is exactly this.

Basically, ... “KNN doesn’t study for the exam — it brings the textbook into the hall. New point arrives, it measures distances to every stored point, takes a vote among the k closest, done. The only decisions you make: how many voters (k) and what ‘close’ means (the metric).”

29.3 The distance metric: what “near” means

Following the lecture, the two metrics used quite often, for points $x^{(1)}, x^{(2)}$ with m features:

Def (Euclidean distance).

$$\delta(x^{(1)}, x^{(2)}) = \sqrt{(x_1^{(1)} - x_1^{(2)})^2 + \dots + (x_m^{(1)} - x_m^{(2)})^2} = \left(\sum_{j=1}^m (x_j^{(1)} - x_j^{(2)})^2 \right)^{1/2}$$

Vectorized: $\delta = ((x^{(1)} - x^{(2)})^T (x^{(1)} - x^{(2)}))^{1/2}$ — the ordinary straight-line distance, §2-style.

Def (Manhattan distance).

$$\delta(x^{(1)}, x^{(2)}) = |x_1^{(1)} - x_1^{(2)}| + \dots + |x_m^{(1)} - x_m^{(2)}| = \sum_{j=1}^m |x_j^{(1)} - x_j^{(2)}|$$

Vectorized (the lecture’s form): $\delta = \mathbf{1}_{1 \times m}^T |x^{(1)} - x^{(2)}|_{m \times 1}$ — sum of absolute per-feature gaps: the “city-block” walk, no diagonals.

- i) **The metric is a hyperparameter** (§29.2(ii)): the lecture lists it alongside k , with no selection procedure given — in practice it joins k in the §29.7 audition (Problem 2 shows it can flip a prediction).
- ii) **Every feature counts** — which is also the sensitivity the lecture warns about (§29.8): all m features enter the sum, relevant or not.

eg 1 (both metrics, one pair). $x^{(1)} = (1, 2)$, $x^{(2)} = (4, 6)$: Euclidean $\sqrt{9 + 16} = \boxed{5}$; Manhattan $|-3| + |-4| = \boxed{7}$.

Basically, ... “Euclidean = as the crow flies (square each gap, add, square-root). Manhattan = as the taxi drives (add the raw gaps). Both turn ‘how alike are these two examples?’ into one number — and that number is the whole algorithm’s raw material.”

29.4 Voting (classification) and averaging (regression)

Classification. The k neighbours take part in voting; **the class that receives the highest number of votes is the predicted class** (majority vote).

Regression. The prediction is the **average** of the neighbours’ outputs:

$$\hat{y} = \frac{1}{k} \sum_{i=1}^k y_i.$$

eg 2 (regression, by hand). A new house’s $k = 3$ nearest neighbours (by area, bedrooms, ...) sold for 2.0, 3.5, 6.5 (crore): $\hat{y} = \frac{1}{3}(2.0 + 3.5 + 6.5) = \boxed{4.0}$.

- i) **Vote for labels, average for numbers.** Discrete $y^{(i)} \rightarrow \text{count}$; real $y^{(i)} \rightarrow \text{mean}$. Same neighbours, different aggregation.
- ii) **Ties are the lecture's silence.** With even k (or $k \rightarrow n$ on balanced classes) the vote can tie; the slides prescribe no tie-break — flagged in the review log, and the chapter's examples keep strict majorities.

Basically, ... “Classification: the neighbours vote, majority wins. Regression: the neighbours' numbers get averaged. That's the entire ‘model’ — there isn't one.”

29.5 Worked: 3-NN on seven 2-D points

Data (class A = blue, B = red), query points $q = (2, 2)$, $q_2 = (4, 2)$:

$$A_1 = (0, 0), A_2 = (1, 4), A_3 = (3, 1), \quad B_1 = (5, 2), B_2 = (4, 5), B_3 = (6, 3), B_4 = (7, 2).$$

Euclidean distance, squared (comparing squares orders the same as roots):

neighbour	$d^2(q, \cdot)$	$d^2(q_2, \cdot)$
A_1	$(2)^2 + (2)^2 = 8$	$(4)^2 + (2)^2 = 20$
A_2	$(1)^2 + (2)^2 = 5$	$(3)^2 + (2)^2 = 13$
A_3	$(1)^2 + (1)^2 = 2$	$(1)^2 + (1)^2 = 2$
B_1	$(3)^2 + (0)^2 = 9$	$(1)^2 + (0)^2 = 1$
B_2	$(2)^2 + (3)^2 = 13$	$(0)^2 + (3)^2 = 9$
B_3	$(4)^2 + (1)^2 = 17$	$(2)^2 + (1)^2 = 5$
B_4	$(5)^2 + (0)^2 = 25$	$(3)^2 + (0)^2 = 9$

The $k = 3$ vote. For q : nearest are A_3 (2), A_2 (5), A_1 (8) — three A votes $\rightarrow$ predict A . For q_2 : nearest are B_1 (1), A_3 (2), B_3 (5) — two B , one $A \rightarrow$ predict B .

Boundary reasoning. Every plane point gets the majority label of its 3 nearest training points, so the plane splits into an A -region (around the blue cluster) and a B -region (around the reds) — the shaded regions in the figure. q sits inside A 's region, q_2 inside B 's; the boundary threads between them.

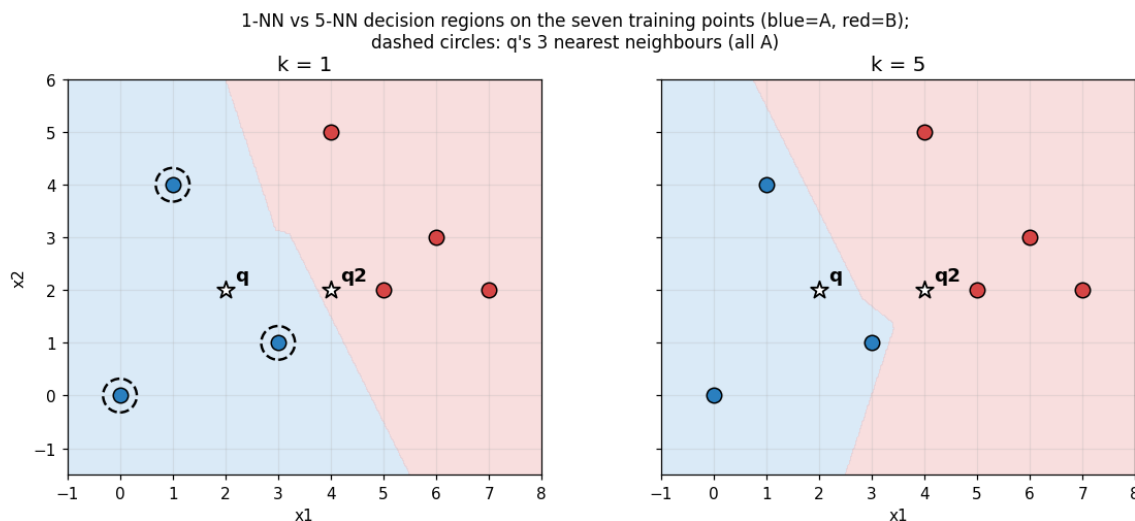


Figure 41: Two panels: 1-NN (left) and 5-NN (right) decision regions on the seven training points of the worked example (blue=A, red=B); query stars $q=(2,2)$ in the blue region and $q_2=(4,2)$ in the red region; q 's three nearest neighbours (all A) circled with dashes. The 1-NN boundary is jagged and intricate; the 5-NN boundary is smoother.

Basically, ... “Measure seven distances, sort, take the top three, count the colours. q ’s three closest are all blue — lands in the blue zone. q_2 ’s are two red, one blue — lands red. The coloured zones in the figure are just this vote repeated for every point of the plane.”

29.6 Decision boundaries: what k does to the shape

Run the §29.5 vote at every plane point and the class regions emerge — the **decision boundary** is where the vote flips. The lecture’s panels (boundaries for $k = 9, \dots, 16$ on its demo data) and the figure above agree on the pattern:

- i) **Small k (1, 2): jagged, overfit.** The model is **sensitive to noise** — it adjusts to tiny wiggles in the data, and the boundary turns very jagged (left panel above: each training point carves its own intricate cell).
- ii) **Large k : smooth, underfit.** The model gets **biased** — it ignores the underlying trend, and the boundary smooths out (right panel: one calm curve).
- iii) $k \rightarrow n$: **majority rules everything.** As k approaches the dataset size, the neighbourhood *is* the dataset, so the model predicts the **majority class for every possible example** — on §29.5’s data (3 A vs 4 B), $k = 7$ labels the entire plane B .

Note (the $k = 1$ boundary, the book’s own observation — not a lecture claim). With $k = 1$ each plane point takes the label of its single nearest training point, so the boundary is exactly the Voronoi tessellation of the training set: each point owns the cell of plane closest to it. “Voronoi-like” is this, softened for larger k — every KNN boundary is **piecewise**, built from distance comparisons, never a single straight cut like §22.9’s separator.

Basically, ... “ k is a smoothness knob. Twist it to 1: the boundary hugs every training point, noise included — jagged memorization. Twist it up: the boundary relaxes into a smooth curve — and at the extreme it gives up and calls everything the majority class. Small k overfits, large k underfits: the same U-story as §28.9’s λ .”

29.7 Choosing k : §28.10’s discipline, reused

k is a hyperparameter (§29.2(ii)) — so §28.12(ii) applies verbatim: run §28.10’s four steps with k in λ ’s seat:

1. **Candidates** — a grid of k values (1, 2, ...).
 2. **Validation error** — train (store) on the training split, score each k on held-out data; pick the k with the **least validation error**.
 3. **Retrain** on the full training set with the winning k .
 4. **Report** on the test set, untouched until now.
- i) **The lecture’s chart is this procedure’s output.** Its “Error vs k ” plot (error rate against $k = 1, \dots, 16$) dips and rises; the lecture’s rule: “*The value k that yields minimum test error is most suitable.*”
 - ii) **“Test” vs “validation” — a wording note.** The slide says *test* error; the book’s standing discipline (§22.10, §28.10) selects on *validation* data and reserves the test set for the final report — selecting on the test set would spend it. The curve’s shape and the rule’s spirit are the lecture’s; the split hygiene is §28.10’s (flagged in the review log).

Basically, ... “Don’t guess k — audition 1, 2, 3, ... on held-out data and hire the one with the smallest error, exactly the §28.10 ritual from the λ chapter. The error-vs- k curve is the audition scorecard.”

29.8 KNN: strengths and limits (as the lecture lists them)

Advantages. i) **Easy to understand and implement** — the whole algorithm is §29.2–§29.4. ii) **Interpretable predictions** — a prediction can be *explained by its neighbours* (“you’re class A because your 3 nearest stored examples are A ”).

Limitations. i) **Slow at prediction time on large training sets** — all computations are performed at runtime (§29.2(iii)): every query measures distance to every stored point. ii) **Sensitive to redundant or irrelevant features** — all m features enter the distance sum (§29.3(ii)), so junk features vote as loudly as signal. iii) **Outperformed on hard tasks** — on significantly difficult problems it can lose to SVMs and neural networks (the lecture’s words).

Basically, ... “KNN’s pitch: trivial to build, and every answer comes with its receipts (the neighbours). KNN’s price: it does all its work at query time, it can’t tell signal features from noise, and on genuinely hard problems the heavy machinery wins.”

29.9 Decision trees: recursive partitioning

Def. Decision trees are **non-parametric supervised** methods for classification and regression. **Tree-based methods partition the feature space into a set of rectangles (or cuboids) and then fit a simple model (like a constant) in each one.** Conceptually simple yet powerful; a **binary** tree splits into two branches at each node.

- i) **Partition, then predict.** The tree asks a sequence of yes/no questions about the features; each root-to-leaf path carves out one rectangle of feature space, and every point landing in the same leaf gets the same prediction (majority class, or region mean — §§29.13, 29.15).
- ii) **Non-parametric** = no fixed parameter vector sized by the modeller: the tree’s shape *is* the model, and it grows with the data (contrast §23’s fixed w).
- iii) **Three questions drive everything** (the lecture poses them up front): **which attribute** to split on? **what splitting criterion?** **what tree size** is optimal? — §§29.10–29.12, §29.16, and §§29.17–29.19 answer them in order.

Basically, ... “A decision tree plays twenty questions with the features: ‘is outlook sunny?’ → yes → ‘is humidity high?’ → ... Each answer narrows down a rectangle of the feature space until the tree reaches a leaf and announces the verdict for everyone inside that box. The learning is choosing the questions.”

29.10 ID3: the greedy top-down algorithm

Most tree-learning algorithms are variations of one core algorithm, **ID3** (Iterative Dichotomizer 3). It employs a **top-down, greedy search** through the space of possible trees.

The procedure. Start with the question “*which attribute should be tested at the root of the tree?*” — each attribute is evaluated with some measure of how well it alone classifies the training examples (§§29.11–29.12):

1. **Select the best attribute**, split at the root node.
2. **Create one descendant per possible value** of that attribute; send each training example to the matching descendant.
3. **Repeat recursively** on each descendant’s examples — pick the best attribute for *that* subset, split again.

- i) **Greedy = never backtracks.** Once an attribute is chosen at a node, the algorithm never reconsiders earlier choices — it takes the locally best split at every step and hopes the tree comes out acceptable.
- ii) **Stopping criterion** — recursion on a subset stops when:
 - every element in the subset belongs to the **same class** → leaf, labelled with that class;
 - **no attributes remain** but classes still mix → leaf, labelled with the **most common class** of the subset;
 - the subset is **empty** (no training example matched that attribute value).

Basically, ... “ID3 builds the tree like you’d actually interrogate a dataset: at each node ask the single most revealing question (§29.12 scores ‘revealing’), branch on every possible answer, and repeat inside each branch. It never second-guesses an old question — greedy, one pass down. Stop when a branch is pure, out of questions, or empty.”

29.11 Measuring impurity: how mixed is this node?

Let $p_{i,k}$ be the **proportion of data points in node i (region R_i) assigned to class k** ($k = 1, \dots, K$):

$$p_{i,k} = \frac{1}{N_i} \sum_{x^{(i)} \in R_i} \mathbf{1}(y^{(i)} = k), \quad N_i = \text{samples in } R_i.$$

Three ways to score a node's impurity:

Misclassification error. The fraction misclassified if the node predicts its majority class:

$$Q_i(T) = 1 - p_{i,k(i)} = \frac{1}{N_i} \sum_{x^{(i)} \in R_i} \mathbf{1}(y^{(i)} \neq k(i)), \quad k(i) = \arg \max_k p_{i,k}.$$

Gini index.

$$G_i = \sum_{k=1}^K p_{i,k}(1 - p_{i,k})$$

Two readings (both the lecture's): (a) instead of predicting the majority class, predict class k **with probability** $p_{i,k}$ — the training error rate of that randomized rule *is* the Gini index; (b) code each observation as 1 for class k and 0 otherwise — the variance of that 0–1 response over the node is $p_{i,k}(1 - p_{i,k})$, and summing over classes gives Gini again.

Entropy.

$$H_i = - \sum_{k=1}^K p_{i,k} \log_2 p_{i,k}$$

0 for a pure node, maximal ($\log_2 K$) for a perfectly mixed one.

- i) **All three are 0 iff the node is pure** and grow as the mix evens out — they agree on “pure vs mixed”, differ in *how much* they care about shades of mixing (§29.16, Problem 7).
- ii) **ID3's original choice is entropy** (via §29.12); CART's classification trees (§29.16) default to Gini.

eg 3 (all three, one node). Node with 6 of class + and 2 of class − ($p = \frac{3}{4}, \frac{1}{4}$): misclassification $1 - \frac{3}{4} = \boxed{\frac{1}{4}}$;

Gini $\frac{3}{4} \cdot \frac{1}{4} + \frac{1}{4} \cdot \frac{3}{4} = \boxed{\frac{3}{8}}$; entropy $-\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} \approx \boxed{0.811}$.

Basically, ... “Impurity = ‘how mixed is the class soup in this node?’ Three thermometers: misclassification (‘fraction I’d get wrong betting the majority’), Gini (‘error rate if I guessed randomly *in proportion* to the mix’), entropy (‘the information-theory measure of mixed-up-ness’). Pure node → all read zero.”

29.12 Information gain: the splitting score

Def. Information gain is the expected **reduction in entropy** from partitioning the examples by an attribute — how much the split un-mixes the soup:

$$IG(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v),$$

where S_v = the examples with $A = v$. In words (the lecture's): $IG(\text{attr})$ = entropy of dataset — entropy of attribute — the weighted child entropy subtracted from the parent's.

ID3's rule: compute IG (equivalently, the child entropy) for each remaining attribute; **split on the attribute with the largest information gain** — equivalently, the smallest weighted child entropy.

- i) **Gain ≥ 0 always:** a split can't increase the weighted entropy — at worst it teaches nothing ($IG = 0$).
- ii) **Why entropy and not misclassification for growing?** Preview of §29.16: entropy/Gini react to *any* probability shift, misclassification only to majority flips — the growing phase wants the sensitive instrument.

Basically, ... “Information gain = ‘how much less mixed-up do things get if I split on this attribute?’ Compute the parent’s entropy, subtract the size-weighted average of the children’s entropies. Biggest drop wins the split. It’s the scoreboard that answers ID3’s root question.”

29.13 Worked: one full split computation by hand

Ten labelled items (defective – vs ok + widgets), two categorical features:

#	colour	size	label
1–3	red	big	+, +, +
4–5	red	small	–, –
6	blue	big	+
7–8	blue	big	–, –
9–10	blue	small	–, –

Parent entropy. 4 +, 6 –:

$$H(S) = -\frac{4}{10} \log_2 \frac{4}{10} - \frac{6}{10} \log_2 \frac{6}{10} \approx 0.529 + 0.442 = \boxed{0.971}.$$

Split on size. Big: 4 +, 2 – $\rightarrow H = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} \approx 0.918$. Small: 0 +, 4 – $\rightarrow H = 0$ (pure).
Weighted: $\frac{6}{10}(0.918) + \frac{4}{10}(0) = 0.551$.

$$\boxed{IG(S, \text{size}) = 0.971 - 0.551 = 0.420}.$$

Split on colour. Red: 3 +, 2 – $\rightarrow H \approx 0.971$. Blue: 1 +, 4 – $\rightarrow H = -\frac{1}{5} \log_2 \frac{1}{5} - \frac{4}{5} \log_2 \frac{4}{5} \approx 0.722$.
Weighted: $\frac{5}{10}(0.971) + \frac{5}{10}(0.722) = 0.846$.

$$\boxed{IG(S, \text{colour}) = 0.971 - 0.846 = 0.125}.$$

Size wins ($0.420 > 0.125$) $\rightarrow$ **root splits on size**. Small branch: pure – $\rightarrow$ leaf “–”. Big branch (4 +, 2 –, $H = 0.918$): split on colour — red-big +, +, + $\rightarrow$ leaf “+”; blue-big +, –, – $\rightarrow$ mixed, no attributes left $\rightarrow$ leaf with the most common class, $\boxed{-}$. Final tree: size=small $\rightarrow$ –; size=big $\rightarrow$ colour=red $\rightarrow$ +, colour=blue $\rightarrow$ –.

Gini cross-check (same verdict): $G(S) = 1 - 0.4^2 - 0.6^2 = 0.48$; size-split weighted Gini $\frac{6}{10}(1 - (\frac{4}{6})^2 - (\frac{2}{6})^2) + 0 = 0.267$; Gini gain $\boxed{0.213}$ — again the biggest drop, so Gini would also pick size first.

Basically, ... “Score both questions: ‘split by size?’ un-mixes the soup by 0.42 bits, ‘split by colour?’ by only 0.125. Ask the size question first. The small branch is pure — done. The big branch still mixes, so ask colour inside it; blue-and-big ends 1-to-2 against, so the leaf calls it – by majority. Two questions, three leaves, every number above recomputed twice.”

29.14 The lecture’s tennis example (numbers verified)

The slides work ID3 on the classic Play-Tennis data: 14 examples, 9 “Yes” / 5 “No”. Recomputed independently (review log) — the lecture’s rounded values check out:

$$H(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} \approx \boxed{0.940}.$$

attribute	split counts	IG
Wind	weak 6Y/2N ($H = 0.811$), strong 3Y/3N ($H = 1$)	$0.940 - \frac{8}{14}(0.811) - \frac{6}{14}(1) = \boxed{0.048}$
Outlook	sunny 2Y/3N (0.971), overcast 4Y/0N (0), rain 3Y/2N (0.971)	$0.940 - \frac{5}{14}(0.971) - \frac{4}{14}(0) - \frac{5}{14}(0.971) = \boxed{0.246}$
Temperature	cold 3Y/1N (0.811), mild 4Y/2N (0.918), hot 2Y/2N (1)	$\boxed{0.029}$
Humidity	high 3Y/4N (0.985), normal 6Y/1N (0.593)	$\boxed{0.151}$

Outlook's 0.246 is the largest $\rightarrow$ **root = Outlook**. Recursing the same way inside each branch gives the lecture's final tree:

The lecture's final tree for the Play-Tennis data (ID3: Outlook first, $IG = 0.246$)

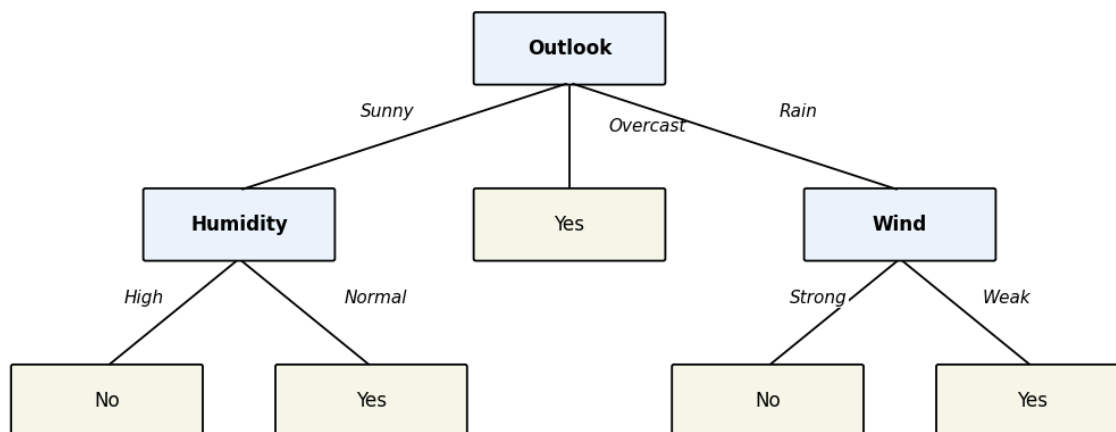


Figure 42: Decision tree: root Outlook; branch Sunny leads to Humidity, which splits into High $\rightarrow$ No and Normal $\rightarrow$ Yes; branch Overcast leads directly to leaf Yes; branch Rain leads to Wind, which splits into Strong $\rightarrow$ No and Weak $\rightarrow$ Yes.

Basically, ... “Four candidate questions, four scores: Outlook un-mixes the most (0.246 bits), so it goes at the top. Same scoring inside each branch grows the rest. The picture is the whole classifier — follow the answers down to a leaf.”

29.15 CART: regression trees — constants per region

CART models **both** regression and classification, learning classification trees very much like ID3 (§29.16). For regression, suppose the space is partitioned into M regions $R_1, \dots, R_M$, with a **constant** c_i predicted in each:

$$\hat{y} = \sum_{i=1}^M c_i \mathbf{1}(x \in R_i)$$

($\mathbf{1}(\cdot)$ is the indicator). Learning = choosing the partition *and* the constants. Three sub-questions (the lecture's): which variable splits each node? what threshold s ? what constant c_i per region?

The greedy answer. Fixing a partition and minimizing the sum-of-squares $J = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$ is easy — but the *best binary partition* for minimum SSE is computationally infeasible, so CART goes greedy: start with all data, consider splitting variable j at split point s into half-planes

$$R_1 = \{x \mid x_j \leq s\}, \quad R_2 = \{x \mid x_j > s\}$$

and solve

$$\min_{j,s} \left[\min_{c_1} \sum_{i: x^{(i)} \in R_1} (y^{(i)} - c_1)^2 + \min_{c_2} \sum_{i: x^{(i)} \in R_2} (y^{(i)} - c_2)^2 \right].$$

The inner minimum is the mean (differentiate $\sum (y^{(i)} - c_1)^2$ w.r.t. c_1 : $-2 \sum (y^{(i)} - c_1) = 0$):

$$\hat{c}_1 = \frac{1}{N_1} \sum_{i: x^{(i)} \in R_1} y^{(i)}, \quad \hat{c}_2 = \frac{1}{N_2} \sum_{i: x^{(i)} \in R_2} y^{(i)}$$

— the region averages, exactly k-means' centroid logic (§26.3(i)) in new clothes. For each variable, the best (j, s) is found by scanning through the inputs; then partition, and **repeat the splitting process on each resulting region**.

- i) **Variance reduction** can replace SSE as the splitting score for regression trees (the lecture's one-line alternative).
- ii) **ID3 vs CART splits.** ID3 branches one-per-attribute-value (multiway, categorical); CART always splits **binary** on $x_j \leq s$ — even categorical-looking splits become thresholded binaries.

eg 4 (region means). Region R_1 holds outputs $\{2, 4, 6\}$: $\hat{c}_1 = \frac{12}{3} = \boxed{4}$ — the constant that minimizes that region's SSE (check: $(2-4)^2 + 0 + (6-4)^2 = 8$; any other c scores worse).

Basically, ... “CART for numbers: chop the space into boxes, predict each box's average. To choose where to chop, try every variable and every cut point, score each by ‘sum of squared misses around the two boxes' averages’, keep the best cut, repeat inside each box. The box's prediction is just its average — calculus says so in one line.”

29.16 Classification trees: swap the loss, keep the growing

For classification, growing (and pruning) works the same — except SSE is replaced by a node-impurity measure. With $p_{i,k}$ from §29.11, classify node i 's observations as class k if

$$p_{i,k} > p_{i,j} \quad \forall j \neq k$$

— the majority class, §29.10's leaf rule in symbols.

Which impurity $Q_i(T)$? The lecture lists three — **misclassification error**, **Gini index**, **cross-entropy** — with a division of labour: i) **For growing the tree: Gini or cross-entropy.** Both are differentiable, hence

more amenable to numerical optimization — and, more importantly, **more sensitive to node probabilities** than the misclassification rate (Problem 7 exhibits the gap: two splits with identical misclassification error but different Gini). ii) **For pruning the tree: misclassification rate.** Once the tree exists and the question is which branches to cut (§29.19), the plain error rate is the criterion.

Basically, ... “Classification CART = regression CART with the squared-error score swapped for an impurity score. Grow with the sensitive instruments (Gini/cross-entropy — they notice probability shifts the raw error rate sleeps through); prune with the honest one (plain misclassification). The leaf’s verdict is always the majority class.”

29.17 Overfitting: deep trees memorize

The problem. Overfitting in trees = the tree is designed to **perfectly fit all training samples** — branches with strict rules and sparse data — which wrecks accuracy on test data. **A very large tree might overfit; a small tree might not capture the important structure.** Without depth control, the tree fits the noise too (the lecture’s figure: an uncontrolled tree’s boundary writhes around every stray point).

Def (maximum depth). The length of the longest path from root to leaf; **the root has depth 0.**

- i) **Tree size is a tuning parameter** — the lecture’s words. Like k (§29.7) and λ (§28.10), depth/max-depth is a hyperparameter: audition candidates by validation error (§28.12(ii)), not by training fit.
- ii) **Rectangles are the mechanism.** Each extra level doubles down on axis-aligned boxes; deep enough, every training point gets its own tiny box — zero training error, pure memorization.

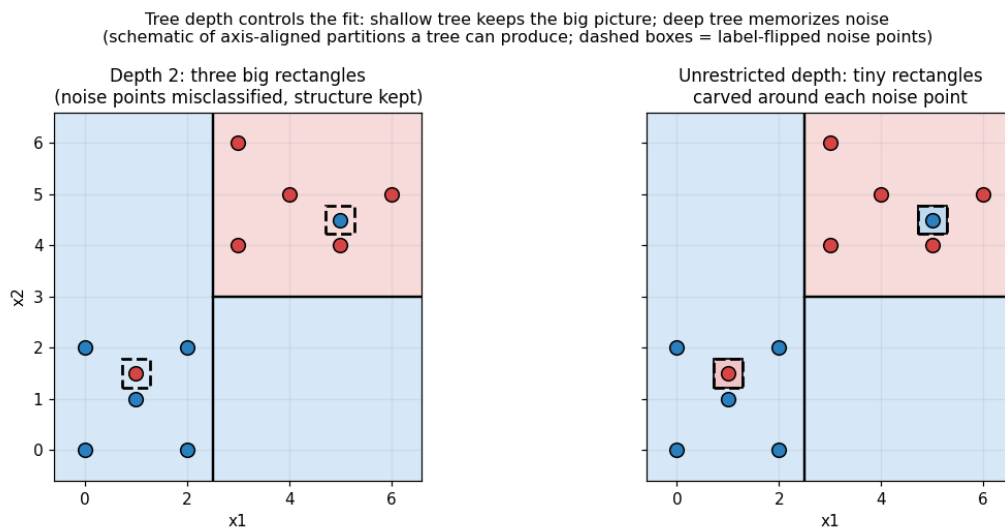


Figure 43: Two panels on the same 2-D data with two label-flipped noise points (dashed boxes). Left: a depth-2 tree’s three big axis-aligned rectangles misclassify the noise but keep the structure. Right: an unrestricted-depth tree carves tiny rectangles around each noise point, fitting them exactly.

Basically, ... “Let the tree grow unchecked and it stops learning the pattern and starts memorizing the roster — one tiny box per training point, noise points included. The figure’s right panel is that pathology: perfect on training, clueless on anything new. Depth is the leash: too long, it memorizes; too short, it can’t see the structure.”

29.18 Pre-pruning: stop before it overgrows

One approach: **stop the algorithm before it becomes a fully-grown tree**. Stopping criteria (the lecture's): - stop if the **number of samples** in a node is below some user-specified threshold; - stop if **expanding the node does not improve impurity**.

- i) **Cheap and direct** — no second phase; the tree simply never grows the dubious branches.
- ii) **But short-sighted**. The lecture's objection: *“a seemingly worthless split might lead to a very good split below it”* — refusing a split with zero immediate gain can block the split beneath it that would have paid off. Greedy stopping can't see around the corner.

Basically, ... “Pre-pruning = quit while you're ahead: don't split tiny nodes, don't split when impurity won't budge. Simple — but it can chicken out one level too early, killing a useless-looking split that was guarding a great one underneath.”

29.19 Post-pruning: grow big, then cut back (cost-complexity)

The lecture's **preferred strategy**: grow a large tree T_0 — stopping the splitting only when some minimum node size is reached — then **prune it back**. Pruning balances residual error against model complexity.

Setup. A **subtree** $T \subset T_0$ is any tree obtainable by pruning nodes off T_0 . For region R_i the prediction is the mean

$$c_i = \frac{1}{N_i} \sum_{x^{(i)} \in R_i} y^{(i)},$$

with residual sum-of-squares contribution

$$Q_i(T) = \sum_{x^{(i)} \in R_i} (y^{(i)} - c_i)^2.$$

The pruning criterion (cost-complexity):

$$C(T) = \sum_{i=1}^{|T|} Q_i(T) + \alpha |T|,$$

$|T|$ = number of leaf nodes. The **regularization parameter** $\alpha \geq 0$ trades residual error against complexity: **large $\alpha \rightarrow$ smaller trees**, small $\alpha \rightarrow$ larger trees — the same shape as §28's λ disciplining weights.

- i) α **is chosen by cross-validation** — yet another hyperparameter joining the §28.12(ii) audition line (k , depth, α).
- ii) **Post- beats pre-** precisely because of §29.18(ii): growing past the “worthless” split lets the good split below it appear; pruning then removes only what the error-vs-complexity trade can't justify (Problem 8 works the arithmetic).

Basically, ... “Post-pruning = let the tree overgrow, then haircut it. Score every candidate haircut by ‘total squared miss + $\alpha \times$ (number of leaves)’ and keep the cheapest. Big α = expensive leaves = severe haircut. α itself gets picked by cross-validation — hyperparameters all the way down.”

29.20 KNN vs decision trees

	KNN (§§29.2–29.8)	Decision trees (§§29.9–29.19)
Model built?	No explicit model — the training set <i>is</i> the model	Explicit model: the partition + leaf constants, built at training time
Work happens...	...at prediction time (all computation at runtime)	...at training time; prediction is a root-to-leaf walk (logarithmic cost, §29.21)
Decision shape	Piecewise, Voronoi-like neighbourhoods (§29.6)	Axis-aligned rectangles (§29.9)
Complexity knob	k (small = jagged/overfit, large = smooth/underfit)	tree size / max depth (deep = memorize, shallow = underfit)
Knob chosen by	validation error (§29.7)	validation error (§29.17(i)); α by CV (§29.19)
Interpretability	“your neighbours are...” (§29.8)	the tree visualizes directly (§29.21)
Weakness	slow queries; junk features pollute distances	high variance; overfits without pruning

- i) **Complementary pathologies.** KNN’s risk is at *query* time (slow, distance-dominated); trees’ risk is at *training* time (variance, overfitting). §29.8(iii) and §29.21’s disadvantages say so respectively.
- ii) **The lecture never says “lazy” or “eager”** — those textbook labels for this exact contrast are omitted here; the table uses the lectures’ own vocabulary (“no explicit model”, “computations performed at runtime”). Flagged in the review log.

Basically, ... “KNN memorizes the data and thinks at query time; trees think at training time and memorize the *rules*. KNN’s boundary is drawn from neighbourhoods, the tree’s from boxes. Both have one complexity knob, both tune it on held-out data — and the lectures describe the contrast without ever saying ‘lazy’ or ‘eager’.”

29.21 Trees: advantages and disadvantages (as the lecture lists them)

Advantages. i) **Versatile** — classification, regression, even multi-output tasks. ii) **Simple to understand and interpret** — trees can be visualized (§29.14’s picture *is* the model). iii) **Little data preprocessing needed** — splits are threshold comparisons, so scaling/monotonic transforms don’t matter. iv) **Logarithmic prediction cost** — using the tree costs log in the number of training samples (walk down, don’t scan).

Disadvantages. i) **Overly complex trees don’t generalize** — pruning (§§29.18–29.19) is needed to address this. ii) **High variance** — small data changes can reshuffle the splits; mitigated by **ensembles** (Chapter 34’s topic — the “Where this goes next” pointer). iii) **Piecewise-linear approximation, poor extrapolation** — constant-per-box fits can’t extend trends beyond the training range.

Basically, ... “Trees: draw the model as a picture, predict by walking down it in log time, barely preprocess anything. The bill: they overfit unless pruned, they’re twitchy (high variance — ensembles calm them, Chapter 34), and boxes can’t extrapolate a trend.”

29.22 Where this goes next

- i) **The toolbox continues.** Next in §25.12(iv)’s list: SVMs (Chapters 32–33) — the “heavy machinery” §29.8(iii) name-checks — then ensembles (Chapter 34), which directly attack trees’ high-variance weakness (§29.21(ii)): the variance-reduction story the tree lecture only gestures at.

- ii) **The hyperparameter discipline, fully generalized.** §28.12(ii)'s promise is now visibly kept: λ (§28.10), k (§29.7), max depth (§29.17(i)), and the pruning parameter α (§29.19(i)) are all chosen the same way — candidates, validation error, retrain, test. One ritual, every knob.
- iii) **From here to deep learning.** Trees partition with axis-aligned boxes and KNN with distance neighbourhoods — both are *fixed* geometries. The coming chapters (and Part V) replace the fixed geometry with *learned* ones: kernels (§27) already hinted at it; neural networks make the feature space itself trainable.

Problem set

- 3-NN vote, by hand.** On §29.5's seven points, for $q_2 = (4, 2)$: (i) list the squared Euclidean distances to all seven points; (ii) name the 3 nearest neighbours; (iii) give the vote and the predicted label. (iv) Repeat the vote for $k = 5$.
- The metric can flip the answer.** Query $q = (0, 0)$; training points $a = (5, 0)$ labelled $+$, $b = (3, 3)$ labelled $-$. With $k = 1$: (i) which point is nearest under Euclidean distance, and what is predicted? (ii) Which under Manhattan distance, and what is predicted? (iii) One line: why does the lecture list the metric as a hyperparameter?
- KNN regression.** A query's $k = 3$ nearest neighbours have outputs 2.0, 3.5, 6.5. (i) Compute $\hat{y}$ from §29.4. (ii) In one line: why average here but vote for classification?
- The effect of k .** On §29.5's data (3 A , 4 B), query $q = (2, 2)$: (i) predict with $k = 1, 3, 5$; (ii) predict with $k = 7$ and explain via the lecture's " k close to n " claim; (iii) using the §29.5 figure, describe in two lines how the boundary changes from $k = 1$ to $k = 5$.
- Verify the lecture's Wind number.** Given $H(S) = 0.940$, $H(S_{\text{weak}}) = 0.811$ (8 examples), $H(S_{\text{strong}}) = 1.0$ (6 examples): recompute $IG(S, \text{Wind})$ from §29.12's formula and confirm the lecture's 0.048.
- Finish the §29.13 comparison.** For the ten-widget dataset: (i) compute H of the red branch (3 $+$, 2 $-$) and the blue branch (1 $+$, 4 $-$); (ii) compute $IG(S, \text{colour})$ and confirm it is $\approx 0.125 < IG(S, \text{size})$; (iii) state which attribute ID3 picks for the root.
- Why Gini beats misclassification for growing.** Parent node: 40 $+$, 40 $-$. Split A: children (30 $+$ / 10 $-$), (10 $+$ / 30 $-$). Split B: children (20 $+$ / 40 $-$), (20 $+$ / 0 $-$). (i) Compute the size-weighted misclassification error of each split. (ii) Compute the size-weighted Gini index of each split. (iii) Which split does Gini prefer, and in one line why is it the better split? (iv) Connect to the lecture's "more sensitive to the node probabilities".
- Cost-complexity arithmetic.** A grown tree T_0 has $|T_0| = 4$ leaves with $\sum Q_i(T_0) = 10$; pruning one branch gives T with $|T| = 2$ leaves and $\sum Q_i(T) = 14$. (i) Compute $C(T_0)$ and $C(T)$ for $\alpha = 1$ — which wins? (ii) Repeat for $\alpha = 3$. (iii) State the general rule relating α to tree size.

Solutions — Chapter 29: KNN and decision trees

Problem 1 — 3-NN vote, by hand ($q_2 = (4, 2)$)

(i) Squared Euclidean distances (from the §29.5 table):

- $d^2(q_2, A_1) = (4 - 0)^2 + (2 - 0)^2 = 16 + 4 = 20$
- $d^2(q_2, A_2) = (4 - 1)^2 + (2 - 4)^2 = 9 + 4 = 13$
- $d^2(q_2, A_3) = (4 - 3)^2 + (2 - 1)^2 = 1 + 1 = 2$
- $d^2(q_2, B_1) = (4 - 5)^2 + (2 - 2)^2 = 1 + 0 = 1$
- $d^2(q_2, B_2) = (4 - 4)^2 + (2 - 5)^2 = 0 + 9 = 9$
- $d^2(q_2, B_3) = (4 - 6)^2 + (2 - 3)^2 = 4 + 1 = 5$
- $d^2(q_2, B_4) = (4 - 7)^2 + (2 - 2)^2 = 9 + 0 = 9$

(ii) Sorted: $B_1(1) < A_3(2) < B_3(5) < B_2(9) = B_4(9) < A_2(13) < A_1(20)$. The 3 nearest: B_1, A_3, B_3 .

(iii) Votes: $B, A, B \rightarrow 2$ for B , 1 for $A \rightarrow$ predict B .

- (iv) $k = 5$: neighbours B_1, A_3, B_3, B_2, B_4 (the 9-tie is between two B 's, so the set is unaffected) $\rightarrow$ votes $B, A, B, B, B \rightarrow 4$ vs $1 \rightarrow \boxed{\text{predict } B}$.

Problem 2 — The metric can flip the answer

- (i) Euclidean (squared): $d^2(q, a) = 25 + 0 = 25$; $d^2(q, b) = 9 + 9 = 18$. Nearest is $\boxed{b} \rightarrow$ predict $\boxed{-}$.
- (ii) Manhattan: $d_1(q, a) = 5 + 0 = 5$; $d_1(q, b) = 3 + 3 = 6$. Nearest is $\boxed{a} \rightarrow$ predict $\boxed{+}$.
- (iii) Because “nearest” — and hence the vote — depends on how distance is measured; the metric is a modelling choice with observable consequences, i.e. a hyperparameter (§29.2(ii), §29.3(i)).

Problem 3 — KNN regression

- (i) $\hat{y} = \frac{1}{3}(2.0 + 3.5 + 6.5) = \frac{12.0}{3} = \boxed{4.0}$.
- (ii) Classification labels are discrete — they can only be counted (voted); regression outputs are numbers, so they can be averaged — the mean is the central value of the neighbourhood's outputs (§29.4(i)).

Problem 4 — The effect of k

- (i) From §29.5's sorted d^2 for $q = (2, 2)$: $A_3(2), A_2(5), A_1(8), B_1(9), B_2(13), B_3(17), B_4(25)$.
- $k = 1$: $\{A_3\} \rightarrow \boxed{A}$.
 - $k = 3$: $\{A_3, A_2, A_1\} \rightarrow 3 A \rightarrow \boxed{A}$.
 - $k = 5$: $\{A_3, A_2, A_1, B_1, B_2\} \rightarrow 3 A, 2 B \rightarrow \boxed{A}$.
- (ii) $k = 7$: the neighbourhood is all 7 points $\rightarrow 3 A$ vs $4 B \rightarrow \boxed{B}$. This is the lecture's endpoint: as $k \rightarrow n$, the vote is the dataset's majority class (B), so every query — q included — is labelled B (§29.6(iii)).
- (iii) At $k = 1$ the boundary is jagged: each training point owns an intricate Voronoi-like cell, and the border weaves between nearby points. At $k = 5$ the border relaxes into one smooth curve separating the blue cluster from the reds — the local wiggles are voted out (§29.6(i)–(ii); see the §29.5 figure).

Problem 5 — Verify the lecture's Wind number

$$IG(S, \text{Wind}) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v):$$

$$IG = 0.940 - \frac{8}{14}(0.811) - \frac{6}{14}(1.0) = 0.940 - 0.4634 - 0.4286 = 0.0480 \approx \boxed{0.048}.$$

Matches the lecture's 0.048 (full-precision recomputation in the review log gives 0.0481).

Problem 6 — Finish the §29.13 comparison

- (i) Red branch (3 +, 2 −):

$$H = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \approx 0.442 + 0.529 = \boxed{0.971}.$$

Blue branch (1 +, 4 −):

$$H = -\frac{1}{5} \log_2 \frac{1}{5} - \frac{4}{5} \log_2 \frac{4}{5} \approx 0.464 + 0.258 = \boxed{0.722}.$$

- (ii) Weighted child entropy: $\frac{5}{10}(0.971) + \frac{5}{10}(0.722) = 0.8465$.

$$IG(S, \text{colour}) = 0.971 - 0.8465 = \boxed{0.125} < 0.420 = IG(S, \text{size}).$$

- (iii) ID3 splits on the largest information gain $\rightarrow$ root attribute is $\boxed{\text{size}}$.

Problem 7 — Why Gini beats misclassification for growing

Parent: 80 points, 40 +, 40 −.

(i) Misclassification error of a child $= 1 - \max(p_+, p_-) = \frac{\min(n_+, n_-)}{n_{\text{child}}}$.

- Split A: left $\frac{10}{40} = 0.25$, right $\frac{10}{40} = 0.25$; weighted: $\frac{40}{80}(0.25) + \frac{40}{80}(0.25) = \boxed{0.25}$.
- Split B: left $\frac{20}{60} = \frac{1}{3}$, right $\frac{0}{20} = 0$; weighted: $\frac{60}{80} \cdot \frac{1}{3} + \frac{20}{80}(0) = 0.25 + 0 = \boxed{0.25}$. Identical — misclassification cannot distinguish the splits.

(ii) Gini of a child $= 2p_+(1 - p_+)$ (binary case).

- Split A: each child $2 \cdot \frac{3}{4} \cdot \frac{1}{4} = 0.375$; weighted $\boxed{0.375}$.
- Split B: left $2 \cdot \frac{1}{3} \cdot \frac{2}{3} = \frac{4}{9} \approx 0.444$, right 0; weighted: $\frac{60}{80} \cdot \frac{4}{9} = \frac{1}{3} \approx 0.333$.

(iii) Gini prefers $\boxed{\text{split B}}$ ($0.333 < 0.375$). It is the better split because its right child is perfectly pure (20 +, 0 −) — the split has isolated a clean region, while split A leaves both children equally mixed.

(iv) This is the lecture's point verbatim: Gini (and cross-entropy) are “more sensitive to the node probabilities” — they register the purity improvement in B's right child, whereas the misclassification rate only cares about majority flips and scores both splits the same (§29.16(i)).

Problem 8 — Cost-complexity arithmetic

$$C(T) = \sum_i Q_i(T) + \alpha|T|.$$

(i) $\alpha = 1$: $C(T_0) = 10 + 1 \cdot 4 = \boxed{14}$; $C(T) = 14 + 1 \cdot 2 = \boxed{16}$. $\boxed{T_0 \text{ wins}}$ (keep the larger tree).

(ii) $\alpha = 3$: $C(T_0) = 10 + 3 \cdot 4 = \boxed{22}$; $C(T) = 14 + 3 \cdot 2 = \boxed{20}$. $\boxed{T \text{ wins}}$ (prune).

(iii) General rule (the lecture's): $\boxed{\text{larger } \alpha \Rightarrow \text{smaller trees}}$ — α prices each leaf, so raising it makes complex trees too expensive to keep (§29.19).

30. Naive Bayes: generative vs discriminative models

Two roads to the same destination. A **discriminative** classifier models $P(y | x)$ directly — “given this email, how likely is it spam?” A **generative** classifier takes the long way round: it models how each class *generates* its data, $P(x | y)$ and $P(y)$, and then flips the conditioning with Bayes’ rule (§14.11) to get $P(y | x)$. Naive Bayes is the lecture’s flagship generative classifier — the intro slide calls it the “**generative counterpart of logistic regression**” (Chapter 31 is the discriminative half of that pair). It makes one strong, usually-false assumption — features are independent given the class — and works shockingly well anyway, especially on text (document classification, spam filtering).

Everything in §§30.1–30.8 comes from the MLT Naive Bayes slides (Dr. Ashish Tendulkar; the shared materials’ “Week 6” folder — actual Naive Bayes content, week numbering notwithstanding) and the Week 8 TA-notes summary, which adds the generative-vs-discriminative framing, the parameter-counting argument, Laplace smoothing, and the linear-boundary derivation. The worked example in §30.9 is the course notes’ six-email dataset (“Get Your Concepts Right” notes, IITM BS MLT SEP 2022 Week 8), with every number recomputed independently — one of its printed lines has an arithmetic slip, corrected here and flagged in the review log. §30.10’s Gaussian example is the chapter’s own.

Notation. The i -th example is $x^{(i)}$ (superscript, §22’s convention) with label $y^{(i)} \in \{1, \dots, k\}$ for k classes; $x_j^{(i)}$ is feature j of example i . $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$. The binary case ($y \in \{0, 1\}$, $x_j \in \{0, 1\}$ — word present/absent) carries all the examples; the formulas are stated for k classes.

30.1 Two philosophies: model $P(y | x)$, or model $P(x, y)$

Setup. Same as §22.9: instances $x^{(i)} \in \mathbb{R}^d$, discrete labels $y^{(i)}$. The running example (from the TA notes): each $x^{(i)}$ is an email, each feature $x_j^{(i)} \in \{0, 1\}$ says whether word j of a dictionary appears, $y^{(i)} \in \{0, 1\}$ says spam or not. The algorithm outputs $f : \mathbb{R}^d \rightarrow \{1, \dots, k\}$, judged by §22.9’s misclassification fraction.

The joint distribution of a (point, label) pair factors two ways (TA notes):

$$P(x, y) = P(y | x) P(x) = P(x | y) P(y).$$

- i) **Discriminative: model $P(y | x)$ directly.** “We just need $P(y | x)$ for prediction” (TA notes) — so model exactly that and nothing else. Given the email, what is the probability it is spam? No need to model what emails look like in general ($P(x)$ is never estimated). The TA notes’ examples: **decision trees** (§29.9) and **KNN** (§29.2).
- ii) **Generative: model the pair $P(x, y) = P(x | y) P(y)$.** Learn two things: $P(y)$ — how often each label occurs (the **class prior**), and $P(x | y)$ — for each class, what its data looks like (the **class-conditional density**). Then obtain $P(y | x)$ from Bayes’ rule at prediction time (§30.2). The examples: **Naive Bayes** and **Gaussian Naive Bayes**.

Basically, ... “Discriminative = learn the boundary straight: ‘given these features, which class?’ Generative = learn each class’s portrait first — ‘what does a spam email look like, what does a ham email look like, and how common is each?’ — then use Bayes’ rule to pick the class whose portrait matches the new point best.”

30.2 Bayes’ rule for classification

Naive Bayes predicts $P(y | x)$ — the **posterior probability** of the label — using §14.11’s theorem (the slides’ four names, verbatim):

$$P(y | x) = \frac{P(x, y)}{P(x)} = \frac{P(x | y) P(y)}{P(x)},$$

where $P(x | y)$ is the **class-conditional density**, $P(y)$ the **class prior**, and $P(x)$ the **evidence**.

The denominator is expanded by the law of total probability over the k classes (the slides do this via the chain rule):

$$P(x) = \sum_{r=1}^k P(x | y_r) P(y_r).$$

The decision rule. Predict the class with the largest posterior (the slides’ inference rule):

$$\hat{y} = \operatorname{argmax}_{y_c} P(y_c | x; w) = \operatorname{argmax}_{y_c} P(x | y_c; w) P(y_c; w),$$

because $P(x; w)$ is the same for every candidate class and drops out of the argmax. This is the **MAP decision rule** — §20.10 maximized the posterior over *parameters*; here we maximize it over *classes*. §20.12(iv) promised the Bayesian story continues wherever priors appear: this is it — the class priors $P(y_c)$ play exactly the prior’s role, weighting each class by its base rate before the evidence is weighed (§14.11’s “base rates dominate” lesson, now inside a classifier).

Basically, ... “Posterior = (how well this class explains the point) $\times$ (how common this class is), divided by (how common such points are overall). To *decide*, the denominator is shared by all classes, so just compare numerator $\times$ prior per class and take the biggest — that is MAP, with the class as the hypothesis.”

30.3 The Naive Bayes assumption

Def (the naive assumption). Features are **conditionally independent given the label**:

$$P(x_1, x_2, \dots, x_m | y) = P(x_1 | y) P(x_2 | y) \cdots P(x_m | y) = \prod_{j=1}^m P(x_j | y).$$

- i) **Why “naive”.** It is a strong assumption and usually false. In the spam story: the presence/absence of one word is assumed to have no impact on any other word — it “does not model the grammatical, semantic and other structures present in a language” (TA notes). Words obviously do correlate (“Viagra” and “cheap” travel together).
- ii) **Why it is worth it.** Without it, the generative story must estimate a probability for *every* (feature-vector, label) combination: $k \cdot 2^d - 1$ parameters for d binary features ($2^{d+1} - 1$ in the TA notes’ binary case) — exponential in d , “computationally infeasible”. With the assumption, each feature needs only its per-class probability: $2d$ conditional probabilities plus one prior — $2d + 1$ numbers (the TA notes’ count) — linear in d . The assumption buys tractability.
- iii) **Why it works anyway.** Classification only needs the *argmax* to be right, not the probabilities to be accurate — even a crude factorization often ranks the classes correctly. (The slides say only that it is “simple yet very powerful”; the argmax-forgiveness is the standard explanation of the empirical success.)

With the assumption, the posterior becomes (the slides' compact form):

$$P(y = y_c | x) = \frac{p(y_c) \prod_{j=1}^m p(x_j | y_c)}{\sum_{r=1}^k p(y_r) \prod_{j=1}^m p(x_j | y_r)}.$$

Parameters. k prior probabilities (one redundant — they sum to 1) plus $k \times m$ class-conditional densities — “ m conditional densities per class and there are k such classes” (slides). How many numbers each density needs “depends on its mathematical form” (§30.4).

Basically, ... “Full generative modelling tries to learn a probability for every possible email — exponentially many, impossible. Naive Bayes cheats: assume words occur independently given the class, and you only need one number per word per class. The assumption is false, but the classifier only needs to pick the winner, and the cheat usually picks right.”

Naive Bayes generative story: learn one bar per feature per class, then flip with Bayes' rule

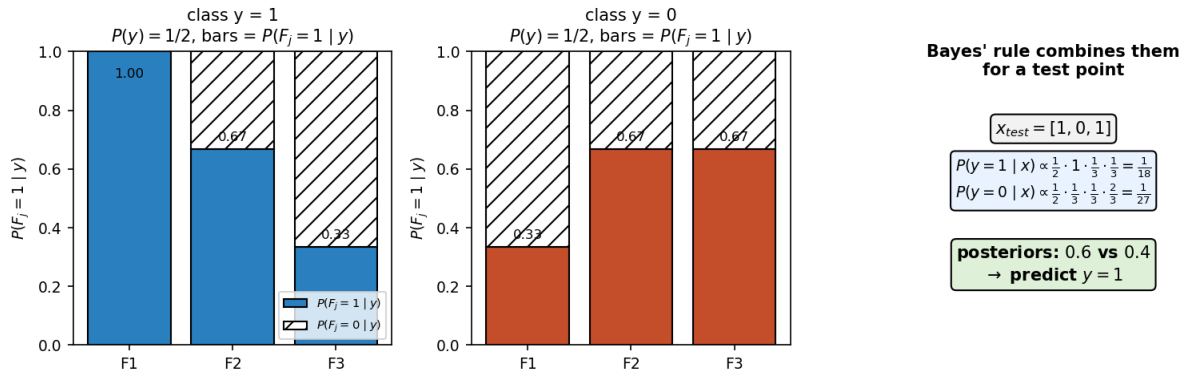


Figure 44: Generative-story schematic: bar charts of $P(F_j = 1 | y)$ for class $y = 1$ (bars 1.00, 0.67, 0.33) and class $y = 0$ (bars 0.33, 0.67, 0.67), each with prior $1/2$; then Bayes' rule combines them for $x_{\text{test}} = [1, 0, 1]$ into scores $1/18$ vs $1/27$, posteriors 0.6 vs 0.4, predicting $y = 1$.

30.4 Modelling $p(x_j | y_c)$: one distribution per feature type

The distribution used for $p(x_j | y_c)$ depends on the feature's nature (the slides' four cases, verbatim):

- i) **Binary feature** (word present/absent) $\rightarrow$ **Bernoulli** (μ_{jc}). $p(x_j = 1 | y_c) = \mu_{jc}$, $p(x_j = 0 | y_c) = 1 - \mu_{jc}$, combined compactly (§15.11's distribution):

$$p(x_j | y_c; \mu_{jc}) = \mu_{jc}^{x_j} (1 - \mu_{jc})^{1-x_j}.$$

Check: $x_j = 1$ gives $\mu_{jc}^1 (1 - \mu_{jc})^0 = \mu_{jc}$; $x_j = 0$ gives $\mu_{jc}^0 (1 - \mu_{jc})^1 = 1 - \mu_{jc}$. For s binary features and k classes: $k \times s$ Bernoulli parameters.

- ii) **Categorical feature** ($e > 2$ values, e.g. {red, green, blue}) $\rightarrow$ **Categorical** $\text{Cat}(\mu_{j1c}, \dots, \mu_{jec})$, $\sum_q \mu_{jqc} = 1$. For $x_j = v_q$: $p(x_j = v_q | y_c) = \mu_{jqc}$; compactly $\prod_q \mu_{jqc}^{\mathbf{1}(x_j=v_q)}$.
- iii) **Counts** (word-count vector, $\sum_j x_j = l$) $\rightarrow$ **Multinomial** ($l, \mu_{1c}, \dots, \mu_{mc}$):

$$p(x | y_c; l, \mu_{1c}, \dots, \mu_{mc}) = \frac{l!}{x_1! \dots x_m!} \prod_{j=1}^m \mu_{jc}^{x_j}, \quad \sum_j x_j = l.$$

Used for documents represented by word counts — the classic spam-filter setup.

iv) **Continuous feature** (e.g. area in sq. feet) $\rightarrow$ **Gaussian** $\mathcal{N}(\mu_{jc}, \sigma_{jc}^2)$:

$$p(x_j | y_c; \mu_{jc}, \sigma_{jc}^2) = \frac{1}{\sqrt{2\pi} \sigma_{jc}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right).$$

Alternately a **multivariate Gaussian** for the whole vector — but under conditional independence every off-diagonal of the covariance is zero, $\Sigma_{jj} = \sigma_j^2$, so it factorizes into the same per-feature Gaussians (slides).

Basically, ... “Pick the distribution that matches the feature’s shape: coin-flip (Bernoulli) for yes/no, die-roll (categorical) for one-of-many, word-counts (multinomial) for bags of words, bell curve (Gaussian) for measurements. One such distribution per feature per class — that is the whole model.”

30.5 Inference: decide in log space

Prediction is the MAP rule from §30.2, expanded with the naive factorization:

$$\hat{y} = \operatorname{argmax}_{y_c} \left(\prod_{j=1}^m p(x_j | y_c; w) \right) p(y_c; w).$$

Products of many small probabilities **underflow** — so compute in log space (the slides’ fix):

$$\hat{y} = \operatorname{argmax}_{y_c} \left[\sum_{j=1}^m \log p(x_j | y_c; w) + \log p(y_c; w) \right].$$

Note (the slides’ warning). The argmax equation gives the *label*, not a probability. If actual probabilities are wanted, use the normalized posterior (§30.3’s fraction) — also computed in log space.

The boundary is linear. For binary y , the log-odds telescopes into a linear function (TA notes):

$$\log \frac{P(y=1 | x)}{P(y=0 | x)} = \sum_{j=1}^d f_j w_j + b, \quad w_j = \log \frac{p_{j1}(1-p_{j0})}{p_{j0}(1-p_{j1})},$$

so Naive Bayes decides by $w^T x + b > 0$ — a **linear decision boundary**, the same shape §22.9 drew. This is the precise content of “generative counterpart of logistic regression”: logistic regression (Chapter 31) learns that linear boundary *discriminatively*, straight from $P(y | x)$; Naive Bayes arrives at the same shape via the generative detour.

Basically, ... “Multiplying hundreds of tiny probabilities underflows to zero, so take logs — products become sums, the argmax is unchanged, and the computer survives. Bonus: in log space the spam/ham decision is just ‘weighted word-score plus bias, check the sign’ — a straight-line boundary, exactly what logistic regression will learn the direct way in Chapter 31.”

30.6 Training = maximizing the log-likelihood

Def (likelihood). The joint probability of the observed data D under parameters w (the slides’ Part 3):

$$L(w) = p(D; w) = \prod_{i=1}^n p(x^{(i)}, y^{(i)}; w),$$

since examples are i.i.d. Take logs for convenience:

$$\ell(w) = \sum_{i=1}^n \log p(x^{(i)}, y^{(i)}; w), \quad J(w) = -\ell(w)$$

— minimize the **negative log-likelihood** (NLL) “to maintain uniformity with other algorithms” (slides).

With the naive factorization, the log-likelihood splits into a prior term and a per-feature term:

$$\ell(w) = \sum_{i=1}^n \log p(y^{(i)}; w) + \sum_{i=1}^n \sum_{j=1}^m \log p(x_j^{(i)} | y^{(i)}; w).$$

Basically, ... “Training = choose the parameters that make the observed (email, label) pairs most probable. Log it, negate it, minimize it — the same NLL ritual every model from here on follows (§20.12(i)).”

30.7 The MLE estimates: training is counting

The slides’ three-step recipe: differentiate $\ell(w)$ w.r.t. each parameter, set to 0 (maxima condition), solve. The solutions are all fractions of counts:

i) **Priors** — class fractions:

$$\hat{p}(y = y_c) = \frac{\sum_{i=1}^n \mathbf{1}(y^{(i)} = y_c)}{n}.$$

ii) **Bernoulli** — fraction of class- y_r examples with feature j equal to 1:

$$\hat{w}_{jyr} = \frac{\sum_{i=1}^n \mathbf{1}(y^{(i)} = y_r) x_j^{(i)}}{\sum_{i=1}^n \mathbf{1}(y^{(i)} = y_r)}.$$

In words: among the training examples labelled y_r , what fraction have $x_j = 1$. (The slides derive this by differentiating the Bernoulli log-likelihood — the standard $\frac{x}{w} - \frac{1-x}{1-w}$ derivative, set to zero.)

iii) **Categorical** — fraction of class- y_r examples with $x_j = v$:

$$\hat{w}_{jvyr} = \frac{\sum_{i=1}^n \mathbf{1}(y^{(i)} = y_r) \mathbf{1}(x_j^{(i)} = v)}{\sum_{i=1}^n \mathbf{1}(y^{(i)} = y_r)}.$$

iv) **Multinomial** — share of feature j ’s total count inside class y_r :

$$\hat{w}_{jyr} = \frac{\sum_{i=1}^n \mathbf{1}(y^{(i)} = y_r) x_j^{(i)}}{\sum_{i=1}^n \mathbf{1}(y^{(i)} = y_r) \sum_{j=1}^m x_j^{(i)}}.$$

v) **Gaussian** — class-conditional mean and variance (n_r = number of class- y_r examples; the MLE divides by n_r):

$$\hat{\mu}_{jr} = \frac{1}{n_r} \sum_{i=1}^n \mathbf{1}(y^{(i)} = y_r) x_j^{(i)}, \quad \hat{\sigma}_{jr}^2 = \frac{1}{n_r} \sum_{i=1}^n \mathbf{1}(y^{(i)} = y_r) (x_j^{(i)} - \hat{\mu}_{jr})^2.$$

Basically, ... “No gradients, no iterations: training Naive Bayes is counting. Priors = class fractions; each conditional = the fraction of that class’s examples showing the feature value. The calculus just confirms what the fractions already say.”

30.8 The zero-count problem and Laplace smoothing

The problem. If no training example of class y_r has $x_j = 1$, the MLE says $\hat{w}_{jyr} = 0$ — so $p(x_j | y_r) = 0$, and the *entire* posterior $P(y_r | x)$ collapses to 0 for any test point with $x_j = 1$. One unseen combination vetoes the whole class. The TA notes’ version: both posteriors can become 0 and “prediction becomes difficult”.

This is §20.4's **zero-count warning** relocated into a classifier — MLE's overconfidence again — and §20.11's Beta prior is its conceptual cousin (a prior “keeps the estimate sane”).

The fix: smoothing. Pretend we saw a few extra pseudo-examples. The slides' **Laplace smoothing** adds $+c$ to the numerator and the matching pseudo-total to the denominator (binary case, $x_j \in \{0, 1\}$ — one pseudo-count per value):

$$\hat{w}_{jyr} = \frac{\sum_{i=1}^n \mathbf{1}(y^{(i)} = y_r) x_j^{(i)} + c}{\sum_{i=1}^n \mathbf{1}(y^{(i)} = y_r) + 2c},$$

with $+ce$ for categorical (e values) and $+cm$ for multinomial (m words). $c = 1$ is **Laplace smoothing** proper; c is a hyperparameter — “too high a value of c leads to underfitting” (slides), i.e. the pseudo-data drowns the real data.

Basically, ... “MLE says ‘never seen it, impossible’ — one zero poisons the whole product. Smoothing pads every count with a little fake data ($c = 1$: pretend you saw each outcome once), so no probability is ever exactly zero. Too much padding and the fake data runs the show — that is underfitting.”

30.9 Worked example: six emails, three words (every number re-computed)

The course notes' dataset (“Get Your Concepts Right”, MLT SEP 2022 Week 8). Three binary features F_1, F_2, F_3 (word present = 1), label $y = 1$ (spam) / 0:

example	F_1	F_2	F_3	y
$x^{(1)}$	0	1	0	0
$x^{(2)}$	1	1	0	1
$x^{(3)}$	0	1	1	0
$x^{(4)}$	1	0	1	0
$x^{(5)}$	1	0	0	1
$x^{(6)}$	1	1	1	1

Step 1 — priors (§30.7(i)). Three of six labels are 1: $\hat{p}(y=1) = 3/6 = 1/2$, $\hat{p}(y=0) = 1/2$.

Step 2 — conditionals (§30.7(ii)). Class 1 = examples 2, 5, 6; class 0 = examples 1, 3, 4:

	$\hat{p}(F_j=1 \mid y=1)$	$\hat{p}(F_j=1 \mid y=0)$
F_1	$3/3 = 1$	$1/3$
F_2	$2/3$	$2/3$
F_3	$1/3$	$2/3$

(F_1 for class 1: examples 2, 5, 6 all have $F_1 = 1$; F_2 for class 0: examples 1, 3 have 1, example 4 has 0 — $2/3$; etc.)

Step 3 — classify $x_{\text{test}} = [1, 0, 1]$. For each feature take $\hat{p}_j$ if the value is 1, $(1 - \hat{p}_j)$ if 0 (the Bernoulli compact form, §30.4(i)):

$$P(x_{\text{test}} \mid y=1) = \underbrace{\frac{1}{3}}_{F_1=1} \cdot \underbrace{\left(1 - \frac{2}{3}\right)}_{F_2=0} \cdot \underbrace{\frac{1}{3}}_{F_3=1} = 1 \cdot \frac{1}{3} \cdot \frac{1}{3} = \boxed{\frac{1}{9} \approx 0.111},$$

$$P(x_{\text{test}} \mid y=0) = \underbrace{\frac{1}{3}}_{F_1=1} \cdot \underbrace{\left(1 - \frac{2}{3}\right)}_{F_2=0} \cdot \underbrace{\frac{2}{3}}_{F_3=1} = \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{2}{3} = \boxed{\frac{2}{27} \approx 0.074}.$$

Step 4 — posteriors. Unnormalized scores ($\times$ prior $1/2$): $y=1$: $1/18 \approx 0.0556$; $y=0$: $1/27 \approx 0.0370$. Normalizing (evidence = $1/18 + 1/27 = 5/54$):

$$P(y=1 \mid x_{\text{test}}) = \frac{1/18}{5/54} = \frac{3}{5} = 0.6, \quad P(y=0 \mid x_{\text{test}}) = \frac{2}{5} = 0.4.$$

Predict $y = 1$.

Note (source slip, corrected). The notes print the $y=1$ likelihood as “ $1.00 \times 0.33 \times 0.67 = 0.22$ ” — the third factor should be $\hat{p}_3^1 = 0.33$ ($F_3 = 1$), not $1 - \hat{p}_3^1 = 0.67$. Correct: $1.00 \times 0.33 \times 0.33 \approx 0.11 = 1/9$. The verdict (predict 1) is unchanged, since $1/9 > 2/27$ either way. Flagged in the review log.

Step 5 — smoothing demo. Note $\hat{p}(F_1=1 \mid y=1) = 1$, i.e. $\hat{p}(F_1=0 \mid y=1) = 0$: any test point with $F_1 = 0$ gets $P(x \mid y=1) = 0$ — §30.8’s pathology, live. With Laplace $c = 1$: $\hat{p}_1^1 = 4/5 = 0.8$, $\hat{p}_2^1 = 3/5 = 0.6$, $\hat{p}_3^1 = 2/5 = 0.4$; $\hat{p}_1^0 = 2/5 = 0.4$, $\hat{p}_2^0 = \hat{p}_3^0 = 3/5 = 0.6$. Then $P(x_{\text{test}} \mid y=1) = 0.8 \cdot 0.4 \cdot 0.4 = 0.128$, $P(x_{\text{test}} \mid y=0) = 0.4 \cdot 0.4 \cdot 0.6 = 0.096$ — still predict 1, but no zero vetoes remain.

Basically, ... “Count: half the emails are spam; among spam, word 1 always appears, word 2 appears two-thirds of the time, word 3 one-third. New email $[1, 0, 1]$: multiply the matching numbers per class, weight by the half-half prior, normalize — spam wins 0.6 to 0.4. And the 1.00 for word-1-in-spam is a warning: one zero anywhere in the product kills the class, which is why smoothing exists.”

30.10 Worked example: Gaussian NB on one continuous feature

Class 0: $x \in \{1, 2, 3\}$; class 1: $x \in \{4, 5, 6\}$; priors $1/2$ each. By §30.7(v):

$$\hat{\mu}_0 = 2, \quad \hat{\mu}_1 = 5, \quad \hat{\sigma}_0^2 = \hat{\sigma}_1^2 = \frac{(1-2)^2 + 0 + (3-2)^2}{3} = \frac{2}{3}.$$

Decision boundary. Equal priors and equal variances: $p(x \mid 1) = p(x \mid 0)$ iff the exponents match,

$$-\frac{(x-5)^2}{2\sigma^2} = -\frac{(x-2)^2}{2\sigma^2} \iff (x-5)^2 = (x-2)^2 \iff \boxed{x = 3.5},$$

the midpoint of the means — a (one-point) linear boundary, as §30.5 promised.

Classify. $x = 4$: $(4-5)^2 = 1 < (4-2)^2 = 4 \rightarrow$ nearer class 1’s centre $\rightarrow$ predict 1. $x = 3$: $(3-5)^2 = 4 > (3-2)^2 = 1 \rightarrow$ predict 0.

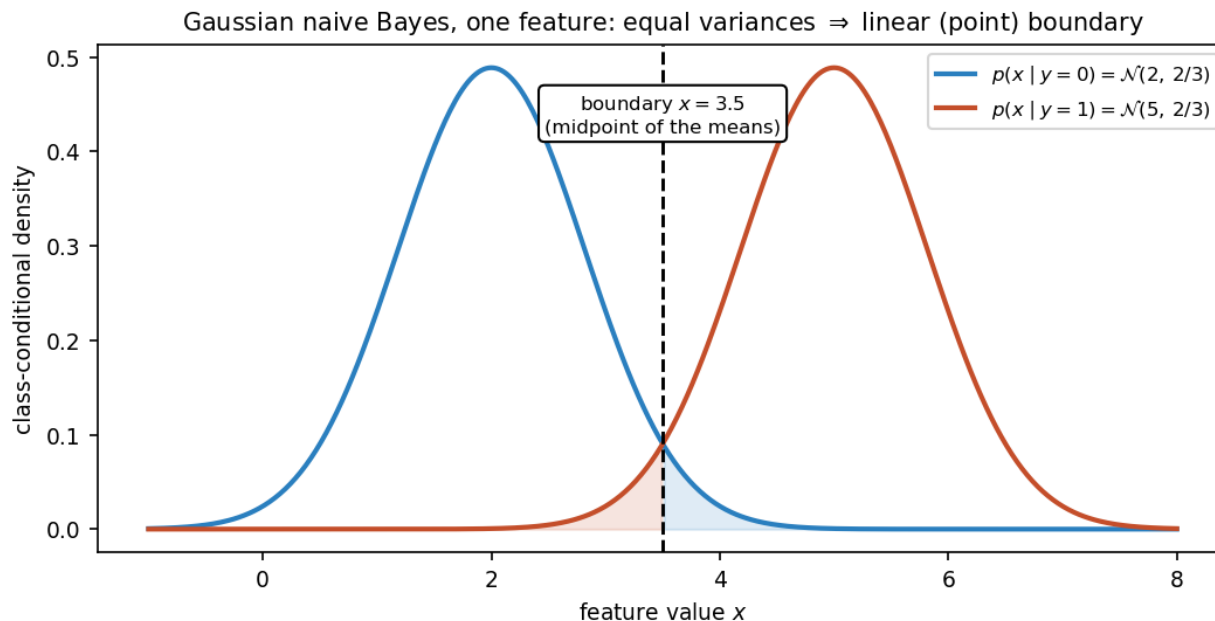


Figure 45: Two Gaussian curves: class 0 centred at 2 and class 1 centred at 5, both variance $2/3$; a dashed vertical line at $x = 3.5$ marks the decision boundary, the midpoint of the means.

Basically, ... “Per class, fit a bell curve (mean and variance of that class’s numbers). New point: whichever bell is taller there wins. Same spread on both bells $\rightarrow$ the boundary is exactly halfway between the humps.”

30.11 Strengths and limits (as the sources list them)

Strengths. i) **Simple and fast** — training is counting (§30.7); prediction is a product of table lookups. ii) **Few parameters** — $2d + 1$ instead of $2^{d+1} - 1$ (§30.3(ii)); works on small data. iii) **Surprisingly strong on text** — “simple yet very powerful ... used extensively in applications like document classification and spam filtering” (intro slide). iv) **Mixed feature types** — binary, categorical, counts, continuous each get their own distribution (§30.4).

Limits. i) **The independence assumption is usually false** — words do correlate; the model “does not model the grammatical, semantic and other structures present in a language” (TA notes). It cannot learn feature interactions at all. ii) **Zero counts veto classes** — needs Laplace smoothing (§30.8), whose strength c is another hyperparameter to tune. iii) **Gaussian NB assumes normality** — skewed or multimodal features per class break the bell-curve model.

Evaluation (the slides’ closing list, brief): confusion matrix, precision/recall/F1, AUC ROC/PR curve, with cross-validation and a held-out test set — §22.10’s discipline. Classification losses get their own chapter (Chapter 35).

Basically, ... “Naive Bayes: cheap to train, few knobs, great first try on text. The bill: it pretends features don’t interact (often false), one unseen combo can zero a class (smooth it), and the Gaussian version believes every class is bell-shaped.”

30.12 Where this goes next

- i) **The discriminative twin.** Chapter 31 (logistic regression) is the intro slide’s “generative counterpart” in reverse: same linear boundary $w^T x + b$, learned by modelling $P(y | x)$ directly instead of the

$P(x | y)P(y)$ detour. Comparing the two is the cleanest way to feel what “generative vs discriminative” buys and costs.

- ii) **The Part IV toolbox continues** (§25.12(iv)’s list): SVMs (Chapters 32–33), ensembles (Chapter 34) — heavier machinery for when the naive assumption costs too much accuracy.

Problem set

- Classify $[0, 0, 1]$ on §30.9’s data.** Using the §30.9 estimates (no smoothing): (i) compute $P(x | y=1)$ and $P(x | y=0)$; (ii) give the unnormalized scores and the posteriors; (iii) state the prediction; (iv) point to the exact factor that makes one likelihood 0 and name the §30.8 concept it illustrates.
- Parameter counting.** $k = 4$ classes, $d = 5$ binary features. (i) How many parameters does the full generative story need (TA notes’ $k2^d - 1$ form)? (ii) How many does Naive Bayes need, counting priors straight (TA-notes style)? (iii) One of those priors is redundant — why, and what is the exact free-parameter count? (iv) In one line each: how do the two totals scale with d ?
- Laplace smoothing in action.** On §30.9’s data, test point $x = [0, 1, 1]$: (i) show the unsmoothed $P(x | y=1) = 0$ and give the (forced) prediction; (ii) recompute all six conditionals with $c = 1$; (iii) recompute both likelihoods, the posteriors, and the prediction.
- Spam posterior (practice-assignment numbers).** $P(\text{spam}) = 0.2$; word likelihoods — spam: Hurray! 0.7, win 0.2, exciting 0.01, prizes 0.3; non-spam: 0.01, 0.02, 0.01, 0.1. Mail: “Hurray! win exciting prizes” (these four words are the only features). Compute $P(\text{spam} | \text{mail})$ to two decimal places.
- Gaussian NB boundary.** Class 0: $\{1, 2, 3\}$; class 1: $\{5, 6, 7\}$; equal priors. (i) Estimate $\hat{\mu}_0, \hat{\mu}_1, \hat{\sigma}_0^2, \hat{\sigma}_1^2$. (ii) Find the decision boundary. (iii) Classify $x = 3$ and $x = 5$.
- Generative or discriminative?** From the TA notes’ classification: for each of {decision trees, KNN, Naive Bayes, Gaussian Naive Bayes}, state whether it models $P(y | x)$ or $P(x, y)$, with a one-line justification.
- Why log space.** (i) Prove the §30.5 claim: $\operatorname{argmax}_{y_c} P(y_c | x)$ equals $\operatorname{argmax}_{y_c} [\sum_j \log p(x_j | y_c) + \log p(y_c)]$. (ii) In one line: why is the log version used in practice?

Solutions — Chapter 30: Naive Bayes: generative vs discriminative models

Problem 1 — Classify $[0, 0, 1]$ (§30.9’s data, no smoothing)

Estimates from §30.9: $\hat{p}(y=1) = \hat{p}(y=0) = 1/2$; $\hat{p}^1 = (1, 2/3, 1/3)$; $\hat{p}^0 = (1/3, 2/3, 2/3)$.

- (i) For $x = [0, 0, 1]$, take $(1 - \hat{p}_j)$ for the 0-valued features and $\hat{p}_j$ for the 1-valued one:

$$P(x | y=1) = \underbrace{(1-1)}_{F_1=0} \cdot \underbrace{(1-\frac{2}{3})}_{F_2=0} \cdot \underbrace{\frac{1}{3}}_{F_3=1} = 0 \cdot \frac{1}{3} \cdot \frac{1}{3} = \boxed{0},$$

$$P(x | y=0) = \underbrace{(1-\frac{1}{3})}_{F_1=0} \cdot \underbrace{(1-\frac{2}{3})}_{F_2=0} \cdot \underbrace{\frac{2}{3}}_{F_3=1} = \frac{2}{3} \cdot \frac{1}{3} \cdot \frac{2}{3} = \boxed{\frac{4}{27} \approx 0.148}.$$

- (ii) Unnormalized scores ($\times$ prior $1/2$): $y=1$: 0; $y=0$: $\frac{1}{2} \cdot \frac{4}{27} = \frac{2}{27} \approx 0.074$. Evidence = $2/27$. Posteriors:

$$\boxed{P(y=1 | x) = 0}, \boxed{P(y=0 | x) = 1}.$$

- (iii) $\boxed{\text{Predict } y = 0.}$

- (iv) The zero factor is $(1 - \hat{p}_1^1) = 1 - 1 = 0$: $F_1 = 0$ never occurs with class 1 in training, so the MLE declares it impossible and the whole product dies. This is §30.8’s **zero-count problem** — one unseen (feature, class) combination vetoes the class entirely, which is why Laplace smoothing exists.

Problem 2 — Parameter counting ($k = 4$, $d = 5$ binary features)

- (i) The full generative story needs one probability per (feature-vector, label) pair: $k \cdot 2^d - 1 = 4 \cdot 2^5 - 1 = 4 \cdot 32 - 1 = \boxed{127}$.
- (ii) Naive Bayes (TA-notes style, priors counted straight): k priors $+k \cdot d$ conditionals $= 4 + 4 \cdot 5 = \boxed{24}$.
- (iii) The k priors sum to 1, so one is redundant: exact free parameters $= (k - 1) + kd = 3 + 20 = \boxed{23}$. (Each Bernoulli conditional is one free number — $\hat{p}$ determines $1 - \hat{p}$ — so no further redundancy.)
- (iv) Full generative: $\boxed{\text{exponential in } d}$ — doubles with each added feature. Naive Bayes: $\boxed{\text{linear in } d}$ — k more numbers per feature. That exponential-to-linear collapse is what the independence assumption buys (§30.3(ii)).

Problem 3 — Laplace smoothing on $x = [0, 1, 1]$

- (i) Unsmoothed, from §30.9's estimates:

$$P(x \mid y=1) = \underbrace{(1-1)}_{F_1=0} \cdot \underbrace{\frac{2}{3}}_{F_2=1} \cdot \underbrace{\frac{1}{3}}_{F_3=1} = \boxed{0}.$$

The $y=1$ score is 0 no matter what the other features say; $P(x \mid y=0) = \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{2}{3} = 8/27$, so the prediction is forced to $\boxed{0}$ (posteriors 0 and 1).

- (ii) With $c = 1$ (§30.8): class 1 ($n = 3$): $\hat{p}_1^1 = \frac{3+1}{3+2} = \frac{4}{5} = 0.8$, $\hat{p}_2^1 = \frac{2+1}{5} = \frac{3}{5} = 0.6$, $\hat{p}_3^1 = \frac{1+1}{5} = \frac{2}{5} = 0.4$.
Class 0 ($n = 3$): $\hat{p}_1^0 = \frac{1+1}{5} = \frac{2}{5} = 0.4$, $\hat{p}_2^0 = \frac{2+1}{5} = \frac{3}{5} = 0.6$, $\hat{p}_3^0 = \frac{2+1}{5} = \frac{3}{5} = 0.6$.
- (iii) Smoothed likelihoods:

$$P(x \mid y=1) = 0.2 \cdot 0.6 \cdot 0.4 = \boxed{0.048}, \quad P(x \mid y=0) = 0.6 \cdot 0.6 \cdot 0.6 = \boxed{0.216}.$$

Scores ($\times$ prior $1/2$): 0.024 vs 0.108; evidence = 0.132. Posteriors: $\boxed{P(y=1 \mid x) = 0.024/0.132 \approx 0.182}$,
 $\boxed{P(y=0 \mid x) = 0.108/0.132 \approx 0.818}$. $\boxed{\text{Predict 0}}$ — same verdict as unsmoothed here, but now class 1 keeps a nonzero 18% instead of being vetoed outright.

Problem 4 — Spam posterior (practice-assignment numbers)

Numerator (spam, prior 0.2):

$$0.2 \times 0.7 \times 0.2 \times 0.01 \times 0.3 = 0.2 \times 0.00042 = 0.000084.$$

Non-spam term (prior 0.8):

$$0.8 \times 0.01 \times 0.02 \times 0.01 \times 0.1 = 0.8 \times 2 \times 10^{-7} = 1.6 \times 10^{-7}.$$

Evidence = 0.000084 + 0.00000016 = 0.00008416.

$$P(\text{spam} \mid \text{mail}) = \frac{0.000084}{0.00008416} \approx 0.99810 = \boxed{1.00} \text{ (two decimals).}$$

Every one of the four “spammy” words multiplies the spam case up and the ham case down; the 0.2 prior is swamped by the likelihood ratio.

Problem 5 — Gaussian NB boundary

- (i) Class 0: $\hat{\mu}_0 = \frac{1+2+3}{3} = \boxed{2}$; $\hat{\sigma}_0^2 = \frac{(1-2)^2+0+(3-2)^2}{3} = \boxed{\frac{2}{3}}$. Class 1: $\hat{\mu}_1 = \frac{5+6+7}{3} = \boxed{6}$; $\hat{\sigma}_1^2 = \frac{(5-6)^2+0+(7-6)^2}{3} = \boxed{\frac{2}{3}}$.

- (ii) Equal priors and equal variances: $p(x | 1) = p(x | 0) \iff (x - 6)^2 = (x - 2)^2$. Expanding: $x^2 - 12x + 36 = x^2 - 4x + 4$, so $-12x + 36 = -4x + 4$, $-8x = -32$, $x = 4$ — again the midpoint of the means.
- (iii) $x = 3$: $(3 - 6)^2 = 9 > (3 - 2)^2 = 1 \rightarrow$ class 0's density is taller $\rightarrow$ predict 0. $x = 5$: $(5 - 6)^2 = 1 < (5 - 2)^2 = 9 \rightarrow$ predict 1.

Problem 6 — Generative or discriminative?

- **Decision trees — discriminative:** model $P(y | x)$ directly (TA notes: $P(y=1 | x) = 1$ if x 's leaf is 1).
- **KNN — discriminative:** the majority vote of the neighbours estimates $P(y | x)$ straight from the data, no model of $P(x, y)$.
- **Naive Bayes — generative:** models the pair $P(x, y) = P(x | y)P(y)$ — class priors plus per-class feature distributions — then flips with Bayes' rule.
- **Gaussian Naive Bayes — generative:** same as NB, with Gaussian class-conditional densities.

Problem 7 — Why log space

- (i) $P(y_c | x) = \frac{P(x | y_c)P(y_c)}{P(x)}$. For the argmax over y_c , the denominator $P(x)$ is constant, so

$$\operatorname{argmax}_{y_c} P(y_c | x) = \operatorname{argmax}_{y_c} [P(x | y_c)P(y_c)].$$

With the naive factorization $P(x | y_c) = \prod_j p(x_j | y_c)$ and log strictly increasing (so it preserves argmax):

$$= \operatorname{argmax}_{y_c} \left[\log \left(\prod_j p(x_j | y_c) P(y_c) \right) \right] = \operatorname{argmax}_{y_c} \left[\sum_{j=1}^m \log p(x_j | y_c) + \log p(y_c) \right]. \quad \blacksquare$$

- (ii) Because a product of hundreds of small probabilities underflows to floating-point zero, while a sum of logs stays in a safe range — same argmax, no underflow.

31. Perceptron and logistic regression

Two linear classifiers, two opposite temperaments. The **perceptron** learns by making mistakes: it guesses, gets corrected, and nudges its weights only when it is wrong. **Logistic regression** never guesses at all — it models the probability $P(y | x)$ directly and tunes its weights to make the observed labels as probable as possible. Both draw the same shape in the end — the linear boundary $w^T x + b = 0$ (§22.9, §30.5) — but they arrive by completely different roads.

This is the chapter §30 promised. Naive Bayes was the intro slide’s “generative counterpart of logistic regression” — the model that reaches the linear boundary via the $P(x | y)P(y)$ detour. Logistic regression is the **discriminative half of that pair**: it skips the detour and learns $P(y | x)$ straight from the data. The perceptron comes first because it is the older, simpler machine (Rosenblatt, 1958) and the direct ancestor of the neural networks in Part VI.

Notation. Two label conventions appear because the two algorithms come from two sources: the perceptron slides use $y^{(i)} \in \{-1, +1\}$; logistic regression (the 6.036 notes in the course materials) uses $y^{(i)} \in \{0, 1\}$, “enabling them to be interpreted as probabilities of being a member of the class of interest”. Translate freely: $+1 \leftrightarrow 1$, $-1 \leftrightarrow 0$. The i -th example is $x^{(i)}$, its label $y^{(i)}$, $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ (§22.9). **Note:** this parenthesized $x^{(i)}$ replaces Part III’s x^i — same meaning; the parens remove the power-collision §22.6 warned about. The $x^{(i)}$ form is used from here on (Chapters 32–36 and beyond).

Everything in §§31.2–31.6 comes from the MLT Perceptron slides (Dr. Ashish Tendulkar; the shared materials’ “Week 4” folder — actual perceptron content, week numbering notwithstanding) and the Week 9 practice assignment. §§31.7–31.12 come from the MITx 6.036 machine-learning notes kept in the MLT “Notes” folder (the logistic-classification and gradient-descent chapters). The linear-boundary geometry (§31.1) is from the “Models of Classification” slides; the generative-vs-discriminative framing (§31.13) leans on §30.1–30.2’s TA-notes account.

31.1 The shared shape: a linear boundary, and what its parts mean

Both algorithms in this chapter learn a **linear discriminant function** — “very similar to the linear regression” (Models of Classification slides):

$$y = w_0 + w_1 x_1 + \cdots + w_m x_m = w_0 + w^T x.$$

The decision rule: predict class 1 if $w_0 + w^T x > 0$, class 0 otherwise. The **decision boundary** is

$$w_0 + w^T x = 0$$

— a hyperplane, one dimension below the feature space (a line in 2-D, a plane in 3-D).

The slides give each part a geometric job:

- i) w = **orientation**. Take two points $x^{(A)}, x^{(B)}$ on the boundary: $w^T x^{(A)} = w^T x^{(B)} = -w_0$, so $w^T(x^{(A)} - x^{(B)}) = 0$. w is orthogonal to every vector lying in the decision surface — it fixes which way the boundary faces.

- ii) $w_0 = \text{location}$. Normalizing by $\|w\|$, the normal distance from the origin to the surface is $-w_0/\|w\|$ — w_0 slides the boundary toward or away from the origin.
- iii) **The value** $w_0 + w^T x = \text{signed distance}$. The discriminant gives a signed measure of the perpendicular distance of x from the surface — positive on one side, negative on the other.

The bias trick. By adding a dummy feature $x_0 = 1$, the bias folds into the weight vector: $w_0 x_0 + w^T x = w^T x$ (augmented). The boundary becomes a hyperplane through the origin in the one-dimension-higher space. §31.6's worked example uses this trick — watch for the leading 1 in every feature vector.

Basically, ... “A linear classifier is a flat wall: w decides which way the wall faces, w_0 decides where it stands, and the sign of $w_0 + w^T x$ tells you which side of the wall a point is on. Everything in this chapter is about where to put that wall.”

31.2 The perceptron: a neuron that learns from mistakes

Def (perceptron). A binary classification algorithm — the slides stress it “can solve only binary classification problems”, so $y^{(i)} \in \{-1, +1\}$. Invented in 1958 by Frank Rosenblatt, “intended to be a machine, rather than a program”, and “meant to be a rough model of how individual neurons work in the brain”.

The **model** (the slides' form):

$$\hat{y} = h_w(x) = f(w^T \phi(x)) = f(z),$$

- i) $\phi(x)$ — a feature transformation of the input (use $\phi(x) = (1, x)$ to smuggle in the bias, §31.1).
- ii) $w^T \phi(x)$ — the linear combination of (transformed) features.
- iii) f — a non-linear activation: the **sign** (threshold) function,

$$f(z) = \begin{cases} +1, & z \geq 0, \\ -1, & z < 0, \end{cases}$$

i.e. $\hat{y} = \text{sign}(w^T \phi(x))$.

So the perceptron is the linear discriminant of §31.1 with its continuous output snapped to ± 1 by a hard threshold. Its decision boundary is $w^T \phi(x) = 0$ — the same $w_0 + w^T x = 0$ shape.

Basically, ... “Take the linear score $w^T x$, then slam it through a sign function: non-negative means class $+1$, negative means class -1 . It is a neuron in the roughest sense — weighted sum in, yes/no out. And it only does two classes, ever.”

31.3 The perceptron loss: pay only for mistakes

Let $\hat{y}^{(i)}$ be the perceptron's prediction and $y^{(i)}$ the true label. The slides define the per-example error:

$$e^{(i)} = \begin{cases} 0, & \hat{y}^{(i)} = y^{(i)}, \\ -w^T \phi(x^{(i)}) y^{(i)}, & \text{otherwise,} \end{cases}$$

compactly,

$$e^{(i)} = \max(0, -w^T \phi(x^{(i)}) y^{(i)}) = \max(0, -\hat{y}^{(i)} y^{(i)}).$$

Why this shape: if the prediction is correct, $\hat{y}^{(i)} y^{(i)} = +1$ and the error is $\max(0, -1) = 0$. If wrong, $\hat{y}^{(i)} y^{(i)} = -1$ and the error is $\max(0, 1) = 1$ (the slides' illustration works both cases out to exactly 0 and 1). The total loss:

$$J(w) = \sum_{i=1}^n e^{(i)} = \sum_{i=1}^n \max(0, -w^T \phi(x^{(i)}) y^{(i)}).$$

Note (the slides' warning). This error is **piecewise linear**: zero in the correctly-classified region, a linear function of w in the misclassified region — and $J(w)$ is **not differentiable** in w . So gradient descent (Chapter 10) cannot be applied directly; the perceptron needs its own optimization procedure (§31.4).

Basically, ... “Right answer: pay nothing. Wrong answer: pay 1 — the bill is all-or-nothing per point. As a function of the weights it has a kink at the boundary — no smooth slope to descend — so the perceptron optimizes by a different trick: fix mistakes one at a time.”

31.4 The perceptron update rule: learn only when wrong

Def (perceptron update rule). Initialize $w^{(0)} = 0$. For each training example $(x^{(i)}, y^{(i)})$:

$$\hat{y}^{(i)} = \text{sign}(w^T \phi(x^{(i)})), \quad w^{(t+1)} := w^{(t)} + \alpha (y^{(i)} - \hat{y}^{(i)}) \phi(x^{(i)}),$$

α the learning rate. The three cases (the slides work each one out):

- i) **Correct:** $y^{(i)} = \hat{y}^{(i)} \rightarrow (y^{(i)} - \hat{y}^{(i)}) = 0 \rightarrow w^{(t+1)} = w^{(t)}$. Nothing happens.
- ii) **True -1, predicted +1:** $w^{(t+1)} = w^{(t)} - 2\alpha \phi(x^{(i)})$ — push the weights *away* from this point.
- iii) **True +1, predicted -1:** $w^{(t+1)} = w^{(t)} + 2\alpha \phi(x^{(i)})$ — pull the weights *toward* this point.

This is **mistake-driven learning** in its purest form: correctly classified examples leave no trace on the weights; only mistakes move them. (The slides' own summary of the loss: “for misclassified example, we can reduce loss by reducing w . And for correctly classified examples, we leave w unchanged.”)

Basically, ... “Guess a label for each point. Right? Move on, change nothing. Wrong? Shove the weights toward the point if it should have been +1, away if it should have been -1. The algorithm has no memory of its successes — only its mistakes teach it.”

31.5 Convergence: guaranteed on separable data, hopeless otherwise

The slides state the convergence behaviour with unusual bluntness:

- i) **Linearly separable data $\rightarrow$ convergence.** “Linear separable examples lead to convergence of the algorithm with zero training loss” — the update loop eventually finds weights that classify every training point correctly and then stops (correct points cause no updates).
- ii) **Non-separable data $\rightarrow$ oscillation, forever.** “Else it oscillates.” The Week 9 practice assignment makes this concrete (Q2): on three points no line can separate, the perceptron cycles through the same weight vectors “and will never converge”.

The mistake bound (practice assignment, Q3–Q4). On separable data the slides' guarantee can be quantified. Let R be the maximum length of the data points and γ the margin — the smallest value of $y^{(i)}(u^T x^{(i)})$ over the data, for a unit-norm separator u . Then the perceptron makes only finitely many mistakes — at most

$$\# \text{mistakes} \leq \left(\frac{R}{\gamma} \right)^2.$$

The assignment's numbers: $R = 4$, $\gamma = 1 \rightarrow$ at most $(4/1)^2 = 16$ mistakes (answer: 16). And a single update can grow the squared weight length by at most R^2 : with $\|w\|^2 = 36$ and $R = 4$, the next iteration's squared length is at most $36 + 16 = 52$ — so 45 is a valid value, 55 is not (Q3's answer).

Note (scope of the claim). The sources give the *statement* — separable implies convergence with zero training loss, plus the $(R/\gamma)^2$ bound as the assignment uses it — not a full proof. Nothing here promises anything about test data (§22.10's warning stands) or about *which* separator is found when many exist.

Basically, ... “If a straight wall *can* separate the classes, the perceptron *will* find one — after at most $(R/\gamma)^2$ mistakes, and then it stops dead, training loss zero. If no such wall exists, it never stops: it just keeps shuffling the same weights around forever. Check separability first (practice Q1/Q8 are exactly this skill).”

31.6 Worked example: three points, two updates (every number recomputed)

Data (augmented $\phi(x) = (1, x_1, x_2)$, bias folded in; labels in $\{-1, +1\}$):

example	$\phi(x^{(i)})$	$y^{(i)}$
m_1	$(1, -2, 1)$	-1
m_2	$(1, 1, -2)$	-1
p_1	$(1, 2, 2)$	$+1$

(Separable: $x_1 + x_2 = 0$ already separates them, but the perceptron must discover a wall on its own.) $\alpha = 1$, $w^{(0)} = (0, 0, 0)$. Cycle m_1, m_2, p_1 repeatedly. Recall $f(z) = +1$ for $z \geq 0$ — so a zero score predicts $+1$.

Sweep 1. - m_1 : $z = 0 \rightarrow \hat{y} = +1 \neq -1$. Mistake (case ii): $w := (0, 0, 0) + 1 \cdot (-1 - 1) \cdot (1, -2, 1) = -2(1, -2, 1) = \boxed{(-2, 4, -2)}$. - m_2 : $z = (-2)(1) + 4(1) + (-2)(-2) = -2 + 4 + 4 = 6 \geq 0 \rightarrow \hat{y} = +1 \neq -1$. Mistake: $w := (-2, 4, -2) + (-2)(1, 1, -2) = \boxed{(-4, 2, 2)}$. - p_1 : $z = (-4)(1) + 2(2) + 2(2) = -4 + 4 + 4 = 4 \geq 0 \rightarrow \hat{y} = +1 = y$. Correct — no change.

Sweep 2 (verify; $w = (-4, 2, 2)$). - m_1 : $z = -4(1) + 2(-2) + 2(1) = -4 - 4 + 2 = -6 < 0 \rightarrow -1$. - m_2 : $z = -4(1) + 2(1) + 2(-2) = -4 + 2 - 4 = -6 < 0 \rightarrow -1$. - p_1 : $z = 4 \geq 0 \rightarrow +1$. All correct — no updates fire, training loss is 0, the algorithm halts. (Every score above was recomputed in numpy independently — see the review log; an early hand pass scored m_1 's second-sweep z as -10 , the recompute corrected it to -6 .)

The learned boundary. $-4 + 2x_1 + 2x_2 = 0$, i.e. $\boxed{x_1 + x_2 = 2}$ — the same $w_0 + w^T x = 0$ shape from §31.1, with $w = (2, 2)$ fixing the orientation (orthogonal to the line) and $w_0 = -4$ fixing the location.

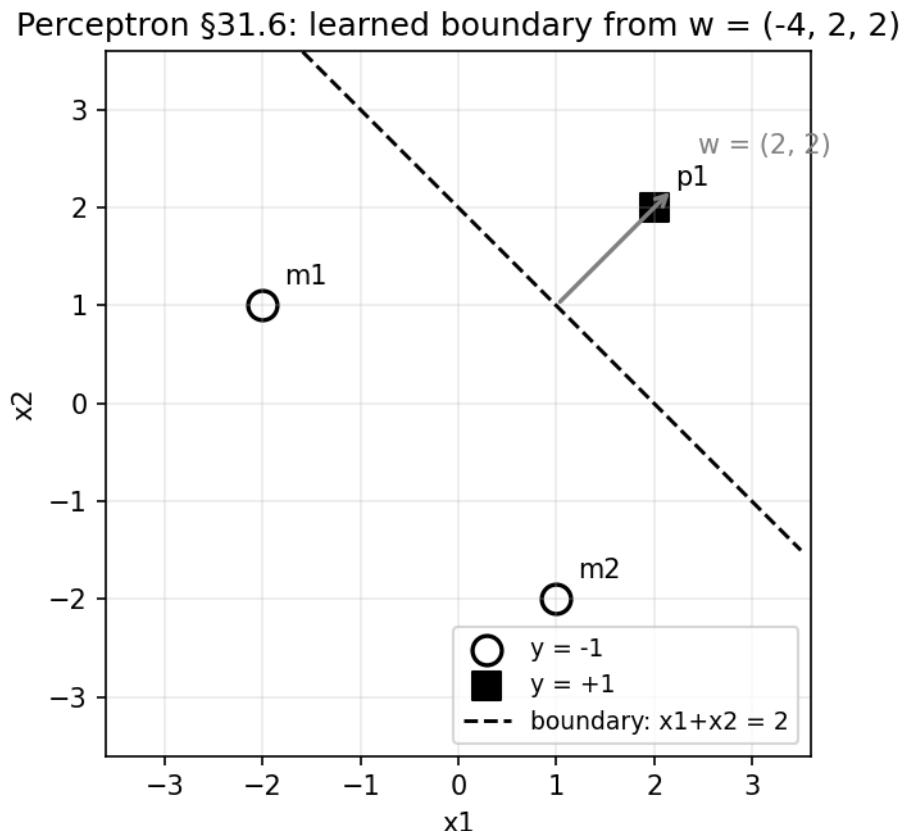


Figure 46: Scatter of the three §31.6 training points: $m_1 = (-2, 1)$ and $m_2 = (1, -2)$ as open circles (label -1), $p_1 = (2, 2)$ as a filled square (label +1); the dashed learned boundary $x_1 + x_2 = 2$; a gray arrow at $(1, 1)$ pointing along $w = (2, 2)$, orthogonal to the boundary.

Basically, ... “Start with zero weights, so everything scores 0 and gets called +1. The two -1 points complain: shove the weights by -2 times each offender, landing at $(-4, 2, 2)$. Second pass: every point is now on the right side of the wall $x_1 + x_2 = 2$, nobody complains, done. Two mistakes, then silence — that is perceptron learning.”

31.7 Why probabilities? The perceptron’s blind spot

A perceptron’s output is categorical: +1 or -1, no in-between. The 6.036 notes list why that hurts:

- i) **No degree of certainty.** “The classifier can’t express a degree of certainty about whether a particular input x should have an associated value y .” A point miles from the boundary and a point grazing it get the same verdict.
- ii) **Flat, uninformative loss.** Any two hypotheses with the same misclassification count “have the same J value” — the loss cannot tell a near-miss from a confident error, “which makes it difficult to design an algorithm that searches through the space of hypotheses for a good one”.

The fix: keep the linear score $z = \theta^T x + \theta_0$, but map it to $(0, 1)$ with a smooth squashing function and read the output **as a probability**. That is logistic regression.

Basically, ... “The perceptron only shouts yes or no. Sometimes you want ‘probably yes, 90% sure’ — and a loss that rewards getting *more* right rather than just counting wrongs. Swap the hard sign for a soft S-curve and the output becomes a probability.”

31.8 Logistic regression: the model

Def (linear logistic classifier). Hypotheses parameterized by a vector θ and a scalar θ_0 (the 6.036 notes' notation), producing outputs in $(0, 1)$:

$$\boxed{h(x; \theta, \theta_0) = \sigma(\theta^T x + \theta_0)}, \quad \boxed{\sigma(z) = \frac{1}{1 + e^{-z}}},$$

σ the **logistic (sigmoid) function**. The output is interpreted as a probability because it always lies in $(0, 1)$.

The probabilistic model (labels now in $\{0, 1\}$):

$$\boxed{P(y = 1 \mid x) = \sigma(\theta^T x + \theta_0) = g, \quad P(y = 0 \mid x) = 1 - g}.$$

This is the discriminative move §30.1 described: model $P(y \mid x)$ directly — “we just need $P(y \mid x)$ for prediction” — with no $P(x)$, no class portraits, no Bayes flip.

The boundary is still linear. Predict 1 when $g > 1/2$: since $\sigma(z) = 1/2 \iff z = 0$,

$$\boxed{\sigma(\theta^T x + \theta_0) > \frac{1}{2} \iff \theta^T x + \theta_0 > 0},$$

so the decision boundary $\sigma = 1/2$ is exactly the line (hyperplane) $\theta^T x + \theta_0 = 0$ — the 6.036 notes pose this as a study question (“convince yourself that the set of points for which $\sigma(\theta^T x + \theta_0) = 0.5 \dots$ is a line”). Same shape §30.5 derived for Naive Bayes ($w^T x + b > 0$), now learned straight from $P(y \mid x)$.

Note (the name lies). “Logistic *regression*” is classification — the “regression” is historical baggage. What it regresses is the *log-odds*: $\log \frac{g}{1-g} = \theta^T x + \theta_0$, a linear function of x .

eg (practice-assignment numbers, Q9). Two points with scores $z = 10$ and $z = -10$: $\sigma(10) = 1/(1 + e^{-10}) \approx 0.99995$, $\sigma(-10) \approx 0.00005$ — the first point’s probability “will be much higher than” the second’s (the assignment’s answer). Far along $+w$, certainty saturates at 1; far along $-w$, at 0.

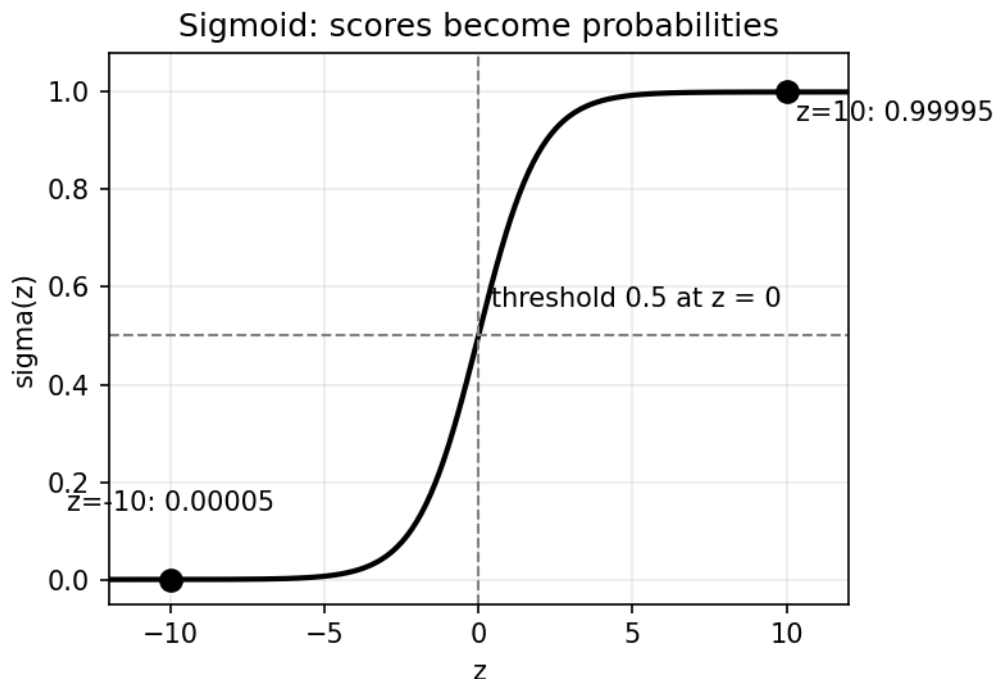


Figure 47: Sigmoid curve from $z=-12$ to 12 rising from 0 to 1, with dashed lines at $z=0$ and $\text{sigma}=0.5$ marking the threshold; marked points at $z=10$ (probability 0.99995) and $z=-10$ (probability 0.00005).

Basically, ... “Keep the perceptron’s linear score, but instead of snapping it to ± 1 , run it through an S-curve that squeezes any number into a probability between 0 and 1. Score 0 $\rightarrow$ fifty-fifty; big positive $\rightarrow$ nearly certain 1; big negative $\rightarrow$ nearly certain 0. The 50/50 line is still a straight wall — $\theta^T x + \theta_0 = 0$.”

31.9 The loss: cross-entropy is the negative log-likelihood

Training means choosing θ, θ_0 to “maximize the probability assigned by the classifier to the correct y values, as specified in the training set” (6.036). With guess $g^{(i)} = \sigma(\theta^T x^{(i)} + \theta_0)$ and labels in $\{0, 1\}$, the probability the model assigns to the *observed* labels (independent examples) is

$$\prod_{i=1}^n g^{(i) y^{(i)}} (1 - g^{(i)})^{1 - y^{(i)}}$$

— the Bernoulli likelihood (§15.11’s distribution, now with $g^{(i)}$ in the μ slot). Check the exponent trick: $y^{(i)} = 1$ picks out $g^{(i)}$; $y^{(i)} = 0$ picks out $1 - g^{(i)}$.

Products are hard to optimize, and log is monotonic, so maximize the log — equivalently **minimize the negative log-likelihood** (the §20.12(i) ritual: logistic regression is listed there as an MLE example, next to least squares and neural nets). Per example, the **negative log-likelihood (NLL)** loss:

$$L_{\text{nll}}(g, y) = -(y \log g + (1 - y) \log(1 - g)),$$

and the training objective (before regularization):

$$J(\theta, \theta_0) = \frac{1}{n} \sum_{i=1}^n L_{\text{nll}}(\sigma(\theta^T x^{(i)} + \theta_0), y^{(i)}).$$

This L_{nll} is the **cross-entropy** (log) loss: $-\log$ of the probability assigned to the truth. Confident and right ($g \approx y$) $\rightarrow$ loss near 0; confident and wrong ($g \approx 1 - y$) $\rightarrow$ loss explodes. Unlike the perceptron's 0/1 bill, it grades *how* wrong.

Basically, ... “Ask: ‘under my current weights, how probable is the *actual* observed label?’ Take the log (products become sums), negate it (we minimize), average over examples. That is the whole loss — cross-entropy is just negative log-likelihood wearing a fancy name, and minimizing it is MLE exactly as §20.12(i) promised.”

31.10 The gradient, and training by gradient descent

Unlike the perceptron's kinked loss, J is smooth — so Chapter 10's machinery applies. The 6.036 notes work the gradient out (posed as a study question; verified below). Write $z^{(i)} = \theta^T x^{(i)} + \theta_0$, $g^{(i)} = \sigma(z^{(i)})$.

Derivation. Chain rule, one example at a time. First the sigmoid:

$$\frac{d\sigma}{dz} = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z)) \quad (\text{check: multiply out}).$$

Then the loss w.r.t. the guess:

$$\frac{\partial L_{\text{nll}}}{\partial g} = -\left(\frac{y}{g} - \frac{1-y}{1-g}\right) = \frac{g-y}{g(1-g)}.$$

Multiply — the $g(1-g)$ cancels beautifully:

$$\frac{\partial L_{\text{nll}}}{\partial z} = \frac{g-y}{g(1-g)} \cdot g(1-g) = \boxed{g-y}.$$

The error signal is just *prediction minus truth*. Finally $z^{(i)} = \theta^T x^{(i)} + \theta_0$ gives $\partial z^{(i)} / \partial \theta = x^{(i)}$, $\partial z^{(i)} / \partial \theta_0 = 1$. Averaging over the n examples:

$$\boxed{\nabla_{\theta} J = \frac{1}{n} \sum_{i=1}^n (g^{(i)} - y^{(i)}) x^{(i)}, \quad \frac{\partial J}{\partial \theta_0} = \frac{1}{n} \sum_{i=1}^n (g^{(i)} - y^{(i)})}.$$

Training = gradient descent (§10.5, §10.7: step opposite the gradient):

$$\boxed{\theta := \theta - \eta \nabla_{\theta} J, \quad \theta_0 := \theta_0 - \eta \frac{\partial J}{\partial \theta_0}},$$

repeated until the change in J is tiny. Each step nudges every weight against its average error-weighted feature — misclassified-by-a-mile points (large $|g^{(i)} - y^{(i)}|$) shout loudest, the opposite of the perceptron's flat treatment.

The regularized objective. The 6.036 notes train $J_{\text{r}} = J + \frac{\lambda}{2} \|\theta\|^2$, adding $\lambda\theta$ to the gradient — the ridge penalty of Chapter 28 (§20.12(ii): MAP with a Gaussian prior). It matters: the notes' study question asks what happens on *separable* data with $\lambda = 0$ — the weights blow up ($\|\theta\| \rightarrow \infty$ drives every $g^{(i)}$ to 0 or 1 and the NLL to 0, but no finite minimizer exists). Regularization keeps the optimum finite.

Basically, ... “The gradient has the cleanest form in ML: average over examples of (how wrong the probability was) $\times$ (the features). Step the weights downhill (§10.5) and repeat. And if the data is perfectly separable, unregularized logistic regression never settles — it keeps inflating the weights toward infinite certainty — so the λ penalty is load-bearing, not decoration.”

31.11 Worked example: one gradient-descent step (every number recomputed)

Two points, one feature: $x^{(1)} = 0, y^{(1)} = 0$; $x^{(2)} = 2, y^{(2)} = 1$. Start $\theta_0 = 0, \theta = 0$; step size $\eta = 1$; no regularization.

Step 0 — scores and loss. $z^{(i)} = 0 \rightarrow g^{(i)} = \sigma(0) = 1/2$ for both.

$$J = \frac{1}{2} \left[-\log(1 - \frac{1}{2}) - \log(\frac{1}{2}) \right] = \frac{1}{2}(\log 2 + \log 2) = \boxed{\log 2 \approx 0.6931}.$$

Step 1 — gradient (§31.10). Errors $g^{(i)} - y^{(i)}$: $1/2 - 0 = 1/2$; $1/2 - 1 = -1/2$.

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{2} \left(\frac{1}{2} - \frac{1}{2} \right) = \boxed{0}, \quad \frac{\partial J}{\partial \theta} = \frac{1}{2} \left(\frac{1}{2} \cdot 0 + \left(-\frac{1}{2}\right) \cdot 2 \right) = \boxed{-\frac{1}{2}}.$$

Step 2 — update. $\theta_0 := 0 - 1 \cdot 0 = 0$; $\theta := 0 - 1 \cdot (-\frac{1}{2}) = \boxed{\frac{1}{2}}$.

Step 3 — check the loss fell. New scores: $z^{(1)} = 0 \rightarrow g^{(1)} = 0.5$; $z^{(2)} = 1 \rightarrow g^{(2)} = \sigma(1) = 1/(1 + e^{-1}) \approx 0.7311$.

$$J_{\text{new}} = \frac{1}{2} [-\log(1 - 0.5) - \log(0.7311)] = \frac{1}{2}(0.6931 + 0.3133) \approx \boxed{0.5032} < 0.6931 \checkmark.$$

(Every number above recomputed in numpy — see the review log. Note the bias gradient came out 0: the two errors $\pm 1/2$ cancel, so this step moves only the slope.)

Basically, ... “Start flat: both points get fifty-fifty, loss $\log 2$. The point at $x = 2$ should be class 1 but scores only 0.5 — error $-1/2$, times feature 2, averaged: slope gradient $-1/2$. Step uphill against it: $\theta = 1/2$. Now $x = 2$ scores $\sigma(1) \approx 0.73$ and the loss drops to 0.50. One step, loss down — gradient descent doing its job.”

31.12 Multiclass: softmax

Binary logistic regression squashes one score into $(0, 1)$. For $k > 2$ classes with one-hot labels $y = (y_1, \dots, y_k)$, the 6.036 notes use the **softmax** activation on a score vector $z = (z_1, \dots, z_k)$:

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{r=1}^k e^{z_r}},$$

which lands in $[0, 1]^k$ and sums to 1 — “we can interpret it as a probability distribution over k items”. The multiclass NLL extends the §31.9 form:

$$L_{\text{nllm}} = - \sum_{r=1}^k y_r \log a_r, \quad a = \text{softmax}(z),$$

($k = 2$ recovers L_{nll} — the notes set this as a study question.) Each class gets its own linear score $z_r = \theta_r^T x + \theta_{r0}$; the class boundaries are the hyperplanes where two scores tie — exactly the k -discriminant fix the Models of Classification slides build after showing one-vs-rest’s ambiguity regions (§31.1’s source: $k - 1$ one-vs-rest walls leave “regions of ambiguity”; k scored classes with argmax do not).

Basically, ... “Softmax = sigmoid for k teams: exponentiate each class’s linear score, divide by the total, and you get k probabilities summing to 1. The loss is still ‘negative log of the probability given to the truth’”

31.13 Generative vs discriminative, revisited: Naive Bayes against logistic regression

§30.1 framed the two philosophies; now both halves of the pair are on the table. Same binary task, same linear boundary $w^T x + b > 0$ (§30.5) — different things estimated, different training:

- i) **What is estimated.** Naive Bayes models the *joint* $P(x, y) = P(x | y)P(y)$ — class portraits plus priors — and Bayes-flips at prediction time. Logistic regression models *only* $P(y | x) = \sigma(\theta^T x + \theta_0)$; $P(x)$ is never estimated (“we just need $P(y | x)$ for prediction”, §30.1).
- ii) **How training works.** NB training is *counting* — closed-form MLE fractions, no iterations (§30.7). Logistic regression has *no closed form* — the weights come from iterative gradient descent on the NLL (§31.10).
- iii) **Parameter count (binary, d features).** NB needs $2d + 1$ numbers (per-class per-feature probabilities plus the prior, §30.3(ii)). Logistic regression needs $d + 1$ (θ, θ_0). Fewer knobs — but each costs iterations.
- iv) **Where the boundary comes from.** NB’s boundary falls out of the log-odds of the *fitted generative* model (§30.5). Logistic regression’s boundary is fitted *directly* to separate the classes in $P(y | x)$ space.

Note (what the sources do not say). The classic “generative learns faster from little data, discriminative wins with lots of data” comparison (Ng & Jordan) appears nowhere in the course materials — so it appears nowhere here either. The honest sourced contrast is i)–iv) above. Flagged in the review log.

Basically, ... “Naive Bayes learns what each class *looks like* (by counting) and derives the wall from the portraits. Logistic regression learns the wall *directly*, as the $P(y | x)$ that best explains the labels (by gradient descent). Same wall shape, opposite directions of travel — that is the whole generative-vs-discriminative idea, in one pair.”

31.14 Strengths and limits (as the sources list them)

Perceptron. - Strengths: i) dead simple — one update rule, no derivatives needed (the loss isn’t differentiable anyway, §31.3); ii) **mistake-driven**: correct points cost nothing, training touches only errors; iii) finite-step convergence with zero training loss on separable data (§31.5). - Limits: i) **binary only** (slides); ii) **linearly separable data only** — otherwise it oscillates forever, never converging (practice Q2); iii) no probabilities, no notion of confidence (§31.7); iv) says nothing about test error (§22.10).

Logistic regression. - Strengths: i) **probabilistic outputs** — expresses certainty, and the loss rewards *how right* not just *whether right* (§31.7); ii) smooth, differentiable objective $\rightarrow$ principled training by gradient descent (§31.10); iii) extends cleanly to k classes via softmax (§31.12). - Limits: i) **still a linear boundary** ($\sigma = 1/2 \iff \theta^T x + \theta_0 = 0$) — XOR-type data defeats it exactly as it defeats the perceptron (practice Q7’s lesson generalizes: any linear wall fails where no straight wall exists); ii) **no closed form** — training is iterative, with a step size to tune (§10.4); iii) on separable data the unregularized weights diverge — the λ penalty is required for a finite solution (§31.10).

Evaluation (both algorithms; the slides’ list, same as §30.11): confusion matrix, then precision, recall, accuracy, F1 — §22.9’s misclassification fraction underneath it all.

Basically, ... “Perceptron: the simplest learner that could possibly work — mistakes only, guaranteed to finish *if* a straight wall exists. Logistic regression: the grown-up version — probabilities, smooth loss, gradient descent — but it draws the same straight wall, so curved problems still beat it. Pick the perceptron for the idea, logistic regression for the probabilities.”

31.15 Where this goes next

- i) **Heavier linear machinery.** Chapters 32–33 (SVMs) keep the linear boundary but choose *which* wall — the max-margin one — instead of the perceptron’s first wall found. The practice assignment’s margin γ (§31.5) is the preview.

- ii) **Beyond linearity.** Ensembles (Chapter 34) and the neural networks of Part VI — the perceptron with a smooth activation and stacked layers is where this whole book is heading; §31.2's neuron remark was the seed.
- iii) **Loss functions.** Classification losses get their own chapter (Chapter 35); L_{nl} was the first specimen.

Problem set

1. **Perceptron by hand.** Augmented data ($\phi(x) = (1, x_1, x_2)$, $y \in \{-1, +1\}$), $\alpha = 1$, $w^{(0)} = (0, 0, 0)$, cycle in the given order: $a = ((1, 0, 2), +1)$, $b = ((1, 2, 0), -1)$, $c = ((1, -1, -1), -1)$.
 - (i) Run sweeps until no update fires; give w after every update.
 - (ii) State the learned boundary in x_1, x_2 .
 - (iii) How many mistakes (updates) were made in total?
2. **Mistake bound.** (i) $R = 3$ (max point length), margin $\gamma = 0.5$: what is the maximum number of mistakes the perceptron can make? (ii) After some iteration $\|w\|^2 = 20$ with $R = 2$: which of $\{21, 23, 25\}$ can be the squared length after the next update, and why? (iii) In one line: what does the bound say about data that is *not* linearly separable?
3. **Flip the labels.** You trained a perceptron to w , then discover every training label was backwards ($+1 \leftrightarrow -1$). You cannot touch the data or the prediction code — only w . What single change to w flips every prediction (assume no point lies exactly on the boundary)? Prove it in two lines.
4. **Sigmoid arithmetic.** (i) Compute $\sigma(10)$ and $\sigma(-10)$ to 4 significant figures. (ii) Find z with $\sigma(z) = 0.9$. (iii) Prove $\sigma(-z) = 1 - \sigma(z)$, and use it to show the decision rule “ $g > 1/2 \iff z > 0$ ” from §31.8.
5. **One GD step, new numbers.** Data: $(x^{(1)}, y^{(1)}) = (1, 0)$, $(x^{(2)}, y^{(2)}) = (3, 1)$; $\theta_0 = 0$, $\theta = 0$, $\eta = 1$, no regularization. (i) Compute J before the step. (ii) Compute ∇J via §31.10's formulas. (iii) Take the step; give the new (θ_0, θ) . (iv) Compute J after and confirm it decreased.
6. **Same wall, two derivations.** Starting from §30.5's Naive Bayes log-odds $\log \frac{P(y=1|x)}{P(y=0|x)} = w^T x + b$ and from §31.8's $P(y = 1 | x) = \sigma(\theta^T x + \theta_0)$, show both decision rules reduce to “predict 1 iff (linear score) > 0 ”. In two lines: what is genuinely different about how the two methods *obtain* their coefficients?
7. **Count the knobs.** Binary task, d features. (i) How many parameters does Naive Bayes fit (§30.3(ii))? (ii) How many does logistic regression fit? (iii) Which training procedure is closed-form and which is iterative — and what does the iterative one need that the closed-form one does not?

Solutions — Chapter 31: Perceptron and logistic regression

Problem 1 — Perceptron by hand

Data (augmented, $y \in \{-1, +1\}$), $\alpha = 1$, $w^{(0)} = (0, 0, 0)$, cycling a, b, c :

	$\phi(x)$	y
a	$(1, 0, 2)$	$+1$
b	$(1, 2, 0)$	-1
c	$(1, -1, -1)$	-1

Sweep 1. - a : $z = 0 \rightarrow \hat{y} = +1 = y$. Correct, no change. - b : $z = 0 \rightarrow \hat{y} = +1 \neq -1$. Mistake: $w := (0, 0, 0) + 1 \cdot (-1 - 1) \cdot (1, 2, 0) = \boxed{(-2, -4, 0)}$. - c : $z = (-2)(1) + (-4)(-1) + 0(-1) = -2 + 4 + 0 = 2 \geq 0 \rightarrow \hat{y} = +1 \neq -1$. Mistake: $w := (-2, -4, 0) + (-2)(1, -1, -1) = (-2 - 2, -4 + 2, 0 + 2) = \boxed{(-4, -2, 2)}$.

Sweep 2 (verify, $w = (-4, -2, 2)$). - a : $z = -4(1) + (-2)(0) + 2(2) = -4 + 0 + 4 = 0 \geq 0 \rightarrow +1$. - b : $z = -4(1) + (-2)(2) + 2(0) = -4 - 4 + 0 = -8 < 0 \rightarrow -1$. - c : $z = -4(1) + (-2)(-1) + 2(-1) = -4 + 2 - 2 = -4 < 0 \rightarrow -1$.

No update fires in sweep 2 — converged (independently confirmed in numpy: same update sequence, final scores $(0, -8, -4)$).

- (i) Weight history: $(0, 0, 0) \rightarrow (-2, -4, 0) \rightarrow (-4, -2, 2)$.
- (ii) Boundary: $-4 - 2x_1 + 2x_2 = 0$, i.e. $x_2 = x_1 + 2$.
- (iii) Total mistakes (updates): $\boxed{2}$.

Problem 2 — Mistake bound

- (i) $\# \text{mistakes} \leq (R/\gamma)^2 = (3/0.5)^2 = 6^2 = \boxed{36}$.
- (ii) One update can increase $\|w\|^2$ by at most $R^2 = 4$, so the next squared length is at most $20 + 4 = 24$. $\boxed{21}$ and $\boxed{23}$ are possible; $25 > 24$ is not.
- (iii) The bound applies only to linearly separable data — for non-separable data the perceptron makes unboundedly many updates: it oscillates forever (§31.5).

Problem 3 — Flip the labels

Negate the weight vector: $w_{\text{new}} = -w$. Proof: $\hat{y}_{\text{new}} = \text{sign}(-w^T \phi(x)) = -\text{sign}(w^T \phi(x)) = -\hat{y}_{\text{old}}$ whenever $w^T \phi(x) \neq 0$ (given: no point sits exactly on the boundary, so the sign is never ambiguous). Every prediction flips, exactly undoing the label swap. (This is the Week 9 practice assignment's Q6, whose answer is the negation.)

Problem 4 — Sigmoid arithmetic

- (i) $\sigma(10) = 1/(1 + e^{-10})$: $e^{-10} \approx 4.5400 \times 10^{-5}$, so $\sigma(10) \approx 1/(1.0000454) \approx \boxed{0.99995}$ (4 s.f.: 0.99995).
 $\sigma(-10) = 1/(1 + e^{10}) \approx e^{-10} \approx \boxed{4.540 \times 10^{-5}}$.
- (ii) $\sigma(z) = 0.9 \iff 1/(1 + e^{-z}) = 0.9 \iff 1 + e^{-z} = 10/9 \iff e^{-z} = 1/9 \iff \boxed{z = \log 9 \approx 2.197}$.
- (iii) $\sigma(-z) = \frac{1}{1+e^z} = \frac{e^{-z}}{e^{-z}(1+e^z)} = \frac{e^{-z}}{1+e^{-z}} = 1 - \frac{1}{1+e^{-z}} = \boxed{1 - \sigma(z)}$. Then $g > 1/2 \iff \sigma(z) > 1/2 \iff 1/(1 + e^{-z}) > 1/2 \iff 1 + e^{-z} < 2 \iff e^{-z} < 1 \iff \boxed{z > 0}$ (using monotonicity of exp and log).

Problem 5 — One GD step, new numbers

$(x^{(1)}, y^{(1)}) = (1, 0)$, $(x^{(2)}, y^{(2)}) = (3, 1)$; $\theta_0 = 0$, $\theta = 0$, $\eta = 1$.

- (i) $z^{(i)} = 0$, $g^{(i)} = \sigma(0) = 1/2$ both. $J = \frac{1}{2}[-\log(1 - \frac{1}{2}) - \log(\frac{1}{2})] = \boxed{\log 2 \approx 0.6931}$.
- (ii) Errors: $g^{(1)} - y^{(1)} = 1/2$, $g^{(2)} - y^{(2)} = -1/2$.

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{2} \left(\frac{1}{2} - \frac{1}{2} \right) = \boxed{0}, \quad \frac{\partial J}{\partial \theta} = \frac{1}{2} \left(\frac{1}{2} \cdot 1 + \left(-\frac{1}{2}\right) \cdot 3 \right) = \frac{1}{2} \left(\frac{1}{2} - \frac{3}{2} \right) = \boxed{-\frac{1}{2}}.$$

$$(iii) \theta_0 := 0 - 0 = \boxed{0}; \theta := 0 - \left(-\frac{1}{2}\right) = \boxed{\frac{1}{2}}.$$

$$(iv) \text{ New scores: } z^{(1)} = \frac{1}{2} \rightarrow g^{(1)} = \sigma(0.5) \approx 0.6225; z^{(2)} = \frac{3}{2} \rightarrow g^{(2)} = \sigma(1.5) \approx 0.8176.$$

$$J_{\text{new}} = \frac{1}{2}[-\log(1-0.6225)-\log(0.8176)] = \frac{1}{2}[-\log(0.3775)-\log(0.8176)] \approx \frac{1}{2}(0.9741+0.2014) \approx \boxed{0.5877} < 0.6931 \checkmark.$$

(All numbers cross-checked in numpy: $J_0 = 0.69314718$, gradient $(0, -0.5)$, $J_1 = 0.58774513$.)

Problem 6 — Same wall, two derivations

Naive Bayes: decide 1 iff posterior odds exceed 1, i.e. $\log \frac{P(y=1|x)}{P(y=0|x)} > 0$. By §30.5 the log-odds equals $w^T x + b$, so: predict 1 $\iff \boxed{w^T x + b > 0}$.

Logistic regression: decide 1 iff $P(y = 1 \mid x) = \sigma(\theta^T x + \theta_0) > 1/2$. By Problem 4(iii), $\sigma(z) > 1/2 \iff z > 0$, so: predict 1 $\iff \boxed{\theta^T x + \theta_0 > 0}$.

The genuine difference: NB's (w, b) are *derived* from counted generative parameters (priors and class-conditionals, closed-form MLE, §30.7); logistic regression's (θ, θ_0) are *fitted directly* by iterative gradient descent on the NLL of $P(y \mid x)$ (§31.10). Same wall shape, opposite estimation philosophies.

Problem 7 — Count the knobs

- (i) Naive Bayes (binary): $\boxed{2d + 1}$ — one Bernoulli parameter per feature per class ($2d$) plus the prior (§30.3(ii)).
- (ii) Logistic regression (binary): $\boxed{d + 1}$ — the weight vector $\theta \in \mathbb{R}^d$ and the scalar bias θ_0 (§31.8).
- (iii) Naive Bayes is **closed-form**: training is counting (§30.7), one pass over the data, no hyperparameters. Logistic regression is **iterative**: gradient descent needs a step size η (§10.4), an initialization, and a stopping criterion — and has no finite closed-form solution.

Chapter 32: Hard-margin SVM

Part IV — Machine Learning Techniques. Builds on: linear boundaries (§31.1), the perceptron’s margin γ (§31.5), Lagrange/KKT machinery (Ch 11, Ch 13), and the SVM preview (§13.15). Sources: MLT Week 10 lecture slides (hard-margin SVM derivation) and the MLT Week 10 practice assignment.

32.1 The problem with “any” wall: which separator?

The perceptron (§31.2–§31.5) finds *a* separating wall and stops. If the data is linearly separable, infinitely many walls separate it — the perceptron just grabs the first one its update order stumbles into, then stops dead (correct points cause no updates, §31.5).

The lecture’s slide 1 asks the uncomfortable question: *given that we prefer classifiers with large margin, can we directly find them?* Its observation:

- i) The perceptron’s mistake bound depends on the margin of the *best possible* separator w^* — the one we never actually compute.
- ii) The wall the perceptron *does* find need not be w^* . It can be any wall that happens to work — including one that grazes the data.

So the perceptron answers “find me a wall”, and this chapter answers the harder question: **of all the walls that separate the classes, which one should we pick?**

The SVM’s answer = the wall **farthest from both classes** — the max-margin wall. The perceptron’s practice-assignment margin γ (§31.5) was the preview: there we measured how much room a separator leaves; here we *optimize* that room.

Basically, ... “The perceptron is the first apartment you toured that had a roof. The SVM is the apartment farthest from both noisy neighbours. Same street (linear walls) — much better address.”

32.2 The margin: functional, geometric, and the canonical form

For a wall $w^T x + b = 0$ and a labelled point $(x^{(i)}, y^{(i)})$, $y^{(i)} \in \{-1, +1\}$:

- i) **Functional margin** = $y^{(i)}(w^T x^{(i)} + b)$. Positive means correct, and its size says “how correct” — but it is *scale-sensitive*: replace (w, b) by $(2w, 2b)$ and every functional margin doubles while the wall never moves.
- ii) **Geometric margin** = $\frac{y^{(i)}(w^T x^{(i)} + b)}{\|w\|}$. This is the true perpendicular distance from the point to the wall, signed by correctness. Scaling (w, b) leaves it unchanged — it measures the wall, not the numbers.
- iii) The **margin of the wall** = the smallest geometric margin over the data: $\gamma = \min_i \frac{y^{(i)}(w^T x^{(i)} + b)}{\|w\|}$.

The scale problem in (i) is the whole game. Fix it by *choosing a scale*: demand that the closest point(s) sit at functional margin exactly 1:

$$\boxed{\min_i y^{(i)}(w^T x^{(i)} + b) = 1} \quad (\text{canonical form})$$

This is always achievable for a separating wall — just rescale (w, b) until the minimum functional margin equals 1. The wall doesn’t move; only the ruler changes.

Note: the canonical form pins down a unique (w, b) per wall. That is why you may rescale the *equation* $w^T x + b = 0$ by a constant but may not rescale w alone — scaling w alone changes which wall the canonical constraints describe (practice assignment, Q3).

With the canonical scale, the geometry becomes clean. The closest points satisfy $w^T x + b = \pm 1$ — two hyperplanes parallel to the wall, one per class. These are the **supporting hyperplanes**. The distance between two parallel planes $w^T x + b = c_1$ and $w^T x + b = c_2$ is $|c_1 - c_2|/\|w\|$ (project the gap onto the unit normal $w/\|w\|$). So:

$$\boxed{\text{width} = \frac{1 - (-1)}{\|w\|} = \frac{2}{\|w\|}}$$

and the margin on each side is $\gamma = 1/\|w\|$. Maximizing the margin is now purely about making $\|w\|$ small.

Basically, ... “Functional margin is the score in made-up points — double the numbers, double the score, same wall. Geometric margin is the score in metres — it can’t be faked. The canonical form just declares ‘the nearest point scores exactly 1’, which fixes the ruler. After that, the whole gap between the two classes is $2/\|w\|$: a shorter w means a wider street.”

32.3 The hard-margin primal: from “widest street” to $\min \frac{1}{2}\|w\|^2$

Maximize the width $2/\|w\|$ over all separating walls:

- i) Maximizing $2/\|w\| \iff$ minimizing $\|w\| \iff$ minimizing $\|w\|^2$ (monotone transforms; the optimum (w, b) is the same).
- ii) The $\frac{1}{2}$ is cosmetic: $\frac{1}{2}\|w\|^2$ differentiates to w , keeping every gradient clean (§13.15’s Hessian remark: the Hessian is the identity).
- iii) “Separating wall” in canonical form = every point’s functional margin is at least 1.

The **hard-margin SVM primal**:

$$\boxed{\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y^{(i)}(w^T x^{(i)} + b) \geq 1 \quad \forall i}$$

Note (when this has no answer). The constraints demand a wall with *every* point on the correct side with functional margin ≥ 1 . If the data is not linearly separable, no such (w, b) exists — the problem is infeasible. The hard-margin SVM, like the perceptron (§31.5), needs separable data; the lecture’s closing question (“how to adapt the SVM when data has outliers?”) is exactly this weakness, and Chapter 33’s soft margin is the answer.

Basically, ... “Widest street = shortest w . The $\frac{1}{2}$ and the square are just tidying up so the calculus comes out nice. And the fine print: this whole problem only makes sense if a separating wall exists at all — one point on the wrong side and there is no feasible answer.”

32.4 The dual: same problem, mirrored

The primal is a quadratic objective with linear constraints — exactly the shape Chapters 11 and 13 were built for (§13.15(i): strong duality holds, so the dual agrees with the primal). The lecture derives the dual in four moves:

Move 1 — the Lagrangian. With one multiplier $\alpha_i \geq 0$ per constraint, written in the $g_i \leq 0$ shape ($g_i(w, b) = 1 - y^{(i)}(w^T x^{(i)} + b)$, §13.2):

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \alpha_i (1 - y^{(i)}(w^T x^{(i)} + b)).$$

Move 2 — the primal as min-max. By §13.3's trick, $\max_{\alpha \geq 0} L$ is $\frac{1}{2} \|w\|^2$ when (w, b) is feasible and $+\infty$ otherwise. So the primal is $\min_{w, b} \max_{\alpha \geq 0} L$.

Move 3 — swap. For convex f and g (quadratic objective, linear constraints), min and max swap (§13.4, §13.7):

$$\min_{w, b} \max_{\alpha \geq 0} L(w, b, \alpha) = \max_{\alpha \geq 0} \min_{w, b} L(w, b, \alpha).$$

The right side is the **dual problem**.

Move 4 — the inner min, by stationarity. Fix $\alpha \geq 0$ and minimize over (w, b) (unconstrained now — set gradients to zero, §11.3):

$$\begin{aligned} \frac{\partial L}{\partial w} = w - \sum_{i=1}^n \alpha_i y^{(i)} x^{(i)} = 0 & \implies \boxed{w^* = \sum_{i=1}^n \alpha_i y^{(i)} x^{(i)}}, \\ \frac{\partial L}{\partial b} = - \sum_{i=1}^n \alpha_i y^{(i)} = 0 & \implies \boxed{\sum_{i=1}^n \alpha_i y^{(i)} = 0}. \end{aligned}$$

Substituting w^* back into L and simplifying (the $\frac{1}{2} \|w\|^2$ and the $-\sum \alpha_i y^{(i)} w^T x^{(i)}$ terms collapse into $-\frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$):

$$\boxed{\max_{\alpha \geq 0, \sum_i \alpha_i y^{(i)} = 0} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}}$$

Note (notation vs the lecture). The lecture folds the bias into w (augmented $\tilde{x}$), so its dual has only $\alpha \geq 0$. Keeping b explicit — as the canonical form of §32.2 does — adds the stationarity condition $\sum_i \alpha_i y^{(i)} = 0$. Same problem, one extra bookkeeping line.

Basically, ... “The primal asks: smallest w among all feasible walls. The dual asks a mirror question: how much ‘vote’ α_i should each datapoint get? Stationarity says the winning w is just a weighted vote of the datapoints — and once you plug that back in, the data appears only as dot products $x^{(i)T} x^{(j)}$. That last fact is the whole reason kernels will work in Chapter 33.”

32.5 KKT and support vectors: only the wall-huggers vote

At the optimum, the four KKT conditions (§13.8–§13.9) hold. For the SVM they say:

- i) **Stationarity:** $w^* = \sum_i \alpha_i^* y^{(i)} x^{(i)}$ and $\sum_i \alpha_i^* y^{(i)} = 0$ (§32.4).

- ii) **Primal feasibility:** $y^{(i)}(w^*{}^T x^{(i)} + b^*) \geq 1$ for all i — every point correctly placed with functional margin ≥ 1 .
- iii) **Dual feasibility:** $\alpha_i^* \geq 0$ for all i .
- iv) **Complementary slackness:** $\alpha_i^* \cdot (1 - y^{(i)}(w^*{}^T x^{(i)} + b^*)) = 0$ for all i (§11.7, §13.11).

From (iv): **if** $\alpha_i^* > 0$ **then** $y^{(i)}(w^*{}^T x^{(i)} + b^*) = 1$ — the point sits exactly on its supporting hyperplane. These points are the **support vectors**: the wall-huggers that pin the margin in place.

The subtle direction (practice assignment, Q1): the converse is *not* guaranteed. A point can sit exactly on a supporting hyperplane and still have $\alpha_i^* = 0$ — complementary slackness only forces the product to zero, and $0 \cdot 0 = 0$ is fine. So: **every support vector lies on a supporting hyperplane, but not every point on a supporting hyperplane is a support vector.**

Consequences (the lecture’s “what have we gained”, slide 7):

- i) w^* is a **sparse** linear combination of the data — only support vectors contribute; every other point gets $\alpha_i^* = 0$ and drops out of the answer entirely.
- ii) The dual sees the data **only through inner products** $x^{(i)T} x^{(j)}$ — the doorway to kernels (Chapter 33).
- iii) The decision function needs only the support vectors:

$$\hat{y}(x_{\text{test}}) = \text{sign} \left(\sum_{i: \alpha_i^* > 0} \alpha_i^* y^{(i)} x^{(i)T} x_{\text{test}} + b^* \right)$$

- iv) Recover b^* from any support vector j : $y^{(j)}(w^*{}^T x^{(j)} + b^*) = 1$ gives $b^* = y^{(j)} - w^*{}^T x^{(j)}$.

Note (prediction inside the margin). A test point landing *between* the supporting hyperplanes still gets a label — the sign of $w^*{}^T x_{\text{test}} + b^*$ decides, margin or no margin (practice assignment, Q6). The constraints shaped the wall during training; they don’t gate predictions.

Basically, ... “Complementary slackness is the bouncer: a datapoint either has voting power ($\alpha_i > 0$) and then it must be pressed against the margin wall, or it sits safely away and gets zero vote. The final wall is built from the wall-huggers alone — delete every other point and nothing changes. (Careful: hugging the wall doesn’t *guarantee* a vote — a point can lean on the margin line with $\alpha_i = 0$.)”

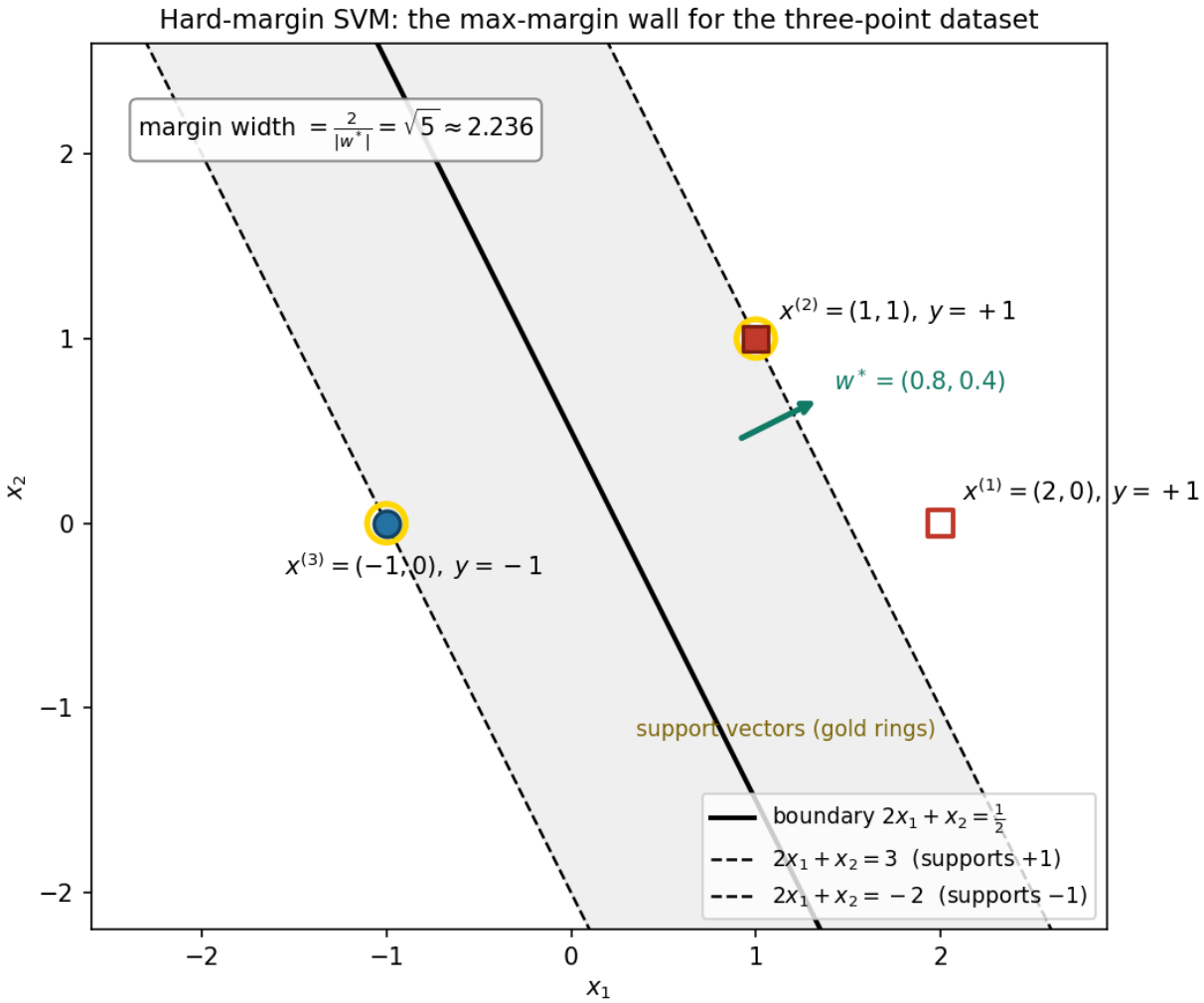


Figure 48: The three §32.6 training points: $\hat{x}^{(1)} = (2, 0)$ and $\hat{x}^{(2)} = (1, 1)$ as red squares (label +1), $\hat{x}^{(3)} = (-1, 0)$ as a blue circle (label -1); the solid max-margin boundary $2x_1 + x_2 = 1/2$, the dashed supporting hyperplanes $2x_1 + x_2 = 3$ and $2x_1 + x_2 = -2$ with the margin band shaded, gold rings marking the support vectors $\hat{x}^{(2)}$ and $\hat{x}^{(3)}$, and a green arrow for $\hat{w} = (0.8, 0.4)$; *margin width* $2/\|\hat{w}\| = \sqrt{5} \approx 2.236$.

32.6 Worked example: three points, the dual solved by hand

Data: $x^{(1)} = (2, 0)$, $y^{(1)} = +1$; $x^{(2)} = (1, 1)$, $y^{(2)} = +1$; $x^{(3)} = (-1, 0)$, $y^{(3)} = -1$.

Step 1 — guess which constraints bind. The margin will be pinned by the closest cross-class pair. Distances: $\|x^{(2)} - x^{(3)}\| = \sqrt{5} \approx 2.236$; $\|x^{(1)} - x^{(3)}\| = 3$. So try $\alpha_1 = 0$ (point 1 sits safely beyond the margin) with points 2, 3 active.

Step 2 — the dual on the active set. With $\alpha_1 = 0$, the constraint $\sum_i \alpha_i y^{(i)} = 0$ gives $\alpha_2 = \alpha_3 =: \alpha$. Dot products: $x^{(2)T} x^{(2)} = 2$, $x^{(3)T} x^{(3)} = 1$, $y^{(2)} y^{(3)} x^{(2)T} x^{(3)} = (-1)(-1) = 1$. The dual objective:

$$\begin{aligned} D(\alpha) &= (\alpha_2 + \alpha_3) - \frac{1}{2}(\alpha_2^2 \cdot 2 + \alpha_3^2 \cdot 1 + 2\alpha_2\alpha_3 \cdot 1) \\ &= 2\alpha - \frac{1}{2}(2\alpha^2 + \alpha^2 + 2\alpha^2) = 2\alpha - \frac{5}{2}\alpha^2. \end{aligned}$$

$$D'(\alpha) = 2 - 5\alpha = 0 \Rightarrow \boxed{\alpha^* = (0, 0.4, 0.4)}, D = 0.8 - 0.4 = 0.4.$$

Step 3 — recover w^*, b^* .

$$w^* = 0.4 \cdot (+1) \cdot (1, 1) + 0.4 \cdot (-1) \cdot (-1, 0) = (0.4, 0.4) + (0.4, 0) = \boxed{(0.8, 0.4)}.$$

From support vector 2 ($y^{(2)}(w^{*T}x^{(2)} + b) = 1$): $0.8 + 0.4 + b = 1 \Rightarrow \boxed{b^* = -0.2}$. Cross-check with support vector 3: $(-1)(0.8 \cdot (-1) + 0 - 0.2) = (-1)(-1) = 1$.

Step 4 — verify everything (KKT). Functional margins $y^{(i)}(w^{*T}x^{(i)} + b^*)$:

- Point 1: $0.8 \cdot 2 + 0 - 0.2 = 1.4 \geq 1$ (inactive, $\alpha_1 = 0$),
- Point 2: $0.8 + 0.4 - 0.2 = 1.0$ (active, $\alpha_2 > 0$),
- Point 3: $(-1)(-0.8 - 0.2) = 1.0$ (active, $\alpha_3 > 0$).

Complementary-slackness products $\alpha_i(1 - \text{margin}_i)$: $0 \cdot (1 - 1.4) = 0$, $0.4 \cdot 0 = 0$, $0.4 \cdot 0 = 0$. Primal value $\frac{1}{2}\|w^*\|^2 = \frac{1}{2}(0.64 + 0.16) = 0.4 = \text{dual value } 0.4$ (strong duality). A numpy grid search over the dual feasible set confirms 0.4 is the maximum, attained at $(0, 0.4, 0.4)$ — see the review log.

Step 5 — the geometry. $\|w^*\| = \sqrt{0.8} = 2/\sqrt{5}$, so $\boxed{\text{width} = 2/\|w^*\| = \sqrt{5} \approx 2.236}$. Supporting hyperplanes $0.8x_1 + 0.4x_2 - 0.2 = \pm 1$, i.e. $\boxed{2x_1 + x_2 = 3}$ and $\boxed{2x_1 + x_2 = -2}$; boundary $\boxed{2x_1 + x_2 = 0.5}$.

Step 6 — support vectors and a prediction. Support vectors = $x^{(2)}, x^{(3)}$ (gold rings in the figure). Point $x^{(1)}$ has $\alpha_1 = 0$: it contributes nothing to w^* and could be deleted without changing the answer. Test point $x_{\text{test}} = (0, 0)$: $\text{sign}(w^{*T}x_{\text{test}} + b^*) = \text{sign}(-0.2) = \boxed{-1}$ — labelled even though it falls inside the margin band (cf. §32.5, Note).

Basically, ... “Three points, and only the two hugging the gap matter. The dual said ‘give the two wall-huggers 0.4 votes each, the safe point gets nothing’, and out popped $w^* = (0.8, 0.4)$, $b^* = -0.2$, street width $\sqrt{5}$. Primal and dual agree at 0.4 — the mirror told the truth.”

32.7 Strengths and limits (as the sources list them)

Strengths.

- A principled wall, not the first one found.** Max-margin is a deliberate choice of separator; the assignment states max-margin classifiers “will generalize better than perceptrons” (practice Q8).
- Sparse answer.** Only the support vectors decide — the rest of the dataset is dead weight after training (lecture, slide 9).
- The dual is the better game.** n variables with “easier” constraints (§13.15(ii)), data entering only via inner products — which “will help one go from linear models to non-linear models in a very smooth way” (kernels, Chapter 33).
- Same family as perceptron and logistic regression.** All three are linear, discriminative models with boundary $w^T x + b = 0$ (practice Q7–Q8) — the SVM is the heavy machinery §29.8(iii) promised would beat KNN on hard tasks.

Limits.

- Separability or nothing.** The hard-margin primal is infeasible the moment one point crosses into the wrong side — it is not robust to outliers (lecture’s closing question; practice Q8 notes the soft margin *is* robust where the perceptron merely “converges only in the linearly separable case”).
- No probabilities.** Distance from the wall carries no confidence — unlike logistic regression, where farther from the boundary means a surer prediction (practice Q8).
- Heavier than the perceptron.** The answer is a quadratic program, not a few online updates — the price of principled.

Basically, ... “SVM’s pitch: the most defensible wall, decided by a handful of points, in a form that kernels can stretch. SVM’s price: it demands cleanly separable data, it never tells you how sure it is, and you pay for the principle with a real optimization problem.”

32.8 Where this goes next

One pointer, as promised: the lecture ends by asking how to adapt the SVM when data has outliers — “kernels can help but is not the right way to solve this” — and answers with the soft-margin primal (slack variables ξ_i , the “bribe” each misbehaving point pays). Soft margins *and* the kernel trick are Chapter 33’s job.

Problem set

- Width and supporting lines.** For the §32.6 solution $w^* = (0.8, 0.4)$, $b^* = -0.2$: (i) compute the margin width from $2/\|w^*\|$; (ii) write the two supporting hyperplanes and the boundary as $2x_1 + x_2 = c$ for suitable c ; (iii) compute the perpendicular distance from $x^{(1)} = (2, 0)$ to the boundary and confirm it exceeds half the width.
- Scaling, canonical and otherwise.** (i) If (w, b) separates the data, show $(2w, 2b)$ gives the same wall, doubled functional margins, and unchanged geometric margins. (ii) In one line: what does the canonical condition $\min_i y^{(i)}(w^T x^{(i)} + b) = 1$ buy you? (iii) Why is rescaling w *alone* (leaving b) forbidden — what breaks?
- The symmetric square.** $x^{(1)} = (1, 1), y^{(1)} = +1; x^{(2)} = (1, -1), y^{(2)} = +1; x^{(3)} = (-1, 1), y^{(3)} = -1; x^{(4)} = (-1, -1), y^{(4)} = -1$. (i) By symmetry set $\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \alpha$; write the dual objective and maximize it. (ii) Recover w^*, b^* , the width, and the supporting hyperplanes. (iii) Which points are support vectors?
- Complementary slackness, both directions.** (i) From $\alpha_i^*(1 - y^{(i)}(w^{*T} x^{(i)} + b^*)) = 0$, prove: $\alpha_i^* > 0 \Rightarrow y^{(i)}(w^{*T} x^{(i)} + b^*) = 1$. (ii) Explain, in the practice assignment’s words (Q1), why a point sitting exactly on a supporting hyperplane may still have $\alpha_i^* = 0$.
- Delete the bystander.** Using §32.6’s numbers: (i) classify $x_{\text{test}} = (0, 0)$ via the support-vector sum $\sum_{i:\alpha_i>0} \alpha_i y^{(i)} x^{(i)T} x_{\text{test}} + b^*$; (ii) delete $x^{(1)}$ from the dataset and argue (no re-solving needed) that w^*, b^* and the prediction are unchanged — which KKT condition guarantees this?
- Three linear classifiers.** List three similarities and three differences between the perceptron, logistic regression, and the hard-margin SVM (practice assignment, Q8).

Solutions — Chapter 32: Hard-margin SVM

Full worked solutions to the Chapter 32 problem set. Every number recomputed independently (hand algebra cross-checked in numpy; see the review log).

Problem 1 — Width and supporting lines

Given $w^* = (0.8, 0.4)$, $b^* = -0.2$.

(i) **Width.** $\|w^*\|^2 = 0.64 + 0.16 = 0.8 = 4/5$, so $\|w^*\| = 2/\sqrt{5}$. Width $= 2/\|w^*\| = 2/(2/\sqrt{5}) = \boxed{\sqrt{5} \approx 2.236}$.

(ii) **The three lines.** Supporting hyperplanes $w^{*T}x + b^* = \pm 1$:

- $0.8x_1 + 0.4x_2 - 0.2 = 1 \Rightarrow 0.8x_1 + 0.4x_2 = 1.2 \Rightarrow \boxed{2x_1 + x_2 = 3}$ (multiply by 2.5),
- $0.8x_1 + 0.4x_2 - 0.2 = -1 \Rightarrow 0.8x_1 + 0.4x_2 = -0.8 \Rightarrow \boxed{2x_1 + x_2 = -2}$.

Boundary $w^{*T}x + b^* = 0$: $0.8x_1 + 0.4x_2 = 0.2 \Rightarrow \boxed{2x_1 + x_2 = 0.5}$.

(iii) **Distance of $x^{(1)} = (2, 0)$ from the boundary.** Distance from (x_1, x_2) to $2x_1 + x_2 - 0.5 = 0$ is $|2x_1 + x_2 - 0.5|/\sqrt{2^2 + 1^2}$. At $(2, 0)$: $|4 - 0.5|/\sqrt{5} = 3.5/\sqrt{5} \approx \boxed{1.565}$. Half-width = $\sqrt{5}/2 \approx 1.118$. Since $1.565 > 1.118$, the point lies strictly beyond the $+1$ supporting line — consistent with its functional margin $1.4 > 1$ (geometric margin $1.4/\|w^*\| = 1.4\sqrt{5}/2 = 0.7\sqrt{5} \approx 1.565$, same number both ways).

Problem 2 — Scaling, canonical and otherwise

(i) Wall from $(2w, 2b)$: $\{x : 2w^T x + 2b = 0\} = \{x : w^T x + b = 0\}$ — same set, same wall. Functional margin: $y^{(i)}(2w^T x^{(i)} + 2b) = 2y^{(i)}(w^T x^{(i)} + b)$ — doubled. Geometric margin: $\frac{2y^{(i)}(w^T x^{(i)} + b)}{\|2w\|} = \frac{y^{(i)}(w^T x^{(i)} + b)}{\|w\|}$ — unchanged. (If (w, b) already satisfied the canonical constraints $y^{(i)}(w^T x^{(i)} + b) \geq 1$, doubling keeps them satisfied a fortiori; either way, the wall is unchanged.)

(ii) One line: the canonical condition fixes the ruler — it selects the *unique* scale (w, b) per wall whose nearest point has functional margin exactly 1, so the primal’s minimizer is a single (w, b) , not a ray.

(iii) Rescaling w alone changes the wall itself: $\{x : (tw)^T x + b = 0\} \neq \{x : w^T x + b = 0\}$ for $t \neq 1$ — the normal direction is the same but the offset-to-normal ratio changes, so the boundary moves. Worse, the canonical constraints $y^{(i)}(tw^T x^{(i)} + b) \geq 1$ now describe a *different* feasible set, and minimizing $\frac{1}{2}\|tw\|^2$ over it no longer finds the max-margin wall (practice assignment, Q3: the scaled weight vector “would not be a solution to the primal problem”).

Problem 3 — The symmetric square

(i) **Dual by symmetry.** $x^{(1)} = (1, 1), y = +1$; $x^{(2)} = (1, -1), y = +1$; $x^{(3)} = (-1, 1), y = -1$; $x^{(4)} = (-1, -1), y = -1$. Set $\alpha_i = \alpha$ (dual constraint $\sum \alpha_i y^{(i)} = \alpha + \alpha - \alpha - \alpha = 0$). $\sum_i \alpha_i = 4\alpha$. Quadratic coefficient: $\sum_{i,j} y^{(i)} y^{(j)} x^{(i)T} x^{(j)} = \|\sum_i y^{(i)} x^{(i)}\|^2$ with $\sum_i y^{(i)} x^{(i)} = (1, 1) + (1, -1) + (-1, 1) + (-1, -1) = (0, 0)$, norm-squared 0. So $D(\alpha) = 4\alpha - \frac{1}{2} \cdot 16\alpha^2 = \boxed{4\alpha - 8\alpha^2}$, maximized at $\alpha^* = 4/16 = \boxed{1/4}$, $D_{\max} = 1 - 1/2 = 0.5$.

(ii) **Recover.** $w^* = \frac{1}{4}(4, 0) = \boxed{(1, 0)}$. From $x^{(1)}$: $1 \cdot (1 + b) = 1 \Rightarrow \boxed{b^* = 0}$ (check $x^{(3)}$: $(-1)(-1 + 0) = 1$). Width = $2/\|(1, 0)\| = \boxed{2}$. Supporting hyperplanes $\boxed{x_1 = 1}$ and $\boxed{x_1 = -1}$; boundary $\boxed{x_1 = 0}$.

(iii) Functional margins: $x^{(1)} : 1$; $x^{(2)} : 1$; $x^{(3)} : (-1)(-1) = 1$; $x^{(4)} : (-1)(-1) = 1$ — all exactly 1, all active, all $\alpha_i = 1/4 > 0$: **all four points are support vectors**. Primal value $\frac{1}{2}\|w^*\|^2 = 0.5 = \text{dual } 0.5$.

Problem 4 — Complementary slackness, both directions

(i) $\alpha_i^*(1 - y^{(i)}(w^{*T} x^{(i)} + b^*)) = 0$ with $\alpha_i^* > 0$: divide by the positive α_i^* to get $1 - y^{(i)}(w^{*T} x^{(i)} + b^*) = 0$, i.e. $\boxed{y^{(i)}(w^{*T} x^{(i)} + b^*) = 1}$ — the point lies exactly on its supporting hyperplane.

(ii) Complementary slackness constrains only the *product*. If $y^{(i)}(w^{*T} x^{(i)} + b^*) = 1$ (point on the supporting hyperplane), then the second factor is 0 and the product is 0 *whatever* α_i^* is — including $\alpha_i^* = 0$. So a point can sit on the margin line yet carry no vote. In the assignment’s words: every support vector lies on a supporting hyperplane, but not every point on a supporting hyperplane is a support vector; the implication $\alpha_i > 0 \Rightarrow (\text{on margin})$ holds, the converse does not.

Problem 5 — Delete the bystander

(i) Support-vector sum at $x_{\text{test}} = (0, 0)$:

$$\sum_{i:\alpha_i>0} \alpha_i y^{(i)} x^{(i)T} x_{\text{test}} + b^* = 0.4 \cdot (+1) \cdot 0 + 0.4 \cdot (-1) \cdot 0 + (-0.2) = -0.2,$$

$\text{sign}(-0.2) = \boxed{-1}$ — matches §32.6’s direct computation.

(ii) $x^{(1)}$ had $\alpha_1^* = 0$, so it appears nowhere in $w^* = \sum_i \alpha_i^* y^{(i)} x^{(i)}$. Deleting it: the remaining $(w^*, b^*, \alpha_2^*, \alpha_3^*)$ still satisfies every KKT condition on the reduced dataset — stationarity is unchanged (the deleted term was zero), primal feasibility and complementary slackness only *drop* a constraint, dual feasibility is untouched. By KKT sufficiency for this convex problem (§13.10), the reduced problem’s optimum is unchanged: same w^*, b^* , same prediction -1 . The guaranteeing condition is the KKT system as a whole — concretely, complementary slackness ($\alpha_1 = 0$ for the inactive constraint) is what certifies the point was irrelevant.

Problem 6 — Three linear classifiers

Similarities (practice assignment, Q8): (1) all three are *linear* models; (2) the decision boundary is $w^T x + b = 0$ for all three; (3) all three are *discriminative* — they model the boundary directly, with no model of how the data was generated.

Differences (practice assignment, Q8): (1) logistic regression attaches an explicit probability to each point, and confidence grows with distance from the boundary — farther means surer; the perceptron and SVM have no such notion. (2) The SVM is the *principled* choice of wall — the max-margin one — and “max-margin classifiers will generalize better than perceptrons”, which just take the first wall that works. (3) The hard-margin SVM demands separable data and is not robust to outliers (the soft-margin formulation fixes this); the perceptron is likewise only guaranteed to converge in the separable case, while logistic regression is more forgiving with outliers.

Chapter 33: Soft-margin SVM and the kernel trick

Part IV — Machine Learning Techniques. Builds on: the hard-margin SVM (Ch 32 — primal, dual, KKT, support vectors), kernel definitions and the Gram matrix (§24.10), the kernel pattern (§27.10(ii)). Sources: MLT Week 8 lecture slides (soft-margin primal, slack variables, the hinge-loss form, kernel SVMs), the MLT Week 11 TA notes (soft-margin dual with the box constraint, the KKT case analysis).

33.1 The lecture’s closing question

Chapter 32 ended (§32.8) with the lecture’s uncomfortable question: *how do we adapt the SVM when the data has outliers?* Two weaknesses force it:

- i) **Infeasible on non-separable data** (§32.3). The hard-margin constraints $y^{(i)}(w^T x^{(i)} + b) \geq 1$ demand *every* point on the correct side with margin ≥ 1 . One mislabelled point and the feasible set is empty — no answer at all.
- ii) **Outlier sensitivity** (§32.7(i)). Even when the data *is* separable, a single outlier can wreck the answer.

eg (one outlier collapses the street). Take the symmetric square of Chapter 32, Problem 3 — $x^{(1)} = (1, 1)$, $x^{(2)} = (1, -1)$ labelled +1; $x^{(3)} = (-1, 1)$, $x^{(4)} = (-1, -1)$ labelled -1 — whose hard-margin answer is the clean street $w^* = (1, 0)$, $b^* = 0$, width 2. Now drop one outlier into it: $x^{(5)} = (0.5, 0)$, $y^{(5)} = -1$, a -1 point sitting on the +1 side. The hard-margin dual (no box yet — §32.4’s dual) solves to $\alpha^* = (4, 4, 0, 0, 8)$:

$$w^* = 4(1, 1) + 4(1, -1) + 8(-1)(0.5, 0) = (4, 0), \quad b^* = -3$$

(check: $y^{(1)}(w^{*T} x^{(1)} + b^*) = 4 - 3 = 1$; $y^{(5)}(w^{*T} x^{(5)} + b^*) = -1(2 - 3) = 1$; points 3, 4 sit at margin $7 \geq 1$ with $\alpha = 0$). The street is now $x_1 \in [0.5, 1]$ — **width** $2/\|w^*\| = 0.5$, down from 2. One point moved the wall from $x_1 = 0$ to $x_1 = 0.75$ and quartered the margin. (A fine grid search over the symmetric feasible set, plus a full 5-D numerical solve — both confirm $D = 8$ is the maximum; see the review log.)

The lecture’s own warning, quoted in §32.8: “kernels can help but is not the right way to solve this.” Kernels bend the wall; they don’t forgive bad points. The answer to outliers is a different one — the **soft-margin** primal.

Basically, ... “Hard-margin SVM is a strict teacher: one kid out of line and class is cancelled (infeasible) — or, if the kid technically fits, the whole seating plan gets rearranged around him (the street collapses from width 2 to width 0.5). The soft margin is the teacher learning to tolerate a little mischief.”

33.2 Slack variables: the bribe

The slide’s fix: give every training point its own **slack variable** $\xi_i \geq 0$ — the “bribe” (§32.8) the point pays to violate the margin. The constraint relaxes from “functional margin ≥ 1 ” to:

$$y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i$$

Each point must now sit only at “distance” $1 - \xi_i$ from the wall (slide 30) instead of the hard distance 1. What ξ_i says about the point:

- i) $\xi_i = 0$ — the point is on or beyond its supporting hyperplane. No bribe paid; behaves exactly like a hard-margin point.
- ii) $0 < \xi_i \leq 1$ — the point slipped *inside* the street but is still on the correct side of the wall. Small bribe.
- iii) $\xi_i > 1$ — the point crossed the wall itself: misclassified. Large bribe.

Note (a slide typo). Slide 30 writes the slack condition as $\xi^{(i)} > 0$; slide 32 corrects it to $\xi^{(i)} \geq 0$. The $\geq$ is the right one — a well-behaved point must be allowed $\xi_i = 0$.

Basically, ... “Slack is a hall pass. $\xi_i = 0$ means the point stayed in its seat (no pass needed). A small ξ_i means it wandered into the aisle but stayed on its side. $\xi_i > 1$ means it crossed into enemy territory. Every pass costs money — which is what the primal charges for.”

33.3 The soft-margin primal

Bribes can’t be free, or every point would take one. The slide (p. 32) penalizes them in the objective:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i$$

The **regularization constant** C sets the exchange rate between “wide street” and “few bribes” — exactly as the slide presents it:

- i) C **large** — bribes are ruinously expensive. The SVM becomes strict and tries to get every point to the correct side (in the limit, the hard margin back).
- ii) C **small** — bribes are cheap. The SVM slacks and allows many misclassifications or points inside the margin, in exchange for a wider street.

Note (the hinge-loss form). For $C \neq 0$, the optimizer pushes each ξ_i as low as its constraint allows (slide 34): $\xi_i = 1 - y^{(i)}(w^T x^{(i)} + b)$ when the margin is < 1 , else 0. That is $\xi_i = \max(1 - y^{(i)}(w^T x^{(i)} + b), 0)$ — the **hinge loss**, which gets its full treatment in Chapter 35. Plugging it in, the primal becomes the unconstrained problem $\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \max(1 - y^{(i)}(w^T x^{(i)} + b), 0)$.

Note (the endpoints, from the TA notes). $C = 0$ makes every bribe free, so the dual forces all $\alpha_i = 0$ and $w^* = 0$ — the classifier gives up on the data entirely. $C = \infty$ is the hard margin (§33.1’s eg is the $C = \infty$ answer).

Basically, ... “The primal now has two bills: the street-width bill $\frac{1}{2} \|w\|^2$ and the bribe bill $C \sum \xi_i$. C is the exchange rate. Crank it up and bribes cost a fortune — everyone behaves, street be damned. Drop it low and bribes are pocket change — the street stays wide and the troublemakers just pay up.”

33.4 The soft-margin dual: the box constraint

Same four moves as §32.4, one extra multiplier per point. With $\alpha_i \geq 0$ on the margin constraints and $\mu_i \geq 0$ on $\xi_i \geq 0$ (the TA notes’ derivation, Week 11):

$$L(w, b, \xi, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i (1 - \xi_i - y^{(i)}(w^T x^{(i)} + b)) - \sum_{i=1}^n \mu_i \xi_i.$$

Stationarity (§11.3):

- i) $\frac{\partial L}{\partial w} = 0 \Rightarrow w^* = \sum_{i=1}^n \alpha_i y^{(i)} x^{(i)}$ — the same vote as §32.4.
- ii) $\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_{i=1}^n \alpha_i y^{(i)} = 0$.
- iii) $\frac{\partial L}{\partial \xi_i} = 0 \Rightarrow C - \alpha_i - \mu_i = 0$, i.e. $\alpha_i + \mu_i = C$. Since $\mu_i \geq 0$, this caps the vote: $0 \leq \alpha_i \leq C$.

The **box constraint**. Substituting back, the ξ_i terms cancel and the dual is:

$$\max_{\substack{0 \leq \alpha_i \leq C \\ \sum_i \alpha_i y^{(i)} = 0}} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

The objective is *identical* to the hard-margin dual (§32.4) — only the feasible set shrank: each α_i now has a ceiling C . No point may cast more than C votes, so no single outlier can drag the whole wall around.

The KKT case analysis (the TA notes' summary page — three kinds of points):

- i) $\alpha_i^* = 0 \Rightarrow \xi_i^* = 0$ and $y^{(i)}(w^{*T} x^{(i)} + b^*) \geq 1$. Safe point beyond the margin: no vote, no bribe.
- ii) $0 < \alpha_i^* < C \Rightarrow \xi_i^* = 0$ and $y^{(i)}(w^{*T} x^{(i)} + b^*) = 1$. On the supporting hyperplane: votes, pays nothing. (These are the only points from which b^* can be read off, as in §32.5(iv).)
- iii) $\alpha_i^* = C \Rightarrow y^{(i)}(w^{*T} x^{(i)} + b^*) \leq 1$ with $\xi_i^* = 1 - y^{(i)}(w^{*T} x^{(i)} + b^*) \geq 0$. Inside the street or misclassified: vote maxed out, pays the bribe.

Read backwards, it is just as useful: margin $> 1 \Rightarrow \alpha_i^* = 0$; margin $= 1 \Rightarrow \alpha_i^* \in [0, C]$; margin $< 1 \Rightarrow \alpha_i^* = C$. So $\alpha_i^* = C$ is the detector for “this point is inside the margin or misclassified” — the TA notes' words: “points where either x_i is incorrectly classified by w^* or barely classified but with margin ≤ 1 .”

Basically, ... “The dual is the same mirror as Chapter 32, except every datapoint's vote is now capped at C . The KKT bouncer sorts points into three lines: no vote and no bribe (safe), a vote but no bribe (on the wall), max vote and pays the bribe (troublemaker). Spot a point with $\alpha_i = C$ and you've found your outlier.”

33.5 Worked example: one outlier, two values of C

The §33.1 dataset: the four square points plus the outlier $x^{(5)} = (0.5, 0)$, $y^{(5)} = -1$.

The hard-margin answer ($C = \infty$) was already computed in §33.1: $\alpha^* = (4, 4, 0, 0, 8)$, $w^* = (4, 0)$, $b^* = -3$, street $x_1 \in [0.5, 1]$, width 0.5. The outlier's vote ($\alpha_5 = 8$) pins the wall.

The soft-margin answer at $C = 1$. Guess from the KKT cases: the outlier hits the cap ($\alpha_5 = C = 1$); the other four sit exactly on the margin, strictly inside the box.

Step 1 — symmetry. The data is symmetric in x_2 , so $\alpha_1 = \alpha_2 =: a$, $\alpha_3 = \alpha_4 =: c$. The constraint $\sum_i \alpha_i y^{(i)} = 0$ gives $2a - 2c - 1 = 0$, i.e. $a - c = 0.5$.

Step 2 — the wall from the on-margin points. Points 1–4 have $0 < \alpha_i < C$, so they sit exactly on the margin: $y^{(1)}(w^T x^{(1)} + b) = 1$ and $y^{(3)}(w^T x^{(3)} + b) = 1$. With $w = a(1, 1) + a(1, -1) + c(1, -1) + c(1, 1) - 1 \cdot (0.5, 0) = (2a + 2c - 0.5, 0)$:

$$y^{(1)}(w^T x^{(1)} + b) = (2a + 2c - 0.5) + b = 1, \quad y^{(3)}(w^T x^{(3)} + b) = (2a + 2c - 0.5) - b = 1.$$

Adding: $2(2a + 2c - 0.5) = 2$, so $w^* = (1, 0)$ and $b^* = 0$ — the clean street is back.

Step 3 — the votes. $2a + 2c = 1.5$ and $a - c = 0.5$ give $\alpha^* = (0.625, 0.625, 0.125, 0.125, 1)$.

Step 4 — verify everything (KKT). Margins $y^{(i)}(w^*T x^{(i)} + b^*)$: points 1–4 exactly 1.0 ($0 < \alpha_i < 1 = C$, $\xi_i = 0$); point 5: $-1(0.5) = -0.5 < 1$ $\xi_5 = 1.5$, $\alpha_5 = C$. Complementary-slackness products $\alpha_i(1 - \xi_i - \text{margin}_i)$: $0.625 \cdot 0$, $0.125 \cdot 0$, $1 \cdot (1 - 1.5 + 0.5) = 0$. Primal $\frac{1}{2}\|w^*\|^2 + C \sum \xi_i = 0.5 + 1.5 = 2.0$; dual $\sum \alpha_i - \frac{1}{2}\alpha^T Q \alpha = 2.5 - 0.5 = 2.0$ (strong duality). A grid search over the symmetric feasible set (plus a full 5-D numerical solve and a 2M-point randomized check) confirms 2.0 is the dual maximum — see the review log.

The moral. Same five points, two exchange rates: at $C = 1$ the outlier is *quarantined* (α_5 capped, pays $\xi_5 = 1.5$, misclassified) and the street is the clean $x_1 = \pm 1$ of width 2; at $C = 10$ (already the $C = \infty$ answer — all $\alpha_i < C$, identical solution at $C = 100$) the outlier *pins* the wall and the street collapses to width 0.5.

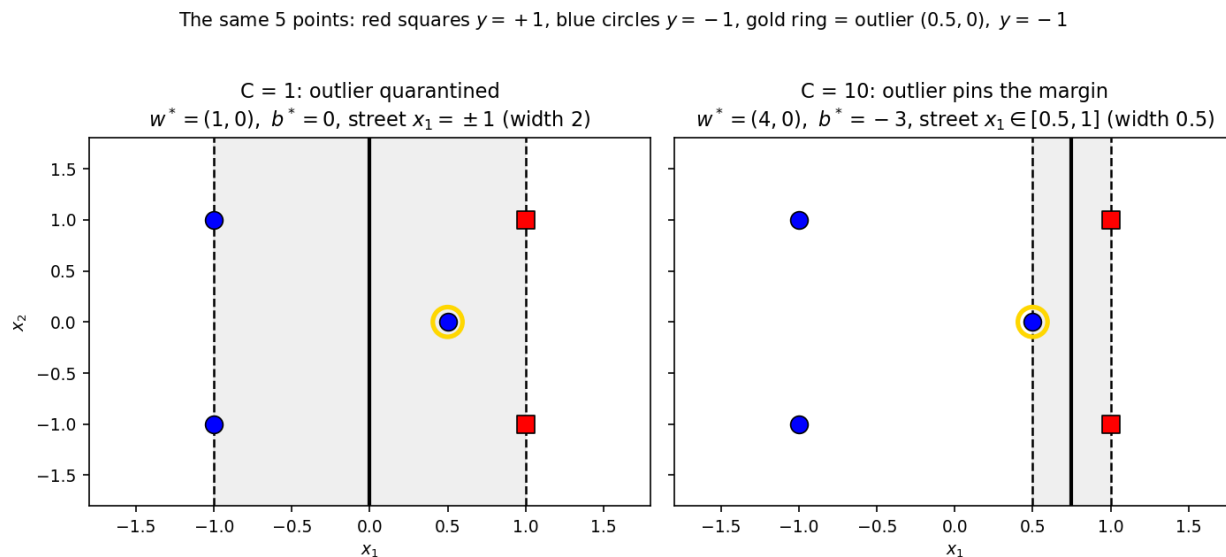


Figure 49: Two panels showing the same five points (red squares $y=+1$, blue circles $y=-1$, gold ring on the outlier $(0.5, 0)$, $y=-1$). Left ($C=1$): the wide street between the dashed lines $x_1=-1$ and $x_1=1$ with the solid boundary $x_1=0$; $w=(1, 0)$, $b=0$, width 2. Right ($C=10$): the narrow street between $x_1=0.5$ and $x_1=1$ with the solid boundary $x_1=0.75$; $w=(4, 0)$, $b=-3$, width 0.5.

Basically, ... “ C is the bouncer’s strictness dial. At $C = 1$ the bouncer lets the troublemaker pay a fine ($\xi_5 = 1.5$) and keeps the wide street. At $C = 10$ the bouncer is so strict the fine might as well be infinite — the troublemaker gets his way and the street shrinks to fit him.”

33.6 The kernel trick for SVMs

The soft-margin dual (§33.4) sees the data **only through inner products** $x^{(i)T} x^{(j)}$ — the same doorway §32.4’s “Basically” pointed at. Suppose the classes aren’t linearly separable at all (no wall in $\mathbb{R}^d$ works, whatever C you pick). The lecture’s second idea: lift $x \mapsto \phi(x)$ into a higher-dimensional space where a wall *does* exist — slide 42’s example $\phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$ turns an elliptical boundary into a flat plane. But explicit ϕ can be brutal (§24.10): cubic features already cost $O(d^3)$, and the RBF kernel’s ϕ is infinite-dimensional.

The kernel trick (§24.10) sidesteps ϕ entirely: the dual never needs $\phi(x^{(i)})$ alone, only $\phi(x^{(i)})^T \phi(x^{(j)})$. So **replace every inner product by a kernel function** $\kappa(x^{(i)}, x^{(j)})$ that computes it directly (slide 44). The kernelized dual:

$$\max_{\substack{0 \leq \alpha_i \leq C \\ \sum_i \alpha_i y^{(i)} = 0}} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y^{(i)} y^{(j)} \kappa(x^{(i)}, x^{(j)})$$

— the same shape as §33.4, with the kernel matrix $K_{ij} = \kappa(x^{(i)}, x^{(j)})$ in place of the Gram matrix. The decision function (slide 46, with κ for the slide's K) needs only support vectors and kernel evaluations — ϕ never appears:

$$\hat{y}(x_{\text{test}}) = \text{sign} \left(\sum_{i: \alpha_i^* > 0} \alpha_i^* y^{(i)} \kappa(x^{(i)}, x_{\text{test}}) + b^* \right)$$

The slide's kernel menu (notation unified with §24.10's κ):

- i) **Linear:** $\kappa(x, z) = x^T z$ — the original SVM (§33.4's dual) back again.
- ii) **Polynomial:** $\kappa(x, z) = (1 + x^T z)^d$ — the wall becomes a degree- d polynomial surface in input space.
- iii) **RBF / Gaussian:** $\kappa(x, z) = \exp(-\|x - z\|^2 / (2\sigma^2))$, $\sigma > 0$ — §24.10's form; its ϕ is infinite-dimensional, yet each kernel evaluation is $O(d)$.

The geometry: a *linear* wall in ϕ -space is a *curved* wall in input space. The algorithm never visits ϕ -space — the kernel evaluations do all the curving.

Basically, ... “The dual only ever asks ‘how similar is point i to point j ?’ as a dot product. The kernel trick swaps in a fancier similarity — one that secretly equals a dot product in a much bigger space. The SVM then draws its straight wall in that bigger space, which looks like a curved wall down here. You get nonlinear boundaries out of a linear algorithm, and you never pay the cost of visiting the bigger space.”

33.7 Worked example: quadratic kernel on three points

Data (1-D, not linearly separable — the -1 point sits between the $+1$ s): $x^{(1)} = -1$, $y^{(1)} = +1$; $x^{(2)} = 0$, $y^{(2)} = -1$; $x^{(3)} = 1$, $y^{(3)} = +1$. Kernel $\kappa(x, z) = (xz + 1)^2$ (the §24.10 polynomial).

Step 1 — the kernel matrix. $K_{11} = ((-1)(-1) + 1)^2 = 4$, $K_{22} = 1$, $K_{33} = 4$, $K_{12} = K_{23} = 1$, $K_{13} = 0$:

$$K = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 4 \end{pmatrix} \quad (\text{eigenvalues} \approx 0.438, 4, 4.562, \text{ all } \geq 0 \text{ — Mercer's condition (b), §24.10})$$

Step 2 — the dual, by symmetry. $\sum_i \alpha_i y^{(i)} = 0$ gives $\alpha_1 - \alpha_2 + \alpha_3 = 0$; the data is symmetric about 0, so $\alpha_1 = \alpha_3 =: a$, $\alpha_2 = 2a$. With $y = (+1, -1, +1)$:

$$\begin{aligned} D(a) &= 4a - \frac{1}{2}(4a^2 + (2a)^2 + 4a^2 - 2a(2a) - 2(2a)a) \\ &= 4a - \frac{1}{2}(4a^2) = 4a - 2a^2. \end{aligned}$$

$D'(a) = 4 - 4a = 0 \Rightarrow \boxed{\alpha^* = (1, 2, 1)}$, $D = 2$. (Grid search over the feasible set confirms 2 is the maximum — review log.)

Step 3 — b^* . From support vector 1 (on the margin): $\sum_j \alpha_j^* y^{(j)} K_{j1} = 1 \cdot 4 + 2(-1) \cdot 1 + 1 \cdot 0 = 2$, so $y^{(1)}(2 + b^*) = 1 \Rightarrow \boxed{b^* = -1}$. Cross-check with support vector 2: $y^{(2)}(1 - 2 + 1 - 1) = (-1)(-1) = 1$.

Step 4 — the decision function. For a test x :

$$\sum_{i=1}^3 \alpha_i^* y^{(i)} \kappa(x^{(i)}, x) = (1-x)^2 - 2(1)^2 + (1+x)^2 = 2x^2,$$

so

$$\hat{y}(x) = \text{sign}(2x^2 - 1).$$

The boundary $2x^2 = 1$ sits at $x = \pm 1/\sqrt{2} \approx \pm 0.707$ — two boundary *points* on the line, i.e. a curved (here: disconnected) boundary in input space. Training check: $\hat{y}(-1) = \text{sign}(1) = +1$, $\hat{y}(0) = \text{sign}(-1) = -1$, $\hat{y}(1) = +1$. A linear SVM in 1-D could never do this; the kernelized one did it with three numbers.

Step 5 — the ϕ -space sanity check. §24.10's explicit map for this kernel (1-D version): $\phi(x) = (x^2, \sqrt{2}x, 1)^T$. The feature-space weight $w_\phi = \sum_i \alpha_i^* y^{(i)} \phi(x^{(i)})$:

$$w_\phi = 1 \cdot (1, -\sqrt{2}, 1) - 2 \cdot (0, 0, 1) + 1 \cdot (1, \sqrt{2}, 1) = (2, 0, 0),$$

and $w_\phi^T \phi(x) + b^* = 2x^2 - 1$ — exactly the decision function. In ϕ -space the wall is the *plane* $2z_1 = 1$; pulled back to the input line it becomes the two points $\pm 1/\sqrt{2}$. We never computed ϕ to train — only to check.

Quadratic kernel $\kappa(x, z) = (xz + 1)^2$: curved boundary in input space = linear wall in ϕ -space

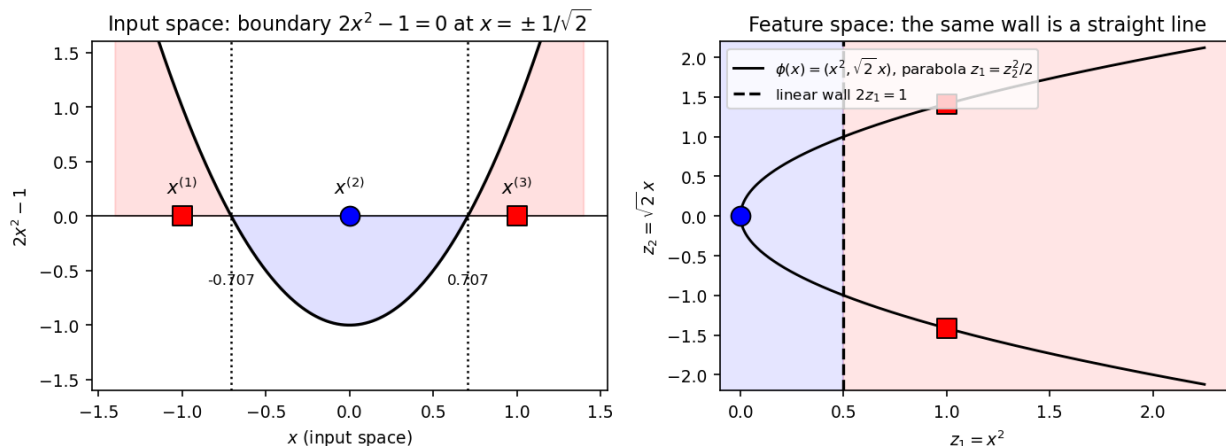


Figure 50: Left: the curve $2x^2 - 1$ over the input line, with the three training points (red squares at $x=-1$ and $x=1$ labelled $+1$, blue circle at $x=0$ labelled -1) and dotted boundary ticks at $x=-0.707$ and $x=+0.707$; the curve is positive (red shaded) outside the ticks and negative (blue shaded) between them. Right: the feature-space parabola $z_1=x^2$ versus $z_2=\sqrt{2}x$ with the three mapped points, cut by the dashed linear wall $2z_1=1$; points right of the wall shaded red ($+1$ region), left of it blue (-1 region).

Basically, ... “Three points on a line, and the middle one is the enemy of both neighbours — no straight cut separates them. The quadratic kernel quietly re-houses each point at $(x^2, \sqrt{2}x)$ on a parabola, where the straight wall $2z_1 = 1$ does the job. Back on the line, that wall shows up as two cut points at ± 0.707 . The SVM never saw the parabola — it only asked the kernel for similarities.”

33.8 The pattern, third time

§27.10(ii) promised this chapter would run the “ $\$w = \$$ (span of data) + Gram matrix K ” pattern a third time. The three appearances, side by side:

- i) **Kernel PCA (§24.10).** The feature-space eigenvector lies in the span of the mapped data: $w_k = X\alpha_k$ (there X holds the $\phi(x_i)$); the eigenproblem becomes $K\alpha_k = (n\lambda_k)\alpha_k$ — data enters only through the Gram matrix K .
- ii) **Kernel ridge (§27.5–§27.6).** The stationary point satisfies $w = \Phi^T\alpha$; substituting gives $(K + \lambda I)\alpha = y$; predictions are $\sum_i \alpha_i \kappa(x_i, x)$.
- iii) **Kernel SVM (this chapter).** The feature-space wall is $w_\phi = \sum_i \alpha_i y^{(i)} \phi(x^{(i)})$ (§33.7, Step 5: $w_\phi = (2, 0, 0)$); the dual (§33.6) touches the data only through $K_{ij} = \kappa(x^{(i)}, x^{(j)})$; predictions are $\text{sign}(\sum_i \alpha_i y^{(i)} \kappa(x^{(i)}, x) + b)$.

Same shape every time: the answer lives in the span of the (mapped) training points, the algorithm manipulates only the $n \times n$ kernel matrix, and swapping the dot product for κ turns a linear method nonlinear — unsupervised (PCA), regression (ridge), classification (SVM).

Basically, ... “Three chapters, one trick. PCA’s directions, ridge’s weights, the SVM’s wall — each turns out to be a weighted vote of the training points, each is computed from the similarity matrix K alone, and each goes nonlinear the moment you let a kernel compute the similarities. Learn the pattern once, recognize it three times.”

33.9 Strengths and limits (as the sources list them)

Strengths.

- i) **Outlier tolerance.** The soft margin learns “a hyperplane with tolerance to a small number of misclassified examples” (slide 28) — the lecture’s closing question answered (§33.1, §33.5).
- ii) **Nonlinear boundaries from a linear algorithm.** The kernel “performs the dot product between input feature vectors in high-dimensional space without actually projecting or transforming the input features in that space” (slide 44) — §33.7’s curved boundary cost three kernel evaluations per prediction, zero ϕ ’s.
- iii) **The margin machinery survives.** The dual keeps §32.7(ii)’s sparsity (support vectors still decide) and the max-margin principle; the box constraint only caps the votes.

Limits.

- i) **Two fixes for two different diseases.** The lecture’s warning stands (§32.8): kernels bend the wall but don’t forgive bad points — “kernels can help but is not the right way to solve this.” Outliers want the soft margin; curved classes want kernels; a dataset with both wants both.
- ii) **Still a quadratic program, still no probabilities.** §32.7(ii)–(iii) carry over unchanged — the dual is the same shape, and distance from the wall is still not a confidence.
- iii) **New knobs, no selection rule in the sources.** C and the kernel’s parameters (d , σ) now have to be chosen; the slides list the kernels without saying how (§27.9’s practical notes discuss the analogous choice for kernel ridge).

Basically, ... “Soft margin + kernels = the SVM grows up: it tolerates bad points and bends around curved classes, while staying sparse and principled. The price: you now tune C and pick a kernel with no recipe in the slides, you still solve a QP, and the wall still won’t tell you how sure it is.”

33.10 Where this goes next

Two pointers. **Chapter 34 (ensembles)** also obsesses over hard-to-classify points — but where the soft margin *tolerates* them via slack, AdaBoost *reweights* them and tries again. Same enemy, opposite tactic. **Chapter 35 (loss functions)** gives the hinge loss $\max(0, 1 - y \cdot \text{score})$ — the ξ_i of §33.3 — its full treatment alongside the other classification losses.

Problem set

- Reading the slack.** Take the §33.5 solution at $C = 1$ ($w^* = (1, 0)$, $b^* = 0$). (i) Compute $\xi_i = \max(1 - y^{(i)}(w^{*T}x^{(i)} + b^*), 0)$ for all five points. (ii) Put each point into one of §33.2's three regimes ($\xi_i = 0$; $0 < \xi_i \leq 1$; $\xi_i > 1$). (iii) Which point pays the largest bribe, and what is it?
- The box, derived.** Starting from the soft-margin primal, write the Lagrangian with multipliers $\alpha_i \geq 0$ (margin constraints) and $\mu_i \geq 0$ ($\xi_i \geq 0$). (i) Compute $\partial L / \partial w$, $\partial L / \partial b$, $\partial L / \partial \xi_i$ and state the three stationarity conditions. (ii) Show $\alpha_i + \mu_i = C$ and deduce the box constraint $0 \leq \alpha_i \leq C$. (iii) Write the final dual.
- Three α regimes.** Using complementary slackness $\alpha_i^*(1 - \xi_i^* - y^{(i)}(w^{*T}x^{(i)} + b^*)) = 0$ and $\mu_i^*\xi_i^* = (C - \alpha_i^*)\xi_i^* = 0$: (i) prove a point with functional margin > 1 has $\alpha_i^* = 0$; (ii) prove $0 < \alpha_i^* < C$ forces the margin to equal 1 with $\xi_i^* = 0$; (iii) prove a point with margin < 1 has $\alpha_i^* = C$.
- The C endpoints.** (i) Argue from the primal that as $C \rightarrow \infty$, any feasible solution with some $\xi_i > 0$ becomes infinitely costly, so the soft-margin primal recovers the hard-margin problem. (ii) The TA notes say $C = 0$ gives $w^* = 0$ — explain why from the dual's box constraint.
- Gram matrix and Mercer.** Points $p_1 = (0, 0)$, $p_2 = (1, 0)$, $p_3 = (0, 1)$; kernel $\kappa(x, z) = (x^T z + 1)^2$. (i) Write the 3×3 kernel matrix K . (ii) Find its eigenvalues (hint: try $(0, 1, -1)^T$ and vectors of the form $(1, t, t)^T$). (iii) Conclude from §24.10's Mercer condition (b) that κ is a valid kernel.
- A two-point kernel dual.** $x^{(1)} = -1$, $y^{(1)} = +1$; $x^{(2)} = 1$, $y^{(2)} = -1$; $\kappa(x, z) = (xz + 1)^2$ (hard-margin form, $\alpha_i \geq 0$). (i) Write K and use $\alpha_1 = \alpha_2 =: \alpha$ (from $\sum_i \alpha_i y^{(i)} = 0$) to get the dual $D(\alpha) = 2\alpha - 4\alpha^2$; maximize it. (ii) Recover b^* . (iii) Simplify the decision function to $\text{sign}(-x)$ and classify $x = -0.5$ and $x = 0.5$.
- True or false, one-line justification:** (i) The soft-margin primal is feasible for every dataset. (ii) $\alpha_i^* = C$ implies point i is misclassified. (iii) The kernelized dual ever needs the explicit $\phi(x^{(i)})$. (iv) A larger C makes the SVM more tolerant of outliers. (v) With the linear kernel $\kappa(x, z) = x^T z$, the kernelized dual is exactly §32.4's hard-margin dual.

Solutions — Chapter 33: Soft-margin SVM and the kernel trick

Full worked solutions to the Chapter 33 problem set. Every number recomputed independently (hand algebra cross-checked in numpy; see the review log).

Problem 1 — Reading the slack

Given $w^* = (1, 0)$, $b^* = 0$; points $x^{(1)} = (1, 1)$, $x^{(2)} = (1, -1)$ ($y = +1$); $x^{(3)} = (-1, 1)$, $x^{(4)} = (-1, -1)$ ($y = -1$); $x^{(5)} = (0.5, 0)$ ($y = -1$).

(i) **The slacks.** $\xi_i = \max(1 - y^{(i)}(w^{*T}x^{(i)} + b^*), 0)$:

- Point 1: $1 - (+1)(1) = 0 \Rightarrow \boxed{\xi_1 = 0}$.
- Point 2: $1 - (+1)(1) = 0 \Rightarrow \boxed{\xi_2 = 0}$.
- Point 3: $1 - (-1)(-1) = 1 - 1 = 0 \Rightarrow \boxed{\xi_3 = 0}$.
- Point 4: $1 - (-1)(-1) = 0 \Rightarrow \boxed{\xi_4 = 0}$.
- Point 5: $1 - (-1)(0.5) = 1 + 0.5 = 1.5 \Rightarrow \boxed{\xi_5 = 1.5}$.

(ii) **Regimes.** Points 1–4: $\xi_i = 0$ (on the supporting hyperplanes, no bribe). Point 5: $\xi_5 = 1.5 > 1$ (crossed the wall — misclassified).

(iii) The outlier pays the largest bribe: $\boxed{\xi_5 = 1.5}$.

Problem 2 — The box, derived

Primal: $\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$ s.t. $1 - \xi_i - y^{(i)}(w^T x^{(i)} + b) \leq 0$ and $-\xi_i \leq 0$.

Lagrangian ($\alpha_i \geq 0, \mu_i \geq 0$):

$$L = \frac{1}{2}\|w\|^2 + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i (1 - \xi_i - y^{(i)}(w^T x^{(i)} + b)) - \sum_{i=1}^n \mu_i \xi_i.$$

(i) **Stationarity.**

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^n \alpha_i y^{(i)} x^{(i)} = 0 \implies \boxed{w^* = \sum_{i=1}^n \alpha_i y^{(i)} x^{(i)}},$$

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^n \alpha_i y^{(i)} = 0 \implies \boxed{\sum_{i=1}^n \alpha_i y^{(i)} = 0},$$

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \implies \boxed{\alpha_i + \mu_i = C}.$$

(ii) Dual feasibility gives $\mu_i \geq 0$, so $\alpha_i = C - \mu_i \leq C$; with $\alpha_i \geq 0$: $\boxed{0 \leq \alpha_i \leq C}$.

(iii) Substituting w^* (the ξ_i terms cancel via $\alpha_i + \mu_i = C$), the dual is

$$\boxed{\max_{\substack{0 \leq \alpha_i \leq C \\ \sum_i \alpha_i y^{(i)} = 0}} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}}.$$

Problem 3 — Three α regimes

KKT at the optimum: (a) $\alpha_i^*(1 - \xi_i^* - m_i) = 0$ where $m_i = y^{(i)}(w^{*T} x^{(i)} + b^*)$; (b) $\mu_i^* \xi_i^* = (C - \alpha_i^*) \xi_i^* = 0$; (c) primal feasibility $m_i \geq 1 - \xi_i^*, \xi_i^* \geq 0$.

(i) $m_i > 1 \Rightarrow \alpha_i^* = 0$. Suppose $\alpha_i^* > 0$. Then (a) gives $\xi_i^* = 1 - m_i < 0$, contradicting $\xi_i^* \geq 0$. Hence $\boxed{\alpha_i^* = 0}$.

(ii) $0 < \alpha_i^* < C \Rightarrow m_i = 1, \xi_i^* = 0$. Since $\alpha_i^* < C, \mu_i^* = C - \alpha_i^* > 0$, so (b) gives $\boxed{\xi_i^* = 0}$. Since $\alpha_i^* > 0$, (a) gives $1 - 0 - m_i = 0$, i.e. $\boxed{m_i = 1}$.

(iii) $m_i < 1 \Rightarrow \alpha_i^* = C$. Suppose $\alpha_i^* < C$. Then $\mu_i^* > 0$, so (b) gives $\xi_i^* = 0$; primal feasibility (c) then needs $m_i \geq 1 - 0 = 1$, contradicting $m_i < 1$. Hence $\boxed{\alpha_i^* = C}$.

Problem 4 — The C endpoints

(i) $C \rightarrow \infty$. The objective is $\frac{1}{2}\|w\|^2 + C \sum_i \xi_i$. Any candidate with some $\xi_i > 0$ has objective $\rightarrow +\infty$ as $C \rightarrow \infty$, while (when the data is separable) feasible hard-margin walls with all $\xi_i = 0$ keep a finite objective. The minimizer is therefore forced to $\xi_i = 0$ for all i , and the problem becomes $\boxed{\min_{w,b} \frac{1}{2}\|w\|^2 \text{ s.t. } y^{(i)}(w^T x^{(i)} + b) \geq 1}$

— the hard-margin primal (§32.3).

(ii) $C = 0$. The box collapses to $0 \leq \alpha_i \leq 0$, so every $\alpha_i^* = 0$. Then $w^* = \sum_i \alpha_i^* y^{(i)} x^{(i)} = \boxed{0}$ — with free bribes, the cheapest answer ignores the data entirely (the TA notes’ “ $C = 0 \Rightarrow w^* = 0$ ”).

Problem 5 — Gram matrix and Mercer

(i) Dots: $p_1^T p_1 = 0$, $p_2^T p_2 = p_3^T p_3 = 1$, all cross dots 0. $K_{ij} = (\text{dot} + 1)^2$:

$$K = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 4 & 1 \\ 1 & 1 & 4 \end{pmatrix}.$$

(ii) Try $v = (0, 1, -1)^T$: $Kv = (0, 3, -3)^T = 3v$, so $\boxed{\lambda = 3}$. Try $v = (1, t, t)^T$: $Kv = (1+2t, 1+5t, 1+5t)^T \stackrel{!}{=} \lambda(1, t, t)^T$ gives $1 + 2t = \lambda$ and $1 + 5t = \lambda t$. Substituting: $1 + 5t = t + 2t^2$, i.e. $2t^2 - 4t - 1 = 0$, so $t = 1 \pm \frac{\sqrt{6}}{2}$ and $\lambda = 1 + 2t = 3 \pm \sqrt{6}$. Eigenvalues: $\boxed{3, 3 + \sqrt{6} \approx 5.449, 3 - \sqrt{6} \approx 0.551}$ (numpy confirms: 0.55051, 3, 5.44949).

(iii) K is symmetric with all eigenvalues ≥ 0 for this dataset — Mercer condition (b) of §24.10 — consistent with $\kappa(x, z) = (x^T z + 1)^2$ being a valid kernel (indeed $\kappa(x, z) = \phi(x)^T \phi(z)$ with an explicit ϕ , §24.10's eg).

Problem 6 — A two-point kernel dual

(i) $K_{11} = ((-1)(-1) + 1)^2 = 4$, $K_{22} = 4$, $K_{12} = ((-1)(1) + 1)^2 = 0$: $K = \begin{pmatrix} 4 & 0 \\ 0 & 4 \end{pmatrix}$. $\alpha_1 - \alpha_2 = 0$ gives $\alpha_1 = \alpha_2 =: \alpha \geq 0$; $D(\alpha) = 2\alpha - \frac{1}{2}(4\alpha^2 + 4\alpha^2) = \boxed{2\alpha - 4\alpha^2}$. $D'(\alpha) = 2 - 8\alpha = 0 \Rightarrow \boxed{\alpha^* = 1/4}$.

(ii) $\sum_j \alpha_j^* y^{(j)} K_{j1} = \frac{1}{4} \cdot 4 + \frac{1}{4}(-1) \cdot 0 = 1$; $y^{(1)}(1 + b^*) = 1 \Rightarrow \boxed{b^* = 0}$.

(iii) $\sum_i \alpha_i^* y^{(i)} \kappa(x^{(i)}, x) = \frac{1}{4}(1-x)^2 - \frac{1}{4}(1+x)^2 = \frac{1}{4}(-4x) = -x$, so $\boxed{\hat{y}(x) = \text{sign}(-x)}$. Then $\hat{y}(-0.5) = \boxed{+1}$ (matches $y^{(1)}$) and $\hat{y}(0.5) = \boxed{-1}$ (matches $y^{(2)}$).

Problem 7 — True or false

- (i) **True.** For any (w, b) , $\xi_i = \max(1 - y^{(i)}(w^T x^{(i)} + b), 0)$ satisfies both constraint sets — e.g. $(w, b) = (0, 0)$ is always feasible, unlike the hard-margin primal.
- (ii) **False.** $\alpha_i^* = C$ means margin ≤ 1 (§33.4(iii)) — that includes $0 < \text{margin} \leq 1$, i.e. inside the street but correctly classified. Misclassification needs margin ≤ 0 .
- (iii) **False.** The kernelized dual (§33.6) and the decision function use only kernel evaluations $\kappa(x^{(i)}, x^{(j)})$; ϕ never appears (that is the whole point of the trick).
- (iv) **False.** Larger C = stricter (§33.3(i), §33.5: at $C = 10$ the outlier pins the wall). Tolerance grows as C *shrinks*.
- (v) **True.** $\kappa(x, z) = x^T z$ makes $K_{ij} = x^{(i)T} x^{(j)}$ the ordinary Gram matrix, and the kernelized dual becomes §32.4's dual (with the $0 \leq \alpha_i \leq C$ box for the soft-margin version, without it for hard margin).

Chapter 34: Ensembles — bagging, boosting, AdaBoost

Everything in §§34.2–34.4 comes from the MLT Week-11 materials (revision slides and practice assignment), ISL §8.2.1, ESL §8.7, and the MITx 6.036 notes ch. 14. §§34.5–34.6 follow the MLT Week-11 revision slides and practice assignment (Q5’s AdaBoost weight update), with the weighted-majority-vote form from ESL ch. 14; §34.8’s contrast and the boosting tuning parameters from ISL §8.2.3. The worked example in §34.7 and the figures use the chapter’s own tiny datasets, with every number recomputed independently (review log).

Notation. The §22.9 convention: instances $x^{(i)}$, labels $y^{(i)} \in \{+1, -1\}$. $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ is the training set. Weak classifiers are written $G_m(x) \in \{+1, -1\}$ (ESL’s notation), and the sample weights in AdaBoost round m are $D_m(i)$, following the MLT practice assignment’s notation.

34.1 The ensemble idea — the answer to trees’ variance problem

§29.21(ii) left a loose end: decision trees have **high variance** — small changes in the data reshuffle the splits, so the fitted tree swings with the sample. §29.22 promised that ensembles attack exactly this weakness. Here is how.

Def (ensemble method). An approach that combines many simple “building block” models to obtain a single, potentially very powerful model (ISL §8.2). The building blocks are sometimes called **weak learners** — models that give mediocre predictions on their own.

- i) **Why combining helps: variance.** §28.9’s picture: high variance = the fit is over-sensitive to *this particular* training sample. Now recall §21.5’s miracle fact: for n independent copies, $\text{Var}(\bar{X}_n) = \sigma^2/n$ — **averaging shrinks variance**. An ensemble exploits exactly this: train many jittery models on many slightly different datasets, average (or vote) their predictions, and the jitters cancel while the signal survives.
- ii) **The weak-learner bargain.** Each block is cheap and weak — a shallow tree, a stump — but the *combination* is strong. ISL’s framing: the blocks are mediocre alone; the ensemble is the product.
- iii) **Two ways to combine** (the chapter’s two halves): **bagging** (§§34.2–34.4) builds the blocks *independently, in parallel*, and averages them; **boosting** (§§34.5–34.6) builds them *sequentially*, each one fixing the previous one’s mistakes.

Basically, ... “One jittery expert guesses; ask a hundred jittery experts trained on slightly different data and average their answers — the jitters cancel out. Bagging asks them all at once; boosting asks them one after another, each new expert told where the last one went wrong.”

34.2 Bagging: bootstrap aggregation

Def (bootstrap sample). A dataset of size n drawn **with replacement** from the n training points — some points appear twice, some never (ISL §8.2.1; MITx 6.036: B such datasets, each of size n).

The bagging algorithm (ISL §8.2.1; ESL §8.7):

- i) For $b = 1, \dots, B$: draw a bootstrap sample Z_b^* and train one model $\hat{f}_b^*$ on it.
- ii) **Regression:** average them —
$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b^*(x).$$
- iii) **Classification: majority vote** — the class predicted by the most of the B models (equivalently: average the one-hot vote vectors and take the arg max; MITx 6.036, ESL §8.7).

Why the variance drops. ISL’s argument is the §21.5 one: averaging a set of observations divides the variance (σ^2/n). Each $\hat{f}_b^*$ is trained on a different resample, so averaging them smooths out the sample-to-sample swings. ESL makes it precise for squared-error loss: **averaging reduces variance and leaves bias unchanged** — and the “ideal” aggregation (resampling from the true population) *never increases* mean squared error. In ESL’s tree experiment, predictors with high correlation made single trees twitchy; bagging “succeeds in smoothing out this variance and hence reducing the test error.”

- i) **Deep, unpruned trees are the ideal base learner** (ISL): each one has high variance but low bias — exactly the profile averaging fixes. (Contrast §29.18–29.19: a *single* tree needs pruning; a bagged forest of unpruned trees does not.)
- ii) **B is not a critical knob** (ISL): a very large B does *not* lead to overfitting — use B big enough that the error settles down (ISL’s example: $B = 100$ suffices).

eg 1 (the vote beats each voter). Three *independent* classifiers, each with error rate $e = 0.3$. The majority is wrong only if ≥ 2 of 3 are wrong:

$$P(\text{vote wrong}) = \binom{3}{2}(0.3)^2(0.7) + (0.3)^3 = 3(0.09)(0.7) + 0.027 = 0.189 + 0.027 = \boxed{0.216 < 0.3}.$$

Each voter errs 30% of the time; the vote errs 21.6%. (This is ESL’s consensus-vote argument: with independent weak learners each beating 0.5, the majority’s win probability $\rightarrow 1$ as B grows — the “wisdom of crowds.”)

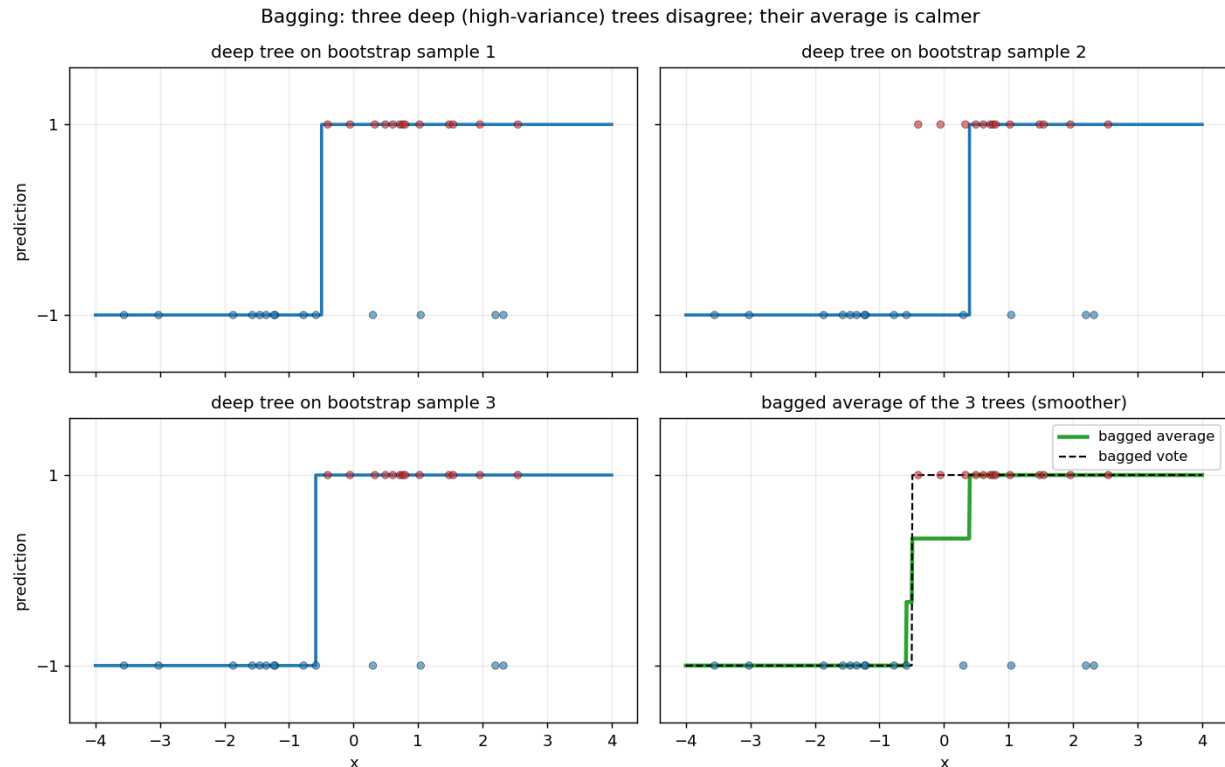


Figure 51: Three deep 1-D trees trained on three different bootstrap resamples of the same noisy data; their step-function predictions disagree on where the splits fall. Bottom right: the bagged average (green staircase, taking fractional values where trees disagree) is calmer, with the dashed black line showing the majority vote.

Basically, ... “Bagging = train the same learner B times on B shuffled-with-replacement copies of your data, then average the answers (or take a vote). Each copy is a slightly different ‘parallel universe’ of your dataset; the average across universes doesn’t twitch when one universe’s noise changes — that’s the variance dying.”

34.3 Bagging needs disagreement — the diversity condition

The MLT Week-11 practice assignment asks exactly when bagging *fails* (Q3): bagging reduces **variance** (not bias), and it is **inefficient if the classifiers are fully correlated** — fully correlated classifiers have the same average output, so aggregating them changes nothing.

- i) **The independence caveat** (ESL): the “wisdom of crowds” math assumes the voters are *independent* — but bagged trees are not; they all see resamples of the same data. The less correlated the trees, the closer the real gain is to the ideal gain.
- ii) **Random forests: bagging + decorrelation** (MLT revision slides; practice Q1). A random forest repeats bagging’s recipe with one twist: each tree is grown on a bootstrap sample **but at every split only a random subset of features is considered**. The revision slides’ steps: 1) create the bootstrapped dataset; 2) grow the tree on it using a random subset of features at each split; 3) repeat — hundreds of trees.
- iii) **The m -knob trade-off** (practice Q1’s answer): *decreasing* the number of features considered per split *reduces the correlation* between trees (a “wide variety of trees” — revision slides), while *increasing* it *improves each individual tree* (more features to choose the split from). The revision slides’ practical rule: start with $\sqrt{d}$ features and try different values.

Note: the sources cover random forests only this far — a decorrelated-bagging variant. The chapter’s topic

stays bagging and boosting; forests appear only where the MLT assignment puts them.

Basically, ... “Averaging only helps if the things you average actually disagree. Ten copies of the same tree vote identically — no gain. Random forests force disagreement on purpose: each tree is only allowed to look at a random handful of features at each split, so the trees come out different and the average has something to smooth.”

34.4 Out-of-bag error: a free test-error estimate

Bagging hands you a validation set for free (ISL §8.2.1):

- i) **Each tree sees only $\sim 2/3$ of the data.** A bootstrap sample leaves about one-third of the observations unused — these are the **out-of-bag (OOB)** observations for that tree.
- ii) **OOB prediction for point i :** predict it using *only* the trees for which it was OOB (about $B/3$ of them), then average (regression) or vote (classification).
- iii) **OOB error $\approx$ test error:** since each point is predicted only by trees that never saw it, the OOB error is a valid estimate of the test error — with B large it is virtually equivalent to leave-one-out cross-validation. Handy when real cross-validation would be computationally onerous.

eg 2 (OOB counting). $n = 100$ points, $B = 300$ trees. A given point lands in a given bootstrap sample with probability $1 - (1 - 1/100)^{100} \approx 1 - 1/e \approx 0.632$ — so each tree trains on ≈ 63 of the 100 points (“around two-thirds,” ISL) and point i is OOB in $\approx 300 \times 0.368 \approx \boxed{110}$ trees, each contributing one vote to its OOB prediction.

Basically, ... “Every bootstrap sample accidentally leaves out a third of your data. Those left-out points are a built-in test set for that tree — predict each point using only trees that never trained on it, and you get an honest test-error estimate without holding out anything.”

34.5 Boosting: the sequential idea — fix mistakes, don’t average them away

The MLT revision slides state the whole philosophy in two lines: **first build a model on the training data; then build a second model to rectify the errors of the first.** ISL §8.2.3 sharpens the contrast with bagging:

- i) **Sequential, not parallel.** Boosting grows trees *one after another*, each tree using information from the previously grown trees. There is **no bootstrap sampling** — each tree is fit on a *modified version of the original dataset*.
- ii) **Learn slowly.** ISL’s regression version: fit a small tree to the current *residuals*, add a shrunk copy of it to the model ($\hat{f} \leftarrow \hat{f} + \lambda \hat{f}_b$), update the residuals, repeat. The shrinkage λ (typically 0.01 or 0.001) slows learning further — and “statistical learning approaches that learn slowly tend to perform well.”
- iii) **Small trees suffice.** Because each tree sees what the earlier ones missed, deep trees are unnecessary — stumps ($d = 1$ split) often work, and a stump ensemble is an *additive* model (interpretable, unlike bagging’s forest).

Def (weak learner). A classifier only **slightly better than a random model** (revision slides) — error rate just below 0.5 (ESL). **Def (stump).** A decision tree with a single split (revision slides).

Basically, ... “Bagging asks a hundred experts independently and averages. Boosting hires one weak expert, looks at exactly which questions it got wrong, hires the next expert *specifically to fix those*, and so on — learning slowly, mistake by mistake. No resampling lottery; the dataset itself is reweighted so the hard questions shout louder each round.”

34.6 AdaBoost: adaptive boosting, the algorithm

Setup: binary labels $y^{(i)} \in \{+1, -1\}$ (§22.9). The base learners $G_m(x) \in \{+1, -1\}$ are usually stumps. The course’s version (revision slides + practice-assignment Q5 solution):

The AdaBoost algorithm.

- i) **Initialize.** $D_0(i) = 1/n$ — equal importance to every sample for the first stump.
- ii) **For** $m = 1, \dots, M$: fit the weak classifier G_m to the data *weighted by* D_{m-1} ; compute its weighted error
$$e_m = \sum_i D_{m-1}(i) \mathbf{1}(G_m(x^{(i)}) \neq y^{(i)})$$
 (the revision slides: “error = proportion of incorrectly classified samples”).
- iii) **Score the classifier** (“performance” in the slides):
$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - e_m}{e_m} \right)$$
. A stump with e_m near 0 earns a huge α_m ; one barely better than guessing ($e_m \rightarrow 0.5$) earns $\alpha_m \rightarrow 0$.
- iv) **Reweight the samples** — increase the misclassified, decrease the correct (practice Q5’s solution):

$$w(i) = D_{m-1}(i) e^{+\alpha_m} \text{ if } G_m(x^{(i)}) \neq y^{(i)}, \quad w(i) = D_{m-1}(i) e^{-\alpha_m} \text{ if } G_m(x^{(i)}) = y^{(i)},$$

then
$$D_m(i) = w(i) / \sum_j w(j)$$
 (normalize to sum 1).

- v) **Output the weighted majority vote** (ESL 14.1):
$$G(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$$
 — more accurate classifiers get higher influence.

Why it works (ESL): observations misclassified by G_m get their weights *increased*, correctly classified ones get them *decreased* — so “as iterations proceed, observations that are difficult to classify receive ever-increasing influence,” and each new classifier is forced to concentrate on the points the previous ones missed.

Note (the $\frac{1}{2}$). ESL’s AdaBoost.M1 box writes $\alpha_m = \log((1 - e_m)/e_m)$ *without* the $\frac{1}{2}$; the MLT course’s revision slides and practice-assignment solution both use $\alpha_m = \frac{1}{2} \ln((1 - e_m)/e_m)$. This chapter follows the course’s form throughout (the $\frac{1}{2}$ only rescales all α_m equally, so the vote’s sign is unaffected).

Basically, ... “Give every point an equal vote and build a stump. The stump misses some points — so shout the misses’ names louder (multiply their weights by $e^{+\alpha}$) and whisper the hits (multiply by $e^{-\alpha}$), then build the next stump listening to the shouting. At the end every stump votes, but accurate stumps get more votes (α_m). The points nobody can classify end up screaming — which is exactly what §34.7 watches happen.”

34.7 Hand-worked: three AdaBoost rounds on eight points

The chapter’s own dataset (labels ± 1 , one planted outlier at $x = 4$):

$x^{(i)}$	−3	−2	−1	0	1	2	3	4
$y^{(i)}$	−1	−1	−1	+1	+1	+1	+1	−1

Round 1. $D_0(i) = 1/8$. The best stump is $G_1(x) = -1$ if $x \leq -0.5$, $+1$ otherwise — it misses only $x = 4$. So $e_1 = 1/8$ and

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - 1/8}{1/8} \right) = \frac{1}{2} \ln 7 \approx \boxed{0.9730}.$$

Update: $x = 4$ gets $\frac{1}{8}e^{+\alpha_1} = \frac{1}{8}\sqrt{7}$; the other seven get $\frac{1}{8}e^{-\alpha_1} = \frac{1}{8}/\sqrt{7}$. Normalizer $Z = \frac{1}{8}\sqrt{7} + 7 \cdot \frac{1}{8}/\sqrt{7} = \frac{2\sqrt{7}}{8} = \frac{\sqrt{7}}{4}$. Hence

$$D_1(4) = \frac{\sqrt{7}/8}{\sqrt{7}/4} = \frac{1}{2}, \quad D_1(i) = \frac{1/(8\sqrt{7})}{\sqrt{7}/4} = \frac{1}{14} \ (i \neq 4).$$

Check: $\frac{1}{2} + 7 \cdot \frac{1}{14} = 1$. The outlier now holds half the total weight.

Round 2. Weights: $1/14$ on the first seven points, $1/2$ on $x = 4$. Reusing G_1 would err $1/2$ (it misses only the now-heavy $x = 4$). The winner is the flipped stump $G_2(x) = +1$ if $x \leq 3.5$, -1 otherwise — it misses $-3, -2, -1$: $e_2 = 3/14 \approx 0.2143$,

$$\alpha_2 = \frac{1}{2} \ln \left(\frac{11/14}{3/14} \right) = \frac{1}{2} \ln \left(\frac{11}{3} \right) \approx \boxed{0.6496}.$$

With $s = e^{\alpha_2} = \sqrt{11/3}$: the three missed points get $\frac{1}{14}s$, the five correct get $\frac{1}{14}/s$ (four of them) and $\frac{1}{2}/s$ ($x = 4$). Normalizer $Z = \frac{3}{14}s + \frac{11}{14}/s = \frac{11}{7}/s$ (using $s^2 = 11/3$), giving

$$D_2(-3:-1) = \frac{1}{6} \text{ each}, \quad D_2(0:3) = \frac{1}{22} \text{ each}, \quad D_2(4) = \frac{7}{22}.$$

Check: $3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{22} + \frac{7}{22} = \frac{1}{2} + \frac{11}{22} = 1$.

Round 3. The best stump is $G_3 = G_1$ again (-1 left of -0.5 , $+1$ right) — it misses only $x = 4$: $e_3 = 7/22 \approx 0.3182$,

$$\alpha_3 = \frac{1}{2} \ln \left(\frac{15/22}{7/22} \right) = \frac{1}{2} \ln \left(\frac{15}{7} \right) \approx \boxed{0.3811}.$$

With $s = \sqrt{15/7}$: $Z = \frac{7}{22}s + \frac{15}{22}/s = \frac{15}{11}/s$ (using $s^2 = 15/7$), giving

$$D_3(-3:-1) = \frac{11}{90} \text{ each}, \quad D_3(0:3) = \frac{1}{30} \text{ each}, \quad D_3(4) = \frac{1}{2}.$$

Check: $3 \cdot \frac{11}{90} + 4 \cdot \frac{1}{30} + \frac{1}{2} = \frac{33+12+45}{90} = 1$. The outlier is back at weight $1/2$ — half of all attention on one point.

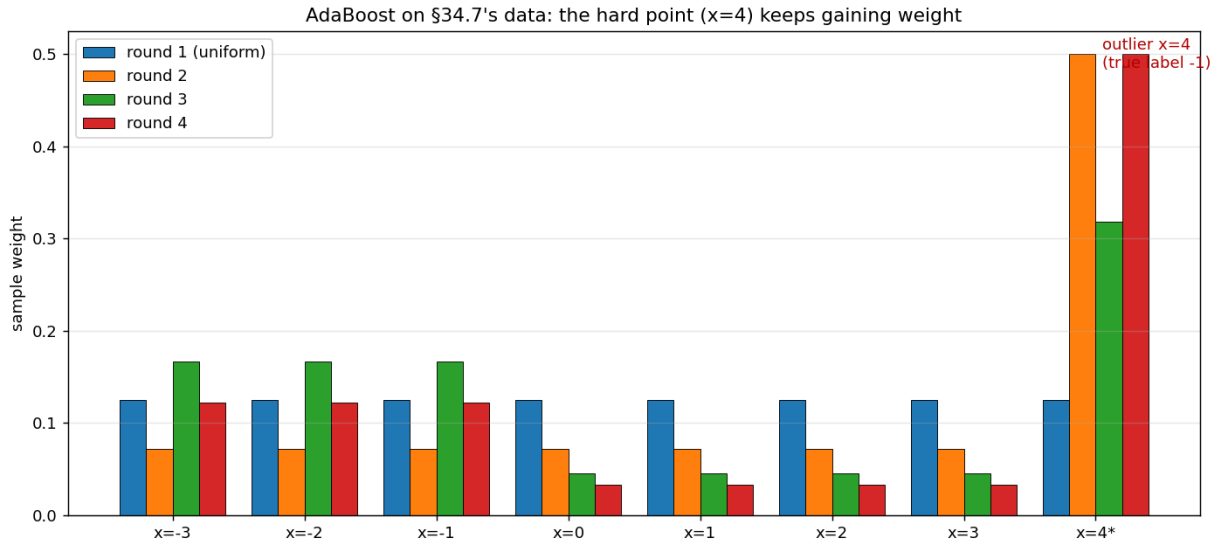


Figure 52: Grouped bar chart of the eight sample weights at the start of AdaBoost rounds 1 through 4 on §34.7's data: all weights start at $1/8$; the outlier $x=4$ (true label -1) rises to $1/2$, dips to $7/22$, and returns to $1/2$, while correctly classified points' weights shrink.

The ensemble after 3 rounds. $G(x) = \text{sign}(\alpha_1 G_1 + \alpha_2 G_2 + \alpha_3 G_3)$: - $x \leq -0.5$: $-(\alpha_1 + \alpha_3) + \alpha_2 = -0.7044 \Rightarrow \boxed{-1}$ (correct for $-3, -2, -1$). - $-0.5 < x \leq 3.5$: $\alpha_1 + \alpha_2 + \alpha_3 = 2.0037 \Rightarrow \boxed{+1}$ (correct for $0, 1, 2, 3$). - $x > 3.5$: $(\alpha_1 + \alpha_3) - \alpha_2 = 0.7044 \Rightarrow \boxed{+1}$ — still wrong on $x = 4$ (true -1).

The §33.10 connection, fulfilled. The soft-margin SVM *tolerates* the hard point via slack (§33.3: it pays ξ_i and the boundary moves on). AdaBoost does the opposite: it *reweights* the hard point — here to weight $1/2$ — and tries again, round after round. Same enemy (the unclassifiable outlier), opposite tactic: tolerance vs. obsession.

Basically, ... “Watch the weights: the stump keeps missing $x = 4$, so its weight balloons to half of everything while the easy points fade to crumbs. The final vote is three stumps arguing — and the loudest argument is still about the one point nobody can classify. That’s AdaBoost’s whole personality: it cannot let a mistake go.”

34.8 Bagging vs boosting — the sources’ contrast

- i) **Parallel vs sequential** (ISL §8.2.3). Bagging: each tree is built on a bootstrap dataset *independent of the other trees*; boosting: the trees are grown *sequentially*, each using information from the previously grown ones — and boosting involves **no bootstrap sampling**.
- ii) **What each fixes.** Bagging reduces *variance* by averaging independent-ish fits (good for high-variance, low-bias deep trees, §34.2). Boosting *rectifies errors*: each round concentrates on the previous round’s mistakes (reweighting) or residuals, learning slowly (revision slides; ISL).
- iii) **More trees.** Bagging: large B does not overfit (ISL). Boosting: *can* overfit if B is too large — choose B by cross-validation (ISL).
- iv) **Base learners.** Bagging wants strong-but-twitchy learners (deep unpruned trees, ISL). Boosting wants *weak* learners — stumps are often enough, and usually better (revision slides; ISL).

Basically, ... “Bagging = democracy: everybody votes independently at the same time, majority wins, and adding more voters never hurts. Boosting = tutoring: one weak student at a time, each drilled specifically on the last one’s mistakes — powerful, but drill too long (B too big) and it starts memorizing.”

34.9 Strengths and limits — as the sources list them

Bagging. - Strengths: dramatically reduces the variance of unstable procedures like trees, improving prediction (ESL’s simulated example; ISL: “impressive improvements” from hundreds of trees); simple to run; B is not a delicate knob; OOB error comes free (§34.4). - Limits: any interpretability of a single tree is **lost** (ESL; MITx 6.036); **stable** procedures like nearest neighbours are barely affected (ESL) — nothing to smooth; under 0–1 loss, bagging a *bad* classifier can make it *worse* (ESL); and it needs **diverse** learners — fully correlated classifiers gain nothing (MLT practice Q3).

Boosting / AdaBoost. - Strengths: can dramatically improve even a *very* weak classifier — ESL’s stump experiment: 45.8% test error (near guessing) $\rightarrow$ $\boxed{5.8\%}$ after 400 rounds, beating a 244-node tree (24.7%); slow learning tends to perform well (ISL); small trees suffice since each sees the earlier ones (ISL). - Limits: unlike bagging, boosting **can overfit if B is too large** — tune B by cross-validation (ISL); it carries **three** tuning parameters (B , shrinkage λ , interaction depth d — ISL’s Algorithm 8.2), so more tuning effort than bagging’s “just crank B .”

Basically, ... “Bagging’s deal: kills variance, costs you the pretty tree picture, useless on already-stable methods. Boosting’s deal: turns stumps into champions by obsessing over mistakes, but the obsession needs supervision — too many rounds and it overfits, and there are three knobs to tune instead of one.”

34.10 Where this goes next

- i) **The variance story closes.** §29.21(ii)’s “mitigated by ensembles” is now concrete: trees twitch (§29.21(ii)), averaging calms them (§34.2), and decorrelated averaging calms them more (§34.3).
- ii) **Same enemy, opposite tactic** (§33.10, fulfilled in §34.7): the soft margin tolerates hard points with slack; AdaBoost reweights them and tries again.
- iii) **Chapter 35 (loss functions).** ESL’s deeper AdaBoost theory shows it fits an *additive model* optimizing the *exponential loss* — the “why do the weights update *this way*” story belongs with the loss functions, next chapter.

Problem set

- Vote arithmetic.** Five independent classifiers, each with error rate 0.2. (i) Write the majority-vote error as a binomial sum and compute it. (ii) Is it below 0.2? (iii) In one line, which assumption of this calculation fails for real bagged trees (§34.3)?
- OOB counting.** $n = 150$ points, $B = 600$ trees. (i) Estimate the expected number of trees for which a fixed point is OOB. (ii) Estimate how many of the 150 points a given tree trains on. (iii) In one line, why is the OOB error a *valid* test-error estimate?
- AdaBoost by hand (Q5-style).** 100 points, first stump misclassifies 30. Using the §34.6 update with $\alpha = \frac{1}{2} \ln((1 - e)/e)$: (i) compute α ; (ii) give the unnormalized weight of a misclassified and of a correctly classified point; (iii) normalize and give the final weight of a misclassified point (leave as an exact expression, then a decimal).
- Reading α .** (i) Show $\alpha_m > 0 \iff e_m < 0.5$. (ii) What is α_m when $e_m = 0.5$, and what does that mean for G_m ’s role in the final vote? (iii) In one line, why must a weak learner beat random guessing for AdaBoost to work?
- Bagging vs boosting, sourced.** For each claim say whether the sources support it and cite the section: (i) “Boosting builds trees on bootstrap samples.” (ii) “A very large B never causes overfitting in boosting.” (iii) “Bagging reduces variance but leaves bias unchanged (squared-error).” (iv) “In boosting, each tree is fit independently of the others.”
- The m -knob.** From the MLT practice assignment (Q1): (i) decreasing the number of features considered per split does what to tree-to-tree correlation? (ii) Increasing it does what to individual tree strength? (iii) In one line, why does (i) help bagging (§34.3)?

Solutions — Chapter 34: Ensembles — bagging, boosting, AdaBoost

Full worked solutions to the Chapter 34 problem set. Every number recomputed independently (hand algebra cross-checked in numpy; see the review log).

Problem 1 — Vote arithmetic

Five independent classifiers, each with error rate $e = 0.2$. Let S = number wrong; $S \sim \text{Bin}(5, 0.2)$.

(i) The majority is wrong iff $S \geq 3$:

$$\begin{aligned}
 P(\text{vote wrong}) &= \sum_{k=3}^5 \binom{5}{k} (0.2)^k (0.8)^{5-k} \\
 &= \binom{5}{3} (0.2)^3 (0.8)^2 + \binom{5}{4} (0.2)^4 (0.8) + (0.2)^5 \\
 &= 10(0.008)(0.64) + 5(0.0016)(0.8) + 0.00032 = 0.0512 + 0.0064 + 0.00032 = \boxed{0.05792}.
 \end{aligned}$$

(ii) Yes: $0.05792 < 0.2$ — the vote is far better than each voter (same moral as §34.2’s eg 1).

(iii) **Independence** fails for real bagged trees: they train on resamples of the *same* data, so they are correlated, and the true gain is smaller than the binomial ideal (§34.3).

Problem 2 — OOB counting

- (i) Each point is OOB for about one-third of the trees: $\approx B/3 = \boxed{200}$ trees.
- (ii) Each tree trains on about $n(1 - 1/e) \approx 150 \times 0.632 \approx \boxed{95}$ distinct points (“around two-thirds,” §34.4).
- (iii) Point i ’s OOB prediction uses only trees that never trained on it — so the OOB error measures performance on genuinely unseen data, like a test set.

Problem 3 — AdaBoost by hand (Q5-style)

$e = 30/100 = 0.3$. With $s = e^\alpha = \sqrt{(1 - e)/e} = \sqrt{7/3}$:

- (i) $\boxed{\alpha = \frac{1}{2} \ln(7/3) \approx 0.4236}$.
- (ii) Unnormalized: misclassified $\frac{1}{100}s = \frac{\sqrt{7/3}}{100} \approx \boxed{0.015275}$; correctly classified $\frac{1}{100}/s = \frac{\sqrt{3/7}}{100} \approx \boxed{0.0065465}$.
- (iii) Normalizer $Z = \frac{30s+70/s}{100}$. Final misclassified weight:

$$\frac{s}{30s + 70/s} = \frac{s^2}{30s^2 + 70} = \frac{7/3}{30 \cdot 7/3 + 70} = \frac{7/3}{140} = \boxed{\frac{1}{60} \approx 0.016667}.$$

(This is the MLT practice assignment Q5’s answer (c), recomputed: $\sqrt{7/3} / (30\sqrt{7/3} + 70\sqrt{3/7}) = 1/60$.)

Problem 4 — Reading α

- (i) $\alpha_m = \frac{1}{2} \ln\left(\frac{1-e_m}{e_m}\right) > 0 \iff \frac{1-e_m}{e_m} > 1 \iff 1 - e_m > e_m \iff \boxed{e_m < 0.5}$.
- (ii) $e_m = 0.5 \Rightarrow \boxed{\alpha_m = 0}$: the classifier contributes *nothing* to the weighted vote — a coin-flip stump is ignored.
- (iii) If $e_m \geq 0.5$ then $\alpha_m \leq 0$ — the round adds no positive signal (or votes against itself), so reweighting cannot steer the ensemble toward the hard points.

Problem 5 — Bagging vs boosting, sourced

- (i) **False.** Boosting does *not* use bootstrap sampling; each tree is fit on a modified (reweighted) version of the original data (ISL §8.2.3; §34.5(i), §34.8(i)).
- (ii) **False.** Unlike bagging, boosting *can* overfit if B is too large — B is chosen by cross-validation (ISL §8.2.3; §34.8(iii)).
- (iii) **True.** Under squared-error loss, averaging reduces variance and leaves bias unchanged (ESL §8.7; §34.2).
- (iv) **False.** Boosting trees are grown *sequentially*, each using information from the previously grown trees (ISL §8.2.3; §34.8(i)).

Problem 6 — The m -knob

- (i) Decreasing the features per split **reduces** the tree-to-tree correlation (wider variety of trees).
- (ii) Increasing the features per split **increases** each individual tree’s strength (more features to pick the split from).
- (iii) Decorrelated trees are what make averaging actually reduce variance — fully correlated trees vote identically and bagging gains nothing (§34.3).

Chapter 35: Loss functions for classification

Everything in §§35.1–35.9 comes from the MLT Week-12 PPT slides (the “loss-function view”: 0/1 loss and its NP-hardness, the four surrogate curves, the perceptron-as-SGD note, the SVM-as-regularization-plus-hinge decomposition, and the logistic-loss margin derivation), with the logistic-regression loss itself from the MITx 6.036 notes (§§31.9–31.10), the hinge loss from §33.3, and the AdaBoost updates from §34.6. §§35.10–35.11 follow ESL ch. 14 (§§14.4–14.6: AdaBoost as forward stagewise exponential-loss minimization, the β_m derivation, the population minimizer, the robustness comparison). The worked comparison in §35.10 and the figure use the chapter’s own tiny datasets, with every number recomputed independently (review log).

Notation. The §22.9 convention: instances $x^{(i)}$, labels $y^{(i)} \in \{+1, -1\}$. A classifier produces a real-valued **score** $s^{(i)} = s(x^{(i)})$ (e.g. $w^T x^{(i)} + b$), and the predicted label is $\text{sign}(s^{(i)})$. The **margin** of example i is $m^{(i)} = y^{(i)} s^{(i)}$: positive means correct, negative means wrong, and its size says *how* correct or wrong. Nearly every classification loss is a function of this single number.

35.1 The ideal loss — and why it’s unusable

The loss you actually care about is the one from §22.9: the **misclassification (0/1) loss**, the fraction of points you get wrong,

$$L(f) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(f(x^{(i)}) \neq y^{(i)}).$$

It is the scoreboard. But nobody trains with it, and the MLT Week-12 slides say why — three reasons:

- i) **It is flat almost everywhere.** §31.7 already made this point for the perceptron: any two hypotheses with the same mistake count “have the same J value” — the loss cannot tell a near-miss from a confident error, “which makes it difficult to design an algorithm that searches through the space of hypotheses for a good one.”
- ii) **It is not differentiable.** §31.3’s note: the 0/1-style bill is piecewise constant in the weights, so gradient descent (Chapter 10) has no slope to follow — the gradient is zero wherever it exists.
- iii) **Minimizing it over linear separators is NP-hard** (the slide’s word, written in red next to the $\min_w$). Even if you could handle i) and ii), the combinatorial problem itself is intractable.

So every working algorithm trains on a **surrogate**: a stand-in loss that is smooth (or at least convex) and easy to minimize, while the 0/1 loss stays on as the judge. That trade is this chapter’s whole subject.

Basically, ... “The 0/1 loss is the exam — but you can’t study for it directly. It’s flat (no slope to descend), jagged (no derivatives), and NP-hard to minimize anyway. So every algorithm picks a *practice* loss that behaves well under optimization and hopes the exam score follows.”

35.2 One yardstick: the margin

Before the zoo, one definition that organizes everything.

Def (margin). For score $s^{(i)}$ and label $y^{(i)} \in \{+1, -1\}$,

$$m^{(i)} = y^{(i)} \cdot s^{(i)}.$$

$m^{(i)} > 0$ = correct; $m^{(i)} < 0$ = wrong; $|m^{(i)}|$ = confidence.

Four regimes a point can be in: - **Confident correct:** $m \gg 0$ (e.g. $m = 2$). - **Correct but shaky:** $0 < m < 1$ (right side of the boundary, but close). - **Wrong:** $m < 0$. - **Confident wrong:** $m \ll 0$.

Each loss below is a rule for turning this one number into a bill. They all agree that negative margins are mistakes to penalize; they disagree — sometimes violently — on how to price the *size*, and even on whether a bigger positive margin is always better (§35.5’s squared loss is the rebel).

Basically, ... “Multiply the label by the score. Positive: right answer. Negative: wrong answer. Bigger: more confident. Every loss in this chapter is just a different pricing scheme for this one number.”

35.3 Zero-one loss: the ideal, restated in margin form

Def (0/1 loss, per-example). $\ell_{0/1}(m) = \mathbf{1}(m < 0)$ — 1 if wrong, 0 if right. The training objective is the average, $\frac{1}{n} \sum_i \ell_{0/1}(m^{(i)})$, i.e. §22.9’s $L(f)$.

Note (the $m = 0$ convention). The MLT slide writes $\mathbf{1}(\hat{w}^T x \cdot y < 0)$ with a *strict* inequality; §22.9’s eg 9 instead defines $\text{sign}(0) = +1$, so a boundary point counts as a +1 prediction. This chapter follows the slide’s strict form; with no margin exactly 0 in any worked example, the two agree everywhere here.

Shape: a step — flat 1 for all $m < 0$, flat 0 for all $m \geq 0$. That is §31.7’s flatness made visible: a point with $m = -3$ (confidently wrong) and one with $m = -0.01$ (barely wrong) pay *identically*.

eg 1 (0/1 on three margins). Margins $m = 1.5, -0.2, 0.3$:

$$\ell_{0/1} = 0, 1, 0 \quad \Rightarrow \quad \text{average} = 1/3.$$

The barely-wrong point costs as much as a disaster would — the loss grades *whether*, never *how*.

Basically, ... “Right: free. Wrong: one rupee — no matter how wrong. The perfect judge and the worst teacher.”

35.4 Hinge loss: the SVM’s bribe

Def (hinge loss). $\ell_{\text{hinge}}(m) = \max(0, 1 - m)$.

This is the ξ_i of §33.3 — Chapter 33 promised (§33.10) its full treatment here. There, $\xi_i = \max(1 - y^{(i)}(w^T x^{(i)} + b), 0)$ was the “bribe” a point pays for violating the margin, and the MLT Week-12 slide re-derives exactly this: pushing each ξ_i as low as its constraint allows gives $\xi_i = \max(0, 1 - \hat{w}_i^T x_i y_i)$, turning the soft-margin primal into

$$\min_{w,b} \underbrace{\frac{1}{2} \|w\|^2}_{\text{model-dependent}} + C \sum_{i=1}^n \underbrace{\max(0, 1 - y^{(i)}(w^T x^{(i)} + b))}_{\text{data-dependent loss}}$$

— “regularization + hinge loss,” in the slide’s words.

How it grades the four regimes: - $m \geq 1$: $\boxed{0}$ — satisfied, pays nothing. The SVM doesn’t just want correct; it wants correct *by a margin of 1*. - $0 < m < 1$: $1 - m > 0$ — **correct, but still charged**. A point barely

on the right side pays almost 1. - $m < 0$: $1 - m > 1$ — wrong, and charged more than the 0/1 loss's flat 1, growing linearly with how wrong.

Note (convex, but not smooth). The subgradient w.r.t. w : $-xy$ when $w^T xy < 1$ (the loss's active region — including correct-but-shaky points, not just mistakes), 0 when $w^T xy > 1$, and the whole interval between $-xy$ and 0 at the kink $w^T xy = 1$ — a subgradient, the standard replacement for a derivative at a convex kink. (The MLT slide's “ $-xy$ on mistakes, 0 elsewhere, kink at $w^T xy = 0$ ” bullet was its *perceptron* statement — the modified hinge of §35.8, not this one.)

eg 2 (hinge on three margins). $m = 1.5, -0.2, 0.3$:

$$\max(0, -0.5) = 0, \quad \max(0, 1.2) = 1.2, \quad \max(0, 0.7) = 0.7.$$

Compare eg 1: the barely-wrong point pays 1.2 (more than 0/1's 1), and the barely-*right* point pays 0.7 where 0/1 paid 0. The SVM taxes uncertainty.

Basically, ... “Being right isn't enough — be right with room to spare. Margin above 1: free. Anything less: you pay $1 - m$, even if you were technically correct. It's the strict teacher who marks down shaky answers.”

35.5 Squared loss: regression doing classification

The MLT slide's “Alg 1: using regression for classification” just fits $(g(x) - y)^2$ with $y \in \{+1, -1\}$ and $\hat{h}(x) = \text{sign}(g(x))$. Expanding:

$$(g - y)^2 = g^2 + y^2 - 2gy = (gy)^2 + 1 - 2gy = \boxed{\ell_{\text{sq}}(m) = (m - 1)^2},$$

using $y^2 = 1$. (The MLT Week-12 practice assignment's Q2 computes exactly this loss on a dataset — answer 7.)

The shape is a parabola with its zero at $m = 1$. Two consequences the slide's plot shows:

- i) **It punishes over-confidence.** At $m = 2.5$ (very confidently correct), $\ell_{\text{sq}} = 2.25$ — more than at $m = 0$! It was built for regression, where the score should *equal* the label; “too right” is as bad as wrong.
- ii) **It is monotone-wrong.** ESL §14.6: squared error “is not a monotone decreasing function of increasing margin ... for margin values $y_i f(x_i) > 1$ it increases quadratically, thereby placing increasing influence (error) on observations that are correctly classified with increasing certainty, thereby reducing the relative influence of those incorrectly classified.” Conclusion: “if class assignment is the goal, a monotone decreasing criterion serves as a better surrogate loss function.”

eg 3 (the confidence tax). $m = 2.5, -1, 0.2$:

$$\ell_{\text{sq}} = 2.25, 4, 0.64 \quad \text{vs hinge:} \quad 0, 2, 0.8.$$

At $m = 2.5$ the hinge is fully satisfied (0) while squared charges 2.25 — the parabola's right arm is the giveaway that this loss doesn't understand classification.

Basically, ... “Squared loss wants the score to *equal* the label, not just agree with it. Nail the answer too confidently and it fines you for showing off. That's why it's a regression loss moonlighting in classification — and not a great one (ESL says so outright).”

35.6 Logistic loss: cross-entropy in margin form

§31.9's loss was the negative log-likelihood with $\{0, 1\}$ labels,

$$\boxed{L_{\text{nl}}(g, y) = -(y \log g + (1 - y) \log(1 - g))},$$

$g = \sigma(\theta^T x + \theta_0)$. The MLT Week-12 slide converts it to the $\{\pm 1\}$ margin form. Writing $z_i = 1$ for $y_i = +1$ and $z_i = 0$ for $y_i = -1$, with $s_i = \hat{w}^T x_i$:

- $z_i = 1$: $-\log \sigma(s_i) = -\log \frac{1}{1+e^{-s_i}} = \log(1 + e^{-s_i}) = \log(1 + e^{-y_i s_i})$.
- $z_i = 0$: $-\log(1 - \sigma(s_i)) = -\log \frac{e^{-s_i}}{1+e^{-s_i}} = \log(1 + e^{s_i}) = \log(1 + e^{-y_i s_i})$.

Both cases collapse to one formula:

$$\boxed{\ell_{\log}(m) = \log(1 + e^{-m}), \quad m = y \cdot s}.$$

The slide boxes exactly this as the “logistic loss,” and the objective becomes $\min_w \sum_i \log(1 + e^{-y_i s_i})$. This is §31.15(iii)’s “first specimen” L_{nll} , now in its margin uniform.

Behavior: at $m = 0$ it pays $\log 2 \approx 0.693$; for large positive m it decays toward 0 (never quite reaching it — §31.10: on separable data the weights inflate toward infinite certainty); for large negative m it grows *linearly* ($\log(1 + e^{-m}) \approx -m$), much gentler than the exponential loss (§35.7). Unlike hinge it never hits exactly 0; unlike squared it never punishes being too right.

eg 4 (the two faces agree). $\{0, 1\}$ form: $g = 0.8$, $y = 1$:

$$L_{\text{nll}} = -\log 0.8 = 0.2231.$$

Margin form: $s = \sigma^{-1}(0.8) = \log \frac{0.8}{0.2} = \log 4 = 1.3863$, $m = (+1)(1.3863)$:

$$\log(1 + e^{-1.3863}) = \log(1 + 0.25) = \log 1.25 = 0.2231. \checkmark$$

Same bill, two notations — the margin form just hides the $\{0, 1\} \leftrightarrow \{\pm 1\}$ bookkeeping.

Basically, ... “Cross-entropy asks: ‘how probable did I say the *true* label was?’ Take negative log of that probability. The margin formula $\log(1 + e^{-m})$ is the same question rewritten so both label conventions give one line. Confident and right: nearly free. Confident and wrong: the bill grows without bound — but only linearly, not exponentially.”

35.7 Exponential loss: AdaBoost’s engine

Def (exponential loss). $\boxed{\ell_{\text{exp}}(m) = e^{-m}}$.

The MLT slide’s “Boosting” bullet gives it in one line: Loss = $e^{-y h(x)}$. §35.11 derives *why* AdaBoost is secretly minimizing exactly this — but the shape already tells the story: at $m = 0$ it pays 1; for $m > 0$ it decays toward 0; for $m < 0$ it **explodes exponentially**. A point with margin -3 pays $e^3 \approx 20.09$ — twenty times the 0/1 loss’s flat 1.

ESL §14.6 compares it with the logistic (binomial deviance) loss: “The penalty associated with binomial deviance increases linearly for large increasingly negative margin, whereas the exponential criterion increases the influence of such observations exponentially.” The price of that steepness: the exponential criterion “concentrates much more influence on observations with large negative margins,” while the deviance is “far more robust in noisy settings ... especially in situations where there is misspecification of the class labels in the training data” — and “the performance of AdaBoost has been empirically observed to dramatically degrade in such situations.” Mislabelled points scream under the exponential bill.

eg 5 (the explosion). $m = 1.5, -0.2, 0.3$:

$$e^{-1.5} = 0.2231, \quad e^{0.2} = 1.2214, \quad e^{-0.3} = 0.7408.$$

The barely-wrong point pays 1.22 (above 0/1’s 1 already at $m = -0.2$); a confident error at $m = -3$ would pay 20.09. Compare logistic’s 3.05 there — the exponential curve is the steepest punisher of confident mistakes in the zoo.

Basically, ... “Exponential loss is the drama queen: correct answers get pocket change, but a confident mistake gets billed *exponentially*. That’s why AdaBoost obsesses over hard points — and why a mislabeled point can hijack it (ESL documents the degradation).”

35.8 The perceptron's loss: hinge without the margin

§31.3's per-example error, $e^{(i)} = \max(0, -w^T \phi(x^{(i)})y^{(i)})$, is the **modified hinge loss** $\ell_{\text{mod-hinge}}(m) = \max(0, -m)$ — the hinge with the margin demand deleted (compare §35.4's $\max(0, 1 - m)$). Correct: 0. Wrong: $-m = |m|$, linear in how wrong.

And the MLT slide draws the consequence: with subgradient $-xy$ on mistakes ($w^T xy < 0$), 0 elsewhere, one SGD step with step size 1 gives

$$w_{t+1} = w_t - 1 \cdot (-x_i y_i) = w_t + x_i y_i \quad (\text{on mistakes}),$$

i.e. **the perceptron update** (§31.4) — “perceptron can be interpreted as SGD with modified hinge loss with step size = 1” (the slide's words; §31.4's $(y^{(i)} - \hat{y}^{(i)}) = \pm 2$ on mistakes folds the extra factor of 2 into the step size).

So the perceptron *does* have a loss after all — just the least demanding one in the zoo: it asks only for the right side of the boundary, no margin, no probabilities. That is §31.7's blind spot in one formula.

Basically, ... “The perceptron's loss is the hinge loss with the ambition removed: right side = free, wrong side = pay $|m|$. No margin demand, no probability. And its famous update rule is just one SGD step on this loss — ‘learn only when wrong’ falls out of the subgradient being zero on correct points.”

35.9 Surrogate losses: the trade

Collecting the MLT slide's conclusions (its closing list, verbatim in spirit):

- i) **0/1 loss is NP-hard to minimize** — the ideal is intractable (§35.1).
- ii) **Different algorithms use different “surrogate” losses** — SVM $\rightarrow$ hinge, logistic regression $\rightarrow$ logistic/cross-entropy, AdaBoost $\rightarrow$ exponential, perceptron $\rightarrow$ modified hinge, least-squares-for-classification $\rightarrow$ squared.
- iii) **Surrogates are convex and hence easy to minimize** — every curve in the zoo (except 0/1) is convex in the margin, so gradient/subgradient methods apply.

The deal, then: **train on the surrogate, evaluate on 0/1**. The surrogate is the practice ground you can optimize; the 0/1 loss is the exam you actually care about. ESL's Figure-14.3 note makes the gap concrete: in boosting, “the training-set misclassification error decreases to zero” after ~ 250 iterations “while the exponential loss continues to decrease” — “Clearly AdaBoost is not optimizing training-set misclassification error; the exponential loss is more sensitive to changes in the estimated class probabilities.”

Note (which surrogates bound 0/1). The slide's plot shows the surrogates sitting *above* the 0/1 step — almost. Hinge, squared, and exponential are genuine upper bounds of $\mathbf{1}(m < 0)$ everywhere (check: $\max(0, 1 - m) \geq 1$ for $m \leq 0$; $(m - 1)^2 \geq 1$ for $m < 0$; $e^{-m} \geq 1$ for $m \leq 0$). The logistic loss is **not**: at $m = -0.5$, $\log(1 + e^{0.5}) = 0.9741 < 1$. It dips a hair below the step for small negative margins — a fine-print exception worth knowing.

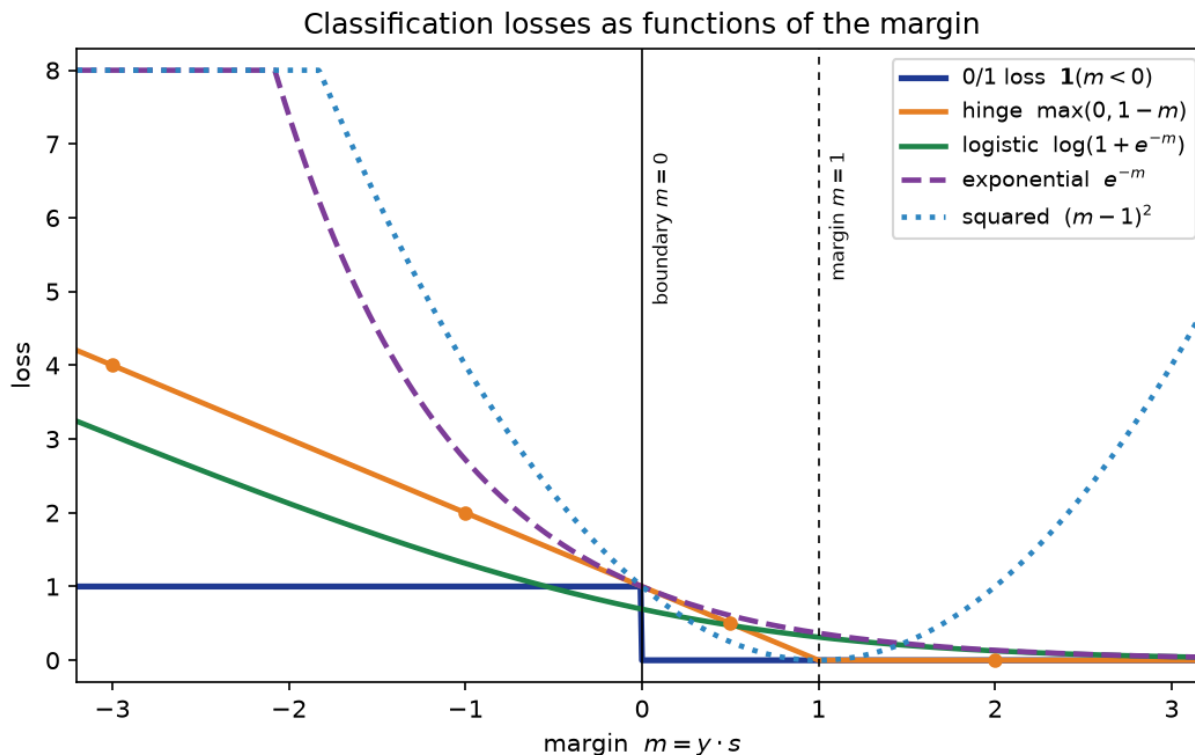


Figure 53: Classification losses as functions of the margin $m = y \cdot s$: the 0/1 step, the hinge $\max(0, 1 - m)$, logistic $\log(1 + e^{-m})$, exponential e^{-m} , and squared $(m - 1)^2$ curves, with margin points from the §35.10 comparison marked.

Basically, ... “You can’t minimize the exam, so you minimize a practice test that’s shaped like the exam but smooth enough to study with. Pick the practice test your algorithm was built for — they’re all convex, so the studying is easy. Just remember the practice score isn’t the exam score: AdaBoost can keep improving its exponential loss long after its mistake count hits zero.”

35.10 The big comparison: one dataset, five losses

Five points, fixed once, scored under every loss. Scores s and labels y :

point	s	y	margin $m = ys$	verdict
A	2.0	+1	2.0	confident correct
B	0.5	+1	0.5	correct, shaky
C	-0.5	-1	0.5	correct, shaky
D	-1.0	+1	-1.0	wrong
E	3.0	-1	-3.0	confident wrong

The bills (every number recomputed by hand and in numpy — review log):

point	m	0/1 $\mathbf{1}(m < 0)$	hinge $\max(0, 1 - m)$	logistic $\log(1 + e^{-m})$	squared $(m - 1)^2$	exponential e^{-m}
A	2.0	0	0	0.1269	1	0.1353
B	0.5	0	0.5	0.4741	0.25	0.6065
C	0.5	0	0.5	0.4741	0.25	0.6065
D	-1.0	1	2	1.3133	4	2.7183
E	-3.0	1	4	3.0486	16	20.0855
total		2 (0.4 avg)	7 (1.4 avg)	5.4369 (1.0874 avg)	21.5 (4.3 avg)	24.1522 (4.8304 avg)

Reading the table:

- i) **B and C (correct but shaky):** 0/1 charges nothing; hinge charges 0.5 — the margin demand made visible. Logistic (0.4741) and exponential (0.6065) also tax shakiness, gently.
- ii) **A (confident correct):** hinge is satisfied (0); squared is the *only* loss that punishes it (1) — the confidence tax of §35.5.
- iii) **E (confident wrong):** 0/1's flat 1 against exponential's 20.09. Every surrogate punishes confident errors harder than 0/1 — that extra slope is exactly what makes them trainable.
- iv) **Ordering of harshness on mistakes:** exponential > squared > hinge > logistic > 0/1 on point E. The losses don't just differ in shape; they differ in *temperament*.

Basically, ... “Same five predictions, five different report cards. 0/1 only counts bodies. Hinge fines you for being right-but-nervous. Squared fines you for being right-and-smug. Exponential goes nuclear on confident mistakes. Logistic stays the calm middle — which is why it's the default.”

35.11 Why AdaBoost's weights update the way they do

§34.10(iii) promised this story: ESL's deeper theory (ch. 14) shows AdaBoost fits an **additive model** optimizing the **exponential loss** — and once you see it, the $e^{\pm\alpha}$ updates stop looking like magic.

Setup. The final classifier is a weighted vote, $G(x) = \text{sign}(f_M(x))$ with

$$f_M(x) = \sum_{m=1}^M \beta_m G_m(x), \quad G_m(x) \in \{+1, -1\},$$

an *additive expansion* in the base classifiers (ESL §14.2). The claim (ESL §14.4): AdaBoost.M1 is **forward stagewise additive modeling** — build f_M one term at a time, never revisiting old terms — with the exponential loss

$$\boxed{L(y, f(x)) = e^{-y f(x)}} \quad (\text{ESL (14.8)}).$$

Step 1 — the weights are the loss. At round m , the model so far is f_{m-1} . Choosing the next pair (β, G) to minimize the total exponential loss:

$$(\beta_m, G_m) = \arg \min_{\beta, G} \sum_{i=1}^n \exp[-y_i(f_{m-1}(x_i) + \beta G(x_i))].$$

Factor out everything independent of (β, G) :

$$= \arg \min_{\beta, G} \sum_{i=1}^n \underbrace{e^{-y_i f_{m-1}(x_i)}}_{w_i^{(m)}} e^{-\beta y_i G(x_i)} \quad (\text{ESL (14.9)}).$$

The weights $w_i^{(m)} = e^{-y_i f_{m-1}(x_i)}$ **are the current per-point exponential loss** — no separate invention needed. Reweighting *is* loss bookkeeping.

Step 2 — fitting G_m means minimizing weighted error. Fix $\beta > 0$. Since $y_i G(x_i) = +1$ on correct points and -1 on wrong ones,

$$\sum_i w_i^{(m)} e^{-\beta y_i G(x_i)} = e^{-\beta} \sum_{\text{correct}} w_i^{(m)} + e^{\beta} \sum_{\text{wrong}} w_i^{(m)} = (e^{\beta} - e^{-\beta}) \sum_i w_i^{(m)} \mathbf{1}(y_i \neq G(x_i)) + e^{-\beta} \sum_i w_i^{(m)},$$

so the minimizer is

$$G_m = \arg \min_G \sum_{i=1}^n w_i^{(m)} \mathbf{1}(y_i \neq G(x_i)) \quad (\text{ESL (14.10)})$$

— “fit the weak classifier to the data *weighted by* D_{m-1} ” (§34.6(ii)) falls out of the loss, not from a heuristic.

Step 3 — the optimal vote weight. Let err_m be G_m ’s weighted error rate (ESL (14.13)). Minimizing the criterion over β is one-variable calculus: with total weight W ,

$$C(\beta) = e^{-\beta} W (1 - \text{err}_m) + e^{\beta} W \text{err}_m, \quad \frac{dC}{d\beta} = 0 \iff e^{2\beta} = \frac{1 - \text{err}_m}{\text{err}_m},$$

$$\beta_m^* = \frac{1}{2} \log \frac{1 - \text{err}_m}{\text{err}_m} \quad (\text{ESL (14.12)})$$

— the course’s α_m (§34.6(iii); the Note there covers ESL’s factor-of-2 convention).

Step 4 — the $e^{\pm\alpha}$ update. The next round’s weights are the updated loss:

$$w_i^{(m+1)} = w_i^{(m)} e^{-\beta_m^* y_i G_m(x_i)} \quad (\text{ESL (14.14)}).$$

Since $y_i G_m(x_i) = +1$ (correct) or -1 (wrong): - correct: multiply by $e^{-\beta_m^*}$, - wrong: multiply by $e^{+\beta_m^*}$.

With the course’s $\alpha_m = \beta_m^*$: **misclassified** $\times e^{+\alpha_m}$, **correctly classified** $\times e^{-\alpha_m}$ — exactly §34.6(iv). (ESL writes it as $w_i^{(m)} e^{\alpha_m \mathbf{1}(\text{wrong})} e^{-\beta_m}$ with $\alpha_m = 2\beta_m$; the common $e^{-\beta_m}$ factor normalizes away — ESL (14.15).)

So the “why”: the weights are the exponential loss of the model built so far, and because the loss is exponential, adding a term to the model *multiplies* each point’s loss by $e^{-\beta y G}$ — i.e. by $e^{+\alpha}$ for mistakes and $e^{-\alpha}$ for hits. Nothing arbitrary; it is the additive model updating its own books.

Two sourced footnotes.

- i) **What it estimates.** The population minimizer of the exponential loss is $f^*(x) = \frac{1}{2} \log \frac{P(Y=1|x)}{P(Y=-1|x)}$ — one-half the log-odds (ESL (14.16), “easy to show (Friedman et al. 2000)” — the proof is cited, not shown, in ESL). Hence $\text{sign}(f_M)$ is a justified classification rule. The binomial deviance (logistic loss, ESL (14.18)) shares this population minimizer — but “ e^{-Yf} itself is not a proper log-likelihood, since it is not the logarithm of any probability mass function” (ESL §14.5).
- ii) **Surrogate scoreboard, again.** ESL’s Figure-14.3 note (§35.9): training misclassification hits zero while the exponential loss keeps decreasing — “AdaBoost is not optimizing training-set misclassification error; the exponential loss is more sensitive to changes in the estimated class probabilities.”

Basically, ... “AdaBoost’s weights aren’t a trick — they’re the current exponential loss, one number per point. Add a stump to the model and each point’s loss gets *multiplied* by $e^{+\alpha}$ (if the stump missed it) or $e^{-\alpha}$ (if it got it). The $e^{\pm\alpha}$ update is just the loss doing its own accounting. And at the population level, all that multiplying is estimating half the log-odds — which is why taking the sign at the end is legitimate.”

35.12 Strengths and limits (as the sources list them)

Strengths. i) **The zoo is honest.** Each loss wears its algorithm’s priorities on its sleeve: hinge demands a margin (SVM), logistic grades probabilities (logistic regression), exponential obsesses over hard points (AdaBoost), modified hinge asks only for the right side (perceptron). ii) **Convexity buys optimizability.** Every surrogate here is convex in the margin — the MLT slide’s whole point: “surrogates are convex and hence

easy to minimize.” iii) **The margin unifies them.** One number, five pricing schemes — the comparison table of §35.10 is the chapter in miniature.

Limits. i) **The surrogate is not the goal.** Training loss and 0/1 loss can diverge (ESL’s Figure-14.3 note, §35.9/§35.11) — §22.10’s “training loss is not the goal” in a new costume. ii) **Surrogates have their own pathologies.** Squared punishes confident-correct points (ESL §14.6: “not a good surrogate”); exponential degrades on mislabeled data (ESL §14.6); logistic has no finite minimizer on separable data without regularization (§31.10); hinge has a kink at $m = 1$ (subgradients needed). iii) **0/1 stays NP-hard.** The slide states it without proof — this chapter reports the claim, not a proof.

Basically, ... “Surrogates are practice tests: convex, gradeable, each with its own bias. None of them *is* the exam, and each has a failure mode. Know which practice test you’re taking and what it lies about.”

35.13 Where this goes next

Part IV closes here. Four promises, kept:

- i) §30.11 (“Classification losses get their own chapter”) — kept: this chapter.
- ii) §31.15(iii) (“ L_{nl} was the first specimen”) — kept: §35.6 puts it in margin form alongside the rest.
- iii) §33.10 (hinge loss “gets its full treatment”) — kept: §35.4, with the ξ_i of §33.3 identified as the hinge loss.
- iv) §34.10(iii) (“the ‘why do the weights update *this* way’ story”) — kept: §35.11.

Part VI picks the thread straight back up: the MLT Week-12 slides close with neural networks trained by **cross-entropy loss** for classification (slide 2’s $p(y | x)$ diagram, “Cross-Entropy Loss”) — the logistic loss of §35.6, scaled up to deep networks. Chapter 41 is where that story continues.

Basically, ... “Every classifier you met in Part IV was secretly a loss function wearing a trench coat. Part V takes the same losses — cross-entropy front and center — and trains neural networks with them.”

Problem set

1. **Margin and 0/1.** Points (s, y) : $A = (1.5, +1)$, $B = (-0.4, +1)$, $C = (-2.2, -1)$, $D = (0.8, -1)$, $E = (0.2, +1)$. (i) Compute each margin $m = ys$. (ii) Using $\ell_{0/1} = \mathbf{1}(m < 0)$, give each point’s loss and the average.
2. **Hinge’s margin demand.** Same five points. (i) Compute each hinge loss $\max(0, 1 - m)$ and the total. (ii) Which points are *correct* ($m > 0$) yet still charged? (iii) In one line: why does the SVM charge them?
3. **Squared loss’s confidence tax.** Margins 2.5, -1 , 0.2. (i) Compute $(m - 1)^2$ and $\max(0, 1 - m)$ for each. (ii) At which margin(s) does squared charge *more* than hinge? (iii) In one line: why would a loss ever dislike $m = 2.5$?
4. **Logistic, two faces.** (i) In $\{0, 1\}$ notation compute L_{nl} for $(g, y) = (0.8, 1)$ and $(0.2, 0)$. (ii) Convert both to ± 1 notation: find each score $s = \sigma^{-1}(g)$, each margin m , and verify $\log(1 + e^{-m})$ equals the (i) answers.
5. **Perceptron = modified-hinge SGD.** For a misclassified point, $\ell(w) = \max(0, -y w^T x)$. (i) Write the loss without the max and compute $\nabla_w \ell$. (ii) Take one SGD step with step size 1 and show it is the perceptron update $w \leftarrow w + yx$ (cf. §31.4). (iii) What happens on a correctly classified point, and which §31.4 phrase does that match?
6. **AdaBoost weights from exponential loss.** Normalized weights w_i ($\sum_i w_i = 1$); fixed G ; $\text{err} = \sum_{i: y_i \neq G(x_i)} w_i$. (i) Write $C(\beta) = \sum_i w_i e^{-\beta y_i G(x_i)}$ in terms of err and show $\beta^* = \frac{1}{2} \ln \frac{1 - \text{err}}{\text{err}}$. (ii) Show the updated weights $w_i e^{-\beta^* y_i G(x_i)}$ equal §34.6(iv)’s update (misclassified $\times e^{+\alpha_m}$, correct $\times e^{-\alpha_m}$) with the course’s α_m . (iii) In one line: which step of the derivation forces G_m to minimize the *weighted* error?
7. **True or false, one-line justification:** (i) The hinge loss is differentiable everywhere. (ii) The logistic loss upper-bounds the 0/1 loss. (iii) A point with margin exactly 1 pays hinge loss 1. (iv) AdaBoost’s

$\text{sign}(f_M(x))$ is justified because f^* estimates half the log-odds. (v) Minimizing 0/1 loss over linear separators is NP-hard (per the MLT slides). (vi) e^{-Yf} is a proper log-likelihood.

Solutions — Chapter 35: Loss functions for classification

Full worked solutions to the Chapter 35 problem set. Every number recomputed independently (hand algebra cross-checked in numpy; see the review log).

Problem 1 — Margin and 0/1

$m = ys$:

(i) A : $(+1)(1.5) = \boxed{1.5}$; B : $(+1)(-0.4) = \boxed{-0.4}$; C : $(-1)(-2.2) = \boxed{2.2}$; D : $(-1)(0.8) = \boxed{-0.8}$; E : $(+1)(0.2) = \boxed{0.2}$.

(ii) $\ell_{0/1} = \mathbf{1}(m < 0)$: $A : 0$, $B : 1$, $C : 0$, $D : 1$, $E : 0$. Average = $\boxed{2/5 = 0.4}$.

Problem 2 — Hinge's margin demand

(i) $\max(0, 1 - m)$: A : $\max(0, -0.5) = \boxed{0}$; B : $\max(0, 1.4) = \boxed{1.4}$; C : $\max(0, -1.2) = \boxed{0}$; D : $\max(0, 1.8) = \boxed{1.8}$; E : $\max(0, 0.8) = \boxed{0.8}$. Total = $0 + 1.4 + 0 + 1.8 + 0.8 = \boxed{4.0}$ (mean 0.8).

(ii) $\boxed{E}$: $m = 0.2 > 0$ (correct) but charged 0.8.

(iii) The SVM doesn't ask "right side?" — it asks "margin ≥ 1 ?"; anything short pays the ξ_i bribe of \$33.3.

Problem 3 — Squared loss's confidence tax

(i) - $m = 2.5$: $(2.5 - 1)^2 = \boxed{2.25}$; hinge $\max(0, -1.5) = \boxed{0}$. - $m = -1$: $(-1 - 1)^2 = \boxed{4}$; hinge $\max(0, 2) = \boxed{2}$. - $m = 0.2$: $(0.2 - 1)^2 = \boxed{0.64}$; hinge $\max(0, 0.8) = \boxed{0.8}$.

(ii) Squared charges more at $\boxed{m = 2.5}$ ($2.25 > 0$) and $\boxed{m = -1}$ ($4 > 2$); at $m = 0.2$ it charges less ($0.64 < 0.8$).

(iii) Squared loss is a regression loss: it wants the score to *equal* the label, so $m = 2.5$ ("too right") counts as an error.

Problem 4 — Logistic, two faces

(i) $L_{\text{nl}}(g, y) = -(y \log g + (1 - y) \log(1 - g))$: - $(0.8, 1)$: $-\log 0.8 = \boxed{0.2231}$ (to 4 s.f.). - $(0.2, 0)$: $-\log(1 - 0.2) = -\log 0.8 = \boxed{0.2231}$.

(ii) $s = \sigma^{-1}(g) = \log \frac{g}{1-g}$: - Case 1: $s = \log \frac{0.8}{0.2} = \log 4 = 1.3863$; $y = +1$ so $m = 1.3863$; $\log(1 + e^{-1.3863}) = \log(1 + 0.25) = \log 1.25 = \boxed{0.2231}$ ✓. - Case 2: $s = \log \frac{0.2}{0.8} = -\log 4 = -1.3863$; $y = -1$ so $m = (-1)(-1.3863) = 1.3863$; same $\boxed{0.2231}$ ✓.

Both notations bill identically — the margin form absorbs the $\{0, 1\} \leftrightarrow \{\pm 1\}$ conversion.

Problem 5 — Perceptron = modified-hinge SGD

(i) Misclassified $\Rightarrow -y w^T x > 0$, so the max drops: $\ell(w) = -y w^T x$. Then $\boxed{\nabla_w \ell = -yx}$.

(ii) One SGD step with step size 1:

$$w \leftarrow w - 1 \cdot \nabla_w \ell = w - (-yx) = \boxed{w + yx},$$

the perceptron update (the MLT slide's form; §31.4's $(y^{(i)} - \hat{y}^{(i)}) = \pm 2$ on mistakes folds the factor of 2 into the step size).

(iii) Correctly classified $\Rightarrow -y w^T x \leq 0 \Rightarrow \ell = 0 \Rightarrow$ gradient 0 $\Rightarrow$ **no update** — §31.4's “learn only when wrong.”

Problem 6 — AdaBoost weights from exponential loss

(i) Split on $y_i G(x_i) = \pm 1$; with normalized weights the correct mass is $1 - \text{err}$, the wrong mass err :

$$C(\beta) = e^{-\beta}(1 - \text{err}) + e^{\beta} \text{err}.$$

$$\frac{dC}{d\beta} = -e^{-\beta}(1 - \text{err}) + e^{\beta} \text{err} = 0 \iff e^{2\beta} = \frac{1 - \text{err}}{\text{err}} \iff \boxed{\beta^* = \frac{1}{2} \ln \frac{1 - \text{err}}{\text{err}}}.$$

(ii) $w_i \leftarrow w_i e^{-\beta^* y_i G(x_i)}$: correct ($y_i G = +1$) gets $\times e^{-\beta^*}$; wrong ($y_i G = -1$) gets $\times e^{+\beta^*}$. The course's α_m (§34.6(iii)) equals β^* , so this is exactly **misclassified** $\times e^{+\alpha_m}$, **correct** $\times e^{-\alpha_m}$ — §34.6(iv). ✓

(iii) Step 2 of §35.11: for any fixed $\beta > 0$, the exponential-loss criterion over G equals $(e^{\beta} - e^{-\beta}) \sum_i w_i \mathbf{1}(y_i \neq G(x_i))$ plus a constant — so minimizing it over G is minimizing the weighted error.

Problem 7 — True or false

(i) **False.** Kink at $m = 1$: the subgradient there is the whole interval $[-1, 0]$ (MLT slide), not a single derivative.

(ii) **False.** Counterexample $m = -0.5$: $\log(1 + e^{0.5}) = \log(2.6487) \approx \boxed{0.9741} < 1 = \ell_{0/1}$. (Hinge, squared, and exponential *do* upper-bound 0/1 everywhere — verified on a grid in the review log.)

(iii) **False.** $\max(0, 1 - 1) = \boxed{0}$ — margin exactly 1 is precisely where the hinge turns off.

(iv) **True.** ESL (14.16): $f^*(x) = \frac{1}{2} \log \frac{P(Y=1|x)}{P(Y=-1|x)}$ — half the log-odds — so $\text{sign}(f_M)$ is a legitimate classification rule.

(v) **True** — as stated (not proved) in the MLT Week-12 slides.

(vi) **False.** ESL §14.5: e^{-Yf} “is not a proper log-likelihood, since it is not the logarithm of any probability mass function.”

Part V — Machine Learning in Practice

Chapter 36: NumPy and Pandas for ML

Everything in §§36.1–36.6 comes from the MLT TA colab notes (the “Intro to Libraries” notebooks — vectors, matrices, more-on-numpy, random-sampling — plus the Week-1 programming-assignment solution for the MNIST data-matrix example), and §§36.7–36.9 from the MLP Week-1 material (Lecture 7 “Data loading”, the Lecture 1–5 end-to-end wine-quality deck, and the Week-1 programming questions notebook with its solutions). Every code block below was re-run in this session (numpy 1.26.4, pandas 2.1.4) and shows the actual output — except the explicitly marked spots where the course notebooks used sklearn, which is not installed on this machine (§36.7’s diabetes/breast-cancer notes and its `SimpleImputer` call). Those are reported as recorded in the course notebooks, not re-run.

This is the chapter where the book’s math becomes code. Parts I–IV worked with $x^{(i)}$, X , $w^T x$ on paper; from here on, those live inside arrays.

Notation. The book’s standing convention (chapters 31–35): the design matrix $X \in \mathbb{R}^{n \times d}$ holds the points $x^{(1)}, \dots, x^{(n)}$ as its rows in order, and $y \in \mathbb{R}^n$ holds the labels — n points, d features. Zero-indexed, so $X[0]$ is $x^{(1)}$ and $X[i]$ is $x^{(i+1)}$.

Note (the convention clash). Two older sources disagree with this. §22.6 wrote the points as x^i (superscript, no parens), and the MLT TA colabs stack data as (d, n) — features as rows, points as columns (“we transpose so that the data-matrix is (d, n) and not (n, d) ”). Chapters 23–24 and the whole sklearn ecosystem use (n, d) . This chapter uses the book’s (n, d) everywhere and flags the TA colab’s (d, n) when it matters (§36.4). Reading either convention into the other is the single most common shape bug in ML code.

36.1 The ndarray: the book’s data structure

Def (ndarray). The TA colab’s one-liner: “Everything in NumPy is an array.” An **ndarray** = an n -dimensional grid of numbers of one type. A vector is a 1-D array, a matrix a 2-D array, an image a 3-D array (MNIST: (60000, 28, 28) in the TA colab’s Week-1 assignment).

- i) **Create:** `np.array(...)`, `np.zeros(n)`, `np.ones((r, c))`, `np.arange(a, b)`, `np.linspace(a, b, k)`, `np.eye(n)`, `np.diag(...)`.
- ii) **Inspect:** `.shape` (the book’s dimensions: (n, d)), `.ndim` (array-dimension, not vector-space dimension — the TA colab warns these differ), `.size`, `.dtype`.

eg 1 (the book’s dataset, as code). §22.3’s six houses, features = rooms, area, distance-to-metro; label = price:

```
X = np.array([[3,9,1.9],[2,7,2.1],[4,12,2.8],
              [5,16,0.9],[5,15,3.1],[4,11,1.6]])
y = np.array([50,32,66,98,85,69])
X.shape, y.shape, X.ndim, X.size, X.dtype
# (6, 3) (6,) 2 18 float64
```

```
X[2]          # row 2 =  $x^{(3)} = [4. \quad 12. \quad 2.8]$ 
```

X is the feature matrix of §23.1 ($n \times d$, rows are points as row vectors); y is the label vector. $X[2]$ is $x^{(3)}$ — zero-indexed, so row i holds $x^{(i+1)}$.

36.2 Vectorized ops vs Python loops

The TA colab contrasts the two ways to add vectors: a Python loop with `zip` + `append`, versus `z = x + y`. Everything element-wise is supported directly:

- i) `x + y`, `3 * x` (scaling), `x * y` = Hadamard product $x \odot y$ (§2.3's ops), `x ** 2`, `np.log10(x)` — element-wise functions of vectors (the colab's $f(x) = x^2$, $f(x) = 5x - 2$ examples).
- ii) `x @ y` = the dot product $x^T y = \sum_j x_j y_j$ (§2.5); the colab “shall stick to this” since it matches the math.
- iii) `np.linalg.norm(x)` (L_2 default; `ord=1` for L_1), `np.abs(A)` element-wise (§2.4's norms).

eg 2 (why ML code never loops). Sum of squares of 10^6 numbers — Python loop vs vectorized, each timed 5 times, best of 5:

```
a = np.arange(10**6, dtype=float)
t_loop = 0.191          # s : s = 0.0
                        #   for v in a: s += v*v
t_vec   = 0.00078       # s : s = (a**2).sum()
# speedup: ~246x; both sums 3.3333e+17, relative difference 1.12e-12
```

Basically, ... “A Python loop does one number at a time through the interpreter; a NumPy op hands the whole array to compiled C in one call. On a million elements that's the difference between 0.19 s and under a millisecond — $\sim 246\times$ here — for the *same* answer. Every training loop in this book (Chapters 10, 37–42) runs millions of such ops, so the vectorized form is not style, it's the only thing that finishes.”

36.3 Broadcasting: the silent shape rule

The TA colab's question: “Sometimes we would have to add a vector to each row or column of a matrix” — but $M + b$ is, strictly, “a slight abuse of notation as we can't add a matrix and a vector together.” NumPy resolves the abuse with one rule:

Def (broadcasting). Align shapes on their *trailing* axes. An axis of size 1 (or a missing axis) is *stretched* — repeated — to match. No copying happens; it's a view trick.

```
M = np.array([[1, 2, 3],
              [4, 5, 6]])          # (2, 3)
M + np.array([1, 2, 3])           # (3,) -> stretched down each row
# array([[2, 4, 6],
#        [5, 7, 9]])
M + np.array([1, 2]).reshape(2, 1) # (2, 1) -> stretched across columns
# array([[2, 3, 4],
#        [6, 7, 8]])
```

eg 3 (centering a data matrix). §24.2's covariance is built from *centered* rows $x^{(i)} - \bar{x}$. Broadcasting does all n subtractions at once:

```
mu = X.mean(axis=0)              # column means: [ 3.83333333 11.66666667  2.06666667]
Xc = X - mu                      # (6,3) - (3,) -> mu stretched down every row
Xc[0]                            # [-0.83333333 -2.66666667 -0.16666667]
Xc.mean(axis=0)                  # [-0.  0. -0.] -> columns now average to 0
```

Basically, ... “Broadcasting = NumPy pretends the small array is as big as the big one, by repeating it. (3,) against (6,3): the 3 numbers get copied down all 6 rows. The rule is always ‘line up the *right* edges, stretch the 1s’. Once you see it, $\mathbf{X} - \mu$ reads as the math $x^{(i)} - \bar{x}$ for all i at once.”

Note. Broadcasting fails loudly when nothing stretches: (2,3) + (4,) is a `ValueError`. That error is your friend — it means you mixed up the (n, d) convention (the Note at the top).

36.4 Indexing, slicing, and the axis semantics

- i) **Indexing:** `M[1, 2]` = element (row 1, col 2), zero-indexed (the colab prefers this over `M[1][2]`).
Slicing: `M[2, :]` = row 2, `M[:, 1]` = column 1, `M[1:3, 2:4]` = submatrix — start included, end excluded, like Python:

```
M = np.array([[1, 3, 0], [4, 2, 5], [9, 8, 7]])
M[1, 2], M[2, :], M[:, 1]
# 5 array([9, 8, 7]) array([3, 2, 8])
```

- ii) **Fancy indexing:** arrays as indices — `x[[1, 3, 6]]` picks positions 1,3,6; **boolean masks** — `x[x > 0]` keeps the positives. The colab’s payoff example: ReLU, three lines,

```
relu = lambda t: np.where(t > 0, t, 0)
relu(np.array([1, -2, 1, 3, -4, -3]))
# array([1, 0, 1, 3, 0, 0])
```

the activation function Chapter 41 runs inside every neural net — a boolean mask wearing a one-line disguise.

- iii) **Reshape:** `x.reshape(3, 2)`, `M.reshape(6)`, `M.reshape(3, -1)` (NumPy infers the -1); `.T` transposes. `x[:, np.newaxis]` turns a $(d,)$ vector into a $(d,1)$ column — the colab’s bridge between the “flat” vector and the matrix forms. **Note:** this is also where you convert the TA colab’s (d, n) habit to the book’s (n, d) : `X_dn.T` is X .
- iv) **Axis semantics** — the one everyone gets backwards once. `axis=k` collapses axis k : the result has one entry per position of every *other* axis.

```
A = np.arange(1, 9).reshape(2, 4) # [[1 2 3 4], [5 6 7 8]]
A.sum(axis=0) # [ 6  8 10 12] -> rows collapsed: one value per COLUMN
A.sum(axis=1) # [10 26] -> columns collapsed: one value per ROW
M = np.arange(1, 10).reshape(3, 3)
M.mean(axis=0) # [4. 5. 6.] -> column means (eg 3's mu)
M.var(axis=1) # [0.66666667 ...] -> variance within each row
```

Note. The TA colab describes `axis=0` as “top-bottom operations ... done on row-vectors” and `axis=1` as “left-right ... done on column-vectors” — loose wording (its own “left-right” example sums *rows*). The collapse rule above is the correct one and matches the verified outputs; the chapter uses it, not the colab’s phrasing.

Basically, ... “`axis=0` eats the rows: you’re left with one number per column. `axis=1` eats the columns: one number per row. Want the mean of each feature? Features are columns, so `X.mean(axis=0)`. Want the length of each data point? Points are rows, so `np.linalg.norm(X, axis=1)`.”

36.5 Reductions, sorting, stacking, comparing

The colab’s “misc functions”, verified:

```
x = np.array([10, -3, 2, 15, 5])
np.max(x), np.argmax(x) # 15, 3 -> value and its POSITION
np.min(x), np.argmin(x) # -3, 1
np.sort(x) # [-3  2  5 10 15]
```

```
np.argsort(x)          # [1 2 4 0 3] -> the permutation that sorts
```

- i) **argmax/argsort** return *indices*, not values — the standard “which class / which example” query (the classification chapters will use **argmax** over score vectors to extract predicted classes).
- ii) **Stacking**: `np.concatenate((A, B), axis=0)` piles rows (4×2 from two 2×2 s — the “add more data points” direction); `axis=1` glues columns (2×4 — “add more features”).
- iii) **Comparing**: `np.array_equal(x, y)` for exact equality; `x == y` gives a boolean *array*, unusable in an `if`. For floats — which are never exact — `np.allclose(x, y, rtol=0, atol=1e-2)`: $|x - y| \leq \text{atol} + \text{rtol} \cdot |y|$ element-wise. **Note**: eg 2’s loop-vs-vectorized check used exactly this idea (relative difference 1.12×10^{-12}). Never test trained weights with `==`; Chapter 40’s debugging discipline starts here.

36.6 Random numbers: seeded and reproducible

Two APIs appear in the course materials, and they are *not* interchangeable calls:

- i) **Legacy**: `np.random.seed(42)` then `np.random.permutation(10)`, `np.random.randn(...)` — global state (the MLP slide’s choice).
- ii) **Modern**: `rng = np.random.default_rng(seed=42)` then `rng.random(3)`, `rng.normal(...)`, `rng.choice(...)`, `rng.multivariate_normal(...)` — an explicit generator object (the TA colab’s choice).

Same seed $\Rightarrow$ same stream, every run:

```
np.random.default_rng(42).random(3)
# [0.77395605 0.43887844 0.85859792]
np.random.seed(42); np.random.permutation(10)
# [8 1 5 0 7 2 9 4 3 6]
```

Note. Pseudo-random, not random: the generator is a deterministic algorithm, so fixing the seed fixes the whole sequence. That is the entire point — reproducibility.

eg 4 (train/test split, the MLP slide’s function). The slide: “Make sure to set the seed so that we get the same test set in the next run.” `np.random.seed(42) + np.random.permutation(len(data))`, first 20% = test:

```
def split_train_test(data, test_ratio):
    np.random.seed(42)
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

(On §36.8’s 10-row table: train (8,4), test (2,4), test rows = ids 9 and 2 — the same two, every run.) The slide’s `sklearn` one-liner is the same idea packaged: `train_test_split(data, test_size=0.2, random_state=42)`.

Basically, ... “Randomness in ML is a controlled substance. You *want* shuffling and sampling (Chapter 39 lives on it), but you want the *same* shuffle every time you re-run, or you can never debug. Set the seed once, and ‘random’ becomes a fixed, replayable script.”

36.7 pandas: the labeled table

Def. A **DataFrame** = a 2-D labeled table (rows = examples, columns = named features); a **Series** = one labeled column. If an `ndarray` is the math, a `DataFrame` is the math *with the metadata attached* (§22.3:

column j means the same thing in every row — the labels are that meaning, written down).

- i) **Load:** `pd.read_csv(url, sep=";")`. The MLP slide’s wine-quality flow: 1599 rows, 12 columns — 11 features + label `quality`, all `float64`, label `int64` (verified against the same UCI file the slide downloads).
- ii) **First look:** `data.head()` (first 5 rows), `data.info()` (1599 entries, dtypes, non-null counts), `data.describe()` (count, mean, std, quartiles per numeric column), `data.shape`.
- iii) **Select:** `data['quality']` (Series), `data[['fixed acidity', 'alcohol']]` (DataFrame), `data.iloc[train_indices] / data.loc[train_index]` (positional vs label — the slide uses both), boolean filter `data[data['price'] > 80]`.
- iv) **Summarize:** `data['quality'].value_counts()` (the label distribution: quality 3: 10, 4: 53, 5: 681, 6: 638, 7: 199, 8: 18 — “lots of samples of average wines”, the slide’s words); `data.groupby('diagnosis').describe()` (the W1 programming notebook: breast-cancer, 569 rows $\times$ 33 cols; benign $B = 357$, malignant $M = 212$ — recorded in the notebook, the data file wasn’t in this dump to re-run).
- v) **Correlate:** `corr_matrix = exploration_set.corr(); corr_matrix['quality']` — alcohol 0.4762, volatile acidity -0.3906 (verified; the slide rounds to 0.48 / -0.38). Standard correlation $\in [-1, 1]$; “only captures linear relationship” (slide’s warning).

NaN semantics. A column with one missing value becomes `float64` (NaN is a float) — verified in §36.8. And NaN is *contagious and unfindable by equality*:

```
np.nan == np.nan    # False
3 + np.nan          # nan
```

So: `df.isna().sum()` counts missing per column (the slide’s check). The slide’s three-way discipline, verbatim in spirit: (a) *recorded-but-absent* $\rightarrow$ impute (`fillna`, or sklearn’s `SimpleImputer(strategy="median")` — its `fit` learns the medians on the training set, `transform` applies them; recorded, sklearn not installed here); (b) *too broken* $\rightarrow$ `dropna()`; (c) *doesn’t exist* (a pregnancy test for a male patient) $\rightarrow$ keep as NaN. Chapter 39 takes up imputation properly; the rule to carry forward is the slide’s: never learn the imputer on the full dataset — train statistics on train only.

Back to NumPy. The round trip the slide shows both ways: `wine_features_tr = pd.DataFrame(tr_features, columns=wine_features.columns)` (ndarray $\rightarrow$ labeled), and the reverse every model needs —

```
X = df[['area', 'rooms']].to_numpy()    # (10, 2) ndarray
y = df['price'].to_numpy()              # (10,) ndarray
```

— the X, y of §36.1, ready for Chapter 37.

Basically, ... “NumPy is the engine room; pandas is the loading dock with labels. You *receive* data as a DataFrame (named columns, mixed types, missing values visible), *explore* it with `head/describe/value_counts/groupby/corr`, *clean* it (`isna`, fill or drop), and *hand it over* as plain X, y arrays the moment the model starts. Every sklearn `fit(X, y)` in Chapters 37–38 is the end of a pandas pipeline.”

36.8 Worked end-to-end: load, clean, describe, extract

One small table, the whole §36.7 pipeline, every number re-run. (The CSV is inline so the example is self-contained.)

```
import pandas as pd, numpy as np, io
csv = """id,area,rooms,price
1,900,3,50
2,700,2,32
3,1200,4,66
4,1600,5,98
```

```

5,1500,5,85
6,1100,4,69
7,950,3,55
8,1300,4,72
9,800,,40
10,1400,5,90
"""
df = pd.read_csv(io.StringIO(csv))

```

Step 1 — look. `df.shape = (10,4)`. `df.dtypes`: `id`, `area`, `price` `int64`; `rooms` `float64` — the empty cell in row 9 forced it. `df.head(3)` shows rows 1–3; row 9’s `rooms` cell is `NaN`.

Step 2 — count the damage. `df.isna().sum()`: `rooms = 1`, everything else 0.

Step 3 — clean. Median of `rooms` = 4.0; `df['rooms'] = df['rooms'].fillna(4.0)`; `df.isna().sum().sum() = 0`. (This is the slide’s `SimpleImputer(strategy="median")` by hand, on a table small enough to check.)

Step 4 — describe. `df.describe()`: mean price 65.70, std 21.64, area mean 1145.0; `df['price'].corr(df['area']) = 0.98` — area nearly determines price, as built. `df.groupby('rooms')['price'].mean()`: 2→32.00, 3→52.50, 4→61.75, 5→91.00.

Step 5 — extract. `X = df[['area','rooms']].to_numpy() → (10,2)`; `y = df['price'].to_numpy() → (10,)`. Done: the labeled table is now the book’s (X, y) , and Chapter 37 can fit on it.

Note. The wine-quality deck runs exactly this pipeline at scale — `read_csv` → `head` → `info/describe` → `value_counts` → `corr` → `split` — and every one of its reported numbers re-verified here: 1599 rows, quality $\in [3, 8]$, alcohol–quality correlation 0.4762, volatile-acidity–quality -0.3906 .

36.9 Where this goes next

- i) Chapter 37 feeds this chapter’s (X, y) to sklearn regressors — `fit`, `predict`, and the transformers of the MLP Lecture-7 slide (`fit` learns parameters on the training set, `transform` applies them; `fit_transform` does both). The slide’s warning stands as the chapter’s last word: *never learn a transformer on the full dataset* — the same discipline as §36.7’s imputation note.
- ii) The vectorization habit (§36.2) is what makes every gradient-descent loop from Chapter 10 onward actually run; broadcasting (§36.3) is how a bias vector joins a score matrix in Chapter 41 without a single loop.
- iii) The (n, d) convention is now load-bearing: Chapter 38’s classifiers expect `X.shape == (n_samples, n_features)`. When a shape error bites, check the convention first (§36.1’s Note) — it is the cheapest bug to find and the most common one to make.

Problem set

1. **Broadcast or break.** For each pair, give the result shape or say it errors, with one line of the broadcasting rule: (i) $(4, 5) + (5,)$; (ii) $(4, 5) + (4, 1)$; (iii) $(4, 5) + (4,)$; (iv) $(6, 1, 5) + (5,)$.
2. **Reproduce the speedup.** On $n = 10^5$ numbers: (i) time a Python loop computing $\sum v^3$ vs `(a**3).sum()` (best of 5 each) and report the speedup; (ii) use `np.allclose` (not `==`) to check the two answers agree and say why `==` is the wrong test.
3. **Read the slice.** `M = np.arange(1, 17).reshape(4, 4)`. By hand, then check in code: (i) `M[1::2, :]`; (ii) `M[:, -1]`; (iii) `M[M > 12]`; (iv) write the mask expression that selects the *even* entries.
4. **Axis by hand.** `M = np.arange(1, 13).reshape(3, 4)`. (i) Compute `M.mean(axis=0)` and `M.sum(axis=1)` by hand. (ii) In one line each: which gives a per-feature statistic and which a per-point statistic? (iii) The TA colab calls `axis=0` “top-bottom”; give the collapse-rule reading that replaces it.

5. **Clean the table.** Load the §36.8 CSV in code. (i) Which column has missing values, and why did its dtype become `float64`? (ii) Fill with the median and verify zero NaNs remain. (iii) Report mean price per rooms via `groupby`, and the area-price correlation.
6. **Seeds.** (i) Run `np.random.default_rng(42).random(3)` twice — the two calls give identical output. Explain in one line why, and contrast with calling `.random(3)` twice on *one* `rng = np.random.default_rng(42)` object. (ii) The MLP slide's `split_train_test` calls `np.random.seed(42)` *inside* the function: what breaks if you call the function twice and expect different splits? (iii) In one line, why does any of this matter for Chapter 39's cross-validation?
7. **Course-sourced.** The MLP Week-1 programming notebook's Q5: `load_diabetes(as_frame=True, return_X_y=True)` returns the feature matrix as a `DataFrame` and the target as a `Series` (recorded in the notebook's solution output). (i) State both types. (ii) In one line, what does the `DataFrame` buy you over a bare `ndarray` here — answer in §22.3's vocabulary.

Solutions — Chapter 36: NumPy and Pandas for ML

Problem 1 — Broadcast or break

Rule: align on trailing axes; size-1 or missing axes stretch; anything else errors.

- i) $(4, 5) + (5,) \rightarrow (4, 5)$. The $(5,)$ aligns with the trailing axis and stretches down the 4 rows (the §36.3 row-vector case).
- ii) $(4, 5) + (4, 1) \rightarrow (4, 5)$. Trailing: 5 vs 1 $\rightarrow$ the 1 stretches; leading: 4 vs 4 $\rightarrow$ fine. (The §36.3 column-vector case.)
- iii) $(4, 5) + (4,) \rightarrow$ **ValueError**. Trailing axes 5 vs 4: neither is 1, nothing stretches.
- iv) $(6, 1, 5) + (5,) \rightarrow (6, 1, 5)$. $(5,)$ aligns with the trailing 5; the middle 1 stretches to 6 (and the leading 6 is untouched).

All four confirmed with `np.broadcast_shapes`.

Problem 2 — Reproduce the speedup

$n = 10^5$, best of 5 runs (numpy 1.26.4):

```
a = np.arange(10**5, dtype=float)
## loop:      s = 0.0
##           for v in a: s += v**3          -> 0.0224 s
## vectorized: s = (a**3).sum()            -> 0.00037 s
## speedup: 60.9x
```

- (i) $\approx 61\times$ on this machine (eg 2's $n = 10^6$ run gave \$ 246\times\$ — the gap widens with n because the per-element interpreter overhead dominates).
- (ii) `np.allclose(s1, s2) $\rightarrow$ True; s1 == s2 $\rightarrow$ False. Both sums print as 2.499950×10^{19} but differ in the last bits: the loop accumulates left-to-right in Python floats while NumPy sums in a different (blocked/pairwise) order, so the round-off differs. == demands bit-identical floats — the wrong test for any two independently computed floating results; allclose (relative + absolute tolerance) is the right one (§36.5, §36.2's 1.12×10^{-12} relative difference).`

Problem 3 — Read the slice

$$M = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}.$$

- (i) `M[1::2, :]` = rows 1, 3 (start at row 1, step 2): $\begin{bmatrix} 5 & 6 & 7 & 8 \\ 13 & 14 & 15 & 16 \end{bmatrix}.$

- (ii) `M[:, -1]` = last column: `[4, 8, 12, 16]`.
- (iii) `M[M > 12]` = flat array of entries `> 12`, in row-major order: `[13, 14, 15, 16]`. **Note:** a boolean mask always returns a *flat* (1-D) array — the selected positions don’t form a rectangle, so no shape is preserved.
- (iv) Even entries: `M[M % 2 == 0]` $\rightarrow$ `[2, 4, 6, 8, 10, 12, 14, 16]`.

Problem 4 — Axis by hand

$$M = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}.$$

- (i) `M.mean(axis=0)`: collapse rows — column means `[5, 6, 7, 8]` (e.g. column 0: $(1 + 5 + 9)/3 = 5$). `M.sum(axis=1)`: collapse columns — row sums `[10, 26, 42]` (e.g. row 0: $1 + 2 + 3 + 4 = 10$).
- (ii) `mean(axis=0)` is the per-*feature* statistic (one number per column); `sum(axis=1)` is the per-*point* statistic (one number per row).
- (iii) Collapse-rule reading: `axis=0` eats the row axis, leaving one value per column; `axis=1` eats the column axis, leaving one value per row. (“Top-bottom” is just a loose name for the same thing — the collapse is what the code does.)

Problem 5 — Clean the table

```
df = pd.read_csv(io.StringIO(csv))    # the §36.8 CSV
```

- (i) `df.isna().sum()` $\rightarrow$ `rooms: 1`, rest 0. The empty cell in row 9 is read as `NaN`; since `NaN` is a float, pandas upcasts the whole column — `rooms` is `float64` while `id/area/price` stay `int64` (§36.7’s `NaN` note).
- (ii) `df['rooms'].median() = 4.0`; `df['rooms'] = df['rooms'].fillna(4.0)`; then `df.isna().sum().sum() = 0`. Verified.
- (iii) `df.groupby('rooms')['price'].mean()`:
 - `rooms 2.0` $\rightarrow$ `32.00` (one house: `id 2`)
 - `rooms 3.0` $\rightarrow$ `52.50` (`ids 1, 7`: $(50 + 55)/2$)
 - `rooms 4.0` $\rightarrow$ `61.75` (`ids 3, 6, 8, 9`: $(66 + 69 + 72 + 40)/4 = 247/4$)
 - `rooms 5.0` $\rightarrow$ `91.00` (`ids 4, 5, 10`: $(98 + 85 + 90)/3 = 273/3$)

`df['price'].corr(df['area']) = 0.98` — area nearly determines price, as the table was built.

Problem 6 — Seeds

- (i) Identical output both times. Each call builds a *fresh* generator from the same seed, and a seeded generator is a fixed deterministic stream — same seed, same first three draws: `[0.77395605, 0.43887844, 0.85859792]`. Contrast: on *one* `rng` object, the second `.random(3)` continues the stream (draws 4–6), so the outputs differ.
- (ii) Nothing varies between the two calls: the seed is reset to 42 *inside* the function, so `np.random.permutation` replays the identical shuffle and both calls return the same train/test split. Expecting different splits breaks; expecting the *same* split (reproducibility) is exactly what the slide wants.
- (iii) Cross-validation re-trains on k different folds and compares scores — if the fold assignment isn’t seeded, every re-run shuffles differently and the scores wobble for no reason; you can’t tell a better model from a luckier shuffle. Seeding makes the comparison fair and replayable.

Problem 7 — Course-sourced

- (i) Feature matrix: `pandas.DataFrame`; target: `pandas.Series` (the W1 programming notebook's recorded solution output for `load_diabetes(as_frame=True, return_X_y=True)`; sklearn was not installed on this machine, so this is the notebook's output as recorded, not re-run).
- (ii) In §22.3's vocabulary: the `DataFrame` carries the **metadata** — the column names say what each coordinate means (`age`, `bmi`, ...) — so the table is human-readable while the numbers stay machine-usable; a bare `ndarray` keeps the numbers but drops the meaning.

Chapter 37: scikit-learn: regression workflows

Everything in §§37.1–37.7 comes from the MLP course’s regression weeks — the Week-3 “Linear Regression” and Week-4 “Polynomial Regression” slide decks (Dr. Ashish Tendulkar, IIT Madras) — and §§37.8–37.9 from the Week-2 “Data Preprocessing” slides, the Week-2 programming-questions notebook, and the “Notes by Sejal” Week-2 summary (pipeline construction, `set_params`, grid search with pipelines). (The MLT TA colab’s “Week 2 Programming Assignment” was checked — it is about kernel matrices, not sklearn workflows, and is not used.) This is the code for Chapter 23’s equations: §23.5 derived $\hat{\theta} = (X^T X)^{-1} X^T y$ on paper; here something calls `.fit()` and returns it.

A word on what was actually run. sklearn is not installed on this machine (see the chapter task note), so no `sklearn` call below was executed here. The code blocks are transcribed from the course slides and notebooks — verbatim unless a slide typo forced a fix (each fix is logged in `reviews/37-sklearn-regression.md`). Every *number* beside the code is either **[recorded]** — printed as output in the course material itself — or **[verified-NumPy]** — computed in this session through the mathematically identical closed form (§23.5’s normal equations, §28.4’s ridge form, the scaler/imputer definitions the slides give), which is provably what the sklearn call computes. The two kinds are marked wherever they could be confused.

Notation. The book’s standing convention (chapters 31–36): $X \in \mathbb{R}^{n \times d}$ holds the points $x^{(1)}, \dots, x^{(n)}$ as rows, $y \in \mathbb{R}^n$ holds the labels. sklearn uses the same convention — its Week-12 slide writes it out: training data shape $\rightarrow$ (n_samples, n_features), labels $\rightarrow$ (n_samples,).

37.1 The estimator contract: fit, predict, score

Def (estimator). An **estimator** = any sklearn object that learns parameters from data. Every estimator honors the same three-method contract:

- i) `fit(X_train, y_train)` — learn the parameters from the training data. For regression, this is §23.5’s job: find $\hat{\theta}$.
- ii) `predict(X_test)` — apply the learned parameters to new data: $\hat{y} = X_{\text{test}} \hat{\theta}$ (§23.1’s model, row by row).
- iii) `score(X_test, y_test)` — evaluate on held-out data (§37.5: for regressors this returns R^2).

The Week-3 slide’s remark, kept because it is the whole point of the chapter: “*These code snippets work for both LinearRegression and SGDRegressor, and for that matter to all regression estimators that we will study in this module. Why? All of them are estimators.*” Learn the contract once; every regressor from here on is a new object with the same three verbs.

eg 1 (the contract, on the §36.8 house data). Features = area, rooms; label = price. The 10-row table of §36.8, rooms imputed with the median 4.0:

```
from sklearn.linear_model import LinearRegression
lin_reg = LinearRegression()           # i)   choose the estimator
lin_reg.fit(X_train, y_train)          # ii)  learn:  theta-hat = (X^T X)^{-1} X^T y
```

```
lin_reg.predict(X_test)      # iii) apply:  $\hat{y} = X_{\text{test}} @ \hat{\theta}$ 
lin_reg.score(X_test, y_test) # iv) evaluate:  $R^2$  on the test set
```

Replace `LinearRegression()` with `Ridge(alpha=1e-3)` or `SGDRegressor()` and nothing else changes — the contract is the API.

Basically, ... “An sklearn estimator is a machine with three buttons: `fit` learns the weights, `predict` uses them, `score` grades them. `LinearRegression`, `Ridge`, `Lasso`, `SGDRegressor` are all the same machine with different engines inside — which is why the Week-3 slide keeps reusing one code shape for all of them.”

37.2 The workflow: split first, then fit

The slide’s four steps, in order — and the order is load-bearing:

- i) **Split** the data into train and test *before anything learns anything*:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

`random_state=42` fixes the shuffle, the same reproducibility discipline as §36.6’s seed (the SGD slide repeats it: “It’s a good idea to use a random seed of your choice... it helps us get reproducible results”). ii) **Fit** the estimator on the *training* set only. iii) **Training error** (a.k.a. *empirical error*) — how the model does on data it saw. iv) **Test error** (a.k.a. *generalization error*) — how it does on data it didn’t. Compare the two: close together is healthy; train tiny + test large is §23.9’s overfitting signature.

Note (the baseline habit). Before any real model, the slide builds a `DummyRegressor`:

```
from sklearn.dummy import DummyRegressor
dummy_regr = DummyRegressor(strategy="mean")
dummy_regr.fit(X_train, y_train)
dummy_regr.predict(X_test)
dummy_regr.score(X_test, y_test)
```

It predicts the training-set mean (strategies: `mean`, `median`, `quantile`, `constant`) for every input — the “constant model” of §37.5. Any model that cannot beat this on the test set is not a model, it is a rounding error with extra steps.

Note (the discipline, again). The imputation rule of §36.7 — learn the imputer on the training set, apply it everywhere — generalizes: *every* learning step (imputer, scaler, polynomial expansion, the regressor itself) is fitted on the training split and merely applied to the test split. §37.8 shows the machinery (`fit` vs `transform`) that enforces this; Chapter 39 makes it a full evaluation doctrine.

37.3 Two engines for the same line

The slide offers two estimators for linear regression — same contract, different internals:

Option A — the normal equation. `LinearRegression()` solves §23.5’s closed form. The appendix is explicit: it uses an SVD-based solve, cost $O(m^2)$ in the number of features m — “if we double the number of features, the training computation grows roughly 4 times” — and it is linear in n “as long as the training set fits in the memory.” Same optimum §23.3 proves, computed directly.

Option B — iterative optimization. `SGDRegressor()` walks downhill the way Chapter 10 does, one sample (or mini-batch) at a time: cost $O(knp)$ for k epochs and p average nonzero features per sample — linear in n , which is why the slide says “Use for large training set up ($> 10k$ samples)”, and adds: “For learning problems with small number of training examples, sklearn user guide recommends Ridge or Lasso.”

Note (SGD’s one demand: scale your features). “SGD is sensitive to feature scaling, so it is highly recommended to scale input feature matrix.” The slide’s fix is a two-step pipeline (full pipeline machinery in §37.8):

```
from sklearn.linear_model import SGDRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
sgd = Pipeline([
    ('feature_scaling', StandardScaler()),
    ('sgd_regressor', SGDRegressor())])
sgd.fit(X_train, y_train)
```

(Exceptions the slide carves out: word frequencies and indicator features “have intrinsic scale” and skip this.)

The SGD knobs, as the slides record them (defaults per the slides — `random_state` is deliberately set in real code even though the slides omit it “for sake of brevity”): i) `random_state=42` — reproducibility (§36.6’s whole sermon, in one argument). ii) `max_iter=100` — one epoch is one full pass over the training data; default is 1000. The slide’s rule of thumb: “SGD converges after observing approximately 10^6 training samples,” so a first guess is `max_iter = np.ceil(1e6 / n)`. iii) `learning_rate` — ‘`invscaling`’ (the slide’s stated default), ‘`constant`’, or ‘`adaptive`’; `eta0` sets the starting rate. `Invscaling` decays as $\eta(t) = \eta_0 / t^{\text{power_t}}$. iv) Stopping — either watch the training loss (`tol`, `n_iter_no_change`: stop when the loss fails to improve by `tol` for `n_iter_no_change` epochs) or hold out a validation fraction and watch the *validation score* (`early_stopping=True`, `validation_fraction=0.2`). v) `loss='squared_error'` — “studied in this course”; ‘`huber`’ is the robust alternative. `average=True` switches on averaged SGD (the returned `coef_` is the average of the iterates, which “works best with a larger number of features and a higher `eta0`”). vi) `warm_start=True` — resume from the previous run’s weights instead of re-initializing; the slide’s monitoring recipe fits one epoch at a time in a loop and evaluates the validation MSE after each.

Basically, ... “`LinearRegression` does the algebra (§23.5) in one shot — exact, but the matrix inverse gets expensive as features grow. `SGDRegressor` does Chapter 10’s hill-walking — cheap per step, scales to millions of rows, but needs scaled features, a seed, and a few knobs set. Same line out the other end.”

37.4 Model inspection: reading the weights back out

After `fit`, the learned parameters live in two attributes — for *every* linear estimator in this chapter:

```
linear_regressor.coef_      # w1, ..., wm (the weights)
linear_regressor.intercept_ # w0          (the bias term)
```

The model is §23.1’s, written the slide’s way: $\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_mx_m = w^T x$. And inference is one line on a properly shaped matrix — “Step 1: arrange data for prediction in a feature matrix of shape (`#samples`, `#features`)” — the (n, d) convention doing its job:

```
# Predict labels for feature matrix X_test
linear_regressor.predict(X_test)
```

eg 2 (the §36.8 houses, inspected) [verified-NumPy]. OLS on all 10 rows (rooms NaN $\rightarrow$ median 4.0, per §36.8), solved through §23.5’s normal equations — the identical computation `LinearRegression().fit(X, y)` performs:

```
lin_reg.coef_      # [0.0643, 1.7601] (area, rooms)
lin_reg.intercept_ # -14.805
```

Read it as the book reads equations: each extra square foot adds 0.0643 to the price, each extra room adds 1.7601, from a base of -14.805 . A 1000-sqft, 3-room house: $\hat{y} = -14.805 + 0.0643 \times 1000 + 1.7601 \times 3 = 54.8$. (Units are §36.8’s; the arithmetic is what matters — `coef_` is §23.5’s $\hat{\theta}$, `intercept_` its θ_0 .) ## 37.5 Evaluation: scores, errors, and R^2

Def (score vs error). The slide draws the line once, and it never moves: a **score** is a metric where *higher is better*; an **error** is one where *lower is better*. sklearn converts any error into a score by negating it — the `neg_` prefix: `neg_mean_squared_error`, `neg_mean_absolute_error`, `neg_root_mean_squared_error`, and so on (needed because cross-validation and grid search always *maximize*). The slide’s table:

error function	scoring name
<code>mean_absolute_error</code>	<code>neg_mean_absolute_error</code>
<code>mean_squared_error</code>	<code>neg_mean_squared_error /</code> <code>neg_root_mean_squared_error</code>
<code>mean_squared_log_error</code>	<code>neg_mean_squared_log_error</code>
<code>median_absolute_error</code>	<code>neg_median_absolute_error</code>

Def (R^2 , the coefficient of determination). `estimator.score(X_test, y_test)` returns

$$R^2 = 1 - \frac{u}{v}, \quad u = (Xw - y)^T(Xw - y), \quad v = (y - \hat{y}_{\text{mean}})^T(y - \hat{y}_{\text{mean}}).$$

u = residual sum of squares (how far the model’s predictions miss); v = total sum of squares (how far the labels miss their *own mean*). So R^2 = “what fraction of the label’s variance the model explains.” Three landmarks, straight from the slides: i) Best possible: 1.0 — $u = 0$, the model nails every point. ii) 0.0 — $u = v$: the model is exactly as good as always predicting the mean. (This is why §37.2’s `DummyRegressor(strategy="mean")` is the baseline to beat.) iii) Negative — “the model can be arbitrarily worse” than predicting the mean. A negative R^2 is the model telling on itself.

eg 3 (R^2 by the numbers, on eg 2’s fit) [verified-NumPy]. Training residuals $u = 160.4$, label variance $v = 4214.1$: $R^2 = 1 - 160.4/4214.1 = 0.9619$. Mean squared error = $u/n = 16.04$. Ninety-six percent of the price variance is explained by area and rooms — on the data it was fitted to, so read it as a goodness-of-fit, not a generalization claim (§37.2’s step iv is what earns that).

The `sklearn.metrics` shelf, with the slide’s one-line character for each: i) `mean_absolute_error(y_test, y_predicted)` — average $|y - \hat{y}|$, in the label’s own units. ii) `mean_squared_error` — the slide’s appendix lists MSE and RMSE as *the* evaluation measures; note the slide text once misspells the import as `mean_squarred_error` — the real name is `mean_squared_error` (fix logged in the review). iii) `r2_score` — “same as output of score.” iv) `mean_squared_log_error` — “useful for targets with exponential growths like population, sales growth”; “penalizes under-estimation heavier than the over-estimation.” v) `mean_absolute_percentage_error` — “sensitive to relative error.” vi) `median_absolute_error` — “robust to outliers” (the median shrugs at the one wild point). vii) `max_error` — the worst case, “only for single output regression.”

Basically, ... “ R^2 answers ‘how much better than just guessing the average?’ — 1 is perfect, 0 is the average-guesser, negative is worse than guessing. Everything else on the shelf is a different way of averaging the misses: squared (MSE) punishes big misses hardest, absolute (MAE) reads in plain units, median (MedAE) ignores outliers, max tells you the worst single embarrassment.”

37.6 Polynomial regression: expand the features, keep the machine

§23.8’s move, as code: polynomial regression = polynomial *transformation* + linear regression. “The only difference with respect to regular linear regression is that we are transforming the features and then performing regression” — §23.8’s lecture line, which the MLP slide re-implements:

```
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
# Two steps:
```

```
# 1. Polynomial features of desired degree (here degree=2)
# 2. Linear regression
poly_model = Pipeline([
    ('polynomial_transform', PolynomialFeatures(degree=2)),
    ('linear_regression', LinearRegression())])
# Train with feature matrix X_train and label vector y_train
poly_model.fit(X_train, y_train)
```

Swap the engine to SGD — “notice that there is a single line code change in two code snippets” (the slide’s words):

```
poly_model = Pipeline([
    ('polynomial_transform', PolynomialFeatures(degree=2)),
    ('sgd_regression', SGDRegressor())])
poly_model.fit(X_train, y_train)
```

eg 4 (what PolynomialFeatures actually builds) [recorded]. The slide’s example, output printed in the deck:

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np
X = np.arange(6).reshape(3, 2)
poly = PolynomialFeatures(degree=2)
poly.fit_transform(X)
# Data matrix:
# [[0 1]
#  [2 3]
#  [4 5]]
# After transformation:
# [[ 1.  0.  1.  0.  0.  1.]
#  [ 1.  2.  3.  4.  6.  9.]
#  [ 1.  4.  5. 16. 20. 25.]]
```

$[x_1, x_2] \rightarrow [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$ — exactly §23.8’s $\phi(x) = (1, x, x^2, \dots)^T$, now with the interaction term x_1x_2 . Two variants: `interaction_only=True` gives $[1, x_1, x_2, x_1x_2]$ (“ $[x_1^2, x_2^2]$ are excluded” — the slide’s note); `include_bias=False` drops the leading 1 when the estimator already fits its own intercept.

Note (capacity bites back). The slide’s appendix warns: “Since the polynomial regression uses more parameters (due to polynomial representation of the input), it is more prone to overfitting. We will study how to detect overfitting with learning curves and use of regularization to mitigate the risk of overfitting.” §23.9 said it on paper; here is the code version.

eg 5 (overfitting, caught red-handed) [verified-NumPy]. Ten noisy points from the smooth curve $f(x) = 4x^3 - 6x^2 + 3x$ on $[0, 1]$ (seed 0; noise $\sigma = 0.15$), judged on 500 fresh points. Each row is the sklearn pipeline shown above, its numbers verified through the identical closed form:

model	$\ w\ _2$	train MSE	test MSE
linear (degree 1)	0.85	0.027	0.048
poly, degree 9	84441.82	0.000	0.226
poly 9 + Ridge($\alpha = 0.5$)	0.38	0.031	0.044
poly 9 + Ridge($\alpha = 50$)	0.03	0.088	0.056

Read it the §23.9 way: degree 9 memorizes the ten training points (train MSE 0.000) with absurd weights ($\|w\|_2 = 84442$ — the wiggle between points from §23.9’s figure, now as a number) and pays on fresh data.

Ridge with $\alpha = 0.5$ shrinks the weights two-hundred-thousand-fold and wins on the test set; $\alpha = 50$ squeezes too hard and underfits. This is §28.6’s “ λ dial” with the dial labeled **alpha** — §37.7 names the translation.

Basically, ... “A degree-9 polynomial on 10 points is a student who memorized the practice test: 0.000 training error, nonsense elsewhere, weights in the tens of thousands. Regularization is the teacher confiscating the cheat sheet — smaller weights, slightly worse practice score, much better real score.”

37.7 Regularization in sklearn: the alpha dial

Chapter 28 derived ridge and lasso with the penalty dial called λ . sklearn calls it **alpha** — same role, and for ridge the translation is exact:

Note (the alpha $\leftrightarrow$ λ convention). The book’s ridge (§28.3) minimizes $\frac{1}{2}\|Xw - y\|^2 + \frac{\lambda}{2}\|w\|^2$; sklearn’s Ridge minimizes $\|Xw - y\|^2 + \alpha\|w\|^2$. Multiply the book’s objective by 2 and the two are the same problem — so **book- λ = sklearn-alpha, exactly**, for ridge. (For lasso the objectives differ in normalization — sklearn’s Lasso divides the squared error by $2n$ — so treat its **alpha** as its own dial, not a port of the book’s λ .)

```
from sklearn.linear_model import Ridge
ridge = Ridge(alpha=1e-3)
# fit, score, predict work exactly like other linear regression estimators
```

or, equivalently, the SGD route (`penalty='l2'` is even SGDRegressor’s default):

```
from sklearn.linear_model import SGDRegressor
sgd = SGDRegressor(alpha=1e-3, penalty='l2')
```

Lasso, the same shape (§28.7’s feature-selector):

```
from sklearn.linear_model import Lasso
lasso = Lasso(alpha=1e-3)
# or: SGDRegressor(alpha=1e-3, penalty='l1')
```

Both at once — elastic net, “a convex combination of L1 (Lasso) and L2 (Ridge)”: `SGDRegressor(penalty='elasticnet', l1_ratio=0.3)` means $0.3 \times \text{L1} + 0.7 \times \text{L2}$ (“L2 takes higher weightage in this formulation” — the slide’s words).

Regularization meets the polynomial pipeline — the slide’s recipe for §37.6’s overfit:

```
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
poly_model = Pipeline([
    ('polynomial_transform', PolynomialFeatures(degree=2)),
    ('ridge', Ridge(alpha=1e-3))])
poly_model.fit(X_train, y_train)
```

(Replace `Ridge` with `Lasso` for the lasso version; the slide gives both.)

Choosing alpha. Two course-sanctioned routes (§28.10’s validation discipline, as code): the built-in path — `RidgeCV` / `LassoCV` / `ElasticNetCV`, which “fit data for a range of values of some parameter almost as efficiently as fitting the estimator for a single value” (`RidgeCV(alphas=[...])` tries a list; the winner is reported in its `alpha_` attribute) — or plain cross-validation over a `Ridge`/`Lasso` grid. The full cross-validation machinery is Chapter 39’s subject; the one-line version lives here because the dial is useless without a way to set it.

Basically, ... “sklearn’s **alpha** is the book’s λ wearing a different name tag (exactly, for ridge). `RidgeCV` is the lazy-good way to pick it: hand it a list of candidates, it cross-validates each, and tells you the winner. The polynomial that memorized your data in eg 5 gets fixed by exactly this.”

37.8 Transformers and pipelines: learn on train, apply everywhere

The Week-2 preprocessing slides give the second contract of the chapter — the **transformer** contract:

Def (transformer). `fit(X)` *learns* parameters from data; `transform(X)` *applies* the learned transformation to (new) data; `fit_transform(X)` does both at once. The slide’s discipline, verbatim in spirit: the same preprocessing must be applied to train and test, so fit on the training set and transform both.

i) **Scaling.** `StandardScaler` learns μ, σ per feature and maps $x' = (x - \mu)/\sigma$ [recorded]:

```
ss = StandardScaler()          # (the slide prints StandardScaler(); the class is
    ↪ StandardScaler)
x = [[4],[3],[2],[5],[6]]      # mu = 4, sigma = sqrt(2)
ss.fit_transform(x)             # [[0],[-1/sqrt(2)],[-2/sqrt(2)],[1/sqrt(2)],[2/sqrt(2)]]
```

transformed mean 0, std 1. Siblings: `MinMaxScaler` ($x' = (x - x_{\min})/(x_{\max} - x_{\min})$, range $[0, 1]$), `MaxAbsScaler` (range $[-1, 1]$). ii) **Imputation.** `SimpleImputer(strategy='mean'|'median'|'most_frequent'|'constant')` [recorded]:

```
si = SimpleImputer(strategy='mean')
# X = [[7,1],[nan,8],[2,nan],[9,6]] -> col means 6 and 5
si.fit_transform(X)      # [[7,1],[6,8],[2,5],[9,6]]
```

`KNNImputer(n_neighbors=k)` fills each gap with the mean of the k nearest rows by Euclidean distance (the slide works one by hand: $[1, 2, \text{nan}]$ ’s two nearest rows give $(3 + 5)/2 = 4$); `MissingIndicator` adds the binary “was missing” columns (§36.7’s discipline: *recorded-but-absent* → impute, and learn the statistics on train only).

Def (pipeline). A Pipeline chains transformers and ends in an estimator: `fit` calls `fit_transform` down the chain and `fit` on the last step; `predict/score` transform-then-predict. The Week-2 programming notebook’s real pipeline (blood-donation data, 5 features; verbatim, lightly trimmed):

```
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import StandardScaler, OrdinalEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, FeatureUnion

num_pipe = Pipeline([("colselect", ColumnTransformer([("select", "passthrough",
    ↪ [0,1,2,3])))),
                    ("impute", SimpleImputer()),
                    ("scale", StandardScaler())])
cat_pipe = ColumnTransformer([("encode", OrdinalEncoder(), [4])])
pipe = FeatureUnion([("numerical", num_pipe),
                    ("categorical", cat_pipe)])
```

`ColumnTransformer` applies *different* transformers to *different* columns and concatenates; `FeatureUnion` runs transformers in parallel and glues the outputs side by side. Steps are reached by name — `pipe.set_params(pca__n_components=2)` — the double underscore addressing that §37.9’s grid search relies on.

Note (the leak this machinery exists to prevent). `fit_transform(X_all)` before the split learns the scaler’s μ, σ (or the imputer’s medians) from the test points too — test information leaking into training statistics. The pipeline fixes it structurally: `pipe.fit(X_train)` then `pipe.predict(X_test)` — the test set only ever sees `transform`. This is §36.7’s imputation note grown into an architecture.

eg 6 (the full workflow, one object). Impute → scale → polynomial → ridge, on the §36.8 houses (the NaN in row 9’s rooms is real this time), fitted on the §36.6 split (seed 42, `test_size=0.2`: test = ids 9 and 2 — “the same two, every run”):

```

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ('impute', SimpleImputer(strategy='median')), # learns medians on TRAIN only
    ('scale', StandardScaler()), # learns mu, sigma on TRAIN only
    ('poly', PolynomialFeatures(degree=2)), # [1, x1, x2, x1^2, x1*x2, x2^2]
    ('ridge', Ridge(alpha=1e-3))] # book-lambda = 1e-3
pipe.fit(X_train, y_train)
pipe.score(X_test, y_test) # R^2 on the held-out rows

```

What each step learned on the 8 training rows [verified-NumPy]: rooms median 4.0 (area median 1250.0, unused — no NaNs there); scaler $\mu = (1243.75, 4.125)$, $\sigma = (236.43, 0.78)$; the first training row’s polynomial expansion [1, -0.61, -0.16, 0.37, 0.10, 0.03]. None of these numbers saw the test rows — that is the entire point of the object.

Basically, ... “A pipeline is the §37.2 workflow with the discipline built in: `fit` learns every step’s parameters from the training data in order, `predict` replays the same transformations on new data. `ColumnTransformer` says ‘different columns, different treatment’; `FeatureUnion` says ‘run these side by side and glue.’ Write the workflow once, and the leak the book keeps warning about becomes un-writable.”

37.9 Choosing the dial: validation, briefly

Two recurring questions now have code answers; both get their full treatment in Chapter 39, and both already appeared on paper in §§23.9 and 28.10:

- i) **Which degree?** `GridSearchCV` over the pipeline’s own parameter — the double underscore reaches inside the named steps (the slide’s code, typo `POLynomialFeatures` fixed):

```

from sklearn.model_selection import GridSearchCV
param_grid = [{'poly__degree': [2, 3, 4, 5, 6, 7, 8, 9]}]
pipeline = Pipeline(steps=[('poly', PolynomialFeatures()),
                           ('sgd', SGDRegressor())])
grid_search = GridSearchCV(pipeline, param_grid, cv=5,
                           scoring='neg_mean_squared_error',
                           return_train_score=True)
grid_search.fit(X_train, y_train)

```

- ii) **Which alpha?** `RidgeCV`/`LassoCV` (built-in, §37.7), or the same grid search with `'ridge__alpha'`.
- iii) **Am I over- or underfitting?** `learning_curve` plots training vs cross-validated error against training-set size — the code version of §23.9’s train/test comparison: both errors high means underfit, a gap that won’t close means overfit.

Note (the HPT data split). The slide is explicit about the three-way split this machinery assumes: tune on train/validation, retrain the winner on train+validation, and report *once* on the test set — “note that the test set was not used in hyper-parameter search and model retraining.” The test set is spent exactly once.

37.10 Where this goes next

- i) **Classification, same contract.** Chapter 38 replays this chapter with labels instead of numbers: the Week-2 notebook already used `LogisticRegression()` inside `RFE/SequentialFeatureSelector` — `fit/predict/score` unchanged, only the estimators and the metrics change.

- ii) **The dial-setting machinery, properly.** Chapter 39 takes §37.9’s sketch — cross-validation iterators (KFold, ShuffleSplit, LeaveOneOut), `cross_val_score/cross_validate`, grid vs randomized search — and makes it a subject.
- iii) **The neural turn.** The Week-12 slide’s `MLPRegressor` “trains using backpropagation... uses the square error as the loss function, and the output is a set of continuous values” — a regressor honoring the same `fit/predict` contract, whose insides are Chapter 41’s neurons. The contract outlives linear models.
- iv) **Debugging the workflow.** Chapter 40: when `score` is negative and the pipeline looks right, the discipline for finding out why.

Problem set

- The contract.** The Week-3 slide claims the `fit/predict/score/coef_/intercept_` snippets “work for both `LinearRegression` and `SGDRegressor`, and for that matter to all regression estimators that we will study in this module.” (i) In one line each, say what `fit`, `predict`, and `score` compute for a regressor, in §23.1’s notation ($\hat{\theta}$, $\hat{y} = X\hat{\theta}$). (ii) Which of the three methods does a *transformer* (e.g. `StandardScaler`) not have, and which does it have instead?
- R^2 by hand.** $y = (2, 4, 5, 4, 5)$, predictions $\hat{y} = (1, 5, 4, 4, 6)$. (i) Compute u and v and hence R^2 . (ii) What would a `DummyRegressor(strategy="mean")` score on this data, and why — answer from the definition of v ?
- Pick the metric.** For each situation, name the slide’s metric and say why in one line: (i) house prices with a few misrecorded mansions in the data; (ii) a model whose contract penalizes the single worst prediction; (iii) forecasting user counts that grow exponentially; (iv) reporting error as “typically off by $x\%$ ”.
- Course-sourced.** The slide’s `PolynomialFeatures` example (eg 4) transforms $[[0,1], [2,3], [4,5]]$ with `degree=2`. (i) Write the transformed matrix by hand and check against the recorded output. (ii) Repeat with `interaction_only=True`. (iii) In §23.8’s notation, what is $\phi(x)$ for each variant?
- Score or error?** `cross_val_score` is maximizing something. (i) Why does sklearn need the `neg_` versions of error metrics at all? (ii) If `cross_val_score(..., scoring='neg_mean_squared_error')` returns -16.04 , what is the MSE? (iii) Why is there no `neg_r2_score`?
- Spot the leak.** A classmate writes:

```
X_imp = SimpleImputer(strategy='median').fit_transform(X)    # X has NaNs
X_train, X_test, y_train, y_test = train_test_split(X_imp, y, random_state=42)
```

- (i) Name the discipline this violates (§36.7 / §37.8’s Note) and say what leaks where. (ii) Rewrite it as a `Pipeline` so the leak is structurally impossible.
- The alpha translation.** The book’s ridge (§28.3) minimizes $\frac{1}{2}\|Xw - y\|^2 + \frac{\lambda}{2}\|w\|^2$; sklearn’s `Ridge` minimizes $\|Xw - y\|^2 + \alpha\|w\|^2$. (i) Show the two minimizers coincide when $\alpha = \lambda$. (ii) Why can’t you port a tuned `alpha` from sklearn’s `Lasso` straight into the book’s §28.7 formula as λ ?
- Diagnose the fit.** A degree-9 polynomial pipeline reports train MSE 0.000, test MSE 0.226, $\|w\|_2 = 84442$ (eg 5’s numbers). (i) Diagnose in §23.9’s vocabulary. (ii) Name two fixes from this chapter and say which hyperparameter each one tunes (`degree` via `GridSearchCV`, or `alpha` via `RidgeCV`).

Solutions — Chapter 37: scikit-learn: regression workflows

Problem 1 — The contract

- (i) In §23.1’s notation: `fit(X_train, y_train)` computes $\hat{\theta}$ — the parameter vector minimizing the training loss (for `LinearRegression`, §23.5’s $(X^T X)^{-1} X^T y$). `predict(X_test)` computes $\hat{y} = X_{\text{test}} \hat{\theta}$, one prediction per row. `score(X_test, y_test)` returns $R^2 = 1 - u/v$ on the held-out pair (§37.5).

- (ii) A transformer has no `predict` — it transforms features, it doesn't model labels. Instead of `predict` it has `transform(X)`: apply the *learned* transformation. (`fit` learns e.g. μ, σ ; `transform` maps $x' = (x - \mu)/\sigma$; `fit_transform` does both.)

Problem 2 — R^2 by hand

- (i) Residuals $y - \hat{y} = (1, -1, 1, 0, -1)$: $u = 1 + 1 + 1 + 0 + 1 = 4$. Mean $\bar{y} = 20/5 = 4$; deviations $(-2, 0, 1, 0, 1)$: $v = 4 + 0 + 1 + 0 + 1 = 6$. $R^2 = 1 - 4/6 = 1/3 \approx 0.333$.
- (ii) It scores exactly 0.0. The dummy predicts $\bar{y} = 4$ for every point, so its residual sum of squares *is* $v = 6$ (each residual is $y_i - \bar{y}$), and $R^2 = 1 - v/v = 0$. That is the §37.5 landmark: the constant model sits at $R^2 = 0$ by construction — v is literally “how far the labels miss their own mean.”

Problem 3 — Pick the metric

- (i) `median_absolute_error` — the median shrugs at the misrecorded mansions (“robust to outliers”); MSE would square them and let them dominate.
- (ii) `max_error` — the worst-case metric, by definition the single largest $|y - \hat{y}|$.
- (iii) `mean_squared_log_error` — the slide’s pick for “targets with exponential growths”; it also “penalizes under-estimation heavier than the over-estimation,” which is the right asymmetry when missing growth hurts more than overshooting it.
- (iv) `mean_absolute_percentage_error` — “sensitive to relative error”: $|y - \hat{y}|/|y|$ averaged, i.e. “typically off by $x\%$ ”.

Problem 4 — Course-sourced

- (i) $[x_1, x_2] \rightarrow [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$: row $[0, 1] \rightarrow [1, 0, 1, 0, 0, 1]$; row $[2, 3] \rightarrow [1, 2, 3, 4, 6, 9]$; row $[4, 5] \rightarrow [1, 4, 5, 16, 20, 25]$ — matches the deck’s recorded output exactly.
- (ii) `interaction_only=True` drops the pure powers: $[x_1, x_2] \rightarrow [1, x_1, x_2, x_1x_2]$: $[1, 0, 1, 0]$, $[1, 2, 3, 6]$, $[1, 4, 5, 20]$.
- (iii) §23.8’s $\phi(x) = (1, x, x^2, \dots)^T$ generalized: full variant $\phi(x_1, x_2) = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2)^T$; interaction-only $\phi(x_1, x_2) = (1, x_1, x_2, x_1x_2)^T$. Both are “linear in θ ” models (§23.8’s Note) — the curvature lives in ϕ , not in the fit.

Problem 5 — Score or error?

- (i) `cross_val_score` (and `GridSearchCV`) always *maximize* the scoring value. An error is “lower is better,” so maximizing raw MSE would pick the *worst* model; the `neg_` prefix flips it into a score where higher is better, and the `argmax` then does the right thing.
- (ii) $\text{MSE} = 16.04$ — negate back.
- (iii) R^2 is already a score (higher is better: 1.0 best, 0.0 baseline, negative bad), so negating it would invert the ordering — `scoring='r2'` is used as-is.

Problem 6 — Spot the leak

- (i) It violates the never-learn-on-full-data discipline (§36.7’s imputation note, §37.8’s Note): `fit_transform(X)` learns the column medians from *all* rows, including the ones that will become the test set. Test information (the medians) leaks into the training data — the test set is no longer unseen.
- (ii)


```

from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
pipe = Pipeline([
    ('impute', SimpleImputer(strategy='median')),
    ('regressor', LinearRegression())])
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
pipe.fit(X_train, y_train)    # medians learned on train only
pipe.score(X_test, y_test)    # test rows only ever see transform

```

The split happens on the raw X (NaNs and all); the imputer is fitted inside `pipe.fit` on the training rows alone. The leak is now unwritable — there is no call sequence that fits the imputer on the test rows.

Problem 7 — The α translation

- (i) The book minimizes $J(w) = \frac{1}{2}\|Xw - y\|^2 + \frac{\lambda}{2}\|w\|^2$; sklearn minimizes $J_{\text{sk}}(w) = \|Xw - y\|^2 + \alpha\|w\|^2$. But $J(w) = \frac{1}{2}J_{\text{sk}}(w)$ when $\alpha = \lambda$ — multiplying an objective by the positive constant $\frac{1}{2}$ does not move its minimizer. Same argmin, so book- $\lambda = \text{sklearn-alpha}$ exactly (for ridge).
- (ii) sklearn's Lasso minimizes $\frac{1}{2n}\|y - Xw\|^2 + \alpha\|w\|_1$ — the squared error is *averaged over the n samples* before the penalty is added. The book's §28.7 objective, $\frac{1}{2}\|Xw - y\|^2 + \frac{\lambda}{2}\|w\|_1$, has no $1/n$. Dividing the book's objective by n shows the two coincide only when $\alpha = \lambda/(2n)$ — so the same numeric dial is a $2n$ -times-stronger penalty in the book's formulation. You cannot copy the number across; re-tune it.

Problem 8 — Diagnose the fit

- (i) Overfitting, textbook (§23.9): training error 0.000 — the degree-9 curve memorizes the ten points — while test error 0.226 is an order of magnitude worse, and $\|w\|_2 = 84442$ is the §28.1 signature of fitting noise with huge canceling weights.
- (ii) Fix 1: shrink the hypothesis class — `GridSearchCV` over `poly__degree` (tunes `degree`; §23.9's "choose a smaller m by validation"). Fix 2: keep degree 9 but penalize the weights — `RidgeCV` over `ridge__alpha` (tunes `alpha`, the book's λ ; §28.10's validation discipline). Either is legitimate; eg 5 shows Fix 2 dropping test MSE from 0.226 to 0.044.

Chapter 38: scikit-learn: classification workflows

Everything in §§38.1–38.7 comes from the MLP Week-5 “Classification functions in sci-kit learn” slide deck (Dr. Ashish Tendulkar, IIT Madras) and the “Notes by Sejal” Week-5 summary; §§38.8–38.10 draw additionally on the Week-5 programming-questions solution notebook (the MNIST perceptron experiments). This is the code for Chapters 29–35’s equations: §31.8 derived $h(x) = \sigma(\theta^T x + \theta_0)$ on paper; here something calls `.fit()` and returns it.

A word on what was actually run. `sklearn` is not installed on this machine (see the chapter task note), so no `sklearn` call below was executed here. The code blocks are transcribed from the course slides and notebooks — verbatim unless a slide typo forced a fix (each fix is logged in `reviews/38-sklearn-classification.md`). Every *number* beside the code is either **[recorded]** — printed as output in the course material itself — or **[verified]** — worked by hand in this session from the recorded confusion-matrix entries (accuracy/precision/recall/F1 are four arithmetic operations on those entries; nothing is invented). The two kinds are marked wherever they could be confused.

Notation. The book’s standing convention (chapters 31–36): $X \in \mathbb{R}^{n \times d}$ holds the points $x^{(1)}, \dots, x^{(n)}$ as rows. For classifiers the labels are discrete: $y \in \{-1, +1\}^n$ (the §22.9 convention), $\{0, 1\}^n$, or plain strings like ‘apple’. `sklearn` accepts all three.

38.1 The classifier contract: `fit`, `predict`, `decision_function`, `predict_proba`, `score`

Def (classifier). A **classifier** = an estimator whose `predict` returns a *label*, not a number. It honors the same contract as §37.1’s regressors, with two additions:

- i) `fit(X_train, y_train)` — learn the parameters from the training data. For logistic regression, this is §31.10’s job: minimize the cross-entropy.
- ii) `predict(X_test)` — apply the learned parameters to new data: return a class label per row.
- iii) `decision_function(X)` — the *confidence score* before the label is snapped on: for a linear classifier, the raw value $w^T x + b$ (whose sign is the prediction — §22.9’s linear separator, un-snapped).
- iv) `predict_proba(X)` — for classifiers that model probabilities (§31.8’s $P(y = 1 | x)$): the probability of each class per row.
- v) `score(X_test, y_test)` — evaluate on held-out data. For classifiers this returns the **mean accuracy** (fraction correct — §22.9’s $1 - L$).

The slide lists these as the common methods “for model training, prediction and evaluation,” and adds the miscellaneous shelf: `get_params` / `set_params` (the §37.8 double-underscore addressing works here too), `densify()` / `sparsify()` (convert the coefficient matrix between dense and sparse formats).

The slide’s two-API split, which organizes the whole chapter:

generic	specific
SGDClassifier — gradient descent; you pick the loss	LogisticRegression, Perceptron, RidgeClassifier (least-squares), KNeighborsClassifier, SVC, GaussianNB — specialized solvers

Basically, ... “Chapter 37’s regressors predicted numbers; classifiers predict labels. The machine is the same — `fit` learns, `predict` labels, `score` grades (accuracy now, not R^2). Two extras: `decision_function` shows the raw score behind the label, `predict_proba` shows the probability. And sklearn splits the shelf in two: one generic engine where *you* name the loss, and one-object-per-algorithm with a purpose-built solver.”

38.2 The zoo, as the course presents it

The slide names five specific classifier families; each is the code for one of the book’s math chapters:

```
from sklearn.neighbors import KNeighborsClassifier      # §29.2's voters: the k nearest
    ↪ decide
from sklearn.tree import DecisionTreeClassifier        # §29.9's recursive partitions
from sklearn.svm import SVC                          # ch32-33's max-margin machines
from sklearn.naive_bayes import GaussianNB           # §30.7's counters: training is
    ↪ counting
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier # ch34's
    ↪ committees
```

(The class names are the standard sklearn API for the algorithms the slide lists; the slide itself codes only the linear family below — §§38.3–38.6 — which is where this chapter spends its pages.)

Note (the zoo’s shared contract). Every object above takes `fit(X, y)`, answers `predict(X)`, and reports `score(X, y)` as accuracy. Learn §§38.3–38.6 once and the pattern transfers: the math chapters (29–34) change what happens *inside* `fit`; the three verbs don’t move.

38.3 Least-squares classification: RidgeClassifier

§35.5’s idea as an estimator: least-squares classification (LSC) treats the labels as numbers and runs ridge regression on them.

The construction (the slide’s, kept whole). Binary classification: i) Convert binary targets to $\{-1, +1\}$. ii) Treat it as a regression task, minimizing the penalized residual sum of squares

$$\min_w \|Xw - y\|_2^2 + \alpha \|w\|_2^2$$

— §28.3’s ridge objective, now with class labels in the y slot. iii) The predicted class is the **sign** of the regressor’s prediction — §22.9’s $\text{sign}(w^T x + b)$ again.

Multiclass: “treated as multi-output regression; predicted class corresponds to the output with the highest value.”

```
from sklearn.linear_model import RidgeClassifier
ridge_classifier = RidgeClassifier()                # step 1: choose the estimator
ridge_classifier.fit(X_train, y_train)              # step 2: learn, on train only
y_pred = ridge_classifier.predict(X_test)           # labels, not numbers
```

The alpha dial (the slide’s): `RidgeClassifier(alpha=0.001)`; default 0.1 per the slide; must be positive; larger α = stronger regularization — the same dial as §37.7’s **Ridge**, and the same book- λ (§28.3) wearing the sklearn name.

Solvers (the slide’s list): `svd` (SVD of the feature matrix), `cholesky` (`scipy.linalg.solve`, closed form), `sparse_cg` (conjugate gradient; “for large scale data”), `lsqr` (“fastest”, regularized least-squares routine), `sag/saga` (“when both `n_samples` and `n_features` are large”; “fast convergence is only guaranteed on features with approximately the same scale” — §37.3’s scaling sermon, repeated), `lbfgs`. The default `solver='auto'` picks by data type (the slide’s pseudocode):

```
if solver == 'auto':
    if return_intercept:
        # only sag supports fitting intercept directly
        solver = "sag"
    elif not sparse.issparse(X):
        solver = "cholesky"
    else:
        solver = "sparse_cg"
```

Intercept: `fit_intercept=True` by default; if the data is already centered, set it `False` and no intercept is used. `RidgeClassifierCV` = the same estimator with built-in cross-validation (Chapter 39 owns the machinery; §37.7’s `RidgeCV` is its regression twin).

Basically, ... “Take ridge regression (§28.3), feed it ± 1 labels instead of prices, and read the *sign* of the output as the class. That’s the whole classifier. `alpha` is the familiar regularization dial; the solver list is just different routes to the same closed form.”

38.4 The perceptron, as code

§31.2’s neuron, one import away:

```
from sklearn.linear_model import Perceptron
perceptron_classifier = Perceptron()
perceptron_classifier.fit(X_train, y_train)
```

The identity that matters (the slide’s): `Perceptron()` shares its implementation with `SGDClassifier`, and

```
Perceptron()
# is
SGDClassifier(loss="perceptron", eta0=1, learning_rate="constant", penalty=None)
```

The perceptron (§31.3’s “pay only for mistakes” loss) is one loss-setting of the generic engine — §38.6’s table makes this a pattern.

The knobs (the slide’s parameter list): `penalty` (default 'l2'), `l1_ratio` (default 0.15), `alpha` (default 10^{-4}), `fit_intercept` (default `True`), `max_iter` (default 1000), `tol` (10^{-3}), `n_iter_no_change` (5), `eta0` (1), `validation_fraction` (0.1), `early_stopping` (`False`), `warm_start` (re-initialize from the previous run’s weights), and `partial_fit` (train one epoch at a time — the course solution uses it to watch the bias evolve).

eg 1 (a real perceptron, on MNIST digits 6 vs 9) [recorded]. The course’s practice solution builds a binary task from MNIST: digit-6 = positive (+1), digit-9 = negative (−1), first 10,000 images for training. The training split holds 1014 sixes and 978 nines [recorded]; the labels are stacked (+1s then −1s) and shuffled with `random_state=1729` [recorded]:

```
from sklearn.linear_model import Perceptron
clf = Perceptron(random_state=1729,
                  eta0=1, max_iter=10,
                  shuffle=False,
                  validation_fraction=0.1,
                  fit_intercept=True,
```

```

        penalty=None,
        warm_start=False)
clf.fit(x_train1, y_train1)
clf.coef_[0, 69]      # the 70th feature's weight (0-based indexing) after 10 epochs
# 605.0 [recorded]

```

`clf.coef_` is §31.1’s weight vector, `clf.intercept_` the bias — the same two attributes §37.4 read off a regressor. And the bias across epochs, via `partial_fit` [recorded]:

```

updates = []
for i in range(5):
    clf.partial_fit(x_train1, y_train1, classes=np.unique(y_train1))
    updates.append(clf.intercept_[0])
# [-1.0, -4.0, -4.0, -6.0, -5.0] [recorded]

```

Each epoch nudges the bias by $\pm\eta$ per mistake (§31.4’s update rule, with $\eta = 1$): the bias walks $-1 \rightarrow -4 \rightarrow -4 \rightarrow -6 \rightarrow -5$ as the perceptron argues with the training rows.

Basically, ... “The perceptron is one line of sklearn — and secretly one *setting* of the generic SGD engine. Its weights live in `coef_` exactly where the regressor’s did. The course’s MNIST eg shows the bias literally walking, epoch by epoch, as mistakes get corrected.” ## 38.5 Logistic regression, as code

§31.8’s model — $P(y = 1 | x) = \sigma(\theta^T x + \theta_0)$ — as an estimator (“also known as logit regression, maximum-entropy classifier, log-linear classifier,” per the slide):

```

from sklearn.linear_model import LogisticRegression
logit_classifier = LogisticRegression()
logit_classifier.fit(X_train, y_train)

```

The objective (the slide’s, kept verbatim in shape): the implementation minimizes

regularization penalty + $C \times$ cross-entropy loss,

i.e. $\arg \min_w \text{penalty} + C \cdot L_{\text{nl}}$ — and §31.9 proved that cross-entropy loss *is* the negative log-likelihood. So `LogisticRegression.fit` is §31.10’s training, outsourced to a solver.

The C dial — read this twice. sklearn’s logistic regression does *not* take `alpha`; it takes **C**, **the inverse of the regularization rate**: i) C multiplies the *loss*, not the penalty. Smaller $C \rightarrow$ the penalty dominates $\rightarrow$ **stronger** regularization; larger $C \rightarrow$ weaker. ii) In the book’s §28.3 convention (objective = loss + $\lambda \cdot$ penalty), dividing the slide’s objective by C gives loss + $\frac{1}{C} \cdot$ penalty — so $\lambda = 1/C$, i.e. **sklearn’s C = 1/alpha**. §37.7’s ridge `alpha` *is* the book’s λ ; logistic regression’s C is its reciprocal. Same dial, flipped name tag.

Penalties and solvers (the slide’s table):

solver	penalties supported
'newton-cg'	12, none
'lbfgs'	12, none
'liblinear'	11, 12
'sag'	12, none
'saga'	elasticnet, 11, 12, none

The slide’s selection rules: small datasets $\rightarrow$ 'liblinear'; large ones $\rightarrow$ 'sag'/'saga'; unscaled data $\rightarrow$ 'liblinear', 'lbfgs', 'newton-cg' are robust; multinomial loss (§31.12) needs 'newton-cg', 'sag', 'saga', or 'lbfgs' — 'liblinear' is one-vs-rest only. Default: `solver='lbfgs'`, `penalty='l2'` (“regularization is applied by default because it improves numerical stability”).

Class imbalance (the slide’s note): `class_weight` in the constructor — “mistakes in a class are penalized by the class weight; higher value here would mean higher emphasis on the class.” Every classifier estimator in sklearn carries it:

```
LogisticRegression(class_weight={0: 1, 1: 10})    # a mistake on class 1 costs 10x
```

LogisticRegressionCV = logistic regression with built-in cross-validation over `C` (and `l1_ratio`): `cv` picks the iterator, `scoring` the metric, `Cs` the candidate strengths. `refit=True` averages *scores* across folds, takes the winning `C`, and refits once; `refit=False` averages the *coefficients*, *intercepts*, and *C* themselves. (Chapter 39 gives CV its full treatment; the stratified iterators it needs are §38.9’s.)

Basically, ... “Logistic regression in sklearn = §31.8’s sigmoid model + §31.9’s cross-entropy loss + a solver doing §31.10’s optimization. The one trap is the dial: it is called `C`, it multiplies the *loss*, and it is the *reciprocal* of the regularization strength — $C = 1/\lambda = 1/\alpha$. Small *C*, strong regularization. And `class_weight` is how you tell it that missing class 1 hurts ten times more.”

38.6 SGDClassifier: one engine, many classifiers

§37.3’s hill-walker, now for labels: “a simple yet very efficient approach to fitting linear classifiers under convex loss functions” — one sample at a time, model updated along the way with a decreasing learning rate. Same demands as its regression twin: **shuffle the training data before fitting, standardize the features** (sensitive to scaling — §37.3’s sermon applies verbatim).

The loss table (the slide’s — this is the whole API):

loss=	classifier trained
'hinge' (default)	(soft-margin) linear SVM
'log'	logistic regression
'modified_huber'	smoothed hinge: outlier-tolerant, with probability estimates
'squared_hinge'	hinge, quadratically penalized
'perceptron'	the perceptron (§38.4)
'squared_error', 'huber', 'epsilon_insensitive', 'squared_epsilon_insensitive'	regression losses (the SGDRegressor shelf, §37.3)

The slide’s equivalences, kept as stated:

```
SGDClassifier(loss='log')      # == LogisticRegression(solver='sgd')
SGDClassifier(loss='hinge')    # == linear support vector machine
```

and multi-class comes free via one-vs-all (§38.7). (Note: real sklearn’s `LogisticRegression` has no `solver='sgd'` — the slide’s equivalence is in the *loss and optimization* sense: `loss='log'` trains the same logistic-regression objective by SGD. Write `SGDClassifier(loss='log')` in code.) It “easily scales up to more than 10^5 training examples and 10^5 features” and handles sparse input — text classification is the slide’s named use case.

```
from sklearn.linear_model import SGDClassifier
SGD_classifier = SGDClassifier(loss='log')    # logistic regression, SGD-flavoured
SGD_classifier.fit(X_train, y_train)
```

Regularization (the slide’s): `penalty='l2'` (default) / `'l1'` / `'elasticnet'` = $(1 - \text{l1_ratio}) \cdot L2 + \text{l1_ratio} \cdot L1$ (default `l1_ratio=0.15`); `alpha` multiplies the penalty term (default 10^{-4} — §37.7’s dial, same name here). `max_iter` = epochs (default 1000). The shared knobs with `SGDRegressor`

(§37.3): `learning_rate ('constant'/'optimal'/'invscaling'/'adaptive')`, `tol`, `n_iter_no_change`, `early_stopping`, `validation_fraction`, `average` (averaged SGD), `warm_start`.

Basically, ... “SGDClassifier is the Swiss-army engine: name a loss, get a classifier. `loss='log'` is logistic regression, `'hinge'` is a linear SVM, `'perceptron'` is §38.4’s perceptron — three chapters of math (31, 33, 35) behind one constructor argument. Price of admission: shuffle, scale, and set the knobs §37.3 taught.”

38.7 Multiclass, multilabel, multioutput

The three setups (the slide’s definitions): i) **Multiclass**: exactly one label per example, more than two classes (iris: setosa/versicolor/virginica; MNIST: ten digits). ii) **Multilabel**: more than one output, each binary (an article tagged {sports, politics}). iii) **Multiclass-multioutput**: more than one output, each with > 2 values. The slide calls all three “multi-learning problems.”

What shape is y ? Ask `type_of_target` (the slide’s diagnostic):

```
from sklearn.utils.multiclass import type_of_target
type_of_target(y)
```

[recorded] examples from the slide: `type_of_target([1, 0, 2])` → 'multiclass'; `type_of_target([1.0, 0.0, 3.0])` → 'multiclass'; `type_of_target(['a', 'b', 'c'])` → 'multiclass'; `type_of_target(np.array([[1, 2], [3, 1]]))` → 'multiclass-multioutput'; `type_of_target(np.array([[0, 1], [1, 1]]))` → 'multilabel-indicator'; `type_of_target([[1, 2]])` → 'multilabel-indicator'. (Plus 'continuous'/'continuous-multioutput' for regression and 'binary' for two-class.)

Representing labels — `LabelBinarizer` turns a label vector into the (n, k) indicator matrix [recorded]:

```
from sklearn.preprocessing import LabelBinarizer
y = np.array(['apple', 'pear', 'apple', 'orange'])
y_dense = LabelBinarizer().fit_transform(y)
# [[1 0 0]
#   [0 0 1]
#   [1 0 0]
#   [0 1 0]]
```

Meta-estimators: wrappers that “transform the multi-learning problem into a set of simpler problems and fit one estimator per problem”: i) **OneVsRestClassifier** — one classifier per class, c -vs-rest; k classifiers; “computationally efficient,” “interpretable”; also handles multilabel (labels as an (n, k) indicator matrix). ii) **OneVsOneClassifier** — one classifier per *pair* of classes: $\binom{k}{2} = k(k-1)/2$ classifiers; predicts the class with the most votes, ties broken by aggregate confidence; “processes subset of data at a time,” useful when the base classifier “does not scale with the data.”

```
from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier
from sklearn.svm import LinearSVC
OneVsRestClassifier(LinearSVC(random_state=0)).fit(X, y)
OneVsOneClassifier(LinearSVC(random_state=0)).fit(X, y)
```

- iii) **MultiOutputClassifier** — one classifier per target (multi-output/multi-label).
- iv) **ClassifierChain** — chains binary classifiers so each sees the previous targets’ predictions: “capable of exploiting correlations among targets” (the slide’s contrast: **MultiOutputClassifier** fits independent targets; the chain lets targets talk to each other).

Built-in support (the slide’s table): many estimators need no wrapper — inherently multiclass: `LogisticRegression(multi_class='multinomial')`, `LogisticRegressionCV(multi_class='multinomial')`, `RidgeClassifier`, `RidgeClassifierCV`; multiclass-as-OVR: `LogisticRegression(multi_class='ovr')`, `SGDClassifier`, `Perceptron`; multilabel: `RidgeClassifier`, `RidgeClassifierCV`. The slide’s advice: “All

classifiers in scikit-learn perform multiclass classification out-of-the-box. Use `sklearn.multiclass` only when you want to experiment with different multiclass strategies.”

Basically, ... “One label of many (multiclass), many yes/no labels (multilabel), many multi-valued labels (multiooutput) — three shapes, checked with `type_of_target`. Most classifiers already handle multiclass internally (logistic regression even does proper multinomial, §31.12). The meta-estimators are the DIY kit: OVR trains k ‘this-vs-everything’ classifiers, OVO trains $k(k-1)/2$ pairwise duels and takes the vote.” ##

38.8 Evaluation: the confusion matrix, and what it implies

§22.9’s 0/1 loss counts mistakes; the confusion matrix *sorts* them. For binary labels with +1 the positive class:

Def (confusion matrix). `confusion_matrix(y_true, y_pred)` returns the 2×2 table whose entry (i, j) = “number of observations actually in group i but predicted to be in group j ” (the slide’s definition — **rows are the truth**):

	pred +1	pred −1
true +1	TP (true positive)	FN (false negative)
true −1	FP (false positive)	TN (true negative)

From these four counts, everything follows (standard definitions, used exactly as the course’s code uses them):

$$\boxed{\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}}, \quad \boxed{\text{precision} = \frac{TP}{TP + FP}}, \quad \boxed{\text{recall} = \frac{TP}{TP + FN}}, \quad \boxed{F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}}$$

Accuracy = $1 -$ §22.9’s 0/1 loss. Precision = “of everything I *called* positive, how many truly were” (purity of the alarms). Recall = “of everything truly positive, how many did I catch” (completeness of the net). F_1 is their harmonic mean — it punishes a classifier that games one at the expense of the other. The slide’s metrics shelf: `accuracy_score`, `balanced_accuracy_score`, `top_k_accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `roc_auc_score`.

eg 2 (the course’s digit-5 vs digit-3 perceptron, graded) [recorded entries, arithmetic verified]. The graded solution trains the §38.4-style perceptron (`random_state=42`, `max_iter=100`, `shuffle=True`, no penalty) on MNIST 5-vs-3 (train: 863 fives, 1032 threes [recorded]) and reports this test confusion matrix [recorded]:

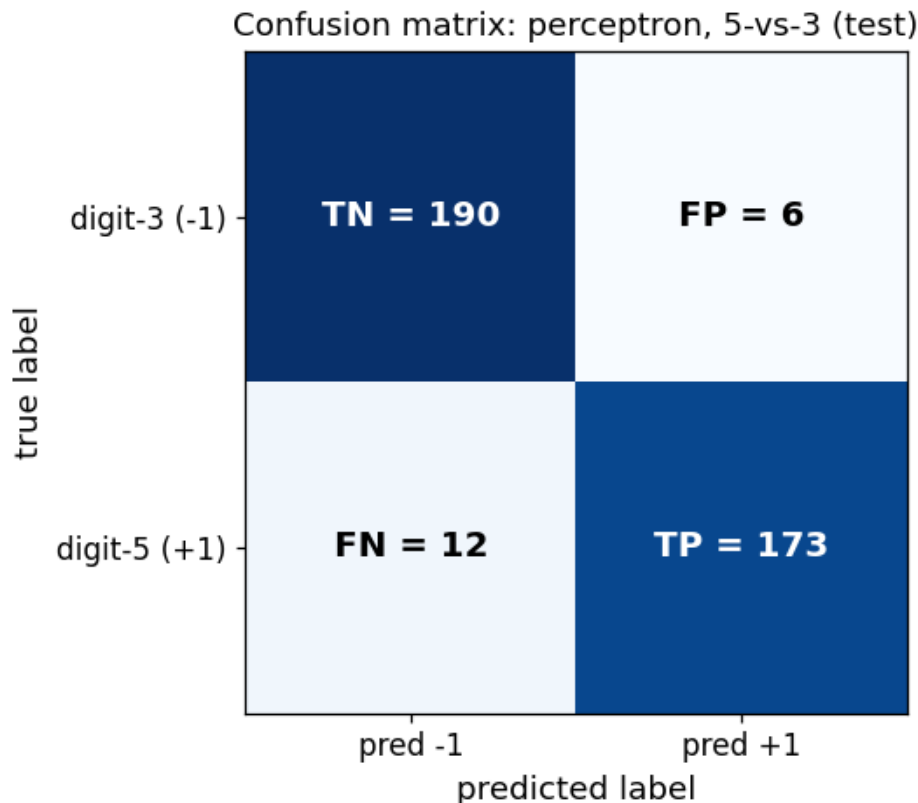


Figure 54: Confusion matrix of the perceptron on MNIST digit-5 vs digit-3 test data. Rows are true labels, columns predicted labels: true -1: $TN=190$, $FP=6$; true +1: $FN=12$, $TP=173$.

Read the four cells: $TN = 190$, $FP = 6$, $FN = 12$, $TP = 173$ (positive = digit-5, label +1 — sklearn’s default `pos_label`). Then:

$$\text{accuracy} = \frac{173 + 190}{381} = \frac{363}{381} \approx \boxed{0.9528},$$

$$\text{precision} = \frac{173}{173 + 6} = \frac{173}{179} \approx \boxed{0.9665}, \quad \text{recall} = \frac{173}{173 + 12} = \frac{173}{185} \approx \boxed{0.9351},$$

$$F_1 = \frac{2 \times 0.9665 \times 0.9351}{0.9665 + 0.9351} \approx \boxed{0.9505}.$$

The accuracy/precision/recall match the notebook’s printed 0.952755905511811, 0.9664804469273743, 0.9351351351351351 [recorded] to every shown digit — the arithmetic is the definition, no magic.

Note (a wrinkle in the course’s own answer keys). The PDF’s answer keys for two graded questions read $FN = 6$, $FP = 12$ — transposed against the confusion matrix above *and* against the notebook’s own printed precision/recall (which use sklearn’s `pos_label=1` convention: $FP = 6$, $FN = 12$). The chapter follows the matrix and the printed metric values; the discrepancy is logged in the review.

The display machinery (the slide’s): `ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=clf.classes_)`, `ConfusionMatrixDisplay.from_estimator(clf, X_test, y_test)`, `ConfusionMatrixDisplay.from_predictions(y_test, y_pred)` — the figure above is that third form’s content. And `classification_report(y_true, y_pred)` prints per-class precision/recall/F1 in one text block.

Basically, ... “The confusion matrix is the mistake ledger: rows = truth, columns = verdict. TP/TN are the agreements, FP/FN the two kinds of disagreement. Accuracy counts agreements; precision asks ‘were

my positive calls right?'; recall asks 'did I find all the positives?'; F_1 keeps the two honest with each other. The course's own perceptron gets 95% accuracy — and the matrix shows you *where* the other 5% went."

38.9 Thresholds, imbalance, and multiclass metrics

A probabilistic classifier (§31.8) doesn't have to snap at 0.5. `predict_proba` gives $P(y = 1 \mid x)$; choosing the threshold that converts it to a label is a decision, not a law:

- i) **Lower the threshold** → more predicted positives → recall rises, precision usually falls.
- ii) **Raise the threshold** → fewer, surer positive calls → precision rises, recall falls.

The slide's tools for watching the trade-off move:

```
from sklearn.metrics import precision_recall_curve, roc_curve
precision, recall, thresholds = precision_recall_curve(y_true, y_predicted)
fpr, tpr, thresholds = roc_curve(y_true, y_scores, pos_label=2)
```

`precision_recall_curve` sweeps the threshold and records precision/recall at each; `roc_curve` sweeps it and records the false-positive rate vs the true-positive rate (recall) — the ROC curve, whose area is `roc_auc_score` (§30.11's closing list names AUC the same way).

Class imbalance, two levers (both course-sourced): i) `class_weight` (§38.5): penalize mistakes on the rare class more, at *training* time. ii) The threshold (§38.9): move the decision boundary at *prediction* time — free, no retraining.

One binary metric, k classes (the slide's `average=` table): treat the problem as k one-vs-rest binaries, then average: - **macro**: plain mean of the per-class scores (every class equal, rare or not). - **weighted**: mean weighted by each class's presence in the true data (frequent classes count more). - **micro**: every sample-class pair contributes equally (big classes dominate). - **samples**: average the metric over samples (multilabel). - **None**: no averaging — return the per-class array.

Basically, ... "Accuracy is one number; the threshold is a dial you turn *after* training. Lower it and you catch more positives but cry wolf more; raise it and your alarms get trustworthy but sparse. `precision_recall_curve` draws that trade-off so you can pick your point. And when there are k classes, `average=` decides whether rare classes get an equal vote (**macro**) or a proportional one (**weighted**)."

38.10 The full classification pipeline: three course experiments

This section replays §37.8's pipeline discipline on classifiers, using the course's own experiments — including two places where the course's first attempt teaches the lesson by failing.

The setup (both course-sourced). The graded MNIST 5-vs-3 data (x_{train1} has shape (1895, 784) [recorded]), and the §38.4 perceptron with `shuffle=True`.

Experiment A — shuffle matters (or: SGD on sorted data). Same perceptron, two shuffles [recorded]:

	accuracy	precision	recall
<code>shuffle=True</code>	0.9528	0.9665	0.9351
<code>shuffle=False</code>	0.5486	1.0000	0.0703

With `shuffle=False` the confusion matrix is $(TN, FP, FN, TP) = (196, 0, 172, 13)$ [recorded]: the perceptron predicts -1 for nearly everything — perfect precision (its 13 positive calls were all right) and catastrophic recall (it found 13 of 185 fives). Why: the training rows were stacked $+1$ s-then- -1 s (§38.4), and SGD (§38.6) "estimates the gradient with one sample at a time" — fed 863 fives first, it walks deep into five-territory, then 1032 threes drag it back and it never recovers. The slide's rule — "it is important to permute (shuffle) the

training data before fitting” — is not hygiene, it is load-bearing. (The course’s Q6 answer, options 3/5/7 [recorded]: with shuffling off, accuracy *decreased*, precision *increased*, recall *decreased*.)

Experiment B — the leak the course committed, then fixed. To reduce 784 pixels to 10, the solution’s first attempt was:

```
x_train1_reduced = pca.fit(x_train1).transform(x_train1)
x_test1_reduced  = pca.fit(x_test1).transform(x_test1)    # fit AGAIN, on the test set
```

Two violations in two lines: fitting on the test set learns the projection from test statistics (§37.8’s Note — the same leak as `fit_transform` before the split), and worse, train and test get *different* projection bases. Result: confusion matrix (119, 77, 92, 93) [recorded], accuracy 0.5564, precision 0.5471, recall 0.5027 [recorded] — barely above coin-flip. The corrected version fits once, on train only:

```
p = pca.fit(x_train1)
x_train1_reduced = p.transform(x_train1)
x_test1_reduced  = p.transform(x_test1)
```

and the structural fix is the pipeline — `fit` learns every step on train, `predict` only transforms (§37.8):

```
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
pca = PCA(n_components=10, random_state=1)
clf = Perceptron(random_state=42, eta0=1, max_iter=100,
                  shuffle=True, validation_fraction=0.1,
                  fit_intercept=True, penalty=None, warm_start=False)
pipe = make_pipeline(pca, clf, verbose=1)
pipe.fit(x_train1, y_train1)
y_pred1 = pipe.predict(x_test1)
```

The corrected run’s confusion matrix: $(TN, FP, FN, TP) = (175, 21, 9, 176)$ [recorded] — accuracy $351/381 \approx 0.9213$, precision $176/197 \approx 0.8934$, recall $176/185 \approx 0.9514$ [verified]. Ten PCA components keep 92% accuracy; the leak had thrown away 36 points of it.

Experiment C — does regularization help here? Same correct-PCA setup, perceptron with `penalty='l2'`, `alpha=0.01` [recorded]: accuracy 0.7480, precision 0.7543, recall 0.7135 [recorded] — from $(TN, FP, FN, TP) = (153, 43, 53, 132)$ [verified]. With `penalty='l1'`, `alpha=0.01` [recorded]: accuracy 0.8688, precision 0.8689, recall 0.8595 [recorded] — from $(172, 24, 26, 159)$ [verified]. Against the unregularized 0.9213: L2 *hurts*, L1 hurts less but still hurts (the course’s Q9 answer: no [recorded]). L1 beats L2 here ($0.8688 > 0.7480$ [verified]) — on 10 dense PCA components there is nothing to sparsify away, so the penalty is pure drag; §28.9’s trade-off cuts both ways.

Basically, ... “Three experiments, three morals the course learned the hard way: (1) shuffle before SGD, or the perceptron memorizes the *order* of your data; (2) `pca.fit` on the test set is the leak §37.8 warned about — fit once on train, or let a pipeline make the leak un-writable; (3) regularization is not fairy dust — on already-compressed features it cost 5–17 points of accuracy. Every one of these numbers is the course’s own output.”

38.11 Where this goes next

- i) **Evaluation doctrine, properly.** Chapter 39 takes §38.9’s sketch — the stratified iterators the slide names (StratifiedKFold, RepeatedStratifiedKFold, StratifiedShuffleSplit: folds that replicate the overall class distribution, because plain folds can starve a class), LogisticRegressionCV’s `refit` semantics, and the full grid/randomized search — and makes it a subject.
- ii) **Debugging the workflow.** Chapter 40: when `score` prints 0.55 and the pipeline looks right, the discipline for finding out why — Experiment A’s shuffle disaster is the canonical first suspect.

- iii) **The neural turn.** The perceptron of §38.4 *is* Chapter 41’s neuron with a sign snapped on top; `MLPClassifier` will honor the same `fit/predict` contract with a hidden layer inside. The contract outlives linear models — again.

Problem set

- The contract.** (i) In one line each, say what `fit`, `predict`, `decision_function`, and `score` compute for a classifier, in the book’s notation (w, b ; $\text{sign}(w^T x + b)$; §22.9’s loss). (ii) Which of these does a regressor’s `score` *not* return, and what does it return instead (§37.5)? (iii) `predict_proba` exists for `LogisticRegression` but is meaningless for `RidgeClassifier` — why, from §38.3’s construction?
- Confusion matrix by hand.** A test set gives $TP = 40$, $TN = 50$, $FP = 10$, $FN = 20$. (i) Compute accuracy, precision, recall, F_1 . (ii) A colleague reports “accuracy 75%, so the model is fine” — the positive class is a rare disease. Using precision and recall, argue in two lines why accuracy alone misleads here.
- The C translation.** The slide’s logistic-regression objective is $\text{penalty} + C \cdot \text{cross-entropy loss}$. (i) Divide through by C and match against the book’s §28.3 form $\text{loss} + \lambda \cdot \text{penalty}$ to show $\lambda = 1/C$. (ii) Explain in one line why *smaller* C means *stronger* regularization. (iii) §37.7 says ridge’s `alpha` *is* the book’s λ — so what is `LogisticRegression(C=...)`’s C in terms of that `alpha`?
- Course-sourced.** The slide’s `LabelBinarizer` example transforms `['apple', 'pear', 'apple', 'orange']`. (i) Write the $(4, 3)$ indicator matrix by hand and check it against the recorded output. (ii) Why does the slide say the result has shape (n, k) — what are n and k here?
- Threshold moving.** `precision, recall, thresholds = precision_recall_curve(y_true, y_predicted)` (the slide’s call). (i) As you walk the `thresholds` array from high to low, what happens to the number of predicted positives, and hence to recall? (ii) A spam filter must almost never flag a legitimate email, but may let some spam through. Should its threshold go up or down from 0.5, and which metric is being protected?
- Spot the leak.** The course solution’s first PCA attempt (Experiment B) ran `pca.fit(x_train1)` and `pca.fit(x_test1)`. (i) Name the discipline this violates (§37.8’s Note / §36.7) and say exactly what leaks where — and what *else* breaks beyond the leak. (ii) Rewrite the two lines so the leak is structurally impossible (one `fit`, or a pipeline).
- OVR vs OVO.** (i) For $k = 5$ classes, how many binary classifiers do `OneVsRestClassifier` and `OneVsOneClassifier` train? (ii) The slide says OVO “processes subset of data at a time and is useful in cases where the classifier does not scale with the data” — explain in one line why training on subsets helps an unscalable base estimator.
- Diagnose the fit.** A perceptron reports test accuracy 0.5486, precision 1.0, recall 0.0703 (Experiment A’s numbers). (i) Reconstruct the approximate confusion matrix counts from these three numbers plus $n = 381$ (positive class = 185 rows). (ii) Diagnose in §38.6’s vocabulary: what single constructor argument most likely caused this, and why does it interact with how the course stacked the training rows?

Solutions — Chapter 38: scikit-learn: classification workflows

Problem 1 — The contract

- `fit(X_train, y_train)` learns (w, b) from the training rows (e.g. the perceptron’s mistake-driven updates, §31.4; logistic regression’s cross-entropy minimization, §31.10). `predict(X_test)` returns one label per row: $\hat{y} = \text{sign}(w^T x + b)$ for the linear family (§22.9). `decision_function(X_test)` returns the *un-snapped* score $w^T x + b$ per row — the confidence behind the sign. `score(X_test, y_test)` returns the mean accuracy, i.e. $1 - L$ with $L =$ §22.9’s 0/1 loss.
- A regressor’s `score` does not return accuracy; it returns $R^2 = 1 - u/v$ (§37.5) — “what fraction of the label’s variance the model explains.”
- `RidgeClassifier` never models $P(y \mid x)$: §38.3’s construction converts labels to $\{-1, +1\}$ and runs

regression; its output is a real number whose sign is read as the class. There is no probability to report — `predict_proba` would be a fiction. `LogisticRegression` genuinely fits §31.8’s $P(y = 1 \mid x) = \sigma(\theta^T x + \theta_0)$, so its probabilities are the model itself.

Problem 2 — Confusion matrix by hand

- (i) $n = 40 + 50 + 10 + 20 = 120$.

$$\begin{aligned} \text{accuracy} &= \frac{40 + 50}{120} = \frac{90}{120} = \boxed{0.75}, & \text{precision} &= \frac{40}{40 + 10} = \frac{40}{50} = \boxed{0.80}, \\ \text{recall} &= \frac{40}{40 + 20} = \frac{40}{60} \approx \boxed{0.6667}, & F_1 &= \frac{2 \times 0.8 \times 0.6667}{0.8 + 0.6667} = \frac{1.0667}{1.4667} \approx \boxed{0.7273}. \end{aligned}$$

- (ii) The disease is rare, so $TN = 50$ dominates the accuracy numerator — a classifier that calls nearly everyone healthy still scores well. Recall 0.67 says a third of the sick are missed (the costly error); precision 0.80 says one in five alarms is false. Accuracy hides both behind the healthy majority.

Problem 3 — The `C` translation

- (i) Slide: minimize penalty + $C \cdot \text{loss}$ over w . Divide the whole objective by the positive constant C (scaling an objective doesn’t move its minimizer):

$$\frac{1}{C} \text{penalty} + \text{loss} \iff \text{loss} + \frac{1}{C} \text{penalty}.$$

Matching against the book’s §28.3 form $\text{loss} + \lambda \cdot \text{penalty}$ gives $\boxed{\lambda = 1/C}$.

- (ii) C multiplies the *loss*: shrinking C shrinks the loss term’s weight, so the penalty term dominates the trade-off — the optimizer prefers smaller weights, i.e. stronger regularization.
- (iii) §37.7’s ridge `alpha` is the book’s λ , so for the identical regularization strength: $\boxed{C = 1/\text{alpha}}$ (with the caveat that the two estimators’ objectives normalize the loss differently — logistic regression averages nothing out front the way the book’s J does — so treat the reciprocal as the dial’s direction and scale, exact only up to that normalization).

Problem 4 — Course-sourced (`LabelBinarizer`)

- (i) Classes sorted: apple, orange, pear ($k = 3$). One-hot per row:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

'apple' → column 0, 'pear' → column 2, 'orange' → column 1. This matches the recorded output `[[1 0 0] [0 0 1] [1 0 0] [0 1 0]]` exactly.

- (ii) $n = 4$ examples (rows), $k = 3$ distinct classes (columns) — one column per class, one row per example.

Problem 5 — Threshold moving

- (i) Lowering the threshold can only flip predictions from negative to positive, never the reverse — so the predicted-positive count is non-decreasing, TP is non-decreasing while the positive pool ($TP + FN$) is fixed, and $\text{recall} = TP/(TP + FN)$ is non-decreasing. (Precision typically falls: the extra positives are the model’s least confident calls.)
- (ii) Threshold **up** from 0.5: fewer positive (“spam”) calls, each surer. Protected metric: **precision** of the spam verdict — “of the emails I flagged, (almost) all were truly spam.” (The cost — missed spam, lower recall — is explicitly accepted.)

Problem 6 — Spot the leak

- (i) The violated discipline is §37.8’s Note (and §36.7’s imputation rule): *fit on train, transform everywhere*. `pca.fit(x_test1)` learns the projection’s mean and principal directions from the **test** rows — test statistics leak into the fitted transformer. Beyond the leak, something worse breaks: train and test are projected onto *different* bases (two independent PCAs), so the classifier trains in one coordinate system and is tested in another — the 0.5564 accuracy is the wreckage.
- (ii) One fit, on train only:

```
p = pca.fit(x_train1)
x_train1_reduced = p.transform(x_train1)
x_test1_reduced = p.transform(x_test1)
```

or structurally leak-proof:

```
pipe = make_pipeline(PCA(n_components=10, random_state=1), clf)
pipe.fit(x_train1, y_train1)      # every step learns from train only
pipe.predict(x_test1)            # test rows only ever see transform
```

Problem 7 — OVR vs OVO

- (i) $k = 5$: `OneVsRestClassifier` trains $\boxed{5}$ classifiers (one per class, *c*-vs-rest). `OneVsOneClassifier` trains $\binom{5}{2} = \frac{5 \times 4}{2} = \boxed{10}$ classifiers (one per pair).
- (ii) Each OVO duel trains on only the two classes’ rows — roughly $2/k$ of the data per classifier — so a base estimator whose training cost blows up with n (the slide’s “does not scale with the data”) sees small, cheap subproblems instead of one big one. (OVR’s k classifiers each still see all n rows.)

Problem 8 — Diagnose the fit

- (i) Positive rows: $TP + FN = 185$. Recall = $TP/185 = 0.0703 \rightarrow TP \approx 0.0703 \times 185 = 13$, $FN = 172$. Precision = $TP/(TP + FP) = 1.0 \rightarrow FP = 0$. Accuracy = $(TP + TN)/381 = 0.5486 \rightarrow TN = 0.5486 \times 381 - 13 = 209 - 13 = 196$. So $(TN, FP, FN, TP) = (196, 0, 172, 13)$ — exactly §38.10’s recorded matrix.
- (ii) The suspect is `shuffle=False`. The course stacked the training matrix as all +1s then all −1s; with shuffling off, SGD (§38.6: “the gradient of the loss is estimated with one sample at a time”) sees 863 fives first and walks far into positive territory, then 1032 threes drag the weights back — the final model sits in negative territory and predicts −1 almost everywhere ($FN = 172$). Shuffling (`shuffle=True`) interleaves the classes so no single class dominates any stretch of the walk — the slide’s “it is important to permute (shuffle) the training data before fitting.”

Chapter 39: Model evaluation, cross-validation, hyperparameter tuning

Everything in §§39.1–39.4 comes from the MLP Week-3 “Linear Regression” slide deck (Dr. Ashish Tendulkar, IIT Madras) and the “Notes by Sejal” Week-3 summary; §§39.6–39.7 from the Week-4 “Polynomial regression / Hyperparameter tuning” deck; §39.5 and the `LogisticRegressionCV` part of §39.8 from the Week-5 classification deck; the course experiments from the Week-3 and Week-4 programming-question solution notebooks. This is the code for §23.9’s overfitting diagnosis, §28.10’s λ -choosing discipline, and §37.9’s hyperparameter sketch — now as full subjects.

A word on what was actually run. `sklearn` is not installed on this machine (see the chapter task note), so no `sklearn` call below was executed here. The code blocks are transcribed from the course slides and notebooks — verbatim unless a slide typo forced a fix (each fix is logged in `reviews/39-model-evaluation-cv.md`). Every *number* beside the code is either **[recorded]** — printed as output in the course material itself — or **[verified]** — worked by hand in this session from definitions (fold index sets, split-MSE arithmetic, variance ratios). The two kinds are marked wherever they could be confused.

Notation. $X \in \mathbb{R}^{n \times d}$ as usual (chapters 31–38). A *hyperparameter* is set before training; a *parameter* is learned by it.

39.1 One test split is a noisy number

The Week-3 deck states the evaluation ritual in four steps:

- i) **STEP 1:** Split data into train and test — `train_test_split(X, y, random_state=42)` (§36.6’s seeded split).
- ii) **STEP 2:** Fit the estimator on the training set.
- iii) **STEP 3:** Calculate the **training error** (a.k.a. *empirical error*) — how wrong the model is on data it already saw.
- iv) **STEP 4:** Calculate the **test error** (a.k.a. *generalization error*) — how wrong it is on data it never saw.

Then compare training and test errors (§23.9’s whole diagnosis lives in that comparison).

Def (score vs error). A **score** = a metric where higher is better (R^2). An **error** = a metric where lower is better (mean squared error). `sklearn` needs scores to maximize, so it converts an error metric into a score metric by negating it — the **neg_ prefix** (§37.5): `neg_mean_absolute_error`, `neg_mean_squared_error`, `neg_root_mean_squared_error`, `neg_mean_squared_log_error`, `neg_median_absolute_error`.

But the deck immediately asks the uncomfortable question: *if train and test performance look comparable on this one split, is that performance guaranteed on other splits too?* Two ways the answer is no:

- i) **Small test set** → **unstable estimate**. The test error is a sample statistic; with few test points it wobbles and “would not reflect the true test error on large test set” (the slide’s words).
- ii) **Lucky split** → **optimistic estimate**. “What is the chance that the easiest examples were kept aside as test by chance? This if happens would lead to optimistic estimation of the true test error.”

eg 1 (how loud one split can be — hand arithmetic) [verified]. The slide’s own baseline estimator, `DummyRegressor(strategy="mean")` (“makes a prediction as specified by the strategy” — here, the training mean), on four labels $y = (1, 2, 10, 11)$. Test mean squared error for three different 50/50 splits:

split	train	test	train mean	test MSE
A	{1, 2}	{10, 11}	1.5	$\frac{(10-1.5)^2 + (11-1.5)^2}{2} = \frac{72.25 + 90.25}{2} = 81.25$
B	{1, 10}	{2, 11}	5.5	$\frac{(2-5.5)^2 + (11-5.5)^2}{2} = \frac{12.25 + 30.25}{2} = 21.25$
C	{1, 11}	{2, 10}	6	$\frac{(2-6)^2 + (10-6)^2}{2} = \frac{16 + 16}{2} = 16.00$

Same model class, same data, one shuffle of luck apart: the estimate swings $16 \rightarrow 81$. (The solutions volume also works out the leave-one-out number, 36.44 — the average over every possible hold-out.) The slide’s remedy: “We use cross validation for robust performance evaluation.”

Note (beat the dummy first). `DummyRegressor(strategy="mean"/"median"/"quantile"/"constant")` exists so every evaluation starts with a floor: a real model must beat the dumbest sensible predictor. The deck’s R^2 page makes the floor quantitative — “a constant model that always predicts the expected value of y would get a score of 0.0.” If your tuned regressor scores 0.05, you have learned almost nothing.

Basically, ... “One split gives one number, and the number depends on which rows drew the short straw. Small test set? Unstable. Easy rows in test? Flattering. The fix is to re-split *many times* and average — that’s all cross-validation is.”

39.2 Cross-validation: the robust estimate

Def (cross-validation). **Cross-validation** = repeated splitting into training and test parts, producing *many* train/test errors instead of one — “this enables us to estimate variability in generalization performance of the model” (the slide’s sentence). sklearn’s iterators, all in `sklearn.model_selection`:

- i) **KFold** — divide the data into k folds; each run uses $k - 1$ folds for training and 1 for evaluation; rotate the held-out fold.
- ii) **RepeatedKFold** — repeat k -fold r times with different shuffles (averages out the fold-luck too).
- iii) **LeaveOneOut** — $k = n$: each point gets its own test fold. Maximum data reuse, n fits — expensive.
- iv) **ShuffleSplit** — “random permutation based”: shuffle, split into train/test, repeat n user-chosen times with a chosen `test_size`. Not a partition — test sets of different iterations can overlap.

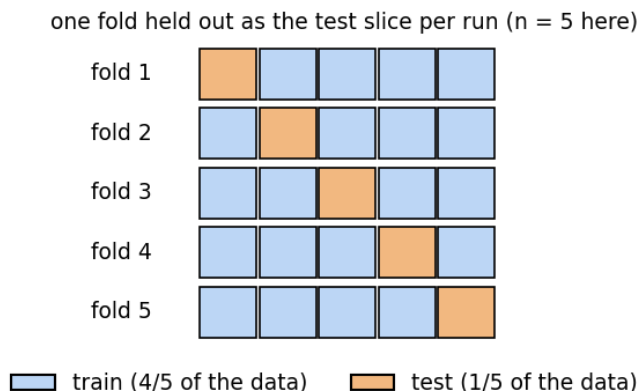


Figure 55: K-fold cross-validation schematic with $k=5$: five rows, one per run; in each row four blocks are blue (train, 4/5 of the data) and one block is orange (test, 1/5 of the data), the orange block rotating across the rows.

The workhorse call. `cross_val_score` runs the estimator through the iterator and returns one score per fold:

```
from sklearn.model_selection import cross_val_score

lin_reg = LinearRegression()                # the slide writes
↳ linear_regression(); the class is LinearRegression
score = cross_val_score(lin_reg, X, y, cv=5) # KFold, 5 folds -> 5 scores
```

With an explicit iterator (the slide's alternate form):

```
from sklearn.model_selection import KFold
kfold_cv = KFold(n_splits=5, random_state=42)
score = cross_val_score(lin_reg, X, y, cv=kfold_cv)
```

Note (random_state without shuffle). The slide's `KFold(n_splits=5, random_state=42)` never sets `shuffle=True` — and with the default `shuffle=False` the folds are deterministic contiguous blocks, so `random_state` is silently inert (it only takes effect when `shuffle=True`). Transcribed as written; the caveat is cross-checked against the sklearn docs, and logged in the review.

eg 2 (fold arithmetic — no sklearn needed) [verified]. $n = 6$ rows, `KFold(n_splits=3)` without shuffling. The three test index sets are forced by the definition: $\{0, 1\}$, $\{2, 3\}$, $\{4, 5\}$ — each row is tested exactly once, trained on exactly twice. `LeaveOneOut` on the same data is `KFold(n_splits=6)`: 6 fits, test sets $\{0\}$, $\{1\}$, ..., $\{5\}$.

The repeated version, as the course ran it [recorded]. The Week-4 graded question runs `RepeatedKFold(n_splits=2, n_repeats=2, random_state=1)` on $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 1 & 2 \\ 3 & 4 \end{bmatrix}$ and asks for the concatenated train+test indices:

```
from sklearn.model_selection import RepeatedKFold
ans = RepeatedKFold(n_splits=2, n_repeats=2, random_state=1)
for train, test in ans.split(X):
    ...
array1 = np.append(train, test) # the course's function returns only the LAST
↳ iteration's split
# [0 2 1 3] [recorded]
```

Read it as: in the final repetition's final split, rows $\{0, 2\}$ trained and rows $\{1, 3\}$ were tested (the graded answer's option (a)). The four splits ran; the function kept only the last one — a quirk of the course's code, not of `RepeatedKFold`.

ShuffleSplit, the slide's third form:

```
from sklearn.model_selection import ShuffleSplit
shuffle_split = ShuffleSplit(n_splits=5, test_size=0.2, random_state=42)
score = cross_val_score(lin_reg, X, y, cv=shuffle_split)
```

“You define the number of splits and the test fraction; each iteration shuffles and re-splits.” The slide adds that it is “robust to class distribution” — read alongside §39.5: repeated random subsampling averages out split luck, but *proportion-matching* is the stratified iterators' job, not this one's.

Basically, ... “KFold is the honest version of the single split: cut the data into k slices, test each slice once, average the k numbers. **LeaveOneOut** takes it to the extreme ($k = n$, n fits — slow). **RepeatedKFold** repeats the cutting to average out fold luck. **ShuffleSplit** rolls its own random splits. All four answer the same question: what does the test error look like *on average*, not on one lucky split?”

39.3 cross_val_score vs cross_validate: what they return

The slide draws a sharp line between the two:

- i) **cross_val_score** — the quick one. Returns a plain array of *test* scores, one per fold ($cv=5 \rightarrow$ five numbers). One metric only.
- ii) **cross_validate** — the instrumented one. Returns a **dictionary** with keys:
 - `fit_time`, `score_time` — how long fitting and scoring took per fold,
 - `test_score` — the per-fold test scores,
 - `estimator` — the fitted estimator per fold (only with `return_estimator=True`),
 - `train_score` — the per-fold training scores (only with `return_train_score=True`).

```
from sklearn.model_selection import cross_validate, ShuffleSplit

cv = ShuffleSplit(n_splits=40, test_size=0.3, random_state=0)
cv_results = cross_validate(regressor, data, target, cv=cv,
                           scoring="neg_mean_absolute_error")
# cv_results['test_score'] -> 40 numbers; cv_results['fit_time'] -> 40 timings
```

Need the fitted models or the train scores (the train/test gap is §23.9's overfitting signal)? Ask for them:

```
cv_results = cross_validate(regressor, data, target, cv=cv,
                           scoring="neg_mean_absolute_error",
                           return_train_score=True,
                           return_estimator=True)
# cv_results['estimator'] -> the 40 fitted estimators
```

And **cross_validate** — unlike **cross_val_score** — accepts a **list of scorings** and computes all of them in the same folds:

```
cv_results = cross_validate(regressor, data, target, cv=cv,
                           scoring=["neg_mean_absolute_error",
                                   "neg_mean_squared_error"],
                           return_train_score=True, return_estimator=True)
```

The `scoring=` menu (the slide's list, regression): `max_error`, `r2`, `neg_mean_absolute_error`, `neg_mean_squared_error`, `neg_mean_squared_log_error`, `neg_median_absolute_error`, `neg_root_mean_squared_error`. Two reading rules, straight from the definitions:

- i) Every `neg_` value is ≤ 0 ; **negate it to get the error**. `cross_val_score(..., scoring='neg_mean_squared_error')` returning -16.04 means $\text{MSE} = 16.04$.
- ii) sklearn maximizes whatever `scoring` names — the `neg_` versions exist (§37.5) so that “higher is better” machinery can optimize an error.

Basically, ... “`cross_val_score` answers ‘how good, on average?’ with one array. `cross_validate` answers ‘how good, how long, and what did each fold’s model look like?’ with a dict — and it can score several metrics at once, which `cross_val_score` refuses to do. And whenever you see `neg_`, flip the sign before reading it as an error.”

39.4 Learning curves: §23.9 as code

§23.9 diagnosed overfitting on paper by comparing training and test error at *one* training size. The slide’s `learning_curve` asks the follow-up question: **how do the two errors move as the training set grows?**

```
from sklearn.model_selection import learning_curve

results = learning_curve(lin_reg, X_train, y_train,
                        train_sizes=train_sizes, cv=cv,
                        scoring="neg_mean_absolute_error")
train_size, train_scores, test_scores = results[:3]
# Convert the scores into errors
train_errors, test_errors = -train_scores, -test_scores
```

Three facts about the shapes (from the call’s semantics): `train_size` is the m training sizes tried; `train_scores` and `test_scores` have shape (m, k) — one row per training size, one column per fold. The slide’s STEP 2: plot the per-size means (usually $\pm$ one standard deviation) of the two errors against n , “and make assessment about model fitment: under/overfitting or right fit.”

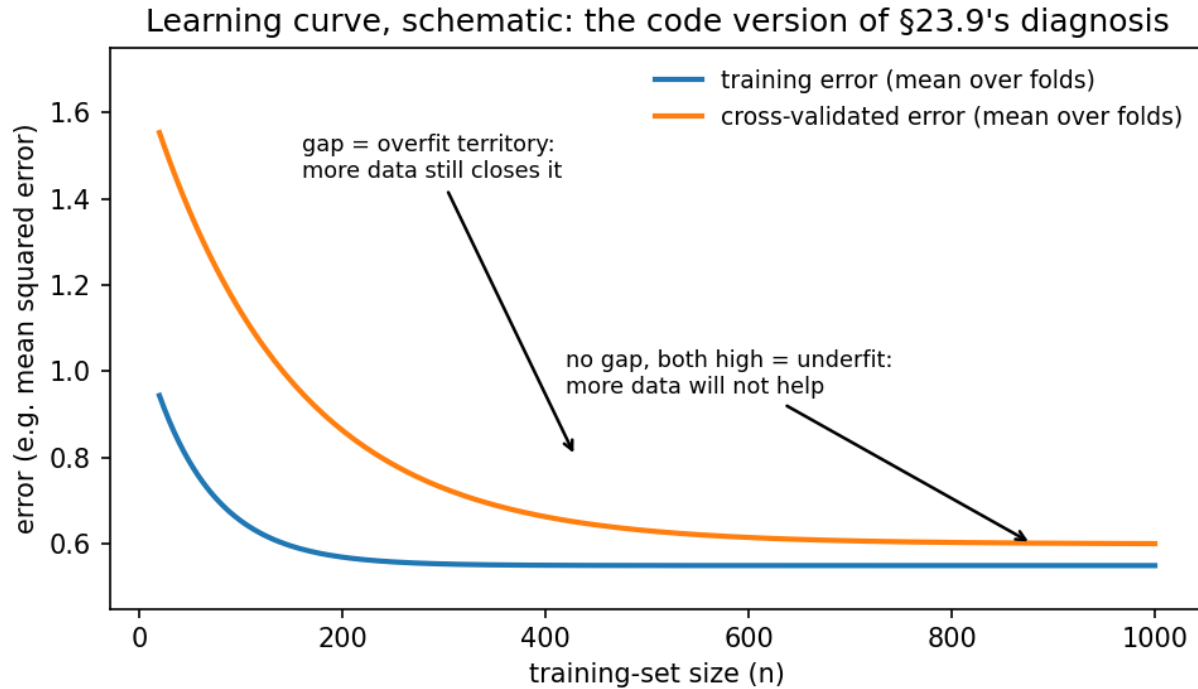


Figure 56: Schematic learning curve: training error falls quickly and plateaus low; cross-validated error starts high and converges toward the training error as n grows. An arrow marks the mid- n gap as overfit territory (“more data still closes it”); an arrow at the flat right end marks the no-gap plateau as underfit (“more data will not help”).

Reading the curves (the standard reading of the slide’s “under/overfitting or right fit”):

- i) **Both errors high and flat** → **underfit** (§23.9’s $m = 1$ line). The model can’t express the pattern; more data won’t help — change the model (more features, less regularization).
- ii) **Train error low, CV error high, gap still closing as n grows** → **overfit that data can cure**. More training rows pull the two curves together.
- iii) **Gap that stays open at the largest n** → **overfit that needs the model changed** — §28.9’s trade-off says the dial is now regularization or capacity, not data.

The slide’s second diagnosis sketch works the *other* axis: “Fit linear models with different number of features ... plot #features vs error graph — one each for training and test errors ... We can replace #features with any other tunable hyperparameter to do this diagnosis for setting that hyperparameter.” (sklearn’s name for exactly this is `validation_curve` — flagged as a standard-resource cross-check in the review log; the slide describes the idea without naming the function.)

Basically, ... “A learning curve is §23.9’s train-vs-test comparison with the x-axis switched from ‘one dataset’ to ‘how much data’. Two lines: training error (optimistic, falls) and cross-validated error (honest, falls toward it). Both stuck high — your model is too simple, buy capacity. A gap that keeps closing — buy data. A gap that won’t close — buy regularization.”

39.5 Stratified iterators: the classification addition

The Week-5 deck opens with a scope note that organizes this whole chapter’s second half: “Cross validation and hyper parameter search for classification works exactly like how it works in regression setting” — with “a couple of CV strategies that are specific to classification.” Those strategies are the **stratified iterators**.

The problem (the slide’s): “There may be issues like class imbalance in classification, which tend to impact the cross validation folds. The overall class distribution and the ones in folds may be different and this has implications in effective model training.” A fold that accidentally contains no positives can’t train (or test) the positive class.

The fix (the slide’s): “`sklearn.model_selection` provides three stratified APIs to create folds such that the overall class distribution is replicated in individual folds”:

- i) **StratifiedKFold** — k -fold with each fold’s class proportions matching the whole dataset.
- ii) **RepeatedStratifiedKFold** — the repeated version.
- iii) **StratifiedShuffleSplit** — the shuffle-split version. **Note** (the slide’s): “Folds obtained via **StratifiedShuffleSplit** may not be completely different” — test sets can overlap across iterations, so it is repeated subsampling, not a partition.

eg 3 (why plain folds starve — hand arithmetic) [verified]. Eight rows, 4 positive / 4 negative, sorted positives-first, `n_splits=2`, no shuffling:

- Plain **KFold**: test fold 1 = rows {0, 1, 2, 3} — **all positive**; test fold 2 = rows {4, 5, 6, 7} — **all negative**. Fold 1’s model trains on negatives only and is tested on positives only.
- **StratifiedKFold**(`n_splits=2`): each test fold holds 2 positives and 2 negatives — the 50/50 overall split replicated exactly.

The arithmetic is definitional: stratification forces each fold to mirror the dataset’s class fractions.

Basically, ... “Plain folds cut the data blindly; on imbalanced or sorted labels a fold can end up with zero examples of a class. The stratified iterators cut *each class* separately and stack the slices, so every fold is a miniature of the whole dataset’s class mix. For classification, reach for these by default — the Week-5 deck says everything else about CV and HPT is identical to the regression case.”

39.6 The three-way split doctrine: train / validation / test

Hyperparameters are the things cross-validation is *for*. The Week-4 deck defines them first:

Def (hyperparameter). **Hyperparameters** = “parameters that are not directly learnt within estimators. In `sklearn`, they are passed as arguments to the constructor of the estimator classes” — e.g. `degree` in `PolynomialFeatures`, the learning rate in `SGDRegressor`. (Parameters are learned by *fit*; hyperparameters are chosen by you.) §28.10 did this on paper for λ ; this section is the code.

Def (hyperparameter search). The slide’s five ingredients: **an estimator** (regressor or classifier); **a parameter space**; **a method for searching or sampling candidates**; **a cross-validation scheme**; and **a score function**.

Choosing hyperparameters on the *test* set would spend the test set — it would become a second validation set, and the final number would be optimistic (§28.10’s discipline). So the slide mandates a **three-way split** — training, validation, test — and five steps:

- i) **STEP 1:** Divide the data into training, validation, and test sets.
- ii) **STEP 2:** For each hyperparameter combination, learn a model on the **training** set. *Tips (the slide’s):* run the combinations in parallel with `n_jobs=-1`; some combinations may fail on some folds — set `error_score=0` (or `np.NaN`) so the search survives a bad combination instead of crashing.
- iii) **STEP 3:** Evaluate each model on the **validation** set; pick the best-scoring combination.
- iv) **STEP 4: Retrain** the winner on training **and** validation combined — the winner deserves the validation rows too, now that their job (choosing) is done.
- v) **STEP 5:** Evaluate the retrained model on the **test set, once**.

The slide’s closing line, the doctrine’s whole point: “Note that the **test set was not used in hyperparameter search and model retraining**.” §37.9’s one-line version: **spend the test set exactly once**.

Basically, ... “Three piles, three jobs: *train* teaches the parameters, *validation* picks the hyperparameters, *test* grades the final model — once, at the very end. Touch the test set during tuning and your grade is rigged. The five steps are: split three ways, train every candidate on train, crown the winner on validation, re-train the winner on train+validation, grade once on test.”

39.7 GridSearchCV vs RandomizedSearchCV

The slide’s two generic search strategies:

	Grid search	Randomized search
What you specify	exact values of each parameter	distributions over parameter values
How candidates are picked	all combinations, exhaustively	sampled from the distributions
Budget	forced: grows as the product of the value lists	chosen by you , independent of the number of parameters — the n_iter argument (default 10)

The course’s grid experiment [recorded]. The Week-4 notebook tunes `SGDRegressor` on the California housing data (70/30 split, `random_state=1`, standardized features — the scale-first-then-search shape \$39.9 will fix):

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import SGDRegressor

sgdregressor = SGDRegressor(random_state=1)
sgdr_model = GridSearchCV(sgdregressor,
                          {'loss': ['squared_error', 'huber'],
                           'penalty': ['l2', 'l1'],
                           'alpha': [0.1, 0.01, 0.001],
                           'max_iter': [1000, 2000, 5000]},
                          cv=4,
                          return_train_score=True)
sgdr_model.fit(X_train_norm, y_train)
sgdr_model.best_params_
# {'alpha': 0.01, 'loss': 'squared_error', 'max_iter': 1000, 'penalty': 'l1'} [recorded]
sgdr_model.score(X_test_norm, y_test)
# 0.5951040704728554 [recorded] (test R2, the once-spent test set)
```

Count the work [verified]: $2 \times 2 \times 3 \times 3 = 36$ combinations $\times 4$ folds = 144 fits. The winner: squared-error loss, L1 penalty, `alpha = 0.01`, `max_iter = 1000` — and `best_params_` hands you exactly that dict.

The course’s ridge experiment [recorded]. Same data, Ridge through `GridSearchCV` over `alpha` $\in \{0.5, 0.1, 0.05, 0.01, 0.005, 0.001\}$ and `fit_intercept` $\in \{\text{True}, \text{False}\}$, `cv=4`:

```
ridge_model = GridSearchCV(Ridge(),
                           {'fit_intercept': [True, False],
                            'alpha': [0.5, 0.1, 0.05, 0.01, 0.005, 0.001]},
                           cv=4)
ridge_model.fit(X_train_norm, y_train)
# best_params_ -> {'alpha': 0.5, 'fit_intercept': True} [recorded]
# test R2 -> 0.5971450612248769 [recorded]
```

The course’s randomized experiment — and its warning [recorded]. Lasso through `RandomizedSearchCV` over a *discrete* space of $2 \times 4 = 8$ combinations (`fit_intercept`, `alpha`

$\in \{1, 0.1, 0.01, 0.001\}$), `cv=6`:

```
lasso_model = RandomizedSearchCV(Lasso(),
                                {'fit_intercept': [True, False],
                                 'alpha': [1, 0.1, 0.01, 0.001]},
                                cv=6)
lasso_model.fit(X_train_norm, y_train)
```

sklearn printed (verbatim from the notebook):

UserWarning: The total space of parameters 8 is smaller than `n_iter=10`.
Running 8 iterations. For exhaustive searches, use `GridSearchCV`.

and then `best_params_` $\rightarrow$ `{'fit_intercept': True, 'alpha': 0.001}` [recorded], test $R^2 = 0.6065831805608592$ [recorded]. The moral: random search on a *tiny discrete* space degenerates into grid search — the warning is the tell. Randomized search earns its keep on large or continuous spaces, where 10 (or 50, or 100) sampled candidates beat an exponential grid.

The slide's pipeline grid (the shape §39.9 adopts — typo `POLynomialFeatures` fixed, logged in the review; §37.9 showed this too):

```
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import SGDRegressor

param_grid = [{'poly__degree': [2, 3, 4, 5, 6, 7, 8, 9]}]
pipeline = Pipeline(steps=[('poly', PolynomialFeatures()),
                           ('sgd', SGDRegressor())])
grid_search = GridSearchCV(pipeline, param_grid, cv=5,
                           scoring='neg_mean_squared_error',
                           return_train_score=True)
grid_search.fit(X_train, y_train) # slide: x_train.reshape(-1, 1) for 1-D input
```

`poly__degree` is §37.8's double-underscore addressing: reach *inside* the named pipeline step and set its hyperparameter. `return_train_score=True` keeps the train-side CV scores too — the train/CV gap is the overfitting read of §39.4.

refit, the quiet default. The slide's HPT STEP 4 (retrain the winner on train+validation) is what `GridSearchCV` does for you automatically: with the default `refit=True`, after the search it refits the best combination on the *whole* training data, so `best_estimator_` is ready to predict and `.score(X_test, y_test)` works straight away. (Cross-checked against standard sklearn behavior; logged in the review.)

Basically, ... “Grid search tries *everything* — 36 combos $\times$ 4 folds = 144 fits in the course's SGD eg, winner `{alpha: 0.01, loss: 'squared_error', max_iter: 1000, penalty: 'l1'}`. Randomized search tries *n_iter samples* — brilliant on big continuous spaces, pointless on an 8-combo space (sklearn literally warns you and runs the 8). The course's own numbers: SGD 0.5951, ridge 0.5971, lasso 0.6066 — all test R^2 , all on the once-spent test set.”

39.8 Model-specific HPT: RidgeCV, LassoCV, ElasticNetCV, LogisticRegressionCV

Grid search is generic; some estimators tune *one* parameter so efficiently that the CV is built in. The slide's principle: “Some models can fit data for a range of values of some parameter **almost as efficiently as fitting the estimator for a single value** of the parameter. This feature can be leveraged to perform more efficient cross-validation used for model selection of this parameter.”

- i) **RidgeCV / LassoCV / ElasticNetCV** (§37.7's one-line version; the slide's details): hand `RidgeCV` the candidate strengths via **alphas** — “the regularization rate must be positive; larger values indicate

stronger regularization” (the §37.7 dial). `cv` picks the splitting strategy: `None` → the efficient Leave-One-Out path; an integer → that many folds; a CV splitter; or an iterable of (train, test) index splits. For binary/multiclass targets with `cv=None` or an integer, the slide says `StratifiedKFold` is used — §39.5’s iterators, inside the estimator. The fitted estimator reports the chosen strength: the slide writes “`alphas` provides the estimated regularization parameter” — in current sklearn the winner lives in `alpha_` (§37.7 already used that name; the slide’s `alphas` is the constructor’s candidate list — logged in the review).

- ii) **LogisticRegressionCV** (Week-5 deck): “logistic regression with in-built cross validation support to find the best values of `C` and `l1_ratio` according to the specified scoring attribute.” Key parameters: `cv` (the iterator), `scoring` (the metric), `Cs` (the candidate regularization strengths — the slide writes “`cs`”). **Note** (cross-check, also in `reviews/38-sklearn-classification.md`): the slide’s `l1_ratio` is the current sklearn keyword `l1_ratios` (a list of candidates, like `Cs`). And the `refit` semantics (also §38.5’s):

- `refit=True`: “scores averaged across folds, values corresponding to the best score are selected, and final refit with these parameters” — one winner `C`, one final model.
- `refit=False`: “the coefs, intercepts and `C` that correspond to the best scores across folds are averaged” — the answer is an *average* of fold-winners, no final refit.

When to use which. The slide’s own two options for ridge’s `alpha`: “[Option #1] built-in cross validation in `RidgeCV`” (cheap, one parameter) vs “[Option #2] Grid search / Randomized search” (general, many parameters). Same pattern for lasso. If the *only* dial is the regularization strength, the `*CV` estimator is the short path; the moment you tune two or more things (say, `degree` and `alpha`), it is `GridSearchCV` over the pipeline.

Basically, ... “For one special dial — the regularization strength — sklearn ships estimators with the search baked in: `RidgeCV`/`LassoCV`/`ElasticNetCV` for regression, `LogisticRegressionCV` for classification (tuning `C`, with `refit` choosing between ‘crown one winner and refit’ and ‘average the fold-winners’). One dial → the `*CV` estimator; several dials → `GridSearchCV` over a pipeline.”

39.9 The pipeline + search combination: leak-proof HPT

Look again at the course’s grid-search notebook (§39.7): it standardized *first* (`scaler.fit_transform(X_train)`), then searched. That works here only because the scaler was fit on the training split. But inside the CV loop the scaler stays fixed across folds while the estimator refits — a mild version of the disease §38.10’s PCA story diagnosed: statistics learned outside the fold loop leaking across it.

The slide’s own poly-degree example (§39.7) shows the structural fix: **put the whole pipeline *inside* the search**. Then every fold — and every candidate — gets a pipeline that is `fit` on that fold’s training rows only, exactly §37.8’s “learn on train, apply everywhere”:

```
param_grid = [{'poly_degree': [2, 3, 4, 5, 6, 7, 8, 9]}]
pipeline = Pipeline(steps=[('scale', StandardScaler()),  # learned per fold, inside the
    ↪ search
                        ('poly', PolynomialFeatures()),
                        ('sgd', SGDRegressor())])
grid_search = GridSearchCV(pipeline, param_grid, cv=5,
                          scoring='neg_mean_squared_error')
grid_search.fit(X_train, y_train)
```

Three disciplines, one object:

- No leak.** The scaler (or imputer, §36.7; or PCA, §38.10) is refit inside every fold on fold-train rows only — the §38.10 leak (“`pca.fit` on the test set”, accuracy 0.5564 [recorded] vs 0.9213 corrected [verified]) becomes structurally un-writable.
- One address space.** `poly__degree`, `sgd__alpha` — the double underscore (§37.8) tunes preprocessing *and* model hyperparameters in a single grid.

- iii) **One verdict.** `best_params_ / best_score_ / best_estimator_` describe the whole workflow, not a naked estimator.

The Week-4 deck closes the arc this chapter opened: “Since the polynomial regression uses more parameters (due to polynomial representation of the input), it is **more prone to overfitting**. We will study how to **detect overfitting with learning curves** and use of **regularization to mitigate the risk** of overfitting.” Detect (§39.4) → mitigate (§28.10, §39.8) → verify (§39.6, test set spent once). That is the chapter in one sentence.

Basically, ... “Never tune around a pipeline — tune *inside* one. The pipeline goes into `GridSearchCV`, the grid addresses steps with `__`, and every fold refits every learning step on its own training rows. That’s the §38.10 leak made impossible by construction, and it’s why the slide’s poly-degree example wraps the pipeline, not the estimator.”

39.10 Where this goes next

- i) **The honest boundary.** The course’s CV story ends here: iterators, `cross_val_score/cross_validate`, learning curves, grid/randomized search, the `*CV` estimators, the three-way split. One natural follow-up the sources do *not* cover is **nested cross-validation** (an outer CV loop around the whole HPT, so the reported score isn’t optimistic from model selection itself) — flagged, not invented; the deck never names it.
- ii) **Debugging the workflow.** Chapter 40: when the CV scores print nonsense and the pipeline looks right — §38.10’s shuffle disaster and the leak story are the first two suspects, and this chapter’s iterators are the instrument panel.
- iii) **The contract survives.** `GridSearchCV` honors the estimator contract (§37.1): it **fits**, **predicts**, and **scores** like any estimator — which is why a search object can sit anywhere an estimator can, including inside another pipeline.

Problem set

1. **Split variance, by hand.** `DummyRegressor(strategy="mean")` on $y = (1, 2, 10, 11)$ (eg 1). (i) Recompute the test MSEs for the three 50/50 splits (A: train $\{1, 2\}$; B: train $\{1, 10\}$; C: train $\{1, 11\}$). (ii) Compute the leave-one-out MSE for the same model: for each y_i , predict the mean of the other three and average the four squared errors. (iii) In two lines: why does the slide prefer the many-split estimate over any one of your three numbers?
2. **Fold arithmetic.** (i) Write the three test index sets of `KFold(n_splits=3)` on $n = 6$ rows, no shuffling. (ii) How many fits does `LeaveOneOut` run on $n = 6$ rows, and why? (iii) The course’s `RepeatedKFold(n_splits=2, n_repeats=2, random_state=1)` function printed `[0 2 1 3]` — say exactly what this array contains, and why only *one* array is returned when $2 \times 2 = 4$ splits were run.
3. **Two functions, two returns.** (i) `cross_val_score(lin_reg, X, y, cv=5)` returns an array of length 5 — of *what*? (ii) List the five keys the slide’s `cross_validate` dict can carry, and say which two need a flag to appear. (iii) `cross_val_score(..., scoring='neg_mean_squared_error')` returns `array([-16.04, -18.90, -15.02])`. Compute the mean MSE. (iv) Why does `cross_validate` accept a *list* for `scoring` while `cross_val_score` does not?
4. **Course-sourced metric.** The Week-4 practice asks for the explained variance of $y_{\text{true}} = [7, 4, 9, 4]$, $y_{\text{pred}} = [8, 7, 12, 5]$. (i) Compute it by hand from $1 - \text{Var}(y - \hat{y}) / \text{Var}(y)$ (population variances) and check against the recorded 0.7777777777777778. (ii) From the formula: what does a perfect predictor score, and what does a mean-predictor (`DummyRegressor(strategy="mean")`) score? (iii) The Week-3 practice’s graded answers give explained variance 0.6605500501742703 vs $R^2 = 0.6605140591531992$ on the California test split [recorded] — in one line, why are they nearly but not exactly equal?
5. **Read the curves.** (i) A learning curve shows training error 0.8 and CV error 0.82, both flat from $n = 200$ to $n = 2000$. Diagnose in §23.9’s vocabulary and name the fix direction. (ii) Training error 0.10, CV error 0.45, and the CV curve is still falling at the largest n . Diagnose, and say whether

the next experiment should buy data or regularization. (iii) Write the slide’s three unpack lines for `learning_curve` results and the two negation lines; for 5 training sizes and 4 folds, what is the shape of `train_scores`?

6. **Grid vs randomized.** (i) The course’s Lasso search ran on an 8-combination space with `n_iter=10` (the default). Quote sklearn’s warning verbatim [recorded] and say what it did instead. (ii) A grid has 3 parameters with 4 values each and `cv=5` — how many fits? (iii) In one line each: when is randomized search the better choice, and when is grid search?
7. **The three-way doctrine.** (i) List the slide’s five HPT steps in order. (ii) Why does STEP 4 retrain on train+validation instead of just keeping the STEP-3 winner? (iii) The course’s notebook called `sgdr_model.score(X_test_norm, y_test)` after the search — which doctrine does this honor, and which `GridSearchCV` default did the STEP-4 retraining for it?
8. **Spot the leak.** The Week-4 notebook ran `scaler.fit_transform(X_train)` before `GridSearchCV`. (i) Name the discipline this bends (§36.7 / §37.8 / §38.10) and say what is fixed across folds that shouldn’t be. (ii) Rewrite the search so the scaler is inside the pipeline and the grid still tunes the estimator’s `alpha` — use the double-underscore addressing and name the pipeline steps.
9. **Refit semantics.** (i) `LogisticRegressionCV(refit=True)` vs `refit=False`: one line each, from the slide. (ii) The slide says `RidgeCV` reports the estimated regularization strength via “`alphas`”, but §37.7 reads it from `alpha_`. Reconcile the two names using the constructor-vs-fitted-attribute distinction. (iii) You must tune *both* the polynomial degree and the ridge `alpha` on a regression task. Name the search object and the estimator choice the slide sanctions, and say why `RidgeCV` alone is insufficient.

Solutions — Chapter 39: Model evaluation, cross-validation, hyperparameter tuning

Worked solutions for the problem set in `chapters/39-model-evaluation-cv.md`. Numbers marked [recorded] are printed in the course material; [verified] are hand arithmetic from this session (numpy used as a calculator only — sklearn was never run).

1. Split variance, by hand. $y = (1, 2, 10, 11)$; model = training mean.

(i) Test MSEs:

- Split A, train $\{1, 2\}$: $\hat{y} = 1.5$. $\text{MSE} = \frac{(10-1.5)^2 + (11-1.5)^2}{2} = \frac{72.25 + 90.25}{2} = \boxed{81.25}$.
- Split B, train $\{1, 10\}$: $\hat{y} = 5.5$. $\text{MSE} = \frac{(2-5.5)^2 + (11-5.5)^2}{2} = \frac{12.25 + 30.25}{2} = \boxed{21.25}$.
- Split C, train $\{1, 11\}$: $\hat{y} = 6$. $\text{MSE} = \frac{(2-6)^2 + (10-6)^2}{2} = \frac{16 + 16}{2} = \boxed{16.00}$. [verified]

(ii) Leave-one-out. The four hold-outs:

held out y_i	mean of other three	squared error
1	23/3	$(1 - 23/3)^2 = (20/3)^2 = 400/9$
2	22/3	$(2 - 22/3)^2 = (16/3)^2 = 256/9$
10	14/3	$(10 - 14/3)^2 = (16/3)^2 = 256/9$
11	13/3	$(11 - 13/3)^2 = (20/3)^2 = 400/9$

$$\text{MSE}_{\text{LOO}} = \frac{1}{4} \cdot \frac{400 + 256 + 256 + 400}{9} = \frac{1312}{36} = \frac{328}{9} \approx \boxed{36.44}.$$

[verified]

- (iii) Any single split’s number is hostage to which rows fell into test — here 16 vs 81 for the same model. Averaging over many splits (LOO: 36.44) washes out that luck; the slide’s point is that the *average* is the honest estimate of generalization, not any one draw.

2. Fold arithmetic.

- (i) `KFold(n_splits=3)`, $n = 6$, no shuffling $\rightarrow$ contiguous blocks: test sets $\{0, 1\}$, $\{2, 3\}$, $\{4, 5\}$. Each row is tested exactly once and trained on exactly twice. [verified]
 - (ii) `LeaveOneOut = KFold(n_splits=n)` = 6 fits: test sets $\{0\}, \{1\}, \dots, \{5\}$, each fit training on the other five rows.
 - (iii) `[0 2 1 3] = np.append(train, test)` for the *final* loop iteration: rows $\{0, 2\}$ trained, rows $\{1, 3\}$ tested in that split. Only one array is returned because the course's function rebinds `train`, `test` on every iteration and calls `np.append` *after* the loop — the first three splits' indices are silently discarded. (The graded answer's option (a).) [recorded]
-

3. Two functions, two returns.

- (i) An array of length 5 holding the five per-fold *test* scores (default scoring for the estimator — R^2 for a regressor).
 - (ii) `fit_time`, `score_time`, `test_score` — always; `estimator` — only with `return_estimator=True`; `train_score` — only with `return_train_score=True`.
 - (iii) Mean of the neg-scores: $\frac{-16.04-18.90-15.02}{3} = \frac{-49.96}{3} \approx -16.6533$; negate for the error: $\text{MSE} \approx 16.65$. [verified]
 - (iv) `cross_val_score` is the thin convenience wrapper: one metric in, one array out. `cross_validate` is the instrumented worker — it loops the folds once and can accumulate as many metrics per fold as you name.
-

4. Course-sourced metric. $y = [7, 4, 9, 4]$, $\hat{y} = [8, 7, 12, 5]$.

- (i) Residuals $r = y - \hat{y} = [-1, -3, -3, -1]$, $\bar{r} = -2$: $\text{Var}(r) = \frac{1^2 + (-1)^2 + (-1)^2 + 1^2}{4} = 1$. $\bar{y} = 24/4 = 6$: $\text{Var}(y) = \frac{1^2 + (-2)^2 + 3^2 + (-2)^2}{4} = \frac{18}{4} = 4.5$.

$$\text{EVS} = 1 - \frac{1}{4.5} = 1 - \frac{2}{9} = \boxed{\frac{7}{9} \approx 0.7778},$$

matching the recorded 0.7777777777777778 to every shown digit. [verified]

- (ii) Perfect predictor: every residual 0 $\rightarrow \text{Var}(r) = 0 \rightarrow \text{EVS} = 1$. Mean-predictor: $\hat{y}_i = \bar{y} \rightarrow r = y - \bar{y} \rightarrow \text{Var}(r) = \text{Var}(y) \rightarrow \text{EVS} = 0$. (This is why the slide's R^2 page gives the constant model 0.0 — same floor.)
 - (iii) $R^2 = 1 - u/v$ uses the *raw* residual sum of squares; explained variance subtracts the residual mean first ($\text{Var}(r) = \frac{1}{n} \sum r_i^2 - \bar{r}^2$). On the *test* set the residuals need not sum to zero (no intercept was fit to them), so the bias correction makes EVS a hair higher: $0.6605500501742703 > 0.6605140591531992$ [recorded].
-

5. Read the curves.

- (i) **Underfit** (§23.9's $m = 1$ line): both errors high and no train/CV gap, so the model lacks capacity — more data will not move either curve. Fix direction: richer model (more features, weaker regularization), not more rows.

- (ii) **Overfit with a closing gap:** train error $0.10 \ll$ CV error 0.45 is §23.9’s overfitting signature, but the CV curve is still falling at max n — the gap is data-starved, not capacity-wrong. Buy *data* first; reach for regularization only if the gap plateaus open.
- (iii) `train_size, train_scores, test_scores = results[:3]; train_errors, test_errors = -train_scores, -test_scores` (the negation undoes the `neg_` scoring). With 5 training sizes and 4 folds, `train_scores` (and `test_scores`) have shape `(5, 4)`.

6. Grid vs randomized.

- (i) Verbatim [recorded]:

UserWarning: The total space of parameters 8 is smaller than `n_iter=10`.

Running 8 iterations. For exhaustive searches, use `GridSearchCV`.

`sklearn` ignored the `n_iter=10` request and ran all 8 combinations ($2 \text{ fit_intercept} \times 4 \text{ alpha}$) — random search degenerated into exhaustive grid search, and the warning says to use `GridSearchCV` for that.

- (ii) $4^3 = 64$ combinations $\times$ 5 folds = `320` fits. [verified]
- (iii) Randomized search wins when the space is large or continuous — the budget (`n_iter`) stays fixed while a grid explodes exponentially. Grid search wins when the space is small and discrete enough to enumerate — then it *is* the exhaustive answer, and random sampling can only miss combinations.

7. The three-way doctrine.

- (i) 1. Split into train/validation/test. 2. Train every hyperparameter combination on train (`n_jobs=-1` to parallelize; `error_score=0/NaN` so one bad combination doesn’t crash the search). 3. Score each on validation; pick the best. 4. Retrain the winner on train+validation combined. 5. Evaluate once on test.
- (ii) The validation rows’ job was *selection*; once the hyperparameters are frozen, there is no reason to withhold those rows from *parameter* learning. Retraining on train+validation gives the final model strictly more data at zero selection cost — the choice is already made.
- (iii) It honors “**spend the test set exactly once**” (§37.9): the test split was never touched during the search, and this single `.score` call is its one and only use. The STEP-4 retraining was done by `GridSearchCV`’s default `refit=True`, which refits the best combination on the full training data before returning.

8. Spot the leak.

- (i) The learn-on-train-only discipline (§36.7’s imputation note, §37.8’s transformer contract, §38.10’s PCA story). `scaler.fit_transform(X_train)` fixes μ, σ *before* the search, so inside the CV loop every fold’s training rows are scaled with statistics that saw the *other* folds’ rows — the fold boundary is porous. (Mild here, since it’s still train-only data; fatal in §38.10’s version, which fit on the test set.)
- (ii) Move the scaler inside the searched object so each fold refits it on its own training rows; address the estimator’s dial through the step name:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDRegressor
from sklearn.model_selection import GridSearchCV

pipeline = Pipeline(steps=[('scale', StandardScaler()),
                           ('sgd', SGDRegressor(random_state=1))])
```

```
search = GridSearchCV(pipeline,
                      {'sgd__alpha': [0.1, 0.01, 0.001]},
                      cv=4)
search.fit(X_train, y_train)
```

`sgd__alpha` is the double-underscore addressing (§37.8): step `sgd`, hyperparameter `alpha`.

9. Refit semantics.

- (i) `refit=True`: average each candidate's scores across folds, take the hyperparameter values behind the best average score, and **refit once** with them — one winning model. `refit=False`: **average the coefs, intercepts, and C** of the per-fold winners instead — the answer is a blend, with no final single fit. (The slide's words, §38.5.)
- (ii) `alphas` is the **constructor's** candidate list — the values you ask it to try. `alpha_` is the **fitted attribute** — the value cross-validation actually chose. The slide names the input side; §37.7 names the output side. Both are right about different objects.
- (iii) `GridSearchCV` over a `Pipeline` of `PolynomialFeatures` + `Ridge` — e.g. `param_grid = [{'poly__degree': [2, 3, 4, 5, 6, 7, 8, 9], 'ridge__alpha': [0.5, 0.1, 0.05, 0.01, 0.005, 0.001]}]`. `RidgeCV` alone is insufficient because its built-in search covers only the regularization strength — it cannot vary the polynomial degree (or any second dial); the slide sanctions exactly this “grid over the pipeline” shape for multi-dial tuning.

Chapter 40: End-to-end model training and debugging

Everything in §40.1 comes from the MLP Week-1 Lecture 1–5 deck, “End to end Machine Learning Project” (the wine-quality project — the course’s own canonical workflow, from problem framing to launch); §§40.2–40.4 replay the debugging cases already built in Chapters 36–39, plus one new catch found by re-reading the deck itself (§40.2’s case 5). This is the last chapter of Part V, and it is a synthesis: the runbook is the course’s, and the playbook below it is **the book’s** — assembled from the chapters’ own experiments, not from the course slides.

A word on what was actually run. The position is the same as Chapters 37–39: sklearn is not installed on this machine, so no `sklearn` call below was executed here. Deck code and numbers are transcribed from the slides — verbatim unless a slide typo forced a fix (each fix is logged in `reviews/40-end-to-end-debugging.md`). Every *number* beside the code is either **[recorded]** — printed as output in the course material itself — or **[verified]** — worked by hand in this session from definitions (confusion-matrix arithmetic, grid-size counting, index sets). `numpy` is installed, so every `numpy`-only check was re-run and is marked **[verified-NumPy]**. The boundary matters more than usual in this chapter: the taxonomy, the checklists, and every “the book’s” label are the book’s synthesis; the cases, code, and numbers are the course’s or the earlier chapters’.

Notation. $X \in \mathbb{R}^{n \times d}$ as usual (n points, d features, points as rows — the book’s (n, d) , §36.1).

40.1 The end-to-end runbook: the course’s eight steps

The deck opens with eight steps and then walks a wine-quality project through every one of them. As a runbook, with the book’s chapters doing each step’s machinery:

i) Look at the big picture. Frame the problem: “What is input and output? What is the business objective? ... Is this a classification, regression or some other task?” Then select the performance measure — the deck’s menu: regression → MSE or MAE (§37.5’s shelf); classification → precision, recall, F1-score, accuracy (§38.8–38.9’s ledger). Then the deck’s underlined rule: “List down various assumptions about the task. Review with domain experts ... **Make sure all assumptions are reviewed and approved before coding!**” (exclamation mark in the original). Debugging lesson zero lives here: a metric chosen *after* seeing results is a metric chosen to flatter.

ii) Get the data. “It’s a good practice to create a function for downloading and extracting the data.” The deck’s wine flow: `pd.read_csv(data_url, sep=";")` → 1599 rows, 12 columns — 11 features + label `quality`, `quality` ∈ [3, 8] [recorded] (§36.7’s verified re-run).

iii) Discover and visualize. `head()`, `info()`, `describe()`, `value_counts()` (“lots of samples of average wines” — qualities 5 and 6 dominate), histograms, `corr()` (“only captures linear relationship”), heatmaps, scatter matrices. The deck’s two discipline lines: “Before any further exploration, **it’s a good idea to separate test set and do not look at it** in order to have a clean evaluation set” — because “when we look at the test set, we are likely to notice patterns in that and based on that we may select certain models. This

leads to biased estimation on test set ... This is called **data snooping bias**.” And the deck’s humility note: “Visualization and data exploration do not have to be absolutely thorough ... **Exploration is an iterative process**: Once we build model and obtain more insights, we can come back to this step.” (§36.6’s seeded split; §39.6’s spend-once doctrine.)

iv) Prepare the data. “It’s a good practice to make a copy of the data and apply preprocessing on that copy. This ensures that in case something goes wrong, we will at least have original copy of the data intact.” The deck’s four moves: separate features and labels; handle missing values and outliers (`isna().sum()`; impute — `SimpleImputer(strategy="median")` learning the medians `[7.9, 0.52, 0.26, ..., 10.2]` on the *training* features [recorded]; drop; or keep-as-NaN when the value can’t exist — §36.7’s discipline); scale features (“Most ML algorithms do not perform well when input features are on very different scales” — “Scaling of target label is generally not required”); transforms (log, square root). Then the deck’s slide-57 law, the sentence this whole part of the book is built on: “**Note that all these transformers are learnt on the training data and then applied on the training and test data to transform them. Never learn these transformers on the full dataset.**” (§37.8’s machinery exists to make this structural.)

v) Select and train — baseline first. “It’s a good practice to build a quick baseline model on the preprocessed data and get an idea about model performance.” The deck’s baseline: `LinearRegression()` → train MSE 0.4207, test MSE 0.3976 [recorded]. Then the deck’s cautionary tale — `DecisionTreeRegressor()`: train MSE 0.0, test MSE 0.5813 [recorded] — “Note that the training error is 0, while the test error is 0.58. **This is an example of an overfitted model.**” (§23.9 on paper; §39.4 as code.) Then robust comparison: `cross_val_score(..., scoring="neg_mean_squared_error", cv=10)` — linear regression 0.4316 ± 0.0836 [recorded]; decision tree 0.6852 ± 0.1667 [recorded]; random forest 0.3457 ± 0.0736 [recorded] — “LinReg has better MSE and more precise estimation compared to DT.” (The deck’s verdict, printed under the numbers.)

vi) Fine-tune. `GridSearchCV(forest_reg, param_grid, cv=5, scoring='neg_mean_squared_error', return_train_score=True)` over 18 combinations ($12+6$, two grids) $\times$ 5 folds = 90 fits [recorded arithmetic]; `best_params_` → `{'max_features': 6, 'n_estimators': 30}` [recorded]; “GridSearchCV is initialized with `refit=True` option, which **retrains the best estimator on the full training set.**” Big spaces: `RandomizedSearchCV` — “enables us to search hyperparameter space with appropriate budget control.” (§39.6–39.7; the $18 \times 5 = 90$ count is the deck’s own.) Then the deck’s debugging-adjacent line, quoted whole because it is the motto of §40.2: “**It is also useful to analyze the errors in prediction and understand its causes and fix them**” — plus `feature_importances_`, “based on this information, we may drop features that are not so important.”

vii) Evaluate once, then present. The test set, spent exactly once (§39.6’s doctrine): transform the test features, predict, one metric — test MSE 0.3535 [recorded] — and the deck’s honest extra: “It’s a good idea to get 95% confidence interval of the evaluation metric,” (0.2916, 0.4153) [recorded] via `stats.t.interval`. Then present: “highlights learnings, assumptions and systems limitation. Document everything, create clear visualizations.”

viii) Launch, monitor, maintain. The deck’s three words: **Launch** — “Plug in input sources and write test cases”; **Monitoring** — “system outages, degradation of model performance, sampling predictions for human evaluation, regular assessment of data quality, which is critical for model performance”; **Maintenance** — “train model regularly every fixed interval with fresh data.” The runbook doesn’t end at the test score; it ends when the model is serving and someone is watching it.

The deck’s framing line, worth keeping above the whole runbook: “ML is usually a small piece in a big project... **Typically 10-15% of time is spent on ML.** A lot more time is spent on capturing and processing data needed for ML and taking decisions based on output of ML module.”

eg 1 (the runbook on one line each). The §36.8 house table through the deck’s steps: (i) big picture — regression, price in dollars, metric MAE; (ii) get — the inline CSV; (iii) explore — `describe()`, `rooms = NaN` in row 9 (§36.8); (iv) prepare — median-impute rooms on the train rows only; (v) baseline — `DummyRegressor(strategy="mean")` floor first, then `LinearRegression` (§37.4, $R^2 = 0.9619$ [verified-NumPy]); (vi) tune — none needed at $n = 10$; (vii) evaluate once — on the §36.6 seeded split’s test rows;

(viii) launch — n/a , it’s ten houses. Every step already done somewhere in Chapters 36–39; this chapter is the index.

Basically, ... “The course’s end-to-end project is eight steps in one sentence: *frame it, fetch it, look at it, clean it, baseline it, tune it, grade it once, ship it and watch it*. The middle six steps are Chapters 36–39; this chapter is the checklist that keeps them in order — and the rest of the chapter is what to check when a step’s output looks wrong.”

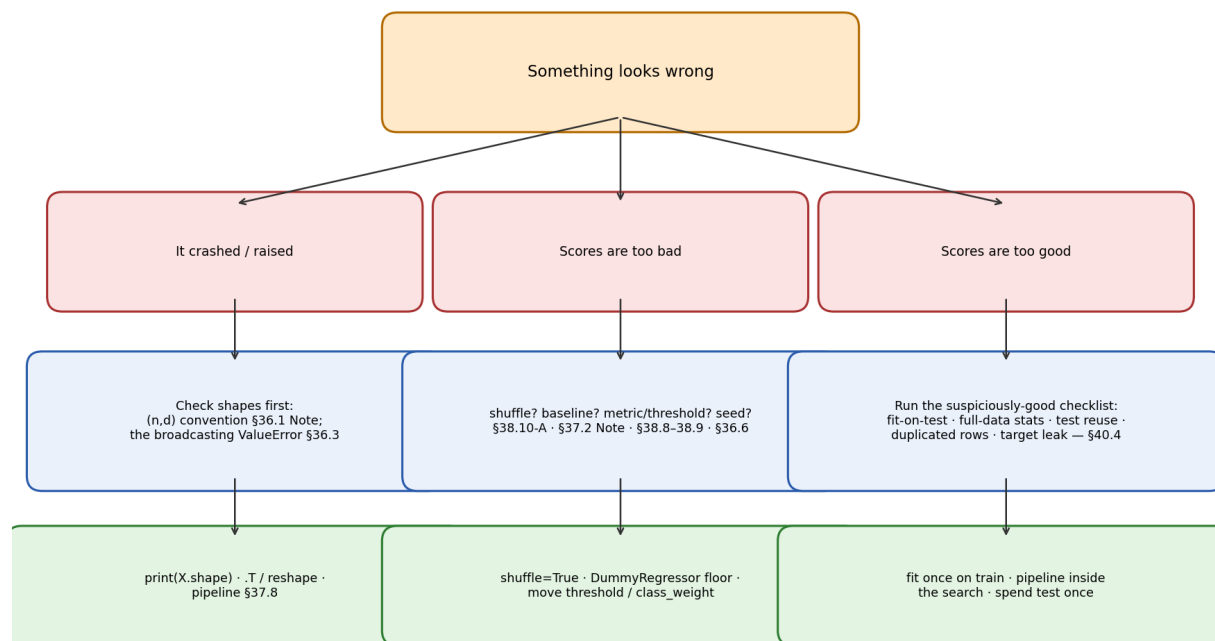


Figure 57: Debugging-playbook flowchart: one entry box (“something looks wrong”) branches into three. “It crashed / raised” points to “check shapes first — the (n,d) convention, §36.1 Note; the broadcasting ValueError, §36.3”. “Scores are too bad” points to “shuffle? baseline? metric/threshold? seed? — §38.10-A, §37.2 Note, §38.8–38.9, §36.6”. “Scores are too good” points to “run the suspiciously-good checklist — leaks, test reuse, duplicated rows, §40.4”.

40.2 The debugging playbook: symptom → diagnosis → fix

Every case below is one the book has already met — the course’s own experiments, or the deck itself. Same shape each time: the **symptom** (what the screen shows), the **diagnosis** (what’s actually wrong), the **fix**.

Case 1 — the shape/convention bug. *Symptom:* `ValueError` at `fit` or `predict`, or — worse — code that runs and silently computes garbage. *Diagnosis:* the (d, n) habit (§36.1’s Note) fed where the (n, d) convention is expected, or an axis collapse read backwards (§36.4’s collapse rule). *Fix:* `print(X.shape)` first, always — in $(60000, 28, 28)$ the *first* axis is the sample axis; convert with `.T` or `reshape` the moment the sample axis isn’t first. §36.3’s rule of thumb: “That error is your friend — it means you mixed up the (n, d) convention.” The cheapest bug to find and the most common one to make.

eg 2 (the broadcast that tells on you) [verified-NumPy]. Mean-centering with a transposed mean:

```

X = np.arange(6).reshape(3, 2)      # (3, 2): 3 points, 2 features
mu = X.mean(axis=0)                 # (2,) per-feature means: [2., 3.]
X - mu                             # (3,2)-(2,) -> fine: mu stretched down the rows

```

```
X - mu[:, np.newaxis]          # (2,1) against (3,2) -> ValueError: shapes (3,2) and
    ↪ (2,1) not aligned
```

The (2,1) column form broadcasts against the *first* axis and dies; the flat (2,) form aligns on the trailing axis and works. If the subtraction had silently succeeded with the wrong alignment, every downstream number would be nonsense — the `ValueError` is the good outcome.

Case 2 — the shuffle disaster (§38.10, Experiment A). *Symptom:* accuracy 0.5486, precision 1.0000, recall 0.0703 [recorded] — the confusion matrix $(TN, FP, FN, TP) = (196, 0, 172, 13)$ [recorded]: the perceptron predicts -1 for nearly everything. *Diagnosis:* `shuffle=False` with training rows stacked `+1s-then-1s`; SGD “estimates the gradient with one sample at a time” (§38.6), so 863 fives drag the weights deep into five-territory before 1032 threes arrive, and it never recovers. *Fix:* shuffle the training data before fitting — the default, and the slide’s rule, “not hygiene, it is load-bearing.”

Case 3 — the PCA double-fit leak (§38.10, Experiment B). *Symptom:* ten PCA components, perceptron, accuracy 0.5564 — barely above coin-flip [recorded]. *Diagnosis:* the solution’s first attempt ran `pca.fit(x_train1)` and `pca.fit(x_test1)` — test statistics learned into the projection, *and* train and test projected onto different bases. *Fix:* fit once, on train only (`p.transform(x_test1)`) — or, structurally, `make_pipeline(pca, clf)` so the leak is un-writable (§37.8’s Note). The corrected run: accuracy 0.9213 [verified] — the leak had cost 36 points.

Case 4 — scale-outside-CV (§39.9). *Symptom:* CV numbers that look a shade rosier than the honest pipeline — nothing crashes, nothing screams. *Diagnosis:* the scaler (or imputer, or PCA) was *fit outside* the fold loop, so every fold’s “training” statistics already saw the validation rows. *Fix:* put the whole pipeline *inside* `GridSearchCV`; the grid addresses steps with `poly__degree`-style double underscores (§37.8), and every fold refits every learning step on its own rows.

Case 5 — the deck’s own slip (found by re-reading). *Symptom:* none visible — the numbers print fine. *Diagnosis:* the wine deck’s own test-evaluation snippet, in *both* places it evaluates the test set, calls `transform_pipeline.fit_transform(wine_features_test)` — refitting the imputer medians and the scaler’s μ, σ on the **test rows**, in direct violation of the deck’s own slide-57 law (“Never learn these transformers on the full dataset,” §40.1-iv). *Fix:* one method call — `.transform(wine_features_test)`, the statistics learned on train replayed on test. The book’s catch, logged in the review; the course’s numbers are kept as [recorded] regardless.

Case 6 — the no-baseline trap (§37.2’s Note, §39.1). *Symptom:* “ $R^2 = 0.05$ — is that good?” *Diagnosis:* no floor was ever built, so the question is unanswerable. *Fix:* `DummyRegressor(strategy="mean")` first — a constant predictor scores exactly 0.0 (§37.5’s second landmark), so anything above zero is learning and anything at-or-below is noise. §39.1’s discipline: beat the dummy first, then tune.

Case 7 — the wrong metric / threshold (§38.8–38.9). *Symptom:* accuracy 99%, model useless — the classic rare-class story (a disease classifier that never predicts disease). *Diagnosis:* the metric doesn’t match the job (§40.1-i’s “select the performance measure” — skipped or chosen late). *Fix:* read the confusion matrix (§38.8’s ledger: precision = purity of the alarms, recall = completeness of the net), move the threshold at *prediction* time (free, no retraining — §38.9’s second lever), or penalize rare-class mistakes at *training* time with `class_weight` (§38.5’s first lever). Multiclass: pick the `average=` (§38.9) that matches the question — `macro` gives rare classes an equal vote.

Case 8 — the silent seed (§36.6, §39.2’s Note). *Symptom:* the numbers change every run; a colleague can’t reproduce your split. *Diagnosis:* no seed was set — or worse, a seed was set where it does nothing: the slide’s `KFold(n_splits=5, random_state=42)` never sets `shuffle=True`, so `random_state` is silently inert (logged in §39’s review). *Fix:* seed deliberately (`random_state=42, np.random.seed()`), and verify the flag actually takes effect — reproducibility is §36.6’s “controlled substance.”

Note (the seed that resets itself). The deck’s own `split_train_test` calls `np.random.seed(42)` *inside* the function body — so every call replays the identical permutation for same-length data (§36.6’s problem 6-ii). Fine for “same test set every run”; a trap the moment you expect two calls to give two *different* splits.

Seed once, at the top, outside the function.

Case 9 — the mis-sized search (§39.7). *Symptom:* sklearn prints, verbatim [recorded]:

UserWarning: The total space of parameters 8 is smaller than n_iter=10.
Running 8 iterations. For exhaustive searches, use GridSearchCV.

Diagnosis: RandomizedSearchCV on a tiny *discrete* space ($2 \times 4 = 8$ combos) degenerates into grid search.
Fix: grid for small/exhaustive spaces, random sampling for large or continuous ones — “with appropriate budget control” is the deck’s phrase, and the warning is the instrument telling you the budget was spent wrong.

Basically, ... “Nine cases, one pattern: *the pipeline looked right and the numbers lied*. Shapes lie silently (case 1) or loudly (the `ValueError` — take the loud one). Order lies to SGD (case 2). `fit` lies to the test set (cases 3, 4, 5 — all the same disease: something learned where it should only have applied). Metrics lie by answering the wrong question (cases 6, 7). Randomness lies by not repeating (case 8). And the search lies by warning you in plain English while you scroll past it (case 9).”

40.3 Reading the instruments

The numbers the workflow prints are a dashboard. What each reading means, in the book’s vocabulary:

i) Negative R^2 . §37.5’s third landmark: the model is *worse than predicting the mean* — “the model can be arbitrarily worse.” Not “a bit off” — a sign error in the pipeline, a target leaked wrong, or a model fit on nonsense. Don’t tune it; debug it (start at case 1).

ii) Train error tiny, test error large. §23.9’s overfitting signature, in the deck’s own numbers: the tree’s train MSE 0.0 vs test 0.5813 [recorded] — “This is an example of an overfitted model.” (§39.4’s learning-curve version: the gap.)

iii) Both errors high and flat. Underfit — the model can’t express the pattern; more data won’t help (§39.4’s first reading). Change the model: features, capacity, less regularization.

iv) The gap that won’t close. Overfit that needs the *model* changed, not the data (§39.4’s third reading) — and note §38.10’s Experiment C cuts the other way: regularization is not fairy dust; on the PCA-compressed digits, L2 *cost* 17 points of accuracy ($0.9213 \rightarrow 0.7480$ [recorded]) and even L1 cost 5. So “the gap won’t close” has two possible prescriptions — more capacity *or* less regularization — and the book’s honest answer is: that’s what the validation set (§39.6) is for.

v) `fit_time` exploding in `cross_validate`. The deck’s own arithmetic: 18 combos $\times$ 5 folds = 90 fits. If each fit is minutes, the search is days — shrink the grid, sample the space (§39.7), or reach for the `*CV` estimators’ efficient paths (§39.8).

eg 3 (three dashboards, read aloud). (a) `learning_curve` prints train error 0.02, CV error 0.61, and the CV curve is still falling at the largest $n \rightarrow$ overfit that data can cure: buy rows. (b) Train error 0.80, CV error 0.82, both flat from $n = 200$ to $n = 2000 \rightarrow$ underfit: buy capacity (§39.4’s readings, applied). (c) Test accuracy 1.0000 on the first run $\rightarrow$ don’t celebrate; run §40.4.

Basically, ... “The dashboard speaks in four sentences: *negative R^2* = something is broken, not untuned. *Train tiny, test huge* = memorizing (the deck’s tree). *Both high and flat* = too simple. *A gap that won’t close* = change the model — but remember the digits: sometimes the fix is *less* regularization, not more. And *perfect* = suspicious, always.”

40.4 The “suspiciously good” checklist (the book’s)

When a score lands near perfection — accuracy 1.0, error ≈ 0 — on the first serious run, the book’s checklist, in suspicion order. (This is the book’s synthesis, not the course’s: the course supplies the cases, the checklist is the assembly.)

i) Was something fit on the test set? The PCA double-fit (§38.10-B) is the canonical form: any transformer or imputer that learned statistics from test rows. Grep for `fit` on anything named `test`.

ii) Were statistics learned on the full dataset? §37.8’s Note, §36.7’s imputation rule, the deck’s slide-57 law: scaler μ/σ , imputer medians, PCA bases — if any were computed before the split (or outside the fold loop, §39.9), test information is in the training statistics. The pipeline is the structural fix; the checklist is the audit.

iii) Was the test set spent more than once? §39.6’s doctrine: tuning on test, then reporting on the same test, is selection bias wearing a lab coat. Retrain-on-train+validation is fine; choosing *anything* on the test set is not.

iv) Are there duplicated rows across the split? The course never names this one, so the book flags it as its own addition: near-duplicate rows (same sensor reading logged twice, a template repeated) put the same information in train and test, and the test score measures memorization. `df.duplicated().sum()` before the split is the two-second check.

v) Is a feature secretly the label? Target leakage in its plainest form: a column that *is* the answer (or is computed from it — a “days until default” column in a default-prediction task). The deck’s Step-1 discipline (“list the assumptions... approved before coding”) is the prevention; the checklist is the autopsy.

Basically, ... “Perfect scores are guilty until proven innocent. The five suspects: *fit on test*, *statistics from the full dataset*, *the test set spent twice*, *duplicated rows*, *a feature that is the label*. Check them in order, and most ‘amazing’ models confess by suspect two.”

40.5 Where this goes next

i) The contract survives — including inside neural nets. GridSearchCV honors the estimator contract (§39.10), MLPClassifier/MLPRegressor honor it too (the Week-12 slide: `fit(X_train, y_train), predict, predict_proba, score` → R^2 [recorded]) — and §37.10/§38.11 already said the perceptron of §38.4 *is* Chapter 41’s neuron with a sign snapped on top. So Part VI inherits this chapter whole: debugging a neural net starts with the same playbook — shapes first (the (n, d) convention is load-bearing in every layer), then scale (§37.3’s scaling sermon returns with a vengeance for SGD-trained nets), then shuffle, then the leak audit.

ii) The runbook doesn’t end at the test score. The deck’s Step 8 is the last word of Part V: launch with test cases, monitor for degradation and data-quality drift, retrain on fresh data on a schedule. A model is a system with a maintenance contract, not a number.

iii) What this part built. Chapters 36–40, one sentence each: 36 turned math into arrays; 37 taught the estimator contract on regression; 38 replayed it on classification; 39 made evaluation honest; 40 is the checklist that keeps the other four honest in practice. Part VI (Chapters 41–47) builds the models this discipline will debug.

Problem set

- The eight steps.** (i) List the deck’s eight end-to-end steps in order. (ii) Which step does “data snooping bias” belong to, and what causes it — quote the deck’s definition. (iii) The deck says “typically 10-15% of time is spent on ML” — in one line, where does the rest go?
- The broadcast that tells on you.** `X = np.arange(6).reshape(3, 2)` (eg 2). (i) Compute `mu = X.mean(axis=0)` by hand. (ii) Give the result of `X - mu` and explain the broadcasting rule that makes it work. (iii) Say exactly why `X - mu[:, np.newaxis]` raises `ValueError`, and why that error is the *good* outcome.
- The shuffle case.** §38.10-A’s confusion matrix: $(TN, FP, FN, TP) = (196, 0, 172, 13)$. (i) Verify accuracy 0.5486, precision 1.0, recall 0.0703 from the four counts. (ii) In two lines, explain why `shuffle=False` produces exactly this signature — near-perfect precision, catastrophic recall — given the stacked +1s-then-1s training rows. (iii) Name the constructor argument and its correct value.

4. **Spot the leak.** The deck evaluates its final model with `transform_pipeline.fit_transform(wine_features_test)` (case 5). (i) The pipeline was already `fit_transformed` on the training features — say exactly what `fit_transform` re-learns on the test rows. (ii) Quote the deck’s own rule this violates. (iii) Write the corrected line, and say which §37.8 discipline it restores.
5. **Read the dashboard.** (i) The deck’s decision tree: train MSE 0.0, test MSE 0.5813 [recorded]. Diagnose in §23.9’s vocabulary and quote the deck’s own verdict. (ii) Ridge reports test $R^2 = -0.4$. Say what this means using §37.5’s landmarks, and whether the next step is tuning or debugging. (iii) A learning curve shows train error 0.8, CV error 0.82, both flat from $n = 200$ to $n = 2000$. Diagnose per §39.4 and name the fix direction — data, capacity, or regularization?
6. **Search arithmetic.** (i) The deck’s grid: two parameter dicts ($3 \times 4 = 12$ and $2 \times 3 = 6$ combos), `cv=5`. Reproduce the $18 \times 5 = 90$ fits count. (ii) Quote sklearn’s `UserWarning` from §39.7’s Lasso search verbatim [recorded] and say what sklearn did instead of the requested `n_iter=10`. (iii) In one line each: when is randomized search the better choice, and when is grid search?
7. **Spend the test set once.** (i) List §39.6’s five HPT steps in order. (ii) The deck reports its final test MSE 0.3535 with a 95% CI of (0.2916, 0.4153) [recorded] — which step of the doctrine does the CI belong to, and what does the interval add over the point estimate? (iii) `GridSearchCV`’s `refit=True` default implements which of the five steps automatically?
8. **Error analysis, the deck’s way.** (i) Quote the deck’s error-analysis line verbatim [recorded]. (ii) `sorted(zip(feature_importances, feature_list), reverse=True)` — say what decision this sorted list supports, in the deck’s words. (iii) The deck’s CV comparison: linear regression 0.4316 ± 0.0836 , decision tree 0.6852 ± 0.1667 , random forest 0.3457 ± 0.0736 [recorded]. State the deck’s verdict and say which two numbers (mean and spread) it rests on.
9. **The suspiciously-good audit.** A teammate’s first serious run reports test accuracy 1.0000. (i) List the book’s §40.4 checklist in order. (ii) For suspects (i) and (ii), name the concrete code smell you’d grep for. (iii) The course never names suspect (iv) — say what it is and give the two-second pandas check.

Chapter 40 solutions: End-to-end model training and debugging

Full worked solutions for the problem set in `chapters/40-end-to-end-debugging.md`. Numbers marked [recorded] are printed as output in the course material itself; [verified] are worked by hand in this session; [verified-NumPy] were re-run with numpy 1.26.4.

1. The eight steps

- (i) The deck’s eight steps, in order: 1. Look at the big picture. 2. Get the data. 3. Discover and visualize the data to gain insights. 4. Prepare the data for ML algorithms. 5. Select a model and train it. 6. Fine-tune your model. 7. Present your solution. 8. Launch, monitor and maintain your system. [recorded]
- (ii) Step 3 (discover/visualize). The deck’s definition: “When we look at the test set, we are likely to notice patterns in that and based on that we may select certain models. This leads to biased estimation on test set, which may not generalize well in practice. This is called data snooping bias.” [recorded] — the remedy is to separate the test set *before* exploring and never look at it.
- (iii) “A lot more time is spent on capturing and processing data needed for ML and taking decisions based on output of ML module” — data plumbing and downstream decisions, with “strong collaboration with domain experts, product managers and eng-teams.” [recorded]

2. The broadcast that tells on you

- (i) `X = [[0,1],[2,3],[4,5]]`; column means: $(0 + 2 + 4)/3 = 2$, $(1 + 3 + 5)/3 = 3 \rightarrow \text{mu} = [2., 3.]$. [verified-NumPy]
- (ii) `X - mu: (3,2) - (2,)` — the `(2,)` aligns on the trailing axis and is stretched down all 3 rows:

```
[[0-2, 1-3],    [[-2., -2.],
 [2-2, 3-3],    = [ 0.,  0.],
 [4-2, 5-3]]    [ 2.,  2.]]
```

[verified-NumPy]

- (iii) `mu[:, np.newaxis]` has shape $(2,1)$. Broadcasting $(3,2)$ against $(2,1)$: trailing axes 2 vs 1 $\rightarrow$ the 1 stretches to 2; next axes 3 vs 2 $\rightarrow$ neither is 1, so it errors: `ValueError: operands could not be broadcast together with shapes (3,2) (2,1)`. [verified-NumPy] It is the good outcome because the alternative — a *silent* wrong-alignment subtraction — would poison every downstream number with no warning; the `ValueError` points at exactly the line where the convention was mixed up (§36.3’s “that error is your friend”).

3. The shuffle case

- (i) $\text{Accuracy} = (TP + TN)/(TP + TN + FP + FN) = (13 + 196)/381 = 209/381 \approx 0.5486$ [verified]. $\text{Precision} = TP/(TP + FP) = 13/(13 + 0) = 1.0$ [verified]. $\text{Recall} = TP/(TP + FN) = 13/(13 + 172) = 13/185 \approx 0.0703$ [verified]. They match the recorded numbers to every shown digit.
- (ii) With `shuffle=False`, SGD sees all 863 +1 rows first: the perceptron walks deep into five-territory, racking up positive predictions it believes in — then the 1032 −1 rows arrive and it over-corrects, ending up predicting −1 almost everywhere. The 13 rows it still calls positive are ones it is sure about (hence precision 1.0), but it finds only 13 of the 185 true positives (hence recall 0.0703).
- (iii) `shuffle=True` in the `Perceptron` constructor (it is the default; the course’s eg set it `False` explicitly).

4. Spot the leak

- (i) `fit_transform` on the test set re-learns everything the pipeline learns: the `SimpleImputer`’s per-feature medians and the `StandardScaler`’s per-feature μ, σ — computed from the *test* rows this time.
- (ii) The deck’s own slide-57 law: “Note that all these transformers are learnt on the training data and then applied on the training and test data to transform them. Never learn these transformers on the full dataset.” [recorded]
- (iii) `wine_features_test_tr = transform_pipeline.transform(wine_features_test)` — replaying the train-learned statistics on test. This restores §37.8’s discipline: `fit` learns on the training set, `transform` applies the learned transformation to new data; the test set only ever sees `transform`.

5. Read the dashboard

- (i) Train MSE 0.0, test MSE 0.5813: textbook **overfitting** — the model memorized the training rows (zero training error) and fails on fresh data. The deck’s own verdict: “Note that the training error is 0, while the test error is 0.58. This is an example of an overfitted model.” [recorded]
- (ii) $R^2 = -0.4$: §37.5’s third landmark — the model is *worse than always predicting the mean* (“the model can be arbitrarily worse” than the constant predictor). This is not an untuned model; it is a broken pipeline (sign error, leak, or nonsense features). Debug first — start at §40.2’s case 1 — don’t tune.
- (iii) Both errors high (≈ 0.8) and flat over a $10\times$ increase in n : **underfit** (§39.4’s first reading) — the model class can’t express the pattern, and more data will not help. Buy capacity (more/better features, a richer model) or reduce regularization — not more data.

6. Search arithmetic

- (i) Grid 1: 3 values of `n_estimators` $\times$ 4 values of `max_features` = 12 combos. Grid 2: 2 values of `n_estimators` $\times$ 3 values of `max_features` = 6 combos. Total $12 + 6 = 18$ combinations; with `cv=5`, each combination is trained 5 times: $18 \times 5 = 90$ fits. [verified]

- (ii) The warning, verbatim [recorded]:

UserWarning: The total space of parameters 8 is smaller than n_iter=10.
Running 8 iterations. For exhaustive searches, use GridSearchCV.

sklearn ran only the 8 existing combinations instead of the requested 10 samples.

- (iii) Randomized search is better on large or continuous spaces, where a fixed budget of `n_iter` samples beats an exponential grid; grid search is better on small discrete spaces where exhaustive enumeration is affordable.

7. Spend the test set once

- (i) §39.6’s five steps: 1. Split train/validation/test. 2. Train every hyperparameter combination on train (`n_jobs=-1`; `error_score` to survive bad combos). 3. Evaluate each on validation; pick the best. 4. Retrain the winner on train+validation. 5. Evaluate once on test.
- (ii) The CI belongs to step 5 — the final, once-spent test evaluation. The point estimate (0.3535) says where the error landed; the 95% interval (0.2916, 0.4153) says how much that number could wobble on a different test draw — the same “one split is a noisy number” honesty as §39.1, quantified.
- (iii) Step 4: with the default `refit=True`, `GridSearchCV` automatically refits the winning combination on the whole training data, so `best_estimator_` is ready to predict.

8. Error analysis, the deck’s way

- (i) “It is also useful to analyze the errors in prediction and understand its causes and fix them.” [recorded]
- (ii) The sorted (`importance`, `feature`) list ranks features by how much the forest relied on them; the deck’s decision: “Based on this information, we may drop features that are not so important.” [recorded]
- (iii) The deck’s verdict: “LinReg has better MSE and more precise estimation compared to DT.” [recorded] It rests on both numbers per model: the *mean* CV MSE (linear 0.4316 < tree 0.6852 — better) and the *standard deviation* (linear 0.0836 < tree 0.1667 — more precise). The forest beats both (0.3457 ± 0.0736), which is why the deck tunes it next.

9. The suspiciously-good audit

- (i) §40.4’s checklist, in order: (1) was something `fit` on the test set? (2) were statistics learned on the full dataset? (3) was the test set spent more than once? (4) are there duplicated rows across the split? (5) is a feature secretly the label?
- (ii) Suspect (1): grep for `fit` (applied to anything named `test` — `pca.fit(x_test)`, `scaler.fit_transform(X_all)` before the split. Suspect (2): any `fit/fit_transform` on the whole `X` before `train_test_split`, or a transformer `fit` outside the CV fold loop.
- (iii) Suspect (iv): near-duplicate rows leaking the same information into both train and test, so the score measures memorization, not generalization. The two-second check: `df.duplicated().sum()` before the split (the book’s addition — flagged as such in the review log).

Part VI — Bridge to Deep Learning

Chapter 41: The artificial neuron and MLPs from scratch

Part VI opens here. The shape of this chapter follows the GenAI Weeks 1–2 deck (“Introduction to ANN,” Balaji Srinivasan and Ganapathy Krishnamurthi), cross-checked against the MLT Week-12 ANN slides (Ashish Tendulkar) and the course’s own notebooks — the XOR lab, the PyTorch fundamentals, the PyTorch workflow, and the Fashion-MNIST classifier. One idea, fully worked: **a neuron is a perceptron with a smooth activation, and an MLP is layers of such neurons trained by gradient descent**. The from-scratch code in this chapter is new numpy written for this book — every number below was re-run here and is marked **[verified-NumPy]**. Notebook numbers are transcribed from the course material and marked **[recorded]** — torch is not installed on this machine, so no PyTorch line below was executed here.

Notation. Per-example (the deck’s, column-vector) form is $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$, $a^{[l]} = g(z^{[l]})$. Batch (the MLT slides’, and this book’s) form is $Z^l = A^{l-1}W^l + b^l$, $A^l = g(Z^l)$, with the book’s (n, d) convention — points as rows (§36.1). §41.4’s Note pins the two conventions to each other; mix them up and you get the transpose bug.

41.1 The neuron’s two steps

The MLT Week-12 slide writes the whole thing in two lines (its notation, single neuron, three inputs):

$$z = w_1a_1 + w_2a_2 + w_3a_3 + b, \quad a = g(z).$$

i) **Pre-activation:** combine the inputs linearly — a weighted sum plus a bias, $z = w^T a + b$. ii) **Activation:** pass the sum through a non-linearity g — $a = g(z)$.

The GenAI deck says the same thing in a bigger font: “This two-step ‘Linear Sum $\rightarrow$ Non-Linear Activation’ process is performed by **every single neuron** in a deep neural network.”

Def (artificial neuron). A function of its inputs, parameterized by weights and a bias: linear combination first, smooth non-linearity second. Everything in this chapter — layers, the MLP, the forward pass — is this two-step process repeated.

Basically, ... “A neuron takes a bunch of numbers, multiplies each by a weight, adds them up plus a bias — that is the linear part — then bends the answer with a squashing function, the activation. Two moves, every neuron, every layer. That is the whole animal.”

41.2 The lineage: a calculator, a learner, and a neuron

The deck tells this as three slides, and they are worth knowing because they say exactly what changed at each step:

- i) **The M-P neuron (a calculator, not a learner).** “The M-P neuron was a fixed logic gate. It could compute, but it could not learn.” Its two lacks, in the deck’s words: **no feature importance** —

“equally important features,” “missing piece: weights”; **no automatic learning** — “threshold (θ) is fixed. Must be set manually.”

- ii) **The perceptron (a learner).** Rosenblatt’s 1958 move was to *learn* the weights: $\hat{y} = \text{sign}(w^T \phi(x))$ (§31.2). The deck’s strengths list: “It Learns! An automatic learning rule adjusts weights from data. Feature Importance: learns which inputs are more important via weights. Convergence Guarantee: if data is linearly separable, it’s guaranteed to find a solution.” Its two limits: **the fatal flaw, linear separability only** — “only problems where a single straight line (or hyperplane) can separate the classes can be solved”; and **the harsh threshold** — “the step function activation is not differentiable, preventing modern gradient-based training.”
- iii) **The artificial neuron.** Take the perceptron’s weighted sum, keep the learned weights, and swap the step function for a *smooth* non-linearity — sigmoid, tanh, ReLU. Logistic regression (§31.8) was already this exact object wearing a statistics costume: $h = \sigma(\theta^T x + \theta_0)$ is a neuron with a sigmoid activation.

Basically, ... “Three generations: the M-P neuron *computed* with hand-set weights (no learning). The perceptron *learned* the weights but could only draw straight walls, and its step function was too harsh to descend. The artificial neuron keeps the learned weighted sum, swaps in a smooth activation — and now gradient descent can train it.”

41.3 Why smooth matters: differentiability $\rightarrow$ gradient descent

The perceptron trained by its own bespoke rule — fix mistakes one at a time (§31.4) — because its loss had a kink at the boundary (§31.3’s Note: $J(w)$ is “not differentiable in w ”). The artificial neuron’s move is to make *everything* differentiable:

- The activation g is smooth (or smooth enough — ReLU is smooth everywhere except a single corner, and gradient descent tolerates the corner).
- The loss is smooth (cross-entropy, squared error — §41.11).
- So the chain rule applies everywhere, gradients exist (§10.7’s multivariate machinery), and Chapter 10’s update $\theta := \theta - \eta \nabla J$ (§10.5) drives training.

This is why the deck’s “harsh threshold” bullet is load-bearing: the *only* thing that kept the perceptron off the gradient-descent highway was a non-differentiable activation. Smooth it, and the whole machinery of Part I applies.

Basically, ... “Gradient descent needs a slope to walk down (§10.5). A step function has no slope — flat, then a cliff. A smooth activation has a slope everywhere, so the loss has a slope, so the gradient exists, so Chapter 10’s machinery trains the network. The smoothness is the ticket onto the highway.”

41.4 From one neuron to a layer: the two vectorized equations

One neuron at a time is “incredibly inefficient” (the deck), so a whole layer is computed at once — the vectorized form (§36.2’s discipline, now paying off). The deck’s two core equations, for a layer l :

$$\boxed{z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}}, \quad \boxed{a^{[l]} = g(z^{[l]})}.$$

The MLT slides write the same two equations for a whole batch at once — the form the code will use:

$$\boxed{Z^l = A^{l-1} W^l + b^l}, \quad \boxed{A^l = g(Z^l)}, \quad A^0 = X.$$

The parts, from the slides: i) $a^{[l-1]}$ (A^{l-1}): activations from the previous layer — the input vector x at $l = 1$. ii) $W^{[l]}$ (W^l): the weight matrix. W_{ij} is the weight from neuron j in layer $l - 1$ to neuron i in layer l (deck’s index order). In the batch form, W^l has size $S_{l-1} \times S_l$ — each neuron in layer l gets S_{l-1} incoming weights, and the layer holds $S_{l-1} \times S_l$ weights total (MLT slides). iii) $b^{[l]}$ (b^l): one bias per neuron — a vector of

length S_l . iv) $z^{[l]}$ (Z^l): pre-activations — the weighted sums. v) $a^{[l]}$ (A^l): post-activations — g applied element-wise in the hidden layers (MLT slides).

Note (the transpose convention). The deck’s per-example form left-multiplies, $W^{[l]}a^{[l-1]}$ — its W is $S_l \times S_{l-1}$. The batch/MLT/code form right-multiplies rows, $A^{l-1}W^l$ — its W is $S_{l-1} \times S_l$. Same weights, transposed layout; pick the one that matches your data layout and don’t mix them. Everything below uses the batch form with the book’s (n, d) rows (§36.1). Shapes, for a layer with $S_{l-1} = m$ inputs and S_l neurons on n points:

$$A^{l-1} : n \times m, \quad W^l : m \times S_l, \quad b^l : (S_l,) \text{ broadcast across the rows (§36.3)}, \quad Z^l, A^l : n \times S_l.$$

Def (MLP). “Simply a feedforward network with one or more hidden layers” (the deck). Information flows one direction — input to output, no loops (MLT slides). The layers are indexed $0 \leq l \leq L$: layer 0 is the input, layer L the output, and L itself is a hyperparameter. Layer sizes: $S_0 = m$ features; $S_1, \dots, S_{L-1}$ are the hidden-layer widths — hyperparameters; $S_L = 1$ for regression, $S_L = k$ for k -class classification (MLT slides).

41.5 Worked example: one layer, every number recomputed

The deck’s worked example (slide 36/93): a hidden layer with 3 neurons receiving input from 2 features, ReLU activation. Every number recomputed here [verified-NumPy]:

eg 1 (the deck’s 2→3 ReLU layer).

$$x = \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix}, \quad W = \begin{bmatrix} 0.2 & 0.7 \\ -0.4 & 0.1 \\ 0.9 & -0.3 \end{bmatrix}, \quad b = \begin{bmatrix} 0.1 \\ 0.2 \\ -0.5 \end{bmatrix}.$$

Step 1 — weighted sum, $z = Wx + b$:

$$\begin{aligned} z_1 &= 0.2(0.5) + 0.7(1.0) + 0.1 = 0.1 + 0.7 + 0.1 = 0.9, \\ z_2 &= -0.4(0.5) + 0.1(1.0) + 0.2 = -0.2 + 0.1 + 0.2 = 0.1, \\ z_3 &= 0.9(0.5) + (-0.3)(1.0) + (-0.5) = 0.45 - 0.3 - 0.5 = -0.35. \end{aligned}$$

Step 2 — element-wise ReLU, $a = \max(0, z)$:

$$a = \begin{bmatrix} \max(0, 0.9) \\ \max(0, 0.1) \\ \max(0, -0.35) \end{bmatrix} = \begin{bmatrix} 0.9 \\ 0.1 \\ 0 \end{bmatrix}.$$

The third neuron fires 0 — a negative pre-activation killed by the threshold (§36.4’s `np.where` one-liner, in matrix form). The numpy re-run prints `z = [0.9 0.1 -0.35]`, `a = [0.9 0.1 0.]` — matching the slide [verified-NumPy].

eg 2 (same layer, batch form, four points at once). The same W, b applied to four rows, $A^0 \in \mathbb{R}^{4 \times 2}$:

```
X = np.array([[0.5, 1.0],
              [0.0, 0.0],
              [-1.0, 2.0],
              [0.5, 0.0]])          # (4, 2): four points as rows
Z = X @ W.T + b                  # (4, 2) @ (2, 3) -> (4, 3)
A = np.maximum(0, Z)
```

The row-first convention means the deck’s per-example W (3×2) gets transposed in batch code (§41.4’s Note). One matrix multiply replaces four loops — and on a GPU, “computationally efficient and perfectly suited for GPUs” (the deck).

41.6 Why depth needs non-linearity

Two facts, in order:

Fact 1 — stacking linear layers is useless. “Without a non-linear activation function, a deep network simply collapses into a single linear model, no matter how many layers it has” (the deck). Two layers without activation:

$$L_2(L_1(x)) = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = W'x + b'.$$

Just another linear function. “A 100-layer linear network has the same power as a 1-layer network.” The activation is what stops the collapse — with g , the composition $g(W_2g(W_1x + b_1) + b_2)$ “cannot be simplified,” which “allows the network to learn arbitrarily complex, ‘wiggly’ functions.”

Fact 2 — the perceptron’s fatal flaw needs depth to fix. The deck’s MLP slide states it plainly: “A single Perceptron fails on non-linear data (like XOR)” — “cannot be separated by one line.” The hidden layer is “an automatic feature engineer”: “it learns to transform the data into a new representation,” and “in this new space, the data becomes linearly separable.” The output layer then draws the (single) straight wall in *that* space.

Basically, ... “Depth without the bend is a lie: a hundred linear layers fold down into one ($W'x + b'$). The activation is the ingredient that makes stacking *do* something — each bent layer re-represents the data, and the last layer separates what the earlier layers arranged.”

41.7 Activations compared

The MLT slides’ three hidden-layer activations, plus the deck’s notes on when each breaks:

$g(z)$	Formula	Range	Why it helps	Why it hurts
Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$(0, 1)$	“Firing rate” reading; probabilities out (§31.8)	Vanishing gradients: “flat at both ends... gradient near zero for large inputs, effectively stops learning”; not zero-centered — “outputs always positive, which can slow down learning”
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Zero-centered — “helps center the data for the next layer, often speeding up convergence”	Same vanishing-gradient flattening — “difficult to use in very deep networks”

$g(z)$	Formula	Range	Why it helps	Why it hurts
ReLU	$\max(0, z)$	$[0, \infty)$	“Computationally efficient: very fast to compute (just a threshold)”; “no vanishing gradient (for $z > 0$)” — constant slope 1 carries the signal deep; sparsity from the zeros	Dying ReLU : “if a neuron’s weights are updated such that its pre-activation z is always negative, it will always output 0... and can never recover”

The ReLU family fixes the dying problem with a small slope on the negative side — the deck’s Leaky ReLU:

$$f(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \leq 0, \end{cases} \quad (\alpha \text{ small, e.g. } 0.01).$$

The deck’s practical guide, verbatim in spirit: hidden layers — “start with ReLU. Most common, fastest, and usually works”; for dying neurons “switch to Leaky ReLU or Parametric ReLU”; “for Transformer-based models, consider GELU or Swish.” The output layer is task-dependent (§41.11): binary classification → sigmoid; multiclass → softmax (§31.12); regression → none (linear/identity).

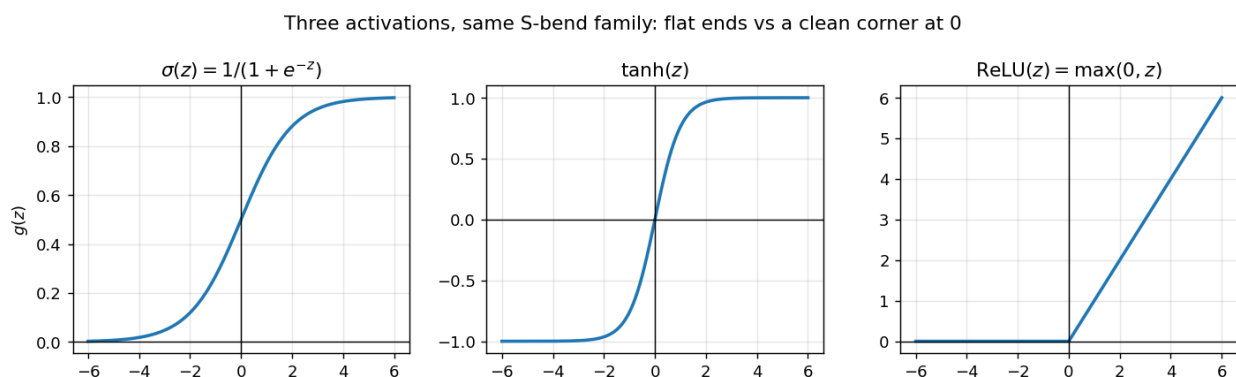


Figure 58: Three activation curves side by side: sigmoid rising 0 to 1 with flat ends, tanh rising -1 to 1 with flat ends, ReLU flat at 0 then a straight line with slope 1.

Basically, ... “Sigmoid and tanh are S-bends — nice for probabilities, but their flat tails choke the gradient in deep stacks. ReLU is a corner: free slope on the right, zero on the left — fast, and the signal survives depth. Its price is neurons that can die (always negative); Leaky ReLU gives them a trickle.”

41.8 XOR: why no linear model can do it

The deck’s proof (and the XOR notebook’s §2.2) — write out what a linear model must do to the four XOR corners and watch it contradict itself:

eg 3 (the proof). XOR: $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 0$. Suppose $f(x_1, x_2) = w_1x_1 + w_2x_2 + b$

fits all four:

$$\begin{aligned} f(0,0) &= b \approx 0, \\ f(0,1) &= w_2 + b \approx 1, \\ f(1,0) &= w_1 + b \approx 1, \\ f(1,1) &= w_1 + w_2 + b \approx 0. \end{aligned}$$

The first gives $b \approx 0$; the second then gives $w_2 \approx 1$; the third gives $w_1 \approx 1$. The fourth then demands $1 + 1 + 0 \approx 0$ — a contradiction. No straight line (hyperplane) separates the corners. The notebook adds the history: “In 1969, Minsky and Papert proved that single-layer perceptrons cannot solve XOR,” and XOR became the “Hello World” of neural networks.

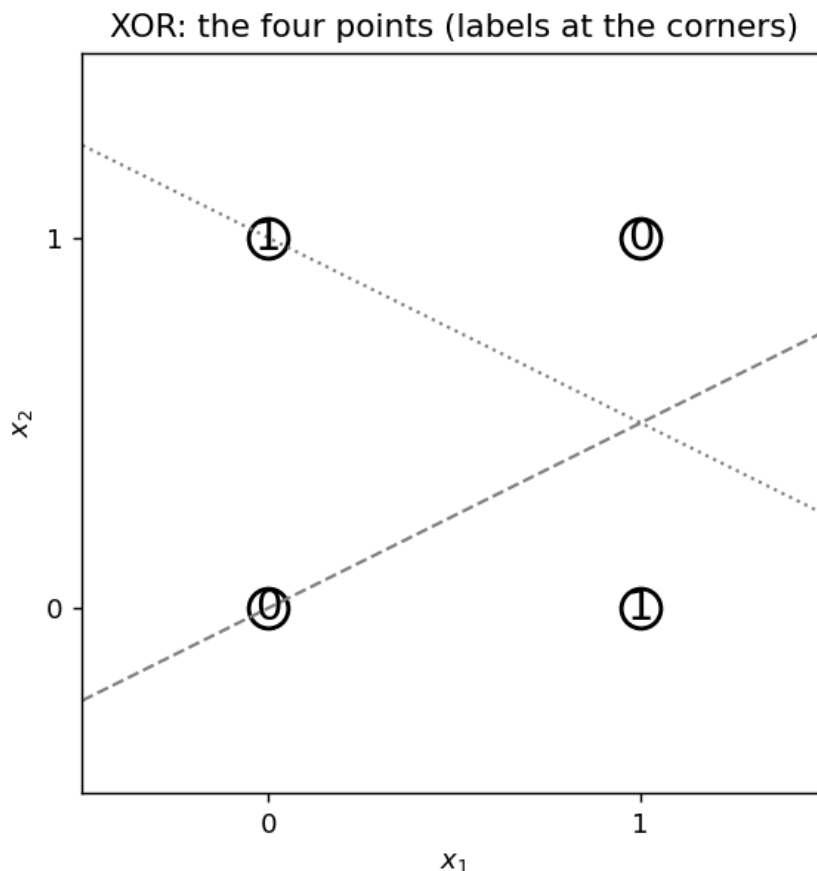


Figure 59: Scatter plot of the four XOR points with labels 0, 0, 1, 1 at the corners of the unit square; two dashed candidate lines each leave opposite-corner same-label points on the wrong side.

Basically, ... “Four corners of a square, opposite corners share a label. Any straight wall you draw puts a mismatched pair on the same side. Four equations, three unknowns — the fourth always contradicts. That is the whole reason hidden layers exist.”

41.9 The 2-2-1 network, worked by hand

The course’s XOR notebook builds a 2-2-1 network — 2 inputs, 2 hidden neurons, 1 output, sigmoid everywhere — and walks one full forward pass by hand for input $(1,0)$ (target $y = 1$, since $\text{XOR}(1,0) = 1$). The setup, from the notebook’s code:

$$W_1 = \begin{bmatrix} 1.0 & -1.0 \\ 1.0 & -1.0 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0.0 \\ 1.0 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 2.0 \\ -1.0 \end{bmatrix}, \quad b_2 = [-1.0].$$

eg 4 (one forward pass, the notebook's numbers, every step recomputed). Input $x_1 = 1$, $x_2 = 0$. Hidden pre-activations:

$$z_{h1} = 1.0(1) + 1.0(0) + 0.0 = 1.0, \quad z_{h2} = (-1.0)(1) + (-1.0)(0) + 1.0 = 0.0.$$

Sigmoid activations ($\sigma(1) = 1/(1 + e^{-1}) = 1/1.36788 \approx 0.7311$):

$$h_1 = \sigma(1.0) \approx 0.7311, \quad h_2 = \sigma(0.0) = 0.5.$$

Output pre-activation and prediction:

$$z_o = 2.0(0.7311) + (-1.0)(0.5) + (-1.0) = 1.4622 - 0.5 - 1.0 = -0.0379,$$

$$\hat{y} = \sigma(-0.0379) = \frac{1}{1 + e^{0.0379}} \approx 0.4905.$$

Binary cross-entropy (§41.11) with $y = 1$:

$$L = -[1 \cdot \log(0.4905) + 0] = 0.7123.$$

The numpy re-run prints `z1 = [1. 0.]`, `h1 = [0.7311 0.5]`, `z3 = -0.0379`, `yhat = 0.4905`, `loss = 0.7123` — the notebook's 0.491 and 0.712 [verified-NumPy].

Basically, ... “This is §41.4's two equations, twice: $X \rightarrow \text{hidden}$ ($Z^1 = XW^1 + b^1$, sigmoid) $\rightarrow$ output ($Z^2 = A^1W^2 + b^2$, sigmoid). The prediction 0.4905 is barely below a coin flip — this untrained net is guessing — and the loss 0.7123 measures exactly how far that guess sits from the true 1.”

41.10 The training loop in numpy: the whole MLP, re-run

The notebook's `XORNeuralNetwork` class is the complete machine: forward (the two equations), binary cross-entropy, backward (one chain-rule step per layer — Chapter 42 derives every line), and gradient-descent weight updates (§10.5). Written here from scratch in the same shape, trained full-batch on all four XOR points, seed fixed (§36.6):

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

class MLP221:
    def __init__(self, seed=42, scale=0.5):
        rng = np.random.default_rng(seed)
        self.W1 = rng.standard_normal((2, 2)) * scale # S0=2 -> S1=2
        self.b1 = rng.standard_normal(2) * scale
        self.W2 = rng.standard_normal((2, 1)) * scale # S1=2 -> S2=1
        self.b2 = rng.standard_normal(1) * scale

    def forward(self, X):
        self.z1 = X @ self.W1 + self.b1 # (4,2) @ (2,2) -> (4,2)
        self.h1 = sigmoid(self.z1)
        self.z2 = self.h1 @ self.W2 + self.b2 # (4,2) @ (2,1) -> (4,1)
        self.out = sigmoid(self.z2).ravel()
        return self.out
```

```

def loss(self, y, yhat):
    yhat = np.clip(yhat, 1e-15, 1 - 1e-15)
    return float(-np.mean(y * np.log(yhat) + (1 - y) * np.log(1 - yhat)))

def backward(self, X, y):
    # sigmoid + BCE: dL/dz2 = out - y
    m = X.shape[0]
    dz2 = (self.out - y).reshape(-1, 1) / m
    dW2 = self.h1.T @ dz2
    db2 = dz2.sum(axis=0)
    dh1 = dz2 @ self.W2.T
    dz1 = dh1 * self.h1 * (1 - self.h1)
    dW1 = X.T @ dz1
    db1 = dz1.sum(axis=0)
    return dW1, db1, dW2, db2

def step(self, grads, lr):
    # theta := theta - lr * grad
    self.W1 -= lr * grads[0]; self.b1 -= lr * grads[1]
    self.W2 -= lr * grads[2]; self.b2 -= lr * grads[3]

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([0,1,1,0], dtype=float)
mlp = MLP221(seed=42)
for epoch in range(1000):
    pred = mlp.forward(X)
    mlp.step(mlp.backward(X, y), lr=5.0)

```

eg 5 (the actual run — every number below is the session’s own output). The trace, epoch 0 / every 100 / last:

epoch	loss	predictions (four corners)	accuracy
0	0.693751	[0.489 0.486 0.493 0.491]	50%
100	0.609766	[0.466 0.723 0.392 0.424]	75%
200	0.030062	[0.029 0.976 0.959 0.024]	100%
300	0.011829	[0.011 0.99 0.984 0.01]	100%
999	0.002181	[0.002 0.998 0.997 0.002]	100%

Final: [0.0021, 0.9981, 0.9971, 0.0018] → binary [0, 1, 1, 0], loss 0.002178 [verified-NumPy]. Three readings: i) **Epoch 0 starts at loss** ≈ 0.693 . Not a coincidence: with untrained weights outputting ≈ 0.5 everywhere, $-\log(0.5) = 0.6931$ — §35.6’s logistic loss at margin 0. ii) **Epoch 100 is honest about the grind**. Accuracy says 75%, but the (1,0) corner predicts 0.392 — still below the 0.5 wall. The network “looks close” while one corner is firmly wrong; the loss (0.6098, barely moved) tells the truer story. iii) **Epoch 200 is the phase change**. Loss collapses to 0.03 and every corner is confident — the hidden layer found the XOR representation. §31.10’s warning applies at the tail: the loss keeps shrinking toward 0 without ever reaching it, because sigmoid only saturates at 0/1 with infinite weights — accuracy is already perfect, the weights keep inflating.

Basically, ... “Forward: the two equations, twice. Loss: how wrong. Backward: how each weight caused it (Chapter 42). Step: nudge every weight downhill (§10.5). Repeat 1000 times and four bent neurons solve what one straight wall never could — the loss trace shows the exact moment it clicks.”

41.11 The output layer + loss pairing

The MLT slides make the output layer *task-dependent*: same forward equations, but the last layer’s activation g and the loss L are chosen as a pair. The deck’s practical guide agrees (sigmoid binary, softmax multiclass, linear regression). Three pairings:

i) Regression — identity output, squared error. Output layer: $S_L = 1$, $g(z) = z$ (the slides: “the activation function in the final layer for regression is just the identity function”), $\hat{y} = A^L$ an $n \times 1$ column. Loss (the slides’ form):

$$L(y, \hat{y}) = \frac{1}{2} (\hat{y} - y)^T (\hat{y} - y),$$

“In Numpy: `L = 0.5 * np.sum((y_hat - y) * (y_hat - y))`” (the slides, verbatim). The $1/2$ is a convenience that cancels the derivative’s 2.

ii) Binary classification — sigmoid output, cross-entropy. $S_L = 1$, $g = \sigma$ (§31.8), loss the NLL (§31.9):

$$L_{\text{nll}}(g, y) = -(y \log g + (1 - y) \log(1 - g)),$$

“penalizes the model for being confident and wrong” (the deck). The worked number from §41.9’s hand calculation: $g = 0.4905$, $y = 1 \rightarrow L = -\log(0.4905) = 0.7123$ [verified-NumPy]. The cleanest fact in the chapter lives here — §31.10’s derivation: for sigmoid + cross-entropy, the gradient w.r.t. the pre-activation is just

$$\frac{\partial L}{\partial z} = g - y \quad (\text{prediction minus truth}),$$

the $g(1 - g)$ from the sigmoid’s derivative canceling the denominator from the loss. This is the `dz2 = (self.out - y)` line in §41.10’s `backward`.

iii) Multiclass classification — softmax output, categorical cross-entropy. Output layer: $S_L = k$ neurons, $g = \text{softmax}$ (§31.12):

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}, \quad \sum_i \hat{y}_i = 1,$$

so $\hat{Y}$ (an $n \times k$ matrix) is a probability distribution over the k classes per row, and inference is $\arg \max_c \hat{y}^c$ (MLT slides). Labels Y are one-hot, $n \times k$. The slides’ matrix form:

$$L(Y, \hat{Y}) = -\mathbf{1}_n^T (Y \odot \log \hat{Y}) \mathbf{1}_k, \quad \text{“In Numpy: } L = -\text{np.sum}(Y * \text{np.log}(Y_hat))\text{”},$$

$\odot$ the element-wise product, $\mathbf{1}_n^T M \mathbf{1}_k$ the sum of all entries. The deck’s worked example (3 classes): true cat $[1, 0, 0]$; good prediction $[0.8, 0.1, 0.1] \rightarrow$ low loss; bad prediction $[0.1, 0.2, 0.7] \rightarrow$ high loss. This is §35.13’s landing point: “the MLT Week-12 slides close with neural networks trained by **cross-entropy loss** for classification” — the hand-annotated slide’s diagram shows a network’s sigmoid feeding “ $p(y | x)$ ” into a “Cross-Entropy Loss” box — “the logistic loss of §35.6, scaled up to deep networks.”

Basically, ... “The hidden layers are the model; the output layer is the task. Counting things: identity activation + squared error. Yes/no: sigmoid + cross-entropy. Pick-one-of- k : softmax + cross-entropy over the one-hot labels. And the pairings are not arbitrary — sigmoid’s derivative cancels cross-entropy’s, leaving prediction-minus-truth.”

41.12 The contract survives: sklearn’s MLPClassifier / MLPRegressor

The Week-12 practice-solution notebook runs both, and both honor the estimator contract of Chapters 37–38 (as §37.10(iii) and §38.11(iii) promised):

eg 6 (classification, diabetes dataset, Outcome target). Pipeline: StandardScaler + MLPClassifier(hidden_layer_sizes=(10,10,10), activation='relu', solver='sgd', alpha=1e-4, learning_rate_init=0.2, max_iter=500, random_state=1); 80:20 split, random_state=1:

```
pipe.fit(X_train, y_train)      # learn the weights
pipe.score(X_train, y_train)    # -> 0.7915309446254072 [recorded]
pipe.score(X_test, y_test)     # -> 0.7662337662337663 [recorded]
```

hidden_layer_sizes=(10,10,10) is three hidden layers of ten neurons each — an $S_0-10-10-10-S_3$ network in §41.4's notation; activation='relu' is the hidden-layer g ; solver='sgd' the engine; alpha the L2 penalty (Chapter 28's λ); max_iter=500 caps the epochs.

eg 7 (regression). Pipeline: StandardScaler + MLPRegressor(hidden_layer_sizes=(50,50,50), tol=1e-2, alpha=1e-4, solver='adam', learning_rate_init=0.1, max_iter=50, random_state=1):

```
pipe.score(X_train, y_train)    # -> 0.999992602958679 [recorded] (R^2)
pipe.score(X_test, y_test)     # -> 0.9999920781245708 [recorded] (R^2)
```

The Week-12 note on this estimator: “trains using backpropagation... uses the square error as the loss function” (§37.10(iii)) — pairing (i) of §41.11, inside the same fit/predict/score contract. Neither snippet ran here (sklearn is not installed) — the numbers are the notebook's printed outputs.

Basically, ... “sklearn's neural nets are just §41.10's class with the same three methods every estimator has had since Chapter 37: fit learns the weights, predict runs the forward pass, score reports the verdict. The inside is neurons; the contract is unchanged.”

41.13 PyTorch: tensors in, the same loop out

The course's two PyTorch notebooks do one job each: *Fundamentals* — tensors as the data structure; *Workflow* — the training loop as the engine. Both transcribe here [recorded]; torch is not installed on this machine.

Tensors (Fundamentals notebook): “the fundamental data structure” — vectors, matrices, and higher-rank arrays, all as one type. The working vocabulary, all transcribed from the notebook: - creation: torch.tensor(...), torch.zeros(), torch.ones(), torch.rand(), torch.arange(); default dtype float32 (a method like .mean() on an integer tensor errors — the notebook hits this); - the central distinction: * is element-wise, torch.matmul / @ is matrix multiplication — the notebook's line: “the difference between element-wise multiplication and matrix multiplication is the addition of values” (§2.10's rule, in code); - shape tools: reshape, view (shares the underlying data — changing the view changes the original), squeeze/unsqueeze, permute (also a view), indexing x[:, 0], x.argmax(); - NumPy interop: torch.from_numpy(array) (keeps the float64 dtype), and .numpy() back — the §36.1 world and the tensor world sharing memory.

The MLP as an nn.Module (Fashion-MNIST notebook — a 784→128→64→10 classifier, the two-core-equations of §41.4 as objects):

```
class FashionMLP(nn.Module):                                     # [recorded]
    """Architecture: 784 -> 128 -> 64 -> 10
    - Input: Flattened 28x28 images (784 features)
    - Hidden layers: ReLU activation for non-linearity
    - Output: 10 class probabilities (softmax applied in loss function)"""
    def __init__(self, input_size=784, hidden1_size=128,
                  hidden2_size=64, num_classes=10):
        super(FashionMLP, self).__init__()
        self.flatten = nn.Flatten()                             # (B,1,28,28) -> (B,784)
        self.hidden1 = nn.Linear(input_size, hidden1_size)      # 784 -> 128
        self.hidden2 = nn.Linear(hidden1_size, hidden2_size)     # 128 -> 64
```

```

self.output = nn.Linear(hidden2_size, num_classes) # 64 -> 10
self.relu = nn.ReLU()
self.dropout = nn.Dropout(p=0.2)

def forward(self, x):
    x = self.flatten(x)
    x = self.hidden1(x); x = self.relu(x); x = self.dropout(x)
    x = self.hidden2(x); x = self.relu(x); x = self.dropout(x)
    return self.output(x)

```

`nn.Linear` is one row of §41.4: $Z = AW + b$ with W, b as learnable parameters. The training configuration the notebook chooses: `nn.CrossEntropyLoss()` (pairing (iii) of §41.11 — “combines” softmax with the loss), Adam at `lr=0.001` with `weight_decay=1e-4` (L2, Chapter 28), 10 epochs.

The five-step training loop (Workflow notebook, transcribed verbatim):

```

for epoch in range(epochs):
    model_0.train()
    y_pred = model_0(X_train)
    loss = loss_fn(y_pred, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

```

Each step maps onto this chapter’s machine: 1 = §41.4’s two equations; 2 = §41.11’s loss; 3–4 = the chain rule (Chapter 42 — §41.10’s **backward**, automated); 5 = §10.5’s update (the perceptron’s era ended, gradient descent is the engine now). The workflow notebook also keeps §40.4’s discipline: train and test phases are *separate* — “assesses model performance on test data without parameter updates... prevents information leakage” — and ends with `state_dict()` save/load, the learned weights as a portable artifact.

Basically, ... “PyTorch changes the notation, not the machine. A tensor is an ndarray that knows about gradients (§36.1’s world, extended). `nn.Linear` is §41.4’s layer as an object. The training loop is §41.10’s loop with the backward step automated. If you understand the numpy version, the PyTorch version is just the same five steps wearing nicer syntax.”

41.14 Four traps worth knowing (the course’s own warnings)

i) Zero-init symmetry. “Why did we not initialize all parameters to zero?” (MLT slides). Because “since the incoming weights are the same for all neurons in a layer, there is nothing to differentiate between two neurons. This symmetry means that they will evolve identically and will not learn different things.” So: “setting all weights to a constant value, especially 0, is a bad idea for neural networks” — the slides sample $\theta \sim \mathcal{N}(0, 1)$ (the MLP class in §41.10 seeds $\mathcal{N}(0, 0.5^2)$ for the same reason; §36.6 on seeding).

ii) Parameter count → overfitting. The MLT slides’ MNIST example, $784 \rightarrow 50 \rightarrow 10$: weights $784 \cdot 50 = 39,200$ plus 50 biases plus $50 \cdot 10 = 500$ plus 10 — “number of parameters = $(39200 + 50) + (500 + 10) = 39,760$ ” (the slides’ arithmetic, rechecked). “With just 1 hidden layer, we have a model with nearly 40,000 parameters... Neural networks are thus prone to overfitting.” The slides’ two fixes: L1/L2 regularization and **dropout** — “in each iteration of GD, randomly choose half the neurons in the hidden layer and ‘drop’ them ‘out’ of the network... at inference, use the full network, but halve the outgoing weights.” The slides’ distinction: “L1/L2 regularization: modifies the loss function. Dropout: modifies the network.”

iii) The loss is not the 0/1 you care about. Chapter 35’s trade (§35.9) rides along: cross-entropy and squared error are *surrogates* — smooth stand-ins optimized so that the true objective (accuracy, the 0/1 loss of §35.1) comes along for the ride.

iv) Local minima, and why nobody panics. The hand-annotated Week-12 slide’s bottom line, quoted

whole: the network “converges to **local minima**” — but “typically works very well in practice, especially for unstructured data.” §10.9’s promises stand: gradient descent walks downhill; nobody guarantees the bottom of the whole landscape.

41.15 Where this goes next

- i) **Backpropagation, end to end (Chapter 42).** The one piece this chapter borrowed without deriving: §41.10’s **backward** — how $\partial L/\partial z$ flows from the output layer back through every weight. The course’s XOR notebook hand-computes the full backward pass for the same 2-2-1 net (§41.9’s numbers: $\partial L/\partial z_3 = -0.509$, hidden deltas -0.200 and 0.127 , one weight update at $\alpha = 0.1$); the GenAI deck derives it generally on a computational graph (“any mathematical expression can be decomposed into a graph of basic operations”). Chapter 42 is both, fully worked.
- ii) **Initialization and optimization (Chapter 43).** §41.14(i)’s symmetry trap is the entry point: *how* to sample the initial weights, why scale matters for deep stacks, and the optimizers (SGD variants, Adam) the notebooks already name.
- iii) **The Part VI arc.** CNNs, RNNs, attention (Chapters 44–46) keep §41.4’s two equations and change what a “layer” computes; the debugging discipline of Chapter 40 rides along whole — “debugging a neural net starts with the same playbook: shapes first, then scale, then shuffle, then the leak audit” (§40.5(i)).

Problem set

1. **Neuron arithmetic.** A neuron takes $a = (2, -1, 0.5)$, $w = (0.4, 0.8, -0.6)$, $b = 0.1$. (i) Compute the pre-activation z . (ii) Compute the post-activation for $g = \sigma$ (to 3 s.f.) and for $g = \text{ReLU}$. (iii) Which activation would the deck’s §41.7 guide recommend if this neuron lived in a hidden layer, and why in one line?
2. **Layer shapes.** An MLP has layer sizes $S = [4, 6, 6, 3]$ (input $m = 4$, two hidden layers, $k = 3$ output) and a batch of $n = 32$ points. (i) Give the shape of each W^l and b^l in the batch convention ($Z^l = A^{l-1}W^l + b^l$). (ii) Count the total parameters (weights + biases); recheck against the MLT slides’ 784–50–10 arithmetic (39,760). (iii) Write the shape chain for Z^2 and A^2 explicitly.
3. **The transpose convention.** The deck’s per-example slide gives $x = [0.5, 1.0]^T$, $W = \begin{bmatrix} 0.2 & 0.7 \\ -0.4 & 0.1 \\ 0.9 & -0.3 \end{bmatrix}$, and computes $z = Wx + b$. (i) Explain in one line why batch code uses $Z = X @ W.T + b$ instead of $X @ W + b$ for this W . (ii) For the batch form, give W ’s shape and the row of Z corresponding to $x = (0.5, 1.0)$. (iii) Name the debugging chapter-section this convention clash would violate if read backwards.
4. **XOR proof, re-derived.** (i) Reproduce §41.8’s contradiction for a linear model $f(x_1, x_2) = w_1x_1 + w_2x_2 + b$ on the four XOR corners. (ii) In two lines, explain why this proof also kills any *stacked* linear network (use §41.6’s collapse). (iii) Give one sentence saying what the hidden layer must do so the output layer can succeed (the deck’s “automatic feature engineer” line).
5. **A new ReLU layer.** $x = (1, -2)$, $W = \begin{bmatrix} 0.5 & -0.5 \\ 1.0 & 2.0 \\ -1.0 & 0.25 \end{bmatrix}$, $b = (0, 0.5, -1)$. (i) Compute $z = Wx + b$ by hand. (ii) Apply ReLU element-wise. (iii) Which neurons fired, which died, and what does the dying one contribute to the next layer?
6. **Loss pairing.** (i) Binary case: prediction $g = 0.2$, true $y = 1$ — compute the cross-entropy loss; then compute it again for $g = 0.8$ and compare. (ii) Multiclass case: $Y = [0, 1, 0]$, $\hat{Y} = [0.2, 0.7, 0.1]$ — compute the categorical cross-entropy. (iii) Regression case: $y = (1, 2, 3)$, $\hat{y} = (1.5, 2.5, 2)$ — compute the slides’ $L = \frac{1}{2}(\hat{y} - y)^T(\hat{y} - y)$.
7. **The $(g - y)$ simplification.** (i) For sigmoid output + cross-entropy, verify $\partial L/\partial z = g - y$ numerically at $g = 0.4905$, $y = 1$ using §31.10’s two-step chain ($\partial L/\partial g$ then dg/dz). (ii) In the §41.10 class, point

- at the exact line this simplification produces. (iii) In one line, say what Chapter 42 must derive when the hidden layer's g is *not* paired with its loss.
8. **Map the two loops.** (i) Match each of the PyTorch workflow's five steps (forward, loss, `zero_grad`, backward, step) to a method or line in §41.10's numpy class. (ii) Which step does the notebook's version do *by hand* that PyTorch automates? (iii) The workflow notebook runs the test phase without parameter updates — name the §40.4 suspect this discipline prevents.
 9. **Zero-init symmetry.** Two hidden neurons start with *identical* incoming weights and bias. (i) In two lines, show their pre-activations stay equal for the same input, so their outputs and gradient updates stay equal forever. (ii) State the fix the MLT slides prescribe (distribution and one-line reason). (iii) §41.10's class uses `rng.standard_normal((2,2)) * 0.5` — name the two choices (distribution, seed) and what each buys.
 10. **The sklearn face.** (i) Read eg 6's constructor: what do `hidden_layer_sizes`, `activation`, `solver`, `alpha`, `max_iter` each mean, in §41.4/§41.7/Chapter 28 terms? (ii) eg 7's `score` returns R^2 — which §41.11 pairing does this estimator use, and how do you know from the Week-12 note? (iii) In one line each: what do `fit`, `predict`, and `score` compute for `MLPClassifier`?

Chapter 41 solutions — The artificial neuron and MLPs from scratch

1. **Neuron arithmetic.** (i) $z = 0.4(2) + 0.8(-1) + (-0.6)(0.5) + 0.1 = 0.8 - 0.8 - 0.3 + 0.1 = -0.2$. (ii) $\sigma(-0.2) = 1/(1 + e^{0.2}) = 1/2.2214 \approx 0.450$; $\text{ReLU}(-0.2) = 0$. (iii) ReLU — the deck's guide: "start with ReLU. Most common, fastest, and usually works"; sigmoid in a hidden layer would invite vanishing gradients for no probabilistic payoff.
2. **Layer shapes.** (i) $W^1 : 4 \times 6$, $b^1 : (6,)$; $W^2 : 6 \times 6$, $b^2 : (6,)$; $W^3 : 6 \times 3$, $b^3 : (3,)$. (ii) $4 \cdot 6 + 6 = 30$, $6 \cdot 6 + 6 = 42$, $6 \cdot 3 + 3 = 21$; total 93. The slides' check: $784 \cdot 50 + 50 = 39,250$, $50 \cdot 10 + 10 = 510$, total 39,760. (iii) $Z^2 = A^1 W^2 + b^2$: $(32 \times 6)(6 \times 6) \rightarrow (32 \times 6)$, bias broadcast down the rows; $A^2 = g(Z^2)$ is (32×6) .
3. **The transpose convention.** (i) The deck's W is 3×2 because it left-multiplies a 2×1 column vector; a batch row is 1×2 and right-multiplies, so it needs the 2×3 layout — W^T . (ii) The batch W is 2×3 (W_{deck}^T); the row for $x = (0.5, 1.0)$ is $z = [0.9, 0.1, -0.35]$ (§41.5's eg 1). (iii) §36.1's Note — the (d, n) -vs- (n, d) convention clash, "the single most common shape bug in ML code."
4. **XOR proof, re-derived.** (i) §41.8's eg 3: $b \approx 0$, $w_2 \approx 1$, $w_1 \approx 1$ force $f(1, 1) \approx 2 \neq 0$ — contradiction. (ii) A stacked linear network is still one linear function, $W'x + b'$ (§41.6), so the same four equations give the same contradiction. (iii) The hidden layer must transform the data into a representation in which the corners are linearly separable — "an automatic feature engineer."
5. **A new ReLU layer.** (i) $z_1 = 0.5(1) + (-0.5)(-2) + 0 = 1.5$; $z_2 = 1.0(1) + 2.0(-2) + 0.5 = -2.5$; $z_3 = -1.0(1) + 0.25(-2) - 1 = -2.5$. (ii) $a = [1.5, 0, 0]$. (iii) Only neuron 1 fired; 2 and 3 died. A dead neuron contributes 0 times its outgoing weights to the next layer — nothing. That is ReLU's sparsity working as intended at one neuron, and the dying-ReLU pathology if the death is permanent (§41.7).
6. **Loss pairing.** (i) $g = 0.2$: $L = -\log 0.2 = 1.6094$; $g = 0.8$: $L = -\log 0.8 = 0.2231$. The wrong-confident guess costs $7.2\times$ the right-confident one — cross-entropy "penalizes the model for being confident and wrong." (ii) $L = -\log 0.7 = 0.3567$ — only the true class's probability enters, via the one-hot product. (iii) $\hat{y} - y = (0.5, 0.5, -1)$, squared length $0.25 + 0.25 + 1 = 1.5$, $L = 1.5/2 = 0.75$.
7. **The $(g - y)$ simplification.** (i) $\partial L / \partial g = (g - y) / (g(1 - g)) = (-0.5095) / (0.24991) = -2.0387$; $dg/dz = g(1 - g) = 0.24991$; product $-0.5095 = g - y$ — §31.10's cancellation, numerically confirmed. (ii) `dz2 = (self.out - y).reshape(-1, 1) / m` in `MLP221.backward`. (iii) Chapter 42 must derive the general backward pass — the error deltas δ^l for every layer and how they chain through each activation — of which this is the output-layer special case.
8. **Map the two loops.** (i) forward $\rightarrow$ `MLP221.forward` (the two equations, both layers); loss $\rightarrow$ `MLP221.loss` (binary cross-entropy); `zero_grad` $\rightarrow$ no analogue — the numpy code computes fresh gradients each call, nothing accumulates; backward $\rightarrow$ `MLP221.backward` (the hand-derived chain rule); step $\rightarrow$ `MLP221.step` ($\theta := \theta - \eta \nabla J$). (ii) The backward step: PyTorch's autograd computes the chain rule

automatically; the notebook and §41.10 derive it by hand. (iii) §40.4’s suspect (iii) — the test set spent more than once: updates on test data would fit the model to the evaluation set, breaking §39.6’s spend-once doctrine.

9. Zero-init symmetry. (i) Identical weights and bias, same input $\rightarrow$ identical pre-activation z , identical activation $g(z)$. The gradients are the same deterministic functions of these identical quantities, so the updates are identical — the two neurons evolve identically, forever. (ii) Sample $\theta \sim \mathcal{N}(0, 1)$ (the MLT slides): random weights break the symmetry, so neurons learn different features. (iii) `default_rng(42)` is the seeded, reproducible RNG (§36.6); `standard_normal * 0.5` is the notebook’s Gaussian-with-small-scale choice — randomness for symmetry-breaking, small so sigmoid starts in its linear-ish region, not saturated.

10. The sklearn face. (i) `hidden_layer_sizes=(10,10,10)`: three hidden layers, 10 neurons each ($S_1 = S_2 = S_3 = 10$). `activation='relu'`: the hidden-layer g (§41.7). `solver='sgd'`: stochastic-gradient-descent engine. `alpha=1e-4`: the L2 penalty (Chapter 28’s λ). `max_iter=500`: the epoch cap. (ii) Pairing (i) of §41.11 — identity output + squared error: the Week-12 note says `MLPRegressor` “uses the square error as the loss function,” and its `score` is R^2 , the regression score (§37.5). (iii) `fit` learns all W^l, b^l from the training set (backprop + SGD); `predict` runs the forward pass and returns labels (threshold at 0.5 / argmax); `score` returns accuracy on the supplied set.

Chapter 42: Backpropagation, worked end-to-end

Part VI continues. This chapter derives the one piece Chapter 41 borrowed without proof: §41.10’s **backward** — how $\partial L/\partial W$ and $\partial L/\partial b$ for *every* weight and bias are computed by flowing one error signal backwards through the network. The shape follows the GenAI Weeks 1–2 deck (“Introduction to ANN,” Balaji Srinivasan and Ganapathy Krishnamurthi, slides 59–70) — the general derivation on a computational graph (“any mathematical expression can be decomposed into a graph of basic operations”) — and the course’s XOR notebook (cells 17–18), which hand-computes the full backward pass for the same 2-2-1 net of §41.9. Cross-checked against the MLT Week-12 ANN slides, which name backprop as the trainer but add no new derivation. One idea, fully worked: **backpropagation is the multivariate chain rule (§9.7) applied to the forward computation graph — one backward sweep yields every parameter’s gradient**. Every number below was re-run here in numpy and is marked [verified-NumPy]; deck and notebook numbers are transcriptions, marked [recorded].

Notation. The deck derives in per-example (column-vector) form, $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$; this book computes in batch form, $Z^l = A^{l-1}W^l + b^l$ (§41.4’s Note — the deck’s $W^{[l]}$ is the book’s $(W^l)^T$). Both forms appear below, pinned to each other at each step; mix them up and you get the transpose bug.

42.1 The question backprop answers

Gradient descent (§10.5, §10.7) needs one number per parameter:

$$\theta := \theta - \eta \frac{\partial L}{\partial \theta} \quad \text{for every weight and bias.}$$

The forward pass gives L ; the update needs the *gradient* of L — how much each weight *caused* the loss. With 39,760 parameters (§41.14(ii)), you cannot afford to guess each one separately.

Def (credit assignment). Determining, for every weight and bias, how much of the final loss it caused — the sign and size of $\partial L/\partial \theta$ for each parameter. The deck poses it as the chain-rule question: “How does a small change in an early parameter (w_1) affect the final loss (L) at the end of a long chain of computations?” — and answers: “The Chain Rule provides the answer by multiplying local derivatives.”

Basically, ... “The network made an error — now who is to blame? Backprop is the blame-assigner: it walks the error signal backwards from the output, asking each layer ‘how much of this loss passed through you?’, until every weight knows its share of the blame. The tool is the multivariate chain rule (§9.7) — the machinery Chapter 10 assumed.”

42.2 The computational graph: any expression is a graph of basic ops

The deck’s general setup (slides 62–63), in its own words: - “Any mathematical expression can be decomposed into a graph of basic operations. This makes the flow of derivatives explicit.” - Two passes of automatic

differentiation: i) **Forward pass**: “We start with input values (a, b, c) and compute the values of all nodes, flowing from left to right, until we get the final output L .” Every intermediate value is *cached*. ii) **Backward pass (backpropagation)**: “We start from the end with the gradient $\partial L / \partial L = 1$. We then go backwards (right to left), using the chain rule at each node to compute the gradient of L with respect to every input of that node.” - The slide’s rule of thumb: each node’s incoming gradient is multiplied by the node’s *local* gradient (how the node’s output changes w.r.t. each input).

eg 1 (the deck’s scalar example, slides 64–65). $\mathcal{L} = (a + b) \cdot c$, inputs $a = 2$, $b = 3$, $c = 6$. Break it down: $d = a + b$, $\mathcal{L} = d \cdot c$.

Forward (green):

$$d = a + b = 2 + 3 = 5, \quad \mathcal{L} = d \cdot c = 5 \cdot 6 = 30.$$

Backward (red), starting $\partial \mathcal{L} / \partial \mathcal{L} = 1$:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial c} &= \frac{\partial \mathcal{L}}{\partial \mathcal{L}} \cdot \frac{\partial \mathcal{L}}{\partial c} = 1 \cdot d = 5, \\ \frac{\partial \mathcal{L}}{\partial d} &= \frac{\partial \mathcal{L}}{\partial \mathcal{L}} \cdot \frac{\partial \mathcal{L}}{\partial d} = 1 \cdot c = 6, \\ \frac{\partial \mathcal{L}}{\partial b} &= \frac{\partial \mathcal{L}}{\partial d} \cdot \frac{\partial d}{\partial b} = 6 \cdot 1 = 6, \\ \frac{\partial \mathcal{L}}{\partial a} &= \frac{\partial \mathcal{L}}{\partial d} \cdot \frac{\partial d}{\partial a} = 6 \cdot 1 = 6. \end{aligned}$$

The slide’s summary table (“Spreadsheet Visualization”): forward values $a = 2$, $b = 3$, $c = 6$, $d = 5$, $\mathcal{L} = 30$; local gradients $\partial d / \partial a = \partial d / \partial b = 1$, $\partial \mathcal{L} / \partial d = c = 6$, $\partial \mathcal{L} / \partial c = d = 5$ [recorded].

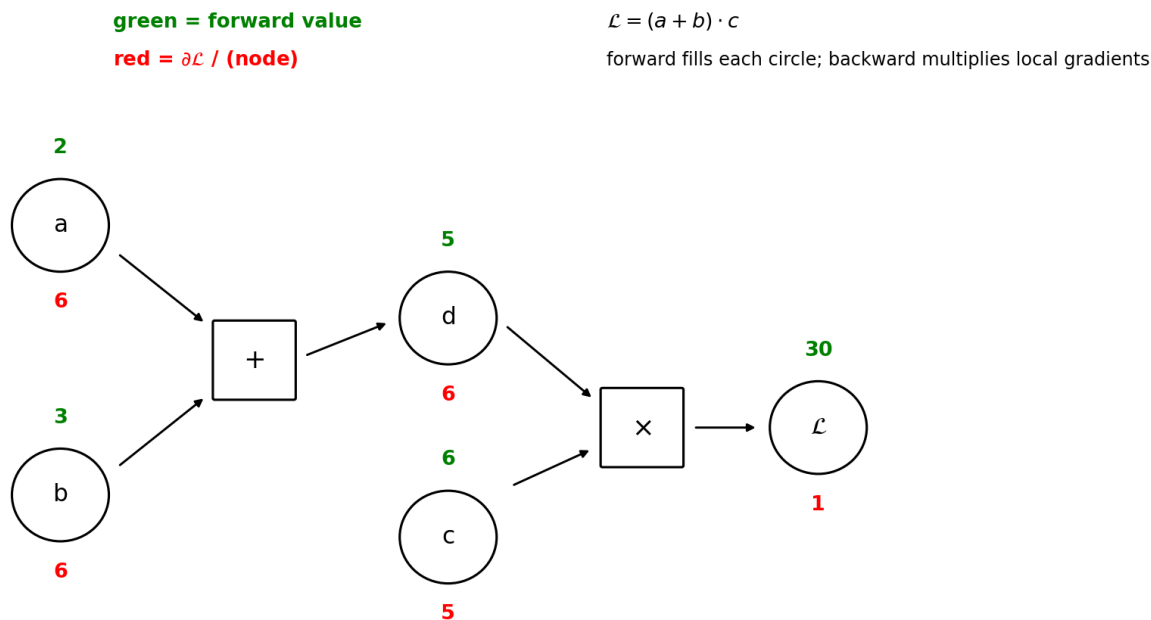


Figure 60: Computational graph for $L = (a+b) \cdot c$: variable circles a, b, c, d, L with values 2,3,6,5,30 in green; gradient values 6,6,5,6,1 in red below each circle; square boxes for $+$ and $\times$ operations.

Note (deck typo, fixed here). The slide’s forward-pass bullet reads “ $\mathcal{L} = d \cdot c = 5 \cdot 4 = 30$.” Since $c = 6$, this should be $5 \cdot 6 = 30$ (and $5 \cdot 4 \neq 30$ anyway). The numbers used above and in the figure are the corrected ones; the derivation and all gradients are unaffected.

Basically, ... “Write any formula as a chain of tiny steps. Forward: fill in each step’s value, left to right, and remember them. Backward: start at the end with ‘the loss changes 1-per-1 with itself’ and walk right to left, multiplying each step’s *local* slope. Every node’s gradient is just (gradient from downstream) $\times$ (local slope) — four multiplications and all four gradients are done.”

42.3 The 2-2-1 backward pass, every number by hand

The notebook’s §4.3 (cell 17) runs the same two-pass idea on the 2-2-1 XOR net, input $(1, 0)$, target $y = 1$. The forward values it inherits from §41.9 (recomputed here [verified-NumPy]):

$$z_1 = [1.0, 0.0], \quad h_1 = [0.73105858, 0.5], \quad z_3 = -0.0379, \quad \hat{y} = 0.4905, \quad L = 0.7123.$$

The notebook’s weight names map to the batch layout ($W_1[i, j]$ = weight from input i to hidden neuron j):

hand-calc name	book matrix entry	value
w_{11}, w_{12}	$W_1[0, 0], W_1[0, 1]$ (from x_1 to h_1, h_2)	1.0, −1.0
w_{21}, w_{22}	$W_1[1, 0], W_1[1, 1]$ (from x_2 to h_1, h_2)	1.0, −1.0
b_1, b_2	$b_1[0], b_1[1]$	0.0, 1.0
w_{31}, w_{32}	$W_2[0, 0], W_2[1, 0]$ (from h_1, h_2 to output)	2.0, −1.0
b_3	$b_2[0]$	−1.0

eg 2 (the notebook’s backward pass, re-derived with exact arithmetic).

Step 1 — output layer. Loss $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = -\frac{1}{0.4905} = -2.0387,$$

$$\frac{\partial \hat{y}}{\partial z_3} = \hat{y}(1 - \hat{y}) = 0.4905 \times 0.5095 = 0.2499,$$

$$\frac{\partial L}{\partial z_3} = (-2.0387)(0.2499) = -0.5095.$$

The notebook prints -2.037 for $\partial L / \partial \hat{y}$ — a rounding artifact of writing $\hat{y}$ as 0.491 first; the exact value is $-1/0.4905 = -2.0387$. The simplification, exact to all shown digits:

$$\boxed{\frac{\partial L}{\partial z_3} = \hat{y} - y = 0.4905 - 1 = -0.5095}$$

(the sigmoid’s $\hat{y}(1 - \hat{y})$ cancels the loss’s denominator — §41.11(ii), §31.10). Output-layer parameter gradients (local gradient = the incoming activation):

$$\frac{\partial L}{\partial w_{31}} = (-0.5095)(0.7311) = -0.3725, \quad \frac{\partial L}{\partial w_{32}} = (-0.5095)(0.5) = -0.2547, \quad \frac{\partial L}{\partial b_3} = -0.5095.$$

Step 2 — hidden layer. Push the error back through W_2 (local gradient = the weight on each path):

$$\frac{\partial L}{\partial h_1} = (-0.5095)(2.0) = -1.0189, \quad \frac{\partial L}{\partial h_2} = (-0.5095)(-1.0) = 0.5095.$$

Through the sigmoid (local gradient $h(1-h)$):

$$\frac{\partial h_1}{\partial z_1} = 0.7311 \times 0.2689 = 0.1966, \quad \frac{\partial h_2}{\partial z_2} = 0.5 \times 0.5 = 0.25.$$

The hidden deltas:

$$\frac{\partial L}{\partial z_1} = (-1.0189)(0.1966) = -0.2003, \quad \frac{\partial L}{\partial z_2} = (0.5095)(0.25) = 0.1274.$$

Hidden-layer parameter gradients (local gradient = the input on each path; $x_2 = 0$):

$$\begin{aligned} \frac{\partial L}{\partial w_{11}} &= (-0.2003)(1) = -0.2003, & \frac{\partial L}{\partial w_{12}} &= (0.1274)(1) = 0.1274, \\ \frac{\partial L}{\partial w_{21}} &= (-0.2003)(0) = 0, & \frac{\partial L}{\partial w_{22}} &= (0.1274)(0) = 0, \\ \frac{\partial L}{\partial b_1} &= -0.2003, & \frac{\partial L}{\partial b_2} &= 0.1274. \end{aligned}$$

Every rounded value matches the notebook's hand calc $(-0.509, -0.200, 0.127)$ [verified-NumPy].

Step 3 — one gradient-descent update at $\alpha = 0.1$ (§10.5: $\theta := \theta - \alpha \partial L / \partial \theta$):

$$\begin{aligned} w_{31} &= 2.0 - 0.1(-0.3725) = 2.0372, & w_{32} &= -1.0 - 0.1(-0.2547) = -0.9745, \\ b_3 &= -1.0 - 0.1(-0.5095) = -0.9491, \\ w_{11} &= 1.0 - 0.1(-0.2003) = 1.0200, & w_{12} &= -1.0 - 0.1(0.1274) = -1.0127, \\ w_{21} &= 1.0 - 0.1(0) = 1.0, & w_{22} &= -1.0 - 0.1(0) = -1.0, \\ b_1 &= 0.0 - 0.1(-0.2003) = 0.0200, & b_2 &= 1.0 - 0.1(0.1274) = 0.9873. \end{aligned}$$

Note. w_{21} and w_{22} do not move: their input $x_2 = 0$, so the chain dies at the local gradient $\partial z / \partial w = x_2 = 0$. This is the credit-assignment logic made visible — a weight whose input is silent gets no blame, because changing it would not have changed z (and the notebook's cell 18 draws the same conclusion: “Zero gradients $\rightarrow$ no change needed”).

Basically, ... “The error at the output is -0.509 (prediction below truth). It flows back: multiplied by the outgoing weights it becomes hidden errors -1.0189 and 0.5095 ; multiplied by each sigmoid's local slope it becomes hidden deltas -0.2003 and 0.1274 ; multiplied by the inputs it becomes each weight's gradient. Every step is (upstream error) $\times$ (local slope) — the §42.2 rule, nine times. Then subtract α times each gradient.”

42.4 The general layer-wise rules (the deck's derivation)

The deck (slides 66–70) derives the update rules for layer l in per-example form. It decomposes each weight's gradient into three components:

$$\underbrace{\frac{\partial L}{\partial W^{[l]}}}_{\text{our goal}} = \underbrace{\frac{\partial L}{\partial a^{[l]}}}_{\text{1. upstream}} \cdot \underbrace{\frac{\partial a^{[l]}}{\partial z^{[l]}}}_{\text{2. activation}} \cdot \underbrace{\frac{\partial z^{[l]}}{\partial W^{[l]}}}_{\text{3. local}}$$

- i) **Upstream gradient** $\partial L / \partial a^{[l]}$: “How the final loss changes with this layer's output. This is passed back from layer $l+1$.”
- ii) **Activation gradient** $\partial a^{[l]} / \partial z^{[l]} = g'(z^{[l]})$: “The derivative of the activation function.”
- iii) **Local gradient** $\partial z^{[l]} / \partial W^{[l]}$: “How this layer's pre-activation changes with its weights.”

Step 1 — the error term δ . The deck defines

$$\delta^{[l]} = \frac{\partial L}{\partial z^{[l]}},$$

“which represents how the loss changes with respect to the pre-activation of layer l .” Combining (i) and (ii):

$$\delta^{[l]} = \frac{\partial L}{\partial a^{[l]}} \odot g'(z^{[l]}), \quad \text{where } \frac{\partial L}{\partial a^{[l]}} = (W^{[l+1]})^T \delta^{[l+1]}.$$

The core recursion (the slide’s boxed line):

$$\delta^{[l]} = ((W^{[l+1]})^T \delta^{[l+1]}) \odot g'(z^{[l]}).$$

“For the output layer L , this is simpler: $\delta^{[L]} = \nabla_{a^{[L]}} L \odot g'(z^{[L]})$.”

Step 2 — the parameter gradients. “Once we have the error $\delta^{[l]}$ for a layer, finding the gradients for its parameters is straightforward”:

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T, \quad \frac{\partial L}{\partial b^{[l]}} = \delta^{[l]}.$$

The same rules in this book’s batch form ($Z^l = A^{l-1}W^l + b^l$, $A^l = g(Z^l)$, mean loss $L = \frac{1}{m} \sum_i L_i$). Write Δ^l for the batch error signal, one row per example: $\Delta_i^l = \frac{1}{m} \partial L_i / \partial z_i^l$. Then:

$$\Delta^l = (\Delta^{l+1} (W^{l+1})^T) \odot g'(Z^l), \quad \frac{\partial L}{\partial W^l} = (A^{l-1})^T \Delta^l, \quad \frac{\partial L}{\partial b^l} = \sum_{\text{rows}} \Delta^l.$$

The shapes: Δ^{l+1} is $n \times S_{l+1}$, $(W^{l+1})^T$ is $S_{l+1} \times S_l$ — the product fans the error back onto the S_l neurons of layer l (§2.10’s rule: the transpose is doing the “push the error backwards through the weights” job); $(A^{l-1})^T \Delta^l$ is $S_{l-1} \times S_l$ — the shape of W^l (§41.4).

Line-by-line map to §41.10’s backward. The numpy class is the batch-form rules above, written out:

backward line	the rule it is
<code>dz2 = (self.out - y).reshape(-1, 1) / m</code>	Δ^2 : output-layer error, §42.5’s $(\hat{y} - y)/m$ pairing
<code>dW2 = self.h1.T @ dz2</code>	$\partial L / \partial W^2 = (A^1)^T \Delta^2$
<code>db2 = dz2.sum(axis=0)</code>	$\partial L / \partial b^2$: row sum of Δ^2 (= row mean of per-example δ ’s, since $dz2$ already carries $1/m$)
<code>dh1 = dz2 @ self.W2.T</code>	upstream: $\partial L / \partial A^1 = \Delta^2 (W^2)^T$
<code>dz1 = dh1 * self.h1 * (1 - self.h1)</code>	$\Delta^1 = \text{upstream} \odot g'(Z^1)$, $g' = \sigma(1 - \sigma)$
<code>dW1 = X.T @ dz1</code>	$\partial L / \partial W^1 = (A^0)^T \Delta^1$
<code>db1 = dz1.sum(axis=0)</code>	$\partial L / \partial b^1$: row sum

The notebook’s own `backward` (cell 11) differs only in style: it divides by m once at the `dW/db` lines (`np.dot(self.h1.T, dL_dz2) / m, np.mean(dL_dz2, axis=0)`) instead of at the Δ line — same gradients, mean loss both ways [recorded].

Basically, ... “Every layer does the same three-move dance: 1) take the error passed back from the layer above, push it through this layer’s *weights* (transpose — that is the ‘backwards through the weights’ move); 2) multiply element-wise by the *activation*’s local slope $g'(z)$ — that is δ , the layer’s error; 3) get the parameter gradients by outer-producting δ with the layer’s *inputs* ($\delta \times$ input for weights, δ alone for biases). Repeat for every layer. The whole `backward` method is these three moves, twice.”

42.5 The output-layer special case: the pairing cancellation

The output layer needs no push-through-weights step — its upstream gradient is the loss’s own derivative, $\nabla_{a^{[L]}} L$. And for the two classification pairings of §41.11, that derivative times the activation’s slope collapses to prediction-minus-truth:

i) **Sigmoid + binary cross-entropy (§41.11(ii)).** $\hat{y} = \sigma(z)$:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \right) (\hat{y}(1-\hat{y})) = \boxed{\hat{y} - y}.$$

The notebook’s cell-17 note: “For binary cross-entropy + sigmoid: $\partial L / \partial z_3 = \hat{y} - y$.” Numerically at the §42.3 values: $-2.0387 \times 0.2499 = -0.5095$ both ways [verified-NumPy]. For mean loss over m points, each row’s δ carries $1/m$: $\Delta_i^2 = (\hat{y}_i - y_i)/m$ — verified numerically against finite differences (max difference 5.0×10^{-10}) [verified-NumPy].

ii) **Softmax + categorical cross-entropy (§41.11(iii)).** $\hat{y}_i = e^{z_i} / \sum_j e^{z_j}$, one-hot y , $L = -\sum_k y_k \log \hat{y}_k$:

$$\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j), \quad \text{so} \quad \frac{\partial L}{\partial z_j} = -\sum_k y_k(\delta_{kj} - \hat{y}_j) = \boxed{\hat{y}_j - y_j},$$

using $\sum_k y_k = 1$. Verified numerically on a 4-class example: finite-difference $\partial L / \partial z$ matches $\hat{y} - y$ to 1.2×10^{-9} [verified-NumPy]. The same “prediction minus truth” — this is why the pairings of §41.11 are chosen together: the activation’s derivative cancels the loss’s.

iii) **Identity + squared error (§41.11(i)).** $g(z) = z$, $L = \frac{1}{2}(\hat{y} - y)^2$ per example:

$$\frac{\partial L}{\partial z} = \boxed{\hat{y} - y},$$

the $\frac{1}{2}$ being the convenience that cancels the squared derivative. No cancellation magic needed — the identity’s slope is 1.

Note. Pairings (i)–(iii) all end at the same slogan, but for different reasons: in (i) and (ii) the activation’s derivative *cancels* the loss’s derivative; in (iii) there is no loss derivative to cancel. An unpaired activation (e.g. ReLU output with cross-entropy) gets no such shortcut — §41’s problem 7 asked exactly what Chapter 42 must do there: fall back to the general $\delta^{[L]} = \nabla_{a^{[L]}} L \odot g'(z^{[L]})$.

Basically, ... “The last layer is special: nothing sits above it, so its error signal comes straight from the loss. And when the loss and the activation are chosen as a pair (the §41.11 pairings), their derivatives cancel into the simplest possible error: prediction minus truth. That is why **backward** opens with **dz2 = (out - y)** — the whole loss-and-sigmoid derivative collapsed into a subtraction.”

42.6 Gradient checking: trust, then verify

Backprop is a long chain of multiplications — one sign error, one missing transpose, and every gradient is wrong while the code runs without complaint. The fix predates deep learning: compare the analytic gradients against a *finite-difference* approximation (§8.3’s derivative-as-limit),

$$\frac{\partial L}{\partial \theta_j} \approx \frac{L(\theta + \varepsilon e_j) - L(\theta - \varepsilon e_j)}{2\varepsilon}, \quad \varepsilon = 10^{-7},$$

which needs no derivation — only forward passes.

eg 3 (gradient check on the §42.3 hand-calc net, all 9 parameters). Analytic gradients from §42.3 vs. centered finite differences:

```
analytic: [-0.200336, 0.127367, 0.0, 0.0, -0.200336, 0.127367,
           -0.372452, -0.254735, -0.509470]
fin-diff: [-0.200336, 0.127367, 0.0, 0.0, -0.200336, 0.127367,
           -0.372452, -0.254735, -0.509470]
max |difference| = 8.18e-10
```


[verified-NumPy]. Agreement to $\sim 10^{-9}$: the hand-derived chain in §42.3 is the true gradient.

Note. Gradient checking is a *debugging* tool, never a training method — see §42.7 for why. And it follows §40.5(i)’s playbook: when the gradients smell wrong, check the machine on a small net with a fixed seed (§36.6) before trusting a big one.

Basically, ... “Don’t trust a chain of nine multiplications just because it looks right. Nudge each weight by a hair, measure how the loss actually moves, and compare with what backprop claimed. If they agree to 10^{-9} , your derivation is the truth. Check once while debugging; never during training.”

42.7 Why one backward pass beats perturbing every weight

The finite-difference formula above *is* a gradient estimator — so why not train with it? Count the forward passes. A net with P parameters:

- **Finite differences:** $2P$ forward passes per gradient (two per parameter).
- **Backpropagation:** 1 forward pass (caching the intermediates, §42.2) + 1 backward pass — each roughly the cost of a forward pass.

eg 4. The MLT slides’ 784–50–10 net: $P = 39,760$ (§41.14(ii)). Finite differences: $2 \times 39,760 = 79,520$ forward passes per update. Backprop: one forward, one backward. That factor of $\sim 40,000$ is the difference between “trains overnight” and “trains before the heat death of the universe” — and it grows with every added parameter.

Basically, ... “Finite differences re-run the whole network twice *per weight* to ask ‘did the loss move?’. Backprop re-runs it once *backwards* and answers that question for *every* weight at the same time — because the chain rule reuses the intermediates instead of rediscovering them. That reuse is the entire trick.”

42.8 Where this goes next

- Optimization for neural nets (Chapter 43).** This chapter computed the gradients; Chapter 43 decides how to *use* them — SGD variants, Adam, and why §41.14(i)’s zero-init symmetry trap is the entry point to initialization: the scale of the starting weights controls the scale of $g'(z)$ in the δ recursion, and hence whether the error signal survives the backward trip.
- The vanishing-gradient reading of §42.4.** Look at the recursion: each layer multiplies the error by $g'(z^{[l]})$. For sigmoid, $g'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$ — every backward step through a saturated sigmoid shrinks the error to at most a quarter (§41.7’s “flat at both ends... effectively stops learning”). ReLU’s $g' = 1$ for $z > 0$ is why “no vanishing gradient (for $z > 0$)” (§41.7) — the error passes through unshrunk. The activation table of §41.7 is, read backwards, a table about backprop.
- PyTorch’s `loss.backward()`.** The workflow notebook’s step 4 (`loss.backward()`, §41.13) automates §42.4: PyTorch builds the computational graph of §42.2 during the forward pass and multiplies the local gradients for you — autograd *is* this chapter, mechanized.
- The traps ride along.** §41.14’s warnings now have derivations behind them: zero-init symmetry (identical weights $\rightarrow$ identical δ ’s $\rightarrow$ identical updates, forever), the parameter-count overfitting, and the surrogate-loss trade.

Problem set

- The deck’s scalar graph, re-derived.** $\mathcal{L} = (a + b) \cdot c$ with $a = 2$, $b = 3$, $c = 6$. (i) Run the forward pass and cache every node value. (ii) Run the backward pass from $\partial\mathcal{L}/\partial\mathcal{L} = 1$; give $\partial\mathcal{L}/\partial a$, $\partial\mathcal{L}/\partial b$, $\partial\mathcal{L}/\partial c$, $\partial\mathcal{L}/\partial d$ with the local gradient used at each step. (iii) Check your $\partial\mathcal{L}/\partial c$ by differentiating $\mathcal{L} = (a + b)c$ directly.

2. **The three components.** For a hidden layer l , the deck writes $\partial L / \partial W^{[l]} = (\partial L / \partial a^{[l]}) \cdot (\partial a^{[l]} / \partial z^{[l]}) \cdot (\partial z^{[l]} / \partial W^{[l]})$. (i) Name each factor in the deck’s words. (ii) Which factor does the $\delta^{[l]}$ definition absorb, and why is that absorption useful? (iii) Write the recursion that computes $\delta^{[l]}$ from $\delta^{[l+1]}$ (per-example form).
3. **Batch-form translation.** (i) Starting from the deck’s $\partial L / \partial W^{[l]} = \delta^{[l]} (a^{[l-1]})^T$ ($S_l \times S_{l-1}$ layout), derive the batch form $\partial L / \partial W^l = (A^{l-1})^T \Delta^l$, stating the shape of every factor for n points. (ii) Explain in one line why the transpose appears on the *other* matrix in the batch form. (iii) For mean loss, explain why `db2 = dz2.sum(axis=0)` in §41.10 equals the row-mean of the per-example δ ’s even though it says `sum`.
4. **The XOR backward pass, a second example.** Take the §42.3 net and input $(0, 1)$ (target $y = 1$). (i) Forward pass: $z_1, h_1, z_3, \hat{y}, L$. (ii) Backward pass: $\partial L / \partial z_3$, both hidden deltas, all weight/bias gradients. (iii) Which input’s weights now get zero gradient, and why?
5. **The $(g - y)$ check.** (i) Reproduce §42.5(i)’s cancellation: at $\hat{y} = 0.4905$, $y = 1$, compute $\partial L / \partial \hat{y}$ and $\partial \hat{y} / \partial z$ separately and show their product equals $\hat{y} - y$. (ii) In §41.10’s **backward**, point at the exact line this produces and explain the `/ m`. (iii) Give one line for when this shortcut is *unavailable* (the §42.5 Note case).
6. **Softmax pairing.** (i) For a 3-class net with $z = (1.0, 0.0, -1.0)$, $y = [0, 1, 0]$: compute $\hat{y}$, L , and $\partial L / \partial z$ via $\hat{y} - y$. (ii) Verify one component with the two-step chain $\partial L / \partial \hat{y}_k \cdot \partial \hat{y}_k / \partial z_j$ (use §31.12’s softmax Jacobian). (iii) In one line, say what cancels.
7. **Gradient check by hand.** A 1-1 net: $z = wx + b$, $\hat{y} = \sigma(z)$, BCE loss, $w = 0.5$, $b = 0.1$, $x = 2.0$, $y = 1$. (i) Compute $\partial L / \partial w$ and $\partial L / \partial b$ by backprop. (ii) Approximate both with centered finite differences at $\varepsilon = 10^{-5}$; report the agreement. (iii) In one line: what breaks in this check if you forget to clip $\hat{y}$ near 0 or 1?
8. **Credit where it’s due.** In §42.3, $\partial L / \partial w_{21} = \partial L / \partial w_{22} = 0$ because $x_2 = 0$. (i) Give a one-line chain-rule reason. (ii) After the $\alpha = 0.1$ update, the net sees $(0, 1)$ — do w_{21}, w_{22} move on *this* step? (iii) In two lines, explain what this says about which training examples teach which weights.
9. **Vanishing, read backwards.** A 5-hidden-layer sigmoid net; every hidden neuron sits at $|z| \geq 3$. (i) Bound $g'(z)$ at $|z| = 3$ ($\sigma(3) \approx 0.9526$). (ii) Bound the shrinkage of $\|\delta^{[1]}\|$ relative to $\|\delta^{[5]}\|$ through four backward steps. (iii) In two lines, connect this to §41.7’s verdict on sigmoid in deep stacks and its ReLU recommendation.
10. **Map the notebook.** The XOR notebook’s **backward** (cell 11) computes `dL_dz2 = dL_dy` then `dL_dw2 = np.dot(self.h1.T, dL_dz2) / m`. (i) Match each of its six gradient lines to a formula in §42.4. (ii) Explain the comment “For sigmoid, $dy/dz = y(1-y)$, but we can simplify” in §42.5 terms. (iii) It divides by m at the `dW/db` lines while §41.10 divides inside `dz2` — prove the two give identical gradients.

Sources: GenAI “Introduction to ANN” deck (Balaji Srinivasan, Ganapathy Krishnamurthi), slides 59–70 (chain rule, computational graph, scalar hand example, MLP weight-update derivation); XOR Problem notebook, cells 14/17/18 (hand-calc weights, full backward pass, verification); MLT Week-12 ANN slides (backprop named as trainer, no new derivation); book chapters 2 (§2.10), 9 (§9.7), 10 (§10.5, §10.7), 31 (§31.10, §31.12), 36 (§36.2, §36.6), 41 (§§41.4, 41.9–41.11, 41.13, 41.14).

Chapter 42 solutions — Backpropagation, worked end-to-end

All numbers below were re-run in numpy [verified-NumPy] unless marked [recorded] (transcribed from the course materials).

Problem 1 — the deck’s scalar graph, re-derived

(i) **Forward pass.** Nodes: $d = a + b$, $\mathcal{L} = d \cdot c$.

$$d = 2 + 3 = 5, \quad \mathcal{L} = 5 \cdot 6 = 30.$$

Cached: $a = 2$, $b = 3$, $c = 6$, $d = 5$, $\mathcal{L} = 30$.

(ii) **Backward pass.** Start $\partial\mathcal{L}/\partial\mathcal{L} = 1$ (“by definition,” slide 63). - At the $\times$ node: local gradients $\partial\mathcal{L}/\partial d = c = 6$, $\partial\mathcal{L}/\partial c = d = 5$. So $\partial\mathcal{L}/\partial c = 1 \cdot 5 = 5$, $\partial\mathcal{L}/\partial d = 1 \cdot 6 = 6$. - At the $+$ node: local gradients $\partial d/\partial a = \partial d/\partial b = 1$. So $\partial\mathcal{L}/\partial a = 6 \cdot 1 = 6$, $\partial\mathcal{L}/\partial b = 6 \cdot 1 = 6$.

(iii) **Direct check.** $\mathcal{L} = (a + b)c$, so $\partial\mathcal{L}/\partial c = a + b = 5$ — matches. (Similarly $\partial\mathcal{L}/\partial a = c = 6$.)

Problem 2 — the three components

(i) $\partial L/\partial a^{[l]} = 1$. **Upstream Gradient** — “How the final loss changes with this layer’s output. This is passed back from layer $l + 1$.” $\partial a^{[l]}/\partial z^{[l]} = g'(z^{[l]}) = 2$. **Activation Gradient** — “The derivative of the activation function.” $\partial z^{[l]}/\partial W^{[l]} = 3$. **Local Gradient** — “How this layer’s pre-activation changes with its weights.”

(ii) $\delta^{[l]} = \partial L/\partial z^{[l]}$ absorbs (i) and (ii): $\delta^{[l]} = (\partial L/\partial a^{[l]}) \odot g'(z^{[l]})$. Useful because $\delta^{[l]}$ is the *single* quantity each layer needs — it is computed once per layer and then reused for both $\partial L/\partial W^{[l]}$ and $\partial L/\partial b^{[l]}$, and it is what gets passed one step further back.

(iii) $\delta^{[l]} = ((W^{[l+1]})^T \delta^{[l+1]}) \odot g'(z^{[l]})$ (the deck’s boxed recursion; for the output layer, $\delta^{[L]} = \nabla_{a^{[L]}} L \odot g'(z^{[L]})$).

Problem 3 — batch-form translation

(i) Per-example: $\partial L/\partial W^{[l]} = \delta^{[l]}(a^{[l-1]})^T$ with $\delta^{[l]}$ $S_l \times 1$, $a^{[l-1]}$ $S_{l-1} \times 1$. The batch layout stores examples as rows: A^{l-1} is $n \times S_{l-1}$, Δ^l is $n \times S_l$. Summing the per-example outer products over the batch:

$$\frac{\partial L}{\partial W^l} = \sum_{i=1}^n \Delta_i^l (A_i^{l-1})^T = (A^{l-1})^T \Delta^l,$$

an $S_{l-1} \times S_l$ matrix — the shape of W^l .

(ii) In per-example form the weights left-multiply ($W^{[l]} a^{[l-1]}$), so the outer product is $\delta(a)^T$; in batch form they right-multiply rows ($A^{l-1} W^l$), so the sum-of-outer-products assembles as $(A^{l-1})^T \Delta^l$ — the transpose moves to keep each factor’s rows/columns aligned with W^l ’s ($S_{l-1} \times S_l$) layout (§41.4’s Note).

(iii) In §41.10, `dz2` already carries the $1/m$: `dz2 = (out - y)/m`. So `dz2.sum(axis=0)` = $\sum_i \delta_i/m$ = the row-mean of the per-example δ ’s. It says `sum` but computes a mean, because the $1/m$ was applied one line earlier. (The notebook does it the other way: unscaled Δ , then `/ m` at the `dW/db` lines — identical result.)

Problem 4 — the XOR backward pass, a second example

Weights as in §42.3. Input $(0, 1)$, target $y = 1$.

(i) **Forward.**

$$z_1 = [0(1) + 1(1) + 0, 0(-1) + 1(-1) + 1] = [1.0, 0.0], \quad h_1 = [0.7311, 0.5],$$

$$z_3 = 2(0.7311) - 1(0.5) - 1 = -0.0379, \quad \hat{y} = 0.4905, \quad L = -\log(0.4905) = 0.7123.$$

Coincidence worth noting: the forward values are identical to the $(1, 0)$ case — with these hand-picked weights, $x_1 + x_2$ always equals 1 on the two $y = 1$ corners. The gradients will differ only through the x factor.

(ii) **Backward.**

$$\frac{\partial L}{\partial z_3} = \hat{y} - y = -0.5095, \quad \frac{\partial L}{\partial z_1} = -0.2003, \quad \frac{\partial L}{\partial z_2} = 0.1274$$

(same deltas as §42.3 — same $\hat{y}$, same h_1). Weight gradients now use $x = (0, 1)$:

$$\frac{\partial L}{\partial W_1} = x^T \delta = \begin{bmatrix} 0 & 0 \\ -0.2003 & 0.1274 \end{bmatrix}, \quad \frac{\partial L}{\partial b_1} = [-0.2003, 0.1274],$$

$$\frac{\partial L}{\partial W_2} = [-0.3725, -0.2547]^T, \quad \frac{\partial L}{\partial b_2} = -0.5095.$$

(iii) Now $x_1 = 0$, so w_{11} and w_{12} (the weights fed by x_1) get zero gradient — the mirror image of §42.3's Note, where $x_2 = 0$ froze w_{21}, w_{22} .

Problem 5 — the $(g - y)$ check

(i) At $\hat{y} = 0.4905$, $y = 1$:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{1}{0.4905} = -2.0387, \quad \frac{\partial \hat{y}}{\partial z} = 0.4905(1 - 0.4905) = 0.2499,$$

$$(-2.0387)(0.2499) = -0.5095 = 0.4905 - 1 = \hat{y} - y \quad \checkmark$$

[verified-NumPy].

(ii) `dz2 = (self.out - y).reshape(-1, 1) / m` — the entire sigmoid-derivative-times-loss-derivative two-step collapses to the subtraction. The `/ m` is there because the loss is the *mean* over the batch: each example's δ is its share $1/m$ of the mean.

(iii) The shortcut is unavailable whenever the output activation and loss are *not* a cancelling pair — e.g. a ReLU (or tanh) output with cross-entropy, or sigmoid with squared error: then use the general $\delta^{[L]} = \nabla_{a^{[L]}} L \odot g'(z^{[L]})$ with no cancellation.

Problem 6 — softmax pairing

(i) $z = (1.0, 0.0, -1.0)$: $e^z = (2.7183, 1.0, 0.3679)$, $\text{sum} = 4.0862$,

$$\hat{y} = (0.6652, 0.2447, 0.0900), \quad L = -\log(0.2447) = 1.4076,$$

$$\frac{\partial L}{\partial z} = \hat{y} - y = (0.6652, -0.7553, 0.0900) \quad [\text{verified} - \text{NumPy}].$$

(ii) Two-step for $j = 1$ (the true class): $\partial L / \partial \hat{y}_k = -y_k / \hat{y}_k$ is nonzero only at $k = 1$: $-1/0.2447 = -4.0862$. $\partial \hat{y}_1 / \partial z_1 = \hat{y}_1(1 - \hat{y}_1) = 0.2447 \times 0.7553 = 0.1848$. Product: $(-4.0862)(0.1848) = -0.7553$ — matches $\hat{y}_1 - y_1$ ✓.

(iii) What cancels: the softmax Jacobian's $\hat{y}_i(\delta_{ij} - \hat{y}_j)$ structure against the log-loss's $1/\hat{y}_k$ — every term with $k \neq j$ folds into $-\hat{y}_j \sum_k y_k = -\hat{y}_j$, leaving $\hat{y}_j - y_j$.

Problem 7 — gradient check by hand

(i) $z = 0.5(2.0) + 0.1 = 1.1$; $\hat{y} = \sigma(1.1) = 0.75026$; $L = -\log(0.75026) = 0.28734$.

$$\frac{\partial L}{\partial z} = \hat{y} - y = -0.24974, \quad \frac{\partial L}{\partial w} = (-0.24974)(2.0) = -0.49948, \quad \frac{\partial L}{\partial b} = -0.24974.$$

(ii) $\varepsilon = 10^{-5}$:

$$\text{FD}_w = -0.499479789, \quad \text{FD}_b = -0.249739894 \quad \Rightarrow \quad \text{agreement to } \sim 10^{-10} \quad \checkmark \quad [\text{verified} - \text{NumPy}].$$

(iii) If $\hat{y}$ hits exactly 0 or 1, $\log \hat{y}$ (or $\log(1 - \hat{y})$) is undefined/ $-\infty$ and the finite-difference evaluation explodes — hence the `np.clip(yhat, 1e-15, 1 - 1e-15)` in every loss implementation (§41.10).

Problem 8 — credit where it’s due

(i) $\partial L / \partial w_{21} = (\partial L / \partial z_1)(\partial z_1 / \partial w_{21}) = (\partial L / \partial z_1) \cdot x_2$, and $x_2 = 0$ — the chain multiplies by the local gradient, which is the input value itself.

(ii) Yes — on input $(0, 1)$ we have $x_2 = 1$, so $\partial L / \partial w_{21} = (\partial L / \partial z_1) \cdot 1 \neq 0$ and w_{21} moves (with the pre-update weights the numbers are Problem 4’s: -0.2003 ; after the update the deltas shift slightly but stay nonzero).

(iii) A weight only learns from examples where its input is nonzero: each weight’s gradient is its delta times its input, so an example teaches exactly the weights on its own active input paths. Sparse/zero inputs mean sparse updates — the same logic behind embedding layers only updating the looked-up rows.

Problem 9 — vanishing, read backwards

(i) $\sigma(3) \approx 0.9526$, $g'(3) = 0.9526 \times 0.0474 \approx 0.0452$.

(ii) Each backward step multiplies by $g' \leq 0.0452$, so over four steps $\|\delta^{[1]}\| \lesssim (0.0452)^4 \|\delta^{[5]}\| \approx 4.2 \times 10^{-6} \|\delta^{[5]}\|$ — the error signal is attenuated by a factor of $\sim 240,000$ [verified-NumPy].

(iii) This is §41.7’s “flat at both ends... gradient near zero for large inputs, effectively stops learning,” now with a number on it: saturated sigmoids don’t just slow learning, they shrink the gradient *exponentially* in depth. ReLU’s $g' = 1$ for $z > 0$ passes the error through unshrunk — hence “start with ReLU.”

Problem 10 — map the notebook

(i) Notebook cell-11 **backward**, with $m =$ batch size: | notebook line | §42.4 formula | |—|—| | $dL_dy = self.y_pred - y_true$ | per-example δ for the sigmoid+BCE pairing (§42.5(i)) | | $dL_dz2 = dL_dy$ (+ reshape) | Δ^2 (unscaled; the $/m$ comes later) | | $dL_dW2 = np.dot(self.h1.T, dL_dz2) / m$ | $\partial L / \partial W^2 = (A^1)^T \Delta^2$, mean over batch | | $dL_db2 = np.mean(dL_dz2, axis=0)$ | $\partial L / \partial b^2 =$ row-mean of Δ^2 | | $dL_dh1 = np.dot(dL_dz2, self.W2.T)$ | upstream $\partial L / \partial A^1 = \Delta^2 (W^2)^T$ | | $dL_dz1 = dL_dh1 * self.sigmoid_derivative(self.z1)$ | $\Delta^1 =$ upstream $\odot g'(Z^1)$ | | $dL_dW1 = np.dot(X.T, dL_dz1) / m$ | $\partial L / \partial W^1 = (A^0)^T \Delta^1$, mean over batch | | $dL_db1 = np.mean(dL_dz1, axis=0)$ | $\partial L / \partial b^1 =$ row-mean of Δ^1 |

(ii) For sigmoid output + BCE, the true $\partial L / \partial z_2 = (\partial L / \partial \hat{y}) \hat{y}(1 - \hat{y})$ *would* need the $y(1 - y)$ factor — but the product collapses to $\hat{y} - y$, so the code skips the two-step and writes the answer directly (§42.5(i)).

(iii) Notebook: Δ unscaled, gradients $= (1/m) \sum_i \cdot$. §41.10: Δ pre-scaled by $1/m$ at **dz2**, gradients $= \sum_i \cdot$. Since $\sum_i (\delta_i / m) = (1/m) \sum_i \delta_i$, the two are algebraically identical — the $1/m$ commutes with the sum. (Note §41.10’s **dh1 = dz2 @ W2.T** then carries the $1/m$ through the whole backward pass consistently.)

Chapter 43: Optimization for neural nets — SGD variants, init, normalization

Chapter 42 derived the gradients: one backward sweep, every $\partial L/\partial \theta$ in hand. This chapter decides what to *do* with them — the optimizers, the learning-rate dial, the starting point, and the input scale. It is the piece §41.15(ii) promised (“§41.14(i)’s symmetry trap is the entry point: *how* to sample the initial weights, why scale matters for deep stacks, and the optimizers (SGD variants, Adam) the notebooks already name”). The spine is the GenAI Week 5 deck (“Optimization Strategies,” Balaji Srinivasan and Ganapathy Krishnamurthi, 78 slides — convex primer, GD algorithms, LR scheduling, momentum, AdaGrad, RMSProp, AdaDelta, Adam) plus its two companion notebooks, *Gradient_Descent.ipynb* (batch/SGD/mini-batch, LR sensitivity, batch-size experiments) and *Advanced_Optimization.ipynb* (momentum, Nesterov, AdaGrad, RMSProp, AdaDelta, Adam, AdamW, with torch implementations and a Rosenbrock/Beale/neural-net comparison). Every number in §43.11 was re-run here in numpy and is marked [verified-NumPy]; deck, notebook, and torch numbers are transcriptions, marked [recorded].

Notation. The deck writes the learning rate as η ; Chapters 10, 41, and 42 wrote α — the same number, different letter. Subscripts t count update steps: $g_t = \nabla_{\theta} J(\theta_t)$.

43.1 Why plain gradient descent needs help in deep nets

Backprop (§42.4) hands the optimizer a gradient vector. The optimizer’s job is to turn gradients into parameter moves. Vanilla GD (§10.5: $\theta_{t+1} = \theta_t - \eta g_t$) has one global step size and no memory — and the deck’s Week 5 material says exactly why that is not enough:

- i) **Ravines.** The deck’s own word, and its own example: $f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$ — a long narrow valley. The gradient is $20\times$ larger in the x_2 direction ($\nabla f = (0.2x_1, 4x_2)$). One η cannot serve both: a large η diverges across the steep direction, a small one crawls along the flat direction. “The optimization path oscillates across the narrow valley instead of moving along it.”
- ii) **Saddle points.** The deck’s definition: “A saddle point is a location where all gradients of a function vanish (i.e., $\nabla f(x) = 0$), but which is neither a local minimum nor a local maximum.” And the deck’s warning, in full caps: “Saddle Points are More Common than Minima in High Dimensions!” Plain GD has no momentum to roll through them.
- iii) **Local minima.** §41.14(iv) already settled this: the net “converges to **local minima**” — but “typically works very well in practice.” The deck agrees: the landscape is non-convex, full of minima and saddles, and nobody panics.

The goal line (deck’s summary, worth keeping). “Optimization in deep learning aims to minimize generalization error, not just training error.” The optimizer that overfits the training loss fastest is not automatically the best optimizer.

Basically, ... “Backprop tells you which way is downhill; it doesn’t tell you how big a step to take, and

one step size can't serve a valley that is steep in one direction and flat in another. This chapter is about smarter stepping: memory (momentum), per-parameter step sizes (adaptive methods), shrinking steps over time (schedules), and not starting in a bad spot (initialization and scaling)."

43.2 The three gradient descents: batch, stochastic, mini-batch

The *Gradient Descent* notebook's framing: "The primary distinction between gradient descent variants lies in how many training samples are used to compute the gradient at each iteration." The deck adds the geometry: vectorization — CPUs and GPUs are parallel processors; computing gradients block-by-block (mini-batch) beats one-by-one (SGD-like) because it keeps data in fast cache, while full-matrix (batch-like) is fastest per pass but touches everything.

Def (the three). - **Batch GD:** gradient over the *entire* dataset each step. Exact gradient of the full loss. Smooth, deterministic — and $O(n)$ per step, "inefficient for redundant data," "does not leverage modern parallel hardware." - **Stochastic GD:** gradient from *one* randomly drawn sample per step. Cheap ($O(1)$), noisy, "can escape shallow local minima," enables online learning — but fluctuates and needs careful LR scheduling. - **Mini-batch GD:** gradient over a batch of b samples. "The Solution: Minibatch Stochastic Gradient Descent. Instead of one example or all examples, we use a small batch of examples. This is the de facto standard for training deep neural networks."

The mini-batch theorem (deck). With batch $B_t \subset \{1, \dots, n\}$, $|B_t| = b$, sampled typically without replacement:

$$\nabla f_{B_t}(x_t) = \frac{1}{b} \sum_{i \in B_t} \nabla f_i(x_t), \quad x_{t+1} = x_t - \eta \nabla f_{B_t}(x_t).$$

"The Best of Both Worlds: Computational Efficiency — significantly faster than one-by-one SGD due to vectorization. Statistical Efficiency — averaging gradients over a minibatch reduces [noise]."

The noise-reduction half is simple: each sample's gradient is an unbiased estimate of the true gradient, and averaging b independent ones divides the variance by b (§21's logic in miniature). That is the entire statistical argument for $b > 1$.

eg 1 (the notebook's batch-size experiment, [recorded]). On the notebook's synthetic linear regression ($n = 1000$, $y = 3.5x + 2 + \varepsilon$), batch sizes 1, 8, 32, 64, 128, 256 with the same $\eta = 0.01$: - Small batches (1–32): more frequent updates, more noise, "better generalization (more noise can help escape local minima)," memory-light. - Large batches (128–256): fewer updates per epoch, stabler gradients, better hardware use — but "may converge to sharp minima (worse generalization)," more memory. - Common choices: 32, 64, 128, 256. The notebook's rule of thumb: "Larger batch sizes need larger learning rates."

PyTorch vocabulary (notebook, [recorded]). In PyTorch, `torch.optim.SGD` is the algorithm and the *DataLoader*'s `batch_size` picks the variant: `batch_size = 1` is stochastic, `batch_size = n` is batch, 32–128 is mini-batch. One optimizer, three regimes.

Basically, ... "How many examples per gradient? All of them (accurate but slow), one (fast but jumpy), or a batch (the sweet spot). Mini-batch wins because averaging b noisy gradients divides the noise by b , and hardware does batches in parallel anyway."

43.3 Momentum: remember where you were going

The physical analogy (deck). "Imagine a ball rolling down a hill. The ball accumulates velocity as it rolls down." In steep directions the velocity keeps changing sign and the oscillations "tend to cancel out"; in flat directions it "consistently accelerates, building up speed." Momentum dampens the ravine's zig-zag and speeds the crawl along the valley floor — §43.1(i)'s fix.

Theorem (momentum, deck). With $g_t = \nabla f(x_t)$, velocity v_t , momentum $\beta \in [0, 1)$:

$$v_t = \beta v_{t-1} + g_t \quad (\text{accumulate velocity}), \quad x_{t+1} = x_t - \eta v_t \quad (\text{update with velocity}).$$

Typical $\beta = 0.9$; the notebook also lists 0.99 for smooth landscapes.

What the velocity really is (deck’s “leaky averages” slide). Expand the recurrence:

$$v_t = \beta v_{t-1} + g_t = \beta^2 v_{t-2} + \beta g_{t-1} + g_t = \dots = \sum_{\tau=0}^t \beta^{t-\tau} g_\tau.$$

v_t is a weighted average of all past gradients, weights $\beta^{t-\tau}$ decaying exponentially — “giving more importance to recent gradients.” The weight sum converges: $\sum_{i=0}^{\infty} \beta^i = 1/(1-\beta)$, so the update step is roughly $\eta/(1-\beta)$ times the averaged gradient. For $\beta = 0.9$ that is $10 \times \eta$ — a momentum-0.9 optimizer with nominal rate η moves, on average, like plain GD with rate 10η [verified-NumPy: $\sum_{k=0}^{60} 0.9^k = 9.984 \approx 10$].

Note. “A larger β (e.g., 0.95) means a longer ‘memory’ of past gradients. A smaller β means a shorter memory.” Longer memory = more smoothing = slower to react when the slope reverses; that is the trade.

Basically, ... “Plain GD has amnesia — every step trusts only the current slope, so it zig-zags across the ravine. Momentum keeps a running average of past slopes (recent ones count most). Consistent directions build up speed; contradictory ones cancel out. The math is a leaky average, the physics is a rolling ball.”

43.4 Nesterov: look before you leap (notebook-only)

Classical momentum computes the gradient at the *current* position, then steps. Nesterov computes the gradient at the *approximate future* position — where the accumulated velocity is about to take you:

$$\boxed{v_t = \beta v_{t-1} + \nabla_{\theta} J(\theta_t - \eta \beta v_{t-1}), \quad \theta_{t+1} = \theta_t - \eta v_t} \quad [\text{recorded}].$$

“Classical Momentum: ‘I’m going in this direction with this velocity, let me check the gradient.’ Nesterov Momentum: ‘I’m about to go in this direction, let me check the gradient there first.’” The look-ahead is a form of error correction — it notices the slope reversing *before* stepping into it, so it overshoots less.

Two facts from the notebook, kept as stated: it converges at $O(1/t^2)$ vs classical momentum’s $O(1/t)$ for *convex functions*, and “for non-convex deep learning, benefits over classical momentum are often marginal.” The notebook’s torch implementation uses the efficient reformulation (equivalent, one gradient evaluation):

$$v_t = \beta v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta(\beta v_t + g_t) \quad [\text{recorded}].$$

Note. Nesterov does not appear in the deck — it is notebook-only. In PyTorch it is a flag on SGD (`nesterov=True`), not a separate optimizer [recorded].

Basically, ... “Momentum looks at the slope where you stand; Nesterov looks at the slope where you’re *about* to stand. Same memory, smarter aim — at the price of one extra conceptual gradient.”

43.5 Adaptive methods I: AdaGrad, RMSProp, AdaDelta

Momentum fixes the *direction* problem. The adaptive family fixes a different one: one global η cannot fit parameters whose gradients live on wildly different scales (§43.1(i) again). Their answer: give every parameter its own learning rate, derived from its own gradient history.

i) AdaGrad. Keep a per-parameter accumulator of squared gradients, $s_t = s_{t-1} + g_t \odot g_t$, and divide the global rate by it (deck’s theorem):

$$\boxed{x_{t+1} = x_t - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t} \quad [\text{recorded}].$$

Intuition (deck): “If a parameter has consistently large gradients, its entry in s_t will grow quickly, rapidly decreasing its effective learning rate. If a parameter has small or zero gradients (like for a sparse feature),

its entry in s_t grows slowly, keeping its effective learning rate high.” On the deck’s ill-conditioned problem, AdaGrad “acts as a preconditioner. It automatically corrects for the different scales of the gradients” — the x_2 accumulator grows faster than the x_1 one, scaling the steep direction down harder, “as if the problem were better conditioned.”

The fatal flaw (deck): “The accumulator s_t only ever increases. After enough time, the learning rate for all parameters will approach zero, stopping training prematurely.” The sum of squared gradients is a one-way ratchet: $\lim_{t \rightarrow \infty} \eta / \sqrt{s_t} = 0$. Fine for short, sparse problems (NLP word embeddings — the notebook’s recommended use); bad for long training runs.

ii) RMSProp. “Fixing AdaGrad’s decaying learning rate”: replace the ever-growing sum with an *exponentially decaying* average, $E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2$ (the deck and notebook; the notebook notes it was “developed by Geoff Hinton in his Coursera lecture (never formally published!)”):

$$x_{t+1} = x_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \odot g_t, \quad \beta \approx 0.9 \quad [\text{recorded}].$$

Old gradients decay away, so the rate stops shrinking — “recent gradients have more influence,” and training “maintains learning capability throughout.” The notebook’s insight: with $\beta = 0.9$, roughly the last 10 gradient updates set the learning rate. Defaults: $\eta = 0.001$, $\beta = 0.9$ [recorded].

iii) AdaDelta. Same decaying averages, but no learning rate at all — the update size comes from the ratio of the RMS of *past parameter updates* to the RMS of *gradients* (deck’s theorem, condensed):

$$s_t = \rho s_{t-1} + (1 - \rho)(g_t \odot g_t), \quad \Delta x_t = -\frac{\sqrt{\Delta x_{t-1} + \epsilon}}{\sqrt{s_t + \epsilon}} \odot g_t, \quad x_{t+1} = x_t + \Delta x_t \quad [\text{recorded}].$$

“The term $\sqrt{\Delta x_{t-1}} / \sqrt{s_t}$ approximates a learning rate, effectively self-adjusting based on the recent dynamics.” The selling point is unit-correctness: the step has the same units as the parameter. The cost: “can be slower to converge than Adam,” extra memory for the update history.

Basically, ... “AdaGrad: each parameter’s step size = global rate divided by (root of its total past squared gradients) — big-history parameters get small steps, rare ones keep big steps; but the denominator only grows, so training eventually freezes. RMSProp: use a *decaying* average instead — old history fades, the freeze never comes. AdaDelta: drop the global rate entirely — let the ratio of past *updates* to past *gradients* set the step.”

43.6 Adam: momentum + RMSProp, with bias correction

“Adam is arguably the most popular ‘go-to’ optimization algorithm. It combines the best of two worlds: Momentum + RMSProp” (deck). Keep the deck’s two-moment notation: v_t = exponentially weighted average of gradients (the momentum part), s_t = exponentially weighted average of squared gradients (the RMSProp part).

Theorem (Adam, deck). With decay rates β_1, β_2 :

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1)g_t \quad (\text{first moment: direction}) \quad (1)$$

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2)(g_t \odot g_t) \quad (\text{second moment: scale}) \quad (2)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1 - \beta_2^t} \quad (\text{bias correction}) \quad (3)$$

$$x_{t+1} = x_t - \eta \frac{\hat{v}_t}{\sqrt{\hat{s}_t + \epsilon}} \quad [\text{recorded}] \quad (4)$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.001$, $\epsilon = 10^{-8}$ [recorded].

Why the bias correction (3). Both moments start at 0, so early on they underestimate the truth — they are “biased towards zero.” The notebook spells out the fix: divide by $1 - \beta^t$, which starts small and approaches 1.

eg 2 (one Adam step, by hand — [verified-NumPy]). Scalar parameter, first step, $g_1 = 0.5$:

$$v_1 = (1 - 0.9)(0.5) = 0.05, \quad s_1 = (1 - 0.999)(0.5^2) = 0.00025,$$

$$\hat{v}_1 = \frac{0.05}{1 - 0.9} = 0.5, \quad \hat{s}_1 = \frac{0.00025}{1 - 0.999} = 0.25,$$

$$\text{step} = \eta \cdot \frac{0.5}{\sqrt{0.25}} = \eta \cdot 1.0.$$

The correction restores the step to $\eta \cdot \text{sign}(g)$ on the very first iteration. Without it, the step would be $\eta \cdot 0.05 / \sqrt{0.00025} \approx 3.16 \eta$ — a directionless $3.16\times$ overshoot cooked up from zero-initialization, not from the data. That is what (3) deletes.

The notebook’s honest caveats. Adam “may not converge to optimal solution in some convex optimization problems,” “can generalize worse than SGD with momentum in some cases,” and “standard L2 regularization doesn’t work as intended” — the last of which gets its own section next.

Basically, ... “Adam = momentum’s running-average direction divided by RMSProp’s running-average scale, per parameter. The direction comes from the average gradient, the step size from the average *squared* gradient, and the bias correction just undoes the zero-start so the first steps aren’t nonsense. Defaults ($\beta_1=0.9$, $\beta_2=0.999$, $\eta=0.001$) work out of the box — which is why it is the default.”

43.7 Weight decay: the λ that rides inside the optimizer

Chapter 28’s ridge penalty and the optimizer’s weight decay are the same idea wearing two hats. The deck’s convex-primer slide gives the penalty view: “Weight decay is a penalty method. Adding $\frac{\lambda}{2}\|w\|^2$ to the loss is equivalent to enforcing a constraint $\|w\|^2 - r^2 \leq 0$ for some radius r .” The λ is the same λ as §28’s $J(w) = \frac{1}{2}(Xw - y)^T(Xw - y) + \frac{\lambda}{2}\|w\|_2^2$, whose gradient picks up $+\lambda w$ (§28: $\nabla_w J = X^T Xw - X^T y + \lambda w$).

In plain SGD the two views coincide. Put the penalty in the loss: the gradient becomes $g_t + \lambda\theta_t$, and the update is

$$\theta_{t+1} = \theta_t - \eta(g_t + \lambda\theta_t) = (1 - \eta\lambda)\theta_t - \eta g_t.$$

Each step shrinks the weights by the fraction $\eta\lambda$ *before* stepping downhill — literally “decay the weights.” Adding $\frac{\lambda}{2}\|w\|^2$ to the loss and multiplying the weights by $(1 - \eta\lambda)$ are the same update.

In Adam they come apart (the notebook’s key point, [recorded]). If you add the penalty to Adam’s loss, the $\lambda\theta_t$ term enters the *gradient*, and the adaptive scaling divides it by $\sqrt{\hat{s}_t}$ — so different parameters get different effective regularization, “not equivalent to L2 regularization.” **AdamW** (the notebook, [recorded]) applies the decay *after* the adaptive update instead:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{v}_t}{\sqrt{\hat{s}_t} + \epsilon} + \lambda\theta_t \right) = (1 - \eta\lambda)\theta_t - \eta \frac{\hat{v}_t}{\sqrt{\hat{s}_t} + \epsilon}.$$

The second form shows it: true weight decay — every parameter shrinks by the constant fraction $\eta\lambda$ per step, independent of its gradient history. “With AdamW, the effective update becomes $\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\hat{v}_t/(\sqrt{\hat{s}_t} + \epsilon)$.”

The notebook’s verdict: AdamW “consistently outperforms Adam in transformer models (BERT, GPT, etc.), computer vision (ResNets, Vision Transformers), fine-tuning pre-trained models” — and the 2025 default is “AdamW (lr=0.001, weight_decay=0.01)” [recorded]. Typical weight-decay values: 0.01–0.1 for vision, 0.001–0.01 for NLP, 0.0001–0.001 for fine-tuning [recorded].

Basically, ... “Weight decay = shrink the weights a little every step $(1 - \eta\lambda)$, the optimizer’s version of Chapter 28’s λ tax. In plain SGD, putting the tax in the loss is identical. In Adam, the tax goes through the adaptive divider and comes out uneven — so AdamW applies the shrink *after* the adaptive step, the same fraction for every parameter.”

43.8 Learning-rate schedules: shrink the steps as you settle

“Adjusting the learning rate η during training is often as important as the choice of optimizer” (deck). Three reasons, the deck’s own list:

- i) **Magnitude:** “If η is too large, the optimizer diverges. If too small, it converges too slowly.”
- ii) **Decay:** “As we approach a minimum, we need to take smaller steps to avoid overshooting and to settle into the minimum.”
- iii) **Warmup:** “At the start of training, parameters are random. Large steps might be harmful. A ‘warmup’ period with a small initial learning rate that gradually increases can improve stability.”

The five common schedules (deck, t = update step; [recorded]). - **Piecewise constant:** $\eta(t) = \eta_i$ for $t_i \leq t < t_{i+1}$ — e.g. “reduce LR by 10× every 30 epochs.” - **Exponential decay:** $\eta(t) = \eta_0 e^{-\lambda t}$. - **Polynomial decay:** $\eta(t) = \eta_0 (\beta t + 1)^{-\alpha}$ — “a popular choice is $\alpha = 0.5$, which gives a decay rate of $O(1/\sqrt{t})$,” with “good theoretical convergence properties for convex problems.” - **Inverse time decay:** $\eta(t) = \eta_0 / (1 + kt)$. - **Cosine annealing:** $\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T_{\max}))$.

eg 3 (the five, side by side — [verified-NumPy]). $\eta_0 = 0.2$, $\lambda = 0.01$, $\alpha = 0.5$, $\beta = 1$, $k = 0.01$, $\eta_{\max} = 0.2$, $\eta_{\min} = 0$, $T_{\max} = 1000$:

t	piecewise	exponential	polynomial ($\alpha=0.5$)	inverse time	cosine
0	0.2000	0.2000	0.2000	0.2000	0.2000
100	0.2000	0.0736	0.0199	0.1000	0.1951
500	0.2000	0.0013	0.0089	0.0333	0.1000
1000	0.2000	0.0000	0.0063	0.0182	0.0000

Exponential dies fastest; cosine stays high longest then lands exactly on $\eta_{\min}$ at $T_{\max}$; inverse time and polynomial sit in the middle.

Note (the §37.3 connection). sklearn’s ‘`invscaling`’ — the stated default of `SGDRegressor` — is the inverse-time shape: $\eta(t) = \eta_0 / t^{\text{power}-t}$ (§37.3). It differs from the deck’s $\eta_0 / (1 + kt)$ only in the +1 in the denominator (sklearn indexes its steps so the division is safe). Same family, same idea — §37.3 was already scheduling; this chapter names the relatives.

Basically, ... “Start bold, finish careful. Big steps early cross the landscape fast; small steps late settle into the minimum without bouncing out. Pick a decay curve (or steps, or cosine), and give training a gentle warmup first — random starting weights plus big steps is how explosions happen.”

43.9 Normalization: fix the scale before the optimizer has to

The adaptive methods of §43.5 are a *reactive* fix for bad scale: they notice the gradient scales differ and compensate. The proactive fix is to never hand the optimizer badly scaled inputs in the first place — and this book already met it.

§37.3’s sermon returns. “SGD is sensitive to feature scaling, so it is highly recommended to scale input feature matrix.” The slide’s fix: `StandardScaler` before `SGDRegressor` (§37.3). Why it matters is now visible in optimizer terms: if feature x_1 lives in $[0, 1]$ and x_2 in $[0, 1000]$, the loss surface is exactly the deck’s ill-conditioned valley — one direction steep, one flat, one η serving both. Standardizing ($x' = (x - \mu)/\sigma$) re-rounds the valley at the source instead of asking AdaGrad to precondition it at runtime.

What the sources don't cover (flagged). The *Advanced Optimization* notebook's classifier stacks `nn.BatchNorm1d` after every hidden layer — but never explains batch normalization: no formula, no derivation, no discussion. The deck is silent too. So: batch normalization is **not derived in this chapter**; the GenAI W5 materials use it without explaining it. What the course *does* support, fully, is the input-scaling story above. Batch norm is a natural candidate for a later pass over Chapter 44 or a dedicated normalization note.

Basically, ... “Don't make the optimizer fight your units. If one feature is $1000\times$ another, the loss landscape is a ravine and you're paying adaptive-optimizer rent to cross it. Scale the features first (z -scores, §37.3) and the valley gets round — the cheapest preconditioner is the one you never need.”

43.10 Initialization: where the weights start

The trap you already know (§41.14(i)). “Why did we not initialize all parameters to zero?” — because “since the incoming weights are the same for all neurons in a layer, there is nothing to differentiate between two neurons... they will evolve identically.” The MLT slides' fix: sample $\theta \sim \mathcal{N}(0, 1)$; the §41.10 class uses $\mathcal{N}(0, 0.5^2)$ (§36.6 on seeding). Randomness breaks the symmetry; that part is settled.

The scale question — what the sources actually support. The deck never names an initialization *scheme* (no Xavier, no He — see the review log). But the scale of the starting weights controls two quantities this book has already derived, so the reasoning below is book-internal, not course-sourced:

- i) **The forward scale.** $z = Wx + b$: if the weights are too large, $|z|$ is large and sigmoid/tanh saturate — the neuron sits on a flat end where nothing changes (§41.7).
- ii) **The backward scale (§42.8(ii)).** The δ recursion multiplies the error by $g'(z^{[l]})$ at every layer. For sigmoid, $g'(z) \leq 0.25$, and at saturation ($|z| \geq 3$) it is ≤ 0.045 — every backward step shrinks the error to at most a twentieth. Too-large initial weights therefore *strangle the gradient before training starts*: the error signal of §42.4 cannot survive the trip back. Too-small weights keep every neuron in the linear region, which survives but learns nothing distinctive — and deep stacks of tiny weights shrink the forward activations layer by layer.

So the initialization problem, in this book's terms: **pick a random scale that keeps z in the region where $g'(z)$ is healthy — neither saturated nor dead — given the fan-in of the layer.** The GenAI W5 materials do not give the formula for that scale; the notebook's troubleshooting table lists “poor initialization” as a cause of “training loss not decreasing” and prescribes “proper initialization” without defining it [recorded]. This is a genuine gap in the course materials, flagged in the review log rather than filled from memory.

Basically, ... “Zero-init is banned (§41.14(i)) — random it is. But *how* random? Too big: neurons saturate, and the §42.8(ii) δ recursion shrinks the error signal to nothing before the first update. Too small: everything stays tiny and timid. The right scale keeps each neuron's z where the activation's slope is alive — the course never writes down the formula, so this chapter states the criterion and flags the gap.”

43.11 The optimizers, traced: numpy on the deck's own problem

Time to watch §43.1–§43.6 happen. The deck's ill-conditioned problem, $f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$ with $\nabla f = (0.2x_1, 4x_2)$, from $x = (-4, 1)$ ($f = 3.6$). All traces below re-run here [verified-NumPy].

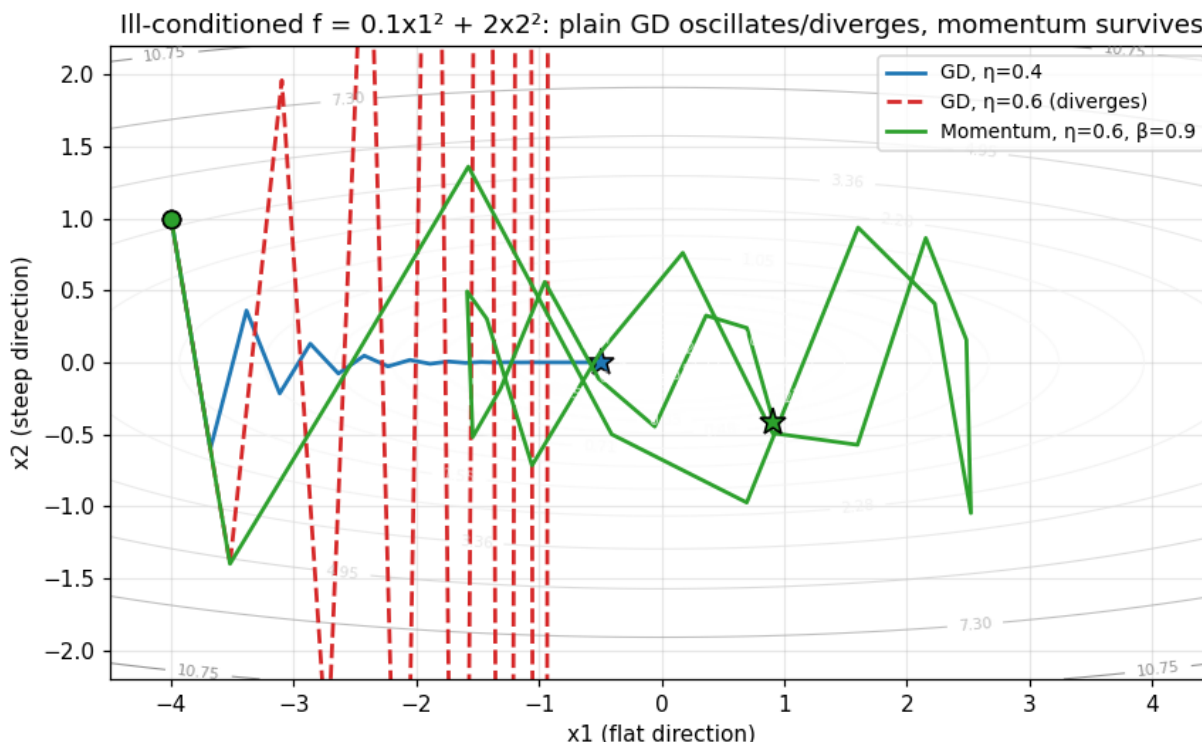


Figure 61: Contours of $f = 0.1x_1^2 + 2x_2^2$ with three optimization trajectories from $(-4,1)$: plain GD with $\eta=0.4$ zig-zags toward the center, plain GD with $\eta=0.6$ shoots off the frame (diverges), momentum with $\eta=0.6$ and $\beta=0.9$ swings widely but survives and heads for the minimum.

eg 4 (plain GD: the oscillation, then the divergence). $\eta = 0.4$:

```
step 0: x=(-4.0000,+1.0000) f=3.6000
step 1: x=(-3.6800,-0.6000) f=2.0742
step 2: x=(-3.3856,+0.3600) f=1.4054
step 3: x=(-3.1148,-0.2160) f=1.0635
step 4: x=(-2.8656,+0.1296) f=0.8547
```

x_2 alternates sign — the ravine zig-zag, decaying by 0.6 per step — while x_1 crawls (factor 0.92). Why: each step multiplies x_1 by $(1 - 0.2\eta) = 0.92$ and x_2 by $(1 - 4\eta) = -0.6$. Now $\eta = 0.6$: the x_2 factor becomes $1 - 2.4 = -1.4$, $|-1.4| > 1$, and

```
step 1: f= 5.1590   step 3: f=15.8021   step 6: f=113.7329
```

— “a high learning rate ($\eta = 0.6$) that caused GD to diverge” (deck, verbatim). One η cannot serve both directions: this is §43.1(i) with numbers.

eg 5 (momentum survives the η that kills GD). Momentum, $\eta = 0.6$, $\beta = 0.9$:

```
step 1: f=5.159040   step 5: f=1.945453   step 10: f=1.958112
step 15: f=1.137399  step 20: f=0.716362   step 25: f=0.423362   step 30: f=0.171884
```

It swings — the velocity overshoots early — but the oscillations cancel instead of compounding, and it converges ($f < 10^{-6}$ at step 120). The deck’s claim, checked: momentum tames the learning rate plain GD cannot survive.

eg 6 (the preconditioner effect, counted in steps). Steps to reach $f < 10^{-6}$ from the same start: - Plain GD, $\eta = 0.4$: **86** steps. - Momentum, $\eta = 0.6$, $\beta = 0.9$: **120** steps (survives the bigger η ; the swings

cost steps). - AdaGrad, $\eta = 2.0$ (the deck's own demo setting): **13** steps — the per-parameter scaling walks almost straight down the valley, “as if the problem were better conditioned.” - RMSProp: $\eta = 0.1 \rightarrow$ **64** steps; $\eta = 0.01$ crawls (still 5.25×10^{-5} after 10,000 steps); $\eta = 0.5$ oscillates ($f = 0.13$ stuck). The notebook's warning, earned: adaptive methods “still require manual LR tuning.” - Adam, $\eta = 0.01$: **1227** steps (the default LR is conservative here; Adam's strength is robustness across problems, not speed on this one).

Note (honest comparison). The deck's story is “momentum fixes GD's oscillation” — the traces refine it: momentum *survives* the η that explodes plain GD, but on this problem the step-count crown goes to AdaGrad's preconditioning. Different fixes for different pathologies: §43.3 fixes direction memory, §43.5 fixes per-parameter scale. Adam does both at once, which is why it is the default — not because it wins every race.

Basically, ... “Same valley, five optimizers, one start. Plain GD zig-zags at $\eta=0.4$ and explodes at $\eta=0.6$. Momentum survives $\eta=0.6$ by averaging the zig-zag away. AdaGrad rescales each direction separately and walks nearly straight to the bottom in 13 steps. Adam is the all-rounder — slower here, reliable everywhere.”

43.12 The practitioner's cheat sheet (notebook, [recorded])

Everything in this section is the *Advanced_Optimization* notebook's tables, transcribed.

Learning-rate starting points.

Optimizer	Typical LR range	Start here
Vanilla GD	0.001–0.1	0.01
Momentum	0.001–0.1	0.01
AdaGrad	0.01–1.0	0.1
RMSProp	0.0001–0.01	0.001
AdaDelta	N/A (adaptive)	default ($\rho = 0.95$)
Adam / AdamW	0.0001–0.01	0.001

Momentum β : 0.9 standard, 0.99 for smooth landscapes. Adam: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. AdamW weight decay: start 0.01.

Which optimizer when.

Scenario	Pick	Why
General deep learning	Adam / AdamW	Best balance of speed and robustness
CNNs (vision)	SGD + momentum, or AdamW	Better generalization, proven track record
NLP / transformers	AdamW	Industry standard; decoupled weight decay
Sparse features	AdaGrad / Adam	Per-parameter rates for rare features
RNNs / LSTMs	RMSProp / Adam	Handles non-stationary objectives
Fine-tuning a pretrained model	AdamW, low LR	Preserves learned features
Convex problem	SGD / Nesterov	Strong convergence guarantees
Fast prototyping	Adam	Works with defaults

Troubleshooting (symptom $\rightarrow$ cause $\rightarrow$ fix).

Problem	Possible causes	Fix
Training loss not decreasing	LR too small, poor initialization	Increase LR, proper init
Loss exploding	LR too large	Decrease LR, gradient clipping
Slow convergence	Wrong optimizer / LR	Try Adam/AdamW, add a schedule
Overfitting	No regularization	AdamW, dropout, augmentation
Oscillating loss	LR too high for vanilla GD	Add momentum or reduce LR
Stuck in a local minimum	No momentum	Momentum-based optimizer
Adam generalizes poorly	L2/weight-decay interaction	Switch to AdamW

The 2025 defaults. Start with AdamW ($\eta = 0.001$, weight decay 0.01). For the best generalization, SGD + momentum + a schedule (cosine annealing, `ReduceLROnPlateau`, warm-up). Clip gradients for RNNs/transformers (`torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`) [recorded — torch API, not re-run]. Mixed precision for large models.

Basically, ... “Prototype with AdamW and its defaults. Chase the last few percent with SGD + momentum + a cosine schedule. When something breaks, read the symptom table before touching the architecture — nine times out of ten it is the learning rate.”

43.13 Where this goes next

- i) **CNNs (Chapter 44).** Same optimizers, new layers. The GenAI W4 material trains convolutions with the machinery of this chapter — nothing here changes, which is the point: the optimizer is layer-agnostic. (Batch normalization, used-but-unexplained in the Week 5 notebook, is the obvious companion topic when convolutions arrive.)
- ii) **RNNs (Chapter 45).** The exploding-gradient half of §42.8’s story: where this chapter scheduled learning rates, Chapter 45 will need gradient *clipping* — the notebook lists it as “essential for RNNs and Transformers.”
- iii) **The debugging playbook rides along.** §40.5(i): “debugging a neural net starts with the same playbook — shapes first, then scale (§37.3’s scaling sermon returns with a vengeance for SGD-trained nets), then shuffle, then the leak audit.” Add this chapter’s row to the playbook: when the loss misbehaves, check η and the schedule before the architecture, and check the feature scales before η .
- iv) **Attention and generative models (Chapters 46–47).** The notebook’s §43.12 defaults — AdamW as the transformer standard, warm-up among the scheduling options — carry over whole: Chapter 46 inherits them without re-deriving.

Problem set

1. **The ravine, by hand.** $f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$, start $(-4, 1)$. (i) Write ∇f and show one GD step multiplies x_1 by $(1 - 0.2\eta)$ and x_2 by $(1 - 4\eta)$. (ii) Compute step 1 by hand for $\eta = 0.4$ and for $\eta = 0.6$. (iii) For what η does the x_2 coordinate start diverging, and why is there no η that is simultaneously fast in x_1 and stable in x_2 ?
2. **Velocity is a leaky average.** (i) Expand $v_t = \beta v_{t-1} + g_t$ ($v_{-1} = 0$) to prove $v_t = \sum_{\tau=0}^t \beta^{t-\tau} g_\tau$. (ii) Prove the weight sum tends to $1/(1 - \beta)$, and evaluate it for $\beta = 0.9$. (iii) If the gradient is a constant g for many steps, what does the momentum update $x_{t+1} = x_t - \eta v_t$ converge to per step, in terms of η , β , g ?
3. **Mini-batch noise.** (i) In one line each: why is one sample’s gradient an unbiased estimate of the full gradient, and why does averaging b of them divide the variance by b ? (ii) The deck says mini-batch is “the de facto standard” — give the computational reason (vectorization) and the statistical reason (your part (i)) in one line each. (iii) Name the downside the notebook lists for very large batches.

4. **AdaGrad vs RMSProp, one step each.** f as in problem 1, at $(-4, 1)$, accumulators starting at 0, $\epsilon = 0$. (i) One AdaGrad step with $\eta = 2.0$: compute s_1 , the per-parameter effective LR, and x_1 . (ii) One RMSProp step with $\eta = 0.1$, $\beta = 0.9$: compute $E[g^2]_1$ and x_1 . (iii) In two lines, say which direction each method stepped harder in, and why that is the preconditioner story.
5. **Adam’s bias correction, with and without.** Scalar, first step, $g_1 = 0.5$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. (i) Compute $v_1, s_1, \hat{v}_1, \hat{s}_1$ and the corrected step as a multiple of η . (ii) Compute the *uncorrected* step $\eta \cdot v_1 / \sqrt{s_1}$ as a multiple of η . (iii) In one line, say what goes wrong in early training without the correction.
6. **Weight decay = L2, until Adam.** (i) Starting from $J_\lambda(\theta) = J(\theta) + \frac{\lambda}{2}\|\theta\|^2$, derive the plain-SGD update and show it equals $(1 - \eta\lambda)\theta_t - \eta g_t$. (ii) Explain in two lines why feeding this same J_λ to Adam does *not* give every parameter the same $(1 - \eta\lambda)$ shrink. (iii) Write AdamW’s update and point at the term that restores the uniform shrink.
7. **Read the schedules.** $\eta_0 = 0.2$, $\lambda = 0.01$, $k = 0.01$, $T_{\max} = 1000$, $\eta_{\min} = 0$, $\eta_{\max} = 0.2$. (i) Compute $\eta(500)$ for exponential, inverse-time, and cosine schedules. (ii) Which schedule hits exactly 0 at $t = 1000$, and why? (iii) sklearn’s ‘invscaling’ is $\eta_0/t^{\text{power}-t}$ (§37.3) — how does it differ from the deck’s $\eta_0/(1 + kt)$, and does the difference matter for large t ?
8. **Init scale through the δ recursion.** A 4-hidden-layer sigmoid net; every hidden neuron starts with $|z| \geq 3$. (i) Bound $g'(z)$ at $|z| = 3$ ($\sigma(3) \approx 0.9526$; show $g'(z) \leq 0.0451$ for $|z| \geq 3$). (ii) Bound how much $\|\delta^{[1]}\|$ shrinks relative to $\|\delta^{[4]}\|$ across the three backward steps (weights ≈ 1 in magnitude). (iii) In two lines, translate this into a rule about how large initial weights may be.
9. **Scaling as preconditioning.** Two features, $x_1 \in [0, 1]$, $x_2 \in [0, 1000]$, squared-error loss on a linear model. (i) In one line, say which direction of the loss valley is steep and why. (ii) Apply **StandardScaler** ($x' = (x - \mu)/\sigma$): what are the new feature ranges, roughly? (iii) In two lines, connect this to the AdaGrad story of §43.5 — what work does scaling do that AdaGrad would otherwise have to?
10. **Triage.** (i) “Training loss not decreasing” — list the notebook’s two possible causes and two fixes. (ii) “Adam generalizes poorly on my fine-tune” — name the cause and the fix from the tables. (iii) You are fine-tuning a transformer: pick the optimizer, LR, and weight decay from §43.12, and give the one-line reason for each choice.

Sources: GenAI Week 5 “Optimization Strategies” deck (Balaji Srinivasan, Ganapathy Krishnamurthi), 78 slides — optimization challenges (ravines, saddle points), convex primer (weight decay as penalty), GD algorithms (batch/SGD/mini-batch theorem, vectorization), LR scheduling (why + five schedules), momentum (theorem, leaky-average derivation, ill-conditioned demo $\eta = 0.6$), AdaGrad/RMSProp/AdaDelta/Adam (theorems, intuitions, drawbacks); Gradient_Descent.ipynb — three GD implementations, batch-size experiment (1–256), LR sensitivity (0.0001–0.1), PyTorch DataLoader vocabulary, manual-vs-PyTorch comparison; Advanced_Optimization.ipynb — optimizer implementations (Vanilla GD, momentum, Nesterov, AdaGrad, RMSProp, AdaDelta, Adam, AdamW), Rosenbrock/Beale/neural-net demos, LR table, selection guide, pitfalls table, 2025 best practices (all torch/sklearn code [recorded], all numeric traces in §43.11 [verified-NumPy]); GenAI Week 5 notes PDF — same deck content; book chapters 10 (§10.5), 21, 28 (ridge λ), 36 (§36.6), 37 (§37.3 scaling + invscaling), 40 (§40.5(i) playbook), 41 (§§41.7, 41.14(i), 41.15(ii)), 42 (§§42.4, 42.8(ii)).

Solutions — Chapter 43: Optimization for neural nets

1. The ravine, by hand

$f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$, start $(-4, 1)$.

(i) $\nabla f = (0.2x_1, 4x_2)$. One GD step $x \leftarrow x - \eta \nabla f$:

$$x_1 \leftarrow x_1 - \eta(0.2x_1) = (1 - 0.2\eta)x_1, \quad x_2 \leftarrow x_2 - \eta(4x_2) = (1 - 4\eta)x_2.$$

(ii) $\eta = 0.4$: $x_1 = -4(1 - 0.08) = -3.68$, $x_2 = 1(1 - 1.6) = -0.60$. $\eta = 0.6$: $x_1 = -4(1 - 0.12) = -3.52$, $x_2 = 1(1 - 2.4) = -1.40$. Both match the §43.11 traces exactly [verified-NumPy].

(iii) The x_2 factor $|1 - 4\eta| > 1$ when $\eta > 0.5$ (or $\eta < 0$) — then $|x_2|$ grows every step: divergence. Fast progress in x_1 wants $(1 - 0.2\eta)$ near 0, i.e. $\eta \approx 5$; stability in x_2 wants $\eta < 0.5$. The two requirements don't overlap — that non-overlap *is* ill-conditioning: no single η is both fast-flat and stable-steep.

2. Velocity is a leaky average

(i) Base: $v_0 = \beta v_{-1} + g_0 = g_0$ (taking $v_{-1} = 0$). Inductive step: assume $v_{t-1} = \sum_{\tau=0}^{t-1} \beta^{t-1-\tau} g_\tau$; then

$$v_t = \beta \sum_{\tau=0}^{t-1} \beta^{t-1-\tau} g_\tau + g_t = \sum_{\tau=0}^t \beta^{t-\tau} g_\tau. \quad \blacksquare$$

(ii) The weights are a geometric series: $\sum_{i=0}^t \beta^i \rightarrow \sum_{i=0}^{\infty} \beta^i = 1/(1 - \beta)$ for $|\beta| < 1$. For $\beta = 0.9$: $1/0.1 = 10$ (partial sum to $k = 60$ is 9.984 [verified-NumPy]).

(iii) $v_t \rightarrow g \sum_{i=0}^{\infty} \beta^i = g/(1 - \beta)$, so the update tends to $x_{t+1} - x_t \rightarrow -\eta g/(1 - \beta)$: momentum with a constant gradient behaves like plain GD with effective learning rate $\eta/(1 - \beta)$ — 10η for $\beta = 0.9$.

3. Mini-batch noise

(i) Sampling i uniformly, $\mathbb{E}[\nabla f_i] = \frac{1}{n} \sum_{i=1}^n \nabla f_i = \nabla f$ — unbiased. For b i.i.d. draws, $\text{Var}(\frac{1}{b} \sum \nabla f_i) = \frac{1}{b^2} \cdot b \text{Var}(\nabla f_i) = \text{Var}/b$.

(ii) Computational: blocks fit in cache and parallelize on CPUs/GPUs (the deck's vectorization slide). Statistical: averaging divides gradient noise by b .

(iii) “May converge to sharp minima (worse generalization)” and needs more memory (notebook's batch-size table [recorded]).

4. AdaGrad vs RMSProp, one step each

At $(-4, 1)$: $g = (0.2 \cdot (-4), 4 \cdot 1) = (-0.8, 4)$.

(i) **AdaGrad**, $\eta = 2.0$. $s_1 = (0.64, 16)$. Effective LR: $2/\sqrt{0.64} = 2/0.8 = 2.5$; $2/\sqrt{16} = 2/4 = 0.5$. Update $= -(2.5 \cdot (-0.8), 0.5 \cdot 4) = (2.0, -2.0)$. New $x = (-4 + 2, 1 - 2) = (-2, -1)$.

(ii) **RMSProp**, $\eta = 0.1$, $\beta = 0.9$. $E[g^2]_1 = 0.1 \cdot (0.64, 16) = (0.064, 1.6)$. Effective LR: $0.1/\sqrt{0.064} \approx 0.3953$; $0.1/\sqrt{1.6} \approx 0.0791$. Update $= -(0.3953 \cdot (-0.8), 0.0791 \cdot 4) \approx (0.3162, -0.3162)$. New $x \approx (-3.6838, 0.6838)$.

(iii) AdaGrad steps the flat direction $5\times$ harder per unit gradient (2.5 vs 0.5); RMSProp does the same (0.3953 vs 0.0791, also $5\times$). That is the preconditioner story: each direction's step is divided by its own typical gradient size, so the $20\times$ scale gap between the directions is cancelled instead of fought.

5. Adam's bias correction, with and without

(i) $v_1 = 0.1 \cdot 0.5 = 0.05$; $s_1 = 0.001 \cdot 0.25 = 0.00025$. $\hat{v}_1 = 0.05/0.1 = 0.5$; $\hat{s}_1 = 0.00025/0.001 = 0.25$. Corrected step $= \eta \cdot 0.5/\sqrt{0.25} = 1.0\eta$.

(ii) Uncorrected: $\eta \cdot 0.05/\sqrt{0.00025} = \eta \cdot 0.05/0.015811 \approx 3.162\eta$.

(iii) Without correction the first steps are inflated by a factor ($3.16\times$ here) that comes from the zero initialization of the moments, not from the data — early training overshoots for no reason.

6. Weight decay = L2, until Adam

(i) $\nabla J_\lambda = \nabla J + \lambda\theta = g_t + \lambda\theta_t$. Plain SGD:

$$\theta_{t+1} = \theta_t - \eta(g_t + \lambda\theta_t) = (1 - \eta\lambda)\theta_t - \eta g_t. \quad \blacksquare$$

(ii) Adam divides the *entire* gradient — including the $\lambda\theta_t$ piece — by $\sqrt{\hat{s}_t} + \epsilon$, which is different for every parameter. So the $\lambda\theta_t$ shrink is no longer the uniform $(1 - \eta\lambda)$ factor; heavily-updated parameters get less decay, rarely-updated ones more.

(iii) AdamW: $\theta_{t+1} = \theta_t - \eta(\hat{v}_t/(\sqrt{\hat{s}_t} + \epsilon) + \lambda\theta_t) = (1 - \eta\lambda)\theta_t - \eta\hat{v}_t/(\sqrt{\hat{s}_t} + \epsilon)$. The $+\lambda\theta_t$ sits *outside* the adaptive fraction — that term restores the uniform per-step shrink.

7. Read the schedules

(i) $\eta(500)$: exponential = $0.2e^{-0.01 \cdot 500} = 0.2e^{-5} \approx 0.2(0.006738) \approx 0.0013$. Inverse time = $0.2/(1 + 0.01 \cdot 500) = 0.2/6 \approx 0.0333$. Cosine = $0 + \frac{1}{2}(0.2)(1 + \cos(\pi \cdot 500/1000)) = 0.1(1 + \cos(\pi/2)) = 0.1(1 + 0) = 0.1$.

(ii) Cosine: at $t = T_{\max}$, $\cos(\pi) = -1$, so $\eta = \eta_{\min} + 0 = 0$ exactly — $T_{\max}$ is defined as the horizon where the curve lands on $\eta_{\min}$.

(iii) sklearn: η_0/t^p ; deck: $\eta_0/(1 + kt)$. sklearn's diverges at $t = 0$ (implementations index steps from $t \geq 1$ or equivalent); the deck's equals η_0 at $t = 0$. For large t with $p = k = 1$: η_0/t vs $\eta_0/(kt)$ — the same $1/t$ tail up to the constant; the difference is negligible.

8. Init scale through the δ recursion

(i) $g'(z) = \sigma(z)(1 - \sigma(z))$, symmetric in z , maximal 0.25 at $z = 0$, decreasing as $|z|$ grows. At $|z| = 3$: $\sigma(3) \approx 0.9526$, so $g'(3) \approx 0.9526 \cdot 0.0474 \approx 0.0451$; for $|z| \geq 3$, $g'(z) \leq 0.0451$.

(ii) $\delta^{[l]} = ((W^{[l+1]})^T \delta^{[l+1]}) \odot g'(z^{[l]})$ (§42.4). With $\|W\| \approx 1$ and $g' \leq 0.0451$ at each of the three backward steps from layer 4 to layer 1:

$$\|\delta^{[1]}\| \lesssim (0.0451)^3 \|\delta^{[4]}\| \approx 9.2 \times 10^{-5} \|\delta^{[4]}\|.$$

The error signal is crushed to $\sim 10^{-4}$ of itself before reaching the first layer — before training has even started.

(iii) Rule: initial weights must be small enough that pre-activations stay where the activation's slope is healthy — $|z| \gtrsim 3$ everywhere in a sigmoid stack means the gradients are dead on arrival. (The course gives no formula for that scale — the criterion, not the constant, is the takeaway; see §43.10's flagged gap.)

9. Scaling as preconditioning

(i) The x_2 direction: its gradients are $\sim 1000\times$ larger, so the loss surface is $1000\times$ steeper along x_2 — the same ravine as §43.1(i), built from units.

(ii) Roughly: both features end up mean 0, standard deviation 1, spanning about $[-3, 3]$ each — the $1000\times$ gap is gone.

(iii) AdaGrad would discover the scale gap through its accumulators (s_t growing $10^6\times$ faster for x_2) and compensate per-parameter; **StandardScaler** removes the gap before the optimizer runs, so no compensation is needed. Scaling is the preconditioner you apply once to the data instead of every step to the gradients.

10. Triage

(i) Causes: learning rate too small; poor initialization. Fixes: increase the LR; use proper initialization (notebook's pitfalls table [recorded]).

(ii) Cause: Adam's interaction with L2/weight decay — the decay goes through the adaptive scaling and isn't true weight decay. Fix: switch to AdamW.

(iii) AdamW, LR 0.001 (low, to preserve the pretrained features), weight decay 0.001 (the notebook's fine-tuning row: 0.0001–0.001). AdamW is the recorded industry standard for transformers because its decoupled decay regularizes uniformly; the low LR keeps fine-tuning from destroying what pretraining learned.

Chapter 44: CNNs — convolution, pooling, transfer learning

Chapter 41 built the neuron and the MLP; Chapter 42 derived the gradients; Chapter 43 picked the optimizers. This chapter changes what a *layer* computes. Nothing else changes: the optimizer is layer-agnostic (Chapter 43’s point), and backprop is the same machinery with new local gradients (§42.4). The spine is the GenAI Weeks 3–4 deck (“Introduction to CNN,” Balaji Srinivasan and Ganapathy Krishnamurthi, 91 slides — vision modelling, convolution vs cross-correlation, stride/padding illustrations, pooling, batch normalization walkthrough, famous architectures, transfer learning) plus its three companion notebooks: *CNN_Fundamentals* (why MLPs fail on images, the convolution definition, pooling, a complete MNIST/Fashion-MNIST CNN in PyTorch), *Transfer_Learning* (feature extraction vs fine-tuning on FoodVision Mini with EfficientNet-B0), and *Image_Segmentation_UNET* (U-Net, used briefly in §44.9). Every numpy number in this chapter was re-run here and is marked [verified-NumPy]; deck, notebook, and torch numbers are transcriptions, marked [recorded].

Notation. I = input image (or feature map), K = convolution kernel (filter), $(I * K)$ = output feature map. Image/feature-map shape is written $H \times W \times C$ (height, width, channels), the notebook’s convention — §36.4’s axis semantics apply to every shape question below.

44.1 Why dense layers fail on images

The *CNN_Fundamentals* notebook’s case against feeding images to MLPs has four counts:

- i) **Parameter explosion.** “A single $224 \times 224 \times 3$ color image has 150,528 pixels. If the first hidden layer has just 100 neurons, we’d need 15,052,800 parameters (weights) just for the first layer!” [recorded]. Set it next to §41.14(ii)’s $784 \rightarrow 50 \rightarrow 10$ MNIST net, where “just 1 hidden layer” already had “nearly 40,000 parameters” — §41.14(ii)’s overfitting warning was about a model almost four hundred times smaller than this dense vision layer.
- ii) **Spatial information loss.** “MLPs flatten images into 1D vectors, destroying spatial relationships. The fact that adjacent pixels are related in an image is completely ignored” [recorded]. The deck’s version: the naive approach is “flatten the image... feed into MLP” — a 28×28 grid becomes a 784×1 vector and the network never learns that pixel (i, j) sits next to pixel $(i, j+1)$.
- iii) **Translation variance.** “MLPs don’t handle translations well — an object shifted slightly in an image appears as a completely different input. A dog in the left corner vs. center would require learning separate representations” [recorded]. Every position needs its own copy of every pattern.
- iv) **Data inefficiency.** With no built-in inductive bias for images, the MLP “require[s] enormous datasets” and is “prone to overfitting with limited training data” [recorded].

The CNN fixes all four at once: a small *learnable filter*, slid across the whole image, with the *same weights reused at every position*.

Basically, ... “An MLP sees a photo as a flat list of 150,528 numbers — no neighbours, no ‘left of’. One

hidden layer of 100 neurons already costs 15 million weights, and a dog in the corner must be learned separately from a dog in the center. CNNs instead learn small pattern-detectors and slide them everywhere, so one edge detector finds edges at every position.”

44.2 Convolution as local pattern matching

Def (the notebook’s). “The convolution operation involves sliding a small filter (or kernel) across an input image and computing element-wise multiplications followed by summation.” For a 2D input I and kernel K :

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n) \quad [\text{recorded}].$$

Each output pixel is the dot product of the kernel with one small patch of the input; move the window one step, repeat. The output is called a **feature map** (or activation map): at each position it says *how strongly the kernel’s pattern matched there*.

A terminology note the deck insists on. Deep learning’s “convolution” is, mathematically, **cross-correlation**: a true convolution flips the kernel first ($I(i - m, j - n)$); DL implementations never flip ($I(i + m, j + n)$). The deck’s slide is blunt: “What we use is technically called cross-correlation,” and then asks “Does it matter?” The honest answer: **no**, because the kernel weights are *learned*. A flipped copy of a learned kernel is just another learnable kernel — the network finds the same patterns either way. From here on this chapter says “convolution” in the deep-learning sense and means cross-correlation.

Three ideas travel with the operation:

- i) **Local receptive field.** “Each neuron in a convolutional layer connects only to a small region of the input” [recorded]. A neuron in a 3×3 layer sees 9 pixels, not 150,528 — the locality the MLP threw away is back.
- ii) **Learnable filters.** “The values in each filter are learnable parameters updated during training” [recorded]. The deck’s learning rule is the same gradient-descent step as ever: $W \leftarrow W - \eta \partial \mathcal{L} / \partial W$, “repeated for every image and for multiple iterations till the loss converges” [recorded]. The filters “evolve to detect features (edges, colors, textures, patterns) that are most useful for the specific task” [recorded] — nobody hand-designs the edge detector; §42.4’s backprop machinery computes $\partial \mathcal{L} / \partial W$ for a conv layer exactly as it did for a dense one.
- iii) **Weight sharing.** “The same set of weights is used regardless of where in the image a feature appears” [recorded] — this is the engine of everything else: parameters (§44.7), translation behaviour (§44.5), and data efficiency all follow from one filter being applied everywhere.

Basically, ... “Put a small stencil over the top-left of the image, multiply its numbers with the pixels underneath, add up — that’s one output pixel. Slide the stencil right, repeat; then down, repeat. The full output says, at every position, ‘how much does my stencil match here?’ The stencil’s numbers are learned like any weights. (Pedant corner: this is cross-correlation, not a true convolution — but since the stencil is learned, the missing flip changes nothing.)”

44.3 eg: a 2D convolution, worked by hand [verified-NumPy]

Input I (4×4), kernel K (2×2), stride 1, no padding:

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}.$$

Position (0,0): the top-left 2×2 patch is $\begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix}$, element-wise with K :

$$y_{0,0} = 1 \cdot 1 + 2 \cdot 0 + 5 \cdot (-1) + 6 \cdot 1 = 1 - 5 + 6 = 2.$$

Position (0,1): patch $\begin{bmatrix} 2 & 3 \\ 6 & 7 \end{bmatrix}$, $y_{0,1} = 2 - 6 + 7 = 3$. Position (0,2): $y_{0,2} = 3 - 7 + 8 = 4$. Row 1: $y_{1,0} = 5 - 9 + 10 = 6$, $y_{1,1} = 6 - 10 + 11 = 7$, $y_{1,2} = 7 - 11 + 12 = 8$. Row 2: $y_{2,0} = 9 - 13 + 14 = 10$, $y_{2,1} = 10 - 14 + 15 = 11$, $y_{2,2} = 11 - 15 + 16 = 12$. Re-run in numpy:

```
[[ 2.  3.  4.]
 [ 6.  7.  8.]
 [10. 11. 12.]]
```

The output is 3×3 — smaller than the 4×4 input, because the 2×2 window only fits in 3 positions per axis. The next section names that rule.

44.4 Channels, stride, padding: the shape bookkeeping

Every shape question about a conv layer is §36.4's axis semantics applied to $H \times W \times C$.

Channels. A filter is as deep as its input. On a $28 \times 28 \times 1$ grayscale image a 3×3 filter has $3 \times 3 \times 1 = 9$ weights; on a $224 \times 224 \times 3$ color image it has $3 \times 3 \times 3 = 27$. The kernel multiplies all three channel-planes at each position and sums to one number — so each *filter* produces one 2D feature map, and F filters produce an $H' \times W' \times F$ output. The notebook's numbers: “for a single 3×3 convolutional filter: only 9 weights (plus 1 bias)” [recorded] — that is per input channel; the +1 bias is per filter, added to every position of its feature map.

Stride. How many pixels the window moves per step. The deck's illustration: 5×5 input, 3×3 kernel, stride 2 $\rightarrow 2 \times 2$ output [recorded].

Padding. Zeros added around the border so the kernel can cover the edge pixels. The deck's illustration: 4×4 input, 3×3 kernel, no padding (“valid”) $\rightarrow 2 \times 2$ output — “(Output shrinks)” [recorded].

Theorem (output size, per axis). With input width W , kernel K , padding P per side, stride S :

$$W_{\text{out}} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1 \quad [\text{verified-NumPy}].$$

Why: the first window starts at pixel 0, the last at pixel $W - K + 2P$ (counting padding), in steps of S — $(W - K + 2P)/S + 1$ positions, floored. **“Same” padding** picks P so the output keeps the input size: for stride 1 and odd K , $P = (K - 1)/2$ (e.g. 3×3 with $P = 1$ on $28 \times 28 \rightarrow 28 \times 28$). **“Valid”** is $P = 0$: the size shrinks by $K - 1$.

eg 4 (the formula against the course's own diagrams). - Deck's padding slide: $W = 4$, $K = 3$, $P = 0$, $S = 1 \rightarrow \lfloor (4 - 3)/1 \rfloor + 1 = 2$. Output 2×2 — the slide's number. - Deck's stride slide: $W = 5$, $K = 3$, $P = 0$, $S = 2 \rightarrow \lfloor (5 - 3)/2 \rfloor + 1 = 2$. Output 2×2 — the slide's number. - Notebook's MNIST net: Conv1 (3×3 , stride 1, valid) on $28 \times 28 \rightarrow 26 \times 26$; 32 filters $\rightarrow 26 \times 26 \times 32$. Same-padding version would stay 28×28 .

Note. Filter sizes are “typically odd numbers like $3 \times 3, 5 \times 5, 7 \times 7$ ” because “an odd size provides a center pixel” [recorded] — symmetric padding around a center is what makes “same” padding exact.

Basically, ... “The kernel is as deep as the input ($3 \times 3 \times 3$ on color), and one filter = one output sheet, so F filters give F sheets. The output shrinks whenever the kernel is bigger than a pixel and has no padding: $(W - K + 2P)/S + 1$ positions per side. Want to keep the size? Pad one ring of zeros ($P=1$ for a 3×3). Want to shrink fast? Stride 2.”

44.5 Pooling: downsampling with a purpose

Def (the deck’s). “A pooling layer performs non-linear downsampling to reduce the spatial dimensions (width and height) of the feature maps.” Its two key purposes: “Reduce Computational Cost” and “Increase Receptive Field — allows subsequent convolutional layers to see a larger area of the original input” [recorded].

The notebook’s three types:

- i) **Max pooling.** “Takes the maximum value from a local region. Most commonly used in practice. Helps extract the most prominent features” [recorded]. Has no learned parameters.
- ii) **Average pooling.** “Takes the average value from a local region. Preserves more background information” [recorded].
- iii) **Global pooling.** “Reduces each feature map to a single value (max or average)” [recorded] — a $H \times W \times F$ stack becomes one F -vector, often right before the dense layers.

Typical settings: 2×2 window, stride 2 — “halves both dimensions” [recorded]; padding is “rarely used in pooling layers” [recorded].

eg 5 (the notebook’s pooling diagram, [verified-NumPy]). 2×2 max pooling, stride 2, on

$$\begin{bmatrix} 1 & 3 & 2 & 5 \\ 9 & 7 & 8 & 6 \\ 4 & 2 & 7 & 3 \\ 1 & 0 & 5 & 2 \end{bmatrix} \rightarrow \begin{bmatrix} 9 & 8 \\ 4 & 7 \end{bmatrix} :$$

top-left window $\{1, 3, 9, 7\} \rightarrow 9$; top-right $\{2, 5, 8, 6\} \rightarrow 8$; bottom-left $\{4, 2, 1, 0\} \rightarrow 4$; bottom-right $\{7, 3, 5, 2\} \rightarrow 7$. Pooling asks “what was the strongest response in this patch?” and throws away *exactly where* — that is the mechanism behind “invariance to small translations” [recorded].

What pooling buys, in three lines.

- i) **Translation robustness.** Convolution is *equivariant* — “if a feature moves in the input, its representation moves correspondingly in the feature maps” [recorded]. Pooling then summarizes each patch, which *enhances translational invariance* [recorded]: the exact position matters less after a max.
- ii) **Downsampling.** Halving H and W quarters the computation of every later layer, and (§44.7’s point) keeps the parameter count in check.
- iii) **Bigger receptive fields downstream.** After pooling, the next conv layer’s 3×3 window covers twice as much of the original image — deep layers see larger patterns built from the small ones.

Note (pooling vs PCA, §24’s contrast). Both shrink representations, but by opposite means. PCA (§24) *learns* a global linear projection from the data’s covariance — optimal reconstruction, but it flattens everything (§24.5) and knows nothing about 2D layout. Pooling is a *fixed, local, nonlinear* summary that preserves the spatial grid. Pooling throws away position-within-patch; PCA throws away low-variance directions. Different losses, different keepers.

Basically, ... “Pooling is the network deliberately squinting: take each 2×2 patch, keep only its loudest pixel (max) or its average. It shrinks the map (cheap!), makes exact positions matter less (robust!), and lets deeper layers see bigger picture chunks through the same small window. Compared to Chapter 24’s PCA: PCA learns one global linear squish from the data; pooling is a fixed local squish that keeps the 2D layout.”

44.6 The CNN stack: feature hierarchy, then a dense head

The notebook’s “standard layer sequence,” with its own MNIST example traced end to end:

- i) **Input:** $H \times W \times C$ raw pixels ($28 \times 28 \times 1$ grayscale, $224 \times 224 \times 3$ color).
- ii) **Conv + activation:** filters extract features, “typically ReLU” [recorded] for the non-linearity (§41.7’s activation logic applies unchanged).
- iii) **Pool:** downsample.
- iv) **Repeat** — conv→pool blocks stacked. The design pattern: “increase filters, decrease spatial dimensions. As we go deeper, number of filters increases while spatial dimensions decrease” [recorded].
- v) **Flatten → dense → output:** “converts 2D feature maps to 1D feature vector” [recorded], then “traditional neural network layers” for the final prediction — §41.4’s two equations ($Z^l = A^{l-1}W^l + b^l$, $A^l = g(Z^l)$) are back, with dropout ($p = 0.5$ in the example, §41.14(ii)’s dropout) between dense layers.

eg 6 (the notebook’s MNIST architecture, shapes traced — [verified-NumPy]).

```
Input: 28x28x1
Conv1: 3x3, 32 filters, stride 1, valid -> 26x26x32
Pool1: 2x2, stride 2 -> 13x13x32
Conv2: 3x3, 64 filters, stride 1, valid -> 11x11x64
Pool2: 2x2, stride 2 -> 5x5x64
Conv3: 3x3, 128 filters, stride 1, valid -> 3x3x128
Flatten -> 1152
FC1: 128, ReLU; Dropout 0.5; FC2: 10, softmax
```

Every arrow is §44.4’s formula: $(28-3)/1+1 = 26$, $(26-2)/2+1 = 13$, $(13-3)/1+1 = 11$, $(11-2)/2+1 = 5$, $(5-3)/1+1 = 3$, $3 \times 3 \times 128 = 1152$.

What each level *learns* is the hierarchy the notebook states plainly: “early layers: detect simple features (edges, corners); middle layers: textures, patterns; later layers: complex objects, parts” [recorded]. The notebook backs it with feature-map and filter visualizations — free from the architecture: first-layer filters are readable directly because they act on raw pixels.

Basically, ... “Stack conv+ReLU+pool blocks: each block shrinks the map and doubles down on features, early blocks finding edges, late blocks finding parts of objects. Then flatten and hand the F -vector to an ordinary dense network (§41.4’s equations, dropout included) for the final call. The traced MNIST stack: $28 \times 28 \rightarrow 26 \times 26 \times 32 \rightarrow 13 \times 13 \times 32 \rightarrow 11 \times 11 \times 64 \rightarrow 5 \times 5 \times 64 \rightarrow 3 \times 3 \times 128 \rightarrow 1152 \rightarrow 128 \rightarrow 10$.”

44.7 The parameter-sharing payoff

The notebook does the count twice, and both are worth keeping:

- i) **64 filters on 28×28 .** “For 64 filters: CNN = 640 parameters vs MLP = 50,176 parameters” [recorded]: CNN = $64 \times (3 \times 3 + 1) = 640$ (the +1 is the bias per filter); MLP = $784 \times 64 = 50,176$ (one weight per input–neuron pair). Roughly an $80\times$ saving — and the CNN’s count does not grow with image size.
- ii) **The big one, re-run here.** A $224 \times 224 \times 3$ image into 100 hidden neurons costs the dense layer $224 \cdot 224 \cdot 3 \cdot 100 + 100 = 15,052,900$ parameters [verified-NumPy]. The same first layer as a 64-filter 3×3 conv on 3 channels: $64 \cdot (3 \cdot 3 \cdot 3 + 1) = 1,792$ parameters [verified-NumPy]. A $8,400\times$ reduction — that is the whole of §44.1(i) answered by one subtraction.

Why sharing is legal. A filter that detects a vertical edge in the top-left detects the same vertical edge in the bottom-right — “the same feature detector applied across entire image” [recorded]. §41.14(ii) counted parameters to warn about overfitting; convolution is the structural answer to that warning: fewer parameters, and they mean the same thing everywhere.

What does not change. The deck’s transfer-learning slides train these layers with the machinery of Chapter 43 — the optimizer is layer-agnostic (Chapter 43’s closing point). And §43.9’s flagged gap is now partly closed: the Weeks 3–4 deck *does* walk through batch normalization — normalize each activation, $\hat{x}_i = (x_i - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$, then scale-and-shift $y_i = \gamma \hat{x}_i + \beta$ with two learned parameters [recorded; deck slide

54]. The full BN treatment (why γ, β restore representational power, train/test behaviour) stays a candidate for its own note.

Basically, ... “Same patterns, fewer numbers. A 3×3 filter has 9 weights no matter how big the image is, and one edge detector works at every position — so a conv layer that replaces a 15-million-parameter dense layer needs under two thousand. That is §41.14(ii)’s overfitting warning being answered by the architecture itself. (Bonus: the CNN deck’s slide 54 fills in the batch-norm formula §43.9 said was missing.)”

44.8 Transfer learning: don’t reinvent the wheel

Def (the deck’s). “Transfer Learning is the process of taking a model pre-trained on a large dataset and fine-tuning it for a new, specific task.” The motivation: “Training a large CNN from scratch requires massive datasets (like ImageNet) and huge computational resources” [recorded]. The notebook’s framing: pre-trained weights are “superior initialization points for optimization” (§43.10’s initialization problem, solved by borrowing someone else’s answer).

When it works. Early conv layers learn generic features — edges, colors, textures — that are useful for almost any vision task; only the late layers are task-specific. So the bottom of the network transfers across domains, and you only teach the top what your classes look like. The standard backbone: “a network (like ResNet50 or MobileNet) that has been pre-trained on the massive ImageNet dataset (1.4 million images, 1000 classes)” [recorded].

The three strategies (notebook’s table, [recorded]).

Strategy	Method	Use case
Feature extraction	Freeze pre-trained layers, train only classifier	Small target datasets
Fine-tuning	Update all parameters with reduced learning rates	Moderate target datasets
Progressive fine-tuning	Gradual unfreezing of network layers	Large target datasets

“To *freeze* layers means to keep them how they are during training” — in PyTorch, `requires_grad=False`, “so PyTorch doesn’t track gradient updates and these parameters won’t be changed by our optimizer” [recorded].

eg 7 (the notebook’s recipe, [recorded] torch). The *Transfer Learning* notebook classifies pizza/steak/sushi (FoodVision Mini) with an ImageNet-pretrained **EfficientNet-B0**:

1. `weights = EfficientNet_B0_Weights.DEFAULT, model = efficientnet_b0(weights=weights).`
2. The model has three parts: `features` (“a collection of convolutional layers... to learn a base representation of vision data”), `avgpool` (“takes the average of the output of the `features` layer(s) and turns it into a feature vector”), `classifier` (turns the feature vector into class scores — `out_features=1000` for ImageNet).
3. Freeze `features` (`requires_grad=False`) — the feature-extraction strategy.
4. Replace `classifier` with a new head for 3 classes (the old head is exactly §44.6(v)’s dense head, rebuilt for the new task).
5. Feed it ImageNet-preprocessed inputs: 224×224 resize, normalize with ImageNet mean `[0.485, 0.456, 0.406]` and std `[0.229, 0.224, 0.225]` [recorded] — “the custom data going into the model should be prepared in the same way as the original training data.”
6. Train the head with Adam ($\eta = 0.001$, Chapter 43’s defaults) on the small dataset.

The notebook’s one-line rule. The “custom data” warning is the whole discipline: a pre-trained backbone only helps if its input distribution matches training — wrong normalization silently degrades the transferred features.

Basically, ... “Don’t train from scratch on a mountain of images when someone already did it on ImageNet. Take their conv stack (it already knows edges and textures), freeze it (set `requires_grad=False`), bolt on your own small classifier for your classes, and train just that head — on your tiny dataset, with Adam at the usual settings. When you have more data, unfreeze and fine-tune the whole thing with a small learning rate. Rule: preprocess your images exactly like ImageNet’s, or the borrowed features go stale.”

44.9 A second head for CNNs: segmentation (the U-Net note)

The *Image_Segmentation_UNET* notebook applies this chapter’s layers to a different output shape: **segmentation** assigns a class to *every pixel* (the notebook’s case study is leaf-disease segmentation), not one label to the whole image. Its **U-Net** is an encoder (the §44.6 conv+pool stack, shrinking) followed by a decoder (upsampling back to full resolution), with **skip connections** joining matching resolutions — early fine-grained maps guide the upsampling. Evaluate with the Dice coefficient, not accuracy [recorded]. The architecture detail is kept for a later pass; the takeaway now: conv and pooling are Lego for more than classification — reshape the head, change the loss, and the same stack segments.

44.10 Where this goes next

- i) **RNNs (Chapter 45).** The same idea, one new dimension: weight sharing across *time* instead of *space*. Chapter 44’s filter is reused at every pixel; Chapter 45’s cell is reused at every timestep — and the price of sharing (vanishing gradients, §42.8’s other half) arrives with it.
- ii) **Attention (Chapter 46).** The hierarchy of §44.6 — local patterns building up to global ones — gets a shortcut: attention lets any position talk to any other directly, instead of waiting for pooling to bring them together.
- iii) **Generative models (Chapter 47).** GANs and diffusion models are built on conv stacks like this chapter’s — the discriminator is a CNN classifier, and the generator is the §44.9 decoder run in reverse.
- iv) **The debugging playbook rides along.** §40.5(i): “shapes first” — for a CNN that means running every layer through §44.4’s formula before training, exactly as eg 6 did. Wrong flatten size is the transpose bug’s (§41.4) convolutional cousin.

Problem set

1. **Parameter explosion, quantified.** A $224 \times 224 \times 3$ image feeds a dense layer with 100 neurons. (i) Count the weights and biases. (ii) Replace it with a conv layer of $64 \ 3 \times 3$ filters: count weights and biases. (iii) Compare with §41.14(ii)’s 39,760 and say in one line what this implies about overfitting risk.
2. **The convolution, by hand.** $I = \begin{bmatrix} 1 & 0 & 2 \\ 3 & 1 & 1 \\ 0 & 2 & 1 \end{bmatrix}$, $K = \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix}$, stride 1, no padding. (i) Compute the 2×2 output entry by entry. (ii) Which input position contributes to all four output entries, and why does that matter for “local pattern matching”? (iii) Add one learned bias b (same for all four output positions): how many learned parameters does this layer have?
3. **Name the operation.** (i) Write the true-convolution formula (flipped kernel) and the deep-learning formula side by side. (ii) In two lines, explain why the missing flip does not hurt a network whose kernels are learned. (iii) The deck asks “Does it matter?” — give your one-line verdict.
4. **Stride and padding.** (i) $W = 7$, $K = 3$, $P = 0$, $S = 2$: compute W_{out} . (ii) Same W, K, S with $P = 1$: compute W_{out} . (iii) A 32×32 image passes through conv (3×3 , stride 1, “same”) then 2×2 max-pool (stride 2): give the feature-map size after each step.

5. **Trace the MNIST stack.** (i) Reproduce eg 6’s $28 \rightarrow 26 \rightarrow 13 \rightarrow 11 \rightarrow 5 \rightarrow 3$ shape chain from the output-size formula. (ii) Why does MaxPool2 give 5 and not 5.5? (iii) If Conv2 used 128 filters instead of 64, how would the flatten size change, and how many weights would Conv2 then have (per $3 \times 3 \times C_{\text{in}} + 1$)?
6. **Pooling vs PCA.** (i) Pooling a 4×4 map with 2×2 max-pool gives 2×2 : how many numbers are discarded, and what information exactly is lost? (ii) In two lines, contrast with §24’s PCA: what does PCA keep that pooling throws away, and vice versa? (iii) Name one task where you’d pick PCA over pooling, and one where you’d pick pooling over PCA.
7. **Equivariance, then invariance.** (i) In one line each, define translation *equivariance* and translation *invariance*. (ii) Say which one a conv layer has and which one pooling adds, in the notebook’s terms. (iii) A cat photo is shifted right by 10 pixels: trace what happens to the conv feature map and then to the pooled map, in words.
8. **Freeze vs fine-tune.** (i) In the notebook’s three-row table: which strategy for 200 training images, and why? (ii) What does `requires_grad=False` on `features` do to the §42.4 backward pass, mechanically? (iii) Why does the notebook insist on ImageNet normalization of the *new* dataset?
9. **The transfer-learning head.** EfficientNet-B0’s `classifier` outputs 1000 scores. (i) For FoodVision Mini (3 classes), what shape must the replacement head have? (ii) Name the model-construction call and the freeze mechanism used in eg 7. (iii) If you unfreeze everything with the same $\eta = 0.001$, name the risk (Chapter 43’s vocabulary) and the fix (fine-tuning row of the table).
10. **Design a small CNN.** Input $64 \times 64 \times 3$, classes = 10. (i) Propose conv/pool blocks ending with a flatten size ≤ 2000 , showing the shape chain. (ii) Count the conv parameters (assume 3×3 filters, +1 bias each). (iii) Name the §41.14(ii) guardrail you would add before the output layer, and why.

Sources: GenAI Weeks 3–4 “Introduction to CNN” deck (Balaji Srinivasan, Ganapathy Krishnamurthi), 91 slides — naive-MLP-flatten slide, translation-invariance principle, convolution-vs-cross-correlation terminology slide, edge-detection slide, kernel-learning slide ($W \leftarrow W - \eta \partial \mathcal{L} / \partial W$), kernel-size note (odd sizes), stride illustration (5×5 , 3×3 , $S = 2 \rightarrow 2 \times 2$), padding illustration (“valid” 4×4 , $3 \times 3 \rightarrow 2 \times 2$, “output shrinks”), pooling definition + purposes, batch-normalization walkthrough (slide 54), data augmentation, ImageNet challenge, ResNet degradation, transfer-learning slides (“don’t reinvent the wheel”, ResNet50/MobileNet backbone, ImageNet 1.4M/1000 classes); CNN_Fundamentals.ipynb — MLP limitations (§44.1’s four counts, 15,052,800 parameters), convolution definition + hyperparameters, weight sharing (64 filters: 640 vs 50,176), equivariance vs invariance, pooling types + worked $4 \times 4 \rightarrow 2 \times 2$ example, MNIST CNN architecture and shape chain, Fashion-MNIST training, feature-map/filter visualizations; Transfer_Learning.ipynb — transfer objective framing, three-strategy table, freeze mechanics (`requires_grad=False`), EfficientNet-B0 anatomy (features/avgpool/classifier), ImageNet preprocessing (means, stds), FoodVision Mini recipe (all torch code [recorded]); Image_Segmentation_UNET.ipynb — U-Net encoder/decoder + skip connections, leaf-disease case study, Dice coefficient (brief §44.9); Week3-CNN.pdf and Week4-CNN notes — mirror of the deck; book chapters 24 (§24.5), 36 (§36.3–§36.4), 40 (§40.5(i)), 41 (§§41.4, 41.7, 41.14(ii)), 42 (§42.4), 43 (layer-agnostic optimizers, §43.9–§43.10, §43.12 defaults).

Chapter 44 solutions

Full worked solutions for the Chapter 44 problem set. (Problems live in `chapters/44-cnns.md`; never inline.) All numpy checks below were re-run here and are marked [verified-NumPy]; course numbers transcribed as [recorded].

Solution 1 — Parameter explosion, quantified

- (i) Dense layer: weights $224 \cdot 224 \cdot 3 \cdot 100 = 15,052,800$, biases 100. Total 15,052,900 [verified-NumPy].
- (ii) Conv layer: each filter is $3 \times 3 \times 3 = 27$ weights (as deep as the 3 input channels) plus 1 bias; 64 filters: $64 \cdot (27 + 1) = 1,792$ [verified-NumPy].

- (iii) §41.14(ii) already warned that 39,760 parameters made a one-hidden-layer MNIST net “prone to overfitting.” The dense vision layer is $378\times$ that; the conv layer is $22\times$ *smaller* than the MNIST net. Parameter count alone says the dense version will memorize, not generalize — the conv architecture is the structural answer to §41.14(ii)’s warning.

Solution 2 — The convolution, by hand

$$I = \begin{bmatrix} 1 & 0 & 2 \\ 3 & 1 & 1 \\ 0 & 2 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix}, \quad \text{stride 1, valid} \Rightarrow 2 \times 2 \text{ output.}$$

- (i) Entry by entry:

- $y_{0,0}$: patch $\begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix} \rightarrow 1 \cdot 1 + 0 \cdot 1 + 3 \cdot 0 + 1 \cdot (-1) = 1 - 1 = 0.$
- $y_{0,1}$: patch $\begin{bmatrix} 0 & 2 \\ 1 & 1 \end{bmatrix} \rightarrow 0 + 2 + 0 - 1 = 1.$
- $y_{1,0}$: patch $\begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \rightarrow 3 + 1 + 0 - 2 = 2.$
- $y_{1,1}$: patch $\begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} \rightarrow 1 + 1 + 0 - 1 = 1.$

Output $\begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix}$ [verified-NumPy].

- (ii) The center pixel $I_{1,1} = 1$ sits in all four 2×2 windows, so it contributes to all four outputs (corners contribute to one each, edge-middles to two). That overlap is the mechanism of “local pattern matching”: neighbouring output positions share evidence, so a pattern is judged by consensus across nearby patches, not by one isolated pixel.
- (iii) $2 \times 2 = 4$ kernel weights plus 1 bias = 5 learned parameters. The same 5 numbers are reused at all four positions — weight sharing in one sentence.

Solution 3 — Name the operation

- (i) True (mathematical) convolution: $(I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$. Deep-learning “convolution”: $(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$ — cross-correlation, no flip.
- (ii) If K is learned, define $K'(m, n) = K(-m, -n)$ (the flipped kernel). The true convolution with K equals cross-correlation with K' ; since training searches over all K , it searches over all K' too. The flip is absorbed into the learned values.
- (iii) The deck’s “Does it matter?” — **no**: a flip is a re-indexing of weights the optimizer would have found anyway.

Solution 4 — Stride and padding

- (i) $W_{\text{out}} = \lfloor (7 - 3 + 0)/2 \rfloor + 1 = \lfloor 2 \rfloor + 1 = 3.$
- (ii) With $P = 1$: $\lfloor (7 - 3 + 2)/2 \rfloor + 1 = \lfloor 3 \rfloor + 1 = 4$. Padding bought one extra position per side.

- (iii) Conv 3×3 , stride 1, “same” on 32×32 : 32×32 (same padding preserves size for odd K , stride 1). Then 2×2 max-pool, stride 2: $\lfloor (32 - 2)/2 \rfloor + 1 = 16 \rightarrow 16 \times 16$ (channels unchanged by pooling).

Solution 5 — Trace the MNIST stack

- (i) Conv1: $\lfloor (28 - 3)/1 \rfloor + 1 = 26$; Pool1: $\lfloor (26 - 2)/2 \rfloor + 1 = 13$; Conv2: $\lfloor (13 - 3)/1 \rfloor + 1 = 11$; Pool2: $\lfloor (11 - 2)/2 \rfloor + 1 = \lfloor 4.5 \rfloor + 1 = 5$; Conv3: $\lfloor (5 - 3)/1 \rfloor + 1 = 3$. Chain $28 \rightarrow 26 \rightarrow 13 \rightarrow 11 \rightarrow 5 \rightarrow 3$ [verified-NumPy].
- (ii) Because the output size is a *count of window positions*, an integer: $\lfloor 4.5 \rfloor = 4$ full steps plus the starting position = 5. The last column of the 11×11 map is simply never covered by a 2×2 stride-2 window (it would start at index 10 and need index 11).
- (iii) Conv2 with 128 filters on 32 input channels: $128 \cdot (3 \cdot 3 \cdot 32 + 1) = 128 \cdot 289 = 36,992$ weights [verified-NumPy]. The flatten size does *not* change: Conv3 still has 128 filters on the 5×5 map, giving $3 \times 3 \times 128 = 1152$.

Solution 6 — Pooling vs PCA

- (i) $16 \rightarrow 4$: 12 numbers discarded. What is lost is *position within each 2×2 patch* — the output records that the max was 9, not that it sat at the bottom-left of its patch.
- (ii) PCA (§24.5) *learns* a global linear projection keeping the top-variance directions and discards low-variance structure — but it flattens the image, losing the 2D layout entirely. Pooling is a *fixed, local, nonlinear* summary that *keeps* the spatial grid and discards within-patch position. PCA keeps variance directions and throws layout; pooling keeps layout and throws exact positions.
- (iii) PCA over pooling: tabular or already-vectorized data with no spatial layout (e.g. compressing 100 correlated features to 10). Pooling over PCA: any image CNN — flattening for PCA would destroy the neighbour structure the conv layers need.

Solution 7 — Equivariance, then invariance

- (i) **Equivariance:** shifting the input shifts the output by the same amount. **Invariance:** shifting the input leaves the output (approximately) unchanged.
- (ii) A conv layer is *equivariant* — “if a feature moves in the input, its representation moves correspondingly in the feature maps” [recorded]. Pooling *enhances invariance* — summarizing each patch means the exact position of a feature matters less [recorded].
- (iii) Shift the cat 10 px right: every conv feature map shifts ~ 10 px right too (equivariance — the cat detector still fires, just 10 px over). After pooling, the pooled values barely change: the strongest response in each 2×2 patch is still the strongest, unless the shift moves a feature *across a pool boundary* — which is why pooling gives *approximate* invariance, not perfect.

Solution 8 — Freeze vs fine-tune

- (i) 200 images $\rightarrow$ **feature extraction:** freeze the backbone, train only the classifier. The backbone’s millions of parameters cannot be re-estimated from 200 images; the classifier’s few thousand can.
- (ii) Mechanically: `requires_grad=False` tells autograd not to track or accumulate gradients for those parameters, so the optimizer has nothing to update for them. Note the forward pass is unchanged, and the *deltas still flow through* the frozen layers to reach earlier trainable layers if any exist — freezing stops $\partial \mathcal{L} / \partial W$ from being computed, not $\partial \mathcal{L} / \partial x$ from propagating.
- (iii) The backbone’s filters were learned on ImageNet-normalized inputs; its feature detectors expect that distribution. Feed raw $[0, 255]$ or wrongly-scaled pixels and the activations land in a regime the weights never saw — the transferred features silently degrade. The notebook’s rule: “the custom data going into the model should be prepared in the same way as the original training data.”

Solution 9 — The transfer-learning head

- (i) The head must output 3 scores: a linear map from the feature-vector dimension to 3 (plus 3 biases). The pretrained 1000-class head is discarded and replaced.
- (ii) `efficientnet_b0(weights=EfficientNet_B0_Weights.DEFAULT)` builds the pretrained model; then set `requires_grad=False` on every parameter in `model.features` to freeze the base [recorded].
- (iii) Same LR everywhere risks **destroying the pretrained features** — large updates to carefully tuned backbone weights (in Chapter 43’s terms: divergence/forgetting of a good initialization). The fine-tuning row’s fix: unfreeze with a *reduced* learning rate so the backbone moves gently while the new head learns fast.

Solution 10 — Design a small CNN

- (i) One valid design (3×3 convs, valid, stride 1; 2×2 pools, stride 2):

```
64x64x3 -> Conv1 (16 filters) -> 62x62x16 -> Pool -> 31x31x16
-> Conv2 (32 filters) -> 29x29x32 -> Pool -> 14x14x32
-> Conv3 (32 filters) -> 12x12x32 -> Pool -> 6x6x32
-> Flatten 1152 -> FC 10, softmax
```

Chain check: $(64 - 3) + 1 = 62$; $(62 - 2)/2 + 1 = 31$; $(31 - 3) + 1 = 29$; $(29 - 2)/2 + 1 = 14$; $(14 - 3) + 1 = 12$; $(12 - 2)/2 + 1 = 6$; $6 \cdot 6 \cdot 32 = 1152 \leq 2000$ [verified-NumPy].

- (ii) Conv1: $16 \cdot (3 \cdot 3 \cdot 3 + 1) = 448$. Conv2: $32 \cdot (3 \cdot 3 \cdot 16 + 1) = 4,640$. Conv3: $32 \cdot (3 \cdot 3 \cdot 32 + 1) = 9,248$. Total conv parameters = 14,336 [verified-NumPy] — versus a dense $64 \cdot 64 \cdot 3 \rightarrow 1152$ layer’s 14,156,928 weights, same moral as §44.7.
- (iii) §41.14(ii)’s guardrail: **dropout** (e.g. $p = 0.5$) before the output dense layer — with only 1152 flattened features and a small dataset, the dense head is the overfitting risk; dropout “modifies the network,” complementing the conv layers’ parameter efficiency.

Chapter 45: RNNs — sequences and vanishing gradients

Chapter 44 shared weights across *space* — one filter, every pixel. This chapter shares them across *time*: the same cell, every timestep (§44.10’s promise). The spine is the GenAI Weeks 7–8 deck (“Introduction to Recurrent Neural Networks,” Balaji Srinivasan and Ganapathy Krishnamurthi, 80 slides — sequence modelling, the RNN, BPTT, vanishing/exploding gradients, LSTM, GRU, bidirectional and encoder–decoder architectures, BLEU, beam search) plus its two companion notebooks: *Time_Series_Analysis_using_RNN* (AMZN stock prediction: data prep, a `torch.nn.RNN` model, Adam at $\eta = 0.001$ for 50 epochs) and *sentiment-analysis* (IMDB: `SentimentRNN` and `SentimentLSTM` with embedding, dropout, and a sigmoid head). Every numpy number below was re-run here and is marked **[verified-NumPy]**; deck, notebook, and torch numbers are transcriptions, marked **[recorded]**.

Notation. The deck writes the timestep as a function argument, $h(t), x(t)$; this book writes it as a subscript, h_t, x_t — the same objects. Weight names follow the deck: W_{xh} (input→hidden), W_{hh} (hidden→hidden, the recurrent weights), W_{hy} (hidden→output), biases b_h, b_y , activation ϕ . LSTM gates keep the deck’s letters: $f_t, i_t, \tilde{c}_t, c_t, o_t$. The deck’s GRU slides switch to row-vector notation ($X_t W_{xr} + H_{t-1} W_{hr}$) while its LSTM slides use column concatenation ($W_f \cdot [h_{t-1}, x_t]$) — a layout change, not a math change; §45.9 flags it.

45.1 Sequence data: order matters

Def (the deck’s). “Sequential data is a type of data where the order of the data points is significant. The sequence itself contains important information. Each data point is dependent on the previous ones (e.g., a sentence).” Three characteristics: **Ordered** (“Data points have a specific position or index in a sequence”), **Variable Length** (“Sequences can have different lengths (e.g., different sentence lengths)”), and **Dependencies** (“Values can depend on other values at different positions in the sequence”).

The modelling target. “A sequence model aims to estimate the joint probability of an entire sequence of observations” — for $x_1, x_2, \dots, x_T$, the model estimates $P(x_1, x_2, \dots, x_T)$, “to assess how plausible a sequence is, or to generate new, similar sequences” [recorded]. Applied to text this is a **language model**: “When a sequence model is applied to text data, it is called a language model. Its goal is to estimate the probability of a sequence of words or characters.”

The autoregressive factorization. Predicting x_t from the past, the joint probability factors by the chain rule of probability (the deck’s term):

$$P(x_1, \dots, x_T) = P(x_1) \prod_{t=2}^T P(x_t \mid x_1, \dots, x_{t-1}) \quad \text{[recorded].}$$

“In words: the probability of a sequence is the product of the conditional probabilities of each element, given the elements that came before it.” Models that “predict future values in a sequence based on past values” this way are **autoregressive**. The catch is stated immediately: “The conditioning term $P(x_t \mid x_1, \dots, x_{t-1})$

becomes computationally expensive and statistically difficult to model as the sequence length t grows” — the full history is too much to carry raw.

Basically, ... “Order is the data: shuffle a sentence and it dies. The model’s job is to score whole sequences — how plausible is this one? The chain rule turns that into ‘predict each token from everything before it’, which is the autoregressive game. The problem: ‘everything before it’ gets long, and long is expensive.”

45.2 Before the RNN: Markov, windows, and why they break

Three simplifications, each buying tractability and each paying for it.

i) The Markov assumption / n-grams. Cut the history: “Assumes that the current state x_t only depends on the immediately preceding state x_{t-1} ,” so $P(x_1, \dots, x_T) \approx P(x_1) \prod_{t=2}^T P(x_t | x_{t-1})$ — the first-order Markov model, “also known as a bigram model in the context of language processing,” extendable to “a fixed-length window of previous states (e.g., trigrams, etc.)” [recorded]. The price: “Markov models and n-grams have a fixed memory; they only look at the last $n - 1$ tokens. To remember more, the number of model parameters grows exponentially, making them impractical for capturing long-range dependencies” [recorded]. Count it: a bigram table over a vocabulary of size $|V|$ needs $|V|^2$ entries; a trigram $|V|^3$ — each extra token of memory multiplies the table by $|V|$.

ii) Linear regression on a fixed window. “It follows an n-gram Markovian assumption. It uses a fixed window of past data and applies a linear model to predict the next value”: $\hat{x}_t = \sum_{i=1}^{\tau} w_i x_{t-i} + b$ [recorded]. The deck lists three limitations:

1. **Linearity.** “They fail to capture complex, non-linear relationships, such as: the interaction between different past events. Sudden shocks or spikes in the data (e.g., a stock market crash).”
2. **Fixed window.** “By conditioning only on a fixed-length window (τ), the model is completely blind to any information that occurred before that window” — fatal “when long-range dependencies are important.”
3. **Error accumulation.** For multi-step prediction the model feeds its own outputs back in: “ $\hat{x}_{t+1}$ will have some error, ϵ_1 . This incorrect value... is then fed back into the model to predict $\hat{x}_{t+2}$. The error in the input leads to a new, often larger, error ϵ_2 ... causing the predictions to diverge rapidly from the true data.”

iii) MLPs and CNNs. Chapter 44’s machinery doesn’t transfer. The deck’s three counts: **Fixed-size vectors** — “They require inputs of a fixed length. This forces us to either truncate long sequences or pad short ones, both of which can lead to loss of information.” **No parameter sharing across time** — “An MLP learns separate weights for features at each position. It cannot generalize a pattern it learns at position ‘i’ to position ‘j’.” A CNN shares parameters spatially, but isn’t designed for capturing temporal dependencies of arbitrary length.” **No memory of context** — “They process each part of the input independently. They have no mechanism to remember previous elements when processing a new one.” The verdict: “We need a model that processes sequences step-by-step, with memory” [recorded].

Basically, ... “Three workarounds, three bills. Markov: only look back $n-1$ steps — but a longer look-back multiplies the parameter table by the vocabulary size each time. Linear window: same blind spot, plus straight-line-only predictions that drift when fed their own errors. MLPs/CNNs: fixed-length inputs, and an MLP learns position 5’s pattern separately from position 50’s — no sharing across time, no memory. The spec for the fix: step-by-step, with memory.”

45.3 The RNN: one cell, reused across time

The latent hidden state. Instead of conditioning on the raw past, “an RNN maintains a compact summary of the entire history in a hidden state, h_t ”:

$$\boxed{P(x_t | x_1, \dots, x_{t-1}) \approx P(x_t | h_{t-1})}, \quad h_t = f(x_t, h_{t-1}) \quad [\text{recorded}].$$

“An RNN processes a sequence by iterating through its elements one by one. It maintains a hidden state (or ‘memory’) that captures information about what it has seen so far. The output at a given time step depends not only on the current input but also on the hidden state from the previous time step.” At each step the network “takes a new input x_t ; updates its hidden state h_t ; produces an output y_t ” [recorded].

The equations (the deck’s). A single recurrent neuron:

$$\boxed{h_t = \tanh(w_{xh} x_t + w_{hh} h_{t-1} + b_h)}, \quad \boxed{y_t = w_{hy} h_t + b_y} \quad [\text{recorded}].$$

w_{xh} weights the current input, w_{hh} “the recurrent or ‘memory’ weight” scales the previous hidden state, w_{hy} reads the output out. A layer vectorizes it (§41.4’s move):

$$\boxed{h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)}, \quad \boxed{y_t = W_{hy} h_t + b_y} \quad [\text{recorded}].$$

This is §41.4’s two equations with one addition: the $W_{hh} h_{t-1}$ term, the memory of the previous step, added to the pre-activation before the nonlinearity. The deck uses $\tanh$ — §41.7’s zero-centered S-bend, “often speeding up convergence” for exactly this inner recurrence.

The key point (the deck’s, §44.10’s twin). “Key Point: The same weight matrix is used at every time step.” Drawn folded, the RNN is a neuron with a self-loop; drawn **unfolded over time**, it is a chain of identical cells, W_{xh}, W_{hh}, W_{hy} repeated at $t-1, t, t+1$. Chapter 44’s filter is reused at every *pixel*; the RNN’s cell is reused at every *timestep* — one set of parameters no matter how long the sequence, which is also what lets it accept variable-length inputs that an MLP cannot.

Note (the sentiment notebook’s variant). Its §1.4 writes the same update with a generic nonlinearity, $h_t = \sigma(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$, σ = “a non-linear activation function” [recorded] — the deck’s neuron fixes $\sigma = \tanh$. Same cell, activation swapped.

Basically, ... “One small network, run in a loop. Each step: mix the new input with a summary of everything seen so far (the hidden state), squash with $\tanh$, read out the prediction. The loop is the memory — and the weights are shared across every step, so a 10-word sentence and a 1000-word sentence use the same parameters. That is Chapter 44’s sharing idea, rotated from space to time.”

45.4 eg: one forward step — next-character prediction [verified-NumPy]

The deck’s worked example. Vocabulary {h:0, e:1, l:2, o:3} (the §45.2 tokenization story in miniature: text $\rightarrow$ integer indices $\rightarrow$ one-hot). Task: input character “e” ($x_t = (0, 1, 0, 0)^T$), memory of having just seen “h” ($h_{t-1} = (0.6, 0.2)^T$, “pre-computed for simplicity”), predict the next character. Ground truth: “l” ($(0, 0, 1, 0)^T$).

Weights:

$$W_{xh} = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.4 & 0.5 & 0.6 & 0.7 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 0.1 & 0.5 \\ 0.5 & 0.1 \end{bmatrix}, \quad W_{hy} = \begin{bmatrix} 2 & -1 \\ 1 & 0 \\ 1 & 3 \\ 0 & -2 \end{bmatrix}, \quad b_h = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}, \quad b_y = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{bmatrix}.$$

Step 1 — the new hidden state. Input contribution: $W_{xh} x_t$ picks out column 1 (the one-hot selects the “e” column): $(0.2, 0.5)^T$. Memory contribution: $W_{hh} h_{t-1} = (0.1 \cdot 0.6 + 0.5 \cdot 0.2, 0.5 \cdot 0.6 + 0.1 \cdot 0.2)^T = (0.16, 0.32)^T$. Add the bias:

$$h_t = \tanh \left(\begin{bmatrix} 0.2 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.16 \\ 0.32 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix} \right) = \tanh \begin{bmatrix} 0.46 \\ 0.92 \end{bmatrix} = \begin{bmatrix} 0.4301 \\ 0.7259 \end{bmatrix}.$$

Step 2 — output logits. $W_{hy}h_t = (2(0.4301) - 0.7259, 0.4301, 0.4301 + 3(0.7259), -2(0.7259))^T = (0.1343, 0.4301, 2.6078, -1.4518)^T$; add b_y :

$$y_t = \begin{bmatrix} 0.2343 \\ 0.6301 \\ 2.9078 \\ -1.0518 \end{bmatrix} \begin{array}{l} \leftarrow \text{'h'} \\ \leftarrow \text{'e'} \\ \leftarrow \text{'l'} \\ \leftarrow \text{'o'} \end{array}.$$

Highest logit $2.9078 \rightarrow \text{'l'}$. “The model predicts ‘l!’” [recorded] — matching the ground truth.

Note (deck arithmetic slip, corrected here). The deck’s slide writes the fourth pre-bias entry as -1.01 and the final logit as -0.61 ; the correct values are -1.4518 and -1.0518 ($0 \cdot 0.4301 - 2 \cdot 0.7259 = -1.4518$, then $+0.4$). The argmax and the conclusion are unaffected. The deck’s rounded $h_t = (0.43, 0.72)$ accounts for the small differences in the first three entries ($0.24, 0.63, 2.89$ vs $0.234, 0.630, 2.908$ here).

Basically, ... “One-hot ‘e’ picks out one column of W_{xh} (the ‘what does e mean’ column); the old hidden state gets mixed through W_{hh} (the ‘what did I remember’ path); add, tanh, done — new memory $(0.43, 0.73)$. Then W_{hy} scores each vocabulary character against that memory, and ‘l’ wins with 2.91. The one-hot trick makes the input matrix multiply into a column lookup — cheap and readable.”

45.5 BPTT: the same backprop, unrolled

Unfold the RNN over T steps and it is a T -layer feedforward network whose layers happen to share weights. So training is Chapter 42’s machinery with a new name: “BPTT is simply the application of the standard backpropagation algorithm to the ‘unrolled’ computational graph of an RNN. It is not a new algorithm, but a name for the process” [recorded]. The loss at the end “depends on computations from every single time step” — h_t depends on h_{t-1} , which depends on h_{t-2} , “and so on” — so the error must flow back through the whole chain.

The heart of BPTT (the deck’s recursion). The gradient of the total loss $L = \sum_{t=1}^T L_t$ w.r.t. a hidden state has *two* components:

$$\frac{\partial L}{\partial h_t} = \underbrace{\frac{\partial L}{\partial y_t} \frac{\partial y_t}{\partial h_t}}_{\text{current loss}} + \underbrace{\frac{\partial L}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t}}_{\text{future loss}} \quad [\text{recorded}].$$

“This recursive formula is the heart of BPTT”: the error at h_t is the error from *this* step’s output plus the error arriving back from *the future* through the recurrent connection. Start at $t = T$ (no future term) and walk backwards.

The parameter gradients (the deck’s). With $\partial L / \partial h_t$ in hand, “we can find the gradients for the weights and biases at that time step. The final gradient for a parameter is the sum of its gradients from all time steps” [recorded]:

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^T \frac{\partial L}{\partial y_t} h_t^T, \quad \frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^T \frac{\partial L}{\partial h_t} \odot (\text{local terms}) x_t^T, \quad \frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L}{\partial h_t} \odot (\text{local terms}) h_{t-1}^T.$$

The local terms are the tanh derivative $\phi'(z) = 1 - \tanh^2(z)$ at each step — §42.4’s three-component product (upstream $\times$ activation $\times$ local), summed over time because the weights are shared across it. Sharing buys parameter efficiency (§45.3); the sum is its bookkeeping cost.

Note. The two-term recursion is §42.4’s δ recursion seen from the side: in a feedforward net the error flows *down through layers*; here it flows *back through time* — the same chain rule, and §42.6’s gradient-checking advice applies to it unchanged.

Basically, ... “Unroll the loop and it’s just a deep net with tied weights — backprop as usual, hence ‘backprop through time’. Each hidden state’s blame has two sources: the output it produced *now*, and the future states it helped build *later*. Add the two, step back one timestep, repeat. Parameter gradients are the per-step gradients *summed* — one weight, many timesteps, so every step gets a vote.”

45.6 eg: BPTT by hand on a 1-neuron net [verified-NumPy]

The deck’s setup, with its arithmetic corrected (see the Note). One neuron, sequence length 2, squared-error loss $L_t = (y_t - \text{target}_t)^2$, biases 0:

$$x_1 = 2, x_2 = 3, h_0 = 0, w_{xh} = 0.5, w_{hh} = 0.2, w_{hy} = 0.4, \text{ targets } 0.5, 0.8, \phi = \tanh.$$

Forward. $t=1$: $h_1 = \tanh(0.5 \cdot 2 + 0.2 \cdot 0) = \tanh(1) = 0.7616$; $y_1 = 0.4 \cdot 0.7616 = 0.3046$; $L_1 = (0.3046 - 0.5)^2 = 0.0382$. $t=2$: $h_2 = \tanh(0.5 \cdot 3 + 0.2 \cdot 0.7616) = \tanh(1.6523) = 0.9292$; $y_2 = 0.4 \cdot 0.9292 = 0.3717$; $L_2 = (0.3717 - 0.8)^2 = 0.1835$. Total $L = 0.2216$.

Backward, from the end. $t=2$: $\frac{\partial L}{\partial y_2} = 2(0.3717 - 0.8) = -0.8567$; $\frac{\partial L}{\partial h_2} = -0.8567 \cdot 0.4 = -0.3427$. The w_{hh} gradient at this step (local: $\phi'(z_2) h_1 = (1 - h_2^2) h_1$):

$$\left. \frac{\partial L}{\partial w_{hh}} \right|_{t=2} = (-0.3427) (1 - 0.9292^2) (0.7616) = -0.0357.$$

$t=1$ — the crucial step, both components:

$$\frac{\partial L}{\partial h_1} = \underbrace{2(0.3046 - 0.5) \cdot 0.4}_{\text{current} = -0.1563} + \underbrace{(-0.3427) (1 - 0.9292^2) (0.2)}_{\text{future} = -0.0094} = -0.1657.$$

Then $\left. \frac{\partial L}{\partial w_{hh}} \right|_{t=1} = (-0.1657) (1 - 0.7616^2) (0) = 0$ — $h_0 = 0$ kills it, the same silent-input logic as §42.3’s Note ($x_2 = 0$ there). Final:

$$\boxed{\frac{\partial L}{\partial w_{hh}} = 0 + (-0.0357) = -0.0357.}$$

The other two, same recipe: $\partial L / \partial w_{hy} = \sum_t \frac{\partial L}{\partial y_t} h_t = (-0.3907)(0.7616) + (-0.8567)(0.9292) = -1.0936$, and $\partial L / \partial w_{xh} = \sum_t \frac{\partial L}{\partial h_t} (1 - h_t^2) x_t = (-0.1657)(0.4200)(2) + (-0.3427)(0.1366)(3) = -0.2796$ [verified-NumPy].

Note (deck arithmetic slip, corrected here). The deck writes $y_2 = 0.4 \cdot 0.93 = 0.744$ — but $0.4 \cdot 0.93 = 0.372$; its 0.744 equals $0.8 \cdot 0.93$, as if the target 0.8 crept in as the weight. Everything downstream in the deck ($L_2 = 0.003$, $\partial L / \partial y_2 = -0.112$, $\partial L / \partial w_{hh} = -0.0046$) follows consistently from the slipped value, so only the one line needed fixing; the corrected pass above re-derives it all. The structural points survive: the two-component recursion, and the $h_0 = 0$ zero gradient.

Basically, ... “Forward: two tanh squashes, two squared errors, $L = 0.222$. Backward: at the last step the blame is just ‘prediction minus target, doubled’ times the output weight; at the first step it’s that *plus* the future’s blame passed back through w_{hh} and the tanh slope — current loss plus future loss, exactly §45.5’s recursion. The $t=1$ memory gradient is zero because $h_0 = 0$: nothing flowed through, nothing to blame.”

45.7 The price of sharing: vanishing and exploding gradients

The product. To reach the early weights, the error crosses every timestep. The deck writes the chain for the hidden-state Jacobian over T steps:

$$\boxed{\frac{\partial h_T}{\partial h_0} = \prod_{t=1}^T \frac{\partial h_t}{\partial h_{t-1}} = \prod_{t=1}^T W_{hh}^T \text{diag}(\phi'(\dots))} \quad [\text{recorded}].$$

Each step multiplies by the *same* W_{hh} and by the tanh slope $\phi'(z) = 1 - \tanh^2(z) \leq 1$ — “This long product is the source of major numerical instability” [recorded]. This is §42.8(ii)’s other half: there the error crossed *layers* and each sigmoid multiplied it by at most 0.25; here it crosses *time* and each step multiplies by $W_{hh}^T \text{diag}(\phi')$.

The twin problems (the deck’s table).

- i) **Gradient explosion.** “If the recurrent weight matrix W_{hh} has large values, the long product of matrices can grow exponentially.” Result: “Gradients become enormous (NaN or infinity)” → “Model weights are updated by huge amounts” [recorded]. Note: the deck’s Week-5 companion (§43.12’s troubleshooting table) lists “Loss exploding → Decrease LR, gradient clipping” — the same fix, now with its derivation.
- ii) **Gradient vanishing.** “If the recurrent weight matrix has small values, the long product of matrices can shrink exponentially towards zero.” Result: “Gradients for early time steps become nearly zero” → “The model cannot learn long-range dependencies” → “The model forgets what happened early in the sequence” [recorded].

The tanh makes (ii) worse even at moderate weights: $\phi'(z) \leq 1$ always, and a saturated neuron ($\tanh(z) \approx \pm 1$) has $\phi'(z) \approx 0$ — §41.7’s tanh row (“same vanishing-gradient flattening — ‘difficult to use in very deep networks’”) is the same sentence, now about depth *in time*. A neuron that saturates stops its own past from being learnable.

eg (the shrink and the blow-up, counted — [verified-NumPy]). Suppose each step multiplies the error by roughly 0.64 (e.g. $\|W_{hh}\| \approx 0.8$, $\phi' \approx 0.8$): after 20 steps the surviving fraction is $0.64^{20} \approx 1.3 \times 10^{-4}$ — four orders of magnitude gone. With 1.5 per step instead: $1.5^{20} \approx 3325$ — the gradient more than triples per step and compounds to thousands. The boundary between the two regimes is a knife’s edge, and the same W_{hh} must serve every timestep — that is why the deck calls the product itself, not any one bad weight, “the source.”

The three fixes (the deck’s).

- i) **Gradient clipping** (for explosion). “If the norm of the gradient exceeds a certain threshold, it is scaled down” [recorded]. No formula in the deck; the torch call lives in §43.12 (`torch.nn.utils.clip_grad_norm_`), which lists clipping as “essential for RNNs/Transformers” — the derivation above is why.
- ii) **Truncated BPTT** (for cost and instability). “Instead of backpropagating through the entire sequence, we detach the gradient history after a fixed number of steps (τ). The model still propagates its hidden state forward through the whole sequence, maintaining its long-term memory. However, during the backward pass, the gradient calculation is cut off after τ steps” [recorded]. “Trade-off: It limits the model’s ability to learn dependencies that span longer than τ time steps.”
- iii) **More complex RNN architectures** (for vanishing). “Solution: More complex RNN architectures” — the gates of §45.8, which keep a gradient path that does not multiply by W_{hh} every step.

Basically, ... “Backprop through 20 timesteps multiplies the error by the same weights 20 times. Slightly-small weights: the signal dies (0.64^{20} is basically zero) and the net forgets the past. Slightly-big: it explodes (1.5^{20} is 3325) and you get NaNs. Clipping caps the explosions, truncation stops the backward chain early (but blinds the net beyond τ steps), and the real fix for vanishing is architectural — gates.”

45.8 LSTM: gates and the additive cell

What it is. “LSTM networks are a type of recurrent neural network (RNN) designed to solve the vanishing gradient problem. They are particularly effective at learning long-term dependencies in sequential data” [recorded]. Three differences from the simple RNN: **dual states** — “a short-term state h_t and a long-term state (or cell state) c_t ”; a **gated mechanism** — “special ‘gates’ to regulate the flow of information into and

out of the cell state”; and **gradient stability** — “The gates and additive interactions within the cell state help combat both vanishing and exploding gradients, making training more stable” [recorded].

The gates (the deck’s equations, [recorded]). Each gate is a sigmoid over the concatenated $[h_{t-1}, x_t]$ — a number in $(0, 1)$ per cell dimension, i.e. a soft valve:

- **Forget gate** — “Decides what information from the old cell state to throw away”: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$. “1 means ‘keep this information completely’ and 0 means ‘forget it completely.’”
- **Input gate** — “Decides what new information from the current input to store in the cell state”: $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$, paired with a **candidate** $\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$ — “tanh... to create a new vector of potential information, scaled between -1 and 1.” The two “are multiplied element-wise to form the new information that will be added to the cell state.”
- **Cell update** — “This is the core of the LSTM’s memory mechanism”:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{[recorded].}$$

“The first term, $f_t \odot c_{t-1}$, represents the old memory with some parts forgotten. The second term, $i_t \odot \tilde{c}_t$, represents the new information being added.”

- **Output gate** — “Decides what part of the cell state to output as the new hidden state”: $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$, $h_t = o_t \odot \tanh(c_t)$. “The hidden state h_t is a filtered version of the cell state.”

Why the gates fix vanishing. “This additive interaction is crucial for preserving gradients. Instead of a series of multiplicative steps, the addition allows gradients to flow directly back, preventing them from vanishing” [recorded]. In symbols: $\partial c_t / \partial c_{t-1} = f_t$ — the error on the cell state passes back through a *single* gate value, not through $W_{hh}^T \text{diag}(\phi')$. If the forget gate stays near 1 (keep everything), the gradient crosses timesteps almost intact — the §45.7 product becomes a sum of f_t ’s instead of a product of W_{hh} ’s.

eg (the gradient path, counted — [verified-NumPy]). Over 20 timesteps with a vanilla step-factor of 0.64, §45.7’s eg left 1.3×10^{-4} . With an LSTM whose forget gates sit at $f_t = 0.95$ (mostly keeping), the cell-state gradient factor is $0.95^{20} \approx 0.36$ — more than a third survives. The gates *learn* where to sit: $f_t \approx 1$ on stretches worth remembering, $f_t \approx 0$ where the past should be dropped. The network learns its own truncation window.

Basically, ... “An LSTM keeps two memories: a long-term cell state and a short-term hidden state. Three sigmoid valves guard the cell — forget (what to erase), input (what new stuff to write, via a tanh candidate), output (what to show the world). The cell update is *addition*, not a matrix multiply, so the gradient flows back through a chain of forget-gate values instead of a chain of weight matrices — and a forget gate near 1 means the past survives. Basically: a memory with an eraser, a pen, and a lid, all learned.”

45.9 GRU: two gates, no separate cell

What it is. “The Gated Recurrent Unit (GRU), introduced in 2014, is a popular alternative to the LSTM. It aims to solve the same vanishing gradient problem but with a simpler architecture and fewer parameters” [recorded]. “Two Gates instead of Three” and “No Separate Cell State: GRU merges the cell state and hidden state into a single hidden state vector, h_t .” The payoff: “Due to its simpler design, a GRU often trains faster than an LSTM and can perform just as well, or sometimes even better, on less complex datasets” [recorded].

The equations (the deck’s, [recorded]). Step 1, the two gates:

- **Reset gate** — “determines how the previous hidden state H_{t-1} is used to compute the new candidate hidden state”: $R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r)$. “When entries in R_t are close to 1, the gate allows the previous hidden state to pass through... close to 0, the gate effectively ‘resets’ or ignores parts of the previous hidden state.”
- **Update gate** — “controls how much of the previous hidden state H_{t-1} is directly carried over to the final hidden state H_t ”: $Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$. “It acts similarly to the forget gate in an LSTM. When entries in Z_t are close to 1, the corresponding dimension in H_{t-1} is almost entirely copied to H_t , preserving long-term memory.”

Step 2, the states. **Candidate** (a vanilla RNN step, but with the reset gate throttling the past): $\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$. **Final state** — “a linear interpolation between the previous hidden state H_{t-1} and the new candidate hidden state $\tilde{H}_t$ ”:

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t \quad [\text{recorded}].$$

“ $Z_t \approx 1$: retain old information; $Z_t \approx 0$: fully replace it with new information.” The same additive trick as the LSTM’s cell update (§45.8): $\partial H_t / \partial H_{t-1}$ carries a Z_t term, so gradients survive when the update gate stays open.

Note (deck notation switch). The GRU slides write row-vector products ($X_t W_{xr}$) while the LSTM slides write $W_f \cdot [h_{t-1}, x_t]$ — the GRU’s H_t is the LSTM section’s h_t , transposed layout, same role. The LSTM-vs-GRU comparison that matters is architectural: three gates + separate cell (LSTM) vs two gates + merged state (GRU).

LSTM vs GRU, one line each. LSTM: more expressive (separate long/short states, three valves) at the cost of more parameters. GRU: fewer parameters, faster training, “just as well, or sometimes even better, on less complex datasets” — and §43.12 files both under “RNNs / LSTMs → RMSProp / Adam.” When in doubt, the course’s notebooks reach for the LSTM (*sentiment-analysis* builds `SentimentLSTM`; the time-series notebook’s class is a plain `nn.RNN`) — the GRU appears in the deck’s theory, not in either notebook’s code.

Basically, ... “GRU = LSTM on a diet. Two valves instead of three (reset: how much past goes into the *candidate*; update: how much past survives into the *final* state), one state instead of two. The final state is a learned mix — update gate near 1 keeps the old memory, near 0 takes the new candidate. Same vanishing-gradient fix (the mix is additive), fewer knobs, faster training.”

45.10 Reading both ways, speaking at length

Bidirectional RNNs. “A standard RNN is unidirectional. The hidden state at time t (H_t) only captures information from past inputs” — “a major drawback for tasks where future context is crucial,” e.g. “The man who ____ movies is a critic,” where “we need the word ‘movies’ to predict ‘reviews’” [recorded]. The fix: “two independent RNNs: one that processes the sequence from left-to-right (a forward layer) and another that processes it from right-to-left (a backward layer). For any time step t , the final hidden state is formed by concatenating the hidden states from both the forward and backward layers”:

$$H_t = [\vec{h}_t, \bar{h}_t], \quad O_t = H_t W_{hq} + b_q \quad [\text{recorded}].$$

The deck’s walkthrough (“I am ____ happy”): the forward pass reaches the blank with only “I am”; the backward pass with only “happy”; the concatenated H_3 “contains a rich representation of the full sentence context” [recorded]. Cost: two passes, and the backward pass must wait for the whole sequence — fine for scoring a sentence, impossible for live generation.

Encoder–decoder (seq2seq). “Standard RNNs are only designed for tasks where the input and output lengths are the same. How do we handle tasks like machine translation or summarization, where the lengths can be completely different?” [recorded]. Two RNNs with different jobs: the **encoder** “reads the entire input sequence step-by-step” and “compress[es] the information of the whole sequence into a single fixed-size vector, often called the **context vector**”; the **decoder** is “initialized with the encoder’s context vector” and “generates the output sequence one token at a time, using the context and the previously generated token” [recorded]. Trained jointly with cross-entropy “at each timestep of the decoder’s output,” comparing “the predicted probability distribution over the vocabulary with the one-hot encoded ground-truth token”; with padding, “the loss function uses a mask to ignore these padding tokens during calculation, ensuring they do not contribute to the gradient updates” [recorded].

Teacher forcing. “During training, instead of feeding the decoder’s own prediction from the previous step as input for the current step, we use the actual ground-truth token from the target sequence.” Benefit: “stabilizes training and helps the model learn the sequence structure more efficiently, as it always receives

correct input.” But “at inference time... the model must use its own previously generated tokens as input, since the ground truth is not available” [recorded] — a train/test mismatch the chapter states and leaves open (Chapter 46’s attention does not remove it either, but it does remove the context-vector bottleneck below).

The context-vector bottleneck. One fixed-size vector must hold a 500-word article for a 50-word summary (“Text Summarization... Machine Translation... Question Answering” are the deck’s applications [recorded]). Everything the decoder knows about the input passes through that single vector — compress a long sequence into it and something is lost. This bottleneck is exactly what §44.10(ii)’s attention shortcut is for: “attention lets any position talk to any other directly, instead of waiting for pooling to bring them together” — here, instead of squeezing through one vector.

Basically, ... “Forward-only RNNs can’t see the future — so run a second one backwards and concatenate: now every position knows both sides. Different input/output lengths (translation, summaries) need two RNNs: an encoder squeezes the input into one context vector, a decoder unrolls the output from it. Training cheats by feeding the decoder the true previous word (teacher forcing); at test time it’s on its own. The weak link: one vector carrying a whole paragraph — remember this when attention arrives.”

45.11 Judging sequences: perplexity, BLEU, beam search

Perplexity. “Perplexity (PPL) is the standard metric for evaluating language models. It measures how ‘surprised’ or ‘perplexed’ a model is by a sequence of text” — “Lower Perplexity is Better!”:

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log P(x_t | x_1, \dots, x_{t-1})\right) \quad [\text{recorded}].$$

It is “the exponentiation of the average negative log-likelihood” — the negative log-likelihood of §35.6’s cross-entropy vocabulary, exponentiated. Intuition: “A perplexity of k means that, on average, the model is as uncertain about the next word as if it had to choose uniformly from k different words” [recorded]. Sanity check [verified-NumPy]: a model assigning probability 0.5 to each of four true next-words has average NLL $-\ln 0.5 = 0.693$, so $\text{PPL} = e^{0.693} = 2.0$ — as confused as a fair coin.

BLEU. “BLEU (Bilingual Evaluation Understudy) is a standard metric for evaluating the quality of a predicted sequence by comparing it to a target (ground truth) sequence,” based on “the precision of matching n-grams... between the two sequences” [recorded]:

$$\text{BLEU} = \exp\left(\min\left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right)\right) \cdot \left(\prod_{n=1}^k p_n\right)^{1/k} \quad [\text{recorded}],$$

p_n = matched n-grams / predicted n-grams. The $\min(0, 1 - \cdot)$ term is the **brevity penalty**: “If the predicted sequence is identical to the target, the BLEU score is 1.”

eg (the deck’s BLEU example — [verified-NumPy]). Target “A B C D E F”, predicted “A B”, $k = 2$: unigrams $\{A, B\}$ both match $\rightarrow p_1 = 2/2 = 1.0$; bigram $\{AB\}$ matches $\rightarrow p_2 = 1/1 = 1.0$. “Without a penalty, the score would be $(1.0 \times 1.0)^{1/2} = 1.0$, which is misleadingly perfect.” Brevity penalty: $\exp(1 - 6/2) = \exp(-2) \approx 0.1353$. Final: $\text{BLEU} = 0.1353 \times 1.0 = 0.1353$ — “correctly lowers the score from a perfect 1.0 to 0.135, reflecting that while the precision is high, the prediction is far too short” [recorded].

Beam search. Generating means finding “the sequence with the highest conditional probability.” **Greedy search** — “at each step, we choose the single most likely token” — is “fast and computationally cheap” but “not optimal. The sequence of most-likely tokens is not necessarily the most-likely sequence overall.”

Exhaustive search “guarantees finding the optimal sequence” but is “computationally infeasible.” **Beam search** is “a practical compromise... controlled by a hyperparameter called the beam size, k ”: at each step keep the top- k tokens, expand each into $|V|$ continuations, keep the best k of the $k \times |V|$ candidates, repeat “until an end-of-sequence token is generated or a maximum length is reached” [recorded]. Final scoring normalizes by length to avoid short-sequence bias: $\text{Score} = \frac{1}{L^\alpha} \sum_{t'=1}^L \log P(y_{t'} | y_{<t'}, c)$, $\alpha \approx 0.75$ [recorded].

“Greedy search is a special case of beam search arising when the beam size is set to 1” [recorded]. (The deck’s beam-search illustration is credited to d2l.ai.)

Basically, ... “Perplexity: how surprised is the model? Lower is better; 10 means ‘as confused as 10 choices’. BLEU: for generated sequences, count matching n-grams (precision) times a brevity penalty — a two-word prediction can’t score 1.0 against a six-word target. Beam search: greedy picks the best word each step (fast, wrong); exhaustive tries everything (right, impossible); beam keeps the k best partial sentences and prunes — greedy is just beam with $k = 1$.”

45.12 The PyTorch face

Both notebooks’ torch code is transcribed as [recorded] (torch is not installed here; nothing re-run).

The time-series notebook’s RNN (Time_Series_Analysis_using_RNN.ipynb, AMZN daily closes, 95/5 split, lookback window reshaped to (samples, timesteps, features)):

```
class RNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_stacked_layers):
        super().__init__()
        self.hidden_size = hidden_size
        self.num_stacked_layers = num_stacked_layers
        self.rnn = nn.RNN(input_size, hidden_size, num_stacked_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, 1)
    def forward(self, x):
        batch_size = x.size(0)
        h0 = torch.zeros(self.num_stacked_layers, batch_size, self.hidden_size)
        out, _ = self.rnn(x, h0)
        out = self.fc(out[:, -1, :]) # last timestep's hidden state -> prediction
        return out
```

[recorded]. Training: `loss_function = nn.MSELoss()`, `optimizer = torch.optim.Adam(model.parameters(), lr=0.001)`, 50 epochs [recorded] — §43.12’s defaults in the wild. The scaling discipline of §43.9 rides along: `MinMaxScaler(feature_range=(-1, 1))` before training, predictions inverse-transformed after [recorded].

The sentiment notebook’s pair (IMDB, binary classification). **SentimentRNN**: `nn.Embedding(vocab_size, embedding_dim) → nn.RNN(embedding_dim, hidden_dim, n_layers, dropout=drop_prob, batch_first=True) → dropout → nn.Linear(hidden_dim, output_size) → sigmoid` [recorded]. **SentimentLSTM**: identical scaffolding with `nn.LSTM(...)` in place of `nn.RNN`, and `forward` takes `hidden` as a tuple (`h`, `c`) — the dual states of §45.8 surfacing in the API [recorded]. Dropout 0.5 is §41.14(ii)’s guardrail, unchanged.

§43.12’s RNN rows, now grounded. “RNNs / LSTMs → RMSProp / Adam — Handles non-stationary objectives” and “Clip gradients for RNNs/transformers (`torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`)” [recorded] — the optimizer choice and the §45.7(i) fix, as the course’s own cheat sheet.

Basically, ... “In torch, an RNN is `nn.RNN(input, hidden, layers, batch_first=True)` plus a linear head on the last timestep; an LSTM is a drop-in swap whose hidden state comes as a (`h`, `c`) pair. Shape your data as (batch, time, features) — §36.4’s axis semantics — scale it (§43.9), train with Adam at the usual 0.001, and clip the gradients (§45.7). The deck’s math and the notebooks’ code are the same object.”

45.13 Where this goes next

- i) **Attention (Chapter 46).** §45.10’s context-vector bottleneck is the problem attention was built for: §44.10(ii)’s “shortcut” — any decoder position talking to any encoder position directly, no single-vector squeeze. The deck’s W9–10 material takes it from here.
- ii) **The sharing moral, complete.** Chapter 44: share across space → translation robustness, parameter counts collapse (§44.7). Chapter 45: share across time → variable-length sequences, one cell — and the

bill arrives as §45.7's product-of- T -matrices, §42.8's vanishing story told along a new axis. Clipping (§45.7(i), §43.12's torch call) and gates (§§45.8–45.9) are the two accepted payments.

- iii) **Generative models (Chapter 47).** The decoder of §45.10 *is* a generator: “assess how plausible a sequence is, or... generate new, similar sequences” (§45.1). Beam search (§45.11) is the decoding algorithm the generative chapters inherit.
- iv) **The debugging playbook rides along.** §40.5(i): “shapes first” — for an RNN that means the (batch, timesteps, features) reshape (§45.12) before anything trains, then scale (§43.9's sermon — the time-series notebook scales to $(-1, 1)$), then the §45.7 watch: NaNs mean the exploding regime, a flat loss on long sequences means the vanishing one.

Problem set

1. **Factor and cut.** For x_1, x_2, x_3, x_4 : (i) write the full autoregressive factorization. (ii) Write the first-order Markov approximation. (iii) A bigram model over $|V| = 10,000$ words needs how many conditional-probability entries? A trigram? In one line, what does the ratio say about “the number of model parameters grows exponentially”?
2. **One recurrent step, by hand.** 1-neuron RNN, $\phi = \tanh$, $w_{xh} = 0.8$, $w_{hh} = -0.3$, $b_h = 0.1$, $x_1 = 1.0$, $h_0 = 0$. (i) Compute h_1 to 4 s.f. (ii) Now $x_2 = -1.0$: compute h_2 to 4 s.f. (iii) In one line: which term in the h_2 computation is “the memory,” and what would h_2 be if $w_{hh} = 0$?
3. **The sharing payoff, counted.** An RNN layer with input dimension $d = 50$ and hidden size $h = 100$ (include biases). (i) Count its parameters (W_{xh}, W_{hh}, b_h). (ii) An MLP applied separately at each of $T = 30$ timesteps with its own weights would need how many? (iii) In one line, connect this to §44.7's $8,400\times$ and say what the RNN count does as T grows.
4. **The two-component recursion.** (i) In words: why does $\partial L / \partial h_t$ have a “current loss” term and a “future loss” term — which part of the unrolled graph does each come from? (ii) At $t = T$, which term vanishes and why? (iii) §42.4's $\delta^{[l]}$ recursion pushes error through *layers*; BPTT pushes it through *time*: in one line each, name the matrix that carries it in each case.
5. **Finish the BPTT eg.** Using §45.6's forward values: (i) derive $\partial L / \partial w_{hy} = \sum_t \frac{\partial L}{\partial y_t} h_t$ from $y_t = w_{hy} h_t$ and evaluate it. (ii) Derive $\partial L / \partial w_{xh} = \sum_t \frac{\partial L}{\partial h_t} (1 - h_t^2) x_t$ and evaluate it. (iii) In one line: why does the $t=1$ term of $\partial L / \partial w_{hh}$ vanish while the $t=1$ term of $\partial L / \partial w_{xh}$ does not?
6. **Name the regime.** Each backward step multiplies the error by ≈ 0.64 . (i) After 20 steps, what fraction survives? (ii) Same question for a per-step factor of 1.5. (iii) In two lines, map (i)/(ii) to the deck's vanishing/exploding rows, and connect (i) to §42.8(ii)'s sigmoid ≤ 0.25 story.
7. **Clip vs truncate.** (i) In one line each: what does gradient clipping fix, and what does it *not* fix? (ii) Truncated BPTT with $\tau = 5$: in one line, what dependency length can the model no longer learn? (iii) In two lines: why does the deck still call truncated BPTT a “compromise” even though the hidden state keeps flowing forward?
8. **The LSTM's gradient path.** $f_t = (0.9, 0.2)$, $i_t = (0.5, 0.8)$, $\tilde{c}_t = (0.6, -0.4)$, $c_{t-1} = (1.0, 2.0)$. (i) Compute c_t . (ii) What is $\partial c_t / \partial c_{t-1}$ (element-wise)? (iii) In two lines: explain why the first dimension's gradient survives far better than the second's over many steps, using your answer to (ii).
9. **The GRU's mix.** $Z_t = (0.8, 0.1)$, $H_{t-1} = (1.0, -1.0)$, $\tilde{H}_t = (0.0, 2.0)$. (i) Compute $H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$. (ii) In one line each: what does $Z_t \approx 1$ do, and $Z_t \approx 0$? (iii) In one line: which LSTM gate does Z_t “act similarly to” (deck's words), and why?
10. **Generate and judge.** (i) Target “A B C D”, predicted “A B”: compute BLEU with $k = 2$ (brevity penalty included). (ii) In two lines: why is the brevity penalty needed — what would the score be without it? (iii) Teacher forcing feeds the decoder ground-truth tokens during training but its own predictions at inference: in one line, state the resulting mismatch; in one line, say whether beam search (§45.11) fixes it.

target $P(x_1, \dots, x_T)$, autoregressive factorization, Markov/ n -gram (fixed memory, exponential parameters), linear window regression ($\hat{x}_t = \sum w_i x_{t-i} + b$; three limitations incl. error accumulation ϵ_1, ϵ_2), MLP/CNN limitations (“No Parameter Sharing Across Time”, “We need a model that processes sequences step-by-step, with memory”), tokenization/vocabulary (the/un k example), random sampling (τ , shift-by-one targets), perplexity (formula, “ k words” intuition), RNN hidden state ($P(x_t | x_{<t}) \approx P(x_t | h_{t-1})$, $h_t = f(x_t, h_{t-1})$), “same weight matrix at every time step”, neuron/layer equations, next-character worked example (corrected per §45.4 Note), BPTT (“not a new algorithm”, two-component recursion “the heart of BPTT”, parameter sums, 1-neuron worked example corrected per §45.6 Note), instability product $\prod W_{hh}^T \text{diag}(\phi')$, twin-problems table (clipping / “more complex RNN architectures”), truncated BPTT (τ , trade-off), LSTM (dual states, four gate equations, additive cell update, gradient-flow interpretation), GRU (2014, two gates, no separate cell, all equations, “trains faster... just as well”), bidirectional RNNs ($H_t = [\vec{h}_t, \overleftarrow{h}_t]$, “I am ____ happy” walkthrough), encoder-decoder (context vector, teacher forcing, masking, cross-entropy per timestep), BLEU (formula + worked example), beam search (steps, k , length normalization, greedy = $k=1$; illustration credited to d2l.ai); **Time_Series_Analysis_using_RNN.ipynb** — AMZN pipeline, **prepare_dataframe_for_lstm**, `MinMaxScaler(-1,1)`, (`samples`, `timesteps`, `features`) reshape, RNN class (`nn.RNN(..., batch_first=True)` + `nn.Linear`), Adam $\eta=0.001$, `nn.MSELoss`, 50 epochs (all torch [recorded]); **sentiment-analysis.ipynb** — IMDB pipeline, $h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$ framework, **SentimentRNN/SentimentLSTM** (embedding $\rightarrow$ recurrent $\rightarrow$ dropout 0.5 $\rightarrow$ fc $\rightarrow$ sigmoid; LSTM hidden as tuple) (all torch [recorded]); deck figure credits — BPTT illustration (DOI 10.13140/RG.2.2.34411.08483), LSTM architecture (DOI 10.1109/ACCESS.2021.3125733), GRU architecture (DOI 10.1007/s11042-023-15571-y); d2l.ai RNN chapter index — live-checked, unrolled-RNN figure (Fig. 9.1, CC-BY-SA) cited as HTML-comment diagram reference only; book chapters 35 (§35.6), 36 (§36.4), 40 (§40.5(i)), 41 (§§41.4, 41.7, 41.14(ii)), 42 (§§42.3–42.4, 42.6, 42.8), 43 (§§43.9, 43.12), 44 (§44.7, §44.10).

Solutions — Chapter 45: RNNs — sequences and vanishing gradients

Full worked solutions. All numbers re-run in numpy [verified-NumPy].

1. Factor and cut

- (i) Autoregressive factorization:

$$P(x_1, x_2, x_3, x_4) = P(x_1) P(x_2 | x_1) P(x_3 | x_1, x_2) P(x_4 | x_1, x_2, x_3).$$

- (ii) First-order Markov: each term conditions only on the immediately previous token:

$$P(x_1, x_2, x_3, x_4) \approx P(x_1) P(x_2 | x_1) P(x_3 | x_2) P(x_4 | x_3).$$

- (iii) Bigram: one distribution over $|V|$ next-words for each of $|V|$ histories $\rightarrow |V|^2 = 10,000^2 = 10^8$ entries. Trigram: $|V|^2$ histories $\rightarrow |V|^3 = 10^{12}$ entries. Ratio $10^{12}/10^8 = 10^4 = |V|$: every extra token of memory multiplies the table by the vocabulary size — the deck’s “grows exponentially” made concrete.

2. One recurrent step, by hand

- (i) $h_1 = \tanh(0.8 \cdot 1.0 + (-0.3) \cdot 0 + 0.1) = \tanh(0.9) = 0.7163$.
- (ii) $h_2 = \tanh(0.8 \cdot (-1.0) + (-0.3)(0.7163) + 0.1) = \tanh(-0.8 - 0.2149 + 0.1) = \tanh(-0.9149) = -0.7235$.
- (iii) The memory term is $w_{hh} h_{t-1} = -0.3 \times 0.7163 = -0.2149$ — the only channel by which x_1 influences h_2 . With $w_{hh} = 0$: $h_2 = \tanh(-0.8 + 0.1) = \tanh(-0.7) = -0.6044$; the network becomes memoryless, each step a function of its own input only.

3. The sharing payoff, counted

- (i) W_{xh} : $50 \times 100 = 5,000$; W_{hh} : $100 \times 100 = 10,000$; b_h : 100. Total 15,100.
- (ii) One timestep's MLP: $50 \cdot 100 + 100 = 5,100$; thirty independent copies: $30 \times 5,100 = 153,000$ — about $10\times$ more, and it still can't share a pattern learned at position 5 with position 25 (§45.2(iii)).
- (iii) The RNN's 15,100 does not move as T grows — same weights, every timestep — the temporal twin of §44.7's $8,400\times$: sharing is what makes the count independent of the input size.

4. The two-component recursion

- (i) In the unrolled graph, h_t has two outgoing edges: one down to this step's output y_t (via W_{hy}), and one right to the next hidden state h_{t+1} (via W_{hh}). The loss reaches h_t along both paths — the chain rule sums over all paths to a node (§42.2's rule), so $\partial L / \partial h_t = (\text{error from } y_t) + (\text{error from } h_{t+1})$.
- (ii) At $t = T$ there is no h_{T+1} : the “future loss” term vanishes, leaving $\partial L / \partial h_T = (\partial L / \partial y_T)(\partial y_T / \partial h_T)$.
- (iii) Feedforward $\delta^{[l]}$ recursion: the error crosses layers through $(W^{[l+1]})^T$ (§42.4). BPTT: the error crosses time through $W_{hh}^T \text{diag}(\phi')$ (§45.7's product) — same chain-rule job, different axis.

5. Finish the BPTT eg

- (i) $y_t = w_{hy}h_t$ gives $\partial y_t / \partial w_{hy} = h_t$, so by the chain rule $\partial L / \partial w_{hy} = \sum_{t=1}^2 \frac{\partial L}{\partial y_t} h_t$ (one shared weight, two votes — §45.5). Numerically: $(-0.3907)(0.7616) + (-0.8567)(0.9292) = -0.2976 - 0.7960 = -1.0936$.
- (ii) $h_t = \tanh(w_{xh}x_t + w_{hh}h_{t-1})$ (biases 0) gives $\partial h_t / \partial w_{xh} = (1 - h_t^2)x_t$, so $\partial L / \partial w_{xh} = \sum_t \frac{\partial L}{\partial h_t}(1 - h_t^2)x_t$. With $\partial L / \partial h_1 = -0.1657$ (current -0.1563 , future -0.0094), $\partial L / \partial h_2 = -0.3427$: $(-0.1657)(1 - 0.7616^2)(2) + (-0.3427)(1 - 0.9292^2)(3) = (-0.1657)(0.4200)(2) + (-0.3427)(0.1366)(3) = -0.1392 - 0.1404 = -0.2796$.
- (iii) $\partial L / \partial w_{hh}|_{t=1}$ carries the local factor $h_0 = 0$ — no signal passed through the memory edge at $t=1$, so nothing to blame (the §42.3 Note logic). $\partial L / \partial w_{xh}|_{t=1}$ carries the local factor $x_1 = 2 \neq 0$ — the input edge was live, so it earns its gradient.

6. Name the regime

- (i) $0.64^{20} \approx 1.33 \times 10^{-4}$ — about one ten-thousandth survives: **vanishing**.
- (ii) $1.5^{20} \approx 3325$ — the gradient compounds to thousands: **exploding**.
- (iii) (i) is the deck's vanishing row (“gradients for early time steps become nearly zero... cannot learn long-range dependencies”); (ii) is the exploding row (“gradients become enormous (NaN or infinity)”). §42.8(ii) is the same story along depth: a saturated sigmoid multiplies the error by at most ≈ 0.045 per layer, so deep stacks strangle the signal the way long sequences do here — the two halves of one phenomenon.

7. Clip vs truncate

- (i) Clipping fixes **explosion** (caps the update size when $\|\nabla\|$ exceeds the threshold); it does **not** fix vanishing — a near-zero gradient stays near-zero after rescaling — and it does not shorten the T -step product.
- (ii) Dependencies longer than $\tau = 5$ steps: the backward chain is cut, so no gradient path connects x_t to y_{t+6} .
- (iii) The hidden state still carries information forward through the full sequence (“maintaining its long-term memory”), so the *model* can in principle use long context — but the *gradient* can't reach back past τ , so the model can never *learn* to use it. Memory without learnability: the compromise.

8. The LSTM's gradient path

- (i) $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t = (0.9 \cdot 1.0 + 0.5 \cdot 0.6, 0.2 \cdot 2.0 + 0.8 \cdot (-0.4)) = (0.9 + 0.3, 0.4 - 0.32) = (1.2, 0.08)$.
- (ii) $\partial c_t / \partial c_{t-1} = f_t = (0.9, 0.2)$ element-wise — the additive update's derivative is the forget gate, no weight matrix involved.
- (iii) Over n steps the first dimension's gradient scales as 0.9^n ($0.9^{10} \approx 0.35$ — a third survives ten steps) while the second scales as 0.2^n ($0.2^{10} \approx 10^{-7}$ — gone). The gate *value* is the gradient highway: open ($f \approx 1$) preserves, closed ($f \approx 0$) erases — and the network learns which, per dimension.

9. The GRU's mix

- (i) $H_t = (0.8 \cdot 1.0 + 0.2 \cdot 0.0, 0.1 \cdot (-1.0) + 0.9 \cdot 2.0) = (0.8, -0.1 + 1.8) = (0.8, 1.7)$.
- (ii) $Z_t \approx 1$: the old hidden state is (almost) copied through — long-term memory preserved. $Z_t \approx 0$: the old state is discarded in favor of the candidate $\tilde{H}_t$ — fresh start.
- (iii) The **forget gate** f_t of the LSTM: both are sigmoids deciding “how much of the past survives,” and both enter an *additive* update ($Z_t \odot H_{t-1}$ here, $f_t \odot c_{t-1}$ there) — the deck's “acts similarly to the forget gate in an LSTM.”

10. Generate and judge

- (i) Predicted unigrams $\{A, B\}$: $p_1 = 2/2 = 1.0$. Predicted bigrams $\{AB\}$: $p_2 = 1/1 = 1.0$. Brevity penalty: $\exp(1 - 4/2) = \exp(-1) \approx 0.3679$. BLEU = $0.3679 \times (1.0 \times 1.0)^{1/2} = 0.3679$.
- (ii) Without the penalty the score would be $(1.0 \times 1.0)^{1/2} = 1.0$ — “misleadingly perfect”: pure n-gram precision rewards a two-word prefix of the target as if it were the whole answer. The penalty prices length.
- (iii) Mismatch: training never shows the decoder its own mistakes (it always gets the true previous token), while inference runs entirely on its own — possibly erroneous — outputs, so errors compound (the §45.2(ii) error-accumulation story, learned). Beam search does not fix it: beam search is a *decoding* strategy (better search over the model's distribution), not a *training* fix; the mismatch is baked in at training time.

Chapter 46: Attention and the transformer

Chapter 45 ended with the bottleneck: one fixed-size context vector must hold a whole paragraph (§45.10). Chapter 44 ended with a promise: “attention lets any position talk to any other directly, instead of waiting for pooling to bring them together” (§44.10(ii)). This chapter cashes both. The spine is the GenAI Weeks 9–10 deck (“Introduction to Attention and Transformer Architecture,” Balaji Srinivasan and Ganapathy Krishnamurthi, 99 slides — word embeddings, attention pooling, scaled dot-product attention, self vs cross attention, masked softmax, multi-head attention, positional encoding, residuals and layer norm, the full encoder–decoder, ViT, and the modern LLM arc), cross-checked against the notes PDF of the same deck (identical content, 20 pages), plus three companion notebooks: *machine_translation* (Bahdanau-attention seq2seq, German→English on Multi30k), *HuggingFace Inference* (sentiment, NER, translation pipelines), and Week 12’s *Transformers* (a nanoGPT built from scratch, trained on a 19-token toy corpus). Every numpy number below was re-run here and is marked [verified-NumPy]; deck, notebook, and torch numbers are transcriptions, marked [recorded].

Notation. A query, key, value pair is $q, k, v \in \mathbb{R}^d$; matrices $Q \in \mathbb{R}^{n \times d}$ (one row per query), $K \in \mathbb{R}^{m \times d}$, $V \in \mathbb{R}^{m \times v}$ stack them. $\alpha(q, k_i)$ is the attention weight of key k_i for query q . The deck writes the scale factor as $\sqrt{d}$; this book writes $\sqrt{d_k}$ — same number (d_k = dimension of the query/key vectors). Multi-head notation keeps the deck’s letters: head i ’s projected matrices are Q_i, K_i, V_i and the final projection is $W^{(o)}$. Position p ’s encoding is $PE(p)$.

46.1 Two promises this chapter cashes

i) The context-vector squeeze. §45.10’s encoder compresses the entire input into one fixed-size vector; the decoder sees nothing else. For short sentences this works; past “typically 30–40 tokens” performance “degrades dramatically” (deck) [recorded]. Attention deletes the squeeze: at every generation step, the decoder computes a *fresh* context — a weighted mix of *all* encoder states, with the weights chosen per step. The sentence is never compressed at all.

ii) The shortcut. §44.10(ii) promised that “attention lets any position talk to any other directly, instead of waiting for pooling to bring them together.” Self-attention does exactly that inside one sequence: token 1 and token 50 interact in a single matrix multiply, with no recurrence chain (§45.7’s T -step product) and no pooling pyramid (§44.6) in between. The deck’s table calls these “direct interactions between any two tokens,” against the RNN’s $O(T)$ sequential depth and the CNN’s slow buildup to global context [recorded].

Basically, ... “Two old limits, one fix. RNNs route everything through a single memory vector and a long relay chain; CNNs shrink the picture layer by layer to connect far-apart things. Attention lets every position look at every other position in one step — each output token picks what it needs from the whole input, afresh, every time.”

46.2 Embeddings: turning words into vectors

Before attention can mix words, words must become vectors. **Def (the deck’s).** “A word embedding is a way of representing a word as a list of numbers, called a vector.” Each vocabulary entry gets one: King $\rightarrow [0.12, -0.45, 0.88, \dots, -0.15]$ [recorded]. The numbers are coordinates in a “meaning space”: words with similar meanings land near each other (“cat” nearer “dog” than “car”), and *directions* capture relations — the deck’s famous relation:

$$\overrightarrow{\text{King}} - \overrightarrow{\text{Man}} + \overrightarrow{\text{Woman}} \approx \overrightarrow{\text{Queen}} \quad [\text{recorded}].$$

Two regimes:

i) Static embeddings. Word2Vec, GloVe, FastText: one fixed vector per word, learned from co-occurrence statistics over billions of words (“a word is known by the company it keeps”). Weakness: “bank” gets the same vector in “river bank” and “money bank” [recorded].

ii) Contextual embeddings. BERT reads the sentence both ways; GPT reads left-to-right. The vector for a word changes with the sentence it sits in. These are what a transformer layer produces — the embedding of “it” in “the animal didn’t cross the street because it was too tired” already knows which “it” it is (§46.9).

How they’re learned. Nobody hand-writes them. Either learn from a big corpus, or, inside a model: **1)** “Initialize Randomly: Assign a vector of random numbers to every word in your vocabulary,” **2)** “Learn Jointly: Train a neural network on a specific task... As the model learns, it simultaneously adjusts the word embedding vectors along with its other weights” [recorded].

Basically, ... “Embeddings are word coordinates. Similar words sit near each other; arrows between them capture relationships. The old ones give each word one fixed address; transformer ones compute the address fresh from the surrounding sentence.”

46.3 Attention, in one formula: queries, keys, values

Def (the deck’s). “Attention can be described as a process of mapping a Query and a set of Key-Value pairs to an output” — “the core of modern self-attention.” The deck’s cookbook analogy: the Query is the dish you want to cook; each recipe has a **Key** (its title, “this is what I am”) and a **Value** (its contents, “this is what I have to offer”). You **1)** compare the query to every key to score relevance, **2)** normalize the scores into focus percentages, **3)** blend the values by those percentages [recorded].

The technical implementation. “Each word in a sentence is disentangled into three vectors”: **Query (Q):** “the current word asking, ‘Who should I pay attention to?’” **Key (K):** “for each word in the sentence saying, ‘This is what I am.’” **Value (V):** “for each word in the sentence saying, ‘This is what I have to offer.’” These are *learned linear projections* of the embeddings [recorded].

The core operation: attention pooling. Given a query q and a database $D = \{(k_1, v_1), \dots, (k_m, v_m)\}$, the output is a weighted sum of values:

$$\text{Attention}(q, D) = \sum_{i=1}^m \alpha(q, k_i) v_i \quad [\text{recorded}].$$

The scalars $\alpha(q, k_i)$ are the **attention weights** — how much weight each value earns. The mechanism “pays attention” to the v_i whose α is significant.

What the weights can be. Three regimes (deck) [recorded]: **i) Convex combination** (the standard setting): $\alpha \geq 0$ and $\sum_i \alpha = 1$ — the output is a genuine average of values. **ii) Hard attention:** one weight is 1, the rest 0 — a database lookup; non-differentiable, so rarely used in deep models. **iii) Average pooling:** all weights equal $1/m$ — every value counted equally.

Computing weights: two steps. **1)** Choose any scoring function $a(q, k)$ measuring query–key relevance. **2)** Apply softmax to turn scores into a convex combination:

$$\alpha(q, k_i) = \frac{\exp(a(q, k_i))}{\sum_{j=1}^m \exp(a(q, k_j))} \quad [\text{recorded}].$$

“Softmax” = exp each score, divide by the sum: non-negative, sums to 1, and differentiable — “its gradient never vanishes,” which is why deep learning uses it.

Basically, ... “Attention is a soft database query. You hand over a question (query); each item advertises itself (key) and holds content (value). Relevance scores get squashed into percentages by softmax; your answer is the weighted blend of contents.”

46.4 From kernels to the dot product

Any scoring function works. The deck starts from the **Gaussian kernel** (one of three similarity functions it lists, with boxcar and Epanechnikov):

$$a(q, k) = \exp(-\tfrac{1}{2}\|q - k\|^2) \quad [\text{recorded}].$$

Now expand the exponent:

$$-\tfrac{1}{2}\|q - k\|^2 = q^\top k - \tfrac{1}{2}\|k\|^2 - \tfrac{1}{2}\|q\|^2 \quad (\text{deck, equation 1}) \quad [\text{recorded}].$$

Two terms simplify away (deck’s argument) [recorded]: **1)** the q term $-\tfrac{1}{2}\|q\|^2$ is identical for all (q, k_i) pairs — under softmax normalization “this term cancels out entirely”; **2)** the key term $-\tfrac{1}{2}\|k_i\|^2$ “can be dropped if the keys have constant norms” — “often the case, such as when keys are generated by layer normalization” (§46.11). What’s left: the plain **dot product** $q^\top k_i$.

Basically, ... “Dot-product attention is Gaussian similarity with the boring parts cancelled: the query-only term dies in softmax, the key-only term dies when keys have equal length, and the pure ‘how alike are they’ part is just the dot product.”

46.5 Why the $\sqrt{d_k}$: scaled dot-product attention

Dot products grow with dimension — and big scores kill softmax. The deck’s variance argument [recorded]: if $q, k \in \mathbb{R}^d$ have independent entries with zero mean and unit variance, their dot product has zero mean but variance d (each of the d terms $q_i k_i$ has variance $E[q_i^2]E[k_i^2] = 1$). Unscaled, the scores spread by $\sqrt{d}$; softmax saturates to a near-one-hot vector, its gradients die, and training stalls. The fix (Vaswani et al., 2017):

$$a(q, k_i) = \frac{q^\top k_i}{\sqrt{d_k}} \quad [\text{recorded}].$$

Dividing by $\sqrt{d_k}$ keeps the score variance at 1 regardless of vector length — softmax stays in its sensitive middle range. Note how this rhymes with §43.9’s scaling sermon: networks break when numbers drift off a sane scale; here the architecture fixes the scale instead of the optimizer.

The final form (matrix notation). Queries $Q \in \mathbb{R}^{n \times d}$, keys $K \in \mathbb{R}^{m \times d}$, values $V \in \mathbb{R}^{m \times v}$:

$$\text{Output} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \in \mathbb{R}^{n \times v} \quad [\text{recorded}].$$

Each of the n queries gets one row: its attention weights over the m keys, times the values.

Basically, ... “Long vectors make dot products huge, huge scores make softmax lazy (everything looks certain), and lazy softmax means dead gradients. Divide by $\sqrt{d}$ and the scores stay a manageable size no matter how wide the vectors get.”

46.6 The matrix formula, by hand

eg (own example — [verified-NumPy]). Two tokens, $d = 3$: $x_1 = (1, 0, 1)$, $x_2 = (0, 1, 1)$. Take

$$W_q = W_k = I_3 \text{ (queries and keys are the raw embeddings) and } W_v = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

- i) Scores $QK^\top$: $x_1^\top x_1 = 2$, $x_1^\top x_2 = 1$, $x_2^\top x_1 = 1$, $x_2^\top x_2 = 2$. Each token resembles itself most — exactly as it should.
- ii) Scale by $\sqrt{3} \approx 1.7321$: rows become $[1.1547, 0.5774]$ and $[0.5774, 1.1547]$.
- iii) Softmax: $e^{1.1547} \approx 3.1736$, $e^{0.5774} \approx 1.7814$, sum 4.9550 $\rightarrow$ weights $[0.6405, 0.3595]$; the rows are symmetric.
- iv) Values: $v_1 = x_1 W_v = (2, 1)$, $v_2 = (1, 2)$. Output row 1: $0.6405(2, 1) + 0.3595(1, 2) = (1.6405, 1.3595)$; row 2: $(1.3595, 1.6405)$.

Each output is a convex mix of both values — mostly its own token, about a third of the other. That is the whole mechanism on paper; numpy agrees to four decimals.

46.7 Self-attention, cross-attention, and the Bahdanau original

i) Self-attention. “Relates different positions of a single sequence... to compute a representation of that same sequence” [recorded]. Q, K, V all come from the same input. The question: “For each token in this sequence, how relevant are the other tokens in the same sequence?” — the core of encoders like BERT.

ii) Cross-attention. “Relates the positions of two different sequences” [recorded]. Queries come from one sequence (the decoder), keys and values from another (the encoder). The question: “While generating a word in the target sentence, which words from the source sentence are most relevant?” — the mechanism that replaced §45.10’s context vector in translation and summarization.

The original: Bahdanau attention. Attention predates the transformer — it was bolted onto §45.10’s RNN encoder-decoder first (the notebook’s class is literally named **Attention**, “Bahdanau attention mechanism”). The deck’s formula for that generation: $\alpha_{ij} = \text{softmax}(\text{score}(s_{j-1}, h_i))$, $c_j = \sum_i \alpha_{ij} h_i$ [recorded]. The *machine_translation* notebook builds exactly this (German $\rightarrow$ English on Multi30k, bidirectional-GRU encoder, Bahdanau decoder): its **energy** = $\tanh(W[h; s])$, **attention** = $\text{softmax}(V \cdot \text{energy})$, **context** = $\sum(\text{attention} \cdot \text{encoder_outputs})$ [recorded — torch]. Trained 20 epochs with teacher forcing 0.6: test loss 3.445, perplexity 31.359, BLEU 0.3203 (BLEU-1 0.6327 down to BLEU-4 0.1672) [recorded]. The transformer (next sections) keeps the attention idea and deletes the RNN underneath.

Basically, ... “Self-attention: one sequence reading itself. Cross-attention: the decoder reading the encoder, position by position. The first one ran on top of RNNs (Bahdanau); the transformer asked what happens if you keep the attention and throw away the RNN — that is the rest of this chapter.”

46.8 The masked softmax

The problem. Batched sequences are padded to equal length (“Hello world <blank> <blank>”), and padding tokens “have no meaning and should not contribute to the attention output” [recorded].

The solution: masking. “Limit the attention summation to only the valid tokens.” Implementation cheat: “Instead of using conditional logic (which is slow on GPUs), we take the scores for the padded positions and set them to a very large negative number (e.g., -10^6).” Then $\exp(-10^6) \approx 0$ — padded tokens get exactly zero weight after softmax [recorded].

eg (own — [verified-NumPy]). Scores $[2.0, 1.0, -10^6]$: $e^2 \approx 7.389$, $e^1 \approx 2.718$, $e^{-10^6} = 0$. Weights: $[7.389, 2.718, 0]/10.107 = [0.7311, 0.2689, 0]$. The pad position is mathematically erased.

Note. Masking reappears in the decoder (§46.12) with a different target: instead of pads, it hides *future* tokens (the causal mask) — same $-\infty$ trick, different job.

Basically, ... “Padding tokens are filler. Adding $-\infty$ to their score before softmax is a cheap way to multiply them by zero: $\exp(-\text{huge}) = 0$.”

46.9 Multi-head attention

The problem. “In the sentence ‘The animal didn’t cross the street because it was too tired’, what does ‘it’ refer to? The animal or the street? A single attention mechanism might get confused” [recorded]. Sentences carry many relationship types at once.

The solution: multiple heads. “Multi-Head Attention runs all these experts at once” — the deck’s committee-of-experts analogy: Expert 1 focuses on syntax (subject–verb agreement), Expert 2 on semantics (similar meanings), Expert 3 on pronouns (linking “it” to the noun) [recorded]. Formally: project Q, K, V into h lower-dimensional subspaces, run attention independently in each, then recombine.

For head $i \in \{1, \dots, h\}$, with learnable projection matrices $W_i^{(q)}, W_i^{(k)}, W_i^{(v)}$:

$$Q_i = QW_i^{(q)}, \quad K_i = KW_i^{(k)}, \quad V_i = VW_i^{(v)}, \quad \text{head}_i = \text{Attention}(Q_i, K_i, V_i) \quad [\text{recorded}].$$

Then concatenate along the feature dimension and mix once more:

$$\boxed{\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^{(o)}} \quad [\text{recorded}].$$

“The final projection layer allows the model to learn how to best combine the information learned from the different attention heads.”

The parameter count. With $d_{\text{model}} = 512$, $h = 8$ heads, $d_k = d_v = 64$: each head’s three projections are $512 \times 64 = 32,768$ each (98,304 per head, 786,432 total), and $W^{(o)}$ is $512 \times 512 = 262,144$. Total $1,048,576 = 4 \cdot 512^2$ [verified-NumPy] — *exactly* what one full-dimensional single head ($3 \cdot 512^2 + 512^2$) costs. Splitting into heads buys expressiveness for free.

Basically, ... “One attention head can only learn one kind of relationship. Eight heads are eight specialists — grammar, meaning, pronouns — whose opinions get mixed by a final vote. And it costs exactly the same as one big generalist.”

46.10 Order is gone — positional encoding

The problem. Self-attention “is permutation-invariant. It sees a sentence as a ‘bag of words’ with no order” [recorded]: “The dog chased the cat” and “The cat chased the dog” are identical to it. Since order *is* meaning, the model needs position injected back.

The solution: add a position vector to each embedding. “Before feeding the words into the Transformer, we add a special vector called a Positional Encoding to each word’s embedding” — “like giving each word a unique timestamp or sequence number” [recorded]. The deck’s version (Vaswani et al.): sinusoidal encodings with per-dimension frequencies. For position p and dimension index i (embedding dimension d):

$$\boxed{PE(p, 2i) = \sin\left(\frac{p}{10000^{2i/d}}\right), \quad PE(p, 2i+1) = \cos\left(\frac{p}{10000^{2i/d}}\right)} \quad [\text{recorded}].$$

Why sines. For any fixed offset δ , $PE(p+\delta)$ is a *linear transformation* of $PE(p)$ — “a constant rotation, which is easy for the model to learn” — from the sum-of-angles identities $\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$

and $\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta$ [recorded]. So relative positions cost the model a linear map, not memorization. Low dimensions (small i) oscillate fast with position; high dimensions change slowly.

eg (own — [verified-NumPy]). $d = 4$: $PE(0) = [\sin 0, \cos 0, \sin 0, \cos 0] = [0, 1, 0, 1]$. For $p = 1$: $[\sin 1, \cos 1, \sin 0.01, \cos 0.01] = [0.8415, 0.5403, 0.0100, 1.0000]$. Check the rotation on the first pair with

$$\delta = 1: \begin{bmatrix} \cos 1 & \sin 1 \\ -\sin 1 & \cos 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = [\sin 1, \cos 1] = [0.8415, 0.5403] \text{ — matches } PE(1)\text{'s first two entries exactly.}$$

Note. The nanoGPT notebook (§46.15) instead *learns* positional embeddings (`nn.Embedding(context_length, n_embeddings)`) — a learned lookup table rather than a fixed formula. Both work; the sinusoidal version extrapolates to longer sequences, the learned version is one less thing to design.

Basically, ... “Attention reads a bag of words, but language is a sentence. So we stamp each word with a position code made of slow and fast sine waves — and because shifting a sine is just a rotation, ‘five words ahead’ is a linear operation the model can actually learn.”

46.11 Training deep stacks: residuals and layer norm

A transformer is “a deep stack of Attention Layers” — dozens or hundreds of sublayers. “Two more components are crucial” (deck): residual connections and layer normalization [recorded].

i) Residual connections. Problem: “the original input information can get distorted or lost after passing through many transformation layers.” Solution: “An ‘Information Express Lane’ — add the sublayer’s input to its output. “This ‘skip connection’ allows the original signal to bypass the transformation, ensuring it’s never lost” [recorded]. Same family as U-Net’s skip connections (§44.9) — and the same gradient argument: the identity path gives every layer a direct gradient route, the architectural answer to §45.7’s vanishing product.

ii) Layer normalization. Problem: “As data passes through layers, the numbers (values in the vectors) can become very large or very small, making the training process unstable.” Solution: rescale each vector to mean 0, variance 1 — “independently for each example in the batch, using only the features of that single example” (the difference from batch norm) [recorded]. The math, for a vector x with H hidden units:

$$\mu = \frac{1}{H} \sum_{i=1}^H x_i, \quad \sigma^2 = \frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2, \quad \hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y_i = \gamma_i \hat{x}_i + \beta_i \quad \text{[recorded].}$$

The learnable gain γ and bias β “restore the expressive power of the network” — plain normalization alone would flatten every layer’s output to the same shape. Why it matters: it “makes the residual connections (‘Add’) effective. Without normalization, repeatedly adding vectors would cause their magnitudes to grow layer by layer.”

eg (own — [verified-NumPy]). $x = [1, 2, 3, 4]$: $\mu = 2.5$, $\sigma^2 = 1.25$, $\sigma \approx 1.1180 \rightarrow \hat{x} = [-1.3416, -0.4472, 0.4472, 1.3416]$.

Basically, ... “Residuals are a skip lane: every sublayer adds its correction to the original signal instead of replacing it, so gradients always have a direct path home. Layer norm keeps the numbers on a sane scale (mean 0, variance 1) after every addition, so stacking a hundred layers doesn’t blow up.”

46.12 The transformer, end to end

The big picture. “The Transformer follows a classic encoder-decoder structure”: the **encoder** “reads the entire input sequence... builds a rich, context-aware representation of each token” (a stack of identical layers); the **decoder** “generates the output sequence one token at a time,” attending to the encoder’s output [recorded]. “Unlike RNNs, the Transformer processes all input tokens at once, relying entirely on self-attention instead of recurrence” — §46.1(ii)’s promise in architecture form.

Inside an encoder layer. Two sublayers, each wrapped as residual + layer norm (“Add & Norm”) [recorded]: **1)** multi-head self-attention — “allows each token to look at all other tokens in the input sequence to understand its context”; **2)** a positionwise feed-forward network — “a simple, fully connected neural network applied independently to each token’s representation” (expand-activate-project, §46.15’s nanoGPT uses $4\times$ expansion with ReLU).

Inside a decoder layer. The same, plus one extra sublayer between them [recorded]: **1)** **masked** multi-head self-attention — “prevents the decoder from looking at future tokens in the output sequence it is trying to predict” (the causal mask: score entries above the diagonal are set to $-\infty$ before the softmax, §46.8’s trick); **2)** **encoder-decoder cross-attention** — “the queries (Q) come from the decoder, while the keys (K) and values (V) come from the encoder’s final output”; **3)** the feed-forward network.

The full data flow. **1)** Input embeddings: tokens $\rightarrow$ vectors. **2)** Positional encoding: add §46.10’s vectors. **3)** Encoder stack: N identical layers refine the representations. **4)** Decoder stack: previously generated tokens (shifted right) plus positional encoding $\rightarrow N$ layers: masked self-attention, then cross-attention on the encoder output. **5)** Final linear layer + softmax: “a probability distribution over the entire vocabulary for the next token” [recorded].

Complexity. The deck’s RNN-vs-transformer table: RNNs cost $O(T)$ sequential steps; the transformer does it in $O(1)$ parallel depth — matrix ops, no relay [recorded]. In FLOPs, attention costs $O(T^2d)$ against the RNN’s $O(Td^2)$; at $T = 1000, d = 512$: 5.12×10^8 vs 2.62×10^8 [verified-NumPy]. So the transformer does roughly *twice* the arithmetic — and wins anyway, because the arithmetic is parallel and the path length is constant. Scaling with hardware beats scaling with patience.

Basically, ... “Encoder: read everything, understand each token in full context. Decoder: write one token at a time, never peeking ahead (causal mask), always consulting the encoder (cross-attention). The whole thing is attention plus two-layer MLPs — no recurrence anywhere — so training runs in parallel.”

46.13 Three flavors: BERT, GPT, and the full stack

The original transformer is encoder-decoder (T5, BART — “Translation, Summarization” [recorded]), but “many tasks do not require this full structure”:

i) Encoder-only (BERT). “Goal: Understand the input. Sees: The full context (bidirectional).” Pre-trained with **masked language modeling**: 15% of tokens are replaced with [MASK] (“the cat sat on the [MASK]”) and the model must recover them from *both* sides — plus **next sentence prediction**: “whether sentence B is the actual sentence that follows A” [recorded]. Best for NLU: classification, question answering, NER.

ii) Decoder-only (GPT). “Goal: Generate an output. Sees: Only the past (unidirectional)” via the causal mask. Pre-trained on one task: **standard language modeling** — given “The cat sat on the”, predict “mat” [recorded]. Everything becomes text generation: “Translation becomes: ‘English: The cat sits. French: ?’” **Classification becomes: ‘Review: Great movie! Sentiment: ’** [recorded]. This is the architecture “the modern LLMs” use — simpler (one stack), scales better, generation is what users want.

BERT vs GPT (deck’s table, condensed) [recorded]:

	BERT (encoder)	GPT (decoder)
Visibility	Bidirectional — all tokens see each other	Causal — tokens see only the past
Pre-training	Masked language modeling	Next-token prediction
Best for	Understanding (NLU)	Generation (NLG)

The scale arc (deck’s table) [recorded]: original transformer — 2017, 65M params, 6 layers, 4.5M sentences; GPT-2 — 2019, 1.5B, 48 layers; GPT-3 — 2020, 175B, 96 layers, 300B tokens; GPT-4 — 2023, $\approx$ 1T (est.), 3–10T tokens; LLaMA 3 — 2024, 8B/70B, 126 layers, 15T tokens. Context windows went $512 \rightarrow 1\text{M}+$ tokens.

The empirical discovery behind it: **scaling laws** — performance “improves predictably with more parameters, training data, compute (GPU hours)” — and **emergent abilities** that “suddenly” appear at scale: basic arithmetic, code generation, few-shot learning, chain-of-thought reasoning [recorded].

The paradigm shift. Old: train from scratch per task. New: **1)** pre-train once on massive unlabeled text (“predict next token” — the model “learns language, facts, reasoning”); **2)** fine-tune cheaply on task data (“thousands, not billions” of examples), optionally with instruction tuning and RLHF — “Instruction Fine-tuning (SFT): Train on (instruction, response) pairs... Reinforcement Learning from Human Feedback (RLHF): Humans rank model outputs and train reward model optimized using RL (PPO)” [recorded]. Two more ingredients the deck names: **tokenization** (“unbelievable” → subword tokens [“un”, “believ”, “able”] via BPE/WordPiece/SentencePiece — “handles rare words, multiple languages, code”) and **massive training data** (“trillions of tokens,” “careful filtering and curation”) [recorded].

Basically, ... “BERT reads both ways and fills blanks — good at understanding. GPT reads left-to-right and predicts the next word — good at writing; it turned out to be good at everything else too, just by framing tasks as text. Train one giant model once, adapt it cheaply for everything — that is the whole LLM economy.”

46.14 The Vision Transformer

The challenge. Transformers eat 1D token sequences; images are 2D pixel grids. “Applying self-attention directly to every pixel in an image is computationally infeasible. For a modest 224×224 image, the number of pixel-to-pixel interactions would be $(224^2)^2 \approx 2.5 \times 10^9$ ” [recorded].

The solution: patches as tokens. “The core idea of the Vision Transformer (ViT) is to break an image down into a series of smaller, fixed-size patches and treat these patches as the tokens in a sequence.” Three steps [recorded]: **1) Patching:** split 224×224 into 16×16 patches. **2) Flattening:** each patch unrolls to a vector ($16 \times 16 \times 3 = 768$ numbers). **3) Linear projection:** an MLP maps it to the model dimension d .

The convolutional shortcut. The whole patchify-flatten-project pipeline “can be efficiently implemented as a single 2D convolution layer”: kernel size = patch size (16×16), stride = patch size (16, non-overlapping), output channels = embedding dim (768) [recorded]. Chapter 44’s convolution as a patch extractor.

eg (the deck’s — [verified-NumPy]). $H \times W = 224 \times 224$, $P \times P = 16 \times 16$, $D = 768$: $N = (224/16) \times (224/16) = 14 \times 14 = 196$ patches, each a 768-dim vector — sequence shape $\mathbb{R}^{196 \times 768}$. Prepend the learnable [CLS] token (below): 197 tokens. The conv shortcut uses $768 \times (16 \times 16 \times 3) + 768 = 590,592$ parameters.

The [CLS] token. “Inspired by the BERT model in NLP,” a learnable token prepended to the patch sequence acts as “a global information aggregator”: after the encoder stack, “the final, output vector corresponding to the ‘[CLS]’ token’s position is considered the representation of the entire image” and goes to “a simple MLP head for the final classification” [recorded]. (ViT uses *learnable* positional embeddings $e_i = z_i + P_{\text{pos},i}$, not sinusoidal ones [recorded].)

ViT vs CNNs (deck’s table, condensed) [recorded]: CNNs carry strong inductive bias — locality, translation equivariance — so they are “data-efficient and powerful on small to medium-sized datasets.” ViTs make “almost no prior assumptions... aside from the initial patching” and learn global patch-to-patch relationships with self-attention — but are “data hungry” (pre-training on JFT-300M / ImageNet-21k), and “when pre-trained on enormous datasets... often outperform state-of-the-art CNNs when fine-tuned on downstream tasks.” This is §44.10(ii)’s shortcut in vision: any patch talks to any other patch in one step, instead of waiting for pooling (§44.6’s pyramid) to bring them together.

Basically, ... “Chop the image into 196 puzzle pieces, flatten each into a vector, and feed the pile to a transformer like it’s a sentence. Self-attention lets the eye in the corner talk directly to the ear in the opposite corner — no pooling pyramid needed. The price: no built-in ‘things are local’ bias, so you need mountains of data to learn it.”

46.15 The notebooks: a mini-GPT and pretrained models in one line

i) **Transformers.ipynb: nanoGPT from scratch** [recorded — torch code]. A decoder-only model on a toy corpus (“lemon tastes sour / apple tastes sweet / ...”, a 19-token word-level vocab (17 words plus UNKNOWN/PADDING specials), `context_length = 12`): embedding dim 20, 2 layers, 2 heads, dropout 0.2, AdamW at 10^{-4} (§43.12’s transformer default, carried over whole), 10{,}000 iterations, batch 64. Architecture mirrors §§46.9–46.12: a **Head** does $K = XW_k$, $Q = XW_q$, $V = XW_v$, weights = $QK^\top/\sqrt{d_k}$, causal-mask with $-\infty$, softmax, output = weights V ; **MultiheadAttention** concatenates heads and projects; **TransformerBlock** is **pre-norm** — LayerNorm $\rightarrow$ attention $\rightarrow$ residual, then LayerNorm $\rightarrow 4\times$ feed-forward (ReLU) $\rightarrow$ residual (note: the deck’s §46.12 diagrams show post-norm “Add & Norm”; this implementation normalizes *before* each sublayer, the modern stable choice); the full model stacks blocks, applies a final LayerNorm, projects to the vocabulary, and trains with cross-entropy. Generation is autoregressive with top- k sampling. Results after training: prompt “i like sour so i like” $\rightarrow$ **lemon at 99.46%**; “i like juicy so i like” $\rightarrow$ apple 61.55% / orange 37.68%; “i like spicy so i like” $\rightarrow$ orange 64.83% / chili 33.29% [recorded]. Even at 20 dimensions and 2 heads, the model picks up the taste fruit pattern — §46.13’s “emergent” phenomenon in miniature.

ii) **HuggingFace Inference.ipynb: the ecosystem** [recorded]. Once trained, transformers are infrastructure: `transformers.pipeline` gives one-line access — sentiment analysis (default pipeline, plus a Twitter-RoBERTa variant), NER (BERT-Large fine-tuned on CoNLL-03), machine translation (T5-base English $\rightarrow$ French, a dedicated English $\rightarrow$ German model), and Gemini API calls for text generation, translation, and summarization. This is §46.13’s pre-train + fine-tune paradigm as a developer tool: the giant model trains once; you call it.

Basically, ... “Twenty dimensions and two attention heads are enough to learn ‘spicy goes with chili’ from 430 characters — scale that thought by ten orders of magnitude and you get GPT. And once someone has done the training, HuggingFace lets you borrow the result in one line of code.”

46.16 Where this goes next

- i) **Generative models (Chapter 47).** The decoder of §46.12 *is* a generator: it scores sequences (the §45.1 modelling target) and samples new ones. Chapter 47 adds GANs and diffusion — two other ways to generate, built on the §44.10(iii) conv stacks and this chapter’s decoder.
- ii) **The optimizer rides along.** Chapter 43’s §43.12 names AdamW the transformer standard; the mini-GPT notebook obeys. No new optimization machinery was needed in this chapter — attention is just another differentiable module (§42.4’s machinery handles it).
- iii) **The sharing moral, updated.** Chapter 44 shared across space, Chapter 45 across time — both bought efficiency and paid with path length. Attention pays the opposite way: direct pairwise interactions (§46.1(ii)), $O(T^2)$ compute, in exchange for hardware parallelism. Pick your poison per §40.5(i)’s shapes-first discipline: if T is huge, that quadratic term is the first thing to check.

Problem set

1. **One query, three keys.** Values $v_1 = (2, 0)$, $v_2 = (0, 4)$, $v_3 = (1, 1)$ with attention weights $\alpha = (0.6, 0.3, 0.1)$. (i) Compute $\text{Attention}(q, D)$. (ii) Show the result is a convex combination (weights ≥ 0 , sum = 1). (iii) Recompute under hard attention, assuming key 2 wins.
2. **Why scaling matters.** (i) Scores [100, 50]: compute softmax to 4 decimals and state what the weight vector approximates. (ii) Scaled scores [10, 5]: compute softmax. (iii) In two lines, connect (i) to the $\sqrt{d_k}$ division of §46.5 — what regime do unscaled dot products in high dimensions land in?
3. **The Gaussian-to-dot-product derivation.** Starting from $a(q, k_i) = -\frac{1}{2}\|q - k_i\|^2$ (no exponential): (i) expand into the three terms; (ii) name which term softmax normalization erases and why; (iii)

state the assumption that erases the key-norm term, and name the chapter-46 component that usually enforces it.

4. **Multi-head parameter count.** $d_{\text{model}} = 512$, $h = 8$, $d_k = d_v = 64$. (i) Count one head's $W^{(q)}, W^{(k)}, W^{(v)}$ parameters. (ii) Add all eight heads and $W^{(o)}$ (512×512). (iii) Compare with a single full-dimensional attention ($d_k = 512$): what is the ratio, and what does it say about the “cost” of splitting into heads?
5. **The causal mask.** (i) Write the 4×4 lower-triangular mask as 1s/0s. (ii) Which positions can token index 2 (0-based) attend to? (iii) In one line, say why the decoder needs this but the encoder does not.
6. **Positional encoding by hand.** $d = 4$. (i) Write $PE(0)$. (ii) Compute $PE(1)$ numerically. (iii) Show the first pair $(PE(0, 0), PE(0, 1))$ rotates into $(PE(1, 0), PE(1, 1))$ for $\delta = 1$ using the sum-of-angles identity, and state what this means for relative positions.
7. **Masked softmax.** Scores $[2.0, 1.0, -10^6]$. (i) Compute the three weights. (ii) What does the third weight equal exactly after masking, and why? (iii) Name the two different jobs masking does in a transformer (§46.8 and §46.12).
8. **Patch math.** $224 \times 224 \times 3$ image, 16×16 patches, $D = 768$. (i) How many patches? (ii) How long is each flattened patch vector? (iii) How many parameters does the conv shortcut use (with bias)? (iv) What is the encoder's input sequence length including [CLS]?
9. **Cross-attention shapes.** Decoder sequence length $T_d = 8$, encoder length $T_e = 30$, model dim 512. (i) What are the shapes of the cross-attention weight matrix? (ii) Show the parameter count of the decoder's cross-attention layer is independent of T_d and T_e ; name the book's term for this property (Chapter 44/45).
10. **The quadratic bill.** (i) $T = 1000$, $d = 512$: compare attention's $O(T^2d)$ with an RNN's $O(Td^2)$ numerically. (ii) If attention does roughly twice the FLOPs here, why does the deck still say transformers “scale well with data + hardware” while RNNs “slow down as sequences grow”? (iii) As T grows at fixed d , which term dominates — and what does that imply for very long contexts?

Sources: GenAI Weeks 9–10 “Introduction to Attention and Transformer Architecture” deck (Balaji Srinivasan, Ganapathy Krishnamurthi), 99 slides — embeddings (meaning space, static vs contextual, king–man+woman queen, learn-from-scratch recipe), attention pooling ($\sum_i \alpha(q, k_i) v_i$), Q/K/V cookbook analogy and projections, Gaussian/boxcar/Epanechnikov kernels, Gaussian→dot-product derivation, $\sqrt{d}$ variance argument, softmax weight formula, matrix form softmax($QK^\top/\sqrt{d}$) V , self vs cross attention, masked softmax (-10^6 trick), RNN-vs-transformer table, multi-head (committee analogy, $Q_i = QW_i^{(q)}$ formulas, concat + $W^{(o)}$), permutation invariance, sinusoidal PE + relative-position rotation, residual connections, layer-norm math ($\mu, \sigma^2, \hat{x}, \gamma, \beta$), encoder/decoder layer diagrams, full data flow, BERT (MLM 15%, NSP) vs GPT (causal LM) table, scaling table (65M→1T est.), pre-train + fine-tune, tokenization, emergent abilities, SFT/RLHF, ViT (patch pipeline, conv shortcut, [CLS] token, 196-patch worked example, ViT-vs-CNN table); notes *Week 9–10.pdf* — same deck content, used as cross-check; *machine_translation.ipynb* — Multi30k pipeline, Bahdanau **Attention** class (energy/attention/context formulas), GRU encoder–decoder, teacher forcing 0.6, 20 epochs, test loss 3.445 / PPL 31.359 / BLEU 0.3203 (all torch [recorded]); *HuggingFace Inference.ipynb* — pipeline API: sentiment (Twitter-RoBERTa), NER (dbmdz/bert-large-cased-finetuned-conll03-english), MT (google-t5/t5-base en→fr, specific en→de), Gemini API examples (all [recorded]); *Week 12 Transformers.ipynb* — nanoGPT: 19-token vocab, context 12, $d = 20$, 2 layers, 2 heads, dropout 0.2, Adam $W 10^{-4}$, 10k iters; *Head/MultiheadAttention/TransformerBlock* (pre-norm, $4 \times$ FFN) classes, causal mask, generation samples (all torch [recorded]); diagram references — Vaswani et al. 2017 Fig. 1 (<https://arxiv.org/abs/1706.03762>), Dosovitskiy et al. 2020 Fig. 1 (<https://arxiv.org/abs/2010.11929>); book chapters 42 (§42.4), 43 (§43.9, §43.12), 44 (§44.6, §44.8, §44.10), 45 (§45.1, §45.7, §45.10, §45.11, §45.13).

Solutions — Chapter 46: Attention and the transformer

Full worked solutions. All numbers re-run in numpy [verified-NumPy].

1. One query, three keys

- (i) Attention = $0.6(2, 0) + 0.3(0, 4) + 0.1(1, 1) = (1.2 + 0 + 0.1, 0 + 1.2 + 0.1) = (1.3, 1.3)$.
- (ii) Every $\alpha_i \geq 0$ and $0.6 + 0.3 + 0.1 = 1.0$ — a convex combination, so the output lies inside the convex hull of $\{v_1, v_2, v_3\}$ (§46.3).
- (iii) Hard attention: key 2 takes all the weight, $\alpha = (0, 1, 0) \rightarrow \text{output} = v_2 = (0, 4)$. The v_1 and v_3 information is discarded entirely — the §46.3 point about differentiability.

2. Why scaling matters

- (i) softmax([100, 50]): $e^{100}/(e^{100} + e^{50}) = 1/(1 + e^{-50}) \approx 1.0000$, the second weight $\approx 1.93 \times 10^{-22} \approx 0.0000$. The vector approximates a one-hot [1, 0].
- (ii) softmax([10, 5]): $e^{10} \approx 22,026.47$, $e^5 \approx 148.41$, sum 22,174.88 $\rightarrow [0.9933, 0.0067]$. Still confident, but the second score earns a real (if small) weight.
- (iii) Unscaled dot products in dimension d spread with standard deviation $\sqrt{d}$ — for $d = 512$ that is ≈ 22.6 , landing squarely in the [100, 50]-style regime where softmax collapses to one-hot and its gradients vanish (§46.5). The $\sqrt{d_k}$ division rescales the spread to ≈ 1 , keeping softmax in its sensitive middle range where small score differences still move the weights.

3. The Gaussian-to-dot-product derivation

- (i) $-\frac{1}{2}\|q - k_i\|^2 = q^\top k_i - \frac{1}{2}\|k_i\|^2 - \frac{1}{2}\|q\|^2$.
- (ii) The $-\frac{1}{2}\|q\|^2$ term is identical for every i , so $\exp(a_i) = \exp(\text{const}) \cdot \exp(a_i - \text{const})$ and the constant factors out of both numerator and denominator of the softmax and cancels (§46.4).
- (iii) Keys must have constant norms (so $-\frac{1}{2}\|k_i\|^2$ is also i -independent and drops out the same way). Layer normalization (§46.11) is the component that usually enforces this — another reason it sits before attention in deep stacks.

4. Multi-head parameter count

- (i) One head: $W^{(q)}, W^{(k)}, W^{(v)}$ are each $512 \times 64 = 32,768 \rightarrow 3 \times 32,768 = 98,304$ parameters.
- (ii) Eight heads: $8 \times 98,304 = 786,432$; plus $W^{(o)}$ ($512 \times 512 = 262,144$): total 1,048,576.
- (iii) Single full-dimensional attention ($d_k = d_v = 512$): three 512×512 projections plus one 512×512 output = $4 \times 262,144 = 1,048,576$. Ratio 1:1 — splitting into eight heads costs exactly nothing extra in parameters (§46.9); the “committee of experts” is free.

5. The causal mask

- (i) Lower-triangular ones:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

- (ii) Token 2 attends to positions 0, 1, 2 — current and past, never future (the $-\infty$ trick of §46.8 erases the zeros).

- (iii) The decoder *predicts* future tokens; seeing them would let it cheat during training and break the train/test match of §45.10’s teacher-forcing discussion. The encoder reads the whole input to *understand* it — there is no future to leak, so no mask.

6. Positional encoding by hand

- (i) $PE(0) = [\sin 0, \cos 0, \sin 0, \cos 0] = [0, 1, 0, 1]$.
- (ii) $PE(1) = [\sin 1, \cos 1, \sin 0.01, \cos 0.01] = [0.8415, 0.5403, 0.0100, 1.0000]$.
- (iii) Sum-of-angles with $\alpha = 0, \beta = 1$: $\sin(0 + 1) = \sin 0 \cos 1 + \cos 0 \sin 1 = \sin 1$; $\cos(0 + 1) = \cos 0 \cos 1 - \sin 0 \sin 1 = \cos 1$. So the first pair of $PE(0)$ rotates by the fixed 2×2 matrix $\begin{bmatrix} \cos 1 & \sin 1 \\ -\sin 1 & \cos 1 \end{bmatrix}$ into the first pair of $PE(1)$ — $[0.8415, 0.5403]$ matches (ii) exactly. Meaning: for a fixed offset δ , position $p + \delta$ is a constant linear function of position p , so the model learns *relative* positions with a linear map instead of memorizing absolute ones (§46.10).

7. Masked softmax

- (i) $e^2 \approx 7.389, e^1 \approx 2.718, e^{-10^6} = 0$ (underflow) $\rightarrow [0.7311, 0.2689, 0]$.
- (ii) Exactly 0: $\exp(-10^6)$ underflows to zero in floating point, so the padded position contributes nothing to the weighted sum — “mathematically erased” (§46.8 eg).
- (iii) Job one (§46.8): erase *padding* tokens in batched inputs. Job two (§46.12): erase *future* tokens in the decoder’s causal mask. Same $-\infty$ mechanism, different targets.

8. Patch math

- (i) $N = (224/16)^2 = 14^2 = 196$ patches.
- (ii) Each flattens to $16 \times 16 \times 3 = 768$ numbers.
- (iii) Conv shortcut: $768 \text{ filters} \times (16 \times 16 \times 3) \text{ weights} + 768 \text{ biases} = 768 \times 768 + 768 = 590,592$.
- (iv) $196 + 1$ ([CLS]) = 197 tokens.

9. Cross-attention shapes

- (i) Queries come from the decoder (8 positions), keys/values from the encoder (30 positions): the weight matrix is 8×30 — each decoder position gets a distribution over all 30 encoder positions.
- (ii) The cross-attention projections are $512 \times d_k, 512 \times d_k, 512 \times d_v$ and $W^{(o)}$ — sized by dimensions only, never by T_d or T_e . Same $4 \cdot 512^2$ parameters for a 3-word caption or a 30-word paragraph: the book’s weight-sharing property (§44.10(i), the §45.2(iii) theme), here across positions rather than pixels or timesteps.

10. The quadratic bill

- (i) Attention: $T^2 d = 1000^2 \times 512 = 5.12 \times 10^8$. RNN: $T d^2 = 1000 \times 512^2 = 2.62 \times 10^8$. Attention does $\approx 1.95\times$ the FLOPs — roughly twice.
- (ii) The deck’s table (§46.12): the RNN’s cost is $O(T)$ *sequential depth* — step 500 cannot start before step 499 finishes, so adding hardware does not help. Attention’s $O(T^2 d)$ is one big matrix multiply: $O(1)$ sequential depth, fully parallel, exactly what GPUs/TPUs are built for. Transformers scale with *hardware*, RNNs scale with *patience*.

- (iii) At fixed d , the T^2 term dominates as T grows — attention becomes quadratic in context length. That is the ceiling the deck’s “512 tokens $\rightarrow$ 1M+ tokens” arc is really about (§46.13): long contexts are an active engineering fight, not a solved problem.

Chapter 47: Generative models: GANs and diffusion

Chapter 46 closed with the observation that the decoder of §46.12 *is* a generator: it scores sequences (the §45.1 modelling target) and samples new ones; §45.13(iii) said the same of the §45.10 decoder, and §44.10(iii) promised that “GANs and diffusion models are built on conv stacks like this chapter’s — the discriminator is a CNN classifier, and the generator is the §44.9 decoder run in reverse.” This chapter cashes all three. The spine is the GenAI Week 6 deck (“Introduction to Generative Adversarial Networks,” Balaji Srinivasan and Ganapathy Krishnamurthi, 65 slides — generative models, latent space, the GAN game, DCGAN, StyleGAN, training challenges) plus its companion notebook *GAN_from_Scratch.ipynb* (a vanilla GAN on 28×28 FashionMNIST images), and the GenAI Week 11 deck (“Introduction to Diffusion Models,” same authors, 94 slides — taxonomy, FID/IS/CLIP, DDPM forward/reverse, reparameterization, schedulers, DDIM, classifier-free guidance, latent diffusion/Stable Diffusion). Every numpy number below was re-run here and is marked [verified-NumPy]; deck, notebook, and torch numbers are transcriptions, marked [recorded]. “The deck” below means the GAN deck when GANs are discussed and the diffusion deck when diffusion is — they agree on everything they share.

Notation. p_{data} is the real data distribution; p_{model} (sometimes p_{θ}) the model’s. $z \sim p(z)$ is a latent/noise vector, usually $\mathcal{N}(0, I)$. The generator is $G(z; \theta_g)$, the discriminator $D(x) \in [0, 1]$. In diffusion, x_t is the image at noise level t , β_t the noise schedule, $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, and $\epsilon_{\theta}(x_t, t)$ the network’s predicted noise.

47.1 Two promises this chapter cashes

i) **The §46.16(i) promise.** The transformer decoder *generates* by sampling next tokens autoregressively. This chapter adds two more generator families: the GAN (learn the mapping noise $\rightarrow$ image in *one* shot, trained adversarially) and the diffusion model (learn to undo noise in *many* small steps). Three ways to turn noise into data — decoder, adversary, denoiser — closing the arc that began at §45.13(iii).

ii) **The §44.10(iii) promise.** Both new families run on Chapter 44’s conv stacks. The GAN’s discriminator becomes a CNN classifier at DCGAN scale (§44.6’s stack with a sigmoid head); its generator is the §44.9 U-Net decoder run in reverse (noise up to full image). The diffusion model’s noise predictor is a U-Net with skip connections (§44.9) — the same Lego, a new game.

iii) **The trilemma that motivated diffusion.** The diffusion deck’s table (condensed) [recorded]: VAEs give **stable training + good diversity** but **blurry samples** (the MSE loss “averages out fine details”); GANs give **sharp samples** but **poor diversity** (mode collapse) and **unstable training**. Diffusion asks: can we keep the stability and coverage of VAEs *with* GAN-level sharpness? Its answer is to replace one hard generation step with many easy denoising steps.

Basically, ... “Three ways to make new data from noise: the transformer writes it word by word (§46.12), the GAN forges it in one shot while a critic judges (§47.4), and diffusion sculpts it by repeatedly removing a little noise (§47.13). All three reuse the machinery of Chapters 41–46 — conv stacks, skip connections, attention, Adam.”

47.2 What a generative model is

Def (the deck’s). “A generative model is a type of statistical model that is capable of learning the underlying distribution of a dataset in order to generate new, synthetic data samples that are similar to the original data.” Formally: the real data comes from $p_{\text{data}}(x)$; the model learns a distribution $p_{\text{model}}(x)$ and training tries to make p_{model} as close to p_{data} as possible.

The generative vs. discriminative split. A discriminative model learns $P(y | x)$ — “given this image, cat or dog?” (logistic regression, SVMs, ordinary classifiers). A generative model learns $P(x)$ — “what makes an image look like a cat?” — and can then *sample* from it (VAEs, GANs, diffusion).

The deck’s taxonomy (condensed) [recorded]: - **i) Explicit density, tractable.** Define and compute $p(x)$ exactly. PixelCNN/RNN, Glow, NADE/MADE — autoregressive pixel-by-pixel models live here. - **ii) Explicit density, approximate.** $p(x)$ is intractable; optimize a proxy. VAEs maximize a lower bound on likelihood (ELBO); diffusion models are classified here too (“systematically add noise to data, then learn to reverse the process”). - **iii) Implicit density.** Never define $p(x)$ at all — just learn to *sample* from it. GANs are “the dominant family in this category.”

Why they’re useful (deck’s list) [recorded]: data generation, understanding latent structure, missing-data imputation (inpainting), anomaly detection, representation learning. The deck also notes the generative-classifier trick: with one generative model $P(x | y = c)$ per class, Bayes’ rule picks the class maximizing $P(x | y = c)P(y = c)$ — Naive Bayes is the textbook instance (§30’s territory).

The deck’s toy (2×2 grid, worked). Take an “image” as a 2×2 grid, each pixel black (0) or white (1): $2^{2 \times 2} = 2^4 = 16$ possible images. A generative model learns $P(x_{11}, x_{12}, x_{21}, x_{22})$ — for this toy, just count occurrences and normalize [recorded].

eg (the deck’s marginalization numbers — [verified-NumPy]). The deck’s hypothetical joint gives eight configs with $x_{11} = 1$: 0.05, 0.10, 0.15, 0.03, 0.02, 0.08, 0.07, 0.01. Marginalizing means summing over the other pixels:

$$P(x_{11} = 1) = 0.05 + 0.10 + 0.15 + 0.03 + 0.02 + 0.08 + 0.07 + 0.01 = 0.51.$$

Same idea, one step harder: **inpainting**. If $x_{11} = 1$ is known and the rest missing, the best fill is $\arg \max_{x_{\text{mis}}} P(x_{\text{mis}}, x_{\text{obs}})$; the deck’s table max is $P(1, 0, 1, 0) = 0.15$, so the completion is $(x_{12}, x_{21}, x_{22}) = (0, 1, 0)$ [recorded].

Note. The toy works because 16 is tiny. A 32×32 grayscale image has 2^{1024} possible binary configurations — explicit enumeration is “computationally impossible,” the deck’s motivating fact for everything that follows. Modern models never store the joint table; they learn a *function* that approximates it or samples from it.

Basically, ... “A generative model learns ‘what kind of data this is’ — the probability pattern — then rolls new dice weighted by that pattern. On a 2×2 toy you can write the whole probability table by hand; on real images the table is astronomically huge, so you need smarter tricks: GANs (§47.4) or diffusion (§47.10).”

47.3 The latent space: from enumeration to noise

For real data, the deck’s pivot is: don’t try to sample from $p(x)$ in pixel space — learn a **mapping** from a simple noise distribution to the data:

Def. Sample $z \sim p(z)$ (usually $\mathcal{N}(0, I)$ — easy). A generator $G(z; \theta_g)$ maps the latent space $\mathcal{Z}$ to data space $\mathcal{X}$: $x = G(z)$. Training finds θ_g^* so the distribution of $G(z)$ matches p_{data} .

Why it works (the deck’s intuition). **i)** Most pixel configurations are meaningless noise; real images sit on a thin “manifold” inside pixel space. **ii)** You cannot “pick random pixels and get a cat,” but you can pick a random point in a small “idea space” and let a trained function grow the idea into an image. **iii)** Each z is a different seed/recipe — the *noise is the source of variety*.

The paradigm shift (deck’s table, condensed) [recorded]: pre-deep-learning latent spaces (PCA’s §24 eigenfaces, factor analysis, LDA topics, SVD matrix factorization) went **data** $\rightarrow$ **latent** and asked “what are the hidden factors in my data?” Modern models flip the arrow — **noise** $\rightarrow$ **data** — and ask “how can I create new data that looks like my data?” The latent space got an *imposed* structure (Gaussian) instead of a discovered one, so any random point in it decodes to something sensible.

Basically, ... “Old latent space: take a real face, squeeze it into a short code (analysis). New latent space: pick a random short code from a nice distribution, grow a face from it (synthesis). Same word, opposite arrow — and the arrow reversal is what makes generation possible.”

47.4 The GAN: a two-player game

Proposed by Goodfellow et al. (2014) [recorded]. The deck’s analogy: an **aspiring artist** (the generator) forges paintings in a master’s style without ever seeing the originals, while an **art critic** (the discriminator) studies real masterpieces and forgeries to learn to tell them apart. The game ends when the forgeries fool the critic half the time.

i) The discriminator D : the critic. Wants to classify correctly — real data as real, fakes as fake. Its objective to maximize:

$$\max_D \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad [\text{recorded}].$$

First term: push $D(x) \rightarrow 1$ on real data. Second term: push $D(G(z)) \rightarrow 0$ on fakes.

ii) The generator G : the forger. Wants to fool D — minimize the second term above:

$$\min_G \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad [\text{recorded}].$$

The combined minimax objective [recorded]:

$$\boxed{\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]}$$

“The Generator tries to minimize the value function that the Discriminator tries to maximize” — a zero-sum game called **adversarial learning**. The ideal end state is a **Nash equilibrium**: G produces perfectly realistic data and D “outputs a 50% probability for all inputs” — the critic is fooled exactly half the time [recorded].

Note (the deck vs. the notebook). The deck teaches the minimax form above. In practice — including this chapter’s notebook (§47.7) — the generator is instead trained to **maximize** $\log D(G(z))$ (label fakes as real), which the deck flags as equivalent (“pushing the Discriminator’s output for fake images towards 1”) but with livelier gradients early on. §47.6 works both.

Basically, ... “Two networks, one lie detector. The forger makes fakes, the critic calls them out; both get better. The score to beat: when the critic is right exactly half the time — no better than a coin flip — the forgeries are perfect.”

47.5 Training: alternate, freeze, repeat

The deck’s training algorithm has two phases, run in turns [recorded]:

Phase 1 — train D (freeze G). Sample real $x \sim p_{\text{data}}$ and noise $z \sim p_z$, make fakes $G(z)$. Gradient **ascent** on the discriminator’s objective (it’s a max):

$$\theta_d \leftarrow \theta_d + \alpha \nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D(x^{(i)}) + \log(1 - D(G(z^{(i)})))]$$

Phase 2 — train G (freeze D). Sample fresh noise $z \sim p_z$. Gradient **descent** on the generator’s loss — gradients flow back *through the frozen discriminator* to reach G :

$$\theta_g \leftarrow \theta_g - \alpha \nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z^{(i)}))).$$

Repeat both phases until the Nash point (or exhaustion). The freezing matters: each network learns against a fixed opponent for one step, so the “moving target” only moves between steps.

Basically, ... “Coach the critic while the forger holds still; then coach the forger while the critic holds still. Two optimizers, two losses, one loop — and each half updates while the other is frozen, so the target stands still for a step.”

47.6 The losses, worked by hand [verified-NumPy]

eg (own). Mini-batch of 2 real, 2 fake. Suppose D outputs $D(x) = [0.9, 0.7]$ on real images and $D(G(z)) = [0.3, 0.6]$ on fakes.

- i) **Discriminator loss (BCE).** Each sample’s binary cross-entropy is $-\log D(x)$ (real) or $-\log(1 - D(G(z)))$ (fake):

$$\begin{aligned} L_D &= -\frac{1}{4} [\log 0.9 + \log(1 - 0.3) + \log 0.7 + \log(1 - 0.6)] \\ &= -\frac{1}{4} [-0.1054 - 0.3567 - 0.3567 - 0.9163] = 0.4338. \end{aligned}$$

- ii) **Generator loss, minimax version.** $L_G = -\frac{1}{2} [\log(1 - 0.3) + \log(1 - 0.6)] = -\frac{1}{2} [-0.3567 - 0.9163] = 0.6365$.

- iii) **Generator loss, non-saturating version** (the one the notebook actually uses). Label fakes as real: $L_G = -\frac{1}{2} [\log 0.3 + \log 0.6] = 0.8574$.

Note. When D is strong — say it gives a fake only 0.05 — the non-saturating loss is $-\log(0.05) \approx 2.9957$, still steep, while the minimax $\log(1 - 0.05) \approx -0.0513$ barely moves. That is the deck’s **vanishing-gradient** failure (§47.9): a critic that wins too early starves the forger of gradients; the non-saturating loss — the one the notebook actually trains with — is the practical patch.

Basically, ... “The critic’s loss is plain binary cross-entropy — real images should score 1, fakes 0. The forger’s smart loss is the same BCE with flipped labels: it pays the critic to score its fakes as 1. And when the critic gets too good, the forger needs the flipped-label version, because the textbook one goes flat.”

47.7 The notebook: a GAN on FashionMNIST, from scratch [recorded — torch]

The `GAN_from_Scratch.ipynb` builds exactly the game above — plain dense networks, no convolutions yet — on 60,000 grayscale 28×28 images. **Note (notebook naming slip).** The code loads `datasets.FashionMNIST`, but the comments call the images “handwritten digits” (wrong — FashionMNIST is clothing items, not digits) and the final comparison panel titles them “MNIST”. The code is authoritative: the dataset is FashionMNIST.

The two networks [recorded]: - **Generator.** Linear($64 \rightarrow 256$) $\rightarrow$ LeakyReLU(0.2) $\rightarrow$ Linear($256 \rightarrow 256$) $\rightarrow$ LeakyReLU(0.2) $\rightarrow$ Linear($256 \rightarrow 784$) $\rightarrow$ tanh. Latent vector $z \in \mathbb{R}^{64}$, tanh output matches the $[-1, 1]$ normalized images. - **Discriminator.** Linear($784 \rightarrow 256$) $\rightarrow$ LeakyReLU(0.2) $\rightarrow$ Linear($256 \rightarrow 256$) $\rightarrow$ LeakyReLU(0.2) $\rightarrow$ Linear($256 \rightarrow 1$) $\rightarrow$ sigmoid — a binary classifier (§44.6’s stack without the convolutions).

Parameter counts [verified-NumPy]. Generator: $(64 \cdot 256 + 256) + (256 \cdot 256 + 256) + (256 \cdot 784 + 784) = 16,640 + 65,792 + 201,488 = 283,920$. Discriminator: $(784 \cdot 256 + 256) + (256 \cdot 256 + 256) + (256 \cdot 1 + 1) = 200,960 + 65,792 + 257 = 267,009$.

The training recipe [recorded]: pixels normalized to $[-1, 1]$ to match the tanh head (denormalize with $(x + 1)/2$ for display); **BCE loss**; **Adam**, $\eta = 0.0002$ with separate optimizers for D and G ; batch 100; 100 epochs $\times$ 600 steps = 60,000 updates. The D step calls `fake_images.detach()` — no gradient flows into G while the critic trains (§47.5’s freeze, implemented as a detach). The G step is the non-saturating form: BCE against `real_labels` (fakes labelled 1).

Results [recorded]: average $L_D = 0.7651$, average $L_G = 2.0232$; final epoch $L_D = 0.8781$, $L_G = 1.5290$ (ratio $G/D = 1.7413$). Early on $D(x) \approx 0.95$, $D(G(z)) \approx 0.05$ — the critic dominates, exactly the regime §47.6’s note describes — and the generator closes the gap over the 100 epochs. The final panel shows generated digits next to real ones.

Basically, ... “Sixty-four random numbers in, a 28×28 digit out — two small MLPs, a coin-flip game, Adam at 0.0002. The critic learns fast (0.95 on real images by epoch 1), the forger catches up slowly, and after 60,000 alternating steps the fake digits look like digits.”

47.8 Scaling up: DCGAN and StyleGAN [recorded]

DCGAN (Deep Convolutional GAN). “A major step forward, successfully using deep convolutional nets for larger images” — and §44.10(iii) in the flesh: a CNN discriminator plus a conv-transpose generator. The deck’s stability guidelines [recorded]: - **i**) Replace pooling: **strided convolutions** in D (downsampling) and **transposed convolutions** in G (upsampling). - **ii**) **Batch normalization** in both nets — but *not* on G ’s output layer or D ’s input layer. - **iii**) Remove fully connected hidden layers (for deeper architectures). - **iv**) Activations: **ReLU** in G everywhere except the output (tanh); **LeakyReLU** in D everywhere.

StyleGAN (Nvidia). The deck’s state-of-the-art step [recorded]. Two nets inside the generator: **1**) a **mapping network** (an MLP) that sends the latent code $z \in \mathcal{Z}$ to an *intermediate* latent space $w \in \mathcal{W}$ — “disentangles the latent space and reduces correlations”; affine transforms of w then produce style vectors that control different visual features (hair colour, face shape) at different scales. **2**) A **synthesis network** that starts from a learned 4×4 constant and upsamples: **AdaIN** (adaptive instance normalization) applies each style vector as scale-and-bias at every conv layer, and explicit **noise injection** at each layer models stochastic detail (freckles, hair placement) “more efficiently than encoding this in the latent code.”

The deck’s payoff claim [recorded]: GANs learn “rich, meaningful latent representations that allow for semantic operations, such as vector arithmetic on faces” — the same embedding arithmetic as §46.2’s king — man + woman, now over images.

Basically, ... “DCGAN = the GAN idea with Chapter 44’s parts: stride instead of pooling, transposed conv to grow the image, batch norm, the right activations. StyleGAN = a two-stage generator: first warp the noise into a better-behaved latent space, then grow the image from a constant while dialling ‘style’ knobs (and sprinkling noise) at every layer.”

47.9 When GANs break [recorded]

“Training GANs can be notoriously difficult and unstable.” The deck’s four failure modes:

- **i) Mode collapse** — “the biggest difficulty.” G finds one output that fools D (the deck’s example: images of shoes) and produces only that, “forgetting how to generate other classes” — or cycles through a few modes, never learning the full distribution. §47.1(iii)’s missing diversity, in one failure.
- **ii) Non-convergence.** Parameters “oscillate, become unstable, and never converge” — the two networks chase each other forever.
- **iii) Vanishing gradients.** “If the discriminator gets too good too quickly, the generator’s gradients can vanish” — §47.6’s note, now as a failure mode.
- **iv) Hyperparameter sensitivity.** GANs “require significant fine-tuning” — §47.7’s Adam-at-0.0002 recipe is typical of how narrow the workable settings are.

Plausible fixes (the deck’s) [recorded]: **WGAN** (Wasserstein GAN) — measures the Wasserstein distance between the real and generated *distributions* and minimizes it, “strongly incentivizing the generator to produce a wide variety of outputs to match the diversity of the real data, thus significantly reducing mode collapse.” **Minibatch discrimination** — lets D see whole batches and measure sample similarity, so a batch of identical fakes “can be easily rejected.” **Progressive growing** — start at 4×4 , then add layers to grow 8×8 , 16×16 , ...; “simplifies the learning task at each stage.”

Basically, ... “GAN training is a balancing act on a knife edge: the forger either collapses to one trick (mode collapse), the critic gets too strong and kills the signal (vanishing gradients), or the two just orbit forever. Fixes: measure distribution distance instead of a classifier score (WGAN), punish boring batches, or grow the image gradually from tiny to full size.”

47.10 The diffusion pitch: many easy steps

The diffusion deck replays the trilemma and then changes the question: instead of *one* hard mapping noise $\rightarrow$ image (GAN) or a *lossy* compression (VAE), do the generation in **many easy steps** — learn to fix a *slightly* noisy image, then iterate.

The origin story (Sohl-Dickstein et al., 2015) [recorded]: ink diffusing in water. **Forward** (nature): the drop “naturally diffuses until it creates a uniform, chaotic distribution” — easy, irreversible. **Reverse** (learned): undo the diffusion, one small step at a time — the hard part a neural network is asked to learn.

- **i) The forward process** q — “the systematic destruction of data.” Slowly add Gaussian noise to a real image over T steps. **Fixed: no learning here**, just follow a schedule. Ends at pure noise $x_T \sim \mathcal{N}(0, I)$.
- **ii) The reverse process** p — “the learned restoration.” Start from pure noise; the network denoises “one step at a time.” **This is the trainable part.**

The deck’s conclusion [recorded]: “We moved from ‘One Hard Step’ (GANs) to ‘Many Easy Steps’ (Diffusion). The trade-off: we gained training stability and mode coverage at the cost of inference speed” — and the fundamental insight: “Don’t learn to create an image from scratch; just learn to fix a slightly noisy one.”

Basically, ... “GANs bet everything on one perfect forgery; diffusion takes a thousand tiny safe bets. Destroying an image with noise needs no learning — it’s a fixed recipe — so all the learning goes into the one easy skill: remove a little noise. That simplicity is why diffusion trains stably where GANs wobble.”

47.11 The forward process: the math

The Markov chain. Each noisy step depends only on the previous one [recorded]:

$$q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1}).$$

One step of noise. At step t , draw from a Gaussian centred on the previous image, scaled slightly down [recorded]:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I).$$

$\beta_t \in (0, 1)$ is the **variance schedule** — typically increasing, $\beta_1 < \beta_2 < \dots < \beta_T$ (“noise increases over time”).

The closed-form shortcut (the reparameterization trick). Reaching x_{500} step by step costs 500 sequential ops — too slow for training. Since sums of Gaussians are Gaussian, there is a one-jump formula. Define $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$; then [recorded]:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

i.e. $q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$. Signal weight $\sqrt{\bar{\alpha}_t}$ fades while noise weight $\sqrt{1 - \bar{\alpha}_t}$ grows — the same Gaussian-sum algebra as the deck’s two-step derivation (two independent Gaussians $a\epsilon_1 + b\epsilon_2$ merge into $\sqrt{a^2 + b^2}\epsilon$).

The schedule. The standard linear schedule: $T = 1000$ steps, β rising linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$. As β_t grows, $\bar{\alpha}_t \rightarrow 0$, and by step T , $x_T \approx \mathcal{N}(0, I)$ — pure noise [recorded].

Basically, ... “Each forward step keeps most of the old image and mixes in a little Gaussian noise (β_t controls how much). Because Gaussians add nicely, you can skip the whole chain: jump straight from the clean image to step t in one formula. Crank β up over 1000 steps and the image melts into pure static.”

47.12 eg: two steps of noise, one jump [verified-NumPy]

Own toy: $\beta_1 = 0.2$, $\beta_2 = 0.25$; take a scalar “image” $x_0 = 1.0$ and sampled noise $\epsilon = 0.7$.

i) $\alpha_1 = 0.8$, $\alpha_2 = 0.75$, so $\bar{\alpha}_2 = 0.8 \times 0.75 = 0.6$.

ii) Jump directly: $x_2 = \sqrt{0.6} \cdot 1.0 + \sqrt{1 - 0.6} \cdot 0.7 = 0.7746 + 0.6325 \times 0.7 = 0.7746 + 0.4427 = 1.2173$.

No 2-step chain needed — the formula carries both noise injections at once. At $t = 2$ the signal still outweighs the noise ($\bar{\alpha}_2 = 0.6$); at $t = 1000$ of the real schedule the signal weight is essentially zero.

Basically, ... “The $\bar{\alpha}$ bookkeeping says: after two steps, 60% of the variance is still signal, 40% is noise — and the one-line formula lands you at $x_2 = 1.2173$ without ever computing x_1 .”

47.13 The reverse process: learning to denoise

The model. If the forward steps are small, the reverse step is (approximately) Gaussian too. Learn it with a network [recorded]:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(t)),$$

input: noisy image x_t and timestep t ; output: the mean of the slightly cleaner x_{t-1} . The variance Σ_θ is usually fixed to a constant (e.g. β_t) — the deck keeps it simple and predicts only the mean.

The training target (the clever workaround). The true reverse $q(x_{t-1} | x_t)$ is intractable — it would need the distribution of *all* images. But during training we *know* x_0 , so the posterior $q(x_{t-1} | x_t, x_0)$ is tractable by Bayes’ rule [recorded]:

$$q(x_{t-1} | x_t, x_0) = \frac{q(x_t | x_{t-1}) q(x_{t-1} | x_0)}{q(x_t | x_0)} = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I).$$

A product of Gaussians is a Gaussian; grinding the algebra gives the target mean [recorded]:

$$\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right).$$

Key insight (the deck’s): x_t is known to the network; the *only unknown* in that formula is ϵ — the noise that was added. “To predict the previous image mean $\tilde{\mu}_t$, the neural network only needs to predict the noise ϵ .”

The loss collapses to MSE. KL divergence between the ground-truth Gaussian and p_θ , with fixed variances, reduces to matching the means; reparameterizing the mean through the predicted noise ϵ_θ cancels the constants [recorded]:

$$L_{\text{simple}} = \|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2.$$

“Take a clean image, add noise, ask the U-Net ‘what noise was added?’, MSE between actual and predicted noise. We are literally teaching the model to separate signal from noise.”

Training (Algorithm 1) [recorded]: repeat — sample x_0 from the data, sample t uniformly from $\{1, \dots, T\}$, sample $\epsilon \sim \mathcal{N}(0, I)$, gradient-descent step on L_{simple} . Note the efficiency: training never unrolls the full chain — one random timestep per step.

Sampling (Algorithm 2) [recorded]: start $x_T \sim \mathcal{N}(0, I)$; for $t = T, \dots, 1$: compute $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \epsilon_\theta(x_t, t) \right)$, sample $z \sim \mathcal{N}(0, I)$ (but $z = 0$ at the final step $t = 1$), set $x_{t-1} = \mu_\theta + \sigma_t z$. Return x_0 . The re-injected noise z is “Langevin dynamics” — it keeps samples diverse instead of collapsing to one average image.

Basically, ... “The network never predicts the clean image directly — it predicts the *noise* that was added, and subtracting the predicted noise is the denoising step. Training is one tidy loop: pick a random moment in the melting process and fix it. Generation runs the film backwards, adding a whisper of fresh noise each frame so every sample comes out different.”

47.14 The denoiser: a U-Net that knows what time it is

The noise predictor ϵ_θ is a **modified U-Net** — §44.9’s encoder-decoder with skip connections, now predicting noise instead of segment masks [recorded]:

- **Structure.** Encoder downsamples ($64 \times 64 \rightarrow 8 \times 8$ in the deck’s figure), bottleneck, decoder upsamples; **skip connections** carry fine detail from encoder to decoder (the §44.9 justification, verbatim).
- **Details (the deck’s table, condensed)** [recorded]: **GroupNorm** (better than BatchNorm for small batches), **SiLU/Swish** activation (“smoother gradients than ReLU”), self-attention at 16×16 resolution (“captures long-range dependencies”), and **noise ϵ as the prediction target** (“more stable than predicting x_0 ”).

The time problem. One network must denoise at *every* noise level — but “timestep is just a number,” and raw $t = 500$ vs 501 “look too different” while they “should be similar.” The fix: **sinusoidal time embeddings**, the §46.10 trick applied to timesteps [recorded]:

$$\text{emb}(t)_i = \begin{cases} \sin\left(\frac{t}{10000^{2i/d}}\right) & i \text{ even,} \\ \cos\left(\frac{t}{10000^{(2i-1)/d}}\right) & i \text{ odd.} \end{cases}$$

Low dimensions oscillate fast (fine time differences), high dimensions slowly (coarse), all values bounded in $[-1, 1]$, deterministic — no learning needed.

The pipeline [recorded]: scalar $t \rightarrow$ sinusoidal vector $\mathbb{R}^{128} \rightarrow$ 2-layer MLP (Linear + SiLU) $\rightarrow \mathbb{R}^{512}$ embedding temb . The embedding is broadcast and **added to the feature maps in every residual block**, “modulating the network’s behaviour” — the same embedding at all spatial locations. The MLP “learns a task-specific representation” on top of the fixed sinusoids.

Basically, ... “The denoiser is a U-Net (§44.9’s shape, new job) that also gets told the clock: a sine-wave fingerprint of the timestep, refined by a small MLP, mixed into every layer — so one network can behave like 1000 different denoisers, one per noise level.”

47.15 eg: a time embedding, by hand [verified-NumPy]

Use the deck’s formula with $d = 4$: $i = 0$ gives $\sin(t)$, $i = 1$ gives $\cos(t/10)$, $i = 2$ gives $\sin(t/10000)$, $i = 3$ gives $\cos(t/10000^{1.25})$.

i) $t = 0$: $[0, 1, 0, 1]$.

ii) $t = 1$: $[\sin 1, \cos 0.1, \sin 0.0001, \cos(1 \times 10^{-5})] = [0.8415, 0.9950, 0.0001, 1.0000]$.

Two adjacent timesteps get nearly identical embeddings in the slow dimensions and visibly different ones in the fast dimension — exactly the “nearby timesteps should be similar” property §47.14 demanded.

Basically, ... “Same sine-wave trick as §46.10’s positional encoding, pointed at time instead of position: $t = 0$ and $t = 1$ are neighbours, and their fingerprints say so.”

47.16 DDIM: sampling 20–100× faster

The bottleneck. Sampling walks $1000 \rightarrow 999 \rightarrow \dots \rightarrow 0$ — strictly sequential (“to calculate step 999, we must have the result of step 1000”), one heavy U-Net call per step. “Generating one image requires running the heavy neural network 1000 times” [recorded].

The insight (Song et al., 2020) [recorded]: “Does the forward process HAVE to be random?” Redefine the schedule so the noise→image mapping is **deterministic** — no randomness, yet the *same training objective* holds. The trajectory becomes smooth and predictable, “we don’t need to take baby steps. We can take huge strides.” **Key benefit: use the exact same pre-trained DDPM model — no retraining required.**

The update rule [recorded]:

$$x_{t-1} = \underbrace{\sqrt{\bar{\alpha}_{t-1}} x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}_{\text{predicted } \hat{x}_0} + \underbrace{\sqrt{1 - \bar{\alpha}_{t-1}} \epsilon_\theta(x_t, t)}_{\text{direction pointing to } x_t}$$

Read it as a weighted mix: $\hat{x}_0$ (the clean image the network thinks it sees) scaled by $\sqrt{\bar{\alpha}_{t-1}}$, plus the predicted noise scaled by $\sqrt{1 - \bar{\alpha}_{t-1}}$ — “we mix clean image and noise according to the noise schedule at $t - 1$.”

Skipping steps. Pick a subsequence $\tau = [1000, 950, 900, \dots, 0]$ — 20 steps instead of 1000; the deck’s timing table [recorded]: DDPM 1000 steps 20 s vs DDIM 20–50 steps 1 s. “DDIM allows us to trade a tiny bit of quality for a massive gain in speed.”

DDIM inversion: image → noise. Determinism buys reversibility: run the update *backwards* and a real image maps to the specific noise x_T that would have generated it [recorded]. That noise “encodes the original image” — composition, structure, layout — so you can edit: invert a cat photo, change the prompt to “dog,” denoise, and get “a dog in the same pose/composition as the original cat.” DDPM cannot do this (“due to randomness”). Pros/cons [recorded]: fast, deterministic, invertible, enables editing — but less diversity, small inversion errors accumulate, and a quality-vs-speed tradeoff.

Basically, ... “DDPM rolls dice at every step, so it must creep forward. DDIM removes the dice — same trained model, same loss — and the path becomes a smooth, predictable road you can drive in 20 strides instead of 1000 baby steps. And because nothing is random, you can drive it backwards: photo in, meaningful noise out, edit, drive forward again.”

47.17 eg: one DDIM step, by hand [verified-NumPy]

Own numbers: $\bar{\alpha}_t = 0.5$, $\bar{\alpha}_{t-1} = 0.6$, $x_t = 1.0$, $\epsilon_\theta(x_t, t) = 0.4$.

i) Predicted clean image: $\hat{x}_0 = \frac{1.0 - \sqrt{1 - 0.5} \times 0.4}{\sqrt{0.5}} = \frac{1.0 - 0.2828}{0.7071} = 1.0142$.

ii) DDIM step: $x_{t-1} = \sqrt{0.6} \times 1.0142 + \sqrt{1 - 0.6} \times 0.4 = 0.7746 \times 1.0142 + 0.6325 \times 0.4 = 0.7856 + 0.2530 = 1.0386$.

No randomness anywhere — rerun this and you get 1.0386 again. That determinism is the whole point.

Basically, ... “Predict the clean image (1.0142), then re-mix it with the predicted noise at the *previous* step’s schedule weights — landing at 1.0386. Same inputs, same output, every time: the anti-DDPM.”

47.18 Guidance: steering the denoiser

So far the model generates *anything*. Guidance steers it: “generate only dogs,” “a dog wearing sunglasses.”

Interlude: the score function [recorded]. The score is the gradient of the log-probability: $\text{score}(x_t) = \nabla_{x_t} \log p(x_t)$ — “which direction increases the probability of x_t ?” Noise prediction and score prediction are the same thing in different clothes [recorded]:

$$\nabla_{x_t} \log p(x_t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}.$$

Score space makes guidance math cleaner: conditioning = adding a gradient that points uphill toward the desired class.

Classifier guidance [recorded]: train a plain unconditional diffusion model, then at each denoising step ask a separate classifier “does this look like class y ?” and push along its gradient. By Bayes:

$$\nabla_{x_t} \log p(x_t | y) = \underbrace{\nabla_{x_t} \log p(x_t)}_{\text{unconditional score}} + \lambda \cdot \underbrace{\nabla_{x_t} \log p(y | x_t)}_{\text{classifier gradient}}$$

λ is the guidance scale. Pros: simple, works with existing models. Cons: needs a *separate* classifier that must work on *noisy* images, limited to the classifier’s fixed classes — “can’t use text prompts.”

Classifier-free guidance (CFG) [recorded] — the current standard (DALL·E 2, Imagen, Stable Diffusion): train *one* model to do both conditional and unconditional generation by randomly **dropping the condition 10% of the time** (null condition $\emptyset$). Rewrite the classifier-gradient term via Bayes — $\nabla_{x_t} \log p(y | x_t) = \nabla_{x_t} \log p(x_t | y) - \nabla_{x_t} \log p(x_t)$ — and substitute:

$$\nabla_{x_t} \log p(x_t | y) = (1 - \lambda) \nabla_{x_t} \log p(x_t) + \lambda \nabla_{x_t} \log p(x_t | y),$$

in noise space [recorded]:

$$\epsilon_{\text{guided}} = \epsilon_{\text{uncond}} + \lambda \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}}).$$

Read it as a vector: find the direction from unconditional to conditional, amplify by λ — “higher $\lambda \rightarrow$ stronger adherence to condition.” The conditioning vector c (text embedding, class label) is injected into the U-Net “just like the time embedding in DDPM” (§47.14).

Classifier vs classifier-free (the deck’s table, condensed) [recorded]: classifier guidance — needs an external model, fixed classes, one forward pass + classifier cost, “less common” today. CFG — single model, “any condition (text, image, etc.),” two forward passes per step, typically better quality, “the dominant approach in modern text-to-image models.”

How the text gets in: cross-attention. At each U-Net layer [recorded]: $Q = W_Q z_t$ (from the image latent), $K = W_K c$, $V = W_V c$ (from the text embedding), then §46.5’s formula $\text{Attention} = \text{softmax}(QK^\top / \sqrt{d}) V$. “Image asks: ‘What features should I have here?’ Text provides: ‘A cat should have whiskers, fur texture...’”

Basically, ... “Unguided diffusion is a slot machine. Classifier guidance bolts on a teacher that nudges each step toward ‘dog’ — but the teacher only knows its ten classes. Classifier-free guidance trains one model on both regimes (condition dropped 10% of the time) and steers by the *difference*: push from ‘whatever’ toward ‘what the prompt says,’ scaled by λ . Text reaches the image through cross-attention — §46.7’s mechanism, pointed at words instead of encoder states.”

47.19 Latent diffusion: Stable Diffusion

The problem. DDPM diffuses in pixel space: a $512 \times 512 \times 3$ image is 786,432 dimensions, processed *every* denoising step — “computational cost... memory intensive... slow training... slow generation” [recorded]. Key observation: “not all pixels are equally important — many pixels are redundant (e.g., clear blue sky).”

The fix. Compress first, diffuse the compression, decompress last. An autoencoder (encoder $\mathcal{E}$, decoder $\mathcal{D}$, trained with reconstruction + regularization) squeezes $512 \times 512 \times 3$ to a $64 \times 64 \times 4$ latent [recorded]: 786,432 dims $\rightarrow$ 16,384 dims — a **48 $\times$ reduction** [verified-NumPy]. Latent space is “like JPEG compression... even more compressed, captures ‘essence.’”

Training (two stages) [recorded]: **1)** train the autoencoder (VAE) on a large image dataset until $\mathcal{D}(\mathcal{E}(x)) \approx x$ with small latents, then **freeze** it; **2)** encode images ($z = \mathcal{E}(x)$), add noise in latent space ($z_t = \sqrt{\alpha_t} z_0 + \sqrt{1 - \alpha_t} \epsilon$), and train the U-Net to predict the noise: $L = \|\epsilon - \epsilon_\theta(z_t, t, c)\|^2$, with condition c = text embeddings, class labels, etc.

Generation [recorded]: sample $z_T \sim \mathcal{N}(0, I)$ at $64 \times 64 \times 4$ (not $512 \times 512 \times 3!$), take a prompt c (“a cat on a sofa”), denoise $z_T \rightarrow z_0$ with the conditioned U-Net, decode $x = \mathcal{D}(z_0)$. Full pipeline: Text $\xrightarrow{\text{CLIP}} c \rightarrow \text{U-Net}(z_t, t, c) \rightarrow z_0 \xrightarrow{\text{Decoder}} \text{Image}$, training objective $\mathbb{E}_{z, \epsilon, t, c} \|\epsilon - \epsilon_\theta(z_t, t, c)\|^2$ [recorded].

Stable Diffusion in practice [recorded]: “Latent Diffusion Model released by Stability AI” — VAE compression factor $f = 8$ ($512 \times 512 \rightarrow 64 \times 64$), 4 latent channels, CLIP ViT-L/14 text encoder (SD 1.x/2.x), cross-attention at multiple U-Net resolutions, trained on LAION-5B; “first high-quality, open-source text-to-image model that can run on consumer GPUs (RTX 3090, even RTX 3060).” Versions (deck’s table, condensed): SD 1.4 (Aug 2022, first public, 512²) $\rightarrow$ 1.5 $\rightarrow$ 2.0 (OpenCLIP encoder) $\rightarrow$ 2.1 $\rightarrow$ SDXL 1.0 (Jul 2023, 1024², dual encoders, refiner) $\rightarrow$ SD 3.0 (2024, diffusion-transformer architecture) — “all based on Latent Diffusion Models (LDM) architecture.”

Basically, ... “Denoising a full-resolution photo 50 times is expensive, so cheat: shrink the photo 48 $\times$ with a VAE, do all the noisy work on the tiny version, then blow it back up. Same math, same training loop (§47.13), one frozen compressor — and suddenly a consumer GPU can paint ‘a cat on a sofa.’”

47.20 Judging the forgeries: FID, IS, CLIP

“Evaluating generative models is like evaluating art” — no single correct output, so the field settled on statistics [recorded].

i) FID — Fréchet Inception Distance [recorded]. Pass real and generated images through Inception-v3, fit a Gaussian to each feature set, and measure the Fréchet distance between the Gaussians:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}).$$

μ_r, Σ_r = real-image feature statistics; μ_g, Σ_g = generated. **Lower is better** (0 = identical distributions); sensitive to *both* quality and diversity; “most widely used metric in practice”; good FID < 10 (domain-dependent).

eg (own — [verified-NumPy]). $\mu_r = (0, 0)$, $\mu_g = (1, 0)$, $\Sigma_r = \Sigma_g = I_2$: $(\Sigma_r \Sigma_g)^{1/2} = I$, so the trace term is $\text{Tr}(I + I - 2I) = 0$ and $\text{FID} = \|(-1, 0)\|^2 = 1.0$. Identical spreads, shifted means — the distance is just the squared shift.

ii) Inception Score (IS) [recorded]. Uses a pretrained Inception-v3 classifier — no real images needed. Two desiderata: each generated image should be *clearly classifiable* (low-entropy $p(y | x)$, quality) and the set should span *many classes* (high-entropy marginal $p(y)$, diversity):

$$\text{IS} = \exp(\mathbb{E}_x[\text{KL}(p(y | x) \| p(y))]).$$

Higher is better (typical range 2–10). Limitation: “biased toward ImageNet classes, doesn’t compare to real data.”

iii) CLIP score [recorded]. For text-to-image: embed the image and the caption with CLIP (“contrastive language-image pre-training,” trained on 400M image-text pairs, shared image-text embedding space) and take cosine similarity:

$$\text{CLIPScore} = \max(100 \cdot \cos(E_I(x), E_T(c)), 0).$$

Measures image–text *alignment*, not realism alone.

Which metric when (the deck’s table, condensed) [recorded]: FID — distribution similarity to real images — best for overall unconditional quality+diversity (needs large samples; implementation-sensitive).

IS — quality + diversity — best for class-conditional generation (ignores real data; ImageNet-biased). CLIP — image-text alignment — best for text-to-image (doesn’t measure realism alone). In practice: combine them (FID + IS unconditional; FID + CLIP text-conditional) — and “human evaluation still crucial.”

Basically, ... “FID asks: do the generated photos have the same *statistics* as real photos? (Lower wins.) Inception Score asks: is each image confidently one thing, and are the images many different things? (Higher wins.) CLIP score asks: does the image match its caption? (Higher wins.) No single number tells the truth — use them together, and trust human eyes last.”

47.21 Where this goes next: closing Part VI (and the book)

- i) **The generator census.** Four ways to generate, four training signals: **1)** the autoregressive decoder (§46.12, via §45.13(iii)) — trained by next-token likelihood, sampling one token at a time; **2)** the GAN generator (§47.4) — trained by a learned critic, one shot from noise; **3)** the diffusion denoiser (§47.13) — trained by noise-prediction MSE, a thousand small steps; **4)** the fast variants — DDIM’s deterministic skips (§47.16) and the latent compressor (§47.19). All four are differentiable modules (§42.4’s machinery); none needed new optimization beyond §43.12’s Adam family (§47.7’s GAN used Adam at 0.0002).
- ii) **The Part VI arc, complete.** Chapter 41 built the neuron; 42 derived backprop; 43 tuned the optimizer; 44 shared weights across space; 45 across time; 46 let any position talk to any other; 47 turns the whole machine around and *generates* — noise in, data out. The book’s one-stop promise lands here: from a single dense layer to Stable Diffusion’s text-to-image pipeline, every step traced to a committed source and every number re-checked.
- iii) **The debugging playbook’s last word.** §40.5(i) — “shapes first” — closes the loop with this chapter: a GAN that dies at step 1 is almost always a 784-vs-28×28 reshape or a tanh-vs-sigmoid range mismatch (§47.7); a diffusion model that explodes is a schedule or time-embedding bug (§§47.11, 47.14). Same discipline, new architectures.
- iv) **Beyond the course.** The decks stop at the fundamentals; the live field keeps moving — through the supplementary shelf in GOAL.md for the follow-up reading. The foundation in this chapter — the forward/reverse split, the noise-prediction loss, guidance as a vector in score space — is the vocabulary those readings are written in.

Problem set

1. **The 2×2 marginal.** The deck’s eight configurations with $x_{11} = 1$ have probabilities 0.05, 0.10, 0.15, 0.03, 0.02, 0.08, 0.07, 0.01. (i) Compute $P(x_{11} = 1)$ by marginalization. (ii) Given $x_{11} = 1$ and the rest missing, which completion (x_{12}, x_{21}, x_{22}) maximizes $P(x_{\text{mis}}, x_{\text{obs}})$? (iii) In one line, say why the same computation is impossible for a 32×32 binary image.
2. **GAN losses by hand.** Real outputs $D(x) = [0.8, 0.95]$, fake outputs $D(G(z)) = [0.4, 0.2]$. (i) Compute the discriminator’s BCE loss. (ii) Compute the generator’s loss in the minimax form. (iii) Compute the generator’s non-saturating loss and explain in one line why it is larger here.
3. **The notebook’s parameter counts.** The generator is $\text{Linear}(64 \rightarrow 256)$, $\text{Linear}(256 \rightarrow 256)$, $\text{Linear}(256 \rightarrow 784)$ with biases; the discriminator is $\text{Linear}(784 \rightarrow 256)$, $\text{Linear}(256 \rightarrow 256)$, $\text{Linear}(256 \rightarrow 1)$ with biases. (i) Compute both totals. (ii) Which network is larger, and by how many parameters? (iii) In one line, say why activations and batch-norm parameters (none here) don’t change the count.
4. **The Nash point.** (i) At Nash, D outputs 0.5 on every input: evaluate the minimax value $V(D, G) = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$. (ii) Early in training $D(G(z)) = 0.05$: compute the generator’s non-saturating loss $-\log D(G(z))$. (iii) In two lines, connect (ii) to the §47.6 Note — which failure mode does the non-saturating loss patch?
5. **Forward diffusion numbers.** $\beta_1 = 0.1$, $\beta_2 = 0.2$. (i) Compute $\bar{\alpha}_2$. (ii) Write $q(x_2 | x_0)$ as $\mathcal{N}(\cdot | x_0, \cdot | I)$ with numbers. (iii) How much signal variance remains at $t = 2$ vs noise variance?

6. **The linear schedule's endpoints.** $\beta_1 = 10^{-4}$, $\beta_{1000} = 0.02$, linear in t . (i) Compute β_{500} . (ii) The signal weight is $\sqrt{\bar{\alpha}_t}$: compute it at $t = 500$ and at $t = 1000$ (values: $\bar{\alpha}_{500} \approx 0.0786$, $\bar{\alpha}_{1000} \approx 4.04 \times 10^{-5}$). (iii) In one line, state what x_{1000} is distributed as.
7. **One DDIM step.** $\bar{\alpha}_t = 0.7$, $\bar{\alpha}_{t-1} = 0.75$, $x_t = 0.5$, $\epsilon_\theta(x_t, t) = -0.3$. (i) Compute the predicted clean image $\hat{x}_0$. (ii) Compute x_{t-1} via the DDIM update. (iii) In one line, say what changes in the answer if the network predicts $\epsilon_\theta = 0$.
8. **FID by hand.** $\mu_r = (0, 0)$, $\mu_g = (2, 0)$, $\Sigma_r = \Sigma_g = I_2$. (i) Compute the FID. (ii) Recompute with $\mu_g = (0, 0)$. (iii) In two lines, say which of the deck's desiderata — quality or diversity — a perfect FID of 0 can still hide.
9. **Classifier-free guidance numbers.** $\epsilon_{\text{uncond}} = (0.2, -0.1)$, $\epsilon_{\text{cond}} = (0.8, 0.3)$, guidance scale $\lambda = 3$. (i) Compute ϵ_{guided} . (ii) Recompute with $\lambda = 1$ and $\lambda = 0$. (iii) In one line, say what $\lambda = 0$ generation corresponds to.
10. **Inception Score by hand.** Three generated images with $p(y | x)$: $(0.9, 0.1)$, $(0.8, 0.2)$, $(0.15, 0.85)$. (i) Compute the marginal $p(y)$. (ii) Compute the three KL divergences. (iii) Compute the IS. (iv) In one line, say whether this model's weakness is quality or diversity, and why.
11. **The book-ender.** In one paragraph, name the four generator families the book built (§46.12's decoder, the GAN generator, the diffusion denoiser, the latent-diffusion pipeline) and for each say what the generator *is* and what training signal teaches it.

Sources: GenAI Week 6 “Generative Models and GAN” deck (Balaji Srinivasan, Ganapathy Krishnamurthi), 65 slides — [sources/course materials/GenAI-20261009T001550Z-1-001/GenAI/Slides/Week - 6/Generative Models and GAN - Lecture Slides.pdf](#) (the 225-page PDF is the same 65 slides with beamer overlays) and the notes PDF [.../GenAI/Notes/Week6-Generative Models and GAN .pdf](#) (identical content, used for cross-check) — generative-model definition and goal, 2×2 toy grid (marginalization numbers 0.05–0.01, inpainting $\arg\max P(1, 0, 1, 0) = 0.15$), computational bottleneck (2^{1024}), generative classifier (Naive Bayes), latent-space paradigm shift (PCA/LDA/SVD examples), explicit/implicit taxonomy, minimax objective and D/G objectives, non-saturating equivalence, Nash equilibrium, two-phase training algorithm, DC-GAN guidelines, StyleGAN (mapping/synthesis networks, AdaIN, noise injection, latent arithmetic), training challenges (mode collapse, non-convergence, vanishing gradients, hyperparameter sensitivity) and fixes (WGAN, minibatch discrimination, progressive growing), [learnopencv.com](#) figure credits; [GAN_from_Scratch.ipynb](#) — vanilla GAN on 60{,}000 grayscale 28×28 images: latent 64, hidden 256, image 784; generator (LeakyReLU 0.2, tanh; 283{,}920 params) and discriminator (LeakyReLU 0.2, sigmoid; 267{,}009 params) architectures; BCE loss; Adam $\eta = 0.0002$ with separate optimizers; $[-1, 1]$ normalization + $(x + 1)/2$ denormalization; `detach()` on D's fake pass; non-saturating G via BCE against real labels; 100 epochs $\times$ 600 steps = 60{,}000 updates; avg $L_D = 0.7651$, avg $L_G = 2.0232$, final 0.8781 / 1.5290, ratio 1.7413; early-epoch $D(x) \approx 0.95$, $D(G(z)) \approx 0.05$ (all torch code and numbers [recorded]); GenAI Week 11 “Introduction to Diffusion Models” deck (same authors), 94 slides — [.../Slides/Week - 11/Week11_GenAI_Diffusion20251127.pdf](#) (339-page PDF with beamer overlays, text-extractable) — generative trilemma (VAE vs GAN table), Sohl-Dickstein 2015 origin, forward/reverse definitions, Markov chain, $q(x_t | x_{t-1})$ formula, $\alpha_t/\bar{\alpha}_t$ notation, reparameterization derivation, linear schedule ($\beta_1 = 10^{-4} \rightarrow \beta_T = 0.02$, $T = 1000$), reverse model p_θ , Bayes posterior and $\tilde{\mu}_t$ formula, $KL \rightarrow MSE$, L_{simple} , Algorithms 1–2 (incl. $z = 0$ at $t = 1$, Langevin noise), U-Net architecture table (GroupNorm, SiLU, 16×16 attention, skip connections), sinusoidal time-embedding formula and MLP pipeline, DDIM (Song et al. 2020; deterministic update, step-skipping, 20–50 steps ≈ 1 s vs 1000 ≈ 20 s, inversion, cat $\rightarrow$ dog editing example, pros/cons), score function and noise-score link, classifier guidance formula + pros/cons, classifier-free guidance (10% condition dropout, ϵ_{guided} formula, comparison table), cross-attention $Q = W_Q z_t$, $K = W_K c$, $V = W_V c$, latent diffusion ($48 \times$ claim: 786,432 $\rightarrow$ 16,384; two-stage training; generation pipeline; training objective), Stable Diffusion specs ($f = 8$, 4 latent channels, CLIP ViT-L/14, LAION-5B) and versions table, FID/IS/CLIP formulas and comparison table, evaluation-metrics guidance (all [recorded]); diagram references — Goodfellow et al. 2014 (<https://arxiv.org/abs/1406.2661>), Ho et al. 2020 DDPM (<https://arxiv.org/abs/2006.11239>), Song et al. 2020 DDIM (<https://arxiv.org/abs/2010.08092>), Rombach et al. 2021 LDM (<https://arxiv.org/abs/2112.10752>); book chapters 24 (PCA), 30 (Naive Bayes), 40 (§40.5(i)), 41–43 (§§42.4, 43.12), 44 (§§44.6, 44.9, 44.10), 45 (§§45.1, 45.10, 45.11, 45.13), 46 (§§46.2, 46.5, 46.7, 46.10, 46.12, 46.16).

Solutions — Chapter 47: Generative models: GANs and diffusion

Full worked solutions. All numbers re-run in numpy [verified-NumPy]; deck/notebook transcriptions marked [recorded].

1. The 2×2 marginal

- (i) $P(x_{11} = 1)$ sums the eight configurations with $x_{11} = 1$: $0.05+0.10+0.15+0.03+0.02+0.08+0.07+0.01 = 0.51$.
- (ii) The denominator $P(x_{\text{obs}})$ is constant across candidates, so maximize the joint: the deck's table gives $P(1, 0, 1, 0) = 0.15$ as the largest of the eight, hence $(x_{12}, x_{21}, x_{22}) = (0, 1, 0)$.
- (iii) A 32×32 binary image has 2^{1024} configurations — half the pixels missing means summing over 2^{512} candidates for the denominator, “computationally impossible” (§47.2).

2. GAN losses by hand

- (i) $L_D = -\frac{1}{4}[\log 0.8 + \log(1-0.4) + \log 0.95 + \log(1-0.2)] = -\frac{1}{4}[-0.2231 - 0.5108 - 0.0513 - 0.2231] = 0.2521$.
- (ii) Minimax: $L_G = -\frac{1}{2}[\log(1-0.4) + \log(1-0.2)] = -\frac{1}{2}[-0.5108 - 0.2231] = 0.3670$.
- (iii) Non-saturating: $L_G = -\frac{1}{2}[\log 0.4 + \log 0.2] = -\frac{1}{2}[-0.9163 - 1.6094] = 1.2629$. It is larger because D is confident the fakes are fake ($0.4, 0.2 \ll 0.5$), so labeling them “real” incurs a steep penalty — the steep slope is exactly the usable gradient the minimax form lacks here.

3. The notebook's parameter counts

- (i) Generator: $(64 \cdot 256 + 256) + (256 \cdot 256 + 256) + (256 \cdot 784 + 784) = 16,640 + 65,792 + 201,488 = 283,920$.
Discriminator: $(784 \cdot 256 + 256) + (256 \cdot 256 + 256) + (256 \cdot 1 + 1) = 200,960 + 65,792 + 257 = 267,009$.
- (ii) The generator is larger, by $283,920 - 267,009 = 16,911$ parameters.
- (iii) Activations (LeakyReLU, tanh, sigmoid) have no learnable parameters; biases were included in each layer's count (+256, +784, +1).

4. The Nash point

- (i) $V = \mathbb{E}[\log 0.5] + \mathbb{E}[\log(1-0.5)] = 2 \log 0.5 = -1.3863$ (natural log).
- (ii) $-\log 0.05 \approx 2.9957$.
- (iii) At (ii) the minimax generator term would be $\log(1-0.05) \approx -0.0513$ — nearly flat, the §47.6 Note's vanishing-gradient failure (critic too strong, forger gets no signal). The non-saturating loss instead stays steep (≈ 2.9957), patching that failure mode.

5. Forward diffusion numbers

- (i) $\alpha_1 = 0.9, \alpha_2 = 0.8, \bar{\alpha}_2 = 0.9 \times 0.8 = 0.72$.
- (ii) $q(x_2 | x_0) = \mathcal{N}(x_2; \sqrt{0.72} x_0, (1-0.72)I) = \mathcal{N}(0.8485 x_0, 0.28 I)$.
- (iii) Signal variance 0.72 vs noise variance 0.28 — 72% of the variance is still the original signal at $t = 2$.

6. The linear schedule's endpoints

- (i) $\beta_t = 10^{-4} + (0.02 - 10^{-4}) \frac{t-1}{999}$: $\beta_{500} = 10^{-4} + 0.0199 \times \frac{499}{999} \approx 0.0100$.
- (ii) $\sqrt{\bar{\alpha}_{500}} = \sqrt{0.0786} \approx 0.2803$ — about 28% of the signal's scale survives at the midpoint. At $t = 1000$: $\sqrt{\bar{\alpha}_{1000}} = \sqrt{4.04 \times 10^{-5}} \approx 0.0064$ — effectively gone.

- (iii) $x_{1000} \sim \mathcal{N}(0, I)$: pure noise, the §47.11 claim verified numerically.

7. One DDIM step

- (i) $\hat{x}_0 = \frac{0.5 - \sqrt{1-0.7} \times (-0.3)}{\sqrt{0.7}} = \frac{0.5 + 0.1643}{0.8367} = \frac{0.6643}{0.8367} = 0.7940$.
- (ii) $x_{t-1} = \sqrt{0.75} \times 0.7940 + \sqrt{1-0.75} \times (-0.3) = 0.8660 \times 0.7940 - 0.5 \times 0.3 = 0.6876 - 0.15 = 0.5376$.
- (iii) If $\epsilon_\theta = 0$, then $\hat{x}_0 = x_t / \sqrt{\alpha_t}$ (the network claims the image is already clean, just rescaled) and the step is $x_{t-1} = \sqrt{\alpha_{t-1}} \hat{x}_0$ — pure rescaling, no noise mixing.

8. FID by hand

- (i) $(\Sigma_r \Sigma_g)^{1/2} = I_2$, so the trace term is $\text{Tr}(I + I - 2I) = 0$; $\text{FID} = \|(0, 0) - (2, 0)\|^2 = 4.0$.
- (ii) With $\mu_g = (0, 0)$: $\text{FID} = 0$ — identical Gaussians.
- (iii) FID measures distribution *statistics*: a model that memorizes and regurgitates a small set of real images can score near 0 while having no true diversity (the deck’s warning that FID is “sensitive to both quality and diversity” cuts both ways — here, memorization games the diversity side).

9. Classifier-free guidance numbers

- (i) $\epsilon_{\text{guided}} = (0.2, -0.1) + 3 \times ((0.8, 0.3) - (0.2, -0.1)) = (0.2, -0.1) + 3 \times (0.6, 0.4) = (2.0, 1.1)$.
- (ii) $\lambda = 1$: $(0.2, -0.1) + (0.6, 0.4) = (0.8, 0.3) = \epsilon_{\text{cond}}$ — pure conditional. $\lambda = 0$: $(0.2, -0.1) = \epsilon_{\text{uncond}}$ — pure unconditional.
- (iii) $\lambda = 0$ ignores the prompt entirely: unguided generation (§47.18’s slot machine).

10. Inception Score by hand

- (i) $p(y) = \frac{1}{3}[(0.9, 0.1) + (0.8, 0.2) + (0.15, 0.85)] = (0.6167, 0.3833)$.
- (ii) $\text{KL}_1 = 0.9 \log \frac{0.9}{0.6167} + 0.1 \log \frac{0.1}{0.3833} = 0.9(0.3788) + 0.1(-1.3445) = 0.2059$; $\text{KL}_2 = 0.8 \log \frac{0.8}{0.6167} + 0.2 \log \frac{0.2}{0.3833} = 0.8(0.2595) + 0.2(-0.6505) = 0.0781$; $\text{KL}_3 = 0.15 \log \frac{0.15}{0.6167} + 0.85 \log \frac{0.85}{0.3833} = 0.15(-1.4145) + 0.85(0.7962) = 0.4648$.
- (iii) $\text{Mean KL} = (0.2059 + 0.0781 + 0.4648)/3 = 0.2496$; $\text{IS} = \exp(0.2496) = 1.2835$.
- (iv) **Diversity** is the weakness: each image is confidently one class (good quality), but two of three images fall in the same class — the marginal $(0.6167, 0.3833)$ is skewed, so the model is near a mode-collapse regime (§47.9).

11. The book-enders

Accept any well-argued paragraph along these lines: **1)** the §46.12 transformer decoder — a next-token distribution trained by likelihood on sequences, generating autoregressively; **2)** the GAN generator — a noise→image map trained adversarially by a discriminator’s binary signal, generating in one shot; **3)** the diffusion denoiser — a U-Net trained by the noise-prediction MSE (§47.13’s L_{simple}), generating by iterating the reverse process from pure noise; **4)** the latent-diffusion pipeline — same denoiser operating on VAE-compressed latents (§47.19), optionally steered by classifier-free guidance and cross-attention (§47.18). All four rest on the same Chapters 41–46 machinery: MLPs, backprop, Adam, conv stacks, skip connections, attention.

